Applications and Integrations

Book 6 — The Applications & Integrations Core

From requirement to working system: design method, instrumentation and data acquisition, embedded and IoT systems, robotics and drones, industrial automation, solar, battery, EV and microgrid systems, communications, signal processing and biomedical instrumentation, engineering software and digital twins, and complete engineering projects.

The Studio of Sadiq · Golam Kuadir Khan Prince

Contents

Part I — Engineering Design Methodology

Chapter 1 — From Requirement to Working System

Chapter 2 — System Architecture

Chapter 3 — Modeling and Simulation Workflow

Chapter 4 — Prototyping and Verification

Chapter 5 — Engineering Documentation

Part II — Instrumentation and Measurement Systems

Chapter 6 — Sensors and Transducers

Chapter 7 — Measurement and Signal Conditioning

Chapter 8 — Data Acquisition Systems

Chapter 9 — Instrumentation Projects

Part III — Embedded and Connected Systems

Chapter 10 — Embedded-System Integration

Chapter 11 — IoT Systems

Chapter 12 — Edge Computing and Embedded AI

Part IV — Robotics and Mechatronics

Chapter 13 — Robotics Fundamentals

Chapter 14 — Mobile Robotics

Chapter 15 — Robot Electronics and Control

Chapter 16 — Drones and Aerial Robotics

Chapter 17 — Robotic System Integration

Part V — Industrial Automation

Chapter 18 — Industrial Control Architecture

Chapter 19 — Industrial Sensors and Actuators

Chapter 20 — Industrial Motor Control

Chapter 21 — Industrial Communication and Safety

Part VI — Renewable Energy and Electric Mobility

Chapter 22 — Solar PV System Design

Chapter 23 — Battery Energy Systems

Chapter 24 — Electric Vehicles

Chapter 25 — Microgrids and Hybrid Energy Systems

Part VII — Communications, Signal Processing, and Biomedical Applications

Chapter 26 — Communication-System Integration

Chapter 27 — Practical Signal Processing

Chapter 28 — Biomedical Instrumentation

Part VIII — Engineering Software and Computational Tools

Chapter 29 — Engineering Software Workflows

Chapter 30 — Scientific and Engineering Computing

Chapter 31 — Engineering Software Architecture

Chapter 32 — AI and Machine Learning for EEE

Chapter 33 — Engineering Simulation and Digital Twins

Part IX — Interdisciplinary Engineering Projects

Chapter 34 — Project: Autonomous Line-Following Robot

Chapter 35 — Project: Smart Energy Monitor

Chapter 36 — Project: Solar Battery System

Chapter 37 — Project: Closed-Loop Motor Drive

Chapter 38 — Project: Wireless Sensor Network

Chapter 39 — Project: Engineering Simulation Tool

Chapter 40 — Capstone Engineering Project

Part I — Engineering Design Methodology

Every system in this book, from a sensor node to a microgrid, is built by one chain of work: problem to requirements, requirements to architecture, architecture to models and prototypes, prototypes to a verified, documented product that someone else can build again. Part I teaches that chain as a method, before the later parts apply it to instrumentation, embedded devices, robots, factories, energy systems, communications and software. Its rule runs through the whole book: every decision is traced to a number written down before the build, and every number is closed by a test with a pass/fail limit. Two small running systems carry the method. Chapters 1–3 follow **CCM-1**, a battery-powered remote temperature monitor for vaccine refrigerators, from a one-sentence brief to a validated architecture. Chapters 4–5 follow **PM-4**, a four-channel 12 V DC power monitor for server racks, through prototypes, tests, calibration, release and a field return.

By the end of this part you should understand

- How to state a problem with evidence and a value, find its stakeholders, and write functional and nonfunctional requirements that are measurable and verifiable.
- Why boundaries and constraints (regulations, networks, shipping rules, budgets) must be listed on day one.
- How requirements are allocated into design specifications and budgets for error, latency, energy and cost, with margins, and traced through design to acceptance tests.
- How a system is decomposed into functions and blocks, partitioned between hardware and software, and described by interfaces and by its flows of data, energy and control.
- How trade studies are run honestly (constraints first, scores from evidence, weights tested for rank reversal) and recorded as decisions.
- How models are built to answer a stated question, simulated with solvers that cannot step over the physics, fitted to data, tested for sensitivity, validated against independent data and made reproducible.
- How each prototype build retires named risks, and how instruments, test plans, calibration, diagnosis and regression tests turn it into evidence.
- How schematics, BOMs, design calculations, test reports and revisions are kept consistent under version control, so that a shipped unit can be traced and rebuilt.

Chapters

Chapter	Central question	What it delivers
Chapter 1 — From Requirement to Working System	How does a vague need become measurable, traceable requirements with acceptance criteria?	Problem statement and value, stakeholder needs, requirement tables, boundary and constraints, error and latency budgets, a machine-checked trace, statistically sized acceptance tests
Chapter 2 — System Architecture	How are requirements turned into blocks, interfaces and flows, and how are the big choices made?	Function tree, block diagram, hardware–software partitioning, interface control document, data, energy and control flows, a radio trade study, ADRs and an architecture FMEA
Chapter 3 — Modeling and Simulation Workflow	How does a model become trustworthy enough to carry a design decision?	Equivalent circuits, system- and component-level simulation, solver selection for stiff and switched models, assumption register, parameter estimation, sensitivity and Monte Carlo analysis, validation domains, run manifests
Chapter 4 — Prototyping and Verification	What must each prototype prove, and how are the instruments, tests and limits chosen that prove it?	Build sequence from breadboard to production prototypes, instrument selection, test plans, verification versus validation, per-unit calibration, fault diagnosis, regression testing and design iteration
Chapter 5 — Engineering Documentation	Which documents does a working system need, and how are they kept consistent with what ships?	Schematics, block and wiring diagrams, BOMs, design calculations, test reports, version control, technical documentation, revision management and reproducible releases

How this part connects to the rest of the book

Part I owns the method; later chapters use it without repeating it. Requirements tables, budgets with margins, component-selection tables, verification tables and FMEAs appear in every chapter of Parts II–VIII, and the Part IX projects follow the full chain end to end, with Chapter 40 turning it into a capstone report template. Chapter 3's modeling workflow is developed further in Chapters 30 and 33 (scientific computing, digital twins) and applied in Chapter 39, and Chapter 5's documentation practice in Chapter 29.

The method rests on theory owned by the sibling books, cited in one line and used. The mathematics of least squares, sensitivity, uncertainty propagation, optimisation and numerical solvers is in *The Mathematics Universe*, Ch. 22–25, and the statistics behind acceptance tests in its Ch. 20. Lumped thermal and electrical networks come from *Engineering Physics*, Ch. 17, and dynamic-system modeling from *Circuits, Signals and Communications*, Ch. 33. Embedded architecture, partitioning, energy budgets and firmware validation come from *Electronics and Computing*, Ch. 25 and Ch. 28. Battery equivalent circuits, installation design and standards compliance come from *Electrical Energy and Machines*, Ch. 16 and Ch. 30.

Chapter 1 — From Requirement to Working System

Part I · Engineering Design Methodology

A national immunisation programme buys a remote alarm for the vaccine refrigerators in its 1,200 rural clinics. The brief to the contractor is one sentence: "a device that sends an SMS when the fridge temperature goes out of range." The contractor builds exactly that. An air-temperature sensor sits on the top shelf, a 2G modem sends a text whenever the reading crosses 8 °C, and the pilot units pass every bench test. In the field, each door opening during a vaccination session pushes the shelf air above 8 °C for a minute or two, so each clinic gets five to eleven messages a day. Within two weeks the nurses have blocked the number. In the third week a cleaner turns a thermostat knob to its coldest setting, a box of freeze-sensitive pentavalent vaccine spends the night at −2 °C, and the alarm that would have saved it is one unread message among sixty. Nothing in the hardware failed. The brief never said *whose* temperature (the air or the vaccine), *what* "out of range" means (how far, for how long), *who* must be told, *how fast*, or *what the user must not be bothered with*. Every one of those gaps was a requirement that nobody wrote down, and the first prototype's test plan could not catch them because it checked the device against a sentence instead of a set of numbers.

This chapter is about writing those numbers down before anything is built, and keeping them connected to the design and the tests until the system is accepted. Its running example, used through all of Part I, is that same product done properly: **CCM-1**, a battery-powered remote temperature monitor for vaccine refrigerators.

> **How does a vague need become a set of measurable, verifiable requirements, and where does the system end?**
> **How are requirements turned into design specifications, traced through the design, and closed by acceptance tests with pass/fail limits?**

Concept	Engineering destination
Problem definition	Project charters, design briefs, tender specifications, the first page of every design report
Stakeholder requirements	Interviews, site surveys and field data that decide what a product must do for whom
Functional requirements	"Shall" statements a design can be built from and a test can check
Nonfunctional requirements	Battery life, accuracy, environment, cost, security, usability: the numbers that size the hardware
System boundaries	Context diagrams; what is supplied, what is assumed, what belongs to someone else
Constraints	Regulations, shipping rules, networks, budgets and schedules that remove design options
Design specifications	Allocated budgets (error, latency, energy) and part-level limits that designers work to
Requirements traceability	Trace matrices, coverage checks and change-impact analysis in every regulated industry
Acceptance criteria	Factory and site acceptance tests, pilot demonstrations, statistically sized field trials

Roadmap. Define the problem with evidence and a value (1.1) → find the stakeholders and their needs (1.2) → write functional (1.3) and nonfunctional (1.4) requirements → draw the system boundary (1.5) and list the constraints (1.6) → allocate requirements into design specifications and budgets (1.7) → trace everything (1.8) → decide, before the build, what result will count as acceptance (1.9).

1.1 Problem definition

What a problem statement contains

A design starts from a problem, not a product. The contractor's brief named a product ("a device that sends an SMS"), and so it fixed a solution before anyone had stated what was wrong. A usable **problem statement** answers six questions, each with evidence:

1. **Who is affected?** The clinic nurse who must act, the district cold-chain officer who must respond, the programme that pays for spoiled vaccine.
2. **What is wrong, measured how?** Vaccines are exposed to temperatures that damage them, and nobody finds out until the monthly download of a 30-day recorder.
3. **How often and how badly?** Here the evidence is an audit of the existing 30-day temperature recorders: in one year, 17 % of refrigerators logged at least one freeze episode (at or below −0.5 °C for 60 minutes or more), and 6 % logged a heat episode (above +8 °C for 10 hours or more).
4. **Why now?** The national 2G network is to be switched off within three years. The programme has a budget line for the replacement, and a new pentavalent and pneumococcal schedule has raised the value of the stock per refrigerator.
5. **What does success look like?** A freeze or heat episode is acted on within the same working day, not discovered a month later.
6. **What is not the problem?** Refrigerator reliability, power supply and vaccine supply logistics are real, but they are other projects. Stating them here stops scope from drifting.

Point 6 matters as much as the other five. A team that lets "and while we are at it, monitor the solar power system" into the problem statement has silently doubled the project.

Putting a value on the problem

A problem with no value attached cannot be traded against cost, so the problem definition should end with a number.

Worked example — what the freeze problem costs. The programme has 1,200 refrigerators, each holding on average $3,000 of vaccine, of which 45 % by value is freeze-sensitive. From the audit, 17 % of refrigerators (204) freeze at least once a year. A shake test after a freeze episode condemns, on average, half of the freeze-sensitive stock. The annual freeze loss is therefore

$$ 204\times3000\times0.45\times0.5=\$137{,}700 . $$

Heat episodes occur in 6 % of refrigerators (72). Each condemns about 30 % of the stock, a loss of $72 \times 3000 \times 0.30 = \$64{,}800$. The total is $202,500 a year, before counting cancelled vaccination sessions. If remote alarms let staff save 70 % of that, the programme saves $141,750 a year. A two-year payback then allows $2 \times 141{,}750/1200 = \$236$ per refrigerator over two years. Take off $36 for 24 months of connectivity at $1.50 a month and $25 for installation, and the device itself may cost at most about $175. That number becomes the cost requirement R11 (Section 1.4). It was derived from the problem, not guessed.

> **Status of the principle: The cost of a requirement defect grows with the phase in which it is found**
> *Empirical.* Industry data in software, aerospace and automotive programmes consistently show a defect found at acceptance or in the field costing one to two orders of magnitude more to fix than the same defect found while writing requirements. The ratio varies widely by domain and is a planning heuristic, not a law.

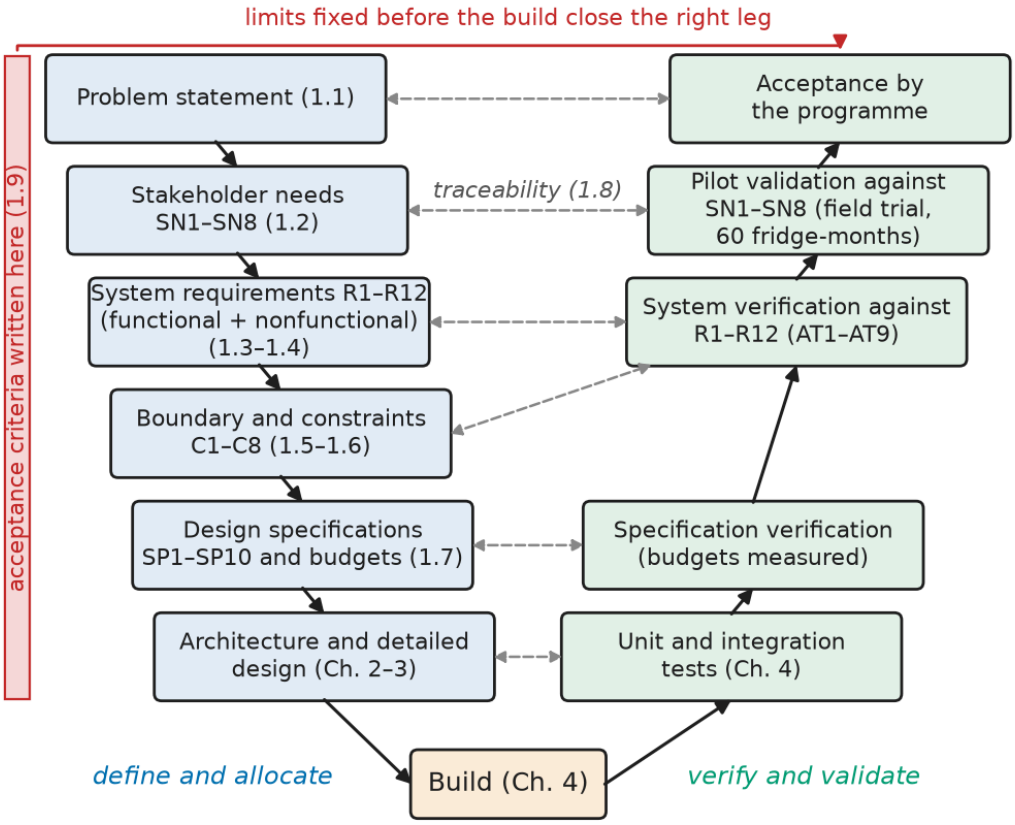


Figure 1.1 — The chain from problem to accepted system, with each left-hand artefact closed by a right-hand verification activity.

1.2 Stakeholder requirements

Finding the stakeholders

A **stakeholder** is anyone who uses, pays for, installs, maintains, regulates or is harmed by the system. The usual mistake is to talk only to the person who signs the order. For CCM-1 the list, and how each was heard, is:

Table 1.1 — CCM-1 stakeholders and elicitation methods.

Stakeholder	Role	How the needs were collected
Clinic nurse / health worker	Primary user; must act on an alarm	Site visits to 9 clinics, interviews, observation of a vaccination session
District cold-chain officer	Receives alarms; dispatches technicians	Interviews; review of 6 months of incident reports
Programme manager (ministry)	Pays; owns the vaccine loss	Workshop; audit data (Section 1.1)
Refrigerator technician	Installs device; diagnoses fridge faults	Interviews; ride-along on two repair trips
Procurement office	Tender, unit price, connectivity contract	Tender rules; past contract prices
Regulator / prequalification body	Accuracy, data integrity, device category	WHO PQS specifications for temperature monitoring devices (category E006); EN 12830 for recorder accuracy classes
Ministry IT and data office	Where data is stored, who may see it	National data-protection policy
Mobile network operators	Network availability and its lifetime	Coverage maps; 2G sunset notices; site survey

The site visits supplied the most important evidence. Figure 1.2 is a two-day trace of the kind the team recorded: shelf air jumps by 5 to 8 K at every door opening, while a sensor in a small vial of liquid, which stands in for a vaccine vial, barely moves. A temperature "of the fridge" does not exist. There is air, there is load, and there is whatever the sensor is coupled to. The prototype's failure in the opening paragraph was decided by this one fact, and only a site visit with a second sensor revealed it.

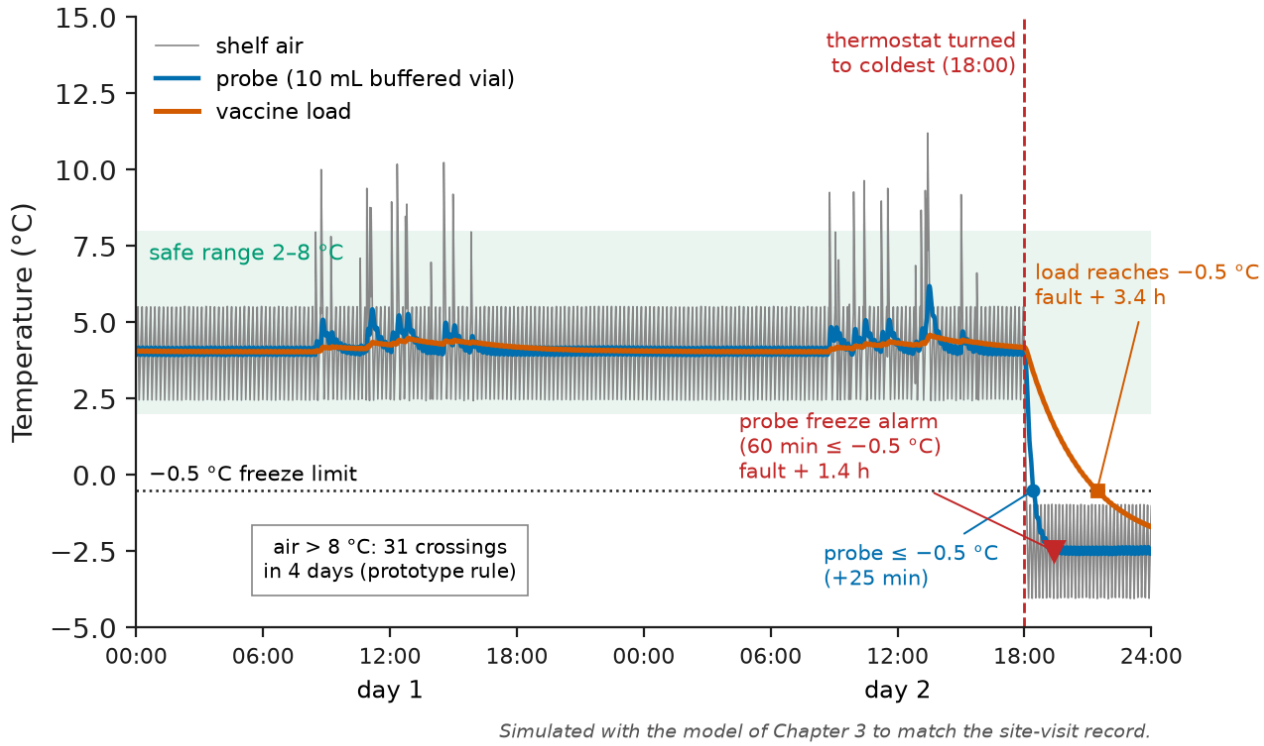


Figure 1.2 — Two days in a clinic refrigerator: shelf air versus a buffered probe and the vaccine load, including a thermostat fault on the second evening.

From stakeholder statements to stakeholder needs

What stakeholders say is raw material. "I want an SMS when the fridge is bad" is a wish, and it has a solution built into it. The analyst rewrites each statement as a **need**: what the stakeholder must be able to do or know, still in their language and still free of design.

Table 1.2 — CCM-1 stakeholder needs.

ID	Stakeholder	Need	Priority
SN1	Nurse	Know at a glance whether the vaccine is safe, and be warned of freezing or overheating in time to save it	Must
SN2	Cold-chain officer	Be told remotely and promptly of every alarm at every clinic, and see daily status of all refrigerators	Must
SN3	Programme manager	No battery change or routine maintenance for at least two years in the field	Must
SN4	Nurse	Not be disturbed by alarms caused by normal use (door openings, restocking)	Must
SN5	Regulator	A complete, accurate, tamper-evident temperature history	Must
SN6	Technician	Install in minutes and diagnose refrigerator faults from the record	Should
SN7	Procurement	Unit and running cost inside the budget of Section 1.1	Must
SN8	Ministry IT	Temperature data hosted in-country and accessible only to authorised staff	Must

Priorities follow the MoSCoW convention (must, should, could, won't this time). Two of the needs conflict. SN2 pulls towards reporting often and alarming early, while SN3 and SN4 pull towards reporting rarely and alarming late. Writing both down is how the conflict gets noticed. Sections 1.3 and 1.4 resolve it with numbers, and Chapter 2 resolves it again in the architecture.

1.3 Functional requirements

What makes a requirement usable

A **functional requirement** states something the system shall *do*: an input it accepts, an output it produces, a transformation, a reaction to an event. ISO/IEC/IEEE 29148 and the INCOSE *Guide to Writing Requirements* set out the properties of a good one. Each statement is *necessary* (it traces to a need), *singular* (one requirement, not three joined by "and"), *unambiguous*, *measurable*, *feasible* and *verifiable*, and it says *what*, not *how*. The quickest way to learn the rules is to repair bad requirements:

Table 1.3 — Repairing requirements.

Defective statement	Defect	Repaired requirement
The device shall alarm when the fridge is out of range	Undefined quantity, threshold and duration	The device shall raise a freeze alarm when the probe temperature has been at or below −0.5 °C continuously for 60 min
The device shall be accurate	Not measurable	The probe temperature error shall be within ±0.5 °C over −10 to +25 °C
Alarms shall be sent quickly and reliably	Two requirements; neither measurable	(a) An alarm notification shall leave the system within 15 min of the alarm condition; (b) for at least 95 % of alarm events
The device shall use an NTC thermistor	Design, not requirement	(Move to the design; keep only the accuracy and range requirement)
The battery shall last as long as possible	Not verifiable; no end point	The device shall operate for at least 2 years at 30 °C mean ambient with hourly reporting
The display should be easy to read	Subjective	Temperature digits shall be readable at 1 m by 9 of 10 test users in clinic lighting

> **Status of the principle: A requirement must carry its own verification**
> *Empirical.* Requirements written without a test in mind are the largest single source of disputes at acceptance. The practical test of a requirement is whether two engineers, working separately, would agree on whether a given measurement passes it.

CCM-1 functional requirements

Requirement IDs are issued in the order the requirements were agreed, so functional (Table 1.4) and nonfunctional (Table 1.6) IDs interleave; an ID is never reused.

Table 1.4 — CCM-1 functional requirements (each with a measurable target and a verification method: T = test, A = analysis, I = inspection, D = demonstration).

ID	Requirement	Target	Verified by	Need
R1	Measure the temperature of a vaccine-simulant probe	Range −30 to +50 °C; error ≤ ±0.5 °C over −10 to +25 °C, ≤ ±1.0 °C elsewhere	T: climatic chamber vs. calibrated reference	SN1
R2	Sample and record	Sample period ≤ 60 s; record period ≤ 5 min	T, I (log inspection)	SN1
R3	Raise freeze and heat alarms locally (sound, light, display)	Freeze: ≤ −0.5 °C for 60 min continuous; heat: > +8 °C for 10 h continuous	T: chamber profiles	SN1, SN2
R4	Notify alarms remotely; send daily status	Notification sent to officers ≤ 15 min after condition, for ≥ 95 % of events; daily report by 09:00	T: pilot and network test	SN2
R7	Retain the record locally	≥ 60 days of 5-min records; survives battery removal; each record integrity-checked	T, I	SN5
R8	Display temperature, alarm, battery and link status	Digits readable at 1 m (Table 1.3 criterion)	D with users	SN4
R9	Time-stamp records	Clock error ≤ 2 min per 30 days without network time	T	SN5

The thresholds in R3 are not invented. They are the defaults that WHO-prequalified 30-day recorders use, and keeping them means a CCM-1 alarm means the same thing to a nurse as the recorder she already knows. The probe in R1 is part of the requirement, not a design choice. The *quantity* to be measured is the temperature of something with the thermal behaviour of a vaccine vial, because Figure 1.2 shows that shelf air is the wrong quantity.

Deriving numbers from other numbers

Some requirements appear to be free choices and are not. The 15-minute remote-notification target in R4 came from the district officers (an alarm acted on within the same working session). The design then has to split that 15 minutes among the parts of the chain.

Worked example — the alarm-latency budget. From the moment the freeze condition becomes true (the 60th consecutive minute at or below −0.5 °C) to the moment the cloud service hands the notification to the operator's SMS gateway, the delay is the sum of the stages in Table 1.5. The sample period adds at most one period, because the condition can become true just after a sample. Evaluation on the device is immediate. The radio must wake from power-saving mode, attach if needed and send, with retries. The cloud service must then forward the alarm as an SMS or app notification.

Table 1.5 — Alarm-latency budget for R4.

Stage	Worst case allocated	Basis
Sampling quantisation	1 min	Sample period of R2
On-device evaluation and queueing	0.1 min	Firmware, Chapter 2
Radio wake, attach and send, up to 3 attempts	5 min	Cellular attach worst case at cell edge, Chapter 2
Cloud processing and hand-off to the SMS gateway	2 min	Cloud design target
Total	**8.1 min**	Requirement 15 min; **margin 6.9 min (46 %)**

The margin is kept on purpose. If the radio trade study in Chapter 2 lands on a network with longer attach times, the budget absorbs it. If it does not, the margin is there for the unknowns of the field.

> **Status of the principle: Worst-case latency of a serial chain is the sum of its stage worst cases**
> *Derived.* Delays in series add, so the sum of stage maxima bounds the end-to-end maximum. It is conservative, because the stage maxima rarely coincide. A statistical budget (percentiles combined by simulation) is tighter but needs stage delay distributions, which come later from test.

1.4 Nonfunctional requirements

Qualities that size the hardware

Nonfunctional requirements say *how well* the system must perform its functions, and under what conditions: performance, energy, environment, reliability, safety, security, usability, maintainability, cost and compliance. Some writers call them quality attributes. In electrical and electronic design these are usually the requirements that set the size of the parts. Accuracy decides the sensor and the calibration. Battery life decides the radio, the power architecture and the reporting rate. Environment decides the enclosure and the cells. The functional requirements decide what the firmware does, and the nonfunctional ones decide whether the hardware can afford to do it.

Table 1.6 — CCM-1 nonfunctional requirements.

ID	Category	Requirement	Target	Verified by	Need
R5	Energy	Battery life without replacement	≥ 2 years at 30 °C mean ambient (43 °C peak), hourly reporting, 4 alarm events per month	A (budget) + T (current profile)	SN3
R6	Usability / performance	False alarms from normal use	≤ 0.1 per refrigerator-month with up to 20 door openings per day of up to 60 s each	T: field pilot	SN4
R10	Maintainability	Installation	≤ 15 min, no tools other than a screwdriver; probe cable through the door seal without degrading it	D: 5 technicians	SN6
R11	Cost	Unit and running cost	≤ $175 per unit at 1,000 pcs; connectivity ≤ $1.50 per month	A: costed BOM, quotes	SN7
R12	Environment	Operating conditions	Body 0 to +43 °C, 95 % RH non-condensing; probe −30 to +50 °C; enclosure IP54 (IEC 60529)	T: chamber, IP test	SN3

R6 is the requirement the first prototype lacked. It is nonfunctional because no function produces "fewer false alarms". It is a property of the whole chain of probe, threshold, duration rule and door behaviour, and it can only be checked in the field with real doors and real nurses.

Security and data protection (SN8) still have no requirement in this table, and the traceability check in Section 1.8 will find that gap.

Turning a lifetime into a current

A lifetime requirement is useless to a circuit designer until it becomes an average current. The conversion needs the battery's *usable* charge, not its label capacity.

Worked example — from two years to a current allocation. Take a candidate pack of two AA lithium–thionyl-chloride cells (ER14505 class, 2.6 Ah each, 3.6 V) in parallel, 5.2 Ah on the label. Chapter 2 will confirm or replace this choice. Derate the label capacity three times. First, about 85 % is deliverable at the pulse loads and temperatures expected. Second, self-discharge at a hot-zone mean temperature is about 3 % a year, so 9 % over three years. Third, 10 % is held back as end-of-life reserve to drive the low-battery warning. The usable charge is

$$Q_{\text{use}} = 5.2 \times 0.85 \times (1 - 0.09) \times 0.90 = 3.62 \text{ Ah}.$$

Designing to the 2-year requirement alone would leave no margin for the unknowns. The team sets a **design target** of 3 years, 50 % above the requirement, and the allowed average current is

$$\bar{I} \le \frac{3.62 \text{ Ah}}{3 \times 8766 \text{ h}} = 138 \ \mu\text{A}.$$

At the requirement itself (2 years) the ceiling would be 206 µA. The 138 µA figure becomes design specification SP6, and every milliamp-second in Chapter 2's energy budget is charged against it.

> **Status of the principle: Battery life from usable charge and average current**
> *Approximate.* $t = Q_{\text{use}}/\bar{I}$ treats the battery as a charge reservoir. It holds when the derating factors are taken from data at the actual load profile and temperature. It fails for pulse loads beyond the cell's rating, where voltage collapse rather than charge sets the end of life. Chapter 3 checks that case by simulation, and *Electronics and Computing*, Ch. 25 §25.5 sets out the average-current budget itself.

1.5 System boundaries

The context diagram

A **system boundary** separates what the project designs, builds and is responsible for from what it uses, assumes or hands over. The tool for drawing it is the **context diagram** (Figure 1.3). The system is one box, every external entity is drawn around it, and every arrow that crosses the boundary is labelled with what flows (data, energy, material or human action). Each arrow is a future interface (Chapter 2), and each entity is someone else's responsibility that the design depends on.

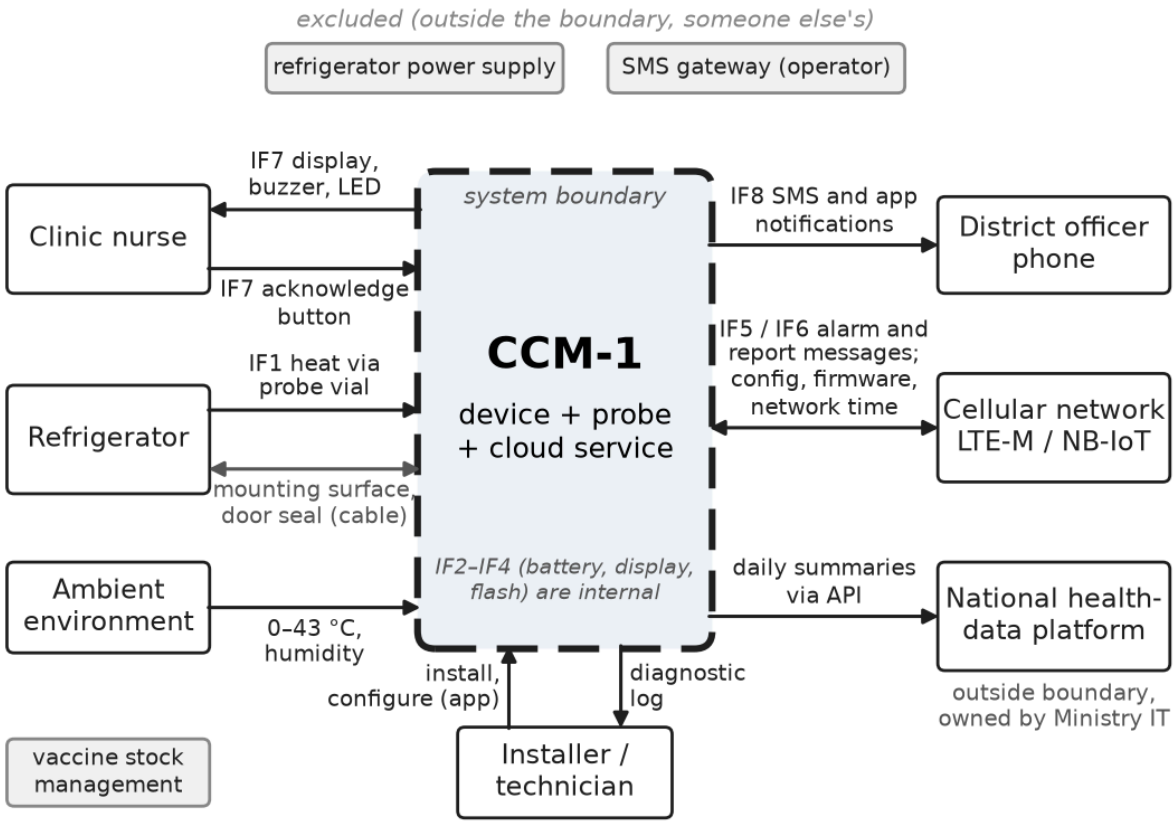


Figure 1.3 — Context diagram of CCM-1: the system boundary and every flow that crosses it.

In scope, out of scope, and assumed

Drawing the boundary forces three lists. Each item on the "assumed" list is a risk, because the design depends on something the project does not control.

Table 1.7 — CCM-1 scope.

In scope (designed and verified by the project)	Out of scope (someone else's)	Assumed (depended on, not controlled)
Device electronics, firmware, enclosure	Refrigerator, its thermostat and power supply	LTE-M or NB-IoT coverage at ≥ 90 % of clinics (verified by survey, Chapter 2)
Probe, buffer vial, flat cable	Vaccine stock management system	SMS gateway delivers within 2 min (operator SLA)
Cloud service: message intake, alarm forwarding, dashboard	National health-data platform	Clinic has a person who can respond during working hours
Installation kit and instructions	Refrigerator repair	Mobile operator keeps LTE-M running ≥ 7 years
Factory calibration and test	Training of nurses (programme does it)	Ambient within R12

The boundary is also a decision about where the requirements stop. CCM-1 does not have to *prevent* freezing, because that is the refrigerator's job. It has to *detect and report* freezing, and so its requirements end at the delivery of the alarm. Whether the nurse moves the vaccine is outside the system and belongs to the programme's training and procedures. This is why R4 ends at "notification sent", with delivery to the phone an operator assumption, and not at "vaccine saved".

1.6 Constraints

Requirements versus constraints

A **constraint** is a limit on the *solution* imposed from outside the problem: a regulation, a standard the customer mandates, a network that exists, a budget, a schedule, a team, a part already in stock. A requirement says what the system must achieve. A constraint removes options for achieving it, and it is not negotiable with the design team. Mixing the two causes trouble. A constraint written as a requirement ("shall use 2G") stops the team looking for better solutions. A real constraint left unwritten (a shipping rule) is found when the first container is refused at the airport.

Table 1.8 — CCM-1 constraints.

ID	Constraint	Source	Consequence for the design
C1	2G networks are switched off within 3 years	Operator notices	2G GSM is eliminated as a long-term radio. It still enters the trade study (Chapter 2) as a reference
C2	Each lithium-metal cell ≤ 1 g lithium content	IATA Dangerous Goods Regulations, PI 970 Section II (cells contained in equipment), to avoid fully regulated Class 9 shipping	Rules out D-size Li-$SOCl_2$ (about 4.5–5 g Li). AA-size (about 0.7 g) allowed
C3	Cells must pass UN 38.3	UN Manual of Tests and Criteria	Buy cells only from suppliers with test summaries
C4	Radio and EMC conformity	ETSI EN 301 908-13 (LTE user equipment), EN 301 489-1/-52 (EMC), operator certification (GCF/PTCRB)	Use a pre-certified cellular module or SiP. A discrete RF design would add months
C5	Electrical safety of the product	IEC 62368-1	Battery-powered SELV design; limited-power source analysis
C6	Data hosted in-country	National data-protection law	The cloud service runs on in-country infrastructure (feeds SN8)
C7	Pilot units in 9 months; team of 3 engineers	Programme plan	Buy modules rather than design; reuse an existing server stack
C8	Device must fit WHO PQS category E006 expectations	Procurement rule	Alarm defaults of R3; accuracy class; record format

Constraint C2 is a good example of a fact that no amount of electrical analysis would reveal. The energy budget alone would pick a single D-size Li-$SOCl_2$ cell (19 Ah, plenty of margin). The air-freight rule makes that cell expensive to ship by air in the quantities needed, so the design moves to AA cells. That move in turn creates the pulse-current problem that Chapter 3 has to simulate. One unwritten constraint would have cost a redesign. Written down on day one, it costs a row in a table.

1.7 Design specifications

From requirement to specification

Requirements describe the system as a black box. **Design specifications** describe how much of each requirement every part of the design is allowed to use. They are written by the design team, for the design team, and they appear once a first architecture exists. The tool for writing them is **allocation**: a system-level number (an error, a latency, an energy, a cost, a mass) is split into contributions, each contribution becomes a limit on one subsystem, and the sum, combined by the right rule, must fit inside the requirement with a margin. Table 1.5 was already an allocation, and Section 1.4 turned R5 into a current. The accuracy requirement R1 shows the method in full.

Worked example — the measurement error budget

R1 asks for ±0.5 °C over −10 to +25 °C. The error at the display has several independent sources. The team lists them, gives each an allocated limit, and combines them twice. A worst-case linear sum covers the case where every error is at its limit with the same sign. A root-sum-square (RSS) combination covers independent errors of random sign, following the GUM.

Table 1.9 — Error budget for R1 (limits, °C).

Contributor	Allocated limit	Becomes specification
Sensor element, interchangeability over −20 to +30 °C	±0.10	SP1
Digitisation (quantisation)	±0.008	SP1
Heat conducted along probe cable into vial	±0.05	SP10
Gradient inside buffer vial	±0.05	SP2
Calibration reference uncertainty at factory test	±0.10	Test spec (Chapter 4)
Long-term drift over 3 years	±0.10	SP1
Display rounding to 0.1 °C	±0.05	SP9
Worst-case sum	**±0.458**	Requirement ±0.5: margin 0.042
RSS of limits	**±0.194**	Requirement ±0.5: margin 0.31

Treating each limit as the half-width of a rectangular distribution, the combined standard uncertainty is $u_c = 0.194/\sqrt{3} = 0.112$ °C, and the expanded uncertainty with coverage factor $k = 2$ is $U = 0.22$ °C. The design passes on both counts. The worst-case margin is thin, though, and it tells the team where to spend effort. The three 0.10 °C items (sensor, reference and drift) carry most of the budget, so the sensor chosen in Chapter 2 must meet SP1 and the factory reference must be better than ±0.10 °C.

> **Status of the principle: Root-sum-square combination of independent errors**
> *Derived.* For independent error sources, variances add (the GUM, JCGM 100, law of propagation of uncertainty for a linear model with unit sensitivities). The RSS is invalid for correlated errors, such as two contributors driven by the same temperature, and it is too optimistic for small numbers of contributors that are each bounded. In those cases the worst-case sum is the safe bound. The mathematics is in *The Mathematics Universe*, Ch. 24 §24.8, and Chapter 7 of this book applies it to measurement chains.

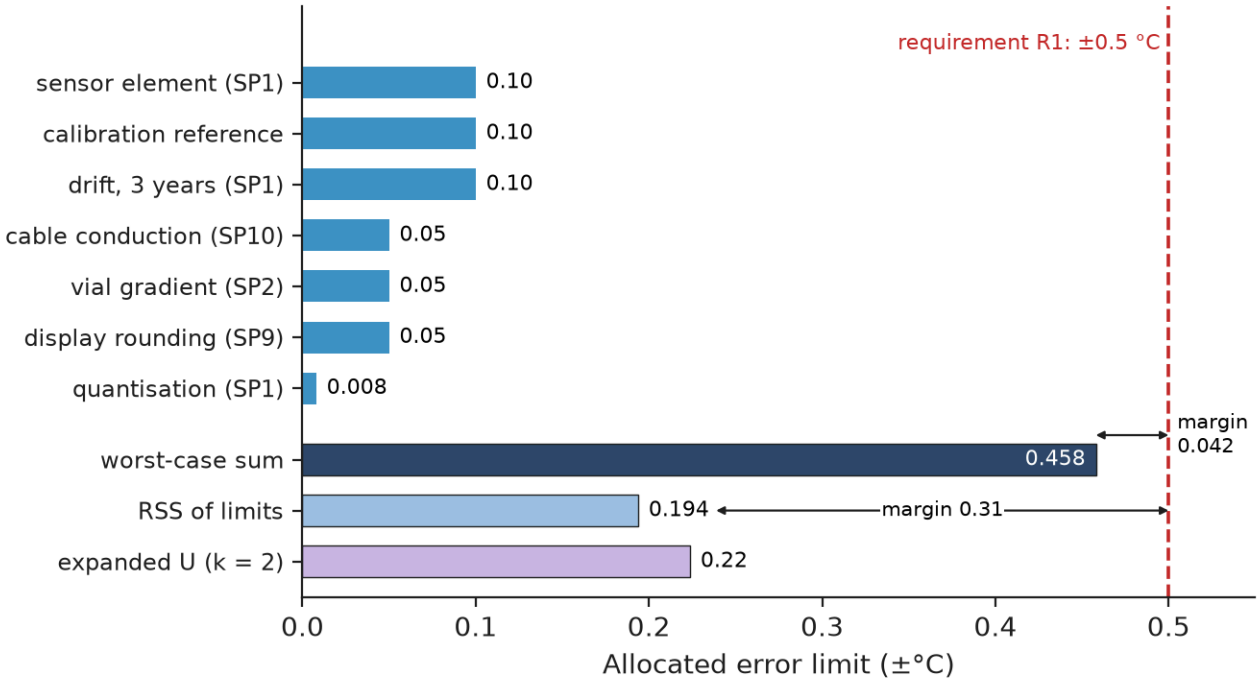


Figure 1.4 — Allocated error budget for R1: contributions, worst-case sum and root-sum-square against the requirement.

The specification set

Allocation produces the specification table that the designers of Chapters 2 and 3 work to. Each specification names its parent requirement, so a change to the requirement shows which specifications move.

Table 1.10 — CCM-1 design specifications (first issue).

ID	Specification	Parent
SP1	Sensor element: error ≤ ±0.10 °C over −20 to +30 °C; drift ≤ 0.03 °C/year; resolution ≤ 0.016 °C	R1
SP2	Buffer vial: 10 ± 1 mL glycol–water; time constant 15–25 min in still air	R1
SP3	Sample period 60 ± 1 s; record period 300 s	R2
SP4	Alarm evaluation every sample; duration counted in whole samples; re-arm hysteresis 0.3 °C	R3
SP5	Radio: up to 3 send attempts within 5 min; alarm messages bypass the hourly queue	R4
SP6	Average supply current ≤ 138 µA at 30 °C ambient, including alarms and updates	R5
SP7	Alarms evaluated on probe temperature only; air temperature never used for alarms	R6
SP8	Non-volatile store ≥ 1 MB; 8-byte records with sequence number; CRC per page	R7
SP9	Display: reflective (e-paper), digits ≥ 12 mm high, value rounded to 0.1 °C	R8
SP10	Probe cable: flat section ≤ 0.6 mm thick over the door seal; length 1.5 m	R10

SP7 is where the opening story's failure is turned into a rule. It is also the specification most likely to be challenged later ("we have an air sensor on the board anyway, why not use it?"), and the trace back to R6, SN4 and Figure 1.2 is the answer.

1.8 Requirements traceability

The trace chain

Traceability is the recorded link from every need to the requirements that serve it, from every requirement to the specifications and design elements that implement it, and to the tests that verify it. In regulated industries it is mandatory evidence. Functional safety (IEC 61508, ISO 26262), medical devices (IEC 62304, ISO 13485) and aerospace (DO-178C) all demand it. Elsewhere it is simply the cheapest way to answer three questions that every project meets. Is every need covered? Is every requirement tested? What breaks if this requirement changes?

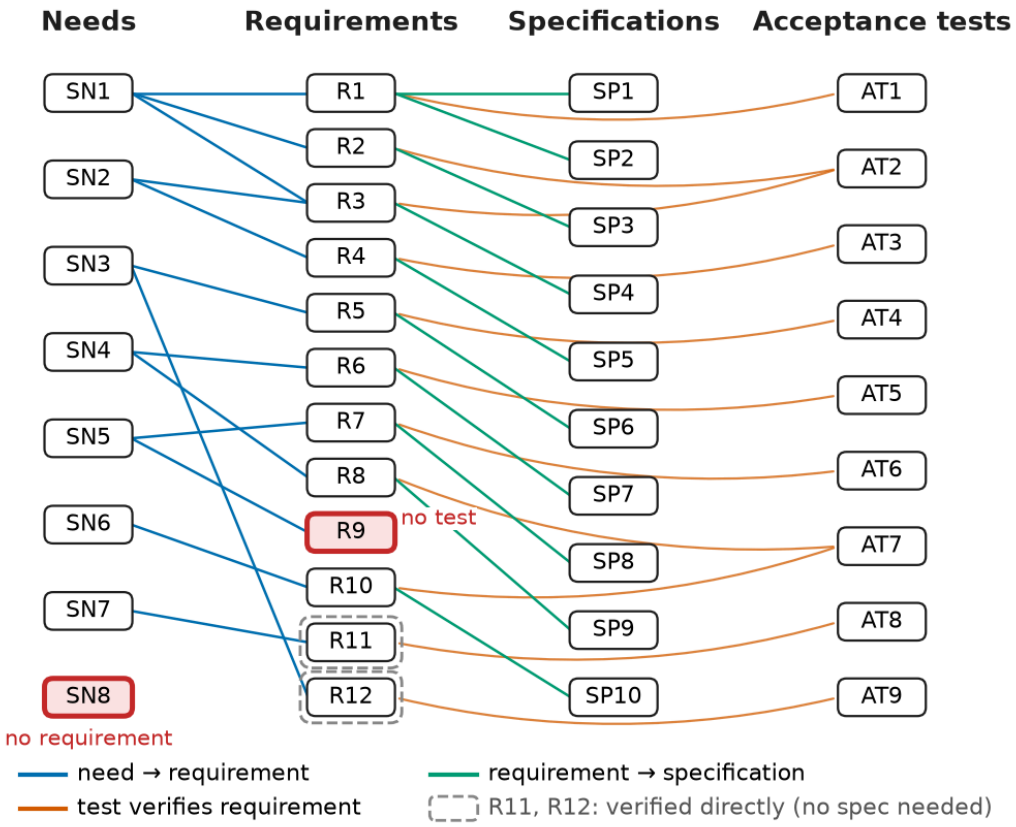


Figure 1.5 — Traceability graph of CCM-1: needs, requirements, specifications and acceptance tests, with the gaps the coverage check found.

A machine-checkable trace

A trace matrix kept in a spreadsheet goes stale within weeks. Kept as data, in the repository next to the design and checked on every change, it stays true. Chapter 5 covers the tooling. The essential check is short:

```
 # Trace check for CCM-1: needs -> requirements -> specs -> tests.
needs = ["SN1", "SN2", "SN3", "SN4", "SN5", "SN6", "SN7", "SN8"]
req_from = {  # requirement -> stakeholder needs it serves
    "R1": ["SN1"], "R2": ["SN1"], "R3": ["SN1", "SN2"], "R4": ["SN2"],
    "R5": ["SN3"], "R6": ["SN4"], "R7": ["SN5"], "R8": ["SN4"],
    "R9": ["SN5"], "R10": ["SN6"], "R11": ["SN7"], "R12": ["SN3"],
}
spec_of = {  # design specification -> requirement it refines
    "SP1": "R1", "SP2": "R1", "SP3": "R2", "SP4": "R3", "SP5": "R4",
    "SP6": "R5", "SP7": "R6", "SP8": "R7", "SP9": "R8", "SP10": "R10",
}
tests = {  # acceptance test -> requirements it verifies
    "AT1": ["R1"], "AT2": ["R2", "R3"], "AT3": ["R4"], "AT4": ["R5"],
    "AT5": ["R6"], "AT6": ["R7"], "AT7": ["R8", "R10"], "AT8": ["R11"],
    "AT9": ["R12"],
}

served = {n for rs in req_from.values() for n in rs}
tested = {r for rs in tests.values() for r in rs}
refined = set(spec_of.values())
unknown = {r for rs in tests.values() for r in rs} - req_from.keys()

print("needs without a requirement:", sorted(set(needs) - served))
print("requirements without a test: ", sorted(req_from.keys() - tested,
                                              key=lambda r: int(r[1:])))
print("requirements without a spec: ", sorted(req_from.keys() - refined,
                                              key=lambda r: int(r[1:])))
print("tests citing unknown reqs:   ", sorted(unknown))
cov = len(req_from.keys() & tested) / len(req_from)
print(f"verification coverage: {cov:.0%}")
```

```
 needs without a requirement: ['SN8']
requirements without a test:  ['R9']
requirements without a spec:  ['R9', 'R11', 'R12']
tests citing unknown reqs:    []
verification coverage: 92%
```

The output says three different things, and each needs a different response:

- **SN8 has no requirement.** In-country hosting and access control were captured as a need and as constraint C6, but nobody wrote the security requirement. The fix is a new requirement R13: device identity by per-device credentials, encrypted transport, role-based access on the dashboard, verified by a security test. Chapter 11 owns its detail.
- **R9 has no test.** Clock accuracy was assumed to follow from the radio's network time. A device that cannot attach for a month would then drift unchecked. The fix is to add an AT10: 30 days in a chamber with the radio disabled, compare time stamps with a reference clock, limit 2 min.
- **R9, R11 and R12 have no specification.** For R11 (cost) and R12 (environment) that is correct, since they are verified directly against the requirement. For R9 it is another symptom of the same gap. A specification for the crystal (±20 ppm gives 52 s per 30 days) would have exposed it.

A coverage check does not judge whether a test is *good*. It only shows which links are missing. That is why the second check in a review is a person reading each requirement next to its test and asking whether the test can actually fail.

Change impact

The second use of the trace appears when a requirement changes. In Chapter 3 the sensitivity analysis will show that the continuous-duration freeze rule of R3 occasionally lets the load freeze before the alarm. The proposed fix is an additional freeze warning based on a 60-minute moving average. Following the trace from R3 gives SP4 (alarm logic), AT2 (alarm test profiles), SN1 and SN2 (users to consult) and, through SP4, the firmware module that implements it (Chapter 2). This set is the *impact* of the change. It is what has to be redesigned, retested and re-approved, and the trace produces it in seconds instead of in a meeting.

1.9 Acceptance criteria

What acceptance means

Acceptance criteria are the conditions, agreed with the customer *before* the design is built, under which the system will be accepted. Each states the test, the conditions, the sample, the pass/fail limit and who witnesses it. They close the V of Figure 1.1, so writing them at the start is the main defence against the opening story. The contractor's prototype would have failed acceptance criterion AT5 on its first day in a clinic, if AT5 had existed.

Table 1.11 — CCM-1 acceptance tests.

Test	Verifies	Conditions and sample	Pass/fail limit
AT1	R1	5 units in a climatic chamber at −25, −10, 0, +5, +8, +25, +45 °C, against a reference PRT with U ≤ 0.05 °C	Every reading within ±0.5 °C (−10 to +25 °C) and ±1.0 °C elsewhere
AT2	R2, R3	Chamber profiles: 59 min at −1 °C (no alarm), 61 min at −1 °C (alarm), 9 h 50 min at +9 °C (no alarm), 10 h 10 min (alarm)	Correct decision in all 4 profiles on all 5 units; alarm time within ±1 sample
AT3	R4	50 injected alarms across 10 pilot sites	≥ 95 % of notifications sent ≤ 15 min
AT4	R5	Current profile over 24 h with a power analyser at 25 and 43 °C, plus the budget analysis	Projected life ≥ 2 years at worst-case coverage
AT5	R6	Field pilot: 20 refrigerators × 3 months	≤ 2 false alarms in 60 refrigerator-months (see below)
AT6	R7	Fill memory, remove battery 24 h, read back	100 % of records intact; CRC and sequence checks pass
AT7	R8, R10	10 users read display at 1 m; 5 technicians install	≥ 9/10 correct readings; every install ≤ 15 min, door seal leak test pass
AT8	R11	Costed BOM at 1,000 pcs with 2 quotes per major part; connectivity contract	≤ $175 unit; ≤ $1.50 per month
AT9	R12	Damp heat 43 °C / 95 % RH 96 h; IP54 test per IEC 60529	Function test pass after each

Sizing a field trial

Some criteria are statistical, and a careless pilot can neither pass nor fail them. R6 asks for no more than 0.1 false alarms per refrigerator-month. A pilot that sees no false alarm in one month at five clinics has shown almost nothing. The question is how many refrigerator-months of exposure are needed, and how many false alarms may be seen, before the rate can be claimed with a given confidence.

Worked example — how long a pilot must run. Model false alarms as a Poisson process with rate λ per refrigerator-month (independent events, constant rate). If k false alarms are seen in total exposure T, the one-sided upper confidence bound on λ at confidence $1-\alpha$ is $\chi^2_{1-\alpha,\,2(k+1)}/(2T)$ (*The Mathematics Universe*, Ch. 20 §20.4 and §20.7). To demonstrate $\lambda \le 0.1$ at 90 % confidence, the exposure must be at least

$$T = \frac{\chi^2_{0.90,\,2(k+1)}}{2 \times 0.1}.$$

For $k = 0$ this gives 23.0 refrigerator-months, for $k = 1$ 38.9, and for $k = 2$ 53.2. A 20-refrigerator, 3-month pilot (60 refrigerator-months) can therefore tolerate up to 2 false alarms and still demonstrate R6. That is the limit written into AT5. The same logic sizes detection tests. To show that at least 95 % of freeze events are detected, with 90 % confidence and no misses allowed, requires $n \ge \ln 0.10 / \ln 0.95 = 44.9$, so 45 injected events. AT3 uses 50.

> **Status of the principle: Exposure needed to demonstrate a failure rate**
> *Derived.* For a constant-rate Poisson process the chi-square bound is exact. It does not hold if false alarms cluster (one bad installation producing many), so the pilot must also be read site by site. A site with repeated false alarms is a failure mode to diagnose, not a sample to average.

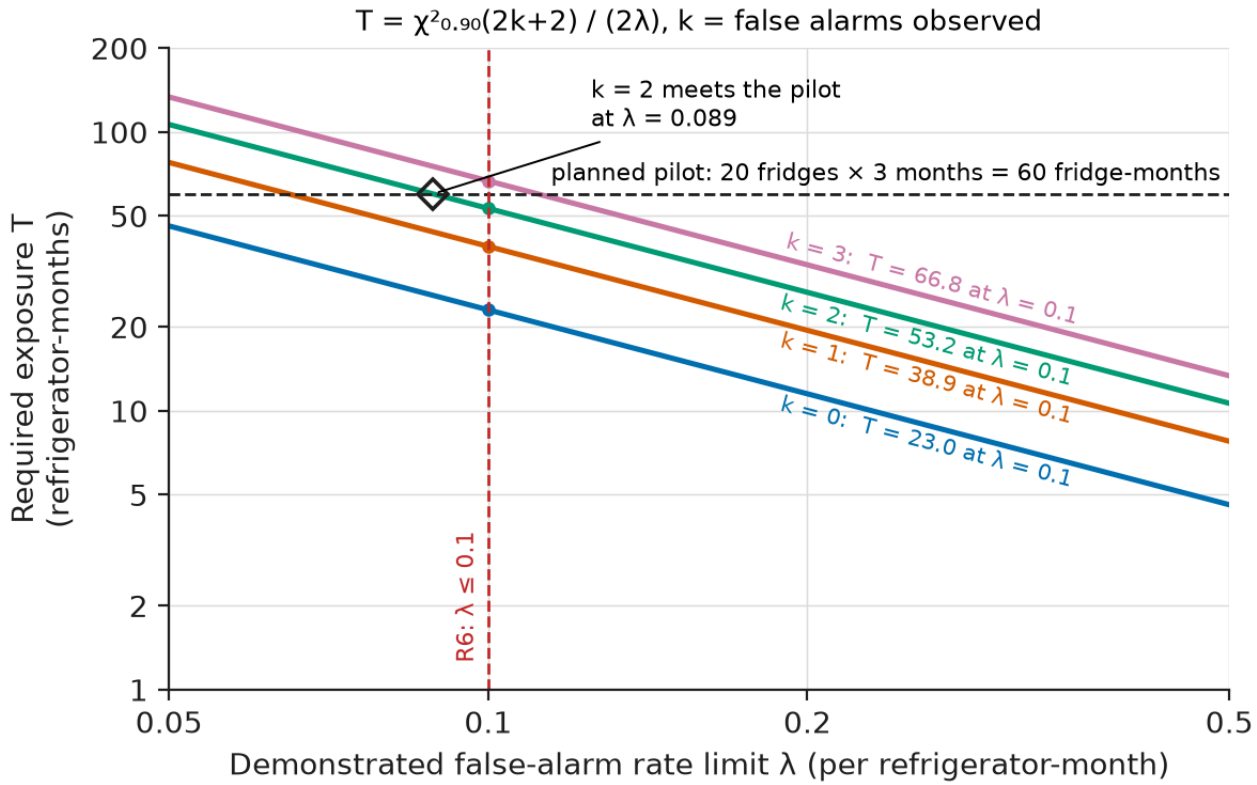


Figure 1.6 — Field exposure needed to demonstrate a false-alarm rate at 90 % confidence, against the number of false alarms observed.

Verification of the first prototype, in hindsight

Running the contractor's Rev A prototype against Table 1.11 shows what written criteria would have caught. The figures for Rev A come from its pilot logs and a bench re-test.

Table 1.12 — Rev A prototype against the CCM-1 acceptance criteria.

Requirement	Test	Limit	Rev A result	Pass/fail
R1 accuracy	AT1	±0.5 °C	±0.3 °C (air sensor, bench)	Pass, but measures the wrong quantity
R3 alarm logic	AT2	4 of 4 profiles	Alarmed instantly at the threshold: 2 of 4 correct	Fail
R4 remote latency	AT3	≥ 95 % ≤ 15 min	2G SMS: 98 % ≤ 15 min	Pass (but C1: network ends)
R5 battery life	AT4	≥ 2 years	Mains adapter only; 0 h on outage	Fail
R6 false alarms	AT5	≤ 0.1 per fridge-month	5–11 per fridge-*day*	Fail
R7 retention	AT6	60 days	None stored on device	Fail

The prototype was built competently to a brief that had none of these rows. Six requirements, written in a week, would have changed what was built.

Failure analysis of the requirements process

Requirements fail in recognisable ways, and each failure has a detection method that costs little if it is used early. Table 1.13 applies FMEA thinking to the process itself, using CCM-1's history.

Table 1.13 — FMEA of the requirements process.

Failure mode	Effect	Cause	Detection	Mitigation
Solution stated as the problem	Wrong product built well (Rev A)	Brief written by the buyer as a product description	Problem-statement review: does it name a product?	Six-question problem statement (Section 1.1)
Wrong measurand	Alarms on air, not vaccine; false alarms	No site data	Site visit with two sensors (Figure 1.2)	Measure the real environment before writing R1
Missing nonfunctional requirement	Users disable the system	Only functions were listed	Stakeholder walk-through of a day of use	Quality-attribute checklist (Table 1.6 categories)
Unwritten constraint	Late redesign (cell shipping)	Regulations not reviewed	Constraint workshop with logistics and compliance	Constraint table at kick-off (Table 1.8)
Untestable requirement	Disputes at acceptance	Adjectives instead of numbers	"Two engineers" test (Section 1.3)	Each requirement written with its test
Need with no requirement	Gap found by auditor or in field (SN8)	Trace not maintained	Automated coverage check (Section 1.8)	Trace as data, checked on every change
Statistically meaningless acceptance	Pilot neither passes nor fails	Trial sized by budget, not by statistics	Compute required exposure first	Size the trial from the claim (Section 1.9)

What remains open

Open question	Where it is answered
How are the requirements turned into an architecture, with interfaces, budgets and a radio and battery chosen by trade study?	Chapter 2
Is the freeze rule of R3 safe for every refrigerator fault, and will the AA cells hold up under radio pulses at end of life?	Chapter 3 (simulation, sensitivity analysis)
How are the acceptance tests of Table 1.11 planned, instrumented, calibrated and repeated as regression tests?	Chapter 4
How are requirements, trace data and test reports kept under version control and released?	Chapter 5; tooling in Chapter 29
How is the probe's sensor chosen and calibrated, and how is the uncertainty budget of Table 1.9 verified?	Chapters 6 and 7
How are device identity, provisioning and transport security (the missing R13) specified and built?	Chapter 11
How does a full project report carry requirements through to final acceptance?	Chapter 40
What theory supports the energy budget and low-power firmware?	*Electronics and Computing*, Ch. 25 and Ch. 28

Next: Chapter 2 takes CCM-1's requirements and builds its architecture: the functions, blocks, interfaces and flows of data, energy and control, and the trade studies that choose the radio and the battery.

Chapter 2 — System Architecture

Part I · Engineering Design Methodology

The second CCM-1 prototype had a good requirement set (Chapter 1) and a bad architecture. One microcontroller ran everything in a single loop: it read the probe, updated the display, evaluated the alarms and then called the cellular modem's blocking "send" function. On the bench, with a strong signal, a send took 2 s. At a clinic at the edge of the cell it took up to 4 min, with retries. During those minutes the loop did not run, so no samples were taken, the buzzer could not sound, and a freeze alarm that was due could not be raised. In the energy budget the radio had been entered at one average figure, measured at the bench, so the cell-edge clinic drained its battery four times faster than planned. Both failures were architectural. The local alarm path depended on the radio path, and the energy flow had been estimated for one operating point. Neither problem showed up in any single module's test. They existed only in the way the blocks had been connected.

This chapter turns CCM-1's requirements into an architecture: the functions, the blocks that perform them, the interfaces between blocks, and the flows of data, energy and control through them. It then makes the major choices by trade study, and records the decisions so that they can be checked, revisited and handed over.

> **How is a set of requirements decomposed into functions and allocated to hardware and software blocks with well-defined interfaces?**
> **How are data, energy and control flows budgeted through the architecture, and how are the big choices made and recorded so that they survive the first surprise?**

Concept	Engineering destination
Functional decomposition	Function trees; completeness checks against requirements; the basis of functional FMEA
Block diagrams	Physical architecture of boards, modules, cables and services
Hardware–software partitioning	Which job goes to a peripheral, an external chip, firmware or the cloud
Interfaces	Interface control documents: electrical, protocol, mechanical and human
Data flow	Sample rates, record formats, storage and airtime budgets
Energy flow	Average-current and peak-power budgets, battery choice and life
Control flow	Operating modes, state machines, independence of safety paths
Modularity	Variants, reuse, swappable radios and probes, testable firmware modules
Trade-off analysis	Weighted decision matrices, sensitivity of a choice to its weights, Pareto fronts
Architecture documentation	Views, ICDs and architecture decision records that survive staff changes

Roadmap. Decompose what the system must do (2.1) → draw what it is made of (2.2) → decide what is hardware and what is software (2.3) → define the interfaces (2.4) → budget the data (2.5), energy (2.6) and

control (2.7) flowing across them → organise it into modules (2.8) → choose among alternatives by trade study (2.9) → record the architecture (2.10).

2.1 Functional decomposition

Functions before boxes

An architecture starts with what the system must *do*, written without reference to any part. A **functional decomposition** breaks the top-level function into subfunctions, each a verb and a noun ("convert temperature", "store record"), until every leaf can be allocated to one block and verified by one test. Missing behaviour then shows up before any hardware exists, and the same function tree serves every candidate physical architecture, so the trade studies of Section 2.9 compare like with like.

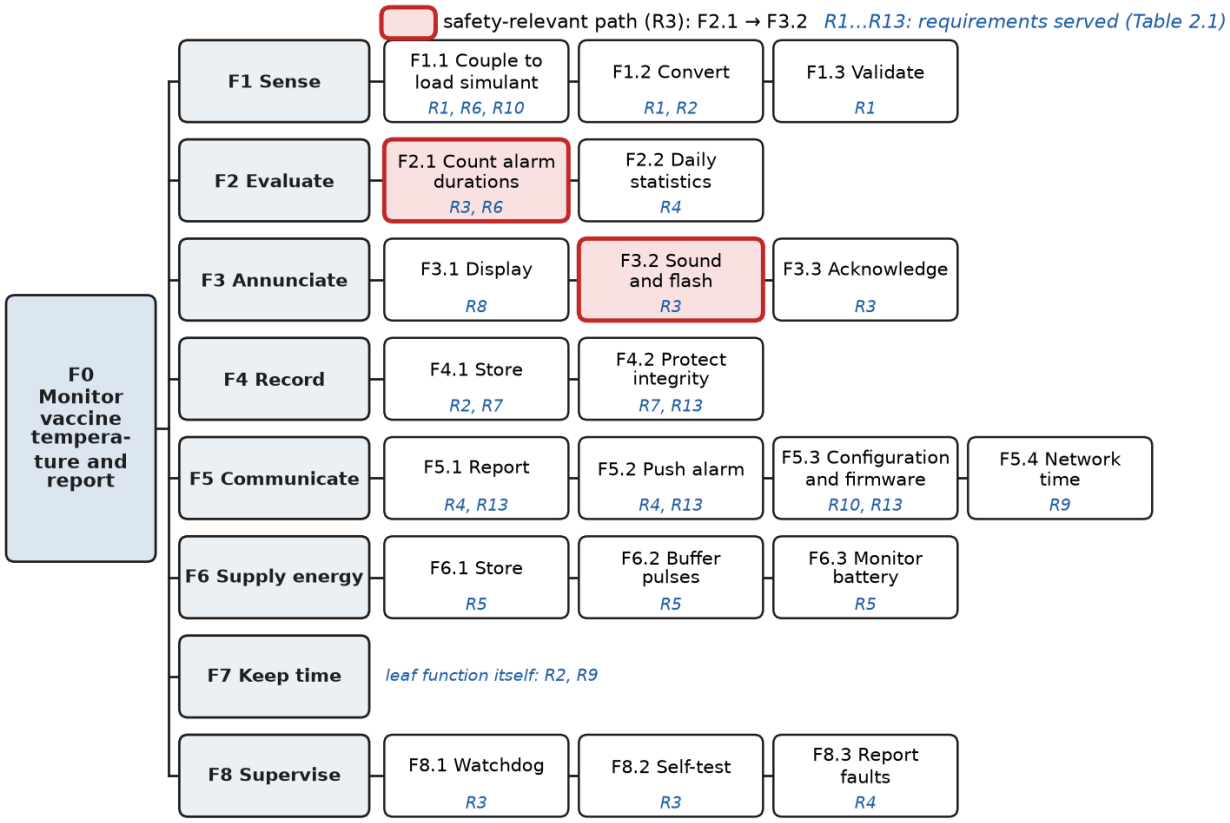


Figure 2.1 — Functional decomposition of CCM-1, from the top-level function to allocatable leaves.

Allocating requirements to functions

Each requirement of Chapter 1 must land on at least one function, and each function must serve at least one requirement. A function that serves none is gold-plating. A requirement served by none is a gap. Table 2.1 is the check.

Table 2.1 — Requirement-to-function allocation for CCM-1.

Requirement	Functions	Note
R1 accuracy, range	F1.1, F1.2, F1.3	Coupling (the vial) is part of the measurement
R2 sample, record	F1.2, F4.1, F7	
R3 local alarms	F2.1, F3.2, F3.3, F8.1, F8.2	Safety-relevant path
R4 remote alarms, daily status	F2.2, F5.1, F5.2, F8.3	
R5 battery life	F6.1, F6.2, F6.3 (and every function's energy)	An emergent property: allocated by budget, Section 2.6
R6 false alarms	F1.1, F2.1	Set by coupling and duration rule together
R7 retention, integrity	F4.1, F4.2	
R8 display	F3.1	
R9 time stamps	F7, F5.4	
R10 installation	F1.1 (cable), F5.3 (configuration)	Mechanical and procedural
R11 cost	All	Allocated by BOM budget
R12 environment	All	Allocated by component ratings and enclosure
R13 security (added in Chapter 1)	F5.1–F5.3, F4.2	Detail in Chapter 11

R5, R11 and R12 have no single function. They are **emergent properties** of the whole system, so they are allocated by budgets (energy, cost) and by derating rules (environment) instead of by function. This is where most architecture failures hide. The opening story's battery drain belonged to no module and to every module at once.

2.2 Block diagrams

From functions to physical blocks

The **physical architecture** assigns functions to blocks: chips, modules, boards, cables, enclosures, software services. A block diagram shows the blocks and the interfaces between them. It is not a schematic. Each block hides its inside, and each line is a named interface with a type (power, digital bus, RF, mechanical, human, network). Figure 2.2 is CCM-1's.

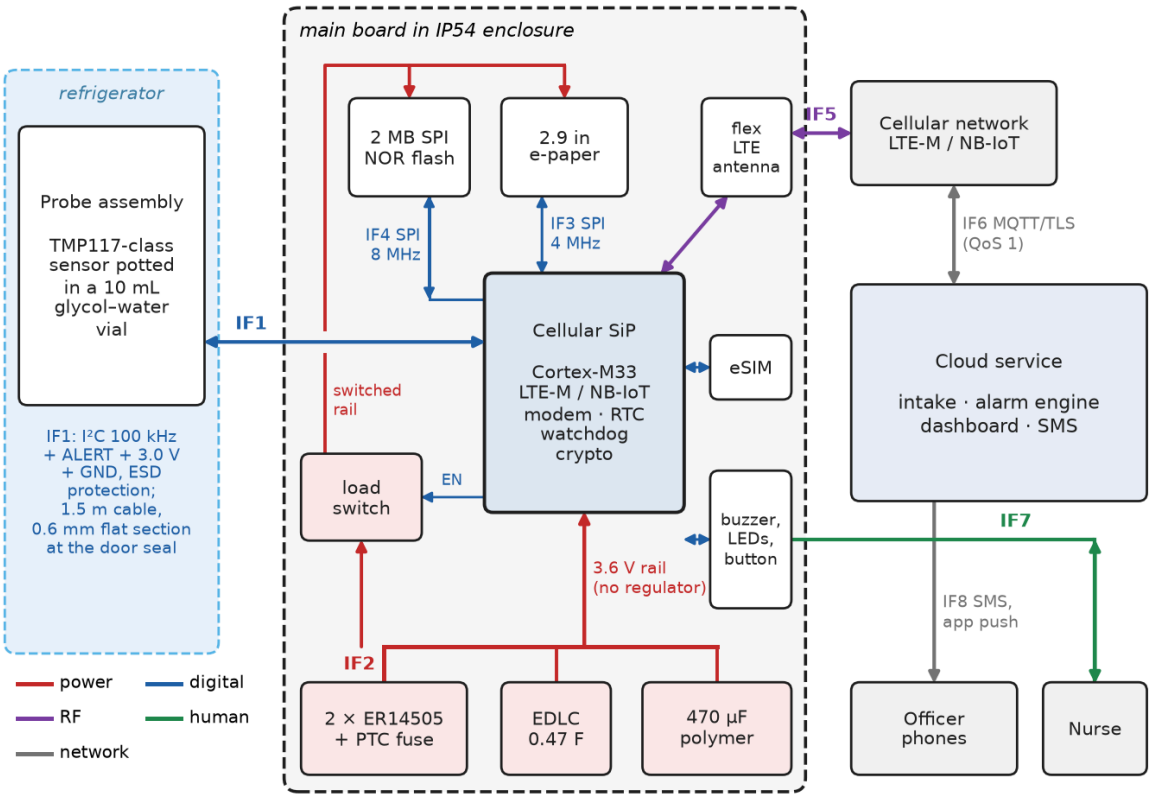


Figure 2.2 — Physical architecture of CCM-1 with every interface labelled.

Several choices are already visible in the diagram, and each has a reason that traces to a requirement:

- **One cellular SiP** (system-in-package: application processor, LTE-M/NB-IoT modem and RF front end in one certified part) replaces a separate microcontroller and modem, removing one firmware image, one interface and one sleep current, and bringing radio certification (C4).
- **No voltage regulator.** The lithium–thionyl-chloride pack sits at 3.3 to 3.65 V under load, and the SiP accepts 3.0 to 5.5 V. A regulator would only add quiescent current and loss.
- **A pulse buffer** (an electric double-layer capacitor, EDLC) carries the radio's current bursts, because AA bobbin cells cannot. Chapter 3 sizes it.
- **A digital probe sensor.** The conversion happens in the vial, and only digital signals cross the cable and the door seal (Section 2.3).
- **Display and flash behind a load switch.** Their standby currents are larger than the SiP's in power-saving mode (Section 2.6).

2.3 Hardware–software partitioning

The rule and its reasons

> **Boundary.**
> Partitioning at microcontroller level (periodic, bit-level and bulk-transfer jobs to peripherals) is set out in *Electronics and Computing*, Ch. 25 §25.2. The quantitative version with accelerators, offload break-even and FPGAs is in Ch. 35 §35.1. This section applies those rules at system level, where the options also include an external chip and the cloud.

At system level a function can live in an external chip, an on-chip peripheral, the firmware or the cloud. Book 4's reasons for moving a job toward hardware (periodic and fast, bit-level, bulk data) gain three more at this level:

1. **Independence.** A function that must still work when another fails belongs in a block that does not share that block's failure modes.
2. **Certification and supply.** A function that needs a certified or calibrated part is bought, not built.
3. **Changeability.** A rule that will change after deployment (thresholds, reporting rates, dashboards) belongs where changing it is cheap: in configuration data, or in the cloud.

Table 2.2 — CCM-1 partitioning decisions.

Function	Options considered	Chosen home	Reason
F1.2 Convert temperature	NTC + SiP ADC + firmware linearisation; digital sensor IC	Digital sensor IC in the vial	Meets SP1 without per-unit calibration; no analog signal across the seal (Table 2.3)
F2.1 Count alarm durations	Firmware; sensor's hardware alert	Firmware, with sensor ALERT as an independent backstop	Duration rules need state; the backstop must not depend on firmware
F3.2 Sound alarm	Firmware tone loop; PWM peripheral	PWM peripheral	Periodic, zero CPU, works while the CPU sleeps
F4.2 Integrity	Firmware CRC; hardware CRC	Firmware CRC-16 per page	Short pages; energy negligible (Section 2.6)
F5.1 Transport security	Firmware TLS; modem-offloaded TLS	Modem-offloaded TLS with keys in secure storage	Protects keys; certified stack
F5.2 Alarm forwarding to people	Device sends SMS; cloud forwards	Cloud	Recipient lists and escalation change often; device sends one message
F7 Keep time	Firmware tick; RTC peripheral + network time	RTC with network time sync	Runs in sleep at about 1 µA
F8.1 Watchdog	Firmware; hardware watchdog	Hardware watchdog, 8 s window	Independence: survives a hung task

Worked example — partition for the right reason. One team member proposes moving alarm evaluation entirely into the sensor's hardware alert to save energy, since the sensor can compare each conversion against a threshold and raise a pin. What does the firmware path cost? Each minute the SiP wakes for about 5 ms at about 3 mA to read the sensor and run the evaluation, a charge of 15 µC. The average is $15 \times 10^{-6}/60 = 0.25\ \mu$A, which is 0.5 % of the 50 µA total found in Section 2.6. Moving evaluation to hardware would save almost nothing, and it would lose the 60-minute duration logic (the hardware alert compares single samples), so R6 would fail. The energy argument is wrong; the *independence* argument is right. If the firmware hangs, the watchdog resets it within 8 s, but a firmware bug that does *not* hang (a counter that never reaches 60) would silently disable freeze alarms. The firmware stays the primary path, and the sensor's ALERT output, set at −3.0 °C, drives the buzzer through a hardware latch. The sensor compares every conversion against its own limit register, so a logic bug in `alarm_eval` cannot hide a deep freeze. Lacking the duration rule, its threshold sits low enough to fire only on a freeze the primary path should already have caught.

> **Status of the principle: Allocate safety-relevant functions to independent blocks**
> *Empirical.* Practice in every safety standard (IEC 61508's independence and diversity, ISO 13849's categories) shows that redundancy within one block shares that block's common-cause failures. The benefit depends on the two paths really failing independently, which must be argued for each common cause (power, clock, firmware image). Chapter 21 covers functional safety.

Component selection for the sensing function

Partitioning the conversion into a sensor IC leaves the question of which sensor. The candidates are compared against the specifications that Chapter 1 allocated: SP1 (±0.10 °C, drift ≤ 0.03 °C/year), SP2 (fits a 10 mL vial), the cable interface and the cost.

Table 2.3 — Sensor element selection (2026 distributor prices, 1,000 pcs, indicative).

Candidate	Accuracy without calibration	Interface over 1.5 m	Calibration needed	Cost incl. calibration	Verdict
NTC 10 kΩ, 1 %, B = 3435 K, epoxy bead	±0.25 °C near 25 °C, worse at the ends	Analog; leakage and seal contamination add error	Yes, 1 point (about 3 min per unit)	$0.60 + $1.50	Meets SP1 only after calibration; analog signal across seal
PT1000, class A (IEC 60751)	±0.15 °C at 0 °C, ±0.19 °C at 20 °C (tolerance grows with temperature)	Analog; needs a precision ADC and reference	Yes, for SP1	$2.50 + $3.00 front end	Fails SP1 uncalibrated; costly front end
Digital, ±0.1 °C class (TMP117-class)	±0.10 °C, −20 to +50 °C	I²C, digital	No (factory-trimmed, traceable)	$1.90	**Chosen**: meets SP1 at its limit with no calibration step
Digital, ±0.5 °C class (DS18B20-class)	±0.5 °C, −10 to +85 °C	1-Wire	Would need calibration and still drift	$1.20	Fails SP1

The chosen sensor meets SP1 exactly at its limit, with no margin on the sensor line. The margin is in the RSS total of the error budget (Table 1.9), and the AT1 chamber test checks the result. That is acceptable only because the budget is written down; without it, a sensor at its datasheet limit looks alarming or fine depending on who reads the sheet.

The processor block was chosen the same way: a cellular SiP with an integrated Cortex-M33 (nRF9160/nRF9151-class, about 3 µA in power-saving mode, $15) beat a pre-certified LTE-M module plus a separate low-power MCU (BG95-class + STM32L4-class, 4–6 µA, $14, two firmware images and an AT-command interface between them) on parts count and sleep current.

2.4 Interfaces

The interface control document

Every line in Figure 2.2 is an **interface**, and every interface is a place where two teams, two suppliers or two assumptions meet. An **interface control document** (ICD) defines each one completely enough that the blocks on either side can be designed and tested separately: electrical levels and timing, protocol, data format, mechanical fit, environmental limits and failure behaviour. The last item, what each side does when the other misbehaves, is the one most often left out.

Table 2.4 — CCM-1 interface control summary.

ID	Between	Type	Definition	Failure behaviour required
IF1	Probe ↔ main board	I²C, 100 kHz, 3.0 V, + ALERT, 5 wires	7-bit address 0x48; one-shot conversion, 8-sample averaging; 4.7 kΩ pull-ups on board	NACK or CRC/range error → invalid sample; 5 in a row → sensor-fault alarm (Section 2.7)
IF2	Battery ↔ board	Power, 3.3–3.65 V, ≤ 100 mA continuous per cell	Keyed connector; PTC fuse 0.5 A	Under-voltage at 3.1 V under load → low-battery warning; 3.0 V → orderly shutdown
IF3	SiP ↔ display	SPI 4 MHz + BUSY, DC, RST	Controller command set; partial refresh	Refresh time-out 5 s → display fault flag; alarms still sound
IF4	SiP ↔ flash	SPI 8 MHz	256-byte pages; 4 kB sectors	Program/erase verify failure → mark sector bad; report
IF5	Antenna ↔ network	LTE-M (Cat-M1), NB-IoT fallback	Bands per national operators; PSM and eDRX enabled	No service → queue; retry schedule (Section 2.7)
IF6	Device ↔ cloud	MQTT 3.1.1 over TLS 1.2, QoS 1	Topics and payloads below; per-device certificate	No acknowledgement → message stays in the device queue
IF7	Device ↔ nurse	Human	Buzzer ≥ 85 dB(A) at 1 m; red LED flash 1 Hz; e-paper text	Acknowledge silences buzzer for 30 min; LED stays on
IF8	Cloud ↔ officer	SMS and app push	Escalation: officer at once; supervisor after 2 h unacknowledged	SMS failure → app push and e-mail

IF6 is the interface between two teams (firmware and cloud), and so it is the one most likely to drift. Fixing it as an example message, in the ICD and in both teams' test suites, makes drift visible:

```
{
  "dev": "ccm1-00417",
  "seq": 18233,
  "type": "alarm",
  "alarm": "freeze",
  "t_start": "2026-03-14T18:24:00Z",
  "t_raise": "2026-03-14T19:24:00Z",
  "probe_c": -1.42,
  "min_c": -2.10,
  "batt_v": 3.52,
  "rsrp_dbm": -118,
  "fw": "1.3.2"
}
```

On the wire the same fields travel in a compact binary encoding (Section 2.5); the JSON form is the readable contract. `seq` lets the cloud detect gaps and duplicates (QoS 1 delivers at least once), and `rsrp_dbm` lets the field team find cell-edge clinics before their batteries do.

Worked example — an I²C bus through a refrigerator door

IF1 runs I²C, a bus designed for one circuit board, over 1.5 m of cable. Is that safe? I²C lines are open-drain, so each rising edge is an RC charge of the bus capacitance through the pull-up. The 30–70 % rise time is $t_r = 0.8473\,R_p C_b$ (*Electronics and Computing*, Ch. 26 §26.4; limits from NXP UM10204). The bus capacitance is estimated as follows: flat cable 1.5 m at about 60 pF/m gives 90 pF; sensor pin 10 pF; SiP pin 10 pF; connector and traces 15 pF; two ESD diodes at 5 pF each. The total is $C_b \approx 135$ pF. Fast mode (400 kHz) allows $t_r \le 300$ ns, which needs $R_p \le 300/(0.8473 \times 0.135) = 2.62$ kΩ. Standard mode (100 kHz) allows 1000 ns, so $R_p \le 8.74$ kΩ. The low end is set by the 3 mA sink limit at $V_{OL} = 0.4$ V from a 3.0 V rail: $R_p \ge (3.0 - 0.4)/3 = 0.87$ kΩ. The design runs at 100 kHz with 4.7 kΩ, which gives $t_r = 538$ ns (46 % margin on the 1000 ns limit). A sensor read moves 6 bytes in under 1 ms, so the slower clock costs nothing that the energy budget can see. Two risks go to the FMEA (Section 2.10): a crushed cable that holds SDA low (bus clear and the sensor-fault path) and condensation leakage at the vial (potted probe PCB).

2.5 Data flow

From sample to server

The **data flow** view follows each piece of information from where it is created to where it is used, with its rate, size and storage at each stage (Figure 2.3). It answers three budget questions: how much memory is needed, how much airtime and data volume, and where data can be lost.

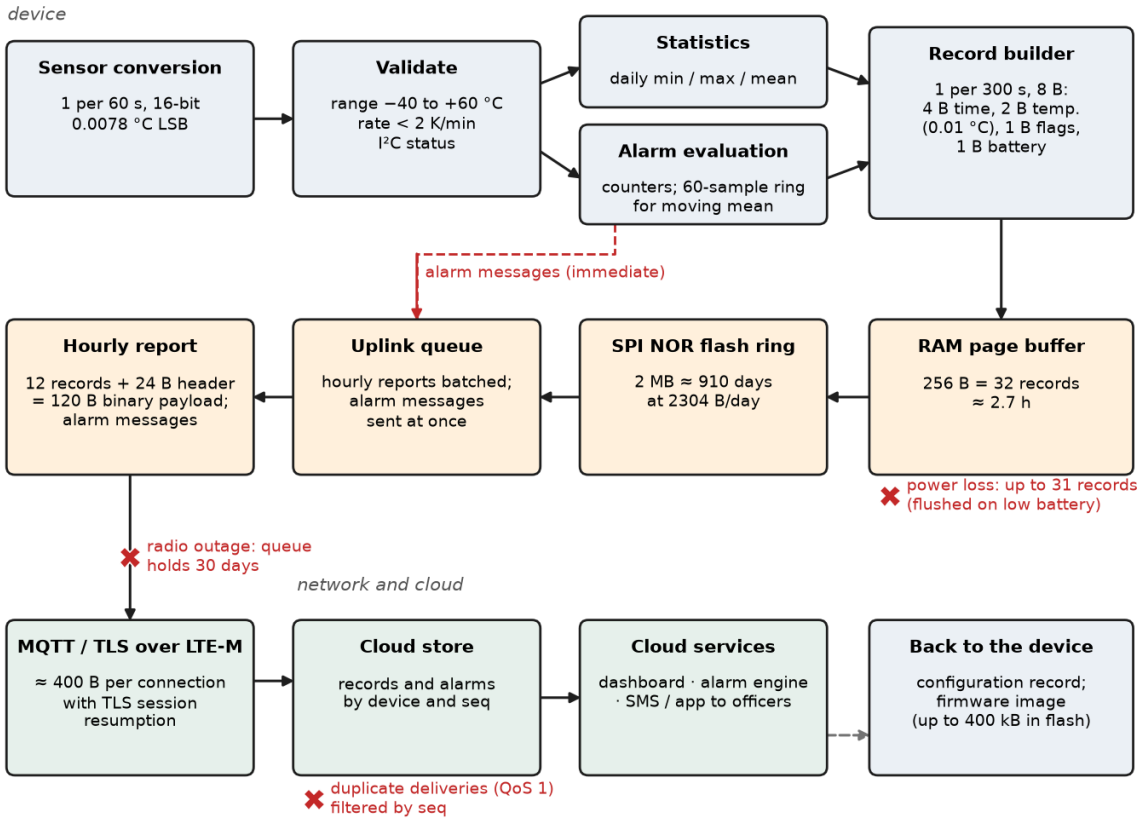


Figure 2.3 — Data flow through CCM-1, with rates and sizes at each stage.

Rates set by physics, not by habit

How fast should CCM-1 sample? The probe vial has a time constant of about 19 min (SP2, confirmed in Chapter 3), so the measured signal has almost no content above $1/(2\pi \times 19\ \text{min}) \approx 0.14$ mHz. A 60 s sample period is about 60 times faster than the Nyquist rate for that bandwidth (*Circuits, Signals and Communications*, Ch. 19 §19.3). The 60 s period is not driven by bandwidth at all. It is driven by the alarm rule (R3 counts minutes) and by the latency budget of Table 1.5. Record and report periods are then set by the users: 5-min records for the regulator's history (R7) and hourly reports for the officer's dashboard (R4).

Worked example — storage and airtime budget. A record is 8 bytes and is written every 5 min, 288 times a day, so a day costs $288 \times 8 = 2304$ bytes. The 60 days of R7 need 138 kB. A 2 MB flash holds about 910 days, so the memory requirement has a margin of about 15×. The flash is chosen at 2 MB anyway, because the next smaller size saves under $0.10, and because firmware update images (up to 400 kB) need space too. The hourly report carries 12 records plus a 24-byte header, a payload of 120 bytes. The payload is not what dominates the airtime. TCP, TLS and MQTT overheads add about 400 bytes per connection with TLS session resumption, and about 4 kB with a full handshake. The monthly volume is

$$720 \times (120 + 400)\ \text{B} \approx 0.37\ \text{MB}$$

with resumption, plus 30 daily summaries for about 0.39 MB in total, inside a 1 MB plan. A full handshake on every connection would need 720×4.12 kB ≈ 3.0 MB a month, three times over the plan, with radio time to match. So session resumption is not an optimisation. It is a requirement on the protocol stack (a specification added to SP5). The data-flow view found it, and no other view would have.

> **Status of the principle: Protocol overhead dominates small telemetry messages**
> *Empirical.* For payloads of tens to hundreds of bytes over TCP/TLS, headers, handshakes and acknowledgements typically make up 70–95 % of the bytes and of the radio-on time. Compact encodings help less than reducing the number of connections and handshakes. Chapter 11 quantifies this for MQTT, CoAP and LwM2M.

2.6 Energy flow

The energy view

The **energy flow** view follows energy from the source through storage, conversion and distribution to every load, at two time scales. The *average* scale sets the battery size and life. The *peak* scale sets the source impedance, the buffers and the brownout margin. CCM-1's source is a 5.2 Ah pack. Its storage includes a pulse buffer. Its distribution is a direct 3.6 V rail plus a load-switched branch. Its loads range from 3 µA (sleep) to about 280 mA (radio transmit bursts), five orders of magnitude.

> **Boundary.**
> The average-current budget $\bar{I} = I_{\text{sleep}} + \sum_k Q_k / T_k$ and the power-mode ladder are set out in *Electronics and Computing*, Ch. 25 §25.5, and the firmware practices that make a budget come true in Ch. 28 §28.5. Here the budget is used as an architecture tool: which block dominates, and what that implies for the design.

The budget

Each line of the budget is a charge per event multiplied by an event rate, or a constant floor. The values are first estimates from datasheets and from a 24-hour current profile of the Rev B board (measurement method: Chapter 4). The supercapacitor line uses the 0.47 F part that Chapter 3 selects.

```
 # ccm1_energy.py -- average-current budget and battery life of CCM-1
SECONDS_PER_MONTH = 30 * 86400
items = [  # (name, charge per event in coulombs, events per month) or a floor in amperes
    ("sleep floor: SiP in PSM + RTC",       None, 3.0e-6),
    ("supercapacitor leakage (0.47 F)",     None, 1.5e-6),
    ("TMP117 conversion + standby",         None, 0.43e-6),
    ("MCU wake, read, evaluate (1/min)",    15e-6, 43200),
    ("e-paper partial refresh (1/10 min)",  2.4e-3, 4320),
    ("e-paper full refresh (1/day)",        12e-3, 30),
    ("LTE-M hourly report",                 0.125, 720),
    ("LTE-M tracking-area update (1/12 h)", 0.10, 60),
    ("alarm events: 3 messages + buzzer",   0.375 + 1.8, 4),
    ("firmware update (2/year)",            3.6, 2 / 12),
]
usable_Ah = 5.2 * 0.85 * (1 - 3 * 0.03) * 0.90   # derated 2 x ER14505 (Section 1.4)

for coverage in (1.0, 4.0):                      # 4 = cell-edge repetitions on LTE-M
    total = 0.0
    for name, q, n in items:
        i = n if q is None else q * n / SECONDS_PER_MONTH
        if name.startswith("LTE-M"):
            i *= coverage
        total += i
        if coverage == 1.0:
            print(f"{name:38s} {i * 1e6:7.2f} uA")
    life = usable_Ah / total / 8766
    print(f"coverage x{coverage:.0f}: average {total * 1e6:6.1f} uA, life {life:4.1f} years, "
          f"margin vs 137.7 uA: {1 - total / 137.7e-6:+.0%}")
```

```
 sleep floor: SiP in PSM + RTC             3.00 uA
supercapacitor leakage (0.47 F)           1.50 uA
TMP117 conversion + standby               0.43 uA
MCU wake, read, evaluate (1/min)          0.25 uA
e-paper partial refresh (1/10 min)        4.00 uA
e-paper full refresh (1/day)              0.14 uA
LTE-M hourly report                      34.72 uA
LTE-M tracking-area update (1/12 h)       2.31 uA
alarm events: 3 messages + buzzer         3.36 uA
firmware update (2/year)                  0.23 uA
coverage x1: average   49.9 uA, life  8.3 years, margin vs 137.7 uA: +64%
coverage x4: average  161.1 uA, life  2.6 years, margin vs 137.7 uA: -17%
```

The 0.125 C per hourly report comes from the upload current profile that Chapter 3 simulates: 2.5 s at a 30 mA baseline, plus 1 s of 250 mA transmit bursts at 20 % duty. The coverage factor models the repetitions

LTE-M uses to reach devices deep inside buildings or at the cell edge. A factor of 4 in energy per message is typical between good coverage and the coverage-enhancement modes.

Table 2.5 — CCM-1 energy budget summary against SP6.

Case	Average current	Life (3.62 Ah usable)	Allocation SP6	Margin	R5 (≥ 2 years)
Good coverage	49.9 µA	8.3 years	137.7 µA	+64 %	Met
Cell edge, ×4 radio energy	161.1 µA	2.6 years	137.7 µA	−17 %	Met, design target (3 years) missed
Cell edge with adaptive reporting (3-hourly)	68.5 µA	6.0 years	137.7 µA	+50 %	Met

The budget makes three facts plain. First, the radio is 74 % of the total in good coverage and 92 % at the cell edge, so optimising the sensor or MCU wake-ups would be wasted. Second, the 3-year target is missed at exactly the remote sites that matter most. The response is architectural, not a bigger battery: the firmware reads the modem's coverage-enhancement level and signal power and, at the cell edge, batches routine reports every 3 h while still pushing alarms at once. The third row shows the result, 6.0 years with R4 untouched. Third, the display costs 4 µA, more than the processor in sleep. Refreshing only on a change of 0.1 °C or of status would cut that about fourfold; it stays an open optimisation because the margin does not need it.

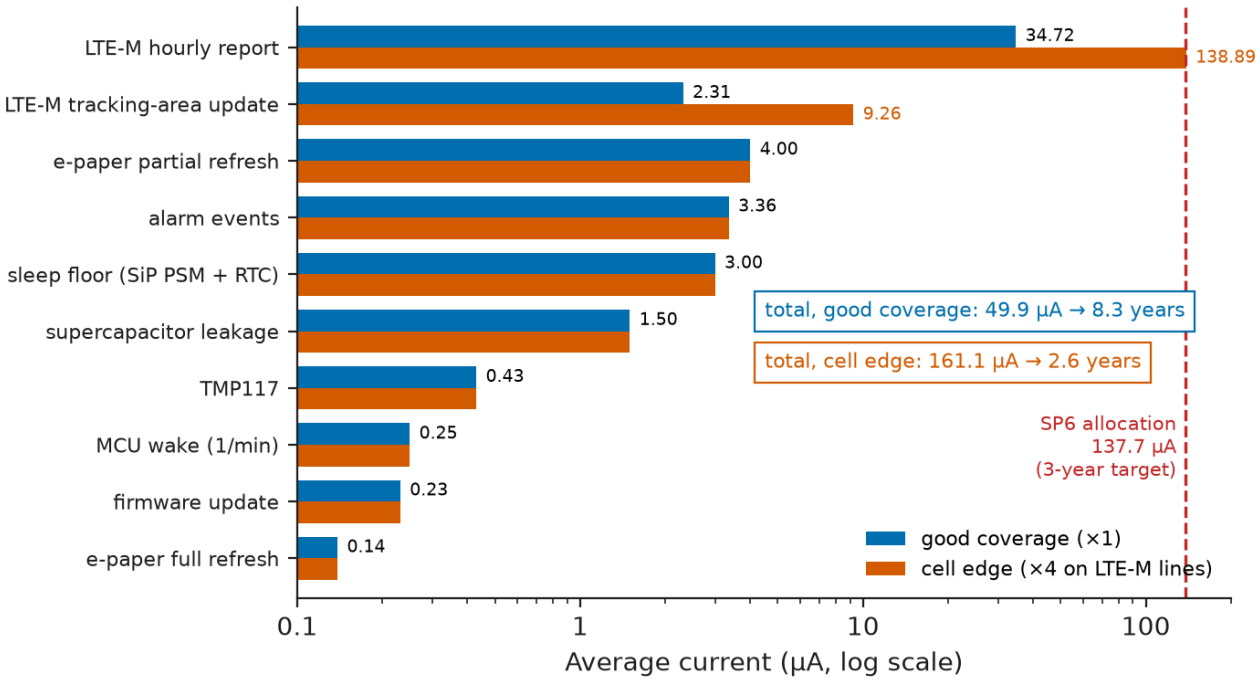


Figure 2.4 — Where CCM-1's charge goes: average current per budget line in good coverage and at the cell edge.

> **Status of the principle: Average-current budget for a duty-cycled device**
> *Approximate.* The budget assumes event charges independent of battery state and temperature, and a battery that delivers its derated charge at any rate. It underestimates consumption when retries cluster (bad coverage days) and when cold raises cell resistance. A budget is a model, and it is checked by a long-duration current measurement (Chapter 4) and against field telemetry (`batt_v` , `rsrp_dbm`).

Choosing the energy source

The battery was a candidate in Chapter 1. With the budget in hand it can be chosen properly. Four architectures were compared. Lithium content per cell is set by constraint C2.

Table 2.6 — Energy-source selection.

Option	Usable charge (3.6 V equivalent)	Life at 49.9 / 161 µA	Pulse capability	Li per cell (C2)	Cost	Verdict
A: 1 × ER14505 (AA Li-$SOCl_2$) + EDLC	1.81 Ah	4.1 / 1.3 years	Needs buffer	0.7 g	$4.50	Fails R5 at cell edge
B: 2 × ER14505 parallel + EDLC	3.62 Ah	8.3 / 2.6 years	Needs buffer (Chapter 3)	0.7 g	$7.80	**Chosen**
C: 3 × AA Li-FeS_2 in series + buck converter	2.87 Ah	6.6 / 2.0 years	Good (2 A pulses)	about 0.98 g	$6.90	Marginal at cell edge; converter adds quiescent loss and a part
D: Li-ion 18650 + mains USB adapter	Unlimited on mains	Depends on mains	Good	n/a (Li-ion, 9 Wh)	$9.00	Fails at clinics with solar fridges and no outlet

Option C handles radio pulses without a buffer and would win if pulse robustness outweighed energy. Option B wins on energy and simplicity: no converter, a flat 3.6 V, a 10-year shelf life. Its weakness, pulse capability as the cells age, is recorded as a risk for Chapter 3 to close by simulation. A decision resting on an unverified assumption is acceptable when the assumption is written down with the test that will check it.

2.7 Control flow

Modes, states and who decides

The **control flow** view shows what the system does in each situation: its operating modes, the events that move it between them, and which block decides. For an embedded device this is a set of state machines, written down explicitly (*Electronics and Computing*, Ch. 28 §28.4). The lesson of the opening paragraph sets the main architectural rule: **the local alarm path must not depend on the communication path**. CCM-1 therefore has two concurrent machines that share only a message queue. The device machine

samples, evaluates, annunciates and records. The communication machine sends whatever is in the queue, whenever it can.

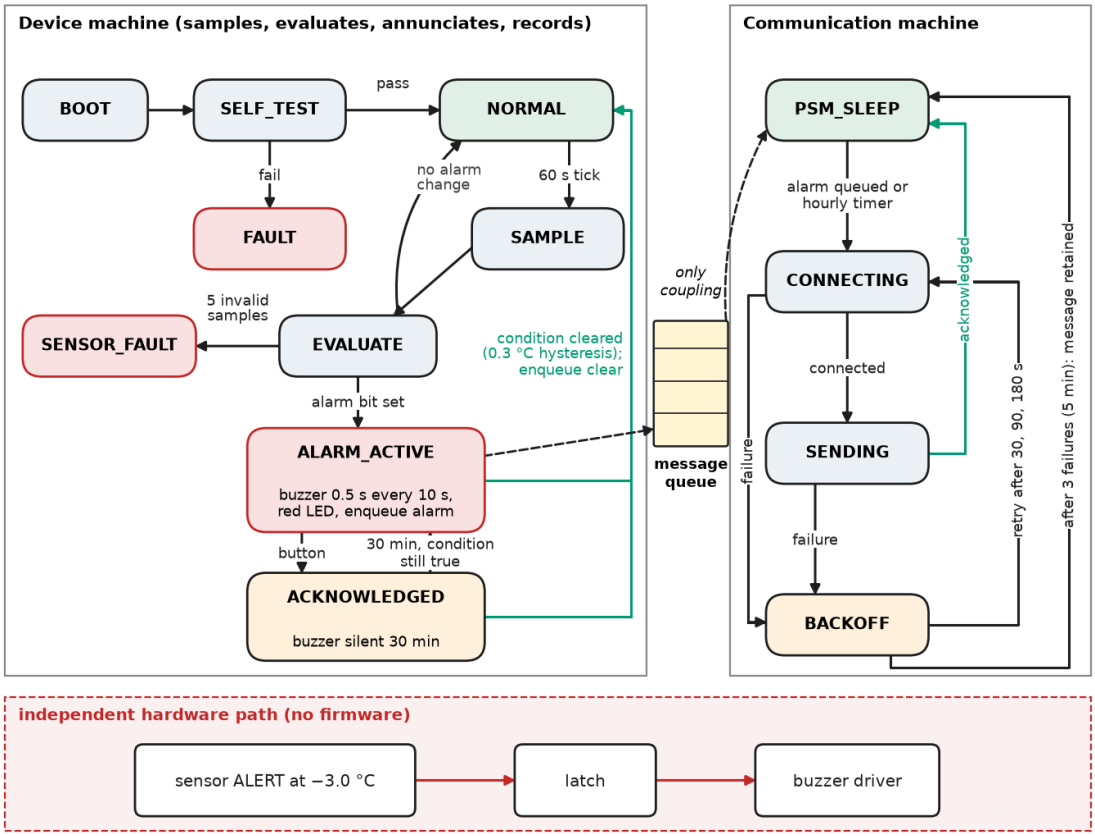


Figure 2.5 — CCM-1 control flow: the device and communication state machines, coupled only through the message queue.

The alarm evaluator

The function at the centre of the safety path, F2.1, is kept small, pure and testable on a host computer. It takes the state, the latest sample and its validity, and it returns the active alarm set. It does no input or output and has no timing of its own. One call per 60 s sample makes "minutes" equal to "calls", and that equivalence is part of the module's contract.

```
 /* alarm_eval.c -- CCM-1 alarm evaluation, one call per 60 s sample (C11) */
#include <stdbool.h>
#include <stdint.h>

#define FREEZE_LIMIT_CC  (-50)   /* -0.50 degC, in 0.01 degC units   (R3)  */
#define HEAT_LIMIT_CC    (800)   /* +8.00 degC                        (R3)  */
#define REARM_HYST_CC    (30)    /*  0.30 degC re-arm hysteresis      (SP4) */
#define FREEZE_SAMPLES   (60u)   /* 60 min continuous                       */
#define HEAT_SAMPLES     (600u)  /* 10 h continuous                         */
#define FAULT_SAMPLES    (5u)    /* consecutive invalid samples             */

enum { ALM_FREEZE = 1u, ALM_HEAT = 2u, ALM_SENSOR = 4u };

typedef struct {
    uint16_t cold_run, hot_run, bad_run;
    uint8_t  active;                       /* OR of ALM_* bits */
} alarm_state_t;

uint8_t alarm_eval(alarm_state_t *s, int16_t t_cc, bool valid)
{
    if (!valid) {                          /* hold duration counters */
        if (s->bad_run < FAULT_SAMPLES) s->bad_run++;
        if (s->bad_run >= FAULT_SAMPLES) s->active |= ALM_SENSOR;
        return s->active;
    }
    s->bad_run = 0u;
    s->active &= (uint8_t)~ALM_SENSOR;

    if (t_cc <= FREEZE_LIMIT_CC) { if (s->cold_run < FREEZE_SAMPLES) s->cold_run++; }
    else s->cold_run = 0u;
    if (t_cc > HEAT_LIMIT_CC) { if (s->hot_run < HEAT_SAMPLES) s->hot_run++; }
    else s->hot_run = 0u;

    if (s->cold_run >= FREEZE_SAMPLES) s->active |= ALM_FREEZE;
    else if (t_cc > FREEZE_LIMIT_CC + REARM_HYST_CC) s->active &= (uint8_t)~ALM_FREEZE;
    if (s->hot_run >= HEAT_SAMPLES) s->active |= ALM_HEAT;
    else if (t_cc < HEAT_LIMIT_CC - REARM_HYST_CC) s->active &= (uint8_t)~ALM_HEAT;
    return s->active;
}
```

Three decisions are visible, each recorded in the module's contract. An invalid sample *holds* the duration counters, so a one-minute probe dropout during a real freeze does not restart the 60-minute clock. The counters saturate, so a 30-day heat episode cannot overflow them. An alarm clears only 0.3 °C back past its limit, so a probe sitting at −0.5 °C does not toggle the alarm, and send a radio message, every minute. Latching the *event* for the record and the nurse is the device state machine's job; this function reports the present condition only.

The device machine works a few milliseconds per 60 s tick, while the communication machine may be busy for minutes. Because they share only the queue, the second cannot delay the first, which is exactly what the single-loop Rev B got wrong.

2.8 Modularity

Boundaries that match change

A **module** is a part of the system with a defined interface that hides its inside, so it can be designed, tested, replaced or varied on its own. Good boundaries follow the lines along which the system will change; for CCM-1 the stakeholders and trade studies name them:

- **Radio.** LTE-M today; NB-IoT where an operator offers only that; LoRaWAN in a few districts with clinic gateways.
- **Probe.** A 2–8 °C probe now; a freezer variant (−25 to −15 °C, other thresholds) is already requested.
- **Rules and cloud.** Thresholds that guidance may revise; dashboards and escalation that change monthly.

So the probe separates at a connector (IF1), thresholds live in a signed configuration record (the constants in the listing are only defaults), and the firmware is layered so that `alarm_eval`, the recorder and the report encoder run on a host PC against fake drivers. Table 2.7 is the firmware design structure matrix (DSM): a mark in row *i*, column *j* means module *i* uses module *j*, and a clean architecture has marks only below the diagonal.

Table 2.7 — Firmware design structure matrix (row uses column).

Module	bsp	drivers	config	alarm_eval	recorder	comms	ui	app_fsm
bsp (board support)	—							
drivers (sensor, flash, display)	×	—						
config		×	—					
alarm_eval				—				
recorder		×	×		—			
comms		×	×		×	—		
ui		×					—	
app_fsm (device state machine)		×	×	×	×	×	×	—

`alarm_eval` uses nothing, so it can be tested exhaustively (Chapter 4); `comms` reads the recorder's queue but the recorder never calls `comms`, the code-level form of Section 2.7's rule.

Worked example — paying for a modular radio. Should the radio sit on a plug-in daughterboard so that a LoRaWAN variant can be fitted later? The connector ($0.80) and second PCB with assembly ($1.10) add $1.90 to every unit, $2,280 over the 1,200 planned. The alternative is a new main-board variant only if it is ever needed, at about $25,000 for layout, test and radio re-certification. The daughterboard pays only if the probability of needing the variant exceeds $2280/25000 \approx 9\,\%$. With NB-IoT fallback already covering most of the cases and only a handful of districts running gateways, the team judged that probability below 5

% and kept the SiP on the main board, with the radio-dependent firmware behind the `comms` interface so that a future variant changes hardware and one module, not the system. Modularity in software is nearly free; in hardware it has a unit cost and must be argued with numbers.

2.9 Trade-off analysis

The method

A **trade study** chooses among alternatives that each satisfy the requirements to different degrees. The common tool is a **weighted decision matrix**. The criteria come from the requirements and constraints. The weights are agreed *before* the options are scored (otherwise the weights get tuned to favour a winner). Scores come from evidence, and the weighted sums are compared. The matrix itself is easy. Doing it honestly is not, and three practices make the difference:

1. **Eliminate first.** An option that violates a must-have requirement or a constraint is removed, not given a low score. A weighted sum can let a fatal flaw be outvoted by many small strengths.
2. **Score from data.** "Coverage = 4" should mean "92 % of surveyed clinics had RSRP better than −120 dBm", not an impression.
3. **Test the weights.** If a small, defensible change of weights changes the winner, the matrix has not decided anything. The decision then rests on judgement, and the record should say so.

The mathematics of weighted sums, dominance and Pareto optimality is in *The Mathematics Universe*, Ch. 23 §23.9. A weighted sum can only find options on the convex part of the Pareto front, which is one reason to plot the alternatives as well as rank them.

The radio trade study

The radio is CCM-1's most consequential choice: it dominates energy (Section 2.6), it sets the alarm latency (Table 1.5) and it decides whether each clinic needs extra infrastructure. 2G GSM fails constraint C1 and would normally be eliminated. It is kept in the matrix as a reference point to show how the matrix treats a fatal flaw.

Table 2.8 — Radio trade study: scores (1 poor to 5 excellent) and evidence.

Criterion (weight)	LTE-M	NB-IoT	LoRaWAN (gateway per clinic)	2G GSM	Evidence
Coverage at clinics (0.25)	4	3	5	5	Survey of 120 clinics: LTE-M 92 %, NB-IoT 78 % (one operator), own gateway 100 %
Energy per message (0.15)	4	5	5	2	Module datasheets and bench profiles
No new infrastructure (0.20)	5	5	1	5	LoRaWAN needs a mains-powered gateway with backhaul at each clinic
Alarm latency (0.10)	5	3	4	5	NB-IoT: longer attach, no connected-mode mobility; 5 min budget tighter
5-year cost (0.15)	3	4	2	4	Module + connectivity; gateway $150 per clinic
Network longevity (0.15)	4	4	4	1	Operator roadmaps; 2G sunset (C1)

```
 # ccm1_trade.py -- weighted decision matrix for the CCM-1 radio, with weight sensitivity
import numpy as np

criteria = ["coverage", "energy", "no new infrastructure", "alarm latency",
            "5-year cost", "network longevity"]
w = np.array([0.25, 0.15, 0.20, 0.10, 0.15, 0.15])
scores = {                    # 1 (poor) .. 5 (excellent), from Table 2.8
    "LTE-M":   [4, 4, 5, 5, 3, 4],
    "NB-IoT":  [3, 5, 5, 3, 4, 4],
    "LoRaWAN": [5, 5, 1, 4, 2, 4],
    "2G GSM":  [5, 2, 5, 5, 4, 1],
}
S = np.array(list(scores.values()), float)
names = list(scores)
total = S @ w
for n, s in sorted(zip(names, total), key=lambda p: -p[1]):
    print(f"{n:8s} {s:.2f}")

# Scale one weight at a time by k (others renormalised) and find the first k that
# changes the winner: how robust is the decision to the weights we argued about?
for j, c in enumerate(criteria):
    for k in np.r_[np.linspace(1, 0, 101), np.linspace(1, 4, 301)]:
        wk = w.copy()
        wk[j] *= k
        wk /= wk.sum()
        win = names[int(np.argmax(S @ wk))]
        if win != "LTE-M":
            print(f"{c:22s}: winner becomes {win} when its weight is x{k:.2f} "
                  f"({wk[j]:.2f} of the total)")
            break
    else:
        print(f"{c:22s}: LTE-M stays first for weight x0 to x4")
```

```
 LTE-M    4.15
NB-IoT   4.00
2G GSM   3.80
LoRaWAN  3.50
coverage              : winner becomes NB-IoT when its weight is x0.39 (0.12 of the total)
energy                : winner becomes NB-IoT when its weight is x2.00 (0.26 of the total)
no new infrastructure : winner becomes LoRaWAN when its weight is x0.18 (0.04 of the total)
alarm latency         : winner becomes NB-IoT when its weight is x0.24 (0.03 of the total)
5-year cost           : winner becomes NB-IoT when its weight is x2.01 (0.26 of the total)
network longevity     : winner becomes 2G GSM when its weight is x0.22 (0.04 of the total)
```

Worked example — reading a close trade study. LTE-M wins with 4.15 against NB-IoT's 4.00, a 4 % lead. The sensitivity scan says what that lead depends on. If the energy weight doubled from 0.15 to about 0.26, or the cost weight did the same, NB-IoT would tie and then win. If the coverage weight fell to under 40 % of its value, NB-IoT would win too. The choice is really 14 percentage points of coverage (92 % against 78 %) against a modest energy and cost advantage: a judgement the matrix exposes but cannot make. 2G comes third *despite* a fatal flaw, its other scores outvoting its 1 for longevity, which is why elimination on

constraints comes before scoring. LoRaWAN wins only if infrastructure is almost ignored. The decision follows: LTE-M primary *with NB-IoT fallback*, which the chosen SiP supports in the same hardware, turning a close call into a configuration setting.

> **Status of the principle: Weighted-sum decision matrices**
> *Empirical.* The weighted sum assumes criteria that are independent and can be traded linearly, and scores on interval scales. Neither is strictly true. The method is a structured way to expose judgements, not to compute a right answer. Its result is trustworthy only when it is insensitive to defensible changes of weights, which is why the sensitivity scan is part of the method.

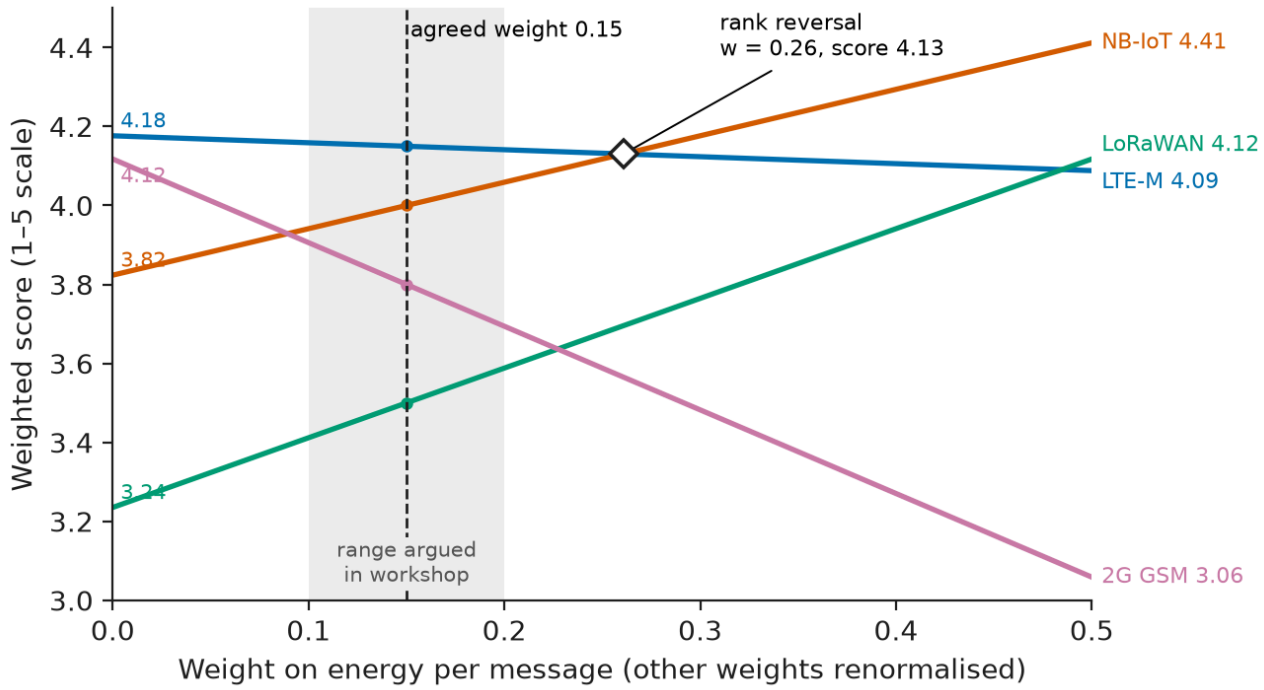


Figure 2.6 — Radio trade study: weighted score of each option as the weight on energy per message varies, with the others renormalised.

2.10 Architecture documentation

Views, not one diagram

No single diagram describes an architecture. ISO/IEC/IEEE 42010 frames the description as a set of **views**, each answering particular stakeholders' concerns (arc42 and the C4 model are lighter templates on the same idea). CCM-1's description is the set of views built in this chapter, kept under version control (Chapter 5):

Table 2.9 — CCM-1 architecture description.

View	Content	Section
Context	System boundary, external entities, flows	1.5
Functional	Function tree, requirement allocation	2.1
Physical	Block diagram, component selections	2.2–2.3
Interface	ICD (Table 2.4), message contract	2.4
Data	Data-flow diagram, storage and airtime budget	2.5
Energy	Energy budget, source selection	2.6
Control	State machines, independence argument	2.7
Module	DSM, variant strategy	2.8
Decisions	Architecture decision records	below

Architecture decision records

The views say *what* the architecture is. They do not say *why*, and the why is the first thing lost when staff change. An **architecture decision record** (ADR) is a short, dated, numbered note: the context, the decision, the alternatives considered, the consequences, and the condition under which the decision should be revisited. ADRs are never edited after acceptance. A changed decision gets a new ADR that supersedes the old one, so the history stays readable.

```
 ADR-003  Radio technology                       Status: accepted 2026-02-11
Context  Must deliver alarms <= 15 min (R4) for >= 2 years on batteries (R5)
         at >= 90 % of clinics; 2G ends within 3 years (C1).
Decision LTE-M primary with NB-IoT fallback on a single cellular SiP.
Options  NB-IoT only; LoRaWAN with gateway per clinic; 2G (eliminated, C1).
Evidence Trade study Table 2.8; coverage survey 2026-01 (120 sites);
         energy budget Table 2.5.
Result   LTE-M 4.15 vs NB-IoT 4.00; reverses if energy weight >= 0.26 or
         coverage weight <= 0.12.
Conseq.  Radio dominates energy: adaptive reporting at cell edge (ADR-005).
Revisit  If an operator withdraws LTE-M, or field RSRP shows < 85 % coverage.
```

Verifying an architecture before it is built

An architecture is verified by analysis against the requirements before hardware exists, and the result is tracked as evidence. Some rows can only be closed later by test, and those rows say so.

Table 2.10 — Architecture verification for CCM-1.

Requirement	Architectural evidence	Limit	Result	Status
R1 accuracy	Error budget with chosen sensor (Tables 1.9, 2.3)	±0.5 °C	±0.458 worst case, ±0.194 RSS	Pass by analysis; AT1 pending
R3 alarm logic	`alarm_eval` contract; host unit tests on the 4 AT2 profiles	4/4 correct	4/4	Pass (host)
R3 independence	Comms coupled only by queue; ALERT backstop	No dependency of alarm path on radio	None found in DSM	Pass by review
R4 latency	Latency budget (Table 1.5) with LTE-M retry schedule	≤ 15 min	8.1 min worst case	Pass by analysis; AT3 pending
R5 battery life	Energy budget (Table 2.5)	≥ 2 years worst case	2.6 years (6.0 with adaptive reporting)	Pass by analysis; pulse capability open (Chapter 3)
R7 retention	Storage budget (Section 2.5)	≥ 60 days	About 910 days	Pass
R11 cost	Costed BOM (main parts as in Tables 2.3, 2.4, 2.7, plus enclosure, PCB, display)	≤ $175	$78 BOM, $112 ex-works	Pass by analysis
IF1 timing	I²C rise time (Section 2.4)	≤ 1000 ns	538 ns	Pass by analysis
Data volume	Airtime budget (Section 2.5)	≤ 1 MB/month	0.39 MB	Pass; requires TLS resumption

Architecture-level failure analysis

A **functional FMEA** at architecture level asks, for each block and interface, how it can fail, what the system does then, and whether the architecture detects and contains it. It is cheaper now than at any later stage, because each mitigation is still a line on a diagram.

Table 2.11 — Architecture FMEA for CCM-1 (selected rows).

Failure mode	Effect	Cause	Detection	Mitigation in the architecture
Radio blocks for minutes	Samples and local alarms stall (Rev B)	Single loop; blocking send	Code review against control-flow view	Separate machines coupled only by queue (Section 2.7)
Cell-edge site drains battery	Device dies in under 2 years	Radio energy ×4; budget at one point	Energy budget with coverage factor; `rsrp_dbm` telemetry	Adaptive reporting (Table 2.5)
Brownout during transmit at end of life	Reset loop; lost alarms	Cell resistance rises; AA cells cannot supply bursts	Peak-energy view; simulation (Chapter 3)	EDLC pulse buffer; low-battery warning at 3.1 V under load
Weak cell reverse-charged by its parallel partner	Cell heating; capacity loss	Unmatched parallel primary cells	Incoming cell inspection	Factory-matched pack from one lot, PTC fuse
Probe cable crushed in door	I²C hang or invalid readings	Mechanical damage at seal	NACK, bus-clear failure, invalid samples	Sensor-fault alarm after 5 samples; flat cable section (SP10)
Firmware bug disables freeze counting	Missed freeze alarm	Logic error not caught by tests	Host unit tests; review	Hardware ALERT backstop at −3.0 °C (Section 2.3)
Cloud and device disagree on message format	Alarms silently dropped	IF6 drift between teams	Contract tests in both CI pipelines	JSON contract in the ICD; `seq` gap detection
Flash sector wears out	Record loss	Repeated rewrites of one sector	Verify after program	Ring buffer, page-level writes (about 2.7 h per page)

What remains open

Open question	Where it is answered
Is the chosen pack (option B) able to carry radio bursts at end of life, and what size of EDLC is needed?	Chapter 3 (component-level simulation)
Is the freeze rule safe across refrigerator faults, and does the 19 min probe constant hold?	Chapter 3 (system simulation, sensitivity, validation)
How are the architecture's analyses confirmed on hardware: current profiles, chamber tests, regression tests of `alarm_eval` ?	Chapter 4
How are ADRs, ICDs and the DSM kept consistent with the design as it changes?	Chapter 5
How are MCU, firmware architecture and timing designed in detail for a product like this?	Chapter 10
How are MQTT, TLS session resumption, provisioning and device identity implemented?	Chapter 11
How is a cellular or LPWAN link budget computed for a cell-edge clinic?	Chapter 26
How do functional-safety standards formalise independence and diagnostic coverage?	Chapter 21

Next: Chapter 3 builds the models that test this architecture before it is built: a thermal model of the refrigerator and probe, an equivalent circuit of the battery and pulse buffer, and the simulation, sensitivity and validation work that turns them into evidence.

Chapter 3 — Modeling and Simulation Workflow

Part I · Engineering Design Methodology

Chapter 2 chose two AA lithium–thionyl-chloride cells for CCM-1 and left one question open: can they, with a pulse-buffer capacitor, carry the modem's transmit bursts at end of life? An engineer modeled the pack, a 0.1 F supercapacitor and the modem load, ran an implicit solver at default settings, and got a minimum supply voltage of 3.33 V against the module's 3.0 V limit. So 0.1 F went into the Rev B build. On the bench, with 12 Ω in series to emulate a passivated end-of-life pack, the module reset during every upload. The circuit, parameters and load profile were all correct. The solver had covered 4 s in 64 steps, stepped straight over all 200 of the 1 ms bursts, and reported the voltage between them. Run properly, the same model gives 2.94 V: a brownout. A simulation is a measurement made on a model, and its method can be wrong in ways the result does not reveal.

This chapter is the workflow that makes model results trustworthy enough to design with. It builds two models of CCM-1, a thermal model of the refrigerator and probe and an electrical model of the battery and pulse buffer, and takes them through simulation, solver choice, assumptions, parameter estimation, sensitivity, validation and reproducibility.

> **How is a model built to answer a specific design question, at the right level of detail, with a solver that cannot silently mislead?**
> **How are model parameters estimated, their influence measured and the model validated, so that a simulation result can carry a design decision and be reproduced later?**

Concept	Engineering destination
Mathematical modeling	Deciding what question a model answers and what it may ignore
Equivalent circuits	Thermal RC networks, battery and supercapacitor models, sensor and cable models
System-level simulation	Whole-product behaviour over days: alarms, false alarms, energy, faults
Component-level simulation	Brownout, ripple, rise times, transients over milliseconds to seconds
Numerical solver selection	Stiff circuits, switching loads, hybrid systems with events
Model assumptions	Assumption registers that state domain of validity and how each was checked
Parameter estimation	Fitting models to test records; identifiability; parameter uncertainty
Sensitivity analysis	Which parameters to measure better, which tolerances to tighten, which rules are fragile
Model validation	Evidence that a model predicts data it was not fitted to, within a stated accuracy
Simulation reproducibility	Configuration, versions, seeds and manifests that let a result be rerun and audited

Roadmap. Frame each model by its question (3.1) → build equivalent circuits (3.2) → simulate the whole system over days (3.3) and the critical component over seconds (3.4) → choose solvers that cannot step over

the physics (3.5) → write down the assumptions (3.6) → estimate parameters from a test (3.7) → find what matters (3.8) → validate against independent data (3.9) → make every run reproducible (3.10).

3.1 Mathematical modeling

A model answers a question

> **Boundary.**
> How lumped models are formed from conservation laws and constitutive relations, and the modeling cycle itself, are treated in *The Mathematics Universe*, Ch. 25 §25.1; the physical-system models and state-variable form used for control in *Circuits, Signals and Communications*, Ch. 33 §33.1–33.2. This chapter is the engineering workflow around them.

A model is built to answer a question, and the question fixes everything else: which quantities appear, which time scales matter, what can be lumped, how accurate the answer must be, and what data will validate it. A model with no stated question grows until nobody can run it or say whether it is right. CCM-1 needs three:

Table 3.1 — CCM-1 model register.

Model	Question it answers	Requirement	Time scale	Accuracy needed	Validated against
M1 Thermal system	Does the probe rule avoid false alarms and catch freezing before the load freezes?	R3, R6, SP2, SP7	Minutes to days	Crossing times ±10 %	Chamber outage and door tests (Section 3.9)
M2 Supply pulse	Does the supply stay above 3.0 V during uploads at end of life, and with what buffer?	R5, Table 2.6 risk	100 µs to 10 s	Minimum voltage ±50 mV	Bench pulse test with emulated pack
M3 Energy budget	Will the battery last?	R5, SP6	Months to years	±20 % on average current	24 h current profile; field telemetry

M3 already exists (the budget of Section 2.6), a reminder that a model need not be a differential equation. This chapter builds M1 and M2 along the workflow of Figure 3.1, a loop in which validation failures return to the assumptions and sensitivity results return to the test plan.

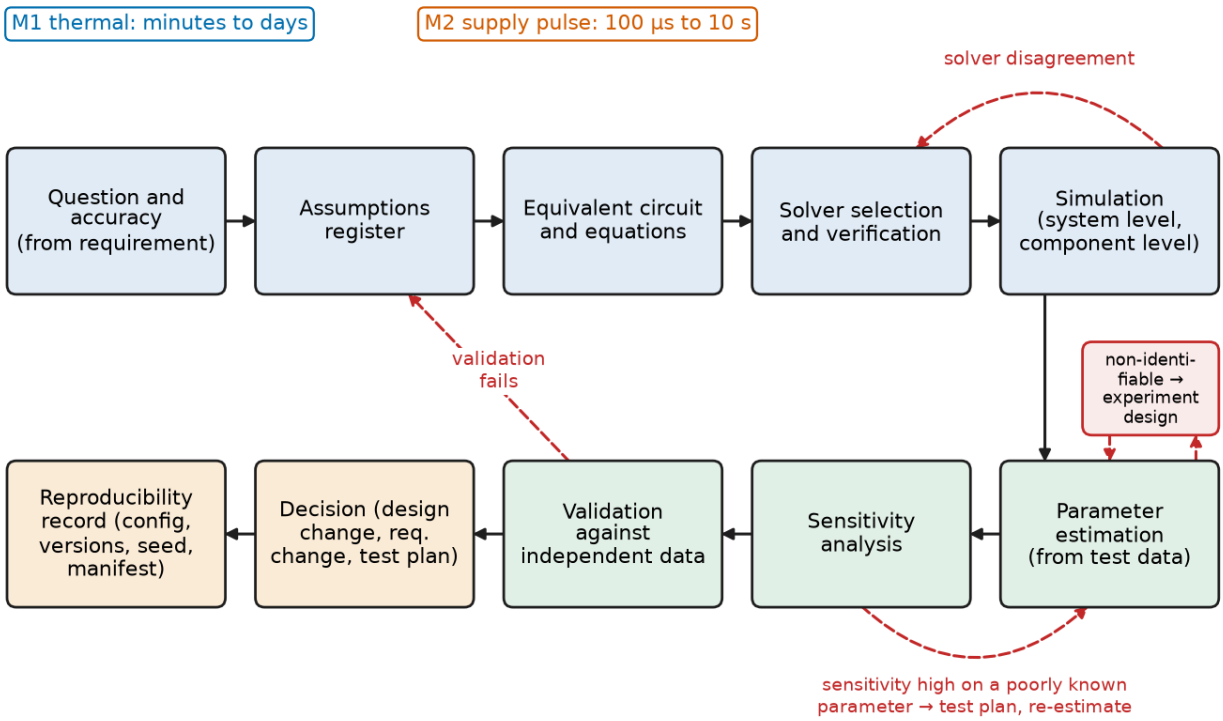


Figure 3.1 — The modeling and simulation workflow used for CCM-1, with the loops that return to earlier steps.

How much model?

Detail costs effort, data and run time, and every extra parameter can be wrong. The right level is the *simplest model whose error is small against the accuracy the question needs*. M1 asks about crossing times of minutes to hours; air, load and probe each have a time constant in that range, so three thermal nodes are the minimum and a CFD model of the cabinet would answer no question anyone has asked. M2 asks about a minimum voltage during 1 ms bursts, so the bulk capacitor's 100 µs time scale stays, while the battery's hour-scale diffusion is dropped.

3.2 Equivalent circuits

The thermal network

> **Boundary.**
> The effort–flow analogy that makes a thermal network an RC circuit (temperature as effort, heat flow as flow, thermal resistance and capacity as R and C) and the lumping criterion are developed in *Engineering Physics*, Ch. 17 §17.1–17.3; thermal resistance networks from conduction in Ch. 14 §14.7. Here the analogy is used, not derived.

M1 is a three-node RC network (Figure 3.2, left). Node T_a is the cabinet air together with the liner and shelves that exchange heat with it quickly. Node T_v is the vaccine load. Node T_p is the probe vial. The

conductances are the walls to ambient (UA), air to load (G_{av}), air to vial (G_{pa}) and, while the door is open, an extra air-exchange conductance G_{door}. The compressor is a heat-current sink Q_c switched by a hysteresis thermostat on the air. In state form, $\dot{\mathbf{x}} = A\mathbf{x} + B\mathbf{u}$ with $\mathbf{x} = [T_a, T_v, T_p]^\mathsf{T}$ and $\mathbf{u} = [T_{amb},\, Q_c\, s(t)]^\mathsf{T}$, where $s(t)$ is the thermostat state.

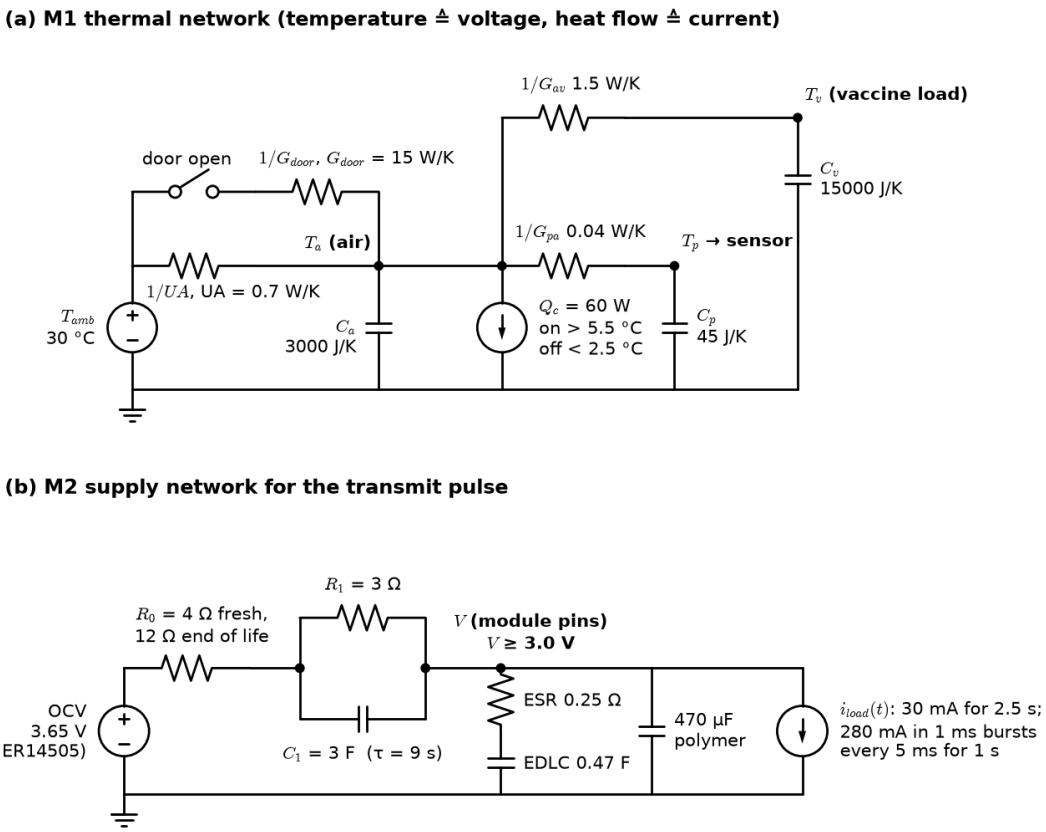


Figure 3.2 — Equivalent circuits of CCM-1: thermal network of refrigerator and probe (left) and supply network for the transmit pulse (right).

Each parameter comes from geometry and handbook data, before any test is run. Section 3.7 then refines the ones the data can determine.

Table 3.2 — First-principles parameters of M1 (100 L compression refrigerator, 30 °C clinic).

Parameter	Value	Basis
C_a	3,000 J/K	Air 100 L × 1.2 kg/m³ × 1005 J/(kg·K) = 120 J/K, plus liner and shelves effectively coupled to air
C_v	15,000 J/K	Vaccine and diluent load, about 3.6 kg water-equivalent
C_p	45 J/K	10 mL glycol–water (about 38 J/K) plus vial and sensor PCB
UA	0.7 W/K	1.5 m² of 50 mm polyurethane at 0.025 W/(m·K) gives 0.75 W/K; less a little for corners
G_{av}	1.5 W/K	Natural convection to boxed load
G_{pa}	0.04 W/K	42 cm² vial surface × about 9.5 W/(m²·K) combined convection and radiation
G_{door}	15 W/K	Air exchange while open; tuned to the 5–8 K air spikes of Figure 1.2
Q_c	60 W	Evaporator capacity at 2–5 °C

Worked example — the probe time constant and whether the vial may be lumped. The probe's time constant in still air is $\tau_p = C_p/G_{pa} = 45/0.04 = 1125$ s, or 18.75 min, inside SP2's 15–25 min window. Treating the vial as a single node requires its internal temperature to be nearly uniform, which means a Biot number $Bi = hL_c/k \ll 1$ (*Engineering Physics*, Ch. 17 §17.1.2). With $L_c = V/A = 10\ \text{mL}/42\ \text{cm}^2 = 2.4$ mm, $h = 9.5$ W/(m²·K) and $k \approx 0.45$ W/(m·K) for glycol–water, $Bi = 9.5 \times 0.0024/0.45 = 0.05$. This is below the usual limit of 0.1, so the lumped vial is justified. The same check fails for the vaccine load, a stack of cartons with air gaps, and Section 3.6 records that as an assumption with a consequence.

The supply network

> **Boundary.**
> The Thevenin equivalent circuit of a cell (OCV source, series R_0, RC pairs for polarisation) and its use in battery management are in *Electrical Energy and Machines*, Ch. 16 §16.1–16.4; capacitor behaviour and first-order transients in *Circuits, Signals and Communications*, Ch. 5–6.

M2 (Figure 3.2, right) has three states: the node voltage across the 470 µF polymer capacitor at the module pins, the EDLC's internal voltage behind its ESR, and the voltage on the battery's polarisation RC pair. The pack's series resistance R_0 is the parameter that matters. For a lithium–thionyl-chloride bobbin cell it is high (several ohms per AA cell) and it grows with the passivation layer that forms on the lithium anode during storage, faster at high temperature (*Materials Science and Engineering*, Ch. 40). Two cases bracket it: 4 Ω for a fresh pack at 25 °C, and 12 Ω for a passivated pack near end of life.

Table 3.3 — Parameters of M2 (representative values for the cell class; to be measured on the purchased cells).

Parameter	Value	Note
OCV	3.65 V	Two ER14505 cells in parallel
R_0	4 Ω fresh / 12 Ω end of life	Pack value; passivation and cold raise it
R_1, C_1	3 Ω, 3 F (τ = 9 s)	Polarisation
EDLC	0.1, 0.47 or 1.0 F; ESR 0.5, 0.25, 0.15 Ω	Candidates; leakage 1–3 µA
Bulk capacitor	470 µF polymer	ESR neglected
Load	30 mA for 2.5 s; 250 mA extra in 1 ms bursts every 5 ms for 1 s; 5 µA otherwise	LTE-M upload profile, 0.125 C per upload

3.3 System-level simulation

Seven days in a clinic

A **system-level simulation** runs the whole product, including its environment and its firmware decisions, over the time scale of the requirement. For M1 that means days: door openings during clinic hours, a compressor cycling under its thermostat, a fault, and the logger's sampling and alarm rules applied to the simulated signals. The firmware logic is part of the model, so a system simulation tests the *rules* as well as the physics.

Between events the network is linear with constant inputs, so it can be advanced exactly by the matrix exponential of the augmented system (the zero-order-hold discretisation). Each 5 s step is then one matrix–vector product, with no truncation error. Listing 3.1 simulates 7 days: 15 door openings of 20–60 s between 08:00 and 16:00 each day, and a thermostat fault at 114 h (day 5, 18:00) that moves the switching points from 2.5/5.5 °C to −4/−1 °C. It then applies two alarm rules: the Rev A prototype's (air above 8 °C) and CCM-1's (probe at or below −0.5 °C for 60 consecutive 1-min samples).

```
 # ccm1_system.py -- refrigerator + buffered probe + logger alarm rules, 7 days
import numpy as np
from scipy.linalg import expm

Ca, Cv, Cp = 3000.0, 15000.0, 45.0          # J/K: air+liner, vaccine load, probe vial
UA, Gav, Gpa, Gdoor = 0.7, 1.5, 0.04, 15.0  # W/K: walls, air-load, air-vial, open door
Tamb, Qc, dt = 30.0, 60.0, 5.0              # degC, W compressor, s

def zoh(g):                                  # exact discretisation, x = [Ta, Tv, Tp]
    A = np.array([[-(g + Gav + Gpa), Gav, Gpa], [Gav, -Gav, 0], [Gpa, 0, -Gpa]])
    A /= np.array([[Ca], [Cv], [Cp]])
    B = np.array([[g / Ca, -1 / Ca], [0, 0], [0, 0]])
    M = expm(np.block([[A, B], [np.zeros((2, 5))]]) * dt)
    return M[:3, :3], M[:3, 3:]
closed, opened = zoh(UA), zoh(UA + Gdoor)

rng = np.random.default_rng(7)
N = 7 * 86400 // 5
door = np.zeros(N, bool)
for t0 in (rng.uniform(8, 16, (7, 15)) * 3600 + np.arange(7)[:, None] * 86400).ravel():
    door[int(t0 / dt):int((t0 + rng.uniform(20, 60)) / dt)] = True  # 15 openings/day
x, on, X = np.array([4.0, 4.0, 4.0]), False, np.empty((N // 12, 3))
for k in range(N):
    lo, hi = (-4.0, -1.0) if k * dt >= 114 * 3600 else (2.5, 5.5)  # thermostat fault
    on = (x[0] > hi) or (on and x[0] > lo)
    Phi, Gam = opened if door[k] else closed
    x = Phi @ x + Gam @ np.array([Tamb, Qc * on])
    if k % 12 == 0:
        X[k // 12] = x                       # logger samples once a minute
air, vac, probe = X.T

n4 = 4 * 1440                                # four normal days of 1-min samples
print("air > 8 degC alarms, days 1-4:", np.sum((air[1:n4] > 8) & (air[:n4 - 1] <= 8)))
print(f"probe range, days 2-4: {probe[1440:n4].min():.2f} to {probe[1440:n4].max():.2f} degC")
run = np.convolve(probe <= -0.5, np.ones(60), "valid")   # 60-min continuous window
print(f"probe freeze alarm at {(np.argmax(run == 60) + 59) / 60:.2f} h; "
      f"vaccine below -0.5 degC at {np.argmax(vac <= -0.5) / 60:.2f} h")
```

```
 air > 8 degC alarms, days 1-4: 31
probe range, days 2-4: 3.92 to 5.53 degC
probe freeze alarm at 115.40 h; vaccine below -0.5 degC at 117.43 h
```

The 15 s run answers M1's question for this scenario. The air rule raises 31 alarms in four normal days, close to the 5–11 a day that killed the Rev A prototype. The probe stays within 3.92–5.53 °C on days 2–4 (6.2 °C at most), so the probe rule raises none. When the thermostat fails, the probe alarm fires 1.40 h after the fault and 2.03 h *before* the load itself reaches −0.5 °C. That is a two-hour window in which a nurse can move the vaccine. SP7, "alarms on probe temperature only", was a requirement-level decision in Chapter 1. Here it gets its first quantitative evidence. Figure 3.3 shows the run.

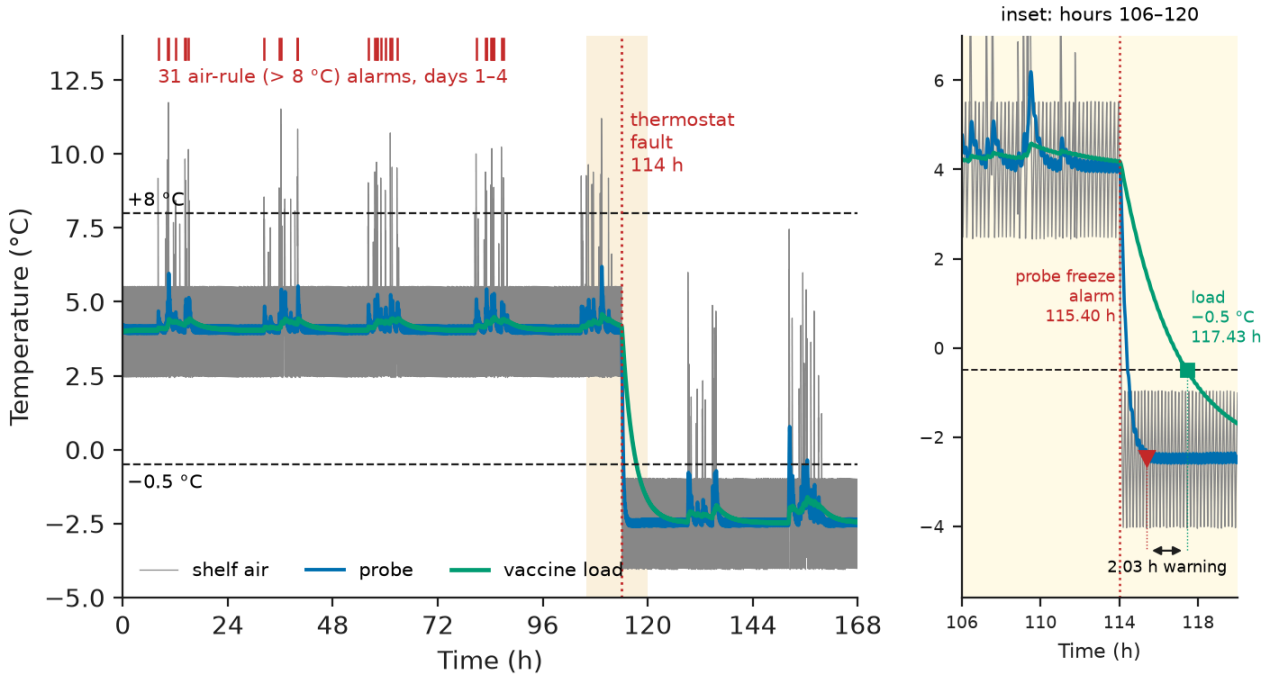


Figure 3.3 — Seven-day system simulation of CCM-1: shelf air, probe and vaccine load, with the thermostat fault on day 5.

Running the simulation with the parameters fitted in Section 3.7 instead of the first-principles values changes the probe range to 3.82–5.70 °C and the warning window to 1.93 h. Both conclusions hold, and that check is itself a small sensitivity analysis.

> **Status of the principle: Exact discretisation of a linear system with piecewise-constant input**
> *Derived.* For $\dot{\mathbf{x}} = A\mathbf{x} + B\mathbf{u}$ with $\mathbf{u}$ constant over a step h, $\mathbf{x}_{k+1} = e^{Ah}\mathbf{x}_k + \int_0^h e^{A\sigma}d\sigma\, B\mathbf{u}_k$ exactly, and both terms come from one exponential of the augmented matrix. It is exact only if every input change falls on a step boundary. Here door openings are rounded to 5 s and the thermostat switches at the step after its threshold, a timing error of at most 5 s in a process with 3-minute time constants.

3.4 Component-level simulation

The supply under a transmit burst

A **component-level simulation** zooms into one part of the system at the time scale of its own failure mode. For M2 that is the 4 s around an upload. The question is the minimum voltage at the module pins, the requirement is ≥ 3.0 V with margin, and the design variable is the pulse buffer. Listing 3.2 integrates the three-state circuit with a stiff solver and *restarts it at every load edge*. Section 3.5 explains why that matters.

```
 # ccm1_pulse.py -- battery + supercapacitor + bulk capacitor under an LTE-M upload
import numpy as np
from scipy.integrate import solve_ivp

OCV, R1, C1 = 3.65, 3.0, 3.0           # 2 x ER14505 in parallel: OCV, polarisation RC
Cb = 470e-6                            # polymer bulk capacitor at the module pins

SEGS = [(0.0, 0.75, 0.030)]            # (start s, end s, load A): 30 mA for 2.5 s ...
for t in 0.75 + 0.005 * np.arange(200):        # ... with 250 mA bursts, 1 ms in 5 ms
    SEGS += [(t, t + 0.001, 0.280), (t + 0.001, t + 0.005, 0.030)]
SEGS += [(1.75, 2.5, 0.030), (2.5, 4.0, 5e-6)]

def rhs(t, y, R0, Csc, Resr, i):        # y = [v_node, v_supercap, v_RC]
    vb, vc, v1 = y
    ib = (OCV - v1 - vb) / R0
    isc = (vc - vb) / Resr if Csc else 0.0
    return [(ib + isc - i) / Cb, -isc / Csc if Csc else 0.0, (ib - v1 / R1) / C1]

def vmin(R0, Csc=0.0, Resr=1.0, restart=True):
    if not restart:                    # one call: the solver never sees the bursts
        s = solve_ivp(lambda t, y: rhs(t, y, R0, Csc, Resr, next(
            i for a, b, i in SEGS if a <= t < b) if t < 4.0 else 5e-6),
            (0, 4.0), [OCV, OCV, 0.0], method="Radau", rtol=1e-6, atol=1e-8)
        return s.y[0].min(), s.nfev
    y, lo, nfev = [OCV, OCV, 0.0], OCV, 0
    for a, b, i in SEGS:               # restart the solver at every load edge
        s = solve_ivp(rhs, (a, b), y, method="Radau", args=(R0, Csc, Resr, i),
                      rtol=1e-6, atol=1e-8)
        y, lo, nfev = s.y[:, -1], min(lo, s.y[0].min()), nfev + s.nfev
    return lo, nfev

print("Radau, one call, R0 = 12, 1 F: Vmin = %.3f V (%d evaluations)" % vmin(12, 1.0, 0.15,
False))
for R0 in (4.0, 12.0):                 # fresh at 25 degC; passivated at end of life
    for Csc, Resr in [(0.0, 1.0), (0.1, 0.5), (0.47, 0.25), (1.0, 0.15)]:
        v, n = vmin(R0, Csc, Resr)
        print(f"R0 = {R0:4.1f} ohm, Csc = {Csc:4.2f} F: Vmin = {v:.3f} V ({n} evaluations)")
```

```
 Radau, one call, R0 = 12, 1 F: Vmin = 3.579 V (516 evaluations)
R0 =  4.0 ohm, Csc = 0.00 F: Vmin = 3.055 V (24443 evaluations)
R0 =  4.0 ohm, Csc = 0.10 F: Vmin = 3.249 V (48450 evaluations)
R0 =  4.0 ohm, Csc = 0.47 F: Vmin = 3.442 V (46923 evaluations)
R0 =  4.0 ohm, Csc = 1.00 F: Vmin = 3.528 V (45489 evaluations)
R0 = 12.0 ohm, Csc = 0.00 F: Vmin = 2.430 V (15963 evaluations)
R0 = 12.0 ohm, Csc = 0.10 F: Vmin = 2.937 V (48675 evaluations)
R0 = 12.0 ohm, Csc = 0.47 F: Vmin = 3.395 V (46888 evaluations)
R0 = 12.0 ohm, Csc = 1.00 F: Vmin = 3.514 V (45456 evaluations)
```

The table closes Chapter 2's open decision. Without a buffer, a fresh pack at 25 °C clears 3.0 V by only 55 mV, which explains why bench tests on new cells passed. At end of life it falls to 2.43 V. The 0.1 F part of the Rev

B build fails at end of life (2.94 V). The 0.47 F part holds 3.40 V, a margin of 395 mV, and 1 F adds only another 120 mV for twice the leakage and size.

Worked example — choosing the buffer. The design rule is to keep at least 0.2 V of margin at end of life, to cover cold and cell-to-cell spread in R_0. The 0.47 F part leaves 0.395 V. Its leakage of about 1.5 µA costs $1.5/49.9 = 3$ % of the average current (Table 2.5), and its 2.6 µA larger sibling would cost 5 %. A rough check confirms the simulation. Over the 2.5 s upload the load draws $0.030 \times 2.5 + 0.25 \times 0.2 \times 1.0 = 0.125$ C. If the 12 Ω battery supplied nothing, the 0.47 F capacitor would drop by $0.125/0.47 = 0.27$ V. The battery supplies part of the charge, so the true drop from 3.65 V to 3.40 V (0.25 V, partly across the ESR) is plausible. A simulation result that a one-line estimate cannot roughly reproduce should not be believed until it is explained. CCM-1 Rev C uses 0.47 F, ESR ≤ 0.25 Ω, and the open risk on option B in Table 2.6 is closed.

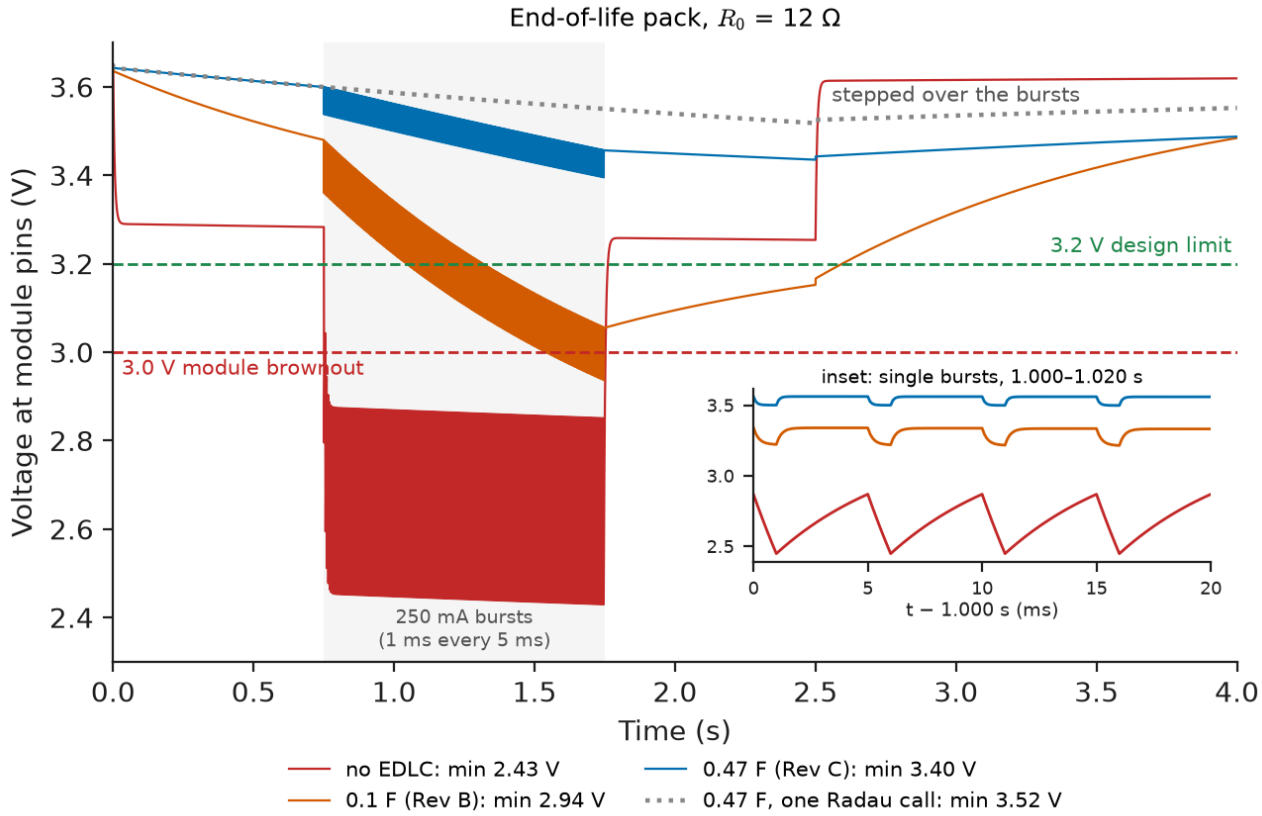


Figure 3.4 — Supply voltage at the module pins during an LTE-M upload with an end-of-life pack (R0 = 12 ohm), for three buffer choices.

A circuit with switching converters, protection diodes and vendor device models belongs in a circuit simulator (LTspice, ngspice) instead. The workflow is unchanged; the usual pattern is to extract compact parameters from SPICE (an effective R_0, an efficiency curve) for the system-level model (Chapter 29).

3.5 Numerical solver selection

Stiffness and discontinuities

> **Boundary.**
> Runge–Kutta and multistep methods, step-size control, absolute stability and stiffness are developed in *The Mathematics Universe*, Ch. 22 §22.9 and §22.11, with the modeling view of stiffness in Ch. 25 §25.9. This section is about choosing among them for a real model.

Two properties of a model decide the solver. The first is **stiffness**: time constants spread over many decades. The eigenvalues of M2's system matrix (with $R_0 = 12\ \Omega$ and 0.47 F) give time constants of 115 µs (bulk capacitor through the parallel source resistances), 4.4 s and 11.8 s (EDLC and battery RC), a ratio of about 10^5. An explicit method's step is limited by *stability* to roughly the smallest time constant, even when nothing is happening on that scale. An implicit method's step is limited only by *accuracy*. The second property is **discontinuity**: the load switches 400 times in one upload. An adaptive solver of any kind assumes a smooth right-hand side between steps. A 1 ms burst that falls between two steps is never seen, and the step-size controller cannot know.

Table 3.4 — Solvers on M2 (R0 = 12 Ω, 0.47 F, 4 s window, rtol 1e-6; desktop PC, SciPy 1.18).

Method	Steps	RHS evaluations	Wall time	V_{min}	Verdict
RK45 (explicit), default	15,012	114,854	15.8 s	3.395 V	Correct, because stiffness forced tiny steps
Radau (implicit), default	62	705	0.17 s	3.518 V	**Wrong** by +123 mV: stepped over the bursts
BDF (implicit), default	112	273	0.06 s	3.518 V	**Wrong**, same reason
LSODA (switching), default	209	631	0.05 s	3.518 V	**Wrong**, same reason
Radau, max step 0.2 ms	28,350	242,269	31.5 s	3.395 V	Correct, wasteful
LSODA, max step 0.2 ms	41,234	87,894	3.9 s	3.395 V	Correct
Radau, restarted at each load edge	—	46,888	3.7 s	3.395 V	Correct by construction
Exact discretisation per segment (linear circuit)	2,015	—	0.03 s	3.395 V	Correct, fastest; needs linearity

The dangerous rows are the fast ones: three implicit solvers return in a twentieth of a second with an answer 123 mV too optimistic and no warning, the opening story's failure. RK45 is right only by accident, because stiffness forces steps of about 0.27 ms, small enough to catch every burst. Extending the run to the full hour between uploads shows the cost of that accident: at 3.8 steps per simulated millisecond, about 14 million steps and close to 4 hours of wall time, against seconds for Radau with restarts or the exact discretisation.

> **Status of the principle: Explicit integrators on stiff systems are stability-limited**
> *Derived.* For the test equation $\dot{y} = \lambda y$, an explicit method is stable only for $h|\lambda|$ below a constant of order 1 to 3, so its step is set by the fastest mode, $h \lesssim 2.8\,\tau_{min}$ for classical RK4-type methods. A-stable implicit methods have no such limit. This holds for linear stability around smooth solutions; discontinuities limit every method equally unless the solver is told where they are.

Rules that follow

For each model, the solver choice is recorded with its reason:

1. **Tell the solver where the discontinuities are**: integrate piecewise between known events, or use event location for state-dependent switches. A small `max_step` is an expensive fallback.
2. **For stiff models use an implicit or switching method** (Radau, BDF, LSODA). Reach for an explicit method only when the stiffness ratio is modest, as it is for M1 (time constants 3 min to 9.6 h).
3. **For linear models with piecewise-constant inputs use exact discretisation.** This is M1's method (Listing 3.1), and it is also how firmware filters are discretised.
4. **Verify the numerics separately from the model**: halve the tolerance or step, cross-check two methods, compare with a one-line estimate. This *solution verification* asks "are the equations solved right?", not validation's "are they the right equations?".

3.6 Model assumptions

The assumption register

Most model failures are assumptions that stopped being true. An **assumption register** lists each one with its justification, a quantitative check, and what happens to the conclusions if it fails, forcing the question "how would I know if this were wrong?"

> **Boundary.**
> Catalogues of standard modeling assumptions, how to make them quantitative and how to check them afterwards are in *The Mathematics Universe*, Ch. 25 §25.6.

Table 3.5 — Assumption register for M1 and M2.

ID	Assumption	Justification and check	If violated
A1	Vial is one lumped node	$Bi = 0.05 < 0.1$ (Section 3.2)	Probe lag underestimated; alarms later than modeled
A2	Vaccine load is one lumped node	Cartons with air gaps: Bi well above 0.1; inner vials lag outer ones	Load crossing times are for an average vial; the outermost vials, nearest the evaporator, freeze earlier. Covered by the margin in Section 3.8
A3	Liner lumped with air in C_a	Liner exchanges heat with air in minutes	Air spikes during door openings mispredicted (found in Section 3.9)
A4	Conductances constant	Natural convection varies about ±15 % over 0–30 K differences	Second-order effect on crossing times; covered by the ±20 % sensitivity
A5	Thermostat is ideal hysteresis on air	Real thermostats sense the evaporator plate	Cycling period wrong; average temperatures unchanged
A6	Door opening = conductance step	Air exchange is turbulent and incomplete	Spike shape approximate; probe response checked in validation
A7	Pack R_0 constant during a pulse	Li-SOCl₂ shows transient voltage delay when passivated	Brownout worse than modeled at first pulse after storage; 0.2 V margin
A8	EDLC is ideal C + ESR	Real EDLCs show distributed (ladder) behaviour at 1 ms	ESR effectively higher at short pulses; measure at 1 ms

A2 and A7 are the assumptions whose violation would make the design *unsafe*, not just inaccurate, so validation and testing target them. A reviewer who doubts A5 can see at once that it affects no decision.

> **Status of the principle: Lumped-node thermal model of a refrigerator**
> *Approximate*. Valid when each node is nearly isothermal ($Bi < 0.1$) and the coupling coefficients are constant. It predicts mean temperatures and crossing times to about 10 % for closed-door operation. It does not predict spatial gradients (top shelf against bottom, vials touching the evaporator plate) or the detailed shape of door transients.

3.7 Parameter estimation

Fitting to a chamber test

> **Boundary.**
> Linear and nonlinear least squares, Gauss–Newton and Levenberg–Marquardt, parameter covariance and identifiability are developed in *The Mathematics Universe*, Ch. 24 §24.7 and Ch. 25 §25.8.

Table 3.2's values come from handbooks. The refining test is a power outage in a climatic chamber at 32 °C: the loaded refrigerator sits at 4 °C, power is cut, and air, load (a reference sensor in a vaccine-simulant carton) and probe are logged each minute for 8 h. Until the chamber is free, Listing 3.3 fits a stand-in record

from a deliberately *richer* 4-node "plant" with the liner as its own node, plus 0.05 °C noise. Such a *recovery test* checks the estimator on data whose truth is known, and the extra node shows how the fit behaves when the model is structurally too simple, as every model is.

```
 # ccm1_fit.py -- estimate the 3-node model from an 8-h power-outage record at 32 degC
import numpy as np
from scipy.linalg import expm
from scipy.optimize import least_squares

t = np.arange(0, 8 * 60 + 1) * 60.0                      # 1-min samples, s

def simulate(A, B, x0, u):                               # constant input u over the test
    n, m = A.shape[0], B.shape[1]
    M = expm(np.block([[A, B], [np.zeros((m, n + m))]]) * 60)
    X = [x0]
    for _ in t[1:]:
        X.append(M[:n, :n] @ X[-1] + M[:n, n:] @ u)
    return np.array(X)

def model3(Ca, UA, Cv, Gav, Gpa, x0, Tamb=32.0):         # states [Ta, Tv, Tp]
    A = np.array([[-(UA + Gav + Gpa), Gav, Gpa], [Gav, -Gav, 0], [Gpa, 0, -Gpa]])
    A /= np.array([[Ca], [Cv], [45.0]])
    return simulate(A, np.array([[UA / Ca], [0], [0]]), x0, np.array([Tamb]))

# Stand-in for the chamber record: 4-node plant (air, liner, load, vial) + 0.05 degC noise
A4 = np.array([[-4.54, 3.0, 1.5, 0.04], [3.0, -3.75, 0, 0], [1.5, 0, -1.5, 0],
               [0.04, 0, 0, -0.04]]) / np.array([[1500], [4000], [15000], [45]])
truth = simulate(A4, np.array([[0], [0.75 / 4000], [0], [0]]),
                 np.array([4.0, 9.0, 4.0, 4.0]), np.array([32.0]))
meas = np.delete(truth, 1, axis=1) + np.random.default_rng(1).normal(0, 0.05, (len(t), 3))

names = ["Ca J/K", "UA W/K", "Cv J/K", "Gav W/K", "Gpa W/K"]
for fixed_Cv in (None, 15000.0):                         # Cv free, then Cv from weighing
    def res(p):
        q = list(p[:2]) + [fixed_Cv if fixed_Cv else p[2]] + list(p[-2:])
        return (model3(*q, meas[0]) - meas).ravel()
    p0 = [3000, 0.7, 1.0, 0.03] if fixed_Cv else [3000, 0.7, 12000, 1.0, 0.03]
    f = least_squares(res, p0, x_scale="jac")
    cov = np.linalg.inv(f.jac.T @ f.jac) * np.sum(f.fun**2) / (f.fun.size - len(p0))
    sd = np.sqrt(np.diag(cov))
    used = [n for n in names if not (fixed_Cv and n.startswith("Cv"))]
    print("Cv fixed" if fixed_Cv else "all free", f"rms = {np.sqrt(np.mean(f.fun**2)):.3f}",
          f"corr(Ca,UA) = {cov[0, 1] / (sd[0] * sd[1]):+.2f}")
    print("   " + ", ".join(f"{n} {v:.4g}+/-{s:.2g}" for n, v, s in zip(used, f.x, sd)))
```

```
 all free rms = 0.052 corr(Ca,UA) = +0.98
   Ca J/K 33.48+/-0.69, UA W/K 0.01045+/-0.00011, Cv J/K 256+/-2.8, Gav W/K 0.02553+/-0.00028, Gpa
W/K 0.03965+/-0.00016
Cv fixed rms = 0.133 corr(Ca,UA) = +0.95
   Ca J/K 3793+/-28, UA W/K 0.5844+/-0.0011, Gav W/K 1.52+/-0.003, Gpa W/K 0.04129+/-0.00045
```

Worked example — reading a fit that looks perfect. The all-free fit has the best residual (0.052 °C, equal to the noise) and tight-looking standard deviations, and it is nonsense. An air capacity of 33 J/K is less than the 120 J/K of the air alone, and UA = 0.010 W/K would mean a wall 70 times better insulated than polyurethane. The reason is **identifiability**. An unforced warming test constrains only the *ratios* G/C, the time constants. The absolute scale of all the capacities and conductances together is pinned only weakly, through the fixed vial capacity. The optimiser exploited that freedom to make the 3-node model mimic the missing liner node, and the Jacobian-based uncertainties, which are local, cannot reveal it. A correlation of +0.98 between C_a and UA is the warning sign. The fix is physical, not numerical: the load is weighed (3.6 kg water-equivalent gives $C_v = 15{,}000$ J/K) and fixed. The second fit is physically sensible. UA = 0.584 W/K is the series combination of the true wall (0.75 W/K) and liner coupling (3 W/K), $0.75 \times 3/3.75 = 0.60$ W/K. $C_a = 3{,}793$ J/K has absorbed part of the liner. G_{av} and G_{pa} come back within 3 % of the plant's values. Its residual (0.133 °C) is now well above the noise, an honest sign of the structural simplification recorded as A3.

> **Status of the principle: Identifiability of lumped thermal parameters from an unforced test**
> *Derived.* The free response of $\dot{\mathbf{x}} = A\mathbf{x} + B\mathbf{u}$ depends only on A and B, whose entries are ratios G/C. Scaling every G and C by the same factor leaves the response unchanged, so absolute values need either a known heat input (a heater test) or an independently measured capacity. The local covariance from $J^{\mathsf{T}}J$ does not show this when the fit settles in a compensating, wrong region.

3.8 Sensitivity analysis

Which parameters matter?

A **sensitivity analysis** asks how much an output of interest changes when each input changes. It decides where measurement effort, tolerances and margins should go. The cheapest form is **one-at-a-time** (OAT): move each parameter by a stated amount (here ±20 %) with the others at nominal, and report the normalised sensitivity $S = (\Delta y/y)/(\Delta p/p)$. The output chosen is the one that carries the safety decision: the **lead time**, from the probe freeze alarm to the load reaching −0.5 °C, in a thermostat-fault scenario starting from steady 4 °C.

> **Boundary.**
> Local sensitivities, adjoint methods and linearised (GUM) and Monte Carlo uncertainty propagation are in *The Mathematics Universe*, Ch. 24 §24.6 and §24.8.

```
 # ccm1_sens.py -- sensitivity of freeze-alarm lead time to the model parameters
import numpy as np
from scipy.linalg import expm

NOM = dict(Ca=3000.0, Cv=15000.0, Cp=45.0, UA=0.7, Gav=1.5, Gpa=0.04,
           Qc=60.0, Tamb=30.0, Toff=-4.0)

def lead_time(p, dt=5.0):
    """Hours from the probe freeze alarm to the load reaching -0.5 degC."""
    A = np.array([[-(p["UA"] + p["Gav"] + p["Gpa"]), p["Gav"], p["Gpa"]],
                  [p["Gav"], -p["Gav"], 0], [p["Gpa"], 0, -p["Gpa"]]])
    A /= np.array([[p["Ca"]], [p["Cv"]], [p["Cp"]]])
    B = np.array([[p["UA"] / p["Ca"], -1 / p["Ca"]], [0, 0], [0, 0]])
    M = expm(np.block([[A, B], [np.zeros((2, 5))]]) * dt)
    x, on, run, t_alarm = np.array([4.0, 4.0, 4.0]), True, 0, None
    for k in range(int(12 * 3600 / dt)):        # thermostat fault from t = 0
        on = (x[0] > p["Toff"] + 3) or (on and x[0] > p["Toff"])
        x = M[:3, :3] @ x + M[:3, 3:] @ np.array([p["Tamb"], p["Qc"] * on])
        if k % 12 == 11:                        # logger: one sample per minute
            run = run + 1 if x[2] <= -0.5 else 0
            if run == 60 and t_alarm is None:
                t_alarm = (k + 1) * dt / 3600
        if x[1] <= -0.5:
            return (k + 1) * dt / 3600 - (t_alarm or np.inf)
    return np.nan

y0 = lead_time(NOM)
print(f"nominal lead time = {y0:.2f} h")
for name in NOM:                                # one at a time, +/- 20 %
    lo, hi = (lead_time({**NOM, name: NOM[name] * f}) for f in (0.8, 1.2))
    print(f"{name:5s} -20%: {lo:5.2f} h  +20%: {hi:5.2f} h  "
          f"S = {(hi - lo) / (0.4 * y0):+.2f}")
```

```
 nominal lead time = 1.92 h
Ca    -20%:  1.91 h  +20%:  1.91 h  S = -0.00
Cv    -20%:  1.28 h  +20%:  2.58 h  S = +1.68
Cp    -20%:  1.97 h  +20%:  1.82 h  S = -0.19
UA    -20%:  1.95 h  +20%:  1.92 h  S = -0.05
Gav   -20%:  2.75 h  +20%:  1.37 h  S = -1.79
Gpa   -20%:  1.81 h  +20%:  1.97 h  S = +0.22
Qc    -20%:  1.89 h  +20%:  1.91 h  S = +0.03
Tamb  -20%:  1.95 h  +20%:  1.92 h  S = -0.04
Toff  -20%:  2.87 h  +20%:  1.37 h  S = -1.95
```

The nominal lead time (1.92 h) differs from Listing 3.1's 2.03 h because this scenario starts from a uniform 4 °C. The ranking is what matters (Figure 3.5). The load's capacity C_v and coupling G_{av} (together its time constant) and the fault severity T_{off} dominate. The probe parameters C_p and G_{pa}, which the design controls through SP2, matter an order of magnitude less, and UA, Q_c, C_a and ambient hardly at all. The lead time is governed by what is in the refrigerator and how badly it fails, which the design cannot control: a margin question, not a tolerance question.

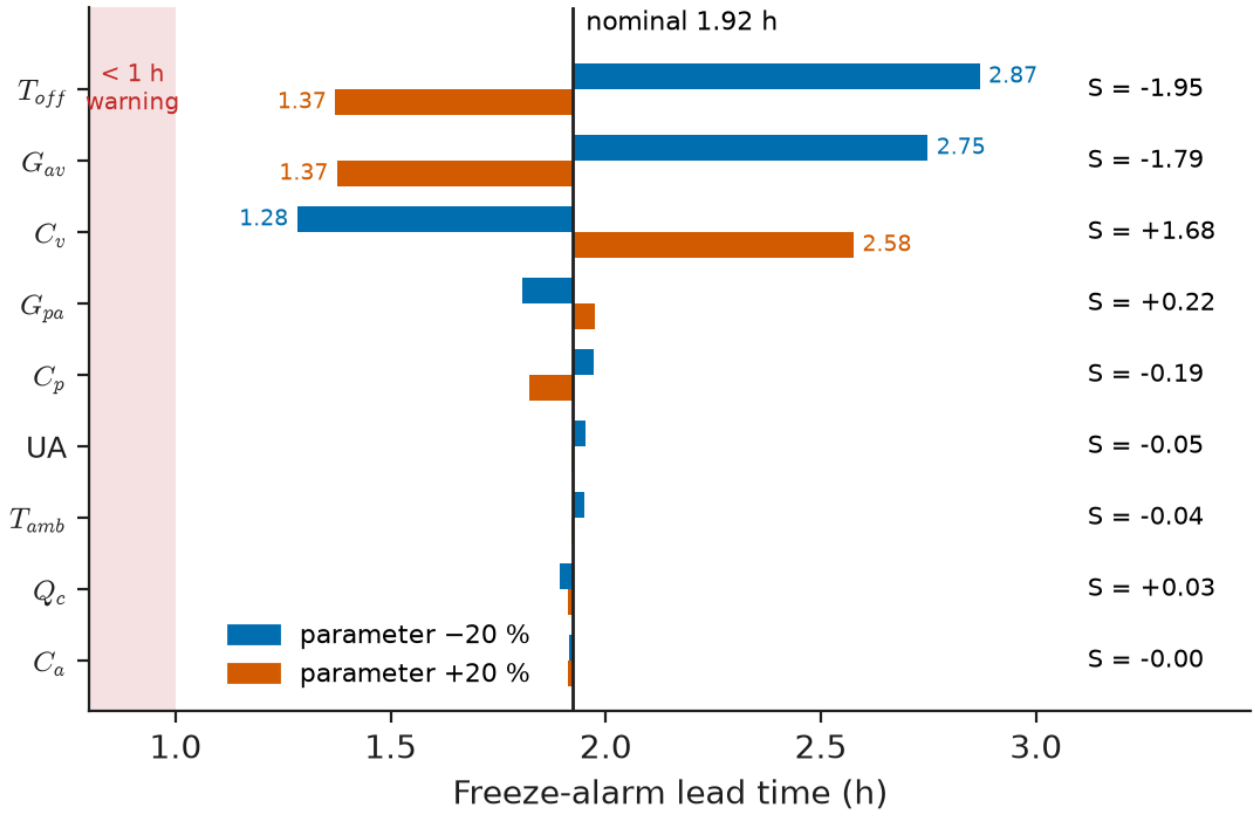


Figure 3.5 — Tornado chart: freeze-alarm lead time for each parameter at -20 % and +20 %, nominal 1.92 h.

Monte Carlo over plausible refrigerators

OAT misses interactions and only explores near the nominal point. A Monte Carlo run samples all the uncertain inputs together over plausible ranges: C_v ±40 %, G_{av} ±30 %, G_{pa} ±20 %, C_p ±10 %, T_{off} uniform from −6 to −1.5 °C, ambient 20–43 °C. Rerunning `lead_time` for 400 samples (seed 3, about 2 min) gives the following. In 56 samples the load never reaches −0.5 °C within 12 h (mild faults). Of the 344 that freeze, the median lead time is 2.08 h and the 5th percentile 0.27 h. In **8 of the 344 the load freezes before the probe alarm**. These are mild faults in which the air cycles around −0.5 °C, so the probe dips below the threshold and back again many times and never completes 60 *continuous* minutes, while the heavy load drifts steadily below freezing.

Worked example — a sensitivity result that changes a requirement. The 8 late cases are a rule problem, not a parameter problem: the continuous-duration rule of R3 resets on any single sample above −0.5 °C. On the same 400 samples the team tested a *freeze warning* raised when the 60-minute moving mean of the probe is at or below −0.5 °C. Late cases fell from 8 to 0, the 5th-percentile lead time rose from 0.27 h to 0.68 h and the median from 2.08 h to 2.51 h. The price was 9 warnings instead of 4 in the 56 non-freezing cases, all situations in which the probe averaged below freezing for an hour, which a nurse would want to know. The change goes back to Chapter 1 as requirement R3a, "raise a freeze warning when the 60-min mean probe temperature is ≤ −0.5 °C", beside the unchanged WHO-compatible alarm of R3, and the trace of Section 1.8 lists what must change: SP4, `alarm_eval`, AT2 and the users to consult. Found now, it costs a code change; found in the field, a box of vaccine.

> **Status of the principle: One-at-a-time sensitivity**
> *Approximate.* OAT normalised sensitivities are first-order and local. They rank parameters correctly when the response is near-linear and interactions are weak over the ranges explored. They miss interactions and threshold effects, such as the late-alarm cases above, which only a global method (Monte Carlo, variance-based indices) reveals.

3.9 Model validation

Verification, calibration, validation

Verification checks that the equations are solved correctly (Section 3.5); **calibration** adjusts parameters to fit data (Section 3.7); **validation** tests whether the calibrated model predicts *other* data, from conditions it was not fitted to, within the accuracy its question needs (*The Mathematics Universe*, Ch. 25 §25.7). A model that fits its calibration data has shown only that it can be bent. Criteria are written *before* the data are seen, from the model's question: for M1, (a) load crossing time of 8 °C within ±10 %, (b) probe crossing time within ±10 min, (c) probe RMS error ≤ 0.3 °C.

Three independent records are used. Until the chamber is available they too come from the 4-node plant: their value now is to rehearse the procedure and expose structural limits, and the procedure is reused unchanged on the chamber records.

Table 3.6 — Validation of M1 (fitted parameters of Section 3.7, C_v set from the weighed load).

Record	Conditions	Probe RMS error	Probe crossing 8 °C (plant / model)	Load crossing 8 °C (plant / model)	Result
V0 (calibration)	Outage, 32 °C, full load	0.133 °C (fit)	n/a	n/a	Not validation
V1	Outage, 43 °C, full load	0.226 °C	25.6 / 30.3 min (+4.7 min)	97.9 / 99.5 min (+1.6 %)	Pass (a), (b), (c)
V2	Outage, 25 °C, half load (C_v = 7,500 J/K)	0.120 °C	49.3 / 52.4 min (+3.1 min)	102.9 / 104.6 min (+1.7 %)	Pass (a), (b), (c)
V3	Single 60 s door opening, 30 °C	Air rise 9.50 K plant / 4.96 K model; probe rise 2.06 / 2.37 K	n/a	n/a	Air: **fail** (−48 %); probe: within +15 %, conservative

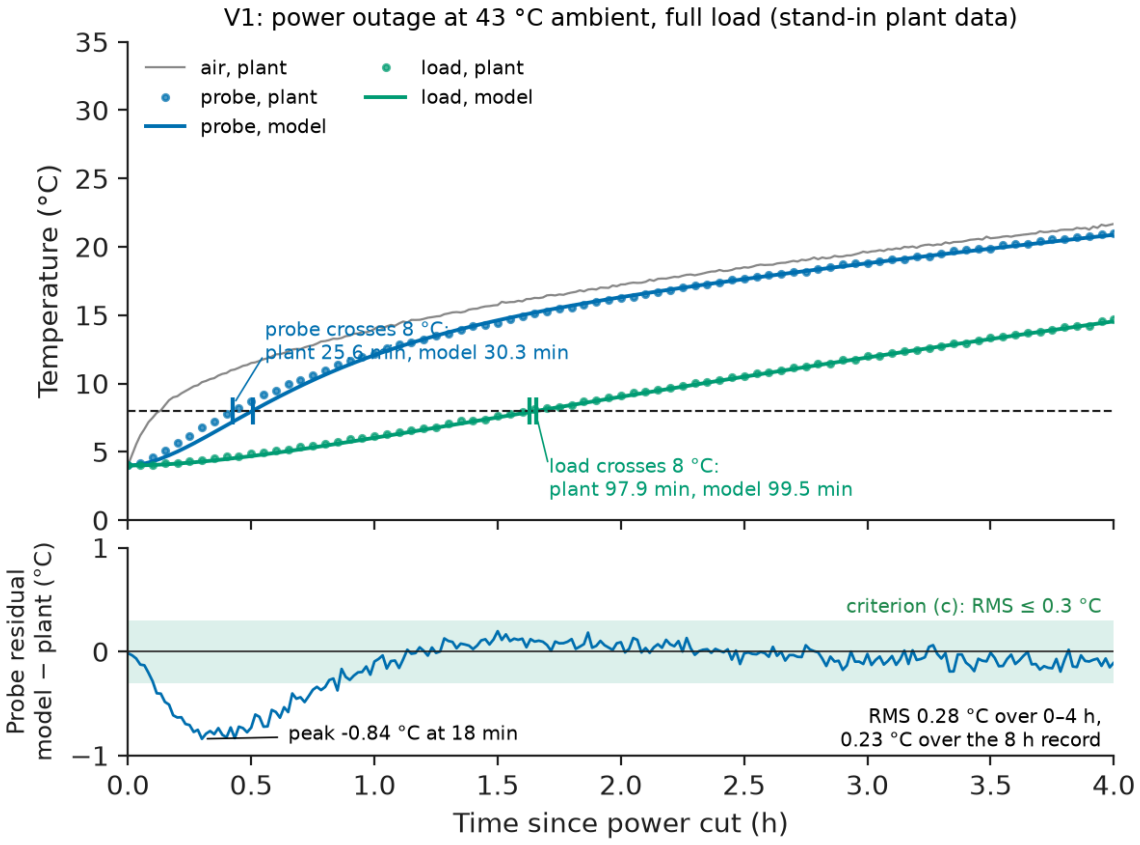


Figure 3.6 — Validation record V1: 43 degC power outage, plant (stand-in for chamber data) against the calibrated 3-node model.

The result is a **domain of validity**, not a yes or no. M1 predicts outage warming, and so probe and load crossing times, well across ambient temperatures and load sizes, and can carry the alarm-timing decisions. It does *not* predict air during door openings: lumping the liner into C_a (A3) makes its air too sluggish, halving the spikes. No decision depends on air spikes, because SP7 removed air from the alarm logic, so the limitation is acceptable *and recorded*; the probe rise it predicts is 15 % high, erring towards false alarms and therefore safe for R6. The model is also 3–5 min *late* on probe crossings, immaterial for a 10-hour heat rule, and freeze timing is not validated at all, since every record is a warming test. A cold-fault chamber test is added to the test plan.

Table 3.7 — Model verification and validation status against the model register.

Model claim	Check	Limit	Result	Status
M1 solved correctly	Exact discretisation; rerun with 1 s step instead of 5 s	Crossing times change ≤ 5 min	Alarm 0 min, load 1.8 min	Pass
M1 warming crossings	V1, V2	±10 % load, ±10 min probe	+1.7 % max, +4.7 min max	Pass (stand-in data)
M1 freeze timing	Cold-fault chamber test	±10 min	Not yet run	Open: chamber test
M1 door transients	V3	Air ±20 %	−48 %	Fail: outside validity domain
M2 solved correctly	Two methods agree (Table 3.4)	±5 mV	0 mV	Pass
M2 minimum voltage	Bench pulse test, emulated pack	±50 mV	Not yet run	Open: bench test

---

3.10 Simulation reproducibility

What must be recorded

A result that cannot be rerun cannot be checked, defended at a review, or compared when the model changes. Five things are recorded with every result that enters a decision: model version (code under version control), configuration (parameters, scenario, solver, tolerances), random seed, software environment, and input data by content hash. Lock files, containers and deterministic reductions are in *Electronics and Computing*, Ch. 24 §24.8; seeded streams and floating-point non-associativity in *The Mathematics Universe*, Ch. 24 §24.9.

Every listing here fixes its seed, which is why "31 alarms" is a reproducible number rather than a lucky draw; another seed gives another door schedule and count, while the *conclusion* (dozens of air alarms, none from the probe) is seed-independent, which a few reruns confirm. Listing 3.5 writes the run record that accompanies Listing 3.1's results.

```
 # run_manifest.py -- record what a CCM-1 simulation run depended on
import hashlib, json, platform, sys
import numpy as np, scipy

config = {                                  # the run's inputs, normally read from TOML
    "model": "ccm1_system", "model_version": "3.2.0",
    "params": {"Ca": 3000.0, "Cv": 15000.0, "Cp": 45.0, "UA": 0.7, "Gav": 1.5,
               "Gpa": 0.04, "Gdoor": 15.0, "Tamb": 30.0, "Qc": 60.0},
    "solver": {"method": "exact-zoh", "dt_s": 5.0},
    "scenario": {"days": 7, "doors_per_day": 15, "fault_h": 114.0},
    "seed": 7,
}
canon = json.dumps(config, sort_keys=True, separators=(",", ":")).encode()
manifest = {
    "config_sha256": hashlib.sha256(canon).hexdigest()[:16],
    "python": platform.python_version(), "numpy": np.__version__,
    "scipy": scipy.__version__, "platform": sys.platform,
    "results": {"air_rule_alarms_4d": 31, "freeze_alarm_h": 115.40,
                "load_below_m0p5_h": 117.43},
}
print(json.dumps(manifest, indent=1))
```

```
 {
  "config_sha256": "735e7d4afc8eec53",
  "python": "3.14.7",
  "numpy": "2.5.4",
  "scipy": "1.18.1",
  "platform": "win32",
  "results": {
   "air_rule_alarms_4d": 31,
   "freeze_alarm_h": 115.4,
   "load_below_m0p5_h": 117.43
  }
}
```

In the project the simulation fills in the results block and the manifest is committed with the report. The hash is taken over a *canonical* serialisation (sorted keys, fixed separators), so identical configurations hash alike and any change, even of one tolerance, gives a new hash. A report can then cite "Figure 3.3, run 735e7d4a" and anyone can check that figure, configuration and code agree. Chapter 5 sets out the version-control and release practice, Chapter 29 the build automation.

> **Status of the principle: Bitwise reproducibility of a simulation run**
> *Approximate.* With the same code, configuration, seed and library versions on the same platform, a run reproduces bit for bit. Across platforms or library versions, results agree only within numerical-error tolerances, because linear-algebra libraries and parallel reductions may sum in different orders. Regression tests therefore compare decision quantities with tolerances, not bit patterns.

Failure analysis of the modeling workflow

Every failure in this chapter came from the workflow, not from the physics, and each has a check that would have caught it.

Table 3.8 — FMEA of the CCM-1 modeling workflow.

Failure mode	Effect	Cause	Detection	Mitigation
Solver steps over load pulses	Brownout margin overstated by 123 mV; 0.1 F built (Rev B)	Adaptive solver blind to discontinuities	Cross-check two methods; one-line charge estimate	Integrate between events; exact discretisation (Section 3.5)
Non-identifiable fit accepted	Physically absurd parameters with tight error bars	Unforced test constrains only ratios	Physical bounds; parameter correlations > 0.95	Fix a measured capacity; design a forced test
Model used outside its domain	Air-spike predictions 48 % low	Liner lumped with air (A3)	Validation record V3	State the domain; decisions avoid air transients
Validation on warming only	Freeze timing untested	All records were outage tests	Validation matrix vs. model claims (Table 3.7)	Cold-fault chamber test added to the test plan
Rule fragility hidden by nominal runs	Load freezes before alarm in 2.3 % of freezing cases	Continuous-duration rule; OAT only	Monte Carlo over plausible refrigerators	Requirement R3a (moving-mean warning)
Lucky seed	Result depends on one random schedule	Single stochastic run	Rerun with several seeds	Seeds recorded; conclusions checked across seeds
Unreproducible result	Figure cannot be regenerated at review	Parameters edited by hand; versions unknown	Manifest and config hash missing	Run manifest committed with every result

What remains open

Open question	Where it is answered
How are prototypes, instruments, test plans and calibrations organised so that open rows like those of Table 3.7 are closed by measurement?	Chapter 4
How are model code, configurations and manifests kept under version control and released with the design?	Chapter 5; Chapter 29
How does a measurement chain's uncertainty propagate into the probe reading used by M1?	Chapter 7
How are numerical solvers, parameter sweeps and optimisation used at scale, and their results verified?	Chapter 30
How are models calibrated continuously against field data, as digital twins with quantified uncertainty?	Chapter 33
How is a complete engineering simulation tool built, verified and documented?	Chapter 39
What theory underlies stiffness, least squares and uncertainty propagation?	*The Mathematics Universe*, Ch. 22, Ch. 24, Ch. 25

Next: Chapter 4 moves from models to hardware: how prototypes are planned, instruments chosen, tests written with pass/fail limits, calibrations done, faults diagnosed and regressions caught, on a four-channel DC power monitor whose first PCB read 3 % high.

Chapter 4 — Prototyping and Verification

Part I · Engineering Design Methodology

A small team builds PM-4, a four-channel 12 V DC power monitor that sits between the 12 V bus of an edge-computing rack and four servers and reports current, voltage, power and energy per port over RS-485. The breadboard reads within 0.1 % of a reference meter. The first printed circuit board reads 3 % high at 10 A, and the error grows another 0.7 % as the board warms. Nothing on the schematic changed between the two builds. What changed was a piece of copper: on the breadboard the current-sense amplifier was wired to the voltage terminals of a four-terminal shunt module; on the PCB its sense traces left the shunt from the edge of the current-carrying pads, so 60 μΩ of copper with a temperature coefficient of 3,930 ppm/°C ended up inside the measurement. The fault took two days to find because nobody had planned which prototype was supposed to answer which question, the test plan checked only one current at room temperature, and the reference meter on the bench was not accurate enough to tell a 0.5 % error from a good board. This chapter is about the work between a model that predicts a design will meet its requirements (Chapter 3) and the evidence that it does.

> **What must each prototype prove, and how do you choose the instruments, tests and limits that prove it?**
> **When a prototype fails, how do you find the cause quickly, fix it, and make sure the same fault never comes back unnoticed?**

Concept	Engineering destination
Prototype planning	A build sequence (breadboard, EVT, DVT, PVT) where each build retires named risks
Breadboarding	Fast proof of firmware, interfaces and circuit concepts before copper is committed
PCB prototyping	Layout-dependent behaviour: Kelvin sensing, thermal paths, EMC, testability
Instrument selection	Reference instruments with a test-uncertainty ratio that makes pass/fail decisions meaningful
Test plans	Test cases traced to requirement IDs, with conditions, limits and pass/fail rules
Verification versus validation	"Built right" against the specification versus "right product" in the user's rack
Calibration	Gain and offset correction of every unit against traceable references, with guard-banded limits
Fault diagnosis	Half-splitting, hypothesis tests and root-cause records instead of trial-and-error
Regression testing	Automated re-runs of the verification suite on every firmware build and board revision
Design iteration	Revision B: changes justified by failures and validation findings, closed by re-test

Roadmap. Plan the builds and what each must prove (4.1) → cheap concept builds (4.2) → the first boards and what only copper can reveal (4.3) → instruments good enough to judge (4.4) → test plans (4.5) and the

difference between verifying and validating (4.6) → calibration of every unit (4.7) → diagnosing what went wrong (4.8) → keeping it fixed (4.9) → the next revision (4.10).

4.1 Prototype planning

The running example: PM-4

PM-4 is small enough to follow in detail and has every ingredient of a larger system: an analog measurement whose accuracy depends on layout, firmware, a field bus, a power supply, an enclosure, a calibration step in production, and users with expectations nobody wrote down. Chapters 1–3 showed how requirements, an architecture and a model are produced; here they are taken as given and turned into hardware. Table 4.1 is the specification the prototypes are built to verify.

Table 4.1 — PM-4 requirements (abridged).

ID	Requirement	Target	Verification method
R1	Current accuracy, each channel, 0–10 A, 0–50 °C ambient	±(0.5 % of reading + 10 mA)	Test against reference shunt, 11 points, 25 °C and 50 °C
R2	Bus-voltage accuracy, 10.8–13.2 V	±(0.2 % of reading + 10 mV)	Test against 6½-digit DMM
R3	Power accuracy, 1–10 A	±1.0 % of reading	Computed from R1, R2 test data; spot test
R4	Update period of all channels	1.00 s, jitter ≤ 10 ms	Logic-analyser capture of Modbus responses
R5	Interface	Modbus RTU over RS-485, 19.2 kbaud, 32 units per segment	Protocol conformance test
R6	Self-consumption from the 12 V bus	≤ 0.5 W	Input power measurement
R7	Insertion voltage drop per channel at 10 A	≤ 30 mV	Four-wire voltage measurement
R8	Ambient temperature range	0–50 °C operating	Chamber test
R9	ESD immunity	IEC 61000-4-2, ±4 kV contact, criterion B	Accredited lab, pre-scan in house
R10	Unit cost at 1,000 pcs	≤ $30.00	BOM roll-up (Chapter 5)

The architecture (Figure 4.1) is one INA228-class 20-bit current/power monitor per channel, each reading a 2 mΩ shunt in the positive rail, all four on one I²C bus to an STM32G071-class microcontroller that runs Modbus on an RS-485 transceiver. A 4.5–36 V buck converter makes 3.3 V from the monitored bus itself.

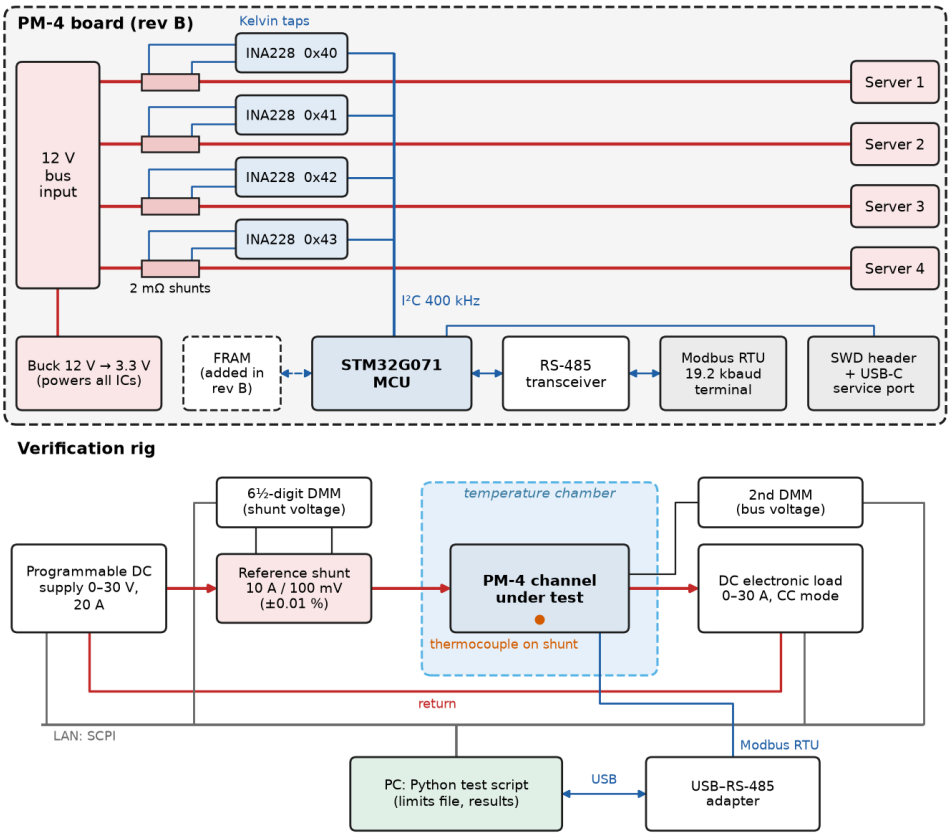


Figure 4.1 — PM-4 architecture (top) and the bench rig used to verify it (bottom).

A prototype answers questions

A prototype is an experiment, and an experiment without a question produces anecdotes. Planning starts from the risk list (every requirement or assumption that could be wrong, ranked by likelihood and by the cost of finding out late), and each build is assigned the risks it must retire and the tests that retire them. Table 4.2 is the usual ladder; its names come from consumer and automotive practice, its logic applies to a 50-unit industrial product as much as to a phone.

Table 4.2 — The prototype ladder for PM-4.

Build	Form	Quantity	Questions it must answer	Exit criterion
Concept (breadboard)	Evaluation boards, shunt module, MCU dev kit	1	Does the INA228 + firmware + Modbus chain work? Is the error budget plausible?	R1 met at 25 °C on 1 channel; Modbus read of all registers
EVT (engineering validation test)	First PCB, hand or quick-turn assembled	5	Does the layout preserve accuracy? Thermal rise? Power consumption? Debug access?	R1–R7 met at 25 °C on all channels of 3 boards
DVT (design validation test)	Final layout, production parts, real enclosure	22	Does the design meet all requirements over temperature, ESD, unit-to-unit spread?	Full verification table passes; no open severity-1 defects
PVT (production validation test)	Built by the contract manufacturer on the production line with the production tester	100	Can it be built and calibrated at yield and cost?	First-pass yield ≥ 95 %; test time ≤ 90 s; R10 met

Two rules keep the ladder honest: a build starts only when the previous exit criterion is met or a reasoned waiver is signed (skipping EVT is how a layout error reaches 22 DVT units), and each quantity is set by what must be estimated, not by what the budget leaves over.

Worked example — How many DVT units? The DVT must show, with 90 % confidence, that at least 90 % of units meet R1 over temperature. If n units are tested and none fails, the success-run relation from the binomial distribution gives the minimum sample size:

$$n = \frac{\ln(1 - C)}{\ln R} = \frac{\ln 0.10}{\ln 0.90} = 21.9 \;\Rightarrow\; 22 \text{ units}$$

Raising the demonstrated reliability to 95 % at the same confidence needs $\ln 0.10 / \ln 0.95 = 44.9$, so 45 units, which doubles the DVT cost; lowering it to 80 % needs 11. The team chose 22, because unit-to-unit spread in PM-4 comes mainly from the shunt tolerance, which calibration removes, and because PVT adds 100 more units of evidence. One failure among the 22 voids the demonstration; the units are then analysed, the cause fixed, and the sample re-run.

> **Status of the principle: Success-run sample size**
> *Derived.* Follows from the binomial probability of zero failures in n independent trials, $R^n \le 1 - C$; it assumes the units are representative of production and that the test actually exercises the failure mechanism, which no formula can check.

PM-4's five largest risks at the start were layout error in the 2 mΩ sense path, shunt self-heating, I²C robustness, buck-converter noise coupling, and cost. The concept build attacks the I²C risk; EVT the layout, heating and noise; DVT everything over temperature; PVT the cost. A risk that no build is scheduled to retire is a risk the project has decided to discover in the field.

4.2 Breadboarding

What a breadboard is for

A breadboard (in the broad sense: solderless board, evaluation modules, dev kits and jumper wires) is the cheapest way to answer questions about *function*: does the chip respond, does the firmware read the right registers, does the protocol stack interoperate, is the error budget of the right order. It is a poor way to answer questions about *parasitics*: contact resistance, stray capacitance, loop inductance and thermal paths are all different from the final board, and sometimes worse by orders of magnitude.

Table 4.3 — Solderless breadboard and jumper parasitics (typical).

Parameter	Typical value	Consequence
Contact resistance per spring contact	10–50 mΩ, varies with insertion and age	Useless for milliohm current paths; unstable in any current path above about 1 A
Current rating of a contact	about 1 A continuous	Contacts heat and loosen at 10 A
Capacitance between adjacent rows	2–5 pF	Limits high-impedance and above-10 MHz circuits
Jumper wire inductance	about 10 nH per cm	Ringing on fast edges; switching converters will not behave as on PCB

Worked example — Why the 10 A path stays off the breadboard. A 10 A test current through one breadboard contact of 20 mΩ drops $10 \times 0.020 = 0.20$ V and dissipates $10^2 \times 0.020 = 2.0$ W in a spring the size of a grain of rice; the plastic softens within a minute. More subtly, if the INA228 sense inputs were taken from breadboard rows rather than from the shunt's own terminals, a 20 mΩ contact in series with a 2 mΩ shunt would make the reading wrong by a factor of eleven. The concept build therefore uses a four-terminal 2 mΩ shunt module bolted to 2.5 mm² wire, with the INA228 evaluation module's sense inputs wired by a twisted pair directly to the module's voltage terminals; only the low-current logic (I²C, power, RS-485) goes through the breadboard.

> **Status of the principle: Breadboards prove function, not parasitics**
> *Empirical.* A breadboard reproduces logic, firmware and low-frequency signal behaviour reliably; any result that depends on milliohms, picofarads, nanohenries or thermal paths must be re-measured on the PCB.

What the PM-4 concept build proved

In two weeks the concept build showed that the firmware reads four INA228s at 400 kHz I²C (rise time 180 ns with 4.7 kΩ pull-ups, against the 300 ns fast-mode limit), that the Modbus stack interoperates with two commercial masters, that the error at 25 °C is within ±0.1 % from 1 to 10 A after a two-point calibration, and that the noise on a 1 s average is 0.3 mA rms. The shunt module rose 18 °C at 10 A, which put shunt drift on the EVT test list with a number attached. The concept build proved nothing about the PCB sense path, and the team's mistake was to treat its 0.1 % result as if it had.

4.3 PCB prototyping

What only copper can reveal

The first PCB build is where layout-dependent requirements are tested for the first time. For PM-4 these are the four-wire (Kelvin) sense connection of each shunt, the thermal path that sets shunt temperature, the coupling of the buck converter's 1 MHz switching node into the sense traces, and ESD paths from the field terminals. Quick-turn fabrication makes the first board cheap compared with the cost of an error discovered later: in 2026 a five-piece order of a 100 × 80 mm four-layer board costs roughly $60–$150 with a five-working-day turnaround, and partial assembly by a quick-turn assembler adds about $200–$400 in set-up.

Design for test and debug

An EVT board is built to be probed, reworked and measured. The PM-4 EVT layout included features that the production board would later drop or shrink:

- **Test points** on every rail, the I²C and RS-485 lines and each alert output, on a 2.54 mm grid that a production bed-of-nails fixture can reuse.
- **Zero-ohm links** in each block's supply, so current can be measured per block and a block isolated during diagnosis.
- **Kelvin test pads** on each shunt body, separate from the INA228 sense traces, so the shunt voltage can be measured independently of the amplifier.
- **An SWD header**, a UART log output and option footprints (a second RC filter stage on the sense lines), populated only if tests demand them.

The error budget the PCB must preserve

The EVT test limits come from the error budget written before layout. Table 4.4 is the budget for one channel at 10 A, after the per-unit calibration of Section 4.7 at 25 °C board temperature, with the worst-case board temperature of 70 °C (50 °C ambient plus 20 °C self-heating) at the hot end.

Table 4.4 — Current error budget, one channel at 10 A, after calibration.

Contribution	Basis	Error at 10 A
Shunt temperature coefficient	50 ppm/°C × 45 °C	0.225 %
INA228 gain drift	20 ppm/°C max × 25 °C die rise	0.050 %
Calibration reference	Reference shunt plus DMM, Section 4.4	0.020 %
Noise on a 1 s average	Concept build measurement	0.010 %
Nonlinearity after two-point calibration	Concept build residuals	0.010 %
Offset after calibration	0.5 mA residual	0.005 %
Worst-case sum		**0.320 %**
Root sum of squares		**0.232 %**
Requirement R1 at 10 A	0.5 % + 10 mA	**0.600 %**
Margin (worst case)	(0.600 − 0.320)/0.600	**47 %**

The budget is the contract between the designer and the layout: the shunt drift entry assumes that the only temperature-sensitive resistance inside the sense loop is the shunt itself. Section 4.8 shows what happened when the layout broke that assumption.

> **Boundary.**
> How individual error terms combine (worst case, root sum of squares, and expanded uncertainty per GUM/JCGM 100) belongs to Chapter 7 of this book, with the statistics in *The Mathematics Universe*, Ch. 20 §20.10. The INA228's internal ADC architecture and the reference behind it are circuit topics of *Electronics and Computing*, Ch. 10 §10.7 and §10.10. Here the budget is a test limit.

4.4 Instrument selection

The instrument must be better than the requirement

A test can only decide pass or fail if the measuring system is substantially more accurate than the tolerance being checked. Practice expresses this as the **test-uncertainty ratio** (TUR): the tolerance of the requirement divided by the uncertainty of the measurement used to check it. Calibration laboratories and most quality systems ask for a TUR of at least 4:1 (ANSI/NCSL Z540.3 uses the closely related probability of false accept, at most 2 %); below that, either the limits must be guard-banded (Section 4.7) or a better instrument found.

> **Status of the principle: The 4:1 test-uncertainty ratio**
> *Empirical.* A convention from metrology practice that keeps the probability of accepting a bad unit to a few percent for typical distributions of unit errors; it is a rule of thumb, not a law, and a decision rule with explicit guard bands (ISO/IEC 17025, ILAC G8) replaces it when the ratio cannot be met.

Worked example — Choosing the current reference. R1 at 10 A allows ±60 mA, that is ±0.60 %. Two ways of measuring the reference current were on the bench:

1. A 6½-digit bench DMM on its 10 A range. A typical 1-year specification for this class is 0.15 % of reading + 0.008 % of range, so 0.158 % at 10 A. TUR $= 0.60/0.158 = 3.8$, below 4:1 and, worse, the DMM's own 10 A shunt self-heats during a long sweep.
2. A 10 A / 100 mV reference shunt with a calibration certificate of ±0.010 %, read by the same DMM on its 100 mV range, typically 0.0050 % of reading + 0.0035 % of range, so 0.0085 % at full scale. The worst-case sum is $0.010 + 0.0085 = 0.0185$ %, giving TUR $= 0.60/0.0185 = 32$.

Option 2 costs one reference shunt (about $400–$900 with a certificate) and makes every later decision clear. It also measures what R1 asks for: current in the external circuit, independent of the DUT.

The PM-4 instrument set

Table 4.5 is the selection made for EVT and DVT. Each row starts from the requirement it serves; a "nice" instrument that serves no requirement is not bought.

Table 4.5 — Instrument selection for PM-4 verification.

Need (requirement)	Candidate classes	Key specification needed	Chosen	Why
Reference current, 0–10 A (R1, R3)	DMM 10 A range; reference shunt + DMM 100 mV range; precision current source	Uncertainty ≤ 0.15 % (TUR 4)	Reference shunt 10 A/100 mV ±0.01 % + 6½-digit DMM	TUR 32; independent of DUT
Bus voltage reference (R2)	4½-digit handheld; 6½-digit DMM	≤ 0.07 % at 12 V	Second 6½-digit DMM, 100 V range	About 0.004 % typical; TUR > 50
Source and sink 0–10 A at 12 V	Bench supply + resistor bank; programmable supply + DC electronic load	Programmable, ≥ 12 A, SCPI over LAN	30 V/20 A programmable supply; 30 A DC electronic load (CC mode)	Automated sweeps, repeatable steps
Timing, jitter (R4)	Oscilloscope; logic analyser with protocol decode	1 µs resolution, Modbus/UART decode, 60 s record	8-channel logic analyser	Long captures, protocol-level timestamps
Temperature (R1 at 50 °C, R8)	Oven; temperature chamber; thermal camera	0–70 °C, ±1 °C uniformity	Benchtop chamber plus type T thermocouples on shunts; thermal camera for EVT	Chamber for limits, camera for finding hot spots
Switching noise coupling	100 MHz scope; 200 MHz 12-bit scope with differential probe	12-bit vertical, 1 mV/div	200 MHz 12-bit oscilloscope	Sees 1 MHz buck ripple on sense lines
ESD (R9)	Accredited lab; in-house ESD gun	IEC 61000-4-2 levels	In-house gun for pre-scan; lab for the formal test	Find failures before paying for the lab

Every instrument used for a verification result carries an asset number and a calibration due date, both recorded in the test report (Chapter 5); an instrument out of calibration on the day of a test turns a pass into "unknown".

4.5 Test plans

Anatomy of a test plan

A test plan converts each requirement into one or more test cases, each of which can be executed by someone other than its author and gives the same verdict. A useful test case has seven fields: **ID and requirement trace** (TC-R1-03 → R1); **purpose**; **set-up** (equipment by asset number, wiring diagram, firmware build, unit serial); **conditions** (temperature, supply voltage, load); **procedure** (steps, settling times, number of readings); **limits** (numerical, including guard band); and **pass/fail rule** (all points within limits, or a statistical rule). A plan also says what happens on failure: stop, record, notify, and whether the unit may be re-tested.

Table 4.6 — Excerpt of the PM-4 DVT test plan.

Test case	Trace	Conditions	Procedure	Limit	Rule
TC-R1-01	R1	25 °C, 12.0 V, each channel	11 points 0–10 A; 30 s settle; mean of 10 readings at 1 s	±(0.5 % + 10 mA) minus guard band	All points pass
TC-R1-02	R1, R8	50 °C ambient, 10 A on all four channels for 30 min	Readings every 10 s; last 10 min evaluated	Same	All readings pass
TC-R2-01	R2	25 °C; 10.8, 12.0, 13.2 V	10 readings each	±(0.2 % + 10 mV)	All pass
TC-R4-01	R4	25 °C	600 consecutive polls	1.00 s ± 10 ms	Max deviation within limit
TC-R6-01	R6	13.2 V, RS-485 active, all LEDs on	Input power, 60 s average	≤ 0.5 W	Pass
TC-R7-01	R7	10 A, 25 °C	Four-wire voltage, input to output terminal	≤ 30 mV	Pass
TC-R9-01	R9	IEC 61000-4-2 set-up	±4 kV contact on terminals and enclosure, 10 discharges each polarity	Criterion B: self-recovery, no data loss	Pass

Coverage of conditions, not only of requirements

Rev A passed its first test because the only current test ran at room temperature with one channel loaded; temperature, and four channels at full load heating the board together, were never exercised. A plan must cover the conditions that drive each term of the error budget: for PM-4 the corners of current, ambient, bus voltage and number of loaded channels. All combinations would be $2^4 = 16$ corners per channel; the plan tests those the budget says are worst (hot, four channels, 10 A) plus the room-temperature sweep.

Automating the bench

A plan executed by hand is executed slightly differently each time. PM-4 tests run from a Python script that drives the supply and load over SCPI, reads the DMMs and the DUT, applies limits from one limits file and stamps each results file with unit serial, firmware hash, instrument asset numbers and ambient temperature. With a different limits file the same script becomes the PVT production test, the cheapest way to make production and design verification agree.

4.6 Verification versus validation

Two different questions

Verification asks whether the product meets its specification: each requirement in Table 4.1, tested against its limit. **Validation** asks whether the product meets the needs of its users in its real environment, including

the needs that never became requirements. The distinction is the same one made for firmware in the V-model of *Electronics and Computing*, Ch. 28 §28.10; here it is applied to the whole device, on the bench for verification and in users' racks for validation.

> **Status of the principle: Verification does not imply validation**
> *Fundamental.* A specification is a finite description of an open-ended need; a product can satisfy every written requirement and still fail the need the requirements were meant to capture, so validation evidence must come from the use environment, not from the specification.

The DVT verification table

Table 4.7 is the summary of DVT verification of 22 Rev B units (the revision produced in Section 4.10). Each result is the worst unit.

Table 4.7 — PM-4 Rev B DVT verification summary (22 units).

Req.	Test	Limit	Worst result	Verdict
R1	TC-R1-01/02, 25 °C and 50 °C, four channels	±52 mA at 10 A (guard-banded)	+21 mA at 10 A, 50 °C	Pass
R2	TC-R2-01	±(0.2 % + 10 mV) = ±34 mV at 12 V	−9 mV	Pass
R3	Derived from R1/R2 data	±1.0 %	0.29 %	Pass
R4	TC-R4-01	1.00 s ± 10 ms	1.4 ms deviation	Pass
R5	Modbus conformance, 32-unit segment emulated	No exceptions, CRC errors 0 in 10^5 polls	0 errors	Pass
R6	TC-R6-01	≤ 0.50 W	0.31 W	Pass
R7	TC-R7-01	≤ 30 mV at 10 A	24 mV	Pass
R8	Chamber 0 °C and 50 °C, 4 h each	Functional, R1 met	Met	Pass
R9	IEC 61000-4-2 ±4 kV contact	Criterion B	Reset on 2 of 40 discharges, recovered in 1.2 s, energy count kept	Pass
R10	BOM roll-up	≤ $30.00	$28.45	Pass

What validation found

Before DVT closed, six EVT units were installed in a customer's pilot rack for four weeks next to the rack's billing meter. They met every number in Table 4.1, and validation still found two needs nobody had written down:

1. **Energy totals must survive a bus outage.** The rack's tenant billing used PM-4's energy registers. During a planned maintenance the 12 V bus went down for 20 minutes; PM-4, powered from the same bus, lost a day of accumulated energy held in RAM. No requirement said the counters must be non-

volatile, because the specification had been written for "monitoring", and billing was the users' real use.
2. **Hot-plug spikes caused nuisance alarms.** A server's power supply draws an inrush spike when plugged in; the 1 s average was fine but a firmware overcurrent alarm sampled single conversions and tripped.

Both became requirements, R11 (energy totals retained through loss of supply, ≤ 1 Wh lost per outage) and R12 (overcurrent alarm on a 100 ms moving average), and entered Revision B (Section 4.10). Six units in one rack found them more cheaply than six hundred in a data centre would have.

4.7 Calibration

What calibration can and cannot fix

Calibration compares the instrument with a reference of known, traceable value under stated conditions and records the relationship; *adjustment* then changes the instrument so its readings agree. In products the two are combined: measure each unit against a reference, compute and store correction coefficients, and verify at points not used for the fit. This removes systematic errors that stay stable between calibration and use (shunt tolerance, gain and offset), but not noise, temperature drift, ageing, or any error that depends on a condition the calibration did not reproduce.

> **Status of the principle: Calibration removes only stable systematic error**
> *Derived.* A correction measured under one set of conditions equals the error under another set only to the extent that the error does not depend on the conditions that changed; the residual is therefore bounded by the drift terms of the budget (Table 4.4), not by the calibration fit.

The PM-4 calibration procedure

The production calibration of each channel is:

1. Soak the unit at 25 ± 2 °C for 10 minutes with the bus at 12.0 V and no load.
2. Apply six reference currents, 0.5, 2, 4, 6, 8 and 10 A, from the electronic load; at each point wait 5 s (shorter than the shunt's thermal time constant, so the shunt stays near 25 °C) and average 10 readings of both the DUT raw current and the reference shunt.
3. Fit raw = gain × reference + offset by least squares; reject the unit if any residual exceeds 5 mA (a sign of nonlinearity or a bad contact, not of calibratable error).
4. Write gain and offset to the unit's non-volatile memory with a CRC and the calibration date, station ID and procedure revision.
5. Verify at five different currents (1, 3, 5, 7, 9 A) against guard-banded limits.

The least-squares fit itself is standard (*The Mathematics Universe*, Ch. 20 §20.9; the same pattern in Python appears in *Electronics and Computing*, Ch. 22 §22.5). What matters here is the procedure around it: points not used for the fit are used for the verdict, and the verdict uses guard-banded limits.

Guard banding

The production station's reference chain (cheaper than the DVT bench, with fixture contact resistance) has an expanded uncertainty U_{95} of 8 mA at 10 A, so a unit truly 61 mA out could read inside the 60 mA tolerance. The guard-banded acceptance limit subtracts the test uncertainty from the tolerance:

$$A = T - U_{95} = 60\ \text{mA} - 8\ \text{mA} = 52\ \text{mA} \quad \text{at } 10\ \text{A}$$

This is the simple guarded-acceptance rule of ILAC G8: it costs some yield near the limit and buys confidence that no accepted unit is outside R1. The station's uncertainty is taken as proportional to current.

Worked example — Calibrating and verifying one channel. The listing applies the procedure to channel 1 of a Rev B board (an illustrative data set of the kind the station records).

```
 import numpy as np

# Calibration points: reference current (A, from reference shunt + DMM)
# and the raw PM-4 channel-1 reading (A, factory-default coefficients).
i_ref = np.array([0.500, 2.000, 4.000, 6.000, 8.000, 10.000])
i_raw = np.array([0.5063, 2.0194, 4.0388, 6.0539, 8.0670, 10.0861])

gain, offset = np.polyfit(i_ref, i_raw, 1)          # raw = gain*ref + offset
resid_mA = 1e3 * (i_raw - (gain * i_ref + offset))
print(f"gain = {gain:.5f}  offset = {1e3*offset:+.2f} mA  "
      f"max |residual| = {np.abs(resid_mA).max():.2f} mA")

def corrected(raw):
    return (raw - offset) / gain

# Verification at points NOT used for the fit, against R1: +/-(0.5 % rdg + 10 mA)
i_chk = np.array([1.000, 3.000, 5.000, 7.000, 9.000])
r_chk = np.array([1.0105, 3.0282, 5.0422, 7.0616, 9.0782])
U_test = 0.008                                      # A, test-station U95 at 10 A
for ref, raw in zip(i_chk, r_chk):
    err = corrected(raw) - ref
    tol = 0.005 * ref + 0.010
    acc = tol - U_test * ref / 10                   # guard-banded acceptance limit
    print(f"{ref:4.1f} A  err {1e3*err:+6.2f} mA  limit +/-{1e3*acc:5.1f} mA  "
          f"{'PASS' if abs(err) <= acc else 'FAIL'}")
```

```
gain = 1.00824  offset = +3.35 mA  max |residual| = 2.48 mA
 1.0 A  err  -1.09 mA  limit +/- 14.2 mA  PASS
 3.0 A  err  +0.12 mA  limit +/- 22.6 mA  PASS
 5.0 A  err  -2.34 mA  limit +/- 31.0 mA  PASS
 7.0 A  err  +0.55 mA  limit +/- 39.4 mA  PASS
 9.0 A  err  +0.66 mA  limit +/- 47.8 mA  PASS
```

The uncalibrated channel read 0.82 % high, mostly the shunt's tolerance (the part is ±1 %) plus a 3.35 mA offset; that alone would fail R1 at 10 A (0.82 % > 0.60 %). After correction the worst verification error is 2.34 mA against a limit of 31.0 mA. The 2.48 mA maximum residual is under the 5 mA reject threshold of step 3. Figure 4.2 shows the error before and after correction across the range, with the guard-banded limit.

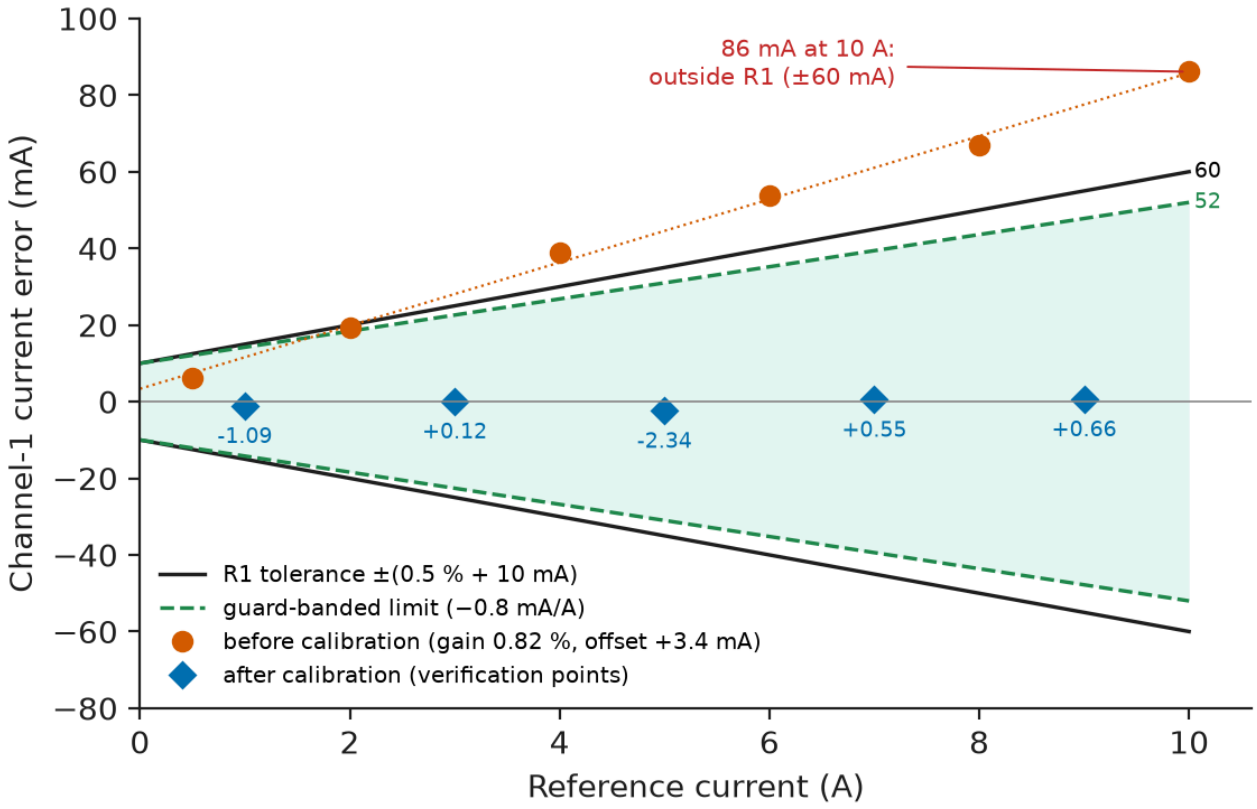


Figure 4.2 — Channel-1 current error of a PM-4 rev B board before and after two-point least-squares calibration, against the R1 tolerance and the guard-banded acceptance limit.

Calibration records and intervals

The calibration data are part of the unit's record (serial, date, station, procedure revision, reference asset numbers and due dates, raw data, coefficients, verdict); a unit whose record cannot be found cannot be shown to meet R1. The bench instruments' own interval (12 months for the DMMs and reference shunt) comes from the manufacturer's stability specification and the laboratory's quality system (ISO/IEC 17025 when accredited), and is shortened if an instrument is found out of tolerance. The calibration of sensors rather than whole instruments, with traceable references for temperature, torque and current, is the subject of §6.11.

4.8 Fault diagnosis

A method, not a mood

Fault diagnosis on a prototype is an experiment on a system whose behaviour has diverged from its model. The efficient method has four steps:

1. **Characterise the symptom with numbers.** "Reads high" cannot be diagnosed; "+3.0 % at 10 A on all four channels of all three boards, rising 0.73 % over 45 °C" can.
2. **List hypotheses that explain all the numbers**, ranked by likelihood and cost of testing.
3. **Split the system** at points where a measurement can tell "good upstream" from "good downstream", choosing points that halve the remaining suspects (half-splitting; Figure 4.3).

4. **Confirm by intervention**: change the suspected cause alone and see the symptom go and come back. Correlation without intervention is a hypothesis, not a root cause.

For N equally likely suspect blocks in a chain, half-splitting finds the faulty one in $\lceil \log_2 N \rceil$ measurements instead of up to $N - 1$ by walking along the chain; for the PM-4 signal chain of eight blocks (terminal, copper, shunt, sense trace, RC filter, INA228, I²C, firmware scaling) that is three measurements instead of seven.

> **Status of the principle: Half-splitting**
> *Derived.* Each measurement that divides the remaining suspects into two equally likely halves gains one bit of information, so $\lceil \log_2 N \rceil$ measurements suffice; the result assumes a single fault and observable intermediate points, which is why EVT boards are designed with test pads and zero-ohm links.

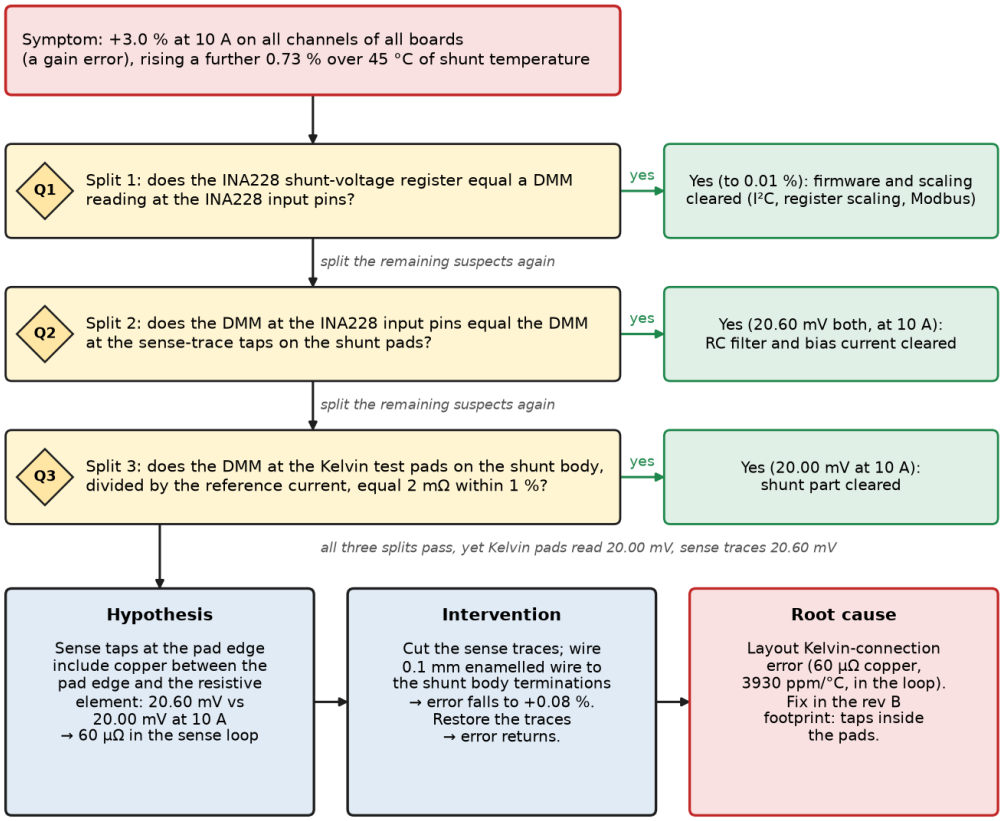


Figure 4.3 — Half-split diagnosis of the PM-4 rev A current error.

The Rev A story

Applying the method to the opening fault:

- **Symptom.** With the *concept* build's coefficients, all twelve channels of the three EVT boards read between +2.7 % and +3.3 % high (mean +3.0 %) at any current, and the error rose by 0.73 % between 25 °C and 70 °C shunt temperature. Proportional to current, it is a gain error, not an offset; present on every channel with a spread no larger than the shunt tolerance, it is a design error, not a manufacturing one.

- **Hypotheses.** (a) Wrong SHUNT_CAL value in firmware (gain scaling); (b) shunt parts out of tolerance or a wrong part fitted; (c) a resistance in series with the shunt inside the sense loop; (d) INA228 input bias current through the RC filter resistors.
- **Split 1.** The INA228 shunt-voltage register (raw, before the firmware's current scaling) agreed with a DMM at the INA228 input pins to 0.01 %: firmware, I²C and scaling cleared, hypothesis (a) gone.
- **Split 2.** The DMM at the INA228 pins agreed with a DMM at the far end of the sense traces where they leave the shunt pads (20.60 mV against 20.60 mV at 10 A), so the RC filter and its bias-current drop were cleared and hypothesis (d) was gone (a bias current would produce an offset, not a gain error). But the separate Kelvin test pads on the shunt body read only 20.00 mV: the 3 % lived between the pad edge and the resistive element.
- **Split 3.** The voltage across the shunt body divided by 2 mΩ matched the reference current within the shunt's 1 % tolerance: hypothesis (b) gone.
- **Intervention.** On one board the sense traces were cut and replaced by fine wires soldered at the shunt's terminations; the error dropped to +0.08 %. Restoring the traces brought it back.

The root cause is a resistance in the sense loop that is not the shunt: the footprint's sense traces left each pad at its outer edge, so the measured path included copper from the pad edge to the resistive element. Its size follows from the 3.0 % error: $0.030 \times 2\ \text{m}\Omega = 60\ \mu\Omega$.

Worked example — Copper in the sense loop. One-ounce copper (35 µm) has a sheet resistance of $\rho/t = 1.72 \times 10^{-8} / 35 \times 10^{-6} = 0.49\ \text{m}\Omega$ per square, so 60 µΩ is only 0.12 squares of copper: a strip 1 mm wide and 0.12 mm long, or the solder-and-copper path under the edge of a 2512 pad. The copper's temperature coefficient (3,930 ppm/°C) then enters the measurement. The effective temperature coefficient of the sensed resistance, referred to the calibrated value, is

$$\alpha_{\text{eff}} = \frac{R_{\text{Cu}}\,\alpha_{\text{Cu}} + R_{\text{sh}}\,\alpha_{\text{sh}}}{R_{\text{sh}} + R_{\text{Cu}}} = \frac{60\ \mu\Omega \times 3930 + 2000\ \mu\Omega \times 50}{2060\ \mu\Omega}\ \text{ppm}/^{\circ}\text{C} = 163\ \text{ppm}/^{\circ}\text{C}$$

Over the 45 °C span of Table 4.4 this gives $163 \times 10^{-6} \times 45 = 0.73$ %, matching the measured drift and more than three times the 0.225 % the budget allotted to the shunt. Calibration could have removed the 3 % gain error; nothing in calibration can remove a drift of 0.73 %, which together with the other budget terms (0.73 + 0.095 = 0.83 %) exceeds R1's 0.60 % at 10 A. Figure 4.4 compares the drift of Rev A and Rev B.

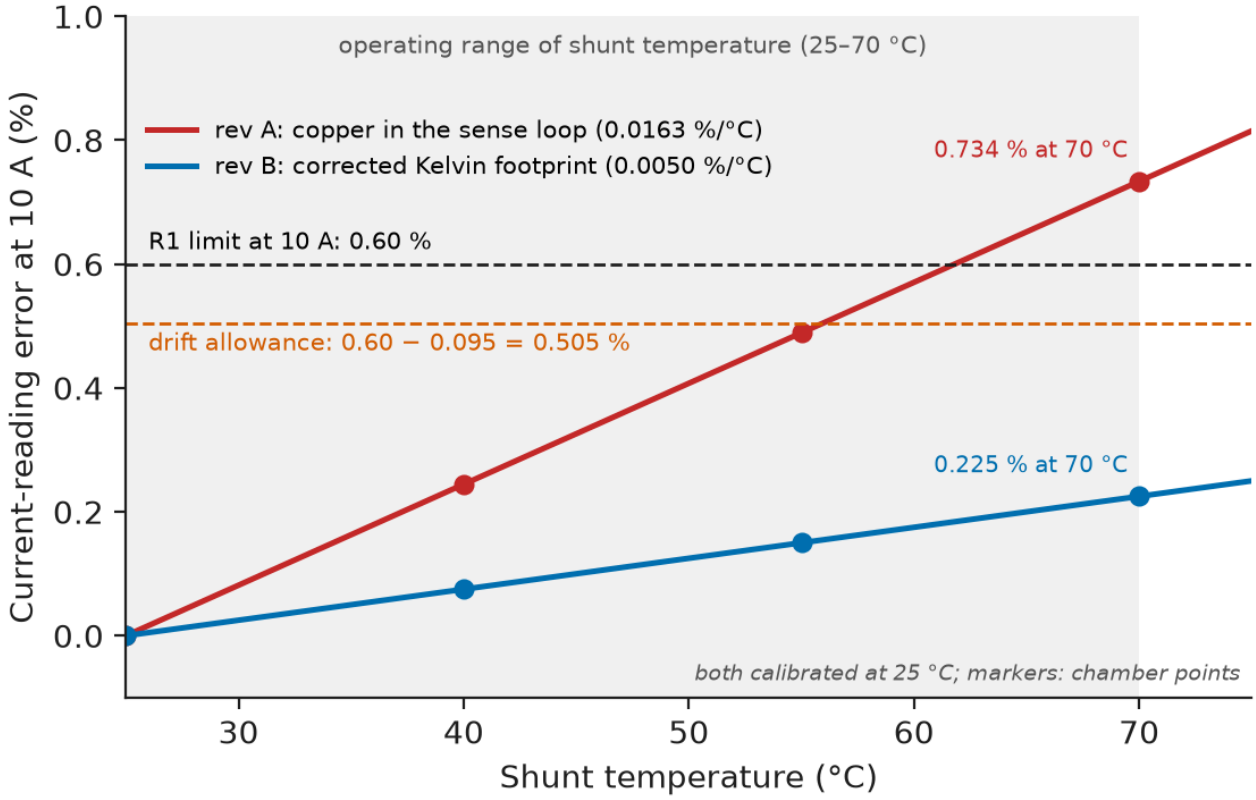


Figure 4.4 — Current-reading error at 10 A versus shunt temperature for rev A (copper in the sense loop) and rev B (corrected Kelvin footprint), both calibrated at 25 C.

> **Status of the principle: Kelvin (four-terminal) sensing**
> *Fundamental.* A voltage measured across two points includes every resistance between them; only when the sense connections touch the resistive element at the points where its value is defined, with no load current in the sense leads, does the measured voltage equal current times the specified resistance.

Recording the failure

A diagnosis ends in a failure record: symptom, data, hypotheses and results, root cause with evidence, corrective action, and the test that proves the fix. Table 4.8 collects PM-4's EVT and pilot failures in FMEA form.

Table 4.8 — FMEA-style record of PM-4 prototype failures.

Failure mode	Effect	Cause	Detection	Mitigation
Gain error +3 % and drift 163 ppm/°C	R1 fails over temperature	Sense traces include 60 µΩ of copper (layout)	EVT chamber test; half-split diagnosis	Rev B footprint with sense taps inside pads; DRC rule; TC-R1-02 at 50 °C on 4 channels
Energy totals lost on bus outage	Billing data lost	Counters in RAM; device powered from monitored bus	Validation pilot	R11; FRAM plus hold-up capacitor (Section 4.10)
Nuisance overcurrent alarms on hot-plug	False alarms to operator	Alarm on single 1 ms conversions	Validation pilot	R12; 100 ms moving average; inrush test case
I²C lock-up after ESD hit	Readings freeze until power cycle	Slave holds SDA low after a glitch	ESD pre-scan, 3 of 40 discharges	Bus-recovery routine (9 clocks), watchdog, I²C timeout
Buck ripple on sense lines	1 mA rms extra noise	Sense traces routed beside the switching node	Scope on EVT, 200 MHz probe	Reroute; ground plane under sense pairs; populate second RC stage
Calibration coefficient corruption	Channel reads with default gain (0.8 % error)	Write interrupted by power loss at station	Production verification step	CRC on coefficients; two copies; reject unit if both invalid

4.9 Regression testing

Why passing once is not enough

Any change of firmware, layout, part or supplier can break something that worked. Regression testing re-runs the affected part of the verification suite after every change and compares the results with the limits and with the previous results. PM-4 runs three tiers:

- **On every firmware commit (minutes):** host-side unit tests of the scaling, Modbus and energy-accumulation code; build of the release binary; static analysis (*Electronics and Computing*, Ch. 28 §28.10 covers these practices for firmware).
- **Nightly (about 40 minutes):** the bench of Figure 4.1 flashes the newest build onto two reference boards and runs TC-R1-01, -R2-01, -R4-01 and -R6-01 against the stored baseline.
- **Each hardware revision or part change:** the full DVT suite on at least 5 units.

Compare with the baseline, not only with the limit

A result that is still inside its limit may still be a regression: a change that moved supply power from 310 mW to 395 mW passes R6 (500 mW) but has used two-thirds of the remaining margin, and the next change may break it. The nightly comparison therefore has two thresholds: the requirement limit (FAIL) and a drift threshold relative to the stored baseline (DRIFT, which holds the build for a human to explain).

```
 import json

LIMITS = {                  # requirement limits on |value|
    "i_err_10A_pct":    0.60,   # R1 at 10 A: 0.5 % + 10 mA
    "v_err_12V_pct":    0.28,   # R2 at 12 V: 0.2 % + 10 mV
    "p_err_10A_pct":    1.00,   # R3
    "jitter_ms":       10.0,    # R4 update-period jitter
    "supply_mW":      500.0,    # R6
}
baseline = json.loads('{"i_err_10A_pct": 0.05, "v_err_12V_pct": 0.04, '
                      '"p_err_10A_pct": 0.09, "jitter_ms": 1.2, "supply_mW": 310}')
current  = json.loads('{"i_err_10A_pct": 0.06, "v_err_12V_pct": 0.04, '
                      '"p_err_10A_pct": 0.10, "jitter_ms": 1.4, "supply_mW": 395}')

def check(value, base, limit):
    if abs(value) > limit:
        return "FAIL"
    if abs(value - base) > 0.10 * limit:      # moved by >10 % of the limit
        return "DRIFT"
    return "ok"

verdicts = {k: check(current[k], baseline[k], LIMITS[k]) for k in LIMITS}
for k, v in verdicts.items():
    print(f"{k:14s} base {baseline[k]:7.2f}  now {current[k]:7.2f}  "
          f"limit {LIMITS[k]:7.2f}  {v}")
print("BUILD", "ACCEPTED" if all(v == "ok" for v in verdicts.values())
      else "HELD FOR REVIEW")
```

```
 i_err_10A_pct  base    0.05  now    0.06  limit    0.60  ok
v_err_12V_pct  base    0.04  now    0.04  limit    0.28  ok
p_err_10A_pct  base    0.09  now    0.10  limit    1.00  ok
jitter_ms      base    1.20  now    1.40  limit   10.00  ok
supply_mW      base  310.00  now  395.00  limit  500.00  DRIFT
BUILD HELD FOR REVIEW
```

The held build had changed the idle loop to poll the RS-485 receiver instead of sleeping until the UART interrupt; the MCU never entered its stop mode, costing 85 mW. The fix restored sleep, and the next nightly run returned to 312 mW. In the real system the baseline and the limits live in version-controlled files next to the test script, so that a change to a limit is itself a reviewed change (Chapter 5). Figure 4.5 shows a month of nightly results.

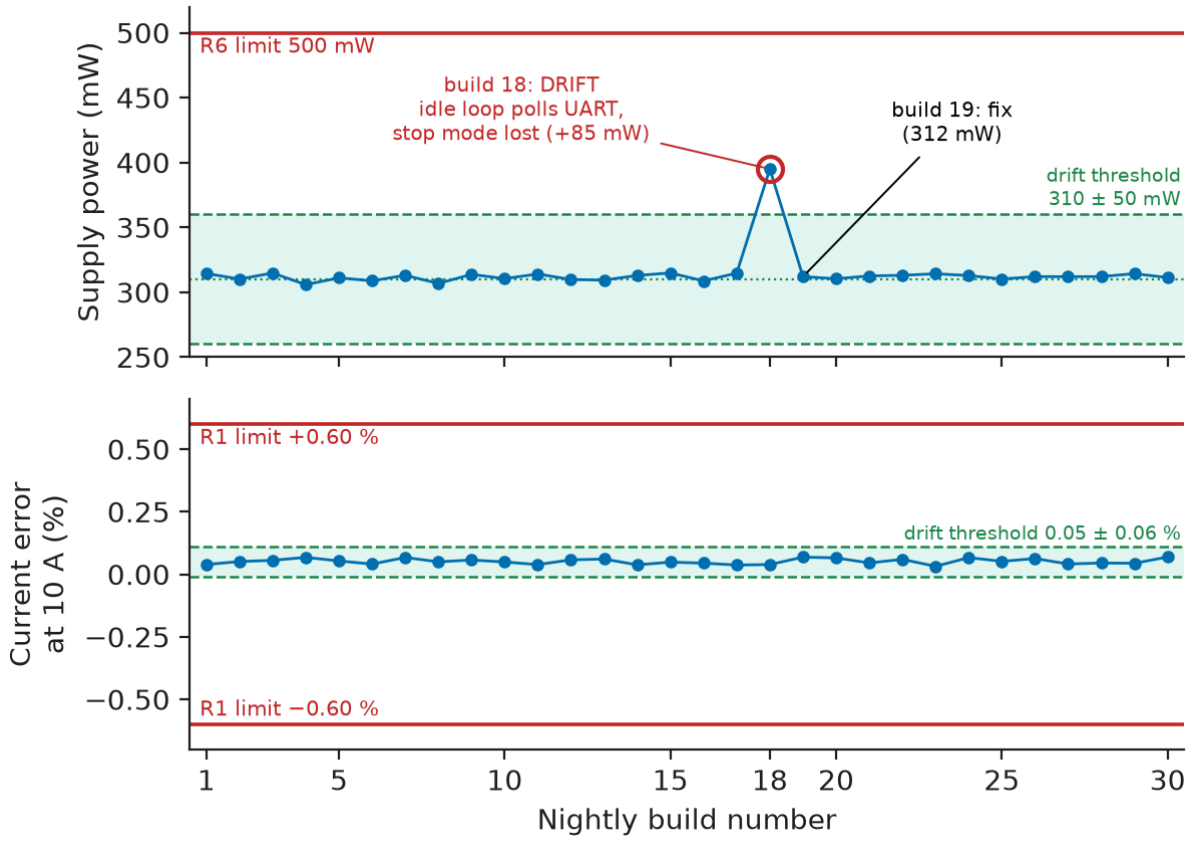


Figure 4.5 — Thirty nightly regression runs of PM-4 firmware on the reference bench: supply power and current error at 10 A, with limits and drift thresholds.

> **Status of the principle: Regression against a baseline**
> *Empirical.* Practice in hardware and software teams shows that most regressions first appear as a loss of margin rather than as a limit violation; tracking the distance from a reviewed baseline catches them while the change that caused them is still known.

Automate whatever is repeated and has a numerical verdict; keep for humans what needs judgement (thermal-camera inspection, ESD discharges, the review of every DRIFT verdict). A suite that reports "ok" for months is checked now and then by injecting a fault, such as a wrong coefficient or a disconnected reference, to prove that it can fail: a test that cannot fail is not a test.

4.10 Design iteration

Revision B

A design iteration is justified by evidence: each change traces to a failure record, a validation finding, or a margin judged too thin, and each is closed by a re-run of the tests that showed the problem. The PM-4 Rev B change list was:

Table 4.9 — PM-4 Rev A → Rev B changes.

Change	Trigger	Closed by
New shunt footprint: sense taps from inside the pads at the resistive element; DRC keep-out of sense traces from current copper	FMEA row 1	TC-R1-02, four channels at 50 °C: drift 0.225 % at 70 °C shunt
FRAM (16 kbit) for energy totals; 47 µF hold-up on 12 V input; brown-out comparator interrupt	R11 (validation)	Power-interruption test: 500 random interruptions, 0 Wh lost
Alarm on 100 ms moving average; inrush test case added	R12 (validation)	Hot-plug of a 1,600 W server PSU 50 times, 0 false alarms
I²C bus-recovery routine, 25 ms transaction timeout, watchdog	ESD pre-scan	R9 at ±4 kV: 0 lock-ups
Sense pairs rerouted on inner layer over ground; buck moved 15 mm away	Noise measurement	0.3 mA rms noise, same as concept build
Coefficients stored twice with CRC	FMEA row 6	Fault-injection test: interrupted writes recovered

Worked example — Sizing the hold-up capacitor for R11. On a bus failure PM-4 must detect the drop and write four 8-byte energy totals plus a sequence number to FRAM before its 3.3 V rail collapses. Detection by the comparator takes under 50 µs; the FRAM write of about 40 bytes at 1 MHz I²C takes about 0.4 ms; allowing for the MCU to finish its current task, 2 ms of hold-up is required. The board draws 0.33 W from its input. Taking the input from 10.8 V (lowest specified bus voltage) down to 4.5 V (the buck converter's minimum input), the stored energy needed is $E = 0.33\ \text{W} \times 2\ \text{ms} = 0.66\ \text{mJ}$, so

$$C \geq \frac{2E}{V_1^2 - V_2^2} = \frac{2 \times 0.66 \times 10^{-3}}{10.8^2 - 4.5^2} = 13.7\ \mu\text{F}$$

A blocking diode keeps the capacitor from feeding the collapsing bus and the servers on it. A 47 µF aluminium-polymer capacitor gives a factor 3.4 margin, covering a −20 % tolerance, end-of-life capacitance loss and a bus that collapses faster than the 2 ms assumed. The power-interruption test (500 interruptions at random instants) is the verification, because the timing assumptions are exactly the kind of thing an analysis gets wrong.

When to stop iterating

Iteration ends when the current exit criterion is met with margin and the remaining risks are accepted in writing, not when the design is perfect. Rev B met every requirement in DVT with at least 20 % margin except R10 (cost, 5 %) and went to PVT, with one accepted open item: the 47 µF capacitor's life at 50 °C, to be re-checked in the first field-return analysis. Recording why each change was made, where it lives and which test closed it is the subject of Chapter 5.

> **Boundary.**
> Simulation of the hold-up transient and of the shunt's thermal behaviour before building (instead of after) belongs to Chapter 3; the electrical-safety rules for bench work on live circuits are in *Electrical Energy and Machines*, Ch. 29 §29.7; on-target firmware debugging (SWD, fault registers, trace) is in *Electronics and Computing*, Ch. 28 §28.8.

What remains open

Open question	Where it is answered
How are schematics, BOMs, test reports and revisions recorded so that Rev B can be rebuilt and audited in five years?	Chapter 5
How are measurement uncertainties combined into an expanded uncertainty and a decision rule?	Chapter 7
How are individual sensors (temperature, torque, current) calibrated against traceable references?	Chapter 6, §6.11
How are calibration rigs and automated test equipment designed as products in their own right?	Chapter 9
How does the energy monitor become a complete metering product with communications and safety approval?	Chapter 35
How is firmware tested continuously and reproducibly in a software pipeline?	Chapter 29
How are hardware-in-the-loop rigs used when the plant is a robot or a motor rather than a resistor load?	Chapter 17

Next: Chapter 5 turns the evidence produced here into documents (schematics, block and wiring diagrams, BOMs, calculations, test reports) under version and revision control, so that PM-4 Rev B can be built, verified and changed by people who were not in the room.

Chapter 5 — Engineering Documentation

Part I · Engineering Design Methodology

Eighteen months after PM-4 Rev B was released (Chapter 4), a data-centre operator returns 40 units from one production lot: in warm racks they read up to 1.4 % high at full load, outside the ±0.6 % of requirement R1. Every unit has a test report that says PASS. It takes the team three weeks to find out why, and almost all of that time goes into reconstructing facts that should have been on paper. The test reports record "PASS" and a date but not the firmware build hash or the calibration procedure revision. The BOM line for the 2 mΩ shunt points to an approved-manufacturer list to which purchasing had added a cheaper "equivalent" thick-film part with a temperature coefficient of 300 ppm/°C instead of 50 ppm/°C; nobody had re-run the temperature test, because nothing in the change process said a new source of a critical part was a design change. Which units carry the alternate is unknown, because the contract manufacturer's lot records were never linked to serial numbers. The design was right and the verification of Chapter 4 was right; the record of *what was actually built* was missing. This chapter treats documentation as part of the product: the set of controlled artefacts from which the product can be built, verified, repaired, changed and audited by people who were not there when it was designed.

> **Which documents does a working system need, and what must each contain so that someone else can build, test and change it?**
> **How are those documents kept consistent with each other and with the hardware and firmware actually shipped, revision after revision?**

Concept	Engineering destination
Schematics	Fabrication, layout, review, repair and failure analysis of every board
Block diagrams	Architecture reviews, interface control, onboarding, safety assessment
Wiring diagrams	Installation, harness manufacture, panel building, field service
Bill of materials	Purchasing, contract manufacturing, cost roll-up, obsolescence and alternates control
Design calculations	Evidence that ratings, scaling and margins were checked; re-use when a part changes
Test reports	Proof of conformity; the basis of release, certification and field-return analysis
Version control	Exact history of firmware, hardware sources, calculations and documents
Technical documentation	Datasheets, register maps, manuals: the interface to users and integrators
Revision management	Part numbers, revision letters, change orders and effectivity by serial number
Reproducibility	Rebuilding any released firmware, board and result bit-for-bit or number-for-number

Roadmap. What the drawings must say (5.1–5.3) → what the product is made of and what it costs (5.4) → why the numbers are right (5.5) → proof that it works (5.6) → keeping every artefact's history (5.7) → documents for users (5.8) → controlled change (5.9) → rebuilding anything that was released (5.10).

5.1 Schematics

What documentation must achieve

Documentation is designed like the product, from requirements. Table 5.1 states what PM-4's documentation set must achieve, each item measurable and verified by an audit or a drill rather than by opinion.

Table 5.1 — Documentation requirements for PM-4.

ID	Requirement	Target	Verification
D1	Every released artefact carries a part number, revision and approval record	100 % of released files	Automated check of the release package
D2	Schematic, BOM and layout of a released revision agree	0 mismatches in reference designators, values, footprints	Scripted comparison in CI (Section 5.4)
D3	Any shipped serial number traces to its PCB revision, BOM revision, approved-source lots, firmware hash and calibration record	Answer within 1 hour	Traceability drill on 5 random serials per quarter
D4	Any released firmware can be rebuilt bit-identically	SHA-256 of rebuild equals released hash	CI rebuild of the last three releases weekly
D5	Every design calculation can be re-run from the repository and reproduces its released numbers	0 differences	CI runs all calculation scripts
D6	Test reports contain the elements of ISO/IEC 17025 §7.8 relevant to a product test, including decision rule	Checklist 100 %	Report review at release
D7	A change to a critical part, its sources or a limit requires an engineering change order (ECO)	0 uncontrolled changes	Quarterly audit of AML and limits files against ECO log

D3, D7 and the missing firmware hash in the test reports are exactly the three gaps that cost three weeks in the opening story.

What a schematic is for

A schematic is the authoritative statement of the circuit's connectivity and component values. It is read by the layout engineer, the reviewer, the test engineer writing the bed-of-nails program, the technician repairing a board and, years later, the engineer analysing a field failure. Each reader needs more than the connectivity a netlist captures:

- **Values with the parameters that matter**: "2 mΩ 1 % 50 ppm/°C 3 W 2512", not "2m"; "47 µF 25 V polymer", not "47u". A tolerance or temperature coefficient that is not on the schematic is not under control.
- **Design intent where layout or assembly can break it**: a net-tie symbol and note at each shunt ("Kelvin: sense taps at resistive element, see D-004 §3"), "place within 3 mm of U1" for decoupling, "matched-length pair" for the RS-485 lines.

- **Population variants**: parts not fitted (DNP) marked as such, so the BOM generator and the assembler agree.
- **Net names that carry meaning**: `CH1_SENSE_P`, `I2C_SDA`, `V12_BUS`, so test points and firmware can refer to the same name.
- **A title block** with part number, revision, sheet number, date and approval, on every sheet.

Large designs are drawn hierarchically: PM-4 has a top sheet (connectors, power, MCU), one channel sheet instantiated four times, and an interface sheet (RS-485, USB, ESD). Figure 5.1 is the channel sheet, after the Rev B changes.

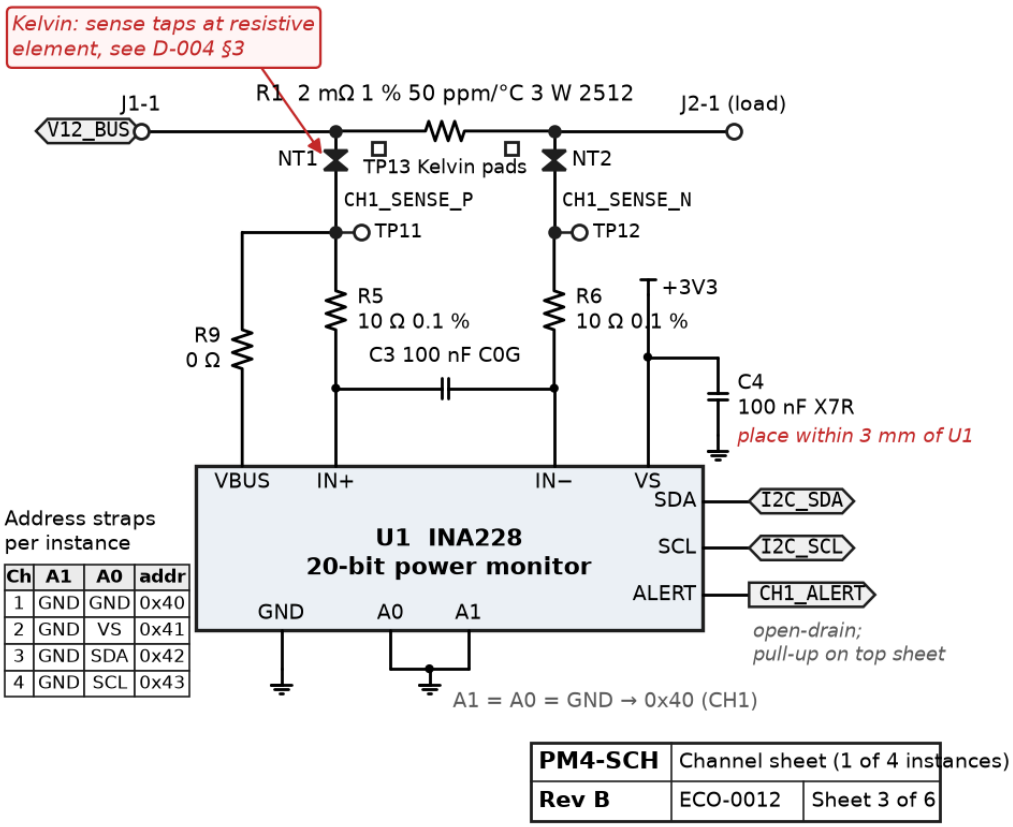


Figure 5.1 — PM-4 rev B channel sheet (one of four instances), with design-intent notes.

Schematic review and electrical rule checks

Before release a schematic passes an electrical rule check (ERC: unconnected pins, outputs driving outputs, power nets without a source) and a human review against a checklist: every IC's absolute maximum ratings against the worst-case voltages on its pins; every pull-up and address strap against the datasheet; every connector pin-out against the wiring diagram; every part against the derating rules (for PM-4, resistors at ≤ 50 % of rated power at 70 °C, capacitors at ≤ 80 % of rated voltage). The review record lists the findings and their disposition; it is a released document in its own right.

5.2 Block diagrams

Architecture on one page

A block diagram shows the system's functions and the interfaces between them at a level where one page holds the whole design (Figure 5.2). Unlike a schematic it is not derived from anything; it is drawn deliberately, and its value is in what it leaves out. Chapter 2 owns the architecture method; here the concern is the block diagram as a controlled document. Three rules make it useful:

1. **Every arrow is an interface with a name and a specification**: "I²C 400 kHz, addresses 0x40–0x43", "Modbus RTU 19.2 kbaud 8E1, register map PM4-ICD rev C", "12 V bus 10.8–13.2 V, 40 A max". An arrow without a specification is a question disguised as an answer.
2. **Each block maps to a schematic sheet, a firmware module or a purchased item**, so the diagram can be checked against the detailed design.
3. **Energy, data and control flows are distinguished** (line style or colour), because they are reviewed by different people.

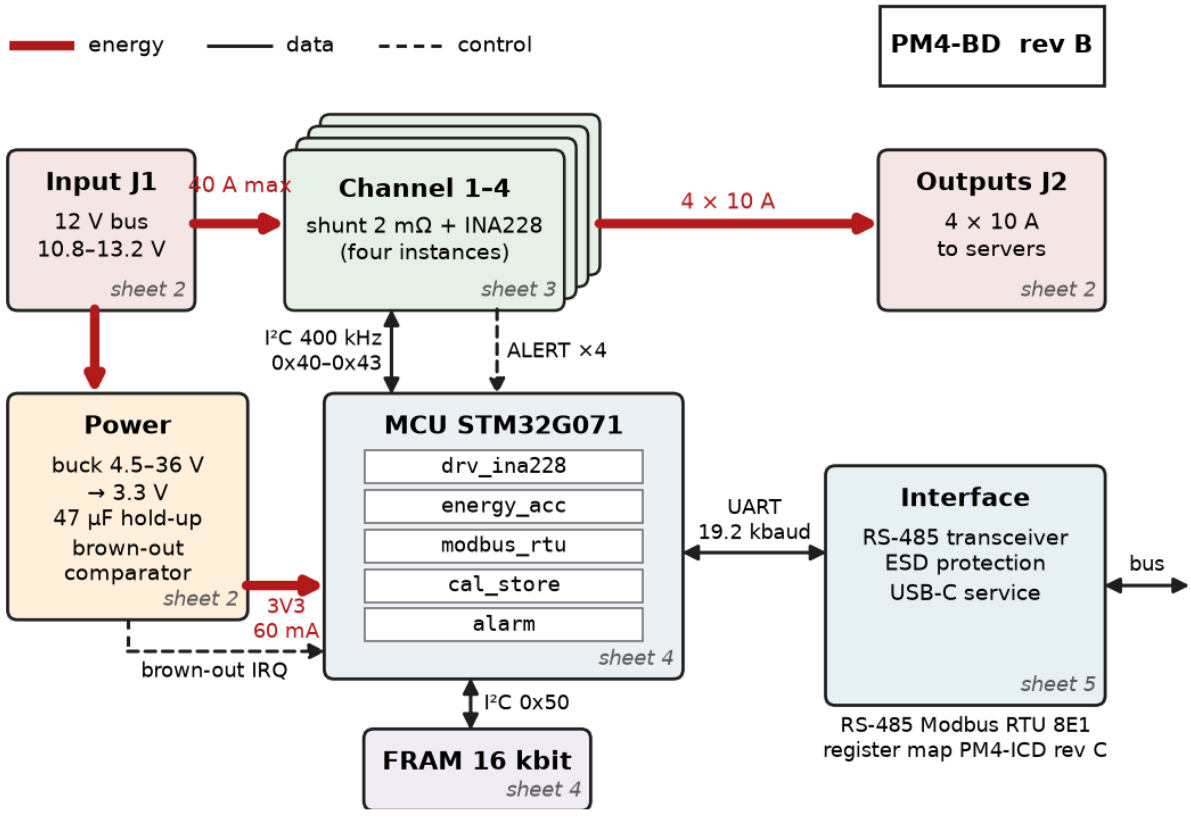


Figure 5.2 — PM-4 block diagram as a controlled document, with named interfaces and the schematic sheet or firmware module behind each block.

The interface control document

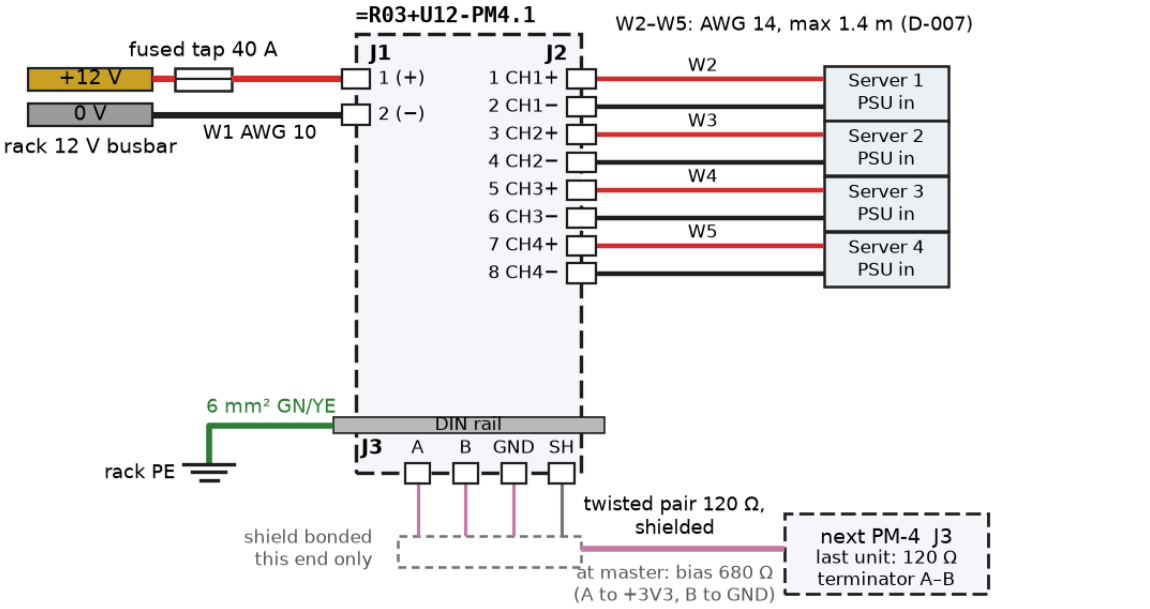
The arrow labelled "Modbus RTU" in Figure 5.2 points to a separate document: the **interface control document** (ICD), here the register map PM4-ICD. It is the one document shared by two teams (PM-4's firmware and the customer's building-management or rack software), so it is versioned independently and changed only by agreement. An excerpt is in Section 5.8.

5.3 Wiring diagrams

From schematic to installation

A wiring diagram shows physical conductors between physical terminals: which wire, of what size, colour and length, from which terminal to which, through which connector. A schematic says CH1 output connects to the load; the wiring diagram says "J2-1 to server 1 PSU input +, AWG 14 red, 1.4 m, ferrule 2.5 mm², tightened to 0.5 N·m". Installers, harness manufacturers and field technicians work from it; a wrong wiring diagram is wired faithfully.

For PM-4 the installation drawing (Figure 5.3) covers the rack-level wiring: the 12 V bus feed to J1, four output pairs to the servers, the RS-485 daisy chain with termination at the last unit, and the protective-earth bond of the DIN rail. Conventions follow IEC 81346 reference designations so that "=R03+U12-PM4.1:J2-1" identifies the same terminal on the drawing, the wire marker and the maintenance record, and IEC 60617 symbols (*Electrical Energy and Machines*, Ch. 30 §30.9 gives the same conventions for installations).



Wire table

Wire	From	To	Size	Col.	L	Ferrule
W1	BUS F1	J1-1/2	AWG 10	RD/BK	site	6 mm²
W2	J2-1/2	SRV1	AWG 14	RD/BK	1.4 m	2.5 mm²
W3	J2-3/4	SRV2	AWG 14	RD/BK	1.4 m	2.5 mm²
W4	J2-5/6	SRV3	AWG 14	RD/BK	1.4 m	2.5 mm²
W5	J2-7/8	SRV4	AWG 14	RD/BK	1.4 m	2.5 mm²

IEC 81346 terminal ID e.g. server 1 feed (+): `=R03+U12-PM4.1:J2-1`

Figure 5.3 — Rack-level wiring of a PM-4 unit: 12 V feed, four server outputs, RS-485 chain and earth bond.

Wire sizing as a documented calculation

The wire sizes on a wiring diagram are not drawing details; each is the result of a calculation that belongs in the design record, because a later installer who lengthens a run changes it.

Worked example — Sizing the server feed. Each output carries up to 10 A at 12 V. The rack designer allows 3 % (360 mV) total drop from the busbar to the server terminals; PM-4's insertion drop (R7) takes 30

mV and four connector contacts of about 1 mΩ each take $4 \times 1\ \text{m}\Omega \times 10\ \text{A} = 40$ mV, leaving 290 mV for the copper of the loop (out and back). At a conductor temperature of 60 °C copper's resistivity rises to $0.0172 \times (1 + 0.00393 \times 40) = 0.0199\ \Omega\,\text{mm}^2/\text{m}$. The longest one-way length allowed is

$$L_{\max} = \frac{\Delta V}{2\, I\, \rho / A}$$

which gives 0.60 m for AWG 18 (0.823 mm²), 0.95 m for AWG 16 (1.31 mm²), 1.52 m for AWG 14 (2.08 mm²) and 2.41 m for AWG 12 (3.31 mm²). The pluggable terminals accept at most 2.5 mm², which rules out AWG 12, so the drawing specifies AWG 14 with a maximum length of 1.4 m: total drop $2 \times 10\ \text{A} \times 1.4\ \text{m} \times 9.57\ \text{m}\Omega/\text{m} + 70\ \text{mV} = 338$ mV, a 6 % margin on the 360 mV budget (Figure 5.4). The note "max 1.4 m" on the wiring diagram is the visible end of this calculation, and the calculation's ID (D-007) is printed beside it.

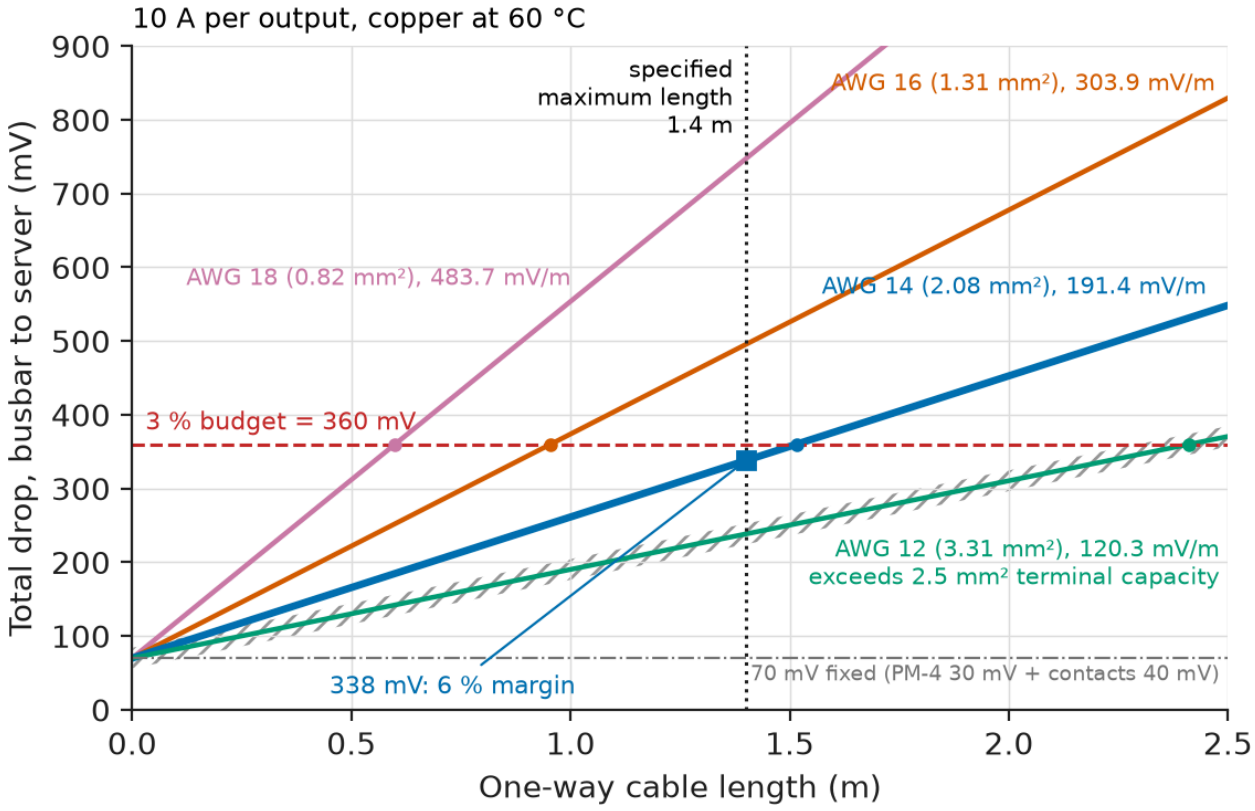


Figure 5.4 — Total voltage drop from busbar to server at 10 A versus one-way cable length for four wire sizes, at 60 C conductor temperature.

Status of the principle: Voltage drop in a feed
Derived. Ohm's law applied to the loop resistance, with resistivity corrected linearly for conductor temperature; it is approximate where contact resistances age or where the cable's temperature is unknown, which is why the budget keeps a margin.

5.4 Bill of materials

The BOM is the product, in purchasable form

The bill of materials lists every item needed to build one unit: reference designators, quantity, an internal part number (IPN), description with the critical parameters, and the approved sources. Two structures are kept apart:

- The **BOM** names IPNs, not manufacturer part numbers. IPN 210-0020 is "Shunt 2 mΩ 1 % 50 ppm/°C 2512 3 W, metal element".
- The **approved manufacturer list (AML)** maps each IPN to the manufacturer parts allowed to fill it, each with a qualification record showing that the part meets the IPN's specification *in this design*.

This split lets purchasing buy from any approved source without an engineering change, and lets engineering control what "approved" means. The opening failure was a hole in that control: an alternate was added to the AML without a qualification record, and nothing checked.

A BOM that checks itself

PM-4's BOM and AML are CSV files in the hardware repository, exported from the schematic tool and edited only through reviewed changes. A script run in continuous integration (CI) rolls up cost at two order quantities and blocks the release if any critical IPN has an unqualified source.

```
 import csv, io

BOM = """ipn,refdes,qty,description,critical,usd_100,usd_1k
310-0228,U1-U4,4,Power monitor 20-bit I2C (INA228),yes,3.10,2.35
210-0020,R1-R4,4,Shunt 2 mOhm 1% 50ppm/K 2512 3 W,yes,0.62,0.38
320-0071,U5,1,MCU Cortex-M0+ 128 kB (STM32G071),yes,1.95,1.42
330-1450,U6,1,RS-485 transceiver 3.3 V,no,1.05,0.78
340-0933,U7,1,Buck 4.5-36 V to 3.3 V 0.5 A,no,0.82,0.58
350-0016,U8,1,FRAM 16 kbit I2C,yes,1.60,1.18
410-0816,J1-J2,2,Pluggable terminal block 2x8 5.08 mm,no,1.90,1.45
410-0105,J3,1,USB-C receptacle,no,0.70,0.48
900-0001,-,1,Passives and TVS (62 places),no,1.80,1.10
600-0004,-,1,PCB 4-layer 100x80 mm ENIG rev B,yes,6.50,1.85
610-0002,-,1,DIN-rail enclosure,no,3.40,2.30
700-0001,-,1,SMT assembly and AOI,no,9.00,3.20
700-0002,-,1,Functional test and calibration,no,4.00,1.40
"""
# Approved manufacturer list: ipn -> [(manufacturer part, qualification record)]
AML = {
    "210-0020": [("WSL2512R0020FEA", "QR-017"), ("CSS2H-2512R-2L00F", "QR-017"),
                 ("GENERIC-2M-2512-TF", "")],        # added by purchasing, not qualified
    "310-0228": [("INA228AIDGSR", "QR-002")],
}
ATTRITION = {"210": 0.02, "900": 0.05}               # extra parts bought, by IPN class

rows = list(csv.DictReader(io.StringIO(BOM)))
for col in ("usd_100", "usd_1k"):
    cost = sum(int(r["qty"]) * float(r[col]) * (1 + ATTRITION.get(r["ipn"][:3], 0.01))
               for r in rows)
    print(f"{col:8s} unit cost {cost:6.2f}")
for r in rows:
    if r["critical"] == "yes":
        for mpn, qr in AML.get(r["ipn"], []):
            if not qr:
                print(f"BLOCK: {r['ipn']} {r['refdes']} source {mpn} has no qualification record")
```

```
 usd_100  unit cost  50.09
usd_1k   unit cost  28.45
BLOCK: 210-0020 R1-R4 source GENERIC-2M-2512-TF has no qualification record
```

Worked example — Cost against requirement R10. At 1,000 pieces the rolled-up cost is $28.45 against the $30.00 target, a margin of $(30.00 - 28.45)/30.00 = 5.2\ \%$. At 100 pieces it is $50.09, 76 % above the 1k price, because set-up-dominated items (PCB, assembly, test) fall steeply with volume: assembly alone drops from $9.00 to $3.20. The thin margin is why the cheaper shunt was tempting: replacing $0.38 parts by $0.12 parts would save $4 \times 0.26 \times 1.02 = \$1.06$ per unit, which looks like 3.7 % of cost and is in fact the difference between meeting R1 and not meeting it. A BOM that carries critical parameters and qualification records makes that trade visible before purchase, not after field return. Figure 5.5 breaks the cost down by category and quantity.

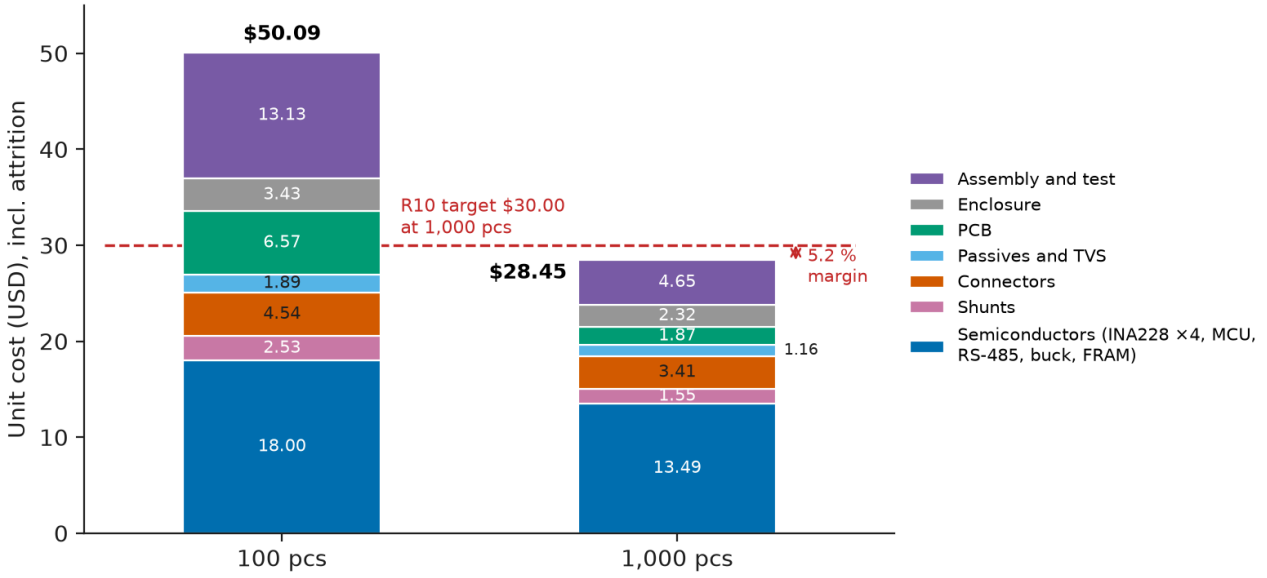


Figure 5.5 — PM-4 rev B unit cost by category at 100 and 1,000 pieces, against the R10 target.

> **Status of the principle: The BOM–AML split**
> *Empirical.* Industry practice (and the configuration-management requirements of ISO 9001 §8.5.2 and IPC-2581/ODB++ data exchange) separates what a part must be from who may supply it; it works only if adding a source is itself a controlled change with a qualification test.

5.5 Design calculations

A calculation is a document

Every number on a schematic that was not copied from a datasheet came from a calculation: the shunt value, the INA228 scaling registers, the hold-up capacitor (Section 4.10), the wire size (Section 5.3), the derating of every part. A design calculation document states its inputs (with sources), its method (with references), its results, and the checks applied; it carries an ID, a revision and an approval like a drawing. The best form for most electronics calculations is a short script in the repository whose output is the document's results section, because it can be re-run when an input changes and checked by CI (requirement D5).

Worked example — INA228 scaling, calculation D-004. The INA228 reports current as a 20-bit two's-complement number scaled by a programmable LSB. Per the device's register description, the current LSB is chosen as $I_{\max}/2^{19}$, the calibration register value is $\mathrm{SHUNT_CAL} = 13107.2 \times 10^{6} \times \mathrm{CURRENT_LSB} \times R_{\mathrm{shunt}}$, multiplied by 4 when the ±40.96 mV input range is selected, and the power and energy LSBs are 3.2 and 16×3.2 times the current LSB.

```
 """calc/ina228_scaling.py -- design calculation D-004, PM-4 rev B."""
R_SHUNT = 0.002        # ohm, IPN 210-0020
I_MAX   = 10.0         # A, requirement R1 full scale
ADCRANGE = 1           # 1 -> +/-40.96 mV full scale

v_fs = 40.96e-3 if ADCRANGE else 163.84e-3
current_lsb = I_MAX / 2**19
shunt_cal = 13107.2e6 * current_lsb * R_SHUNT * (4 if ADCRANGE else 1)
v_max = I_MAX * R_SHUNT
p_shunt = I_MAX**2 * R_SHUNT

print(f"CURRENT_LSB = {current_lsb*1e6:.4f} uA")
print(f"SHUNT_CAL   = {shunt_cal:.1f}  (register value {round(shunt_cal)})")
print(f"POWER_LSB   = {3.2*current_lsb*1e6:.2f} uW, ENERGY_LSB = {16*3.2*current_lsb*1e6:.1f} uJ")
print(f"V_shunt max = {v_max*1e3:.1f} mV = {100*v_max/v_fs:.1f} % of ADC range")
print(f"P_shunt     = {p_shunt:.2f} W (rating 3 W, derated to 1.5 W at 70 C)")
assert v_max < 0.9 * v_fs, "shunt voltage exceeds 90 % of ADC range"
assert p_shunt < 0.5 * 1.5, "shunt dissipation above 50 % of derated rating"
assert round(shunt_cal) < 2**15, "SHUNT_CAL does not fit its 15-bit field"
```

```
 CURRENT_LSB = 19.0735 uA
SHUNT_CAL   = 2000.0  (register value 2000)
POWER_LSB   = 61.04 uW, ENERGY_LSB = 976.6 uJ
V_shunt max = 20.0 mV = 48.8 % of ADC range
P_shunt     = 0.20 W (rating 3 W, derated to 1.5 W at 70 C)
```

The register value 2000 goes into the firmware's configuration header, which cites D-004 rev B; the firmware's unit test asserts the same number, so a change of shunt value that is not carried into firmware fails CI. The `assert` lines are the calculation's acceptance criteria: the 20 mV full-scale signal uses 48.8 % of the ADC range (headroom for 2× overload alarms), and 0.20 W is 13 % of the derated rating. The choice of 2 mΩ against R7's 30 mV insertion limit is in the same document, so a later proposal to use 5 mΩ for better resolution meets the 30 mV limit on the same page ($10 \times 5 = 50$ mV, fail).

> **Status of the principle: Executable calculations**
> *Empirical.* Calculations kept as scripts with assertions and run in CI catch stale inputs and transcription errors that hand calculations and spreadsheets miss; the script is still only as right as its equations, which is why each one cites its source and is reviewed once by a second engineer.

5.6 Test reports

What a test report must contain

A test report is the evidence that a specific item met (or did not meet) specific requirements under specific conditions. Its value lies in the details that make the result reproducible and traceable. ISO/IEC 17025 §7.8 lists the content of a laboratory report; for a product verification report the relevant elements are:

- **Identification**: report number, revision, page *n* of *N*, issuing organisation, approval.
- **Item under test**: part number, revision, serial numbers; **firmware build hash**; calibration-coefficient file hash; any deviations from the released configuration.
- **Procedure**: test plan and test-case IDs with their revisions; limits file revision; any deviation from the procedure.
- **Equipment**: every instrument by asset number with its calibration due date; test software version.
- **Conditions**: ambient temperature and humidity, supply voltages, warm-up times.
- **Results**: measured values with units and expanded uncertainty; the **decision rule** (e.g. guarded acceptance, Section 4.7); the verdict per test case.
- **Raw data reference**: the path and hash of the raw data file, kept for the product's life.

The PM-4 reports from the opening story had the verdicts and the serial numbers but not the firmware hash, the coefficient-file hash, the limits revision or the raw data, so a question about a single unit's calibration could not be answered from the report. The Rev B report header after the change is shown below; it is generated by the test script itself, not typed.

```
 TEST REPORT TR-PM4-0217  rev 0            page 1 of 4
Item      PM4-100 rev B   serials 2410-0441 .. 2410-0462 (22 units)
Firmware  pm4-fw v1.3.0   sha256 9f2c41e7...b07a   (release build)
Cal file  per unit, sha256 listed on page 3
Procedure TP-PM4-DVT rev C; limits limits_dvt.toml rev 5 (git 4e81c0d)
Equipment AS-0112 DMM 6.5 digit (cal due 2027-03-14)
          AS-0113 DMM 6.5 digit (cal due 2027-03-14)
          AS-0201 shunt 10 A/100 mV, 0.01 % (cal due 2027-01-30)
          AS-0310 chamber; AS-0405 load 30 A; AS-0406 supply 20 A
Software  pm4-bench v2.4.1 (git a7d93f2)
Ambient   23.4 C, 41 % RH
Decision  guarded acceptance, A = T - U95 (ILAC G8); U95 stated per result
Raw data  data/dvt/2026-10-12/ sha256 3c1e...9a40
```

Table 5.2 — Excerpt of results, TR-PM4-0217 (worst of 22 units).

Test case	Requirement	Limit (guarded)	Result ± U_{95}	Verdict
TC-R1-02	R1, 10 A, 50 °C	±52 mA	+21 ± 3 mA	Pass
TC-R2-01	R2, 12.0 V	±33 mV	−9 ± 1 mV	Pass
TC-R6-01	R6	≤ 0.50 W	0.31 ± 0.01 W	Pass
TC-R7-01	R7	≤ 30 mV	24.0 ± 0.2 mV	Pass
TC-R11-01	R11, 500 interruptions	≤ 1 Wh lost	0 Wh lost	Pass

A failed test is reported with the same care as a passed one, and a report is never edited after issue: a correction is a new revision with the reason stated. A report that has been quietly corrected is worth nothing in a dispute or an audit.

5.7 Version control

Everything that defines the product goes in the repository

Version control records every change to every file with author, time, reason and parent, and lets any past state be recovered exactly. Firmware engineers take it for granted; electronics teams too often keep schematics on a shared drive as `PM4_final_v3_JB.sch`. Modern EDA tools (KiCad, and Altium or OrCAD with their own data managers) store designs as files that can be version-controlled; KiCad's are text, so changes can even be reviewed as diffs. PM-4 uses one Git repository per product with this layout:

```
pm4/
  hw/          KiCad project (schematic, layout), bom.csv, aml.csv, fab notes
  fw/          firmware sources, CMake build, unit tests
  calc/        design calculations D-001..D-012 as Python scripts
  test/        bench scripts, test plans, limits_*.toml, report generator
  docs/        ICD, user manual, installation guide (Markdown -> PDF)
  ci/          container definition, CI workflow, release script
  CHANGELOG.md, ECO/ (one file per change order)
```

Large binary outputs (Gerbers, PDFs, firmware images) are not committed by hand; they are produced by the release pipeline (Section 5.10) and attached to the release, with their hashes recorded. Measurement raw data, which can be large, goes to an archive whose path and hash the test report records.

Commits, branches and tags

Three conventions make the history useful rather than merely complete:

- **Commit messages cite the reason**: `fw: alarm on 100 ms moving average (R12, ECO-0014)`. A history searchable by requirement and ECO answers "why is it like this?" years later.
- **Changes arrive through reviewed merges**: work happens on a branch; merging to `main` requires CI to pass and one reviewer to approve. Limits files and the AML are under the same rule, which closes the gap that let an unqualified source in (requirement D7).
- **Releases are annotated tags**: hardware and firmware are tagged separately because they are released separately.

```
 git tag -a hw-pm4-revB -m "PM4-100 rev B, ECO-0012..0016, released 2026-09-02"
git tag -a fw-v1.3.0 -m "Firmware 1.3.0 for PM4-100 rev B (R11, R12)"
git push origin hw-pm4-revB fw-v1.3.0
git describe --tags --dirty        # embedded in the build: fw-v1.3.0, or fw-v1.3.0-4-g7c1d2e9-
dirty
```

The firmware embeds the `git describe` string and the build refuses to produce a release image from a dirty tree (uncommitted changes), so no shipped binary can come from code that is not in the repository.

Choosing the documentation tools

The tool choices are themselves design decisions against the requirements of Table 5.1. Table 5.3 compares the classes considered.

Table 5.3 — Documentation tool selection for a small hardware team.

Need	Options compared	Strengths	Weaknesses	Choice for PM-4
Schematic and layout	KiCad (open, text files); Altium (commercial, own vault)	KiCad: diffable files, scriptable `kicad-cli`, no licence cost. Altium: mature library and supply-chain tools	KiCad: smaller library ecosystem. Altium: binary files, licence per seat	KiCad 8 (D2, D5 automation)
Source history	Git; PDM/PLM system	Git: branches, review, free hosting. PLM: part numbers, ECO workflow, AML	Git: no item master. PLM: cost, poor diffing of sources	Git for sources; ECO and IPN registers as files in Git (spreadsheet-sized team)
Documents	Word processor; Markdown/AsciiDoc built to PDF	Markdown: diffable, built in CI, same review as code	Markdown: weaker page layout	Markdown + PDF build
Calculations	Spreadsheet; scripts	Scripts: testable, re-runnable (D5)	Spreadsheets hide formulas and invite copy-paste errors	Python scripts with asserts

A team that grows past a few products usually adds a product-lifecycle-management (PLM) system for the item master and the change workflow and keeps Git for sources; the principles are the same.

5.8 Technical documentation

Documents for people outside the team

The documents so far are for the people who build and change PM-4. Technical documentation is for those who use, integrate and service it: a **datasheet** (what it does, ratings, accuracy, dimensions), an **installation guide** (wiring diagram, torque values, termination, earthing), a **user manual** (configuration, alarms, troubleshooting), the **ICD** (register map, protocol behaviour, error codes) and a **service note** for recalibration. Each states the product revision and firmware versions it applies to.

Two qualities matter more than style. First, **every number in a user document must trace to a verified value**: the datasheet's "±0.5 % + 10 mA, 0–50 °C" is requirement R1, verified in TR-PM4-0217; a datasheet that rounds a verified ±0.6 % to "better than 0.5 %" creates a warranty problem. Second, **interface documents are exact and versioned**: integrators write code against them.

Table 5.4 — Excerpt of the register map, PM4-ICD rev C (Modbus input registers).

Address	Name	Type	Unit / scale	Notes
30001–30002	CH1_CURRENT	int32, big-endian word order	1 mA	Signed; positive for current into J1
30003–30004	CH1_VOLTAGE	uint32	1 mV	Bus voltage at channel output
30005–30006	CH1_POWER	int32	10 mW	
30007–30010	CH1_ENERGY	uint64	1 Wh	Non-volatile since firmware 1.3.0 (R11)
30081	STATUS	bitfield	—	bit 0 cal valid, bit 1 FRAM ok, bit 2 brown-out seen, bits 4–7 alarm per channel
30090–30091	FW_HASH	uint32	—	First 32 bits of the firmware SHA-256

The last row came from the field-return investigation: an operator can now read the firmware identity over the network, which links a unit's behaviour to its build without opening the rack (requirement D3).

Docs as code

PM-4's user documents are Markdown in `docs/`, reviewed in the same merge requests as the firmware changes that affect them, and built to PDF by the release pipeline with the product revision and firmware version filled in from the tags. The register-map table is generated from the same header file the firmware uses, so the ICD cannot disagree with the code.

5.9 Revision management

Part numbers, revisions and the interchangeability rule

Every controlled item (assembly, PCB, drawing, document, firmware) has an identifier and a revision. The rule that decides whether a change needs a new revision letter or a new part number is **form, fit and function** (F/F/F) interchangeability: if the changed item can replace the old one in every application without any other change, it keeps its part number and takes a new revision; if not, it gets a new part number. PM4-100 rev A → rev B kept its part number because rev B is a drop-in replacement (same terminals, same Modbus map for existing registers, better accuracy). Had the register map changed incompatibly, the product would have become PM4-200.

> **Status of the principle: Form, fit and function interchangeability**
> *Empirical.* A configuration-management convention (ISO 10007, EIA-649) that keeps part numbers meaningful to stores, service and customers; its value depends on judging "function" from the user's side, including software interfaces, not only from the drawing.

The change workflow

A change passes through a request (ECR: what and why), an impact analysis (which documents, stock, units in the field, tests), an approved change order (ECO: what exactly changes, from which serial number or date, what happens to existing stock), implementation, and verification. The ECO's **effectivity** (the first serial, lot or date to which it applies) is what makes traceability possible.

Table 5.5 — PM-4 revision history (extract of the ECO register).

ECO	Change	Items affected	Effectivity	Verification
ECO-0012	Kelvin footprint for shunts	PM4-PCB rev B, PM4-SCH rev B	Serial 2409-0001	TC-R1-02
ECO-0013	FRAM, hold-up capacitor, brown-out interrupt (R11)	PCB rev B, BOM rev B, fw 1.3.0	Serial 2409-0001	TC-R11-01
ECO-0014	Alarm moving average (R12)	fw 1.3.0, ICD rev C	fw 1.3.0, field update allowed	TC-R12-01
ECO-0019	Remove unqualified shunt source from AML; add QR rule to CI	AML rev D, ci/bom_check	Immediate; purge lots 2502-xx	Audit; CI BLOCK test
ECO-0020	FW_HASH register; report header with firmware and cal hashes	fw 1.3.1, ICD rev D, pm4-bench 2.4	fw 1.3.1	Traceability drill (D3)

Worked example — Disposition of existing stock. When ECO-0012 was approved, 350 assembled Rev A boards were in stock. Two dispositions were costed. Rework (cut eight sense traces, add eight fine wires, re-test) takes about 20 minutes at a loaded labour rate of $36 per hour, so $12.00, plus $1.40 for re-test: $350 \times 13.40 = \$4{,}690$. Scrapping writes off the board cost at 1,000-piece prices without enclosure and test, $28.45 - 2.32 - 1.41 = \$24.71$ per board (with the 1 % attrition): $350 \times 24.71 = \$8{,}650$. Rework is cheaper by about $3,960, but the wired boards differ from the released design, would need their own qualification (wire fatigue under vibration, consistency of hand-made Kelvin taps), and would leave a population in the field that behaves differently from Rev B. The ECO therefore scrapped them for products and reworked 30 as engineering and test-fixture units, labelled "REV A-ENG, not for sale", recorded by serial number. The decision and its arithmetic are part of the ECO, so the next engineer does not re-argue it.

Release states

Each document carries a state: **draft** (editable, not for use), **in review**, **released** (frozen; changes only through an ECO), **superseded** and **obsolete**. Manufacturing and test use released documents only. In the Git workflow, "released" means a tagged commit and a release package built by the pipeline; a PDF on a shared drive is never authoritative.

5.10 Reproducibility

What must be reproducible

Reproducibility means that anyone with the repository and the archived inputs can regenerate any released artefact and get the same result: the firmware binary bit for bit, the fabrication outputs, every calculation's numbers, and every test report from its raw data. It requires that *every* input is recorded and pinned: source code, but also compiler version, library versions, tool versions, the limits files, the calibration coefficients and the instrument identities. The mathematics of reproducible numerical work is in *The Mathematics Universe*, Ch. 24 §24.9; here the inputs are a product's.

> **Status of the principle: A build is a function of its inputs**
> *Derived.* A deterministic toolchain maps identical inputs to identical outputs, so a differing output proves a differing input; bit-identical rebuilds therefore require every input, including tool versions and timestamps, to be pinned, and fail as soon as one is left implicit.

The release pipeline

PM-4's CI runs the same pipeline on every merge and, for a tagged commit, produces the release package. It runs inside a container image pinned by digest, so the compiler, KiCad and Python versions are fixed; the build date embedded in the firmware comes from the commit time, not the clock.

```
 # ci/release.yml -- CI workflow (GitHub Actions syntax)
name: release
on: { push: { tags: ["hw-*", "fw-*"] }, pull_request: {} }
jobs:
  build:
    runs-on: ubuntu-24.04
    container: ghcr.io/example/pm4-
tools@sha256:5b1f0c9e2d4a7b3c6e8f1a2b3c4d5e6f7a8b9c0d1e2f3a4b5c6d7e8f9a0b1c2d
    steps:
      - uses: actions/checkout@v4
        with: { fetch-depth: 0 }
      - name: Design calculations reproduce (D5)
        run: for f in calc/*.py; do python "$f" > "out/${f##*/}.txt"; done && git diff --exit-code
out/
      - name: ERC, DRC and BOM/AML checks (D2, D7)
        run: |
          kicad-cli sch erc --exit-code-violations hw/pm4.kicad_sch
          kicad-cli pcb drc --exit-code-violations hw/pm4.kicad_pcb
          python ci/bom_check.py hw/bom.csv hw/aml.csv
      - name: Firmware build, unit tests, reproducibility (D4)
        run: |
          export SOURCE_DATE_EPOCH=$(git log -1 --format=%ct)
          cmake -S fw -B build -DCMAKE_BUILD_TYPE=Release && cmake --build build
          ctest --test-dir build --output-on-failure
          sha256sum build/pm4-fw.bin | tee out/fw.sha256
      - name: Fabrication outputs and documents
        run: |
          kicad-cli sch export pdf -o out/PM4-SCH.pdf hw/pm4.kicad_sch
          kicad-cli pcb export gerbers -o out/gerbers/ hw/pm4.kicad_pcb
          python docs/build.py --rev "$(git describe --tags)" -o out/docs/
```

The calculation step is the CI form of requirement D5: the scripts' outputs are committed, the pipeline re-runs them, and `git diff --exit-code` fails the build if any number changed without the change being committed and reviewed. The weekly job rebuilds the last three firmware tags and compares their hashes with the released ones (D4).

Traceability of the built unit

Reproducing a design is half the job; the other half is knowing which design each shipped unit is. The as-built record joins, per serial number: PCB revision and fabrication lot; BOM revision and the AML source and lot of each critical part (from the contract manufacturer's component-traceability data, now required by the manufacturing agreement); firmware hash; calibration record and coefficient hash; test report number. With it, the opening question ("which units carry the 300 ppm/°C shunt?") becomes a query: the alternate's lots map to serials 2502-0118 to 2502-0377 (260 units), which were recalled and recalibrated with the qualified part fitted, instead of recalling the full 1,800 units shipped that year.

Verification of the documentation set

The documentation requirements of Table 5.1 are verified like any others. Table 5.6 is the result of the first audit after ECO-0019 and ECO-0020.

Table 5.6 — Verification of the PM-4 documentation set.

Req.	Test	Limit	Result	Verdict
D1	Release-package checker over 146 files	100 % with part number, revision, approval	146 of 146	Pass
D2	`bom_check.py` and KiCad DRC/ERC in CI	0 mismatches	0	Pass
D3	Traceability drill, 5 random serials	≤ 1 h each	6–22 min	Pass
D4	Rebuild fw-v1.2.0, v1.3.0, v1.3.1	Hashes equal	3 of 3 equal	Pass
D5	Re-run calc/*.py	0 differences	0	Pass
D6	Checklist review of TR-PM4-0217	100 %	100 %	Pass
D7	AML and limits files against ECO log	0 uncontrolled changes	0 since ECO-0019	Pass

Failure analysis of the documentation system

Table 5.7 lists the documentation failures this chapter has met, in the same FMEA form used for hardware in Chapter 4: a documentation system fails by mechanisms just as definite as a circuit's.

Table 5.7 — FMEA of the PM-4 documentation system.

Failure mode	Effect	Cause	Detection	Mitigation
Unqualified alternate on a critical part	Out-of-spec units shipped (1.4 % error)	AML editable without ECO; no qualification field	Field returns	ECO-0019: QR field mandatory, CI BLOCK, AML under review rule
Test report lacks build and calibration identity	Days lost reconstructing configuration	Report template typed by hand	Field-return investigation	Report header generated by test script with hashes (ECO-0020)
Units not traceable to component lots	Recall of 1,800 instead of 260 units	CM lot data not linked to serials	Field-return investigation	Traceability clause in CM agreement; as-built record per serial
Firmware not rebuildable	Cannot reproduce a field bug or patch an old release	Unpinned toolchain, build date from clock	Weekly rebuild job	Container by digest; SOURCE_DATE_EPOCH; dirty-tree refusal
Calculation out of date after a part change	Wrong register value or rating	Hand calculation not linked to BOM	CI calc re-run; firmware unit test of SHUNT_CAL	Executable calculations with asserts (D5)
Wiring length exceeded in the field	Server PSU undervoltage at load steps	Installer unaware of length limit	Commissioning check of voltage at server	"max 1.4 m, D-007" printed on wiring diagram and label

What remains open

Open question	Where it is answered
How are requirements written so that test cases and documents can trace to them?	Chapter 1
How is an architecture decomposed and its interfaces specified before the block diagram is drawn?	Chapter 2
How are measurement uncertainties stated in a test report and combined into a decision rule?	Chapter 7
How are industrial control panels documented (circuit diagrams, terminal plans, IEC 81346 designations)?	Chapters 18 and 20
How are engineering software projects version-controlled, built and tested reproducibly at scale?	Chapter 29
How do file formats and data interoperability of engineering tools affect long-term documentation?	Chapter 31
How is a full project report assembled from these documents?	Chapter 40
How are installation documents for low-voltage electrical systems prepared?	*Electrical Energy and Machines*, Ch. 30 §30.9

Next: Chapter 6 opens Part II with the sensors and transducers that turn temperature, pressure, position, speed, force, light, magnetic field, current and voltage into signals, selected and calibrated for an instrumented motor test bench.

Part II — Instrumentation and Measurement Systems

Every system in this book is judged by measurements: an efficiency map, a battery's capacity, a robot's position error, a power-quality report, a verification table with a pass or a fail in its last column. Part II is about making those numbers trustworthy. It follows a measurement from the physical quantity to a stored, timestamped, validated record — sensor, conditioning, conversion, acquisition, storage — and treats each link the way Part I taught: as a requirement with a target, a budget with a margin, a part chosen against alternatives, a test with a limit and a failure analysed until its mechanism is known. Four running systems carry the method: a 15 kW motor test bench (sensors), an isolated strain-gauge module for a fatigue rig (conditioning), a three-node acquisition system for a 150 kW drive dynamometer (acquisition), and the instrument suite of the laboratory that houses them (complete instruments).

By the end of this part you should understand

- how to select a sensor against range, accuracy, dynamics, environment and interface, and how to calibrate it traceably;
- how to turn a range and a resolution into gain, headroom and converter bits, and how to write an uncertainty budget before a build and close it with a test;
- how bridges, instrumentation amplifiers, filters and isolation barriers are chosen and checked in a real installation, where common-mode voltage, drive edges and ground loops decide the result;
- how to choose between SAR and delta-sigma, simultaneous and multiplexed conversion, and how sample rates follow from analysis bandwidth;
- how distributed acquisition systems are architected, synchronized with PTP, logged with their metadata and protected against silent sample loss, clock faults and corrupted files;
- how automated validation — completeness, time base, range, sensor health and physical consistency — catches errors before analysis does;
- how compact instruments for temperature, energy, speed, power quality and battery monitoring, calibration rigs and automated test stations are built and verified.

Chapters

Chapter	Central question	What it delivers
6 — Sensors and Transducers	How do you choose a sensor so that the measurement, not the catalogue, meets the requirement?	Sensor families for temperature, pressure, position, speed, force, light, magnetic field, current and voltage, compared on a 15 kW motor bench; a selection procedure; traceable calibration of the bench's sensors
7 — Measurement and Signal Conditioning	How is a chain from a millivolt sensor to a digital number designed so that its range, resolution and uncertainty are known before it is built?	An 8-channel isolated strain-gauge module: requirements, GUM budget, bridge, in-amp, four filters, isolation, converter and DAC selection, sampling, noise and interference diagnosis, verification and FMEA
8 — Data Acquisition Systems	How is a multi-node acquisition system built so that every sample has a known time, origin and validity?	A three-node dynamometer DAQ: architecture, sensor interfaces, measured ENOB, rates and data budget, windowed logging, PTP synchronization and filter-delay compensation, DMA real-time transfer, network and storage design, automated validation
9 — Instrumentation Projects	What does it take to turn a sensor, a converter and a microcontroller into an instrument that can be trusted?	Eight compact laboratory projects — temperature, energy, speed, power quality, battery monitoring, a PT100 calibration rig, an ATE station, the laboratory DAQ — each with requirements, key calculation, part choice, verification and failure modes

How this part connects to the rest of the book

Part II applies the method of Part I: requirements (Chapter 1), architecture (Chapter 2), calibration and test plans (Chapter 4) and documentation (Chapter 5). Chapter 10 takes up the microcontroller inside every instrument here and Chapter 11 their connectivity; robots (Part IV) and plants (Part V, with industrial transmitters in Chapter 19) use the same sensors and acquisition principles. The battery monitor points to Chapter 23 for battery management, the filtering of Chapter 7 continues in Chapter 27, and Chapter 28 applies the same conditioning under patient-safety rules. In Part IX the smart energy monitor of Chapter 35 is the full version of Chapter 9's compact energy meter.

The theory comes from the sibling books, cited and used rather than re-derived. Sensor physics comes from *Engineering Physics* (Ch. 14) and sensor materials from *Materials Science and Engineering*. Instrumentation amplifiers, active filters, references and converter architectures come from *Electronics and Computing* (Ch. 8, Ch. 10), with embedded interfaces in its Ch. 26. Sampling, anti-aliasing, the DFT, digital filters and noise come from *Circuits, Signals and Communications* (Ch. 19, Ch. 22–23, Ch. 29). Power-quality phenomena, energy storage and electrical safety come from *Electrical Energy and Machines* (Ch. 16, Ch. 25, Ch. 29). Uncertainty statistics come from *The Mathematics Universe* (Ch. 20).

Chapter 6 — Sensors and Transducers

Part II · Instrumentation and Measurement Systems

A motor supplier's test bench reports a peak efficiency of 96.1 % for a 15 kW permanent-magnet pump motor at 20 N·m and 3,000 rpm. The customer's laboratory measures 94.9 % on the same motor. Neither bench has a broken sensor. The supplier's bench uses a 500 N·m torque flange left over from a larger motor; its zero drifts by 0.05 % of full scale per 10 K, and as the shaft coupling warms by 10 K during the run the zero moves by 0.25 N·m, which is 1.25 % of the 20 N·m being measured and almost exactly the 1.2 percentage points of disagreement. Its winding temperature comes from a type K thermocouple taped to the end winding, which lags the slot temperature by 15 °C, so the motor is mapped "cold" and its copper losses are underestimated. The sensors were chosen from what was in the cupboard, not from an uncertainty budget. This chapter rebuilds that bench (a 15 kW motor test bench for efficiency mapping and thermal characterisation) as the running reference system, and uses it to cover every major sensor family: temperature, pressure, position, speed, force and strain, light, magnetic field, current and voltage, then selection and calibration.

> **How do you choose a sensor so that the measurement, not the catalogue, meets the requirement, including range, accuracy, dynamics, environment and interface?**
> **What does each sensor family really deliver on a bench, where does it fail, and how is it calibrated traceably?**

Concept	Engineering destination
Temperature sensors	Winding, bearing, coolant and electronics temperatures; thermal derating and protection
Pressure sensors	Coolant, hydraulic and pneumatic circuits; process transmitters; leak tests
Position sensors	Rotor angle for commutation and back-EMF tests; robot joints; valve positions
Speed sensors	Efficiency maps, speed loops, overspeed protection, vibration orders
Force and strain sensors	Torque flanges, load cells, weighing, structural monitoring
Light sensors	Optical encoders and tachometers, IR thermometry, light curtains, ambient-light control
Magnetic sensors	Angle sensors, current transducers, proximity, commutation
Current and voltage sensors	Power analysers, drive control, protection, battery and PV monitoring
Sensor selection	A traceable chain from requirement to part number with the margin shown
Sensor calibration	Traceable references, calibration procedures and intervals for every channel

Roadmap. The bench and its requirements (6.1) → the sensor families it needs (6.2–6.9) → choosing among them (6.10) → calibrating them and proving the bench (6.11).

6.1 The test bench and its measurement requirements

The reference system

The bench tests permanent-magnet synchronous motors up to 15 kW (rated 35.8 N·m at 4,000 rpm, peak 60 N·m, maximum 12,000 rpm, four pole pairs) driven by their own inverter from an 800 V-capable DC supply. The motor under test (DUT) is coupled through a torque flange to a 30 kW regenerative load machine. The bench produces efficiency maps over the torque–speed plane, thermal characterisation (winding, magnet, bearing and coolant temperatures) and back-EMF and angle-offset measurements. Table 6.1 is its measurement specification.

Table 6.1 — Measurement requirements of the 15 kW motor test bench.

ID	Quantity	Range	Required accuracy	Verification
R1	Shaft torque	±100 N·m, motoring and generating	Class 0.05 (±0.05 N·m)	Lever-and-weights calibration (6.11)
R2	Shaft speed	0–12,000 rpm, 1 kHz updates	≤ 1 rpm resolution and error	Comparison with optical tachometer and frequency reference
R3	Rotor angle	0–360°	Resolution ≤ 0.025°, accuracy ±0.05° mechanical (±0.2° electrical)	Manufacturer's certificate; index check
R4	Winding temperature	−20 to 180 °C	±1 °C up to 150 °C; insulation tested at 3 kV	Dry-block comparison with reference PRT
R5	Bearing, housing, coolant temperatures	0–150 °C	±1.5 °C	Dry-block comparison
R6	Coolant pressure, inlet and outlet	0–4 bar (gauge)	±0.5 % FS (±20 mbar)	Pressure-calibrator comparison
R7	Rotor magnet temperature	20–150 °C, non-contact	±5 °C	Comparison with embedded probe on a test rotor
R8	Phase currents; DC-link current and voltage	300 A peak, DC–100 kHz; 100 A DC; 0–800 V	Power uncertainty ≤ 0.04 % of reading (standard)	Power-analyser calibration certificate
R9	Efficiency (derived)	Torque ≥ 20 N·m	Expanded uncertainty ≤ 0.3 percentage points (k = 2)	Uncertainty budget (6.6) and round-robin
R10	Traceability	All channels	Calibration interval ≤ 12 months, ISO/IEC 17025 certificates	Calibration register

Architecture

Figure 6.1 shows the sensors, where they sit, and the interfaces that carry their signals to the data-acquisition (DAQ) system and the power analyser. Conditioning (bridges, amplifiers, filters, isolation and ADCs) is the subject of Chapter 7 and the DAQ itself of Chapter 8; this chapter ends at the sensor's electrical output and its specification.

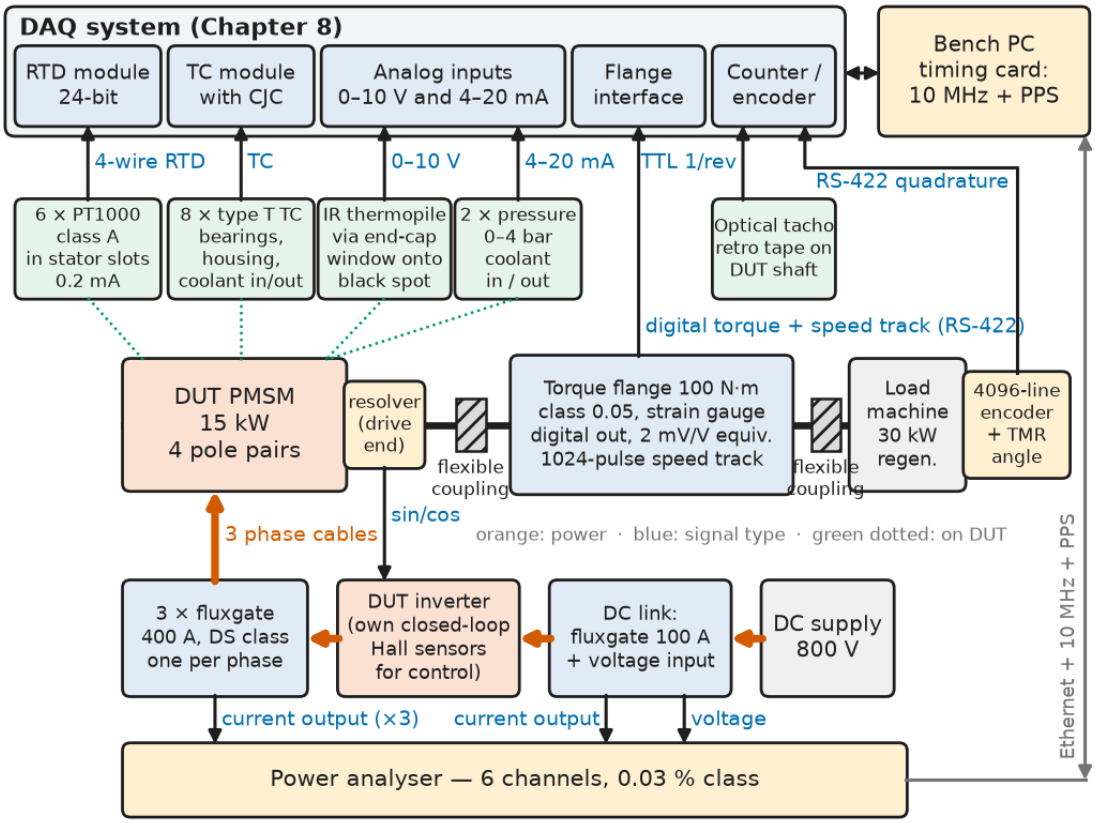


Figure 6.1 — Sensor architecture of the 15 kW motor test bench, with the output interface of each sensor.

The interfaces fall into the classes summarised in *Electronics and Computing*, Ch. 26 §26.9: resistance (RTDs), thermocouple EMF, 4–20 mA loops, bridge outputs in mV/V, pulse and quadrature trains, sin/cos analog, current outputs from closed-loop transducers, and digital buses. The choice of interface is part of the sensor choice, because it decides how much error the cable and the conditioning add.

6.2 Temperature sensors

The families and what each is good for

Five families cover almost every engineering temperature measurement (Table 6.2). Their physics (resistance of metals and semiconductors, the Seebeck effect, junction voltage, thermal radiation) is in *Engineering Physics*, Ch. 14 §14.1 and §14.3; what matters for selection is range, interchangeability (tolerance without calibration), stability, size and interface.

Table 6.2 — Temperature sensor families (typical values).

Family	Range	Interchangeability	Stability	Interface	Typical use
Platinum RTD (Pt100, Pt1000), IEC 60751	−200 to 600 °C (thin film −50 to 300 °C)	Class A $\pm(0.15 + 0.002\lvert t\rvert)$ °C	0.05 °C/year	Resistance, 3- or 4-wire, 0.1–1 mA excitation	Windings, process, reference work
Thermocouple, IEC 60584 (types K, T, J, N)	−200 to 1,300 °C (type K)	Class 1 type T ±0.5 °C or $\pm 0.004\lvert t\rvert$	Drifts at high temperature (type K ageing)	µV signal plus cold-junction compensation	Surfaces, high temperature, small and fast
NTC thermistor	−40 to 150 °C	±0.2 to ±1 °C at 25 °C, worse elsewhere	Good if glass-encapsulated	Resistance, strongly nonlinear	Electronics, batteries, low cost
Silicon IC sensor (analog or I²C/SPI)	−55 to 150 °C	±0.1 to ±1 °C	Good	Digital bus or voltage	Board temperatures, ambient
Infrared (thermopile, Section 6.7)	−50 to 1,000 °C	Depends on emissivity	Good	Voltage or digital	Rotating and live parts

The winding: RTD or thermocouple?

Requirement R4 asks for ±1 °C up to 150 °C inside a winding that sits at up to 800 V DC-link potential with fast PWM edges. The tolerance classes of IEC 60751 for platinum RTDs, at temperature t in °C, are AA: $\pm(0.1 + 0.0017\lvert t\rvert)$, A: $\pm(0.15 + 0.002\lvert t\rvert)$, B: $\pm(0.3 + 0.005\lvert t\rvert)$.

Worked example — Which class meets R4? At 150 °C: class AA gives ±0.36 °C, class A ±0.45 °C and class B ±1.05 °C. Class B fails R4 on its own; class A leaves 0.55 °C for the measurement chain (4-wire connection, excitation, ADC), which a 24-bit RTD module meets easily. A type T thermocouple of IEC 60584 class 1 has $\max(0.5,\ 0.004\lvert t\rvert) = 0.6$ °C at 150 °C, before a cold-junction compensation error of typically 0.3–0.5 °C; type K class 1 is ±1.5 °C, failing R4 outright. Self-heating of a PT1000 at 0.2 mA is $I^2R = (0.2\ \text{mA})^2 \times 1573\ \Omega = 63\ \mu\text{W}$, negligible against a winding's thermal mass, while the signal is still $0.2\ \text{mA} \times 3.74\ \Omega/^\circ\text{C} = 0.75\ \text{mV/°C}$. The choice is PT1000 class A thin-film elements, two per phase, embedded in the slots during winding with insulation rated for the winding's own test voltage, wired 4-wire with shielded twisted pairs.

Figure 6.2 plots the tolerance bands against R4.

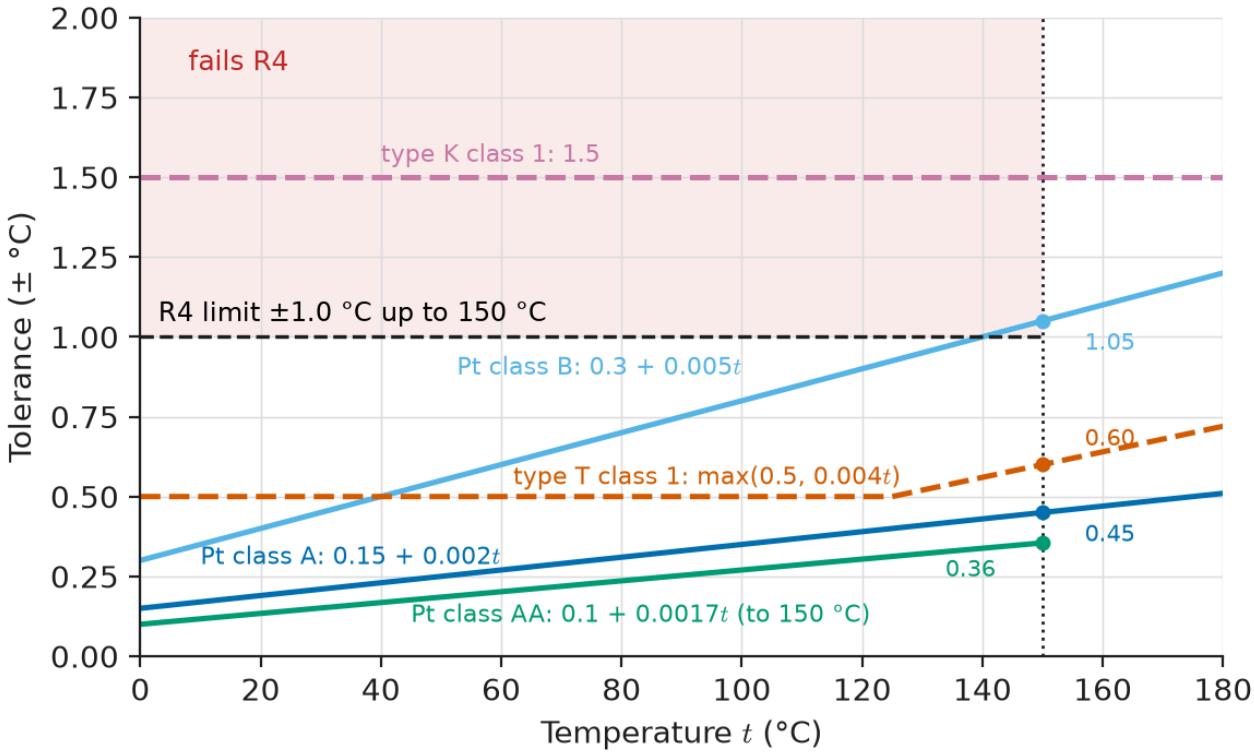


Figure 6.2 — Interchangeability tolerance of platinum RTD classes (IEC 60751) and class 1 thermocouples (IEC 60584) from 0 to 180 C, against the winding-temperature requirement.

> **Status of the principle: Interchangeability tolerance is not accuracy**
> *Empirical.* The IEC 60751 and IEC 60584 classes bound the deviation of an uncalibrated sensor from the standard reference function when new; ageing, installation (immersion, contact, lead resistance, cold-junction error) and the measuring chain add to it, so the class is the starting entry of an error budget, not the result.

Installation decides the reading

A sensor measures its own temperature. A probe taped to an end winding measures the tape, the air and the end winding, and its time constant can be minutes; the old bench's 15 °C lag was an installation error, not a sensor error. Bearing thermocouples are fitted in drilled holes with thermal paste and a spring, because surface contact resistance otherwise dominates; coolant thermocouples are immersed at least ten probe diameters to keep stem conduction below 0.1 °C. The thermal response of each installation follows a first-order lag whose analysis is in *Circuits, Signals and Communications*, Ch. 6: with a time constant of 8 s for a slot RTD, a reading taken 40 s (five time constants) after a step is within 0.7 % of the step, which sets the dwell time per efficiency-map point.

6.3 Pressure sensors

Technologies and references

Pressure is measured as **gauge** (relative to atmosphere), **absolute** (relative to vacuum) or **differential** (between two ports). Coolant pressure drop across the motor jacket (R6) is a differential quantity, but two gauge transmitters are used so that each port can also be checked for over-pressure. Four sensing technologies dominate:

Table 6.3 — Pressure-sensing technologies (typical industrial transmitters).

Technology	Principle	Typical accuracy	Strengths	Weaknesses
Piezoresistive silicon (oil-filled, stainless diaphragm)	Doped-Si bridge on diaphragm, gauge factor about 100	0.1–0.5 % FS	Cheap, small, high output	Temperature drift needs compensation; fill-fluid limits
Thin-film on steel	Sputtered metal bridge on steel diaphragm	0.1–0.5 % FS	No fluid, robust, high overpressure	Lower output
Ceramic capacitive	Ceramic diaphragm capacitance	0.1–0.5 % FS	Corrosion resistant, high overload	Larger, electronics in sensor
Piezoelectric	Charge from quartz or ceramic under stress	Dynamic only	Very high bandwidth (combustion, cavitation)	No static measurement

The piezoresistive and piezoelectric materials and their coefficients are in *Materials Science and Engineering*, Ch. 35 §35.9 and Ch. 27 §27.3.

The bench choice

R6 needs ±20 mbar on 0–4 bar. A 0–4 bar gauge thin-film transmitter with 0.25 % FS accuracy (±10 mbar, including nonlinearity, hysteresis and repeatability at 25 °C) and a total error band of 0.5 % FS over 0–80 °C media temperature meets R6 with the 4–20 mA output preferred for the 5 m cable to the DAQ: 4 mA per bar, and a broken wire reads below 3.6 mA (NAMUR NE 43) instead of "0 bar". The total error band, not the accuracy at 25 °C, is the specification to compare with R6, because coolant runs at 25–65 °C during a thermal test. Pump pulsation (about 50 Hz) is averaged out over 1 s in the DAQ.

> **Boundary.**
> Process-industry pressure transmitters, their HART and 4–20 mA installation, and flow measurement belong to Chapter 19; loop conditioning and the shunt-resistor interface are in Chapter 7.

6.4 Position sensors

Linear and angular, incremental and absolute

Position sensors report either a change (incremental: counts from a reference) or an absolute value (each position has its own code). For rotating machines the relevant families are:

Table 6.4 — Angular position sensors for motor work (typical).

Sensor	Resolution	Accuracy	Speed limit	Environment	Interface
Optical incremental encoder, 4096 lines	16,384 counts/rev (×4) = 0.022°	±20″ (0.0056°)	12,000 rpm (300 kHz line frequency)	Clean, 0–70 °C	RS-422 A/B/Z
Optical absolute encoder, 23 bit	0.00004°	±20″	6,000–12,000 rpm	Clean	BiSS-C, EnDat 2.2
Resolver (transformer type)	12–16 bit after converter	±5 to ±10′ (0.08–0.17°)	> 20,000 rpm	−55 to 155 °C, oil, vibration	sin/cos carrier, needs RDC
Magnetic (Hall or TMR) on-axis angle IC	12–14 bit	±0.3° to ±1° after calibration	> 20,000 rpm	Robust, sensitive to stray fields (6.8)	SPI, ABI, PWM
Inductive (PCB coil) encoder	14–18 bit	±0.05° to ±0.2°	> 20,000 rpm	Robust, immune to DC fields	SPI, sin/cos
Potentiometer (linear or rotary)	Analog	0.1–1 % FS linearity	Low; wear	Contact wear	Ratiometric voltage

What the bench needs

The DUT's own inverter uses a resolver, because resolvers survive the motor's heat and vibration; the drive control behind that choice is in *Electrical Energy and Machines*, Ch. 12 §12.6. The bench needs more: R3 asks for ±0.05° mechanical, which is ±0.2° electrical with four pole pairs, to measure the resolver's offset against the back-EMF zero crossing and to map torque ripple against angle.

Worked example — Does the resolver meet R3? A ±10′ resolver is $\pm 10/60 = \pm 0.167^\circ$ mechanical, so $\pm 0.67^\circ$ electrical with $p = 4$: more than three times R3. A 4096-line optical encoder has a resolution of $360^\circ/16384 = 0.022^\circ$ and an accuracy of ±20″ = ±0.0056°, both inside R3, so it goes on the load-machine end of the shaft line as the bench reference. The angle between DUT rotor and encoder is then fixed only if the couplings are torsionally stiff: a coupling of 10 kN·m/rad twisting under 60 N·m deflects $60/10^4 = 6$ mrad $= 0.34^\circ$, so angle-sensitive tests are run at no load (back-EMF) or corrected with the measured torque and the coupling's stiffness.

> **Status of the principle: Electrical angle is pole pairs times mechanical angle**
> *Fundamental.* In a machine with p pole pairs, one mechanical revolution is p electrical cycles, so every mechanical angle error is multiplied by p in the commutation and torque-angle the drive sees.

> **Boundary.**
> Encoder and resolver interfaces in firmware (timer encoder mode, resolver-to-digital conversion) are implemented in Chapter 15; linear position sensors and limit switches for machines are in Chapter 19.

6.5 Speed sensors

Speed from position, and speed of its own

Speed is measured either by differentiating position (encoder, resolver) or by a sensor of its own: a magnetic or Hall gear-tooth pickup counting teeth, an optical reflective tachometer reading tape on the shaft, a DC or AC tachogenerator (now rare), or the speed track built into many torque flanges. On the bench the encoder supplies speed at 1 kHz; an optical tachometer (1 pulse per revolution) gives an independent check that does not depend on the encoder's electronics.

Counting methods

Three ways of turning edges into speed are in use (Figure 6.3), with N counts per revolution and a timer clock f_c:

- **M-method** (count edges in a fixed window T_w): resolution $\Delta n = 60/(NT_w)$, constant with speed; with $N = 16384$ and $T_w = 1$ ms, 3.66 rpm, which fails R2 at every speed.
- **T-method** (time one edge period with f_c): relative resolution $Nn/(60 f_c)$, excellent at low speed and poor at high speed; with $f_c = 170$ MHz at 12,000 rpm only 51.9 clock ticks fall in one edge period, a resolution of 231 rpm.
- **M/T method** (count the edges in about one window, and time exactly from the first to the last of them): relative resolution $1/(f_c T_w) = 5.9 \times 10^{-6}$, that is 0.07 rpm at 12,000 rpm and better below.

Worked example — Meeting R2. R2 asks for ≤ 1 rpm with an estimate every 1 ms. The M-method gives 3.66 rpm and is rejected; the T-method meets 1 rpm only below about 790 rpm ($n^2 \le 60 f_c/N$, so $n \le \sqrt{60 \times 170 \times 10^6 / 16384} = 789$ rpm). The M/T method meets it from about 10 rpm (below which fewer than three edges fall in a 1 ms window and the window is lengthened) to 12,000 rpm with a 14-fold margin at top speed. The encoder's own ±20″ graduation error adds a once-per-revolution speed ripple that averages out over whole revolutions; for efficiency points the speed is averaged over 1 s anyway.

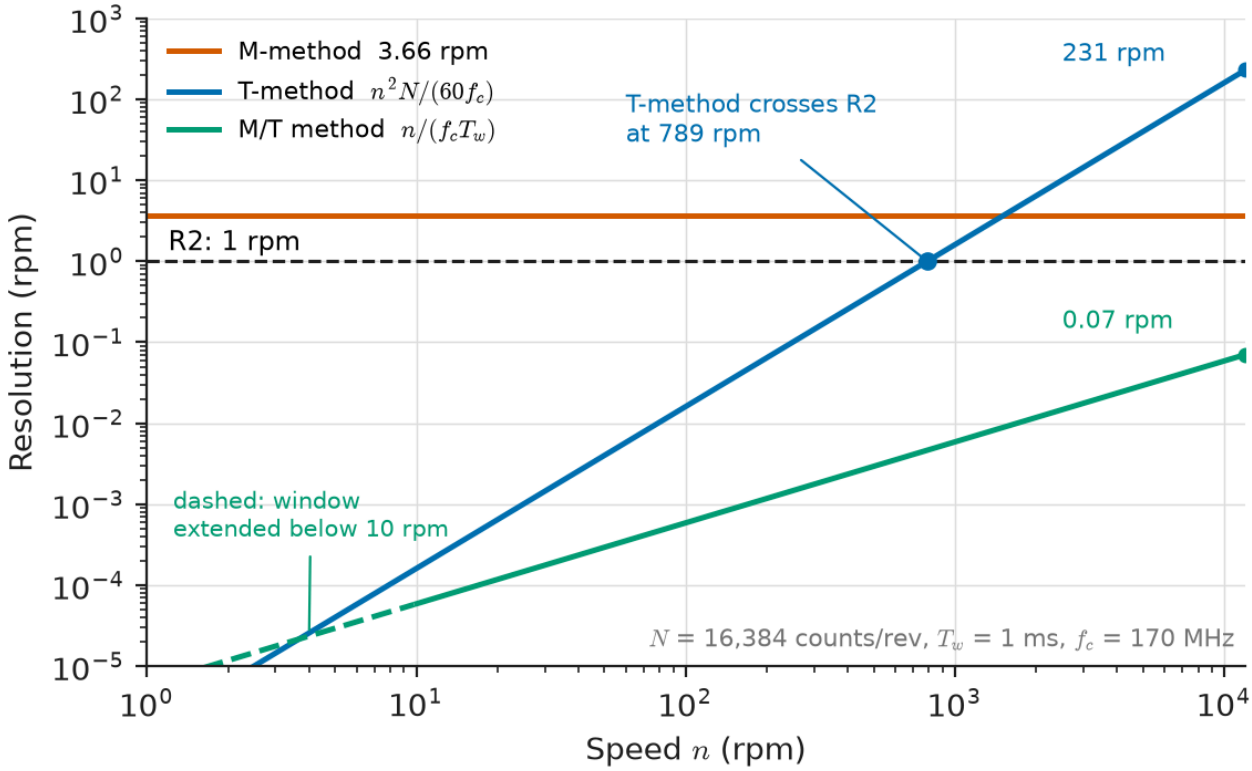


Figure 6.3 — Speed-measurement resolution of the M, T and M/T methods for a 16,384-count encoder, 1 ms window and 170 MHz timer clock, against requirement R2.

The firmware of the bench's speed module implements the M/T method in a few lines. The hardware latches the encoder count and a 170 MHz timestamp on each edge (how that latch is made on a given timer is a Chapter 15 topic); the 1 ms task does the arithmetic.

```
/* mt_speed.c -- M/T speed estimate from a quadrature encoder (C11).
 * Illustrative for an STM32G4-class MCU: one timer in encoder mode counts
 * edges, a 32-bit timer at 170 MHz timestamps the most recent edge.      */
#include <stdint.h>

#define F_TICK_HZ    170000000.0f   /* timestamp clock                    */
#define COUNTS_REV   16384.0f       /* 4096 lines x 4                     */
#define STALL_TICKS  17000000u      /* 100 ms without an edge -> speed 0  */

typedef struct {
    uint16_t cnt_prev;   /* encoder count at the previous edge used */
    uint32_t t_prev;     /* timestamp of that edge                  */
    float    rpm;        /* last estimate                           */
} mt_t;

/* Call every 1 ms with the count and timestamp latched at the latest edge
 * and the present timestamp. Wrap-around of both counters is handled.    */
float mt_update(mt_t *s, uint16_t cnt_edge, uint32_t t_edge, uint32_t t_now)
{
    int32_t  m  = (int16_t)(uint16_t)(cnt_edge - s->cnt_prev); /* signed edges */
    uint32_t dt = t_edge - s->t_prev;                          /* ticks        */

    if (m == 0 || dt == 0u) {            /* no new edge in this window      */
        if ((uint32_t)(t_now - s->t_prev) > STALL_TICKS) {
            s->rpm = 0.0f;               /* declare standstill              */
        }
        return s->rpm;                   /* otherwise hold the last value   */
    }
    s->rpm = (60.0f * F_TICK_HZ / COUNTS_REV) * (float)m / (float)dt;
    s->cnt_prev = cnt_edge;
    s->t_prev   = t_edge;
    return s->rpm;
}
```

Signed edge counts give direction; unsigned subtraction makes both counters wrap safely (the conversion of an out-of-range value to `int16_t` is implementation-defined in C11 but modular on every embedded compiler in use). At 12,000 rpm 3,277 edges fall in 1 ms, well inside the 16-bit range. Holding the last value until a 100 ms stall timeout means the estimate is stale at very low speed; the bench flags points below 10 rpm as "low-speed mode".

> **Status of the principle: Quantisation of counted speed**
> *Derived.* Counting an integer number of edges in a window, or an integer number of clock ticks in a period, bounds the resolution by one count; the M/T method makes both counts large by timing exactly the interval spanned by whole edges.

> **Boundary.**
> Motor-speed measurement as a stand-alone instrumentation project (gear-tooth pickups, period counters, display) is in Chapter 9; the speed loop that uses this estimate is in Chapter 37.

6.6 Force and strain sensors

Strain gauges, load cells and torque transducers

A metal-foil strain gauge changes resistance with strain by its gauge factor (about 2; silicon gauges 100–175, *Materials Science and Engineering*, Ch. 35 §35.9). Four gauges in a full Wheatstone bridge on a machined spring element make a load cell or a torque transducer whose output is specified in mV/V of excitation, typically 1–3 mV/V at rated load; the loaded-bridge circuit analysis is in *Circuits, Signals and Communications*, Ch. 4 §4.2. With 10 V excitation and 2 mV/V, full scale is 20 mV, and 0.01 % of full scale is 2 µV, which is why thermal EMFs, excitation stability and ratiometric conversion (Chapter 7) matter as much as the gauge.

For shaft torque there are three options:

Table 6.5 — Torque-measurement options for the bench (typical).

Option	Accuracy class	Speed limit	Signal transfer	Comments
In-line torque flange, strain gauge, contactless telemetry	0.03–0.1 % FS	10,000–20,000 rpm	Digital or frequency (10 kHz ± 5 kHz), integrated speed track	Measures shaft torque directly; stiff, short
Slip-ring torque shaft	0.1–0.25 % FS	3,000–8,000 rpm	Analog through rings	Ring wear and noise; obsolescent
Reaction torque (cradled load machine + load cell on an arm)	0.05–0.1 % of FS of load cell	Unlimited	Load cell analog	Includes load machine bearing friction; slow dynamics
Magnetoelastic or SAW shaft sensor	0.5–1 % FS	High	Analog/digital	Cheap, automotive; not for efficiency work

The range trap

The old bench's failure was a range error, not a technology error. A transducer's accuracy class is a percentage of *full scale*, so the absolute error is fixed by the rated torque and the relative error grows as the measured torque falls.

Worked example — Choosing the flange's rated torque. The DUT's peak torque is 60 N·m. A 100 N·m class 0.05 flange has an error limit of ±0.05 N·m; a 200 N·m flange (bought "for future motors") has ±0.10 N·m, and the old 500 N·m flange ±0.25 N·m. Combined with a zero error of 0.01 % FS after re-zeroing, converted to standard uncertainty for rectangular limits, the 100 N·m flange contributes $u_T = \sqrt{0.05^2 + 0.01^2}/\sqrt{3} = 0.029$ N·m, which is 0.147 % at 20 N·m. The flange's safe overload (typically 200 % of rated torque) covers a short-circuit transient of the DUT. The 100 N·m flange is the choice; motors above 50 N·m peak will need their own.

The efficiency budget

R9 is a derived requirement: efficiency $\eta = P_{\mathrm{mech}}/P_{\mathrm{elec}}$ with $P_{\mathrm{mech}} = T\omega$. For independent inputs the relative uncertainties add in quadrature (the GUM law of propagation; Chapter 7 owns its use, *The Mathematics Universe*, Ch. 24 §24.8 its mathematics):

$$\frac{U(\eta)}{\eta} = 2\sqrt{\left(\frac{u_T}{T}\right)^2 + \left(\frac{u_n}{n}\right)^2 + \left(\frac{u_P}{P_{\mathrm{elec}}}\right)^2}$$

Table 6.6 — Efficiency uncertainty budget at 3,000 rpm, η = 0.94 (100 N·m flange).

Contribution	Basis	Relative standard uncertainty at 20 N·m	At 35.8 N·m
Torque	Class 0.05 and 0.01 % zero, rectangular	0.147 %	0.082 %
Speed	0.1 rpm, rectangular	0.002 %	0.002 %
Electrical power	Power analyser and fluxgate sensors (6.9)	0.040 %	0.040 %
Combined		**0.152 %**	**0.091 %**
Expanded, k = 2, in percentage points	2 × 0.94× combined	**0.287 pp**	**0.172 pp**
R9 limit		0.30 pp	0.30 pp
Margin		**4 %**	**43 %**

The listing evaluates the budget over torque and finds the lowest torque at which R9 is met for each flange size.

```
 import numpy as np

SQ3 = np.sqrt(3.0)

def U_eta(T, n_rpm, eta, T_fs=100.0, cls=0.05e-2, zero=0.01e-2,
          p_rel=0.04e-2, n_abs=0.1):
    """Expanded (k=2) uncertainty of efficiency, in percentage points."""
    u_T = np.hypot(cls * T_fs, zero * T_fs) / SQ3    # N m, rectangular limits
    u_n = n_abs / SQ3                                  # rpm
    rel = np.sqrt((u_T / T)**2 + (u_n / n_rpm)**2 + p_rel**2)
    return 2 * 100 * eta * rel

for T in (5, 10, 20, 35.8, 60):
    print(f"T = {T:5.1f} N m   U(eta) = {U_eta(T, 3000, 0.94):.2f} pp (100 N m)"
          f"   {U_eta(T, 3000, 0.94, T_fs=200):.2f} pp (200 N m)")

T = np.linspace(1, 60, 5901)
for fs in (100, 200):
    ok = T[U_eta(T, 3000, 0.94, T_fs=fs) <= 0.30]
    print(f"{fs} N m flange meets 0.30 pp for T >= {ok.min():.1f} N m")
```

```
 T =   5.0 N m   U(eta) = 1.11 pp (100 N m)   2.22 pp (200 N m)
T =  10.0 N m   U(eta) = 0.56 pp (100 N m)   1.11 pp (200 N m)
T =  20.0 N m   U(eta) = 0.29 pp (100 N m)   0.56 pp (200 N m)
T =  35.8 N m   U(eta) = 0.17 pp (100 N m)   0.32 pp (200 N m)
T =  60.0 N m   U(eta) = 0.12 pp (100 N m)   0.20 pp (200 N m)
100 N m flange meets 0.30 pp for T >= 19.1 N m
200 N m flange meets 0.30 pp for T >= 38.1 N m
```

This is why R9 is stated "for torque ≥ 20 N·m": below that the torque term dominates and no power analyser can rescue the measurement. The map is still measured at low torque, but its points are drawn with their larger uncertainty (Figure 6.4) instead of being presented as equally good. The 200 N·m flange would have met R9 only above 38 N·m, above the motor's rated torque.

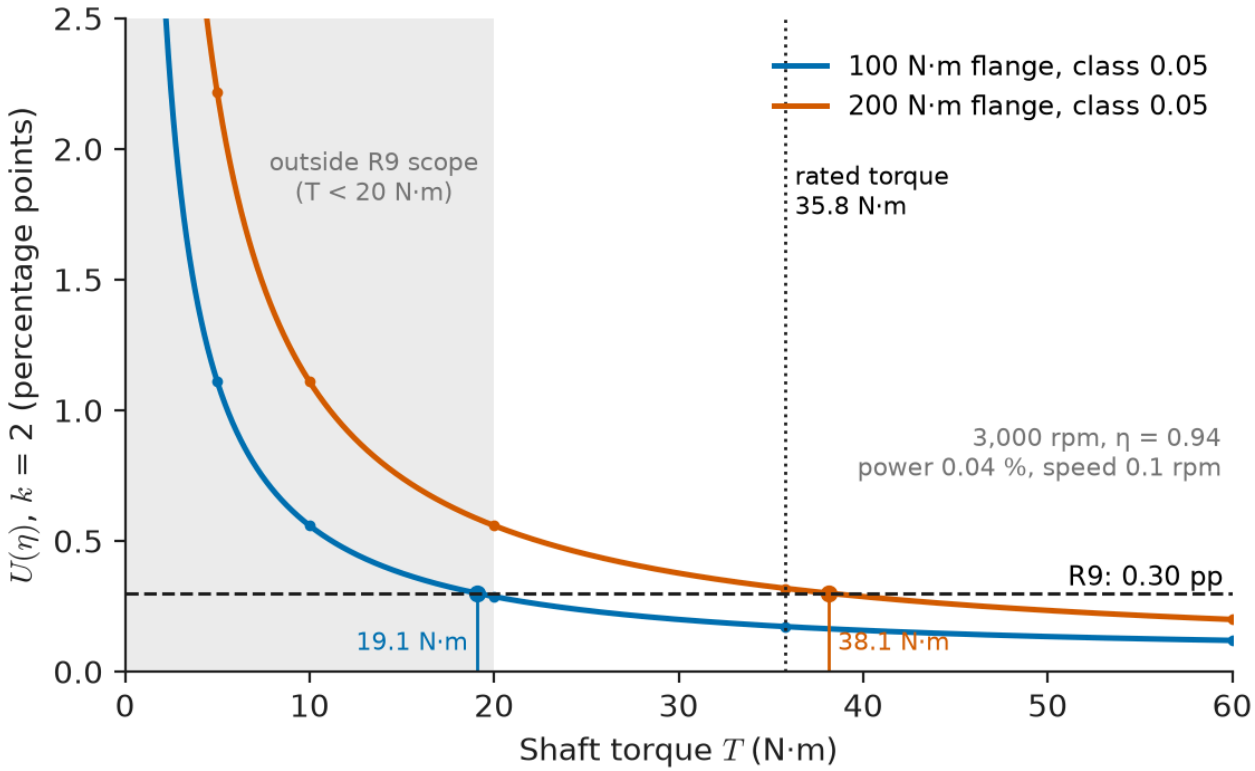


Figure 6.4 — Expanded uncertainty of measured efficiency versus shaft torque at 3,000 rpm for 100 N m and 200 N m class 0.05 torque flanges.

> **Status of the principle: Full-scale-specified errors grow at low reading**
> *Derived.* An error limit fixed as a fraction of full scale is a constant absolute error, so its relative size is inversely proportional to the reading; the useful range of a transducer for a relative-accuracy requirement is therefore bounded below, typically at 20–30 % of full scale.

6.7 Light sensors

Photodiodes, phototransistors and thermopiles

Light sensors convert optical power into current or voltage. **Photodiodes** give a photocurrent proportional to optical power (about 0.5 A/W for silicon near 850 nm), linear over many decades and fast (nanoseconds with a reverse bias); **phototransistors** add gain but are slower (microseconds) and less linear; **integrated ambient-light sensors** add a filter matched to the eye and a digital interface; **thermopiles** are chains of thermocouples that absorb infrared radiation and are the basis of non-contact thermometers. Device physics is in *Electronics and Computing*, Ch. 2 §2.6 and Ch. 5 §5.7; detector materials in *Materials Science and Engineering*, Ch. 35 §35.5. On the bench, light sensors do three jobs: the optical encoder (Section 6.4) is a photodiode array behind a grating; the reflective optical tachometer is an infrared LED and phototransistor reading a strip of retro-reflective tape; and the rotor magnet temperature (R7) is measured by a thermopile through a window in the end cap.

Infrared thermometry of the rotor

The rotor's magnets are the part of the motor most at risk thermally (demagnetisation above about 150 °C for common NdFeB grades) and the hardest to reach: they rotate at up to 12,000 rpm and sit at no fixed potential. A thermopile with an 8–14 µm filter viewing the rotor's end face through a germanium window measures the radiated power, which depends on temperature *and on emissivity*. Bare magnet coatings and machined steel have emissivities of 0.1–0.3 that change with polishing and oxidation; the rotor therefore gets a spot of high-emissivity black paint (ε ≈ 0.95).

Worked example — The cost of an emissivity error. The thermometer is set to ε = 0.95; the paint, after a few hundred hours, is really 0.90. Neglecting reflected radiation, the indicated temperature scales as $(\varepsilon/\varepsilon_{\text{set}})^{1/n}$ with $n \approx 4$ for a broadband detector (Stefan–Boltzmann; *Engineering Physics*, Ch. 10 §10.8), or $n \approx c_2/(\lambda T) = 14388/(10 \times 373) = 3.86$ for an 8–14 µm band at 100 °C. At a true 100 °C (373.15 K) the reading is $373.15 \times (0.90/0.95)^{1/3.86} = 367.96$ K, that is 5.2 °C low, beyond R7's ±5 °C limit from this one term alone. The bench therefore re-checks the painted spot's emissivity every 200 hours against a test rotor with an embedded, telemetered thermocouple, and the window's transmission is calibrated in place.

> **Status of the principle: Radiation thermometry depends on emissivity**
> *Fundamental.* Emitted radiance is emissivity times the black-body radiance at the surface temperature, so a non-contact thermometer measures temperature only through an assumed emissivity; every error in that assumption is an error in temperature, larger at longer wavelengths and lower temperatures.

Optical tachometer and ambient light

The tachometer's phototransistor also sees ambient light: lighting flickering at 100 Hz produced spurious pulses near 6,000 rpm during commissioning. Synchronous detection of a modulated LED, as in industrial optical sensors, cures this; the bench used a hood and a 940 nm daylight filter.

6.8 Magnetic sensors

Families

Magnetic-field sensors underlie current transducers, angle sensors, proximity switches and gear-tooth speed pickups. The families differ by orders of magnitude in sensitivity:

Table 6.7 — Magnetic sensor families (typical).

Family	Range	Resolution	Offset and drift	Typical use
Hall effect (CMOS)	±10 mT to ±2 T	about 10 µT	Tens of µT offset; chopper-stabilised versions better	Current sensors, switches, angle ICs, BLDC commutation
AMR (anisotropic magnetoresistance)	±1 mT (sensing), saturated mode for angle	about 10 nT	Needs set/reset pulses	Compasses, angle (180° periodic)
GMR / TMR (giant, tunnel magnetoresistance)	±1 to ±10 mT, or saturated angle mode	1–10 nT	Low; TMR has high output	Angle sensors, current sensors, read heads
Fluxgate	±100 µT (field) or as zero-flux current transducer	about 0.1 nT	Very low drift	Precision current transducers, magnetometers
Inductive (eddy-current) proximity	Distance 1–20 mm to metal	µm	Temperature dependent	Industrial proximity, shaft displacement

The physics of Hall and magnetoresistive effects is in *Engineering Physics*, Ch. 14 §14.2 and §14.9, and the magnetic materials behind fluxgate cores and bias magnets in *Materials Science and Engineering*, Ch. 29 §29.9.

Stray fields on a power bench

A magnetic angle sensor reads the direction of the field from a diametrically magnetised magnet on the shaft end, typically 40–60 mT at the sensor. Any external field adds vectorially and turns the measured angle. A motor bench is full of external fields: phase cables carrying 300 A peak.

Worked example — A cable beside the angle sensor. The field of a long straight conductor at distance r is $B = \mu_0 I/(2\pi r)$. A 300 A phase cable 20 mm from the sensor produces $4\pi \times 10^{-7} \times 300/(2\pi \times 0.02) = 3.0$ mT. Perpendicular to a 50 mT magnet field, it turns the measured angle by up to $\arctan(3.0/50) = 3.4^\circ$, at the PWM fundamental frequency, so it shows up as angle noise rather than an offset. At 50 mm the error is 1.4°, at 100 mm 0.69°. The TMR angle sensor on the load machine is therefore used only for coarse position (its role is a plausibility check of the optical encoder), cables are routed at least 100 mm away and twisted as a three-phase set so their fields largely cancel, and for the DUT drive the resolver, being an inductive transformer with a high-amplitude carrier, is far less susceptible. Differential (gradiometric) Hall angle sensors, which subtract the uniform part of an external field, are the production solution where a magnetic sensor must sit near power cables.

> **Status of the principle: Superposition of magnetic fields at the sensor**
> *Fundamental.* A magnetic sensor responds to the total field at its location; any external source adds vectorially to the field it is meant to measure, so stray-field immunity must be designed by distance, shielding, cancellation or gradiometric sensing, not assumed.

6.9 Current and voltage sensors

Current: five ways

Table 6.8 — Current-sensing options for a motor bench (typical).

Sensor	Bandwidth	Accuracy	Offset	Isolation	Notes
Shunt resistor	DC to MHz (coaxial)	0.1 % (calibrated 0.01 %)	µV-level	None; needs isolated amplifier	Heating at 300 A; best for low current
Current transformer	50 Hz to kHz	0.2–1 % (IEC 61869-2 classes)	No DC	Inherent	AC only; saturates with DC
Open-loop Hall	DC to 50 kHz	1–2 %	0.5–1 % FS, drifts	Inherent	Cheap, low power
Closed-loop (compensated) Hall	DC to 100–200 kHz	0.3–0.5 %	about 0.1–0.2 % FS	Inherent	Drive control standard
Fluxgate (zero-flux) transducer	DC to 300 kHz–1 MHz	0.01 % or better, linearity ppm	about 10–20 ppm FS	Inherent	Power analysers, calibration
Rogowski coil	Hz to MHz	0.5–1 %	No DC	Inherent	Fast transients, large conductors

Current transformers as instrument transformers are treated in *Electrical Energy and Machines*, Ch. 4 §4.13; in drive power stages the closed-loop Hall sensor is the usual choice (*Electronics and Computing*, Ch. 32 §32.4).

Why the power measurement uses fluxgates

R8 asks for a power uncertainty of 0.04 % of reading, and the efficiency map extends to light loads, where offsets dominate.

Worked example — Offset at light load. The DC-link current transducer is rated 100 A. A closed-loop Hall transducer with an offset of 0.2 % of rating has a 0.2 A offset; at a light-load point drawing 2 A from the DC link, that is a 10 % error in input power, and it drifts with temperature so it cannot simply be zeroed out once. A fluxgate transducer with a 20 ppm offset has $20 \times 10^{-6} \times 100\ \text{A} = 2\ \text{mA}$, 0.1 % at 2 A and negligible above 20 A. On the AC side the same argument applies to the rms currents and to the power factor at light load. The bench therefore uses fluxgate transducers (400 A rating on the phases, 100 A on the DC link) feeding a precision power analyser, while the DUT inverter keeps its own closed-loop Hall sensors for control, which needs bandwidth and robustness rather than 0.01 %.

Voltage

Voltage up to 800 V is measured by the power analyser's own isolated 1,000 V inputs, specified together with the analyser (typical basic accuracy 0.02–0.05 % of reading at the fundamental); a passive divider is used only where the analyser cannot be connected. For control and protection inside an inverter the standard solution is a high-ohmic resistive divider (several resistors in series to share voltage and creepage) into an isolated amplifier or isolated sigma-delta modulator. Working on an 800 V bench imposes the safe-testing

practice in *Electrical Energy and Machines*, Ch. 29 §29.7: CAT-rated probes and leads, interlocked guards and a discharge check of the DC link before contact.

> **Boundary.**
> Isolation amplifiers, bridge conditioning and anti-alias filtering are designed in Chapter 7; power-analyser-grade measurement of harmonics and power quality is the subject of Chapter 9; the current sensing of a production drive is applied in Chapter 37.

6.10 Sensor selection

A procedure

The sections above applied the same steps to each quantity. Written as a procedure:

1. **Quantity and reference**: what exactly is measured (gauge or absolute pressure; slot or end-winding temperature; shaft or reaction torque).
2. **Range including abnormal conditions**: overload, short-circuit torque, over-temperature, plus the low end where full-scale errors dominate (Section 6.6).
3. **Accuracy allocated from a budget**: the sensor's share of the derived requirement (R9 for torque and power), not a catalogue class chosen by habit.
4. **Dynamics**: bandwidth or time constant against the signal (100 kHz for currents with PWM ripple, 8 s for a slot RTD against a 40 s dwell).
5. **Environment**: temperature, vibration, EMI and stray fields, potential and insulation (an RTD in an 800 V winding), ingress protection, cleanliness (optical encoders).
6. **Interface and cabling**: 4-wire RTD, 4–20 mA, RS-422, digital; what the cable adds and how faults are detected.
7. **Installation**: mounting, thermal contact, coupling stiffness, alignment.
8. **Calibration and traceability**: can it be calibrated in place or must it be removed; interval; who certifies it.
9. **Cost, availability and lifetime**, including the cost of a wrong measurement.

The bench's selection

Table 6.9 summarises the result for every channel of the bench, each traced to its requirement.

Table 6.9 — Sensor selection for the motor test bench.

Quantity (req.)	Candidates compared	Chosen	Deciding reasons
Torque (R1, R9)	Flange 100 N·m class 0.05; flange 200 N·m class 0.05; reaction load cell	100 N·m flange, digital output, speed track	Meets R9 from 19.1 N·m; reaction method adds bearing friction
Speed (R2)	Encoder M/T; flange speed track 1024 pulses; optical tach	Encoder M/T at 1 kHz; tach as independent check	0.07 rpm at 12,000 rpm; tach catches encoder faults
Angle (R3)	Resolver ±10′; 4096-line optical; TMR angle IC	4096-line optical encoder	±0.0056° against ±0.05° required
Winding temp. (R4)	PT1000 class A; PT1000 class B; type T; type K	PT1000 class A, 4-wire, slot-embedded	±0.45 °C at 150 °C; insulation for 3 kV
Surface and coolant temp. (R5)	Type T class 1; type K class 1; NTC	Type T class 1	0.6 °C at 150 °C plus CJC within ±1.5 °C
Coolant pressure (R6)	Piezoresistive oil-filled; thin-film on steel; ceramic	Thin-film 0–4 bar, 4–20 mA	0.5 % FS total error band 0–80 °C; wire-break detection
Magnet temp. (R7)	Thermopile 8–14 µm; telemetry thermocouple	Thermopile + painted spot, telemetry for periodic check	Non-contact at 12,000 rpm; telemetry too costly for routine use
Currents (R8)	Closed-loop Hall; fluxgate; coaxial shunt	Fluxgate 400 A and 100 A	Offset 20 ppm; 0.01 % class
Voltage (R8)	Analyser direct input; divider + isolated amplifier	Analyser direct 1,000 V input	Accuracy specified with analyser

6.11 Sensor calibration

Traceability chains

Each sensor is calibrated against a reference whose own calibration traces, through an unbroken chain of comparisons with stated uncertainties, to national standards (ISO/IEC 17025 laboratories issue the certificates). The calibration procedure itself (points, dwell, fit, verification at independent points, guard bands, records) is the one developed in Chapter 4 (§4.7); here it is applied to each sensor family:

Table 6.10 — Calibration methods for the bench sensors.

Sensor	Reference	Method	Typical reference uncertainty	Interval
Torque flange	Lever arm and calibrated weights (DIN 51309 / EURAMET cg-14 procedures) or reference transducer	Static steps up and down, both directions	0.01–0.02 %	12 months
PT1000, thermocouples	Dry-block or liquid-bath calibrator with reference PRT	Comparison at 3–5 temperatures	0.03–0.1 °C	12 months; in-situ check every 3 months
Pressure transmitters	Pressure calibrator or dead-weight tester	5 points up and down	0.02–0.05 % FS	12 months
Fluxgate transducers and analyser	Accredited power calibration	Ratio and phase at several currents and frequencies	0.005–0.01 %	12 months
Encoder	Manufacturer's certificate; index and count check in situ	Count per revolution against optical tach	—	On installation
IR thermopile	Black-body source; test rotor with telemetry	3 temperatures; emissivity of spot	0.5–1 °C	200 h (emissivity), 12 months

Calibrating the torque flange

The flange is calibrated in place with a calibrated lever arm clamped to the shaft and certified weights hung from it, with the load machine's shaft locked. The applied torque is $T = m\,g\,L$, and g **is the local acceleration of gravity, not the standard value**.

Worked example — Local gravity. The bench is in Dhaka (latitude 23.8° N, about 10 m above sea level). The international gravity formula with the free-air correction gives $g = 9.7887$ m/s², against the standard $g_n = 9.80665$ m/s². Using g_n would overstate every applied torque by $9.80665/9.7887 - 1 = 0.183$ %, and the calibrated flange would then read 0.183 % high forever, almost four times its class. A 10 kg weight on a 1.0000 m arm applies $10 \times 9.7887 \times 1.0000 = 97.89$ N·m, not 98.07 N·m. (The difference between the equator and the poles is about 0.5 %, so this error is never negligible for class 0.05 work.)

The calibration data are analysed for sensitivity, linearity and hysteresis as a percentage of full scale:

```
 import numpy as np

G_LOCAL = 9.78871          # m/s^2 at the lab (23.8 deg N, 10 m above sea level)
ARM = 1.0000               # m, calibrated lever arm
mass = np.array([0, 2, 4, 6, 8, 10.0])                  # kg, OIML class M1 weights
torque = mass * G_LOCAL * ARM                            # N m applied
up   = np.array([0.0, 0.39369, 0.78759, 1.18168, 1.57597, 1.97047])  # mV/V
down = np.array([0.00012, 0.39419, 0.78819, 1.18228, 1.57637, 1.97047])

T_FS = 100.0
mean = (up + down) / 2
k, z = np.polyfit(torque, mean, 1)                       # best-fit straight line
span = k * T_FS                                          # mV/V at full scale
lin = np.max(np.abs(mean - (k * torque + z))) / span * 100
hys = np.max(np.abs(down - up)) / span * 100
print(f"sensitivity {k*1e3:.3f} uV/V per N m, span {span:.4f} mV/V at {T_FS:.0f} N m")
print(f"linearity {lin:.3f} % FS, hysteresis {hys:.3f} % FS, "
      f"zero return {1e3*(down[0]-up[0]):.2f} uV/V")
print(f"span error had standard g been used: {100*(9.80665/G_LOCAL-1):+.3f} %")
```

```
 sensitivity 20.129 uV/V per N m, span 2.0129 mV/V at 100 N m
linearity 0.009 % FS, hysteresis 0.030 % FS, zero return 0.12 uV/V
span error had standard g been used: +0.183 %
```

Linearity (0.009 % FS) plus hysteresis (0.030 % FS) is 0.039 % FS, inside class 0.05; the calibration is repeated in the generating (negative torque) direction with the arm on the other side. The new sensitivity, 20.129 µV/V per N·m, is entered into the DAQ with the certificate number. Figure 6.5 shows the deviations.

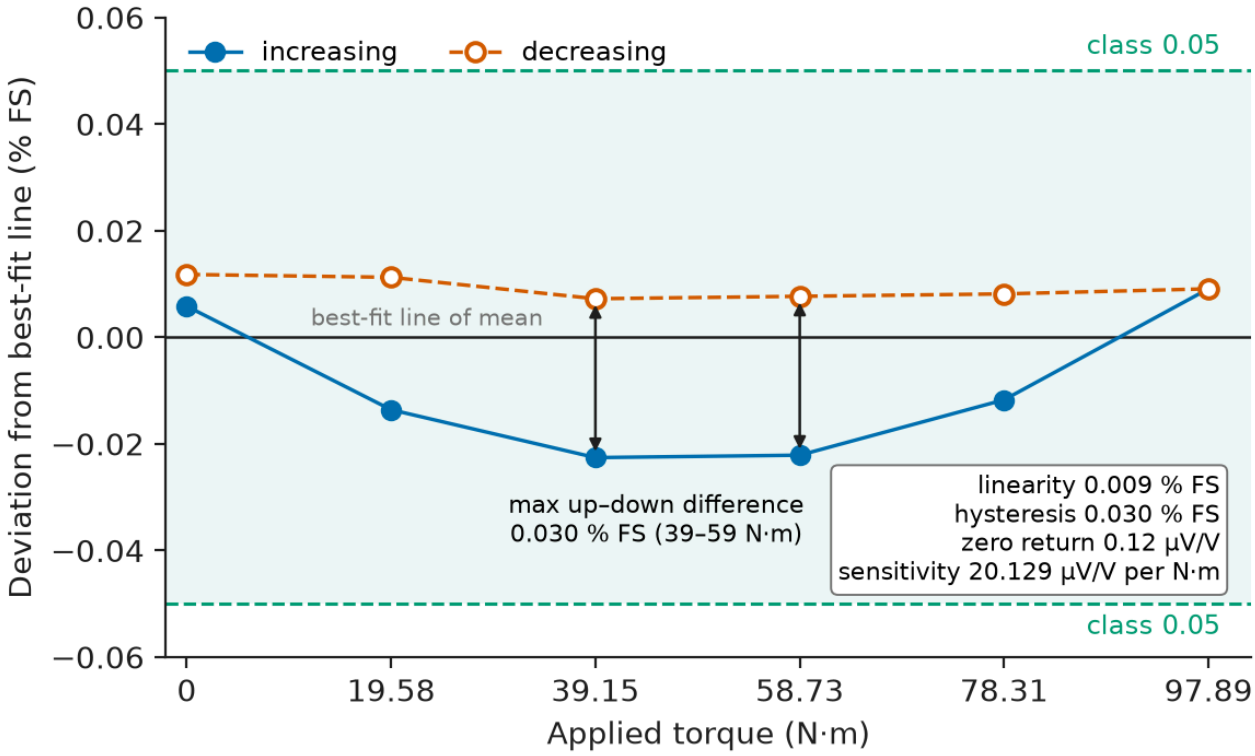


Figure 6.5 — Torque-flange calibration: deviation of increasing and decreasing readings from the best-fit line, in percent of full scale, with the class 0.05 band.

> **Status of the principle: Traceable calibration**
> *Empirical.* A measurement result is comparable with others only through an unbroken, documented chain of calibrations to national standards with uncertainties at each step (the metrological traceability of ISO/IEC 17025 and the International Vocabulary of Metrology); a calibration with an unstated reference, such as an unstated value of g, is not traceable however careful it was.

Commissioning verification

After calibration the bench was verified against Table 6.1 and by a round robin: the same reference motor measured on the customer's bench, whose efficiency at 3,000 rpm and 30 N·m was 94.62 %.

Table 6.11 — Verification of the rebuilt bench.

Req.	Test	Limit	Result	Verdict
R1	Calibration, both directions	Class 0.05	0.039 % FS (lin + hys)	Pass
R2	Encoder speed against tach and frequency reference, 100–12,000 rpm	≤ 1 rpm	0.3 rpm worst (1 s averages)	Pass
R3	Index position repeatability, 100 revolutions	±0.05°	±0.006°	Pass
R4	Slot PT1000 against reference PRT in dry block, 25/100/150 °C (before winding)	±1 °C	±0.31 °C worst	Pass
R5	Type T against reference, 25/100/150 °C	±1.5 °C	±0.7 °C worst	Pass
R6	Pressure calibrator, 5 points, 25 and 65 °C media	±20 mbar	±11 mbar	Pass
R7	Thermopile against telemetry test rotor, 60–140 °C	±5 °C	±2.8 °C	Pass
R8	Analyser and fluxgates, accredited certificate	0.04 % (standard)	0.03 %	Pass
R9	Round robin, reference motor, 3,000 rpm, 30 N·m	Agreement within combined U of both benches (0.27 pp)	94.71 % vs 94.62 %: 0.09 pp	Pass
R10	Calibration register audit	All channels ≤ 12 months	42 of 42	Pass

Failure analysis of the bench sensors

Table 6.12 — FMEA of the motor-bench sensor set.

Failure mode	Effect	Cause	Detection	Mitigation
Torque zero drift during a run	Efficiency biased, 1 pp at 20 N·m with a 500 N·m flange	Coupling heating; oversized flange	Zero check at standstill before and after each map; round robin	100 N·m flange; automatic re-zero every 15 min at standstill
Winding temperature lags	Thermal map optimistic by 15 °C	Probe taped to end winding	Comparison of slot and end-winding probes	Slot-embedded PT1000 class A
Encoder miscount	Speed and angle wrong; efficiency wrong	Electrical noise on RS-422 lines; contamination	Count per revolution checked against index and optical tach	Shielded twisted pairs, differential receivers, index check every revolution
Angle sensor disturbed by cable field	Angle noise up to 3.4°	Phase cable 20 mm from magnetic sensor	Angle noise at PWM frequency	Route cables 100 mm away, twisted set; optical encoder as reference
IR reading low	Magnet temperature underestimated by 5 °C	Emissivity of painted spot fell from 0.95 to 0.90	200 h check against telemetry rotor	Repaint and re-check; window kept clean
Pressure transmitter wire break	Reads 0 bar; overpressure undetected	Connector vibration	Loop current below 3.6 mA (NE 43)	Alarm on loop fault; locking connectors
Fluxgate transducer loses zero-flux lock	Current reading saturates	Overcurrent above rating during a DUT fault	Transducer status output	Status wired to bench interlock; DUT trip at 350 A
Calibration with standard g	Torque reads 0.183 % high	Local gravity not applied	Calibration review	Local g on procedure and certificate

The opening bench failed through two of these rows. The rebuilt bench agrees with the customer's within 0.09 percentage points, well inside the combined uncertainty of the two benches.

What remains open

Open question	Where it is answered
How are bridges, RTDs, thermocouples and 4–20 mA loops conditioned, filtered, isolated and digitised without losing the accuracy chosen here?	Chapter 7
How are expanded uncertainties computed per GUM and used in decision rules?	Chapter 7
How are the 40-odd bench channels sampled, synchronised with the power analyser and logged?	Chapter 8
How are motor-speed, temperature and power-quality instruments built as stand-alone projects?	Chapter 9
How are encoders and resolvers read by timers and RDCs in firmware?	Chapter 15
Which industrial proximity, pressure, flow and limit sensors are used on machines and process plants?	Chapter 19
How is a production drive's current and speed sensing designed and protected?	Chapter 37
How are the motor's losses and efficiency map explained physically?	*Electrical Energy and Machines*, Ch. 8

Next: Chapter 7 takes the sensor outputs chosen here (millivolt bridges, RTD resistances, thermocouple EMFs, loop currents) through bridges, instrumentation amplifiers, filters, isolation and ADCs, and puts a number on the uncertainty each stage adds.

Chapter 7 — Measurement and Signal Conditioning

Part II · Instrumentation and Measurement Systems

A materials laboratory commissions a servo-hydraulic fatigue rig for aluminium suspension arms. Eight foil strain gauges on each arm feed a new conditioning module, and on the bench, with a resistor box standing in for the gauges, the module is quiet to a fraction of a microstrain. On the rig it is not. When the 55 kW hydraulic pump starts on its variable-frequency drive, every channel jumps by 30 to 40 με and stays there, a 14 με peak-to-peak 50 Hz ripple appears, and narrow spikes follow each switching edge of the drive. Nothing in the module is broken. The bench's resistor box floated; the rig's frame sits about 10 V rms from the module's ground and carries the drive's common-mode edges; the cable shields were bonded at both ends; and the in-amp inputs rectified radio-frequency energy into a DC shift. This chapter designs that module properly and verifies it against numbers written down before the build.

> **How is a chain from a millivolt-level sensor to a digital number designed so that its range, resolution and uncertainty are known before it is built?**
> **Which conditioning decisions — bridge, amplifier, filters, isolation, converter, sampling — decide whether a measurement survives a real installation rather than a bench?**

Concept	Engineering destination
Measurement range and resolution	Choosing gain, headroom and converter bits for load cells, pressure dies, current shunts
Accuracy and precision	Separating bias from scatter in calibration data; deciding what a calibration can fix
Error and uncertainty	Uncertainty budgets for test reports, calibration certificates and acceptance limits
Bridge circuits	Strain gauges, load cells, pressure sensors, RTDs, magnetoresistive sensors
Instrumentation amplifiers	Bridge, thermocouple, shunt and biopotential front ends
Filtering	RFI protection, anti-aliasing, decimation, low-latency control outputs
Isolation	Rig and plant grounds, motor drives, mains-referenced and high-voltage measurements
ADC and DAC selection	Simultaneous versus multiplexed, SAR versus delta-sigma, analog outputs to controllers
Sampling	Sample rate, oversampling, inter-channel skew
Noise reduction	Noise budgets, shielding, grounding, guarding, averaging

Roadmap. The reference system and what range and resolution it needs (7.1) → what accuracy and precision mean in its calibration data (7.2) → the uncertainty budget (7.3) → the bridge (7.4) → the in-amp

(7.5) → four filters with four jobs (7.6) → isolation (7.7) → converter selection (7.8) → sampling (7.9) → noise reduction and the rig failure explained (7.10) → implementation, verification and failure analysis (7.11).

7.1 Measurement range and resolution

> **Boundary.**
> Strain-gauge physics, gauge factor, transverse sensitivity and thermal output belong to Chapter 6; the conductor and alloy properties behind them to *Materials Science and Engineering*, Ch. 25. This chapter takes the gauge as a 350 Ω resistor whose value changes by $\Delta R/R = \mathrm{GF} \cdot \varepsilon$ and designs everything after it.

The reference system

The running system of this chapter is an **8-channel isolated strain-gauge conditioning module** for the fatigue rig. Each channel serves one 350 Ω foil gauge with gauge factor 2.0 in a quarter-bridge, 3-wire connection over 5 m of shielded cable. The module samples all channels simultaneously, logs them at 2 kSa/s to the data-acquisition system of Chapter 8, and drives one selected channel to the rig's servo controller as a ±10 V analog signal for strain-controlled tests. Table 7.1 is the requirement set it is built to.

Table 7.1 — Requirements for the strain-gauge conditioning module.

ID	Requirement	Target	Verification
R1	Measurement range	±3000 με per channel without clipping, after ±500 με of initial bridge imbalance	Bridge calibrator, ±3500 με steps
R2	Noise	≤ 1 με rms in the 2 kSa/s logged stream, rig running	60 s record at zero load, pump on
R3	Module uncertainty	≤ 0.15 % of reading + 4 με (k = 2), 15–35 °C module ambient	Bridge calibrator at 9 points in a climatic chamber
R4	Bandwidth	0–200 Hz; amplitude error ≤ 0.1 % at 30 Hz	Sine from calibrator, 1–200 Hz
R5	Inter-channel skew	≤ 10 µs	Common sine into all channels, cross-correlation
R6	Common-mode immunity	≤ 1 με shift for 10 V rms 50 Hz and 1 kV/µs edges between rig and module ground	Injection test
R7	Zero drift	≤ 0.5 με/°C	Climatic chamber, dummy bridges
R8	Analog output	±10 V for ±3000 με, 16-bit, latency ≤ 1 ms	Step input, oscilloscope
R9	Shunt calibration	Built-in 1000 με equivalent step, ±0.1 %	Compare with bridge calibrator
R10	Isolation	Channel bank to system: 1.5 kV rms for 60 s (functional), CMTI ≥ 50 kV/µs	Hipot, datasheet

The architecture (Figure 7.1) follows the signal: bridge, RFI filter, instrumentation amplifier, analog anti-aliasing filter and one 8-channel simultaneous-sampling ADC inside an isolated domain; a digital isolator and isolated DC–DC converter across the barrier; a microcontroller that decimates, scales and logs; and a DAC for the controller output. The interfaces are ±2.5 V excitation and three wires per gauge, ±5 V full scale at the ADC, SPI at 20 MHz across the barrier, Ethernet to the DAQ.

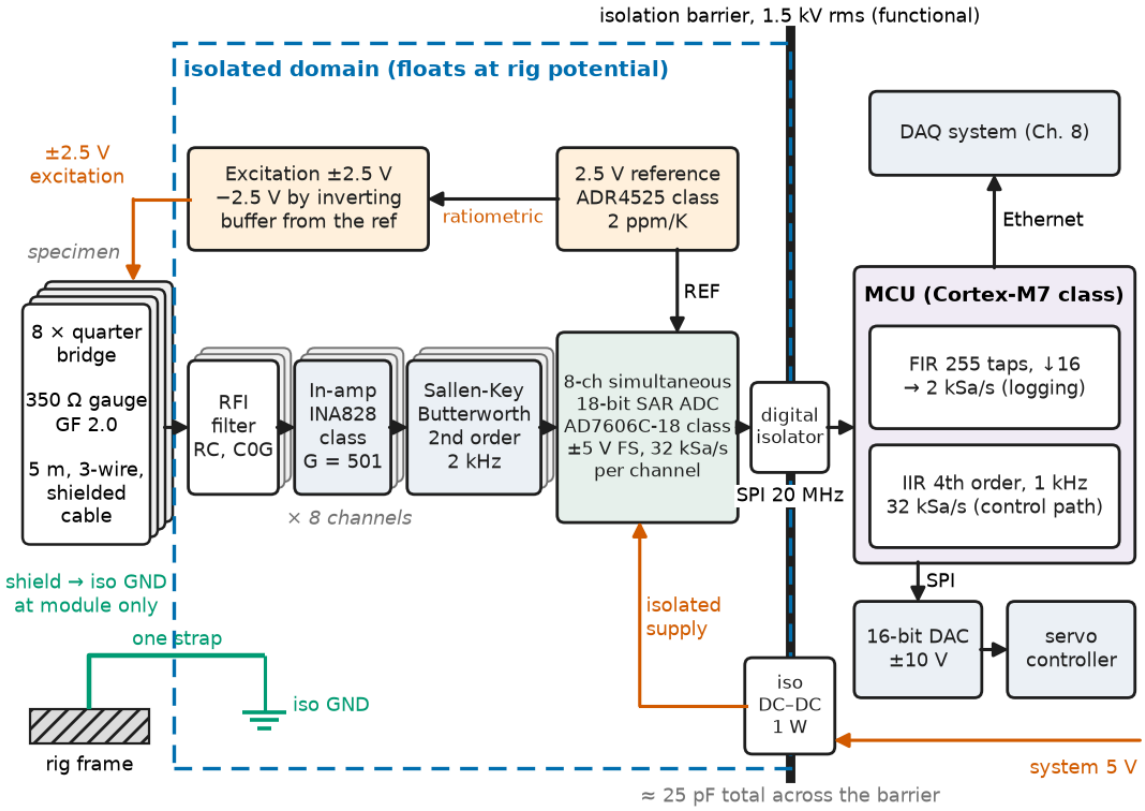


Figure 7.1 — Architecture of the 8-channel isolated strain-gauge conditioning module for the fatigue rig.

Range from the material and the bridge

The range comes from the specimen, not the electronics. A 6061-T6 arm yields near 276 MPa; with $E = 69$ GPa that is about 4000 με, and a fatigue test stays below roughly three quarters of yield, so ±3000 με covers every planned load case. A quarter bridge excited with $V_{ex} = 5$ V gives about $V_{ex}\,\mathrm{GF}\,\varepsilon/4 = 2.5$ μV per microstrain, 7.5 mV at 3000 με. The gauge, its solder tabs and the completion resistor are never exactly 350 Ω, so the bridge starts unbalanced; ±500 με of initial imbalance (0.1 % resistance mismatch) is the allowance in R1. The signal chain must therefore carry ±3500 με, ±8.75 mV, without clipping anywhere — including at the in-amp's internal nodes (Section 7.5).

Resolution: bits needed and bits bought

Resolution is the smallest change the chain can report; it is limited by quantization or by noise, whichever is larger. The ratio of range to resolution sets the minimum effective number of bits:

$$N_{\min} = \log_2 \frac{\text{span}}{\text{resolution}}$$

> **Status of the principle: Effective bits from range and resolution**
> *Derived.* Follows from dividing the span into equal steps; it gives the minimum *effective* resolution, so the converter's ENOB after noise, not its nominal bit count, must meet it, and headroom for imbalance and overload is added on top.

Worked example — How many bits does the strain module need, and how many does it get? The span is 6000 με and the resolution target 1 με, a ratio of 6000 or 75.6 dB, so $N_{\min} = 12.6$ bits. With ±500 με of imbalance and 20 % overload headroom the converter must cover about 8400 με at 1 με, 13.0 bits effective. The chosen chain has an in-amp gain of 501, so 3000 με gives $7.48\ \text{mV} \times 501 = 3.75\ \text{V}$ at a ±5 V, 18-bit converter: one LSB is $10\ \text{V}/2^{18} = 38.1\ \mu\text{V}$, or 76.1 nV referred to the input, 0.03 με. The quantization step is thirty times finer than needed, and Section 7.10 shows the noise is below it as well. The extra bits are not waste: they let one gain cover gauges with very different imbalance without trimming, and they make the noise, not the converter, the resolution limit.

Bits follow from range and resolution at the *input*, after the gain: a "24-bit" converter behind a chain whose noise is 10 με rms resolves 10 με.

7.2 Accuracy and precision

Precision is the closeness of repeated readings to each other under stated conditions (repeatability over a short time, reproducibility across operators, days or rigs). **Trueness** is the closeness of their mean to the true value; its opposite is **bias**. **Accuracy** combines both. A channel can be precise and wrong, and calibration can remove bias but never scatter.

Worked example — A precise channel with a bias. A precision bridge calibrator is connected at the end of a 5 m gauge cable in place of the gauge and set to +1000.0 με (GF = 2.0, uncertainty 0.02 % of setting). Channel 3 is read 25 times at one-second intervals: the mean is 998.1 με and the sample standard deviation 0.22 με. The standard error of the mean is $0.22/\sqrt{25} = 0.044$ με, and with Student's $t_{0.975,24} = 2.064$ the 95 % interval for the mean is 998.1 ± 0.09 με (*The Mathematics Universe*, Ch. 20 §20.7). The channel is repeatable to a fifth of a microstrain, yet reads 1.9 με (0.19 %) low, twenty times its half-width. The cause is the 0.67 Ω of lead resistance, which desensitizes a 3-wire quarter bridge by $R_L/(R_g + R_L) = 0.19\ \%$ (Section 7.4). Corrected, the mean moves to 1000.0 με; the 0.22 με scatter does not change.

A bias that is stable and explained is corrected, and its *correction* carries an uncertainty; scatter enters the budget as a Type A term; a bias that is not understood is a fault to be found, never a number to be subtracted.

7.3 Error and uncertainty

> **Boundary.**
> The GUM vocabulary, Type A and Type B evaluation, the law of propagation of uncertainty, effective degrees of freedom and the coverage factor are set out in *The Mathematics Universe*, Ch. 20 §20.10 (after JCGM 100). Here they are applied to a design: the budget is written before the build, and each line is a design decision.

The error model of a conditioning chain

Every line of a conditioning chain contributes to one of four error shapes:

- **Offset** errors, independent of reading: in-amp offset and its drift, thermal EMFs at connectors, bridge imbalance and the drift of the completion network, residual interference.
- **Gain** errors, proportional to reading: excitation, reference, in-amp gain and its drift, lead desensitization, the calibration reference itself.
- **Nonlinearity**: the quarter bridge's own (Section 7.4), the converter's INL, amplifier distortion.
- **Noise**: random, characterized by a spectral density and a bandwidth.

Zeroing and shunt calibration (R9) remove initial offset and gain; the budget holds the uncertainty of those corrections, the drift after them, and noise.

The budget for the strain module

The listing evaluates the module's budget in the GUM manner. Tolerances and drift specifications are Type B terms with rectangular distributions (half-width divided by $\sqrt{3}$); the at-rest noise, measured on the first rig run after the fixes of Section 7.10, is Type A. Gain terms are combined in percent of reading, offset terms in microstrain, and the two are combined at each reading.

```
 import math

RECT, NORM = math.sqrt(3), 1.0          # divisors: rectangular half-width, standard unc.
# (contribution, type, value, divisor); type 'g' = % of reading, 'o' = microstrain
budget = [
    ("shunt-cal resistor 0.05 %",       "g", 0.05,  RECT),
    ("lead-resistance correction",      "g", 0.014, RECT),
    ("gain drift after cal, +/-10 K",   "g", 0.06,  RECT),
    ("excitation ratio residual",       "g", 0.01,  RECT),
    ("completion-network tracking",     "o", 2.5,   RECT),
    ("in-amp offset drift, +/-10 K",    "o", 1.6,   RECT),
    ("thermal EMF at connectors",       "o", 0.4,   RECT),
    ("random noise (Type A)",           "o", 0.3,   NORM),
    ("residual 50 Hz after isolation",  "o", 0.2,   NORM),
]
def rss(kind):
    return math.sqrt(sum((v/d)**2 for _, k, v, d in budget if k == kind))

u_g, u_o = rss("g"), rss("o")
print(f"gain terms: u = {u_g:.4f} % of reading; offset terms: u = {u_o:.2f} ue")
for eps in (100, 1000, 3000):
    U = 2*math.hypot(u_g/100*eps, u_o)            # expanded, k = 2
    limit = 0.0015*eps + 4.0                       # R3: 0.15 % of reading + 4 ue
    print(f"{eps:5d} ue: U(k=2) = {U:4.2f} ue, limit {limit:4.2f} ue, "
          f"margin {100*(1-U/limit):3.0f} %")
```

```
gain terms: u = 0.0462 % of reading; offset terms: u = 1.77 ue
  100 ue: U(k=2) = 3.53 ue, limit 4.15 ue, margin  15 %
 1000 ue: U(k=2) = 3.65 ue, limit 5.50 ue, margin  34 %
 3000 ue: U(k=2) = 4.49 ue, limit 8.50 ue, margin  47 %
```

Table 7.2 is the same budget with the origin of each value; Figure 7.2 plots the contributions.

Table 7.2 — Uncertainty budget of one channel at 3000 με (25 ± 10 °C, after zero and shunt calibration).

Contribution	Basis	Half-width or s	Standard uncertainty at 3000 με
Shunt-cal resistor	0.05 % tolerance, 175.0 kΩ	0.05 %	0.87 με
Lead-resistance correction	±0.05 Ω on 350 Ω	0.014 %	0.24 με
Gain drift after calibration	in-amp and R_G, 60 ppm/K × 10 K	0.06 %	1.04 με
Excitation ratio residual	ratiometric, 0.01 % divider ratio	0.01 %	0.17 με
Completion-network tracking	0.5 ppm/K × 10 K, ÷ GF	2.5 με	1.44 με
In-amp offset drift	0.4 µV/K × 10 K = 4 µV	1.6 με	0.92 με
Thermal EMF	1 µV at dissimilar contacts	0.4 με	0.23 με
Random noise	measured, 2 kSa/s, pump on	0.3 με	0.30 με
Residual 50 Hz	measured after fixes	0.2 με	0.20 με
Combined, expanded (k = 2)			**4.49 με**
R3 limit / margin	0.15 % + 4 με		**8.50 με / 47 %**

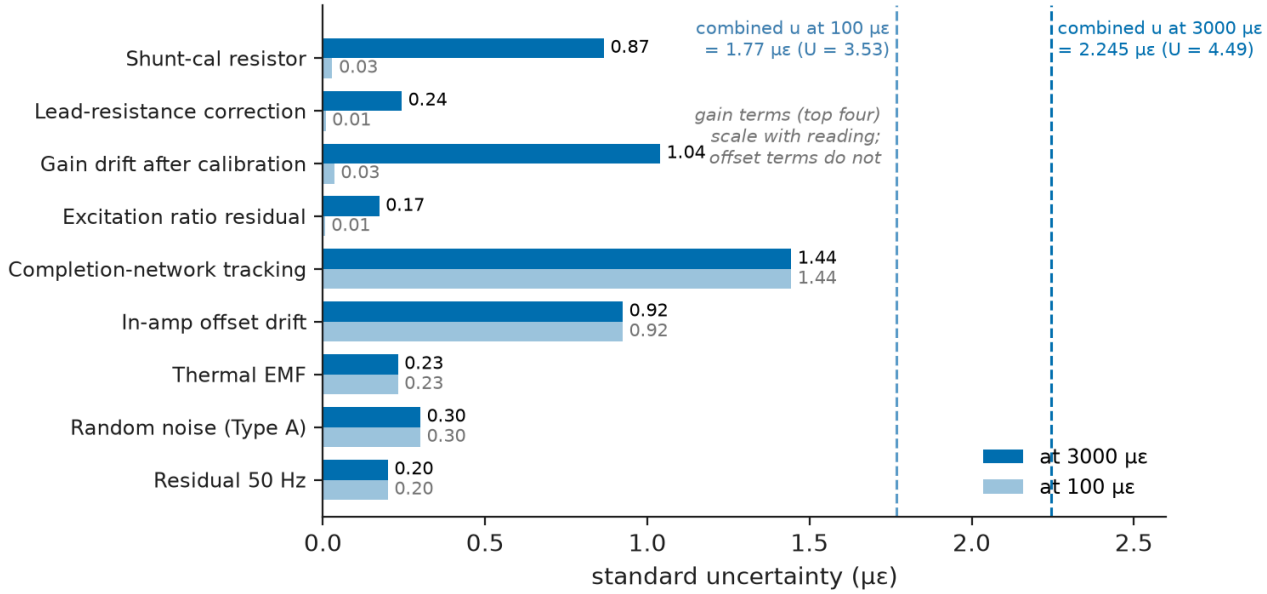


Figure 7.2 — Standard-uncertainty contributions of one strain channel at 3000 and at 100 microstrain.

> **Status of the principle: Root-sum-square combination of independent contributions**
> *Derived.* The law of propagation of uncertainty for a linear model with uncorrelated inputs; correlated terms (two channels sharing one reference, a gain drift and an offset drift driven by the same temperature) need their covariance, and a worst-case linear sum replaces the RSS where a requirement is a safety limit rather than a statistical one.

Two things in the budget change the design. First, the largest term at low strain is not the converter, not noise, but the **temperature tracking of the bridge-completion network**: a 0.5 ppm/K ratio drift over ±10 K is 5 ppm of resistance, $5 \times 10^{-6}/2.0 = 2.5$ με. A pair of ordinary 25 ppm/K thin-film resistors with 10 ppm/K tracking would contribute 50 με and fail R3 and R7 by a factor of ten; the module uses a hermetic metal-foil network with 0.5 ppm/K tracking at about $18 per channel, the most expensive analog part on the board. Second, a 0.05 % shunt-calibration resistor contributes more than the whole lead correction; a 0.01 % part (another $6) would cut that line by five, and is noted for Revision B.

7.4 Bridge circuits

> **Boundary.**
> The Wheatstone bridge as a circuit — its balance condition and Thévenin equivalent — is network analysis from *Circuits, Signals and Communications*, Ch. 3. This section chooses a configuration, excitation and wiring for a measurement, and computes what each choice costs.

Choosing the configuration

A bridge converts a small resistance change into a small differential voltage riding on a large common-mode voltage of $V_{ex}/2$. Which arms are active decides sensitivity, linearity and temperature compensation (Table 7.3).

Table 7.3 — Strain-bridge configurations (350 Ω gauges, GF 2.0, 5 V excitation, typical).

Configuration	Active gauges	Output at 1000 με	Nonlinearity at 3000 με	Temperature compensation	Typical use
Quarter, 2-wire	1	2.50 mV	−0.30 %	None; leads in one arm	Avoid beyond a few centimetres of lead
Quarter, 3-wire	1	2.50 mV (−0.19 % with 5 m leads)	−0.30 %	Leads in adjacent arms cancel	Strain surveys on specimens
Half, bending	2 (top and bottom faces)	5.00 mV	0	Gauges compensate each other	Beams, shafts in bending
Full, bending or Poisson	4	10.0 mV (bending)	0 (bending)	Full	Load cells, torque shafts, pressure dies

The fatigue rig needs strain at eight specific points on an irregular part, so quarter bridges are unavoidable; the completion half-bridge and the dummy arm live in the module (Figure 7.3).

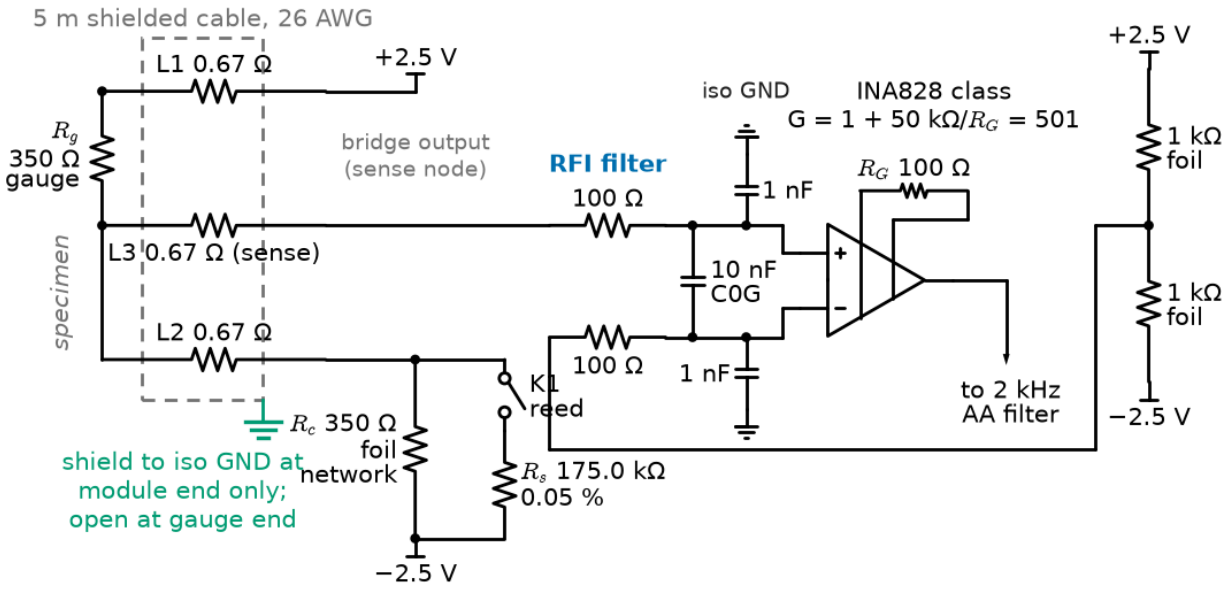


Figure 7.3 — One channel of the quarter-bridge front end with 3-wire gauge connection, symmetric excitation, shunt calibration and RFI filter.

Output, nonlinearity, leads and shunt calibration

For a quarter bridge with all arms nominally equal and $x = \mathrm{GF}\,\varepsilon$, the bridge ratio is

$$\frac{V_o}{V_{ex}} = \frac{x/4}{1 + x/2}$$

so the output falls short of the linear approximation $V_{ex}x/4$ by a fraction of about $x/2$, and a linear conversion reads $-\mathrm{GF}\,\varepsilon^2/2$ in error: low by 0.10 % at 1000 με and 0.30 % at 3000 με in tension, and too compressive by the same amount in compression. The error is exactly known and is removed in firmware by inverting the expression (Section 7.11), so it is a correction, not an uncertainty.

> **Status of the principle: Quarter-bridge nonlinearity**
> *Derived.* Exact for a bridge of four equal nominal arms with one varying; with unequal arms or lead resistance the expression changes slightly, and the firmware correction must use the same arm model as the hardware.

Worked example — Two wires or three, and the shunt resistor. Each conductor of the 5 m cable is 26 AWG copper, 0.134 Ω/m, so $R_L = 0.67\ \Omega$.

- *2-wire.* Both leads sit in the gauge arm and add $2R_L = 1.34\ \Omega$ to 350 Ω, an apparent strain of $1.34/350/2.0 = 1914$ με. Zeroing removes it, but the copper changes resistance by 0.393 %/K: a 10 K change of cable temperature along the rig moves the zero by $0.00393 \times 10 \times 1.34/350/2.0 = 75$ με, twenty times the R3 budget.
- *3-wire.* One lead sits in the gauge arm and one in the adjacent completion arm, so their changes cancel to first order; the third carries no current and only senses. What remains is a gain error: the gauge's change is now measured against $R_g + R_L$ rather than R_g, so the reading is low by $R_L/(R_g + R_L) = 0.19\ \%$, the bias found in Section 7.2. The firmware multiplies by $1 + R_L/R_g$, with R_L measured at installation by the module's lead-resistance check.
- *Shunt calibration.* A resistor R_s across the 350 Ω completion arm lowers it to $R_g R_s/(R_g + R_s)$. The bridge responds only to the ratio of the gauge arm to the completion arm, so this is exactly equivalent to raising the gauge by the fraction R_g/R_s: a tensile step of $R_g/(R_s\,\mathrm{GF})$. For 1000 με at GF 2.0, $R_g/R_s = 0.002$ and $R_s = 175.0$ kΩ. (The often-quoted $R_g(1/(\mathrm{GF}\,\varepsilon) - 1) = 174.65$ kΩ is for a shunt across the gauge itself; used across the completion arm it gives 1002 με, a 0.2 % calibration error.) Its 0.05 % tolerance is the first line of Table 7.2.

Figure 7.4 shows the errors against strain: the nonlinearity is the largest but costs nothing once corrected; the 2-wire lead drift, invisible on a bench, is what ruins a test.

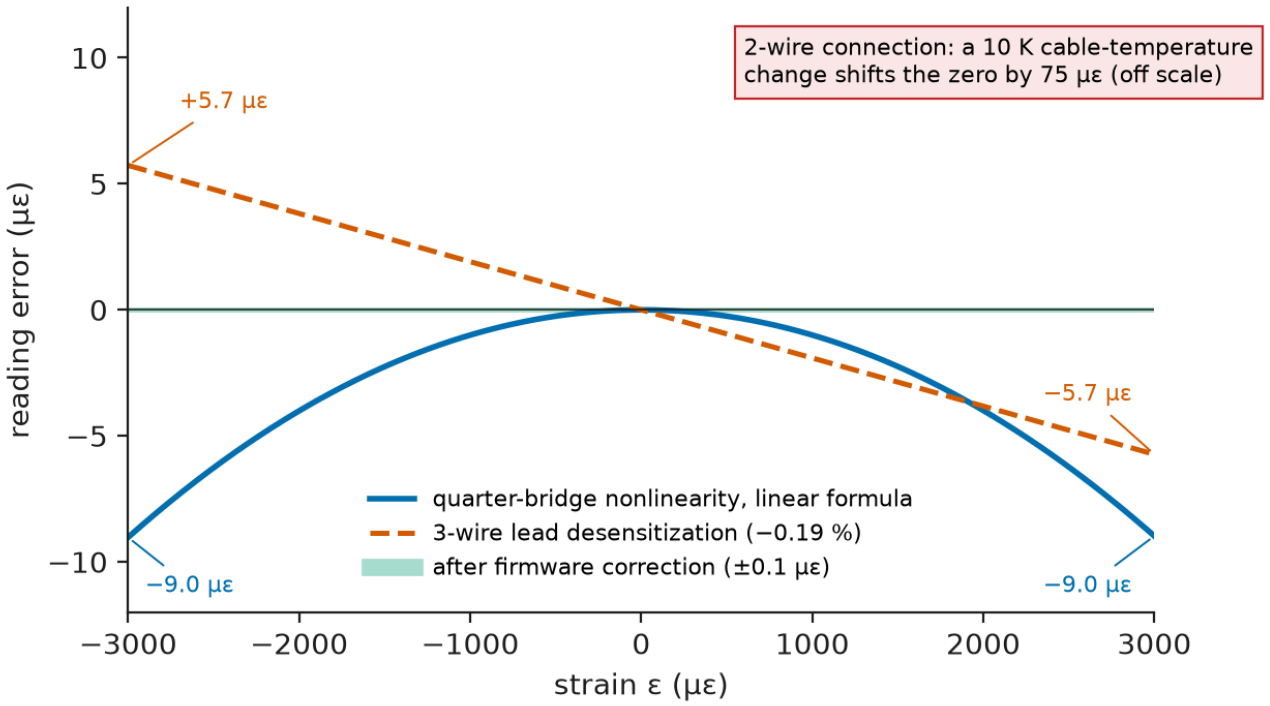


Figure 7.4 — Quarter-bridge reading errors against strain before correction.

Excitation

Excitation trades signal against self-heating. At 5 V the gauge carries 2.5 V/350 Ω = 7.1 mA and dissipates 17.9 mW, acceptable on aluminium, which conducts the heat away; on a composite specimen excitation would drop to 1 to 2 V. Two further decisions:

- **Symmetric excitation**, +2.5 V and −2.5 V, puts the bridge outputs near 0 V instead of 2.5 V. Section 7.5 shows why the in-amp needs this.
- **Ratiometric excitation**: both rails are derived from the ADC's own 2.5 V reference, so a reference error scales signal and full scale together and cancels. The residual is the ratio error of the inverting buffer's resistors, 0.01 %, the fourth line of Table 7.2. Full bridges at the end of long cables add **remote sense** lines (6-wire connection) so the excitation actually reaching the bridge is measured; a 3-wire quarter bridge carries its lead correction instead.

7.5 Instrumentation amplifiers

> **Boundary.**
> The three-op-amp instrumentation amplifier, its gain equation, its gain-proportional CMRR and the common-mode "diamond plot" are in *Electronics and Computing*, Ch. 8 §8.6; offset drift, zero-drift amplifiers and input protection in §8.10. Here they decide a part, a gain resistor and a supply.

Selecting the in-amp

Table 7.4 compares four in-amp classes against R2, R6 and R7. The in-amp's own noise is not decisive (Section 7.10 shows 7 nV/√Hz contributes 0.06 με rms); offset drift, CMRR at 50 Hz and common-mode

range are.

Table 7.4 — In-amp classes for a 350 Ω bridge at G ≈ 500 (datasheet-class values, 2026 distributor prices, 100 pcs).

Class	Noise density	Offset drift (max)	CMRR at G ≥ 100 (min)	Notes	Price	Verdict
INA828 class, bipolar, ±18 V	7 nV/√Hz	0.2–0.4 µV/K	120 dB	Robust inputs, low 1/f noise	$4.50	**Chosen**
AD8421 class, low noise	3.2 nV/√Hz	0.2–0.4 µV/K	126 dB	Lowest noise, 2 mA supply	$7.80	Noise advantage not needed
INA333 class, zero-drift CMOS	50 nV/√Hz	0.1 µV/K	100 dB	5.5 V max supply, chopper ripple near 350 kHz, BW about 3.5 kHz at G = 100	$3.20	Lower CMRR; ripple must be filtered
Discrete difference amplifier, op-amp and 0.01 % resistors	Depends	Depends	74 dB (resistor-limited)	Cheap, inputs load the bridge	$2	Fails R6

The gain of the chosen class is $G = 1 + 50\,\text{k}\Omega / R_G$. For a 3.75 V output at 3000 µε, $G \approx 500$, so $R_G = 50\,\text{k}\Omega / 499 = 100.2\ \Omega$: a 100 Ω, 0.1 %, 10 ppm/K resistor gives $G = 501$. Its tolerance does not matter, because shunt calibration measures the actual gain; its temperature coefficient does, and is part of the 60 ppm/K gain-drift line in Table 7.2.

The common-mode check

Worked example — Why the bridge is excited at ±2.5 V. In the first schematic the bridge was excited from 0 and +5 V, so both inputs sit at a common mode of 2.5 V, and the in-amp ran from an isolated ±5 V supply. The first-stage amplifiers of a three-op-amp in-amp swing to $V_{cm} \pm G\,v_d/2$. At full range plus imbalance, $v_d = 8.75$ mV and $G\,v_d/2 = 501 \times 8.75\ \text{mV}/2 = 2.19$ V, so the internal nodes reach $2.5 + 2.19 = 4.69$ V — inside a ±5 V supply only if the internal amplifiers swing within 0.31 V of the rail, which this class does not guarantee across temperature. The output looks normal until the strain approaches the top of its range and then flattens: a silent clip that a bench test with small signals never shows. With symmetric excitation the common mode is about 0 V, the internal nodes stay within ±2.2 V, and there is 2.5 V of headroom on each side. The check is made against the manufacturer's diamond plot for G = 501 and ±5 V, not against the input-range line on the first page of the datasheet.

Common-mode rejection and the RFI filter

With 120 dB of CMRR at 50 Hz, a 10 V rms common-mode voltage leaves $10\,\text{V}/10^6 = 10$ µV rms of differential error, 4 µε rms — four times R6. CMRR alone cannot meet R6; isolation (Section 7.7) removes the common mode from the in-amp inputs instead.

At radio frequencies a bipolar input stage rectifies interference and produces a DC offset; this was the 30–40 με zero shift seen when the pump started. The cure is a passive RFI filter at the inputs (Figure 7.3): 100 Ω in series with each input, 1 nF from each input to ground and 10 nF across them. The differential bandwidth is $1/(2\pi \cdot 2R(C_d + C_c/2)) = 75.8$ kHz, far above the 200 Hz signal, and the common-mode bandwidth $1/(2\pi R C_c) = 1.59$ MHz. The differential capacitor is ten times the common-mode ones so that mismatch between the two 1 nF parts, which converts common mode into differential mode, is swamped. All three are C0G: X7R capacitors change value with voltage and temperature and are piezoelectric, so they turn vibration on a fatigue rig into signal (*Materials Science and Engineering*, Ch. 27).

7.6 Filtering

> **Boundary.**
> The anti-aliasing stopband rule $f_{stop} = f_s - f_p$ and Butterworth order selection are in *Circuits, Signals and Communications*, Ch. 19 §19.9; FIR and IIR design in Book 3 Ch. 23 §23.5–23.6; Sallen-Key component selection in *Electronics and Computing*, Ch. 8 §8.8.

Four filters, four jobs

The module has four filters, and confusing their jobs is the commonest filtering mistake:

1. **RFI filter** (Section 7.5): keeps megahertz energy out of the in-amp. Corner 75.8 kHz differential, 1.6 MHz common mode.
2. **Analog anti-aliasing filter**: a 2nd-order Sallen-Key Butterworth at 2 kHz after the in-amp, ahead of an ADC sampling each channel at 32 kSa/s. It must attenuate everything that would fold into 0–200 Hz, i.e. above 32 kHz − 200 Hz = 31.8 kHz.
3. **Logging decimator**: a 255-tap linear-phase FIR at 32 kSa/s, decimating by 16 to 2 kSa/s, passband 0–200 Hz, stopband from 800 Hz.
4. **Control-path filter**: a 4th-order Butterworth IIR at 1 kHz on the 32 kSa/s stream, feeding the DAC without decimation.

Oversampling lets the analog filter be gentle; the sharp filtering is digital and identical in every channel. The listing checks the chain against R4, R5 and R8.

```
 import numpy as np
from scipy import signal

FS_ADC, DEC = 32_000, 16            # ADC rate per channel, decimation to 2 kSa/s
FC_AA = 2_000                       # analog 2nd-order Butterworth (Sallen-Key)

# Analog anti-alias filter
b_a, a_a = signal.butter(2, 2*np.pi*FC_AA, analog=True)
def analog(f):
    w, h = signal.freqs(b_a, a_a, worN=2*np.pi*np.atleast_1d(f))
    return h
att_alias = -20*np.log10(abs(analog(FS_ADC - 200)))[0]
gd_aa = np.sqrt(2)/(2*np.pi*FC_AA)          # DC group delay, 2nd-order Butterworth

# Logging path: linear-phase FIR decimator 32 -> 2 kSa/s
fir = signal.remez(255, [0, 200, 800, FS_ADC/2], [1, 0], weight=[1, 3], fs=FS_ADC)
w, h = signal.freqz(fir, worN=2**16, fs=FS_ADC)
ripple = np.ptp(20*np.log10(abs(h[w <= 200])))
stop = -20*np.log10(abs(h[w >= 800]).max())
gd_fir = (len(fir) - 1)/2/FS_ADC

# Control path: 4th-order Butterworth IIR at 1 kHz, no decimation
sos = signal.butter(4, 1_000, fs=FS_ADC, output='sos')
b, a = signal.sos2tf(sos)
_, gd = signal.group_delay((b, a), w=[30], fs=FS_ADC)
gd_iir = gd[0]/FS_ADC

# Channel skew from a 1 % shift in analog cut-off frequency
skew = gd_aa - np.sqrt(2)/(2*np.pi*FC_AA*1.01)

print(f"analog AA: {att_alias:.1f} dB at {FS_ADC-200} Hz, "
      f"droop at 200 Hz {20*np.log10(abs(analog(200)))[0]:.5f} dB, delay {gd_aa*1e6:.1f} us")
print(f"FIR 255 taps: ripple {ripple:.4f} dB, stopband {stop:.1f} dB, delay {gd_fir*1e3:.2f} ms")
print(f"IIR control path: group delay at 30 Hz {gd_iir*1e3:.3f} ms")
print(f"skew per 1 % cut-off mismatch: {skew*1e6:.2f} us")
```

```
 analog AA: 48.1 dB at 31800 Hz, droop at 200 Hz -0.00043 dB, delay 112.5 us
FIR 255 taps: ripple 0.0024 dB, stopband 86.5 dB, delay 3.97 ms
IIR control path: group delay at 30 Hz 0.415 ms
skew per 1 % cut-off mismatch: 1.11 us
```

The FIR's 0.0024 dB peak-to-peak ripple is ±0.014 %, a seventh of R4's 0.1 %; its 86.5 dB stopband removes the drive's 4 kHz switching components and everything that would fold into the passband at 2 kSa/s. Figure 7.5 shows the cascade.

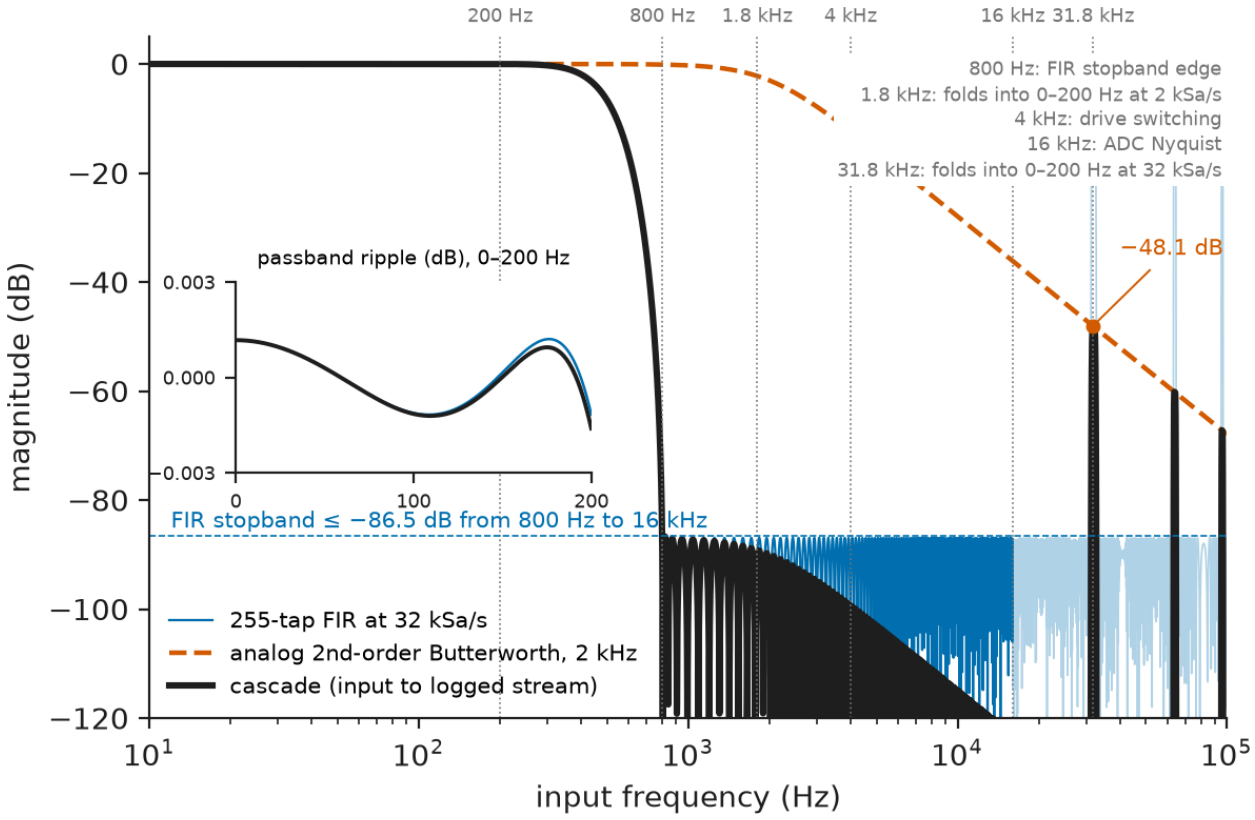


Figure 7.5 — Magnitude response of the filter chain: analog anti-aliasing filter, FIR decimator and their cascade.

> **Status of the principle: Linear-phase FIR delay**
> *Derived.* A symmetric FIR of N taps delays every frequency by exactly $(N-1)/2$ samples; the delay is fixed and correctable in logged data, but it is latency, and it rules a long linear-phase FIR out of any feedback path.

The control path and its latency budget

R8 asks the analog output to follow strain within 1 ms, because the servo controller closes a strain loop on it. Fed from the logging FIR, the DAC would lag 3.97 ms, a phase lag of $360 \times 30 \times 3.97\ \text{ms} = 43°$ at 30 Hz. The control path therefore taps the 32 kSa/s stream through a 4th-order IIR with 0.415 ms of group delay at 30 Hz; its nonlinear phase does not matter to a controller, so the two paths are kept separate (Table 7.5).

Table 7.5 — Latency budget of the analog output path (R8).

Stage	Latency	Basis
Analog anti-aliasing filter	0.113 ms	Group delay, 2nd-order Butterworth, 2 kHz
Wait for next sample (worst case)	0.031 ms	One period at 32 kSa/s
Conversion and SPI read across isolator	0.010 ms	8 channels, 18 bits, 20 MHz
IIR group delay at 30 Hz	0.415 ms	Listing above
Scaling, channel selection	0.020 ms	Measured on target, worst case
DAC SPI write and settling to 16 bits	0.010 ms	Datasheet class
Total / requirement / margin	**0.599 ms / 1.0 ms / 40 %**	

7.7 Isolation

Why the module is isolated

Section 7.5 showed that CMRR leaves 4 με of error from the rig's 50 Hz ground difference; the drive's edges are worse, and a fault could put mains voltage on the rig frame. An **isolation barrier** lets the whole front end float at the rig's potential, so the common-mode voltage appears across the barrier, not across the in-amp inputs.

Functional isolation lets the circuit work; **basic** isolation gives one level of shock protection; **reinforced** two. Grade, creepage, clearance and test voltages come from the product standard, IEC 61010-1 for measurement equipment, with measurement categories for mains-connected inputs (*Electrical Energy and Machines*, Ch. 29 §29.7). The strain inputs are not mains-connected and the rig frame is earthed, so functional isolation with a 1.5 kV test (R10) is adequate; Section 9.4 shows an instrument where it is not.

Two numbers describe a barrier in a noisy installation: its **common-mode transient immunity** (CMTI), the dv/dt the data path survives without errors, and its **barrier capacitance**.

Table 7.6 — Isolation options for a multichannel analog front end (typical).

Option	Example class	What crosses the barrier	Barrier capacitance	Comment
Digital isolator + isolated DC–DC, bank isolation	ISO7741-class 4-channel isolator, 1 W isolated module	SPI and power for all 8 channels	2 pF (isolator) + 3–50 pF (DC–DC)	Chosen; lowest cost per channel
Isolated ΔΣ modulator per channel	AMC1306-class	1-bit stream, decimated in MCU	about 1–2 pF per channel	Channel-to-channel isolation; for shunts on separate potentials
Isolation amplifier per channel	AMC1311-class	Analog voltage	about 1–2 pF	Simple, but ±0.2–0.3 % gain error adds to the budget

The rig's eight gauges sit on one specimen bolted to one frame, so they share a potential and do not need isolation from each other: one barrier for the bank is enough. A test on a battery string, where each sensor sits at a different cell potential, would need per-channel isolation (Section 9.5).

Worked example — What the barrier carries. The chosen isolator and isolated DC–DC converter together present at most about 22 pF across the barrier (20 pF for the DC–DC plus 2 pF for the isolator, budgeted here as 25 pF). With the isolated ground strapped to the rig frame at the specimen end, the 10 V rms, 50 Hz ground difference now appears across that capacitance, whose reactance at 50 Hz is $1/(2\pi \cdot 50 \cdot 25\ \text{pF}) = 127\ \text{M}\Omega$, so it drives only $10\ \text{V}/127\ \text{M}\Omega = 79$ nA, which returns through the strap and shield, not through the signal wires. The in-amp sees essentially no 50 Hz common mode; the residual measured after the fix was 0.2 με. The drive edges are a different matter: at 1 kV/μs the same 25 pF passes $i = C\,dv/dt = 25\ \text{pF} \times 10^9\ \text{V/s} = 25$ mA for the duration of the edge. That current must have a low-impedance path that avoids the analog ground — a dedicated strap from isolated ground to the rig frame and a stitching capacitor across the barrier on the PCB — and it is why a DC–DC module with 50 pF of barrier capacitance was rejected in favour of one with under 20 pF.

> **Status of the principle: Isolation converts common-mode voltage into barrier current**
> *Derived.* With the front end floating, a ground-potential difference divides between the barrier impedance and the impedances to the rig; when the barrier impedance is much larger, almost all of it appears across the barrier and the remaining current is $C_{barrier}\,dv/dt$. It fails when the front end has a second, unintended path to system ground — a shield bonded at both ends, a USB debugger plugged in during the test.

7.8 ADC and DAC selection

> **Boundary.**
> Converter architectures, static accuracy (offset, gain, DNL, INL), quantization and ENOB are in *Electronics and Computing*, Ch. 10 §10.10 and the converter interface (reference, acquisition time, ratiometric measurement) in Ch. 26 §26.8. Here four real part classes are compared against R1–R8.

The ADC

Table 7.7 compares four ways to digitize eight bridges.

Table 7.7 — Converter options for 8 bridge channels (datasheet-class values; indicative 2026 prices, 100 pcs).

Option	Example class	Simultaneous (R5)	Resolution at input	Common mode	Cost per 8 ch	Verdict
(a) In-amp per channel + 8-ch simultaneous 18-bit SAR	INA828 + AD7606C-18	Yes	≪ 0.1 με	Handled by in-amp, 120 dB min	about $66	**Chosen**
(b) Multiplexed 24-bit ΔΣ with PGA and excitation	AD7124-8	No	0.05 με at 50 Sa/s	Direct bridge, ratiometric	about $12	Fails R5; throughput far below 8 × 2 kSa/s
(c) 8-ch simultaneous 24-bit ΔΣ with PGA	ADS131M08	Yes	0.1–0.3 με at gain 128	Limited range; CMRR about 100 dB	about $6	Revision B candidate
(d) Industrial bridge modules + generic DAQ	DIN-rail, ±10 V out	Depends on DAQ	0.1 % class	Module	about $1200	Cost; second analog stage

Option (a) wins because its common-mode rejection is a guaranteed minimum at the operating gain. Option (c) would remove eight in-amps and eight filters, and is the obvious cost-down once Revision A has measured how little common mode remains behind the barrier.

The ADC's 2.5 V reference (ADR4525 class, 2 ppm/K) also generates the excitation, so its drift cancels in the ratio (Section 7.4).

The DAC

Worked example — How many bits for the controller output? R8 maps ±3000 με to ±10 V, so 1 με is 3.33 mV. A microcontroller's 12-bit DAC behind a ±10 V scaling amplifier has steps of 20 V/4096 = 4.88 mV, 1.46 με: the controller would see a staircase coarser than the measurement's resolution, and its integrator would hunt between steps. A 16-bit bipolar DAC (AD5761R class, internal reference, ±10 V output, settling about 8 μs) has steps of 0.305 mV, 0.09 με, with INL of a few LSB. A PWM output with an RC filter cannot meet R8 at all: a ripple below one 16-bit step requires a filter time constant of several milliseconds. The 16-bit DAC at $11 is chosen, on the non-isolated side, because the servo controller's input is differential and referenced to the controller ground.

7.9 Sampling

> **Boundary.**
> The sampling theorem, aliasing, jitter-limited SNR and the sample-and-hold are in *Circuits, Signals and Communications*, Ch. 19 §19.2–19.7; oversampling and its resolution gain in Book 3 Ch. 21 §21.2.

Choosing the rate

The signal of interest is the strain waveform of a fatigue cycle at up to 30 Hz with harmonics to 200 Hz. The logged rate of 2 kSa/s gives ten samples per period of the 200 Hz component and a margin between 200 Hz and the 1 kHz Nyquist frequency that the FIR uses as its transition band. The ADC rate of 32 kSa/s is chosen for the analog filter's sake: it puts the first frequency that folds into the passband at 31.8 kHz, sixteen times above the 2 kHz analog corner. At 4 kSa/s instead, the same filter would give $10\log_{10}(1+(3.8/2)^4) = 11.5$ dB at the folding frequency of 3.8 kHz — and the drive's 4 kHz switching component would alias to 0 Hz, straight into the passband.

> **Status of the principle: Oversample, filter gently, decimate digitally**
> *Derived.* Raising the converter rate moves the folding frequency $f_s - f_p$ away from the passband, so a low-order analog filter suffices and the sharp filter becomes a digital one with exact, identical coefficients in every channel; the cost is converter power, data rate and processor load, and it assumes the converter's own noise does not rise faster than the oversampling ratio recovers.

What can still alias

Components near multiples of the ADC rate fold to near DC. The drive's 8th switching harmonic at 32 kHz would alias to a slowly wandering offset, indistinguishable from drift; only the analog filter's 48 dB prevents it, and no digital filter can. Verification therefore sweeps the drive's switching frequency from 2 to 8 kHz while a channel at zero load is watched for any offset change.

Simultaneity and skew

The simultaneous-sampling ADC holds all channels at one instant, so the remaining skew comes from the analog filters: 1.11 µs per 1 % of cut-off mismatch (Section 7.6). With 1 % C0G capacitors and 0.1 % resistors the worst case is about 2 %, 2.2 µs, inside R5; it matters when principal strains are computed from a rosette. A multiplexed converter at 32 kSa/s per channel would skew the first and last channels by 27 µs.

Clock jitter is irrelevant here: 1 ns rms on a 200 Hz signal limits SNR to $-20\log_{10}(2\pi \cdot 200 \cdot 10^{-9}) = 118$ dB. It is worth computing again for the 1 MSa/s power channels of Chapter 8.

7.10 Noise reduction

> **Boundary.**
> Thermal noise, noise spectral density, noise bandwidth and the noise of filters are from *Circuits, Signals and Communications*, Ch. 29 §29.3–29.4; in-amp noise models from *Electronics and Computing*, Ch. 8.

Random noise: the budget nobody needed

Worked example — Input-referred noise of one channel. Referred to the in-amp input, three white-noise sources add in quadrature:

- the bridge's 350 Ω output resistance plus the 2 × 100 Ω RFI resistors, 550 Ω, give $\sqrt{4kTR} = 3.0$ nV/√Hz at 300 K;
- the in-amp contributes 7 nV/√Hz;
- the ADC, with about 90 dB SNR on its ±5 V range (112 µV rms spread over 0–16 kHz at 32 kSa/s), contributes $112\ \mu\text{V}/\sqrt{16\,000\ \text{Hz}}/501 = 1.8$ nV/√Hz.

The total is 7.8 nV/√Hz. The equivalent noise bandwidth of the 255-tap FIR, computed from its coefficients, is 432 Hz, so the logged stream carries $7.8\ \text{nV}/\sqrt{\text{Hz}} \times \sqrt{432\ \text{Hz}} = 0.16$ µV rms, or 0.065 µε rms. The in-amp's 1/f noise, 0.14 µV peak-to-peak from 0.1 to 10 Hz, adds about 0.01 µε rms. Random noise is fifteen times below R2's limit.

A lower-noise in-amp would therefore buy nothing. What fills the rig's noise budget is **interference** and **drift**, reduced by layout, wiring and grounding, not by part selection.

Interference and the rig failure explained

Interference couples by four paths, each with its own cure (Table 7.8). The diagnosis on the rig followed them in order, with the pump switched on and off and a battery-powered oscilloscope with a differential probe.

Table 7.8 — Interference paths found on the fatigue rig and the fix for each.

Path	Mechanism on the rig	Symptom before	Fix	After
Conductive (ground loop)	Shield bonded at module and at rig frame; 50 Hz frame current shared the shield and the module ground	14 με pk-pk at 50 Hz and 150 Hz	Isolated front end; shield bonded at module end only; one strap from isolated ground to frame	0.2 με rms residual
Common-mode to differential conversion	Drive edges, 1 kV/μs, through unequal input impedances	Spikes up to 60 με at each edge	RFI filter with matched CoG parts; twisted-pair leads	Not detectable at 2 kSa/s
RF rectification	In-amp inputs demodulated MHz ringing into DC	30–40 με zero shift with pump on	RFI filter (Section 7.5)	< 0.5 με shift
Magnetic (normal mode)	Untwisted lead loop of 30 cm² near the pump motor cable	2 με at 50 Hz on two channels	Twisted leads; cable rerouted 0.5 m from motor cable	Below noise

Figure 7.6 shows a channel's spectrum at zero load with the pump running, before and after.

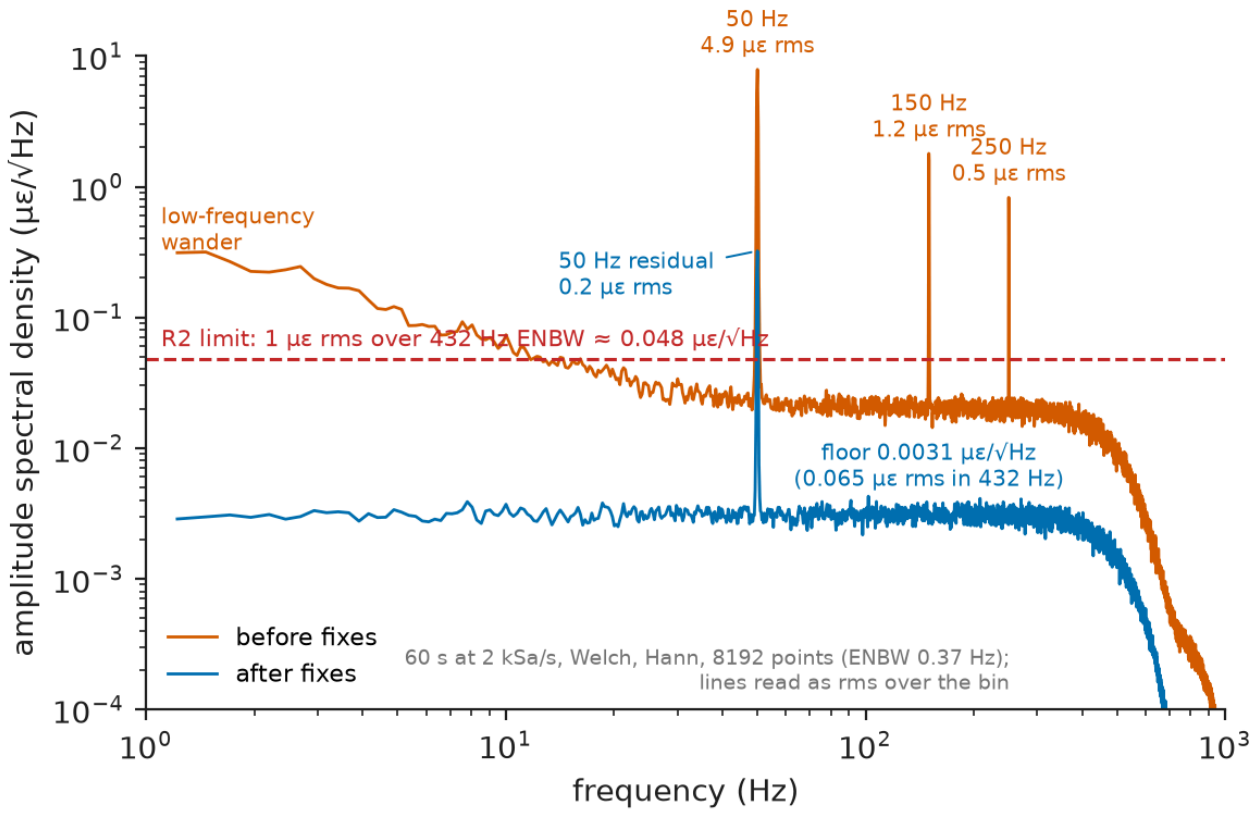


Figure 7.6 — Spectrum of one channel at zero load with the pump running, before and after the isolation and wiring fixes.

> **Status of the principle: Single-point shield grounding at low frequency**
> *Empirical.* Bonding a cable shield at one end only breaks the low-frequency ground loop through it, which is what a bridge at 0–200 Hz needs; above a few megahertz a shield open at one end becomes an antenna, so RF practice bonds both ends, and the compromise for mixed environments is one end direct and the other through a small capacitor.

Averaging and its limit

Averaging n independent readings of a static quantity reduces random noise by $\sqrt{n}$ (Section 7.2 located a mean to 0.044 με with 25 readings). It does nothing for a 50 Hz line unless the window is a whole number of mains periods, and nothing for drift or bias.

7.11 Implementation, verification and failure analysis

Strain from codes

The firmware converts each decimated code to strain in three steps: form the bridge ratio V_r, invert the quarter-bridge expression exactly and subtract the zero taken before the test, then apply the lead and shunt-calibration corrections. Because the excitation is ratiometric, the reference voltage cancels: with a ±5 V ADC range equal to twice the 2.5 V reference and a 5 V excitation equal to twice the same reference, $V_r = \text{code}/(2^{17}G)$ for an 18-bit signed code. The C17 listing is the per-channel conversion; the shunt-calibration routine computes `k_cal` as the ratio of the expected to the measured step.

```
 #include <stdint.h>

#define IA_GAIN     501.0f          /* in-amp gain, 1 + 50 kOhm / 100 Ohm */
#define CODE_FS     131072.0f       /* 2^17: 18-bit signed full scale      */

typedef struct {
    float gf;                       /* gauge factor from the gauge data sheet       */
    float rl_over_rg;               /* lead resistance / gauge resistance (3-wire)  */
    float k_cal;                    /* gain correction from shunt calibration       */
    float eps_zero;                 /* strain reading at zero load (tare)           */
} qb_channel_t;

/* Bridge ratio Vout/Vex from a ratiometric ADC code: the reference cancels. */
static inline float bridge_ratio(int32_t code)
{
    return (float)code / (CODE_FS * IA_GAIN);
}

/* Quarter bridge, exact inversion of Vr = (x/4)/(1 + x/2), x = GF * strain. */
float strain_from_code(const qb_channel_t *ch, int32_t code)
{
    const float vr  = bridge_ratio(code);
    const float eps = 4.0f * vr / (ch->gf * (1.0f - 2.0f * vr)) - ch->eps_zero;
    return eps * (1.0f + ch->rl_over_rg) * ch->k_cal;
}

/* Shunt R across the completion arm simulates eps = (Rg/Rs)/GF in tension. */
float shunt_cal_factor(const qb_channel_t *ch, float rg, float rs, float eps_measured)
{
    const float eps_expected = (rg / rs) / ch->gf;
    return ch->k_cal * eps_expected / eps_measured;
}
```

Two details matter. The tare is subtracted in the strain domain, after the inversion: subtracting the zero-load *ratio* first would leave a 3 με error at 3000 με for 500 με of imbalance. An imbalance in the completion arm also changes the bridge's sensitivity by about 0.1 % per 500 με, and the shunt calibration, made with the same imbalance present, absorbs it into `k_cal`. Second, `shunt_cal_factor` is called with the strain computed with the old `k_cal` but with the lead correction disabled, because a shunt at the module calibrates the electronics, not the leads.

Verification

Table 7.9 closes each requirement with a test; the bridge calibrator is a 350 Ω precision strain-bridge simulator with 0.02 % of setting uncertainty, itself calibrated by an accredited laboratory, and the climatic chamber runs 15, 25 and 35 °C.

Table 7.9 — Verification of the strain-gauge module (Revision A, after fixes).

Req.	Test	Limit	Result	Pass/fail
R1	Calibrator steps to ±3500 με, all channels; check ADC codes and in-amp internal nodes	No clipping; linearity ≤ 0.05 %	Max +4.37 V / −4.40 V at ADC (±5 V range); linearity 0.012 %	Pass
R2	60 s at zero load, pump and drive running, 8 channels	≤ 1 με rms	0.25 με rms worst channel (0.07 με with dummy bridge on bench)	Pass
R3	9 points from −3000 to +3000 με at 15, 25, 35 °C	0.15 % + 4 με (k = 2)	Worst error 3.1 με at +3000 με, 35 °C	Pass
R4	Calibrator sine 1–200 Hz, 1000 με	≤ 0.1 % at 30 Hz	−0.004 % at 30 Hz; −0.03 % at 200 Hz	Pass
R5	Same 30 Hz sine into 8 channels; cross-correlation	≤ 10 µs	1.4 µs worst pair	Pass
R6	10 V rms 50 Hz injected rig-to-module; 1 kV/µs pulses; drive sweep 2–8 kHz	≤ 1 με shift	0.2 με; 0.4 με; no offset change	Pass
R7	Dummy bridges, 15–35 °C	≤ 0.5 με/°C	0.18 με/°C worst channel	Pass
R8	1000 με step from calibrator to DAC output	≤ 1 ms; 16 bits	0.62 ms to 50 %; 16 bits monotonic	Pass
R9	Shunt step vs calibrator 1000 με	±0.1 %	999.6 με (−0.04 %)	Pass
R10	1.5 kV rms, 60 s, barrier; isolator CMTI from datasheet	No breakdown; ≥ 50 kV/µs	0.4 µA leakage; 100 kV/µs	Pass

The R6 drive sweep would have caught the original failure on day one; it is now in the regression set (Chapter 4).

Failure analysis

Table 7.10 — FMEA of the strain-gauge module (extract).

Failure mode	Effect	Cause	Detection	Mitigation
Ground loop through shield	50 Hz error of several με	Shield bonded at both ends during rig rework	R6 injection test; 50 Hz line in at-rest spectrum	Shield termination marked on drawing; at-rest spectrum logged before every test
Silent clipping of in-amp internal node	Reading flattens near full range	Asymmetric excitation; gain raised for a low-strain specimen	Diamond-plot check in design review; R1 step test	Symmetric excitation; firmware flags codes beyond ±4.5 V
RF rectification	DC zero shift with drive running	RFI filter omitted or wrong dielectric	Pump on/off zero comparison	RFI filter with C0G parts; test in R6
Open gauge or broken lead	Channel saturates or reads nonsense	Debonded gauge, fatigued lead at the specimen	Bridge-resistance check at start of test; out-of-range flag	Lead strain relief; automatic channel invalidation in the log
Completion-network drift	Zero drift of tens of με over temperature	Thin-film resistors substituted for foil network at purchasing	R7 chamber test on first article	Part number locked in BOM with "no substitution"
Aliased drive harmonic	Slow, wandering offset	Drive frequency multiple of ADC rate; weak analog filter	Drive sweep in R6	48 dB analog filter; ADC rate not a round multiple of common drive frequencies

Every failure in the table is invisible on a bench with a floating resistor box; the two tests that make the module trustworthy — injecting the rig's common-mode voltage and sweeping the drive frequency — come from asking what the installation does that the bench does not.

What remains open

Open question	Where it is answered
How are the eight channels timestamped, logged, stored and synchronized with the rig controller's load and displacement channels?	Chapter 8
How are thermocouple, energy, speed, power-quality and battery measurements conditioned in compact instruments?	Chapter 9
How are the strain module's production units tested automatically, with guard-banded limits?	Chapter 9 §9.7
How are gauges, thermocouples and other sensors chosen and calibrated in the first place?	Chapter 6
How is a current and voltage front end designed for a mains energy meter, with calibration and safety?	Chapter 35
How are the same conditioning principles applied to biopotentials with patient-safety isolation?	Chapter 28

Next: Chapter 8 takes the conditioned, digitized channels and builds the acquisition system around them — architecture, synchronization across nodes, real-time transfer, logging, storage and the checks that prove a recorded data set is valid.

Chapter 8 — Data Acquisition Systems

Part II · Instrumentation and Measurement Systems

An engineering team maps the efficiency of a 150 kW traction drive on a dynamometer: an 800 V silicon-carbide inverter feeding a permanent-magnet motor whose shaft drives an absorbing machine through a torque flange. Electrical power is measured by one acquisition node next to the inverter, mechanical power by another at the shaft, temperatures by a third, and each node timestamps its data from its own crystal, corrected by NTP. Steady-state points look plausible. During torque ramps the computed efficiency swings by more than a percent from ramp to ramp and occasionally exceeds 100 %, and the vibration order maps at high speed are smeared. Every sensor is in calibration and every converter meets its datasheet. The fault is in the acquisition system: the nodes disagree about time by milliseconds, with steps whenever NTP corrects a clock, and the vibration converter's decimation filter delays its samples by 664 µs that nobody subtracted. This chapter builds the acquisition system for that test cell so that every sample has a known time, a known origin and a known validity.

> **How is a multi-node acquisition system architected so that thousands of channels at very different rates arrive complete, aligned in time and traceable to their calibration?**
> **How are real-time transfer, logging, storage and validation designed so that a lost sample, a clock fault or a bad sensor is detected rather than analysed?**

Concept	Engineering destination
DAQ architectures	Choosing centralized, distributed or embedded acquisition for test cells, vehicles, plants and labs
Sensor interfaces	IEPE, thermocouple, RTD, current-transducer, frequency, encoder and bus inputs
ADC performance	ENOB, SINAD and SFDR measured on the assembled channel, not quoted from the datasheet
Sampling rates	Per-channel rates from signal bandwidth, and the data rates they imply
Data logging	Triggered windows, ring buffers, metadata, continuous low-rate records
Synchronization	PTP, shared clocks, GNSS, filter-delay compensation
Real-time acquisition	DMA ping-pong buffers, deadlines, overrun detection
Serial and network interfaces	SPI, CAN FD, USB, Ethernet, EtherCAT throughput and latency
Data storage	File formats, storage budgets, integrity under power loss
Measurement validation	Automated checks, injected references, energy balances

Roadmap. Requirements and architecture (8.1) → what each sensor needs at the node (8.2) → how good the digitized channel really is (8.3) → rates and data volume (8.4) → logging (8.5) → time alignment across nodes (8.6) → moving samples in real time (8.7) → the links (8.8) → storage (8.9) → proving the data valid (8.10) → verification and failure analysis (8.11).

8.1 DAQ architectures

> **Boundary.**
> Conditioning, uncertainty and converter selection for individual channels are in Chapter 7; embedded interfaces (SPI, CAN, USB, Ethernet) at the protocol level in *Electronics and Computing*, Ch. 26; scheduling and latency theory in Ch. 27. This chapter is the system around the channels.

Four architectures

Table 8.1 — DAQ architectures (typical).

Architecture	Example	Strengths	Weaknesses	Fits
Centralized chassis	PXI or cDAQ-class chassis with plug-in modules, shared backplane clock	Sample-clock sharing, one software stack	Long analog cables from sensor to chassis; one chassis per location	Bench and small test cells
Distributed networked nodes	Ethernet nodes near sensor groups, PTP time	Short analog runs, galvanic separation by location, scalable	Time sync must be engineered; more firmware	Large test cells, vehicles, plants
Fieldbus I/O	EtherCAT terminals, distributed clocks	Deterministic cycle, cheap channels	Cycle-based, limited to kSa/s per channel	Process and machine data, slow channels
Embedded logger	MCU board with local ADC and SD card	Low cost and power, standalone	Limited rate and storage; validation is on you	Field monitoring, IoT (Chapter 11)

The reference system: a three-node dynamometer DAQ

The dynamometer cell's sensors sit in three places with three very different signal types, so the architecture is **distributed** (Figure 8.1). **Node P** (power) stands beside the inverter: eight high-voltage channels — three phase currents, three phase voltages, DC-link voltage and current — each with its own isolated 16-bit 2 MSa/s SAR converter run at 1 MSa/s, all triggered by one convert signal from a SoC FPGA. **Node M** (mechanical) at the shaft: six IEPE accelerometers and two microphones on an 8-channel simultaneous 24-bit delta-sigma converter at 51.2 kSa/s, plus torque-flange frequency output, a 1024-line encoder and the inverter's CAN FD telemetry, on a Cortex-M7 MCU. **Node T** (thermal) on the coolant manifold: 32 type-K thermocouples and 8 Pt100s on five multiplexed 24-bit delta-sigma converters at 10 Sa/s. All three connect to a PTP-aware Gigabit switch, a grandmaster clock and a host server with 2 TB of NVMe storage. Safety interlocks (over-speed, over-temperature) are hardwired through the cell's safety PLC (Chapter 21); the DAQ observes them but is not part of the safety function.

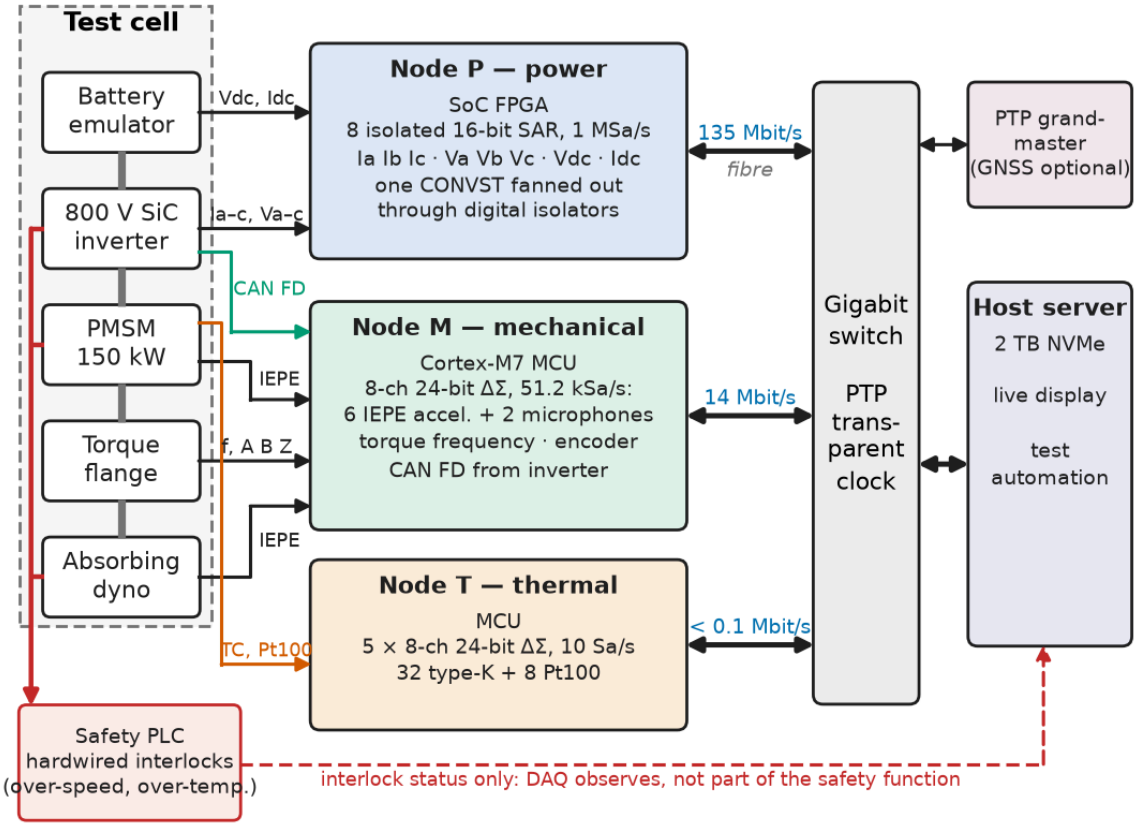


Figure 8.1 — Distributed acquisition system of the 150 kW electric-drive dynamometer cell.

Table 8.2 — Requirements for the dynamometer DAQ.

ID	Requirement	Target	Verification
R1	Channels and rates	P: 8 × 1 MSa/s; M: 8 × 51.2 kSa/s + torque, speed 1 kSa/s + CAN; T: 40 × 10 Sa/s	Configuration audit, rate counters
R2	Node P channel quality	ENOB ≥ 14.0 bits at 1 kHz, −0.5 dBFS, front end included; intra-node skew ≤ 50 ns	FFT test with coherent sampling; common pulse
R3	Inter-node alignment	P–M ≤ 1 µs; P–T ≤ 1 ms, for 8 h	Shared reference sine, sine-fit phase
R4	No silent loss	0 undetected gaps in 8 h; every gap flagged in the file	Sequence counters; forced overload test
R5	Live view	≤ 250 ms from sample to display	Timestamped test pulse
R6	Storage	One 8 h test on local NVMe with ≥ 20 % free; self-describing files with checksum	Storage audit; reader test
R7	Power-loss integrity	≤ 1 s of data lost; all closed files readable	Pull-the-plug test
R8	Validation	Automated range, stuck, gap, clock and energy-balance checks on every run	Seeded faults
R9	Network load	≤ 50 % on any link	Switch counters
R10	HV safety	Node P inputs reinforced-isolated for 1000 V DC working (IEC 61010-1, -2-030)	Type test, hipot

8.2 Sensor interfaces

Each sensor family arrives with its own electrical interface, and the node must supply what the sensor needs and detect when it fails (Table 8.3); how the sensors themselves were chosen — fluxgates for power, encoders and torque flanges for the shaft — is in Chapter 6.

Table 8.3 — Sensor interfaces of the dynamometer DAQ.

Signal	Sensor class	Interface	Conditioning at the node	Fault detection
Phase and DC currents	Closed-loop fluxgate transducer, ±700 A, DC–100 kHz class	Current output, 1:1750	12.5 Ω 0.01 % burden, ±5 V	Transducer status line; zero check at standstill
Phase and DC voltages	Compensated 1000:1 HV divider	±1 V differential	Isolated channel per input, 300 kHz anti-alias	Divider open: reads 0 with inverter running
Vibration	IEPE accelerometer, 100 mV/g, ±50 g	2-wire, constant current, bias about 12 V	4 mA source, 24 V compliance, AC coupling 0.3 Hz	Bias outside 2–20 V: short or open
Sound	IEPE microphone	As above	As above	As above
Torque	Torque flange, 10 kHz ± 5 kHz frequency output	RS-422	Timer reciprocal counting	Frequency outside 4–16 kHz
Speed and angle	1024-line encoder	RS-422 A, B, Z	Quadrature timer, M/T speed (Section 9.3)	Missing Z pulse count per revolution
Temperatures	Type K thermocouple; Pt100 4-wire	µV; resistance	24-bit ΔΣ, CJC on isothermal block (Section 9.1); 1 mA excitation	Burnout current; resistance out of range
Inverter telemetry	Inverter controller	CAN FD, 500 kbit/s / 2 Mbit/s	Transceiver, hardware timestamp on receipt	Message-rate watchdog

Worked example — How long can an IEPE cable be? An IEPE accelerometer's internal amplifier is powered by the constant current I_{cc} that also carries the signal; it can only charge the cable capacitance as fast as the spare current allows. The usual limit is

$$f_{\max} = \frac{I_{cc} - 1\ \text{mA}}{2\pi\, C_{cable}\, V_{pk}}$$

where 1 mA is reserved for the sensor's own amplifier. With $I_{cc} = 4$ mA, a 30 m cable at 100 pF/m (3 nF) and a 5 V peak signal (50 g at 100 mV/g), $f_{\max} = 3\ \text{mA}/(2\pi \cdot 3\ \text{nF} \cdot 5\ \text{V}) = 31.8$ kHz, above the 20 kHz analysis band. A 100 m run through the cell's cable trays (10 nF) drops it to 9.5 kHz: the waveform would be slew-limited, distorting peaks without any visible fault. The cell layout therefore puts Node M within 30 m of every accelerometer, an architecture decision driven by one line of arithmetic.

> **Status of the principle: IEPE slew limit**
> *Approximate.* Treats the cable as a lumped capacitance charged by the spare constant current; it ignores cable resistance and the sensor's output impedance, and it is conservative at low amplitude because the limit scales with V_{pk}.

The phase-current interface shows a different kind of choice. A fluxgate transducer's current output is immune to the inverter's electric fields over the 5 m run to Node P, and the burden resistor that converts it to voltage sits at the node, so the accuracy-setting part (0.01 %, 5 ppm/K) is inside the instrument's

temperature-controlled enclosure. Voltage-output transducers are cheaper but pick up common-mode noise from the inverter's 50 V/ns edges along the cable.

8.3 ADC performance

> **Boundary.**
> Quantization noise, SQNR $= 6.02N + 1.76$ dB and ENOB are defined in *Circuits, Signals and Communications*, Ch. 19 §19.8; leakage and coherent sampling in Book 3 Ch. 22 §22.5; converter static errors in *Electronics and Computing*, Ch. 10 §10.10.

What is measured, on what

A datasheet characterizes the converter with an ideal source; the system's ENOB includes the divider, the isolated driver, the reference and the board. R2 therefore applies to the **assembled channel**, measured with a low-distortion source (a calibrator with < −110 dBc harmonics, followed by a passive band-pass filter) at the channel's input terminals. The dynamic figures are SNR (noise only), THD (harmonics 2 to 6), SINAD (both), ENOB $=$ (SINAD $-$ 1.76)/6.02, and SFDR (signal to largest spur). The test follows IEEE Std 1241 with **coherent sampling**: the tone is set to a prime number of cycles in the record, $f_0 = J f_s / N$, so all its energy falls in one bin with no window.

The listing processes a record exactly as the production test does; here the record is synthesized with the channel's measured noise (180 µV rms) and harmonic levels so that the analysis can be checked against known inputs.

```
 import numpy as np

FS, N, NBITS, VFS = 1_000_000, 65_536, 16, 10.0      # +/-10 V, 16-bit, 1 MSa/s
CYCLES = 67                                           # prime: coherent, no window
f0 = CYCLES*FS/N
rng = np.random.default_rng(1)

t = np.arange(N)/FS
a = VFS*10**(-0.5/20)                                 # -0.5 dBFS test tone
v = a*np.sin(2*np.pi*f0*t)
v += a*10**(-98/20)*np.sin(2*np.pi*2*f0*t)            # HD2 model, -98 dBc
v += a*10**(-94/20)*np.sin(2*np.pi*3*f0*t)            # HD3 model, -94 dBc
v += rng.normal(0, 180e-6, N)                         # front-end + ADC noise
lsb = 2*VFS/2**NBITS
code = np.clip(np.round(v/lsb), -2**15, 2**15 - 1)    # ideal 16-bit quantizer

P = np.abs(np.fft.rfft(code*lsb))**2
P[0] = 0
sig = P[CYCLES]
harm = sum(P[(h*CYCLES) % N] for h in range(2, 7))    # HD2..HD6 (no folding here)
noise = P.sum() - sig - harm
spur = np.delete(P, [0, CYCLES]).max()

snr, thd = 10*np.log10(sig/noise), 10*np.log10(harm/sig)
sinad = 10*np.log10(sig/(noise + harm))
print(f"f0 = {f0:.2f} Hz  SNR {snr:.1f} dB  THD {thd:.1f} dBc  SINAD {sinad:.1f} dB")
print(f"ENOB {(sinad - 1.76)/6.02:.2f} bits (re full scale "
      f"{(sinad + 0.5 - 1.76)/6.02:.2f})  SFDR {10*np.log10(sig/spur):.1f} dBc")
```

```
 f0 = 1022.34 Hz  SNR 90.5 dB  THD -92.5 dBc  SINAD 88.4 dB
ENOB 14.39 bits (re full scale 14.47)  SFDR 94.0 dBc
```

The channel meets R2 (14.0 bits) with 0.39 bits of margin. The listing makes two practical points. The noise term, 180 µV rms, is twice the 16-bit quantization noise ($305\ \mu\text{V}/\sqrt{12} = 88\ \mu\text{V}$): the front end, not the converter, sets the ENOB, which is why a 16-bit converter was chosen rather than an 18-bit one. And ENOB must be quoted with its test level: referred to full scale the same channel is 14.47 bits. Figure 8.2 shows the spectrum.

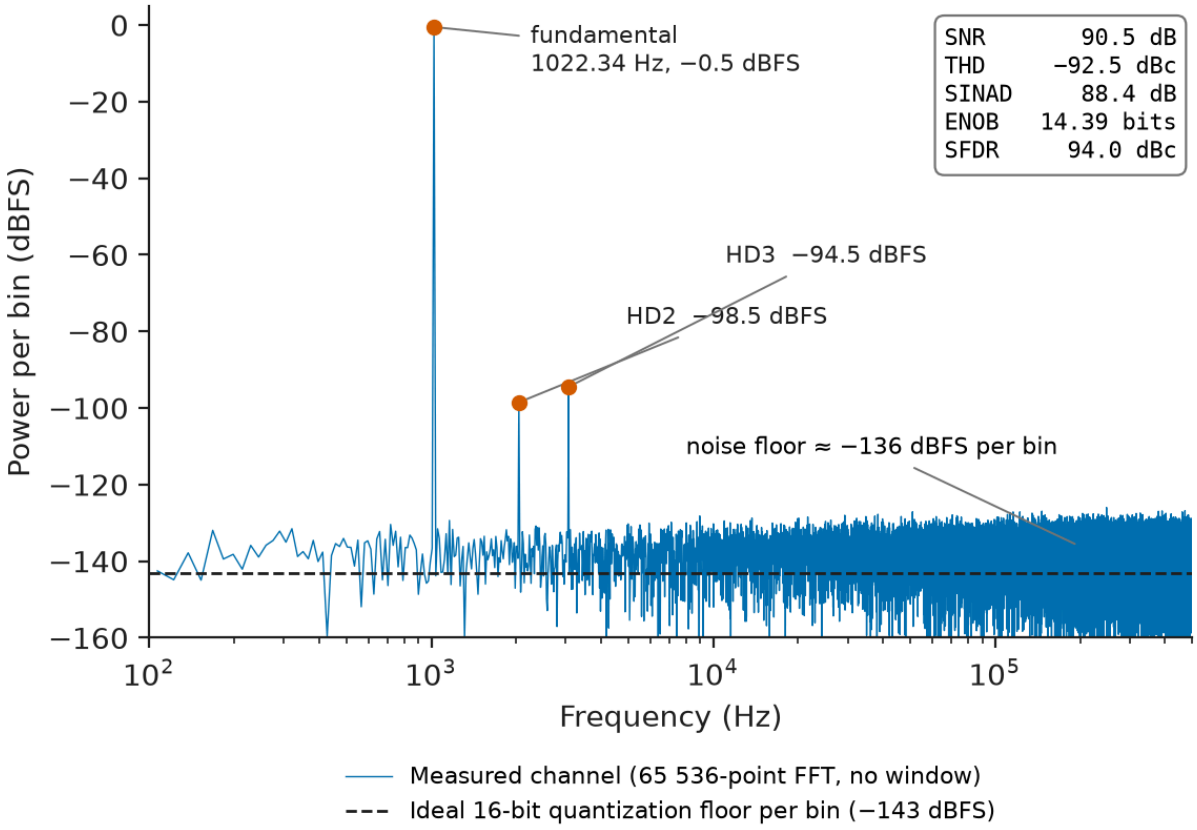


Figure 8.2 — Coherently sampled FFT test of one Node P channel: 1022.34 Hz at -0.5 dBFS, 65 536 points at 1 MSa/s.

> **Status of the principle: ENOB from SINAD**
> *Derived.* Inverts the ideal-quantizer SNR relation, so it summarizes noise and distortion as an equivalent ideal converter; it hides where the error comes from (THD and SNR must be read separately) and it depends on input frequency and level, which must be stated with it.

Other performance figures matter for multichannel systems and are measured the same way: **crosstalk** (a full-scale tone on one channel, the FFT of its neighbours: −110 dB measured between Node P channels thanks to separate isolated islands), **DC offset and gain drift** over the enclosure's temperature range, and for multiplexed converters such as Node T's, **settling after a channel switch**, measured by alternating a full-scale and a zero-scale input on adjacent channels.

8.4 Sampling rates

> **Boundary.**
> The sampling theorem, anti-aliasing filter order and jitter-limited SNR are in *Circuits, Signals and Communications*, Ch. 19 §19.2–19.9.

Each rate starts from the highest frequency the analysis needs, adds a filter transition band, and is then rounded to what the converter offers (Table 8.4).

Table 8.4 — Sampling-rate decisions for the dynamometer DAQ.

Group	Analysis band	Rule	Rate	Margin
Phase currents and voltages	To 100 kHz: 1 kHz fundamental, 20 kHz switching and sidebands to 5th order	Analog 3rd-order Bessel at 300 kHz; folding starts at $f_s - f_p = 900$ kHz	1 MSa/s	Bessel −21 dB at 900 kHz plus transducer roll-off; filter chosen for flat group delay
Vibration, sound	To 20 kHz (gear mesh, bearing orders)	$f_s = 2.56\, f_{\max}$, ΔΣ brick-wall filter	51.2 kSa/s	Converter's digital filter > 100 dB above 25.6 kHz
Torque	To 1 kHz	One reciprocal count per carrier period, averaged	1 kSa/s	5–15 periods per sample
Speed	To 1 kHz	M/T method (Section 9.3)	1 kSa/s	±0.05 % at ≥ 1200 rpm
Temperatures	< 1 Hz (thermal time constants ≥ 5 s)	20 ms integration rejects 50 Hz	10 Sa/s	50×
CAN telemetry	Event-driven, about 500 frames/s	Timestamp on receipt	—	—

> **Status of the principle: Sample rate of 2.56 times the analysis bandwidth**
> *Empirical.* Vibration-analyser practice: a delta-sigma converter's decimation filter is flat to about 0.45 f_s and fully attenuating by 0.55 f_s, so $f_{\max} = f_s/2.56$ leaves its transition band outside the analysis band; it assumes that filter shape and is not a sampling-theorem limit.

The jitter check is worth doing for Node P: at 100 kHz, 50 ps rms of convert-clock jitter limits SNR to $-20\log_{10}(2\pi \cdot 10^5 \cdot 50\ \text{ps}) = 90.1$ dB, comparable to the channel's own 90.5 dB, so the FPGA's convert signal is generated from a low-jitter clock and re-timed in each isolated island, not driven straight from fabric logic through the isolators.

The rates fix the data volume. Node P produces $8 \times 10^6 \times 2$ bytes = 16.0 MB/s; Node M $8 \times 51\,200 \times 4 =$ 1.64 MB/s; Node T 1.6 kB/s; CAN about 8 kB/s. The total, 17.6 MB/s, is 63.5 GB per hour of continuous recording, the number that drives Sections 8.5, 8.8 and 8.9.

8.5 Data logging

Windows, rings and continuous records

Continuous recording of every channel at full rate is rarely needed and expensive. The cell's test automation steps the drive through 400 operating points in an 8 h efficiency map; at each it waits for thermal and electrical settling and then asks for a measurement. The logging design therefore has three layers:

- **Triggered full-rate windows**: Node P records 2 s at 1 MSa/s per test point, with 0.5 s of **pre-trigger** taken from a ring buffer that runs continuously, so events just before a trigger (a protection trip, a torque step) are captured.

- **Continuous high-rate where it is cheap**: Node M records continuously, because vibration events are not scheduled.
- **Continuous aggregates**: Node P computes per-electrical-cycle RMS values, active and reactive power per phase and DC power on the FPGA and logs them at up to 1000 records per second; Node T logs everything continuously.

The logger in each node is a state machine (Figure 8.3) whose states and transitions are logged themselves, so that a file's history — when it was armed, triggered, flushed, and whether anything went wrong — is part of the data.

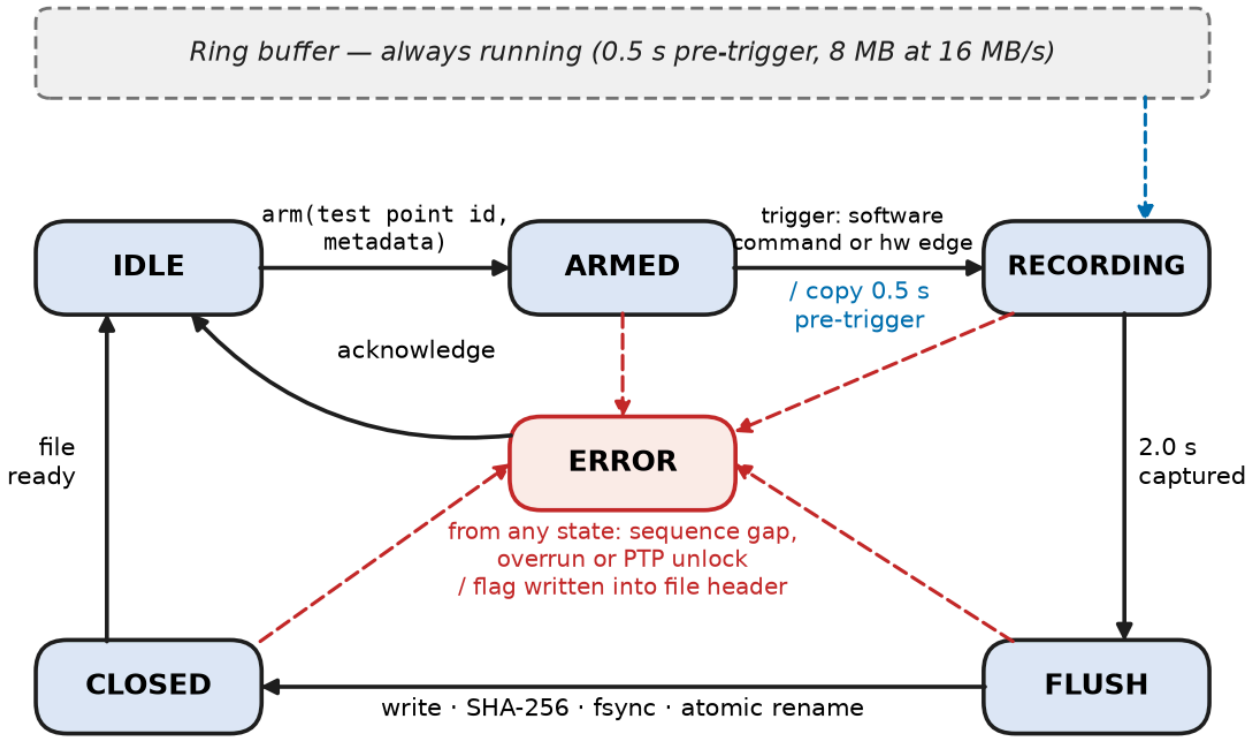


Figure 8.3 — State machine of the windowed logger on Node P.

Metadata is part of the measurement

A number without its context cannot be validated or reused. Every file carries, in its header:

- channel names, units, sensor type and serial number, calibration coefficients and calibration date;
- the conditioning configuration: range, filter settings, decimation and the resulting **group delay** per channel (Section 8.6);
- the time base: PTP grandmaster identity, lock state at start and end, maximum offset during the record;
- the test context: test-point ID, commanded speed and torque, firmware versions, operator;
- integrity data: block sequence range, any gaps, overrun counters, clipping counters per channel.

The calibration coefficients are recorded, not just applied, so that a calibration error found later can be corrected in archived data without re-running a test.

8.6 Synchronization

Why time is the hardest channel

Efficiency is $\eta = T\omega / P_{elec}$, with $T\omega$ from Node M and P_{elec} from Node P. If the two nodes disagree about time, every transient converts directly into an error.

Worked example — What a timebase error costs. Two nodes with free-running ±50 ppm crystals can drift apart by up to 100 ppm, $10^{-4} \times 3600\ \text{s} = 0.36\ \text{s}$ per hour. NTP holds a LAN within a few milliseconds but corrects in steps. Suppose a residual 10 ms misalignment during a torque ramp of 200 N·m/s at 6000 rpm ($\omega = 628.3$ rad/s): the torque value paired with a given electrical sample is wrong by $200 \times 0.01 = 2$ N·m, and mechanical power by $2 \times 628.3 = 1.26$ kW — 1.3 % of a 100 kW operating point, the scatter seen in the opening paragraph. To keep the same error below 0.01 %, alignment must be better than about 80 µs; for vibration-to-current phase analysis at a 2.3 kHz gear-mesh frequency, 1 µs is 0.8°. R3's 1 µs follows.

Methods

Table 8.5 — Time-synchronization methods (typical achievable alignment).

Method	Alignment	Infrastructure	Typical use
Shared sample clock and trigger lines	< 10 ns	Coax to each module; short distances	Chassis DAQ, oscilloscopes
IEEE 1588 PTP, hardware timestamping, PTP-aware switches	20–200 ns	Grandmaster; transparent or boundary-clock switches	Distributed test cells; substations (IEC 61850-9-3 profile)
White Rabbit (PTP extension)	< 1 ns	Dedicated switches, fibre	Accelerators, large physics experiments
GNSS 1PPS at each node	20–100 ns	Antenna per node; sky view	Outdoor and wide-area systems
IRIG-B	1–10 µs	Coax distribution	Legacy test ranges
NTP	0.1–10 ms on a LAN	None	Logs, slow channels

The cell uses **PTP** (IEEE 1588-2019, default profile) with hardware timestamping in each node's Ethernet MAC and a transparent-clock switch that corrects for its own queueing delay. PTP gives each node a clock, but a clock is not a sample: the node's **sample clock** must be derived from it. Node P's FPGA generates its 1 MSa/s convert pulses from a counter clocked by a PLL locked to the PTP clock, so sample 0 of every second falls on the PTP second. Node M feeds the delta-sigma converter's master clock from a VCXO disciplined to PTP and resynchronizes the converter's filter on a PTP second edge. Node T, whose requirement is 1 ms, simply timestamps each conversion with the PTP clock.

> **Status of the principle: Time transfer by two-way message exchange**
> *Derived.* PTP estimates the offset between two clocks from the timestamps of a message exchange in both directions, assuming the path delay is the same each way; asymmetric delays (a switch that is not PTP-aware, different cable lengths for transmit and receive pairs) appear directly as half the asymmetry in offset error.

Filter delay is a timing error too

Worked example — The delta-sigma converter's delay. Node M's converter uses a wideband linear-phase decimation filter with a group delay of about 34 output periods (datasheet-class value; the exact figure depends on the filter and rate selected and is read from the datasheet), $34/51\,200 = 664$ µs. Node P's SAR channels add only their analog filters' 1–2 µs. Left uncorrected, a 100 Hz shaft-order component in the vibration data appears $360 \times 100 \times 664\ \mu\text{s} = 23.9^\circ$ late relative to the phase currents, and at the 2.3 kHz gear mesh the error is 550°, more than a full cycle. The correction is exact and costs nothing: every Node M sample is timestamped at the time its input was *sampled*, the converter's output time minus the filter delay, and the delay used is written in the metadata. A firmware change of filter mode must change that number in the same commit.

The time budget

Table 8.6 — Time-alignment budget, Node P to Node M (R3: 1 µs).

Contribution	Value (bound)	Basis
PTP offset between the two nodes (each a slave to the same grandmaster)	100 ns	Hardware timestamps, one transparent-clock hop, measured
Timestamp granularity	8 ns	125 MHz timestamp clock
Sample-clock PLL residual	50 ns	PLL/VCXO tracking jitter
Convert-edge to timestamp latency	20 ns	Calibrated, residual after correction
Analog front-end delay mismatch after calibration	200 ns	Transducer and filter delays measured and subtracted
RSS / linear sum	**230 ns / 378 ns**	
Requirement / margin (RSS)	**1 µs / 77 %**	

PTP must also fail safely. If the grandmaster is lost, each node holds over on its own oscillator; a 0.5 ppm VCXO drifts 1 µs in 2 s, so R3 is violated within seconds. The nodes therefore write the PTP state and maximum offset into every block header, and the validation of Section 8.10 marks data recorded outside lock rather than letting an analyst discover it. Figure 8.4 compares the alignment achieved by the three timing schemes the cell has used.

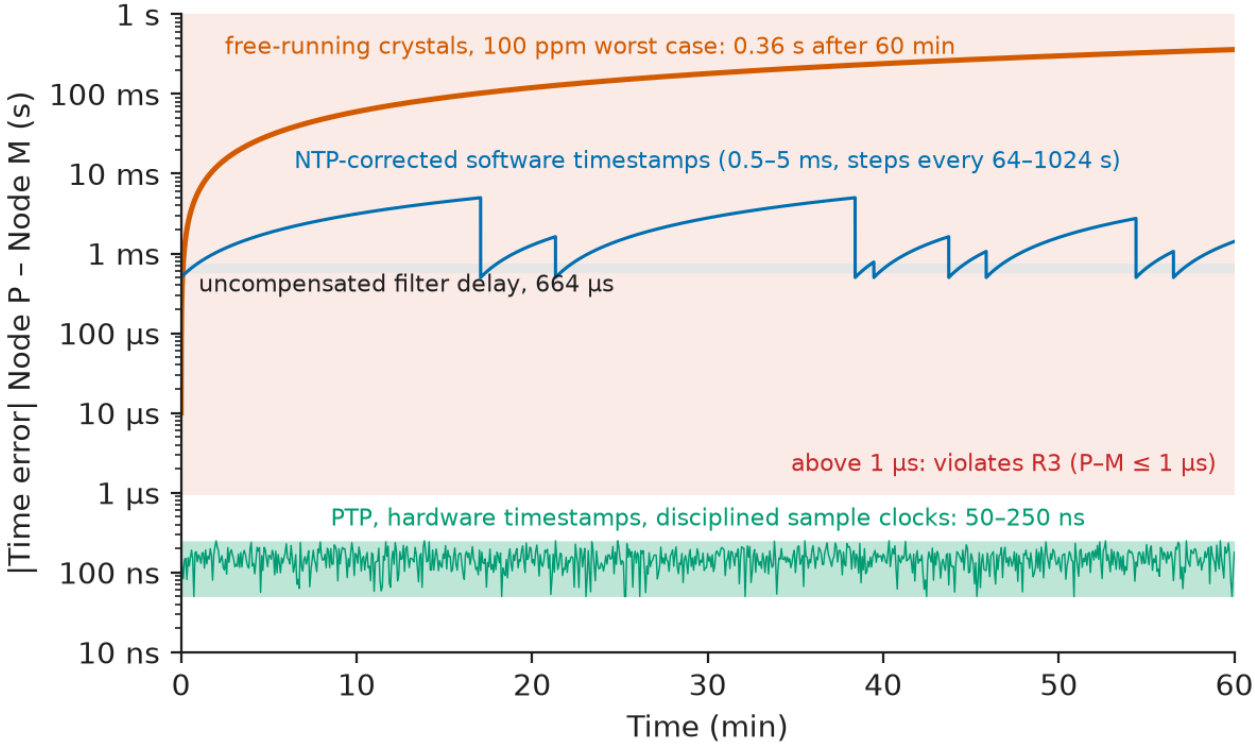


Figure 8.4 — Time offset between Node P and Node M over one hour with three timing schemes.

8.7 Real-time acquisition

> **Boundary.**
> Interrupt latency, priorities, response-time analysis and latency and jitter measurement are in *Electronics and Computing*, Ch. 27 §27.2–27.10; timer-triggered ADC with DMA at the architecture level in Ch. 25 §25.2.

The deadline is the buffer

Acquisition is real-time in a specific sense: the converter will not wait. Samples arrive at a fixed rate and are written by DMA into a circular buffer split into two halves (**ping-pong**); when one half fills, the processor must finish with it before the DMA wraps around and starts overwriting it. The deadline for processing one half is therefore one half-buffer period, and missing it loses data silently unless the firmware detects it. Figure 8.5 shows the timing on Node M.

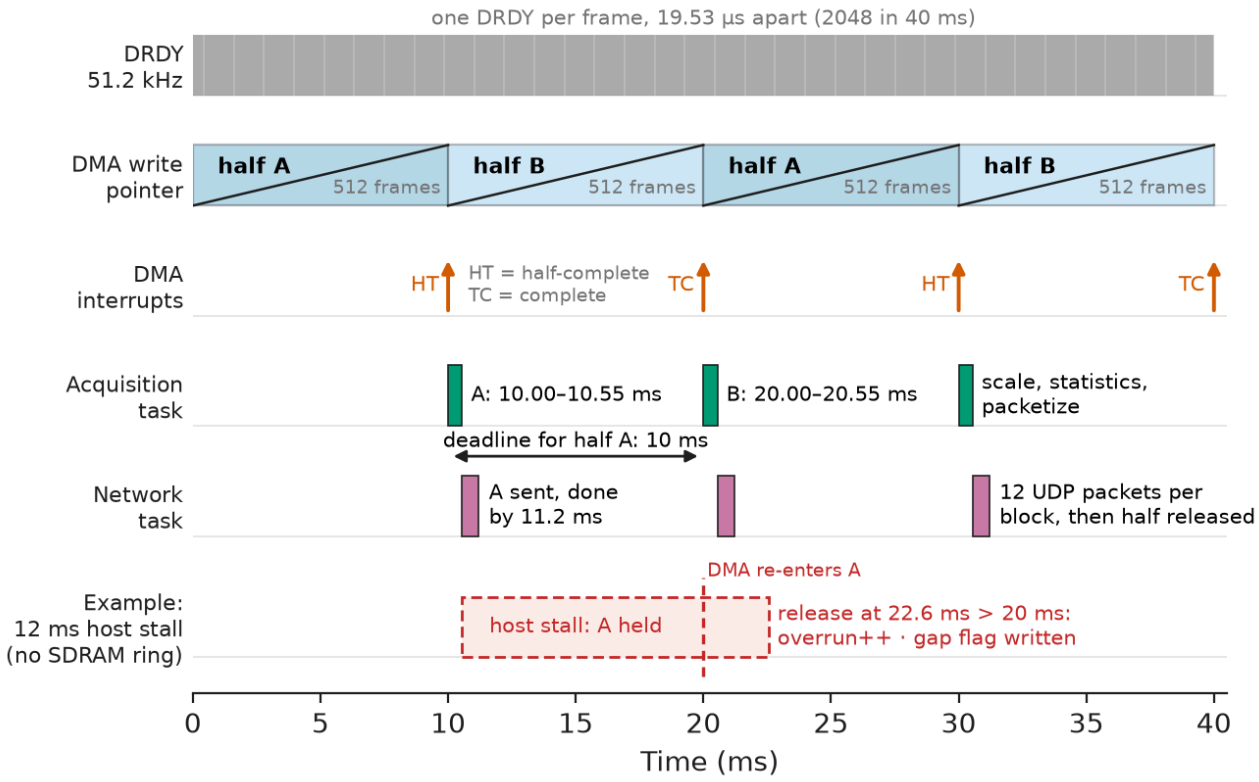


Figure 8.5 — Ping-pong DMA acquisition on Node M: 512-frame halves at 51.2 kSa/s.

The C17 listing is the DMA side of that design for an STM32H7-class MCU (HAL callback names as in STM32Cube; illustrative). The converter's 24-bit samples arrive in 32-bit words, eight channels interleaved; each half holds 512 frames, 10 ms. The completion interrupt does almost nothing: it marks the half as owned by the consumer, invalidates the data cache over it (the Cortex-M7 cache would otherwise return stale data), attaches the block's sequence number and the PTP time of its first sample, and posts it to a queue. If the consumer has not released the half from the previous lap, the overrun is counted, not hidden.

```
 #include <stdint.h>
#include <stdbool.h>
#include "stm32h7xx_hal.h"               /* vendor HAL; illustrative names */

#define NCH        8u
#define FRAMES     512u                  /* 10 ms at 51.2 kSa/s */
#define HALF_WORDS (NCH * FRAMES)

typedef struct {
    uint32_t seq;                        /* block sequence number             */
    uint64_t t0_ns;                      /* PTP time at which frame 0 sampled */
    const int32_t *data;                 /* HALF_WORDS samples, interleaved   */
} block_t;

static int32_t dma_buf[2u * HALF_WORDS] __attribute__((aligned(32), section(".dma_ram")));
static volatile bool half_busy[2];
static volatile uint32_t blocks_done, overruns;

extern SPI_HandleTypeDef hspi2;
extern uint64_t ptp_sample_time_ns(uint64_t frame_index);  /* DRDY capture minus filter delay */
extern bool queue_post_from_isr(const block_t *b);         /* RTOS queue, non-blocking        */

static void on_half(unsigned h)
{
    if (half_busy[h]) {
        overruns++;                       /* consumer still owned it: data overwritten */
    }
    half_busy[h] = true;
    SCB_InvalidateDCache_by_Addr(&dma_buf[h * HALF_WORDS], (int32_t)(HALF_WORDS *
sizeof(int32_t)));
    const block_t b = { blocks_done, ptp_sample_time_ns((uint64_t)blocks_done * FRAMES),
                        &dma_buf[h * HALF_WORDS] };
    if (!queue_post_from_isr(&b)) {
        overruns++;                       /* queue full: block dropped, gap recorded */
        half_busy[h] = false;
    }
    blocks_done++;
}

void HAL_SPI_RxHalfCpltCallback(SPI_HandleTypeDef *hspi) { if (hspi == &hspi2) on_half(0u); }
void HAL_SPI_RxCpltCallback(SPI_HandleTypeDef *hspi)     { if (hspi == &hspi2) on_half(1u); }

void block_release(const block_t *b)     /* called by the network task when sent */
{
    half_busy[(b->data == dma_buf) ? 0u : 1u] = false;
}
```

The sequence number travels with the block to the host, which detects any gap by a jump in `seq`; the overrun counter travels in the block header. Together they make R4 testable: a forced overload (the host deliberately stalls) must produce a flagged gap, never a silently shortened record.

Table 8.7 — Node M processing budget per 10 ms block (480 MHz Cortex-M7, measured with a cycle counter).

Task	Time	Basis
DMA interrupt, cache invalidate, queue post	0.009 ms	4096 words, 32-byte lines
Scale 4096 samples to engineering units	0.085 ms	about 10 cycles per sample
Block statistics (min, max, RMS, clip count)	0.040 ms	
Packetize and send 16 kB over UDP	0.350 ms	12 packets, lwIP and Ethernet DMA
CAN FD receive and timestamp (about 5 frames)	0.050 ms	
Torque and speed timer reads	0.015 ms	
Total / deadline / margin	**0.55 ms / 10 ms / 94 %**	

The large margin is intentional. The deadline that matters is not the processor's but the **network's**: the half must be released only after it has been sent or copied, so a host or switch stall of more than 10 ms would overrun. Node M therefore copies each block into a 4 MB ring in external SDRAM (about 2.4 s of data) and releases the DMA half immediately; the network task drains the ring. Node P, at 16 MB/s, uses 1 GB of DDR as the same kind of ring, 62 s of ride-through.

8.8 Serial and network interfaces

> **Boundary.**
> Signalling, framing, arbitration and bit timing of SPI, I²C, CAN, USB and Ethernet are in *Electronics and Computing*, Ch. 26 §26.3–26.7; industrial Ethernet and fieldbuses in Chapter 21.

Inside the node the converter links are chosen for bandwidth and determinism; between nodes, for distance, throughput and time transfer (Table 8.8).

Table 8.8 — Interfaces in and around the DAQ (practical payload rates, typical).

Interface	Raw rate	Practical payload	Latency	Used here
SPI to converter	25–50 Mbit/s	Near raw	µs	Node M ΔΣ converter; Node T converters
LVDS or parallel to FPGA	100+ Mbit/s per lane	Near raw	ns	Node P SAR channels (through isolators)
CAN FD	0.5 Mbit/s arbitration, 2 Mbit/s data	about 1 Mbit/s for 64-byte frames	Frame time, sub-ms	Inverter telemetry
USB 2.0 high speed	480 Mbit/s	30–40 MB/s bulk	125 µs microframes	Bench DAQ, not here
100BASE-TX Ethernet	100 Mbit/s	about 11.5 MB/s TCP	Sub-ms on a quiet LAN	Node T
1000BASE-T Ethernet	1 Gbit/s	about 115 MB/s TCP	Sub-ms on a quiet LAN	Nodes P and M, host
EtherCAT	100 Mbit/s	Cyclic process image	µs jitter, distributed clocks	Alternative for slow channels

Worked example — Node P on Gigabit Ethernet. Node P sends 16.0 MB/s in packets carrying 1456 bytes of samples after a 16-byte block header inside a 1472-byte UDP payload: 10 989 packets per second. Each occupies 1538 bytes on the wire (payload, UDP, IP and Ethernet headers, preamble and inter-frame gap), so the link carries $10\,989 \times 1538 \times 8 = 135$ Mbit/s, 13.5 % of the link — within R9's 50 %, and Node M adds 14 Mbit/s. The bottleneck is the host, not the wire: 11 000 packets per second through a default 200 kB socket buffer overflow it in about 12 ms if the receiving thread is descheduled. The host therefore uses a 64 MB socket buffer and a dedicated receive thread, and the node's 1 GB ring with a retransmission request by sequence number covers anything longer. UDP with application-level sequence numbers and retransmission was chosen over TCP for Node P so that live-view latency (R5) is not held up behind a retransmission of archived data; Node M and T use TCP.

8.9 Data storage

Formats

Table 8.9 — Storage formats for measurement data.

Format	Strengths	Weaknesses	Used here
CSV	Universal, readable	3–4× larger than binary, slow, lossy float printing, no typed metadata	Small exports only
Raw binary blocks + JSON header	Fastest to write, simple, append-only	Must be documented (endianness, scaling)	Node ring buffers, transfer
HDF5	Chunked, compressed, typed metadata, partial reads	Library dependency; needs care for crash consistency	Archive of every test
TDMS	Streaming with metadata	Tied to one vendor's ecosystem	Not used
ASAM MDF4	Automotive standard, bus and signal data together	Tooling-specific	Inverter CAN logs for the drive supplier
Parquet	Columnar, compressed, fast queries	Not suited to streaming waveforms	Per-test-point results table

A 16-bit sample written as text ("-12345,") takes seven bytes against two in binary; for Node P's 12.8 GB of windows per test, CSV would mean 45 GB and a float formatting choice that could lose resolution. Raw binary blocks written by the host's receive thread, then converted to HDF5 after each test point with the metadata of Section 8.5, give both fast writes and a self-describing archive.

The storage budget

Worked example — How much disk does a test need? For one 8 h efficiency map (Table 8.10), Node P windows are $400 \times 2\ \text{s} \times 16\ \text{MB/s} = 12.8\ \text{GB}$; its per-cycle aggregates, 20 quantities × 4 bytes × 1000 records/s for 28 800 s, 2.3 GB; Node M continuously, $1.64\ \text{MB/s} \times 28\,800\ \text{s} = 47.2\ \text{GB}$; Node T and CAN together 0.3 GB. The total is 62.6 GB. A 2 TB NVMe disk kept 20 % free holds 1.6 TB, 25 tests, before data must move to the archive; the peak write rate of 17.6 MB/s is under 4 % of a sustained 500 MB/s NVMe rate, so writing never limits acquisition.

Table 8.10 — Storage budget per 8 h test.

Stream	Volume	Share
Node P windows (400 × 2 s)	12.8 GB	20 %
Node P per-cycle aggregates	2.3 GB	4 %
Node M continuous	47.2 GB	75 %
Node T and CAN	0.3 GB	< 1 %
Total / usable disk / tests per disk	**62.6 GB / 1.6 TB / 25**	

The budget also says where to save: Node M's continuous record is three quarters of the volume. Revision B stores Node M at full rate only within ±5 s of each test point and anomaly flag, and keeps 1-second spectra otherwise, cutting the test to about 22 GB.

Integrity

R7 requires that a power failure costs at most one second and never corrupts closed files. The host writes each window to a temporary file, flushes it to disk, computes its SHA-256, writes the hash into the header and renames the file atomically into place; a file that exists under its final name is complete by construction. The continuous streams are written as one-minute segments the same way, and the host runs on a UPS that gives it 5 minutes to close everything cleanly. The nodes themselves store nothing permanently: their flash is used only for configuration, which avoids the write-endurance failures of loggers that write samples to internal flash.

8.10 Measurement validation

A data set is valid when it can be shown to be complete, correctly timed, within the sensors' ranges and physically consistent. Each check runs automatically after every test point, and its result is written into the file (Table 8.11).

Table 8.11 — Automated validation checks.

Check	Rule	Catches
Completeness	No gap in block sequence; overrun counters zero	Lost samples (R4)
Time base	PTP locked throughout; offset < 250 ns; timestamps monotonic	Clock faults (R3)
Range and clipping	No sample at converter full scale; values inside sensor range	Overload, wrong range setting
Stuck or dead channel	Standard deviation over 1 s above a floor for running conditions	Frozen sensor, broken wire
Sensor health	IEPE bias 2–20 V; thermocouple burnout; current-transducer status	Cable faults, open sensors
Physical consistency	Motoring: $P_{dc} \geq P_{ac} \geq T\omega$; inverter efficiency 95–99.5 %; motor 85–98 %	Calibration, sign or alignment errors
Reference injection	Shared reference sine phase between nodes within 1 µs	Inter-node skew (R3)

The consistency check is the most powerful: an efficiency above 100 % in the opening paragraph was a failed physical-consistency check that nobody had automated. The reference-injection check measures alignment directly. A 97 Hz sine from a signal generator is fed into a spare channel on every node; the listing estimates each node's phase by a three-parameter least-squares sine fit (IEEE Std 1057) using the node's own sample timestamps, so the result is the timing error of the whole chain including filter delay. The simulated case includes Node M's 664 µs filter delay and a 0.35 µs residual timing error.

```
 import numpy as np

F_REF = 97.0                        # shared reference sine, injected into one channel per node
rng = np.random.default_rng(7)

def capture(fs, n, true_delay, noise):
    """Timestamps the node believes, and samples it really took true_delay later."""
    t = np.arange(n)/fs
    x = np.sin(2*np.pi*F_REF*(t - true_delay)) + rng.normal(0, noise, n)
    return t, x

def phase(t, x):
    """Three-parameter sine fit (IEEE 1057) at known frequency; returns phase in rad."""
    A = np.column_stack([np.sin(2*np.pi*F_REF*t), np.cos(2*np.pi*F_REF*t), np.ones_like(t)])
    s, c, _ = np.linalg.lstsq(A, x, rcond=None)[0]
    return np.arctan2(c, s)

GD_M = 34/51_200                    # node M decimation-filter group delay (datasheet class)
tP, xP = capture(1_000_000, 200_000, 0.0, 1e-3)          # node P, SAR, 0.2 s
tM, xM = capture(51_200, 10_240, GD_M + 0.35e-6, 1e-3)   # node M, delta-sigma + PTP error

def skew(tm):
    d = (phase(tP, xP) - phase(tm, xM))/(2*np.pi*F_REF)
    return (d + 0.5/F_REF) % (1/F_REF) - 0.5/F_REF        # unwrap to +/- half a period

print(f"raw skew M vs P:         {skew(tM)*1e6:8.2f} us")
print(f"after group-delay shift: {skew(tM - GD_M)*1e6:8.2f} us")
```

```
 raw skew M vs P:           664.42 us
after group-delay shift:     0.36 us
```

The method resolves the 0.35 µs residual to about 0.01–0.02 µs (repeated runs with different noise seeds give 0.32 to 0.36 µs), well inside R3. The reference frequency is chosen low, 97 Hz, so that the phase is unambiguous over ±5.2 ms: at 997 Hz the 664 µs delay would wrap and read as −339 µs. Figure 8.6 shows the two captured sines before and after compensation.

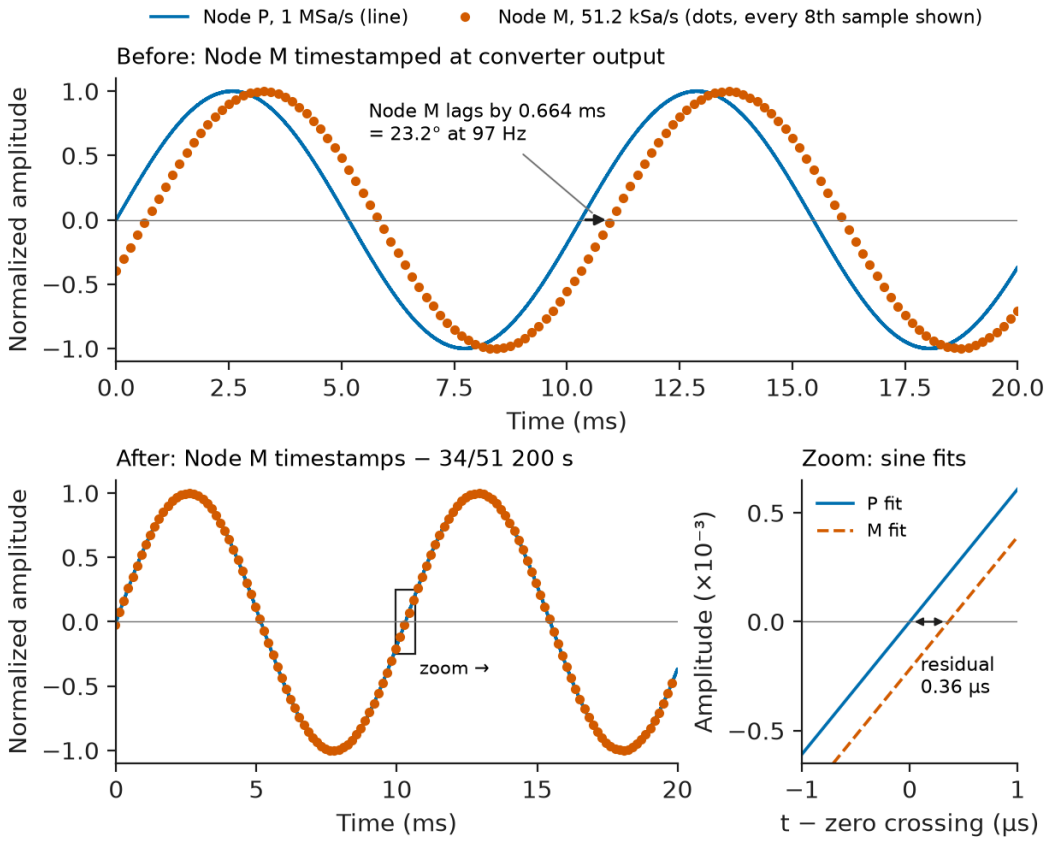


Figure 8.6 — Shared 97 Hz reference sine captured by Node P and Node M, before and after filter-delay compensation.

> **Status of the principle: Energy-balance plausibility check**
> *Fundamental.* Energy conservation requires DC input power to exceed AC output power and AC power to exceed shaft power in motoring; a measured efficiency above unity proves an error in calibration, sign convention or time alignment, although an efficiency below unity does not prove the data correct.

8.11 Verification and failure analysis

Table 8.12 — Verification of the dynamometer DAQ.

Req.	Test	Limit	Result	Pass/fail
R1	Configuration audit; rate counters over 1 h	Rates as specified	All exact to PTP-second boundaries	Pass
R2	FFT test per channel; 1 µs pulse into all 8 P channels	ENOB ≥ 14.0; skew ≤ 50 ns	14.31–14.42 bits; 18 ns worst	Pass
R3	97 Hz reference into P, M, T for 8 h	P–M ≤ 1 µs; P–T ≤ 1 ms	P–M 0.15–0.42 µs; P–T 0.2 ms	Pass
R4	8 h run; forced 15 s host stall	0 undetected gaps	0 gaps in normal run; 15 s stall: 0 lost on Node P (62 s ring), gap flagged on Node M after its 2.4 s ring filled; 70 s stall: gap flagged on both	Pass
R5	Test pulse to display, timestamp comparison	≤ 250 ms	120 ms worst	Pass
R6	Full 8 h test; reader opens every file	≥ 20 % free; all files self-describing	62.6 GB used; 100 % read	Pass
R7	Mains pulled at random 10 times	≤ 1 s lost; files readable	UPS shutdown clean; with UPS bypassed, ≤ 0.8 s lost, no corrupt closed file	Pass
R8	Seeded faults: open IEPE, swapped current sign, 1 ms time step, stuck TC	All detected	4/4 detected and flagged	Pass
R9	Switch counters at full rate	≤ 50 %	13.5 % (P), 1.4 % (M)	Pass
R10	Type test 1000 V DC working; routine hipot	IEC 61010-1/-2-030 limits	Pass at accredited lab	Pass

Table 8.13 — FMEA of the dynamometer DAQ (extract).

Failure mode	Effect	Cause	Detection	Mitigation
Uncompensated filter delay	Phase and efficiency errors in transients	Filter mode changed in firmware without updating delay	Reference-sine check (R3) after every firmware change	Delay derived from filter configuration in code, written to metadata
PTP loss and holdover	Inter-node drift of µs per second	Grandmaster or switch reboot	PTP state in block headers; validation check	Data outside lock flagged; grandmaster on UPS
Silent sample loss	Shortened record, wrong time base	Host stall longer than node buffer	Sequence gaps, overrun counters	Node rings (62 s P, 2.4 s M); gap flags
Asymmetric network path	Constant time offset	Non-PTP-aware switch inserted during maintenance	Reference-sine check	Switch model locked in drawings; check in commissioning
IEPE cable fault	Distorted or missing vibration	Damaged cable, connector water ingress	Bias voltage outside 2–20 V	Health check per block; cable routing
Calibration coefficients wrong	Systematic error in a channel	Sensor swapped without updating record	Physical-consistency check; sensor serial read back where possible	Calibration database keyed by sensor serial
File corruption on power loss	Lost or unreadable test data	Write interrupted	Hash check on read	Atomic rename, fsync, UPS
Ground loop through Ethernet shields	Noise on HV channels	Shielded cables bonded at both ends across cell grounds	Noise floor check at standstill	Fibre link from Node P to switch

The last line records a Revision A change: Node P's copper Ethernet link carried 2 mV of 50 Hz noise into the isolated channel references through the cable shield, raising the noise floor by 3 dB; a fibre link removed it and broke a ground path between the inverter cabinet and the control room.

What remains open

Open question	Where it is answered
How are the cell's thermocouples, speed sensor and energy measurements built and calibrated as instruments?	Chapter 9
How is a whole laboratory's acquisition organised around shared time, storage and test automation?	Chapter 9 §9.8
How are firmware architecture, fault handling and timing designed across many embedded nodes?	Chapter 10
How is telemetry moved from remote devices over MQTT or HTTP with authentication?	Chapter 11
How do EtherCAT, PROFINET and other industrial networks handle cyclic data and safety?	Chapter 21
How are acquired data processed, versioned and managed in engineering software workflows?	Chapter 29, Chapter 30
How is a drive's efficiency map used in a vehicle energy model?	Chapter 24

Next: Chapter 9 applies conditioning and acquisition to eight compact instruments for an electric-drive and battery laboratory — temperature, energy, speed, power quality, battery monitoring, calibration rigs, automated test and the laboratory DAQ that ties them together.

Chapter 9 — Instrumentation Projects

Part II · Instrumentation and Measurement Systems

An electric-drive and battery laboratory installs a new 16-channel thermocouple monitor in each of its battery climatic chambers. On the bench it agrees with a reference thermometer to 0.2 °C. In service, an hour after the enclosure lid is screwed down, every channel reads about 2 °C high, and a cell that is actually at 58 °C trips the 60 °C test-abort alarm. The thermocouples are within tolerance and the converter is fine. The cold-junction sensor is a thermistor on the circuit board, 4 cm from a linear regulator; with the lid closed the board warms by 2 °C more than the terminal block where the thermocouple wires actually end, and a thermocouple measures only the difference between its two junctions. The fix — an isothermal aluminium terminal block with the reference sensor inside it — costs $14. The failure is typical of small instruments: each is simple, and each fails at an interface its designer did not measure. This chapter builds eight of them for the same laboratory, each as a compact project with requirements, architecture, a key calculation, a part choice, a test with a limit and a failure.

> **What does it take to turn a sensor, a converter and a microcontroller into an instrument whose readings can be trusted in a real laboratory?**
> **Which few design decisions and tests dominate each common instrument — temperature, energy, speed, power quality, battery, calibration rig and test station?**

Concept	Engineering destination
Temperature monitoring	Climatic chambers, battery cycling, data centres, cold chain, process monitoring
Energy monitoring	Sub-metering, building energy management, lab cost allocation
Motor-speed measurement	Drive test benches, conveyors, pumps, machine tools
Power-quality measurement	Compliance surveys, troubleshooting drive trips, utility contracts
Battery monitoring	Cell and module testing, energy-storage commissioning, BMS validation
Sensor calibration systems	In-house calibration, traceability, ISO/IEC 17025 practice
Automated test equipment	Production test of boards and modules, incoming inspection
Laboratory data-acquisition systems	Shared time, storage and test automation for a whole lab

Roadmap. The laboratory's instrument suite, its requirements and the temperature monitor (9.1) → energy (9.2) → speed (9.3) → power quality (9.4) → battery (9.5) → calibration rig (9.6) → automated test (9.7) → the laboratory DAQ that ties them together (9.8) → verification and failure analysis across the suite (9.9).

9.1 Temperature monitoring

> **Boundary.**
> Thermocouple physics and the Seebeck effect are in *Engineering Physics*, Ch. 14 §14.3; sensor types and selection in Chapter 6; RTD excitation and lead resistance in *Electronics and Computing*, Ch. 26 §26.9.

The laboratory and its instruments

The running system is the instrument suite of the laboratory that houses the dynamometer cell of Chapter 8 (Figure 9.1). Every instrument publishes to one laboratory data server with shared time, and each is small enough to be designed, built and verified by one engineer in a few weeks. Table 9.1 collects their requirements; IDs carry the project's initial.

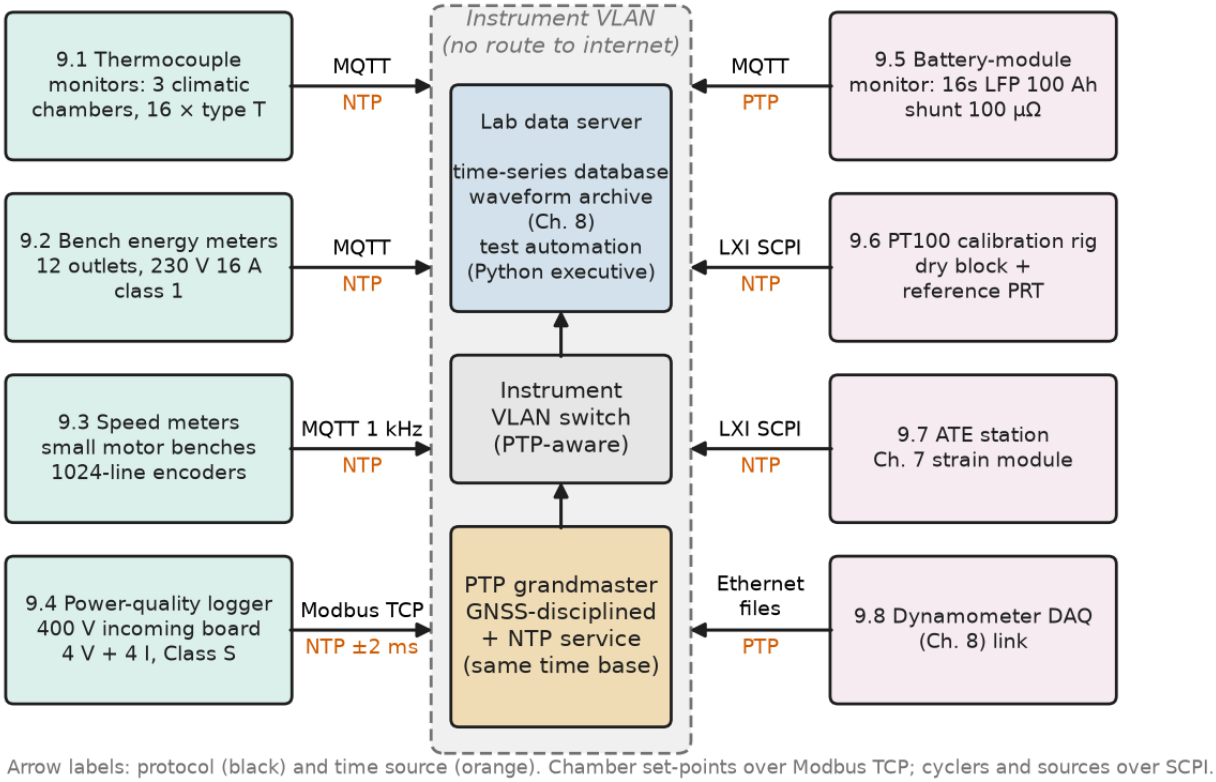


Figure 9.1 — Instrument suite of the electric-drive and battery laboratory around its shared data backbone.

Table 9.1 — Requirements for the laboratory instrument suite.

ID	Requirement	Target	Verification
T1	Chamber thermocouple accuracy	16 × type T, −20 to +60 °C, U ≤ 1.0 °C (k = 2) including sensor	Comparison with reference PRT in chamber, lid closed, 4 h
T2	Over-temperature alarm	Relay opens within 2 s of any channel > 60 °C, independent of server	Heated-junction test
E1	Bench energy meter accuracy	Class 1 (IEC 62053-21), 0.05–16 A, PF 0.5 inductive to 1, 230 V	Reference meter on calibration bench
S1	Shaft speed	±0.1 % of reading, 300–6000 rpm, new value every 1 ms	Speed from precision frequency source
Q1	Power-quality methods	IEC 61000-4-30 Class S: 10-cycle RMS ±0.5 % of declared voltage; harmonics to the 40th (IEC 61000-4-7)	Calibrator with harmonics, 49.5–50.5 Hz
Q2	Event capture	Sags and swells by half-cycle RMS; timestamps ±10 ms	Programmable source sags
B1	Cell voltages	16 cells, ±2 mV, all cells within 10 ms of each other	Precision DMM per cell
B2	Module current	±0.1 % of reading ±20 mA (k = 2), −300 to +300 A, aligned with voltages within 1 ms	Reference DC current transducer
B3	Charge counting	Offset-driven drift ≤ 0.1 Ah per 24 h	24 h at zero current
C1	PT100 calibration	U ≤ 0.075 °C (k = 2), 0–125 °C	Uncertainty budget; proficiency comparison
A1	Strain-module production test	≤ 120 s per unit; guard-banded limits	Timed runs; seeded faulty units
A2	ATE repeatability	%GRR ≤ 30 % of gain-error tolerance	Gauge R&R study, 10 units × 3 repeats × 2 operators
L1	Time and data	Every record timestamped with stated accuracy; one catalogue for all instruments	Audit of 1 week of data

The temperature monitor: architecture and part choice

Battery cells in the chambers run from −20 to +60 °C; the monitor reads 16 thermocouples bonded to cells, logs at 1 Sa/s and opens an alarm relay wired into the chamber's interlock (T2). Type T (copper–constantan) is chosen over the more common type K because its IEC 60584-1 class 1 tolerance is ±0.5 °C from −40 to 125 °C, against ±1.5 °C for type K, and T1 cannot be met with a ±1.5 °C sensor. A thermocouple measures only the temperature *difference* between its measuring junction and the point where its wires meet copper, so the second sensor in the system — the cold-junction compensation (CJC) sensor — has an error that adds one-for-one to every reading.

Table 9.2 — Front-end options for 16 thermocouples (datasheet-class values; indicative 2026 prices).

Option	Class	CJC	Resolution and accuracy	Cost for 16 ch	Verdict
Thermocouple-to-digital IC per channel	MAX31856	On-die sensor, ±0.7 °C, at the IC's location	19-bit; linearization built in	about $130	CJC on the PCB, not at the terminals
16-bit ΔΣ with PGA	ADS1118	Internal temperature sensor	16-bit, 2 channels per IC	about $45	Same CJC weakness; 8 ICs
24-bit ΔΣ with PGA, external RTD for CJC	AD7124-8 ×3	PT1000 class A inside isothermal block	Noise ≪ 0.01 °C; burnout detection	about $40 + $14 block	**Chosen**
Commercial 16-ch module	NI-9213 class	Built-in, at terminals	±0.5–1 °C typical system accuracy	about $1100	Meets T1 marginally; cost

In the chosen design both thermocouple terminal blocks are bolted to one 10 mm aluminium plate with a 4-wire PT1000 cemented into a hole between them; the plate is thermally separated from the PCB by nylon standoffs and from the enclosure air by a foam cover. Two converters take seven thermocouples each plus one RTD input; a third takes the remaining two thermocouples and a second PT1000 at the other end of the block, which measures the block's own gradient.

The uncertainty budget

Table 9.3 — Uncertainty of one chamber channel at 25 °C cold junction.

Contribution	Half-width or s	Distribution	Standard uncertainty
Type T class 1 tolerance	0.5 °C	Rectangular	0.289 °C
CJC PT1000 class A, $\pm(0.15 + 0.002\lvert t\rvert)$ at 25 °C	0.20 °C	Rectangular	0.115 °C
Gradient terminal to PT1000 (measured ≤ 0.1 °C)	0.10 °C	Rectangular	0.058 °C
Converter, reference, linearization (NIST ITS-90 polynomial)	—	Normal	0.030 °C
Noise at 1 Sa/s	—	Normal	0.020 °C
Combined; expanded (k = 2)			**0.318 °C; 0.64 °C**
T1 limit; margin			**1.0 °C; 36 %**

The sensor's own tolerance is 80 % of the variance; the next improvement would be calibrating the thermocouple batch on the rig of Section 9.6, not a better converter.

Worked example — How fast is the alarm? Each AD7124-class converter runs its 50/60 Hz-rejecting sinc filter, which settles in about 80 ms after a channel switch, so eight inputs take $8 \times 80 = 640$ ms per scan. The firmware compares each new value with 60 °C and debounces over two consecutive readings, so the worst case is about two scans, 1.3 s, inside T2's 2 s. The comparison and relay drive run in the instrument, not on the server; if the network or server is down, the alarm still works, and the relay is energized-when-healthy so that a dead instrument also opens the interlock.

The failure and its test

Figure 9.2 shows the opening paragraph's failure as measured, and the fix. The error rose with a time constant of about 22 min after the lid closed, because the board's thermistor followed the regulator's heating while the terminal block followed the chamber air. The verification test for T1 is therefore run with the lid closed for 4 h, the instrument's worst thermal state, never on an open bench.

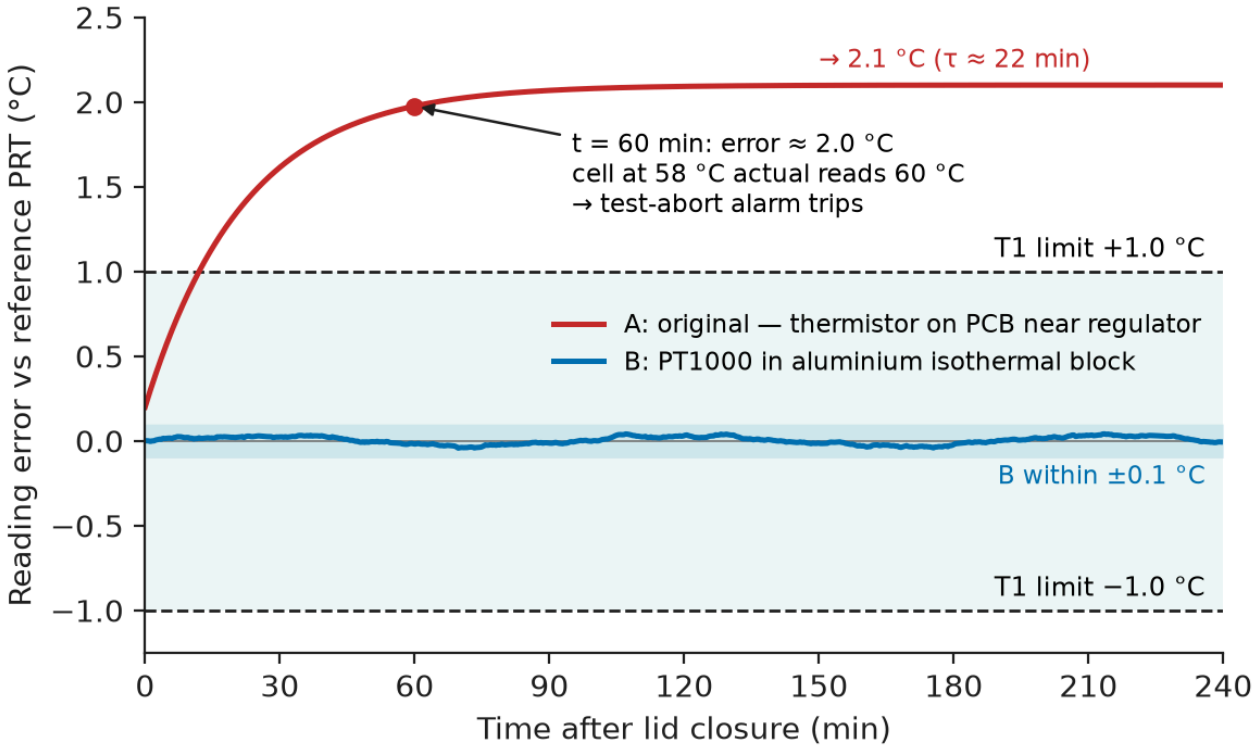


Figure 9.2 — Cold-junction compensation error after the enclosure lid is closed, original design and isothermal-block fix.

9.2 Energy monitoring

> **Boundary.**
> AC power, power factor and three-phase power are in *Circuits, Signals and Communications*, Ch. 9. This section is a compact project; the full design of a smart energy monitor, with front-end design, firmware, calibration, communications and safety, is Chapter 35.

The laboratory allocates energy costs to projects through meters on its 12 bench outlets (230 V, 16 A). Each meter is a single-phase **metering IC** (ADE7953 or STPM32 class: delta-sigma converters, digital integrator and power computation, ±0.1 % typical over a 1000:1 range) with a 2000:1 current transformer and a resistive voltage divider, and a small MCU that reads energy registers every second and publishes them. The CT with a 10 Ω burden gives 80 mV at 16 A; a 1 MΩ / 1 kΩ divider gives 230 mV at 230 V, both inside a ±500 mV input range. The IC's analog side sits at mains potential through the divider, so the MCU's link to the network crosses a reinforced digital isolator, and creepage and clearance follow IEC 61010-1 for a CAT III 300 V circuit.

The dynamic range sets the converter: from the 16 A maximum down to the starting current of a class 1 meter, 0.004 of its base current (here 1 A), or 4 mA, is 4000:1, and the meter must still be within ±1.5 % at 5 % of its base current, 0.05 A. That is why metering ICs are delta-sigma devices with programmable gain, and why the CT burden is kept small enough that the CT stays linear at the bottom of the range.

Worked example — The CT's phase error at power factor 0.5. Active power is $P = VI\cos\varphi$. A CT whose secondary current leads the primary by δ makes the meter compute $VI\cos(\varphi - \delta)$, a relative error of about $\delta \tan\varphi$. At PF 0.5 ($\varphi = 60°$, $\tan\varphi = 1.732$) a typical small CT's 0.5° phase error gives $1.732 \times 0.00873 = 1.51$ %, beyond class 1 limits on its own, while at PF 1 it contributes nothing. The metering IC has a phase-calibration register that delays the voltage channel; set during calibration at PF 0.5, it leaves a residual of about 0.02°, 0.06 %. The lesson generalizes to every power measurement in this book: amplitude calibration at unity power factor proves nothing about the phase path.

> **Status of the principle: Power error from a phase error**
> *Derived.* First-order expansion of $\cos(\varphi - \delta)$ for small δ; the error grows without bound as φ approaches 90°, which is why reactive-energy and low-power-factor measurements need phase matching an order of magnitude better than active-energy ones.

Verification follows the class 1 test points (0.05, 0.1, 1, 5 and 16 A at PF 1; 0.1, 5 and 16 A at PF 0.5 inductive and 0.8 capacitive) against a reference meter of class 0.1 on the calibration bench; the results are in Table 9.7.

9.3 Motor-speed measurement

> **Boundary.**
> The M, T and M/T counting methods, their quantization for a 16 384-count encoder and an M/T firmware listing are in Chapter 6 §6.5; encoder interfacing for control loops is in Chapter 15. This section builds a stand-alone speed meter for cheaper sensors, where a different error dominates.

The instrument

The laboratory's small motor benches and conveyor rigs (up to 6000 rpm) need a portable speed meter: a 5-digit display updated four times a second for the operator, and a 1 kHz stream to the lab DAQ (S1: ±0.1 % of reading, 300–6000 rpm). It accepts either the rig's existing 1024-line encoder (RS-422 A channel) or a Hall gear-tooth sensor reading a 60-tooth steel wheel where no encoder is fitted. The input passes an RC filter and a Schmitt trigger into a timer of an STM32G4-class MCU with a 170 MHz capture clock; the timer's digital glitch filter rejects pulses shorter than 0.5 µs, which removes drive switching spikes and is still ten times shorter than the 4.9 µs half-period of a 1024-line encoder at 6000 rpm.

The quantization analysis of Chapter 6 assumes perfectly spaced edges. A cheap encoder or a machined gear does not have them: each line or tooth sits off its ideal angle by a **pitch error**, about 1 % of a line period for a small optical encoder, and a single interval inherits it in full at both ends. The listing adds that term to the three methods, with an M/T estimator that spans at least 40 lines over a sliding window updated every 1 ms.

```
 import math

LINES, F_CLK, T_WIN = 1024, 170e6, 1e-3   # A-channel rising edges, timer clock, window
PITCH_ERR = 0.01                          # edge-placement error, fraction of a line period
M_MIN = 40                                # M/T: minimum lines per estimate

print(" rpm   M-method   T-method   M/T quant   M/T pitch   M/T window")
for rpm in (100, 300, 1200, 6000):
    f_line = rpm/60*LINES                         # edges per second
    m_rel = 1/(f_line*T_WIN)                      # one count in the window
    t_rel = f_line/F_CLK + 2*PITCH_ERR            # one clock tick per period + pitch
    lines = max(M_MIN, math.floor(f_line*T_WIN))   # whole lines inside the estimate
    win = max(T_WIN, lines/f_line)                # window stretches at low speed
    mt_q = 1/(F_CLK*win)                          # one clock tick over the window
    mt_p = 2*PITCH_ERR/lines                      # two end edges misplaced
    print(f"{rpm:5d}  {100*m_rel:8.3f} %  {100*t_rel:7.3f} %  {100*mt_q:8.5f} %"
          f"  {100*mt_p:8.3f} %  {1e3*win:7.2f} ms")
```

```
  rpm   M-method   T-method   M/T quant   M/T pitch   M/T window
  100    58.594 %    2.001 %   0.00003 %     0.050 %    23.44 ms
  300    19.531 %    2.003 %   0.00008 %     0.050 %     7.81 ms
 1200     4.883 %    2.012 %   0.00030 %     0.050 %     1.95 ms
 6000     0.977 %    2.060 %   0.00059 %     0.020 %     1.00 ms
```

Worked example — Reading the table against S1. The M method misses S1 at every speed: even at 6000 rpm one count in a 1 ms window is 0.98 %. The T method's clock resolution is excellent (0.06 % at 6000 rpm) but the pitch error puts every single-interval estimate at about ±2 %. The M/T method meets ±0.1 % everywhere: clock quantization is negligible, and pitch error is 0.05 % at 40 lines, falling to 0.02 % at 6000 rpm where 102 lines fit in 1 ms. The price is latency at low speed: at 300 rpm the 40-line window spans 7.8 ms, so a new value still appears every 1 ms but describes the last 7.8 ms. Below 300 rpm S1 is not claimed. Figure 9.3 plots the three methods: with real edges the deciding term is pitch error, not clock resolution.

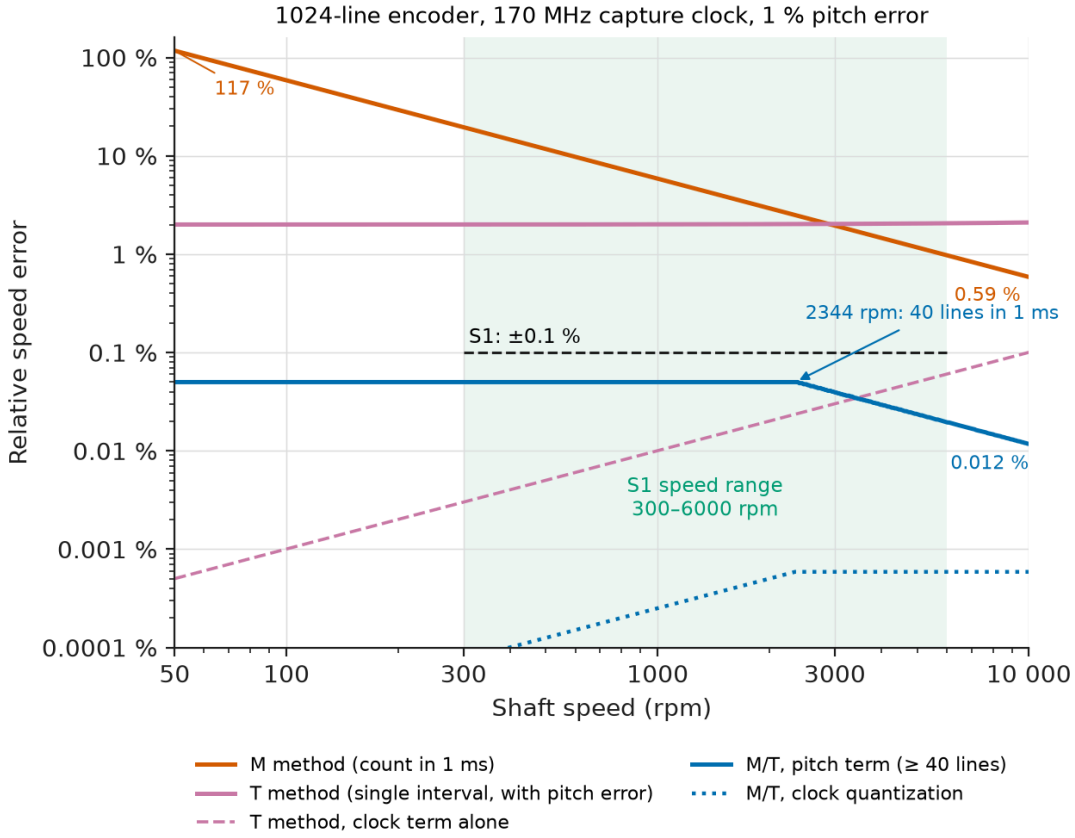


Figure 9.3 — Relative speed error of M, T and M/T methods against shaft speed for a 1024-line encoder and 170 MHz timer.

> **Status of the principle: M/T speed estimation**
> *Derived.* Counting whole edges over a clock-timed interval that begins and ends on edges makes clock quantization relative to the whole interval and divides end-point placement error by the number of lines spanned; it assumes speed is nearly constant over the window, so the window length is also the estimator's low-pass filter.

Worked example — The gear-tooth wheel. With the 60-tooth wheel, 40 teeth take 133 ms at 300 rpm and 6.7 ms at 6000 rpm, so the 1 kHz stream is heavily overlapped at low speed. A second error appears: if the wheel is mounted 0.05 mm off-centre on a 50 mm pitch radius, each tooth's angle is displaced by up to $e/r = 0.001$ rad in a once-per-revolution pattern, and the measured speed carries a ±0.1 % ripple at shaft frequency — the whole of S1. Averaging over whole revolutions cancels it exactly, so in gear-tooth mode the estimator's window is rounded up to an integer number of revolutions (200 ms at 300 rpm, 10 ms at 6000 rpm), and the meter states that mode in every record.

The verification source is a function generator producing a square wave at the line frequency for 300, 1200 and 6000 rpm (5.12, 20.48 and 102.4 kHz) with a rubidium-referenced timebase, which tests the timer chain; the pitch-error term is then checked against a 10 000-line reference encoder on the same shaft. The same estimator feeds the motor drive of Chapter 37.

9.4 Power-quality measurement

> **Boundary.**
> Power-quality phenomena, limits (EN 50160, IEC 61000-3-x) and IEC 61000-4-30 instrument classes are in *Electrical Energy and Machines*, Ch. 25 §25.9; the leakage of a fixed-rate 200 ms record at 49.8 Hz is worked through in *Circuits, Signals and Communications*, Ch. 22 §22.3. This section builds the instrument.

The laboratory's drives and cyclers draw distorted currents, and the facilities team wants to know whether the 400 V incoming supply stays inside EN 50160 and which equipment is responsible when a cycler trips. A permanently installed **Class S** logger (IEC 61000-4-30) at the incoming board measures four voltages (L1, L2, L3, N to PE) and four currents through split-core CTs with 333 mV outputs.

Architecture and safety

An 8-channel simultaneous delta-sigma converter (ADS131M08 class) samples all channels at a fixed 10.24 kSa/s, 2048 samples per 10 cycles at exactly 50 Hz. The voltage inputs are 2 MΩ resistor strings (five 400 kΩ resistors in series, each rated for its share of the impulse) into 2.7 kΩ, giving ±438 mV peak at 230 V rms, with headroom to the converter's ±1.2 V for swells. Unlike the strain module of Chapter 7, this front end *is* mains-connected: the incoming board is a CAT III location, so the inputs must withstand a 4 kV impulse (IEC 61010-1 for CAT III 300 V; *Electrical Energy and Machines*, Ch. 29 §29.7), and the barrier between the measuring circuit and the Ethernet port must be **reinforced**: a reinforced digital isolator, a reinforced isolated DC–DC converter and 6 mm of creepage on the PCB with a slot under the barrier.

Synchronization to the mains

IEC 61000-4-7 computes harmonics over a 10-cycle window that must contain exactly ten mains cycles, with harmonic **groups** that collect the energy of neighbouring bins. A fixed-rate converter delivers 2048 samples per 200.0 ms, but at 49.8 Hz ten cycles last 200.8 ms. Instead of a PLL that steers the converter clock, this design measures the frequency from the zero crossings of the fundamental and **resamples** each 10-cycle window to 2048 points by cubic interpolation in firmware. The listing compares the two on a voltage with 4 % fifth, 3 % seventh and 1.5 % eleventh harmonics.

```
 import numpy as np
from scipy.interpolate import CubicSpline

FS, N = 10_240, 2048                       # fixed ADC rate; 2048 samples = 10 cycles at 50 Hz
H = {1: 1.0, 5: 0.04, 7: 0.03, 11: 0.015}  # harmonic amplitudes, per unit of fundamental

def grid_voltage(t, f):
    return 325.27*sum(a*np.sin(2*np.pi*h*f*t + 0.3*h) for h, a in H.items())

def harmonics(x):
    """IEC 61000-4-7 harmonic groups from one 10-cycle window, per unit of H1."""
    X = 2*np.abs(np.fft.rfft(x))/len(x)
    g = {h: np.sqrt(X[10*h]**2 + 0.5*X[10*h - 5]**2 + 0.5*X[10*h + 5]**2
                    + sum(X[10*h + k]**2 for k in (-4, -3, -2, -1, 1, 2, 3, 4)))
         for h in range(1, 41)}
    thd = np.sqrt(sum(g[h]**2 for h in range(2, 41)))/g[1]
    return g[2]/g[1], g[5]/g[1], thd

for f in (50.0, 49.8):
    t = np.arange(int(1.1*N))/FS
    x = grid_voltage(t, f)
    fixed = harmonics(x[:N])                           # window assumes exactly 50 Hz
    t_sync = np.arange(N)*(10/f)/N                     # 2048 points over exactly 10 cycles
    sync = harmonics(CubicSpline(t, x)(t_sync))
    for name, r in (("fixed", fixed), ("sync ", sync)):
        print(f"{f:4.1f} Hz {name}: H2 {100*r[0]:.3f} %  H5 {100*r[1]:.3f} %  THD {100*r[2]:.3f}
%")
```

```
 50.0 Hz fixed: H2 0.000 %  H5 4.000 %  THD 5.220 %
50.0 Hz sync : H2 0.000 %  H5 4.000 %  THD 5.220 %
49.8 Hz fixed: H2 1.813 %  H5 4.053 %  THD 5.823 %
49.8 Hz sync : H2 0.000 %  H5 4.000 %  THD 5.220 %
```

Worked example — What a 0.2 Hz deviation does to a compliance report. With the fixed-rate window at 49.8 Hz, the fundamental's leakage lands in the second-harmonic group as a fictitious 1.81 % H2 — close to the 2 % EN 50160 limit for that order — and THD reads 5.82 % instead of 5.22 %, 11.5 % high in relative terms. Harmonic grouping alone does not prevent this, because the fundamental is 25 times larger than the largest harmonic and its leakage dominates the low-order groups. Resampled to exactly ten cycles, all values return to their true levels to the printed precision (Figure 9.4). The resampling costs about 2048 cubic evaluations per channel per 200 ms window, under 2 % of a Cortex-M4 at 170 MHz for eight channels, and avoids a clock PLL that would also need to keep the converter's digital filter settled across frequency changes.

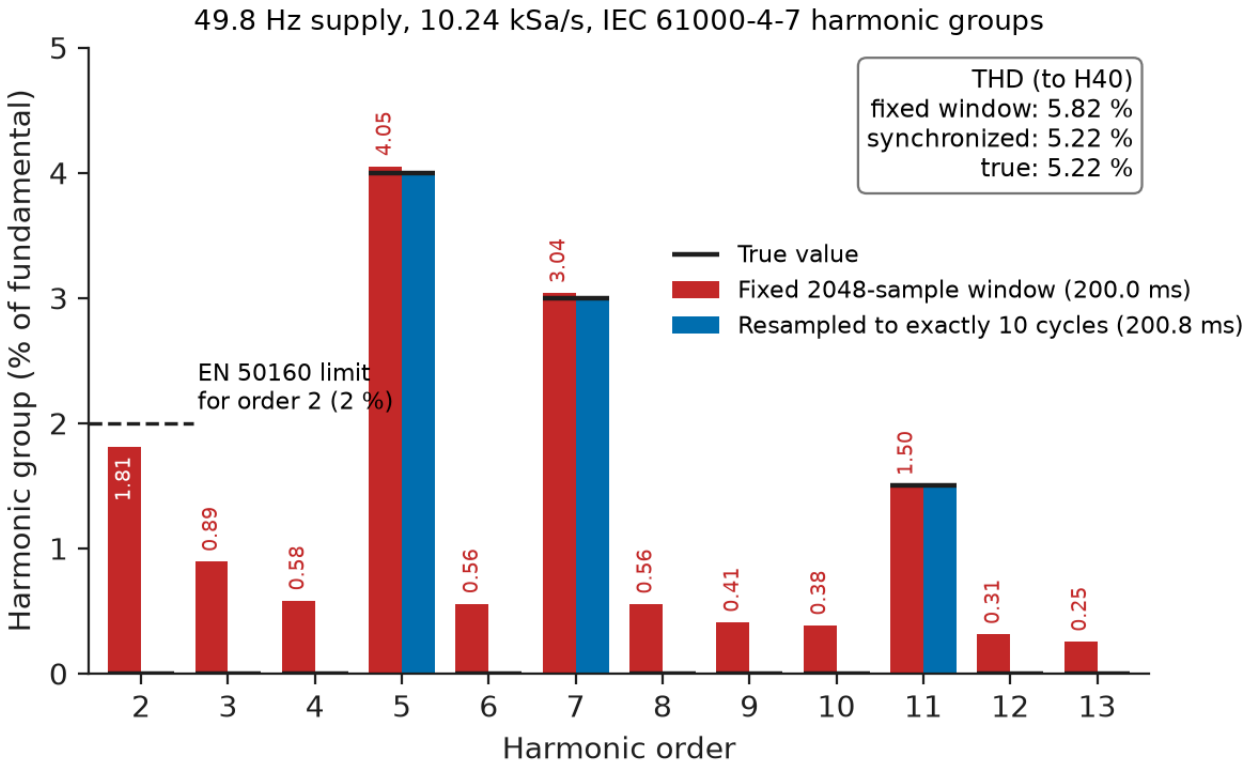


Figure 9.4 — Harmonic groups of a 49.8 Hz voltage measured with a fixed 2048-sample window and with the window resampled to exactly ten cycles.

> **Status of the principle: Window synchronization for harmonic measurement**
> *Derived.* The DFT of a record holding an integer number of cycles has no leakage between harmonic bins; with a non-integer number the fundamental's energy spreads into every group. It assumes the frequency is constant over the window, so the frequency estimate is updated every window, and fast frequency changes still produce small errors.

Events

Sags, swells and interruptions are detected from the half-cycle RMS, $U_{rms(1/2)}$, recomputed every half cycle over one full cycle; an event starts when any phase falls below 90 % of the declared 230 V and ends when all return above 92 % (2 % hysteresis), and its duration and residual voltage are logged with a 1-cycle waveform before and after. The logger's clock is NTP-disciplined to ±2 ms against the lab grandmaster, inside Q2's ±10 ms; correlating a sag with a cycler trip needs no better, because the cycler's own fault log is stamped to the nearest 10 ms.

9.5 Battery monitoring

> **Boundary.**
> Battery-pack architecture, BMS functions, state-of-charge and state-of-health estimation are owned by Chapter 23; the system-level principles (Coulomb counting, OCV methods, balancing) are in *Electrical Energy and Machines*, Ch. 16 §16.3–16.4. This section builds the *measurement* instrument that validates a module under test, independent of the cycler and the module's own BMS.

The laboratory cycles 16s LFP modules of 100 Ah (51.2 V nominal) for weeks at a time. The cycler reports its own current and voltage, but the test standard requires independent confirmation of cell voltages, module current and temperature, and the lab wants DC internal resistance (DCIR) from current pulses. LFP makes the voltage requirement hard: on its flat plateau the open-circuit voltage changes by well under a millivolt per percent of state of charge, so B1's ±2 mV is the loosest tolerance that is still useful.

Architecture. A 16-cell monitor IC of the automotive class (ADBMS6830 or BQ79616 class; about ±1.5–2 mV total error over temperature, all cells converted simultaneously) sits on the module with each sense lead protected by a 100 Ω series resistor and a fuse at the tap, because a sense wire shorted across a 100 Ah cell is a fire, not a measurement error. It talks over an isolated daisy-chain link to the instrument's MCU. Current is measured on the low side by a 100 µΩ manganin shunt and a 20-bit digital power monitor (INA228 class: ±163.84 mV range, ±1 µV maximum offset, 312.5 nV per LSB, which is 3.1 mA at 100 µΩ). Both converters are triggered from one timer so that voltage and current samples are aligned within 1 ms (B2), which DCIR needs: $R_{DC} = \Delta V / \Delta I$ at the edges of a 100 A pulse.

Worked example — Choosing the shunt value. At 300 A a 100 µΩ shunt dissipates $300^2 \times 100\ \mu\Omega = 9.0$ W and develops 30 mV; a 50 µΩ shunt halves the heat to 4.5 W but also halves the signal. The converter's ±1 µV offset is $1\ \mu\text{V} / 100\ \mu\Omega = 10$ mA with the larger shunt and 20 mA with the smaller. Uncorrected, 10 mA integrates to $0.010 \times 24 = 0.24$ Ah in 24 h, 0.24 % of the module's capacity, and 20 mA to 0.48 Ah; both miss B3's 0.1 Ah. The instrument therefore **auto-zeros** during every rest step of the test profile, when the cycler's output is open and the current is truly zero, leaving about 2 mA of residual drift: 0.048 Ah per 24 h with the 100 µΩ shunt. The 100 µΩ shunt is kept, mounted on a finned heat sink that limits its rise to 30 K so that its temperature coefficient stays inside the budget below.

Table 9.4 — Current-measurement budget (B2: ±0.1 % of reading ±20 mA, k = 2).

Contribution	Half-width	Distribution	Standard uncertainty
Shunt value after in-situ calibration against a reference DCCT	0.01 %	Rectangular	0.006 %
Shunt temperature coefficient, 20 ppm/K × 30 K	0.06 %	Rectangular	0.035 %
Converter gain drift after calibration	0.02 %	Rectangular	0.012 %
Converter offset (before auto-zero)	10 mA	Rectangular	5.8 mA
Thermal EMF at shunt sense terminals, 1 µV	10 mA	Rectangular	5.8 mA
Expanded (k = 2) at 10 A / 100 A / 300 A			**18 mA / 76 mA / 222 mA**
B2 limit at 10 A / 100 A / 300 A; margin			**30 / 120 / 320 mA; 30–40 %**

The table shows why the shunt's temperature, not the converter, dominates at high current — and why the shunt is calibrated in place, mounted and warm, against a reference DC current transducer rather than trusted from its nameplate.

9.6 Sensor calibration systems

> **Boundary.**
> Calibration procedure, calibration intervals and the place of calibration in a test plan are owned by Chapter 4; traceability chains and calibration methods per sensor type by Chapter 6 §6.11; least-squares fitting and its uncertainty by *The Mathematics Universe*, Ch. 20 §20.9–20.10. This section builds a calibration rig.

Traceability and the rig

For temperature the traceability chain runs from the ITS-90 fixed points at a national laboratory, through an ISO/IEC 17025-accredited laboratory that calibrates the lab's **reference platinum resistance thermometer** (PRT) with an expanded uncertainty of 0.020 °C, to the in-house rig, which calibrates the lab's PT100 sensors and, with an electrical thermocouple simulator, its thermocouple channels.

The rig is a dry-block calibrator (−25 to 140 °C, an aluminium block with drilled wells), the reference PRT, and a readout that measures each sensor by 4-wire ratio against a 100 Ω standard resistor (0.005 %, 2 ppm/K) using an AD7124-class converter, so that the excitation current cancels. Eight PT100s are calibrated together at six points, 0 to 125 °C, waiting at each for the reference to be stable within 5 mK over 10 min.

Fitting the coefficients

IEC 60751 gives the Callendar–Van Dusen equation and nominal coefficients for any Pt100; an individual sensor deviates from them, and the rig fits its own R_0, A and B (for $t \geq 0$ °C, $R = R_0(1 + At + Bt^2)$) by linear least squares in R_0, R_0A and R_0B. The listing simulates one sensor and the rig's measurement noise, fits it, and shows how far the nominal curve would have been off.

```
 import numpy as np

A0, B0 = 3.9083e-3, -5.775e-7                # IEC 60751 coefficients (t >= 0 degC)
rng = np.random.default_rng(3)

def r_cvd(t, r0, a, b):
    return r0*(1 + a*t + b*t*t)

def t_from_r(r, r0, a, b):                    # inverse of the quadratic
    return (-a + np.sqrt(a*a - 4*b*(1 - r/r0)))/(2*b)

# Unit under test (unknown to the rig) and the comparison run
true = (100.031, 3.9050e-3, -5.79e-7)
t_ref = np.array([0.0, 25, 50, 75, 100, 125])           # reference PRT readings, degC
t_act = t_ref + rng.normal(0, 0.010, t_ref.size)        # block gradient + reference unc.
r_meas = r_cvd(t_act, *true) + rng.normal(0, 0.0005, t_ref.size)   # 4-wire readout

# Linear least squares in (R0, R0*A, R0*B)
M = np.column_stack([np.ones_like(t_ref), t_ref, t_ref**2])
c0, c1, c2 = np.linalg.lstsq(M, r_meas, rcond=None)[0]
fit = (c0, c1/c0, c2/c0)
resid_mK = 1e3*(t_from_r(r_meas, *fit) - t_ref)
print(f"R0 = {fit[0]:.4f} ohm  A = {fit[1]:.5e}  B = {fit[2]:.3e}")
print("residuals (mK):", np.array2string(resid_mK, precision=1))

for t in (0.0, 60.0, 125.0):                  # error if IEC nominal coefficients were used
    err = t_from_r(r_cvd(t, *true), 100.0, A0, B0) - t
    tol = 0.15 + 0.002*abs(t)                 # IEC 60751 class A
    print(f"{t:5.1f} degC: nominal-curve error {err:+.3f} degC (class A +/-{tol:.3f})")
```

```
 R0 = 100.0336 ohm  A = 3.90333e-03  B = -5.679e-07
residuals (mK): [ 11.1 -24.3   9.4   6.3   1.   -3.5]
  0.0 degC: nominal-curve error +0.079 degC (class A +/-0.150)
 60.0 degC: nominal-curve error +0.047 degC (class A +/-0.270)
125.0 degC: nominal-curve error +0.006 degC (class A +/-0.400)
```

The fitted coefficients differ from the simulated truth in the fourth digit because six noisy points constrain three parameters; what matters is that the fitted curve reproduces the calibration points to within the residuals (13 mK standard deviation), and that the sensor, while inside class A, sits 0.079 °C off the nominal curve at 0 °C. Using the individual coefficients removes that bias for every later measurement. Figure 9.5 shows the residuals with their uncertainty.

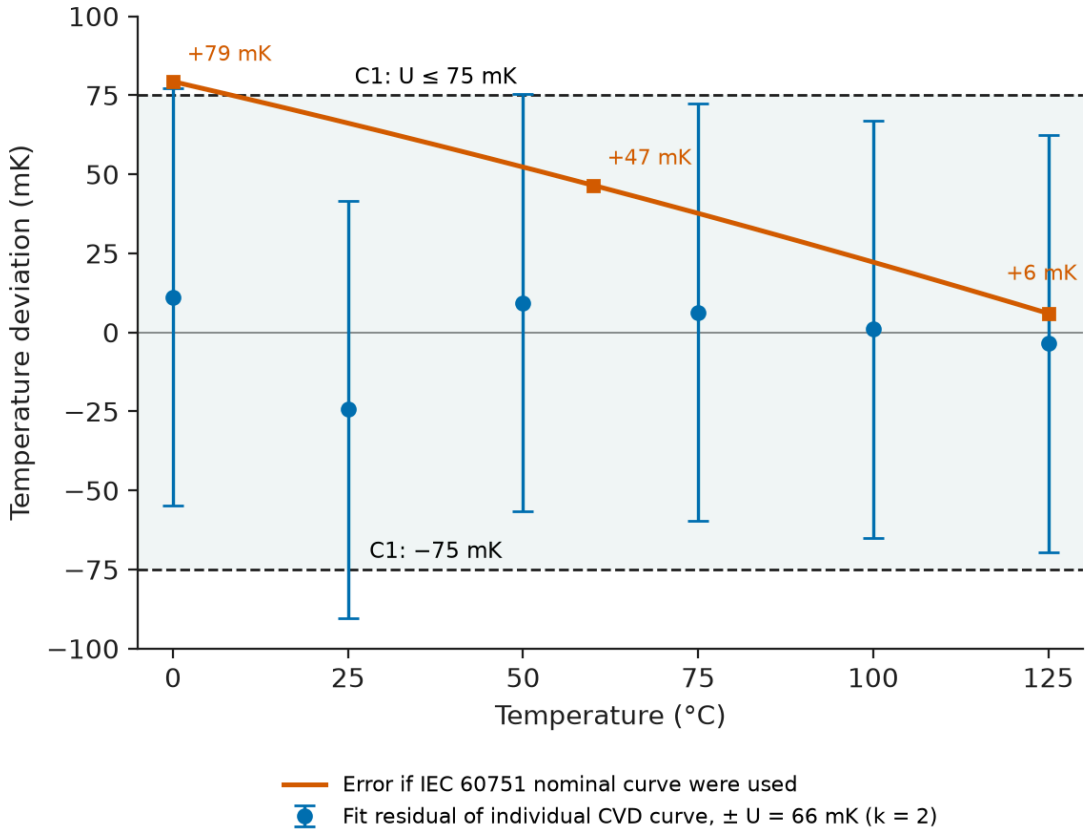


Figure 9.5 — PT100 calibration on the dry-block rig: residuals of the fitted Callendar-Van Dusen curve and deviation of the IEC nominal curve.

The rig's uncertainty

Table 9.5 — Uncertainty budget of the PT100 calibration rig (per point, 0–125 °C).

Contribution	Half-width or s	Distribution	Standard uncertainty
Reference PRT, from certificate (U = 0.020 °C, k = 2)	0.010 °C	Normal	0.0100 °C
Reference drift since calibration	0.010 °C	Rectangular	0.0058 °C
Block axial uniformity over sensing length	0.040 °C	Rectangular	0.0231 °C
Block radial (well-to-well) uniformity	0.020 °C	Rectangular	0.0115 °C
Immersion and stem conduction	0.010 °C	Rectangular	0.0058 °C
Readout ratio, 10 ppm of 100 Ω	0.0026 °C	Rectangular	0.0015 °C
Self-heating difference at 1 mA	0.005 °C	Rectangular	0.0029 °C
Repeatability (Type A, 10 readings)	—	Normal	0.0080 °C
Fit residual	—	Normal	0.0130 °C
Combined; expanded (k = 2)			**0.0328 °C; 0.066 °C**
C1 limit; margin			**0.075 °C; 12 %**

The block's axial uniformity is the largest term, which is typical of dry blocks and is why the reference PRT and the sensors under test are inserted to the same depth, and why the next upgrade is a stirred liquid bath, not a better readout. The ratio of the class A tolerance at 0 °C (±0.15 °C) to the rig's 0.066 °C is 2.3: enough to verify class A with guard bands, not enough to verify class AA (±0.10 °C) without a large rejection zone.

> **Status of the principle: Test uncertainty ratio and guard banding**
> *Empirical.* Practice (ILAC-G8, ANSI/NCSL Z540.3) treats a ratio of tolerance to expanded calibration uncertainty of 4:1 as making measurement uncertainty negligible in pass/fail decisions; below that, acceptance limits are tightened by a guard band derived from the uncertainty, trading false rejects for a stated probability of false accept.

9.7 Automated test equipment

> **Boundary.**
> Test plans, verification versus validation and regression testing are owned by Chapter 4; automated testing of software by Chapter 29. This section builds a production test station.

The strain-gauge module of Chapter 7 moves from prototype to a batch of 40 units. Testing each by hand with a bridge calibrator takes 40 minutes and depends on the operator; an ATE station must do it in two minutes (A1) and with a known repeatability (A2).

The station

The station consists of a bed-of-nails fixture that contacts the module's power, isolated-side test points and DAC output; eight **bridge simulators**, each a 350 Ω precision network with relay-switched shunts giving 0, ±1000 and ±3000 με equivalents (0.005 % resistors, calibrated against a resistance bridge traceable through the lab's standard resistor); a programmable power supply with current readback; a 6.5-digit DMM on a relay matrix; and a PC running the test sequencer. Instruments are controlled over LXI with SCPI commands (with PyVISA as an optional dependency); the module under test streams its readings over its own Ethernet port, as it will in service. The hipot test of R10 runs on a separate safety-interlocked station.

Table 9.6 — Test-time budget per strain module (A1: ≤ 120 s).

Step	Time	Note
Load, clamp, scan serial number	10 s	Operator
Firmware programming and boot check	30 s	SWD, verify CRC
Supply current, excitation voltages	4 s	DMM via matrix
Zero and gain at ±1000, ±3000 με, all 8 channels in parallel	8 s	5 points × (0.5 s settle + 0.5 s average), plus 3 s relay switching
Noise record, 10 s	10 s	All channels at once
Shunt-cal check	2 s	Built-in relay
DAC output at −10, 0, +10 V	3 s	DMM
Report, label, unclamp	8 s	Database write
Total / limit / margin	**75 s / 120 s / 38 %**	

Running the channels in parallel is what makes the budget: the module samples all eight simultaneously, so testing them one at a time would add about two minutes for no information.

Limits, guard bands and the sequencer

Each measurement has a specification limit and a measurement uncertainty. The station applies **guard-banded acceptance**: a value passes only if it is inside the limits shrunk by the expanded uncertainty of that measurement; a value between the guard-banded and the specification limit is **marginal** and triggers a retest, and only a value outside the specification is a fail. The listing is the core of the sequencer, with a simulated station standing in for the instruments; channel 6 of this unit has been given a gain error of +0.06 % to show the marginal path.

```
 import numpy as np
from dataclasses import dataclass

@dataclass
class Step:
    name: str
    lo: float
    hi: float
    u_meas: float          # expanded uncertainty of this measurement (k = 2)
    unit: str

    def judge(self, x):
        lo_g, hi_g = self.lo + self.u_meas, self.hi - self.u_meas   # guard-banded limits
        if lo_g <= x <= hi_g:
            return "PASS"
        return "FAIL" if (x < self.lo or x > self.hi) else "MARGINAL"

class FakeStation:                       # stands in for SMU, DMM and bridge simulator
    rng = np.random.default_rng(11)
    def measure(self, step, ch):
        nominal = {"I_supply": 182.0, "V_exc": 5.0000, "gain_err": 0.0, "noise": 0.08}[step]
        spread = {"I_supply": 6.0, "V_exc": 0.0006, "gain_err": 0.035, "noise": 0.02}[step]
        return nominal + self.rng.normal(0, spread) + (0.06 if (step, ch) == ("gain_err", 6) else
0)

plan = [Step("I_supply", 150, 220, 1.0, "mA"), Step("V_exc", 4.998, 5.002, 0.0003, "V"),
        Step("gain_err", -0.10, 0.10, 0.02, "%"), Step("noise", 0.0, 0.25, 0.02, "ue rms")]
st, verdicts = FakeStation(), []
for s in plan:
    for ch in (range(8) if s.name in ("gain_err", "noise") else [0]):
        x = st.measure(s.name, ch)
        verdicts.append(v := s.judge(x))
        if v != "PASS":
            print(f"{s.name:9s} ch{ch}: {x:+.4f} {s.unit} -> {v} (limits {s.lo}..{s.hi})")
print(f"{len(verdicts)} measurements, unit result:",
      "PASS" if all(v == "PASS" for v in verdicts) else "RETEST/REJECT")
```

```
 gain_err  ch6: +0.0861 % -> MARGINAL (limits -0.1..0.1)
18 measurements, unit result: RETEST/REJECT
```

Worked example — How many good units does the guard band reject? The gain error of production units has a spread of about 0.035 % (one standard deviation, from the first 20 units) around zero, and the guard band narrows the ±0.10 % limit to ±0.08 %. The fraction of good units falling between 0.08 % and 0.10 % on either side is $2[\Phi(0.10/0.035) - \Phi(0.08/0.035)] = 2(0.99786 - 0.98886) = 1.8\ \%$, where Φ is the standard normal distribution; these are retested, and most pass on the second measurement because part of the spread is measurement noise. Without the guard band the station would accept units whose true error is up to 0.12 % with a probability that cannot be stated. The gauge R&R study (10 units, 3 repeats, 2 operators) gave a measurement-system standard deviation of 0.006 %, a %GRR of $6 \times 0.006/0.20 = 18\ \%$ of the tolerance, within A2.

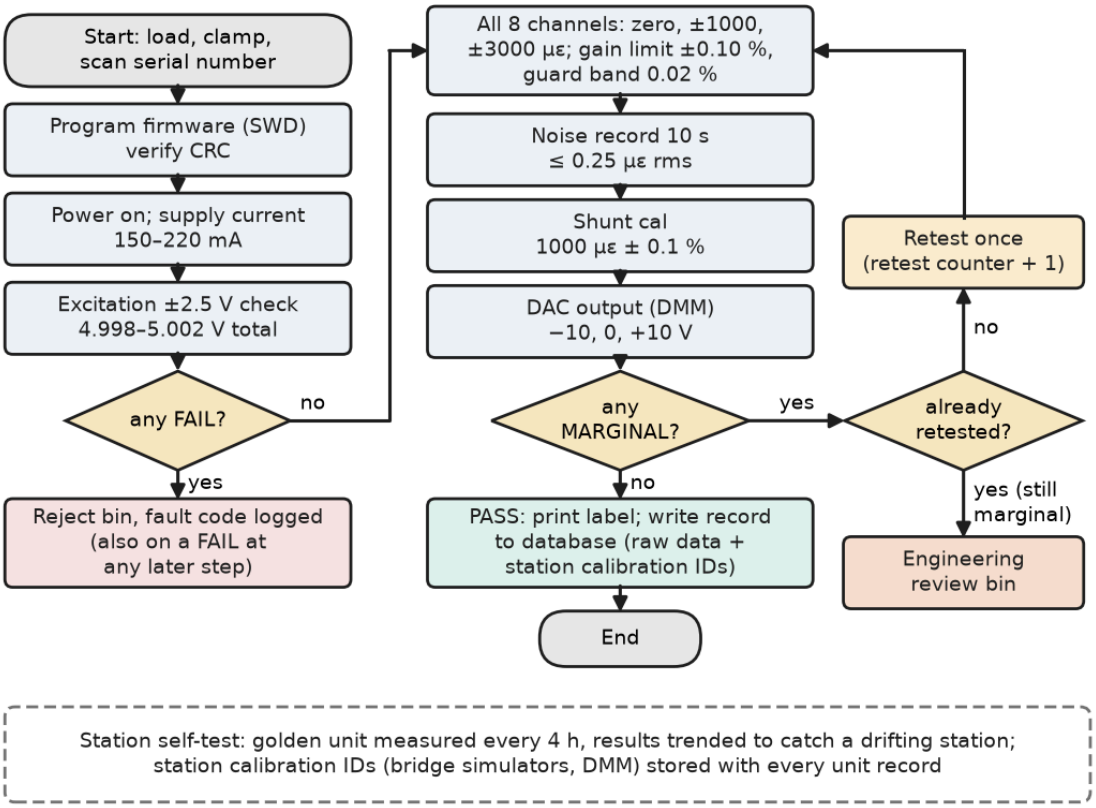


Figure 9.6 — Test flow of the ATE station for the strain-gauge module.

The sequence in Figure 9.6 adds two practices that make ATE trustworthy over months: a **golden unit** measured every four hours, whose results are trended to catch a drifting station, and the station's own calibration IDs written into every unit's record, so that if a bridge simulator is later found out of calibration, every affected unit can be identified.

9.8 Laboratory data-acquisition systems

> **Boundary.**
> DAQ architectures, synchronization, logging and storage are owned by Chapter 8; device connectivity, MQTT and provisioning by Chapter 11; scientific data management by Chapter 30. This section integrates the laboratory's instruments.

Eight instruments built by different people become a laboratory system only when they share four things: **time**, **naming**, **storage** and **automation**.

- **Time.** The lab runs one GNSS-disciplined PTP grandmaster. Instruments with sub-millisecond needs (the dynamometer DAQ, the battery monitor's current–voltage alignment across instruments) use PTP; the others use NTP served by the same grandmaster, and every record carries its time source and estimated accuracy (L1).
- **Naming and metadata.** Every channel has a unique identifier of the form `lab/chamber2/tc07`, a unit, a sensor serial and a calibration record linked by that serial, so a Section 9.6 recalibration propagates to every later reading.

- **Storage.** Slow data (temperatures, energy, PQ aggregates, battery monitor) go to a time-series database; waveforms and dynamometer windows to the file archive of Chapter 8; test results to a relational table keyed by test ID.
- **Automation.** Tests are scripts: a Python test executive sets chamber temperatures over Modbus TCP, commands cyclers and power sources over SCPI, arms the DAQ and records which instrument, firmware and calibration produced each file.

Worked example — How much data does the lab produce? The slow instruments produce, per day: chamber thermocouples 3 × 16 channels × 1 Sa/s × 8 bytes, 33 MB; bench energy meters 12 × 6 quantities × 1/s × 8 bytes, 50 MB; the PQ logger about 150 quantities at five 200 ms values per second × 4 bytes, 259 MB; four battery monitors with 21 channels at 10 Sa/s × 8 bytes, 581 MB. The total, about 0.92 GB/day or 340 GB/year, is small. Two dynamometer tests a week at 62.6 GB each add 6.5 TB/year. The archive design is therefore driven entirely by waveforms, and the decimation and windowing decisions of Chapter 8 are worth more than any database tuning.

The instrument network is a separate VLAN with no route to the internet, and instruments that cannot authenticate (most bench instruments) are reachable only from the test-automation server; the energy meters and battery monitors, which publish over MQTT, use per-device credentials as in Chapter 11.

9.9 Verification and failure analysis

Table 9.7 — Verification of the instrument suite.

ID	Test	Limit	Result	Pass/fail
T1	16 channels vs reference PRT at −20, 25, 60 °C, lid closed 4 h	±1.0 °C	Worst −0.42 °C	Pass
T2	Heated junction stepped to 65 °C	≤ 2 s; works with server down	1.2 s worst; yes	Pass
E1	Class 1 test points vs class 0.1 reference meter	±1.0 % (±1.5 % at 0.05 A)	Worst +0.41 % (PF 0.5, 0.1 A)	Pass
S1	Frequency source at 300, 1200, 6000 rpm equivalents; reference encoder	±0.1 %	0.004 % (timer chain); 0.06 % worst (with encoder)	Pass
Q1	Calibrator, 230 V with 4 % H5, 49.5–50.5 Hz	±0.5 % RMS; IEC 61000-4-7 class II harmonics	RMS +0.12 %; H5 within 0.6 % of reading	Pass
Q2	Programmable sags to 70 % for 100 ms	Detected; ±10 ms	All detected; timestamps ±3 ms	Pass
B1	16 cells vs 6.5-digit DMM at 2.8, 3.3, 3.6 V	±2 mV	Worst 1.3 mV	Pass
B2	Reference DCCT at 10, 100, 300 A, both directions	±0.1 % ±20 mA	Worst 0.06 % at 300 A, warm	Pass
B3	24 h at zero current with auto-zero	≤ 0.1 Ah	0.031 Ah	Pass
C1	Budget (Table 9.5); bilateral comparison with accredited lab	U ≤ 0.075 °C; $\lvert E_n \rvert \le 1$	0.066 °C; E_n = 0.4	Pass
A1	20 timed runs; 3 seeded faulty units	≤ 120 s; all faults caught	74–79 s; 3/3	Pass
A2	Gauge R&R, 10 × 3 × 2	%GRR ≤ 30 %	18 %	Pass
L1	Audit of one week of records	Every record has time source and calibration link	100 %	Pass

Table 9.8 — FMEA across the instrument suite (extract).

Instrument	Failure mode	Effect	Cause	Detection	Mitigation
Temperature	CJC not at junction temperature	All channels offset by up to 2 °C	Reference sensor on warm PCB	Lid-closed comparison test	Isothermal block with PT1000; block-gradient RTD
Temperature	Thermocouple open	Reading drifts or rails	Broken wire at cell	Burnout current, out-of-range flag	Alarm treats open as hot (fail-safe)
Energy	CT phase error	Error grows at low power factor	Uncalibrated phase path	PF 0.5 test point	Phase register calibrated per meter
Speed	Encoder pitch error	±2 % ripple in single-interval speed	T method on uneven lines	Reference-encoder comparison	M/T over ≥ 40 lines
Power quality	Fixed window at off-nominal frequency	Fictitious even harmonics, THD high	No window synchronization	Calibrator at 49.8 Hz	Resampling to 10 cycles
Battery	Sense lead short	Arc or fire at cell tap	Chafed harness	Cell voltage implausible	Tap fuse and 100 Ω resistor at every lead
Battery	Coulomb drift	SoC reference wrong after days	Offset integration	24 h zero-current test	Auto-zero at rest steps
Calibration	Block gradient underestimated	Certificates optimistic	Different immersion of reference and UUT	Two references swapped between wells	Equal immersion; liquid bath for Revision B
ATE	Station drift	Good units failed or bad passed	Bridge simulator relay contact wear	Golden unit every 4 h	Relay replacement schedule; calibration IDs in records

What remains open

Open question	Where it is answered
How is the bench energy meter designed in full, with firmware, communications, calibration and safety?	Chapter 35
How are battery packs, BMS functions and state estimation designed around the measurements of Section 9.5?	Chapter 23
How is the speed measurement closed into a motor speed loop?	Chapter 15, Chapter 37
How are microcontroller selection, firmware architecture and fault handling done for instruments like these?	Chapter 10
How do the instruments connect securely to a server over MQTT?	Chapter 11
How are power-quality limits set and harmonics mitigated?	*Electrical Energy and Machines*, Ch. 25
How is a whole project's test report and revision history organised?	Chapter 5, Chapter 40

Next: Chapter 10 opens Part III by taking the microcontroller inside every one of these instruments as its subject: how it is chosen, how its firmware is structured, how its timing and power are budgeted and how it is debugged when the system around it misbehaves.

Part III — Embedded and Connected Systems

Part II built measurement chains that end in a number. This part puts a processor at the end of the chain and makes it act: drive a motor, join a network, and finally decide on its own what its sensors are telling it. Its three systems are a battery-powered radiator valve, a cold-chain monitoring fleet for 120 pharmacies, and a vibration node that classifies a water pump's condition. None of their field failures came from a broken part. A valve that emptied its batteries overnight, an alarm that never reached anyone and a classifier that called healthy pumps unbalanced all failed in the joins, at an assumption about a battery, a connection or a scale factor that was never written down and tested. Each chapter carries its system from requirements through budgets, component choice and implementation to verification, failure analysis and the revision the failures forced.

By the end of this part you should understand

- How a microcontroller is chosen by eliminating candidates on hard constraints before comparing the rest, and how flash and RAM are budgeted for updates and growth.
- How sensors and an actuator are interfaced so that the faults that matter are detected, with safety-relevant reactions in hardware.
- How firmware is partitioned between hardware, interrupts and threads, and how an end-to-end latency budget is written across all of them.
- Why battery life is a charge budget but brownout is a source-resistance problem, and how both are tested at the corners where they meet.
- How a fleet is partitioned between device, gateway and cloud, and how connectivity and payloads are chosen against a data budget.
- What MQTT and HTTP each guarantee, and how an alarm path is made to survive silent network failures.
- How devices are provisioned, authenticated and authorised, and kept from collapsing a broker when they all reconnect at once.
- When computation belongs at the edge, and how a learned model is deployed, quantised, validated on held-out machines and guarded against inputs it has never seen.

Chapters

Chapter	Central question	What it delivers
10 — Embedded-System Integration	How are an MCU, sensors, an actuator, a radio and a battery integrated into a device whose timing, energy and faults are designed rather than discovered?	MCU selection, memory budget, actuator and sensor interfaces, firmware partition and state machine, stall-latency budget, charge budget and brownout model, FMEA, HIL rig and verification, Revision B
11 — IoT Systems	How is a fleet of connected devices designed so that data and alarms arrive complete, on time and authenticated across links that fail silently?	Device–gateway–cloud architecture, connectivity choice, telemetry and data budget, MQTT and HTTP in practice, store-and-forward gateway, three-tier storage, alarm-latency budget, provisioning, mutual-TLS authentication, threat model, reconnection-storm analysis, availability and verification
12 — Edge Computing and Embedded AI	When should data become decisions on the device, and how is a learned model fitted, validated and made safe on a microcontroller?	Edge-versus-cloud analysis, platform selection, preprocessing contract, sensor fusion and k-of-n voting, model deployment with schema checks, int8 agreement testing, cycle and energy budgets, grouped validation and shadow mode, layered failure handling and verification

How this part connects to the rest of the book

Part III applies the method of Chapters 1–5 to devices fed by the sensors and conditioning of Chapters 6–8. Forward, the integration practice of Chapter 10 is reused by the robot controllers of Chapters 15–17 and by the projects in Chapters 34, 35, 37 and 38. The connectivity and security of Chapter 11 reappear in the energy monitor of Chapter 35 and the sensor network of Chapter 38. Chapter 12 is the deployment half of the machine learning that Chapter 32 trains, and it shares its pump fleet with the signal-processing chain of Chapter 27. Radio front ends and link budgets are owned by Chapter 26, and industrial-network security by Chapter 21.

Among the sibling books, *Electronics and Computing* supplies almost all of the underlying theory: embedded architecture and MCU selection (Ch. 25), interfaces (Ch. 26), real-time scheduling and watchdogs (Ch. 27), firmware design and HIL testing (Ch. 28), and quantised inference on embedded hardware (Ch. 35). *Circuits, Signals and Communications* supplies sampling, the DFT and filters (Ch. 19, Ch. 22) and wireless propagation (Ch. 30). *Electrical Energy and Machines* supplies battery ratings (Ch. 16) and the motor characteristics behind the valve actuator and the pump (Ch. 5, Ch. 6). *The Mathematics Universe* supplies the probability behind availability, voting and confidence bounds (Ch. 19, Ch. 20).

Chapter 10 — Embedded-System Integration

Part III · Embedded and Connected Systems

A manufacturer of smart radiator valves ships 40,000 units in October. In late November the returns start: two AA cells flat after six weeks instead of the two heating seasons on the box. Every returned unit passes bench test with fresh cells. The cause appears only when a unit is put in a 0 °C chamber with cells two-thirds used. When the valve drives into its end stop, the stalled motor pulls the rail from 2.5 V to 2.17 V. The power-fail comparator, set at 2.30 V to protect flash writes, had a handler that reset the processor "to be safe". The reset lost the position held in RAM, so the valve homed by driving into the end stop again, which reset it again. The loop ran at about 60 mA until the cells were empty, within a day. No component was faulty. The failure lay in the joins: an unwritten battery model for a cold hallway, an interrupt handler with the wrong policy, position kept in the wrong memory, and a test plan that never put a half-used cell in a cold room. This chapter integrates that device, a battery-powered smart thermostatic radiator valve (TRV), from microcontroller choice to the second revision.

> **How are a microcontroller, sensors, an actuator, a radio and a battery integrated into a device whose timing, energy and fault behaviour are designed rather than discovered? What has to be measured, injected and recorded before an embedded product can be trusted to run unattended for years?**

Concept	Engineering destination
Microcontroller selection	A part that fits the peripherals, radio, memory, sleep current and supply range of the product, with margin
Sensor and actuator interfaces	Shunt, ADC, encoder and driver circuits sized against the signal and the fault they must detect
Firmware architecture	An event-driven state machine on an RTOS, with the safety-relevant paths in hardware
Timing constraints	A stall-detection chain with a written latency budget and measured margin
Power management	An average-current budget that predicts battery life, and a load-aware supply model that predicts brownout
Fault handling	A defined safe state, a reset policy, persistent state and a fault log for every failure the FMEA lists
Communication interfaces	I²C, SPI, UART and an IEEE 802.15.4 radio, each with a stated rate, owner and power cost
Debugging and testing	SWD trace, current profiling, a hardware-in-the-loop rig with fault injection, and a verification table

Roadmap. The reference valve and its requirements (10.1) → choosing the microcontroller (10.2) → the sensor and actuator interfaces (10.3) → firmware architecture (10.4) → timing constraints and the stall budget (10.5) → power management and the brownout model (10.6) → fault handling and the FMEA (10.7) → communication interfaces (10.8) → debugging, testing and Revision B (10.9).

10.1 The reference system: a battery-powered radiator valve

> **Boundary.**
> The embedded architecture this chapter applies — system models, partitioning, MCU selection criteria, memory and power constraints, firmware structures, boot and update — is set out in *Electronics and Computing*, Ch. 25; the bus protocols in Ch. 26; scheduling, latency and watchdogs in Ch. 27; drivers, state machines, low-power practice, debugging and HIL testing in Ch. 28. This chapter does not repeat them; it takes one product through those decisions. Requirements practice is from Chapter 1 and architecture documentation from Chapter 2.

A TRV replaces the wax-capsule head of a radiator valve: a DC gear motor turns a lead screw that pushes the valve pin through its 4.5 mm stroke, a sensor measures the room, and a radio exchanges setpoints, temperature, position and battery state with a building hub. It runs on two AA alkaline cells from 0 °C to about 50 °C at the head, unattended for at least two years. It contains every integration problem of a small embedded product: an actuator drawing fifty thousand times the sleep current, a sensor that its own electronics can heat, a radio that must be reachable yet nearly always asleep, a battery whose resistance changes threefold with temperature and charge, and an end stop found by stalling the motor against it.

Table 10.1 — Requirements for the reference TRV.

ID	Requirement	Target	Verification
R1	Room-temperature measurement	±0.3 °C, 15–30 °C, sensor shielded from the motor and the hot valve body	Chamber test against a calibrated PT100 reference
R2	Valve-position accuracy	±0.05 mm over the 4.5 mm stroke, repeatable after reset	Dial-gauge fixture, 200 moves
R3	Stall protection	Motor de-energised ≤ 20 ms after stall on the primary path, ≤ 100 ms on the backup path	Locked-rotor HIL test, oscilloscope on motor current
R4	Battery life	≥ 2 years on 2 × AA alkaline at 48 moves per day, margin ≥ 25 %	Charge per event measured, budget computed
R5	Supply range	Full function from 3.2 V down to 2.2 V loaded rail, 0–50 °C; no reset caused by the motor	Programmable source with series resistance, 0 °C chamber
R6	Command latency	Setpoint from hub applied ≤ 10 s after it is sent	Network test, 500 commands
R7	Safe state	On any unrecoverable fault: valve parked at the frost-protection position (1.0 mm open), fault reported	Fault-injection campaign, Table 10.10
R8	Recovery	Watchdog recovery to normal operation ≤ 2 s; no homing after a reset if position is known	Forced-reset test
R9	Firmware update	Signed over-the-air update with rollback; device never unbootable	Power-cut during update, 50 trials
R10	ESD robustness	±8 kV contact discharge on buttons and housing with no reset, IEC 61000-4-2 level 4	Accredited ESD test

Each requirement implies an interface or a budget: R3 a fast current path, R4 and R5 a battery that is modelled and not just counted, R8 position that survives a reset, R7 a frost position reachable on a nearly empty battery.

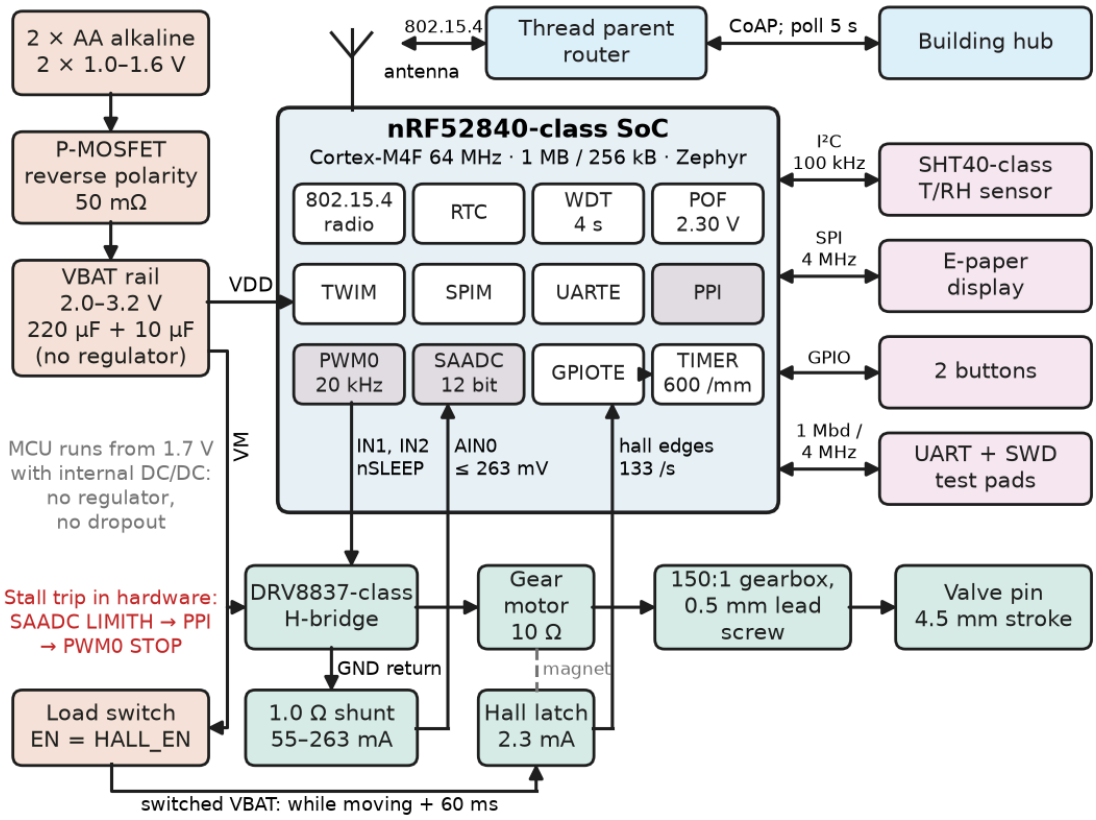


Figure 10.1 — Architecture of the reference smart radiator valve, with the interfaces between blocks.

In the architecture of Figure 10.1 everything sits on one battery rail with no regulator: the MCU runs from 1.7 V and has an internal DC/DC converter, so a regulator would only add quiescent current and dropout. Motor, driver and shunt form the actuator chain; a hall latch forms the position chain; the temperature sensor sits at the front, away from motor and valve body. The 802.15.4 radio runs Thread as a *sleepy end device* that polls its parent router for queued messages.

10.2 Microcontroller selection

> **Boundary.**
> The selection criteria (peripherals and pins, compute, memory, energy, security, qualification, ecosystem) and a table of representative parts are in *Electronics and Computing*, Ch. 25 §25.3. Here they are applied to one product, with the numbers that decide it.

Requirements that eliminate candidates

Three requirements remove most microcontrollers before any comparison of cores. The radio: the building system uses IEEE 802.15.4 (Thread), so the MCU contains that radio or pairs with a transceiver that adds a

second sleep current, an SPI link and a second firmware image. The supply range: R5 requires full function on a 2.2 V loaded rail, in practice to 2.0 V, with no regulator. The sleep floor: the budget of Section 10.6 allows about 3 µA for everything always on, including RAM retention and the RTC.

Table 10.2 — Microcontroller candidates for the TRV (typical datasheet values at 3 V, 25 °C; check current datasheets).

Candidate	Core, Flash / RAM	Radio	Supply range	Sleep with RTC and RAM	Radio RX / TX 0 dBm	Assessment
nRF52840-class SoC	Cortex-M4F 64 MHz, 1 MB / 256 kB	BLE 5, 802.15.4	1.7–5.5 V (VDDH)	1.5–3 µA	about 4.6 / 4.8 mA with DC/DC	Mature Thread stack, power-fail comparator, PPI event routing; **chosen**
EFR32MG24-class SoC	Cortex-M33 78 MHz, 1.5 MB / 256 kB	BLE 5, 802.15.4	1.71–3.8 V	about 1.3 µA (EM2)	about 5 / 5 mA	Close second; larger flash makes A/B updates easier; less team experience
ESP32-C6-class SoC	RISC-V 160 MHz, external flash / 512 kB	Wi-Fi 6, BLE, 802.15.4	3.0–3.6 V	about 7 µA deep sleep	tens of mA	Fails R5: needs a regulated 3.0 V or more
STM32L0 + separate 802.15.4 transceiver	Cortex-M0+ 32 MHz, 192 kB / 20 kB	external	1.65–3.6 V	about 1 µA + transceiver	per transceiver	20 kB RAM cannot hold a Thread stack with margin

The nRF52840-class part is chosen, although it does not have the lowest sleep current. It passes every filter with margin and has three integration features: a power-fail comparator (POF) that warns before the rail is too low for a safe flash write; a programmable peripheral interconnect (PPI) that lets one peripheral's event trigger another's task without the CPU (the stall path of Section 10.5); and an ADC (SAADC) with hardware limit comparators. The EFR32MG24 has equivalents and would also have worked; the team's existing Thread code decided.

> **Status of the principle: Eliminate, then compare**
> *Empirical.* Selecting on hard constraints first (radio, supply range, sleep floor, memory for the stack) and only then on cost and convenience is established practice. It avoids choosing on a headline figure such as MHz or µA/MHz that does not bind for the product.

The memory budget

The firmware is Zephyr with OpenThread and a small CoAP application. A Matter layer was rejected: its image, around 700 kB, would not fit twice in 1 MB for A/B updates (R9) without external flash.

Table 10.3 — Flash and RAM budget (nRF52840-class, Zephyr + OpenThread, release build).

Region	Allocation	Used / estimated	Margin
MCUboot bootloader	48 kB	38 kB	21 %
Primary image slot	464 kB	340 kB	27 %
Secondary (update) slot	464 kB	340 kB	27 %
Settings / position journal (NVS)	32 kB (8 pages)	1 kB live data	—
Fault log	16 kB	4 kB per 64 records	—
Flash total	**1024 kB**		
OpenThread, 802.15.4 driver, buffers		54 kB	
Kernel, thread stacks (measured high-water + 25 %)		28 kB	
mbedTLS heap (commissioning, DTLS)		36 kB	
Application, FSM, display buffer		14 kB	
Logging (RTT) and retained-RAM block		9 kB	
RAM total	**256 kB**	**141 kB**	**45 %**

Worked example — Will an update still fit in two years? Firmware images of connected products typically grow 10–20 % a year as features and fixes are added. At 15 % per year the 340 kB image becomes $340 \times 1.15^2 = 450$ kB after two years, still inside the 464 kB slot but with only 3 % margin left. The 27 % release margin buys about two years of growth, not five, which is why CI records every build's image size (Chapter 29): the partition table cannot change in the field, because the bootloader is never updated over the air.

10.3 Sensor and actuator interfaces

> **Boundary.**
> Sensor principles and selection are in Chapter 6, signal conditioning and ADC selection in Chapter 7. The brushed DC motor's linear torque–current and speed–voltage characteristics are from *Electrical Energy and Machines*, Ch. 5 §5.8, and H-bridge operation is from *Electronics and Computing*, Ch. 32. Motor drivers and encoder interfaces for robots are developed further in Chapter 15.

The interface circuit of Figure 10.2 brings four signals to the MCU (motor current, shaft position, room temperature, battery voltage) and drives one actuator.

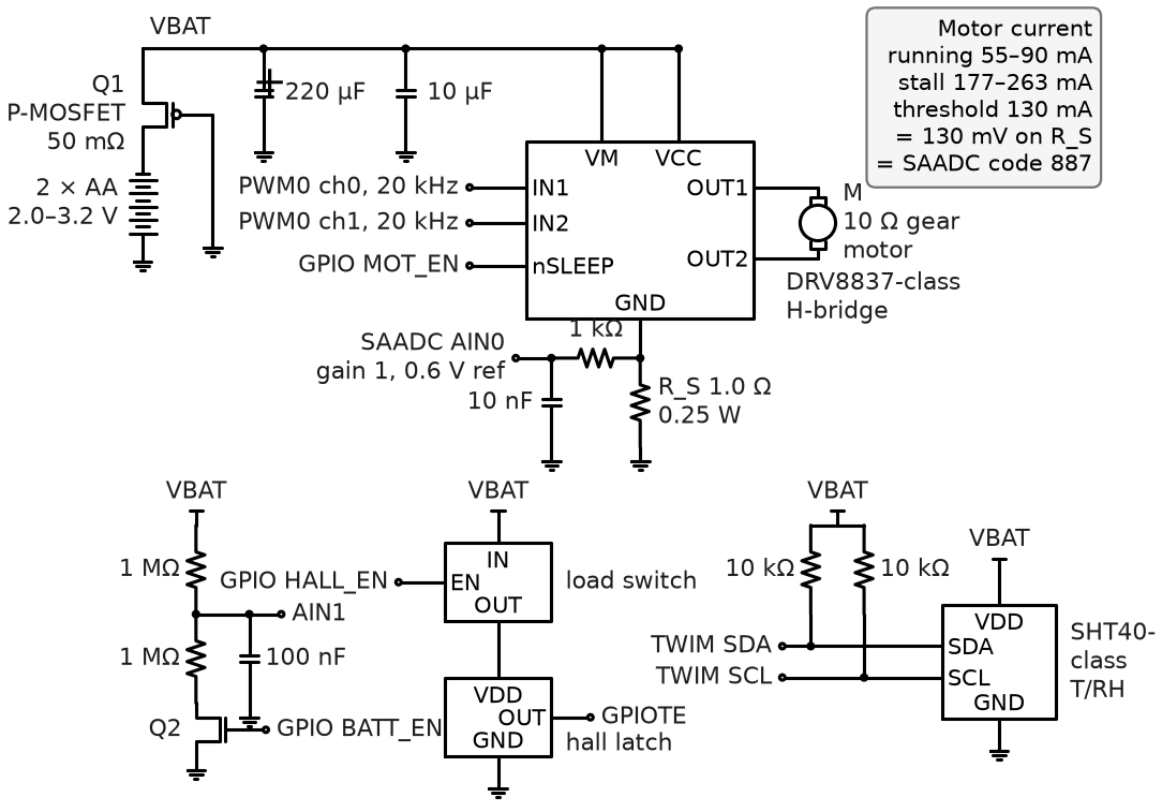


Figure 10.2 — Power path and actuator interface of the TRV.

Actuator: driver choice and a lesson about minimum voltage

The first choice of motor driver, a DRV8833-class dual H-bridge common on evaluation boards, has a motor-supply minimum of 2.7 V: it would stop driving long before the cells reach the 2.2 V of R5. The rule is to check every part's minimum operating voltage against the *loaded end-of-life* rail, not the nominal one. A DRV8837-class bridge was chosen instead (Table 10.4).

Table 10.4 — Actuator-chain component selection.

Function	Candidates	Deciding numbers	Choice
H-bridge	DRV8833 class (VM 2.7–10.8 V); DRV8837 class (VM 0–11 V, logic 1.8–7 V, 0.28 Ω); discrete N/P MOSFET bridge	Must drive at 2.0 V; sleep < 0.1 µA	DRV8837 class
Current sense	Low-side shunt + MCU ADC; high-side current-sense amplifier (INA180 class, about 80 µA when on)	Stall must be detected without a powered amplifier	1.0 Ω low-side shunt into SAADC
Position sensor	Hall latch on motor shaft (DRV5011 class, about 2.3 mA); slotted optical switch (LED 5–10 mA); none (time-based)	Count accuracy and current while moving	Hall latch, powered only while moving
Temperature sensor	10 kΩ NTC + ADC (±0.5 °C with calibration); SHT40 class digital (±0.2 °C typ, 0.08 µA idle); TMP117 class (±0.1 °C)	R1 ±0.3 °C, no calibration step in production	SHT40 class

Current sensing for stall detection

The motor is a 10 Ω brushed DC motor driving a 150:1 gearbox. At standstill only its winding resistance limits its current, so the start and stall current is

$$I_{\text{stall}} = \frac{V_{\text{OC}}}{R_{\text{src}} + R_{\text{bridge}} + R_{m} + R_{\text{shunt}}}$$

where V_{OC} is the battery open-circuit voltage and R_{src} the battery's internal plus contact resistance. Running against the valve load it draws 55–90 mA, rising as the pin compresses the valve spring near the closed position.

Worked example — Sizing the shunt and the stall threshold. Over the battery states of Section 10.6, the stall current ranges from 177 mA (cells at 1.15 V each, 0 °C) to 263 mA (fresh cells, 25 °C). The running current peaks at 90 mA when closing. A threshold of 130 mA sits between the two: it is 1.44 times the highest running current and 0.73 times the lowest stall current, a 27 % margin below the weakest stall. With a 1.0 Ω shunt the threshold is 130 mV. The SAADC is set to gain 1 with its internal 0.6 V reference and 12-bit resolution, so one code is $0.6/4096 = 0.146$ mV, or 0.146 mA, and the threshold is code 887. The shunt dissipates at most $0.263^2 \times 1.0 = 69$ mW in a stall and drops 55–90 mV while running, about 3 % of the motor voltage. A 0.5 Ω shunt would halve that loss but put the threshold at 65 mV, closer to the ADC offset and to ground-bounce errors. The input RC (1 kΩ, 10 nF, 10 µs) softens the 20 kHz PWM edges and is fast against the 0.5 ms sampling period.

The motor also draws stall current every time it *starts*, before it builds back-EMF; with a mechanical time constant near 10 ms the transient decays within 30–50 ms. The design ramps the PWM duty from 30 % to 100 % over 30 ms and **blanks** the detector for the first 50 ms of every move. The blanking leaves a hole: a valve already seized at the start is caught only by the backup encoder path of Section 10.5.

Position: counting hall edges

A two-pole magnet on the motor shaft and a hall latch give two edges per revolution. Through the 150:1 gearbox and 0.5 mm lead screw that is $2 \times 150/0.5 = 600$ counts per millimetre, so R2 (±0.05 mm) is ±30 counts. The risk is not resolution but *lost counts*, which accumulate until the next homing. Three details prevent them:

- The latch draws about 2.3 mA, so a load switch powers it only while moving, and for 60 ms *after* the stop, because the coasting rotor's edges are real travel.
- Edges are counted by a hardware timer fed through GPIOTE and PPI, not by an interrupt per edge. At 4,000 rpm loaded they arrive at 133 per second, but a radio critical section could still drop one; hardware counting removes the dependence.
- A single latch cannot sense direction, which is inferred from the command. That is safe only because the lead screw is self-locking, confirmed by pushing on the pin with 80 N.

Temperature and battery

The SHT40-class sensor sits on a flex at the front of the housing, a thermal decision: on the main board of the first prototype it read 1.6 °C high whenever the radiator was hot. The 2 × 1 MΩ battery divider is switched by a MOSFET, because left connected it would draw 1.5 µA, half the sleep budget. The battery is measured unloaded once an hour and *loaded* during the first 100 ms of each move; only the loaded value predicts the brownout of Section 10.6.

10.4 Firmware architecture

> **Boundary.**
> Super-loop, event-driven and RTOS structures and their worst-case response are compared in *Electronics and Computing*, Ch. 25 §25.6; layering, non-blocking drivers and run-to-completion state machines in Ch. 28 §28.1–28.4. The choices below apply them.

Structure

The radio stack decides the structure. OpenThread needs an RTOS or a carefully managed event loop, and its radio driver needs the highest interrupt priority to meet 802.15.4 acknowledgement timing (an ACK must start 192 µs after the frame ends). The firmware therefore runs on Zephyr with five threads and a strict rule about what runs where.

Table 10.5 — Firmware partition: what runs where.

Context	Content	Priority	Rule
Hardware (PPI, SAADC limit, PWM STOP)	Primary stall trip; hall edge counting	—	No CPU involvement; works even if firmware is stuck
Radio ISR and OpenThread thread	802.15.4 MAC, Thread, CoAP	Highest	Owned by the stack; no application code
ISRs (SAADC, GPIOTE, POF, RTC)	Capture data, post an event; nothing else	High	≤ 20 µs each; no blocking, no flash access
Control thread	Valve state machine, setpoint control	Above normal	Run-to-completion event handler, one event queue
Sensor and UI threads	Temperature read, buttons, e-paper	Normal	Non-blocking drivers with timeouts
Housekeeping thread	Fault log, position journal, battery, watchdog supervisor	Low	Only thread that writes flash

Two rules carry the design. **The safety-relevant reaction is in hardware** and firmware only confirms it, so nothing in software can delay the motor stop. **Only one thread writes flash**: a page erase stalls the CPU for tens of milliseconds and must not run below the power-fail threshold, and one low-priority writer can be gated by the power monitor without affecting control.

The valve state machine

The valve's behaviour is a six-state machine (Figure 10.3); the control thread feeds events from ISRs, radio and timers into one handler, whose core is listed below.

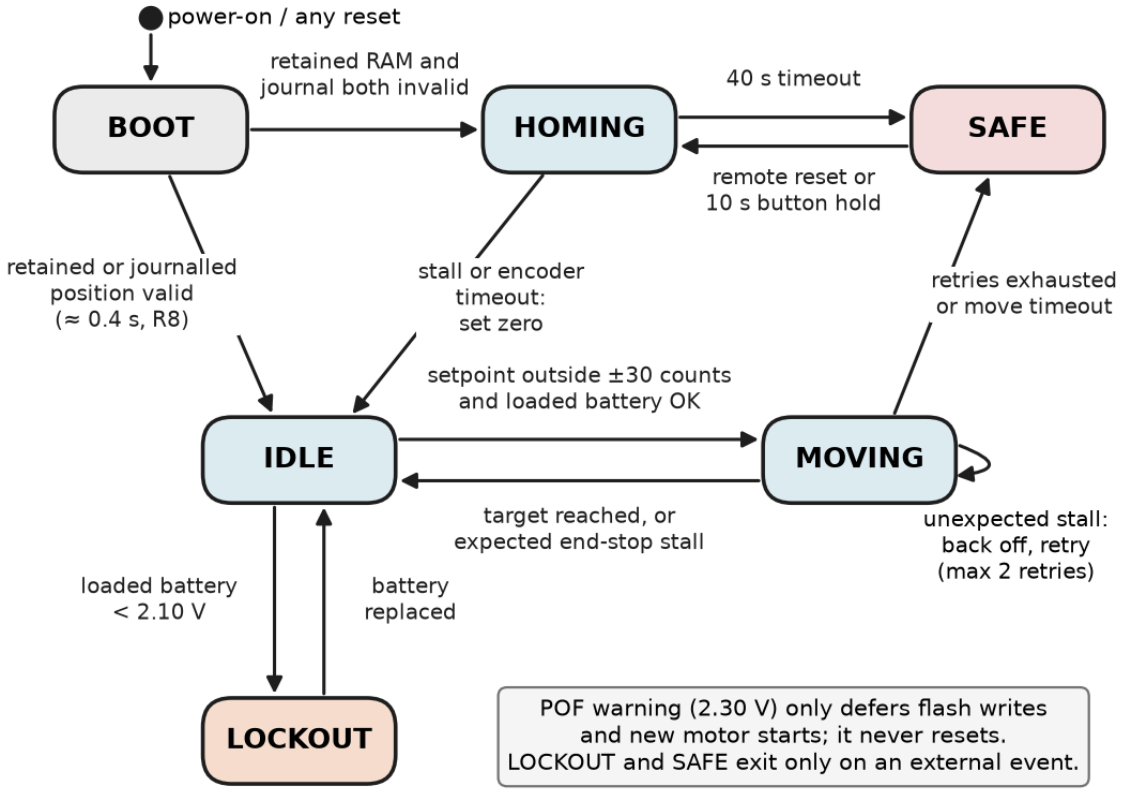


Figure 10.3 — Valve control state machine of the TRV after the Revision B changes.

```
 /* valve_fsm.c -- valve control state machine (C11; illustrative driver API) */
#include <stdint.h>
#include "valve_io.h"   /* motor_start(), motor_stop(), pos_counts(), pos_set_zero(),
                           batt_loaded_mv(), report_fault(), safe_park(),
                           event_t, fault_t, EV_*, F_*, DIR_*                    */

typedef enum { ST_BOOT, ST_HOMING, ST_IDLE, ST_MOVING, ST_LOCKOUT, ST_SAFE } vstate_t;

#define POS_TOL      30      /* counts, +/-0.05 mm at 600 counts per mm */
#define FROST_POS    600     /* 1.0 mm open                             */
#define V_LOCKOUT_MV 2100u   /* loaded-rail threshold                   */
#define MAX_RETRIES  2u

static vstate_t st = ST_BOOT;
static int32_t target;
static uint8_t retries;

static int32_t iabs32(int32_t v) { return v < 0 ? -v : v; }

static void go_safe(fault_t f)
{
    motor_stop();
    report_fault(f);
    safe_park(FROST_POS);           /* timed open-loop move, own stall trip   */
    st = ST_SAFE;
}

static void start_move(void)
{
    if (batt_loaded_mv() < V_LOCKOUT_MV) { report_fault(F_LOW_BATT); safe_park(FROST_POS); st =
ST_LOCKOUT; return; }
    motor_start(target > pos_counts() ? DIR_OPEN : DIR_CLOSE);  /* 30 ms soft start */
    st = ST_MOVING;
}

void valve_on_event(const event_t *e)
{
    switch (st) {
    case ST_BOOT:                   /* arg: position restored */
        if (e->id != EV_BOOT) break;
        if (e->arg) st = ST_IDLE; else { motor_start(DIR_CLOSE); st = ST_HOMING; }
        break;
    case ST_HOMING:
        if (e->id == EV_STALL || e->id == EV_ENC_TIMEOUT) { motor_stop(); pos_set_zero(); st =
ST_IDLE; }
        else if (e->id == EV_MOVE_TIMEOUT) go_safe(F_HOMING);
        break;
    case ST_IDLE:
        if (e->id == EV_SETPOINT && iabs32(e->arg - pos_counts()) > POS_TOL) {
            target = e->arg; retries = 0; start_move();
        }
        break;
```

```
    case ST_MOVING:
        if (e->id == EV_TARGET_REACHED) { motor_stop(); st = ST_IDLE; }
        else if (e->id == EV_STALL && iabs32(pos_counts()) <= POS_TOL) { motor_stop(); st =
ST_IDLE; }
        else if (e->id == EV_STALL || e->id == EV_ENC_TIMEOUT) {
            motor_stop();
            if (++retries > MAX_RETRIES) go_safe(F_STALL); else start_move();
        } else if (e->id == EV_MOVE_TIMEOUT) go_safe(F_MOVE_TIMEOUT);
        break;
    default:                        /* LOCKOUT, SAFE: exits on explicit commands, elided */
        break;
    }
}
```

A stall *at the closed end stop* is the expected end of a closing move; a stall anywhere else is a fault, retried twice (production code first backs off 0.2 mm to free a sticky pin) and then escalated to SAFE. The battery is checked before every start. LOCKOUT and SAFE leave only on explicit external events, so the device cannot oscillate between faulted and running, the loop that killed the Revision A batteries.

> **Status of the principle: Every fault state has an exit that needs a new fact**
> *Derived.* If a state machine can leave a fault state only on an event that the fault itself cannot generate, it cannot form a fault–recover–fault cycle without outside input. This follows from the transition structure and holds for any machine whose events are correctly classified.

10.5 Timing constraints

> **Boundary.**
> Deadlines, worst-case execution time, interrupt latency, jitter and response-time analysis are developed in *Electronics and Computing*, Ch. 27 §27.1–27.3 and §27.10. This section writes a latency budget across hardware and software for one safety function.

Which deadlines are hard

Two deadlines are hard: the 192 µs 802.15.4 acknowledgement turnaround, owned by the radio stack, and R3. A stalled gear motor delivers its stall torque, multiplied by 150, into the screw and end stop; gearbox tests showed tooth damage after about 400 stalls longer than 200 ms, and every homing and full close ends in one.

The stall-detection budget

Figure 10.4 shows a closing move that ends in the end stop. The detection chain runs from the mechanical event to the motor being de-energised, and each link has a worst-case time.

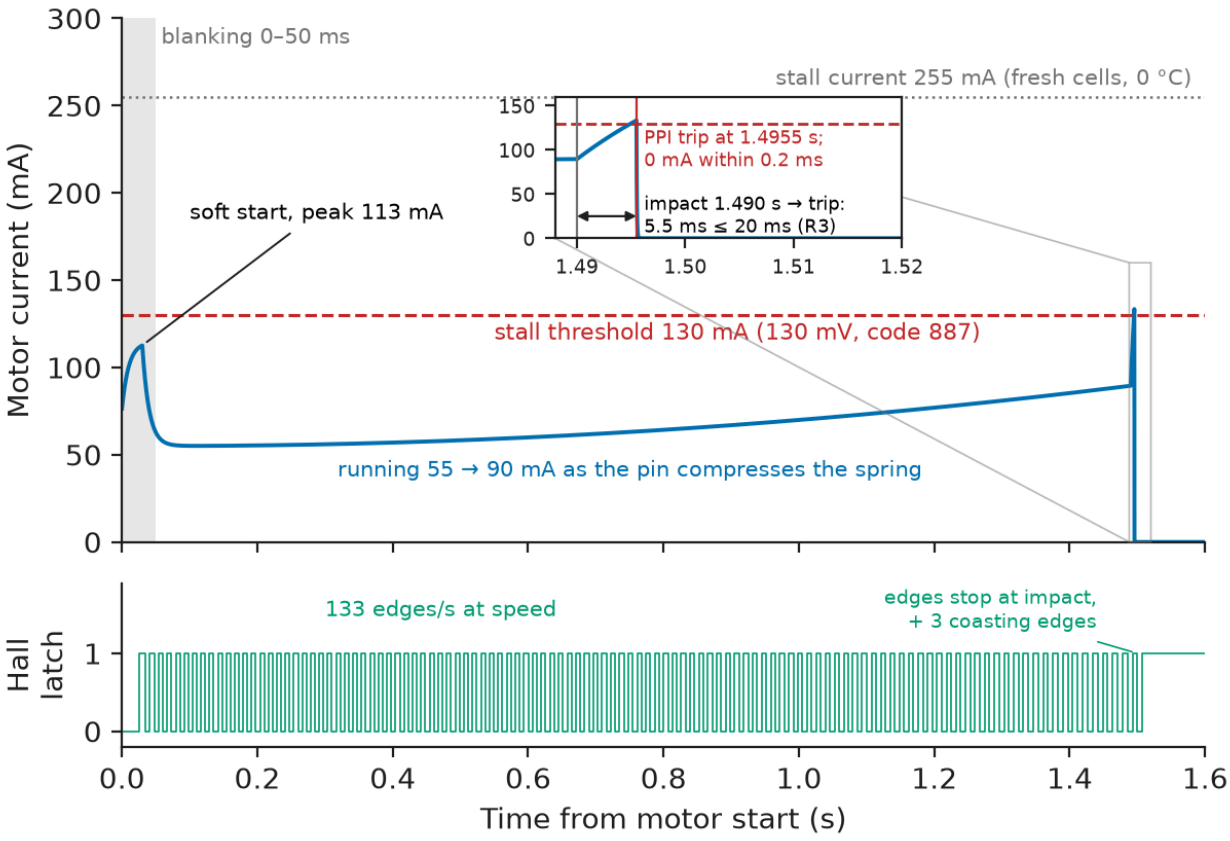


Figure 10.4 — Motor current during a closing move that ends at the end stop, with soft start, blanking window, stall threshold and hall edges.

Table 10.6 — Stall-detection latency budget (R3: 20 ms primary, 100 ms backup).

Link	Primary path (hardware)	Software-confirm path	Backup path (encoder timeout)
Current rises from running level to threshold after impact	5.0 ms	5.0 ms	—
Last hall edge to timeout	—	—	40 ms timeout + 7.5 ms edge period
SAADC sample period (timer-triggered via PPI)	0.5 ms	0.5 ms	—
Conversion (40 µs acquisition and conversion)	0.04 ms	0.04 ms	—
Confirmation filter (4 consecutive samples)	—	1.5 ms	—
Limit event → PPI → PWM STOP, or ISR latency incl. radio masking	< 0.001 ms	1.0 ms (assumed, then measured)	1.0 ms
H-bridge turn-off	< 0.001 ms	< 0.001 ms	< 0.001 ms
Worst-case total	**5.5 ms**	**8.0 ms**	**48.5 ms**
Limit	20 ms	20 ms	100 ms
Margin	**72 %**	**60 %**	**52 %**

The primary path never involves the CPU: a timer triggers the SAADC every 0.5 ms through PPI, the channel limit comparator raises LIMITH above code 887, and a PPI channel routes that event to the PWM STOP task. The software-confirm path takes the same event as an interrupt, checks the next samples and posts `EV_STALL`. If the shunt opened or the SAADC were misconfigured, the backup catches the stall: no hall edge for 40 ms (five nominal edge periods) while the motor is on.

```
/* stall_trip.c -- hardware stall trip: SAADC limit -> PPI -> PWM0 STOP
   (nRF52840 register names; illustrative, error handling omitted) */
#include <stdint.h>
#include "nrf.h"

#define STALL_CODE 887u      /* 130 mV / (0.6 V / 4096): 130 mA on the 1-ohm shunt */
#define PPI_CH_TRIP 3u

void stall_trip_configure(void)
{
    /* High limit = STALL_CODE, low limit = most negative value (disabled). */
    NRF_SAADC->CH[0].LIMIT = (STALL_CODE << SAADC_CH_LIMIT_HIGH_Pos) | 0x8000u;
    NRF_SAADC->INTENSET    = SAADC_INTENSET_CH0LIMITH_Msk;   /* software confirm path */
    NRF_PPI->CH[PPI_CH_TRIP].EEP = (uint32_t)&NRF_SAADC->EVENTS_CH[0].LIMITH;
    NRF_PPI->CH[PPI_CH_TRIP].TEP = (uint32_t)&NRF_PWM0->TASKS_STOP;
}

/* Called by the blanking timer 50 ms after each motor start, and on motor stop. */
void stall_trip_arm(int on)
{
    if (on) NRF_PPI->CHENSET = 1u << PPI_CH_TRIP;
    else    NRF_PPI->CHENCLR = 1u << PPI_CH_TRIP;
}
```

The entry "assumed, then measured" is how such a budget is used honestly: the radio stack masks application interrupts in short critical sections that no datasheet bounds, so the budget *assumes* 1.0 ms and Section 10.9 *measures* it (0.31 ms worst case over 10^6 samples under continuous Thread traffic).

> **Status of the principle: End-to-end latency is the sum of worst-case link latencies**
> *Derived.* For a chain of sequential stages, the worst-case end-to-end latency is bounded by the sum of each stage's worst case. The bound is conservative, because the worst cases rarely coincide. It is valid only if every stage is in the table, including the physical one.

Worked example — Why the software-only path was not enough in Revision A. Revision A detected stalls in a 1 kHz timer interrupt averaging 8 samples: 5 + 8 + 1 = 14 ms, apparently inside 20 ms. But its control thread also wrote a statistics record to flash at the end of each move, and a page erase blocks the CPU for up to 85 ms on this part class. When an erase coincided with an end-of-move stall, the measured stall lasted up to 97 ms. A budget that omits the flash stall is wrong, not just tight. The hardware trip and the single flash-writing thread each remove the problem on their own.

10.6 Power management

> **Boundary.**
> The average-current model of battery life, the power-mode ladder and supply integrity are in *Electronics and Computing*, Ch. 25 §25.5, and the firmware practices that make a budget true in Ch. 28 §28.5. Battery capacity and rate ratings are from *Electrical Energy and Machines*, Ch. 16 §16.2.

Power management here asks two separate questions, and Revision A answered only the first. **How long will the battery last** is about *charge*, answered by an average-current budget. **Will the device keep working as the battery ages** is about *source resistance and peak current*, answered by a load model of the battery.

The charge budget

The listing computes the average current from the charge per event, measured on prototypes with a current profiler, and converts it to life with 80 % usable capacity and 2 % per year self-discharge.

```
 # Average-current and life budget of the smart radiator valve (TRV)
DAY = 86400.0
loads = [  # name, current_A, duration_s, events_per_day
    ("sleep floor",       3.0e-6, DAY,   1),
    ("temperature read",  1.7e-3, 6e-3,  1440),
    ("Thread data poll",  5.5e-3, 3.5e-3, DAY / 5),
    ("status report",     6.0e-3, 18e-3, 288),
    ("valve moves",       70e-3,  1.5,   48),
    ("weekly exercise",   70e-3,  40.6,  1 / 7),
    ("display updates",   3.0e-3, 2.0,   10),
]
total = 0.0
for name, i, t, n in loads:
    ua = i * t * n / DAY * 1e6
    total += ua
    print(f"{name:<17s} {ua:6.2f} uA")
cap_mah, usable, sd = 2500.0, 0.80, 0.02          # 2 x AA alkaline in series
i_sd = cap_mah * sd / 8760 * 1e3                  # self-discharge, uA
life_h = usable * cap_mah / ((total + i_sd) / 1e3)
print(f"total load {total:.1f} uA + self-discharge {i_sd:.1f} uA")
print(f"life {life_h / 8760:.2f} years; motor share "
      f"{(loads[4][1]*loads[4][2]*loads[4][3] + 70e-3*40.6/7) / DAY * 1e6 / total:.0%}")
```

```
 sleep floor         3.00 uA
temperature read    0.17 uA
Thread data poll    3.85 uA
status report       0.36 uA
valve moves        58.33 uA
weekly exercise     4.70 uA
display updates     0.69 uA
total load 71.1 uA + self-discharge 5.7 uA
life 2.97 years; motor share 89%
```

Table 10.7 — Battery-life budget summary (R4).

Item	Value
Average load current	71.1 µA (motor 89 %, radio polling 5 %, sleep floor 4 %)
Self-discharge equivalent	5.7 µA
Usable capacity	2,000 mAh (80 % of 2,500 mAh at this load and 0–20 °C)
Predicted life	2.97 years
Requirement	≥ 2 years with ≥ 25 % margin
Margin	**49 %**

Worked example — Where the next microamp comes from. The motor is 89 % of the load. Halving the sleep floor to 1.5 µA, which would take a new MCU, buys 2 % of life; halving the number of moves by widening the controller's deadband buys about 60 %. Figure 10.5 shows life falling from 6.9 years at 12 moves per day to 2.97 years at 48 and 1.69 years at 96, crossing the requirement near 80. The heating-control algorithm therefore carries an electronics limit, at most 60 moves per day on average (25 % margin), written into the interface specification with the control team and checked by the HIL rig's 30-day replay of a real heating season.

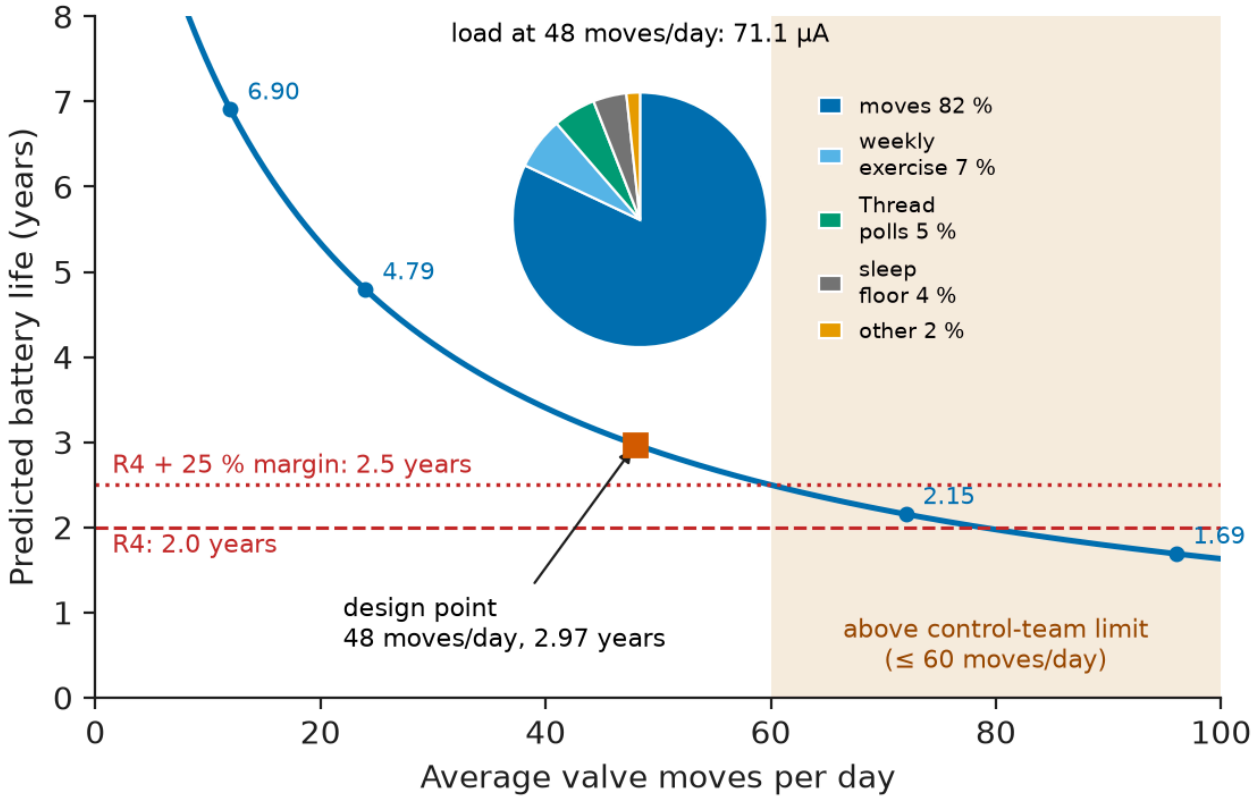


Figure 10.5 — Predicted battery life of the TRV against the average number of valve moves per day.

The load model and brownout

For brownout the battery is a voltage source with a resistance; the open-circuit voltage falls with discharge, and the resistance rises with discharge and roughly doubles from 25 °C to 0 °C for alkaline cells. Under a current pulse I the rail is

$$V_{\text{rail}} = V_{\text{OC}} - I\,R_{\text{src}}$$

and the largest I is the stall current. The listing computes the rail at stall for four states of charge at 25 °C and 0 °C and compares it with the 2.30 V threshold that Revision A treated as a reset.

```
 # Battery-rail dip at motor start or stall: 2 x AA alkaline, Rev A power path
R_MOTOR, R_BRIDGE, R_SHUNT, R_CONTACTS = 10.0, 0.28, 1.0, 0.10
POF_REV_A = 2.30          # V, power-fail threshold that Rev A treated as a reset

def r_cell(ocv, temp_c):
    """Representative AA alkaline internal resistance (ohm)."""
    r = 0.20 if ocv >= 1.45 else 0.30 if ocv >= 1.30 else 0.40
    return r * (2.0 if temp_c <= 0 else 1.0)

print(" OCV/cell  T(C)  R_src   I_stall   V_rail  Rev A")
for ocv in (1.55, 1.35, 1.25, 1.15):
    for temp in (25, 0):
        r_src = 2 * r_cell(ocv, temp) + R_CONTACTS
        i = 2 * ocv / (r_src + R_BRIDGE + R_MOTOR + R_SHUNT)
        v = 2 * ocv - i * r_src
        flag = "RESET" if v < POF_REV_A else "ok"
        print(f"  {ocv:4.2f}    {temp:3d}  {r_src:4.2f}  {i*1e3:6.0f} mA  {v:5.2f} V  {flag}")
```

```
 OCV/cell  T(C)  R_src   I_stall   V_rail  Rev A
  1.55     25  0.50     263 mA   2.97 V  ok
  1.55      0  0.90     255 mA   2.87 V  ok
  1.35     25  0.70     225 mA   2.54 V  ok
  1.35      0  1.30     215 mA   2.42 V  ok
  1.25     25  0.90     205 mA   2.32 V  ok
  1.25      0  1.70     193 mA   2.17 V  RESET
  1.15     25  0.90     189 mA   2.13 V  RESET
  1.15      0  1.70     177 mA   2.00 V  RESET
```

Worked example — Reproducing the field failure. At room temperature Revision A survives until the cells are nearly exhausted (1.15 V each), which is why bench tests passed. At 0 °C it fails at 1.25 V per cell, about two-thirds through the capacity, reached after a few weeks of heavy winter use. The loop then draws about 60 mA (homing, reboot, network rejoin), so the remaining $0.33 \times 2{,}500 = 825$ mAh lasts about $825/60 \approx 14$ h. The "six weeks" was the time to reach cold and two-thirds used; the loop emptied the cells overnight.

> **Status of the principle: Battery as an open-circuit voltage behind a resistance**
> *Approximate.* A Thevenin model with state-of-charge- and temperature-dependent V_{OC} and R_{src} predicts the dip under pulses of milliseconds to seconds. It ignores the cell's diffusion dynamics (the slow additional sag under long pulses), so it is used here with resistances measured at 100 ms pulse length on cells of the chosen brand at each temperature.

Revision B power policy

The fix broke the failure chain at four links:

1. **Power-fail warning changes behaviour, not state.** The POF handler now only blocks flash writes and new motor starts; the hardware brownout reset (about 1.7 V) is the only reset.
2. **Position survives reset**, in a CRC-protected retained-RAM block and a flash journal written after each move; homing happens only if both are invalid.
3. **Loaded-voltage lockout.** If the rail measured early in the previous move was below 2.10 V, the valve parks at the frost position and reports low battery. At 0 °C and 1.15 V per cell the running rail is $2.30 - 0.07 \times 1.70 = 2.18$ V, so the lockout trips shortly after that, near the point where a full stroke is no longer guaranteed.
4. **Gentler stalls.** Homing runs at 50 % PWM duty and uses the encoder timeout. With the 220 µF reservoir smoothing the 20 kHz PWM, the battery sees roughly D^2 of the stall current, a quarter at $D = 0.5$, so the worst homing dip falls from 0.33 V to about 0.08 V.

10.7 Fault handling

> **Boundary.**
> Watchdog types, fault handlers and safe states at the processor level are in *Electronics and Computing*, Ch. 27 §27.11; power-loss-safe logging in Ch. 28 §28.6. Functional safety in the formal sense (IEC 61508, ISO 13849) is the subject of Chapter 21. A TRV is not a safety function under those standards, but the method of naming each failure, its detection and its reaction is the same.

Safe state and reset policy

For a radiator valve, fully closed is not safe in winter (pipes freeze) and fully open overheats rooms, so the safe state is *parked at 1.0 mm open*. A fault never leaves the motor energised, nor the valve closed if it can still move.

The reset policy is the second decision, and Revision A got it wrong. A reset is the right answer when the software state is untrustworthy (hard fault, watchdog timeout, stack overflow) and the wrong answer to an *environmental* condition such as a low rail, which survives the reset. In Revision B environmental faults move the state machine to a degraded state and only software faults reset. Every reset stores its cause from the reset-reason register, with the last 16 events, in no-init RAM; with a valid retained position the device is back in IDLE in about 0.4 s (R8).

The housekeeping thread kicks the watchdog (4 s timeout) only when the control, sensor and radio threads have each reported progress within 2 s, the supervisor pattern of Book 4 §27.11; the timeout exceeds the worst flash erase plus the longest radio operation.

FMEA

Table 10.8 — FMEA of the reference TRV (Revision B).

Failure mode	Effect	Cause	Detection	Mitigation
Motor stall not detected	Gearbox damage after repeated long stalls	Shunt open, SAADC misconfigured, blanking left on	Encoder-timeout backup path (≤ 48.5 ms); self-test at boot reads shunt offset	Independent backup path; move timeout 30 s
Hall sensor failure	Position lost; valve runs to end stop	Sensor dead, magnet cracked, load switch failed	No edges within 100 ms of start while current shows running	Stop, report F_ENCODER, go SAFE using timed open-loop move to frost position
Lost counts	Position drifts; R2 violated	Coasting edges ignored; back-driving	Homing offset > 60 counts at weekly exercise	Hall powered 60 ms after stop; self-locking screw; weekly re-zero
Brownout reset loop (Rev A)	Battery flat in hours	POF handler reset + RAM-only position + homing stall	Reset counter > 3 per hour in fault log	Rev B power policy (Section 10.6)
Temperature sensor I²C hang	No room temperature; control stops	SDA held low after a reset mid-transfer	Driver timeout 20 ms, NACK on probe	Bus-clear (9 SCL pulses), sensor power-cycle, fall back to last value for 30 min then SAFE
Flash wear of position journal	Settings lost after years	Excessive journal writes	NVS erase counter	48 writes/day of 16 B over 8 pages of 4 kB: each page erased every 43 days, about 1,200 years at 10,000 cycles
Radio parent lost	No commands; device keeps last schedule	Border router off, interference	No poll response for 3 polls	Keep local schedule; reattach with back-off; Ch. 11 covers fleet behaviour
Seized valve pin after summer	Valve will not move in October	Limescale, idle pin	Stall within 0.2 mm of start	Weekly exercise stroke all year
ESD on buttons	Reset or latch-up	Discharge to button frame	Reset cause = brownout or pin reset	TVS on button lines; frame bonded to ground plane; R10 test
Reversed battery	Damage	User error	—	P-MOSFET reverse-polarity protection

Two rows deserve comment. An early prototype journalled position on every hall *edge* instead of every *move*: about 9,600 writes per day, wearing the pages out in under six years, inside the service life. And the seized-pin row is mitigated by a *behaviour*, the weekly exercise, which the power budget then has to pay for (4.7 µA).

10.8 Communication interfaces

> **Boundary.**
> The electrical and protocol details of UART, SPI, I²C, and the troubleshooting method are in *Electronics and Computing*, Ch. 26; the communication stack layering in Ch. 28 §28.7. The radio link budget belongs to Chapter 26 of this book, and the connected-device side (provisioning, MQTT, fleet security) to Chapter 11.

Every interface gets a line recording its rate, its owner (the one driver that may use it) and its power cost, the column that matters most for battery life.

Table 10.9 — Interface table of the TRV.

Interface	Peer	Rate / format	Owner	Power cost and rule
I²C (TWIM)	SHT40-class sensor	100 kHz, CRC-8 on data	Sensor driver	Pull-ups 10 kΩ: 0.3 mA only while a line is low; peripheral disabled between reads
SPI (SPIM)	E-paper display	4 MHz, write-only	Display driver	Display powered only during updates (about 2 s, 3 mA)
UART (UARTE)	Factory tester, debug	1 Mbaud, framed with CRC-16	Test shell	Disabled in field firmware: an enabled receiver keeps the high-frequency clock running, about 0.5 mA
SAADC	Shunt, battery divider	2 kHz while moving; 1 per hour otherwise	Motor and battery drivers	Off between conversions
802.15.4 radio	Thread parent router	250 kbit/s; CoAP over UDP over 6LoWPAN	OpenThread	Sleepy end device, poll every 5 s: 3.85 µA
SWD	Debugger	4 MHz	—	Pads only; debug interface locked in production (APPROTECT)

Worked example — I²C pull-ups chosen for energy, not speed. The bus to the temperature sensor is 30 mm long with one device, about 50 pF in total. Book 4 §26.4 gives the rise time $t_r = 0.8473\,R_p C_b$. With 10 kΩ pull-ups, $t_r = 0.8473 \times 10\ \text{k}\Omega \times 50\ \text{pF} = 0.42\ \mu\text{s}$. That meets the 1 µs standard-mode limit but not the 0.3 µs fast-mode limit, so the bus runs at 100 kHz. A read is about 0.9 ms of bus activity, roughly half of it with a line low at $3.0/10\ \text{k}\Omega = 0.3$ mA: about 0.14 µC, against 10 µC for the whole measurement event. Fast-mode 2.2 kΩ pull-ups would cost 1.4 mA while low and gain nothing.

The radio couples timing to power. A sleepy end device is reachable only at its polls, so the parent router holds the hub's command until the next one: worst-case latency is 5 s plus about 0.2 s, against R6's 10 s. A 10 s poll would save 1.9 µA but leave no margin. On a battery-powered radio device latency is bought with energy; Chapter 11 takes that trade-off to fleet scale.

10.9 Debugging and testing

> **Boundary.**
> On-chip debug (SWD, breakpoints, trace), fault analysis, HIL testing and firmware validation are in *Electronics and Computing*, Ch. 28 §28.8–28.10. Test plans, verification versus validation and regression are in Chapter 4; HIL for robots is in Chapter 17.

The instruments

Three instruments carry most of the integration work, each answering a question the others cannot.

- **SWD debug probe with RTT logging**: what is the firmware doing? RTT logs through a RAM buffer without stopping the core. The probe is unplugged for power measurements, because an attached debugger keeps debug domains powered (hundreds of microamps).
- **Current profiler** (PPK2 or Otii class, 100 kS/s, sub-microamp resolution, also the supply): where does the charge go? The budget of Section 10.6 is built from its charge-per-event measurements, and it found the 0.5 mA UART receiver left enabled on the first prototype.
- **Mixed-signal oscilloscope** with a current probe on the motor lead and digital channels on hall, PWM and a firmware-toggled GPIO: how long does it take? Every link of Table 10.6 was measured as the time between two edges.

The HIL rig and fault injection

The integration rig puts a production head on a real valve body mounted on a load cell, inside a climate chamber. It is powered from a source with programmable output resistance, which emulates a battery at any V_{OC} and R_{src}, so the Section 10.6 table becomes a test matrix. A Thread border router and a scripted hub send setpoints; relays inject faults (open hall line, SDA shorted to ground, motor disconnected, locked rotor). A Python runner (Chapter 29) sequences the cases against limits nightly for every firmware build, so the rig is also the regression test.

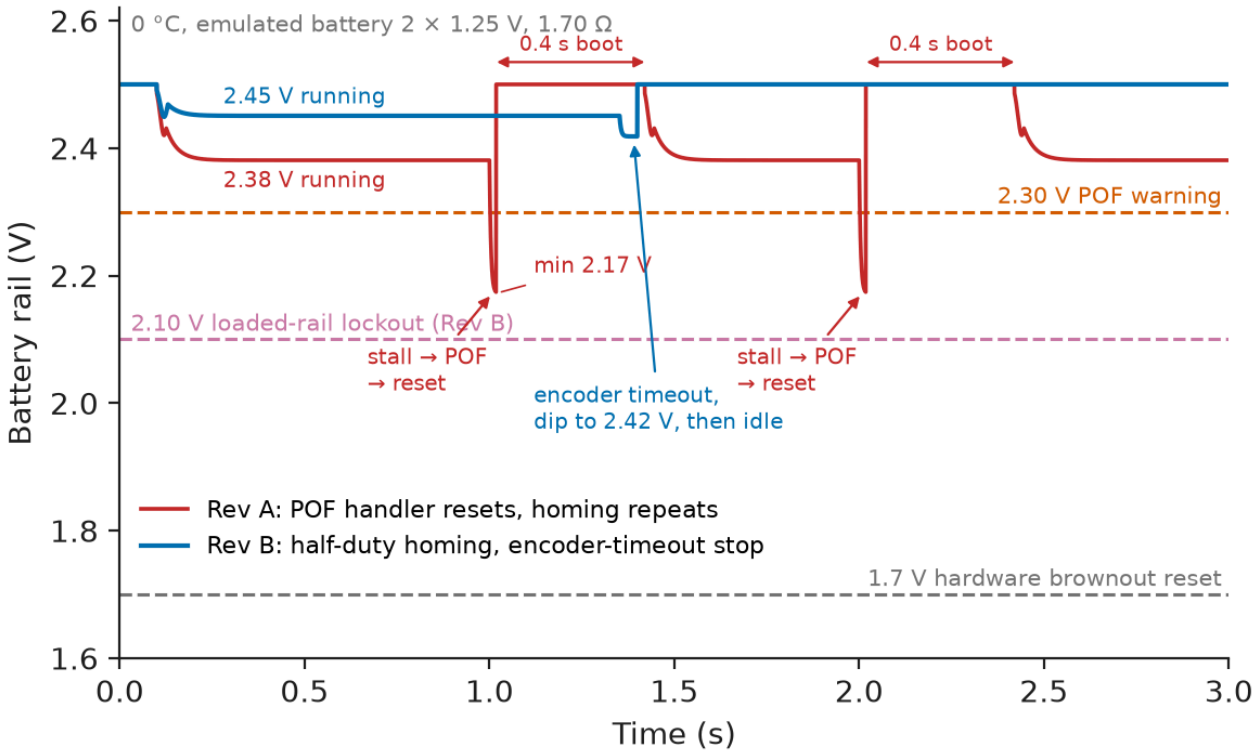


Figure 10.6 — Battery rail at 0 C with cells at 1.25 V each during homing, Revision A against Revision B, on the HIL rig with emulated battery.

Figure 10.6 shows the rig reproducing the field failure on the first day it was run: the emulated battery at 2.50 V and 1.70 Ω, chamber at 0 °C, Revision A firmware. It looped like the returned units; Revision B homes once, cleanly.

Verification against the requirements

Table 10.10 — Verification of the Revision B TRV (5 prototype units; rig measurements).

Req.	Test	Limit	Result	Pass/fail
R1	Chamber 15, 22, 30 °C against PT100 reference, radiator hot and cold	±0.3 °C	max error 0.21 °C	Pass
R2	200 random moves, dial gauge, 2 resets in sequence	±0.05 mm	max error 0.032 mm	Pass
R3	Locked rotor at 25 °C and 0 °C, 5 battery states, 100 stalls each	≤ 20 ms primary, ≤ 100 ms backup	5.1 ms worst primary; 44 ms backup (shunt open)	Pass
R4	Charge per event measured; budget recomputed	≥ 2.5 years (2 years + 25 %)	2.97 years predicted	Pass
R5	Emulated battery 2.20–3.20 V loaded, 0 and 50 °C, 1000 moves	No motor-caused reset	0 resets; lockout at 2.09–2.11 V	Pass
R6	500 setpoint commands from hub	≤ 10 s	max 5.4 s, median 2.7 s	Pass
R7	Fault-injection set of Table 10.8 (9 modes × 10 trials)	Safe state reached, fault reported	90/90	Pass
R8	Forced watchdog reset during move	≤ 2 s to IDLE, no homing	0.42 s, no homing	Pass
R9	Power cut at random points during OTA, 50 trials	Always boots a valid image	50/50	Pass
R10	IEC 61000-4-2, ±8 kV contact	No reset	1 reset at frame seam on unit 3 (fixed by bonding, retested)	Pass after fix
Timing	Radio-induced ISR latency, 10^6 samples under traffic	≤ 1.0 ms assumed in budget	0.31 ms worst case	Pass

The R10 row shows what the table is for: the first ESD run failed, the fix was traced to the failure, and the test was repeated. A table in which every first attempt passes usually means the limits were written after the results.

What Revision B changed, and why

Each Revision B change traces to a mechanism: the hardware stall trip (flash-erase stalls, Section 10.5); POF policy, retained position, loaded-voltage lockout and half-duty homing (the reset loop, Section 10.6); the DRV8837-class driver (2.7 V minimum of the original, Section 10.3); the front-flex sensor (1.6 °C self-heating error); the bonded button frame (ESD reset); and the disabled UART receiver (0.5 mA, which would have cut life to under five months).

> **Status of the principle: Test the corners the budget depends on**
> *Empirical.* Integration failures cluster where two budgets meet at an extreme: cold battery with peak current, flash erase with stall, radio traffic with interrupt latency. A test matrix built from the budgets' own variables (temperature, state of charge, concurrent activity) finds them before the field does.

What remains open

Open question	Where it is answered
How the TRV is provisioned onto a network, authenticated and updated as one of thousands of devices	Chapter 11
How a learned model could replace the fixed deadband controller, and what it costs on this MCU	Chapter 12
The 802.15.4 link budget through walls and around radiators	Chapter 26
Motor drivers, encoders and PID loops when the actuator is a robot joint, not a valve	Chapter 15
The same integration method in a complete project: firmware, budgets and tests for a robot, an energy monitor, a motor drive and a sensor network	Chapters 34, 35, 37, 38
Battery chemistry choice and cell models beyond the Thevenin approximation	Chapter 23; *Electrical Energy and Machines*, Ch. 16
Formal functional safety when the actuator can hurt someone	Chapter 21

Next: Chapter 11 takes the single connected device and multiplies it into a fleet, and the questions change from timing and microamps to telemetry, protocols, gateways, provisioning and security.

Chapter 11 — IoT Systems

Part III · Embedded and Connected Systems

At 22:40 on a Saturday the compressor of a vaccine refrigerator in a suburban pharmacy fails. The wireless probe inside reports the rising temperature every minute. At 23:12 the probe passes 8 °C for the fifteenth consecutive minute, and the gateway on the shelf publishes an alarm to the cloud. The alarm never arrives. To save cellular data, the gateway uploaded telemetry in 15-minute batches and its MQTT keepalive was stretched to 20 minutes; in the idle gap the pharmacy's broadband router had silently dropped the gateway's TCP mapping, and the gateway had not noticed. The alarm was published at QoS 0 into a dead socket, and the gateway marked it as sent. When the connection was re-established, the routine telemetry flowed again and was stored correctly, but the cloud raised alarms only from alarm messages, not from the data. The pharmacist found the fridge at 14 °C on Monday morning, and $18,000 of vaccines were destroyed. Every component met its datasheet; the probe, the gateway and the broker all did what they were told. The failure was in the system's assumptions: that a TCP connection is alive until told otherwise, that a published message is a delivered message, and that one alarm path is enough. This chapter designs that monitoring system properly. The reference system is a cold-chain monitoring fleet for a chain of 120 pharmacies: about 1,000 refrigerator and freezer probes, 120 gateways, and a cloud back end.

> **How is a fleet of connected devices designed so that data and alarms arrive complete, on time and authenticated, across links that fail silently?**
> **Which choices of connectivity, protocol, storage, provisioning and security decide whether the system can be trusted for regulated records over five years?**

Concept	Engineering destination
Device architecture	A probe, gateway and cloud partition that keeps each function where its data and power are
Connectivity	BLE inside the pharmacy, Ethernet with LTE-M failover to the cloud
Telemetry	A batched payload with sequence numbers and UTC timestamps that fits a cellular data budget
MQTT and HTTP	Publish–subscribe for telemetry, alarms and commands; HTTPS for provisioning and firmware
Gateways	Store-and-forward for 72 h, local alarms, direct SMS when the cloud is unreachable
Data storage	A probe ring buffer, an SQLite queue on the gateway, a compressed time-series store in the cloud
Remote monitoring	Heartbeats, last-will messages, stale-data alarms and an end-to-end alarm acknowledgement
Device provisioning	Factory keys in a secure element, certificates, claiming by QR code
Authentication	Mutual TLS with per-device X.509 certificates, topic-level access control
Security and reliability	A threat model, ETSI EN 303 645 provisions, signed updates, jittered back-off, availability budgets

Roadmap. Requirements and device architecture (11.1) → connectivity (11.2) → telemetry design (11.3) → MQTT and HTTP fundamentals (11.4) → the gateway (11.5) → data storage (11.6) → remote monitoring and the alarm path (11.7) → device provisioning (11.8) → authentication (11.9) → security, reliability and verification (11.10).

11.1 Requirements and device architecture

> **Boundary.**
> The single device is the subject of Chapter 10: MCU choice, timing, power budget, fault handling. This chapter treats devices as members of a fleet. Radio link budgets and wireless sensor networks are owned by Chapter 26; the theory of propagation and path loss by *Circuits, Signals and Communications*, Ch. 30. A complete wireless sensor network project, with its own security section, is Chapter 38. Industrial cybersecurity (IEC 62443) for control systems is in Chapter 21.

Cold-chain monitoring for medicines is regulated. In the EU, good distribution practice (GDP, 2013/C 343/01) requires temperature-controlled storage to be monitored, deviations to be alarmed and investigated, and records to be kept. Electronic records must meet data-integrity rules such as EU GMP Annex 11 or US 21 CFR Part 11: audit trails, protection against alteration, and retention. The requirements below come from the pharmacy chain's quality team and its IT security policy.

Table 11.1 — Requirements for the cold-chain monitoring fleet.

ID	Requirement	Target	Verification
R1	Temperature measurement	±0.5 °C from −30 to +15 °C (glycol-buffered probe)	Calibration against reference, Ch. 9 procedure
R2	Logging interval and time base	One reading per minute; UTC timestamps within ±2 s	Log audit against GNSS-disciplined reference
R3	Alarm latency	Local buzzer ≤ 2 min and on-call notification ≤ 5 min after the alarm condition is met	End-to-end injected excursions, 200 trials
R4	Data completeness	100 % of readings stored for WAN or cloud outages ≤ 72 h and gateway power cuts ≤ 2 h	Outage injection, sequence-gap audit
R5	Probe battery life	≥ 3 years	Charge-per-event measurement and budget
R6	Cellular data	≤ 50 MB per gateway per month in continuous LTE backup	Byte count on modem over 7 days
R7	Records	Retained 5 years, tamper-evident, with audit trail	Data-integrity audit
R8	Device security	Unique identity per device, mutual authentication, no default passwords, signed updates (ETSI EN 303 645)	Penetration test, provision checklist
R9	Availability	Alarm delivery ≥ 99.9 % per year; remote dashboard ≥ 99.8 %	Availability model plus 6-month field data
R10	Fleet recovery	After a broker restart, all gateways reconnected within 10 min without overload	Load test with the vendor's full fleet of 12,000 gateways

R3 and R4 are the requirements Revision A broke. R10 extends beyond this chain, because the monitoring vendor runs one broker cluster for 12,000 gateways across many customers.

Partition: probe, gateway, cloud

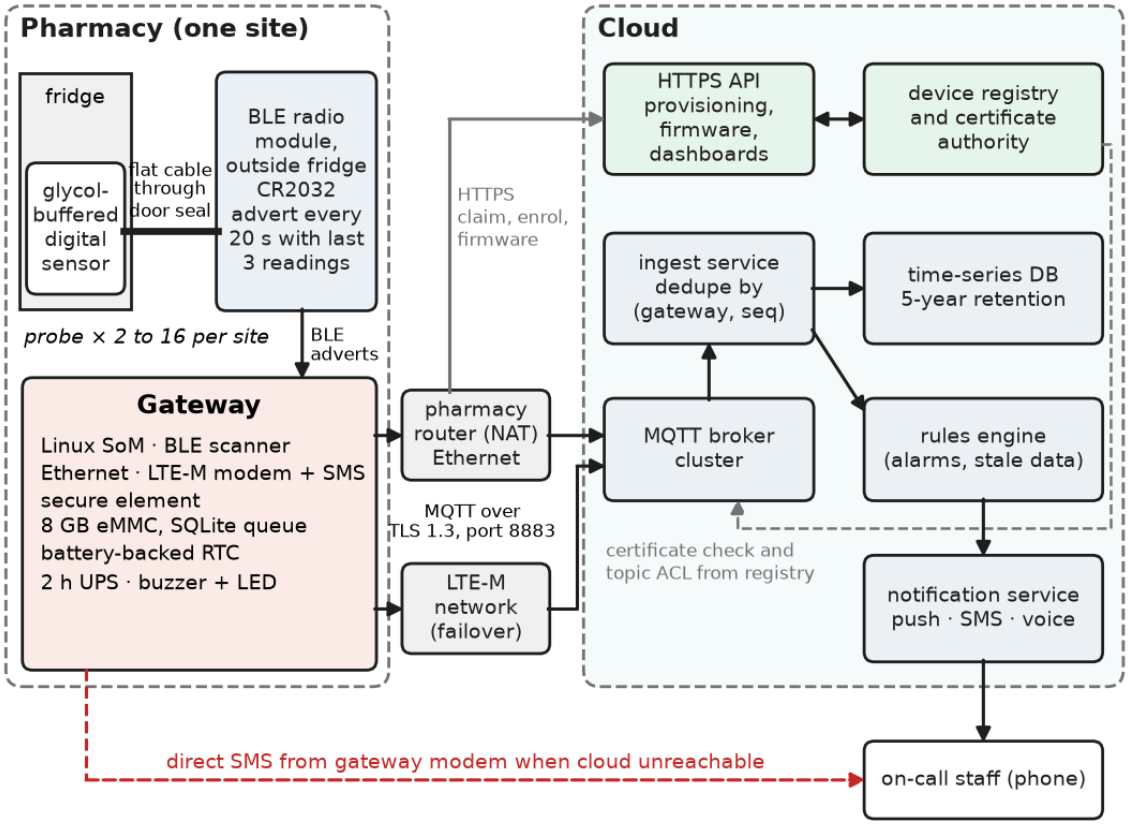


Figure 11.1 — Architecture of the pharmacy cold-chain monitoring fleet.

The architecture of Figure 11.1 puts each function where its information and energy are.

- **Probe.** It measures, timestamps relative to its own counter, and broadcasts. It is a coin-cell device, so it never holds a connection. The sensor sits inside the fridge in a glycol bottle that mimics the thermal mass of a vial. The radio sits outside the fridge door, because a refrigerator is a metal box that attenuates 2.4 GHz by 20–40 dB.
- **Gateway.** It owns everything that needs mains power, storage and a clock: scanning, UTC timestamping, alarm evaluation, store-and-forward, the cloud connection, and a second alarm path that does not depend on the cloud.
- **Cloud.** It owns everything that spans sites: the record of truth, the audit trail, dashboards, alarm escalation to people, the device registry and firmware distribution.

The alarm decision is made twice, on purpose: once at the gateway, from the readings it has, and once in the cloud, from the stored data stream. The cloud evaluation also raises a "stale data" alarm when a gateway's data stops arriving. Revision A made the decision only at the gateway and trusted its single message.

> **Status of the principle: Decide where the data is; confirm where the record is**
> *Empirical.* Placing time-critical decisions at the edge, where data arrives first and does not depend on the WAN, and repeating them centrally on the authoritative record, is standard practice in monitoring and alarm systems. It turns a single point of failure in the alarm path into two independent paths.

11.2 Connectivity

> **Boundary.**
> Data rates, sensitivities and range follow from modulation, noise and path loss, owned by *Circuits, Signals and Communications*, Ch. 27 and Ch. 30 §30.10. Ethernet's PHY and MAC are in *Electronics and Computing*, Ch. 26 §26.7. Here the technologies are compared as integration options.

There are two connectivity decisions: probe to gateway (short range, coin cell, inside a building) and gateway to cloud (wide area, mains powered, through networks the system does not own).

Table 11.2 — Connectivity options (typical values; check current datasheets and operator terms).

Technology	Rate	Indoor range	Device current (TX / RX / sleep)	Infrastructure	Fit for this system
BLE 5 advertising, 1M PHY	1 Mbit/s	10–30 m	5 / 5 mA / 1–2 µA	Gateway scanner	**Probe link, chosen**: lowest probe energy, phones can read it for service
BLE Coded PHY (S8)	125 kbit/s	2–4 × 1M range	same, 8 × airtime	Gateway scanner	Fallback for large stores
IEEE 802.15.4 / Thread	250 kbit/s	10–30 m, mesh	5 / 5 mA / 1–3 µA	Border router, routers	Mesh unnecessary for 2–16 probes per room
Sub-GHz 868 MHz (LoRa, FSK)	0.3–50 kbit/s	50–500 m	20–120 / 5–10 mA / 1 µA	Gateway; 1 % duty cycle (ETSI EN 300 220)	Range not needed; duty-cycle limit constrains retries
Wi-Fi 4	1–72 Mbit/s	20–40 m	200–300 / 50–100 mA / 10–100 µA	Pharmacy AP	Probe energy too high; pharmacy IT will not admit IoT devices
Ethernet	100 Mbit/s	cable	mains	Pharmacy router	**Gateway primary uplink**
LTE-M (Cat-M1)	up to about 1 Mbit/s	operator	100–500 / 30–60 mA / PSM 3–10 µA	Operator, SIM	**Gateway failover uplink, chosen**; supports SMS
NB-IoT	tens of kbit/s	operator, deep indoor	similar, longer airtime	Operator	Latency of seconds and patchy SMS support; not for alarms

Two non-technical facts decide as much as the table does. Pharmacy IT policy forbids IoT devices on the store's Wi-Fi, but allows a separate VLAN port on the router. And a store's broadband is consumer-grade, with outages of hours a few times a year. That is why the gateway has an LTE-M modem as a *warm* standby: attached to the network, with a TLS session ready to open, so failover takes seconds rather than the 10–60 s of a cold attach.

Worked example — Will the probe battery last? A probe wakes every 60 s to read its sensor, about 2 µC, and transmits an advertising event every 20 s, about 15 µC each for three channels at 0 dBm. Each advert carries the last three readings, so a single lost advert loses nothing. The average current is $1.5 + 2/60 + 3 \times 15/60 = 2.28$ µA, plus self-discharge of 1 % per year of a 225 mAh CR2032 (0.26 µA). With 80 % usable capacity, life is $0.8 \times 225\ \text{mAh} / 2.54\ \mu\text{A} = 70{,}900$ h, or 8.1 years. R5 is met with a large margin, which is spent on the triple-redundant adverts. The method is that of Chapter 10's budget and is not repeated here.

11.3 Telemetry

Telemetry design decides three things: what is sent, how often, and in what shape. Each decision is constrained by the requirements, and each one costs bytes.

What. Each reading carries the probe ID, the temperature, the probe's battery voltage and the received signal strength. The last two are not needed for the record, but they are the fleet's health data, and they predict failures weeks ahead. Every gateway message also carries a **monotonic sequence number** and a **UTC timestamp**. The sequence number lets the cloud detect gaps (R4) and discard duplicates. The timestamp is assigned by the gateway, whose clock is disciplined by NTP over the WAN and held by a battery-backed RTC when offline. Probe readings are timestamped at reception and corrected by the probe's relative counter: a reading that was "two adverts old" is backdated by 40 s.

How often. R2 fixes one reading per minute. The temptation is to report *on change* only, which can cut traffic tenfold for a stable fridge. That temptation is rejected for the record stream, because a missing reading must be distinguishable from an unchanged one: under GDP an absent value is a gap that must be investigated. The heartbeat property of continuous telemetry is also what lets the cloud detect a dead gateway (Section 11.7).

Shape. Readings can be published one per message or batched per gateway, as text (JSON) or binary (CBOR, Protocol Buffers). The listing computes the bytes exchanged per minute, including MQTT, TLS 1.3 and TCP/IP overheads and the acknowledgements, for a gateway with 8 probes, and the resulting monthly volume on LTE.

```
 # Monthly uplink+downlink bytes per gateway (8 probes, one reading per probe per minute)
TLS, TCPIP = 22, 52            # TLS 1.3 record overhead; IPv4 + TCP with timestamps
MIN_PER_MONTH = 60 * 24 * 30

def mqtt_qos1(topic_len, payload_len):
    publish = 2 + 2 + topic_len + 2 + payload_len + TLS + TCPIP
    puback = 4 + TLS + TCPIP
    return publish + puback + TCPIP          # + bare TCP ACK of the PUBACK

options = {   # name: bytes exchanged per minute
    "JSON, one publish per reading": 8 * mqtt_qos1(36, 75),
    "JSON, batched per gateway":     mqtt_qos1(28, 8 * 40 + 30),
    "CBOR, batched, delta-coded":    mqtt_qos1(28, 8 * 10 + 15),
    "HTTPS POST, batched (keep-alive)": 250 + 8 * 40 + 30 + 150 + 2 * TLS + 3 * TCPIP,
}
for name, b in options.items():
    print(f"{name:<34s} {b:5d} B/min  {b * MIN_PER_MONTH / 1e6:6.1f} MB/month")
```

```
JSON, one publish per reading       2568 B/min   110.9 MB/month
JSON, batched per gateway            588 B/min    25.4 MB/month
CBOR, batched, delta-coded           333 B/min    14.4 MB/month
HTTPS POST, batched (keep-alive)     950 B/min    41.0 MB/month
```

Worked example — Choosing the payload against the data budget. One message per reading would cost 110.9 MB per month on LTE, more than twice R6's 50 MB. The payload is not the problem; overhead is. Each 75-byte reading carries about 246 bytes of headers and acknowledgements. Batching the 8 readings into one message cuts the volume to 25.4 MB, a 49 % margin against R6. CBOR with delta coding would reach 14.4 MB (71 % margin), but it makes every message unreadable in a broker console during an incident. The design takes batched JSON: the margin is enough, and debuggability is worth more than the extra 11 MB. The decision is recorded together with its trigger for revisiting: switch to CBOR if a gateway's probe count makes JSON exceed 40 MB, which happens above 16 probes (Figure 11.2). Batched HTTPS POSTs, at 41.0 MB, fit as well, but with only 18 % margin, and they cannot carry commands *to* the gateway, which is the deciding point for MQTT in Section 11.4.

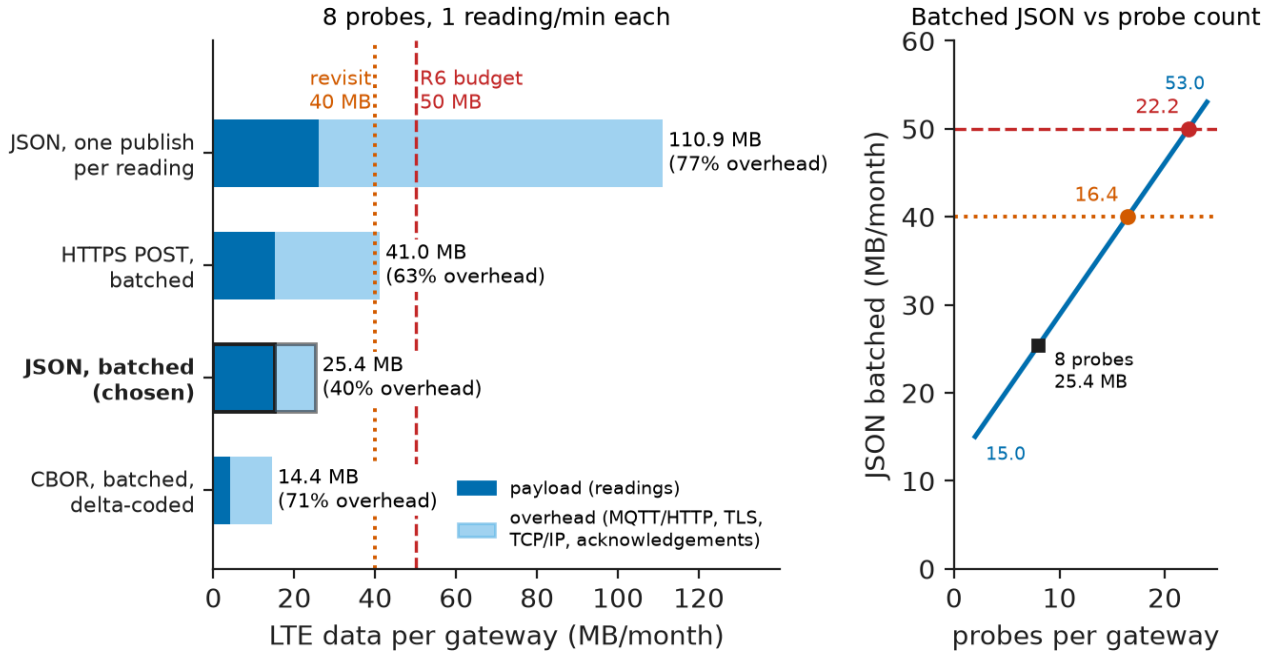


Figure 11.2 — Monthly LTE data per gateway for four telemetry formats, against the 50 MB budget.

The batched JSON payload for one minute from one gateway looks like this (two of eight readings shown):

```
{
  "v": 1,
  "gw": "GW-0A17F3",
  "seq": 1048213,
  "ts": "2026-10-24T23:12:00Z",
  "r": [
    {"p": "P-03C1", "t": 8.21, "b": 2.98, "q": -71, "age": 0},
    {"p": "P-03C4", "t": 4.62, "b": 3.01, "q": -64, "age": 20}
  ]
}
```

The `v` field versions the schema, because a fleet in the field always runs several firmware versions at once and the ingest service must accept all of them. `age` is the reading's age in seconds at reception, from the redundant-advert scheme.

11.4 MQTT and HTTP fundamentals

> **Boundary.**
> TCP, TLS and the internet protocol suite are used here as given. CRC and error-detection theory is in *Circuits, Signals and Communications*, Ch. 28 §28.6. The protocols are specified by OASIS (MQTT 3.1.1 and 5.0) and the IETF (HTTP semantics, RFC 9110; TLS 1.3, RFC 8446).

MQTT in one page

MQTT is a publish–subscribe protocol over a long-lived TCP connection to a **broker**. Clients do not address each other. They publish messages to **topics**, hierarchical strings such as `cc/v1/acme/s042/GW-0A17F3/telemetry`, and subscribe to topic filters with wildcards (`+` for one level, `#` for the rest). The features that matter for integration are few:

- **QoS levels.** QoS 0 is at most once: sent, never acknowledged. QoS 1 is at least once: the sender stores the message until it receives a PUBACK and resends it otherwise, so duplicates are possible. QoS 2 is exactly once, through a four-step handshake. QoS applies *per hop*, client to broker and broker to subscriber, never end to end.
- **Sessions.** With a persistent session (MQTT 3.1.1 `cleanSession=0`; MQTT 5 `Session Expiry Interval` > 0) the broker keeps a client's subscriptions and queues its QoS 1/2 messages while it is offline.
- **Keepalive.** The client promises to send *some* control packet within the keepalive interval, and sends PINGREQ if it has nothing else to send. The broker drops a client that is silent for 1.5 times the interval.
- **Retained messages and Last Will.** A retained message is stored per topic and delivered to every new subscriber, which suits "current status". A **Last Will and Testament** (LWT) is registered at connect time and published *by the broker* if the client vanishes without a clean disconnect.
- **MQTT 5 additions.** Message expiry, reason codes on every acknowledgement, user properties, and request–response correlation are all useful for commands and acknowledgements.

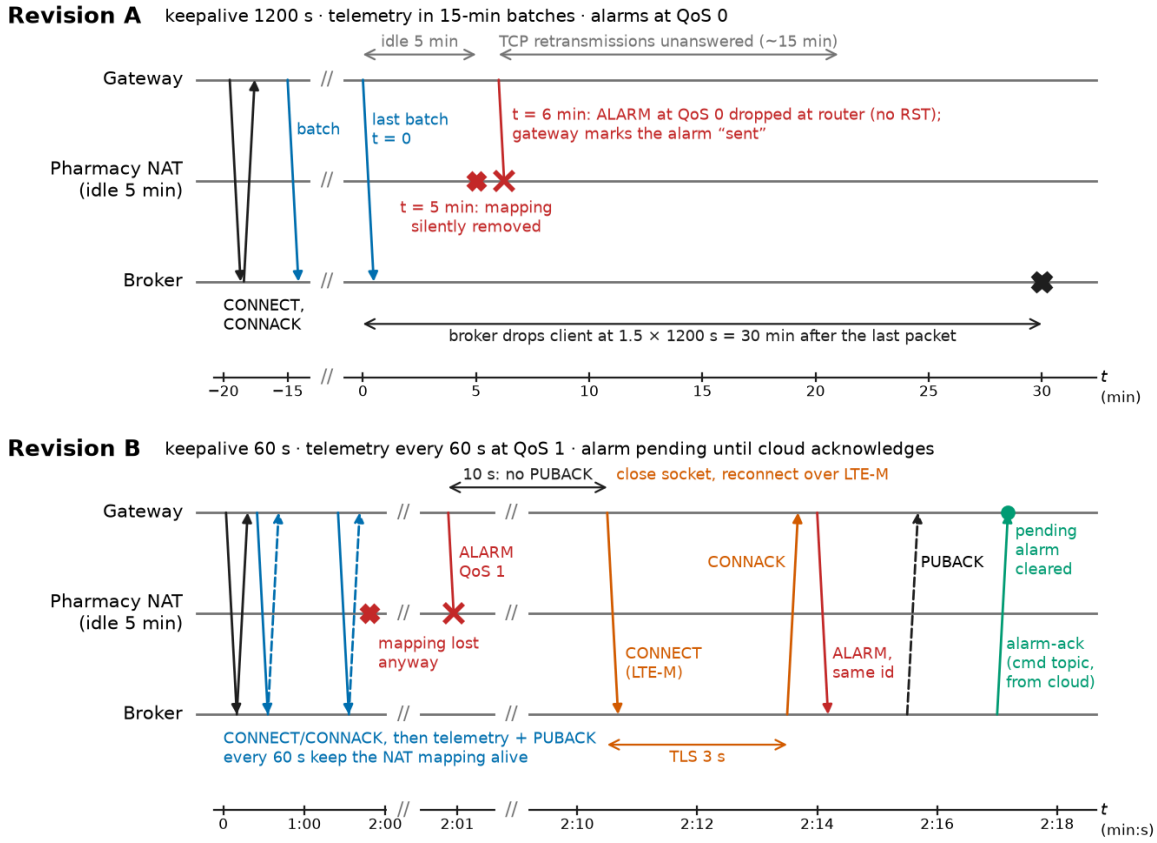


Figure 11.3 — MQTT session through a NAT router that drops idle mappings: Revision A loses the alarm, Revision B recovers it.

Figure 11.3 replays the opening failure on these mechanisms. Three properties of TCP and NAT combined against Revision A. A NAT router or carrier-grade NAT forgets an idle TCP mapping after some minutes, often 2–30, without telling either end. A TCP sender learns that its peer is gone only after its retransmissions time out, about 15 minutes with Linux defaults. And QoS 0 discards the message as soon as it is written to the socket. The rules that follow are general:

1. **Keepalive must be shorter than the shortest NAT idle timeout on the path.** This design uses 60 s. Telemetry every minute is itself a keepalive, so the setting costs no extra data in normal operation.
2. **Anything that must arrive is QoS 1**, on a persistent session, with a short application timeout for the PUBACK. Here the gateway waits 10 s for a PUBACK and then declares the connection dead.
3. **"Delivered to the broker" is not "handled".** The alarm stays *pending* at the gateway, and is re-published with the same alarm ID, until the cloud returns an application-level acknowledgement on the gateway's command topic.
4. **QoS 1 means duplicates**, so every consumer is idempotent: the ingest service stores a reading once per (gateway, sequence) pair.

> **Status of the principle: Delivery guarantees are per hop**
> *Fundamental.* By the protocol's definition an MQTT acknowledgement covers one hop, client to broker or broker to client. End-to-end assurance therefore needs an application acknowledgement from the final consumer. This holds for any store-and-forward messaging protocol, not only MQTT.

The topic tree and the command channel

Topics are an interface and are designed like one: versioned, scoped by tenant and site so that access control can be written as patterns, and separated by direction.

```
 cc/v1/{tenant}/{site}/{gw}/telemetry     gateway -> cloud   QoS 1   batched readings
cc/v1/{tenant}/{site}/{gw}/alarm         gateway -> cloud   QoS 1   alarm raise/clear, alarm ID
cc/v1/{tenant}/{site}/{gw}/status        gateway -> cloud   QoS 1   retained; LWT = "offline"
cc/v1/{tenant}/{site}/{gw}/cmd/#         cloud -> gateway   QoS 1   alarm-ack, config, ota
cc/v1/{tenant}/{site}/{gw}/cmd-result    gateway -> cloud   QoS 1   result of each command
```

An alarm message carries enough to be handled without any other context, and an ID that makes it idempotent:

```
{
  "v": 1,
  "gw": "GW-0A17F3",
  "alarm_id": "GW-0A17F3-000731",
  "state": "raised",
  "probe": "P-03C1",
  "rule": "T>8.0C for 15 min",
  "t_now": 8.21,
  "since": "2026-10-24T22:57:00Z",
  "ts": "2026-10-24T23:12:00Z"
}
```

Where HTTP fits

HTTP is request–response: the client asks, the server answers, and the connection can be closed between requests. That suits operations that the device initiates, that are large, or that are rare.

Table 11.3 — MQTT and HTTP in this system.

Property	MQTT	HTTPS
Connection	Long-lived, one per device	Per request, or kept alive for a while
Direction	Both ways; the cloud can push to the device at any time	Device asks, server answers; push needs polling or long-polling
Overhead per small message	2–5 B header plus topic	150–500 B of headers
Delivery semantics	QoS 0/1/2 per hop, persistent sessions, LWT	Status codes; retry is the client's job
Large transfers	Possible, but awkward (whole message in memory)	Range requests, resumable, CDN-cacheable
Used here for	Telemetry, alarms, status, commands	Provisioning (certificate signing), firmware download, probe backfill uploads, dashboards and API

Firmware images (40–80 MB root-filesystem deltas for the Linux gateway) are downloaded over HTTPS with range requests from a content-delivery network, which is resumable across the store's broadband drops. The MQTT command channel only tells the gateway *which* signed image to fetch.

11.5 Gateways

A gateway is where protocols, trust domains and failure domains meet. In this system it does six jobs: protocol translation (BLE adverts to MQTT), time-stamping, local alarm evaluation, store-and-forward, WAN failover, and a second alarm path.

Table 11.4 — Gateway platform selection.

Option	Compute and storage	Power	Indicative cost (2026, 1k pcs)	Assessment
MCU gateway (ESP32-S3 class + Ethernet PHY + LTE-M module)	240 MHz, 8 MB PSRAM, 16 MB flash	0.5 W	$38	Cheapest. Store-and-forward and queries in raw flash, SMS fallback and certificate rotation all hand-written
Linux SoM (i.MX 6ULL / i.MX 93 class) + LTE-M module	Cortex-A7/A55, 512 MB DDR, 8 GB eMMC	1–1.5 W	$56	**Chosen.** SQLite queue, A/B root filesystem updates, mature TLS and modem tooling, 10-year SoM availability
Single-board computer (Raspberry Pi CM4 class)	4 × Cortex-A72, 1–8 GB, eMMC	2–4 W	$60–80	Capable, but higher power for the 2 h UPS, and consumer-grade supply terms
Commercial industrial gateway	Varies	3–8 W	$250–600	Fast to deploy, but per-unit cost across 120 sites and vendor lock-in of the edge software

The Linux SoM costs $18 more per site than the MCU option, $2,160 across the chain. That is about one-eighth of one lost vaccine fridge, and it buys a database, robust updates and maintainable alarm logic. The MCU option would be right for a fleet of 50,000 consumer devices. It is not right for 120 regulated sites.

Store-and-forward

Worked example — Sizing the queue for R4 (Figure 11.4). The largest store has 16 probes, so a 72 h outage accumulates $16 \times 60 \times 72 = 69{,}120$ readings. At about 64 bytes per row in SQLite, including indices, that is 4.4 MB, trivial for an 8 GB eMMC. Sizing for storage is easy. The design questions are elsewhere. First, *commit granularity*: one SQLite transaction per minute-batch, not per reading, which cuts eMMC writes sixty-fold and loses at most one minute if power is cut without warning. Second, *power*: a 2 h UPS covers R4's power-cut case, and the probes' own 24 h ring buffers allow backfill over a BLE connection if the gateway is off longer. Third, *drain*: after the outage the backlog is uploaded in batches of 500 readings every 2 s, so 139 messages, about 4.6 minutes. Live readings take priority over backfill, so real-time alarms are never queued behind history. The drain also starts after a random 0–60 s delay, for the fleet reason in Section 11.10.

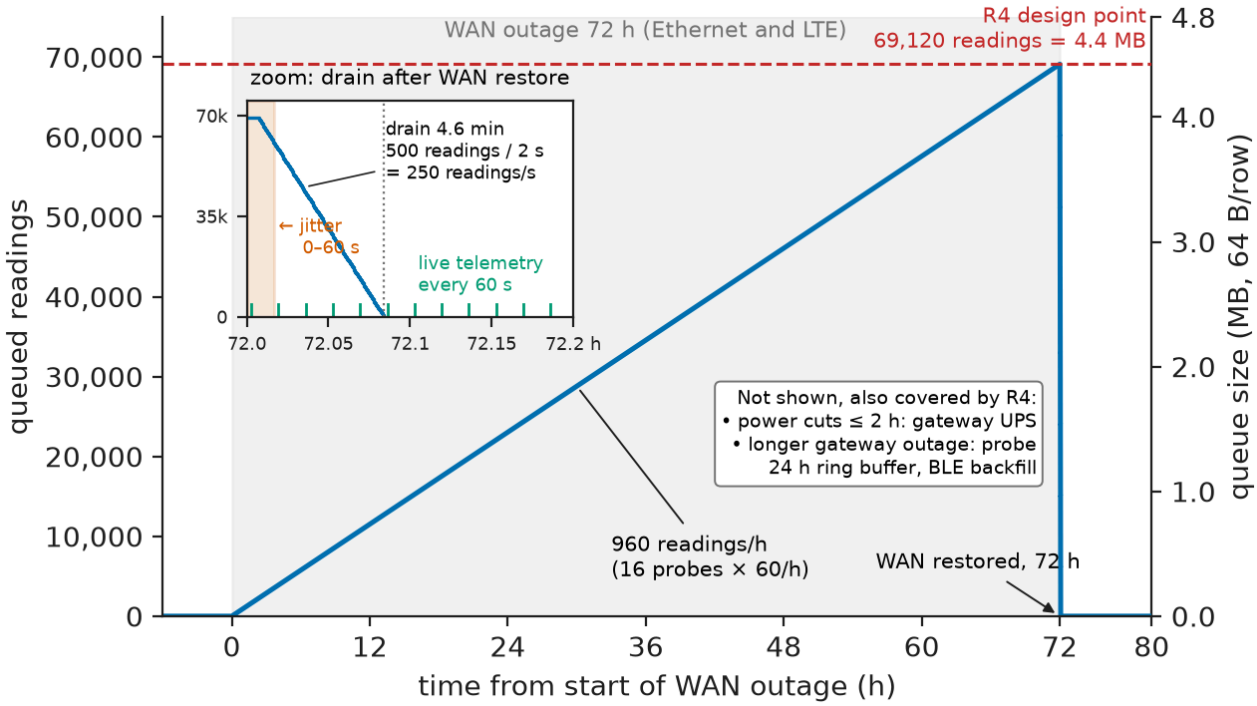


Figure 11.4 — Gateway queue occupancy during a 72 h WAN outage at the largest store and its drain afterwards.

Local alarms and the second path

The gateway evaluates the alarm rules locally, so its buzzer and LED respond within R3's 2 min whether or not any network exists. If an alarm is raised and the cloud has not acknowledged it within 3 minutes over either uplink, the gateway sends an SMS directly through its LTE-M modem to the store's on-call number. The SMS path shares the gateway and the modem with the main path, but not the router, the broker, the cloud or the notification service, which is where most outages occur.

> **Status of the principle: A gateway is a failure-domain boundary**
> *Empirical.* Experience across monitoring fleets shows that WAN and cloud outages outnumber gateway hardware failures by an order of magnitude or more. Functions that must survive them (buffering, local alarming, a direct notification path) therefore belong on the gateway. The ratio varies with site and provider, so it is checked against field data (Table 11.8).

11.6 Data storage

Data is stored at three tiers with different jobs, and each tier's design follows from its job.

Table 11.5 — Storage tiers.

Tier	Store	Holds	Design rule
Probe	24 h ring buffer in internal flash, 1,440 records	Raw readings with relative timestamps	Wear-levelled; survives gateway outage
Gateway	SQLite in WAL mode on eMMC	Unacknowledged batches, alarm state, config	Delete only after broker PUBACK *and* ingest confirmation
Cloud	Time-series database, 1-minute resolution, plus object storage archive	The regulated record, 5 years	Append-only; corrections as new annotated records; audit trail

The gateway rule "delete only after ingest confirmation" is a second application-level acknowledgement. A broker PUBACK means only that the broker has the message. If the ingest service then crashes before writing to the database, the reading would be lost, because no one holds it any more. The ingest service therefore confirms the highest contiguous sequence number it has persisted, in each minute's `cmd/ack`, and the gateway trims its queue up to that number.

Worked example — Five years of records. The chain's 1,000 probes produce one reading per minute: $1{,}000 \times 525{,}600 \times 5 = 2.63 \times 10^9$ points over five years. Stored as rows in a general-purpose relational table at about 40 bytes per row with indices, that is 105 GB. A time-series database that compresses timestamps with delta-of-delta coding and values with XOR coding typically stores slowly changing sensor data at 1–2 bytes per point, about 3.9 GB at 1.5 B/point. Both are affordable. The difference that matters is query speed: a regulatory audit asks for "every excursion in store 42 in March 2027" and expects the answer in seconds. The record is kept at full resolution. Dashboards read pre-computed 15-minute and 1-hour roll-ups.

```
 # Storage for 5-year retention, and availability of the remote view and of alarm delivery
probes, years = 1000, 5
points = probes * 525_600 * years                       # one reading per minute
for name, b in (("row store, ~40 B/row", 40), ("TSDB compressed, ~1.5 B/pt", 1.5)):
    print(f"{name:<28s} {points * b / 1e9:7.1f} GB for {points / 1e9:.2f} G points")

gw, eth, lte, cloud, notify, sms = 0.9995, 0.995, 0.99, 0.9995, 0.999, 0.99
wan = 1 - (1 - eth) * (1 - lte)                         # Ethernet with LTE failover
remote = gw * wan * cloud
alarm = gw * (1 - (1 - wan * cloud * notify) * (1 - sms))   # cloud path OR direct SMS
for name, a in (("remote view", remote), ("alarm delivery", alarm)):
    print(f"{name:<15s} {a:.5%}  downtime {(1 - a) * 8760 * 60:6.0f} min/year")
```

```
 row store, ~40 B/row           105.1 GB for 2.63 G points
TSDB compressed, ~1.5 B/pt       3.9 GB for 2.63 G points
remote view     99.89503%  downtime    552 min/year
alarm delivery  99.94845%  downtime    271 min/year
```

The second half of the listing is used in Section 11.10.

Data integrity is a storage property as well as a security one. Records are never updated in place. A calibration correction or an annotation ("door left open during delivery, product checked") is a new record that references the original, with user and time. The archive tier writes daily files with a hash chain: each

day's file includes the hash of the previous day's file, so any later alteration of history is detectable. This is the "tamper-evident" of R7, implemented with nothing more exotic than SHA-256.

11.7 Remote monitoring

Remote monitoring is the part of the system that people see: dashboards, alarms, escalation. Its engineering problem is to tell the difference between "the fridge is fine" and "we have not heard from the fridge", and to bound the time from an event to a person.

Liveness: heartbeats, LWT and stale data

Three mechanisms report liveness, at three levels:

- **Gateway connection.** The gateway publishes `status = "online"` as a retained message at connect and registers an LWT of `"offline"`. If its connection dies, the broker publishes the LWT after 1.5 keepalives, 90 s.
- **Data freshness.** The rules engine raises a *stale data* alarm when no telemetry has been ingested from a gateway for 5 minutes, *whatever the connection state says*. This catches a gateway that is connected but stuck, and an ingest pipeline that is silently failing.
- **Probe freshness.** The gateway reports each probe's last-seen age. A probe silent for 10 minutes raises a *probe lost* alarm: a flat battery, a probe knocked off its bracket, or a fridge moved.

Revision A had only the first, with a 30-minute effective timeout.

The alarm-latency budget

Table 11.6 — Alarm-latency budget from alarm condition met to notification (R3: local ≤ 120 s, remote ≤ 300 s).

Link	Mechanism	Worst case
Reading that meets the condition is taken	1 reading per minute	60 s
Lost adverts	3 copies per minute; budget for 2 lost	40 s
Gateway rule evaluation, buzzer on	On reception	1 s
Local alarm subtotal		**101 s (margin 16 %)**
Publish fails on dead connection, failover to LTE-M	10 s PUBACK timeout + 5 s TLS budget on warm modem (3 s typical)	15 s
Broker → ingest → rules engine	Measured p99.9	2 s
Notification service → phone (push, SMS)	Provider p99, budgeted	60 s
Remote alarm total		**178 s (margin 41 %)**

The budget shows that R3's local limit is tight. Its 16 % margin comes almost entirely from the choice of a 60 s sampling interval. A 2-minute interval would break it, which is why the sampling rate is fixed by R3 and not just by R2.

11.8 Device provisioning

Provisioning is everything that turns an anonymous piece of hardware into a known, authorised member of the fleet, bound to a customer and a site. Its two halves, at the factory and at the pharmacy, are shown in Figure 11.5.

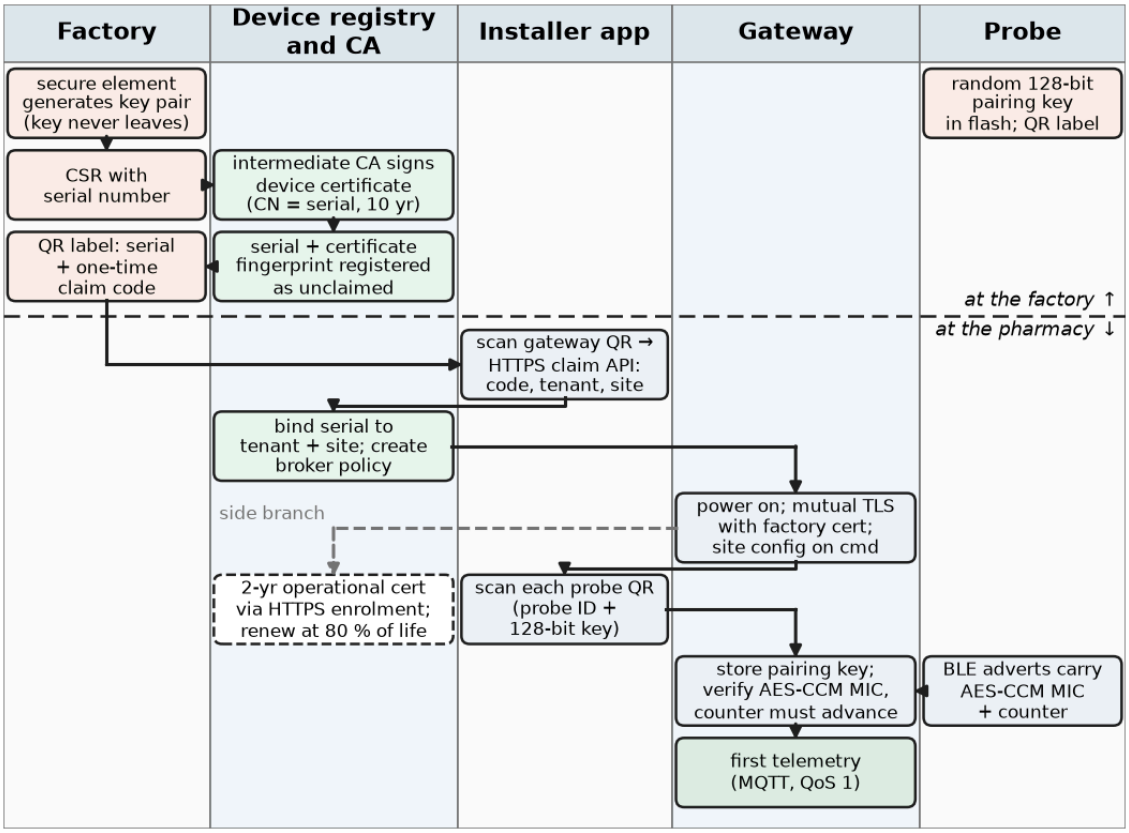


Figure 11.5 — Provisioning and claiming of a gateway and its probes, from factory to first telemetry.

At the factory. Each gateway carries a secure element (ATECC608B or SE050 class) that generates its own key pair internally. The private key never exists outside the chip. The secure element produces a certificate signing request with the device serial number, and the manufacturer's intermediate certificate authority signs it. The serial number and certificate fingerprint are registered as *unclaimed*, and a label carries the serial and a one-time claim code as a QR code. Probes get a random 128-bit pairing key, written into their flash and printed on their QR label. There are no default passwords anywhere, because there are no passwords. That is ETSI EN 303 645 provision 5.1 met by design.

At the pharmacy. The installer's app scans the gateway QR and calls the claim API, which binds the serial to the tenant and site and creates the gateway's broker permissions (Section 11.9). The gateway connects with its factory certificate and receives its site configuration over its command topic. It then *enrols* for an operational certificate with a 2-year validity, using the factory identity to authenticate the request. The installer scans each probe's QR, and the gateway stores the probe's pairing key. From then on the gateway

accepts a probe's adverts only if the 4-byte AES-CCM message integrity code verifies and the advert counter has advanced. That stops a spoofed or replayed probe from faking a healthy fridge.

Worked example — Why certificates expire on different days. If 12,000 gateways had been given operational certificates with the same validity dates in a single provisioning campaign, the whole fleet would try to renew in the same hour two years later. If the renewal service were down that day, the whole fleet would also go dark at once. The design renews at 80 % of the certificate's lifetime plus a random offset of up to 30 days. Expiries are then spread over a month, at most about 400 renewals a day. A failed renewal leaves at least 116 days of validity, about 16 % of two years, to fix the service before any gateway loses its certificate.

11.9 Authentication

Authentication answers "who is this?" in both directions, and authorisation answers "what may it do?".

Mutual TLS. The gateway verifies the broker's server certificate against a pinned root, so it cannot be redirected to an impostor broker. The broker verifies the gateway's client certificate against the manufacturer CA and the registry, which records claimed, revoked or replaced devices. TLS 1.3 with ECDHE key exchange provides forward secrecy. Session resumption keeps the handshake on LTE to about 1.5 kB.

Authorisation by topic. The broker maps the certificate's serial number to the tenant and site from the registry, and allows a gateway to publish only under its own prefix and to subscribe only to its own command topic. Expressed in the broker's policy language (a generic JSON form shown here):

```
{
  "client_id_must_equal": "${cert.subject.cn}",
  "allow": [
    {"action": "publish",   "topic": "cc/v1/${tenant}/${site}/${cert.subject.cn}/+"},
    {"action": "subscribe", "topic": "cc/v1/${tenant}/${site}/${cert.subject.cn}/cmd/#"}
  ],
  "deny_all_else": true
}
```

A stolen gateway can therefore lie only about its own site, and revoking its certificate in the registry cuts it off on its next connection. The rule "client ID must equal the certificate CN" stops a device from taking over another device's session.

People and services. Dashboard users and integrations authenticate to the HTTPS API with OAuth 2.0 / OpenID Connect tokens from the chain's identity provider, with multi-factor authentication for anyone who can acknowledge alarms or annotate records. Acknowledging an alarm is a regulated action that enters the audit trail.

11.10 Security, reliability and verification

> **Boundary.**
> Industrial security levels and zones (IEC 62443) for control systems are applied in Chapter 21; secure boot, signed images and A/B updates at the device level in *Electronics and Computing*, Ch. 25 §25.9. Series and parallel combination of independent availabilities uses the probability of independent events, *The Mathematics Universe*, Ch. 19 §19.4.

Threats and the controls that answer them

Table 11.7 — Threat model (abridged) and controls.

Threat	Asset at risk	Control
Spoofed or replayed probe advert hides a warm fridge	Record integrity, patient safety	AES-CCM MIC with per-probe key and monotonic counter
Impostor broker or MITM on store network	Data, commands	Mutual TLS, pinned root, no plaintext fallback
Stolen or cloned gateway	Other sites' data	Key in secure element; topic ACL per certificate; registry revocation
Malicious or corrupted firmware	Whole fleet	Signed images verified by bootloader; A/B rootfs with rollback; staged rollout 1 % → 10 % → 100 %
Default credentials, open debug ports	Gateway	No passwords; SSH disabled in production; debug UART behind tamper screw
Denial of service by reconnection storm	Broker, whole fleet	Jittered exponential back-off (below); per-client rate limits
Unpatched vulnerability	Fleet over 5+ years	Vulnerability-disclosure policy and update commitment (ETSI EN 303 645 provisions 5.2, 5.3; EU Cyber Resilience Act)

Reliability at fleet scale: the reconnection storm

The vendor's broker cluster serves 12,000 gateways. When a broker node restarts, every gateway connected to it sees its connection drop within a second or two and tries to reconnect. A TLS handshake costs the broker far more CPU than a message, and failed handshakes still cost something. Suppose one node completes 200 handshakes per second, and each rejected attempt costs a quarter of a handshake. Then above 800 attempts per second the node completes *none*. The listing simulates three retry policies against that broker model.

```
 # Reconnection storm after a broker restart: 12,000 gateways, one broker node.
# Budget: 200 full TLS handshakes/s; a rejected attempt still costs 0.25 of one.
import numpy as np

N, CAP, COST_FAIL, DT, T_END = 12_000, 200.0, 0.25, 0.1, 3600.0

def simulate(delay, seed=7):
    rng = np.random.default_rng(seed)
    fails = np.zeros(N, int)
    next_try = rng.uniform(0, 2.0, N) + delay(fails, rng)   # drop seen within 2 s
    done = np.zeros(N, bool)
    per_step = []
    for k in range(int(T_END / DT)):
        t = k * DT
        idx = rng.permutation(np.flatnonzero(~done & (next_try <= t)))
        per_step.append(idx.size)
        s = int(max(0.0, CAP * DT - COST_FAIL * idx.size) / (1 - COST_FAIL))
        done[idx[:s]] = True
        bad = idx[s:]
        fails[bad] += 1
        next_try[bad] = t + delay(fails[bad], rng)
        if done.all():
            break
    rate = np.convolve(per_step, np.ones(10), "valid")      # attempts per 1 s window
    return done.mean(), t, rate.max(), sum(per_step) / N

policies = {
    "fixed 5 s":               lambda f, r: np.full(f.size, 5.0),
    "exponential, no jitter":   lambda f, r: np.minimum(300.0, 2.0 * 2.0 ** f),
    "exponential, full jitter": lambda f, r: r.uniform(0, np.minimum(300.0, 60.0 * 2.0 ** f)),
}
for name, pol in policies.items():
    frac, t, peak, att = simulate(pol)
    print(f"{name:<25s} {frac:6.1%} connected at {t:5.0f} s  peak {peak:6.0f} attempts/s  "
          f"{att:4.1f} attempts per gateway")
print(f"ideal (capacity-limited): {N / CAP:.0f} s")
```

```
 fixed 5 s                  0.0% connected at  3600 s  peak   6025 attempts/s  719.0 attempts per
gateway
exponential, no jitter      0.0% connected at  3600 s  peak   6025 attempts/s  18.0 attempts per
gateway
exponential, full jitter  100.0% connected at   289 s  peak    246 attempts/s   1.1 attempts per
gateway
ideal (capacity-limited): 60 s
```

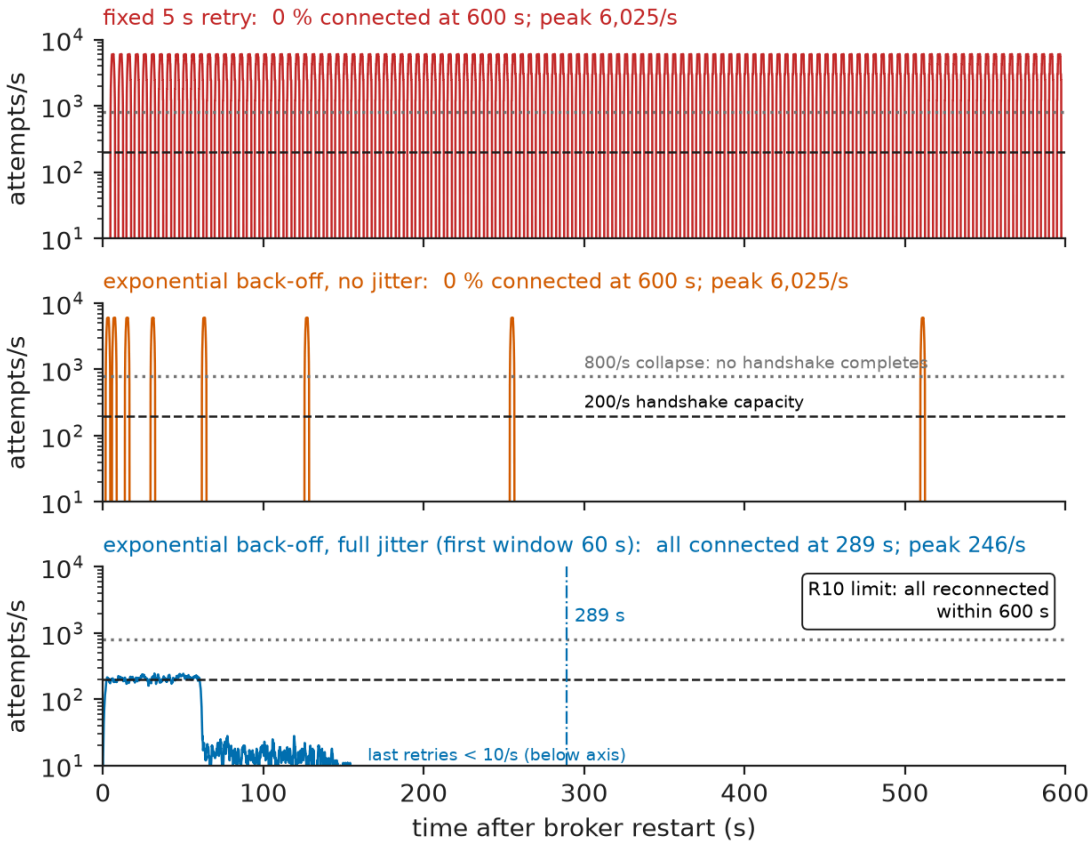


Figure 11.6 — Reconnection attempts per second after a broker restart for three retry policies, from the simulation of 12,000 gateways against a broker completing 200 handshakes per second.

Worked example — Why jitter, and how wide. Figure 11.6 shows the three policies. A fixed 5 s retry keeps every retry wave above the collapse threshold indefinitely: after an hour, no gateway has connected and each has tried 719 times. Exponential back-off *without* jitter fails differently. Every gateway that failed together waits the same time and retries together, so the waves stay synchronised and each one collapses the broker. Exponential back-off with *full jitter*, a delay drawn uniformly between zero and an exponentially growing cap, spreads the attempts. Its first window is the important parameter. Set to $N/C = 12{,}000/200 = 60$ s, it offers the broker roughly its capacity from the first second. The fleet is reconnected in 289 s, with 1.1 attempts per gateway, inside R10's 10 minutes. With a 5 s first window, the first wave would offer 2,400 attempts per second and collapse the broker before the back-off could help.

> **Status of the principle: Retries must be desynchronised and sized to capacity**
> *Derived.* If N clients retry against a service that completes C requests per second, and failed attempts consume capacity, the offered load must be kept near C. Randomising each client's delay over a window of at least N/C achieves that, and synchronised retries cannot. The result follows from the load model and holds whenever rejected work is not free.

Availability

The listing in Section 11.6 combines the availabilities of the blocks in Figure 11.1: gateway 99.95 %, Ethernet 99.5 %, LTE-M 99.0 %, cloud platform 99.95 %, notification provider 99.9 %, direct SMS 99.0 %. Series blocks multiply; parallel paths multiply their unavailabilities.

Worked example — Which availability requirement does the design meet? The remote dashboard depends on gateway, WAN (Ethernet *or* LTE) and cloud in series, giving 99.895 %, or 552 minutes of downtime a year. That misses a 99.9 % target, and nothing short of a second gateway per site would fix it. The *alarm*, however, has two paths, the cloud path and the direct SMS. Its availability is 99.948 % (271 min/year), and almost all of that unavailability is the gateway itself. This is why R9 was written as two requirements rather than one: the alarm, which protects patients, at 99.9 %, and the dashboard at 99.8 %. The quality team accepted the split because the record itself is protected separately, by store-and-forward. During a dashboard outage data is late, not lost.

FMEA

Table 11.8 — FMEA of the monitoring fleet (Revision B).

Failure mode	Effect	Cause	Detection	Mitigation
Half-open TCP connection	Alarm lost (Rev A)	NAT idle timeout shorter than keepalive	PUBACK timeout 10 s	Keepalive 60 s; QoS 1; pending alarm until cloud ack; SMS fallback
Reconnection storm	Fleet offline for hours	Synchronised retries after broker restart	Broker connection-rate metric	Full-jitter back-off, first window 60 s; rate limits
Duplicate readings	Inflated records, false statistics	QoS 1 redelivery	Duplicate (gateway, seq) at ingest	Idempotent ingest
Gateway clock lost	Records stamped 1970 or skewed	RTC battery flat and no NTP	Timestamp sanity check against broker time	Battery-backed RTC; refuse to timestamp before NTP or RTC valid; mark as "time uncertain"
Probe silent	Gap in record	Flat battery, moved fridge, metal shielding	Probe-lost alarm at 10 min	Battery telemetry predicts end of life; radio module outside fridge
SIM data exhausted	No failover	Runaway logging over LTE	Modem byte counter, cloud usage alert at 70 %	Logs never uploaded over LTE; pooled data plan
Fleet-wide certificate expiry	Fleet offline	Same validity dates	Expiry monitor	Renew at 80 % + random 0–30 days
Bad firmware update	Gateways down	Defective image	Health check after boot	A/B rootfs with automatic rollback; staged rollout
eMMC full or worn	Queue lost	Unbounded logs, per-reading commits	Free-space and wear-indicator telemetry	Bounded logs in RAM; per-minute commits; queue limit 30 days
Ingest crash after PUBACK	Silent data loss	Broker ack treated as persistence	Sequence-gap audit	Delete from gateway only after ingest confirmation

Verification

Table 11.9 — Verification of the Revision B fleet (pilot: 12 stores, 104 probes, 8 weeks; load test on staging broker).

Req.	Test	Limit	Result	Pass/fail
R2	Timestamp audit against GNSS-disciplined clock	±2 s	max 0.8 s	Pass
R3	200 injected excursions (heated probe), WAN up and WAN cut	Local ≤ 120 s, remote ≤ 300 s	Local max 96 s; remote max 171 s (WAN cut)	Pass
R4	72 h WAN cut at the 16-probe store; 2 h power cut	0 gaps	0 gaps; drain 4.9 min	Pass
R5	Probe charge per event measured; budget	≥ 3 years	8.1 years predicted	Pass
R6	Modem byte count, 7 days forced LTE	≤ 50 MB/month	26.8 MB/month equivalent	Pass
R8	Third-party penetration test	No high-severity finding	1 medium (verbose TLS error text), fixed	Pass after fix
R9	Availability model + pilot data	Alarm ≥ 99.9 %	Model 99.948 %; pilot: no missed alarm	Pass (model)
R10	Kill broker node with 12,000 simulated clients	All reconnected ≤ 600 s	312 s; broker CPU peak 84 %	Pass
Regression	Rev A scenario: NAT drop 5 min, alarm during idle period	Alarm delivered	Delivered via LTE at +14 s	Pass

The measured 26.8 MB per month is above the computed 25.4 MB. The difference was traced to TCP retransmissions on the LTE link and to the daily status and configuration messages that the budget had left out. It is recorded in the budget as an 8 % allowance, not explained away.

What remains open

Open question	Where it is answered
Moving more computation onto the device or gateway: anomaly detection on the temperature curves, compressor-failure prediction	Chapter 12; training side in Chapter 32
The radio link budget for probes in large stores and cold rooms	Chapter 26
A complete energy monitor that uses this connectivity stack	Chapter 35
A complete multi-hop wireless sensor network with its own security and deployment tests	Chapter 38
Security of industrial control networks (zones, conduits, IEC 62443 security levels)	Chapter 21
Probe calibration and the uncertainty of the glycol-buffered measurement	Chapter 7, Chapter 9

Next: Chapter 12 asks what happens when the data is too large to send at all, and the device must decide on its own what it is looking at.

Chapter 12 — Edge Computing and Embedded AI

Part III · Embedded and Connected Systems

A water utility fits 400 battery-powered condition-monitoring nodes to the 7.5 kW centrifugal pumps of its booster stations. Each node listens to its pump's vibration, classifies the pump's condition on the device and sends a few bytes over LoRaWAN. In the lab, on a test rig with seeded faults, the classifier scored 98.7 % accuracy. In the first week of service, 152 of the 400 pumps, all of them healthy, reported "imbalance". Two causes were found, and neither was in the model. The training data had been recorded with the accelerometer at ±16 g full scale. The production firmware configured ±4 g but kept the ±16 g conversion constant, so every amplitude it computed was four times too large: +12 dB on every band energy. A healthy pump with a four-times-too-large running-speed peak looks exactly like an unbalanced one. When that was fixed, field accuracy against technicians' inspections settled at 81 %, not 98.7 %. The lab score had been computed on randomly split windows, so every test window had near-identical siblings from the same pump in the training set, and the score measured how well the model recognised pumps, not faults. This chapter integrates such a node properly, from the decision to compute at the edge to the failure handling that keeps a learned model from becoming a confident source of false alarms.

> **When should data be turned into decisions on the device, and how is a learned model fitted into the memory, time and energy of a microcontroller?**
> **How is an embedded model validated so that the number measured in the lab is the number seen in the field, and what does the device do when the model is wrong?**

Concept	Engineering destination
Edge versus cloud processing	Computing locally when the raw data is too large, too slow or too costly in energy to move
Resource constraints	MCU, NPU or Linux edge platform chosen against flash, SRAM, cycles and sleep current
Data preprocessing	A DSP front end (FFT, envelope, band energies) whose contract with the training pipeline is enforced
Model deployment	Model as a signed, versioned blob with a feature-schema hash, updatable without new firmware
Quantisation	int8 weights and activations, verified for agreement with the float reference
Inference performance	A cycle budget measured with the cycle counter, stage by stage
Memory and power constraints	Flash, SRAM and energy budgets with margins, and battery life
Sensor fusion	Shaft speed from magnetic flux to locate fault frequencies; temperature trend; k-of-n temporal voting
Model validation	Split by pump, golden-vector equivalence, shadow-mode field trial
Failure handling	Sensor self-checks, out-of-distribution detection, ISO 20816 fallback, "unknown" as an answer

Roadmap. The node and its requirements (12.1) → edge versus cloud (12.2) → resource constraints and platform choice (12.3) → data preprocessing (12.4) → sensor fusion (12.5) → model deployment (12.6) → quantisation (12.7) → inference performance (12.8) → memory and power constraints (12.9) → model validation (12.10) → failure handling and verification (12.11).

12.1 The reference system: a pump condition-monitoring node

> **Boundary.**
> The deployment flow, int8 affine quantisation, integer requantisation and accelerator trade-offs are developed in *Electronics and Computing*, Ch. 35 §35.5–35.7. Training, feature engineering, classification, anomaly detection and model evaluation are the subject of Chapter 32; spectral analysis and feature extraction as DSP practice belong to Chapter 27, whose mains-powered node for the same utility's 37 kW pump sets is the larger sibling of the battery node designed here. This chapter is the deployment side: the model as one component of an embedded system. The device-level integration method is that of Chapter 10, and the connectivity that of Chapter 11.

The node clamps onto the pump's drive-end bearing housing with a stud. It contains a wide-band MEMS accelerometer, a three-axis magnetometer that senses the motor's stray flux, a temperature sensor coupled through the stud, a microcontroller, a LoRaWAN radio and an AA lithium–thionyl-chloride cell. Its architecture and pipeline are shown in Figure 12.1. Every 10 minutes it wakes, records 1.23 s of vibration and 1 s of flux, computes features, runs a classifier and an anomaly detector, and updates its state. Once an hour,

or at once if the state changes, it sends a short message: condition class, confidence, an anomaly score and the ISO 20816 vibration severity.

Table 12.1 — Requirements for the condition-monitoring node.

ID	Requirement	Target	Verification
R1	Fault classification	Macro-averaged recall ≥ 90 % over normal, imbalance, misalignment, looseness, bearing defect and cavitation, on pumps not used in training	Grouped validation; field inspections
R2	False alarms	≤ 1 per node per year (95 % upper bound)	Pilot fleet, Poisson bound
R3	Detection delay	A developing fault reported ≤ 2 h after it is detectable in single windows	Seeded-fault rig
R4	Battery life	≥ 5 years on one AA Li-SOCl2 cell (2.6 Ah)	Charge-per-event measurement and budget
R5	Uplink	≤ 24 messages/day, ≤ 12 B payload; within EU868 1 % duty cycle and 30 s/day network fair use	Airtime calculation; network logs
R6	Numerical equivalence	On-device features within 0.1 dB of the reference pipeline; int8 model agrees with float on ≥ 99 % of windows	Golden-vector test
R7	Resources	Flash ≤ 75 %, SRAM ≤ 60 %	Linker map, stack watermark
R8	Graceful degradation	Sensor faults detected within one cycle; out-of-distribution windows reported as "unknown"; ISO 20816 severity always reported	Fault injection
R9	Model update	Signed, versioned, schema-checked; updatable without a firmware update	Update test, schema-mismatch test

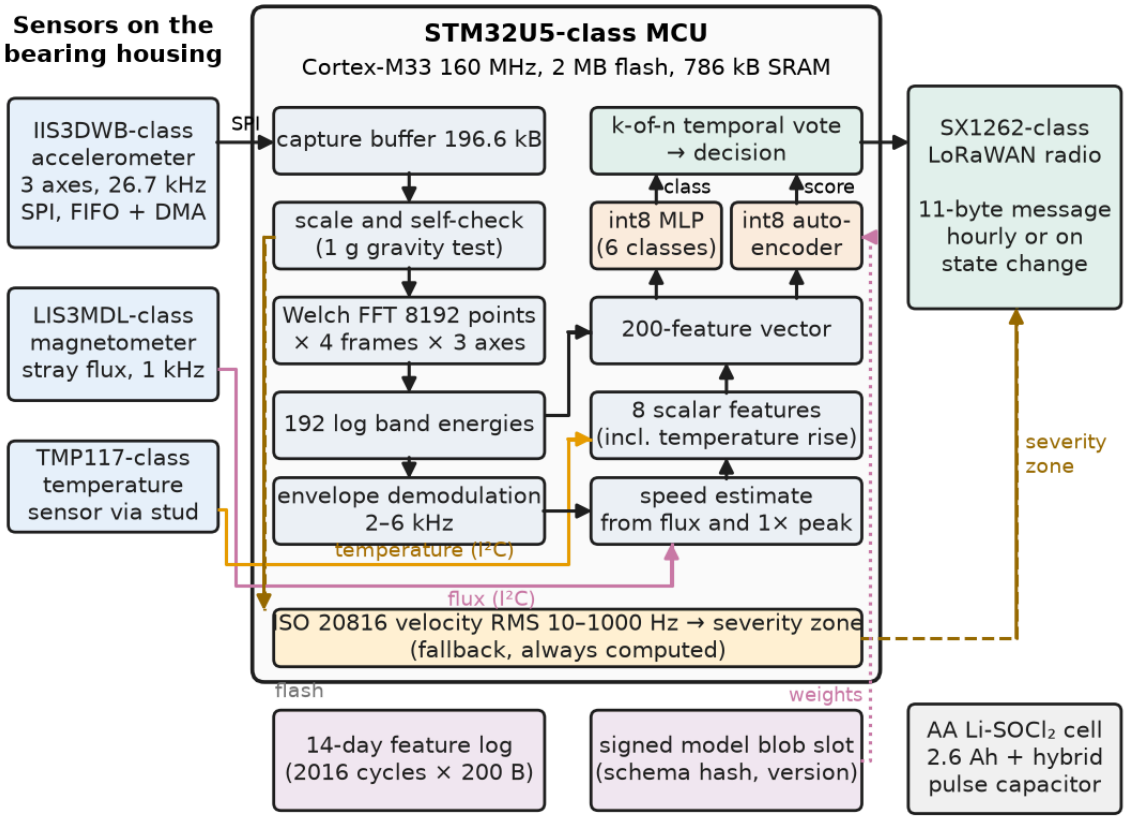


Figure 12.1 — Architecture and processing pipeline of the pump condition-monitoring node.

12.2 Edge versus cloud processing

The first architecture decision is *where* the vibration becomes a decision. Four placements are possible, and the numbers decide quickly.

Worked example — Can the raw data leave the device? The accelerometer produces 26,667 Hz × 3 axes × 16 bit = 1.28 Mbit/s while it runs. One 1.23 s capture of 32,768 samples per axis is 196.6 kB. Sent once per 10-minute cycle, that is an average of 196,608 × 8/600 = 2.62 kbit/s. LoRaWAN in the EU868 band is limited to a 1 % duty cycle per sub-band. At the fastest data rate (SF7, 125 kHz), a maximum 222-byte frame occupies 369 ms of air, so the legal average throughput is about 48 bit/s, 54 times too little *before* any network fair-use policy. Over LTE-M the data could move, but at a cost in energy: about 7 s of radio time per capture at roughly 0.2 A, 1.45 C per cycle. The budget listing in Section 12.9 puts the average current at 2.42 mA, which would empty the AA cell in about 31 days instead of lasting 13 years. The raw data cannot leave the device. The decision has to be made on it.

Table 12.2 — Placement options for the analysis.

Option	Data leaving the node	Node energy	Decision latency	Model update	Verdict
Stream raw to cloud (LTE-M)	196.6 kB per cycle	About 2.4 mA average: 1 month on AA	Seconds	Trivial (cloud)	Fails R4 by 60×
Features on node, model in cloud	200 B per cycle (2.7 bit/s)	Low	Minutes	Trivial	Fails R5: 144 messages/day of 200 B exceed fair use
Features and model on node	11 B per hour or on change	12.6 µA average	≤ 10 min per window, ≤ 1 h after voting	Signed model blob over the air (Section 12.6)	**Chosen**
Wired sensors to a Linux edge box per station	Raw data on a local bus	Mains (5–15 W)	Seconds	Easy	Needs cabling and power in every station; kept for the 5 largest stations

Status of the principle: Move the computation to the data when the data is the expensive thing

Derived. If moving a sample costs more energy or bandwidth than processing it, as it does whenever the radio's energy per bit times the data volume exceeds the energy of the computation, local processing minimises the total cost. The comparison is a ratio of measured quantities, and it reverses when links are wired and powered.

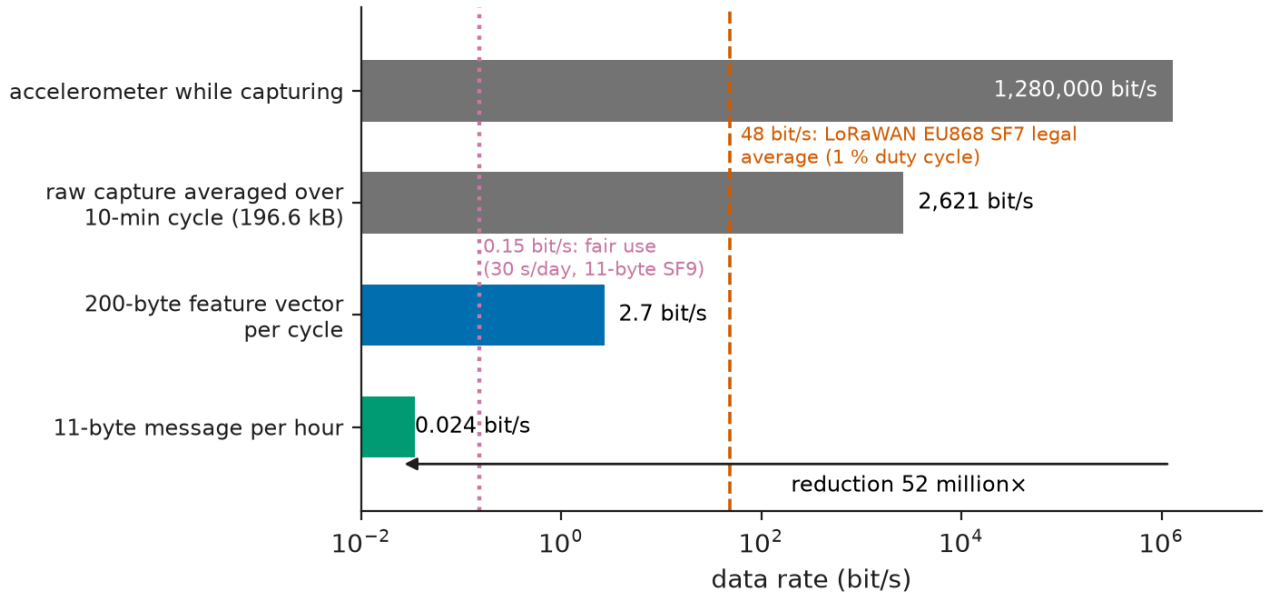


Figure 12.2 — Data rate at each stage of the edge pipeline against the LoRaWAN capacity.

Figure 12.2 puts the stages on one axis: from sensor to message the data shrinks 52-million-fold. The pipeline is a compressor that keeps only the evidence the decision needs. Its design, Sections 12.4 and 12.5, matters more to the result than the network that follows.

12.3 Resource constraints

> **Boundary.**
> MCU selection criteria are in *Electronics and Computing*, Ch. 25 §25.3; the roofline model and the cost of data movement in Ch. 35 §35.4–35.5.

The constraints come from the requirements. SRAM must hold a 196.6 kB capture plus FFT work buffers. The CPU must finish the processing within seconds, and the energy of the processing must be small next to the capture and the radio. The sleep current must be in single microamps. The platform must survive −20 to +60 °C in an unheated pump station.

Table 12.3 — Compute-platform candidates (typical values; check current datasheets).

Platform	Compute	Flash / SRAM	Run current / sleep	ML acceleration	Assessment
STM32U5-class MCU	Cortex-M33 160 MHz, FPU, DSP extension	2 MB / 786 kB	about 20 µA/MHz / 2–5 µA Stop2 with SRAM	CMSIS-NN on CPU, 0.5 MAC/cycle sustained	**Chosen**: holds capture and buffers, low sleep current, mature DSP libraries
nRF5340-class MCU	Cortex-M33 128 MHz + network core	1 MB / 512 kB	similar / 2–4 µA	CPU only	Integrated BLE not needed; SRAM tight for full 3-axis capture
MCU with NPU (MCX N94x, Alif Ensemble E3, STM32N6 classes)	M33/M55 + NPU, 100–600 GOPS int8	1–4 MB / 1–13 MB	higher run, 10–100 µA sleep for large SRAM retention	NPU, 50–250 MAC/cycle on CNNs	Kept for Revision B if a CNN is adopted (Section 12.10)
Linux edge SoC (i.MX 8M Plus class)	4 × Cortex-A53 + 2.3 TOPS NPU	eMMC / 1–4 GB DRAM	2–4 W / no µA sleep	NPU, GPU	For the wired edge boxes only

The decisive figure is not compute but SRAM, because the capture must sit in memory while it is transformed, and sleep current. The chosen part's compute is ample, as Section 12.8 shows: the network itself takes 0.35 ms per cycle. Choosing an NPU for this model would have bought a 100-fold speed-up of 0.4 % of the work.

12.4 Data preprocessing

> **Boundary.**
> Sampling, aliasing and anti-alias filtering are owned by *Circuits, Signals and Communications*, Ch. 19; the DFT, windows and spectral leakage by Ch. 22; filter design by Ch. 23. The filter chain, envelope spectrum, bearing kinematics and feature set are designed in Chapter 27 §27.2–27.4 for a mains-powered node; here they are cut down to a battery budget and fixed as a contract with the training pipeline.

Preprocessing turns 196.6 kB of samples into a 200-number feature vector. Each step is a design choice with a number behind it.

- **Sampling.** The accelerometer's internal digital filter limits the band to about 6.3 kHz at 26.7 kHz output rate. Bearing-defect impacts excite housing resonances at 2–6 kHz, which the envelope stage demodulates, so this band is needed. Samples stream through the sensor's FIFO by SPI and DMA while the CPU sleeps.
- **Scaling and self-check.** Raw codes are converted to g with the sensitivity of the *configured* full scale, read back from the sensor's control register rather than assumed. The mean of each axis over the capture gives the gravity vector, whose magnitude must be 1 g (Section 12.11).
- **Spectrum.** For each axis, four 8,192-point frames (Hann window, 3.26 Hz bins) are averaged as a Welch estimate. Averaging four frames reduces the variance of each bin's power estimate fourfold, and that is what makes features repeatable from cycle to cycle.
- **Band energies.** 64 logarithmically spaced bands from 10 Hz to 6 kHz per axis, in dB re 1 $(mg)^2$. That gives 192 features that keep the shape of the spectrum without its fine detail.
- **Scalar features.** Velocity RMS 10–1,000 Hz (the ISO 20816 quantity), kurtosis and crest factor of the radial axis, envelope-spectrum energy at the bearing outer- and inner-race defect frequencies, the 1× and 2× running-speed amplitudes, and the bearing-housing temperature rise over ambient. Eight features. Several of them depend on knowing the shaft speed, which is where fusion enters (Section 12.5).

The most important property of the preprocessing is not any single step. It is that the device computes *exactly the same function* as the Python pipeline that produced the training data. Every parameter that defines that function goes into one feature-schema file: output data rate, full scale, window, frame length, band edges, units, feature order. A hash of that file is compiled into both the firmware and the model metadata (Section 12.6). The opening failure was a broken contract: the model had been trained on one function and was fed another.

> **Status of the principle: Training and serving must compute the same features**
> *Fundamental.* A learned model is a function of its inputs as they were defined during training. If deployment changes the definition (scale, units, window, order), the model's outputs are undefined, whatever its validated accuracy. This holds by construction for any model, and no amount of model quality compensates for it.

12.5 Sensor fusion

> **Boundary.**
> Induction-motor slip is from *Electrical Energy and Machines*, Ch. 6 §6.3. Kalman and complementary filters for fusing inertial sensors in robots and drones are applied in Chapter 15 and Chapter 16. Binomial probabilities and independence are from *The Mathematics Universe*, Ch. 19 §19.4.

The node fuses information at three levels. Each one fixes a specific weakness of vibration alone.

Signal level: shaft speed from flux

Fault frequencies scale with shaft speed. Imbalance appears at 1×, misalignment at 2×, a bearing's outer-race defect at about 3.58× for the drive-end bearing of these motors, and blade-pass at 6×. Half of the utility's pumps run on variable-frequency drives, between 30 and 50 Hz supply. A feature computed at a fixed nominal speed therefore looks in the wrong place.

Worked example — Locating the outer-race frequency. A 2-pole motor on a drive set to 40.00 Hz runs at 2 % slip, so the shaft turns at $40.00 \times 0.98 = 39.20$ Hz and the outer-race defect frequency is $3.58 \times 39.20 = 140.3$ Hz. A pipeline that assumed the nameplate speed of 2,970 rpm (49.5 Hz) would look at $3.58 \times 49.5 = 177.2$ Hz, 37 Hz and 11 bins away, and would never see the fault. Vibration alone gives a speed estimate from the largest peak near 1×, but at low load the 1× peak can be smaller than the blade-pass peak or a structural resonance. The magnetometer gives the supply frequency directly from the stray-flux spectrum, 40.00 Hz here, and for a 2-pole induction motor the shaft speed must lie between 0.96 and 1.00 times it. The fused estimate searches for the 1× vibration peak only inside that window, 38.4–40.0 Hz (Figure 12.3), and the peak found there, 39.20 Hz, sets all the order-based features. On the test rig this fusion cut speed-estimate failures from 6.1 % of windows (vibration alone, at low load) to 0.2 %.

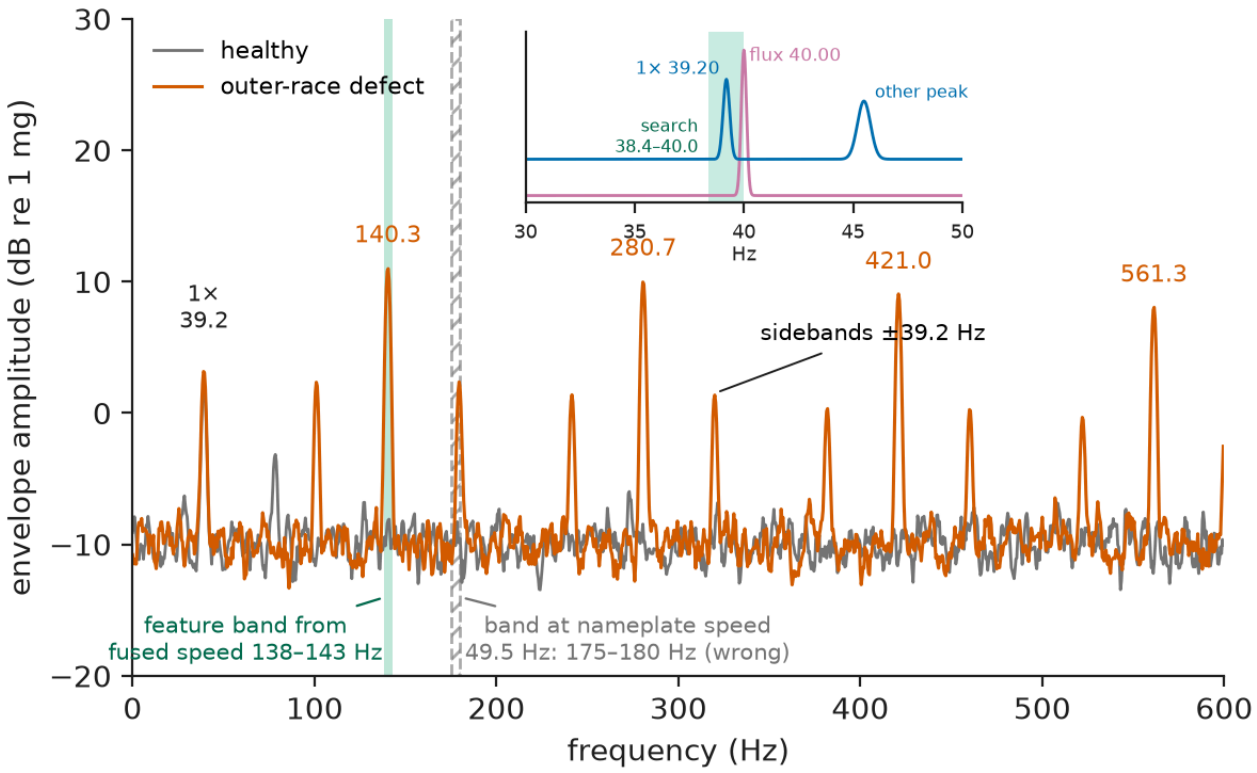


Figure 12.3 — Envelope spectra of the radial acceleration for a healthy pump and for a pump with an outer-race defect, at a shaft speed of 39.20 Hz on a 40 Hz drive.

Feature level: temperature trend

Bearing temperature rises with friction, but it also rises with ambient temperature and load. The feature used is the bearing-housing temperature *minus* the station's ambient temperature, the latter estimated from the node's internal sensor at the end of the sleep period, smoothed over 24 h. It enters the classifier as one of the eight scalar features. Lubrication faults show in it days before any vibration change.

Decision level: temporal voting

A per-window classifier with a 2 % false-positive rate, run 52,560 times a year, would raise about 1,050 false alarms per node per year, three orders of magnitude over R2. Faults develop over days and false positives are mostly isolated, so the node reports a fault only when at least k of the last $n = 6$ windows (one hour) agree.

Worked example — Choosing k. If window errors were independent with $p = 0.02$, the probability that a sliding block of 6 windows contains at least k false positives is a binomial tail. Multiplied by 52,560 evaluations per year (a union bound), it gives about 8 false alarms per year for $k = 3$, 0.12 for $k = 4$ and 0.001 for $k = 5$. A real fault detected in single windows with recall 0.9 is reported within the hour with probability 0.9987, 0.984 and 0.886 for the same three values. $k = 4$ meets R2 and R3 together. The independence assumption is optimistic, because errors cluster when a pump's operating point sits near a class boundary for hours. The computed 0.12 per year is therefore a design target, and R2 is verified on field data (Table 12.8), not on this calculation.

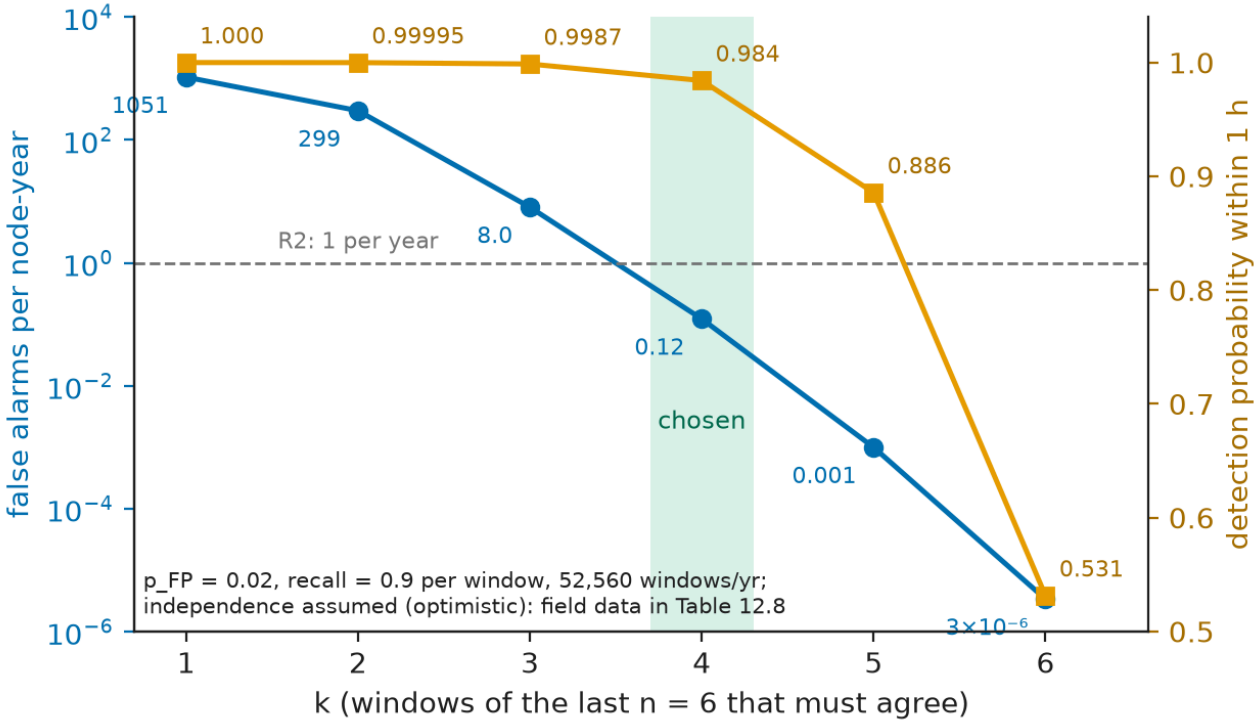


Figure 12.4 — False alarms per node-year and detection probability within one hour for k-of-6 voting.

Figure 12.4 shows the trade-off. Below $k = 4$ false alarms dominate; above it, missed and late detections do.

12.6 Model deployment

A model on a device is a piece of firmware with unusual properties. It is produced by a data pipeline rather than by a compiler. Its correctness depends on inputs defined elsewhere. And it will be replaced more often than the code around it. The deployment design makes those properties explicit.

The artefact. The trained model (Chapter 32) is exported as an int8 TensorFlow Lite flatbuffer. For a model this small, a generated C array of weights run by CMSIS-NN fully connected kernels would do just as well, at a smaller runtime cost. The blob carries a header with a magic number, model version, feature-schema hash, input and output dimensions, the int8 quantisation parameters, the out-of-distribution threshold, the training-data identifier, and a signature over all of it.

The slot. Firmware and model live in separate flash regions. A firmware update (A/B slots with signature check, Book 4 §25.9) changes code. A model update changes only the 64 kB model slot and can be delivered over LoRaWAN firmware-update-over-the-air multicast in about 900 fragments, or by a technician's phone over a maintenance BLE link. At boot and after every model update, the firmware checks the model's signature and its compatibility with the firmware's own feature schema, and refuses a mismatch. The previous model is kept, so a refused or failed update leaves the old one running.

```
 /* model_guard.c -- model compatibility and sensor-scale checks (C11, illustrative) */
#include <math.h>
#include <stdbool.h>
#include <stdint.h>

typedef struct {                  /* header of the signed model blob */
    uint32_t magic;               /* 0x4D4F444C, "MODL" */
    uint32_t model_version;
    uint32_t schema_hash;         /* hash of ODR, full scale, window, bands, units, order */
    uint16_t n_in, n_out;
    float    ood_threshold;       /* autoencoder error at the 99.5th training percentile */
} model_hdr_t;

/* Generated at build time from the same schema file that the training pipeline uses. */
extern const uint32_t FEATURE_SCHEMA_HASH;
bool signature_valid(const void *blob, uint32_t len);   /* ECDSA P-256, in bootloader lib */

bool model_compatible(const model_hdr_t *h, uint32_t len)
{
    return signature_valid(h, len) && h->magic == 0x4D4F444Cu
        && h->schema_hash == FEATURE_SCHEMA_HASH && h->n_in == 200u && h->n_out == 6u;
}

/* Static acceleration must be 1 g in magnitude at any mounting angle.
   mean_mg: per-axis mean of the capture in milli-g after scaling. */
bool accel_scale_ok(const float mean_mg[3])
{
    float g = sqrtf(mean_mg[0] * mean_mg[0] + mean_mg[1] * mean_mg[1]
                    + mean_mg[2] * mean_mg[2]);
    return g > 950.0f && g < 1050.0f;      /* +/-5 %: a x4 scale error gives 4000 or 250 */
}
```

The two functions are the deployment-side answers to the opening failure. The first makes a model and a pipeline that disagree about the feature definition refuse to run together. The second detects at run time a scale error that a schema check cannot see, because the configuration was right in the file and wrong in the register. Either one alone would have stopped the 152 false reports in the first cycle.

12.7 Quantisation

> **Boundary.**
> The affine int8 mapping $r = S(q - Z)$, per-channel weight scales, int32 accumulation, fixed-point requantisation and the comparison of post-training quantisation with quantisation-aware training are in *Electronics and Computing*, Ch. 35 §35.6; quantisation noise and fixed-point formats in *Circuits, Signals and Communications*, Ch. 21 §21.2 and §21.5. Here quantisation is a verification question: does the deployed model still make the same decisions?

The classifier is a multilayer perceptron, 200 → 64 → 32 → 6, with 15,142 parameters. The anomaly detector is an autoencoder, 200 → 32 → 8 → 32 → 200, with 13,584 parameters. At int8 they need 28.7 kB of flash instead of 115 kB at float32. Post-training quantisation with per-channel weight scales is used, calibrated on 2,000 training windows. The question that matters for integration is whether the int8 model *agrees* with the float model it was validated as, on data it has not seen. The answer must come from a test, not from an expectation.

The listing demonstrates the test on a synthetic stand-in for the real feature set: dB band energies from 48 pumps, six conditions, with pump-specific offsets. A softmax classifier is trained on 36 pumps and tested on 12 held-out pumps, in float and with int8 weights and inputs. The listing then repeats the int8 evaluation with the opening failure's +12 dB scale error applied to every feature.

```
 # Softmax classifier on dB band features: float vs int8, and a +12 dB scaling fault
import numpy as np

rng = np.random.default_rng(3)
P, W, D, C = 48, 60, 24, 6
cls = np.arange(P) % C
centres, sig = rng.normal(0, 1, (C, D)), rng.normal(0, 1.6, (P, D))
X = 6.0 * np.concatenate([centres[cls[p]] + sig[p] + rng.normal(0, 0.3, (W, D))
                          for p in range(P)])                       # dB
y, pump = np.repeat(cls, W), np.repeat(np.arange(P), W)
te = np.isin(pump, rng.choice(P, 12, replace=False))              # held-out pumps
mu, sd = X[~te].mean(0), X[~te].std(0)
Z = (X - mu) / sd
Wt, b = np.zeros((D, C)), np.zeros(C)
Y = np.eye(C)[y[~te]]
for _ in range(500):                                               # gradient descent
    p = np.exp(Z[~te] @ Wt + b); p /= p.sum(1, keepdims=True)
    g = (p - Y) / len(Y)
    Wt -= 0.5 * (Z[~te].T @ g + 1e-3 * Wt); b -= 0.5 * g.sum(0)

s_x = np.abs(Z[~te]).max() / 127                                   # per-tensor input scale
s_w = np.abs(Wt).max(0) / 127                                      # per-class weight scales
q_w = np.round(Wt / s_w).astype(np.int32)
q_b = np.round(b / (s_x * s_w)).astype(np.int32)

def predict_int8(Zin):
    q_x = np.clip(np.round(Zin / s_x), -128, 127).astype(np.int32)
    return ((q_x @ q_w + q_b) * s_w).argmax(1)                     # int32 acc, dequantised

f32 = (Z[te] @ Wt + b).argmax(1)
i8 = predict_int8(Z[te])
print(f"float accuracy {np.mean(f32 == y[te]):.1%}, int8 accuracy {np.mean(i8 == y[te]):.1%}, "
      f"agreement {np.mean(f32 == i8):.2%}, max |acc| {np.abs(np.clip(np.round(Z / s_x), -128,
127) @ q_w).max():.0f}")
bad = predict_int8((X[te] + 12.04 - mu) / sd)                      # full-scale mismatch, x4
print(f"+12 dB fault: accuracy {np.mean(bad == y[te]):.1%}, "
      f"predicted-class histogram {np.bincount(bad, minlength=C)}")
```

```
 float accuracy 81.1%, int8 accuracy 81.1%, agreement 100.00%, max |acc| 49646
+12 dB fault: accuracy 42.4%, predicted-class histogram [ 97  59   6   0 179 379]
```

Worked example — Reading the quantisation result. On the 720 held-out windows the int8 model agrees with the float model on every window, and its accuracy (81.1 % on this deliberately hard synthetic set) is unchanged. The largest accumulator magnitude, 49,646, needs 17 bits including sign, far inside int32. Quantisation therefore costs nothing measurable here. The real model met R6 with 99.7 % agreement on 6,400 validation windows; the 0.3 % of disagreements were all windows whose two top classes had confidences within 0.02 of each other. The same listing shows where the real risk is. A +12 dB error *in the input* halves the accuracy and drives 558 of 720 windows into just two classes, which is the signature of the field failure. In int8 there is a second effect: the shifted inputs saturate at the edge of the calibrated input range, so the model is not even computing on the shifted values correctly. Quantisation error is a fraction of a percent; a units error is a factor of two in accuracy.

> **Status of the principle: int8 post-training quantisation preserves decisions of small models**
> *Empirical.* For classifiers on well-scaled features and for convolutional networks, int8 post-training quantisation with per-channel weights typically changes accuracy by less than 1 %. The result must still be verified per model by an agreement test on held-out data, and it does not hold for inputs outside the calibration range.

12.8 Inference performance

"Inference performance" on a microcontroller means the time and energy of the *whole* decision, not of the network. The pipeline was profiled on the target with the Cortex-M33's DWT cycle counter around each stage, with caches warm and with the capture in SRAM. The listing in Section 12.9 reproduces the cycle budget from the per-stage estimates.

Table 12.4 — Per-cycle processing budget (Cortex-M33 at 160 MHz).

Stage	Estimate	Measured	Share
12 × 8,192-point real FFT (CMSIS-DSP, f32)	71.25 ms	69.8 ms	81 %
Envelope demodulation (band-pass, rectify, decimate, 2,048-pt FFT)	11.88 ms	12.6 ms	14 %
Band energies and scalar features	4.00 ms	3.7 ms	4 %
MLP + autoencoder, int8 (28,352 MACs)	0.35 ms	0.41 ms	0.5 %
Total	**87.5 ms**	**86.5 ms**	
Limit (soft: must finish well inside the 10-min cycle)	1 s		margin 91 %

The network is half a percent of the computation. The design consequence: optimising inference (a faster kernel, an NPU) is pointless, and the right lever is the FFT. Two options were evaluated. Fixed-point q15 FFTs are about 1.6 times faster on this core but lose tens of decibels of dynamic range to per-stage scaling, which buries the weak low-frequency bands. Analysing only the radial axis at full resolution and the other

two at 2,048 points would cut FFT time by 50 %, but it costs a measured 1.8 points of macro recall. Neither was needed. At 87 ms per 10 minutes, processing is 0.66 µA of the 12.6 µA budget (Figure 12.5).

Worked example — The CNN alternative. A 2-D convolutional network on a 64-band × 32-frame × 3-axis log-spectrogram was the obvious alternative. Its layers (three 3 × 3 convolutions to 16, 32 and 64 channels, a fourth at 64 channels, global pooling and a 6-way output) need 6.78×10^6 MACs and 60,902 parameters. At 0.5 MAC per cycle that is $6.78 \times 10^6 / (0.5 \times 160 \times 10^6) = 85$ ms, which would double the processing time, and its peak activation memory is 49 kB. Both fit. The CNN was not rejected for resources. It was rejected in validation (Section 12.10), and it is the candidate that would justify an NPU part in Revision B.

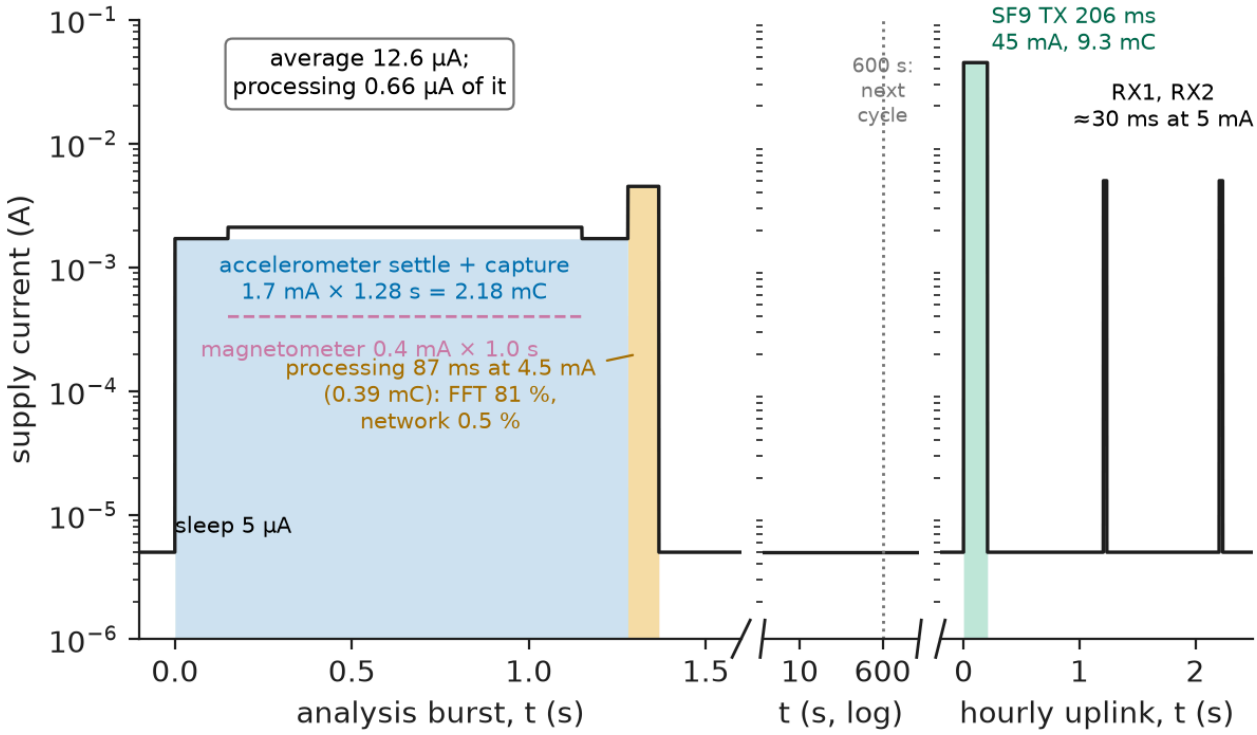


Figure 12.5 — Supply current of the node over one 10-minute analysis cycle and one hourly uplink, on a logarithmic current axis.

12.9 Memory and power constraints

> **Boundary.**
> The average-current battery model and the power-mode ladder are from *Electronics and Computing*, Ch. 25 §25.5, applied as in Chapter 10. Li-SOCl₂ cell behaviour is from *Electrical Energy and Machines*, Ch. 16 §16.2.

Table 12.5 — Memory budget (STM32U5-class, 2 MB flash, 786 kB SRAM).

Region	Size	Notes
Bootloader	64 kB	Secure boot, signature verification
Firmware slots A and B	2 × 448 kB	Image 236 kB (LoRaWAN stack, CMSIS-DSP/NN, RTOS, application)
Model slots current and previous	2 × 64 kB	Model blob 35 kB (28.7 kB weights + metadata)
Configuration and calibration	16 kB	
Feature log, 14 days	400 kB	2,016 cycles × 200 B, for re-analysis on request
Flash used / total	**1,504 / 2,048 kB**	**73 % (limit 75 %)**
Capture buffer (3 × 32,768 × int16)	196.6 kB	Filled by DMA
FFT input and output (f32, 8,192 points)	64 kB	Reused per frame
Features, band tables, envelope buffers	12 kB	
Inference arena (MLP + autoencoder)	8 kB	Largest pair of live tensors
LoRaWAN stack, RTOS, stacks (watermark + 25 %)	36 kB	
SRAM used / total	**317 / 786 kB**	**40 % (limit 60 %)**

The flash budget is the tight one, because of the feature log. A 14-day log allows the utility's analysts to download two weeks of features from a suspicious pump and re-analyse them with a newer model before deciding on a site visit. That capability was judged worth sitting just under the 75 % limit of R7. Flash margin is not left for firmware growth here, because the slots are sized separately with 47 % headroom over the image.

The energy budget computes the cycle's processing time, the average current and the life, and compares them with streaming raw data.

```
 # Edge pipeline budget: latency per cycle, average current, life; versus streaming raw data
F_CPU = 160e6
stages = {                        # cycles per analysis cycle (Cortex-M33, CMSIS-DSP/NN)
    "12 x 8192-pt real FFT":     12 * 0.95e6,
    "envelope demodulation":     1.9e6,
    "band energies, features":   0.64e6,
    "MLP + autoencoder (int8)":  28_352 / 0.5,          # MACs at 0.5 MAC/cycle
}
t_proc = sum(stages.values()) / F_CPU
for k, c in stages.items():
    print(f"{k:<26s} {c / F_CPU * 1e3:6.2f} ms")
CYCLE, HOUR = 600.0, 3600.0
charge = {                        # coulombs per period -> average current
    "sleep floor":        (5.0e-6 * CYCLE, CYCLE),
    "accel capture":      (1.7e-3 * 1.28, CYCLE),
    "flux capture":       (0.4e-3 * 1.0, CYCLE),
    "processing":         (4.5e-3 * t_proc, CYCLE),
    "LoRaWAN uplink SF9": (45e-3 * 0.206 + 0.3e-3, HOUR),
}
i_avg = sum(q / T for q, T in charge.values())
i_sd = 2.6 * 0.01 / 8760                                 # Li-SOCl2 AA self-discharge, A
life = 0.7 * 2.6 / (i_avg + i_sd) / 8760
i_raw = 1.45 / CYCLE + 5.0e-6                            # LTE-M upload of 196.6 kB per cycle
print(f"processing {t_proc * 1e3:.1f} ms; average {i_avg * 1e6:.1f} uA "
      f"(+{i_sd * 1e6:.1f} uA self-discharge); life {life:.1f} years")
print(f"stream raw over LTE-M: {i_raw * 1e3:.2f} mA, "
      f"life {0.7 * 2.6 / (i_raw + i_sd) / 24:.0f} days, {i_raw / i_avg:.0f}x the edge design")
```

```
 12 x 8192-pt real FFT       71.25 ms
envelope demodulation       11.88 ms
band energies, features      4.00 ms
MLP + autoencoder (int8)     0.35 ms
processing 87.5 ms; average 12.6 uA (+3.0 uA self-discharge); life 13.3 years
stream raw over LTE-M: 2.42 mA, life 31 days, 192x the edge design
```

Table 12.6 — Energy budget summary (R4).

Item	Average current
Sleep floor (MCU Stop2 with SRAM retention, sensors off, radio sleep)	5.0 µA
Accelerometer capture, 1.28 s per cycle	3.6 µA
Uplink, SF9, 206 ms at 45 mA hourly	2.7 µA
Magnetometer capture	0.7 µA
Processing, 87.5 ms per cycle	0.7 µA
Total load / with self-discharge	**12.6 µA / 15.6 µA**
Usable capacity (70 % of 2.6 Ah: pulse load, −20 °C)	1.82 Ah
Predicted life / requirement / margin	**13.3 years / 5 years / 166 %**

The large margin is deliberate and is spent in two places. The first is a 5-minute cycle at pump stations that request closer watch, which costs about 5 µA more. The second is a hybrid pulse capacitor across the cell, because a bare bobbin-type Li-SOCl₂ AA cell cannot deliver the 45 mA transmit pulse at −20 °C without a voltage delay that browns out the radio. That is the battery-as-a-resistance lesson of Chapter 10 in another chemistry.

The uplink message is 11 bytes: class (1), confidence (1), anomaly score (1), ISO 20816 velocity RMS (2), severity zone and flags (1), shaft speed (2), temperature rise (1), model version (1) and sequence (1). At SF9 its time on air is 206 ms, so 24 messages a day use 4.9 s of airtime, inside a 30 s/day fair-use policy, and a 0.006 % duty cycle, 0.6 % of the 1 % limit (R5).

12.10 Model validation

> **Boundary.**
> Metrics, cross-validation and uncertainty for trained models are developed in Chapter 32; confidence intervals in *The Mathematics Universe*, Ch. 20 §20.7. This section is about validating the *deployed* system.

Validating an embedded model is three different activities, and the opening failure skipped two of them.

1. Is the validation data independent of the training data?

Windows from one pump are not independent samples. They share the pump's mounting stiffness, resonances, impeller wear and sensor placement. A random split puts sibling windows of every pump on both sides, and a model that memorises pumps scores as if it had learned faults. The listing compares a memorising classifier (1-nearest-neighbour) with an averaging one (nearest class centroid) on the same synthetic feature set as Section 12.7, under a random-window split and under a split that holds out whole pumps.

```
 # Random-window split versus split by pump: how validation leaks
import numpy as np

rng = np.random.default_rng(3)
P, W, D, C = 48, 60, 24, 6                    # pumps, windows per pump, features, classes
cls_of_pump = np.arange(P) % C                # each pump recorded in one condition
centres = rng.normal(0, 1.0, (C, D))          # class signatures (log band energies)
sig = rng.normal(0, 1.6, (P, D))              # pump-specific mounting/resonance offset
X = np.concatenate([centres[cls_of_pump[p]] + sig[p] + rng.normal(0, 0.3, (W, D))
                    for p in range(P)])
y = np.repeat(cls_of_pump, W)
pump = np.repeat(np.arange(P), W)

def nn1(Xtr, ytr, Xte):                       # 1-nearest neighbour: memorises
    d = ((Xte[:, None, :] - Xtr[None, :, :]) ** 2).sum(-1)
    return ytr[d.argmin(1)]

def centroid(Xtr, ytr, Xte):                  # nearest class mean: averages pumps out
    m = np.stack([Xtr[ytr == c].mean(0) for c in range(C)])
    return ((Xte[:, None, :] - m[None]) ** 2).sum(-1).argmin(1)

test_rand = rng.random(len(y)) < 0.25
test_pump = np.isin(pump, rng.choice(P, 12, replace=False))
for split, te in (("random windows", test_rand), ("held-out pumps", test_pump)):
    for name, f in (("1-NN", nn1), ("class centroid", centroid)):
        acc = (f(X[~te], y[~te], X[te]) == y[te]).mean()
        print(f"{split:<15s} {name:<15s} accuracy {acc:6.1%}")
```

```
 random windows  1-NN            accuracy 100.0%
random windows  class centroid  accuracy  96.5%
held-out pumps  1-NN            accuracy  69.3%
held-out pumps  class centroid  accuracy  83.3%
```

Worked example — The split reverses the ranking. Under the random split (Figure 12.6) the memorising classifier wins, 100 % against 96.5 %, and a team choosing on that number would ship it. Under the pump split it loses, 69.3 % against 83.3 %, and both numbers fall by 13–31 points. The real project saw the same reversal. On randomly split windows the CNN of Section 12.8 scored 98.9 % and the feature MLP 97.1 %. With 9 pumps held out (grouped 5-fold cross-validation by pump, repeated), the MLP's macro recall was 92.4 % and the CNN's 89.6 %. The CNN had learned pump-specific spectral texture from only 38 training pumps. The MLP, fed speed-normalised physics features, generalised better. The MLP was chosen, and the CNN stays a Revision B candidate for when the labelled fleet data reaches a few hundred pumps.

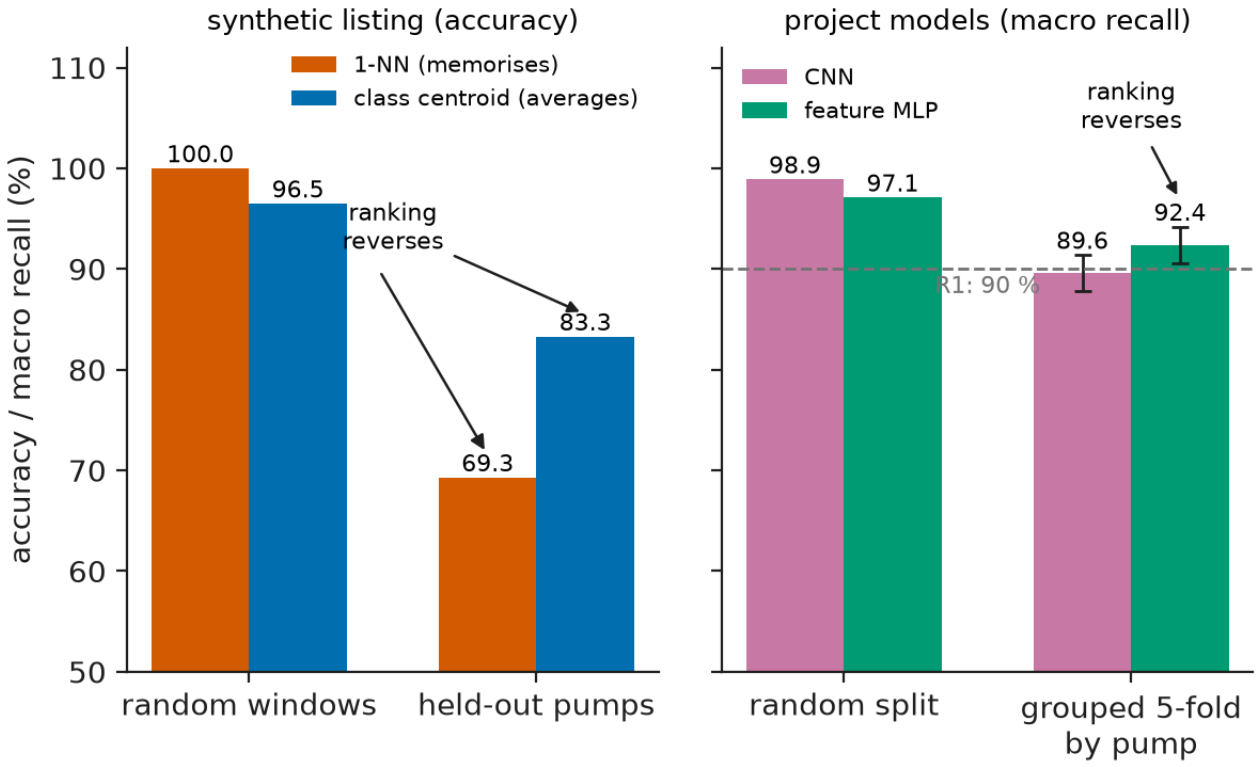


Figure 12.6 — Accuracy of a memorising and an averaging classifier under a random-window split and a held-out-pump split.

2. Does the device compute what the reference computes?

Golden vectors. 500 raw captures, from the test rig and from field nodes, are stored with the firmware's test suite. The reference Python pipeline computes their features and model outputs. The firmware, run on the target in a HIL fixture that injects the raw capture through the sensor's SPI interface, must reproduce every feature within 0.1 dB and every class decision in at least 99 % of cases (R6). This test runs in CI for every firmware and every model build. It is the test that would have caught the full-scale error before deployment, because the HIL fixture configures the sensor through the same driver as production.

3. Does it work on the pumps it is installed on?

Shadow mode. For the first four weeks on each new station, the node reports its classes but raises no alarms. Its outputs are compared with technicians' scheduled inspections and with portable-analyser surveys. Field macro recall from shadow mode on 60 pilot pumps was 90.8 %, consistent with the grouped estimate within its uncertainty. Shadow mode also produced the first labelled field data for retraining, the loop that Chapter 32 and the digital-twin methods of Chapter 33 take further.

> **Status of the principle: Validate on the unit of deployment**
> *Empirical.* When data comes in correlated groups (machines, patients, sites, vehicles), held-out performance must be measured on held-out *groups*, because deployment always meets new groups. Grouped estimates are consistently lower than random-split estimates on such data, and the gap grows with model capacity.

12.11 Failure handling and verification

> **Boundary.**
> Fault handling on the device (watchdogs, safe states, reset policy) is from Chapter 10 §10.7; fleet-level detection of silent nodes from Chapter 11 §11.7. Vibration-severity zones are from ISO 20816-1 (formerly ISO 10816-1).

A model can be wrong in two ways. It can be wrong on data like its training data, which is measured by validation and contained by voting. Or it can be confidently wrong on data unlike anything it has seen, which validation cannot measure. The node's failure handling is designed for the second kind.

Layered defences.

1. **Sensor self-checks, every cycle.** Gravity magnitude 1 g ± 5 % (scale and orientation). Noise floor above the sensor's datasheet floor (a dead or disconnected sensor reads too quiet). Saturation count below 0.01 % of samples (a loose node hammering its stud reads too loud). Flux supply frequency between 25 and 55 Hz (pump running). A failed check produces the outcome "sensor fault", never a class.
2. **Out-of-distribution detection.** The autoencoder's reconstruction error is compared with a threshold at the 99.5th percentile of the training distribution, stored in the model header. Above it, the window's class is reported as "unknown". The pump is doing something the model has not been trained on, perhaps a new pump type, a cracked baseplate or a different operating point.
3. **A physics fallback that is always computed.** The ISO 20816 velocity RMS (10–1,000 Hz) and its severity zone go into every message, whatever the model says. For this machine class the zone boundaries used are 0.71, 1.8 and 4.5 mm/s (A/B, B/C, C/D). A pump in zone D is reported as needing action even if the model says "normal" or "unknown".
4. **Temporal voting** (Section 12.5), so that a single bad window does not become an alarm.
5. **Model integrity and compatibility** (Section 12.6). A model that fails its signature or schema check is not loaded, and the node runs on the previous model or, if there is none, on the ISO fallback alone.

Table 12.7 — FMEA of the condition-monitoring node.

Failure mode	Effect	Cause	Detection	Mitigation
Feature-scale mismatch (opening failure)	Healthy pumps classified as imbalance	Register configuration differs from training pipeline	Gravity check fails (4 g or 0.25 g); golden-vector HIL test	Full scale read back from register; schema hash; CI golden vectors
Optimistic validation	Field accuracy far below lab	Random-window split	Grouped vs random gap	Group k-fold by pump; shadow mode
Wrong speed estimate	Fault features computed at wrong frequency	Low load: 1× peak below blade-pass peak	Flux and vibration speed disagree by > 3 %	Flux-bounded search window
Out-of-distribution input	Confident wrong class	New pump type, structural fault	Autoencoder error above threshold	Report "unknown" plus ISO severity
Node loosened on stud	Spurious looseness and impacts	Vibration, thermal cycling	Saturation count, gravity vector direction changed > 10°	Report "sensor fault"; maintenance ticket
Model update incompatible	Undefined outputs	Model trained on new schema shipped to old firmware	Schema hash mismatch	Refuse model, keep previous
Brownout at transmit	Reset, lost uplink	Li-SOCl₂ voltage delay at −20 °C	Reset cause log	Hybrid pulse capacitor; transmit only after supply check
Drift of normal baseline	Slow increase in false alarms	Impeller wear, seasonal temperature	Rising share of low-confidence windows	Per-pump baseline normalisation after commissioning; retraining from shadow data

Verification

Table 12.8 — Verification of the Revision B node (rig, HIL and 60-node pilot over 6 months).

Req.	Test	Limit	Result	Pass/fail
R1	Grouped 5-fold CV by pump (47 pumps); shadow-mode field check (60 pumps)	Macro recall ≥ 90 %	92.4 % (CV); 90.8 % (field)	Pass
R2	Pilot: 30 node-years, 3 false alarms	≤ 1 per node-year (95 % upper bound)	0.10 observed; upper bound 7.75/30 = 0.26	Pass
R3	Seeded outer-race defect on rig, 20 runs	≤ 2 h after first detectable window	Median 50 min, max 80 min	Pass
R4	Charge per event measured; budget	≥ 5 years	13.3 years predicted	Pass
R5	Network server logs, 30 days	≤ 24 messages/day, ≤ 30 s airtime/day	24.3 messages/day 30-day mean, 31 on 2 stormy days (state changes); 5.0 s/day mean	Pass after rate limiter added
R6	500 golden captures on target via HIL	Features ≤ 0.1 dB; agreement ≥ 99 %	Max feature error 0.04 dB; 99.6 % agreement	Pass
R7	Linker map, SRAM watermark	Flash ≤ 75 %, SRAM ≤ 60 %	73 %, 40 %	Pass
R8	Fault injection: ±16 g/±4 g swap, sensor unplugged, loose stud, foreign pump type	Detected in one cycle; no class reported	4/4 detected in first cycle	Pass
R9	Model with wrong schema hash pushed over the air	Refused, previous model kept	Refused; event reported	Pass

R2's row shows how a false-alarm requirement is verified honestly. Three events in 30 node-years is an observed rate of 0.10. The one-sided 95 % upper confidence bound on a Poisson rate with three events is 7.75 events, or 0.26 per node-year, and the requirement is judged on that bound. R5 initially failed: on two stormy days, cavitation alarms toggled state and generated 31 messages. A rate limiter that sends at most one state change per hour fixed it.

Revision B

The changes from the first deployment are each traced to a mechanism: the full scale read back from the register, the gravity check and the schema hash (training–serving skew); grouped validation and shadow mode (leaky validation); flux-bounded speed search (wrong fault frequencies at low load); the "unknown" outcome and the ISO fallback (confident errors out of distribution); and the uplink rate limiter (fair-use overrun on unstable states). The next revision will reconsider the CNN on an NPU-equipped MCU once the fleet has provided labelled data from a few hundred pumps.

What remains open

Open question	Where it is answered
Training the classifier and the anomaly model: datasets, cleaning, features, evaluation, uncertainty	Chapter 32
Spectral analysis, envelope demodulation and feature extraction as DSP practice	Chapter 27
A digital twin of the pump that turns condition data into remaining-life estimates	Chapter 33
Sensor fusion with Kalman filters for attitude and localisation	Chapter 15, Chapter 16
Edge processing inside a complete multi-node wireless sensor network	Chapter 38
Motor-drive context of the monitored pumps: VFDs and their effect on vibration and flux	Chapter 20; *Electrical Energy and Machines*, Ch. 12

Next: Chapter 13 leaves devices that only sense and decide and turns to machines that move, beginning with the architecture, frames and kinematics of robots.

Part IV — Robotics and Mechatronics

A robot is where the rest of this book meets in one machine: sensors and acquisition from Part II, a microcontroller and firmware from Part III, motors and power electronics from the sibling books, and a mechanism that turns it all into force and motion. It is also where integration mistakes become physical: a shoulder sized at the wrong pose drops a tube, a wheel radius 1.5 % off sends a robot into a wall 60 m later, a bus with nowhere to put regenerated energy lets a robot roll down a ramp. Part IV calculates such failures before the build. Its systems are a six-axis laboratory arm, a hospital delivery robot and its motor-control board, a bridge-inspection quadcopter, and a collaborative assembly arm taken through integration and field test, each carried from requirements through budgets, part selection and implementation to verification, failure analysis and the second version the failures forced.

By the end of this part you should understand

- How a task sets a robot's architecture and joint count, and how frames, kinematics, the Jacobian and singularities serve as integration tools.
- How joint actuators are sized at the worst pose with acceleration and duty, and how error budgets decide gears, encoders and compensation.
- How a wheeled robot estimates its pose, how odometry error grows and is calibrated, and how planning, localization, stopping fields and navigation are sized.
- How drivers, encoder interfaces, sensor fusion and cascaded loops are implemented on an embedded controller, and why a loop belongs where its worst-case delay fits its phase margin.
- How power distribution handles regeneration and brown-out, and which faults may retry and which must latch.
- How a multirotor is sized and tested, and how a robot is integrated, calibrated, identified and tested in the loop and in the field.

Chapters

Chapter	Central question	What it delivers
13 — Robotics Fundamentals	How do a task and a bench become an arm architecture, a kinematic model and joint actuators with known margins?	A 6R lab arm: architectures, degrees of freedom, frames, DH model, inverse kinematics, joint and Cartesian moves, dexterous workspace, worst-pose actuator sizing, error budgets, mechanical–electrical integration, verification and FMEA
14 — Mobile Robotics	How does a wheeled robot know where it is, how wrong does that become, and how are its drive, battery and obstacle sensing sized?	A hospital delivery robot: differential drive, encoders, line following, stopping fields, odometry calibration, A* planning, localization, navigation, motor and battery sizing, testing and FMEA
15 — Robot Electronics and Control	Which drivers, interfaces, loops and timing turn velocity commands into safe wheel motion, and what happens when something fails?	The robot's motor-control board: driver ratings, M/T encoder speed, gyro fusion, MCU and CPU budget, PI implementation, cascaded loops and delay budget, buses, power tree with bus clamp, fault manager, verification and FMEA
16 — Drones and Aerial Robotics	How are a multirotor's motors, propellers, ESCs, battery and flight controller sized together, and made to fly on a vibrating airframe?	A 2.3 kg inspection quadcopter: architecture and mass budget, motors, ESCs, propellers, flight controller and IMU, attitude estimation, flight loops, battery sizing, safety and staged testing
17 — Robotic System Integration	Which interfaces between a robot's modules must be specified, and how is the integrated robot calibrated, identified and tested?	A collaborative assembly arm: mechanical, electrical, firmware, control, power and communication interfaces, calibration, system identification, hardware-in-the-loop and field testing

How this part connects to the rest of the book

Part IV applies the method of Part I (requirements, architecture, modelling, test plans) to the sensors of Part II and the embedded practice of Part III; Chapter 15 reuses Chapter 10's MCU selection, timing and fault handling. Inside the part, Chapter 13 owns kinematics and actuator selection, Chapter 14 differential drive, odometry and line following, and Chapter 15 motor drivers, encoders, PID implementation and robot buses; Chapter 17 integrates them. Forward, the line-following robot of Chapter 34 and the motor drive of Chapter 37 apply them as complete projects; safety functions belong to Chapter 21, battery packs to Chapter 23 and model calibration to Chapter 33.

The theory is cited, never re-derived. *Electrical Energy and Machines* supplies motors, servo sizing and drive control (Ch. 8, Ch. 10, Ch. 12). *Electronics and Computing* supplies interfaces, real-time computing and drive power stages (Ch. 26, Ch. 27, Ch. 32). *Circuits, Signals and Communications* supplies feedback and PID design (Ch. 35), *Engineering Physics* gears and vibration (Ch. 19), and *The Mathematics Universe* the linear algebra, numerical methods and probability behind kinematics and estimation (Ch. 9, Ch. 22, Ch. 19).

Chapter 13 — Robotics Fundamentals

Part IV · Robotics and Mechatronics

A clinical laboratory wants a small arm that lifts sample tubes out of a rack, presents them to a decapper and loads them into the 45° slots of a benchtop centrifuge, on a 600 × 900 mm bench next to a technician. The first prototype, built in six weeks around a hobby-arm geometry with NEMA 17 steppers and planetary gearboxes, passed every bench test at its home pose and failed on its first real cycle: reaching for the far corner of the rack with a full tube, the shoulder lost steps and dropped the gripper 14 mm; with more current, tubes still landed 0.6 mm low at full reach and jammed on the rotor lip. Neither failure was bad luck. The shoulder had been sized from its static torque at the home pose, not at full extension with payload and acceleration, and the gripper position came from a rigid model with no budget for gearbox compliance. This chapter follows the second design of that arm, its reference system, from task to architecture, kinematics, motion, workspace, actuators and the mechanical–electrical details that decide whether it works.

> **How do a task and a bench turn into a robot architecture, a number of joints, a kinematic model and a set of joint actuators with known margins?**
> **Where do the position errors and the torque peaks of a real arm come from, and how are they budgeted, verified and designed out before the robot meets a person?**

Concept	Engineering destination
Robot architectures	Choosing articulated, SCARA, delta, Cartesian or collaborative arms for a task and a budget
Degrees of freedom	Matching the joint count to the constraints a task actually imposes
Coordinate frames	Base, tool, work-object and camera frames; hand–eye and fixture calibration
Forward kinematics	Tool pose from joint readings; simulation, collision checks, error propagation
Inverse kinematics	Joint targets for a desired tool pose; branch selection; singularity handling
Joint and Cartesian motion	Time-optimal moves, straight-line insertions, speed limits near people
Workspace	Placing racks, instruments and the robot on a bench; reach and dexterity checks
Actuator selection	Motor, gearbox, encoder and brake per joint from torque, speed, stiffness and duty
Mechanical–electrical integration	Cabling through joints, encoders, brakes, bus power, resonance, safety functions

Roadmap. Requirements and architecture (13.1) → degrees of freedom the task needs (13.2) → frames and transforms (13.3) → forward kinematics (13.4) → inverse kinematics (13.5) → joint and Cartesian motion (13.6) → workspace and placement (13.7) → actuator selection and the error budget (13.8) → mechanical–electrical integration (13.9) → verification and failure analysis (13.10).

13.1 Robot architectures

The task and its requirements

The arm's job is a pick-and-place cycle: take a capped 13 × 75 mm tube from a 96-position rack, present it to a decapper, place it in a 45° fixed-angle rotor slot, and repeat. Two people share the bench, so the arm must be able to work slowly and softly when a person is inside its reach, and it must hold its load if power fails. Table 13.1 states the requirements the second design was built against (the method of writing them is Chapter 1's).

Table 13.1 — Requirements of the desktop lab arm.

ID	Requirement	Target	Verification
R1	Payload	0.5 kg at the tool centre point (TCP), any pose in the work zone	Static hold and full-speed cycle with 0.5 kg test mass
R2	Reach	Tool pointing down, TCP 60 mm above the bench, up to 390 mm from the base axis	Pose sweep out to 390 mm at 60 mm height
R3	Repeatability	±0.10 mm at the TCP (ISO 9283 pose repeatability)	30-cycle dial-indicator test at three poses
R4	Absolute accuracy	±0.5 mm after calibration, anywhere in the work zone	Touch-off on a calibrated grid plate
R5	Cycle time	Rack → decapper → centrifuge → rack in ≤ 6 s	Timed run of 100 cycles
R6	Collaborative speed	TCP ≤ 250 mm/s while a person is inside the reach (ISO 10218-1 reduced speed)	Speed log during a light-curtain trigger
R7	Power-loss hold	TCP drop < 1 mm when 24 V is removed at full reach with payload	Pull-the-plug test, dial indicator
R8	Thermal	Joint motor winding rise < 40 K over 2 h of the R5 cycle	Thermocouples on the shoulder and elbow motors
R9	Stop	Stop category 1 within 300 ms from E-stop; safe torque off afterwards	High-speed log of joint speed and STO line
R10	Cost	Bill of materials ≤ $4,500 at 10 units	Costed BOM

Architectures and where each fits

Architecture comes before any joint is sized, because it fixes how reach, payload, stiffness and cost trade. Table 13.2 compares the candidates.

Table 13.2 — Robot architectures for small-part handling (typical catalogue values, 2026).

Architecture	Payload	Reach	Repeatability	Fit and limits	Price
Articulated 6R industrial (IRB 1200 class)	5–7 kg	0.7–0.9 m	±0.02 mm	Any pose; needs guarding	$25k–40k
Collaborative 6R (UR5e class)	3–5 kg	0.5–0.85 m	±0.03 mm	Works beside people (ISO/TS 15066); slower	$20k–35k
SCARA (T6 class)	3–6 kg	0.4–0.6 m	±0.02 mm	0.4 s cycles; tool always vertical (4 DOF)	$8k–15k
Delta (parallel)	1–3 kg	0.8–1.3 m dia.	±0.05 mm	150+ picks/min; small stroke, 4 DOF	$20k+
Cartesian gantry	any	any	±0.01–0.05 mm	Stiff, simple; covers the bench	$3k–20k
Desktop stepper arm	0.25–0.5 kg	0.3–0.45 m	±0.2 mm	Cheap; lost steps, compliance, no safety rating	$1.5k–4k

The bench rules out a gantry (it would cover the bench the technicians need), and the 45° rotor slots rule out SCARA and delta arms, whose tools stay vertical. A commercial collaborative arm would work but costs five to eight times R10 and is oversized at 0.5 kg. The design is therefore a **custom 6R articulated arm**: a base yaw joint, shoulder and elbow pitch joints, and a three-axis wrist whose axes intersect at one point (a *spherical wrist*), which makes the inverse kinematics solvable in closed form (Section 13.5).

Architecture of the arm and its cell

Figure 13.1 shows the architecture. Each joint is a self-contained module — motor, gear, two encoders, brake and driver — on a 24 V bus and a CAN FD network, so the six modules differ only in size and parameters. A motion controller (a Linux single-board computer running the trajectory generator and kinematics at 1 kHz) sends position, velocity and feed-forward torque set points to the joints; each joint closes its own current, velocity and position loops (Chapter 15). A safety relay drives the safe-torque-off (STO) input of every joint driver from the E-stop and the light curtain.

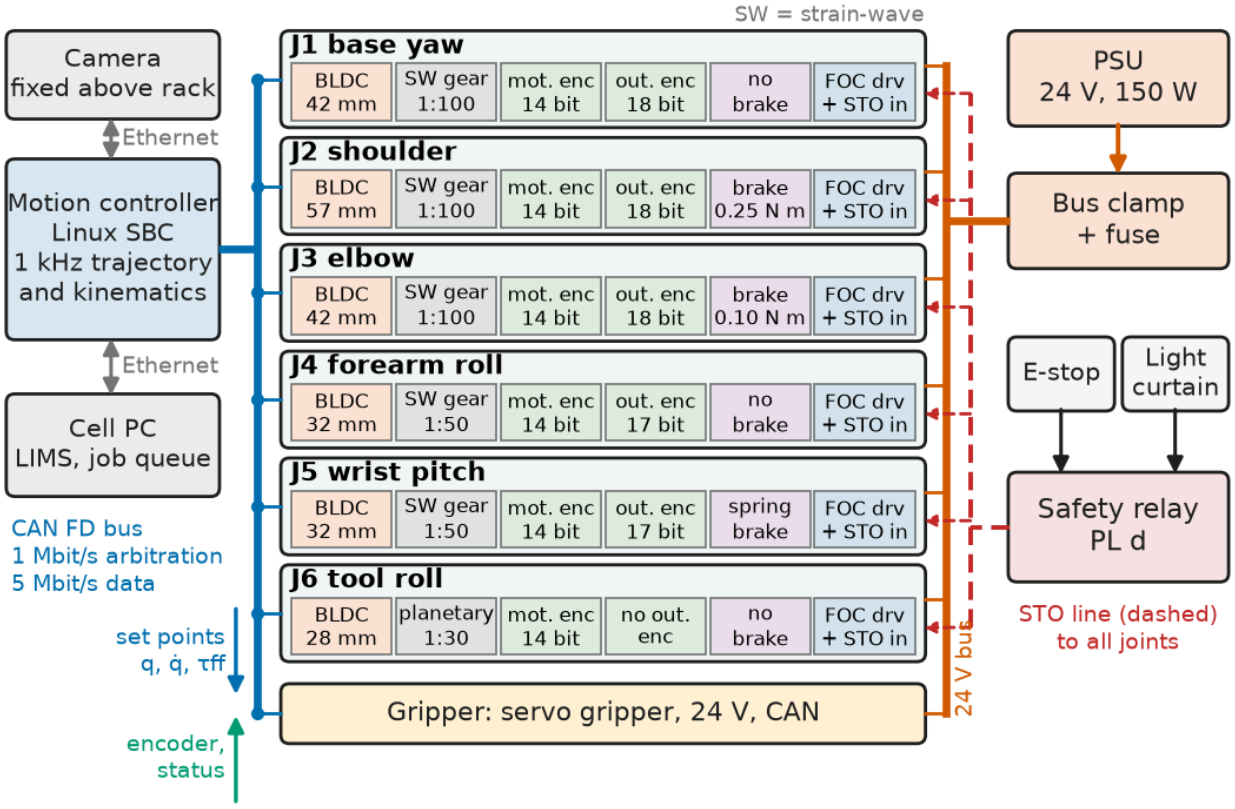


Figure 13.1 — Architecture of the desktop lab arm: motion controller, six joint modules on CAN FD and a 24 V bus, gripper, camera and the safety chain.

13.2 Degrees of freedom

What the task constrains

A rigid object has six degrees of freedom. The joints a robot needs are the freedoms the task constrains, plus any redundancy deliberately added; each extra joint costs a module, mass at the end of the arm and a few hundred dollars.

Table 13.3 — Constrained degrees of freedom per operation.

Operation	Position (x, y, z)	Tool tilt (two angles)	Rotation about tool axis	Total
Pick from vertical rack	3	2 (must be vertical)	free (tube is round)	5
Present to decapper (grips cap from above)	3	2	1 (cap thread start, ±20°)	6
Insert in 45° rotor slot, slot at any rotor angle	3	2 (45° tilt toward rotor axis)	free	5
Return to rack	3	2	free	5

The rotor slots need two tilt angles in a direction that depends on where the rotor stopped, and the decapper needs a controlled roll; together they need all six. A five-axis arm would serve the rotor but not the decapper.

Mobility of the mechanism

For a serial chain the mobility is the joint count; for closed chains (a delta robot, a parallelogram forearm) the Grübler–Kutzbach count gives it:

$$M = 6(n_l - 1 - n_j) + \sum_{i=1}^{n_j} f_i$$

with n_l links including the ground, n_j joints and f_i the freedoms of joint i.

> **Status of the principle: Grübler–Kutzbach mobility count**
> *Derived.* It follows from counting the six freedoms of each moving link and the constraints each joint removes; it can undercount when the geometry is special (parallel axes in a planar loop, overconstrained linkages that only move because of exact dimensions), so a real linkage is always checked in CAD as well.

For the arm, seven links and six revolute joints give $M = 6(7 - 1 - 6) + 6 = 6$; a redundant seventh joint would dodge some singularities, but the bench is uncluttered and the cost is not justified.

13.3 Coordinate frames

The frames of the cell

Every position the arm uses is expressed in some frame, and most integration bugs in robot cells are frame bugs: the right number in the wrong frame. The cell defines five frames, all right-handed with z up where it applies (the ROS REP 103 convention):

- **Base** $\{B\}$: on the J1 axis at the bench surface, x toward the rack.
- **Flange** $\{F\}$ and **tool** $\{T\}$: the J6 flange, and the TCP between the gripper fingers, 60 mm out along the flange z axis.
- **Work object** $\{W\}$: one per fixture, taught by touching three points, so moving a fixture means re-teaching one frame, not every point.
- **Camera** $\{C\}$: the fixed camera above the rack that reads barcodes and checks for empty slots.

A pose of frame $\{b\}$ in frame $\{a\}$ is a 4×4 homogeneous transform, a rotation matrix and a translation, and poses chain by matrix multiplication:

$${}^{B}\mathbf{T}_{T} = {}^{B}\mathbf{T}_{F}\,{}^{F}\mathbf{T}_{T}, \qquad {}^{a}\mathbf{T}_{b} = \begin{bmatrix} \mathbf{R} & \mathbf{p} \\ \mathbf{0}^{\mathsf{T}} & 1 \end{bmatrix}$$

> **Boundary.**
> Rotation matrices are orthogonal matrices with determinant +1, and a homogeneous transform is a linear map composed with a translation; their algebra (composition, inverse as transpose, change of basis) is in *The Mathematics Universe*, Ch. 9 §9.5 and §9.7. This section uses it.

Orientation representations in practice

The controller stores orientations as unit quaternions (no singularity, cheap interpolation), shows them to technicians as roll–pitch–yaw angles, and never interpolates Euler angles, which jump near ±90° pitch. Every pose variable carries its frames in its name (`T_base_tool`, `p_cam_slot`), and every new frame is checked by jogging the robot 50 mm along each of its axes.

Worked example — from camera to base, and what a half-degree costs. Hand–eye calibration put the camera at $\mathbf{p} = (0.300,\ 0.000,\ 0.750)$ m in the base frame, looking straight down, so its rotation relative to the base is $\mathbf{R} = \mathrm{diag}(1, -1, -1)$ (camera x along base x, camera y and z reversed). The vision system finds a slot centre at ${}^{C}\mathbf{p} = (0.000,\ 0.100,\ 0.690)$ m. Then

$$ {}^{B}\mathbf{p} = \mathbf{p} + \mathbf{R}\,{}^{C}\mathbf{p} = (0.300 + 0,\ 0 - 0.100,\ 0.750 - 0.690) = (0.300,\ -0.100,\ 0.060)\ \text{m}, $$

the rack slot used throughout this chapter. If the calibration has a 0.5° tilt error about the camera x axis, the slot at 0.690 m range moves sideways by $0.690 \sin 0.5^\circ = 6.0$ mm — twelve times R4. Orientation errors are multiplied by distance, so the camera is calibrated against a target at the working height and mounted as stiffly as the arm.

13.4 Forward kinematics

The Denavit–Hartenberg model of the arm

Forward kinematics gives the tool pose from the joint angles. For a serial arm it is a product of one transform per joint; the Denavit–Hartenberg (DH) convention describes each with four numbers: link length a_i, link twist α_i, link offset d_i and joint angle θ_i,

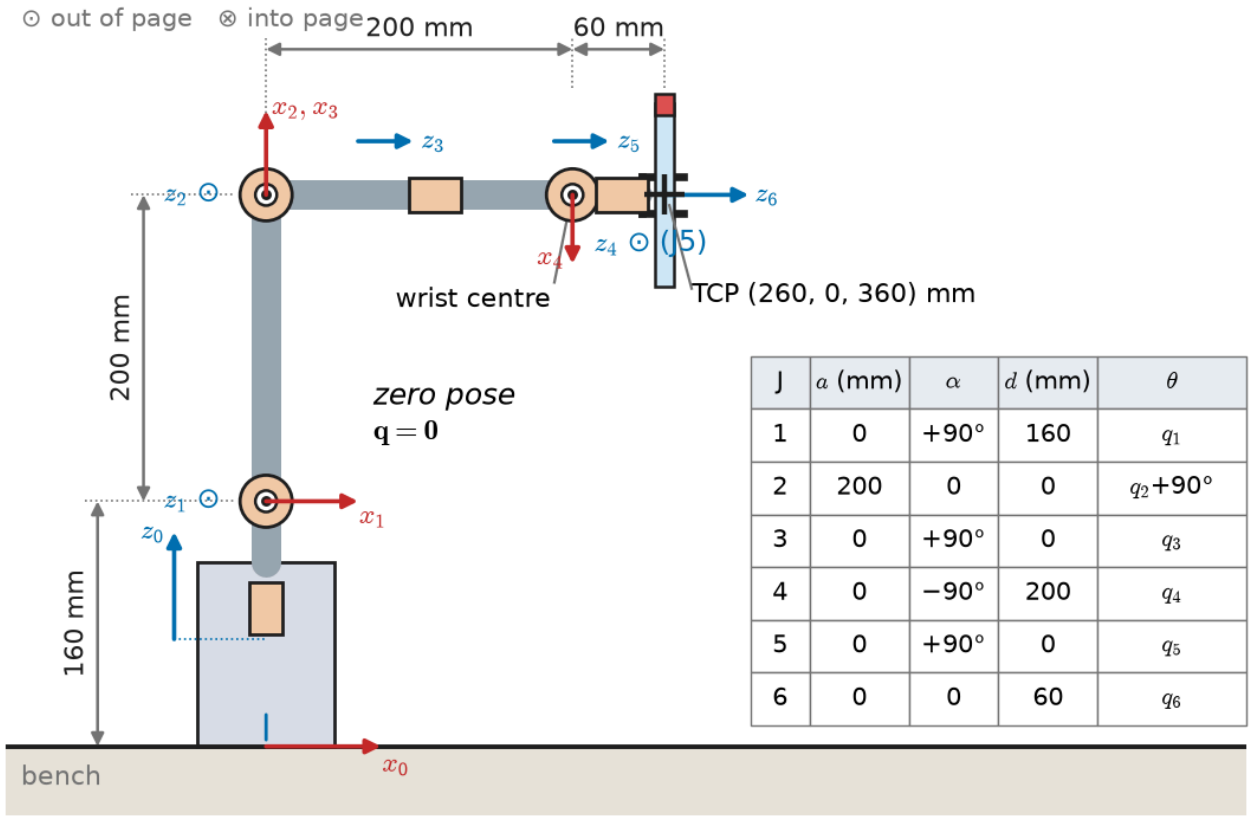
$$ {}^{i-1}\mathbf{T}_i = \mathrm{Rot}_z(\theta_i)\,\mathrm{Trans}_z(d_i)\,\mathrm{Trans}_x(a_i)\,\mathrm{Rot}_x(\alpha_i), \qquad {}^{0}\mathbf{T}_6 = \prod_{i=1}^{6} {}^{i-1}\mathbf{T}_i . $$

Table 13.4 is the DH table of the arm (standard convention). The shoulder is 160 mm above the bench, the upper arm and the forearm are each 200 mm, and the TCP is 60 mm beyond the wrist centre. The 90° offset on J2 makes the *zero pose* an inverted L: upper arm vertical, forearm horizontal, tool pointing forward, which puts every joint mid-range at power-up.

Table 13.4 — DH parameters of the desktop arm.

Joint	a_i (mm)	α_i	d_i (mm)	θ_i	Range
J1 base yaw	0	+90°	160	q_1	±170°
J2 shoulder	200	0	0	$q_2 + 90^\circ$	−110° to +30°
J3 elbow	0	+90°	0	q_3	−50° to +140°
J4 forearm roll	0	−90°	200	q_4	±180°
J5 wrist pitch	0	+90°	0	q_5	±120°
J6 tool roll	0	0	60	q_6	±270°

Figure 13.2 draws the frames. J4, J5 and J6 axes intersect at the wrist centre, 200 mm along the forearm from the elbow.



J	a (mm)	α	d (mm)	θ
1	0	+90°	160	q_1
2	200	0	0	q_2+90°
3	0	+90°	0	q_3
4	0	−90°	200	q_4
5	0	+90°	0	q_5
6	0	0	60	q_6

Figure 13.2 — Kinematic diagram of the 6R desktop arm with its DH frames in the zero pose.

Listing 13.1 implements the model. It is the module every other tool in the cell imports: the simulator, the collision checker, the calibration fit and the inverse kinematics.

```
 import numpy as np
# Standard DH rows of the desktop arm: a [m], alpha [rad], d [m], joint offset [rad]
DH = [(0.000,  np.pi/2, 0.160, 0.0), (0.200, 0.0, 0.000, np.pi/2),
      (0.000,  np.pi/2, 0.000, 0.0), (0.000, -np.pi/2, 0.200, 0.0),
      (0.000,  np.pi/2, 0.000, 0.0), (0.000, 0.0, 0.060, 0.0)]

def dh(a, al, d, th):
    ca, sa, ct, st = np.cos(al), np.sin(al), np.cos(th), np.sin(th)
    return np.array([[ct, -st*ca, st*sa, a*ct], [st, ct*ca, -ct*sa, a*st],
                     [0.0, sa, ca, d], [0.0, 0.0, 0.0, 1.0]])

def fk(q):
    T = np.eye(4)
    for (a, al, d, off), qi in zip(DH, q):
        T = T @ dh(a, al, d, qi + off)
    return T

def pose_error(Tg, T):                    # 6-vector: position, small rotation
    dR = Tg[:3, :3] @ T[:3, :3].T
    w = 0.5*np.array([dR[2, 1]-dR[1, 2], dR[0, 2]-dR[2, 0], dR[1, 0]-dR[0, 1]])
    return np.concatenate([Tg[:3, 3] - T[:3, 3], w])

def jacobian(q, h=1e-6):                  # numerical geometric Jacobian, 6x6
    T0 = fk(q)
    return np.column_stack([pose_error(fk(q + h*np.eye(6)[i]), T0)/h for i in range(6)])

def ik(Tg, q, lam=0.01, iters=200):       # damped least squares
    for k in range(iters):
        e = pose_error(Tg, fk(q))
        if np.linalg.norm(e) < 1e-9:
            break
        J = jacobian(q)
        q = q + J.T @ np.linalg.solve(J @ J.T + lam**2*np.eye(6), e)
    return q, k, np.linalg.norm(e[:3])

if __name__ == "__main__":
    Tg = np.eye(4); Tg[:3, :3] = np.diag([1.0, -1.0, -1.0])   # tool pointing down
    Tg[:3, 3] = [0.300, -0.100, 0.060]                         # tube rack slot
    q, n, err = ik(Tg, np.radians([0, -45, -45, 0, -90, 0]))
    print("rack pose q [deg]:", np.round(np.degrees(q), 2))
    print(f"iterations {n}, residual {err*1e6:.4f} um, cond(J) {np.linalg.cond(jacobian(q)):.1f}")
    q[4] = np.radians(0.5)                                      # wrist almost straight
    print(f"cond(J) with q5 = 0.5 deg: {np.linalg.cond(jacobian(q)):.0f}")
```

Saved as `arm_kin.py`, it prints:

```
 rack pose q [deg]: [-18.43 -60.04  15.66   0.   -45.62 -18.43]
iterations 7, residual 0.0004 um, cond(J) 18.0
cond(J) with q5 = 0.5 deg: 730
```

The zero pose evaluates to a TCP at (260, 0, 360) mm, the first check of any kinematic model against a tape measure on the real arm.

The Jacobian and why it is the integration tool

The Jacobian $\mathbf{J}(\mathbf{q})$ maps joint velocities to the tool's linear and angular velocity, $\dot{\mathbf{x}} = \mathbf{J}\dot{\mathbf{q}}$, and does three integration jobs. It **propagates errors** ($\delta\mathbf{x} = \mathbf{J}\,\delta\mathbf{q}$, the basis of the budget in Section 13.8); it **maps forces** ($\boldsymbol{\tau} = \mathbf{J}^{\mathsf{T}}\mathbf{f}$, sizing joints for an insertion load); and it **flags singularities**, where it loses rank. The condition number printed by Listing 13.1 is the alarm: 18 at the rack pose, 730 with the wrist almost straight ($q_5 = 0.5^\circ$).

> **Status of the principle: Velocity and force mapping through the Jacobian**
> *Derived.* Differentiating the forward kinematics gives $\dot{\mathbf{x}} = \mathbf{J}\dot{\mathbf{q}}$, and virtual work across a frictionless, rigid chain gives $\boldsymbol{\tau} = \mathbf{J}^{\mathsf{T}}\mathbf{f}$; both are linearisations valid for small displacements about the current pose and ignore link and gear compliance.

13.5 Inverse kinematics

Closed form for a spherical wrist

The controller needs the opposite map: joint angles that put the tool at a commanded pose. Because the wrist axes intersect, position and orientation separate (Pieper's decoupling). The wrist centre is 60 mm behind the TCP along the tool axis; J1, J2 and J3 place the wrist centre, then J4, J5 and J6 orient the tool.

Worked example — the rack slot in closed form. The TCP target is $(0.300,\ -0.100,\ 0.060)$ m with the tool pointing straight down, so the wrist centre is 60 mm higher: $\mathbf{p}_w = (0.300,\ -0.100,\ 0.120)$ m.

- Base yaw: $q_1 = \operatorname{atan2}(-0.100,\ 0.300) = -18.43^\circ$.
- In the arm's vertical plane the wrist centre is at horizontal distance $r = \sqrt{0.300^2 + 0.100^2} = 0.3162$ m and height $s = 0.120 - 0.160 = -0.0400$ m from the shoulder, at distance $D = \sqrt{r^2 + s^2} = 0.3187$ m.
- The elbow bend ψ (forearm angle relative to the upper arm's extension) follows from the cosine rule with two 0.200 m links: $\cos\psi = (D^2 - 0.200^2 - 0.200^2)/(2 \cdot 0.200 \cdot 0.200) = 0.2700$, so $\psi = 74.34^\circ$, elbow up.
- Upper-arm elevation: $\beta = \operatorname{atan2}(s, r) + \operatorname{atan2}(0.200\sin\psi,\ 0.200 + 0.200\cos\psi) = -7.21^\circ + 37.17^\circ = 29.96^\circ$.
- With the DH offsets of Table 13.4, $q_2 = \beta - 90^\circ = -60.04^\circ$ and $q_3 = 90^\circ - \psi = 15.66^\circ$.

The wrist then solves ${}^{3}\mathbf{R}_{6} = ({}^{0}\mathbf{R}_{3})^{\mathsf{T}}\,\mathbf{R}_{\text{goal}}$ as a Z–Y–Z Euler triple, giving $q_4 = 0$, $q_5 = -45.62^\circ$, $q_6 = -18.43^\circ$. Listing 13.1's numerical solver, started from a generic seed, converged in 7 iterations to the same six angles. That agreement is the regression test that guards both implementations.

Branches, joint limits and the numerical fallback

A 6R arm with a spherical wrist has up to eight closed-form solutions for one pose: shoulder left or right, elbow up or down, wrist flipped or not. The controller picks the branch that respects joint limits and is closest to the current joint vector, and it **never changes branch in the middle of a Cartesian move**, because a branch change is a large, fast joint motion the TCP path does not show (Section 13.6 shows one).

The damped-least-squares iteration of Listing 13.1, $\Delta\mathbf{q} = \mathbf{J}^{\mathsf{T}}(\mathbf{J}\mathbf{J}^{\mathsf{T}} + \lambda^2\mathbf{I})^{-1}\mathbf{e}$, is kept as the general solver: for tool offsets that change with each gripper, for calibrated kinematics in which the wrist axes no longer intersect exactly, and near singularities, where the damping λ trades a small tracking error for bounded joint speeds. It is Newton-type iteration on the pose error; its convergence behaviour is in *The Mathematics Universe*, Ch. 22 §22.4.

> **Status of the principle: Damped least-squares inverse kinematics**
> *Approximate.* It converges in a few iterations (close to quadratically for small λ) near a regular solution when started close to it, and it remains bounded near singularities at the price of a tracking error that grows with λ; it finds one solution near the seed, not all branches, so the seed decides the branch.

Singularities of this arm

Three singular configurations matter, each with a physical meaning and a design response:

Singularity	Condition	What happens	Response in the controller
Wrist	$q_5 = 0$ (J4 and J6 aligned)	Tool roll can be made by J4 or J6 in any mix; near it, small orientation changes need large J4/J6 speeds	Keep $\lvert q_5\rvert > 10^\circ$ in taught poses; DLS with speed clamp in Cartesian moves
Elbow	Arm fully stretched, $\psi = 0$	No motion toward or away from the shoulder	Elbow kept at least 8° from straight: wrist distance ≤ 0.399 m of the 0.400 m physical maximum
Shoulder	Wrist centre on the J1 axis	J1 undefined	Fixtures placed at least 120 mm from the J1 axis

13.6 Joint and Cartesian motion

Two kinds of move

A **joint move** interpolates every joint with the same time scaling: fast, free of singularities, with a TCP path that is not obvious from the endpoints. A **Cartesian move** interpolates the TCP along a straight line (orientation by quaternion interpolation) and solves inverse kinematics at every sample; it is needed wherever the path matters, such as a 45° slot insertion, and can demand very high joint speeds near singularities.

Both use the quintic time scaling $s(\tau) = 10\tau^3 - 15\tau^4 + 6\tau^5$, $\tau = t/T$, which starts and ends at zero velocity and acceleration (no torque step to excite the 23 Hz gear-compliance mode of Section 13.9); its peak rate is $1.875/T$.

Listing 13.2 compares the two for the transfer from the rack to the centrifuge, both at 150 mm above the bench, 381 mm apart, in $T = 1.0$ s.

```
 import numpy as np
from arm_kin import fk, ik

def tool_down(p):
    T = np.eye(4); T[:3, :3] = np.diag([1.0, -1.0, -1.0]); T[:3, 3] = p
    return T

A, B, T_move = np.array([0.300, -0.100, 0.150]), np.array([0.150, 0.250, 0.150]), 1.0
seed = np.radians([0, -45, -45, 0, -90, 0])
qA, _, _ = ik(tool_down(A), seed)
qB, _, _ = ik(tool_down(B), qA)
qB = qA + (qB - qA + np.pi) % (2*np.pi) - np.pi                # nearest equivalent angles
t = np.linspace(0.0, T_move, 201)
s = 10*(t/T_move)**3 - 15*(t/T_move)**4 + 6*(t/T_move)**5       # quintic time scaling

# 1) joint-space move: every joint interpolated with the same s(t)
pj = np.array([fk(qA + si*(qB - qA))[:3, 3] for si in s])
u = (B - A)/np.linalg.norm(B - A)
dev = np.linalg.norm((pj - A) - np.outer((pj - A) @ u, u), axis=1)
print(f"joint move: max deviation from straight line {dev.max()*1e3:.1f} mm, "
      f"lowest z {pj[:, 2].min()*1e3:.1f} mm")

# 2) Cartesian move: IK at every sample, warm-started from the previous one
q, Q = qA.copy(), []
for si in s:
    q, _, _ = ik(tool_down(A + si*(B - A)), q)
    Q.append(q)
qd = np.gradient(np.array(Q), t, axis=0)
print("Cartesian move: peak joint speeds [deg/s]:", np.round(np.degrees(np.abs(qd).max(axis=0)),
0))
print("joint move:     peak joint speeds [deg/s]:", np.round(np.degrees(np.abs(qB -
qA)*1.875/T_move), 0))
```

```
 joint move: max deviation from straight line 67.5 mm, lowest z 150.0 mm
Cartesian move: peak joint speeds [deg/s]: [172.  63. 107.   0.  44. 172.]
joint move:     peak joint speeds [deg/s]: [145.  12.  21.   0.   9. 145.]
```

Worked example — choosing the move type and its duration. The joint move swings the TCP up to 67.5 mm off the straight line, away from the base (it sweeps an arc around J1), and keeps its height. That is acceptable in free space above the fixtures, but not near the decapper, which stands 50 mm from the line. Both moves ask more than the 135 °/s joint limit of J1 and J6 (the sizing speed of Listing 13.3) at $T = 1.0$ s: the joint move needs 145 °/s and the Cartesian move 172 °/s. Because every speed scales as $1/T$, the minimum durations are $1.0 \times 145/135 = 1.07$ s for the joint move and $1.0 \times 172/135 = 1.27$ s for the Cartesian move. The cycle uses a joint move of 1.1 s for the transfer and Cartesian moves only for the last 40 mm of each approach. The peak TCP speed of the Cartesian transfer would be $1.875 \times 0.381/1.27 = 0.56$ m/s; in collaborative mode (R6) the same path is scaled to 250 mm/s, which stretches it to $1.875 \times 0.381/0.25 = 2.86$ s.

Two details of the listing are design rules. The line that wraps `qB` to the nearest equivalent angle exists because the first version lacked it: the solver returned −301° for J1, which demanded 530 °/s and a swing through the technician's side of the bench. And J6 counter-rotates as much as J1 (both 172 °/s) to keep the gripper's orientation fixed while the base turns; a symmetric grip would let J6 stay still.

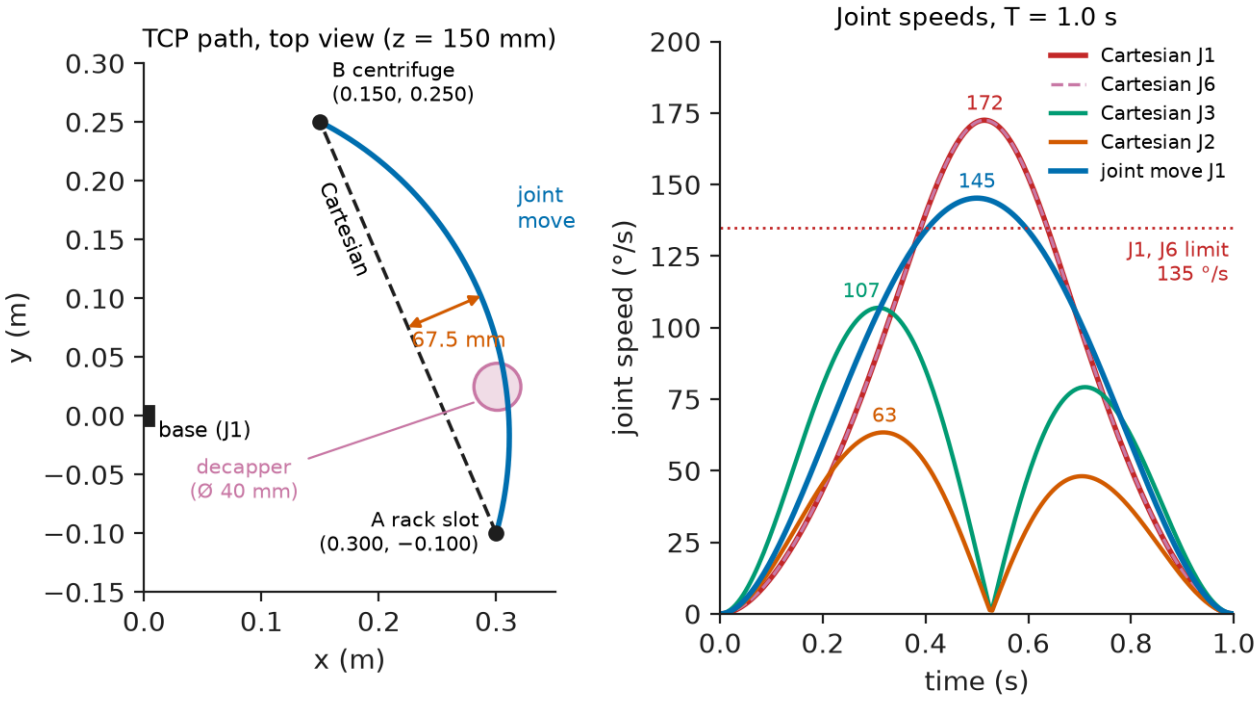


Figure 13.3 — Joint-space versus Cartesian transfer from rack to centrifuge: TCP paths seen from above, and joint speeds against time.

> **Status of the principle: Speed scaling of a time-parameterised path**
> *Derived.* For a fixed geometric path $\mathbf{q}(s)$ traversed with $s(t/T)$, every joint velocity scales as $1/T$ and every acceleration as $1/T^2$, so the shortest admissible duration is set by the most limited joint; torque limits that depend on speed and gravity make the real minimum time a little longer.

Limits a real controller adds

The trajectory generator also enforces per-joint velocity, acceleration and jerk limits, TCP speed limits per zone (R6), 10 mm corner blending between successive moves, and a rule that a Cartesian move whose DLS residual exceeds 0.05 mm or whose condition number exceeds 300 is refused before it starts, not abandoned halfway.

13.7 Workspace

Reachable and dexterous workspace

The **reachable workspace** is every point the TCP can reach in some orientation; the **dexterous workspace** is where it can reach with the orientation the task needs. For this arm the wrist centre must lie

in a spherical shell around the shoulder between the folded and the stretched elbow, and with the tool pointing down the TCP is 60 mm below the wrist centre.

Worked example — how far can the arm reach on the bench? With the elbow kept at least 8° from straight, the wrist centre can be up to $0.400 \cos 4^\circ = 0.399$ m from the shoulder; with the elbow folded to its 140° limit, no closer than $\sqrt{0.08 + 0.08 \cos 140^\circ} = 0.137$ m. A tool-down TCP 60 mm above the bench has its wrist centre at 0.120 m, 0.040 m below the shoulder, so the horizontal reach is $\sqrt{0.399^2 - 0.040^2} = 0.397$ m and the inner limit is $\sqrt{0.137^2 - 0.040^2} = 0.131$ m. R2 asks for 0.390 m: the margin is 7 mm, which is thin but acceptable because no fixture needs it: the farthest pick point is at 0.36 m (Section 13.7), and R2 exists so that a rearranged bench still has room. A tool-down target at 0.420 m — inside the 0.46 m reachable envelope — has no solution, and the numerical solver of Listing 13.1 confirms it: after 200 iterations it stops with the elbow stretched and 53 mm of residual error. The gap between catalogue reach (0.46 m) and dexterous reach at the working height (0.397 m) is the most common surprise in robot-cell layout.

> **Status of the principle: Dexterous reach from the wrist-centre shell**
> *Derived.* For a spherical-wrist arm the wrist centre is fixed by the TCP pose and the tool length, so the dexterous workspace for a given orientation is the wrist-centre shell shifted by the tool vector; joint limits and self-collision then remove further regions.

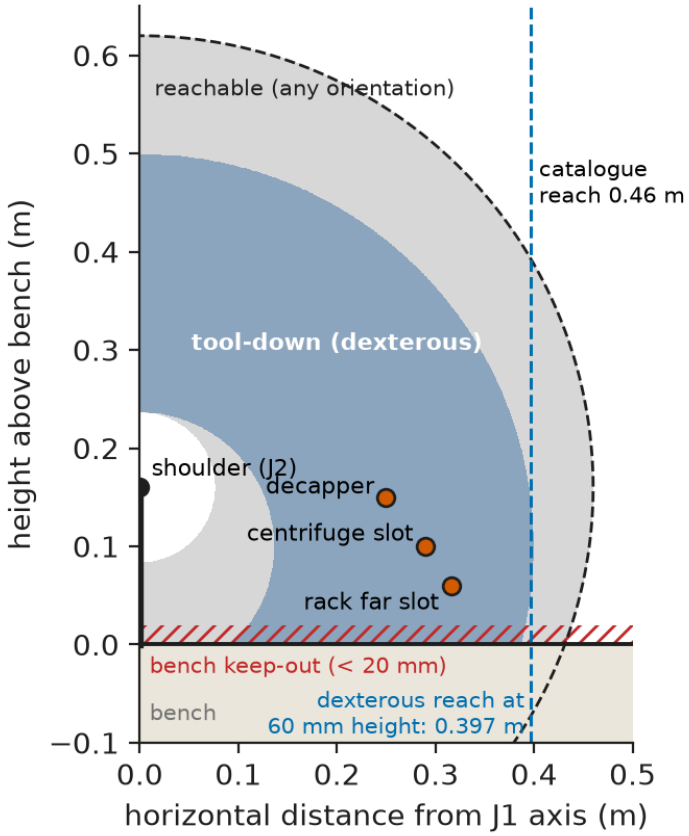


Figure 13.4 — Side section of the arm's workspace: reachable region, dexterous region with the tool pointing down, and the fixtures.

Placing the fixtures

Workspace analysis decides the bench layout. Every pick and place point lies between 0.15 and 0.36 m from the J1 axis, where the Jacobian condition number stays below 30 for tool-down poses (computed with Listing 13.1: 21.8 at 0.15 m, 25.6 at 0.36 m, 54.8 at 0.39 m, all at 60 mm height); the rack's long side faces the arm, so its far corner is at 0.32 m; and the centrifuge is turned so the loading slot faces the arm, keeping q_5 away from zero. A layout needing a wrist flip between decapper and rotor was rejected in simulation: it swept the open tube through 180°.

13.8 Actuator selection

From the motion to joint torque and speed

Joint sizing follows the duty cycle, not a steady load, using the peak-torque and RMS-torque checks and the inertia referred through the gear. Those rules are owned by *Electrical Energy and Machines*: referred inertia J_L/n^2 and the matching ratio in Ch. 10 §10.1, RMS torque against continuous torque in Ch. 10 §10.2 and the servo-sizing procedure in Ch. 8 §8.7. What a robot adds is that the load torque depends on the **pose**. The shoulder carries everything outboard of it, and its gravity torque is largest with the arm stretched horizontally, not at the zero pose where the first prototype was sized.

> **Boundary.**
> Motor types (BLDC, PMSM, stepper), their torque–speed curves and lost-step behaviour are in *Electrical Energy and Machines*, Ch. 8 §8.2, §8.4; gear trains as mechanical transformers in *Engineering Physics*, Ch. 19 §19.2.2. The drive electronics of each joint are Chapter 15's.

Listing 13.3 sizes the shoulder for the worst case: arm horizontal with the 0.5 kg payload, a 90° move in 1.0 s with a trapezoidal velocity profile (one third accelerating, one third at constant speed, one third decelerating), then a 1.0 s hold. The masses come from the CAD model of the second design.

```
 import numpy as np
g = 9.81
# (mass kg, distance from shoulder axis with arm horizontal, m)
parts = {"upper link": (0.35, 0.10), "elbow module": (0.55, 0.20),
         "forearm": (0.25, 0.30), "wrist modules": (0.60, 0.40),
         "gripper": (0.20, 0.43), "payload": (0.50, 0.46)}
T_g = sum(m*r*g for m, r in parts.values())          # worst-case gravity torque
J_L = sum(m*r*r for m, r in parts.values())          # point-mass load inertia
n, eta, J_m = 100, 0.70, 4.8e-6                     # gear ratio, efficiency, rotor
theta, t_move, t_dwell = np.pi/2, 1.0, 1.0          # 90 deg move, then hold
t_a = t_move/3
w_max = theta/(t_move - t_a); alpha = w_max/t_a
T_acc = (J_L + J_m*n**2)*alpha
segs = [(T_g + T_acc, t_a), (T_g, t_a), (T_g - T_acc, t_a), (T_g, t_dwell)]
T_pk = max(abs(T) for T, _ in segs)
T_rms = np.sqrt(sum(T*T*t for T, t in segs)/sum(t for _, t in segs))
print(f"gravity {T_g:.2f} N m, J_load {J_L:.4f} kg m2, J_rotor referred {J_m*n**2:.4f}")
print(f"w_max {w_max:.3f} rad/s ({np.degrees(w_max):.0f} deg/s), alpha {alpha:.2f} rad/s2")
print(f"T_acc {T_acc:.2f}  T_peak {T_pk:.2f}  T_rms {T_rms:.2f} N m (joint output)")
print(f"motor: peak {T_pk/(n*eta):.3f} N m, rms {T_rms/(n*eta):.3f} N m, "
      f"speed {w_max*n*60/(2*np.pi):.0f} rpm")
print(f"peak output power {T_pk*w_max:.1f} W, inertia ratio {J_L/(J_m*n**2):.1f}")
```

```
 gravity 7.61 N m, J_load 0.2868 kg m2, J_rotor referred 0.0480
w_max 2.356 rad/s (135 deg/s), alpha 7.07 rad/s2
T_acc 2.37  T_peak 9.98  T_rms 7.73 N m (joint output)
motor: peak 0.143 N m, rms 0.110 N m, speed 2250 rpm
peak output power 23.5 W, inertia ratio 6.0
```

Worked example — why the first shoulder failed and what the second one needs. At the zero pose the shoulder carries only the forearm side, 3.15 N·m; the first prototype doubled that to about 6 N·m. The real worst case is 7.61 N·m of gravity at full horizontal reach and 9.98 N·m while accelerating, 1.66 times the prototype's capacity. Because gravity dominates, the RMS torque (7.73 N·m) is close to the gravity torque: the shoulder is a *holding* problem more than an *acceleration* problem. Through a 1:100 strain-wave gear the motor needs 0.143 N·m peak and 0.110 N·m RMS at 2250 rpm. A 42 mm, 24 V BLDC rated 0.125 N·m continuous (0.38 N·m peak, 4000 rpm) has 2.7 times peak margin but only 12 % thermal margin, below the 20 % design rule. Engaging the spring brake whenever a hold lasts more than 0.5 s removes the gravity term from the dwell, and the RMS falls to $\sqrt{(9.98^2 + 7.61^2 + 5.24^2) \times 0.333/2} = 5.55$ N·m at the output, 0.079 N·m at the motor, a 37 % margin. The design uses brake management on every gravity-loaded joint and, on J2 only, also a 57 mm motor (about 0.3 N·m continuous), whose extra 0.45 kg sits on the base turntable where it costs nothing.

The 1:100 ratio also sets the inertia ratio: referred load inertia (0.287 kg·m²) is 6.0 times the referred rotor inertia (0.048 kg·m²), inside the 1–10 band that tunes well, where a 1:50 gear would give 23.9. Its penalty is speed: the gear's 3500 rpm input rating caps the output at 210 °/s, above the 135 °/s sizing speed used as the working limit in Section 13.6.

Actuator options compared

Table 13.5 — Joint actuator options for the shoulder (2026 distributor prices, 10 pcs).

Option	Output torque (cont./peak)	Lost motion	Feedback	Mass	Cost	Verdict
NEMA 23 stepper + 1:20 planetary	17 / 24 N·m, halves by 600 rpm	15–30 arcmin	Open loop	1.4 kg	$110	Lost steps unless oversized; backlash fails R3
57 mm BLDC + 1:100 strain-wave + dual encoders + brake	~20 / 34 N·m (gear-limited)	< 1 arcmin	14-bit motor, 18-bit output	1.1 kg	$620	**Chosen for J2**: stiff, precise; brake for R7
Smart servo (XM540 class)	~4 / 10 N·m	~0.25°	12-bit output	0.17 kg	$500	Too weak for J2; fits J5–J6
Quasi-direct drive (90 mm BLDC + 1:9)	6 / 18 N·m	5–10 arcmin	14-bit motor	0.9 kg	$350	Ideal for force control; too compliant for ±0.1 mm

The error budget decides the encoders

Requirement R3 (±0.10 mm repeatability) and R4 (±0.5 mm accuracy) are budgets of small angles multiplied by lever arms through the Jacobian. Table 13.6 is the repeatability budget at full reach (lever 0.46 m for J1 and J2, 0.26 m for J3), with each contributor as a ± bound combined by root-sum-square.

Table 13.6 — TCP repeatability budget at full reach.

Contributor	Basis	TCP error (µm)
Output encoders J1–J3	18-bit absolute (24.0 µrad per count), ±1 count, three joints RSS	16.8
Servo settling window	±2 counts of the 14-bit motor encoder through 1:100 (3.83 µrad), three joints	5.4
Strain-wave hysteresis left over by the output-encoder loop	10 % of 1 arcmin lost motion, J1 and J2	18.9
Bearing runout, frame micro-slip	Measured on the first prototype	20.0
Warm-up thermal drift	2 K over the hour after a 15 min warm-up, aluminium links, 0.46 m	21.2
Total (RSS)		**39.0**
Requirement R3		100
Margin		2.6 ×

Worked example — one encoder or two? If each joint closed its position loop on the motor encoder only, the gear's lost motion would appear in full: 1 arcmin is 291 µrad, 134 µm at 0.46 m, and $\sqrt{2} \times 134 = 189$ µm for J1 and J2 together, failing R3 on that line alone. A 14-bit output encoder would not do either (384 µrad, 176 µm at the TCP). An 18-bit off-axis absolute encoder on the output (AksIM-2 class, about $95) lets the position loop see the output shaft; the motor encoder keeps commutation and the velocity loop. That dual-encoder scheme on J1–J3 is the cheapest way to meet R3.

The accuracy budget (R4) adds systematic terms that calibration reduces but does not remove: residual DH errors after kinematic calibration (150 µm), gravity sag after model compensation (20 % of 583 µm, 117 µm), twist from an unknown payload of 0–0.5 kg (86 µm), 10 K of uncompensated thermal expansion (106 µm) and the repeatability (39 µm). Their RSS is 0.24 mm against ±0.5 mm; their linear sum, 0.50 mm, just meets it. Gravity sag is the line prototype A ignored: at about 6000 N·m/rad, the shoulder's 7.61 N·m twists the gear by 1.27 mrad, 0.58 mm at the TCP, so the controller now adds the computed twist to each joint target.

> **Status of the principle: Root-sum-square error budget**
> *Derived.* Independent, zero-mean contributors combine in variance, so their standard uncertainties (or matched-confidence bounds) add as a root-sum-square; correlated or systematic terms must be added linearly or removed by calibration, which is why the accuracy budget is also checked as a linear sum.

The rest of the arm

Run for each joint at its own worst pose, the same procedure gave Table 13.7.

Table 13.7 — Joint actuator set of the second design.

Joint	Output torque peak / RMS	Motor	Gear	Brake
J1 base yaw	3.1 / 1.6 N·m (inertia only)	42 mm BLDC	Strain-wave 1:100, size 17	No
J2 shoulder	9.98 / 5.55 N·m (brake in dwell)	57 mm BLDC	Strain-wave 1:100, size 17	0.25 N·m
J3 elbow	4.3 / 3.0 N·m	42 mm BLDC	Strain-wave 1:100, size 14	0.10 N·m
J4 forearm roll	0.6 / 0.3 N·m	32 mm BLDC	Strain-wave 1:50, size 11	No
J5 wrist pitch	1.6 / 0.9 N·m	32 mm BLDC	Strain-wave 1:50, size 11	Yes
J6 tool roll	0.3 / 0.1 N·m	28 mm BLDC	Planetary 1:30	No

J1–J3 carry 14-bit motor and 18-bit output encoders, J4–J5 a 17-bit output encoder, J6 the motor encoder only.

The brake on J2 must hold 1.5 times the worst gravity torque with power off (R7): $1.5 \times 7.61 = 11.4$ N·m at the output, 0.114 N·m at the motor, so a 24 V spring-applied brake rated 0.25 N·m is used, 2.2 times the need, because brake torque falls with wear and temperature.

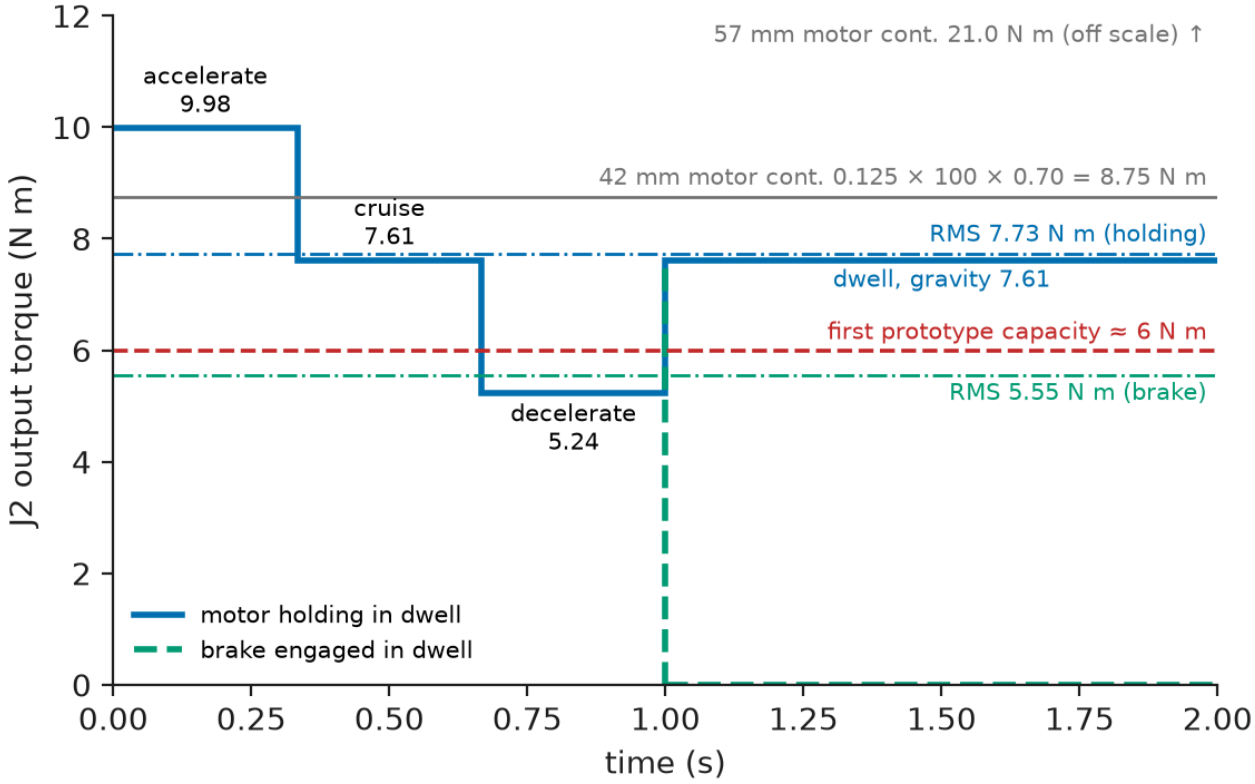


Figure 13.5 — Shoulder torque over the sizing cycle at full horizontal reach, with and without the brake engaged during the dwell.

13.9 Mechanical–electrical integration

Cables through moving joints

One six-core cable (2 × 0.5 mm² for 24 V and 0 V, a twisted pair each for CAN FD and STO, overall braided shield) runs through a 10 mm hollow-shaft bore at every joint. It is a torsion-rated robot cable specified for 5 million cycles at ±180° over 300 mm of free length; J1, J4 and J6 have service loops sized for that twist. A slip ring for continuous J6 rotation was rejected: contact noise on CAN FD at 5 Mbit/s, and the ±270° limit costs the task nothing. M8 connectors at every module boundary let a joint be replaced in ten minutes.

Encoders, magnets and noise

The 14-bit motor encoders are on-axis magnetic sensors (AS5047P class) read by the joint driver at 20 kHz; the 18-bit output encoders are off-axis rings around the hollow output shaft. Three details decided whether they met the datasheet: the magnet gap (1.0 ± 0.3 mm, set by a machined shoulder, not a shim); the brake coil, which shifted the motor encoder by 0.4° on the first module until it was moved 15 mm from the magnet; and PWM noise from the motor phases, cured by giving the output encoder its own shielded pair with a 360° shield clamp at the housing.

Stiffness, resonance and servo bandwidth

The gear's torsional compliance and the arm's inertia form a spring–mass resonance that bounds the position-loop bandwidth. At the shoulder, $f_r = \frac{1}{2\pi}\sqrt{K/J_L} = \frac{1}{2\pi}\sqrt{6000/0.287} = 23.0$ Hz with the arm

stretched; with the arm folded the inertia falls and the resonance rises to about 40 Hz. A position loop closed on the output encoder through this compliance is non-collocated, so its crossover is held at about a third of the lowest resonance, 7–8 Hz, with a notch filter at 23 Hz in the velocity loop (the loop design is Chapter 15's; resonance in two-mass systems is in *Engineering Physics*, Ch. 19 §19.3). The quintic time scaling of Section 13.6 keeps the trajectory's spectral content well below 23 Hz.

> **Status of the principle: Bandwidth below the first structural resonance**
> *Empirical.* A position loop closed through a compliant transmission is usually given a crossover below one third to one fifth of the first anti-resonance or resonance; above that, phase lost to the resonance makes the loop oscillate or ring, and the ratio depends on damping and on whether the sensor is collocated with the motor.

Bus power and regeneration

The six joints share a 24 V bus from a 150 W supply. The power budget (Table 13.8) is set by the simultaneous peak of J1–J3 during a fast transfer.

Table 13.8 — Power budget of the arm on the 24 V bus.

Load	Average (W)	Peak (W)	Basis
Joint logic and encoders, 6 modules	7.2	7.2	1.2 W each
Brakes released (J2, J3, J5)	12.0	12.0	4 W each while moving
J2 shoulder motion	14	42	23.5 W mechanical / (0.70 gear × 0.80 motor and driver)
J3 elbow motion	6	18	Same method
J1, J4–J6 motion	5	20	Same method
Gripper	1	6	Servo gripper, closing
Total	**45**	**105**	
Supply rating	150	150	24 V, 6.25 A
Margin	3.3 ×	1.43 ×	

Lowering the stretched arm returns energy to the bus: dropping the 2.45 kg outboard of the shoulder, whose combined centre of mass is about 0.32 m out, from horizontal to vertical releases $mgh \approx 2.45 \times 9.81 \times 0.32 = 7.6$ J, minus gear and motor losses. A supply cannot absorb it, so the bus has 2200 µF of bulk capacitance and a 28 V shunt clamp in the power-entry module; Chapter 15 sizes such a clamp for a robot that returns much more energy.

Safety functions

The E-stop and light curtain feed a safety relay rated for ISO 13849-1 performance level d. On a trip the controller makes a controlled stop on the trajectory (stop category 1 of IEC 60204-1), the relay removes gate-

driver power through each joint's STO input after 300 ms, and the brakes drop. While a person is inside the reach, the R6 speed limit applies, and a torque monitor stops the arm if any joint's measured torque departs from the gravity model by more than 1.5 N·m. Transient contact limits come from ISO/TS 15066; validating the stop functions is Chapter 21's subject.

13.10 Verification and failure analysis

Verification against the requirements

Table 13.9 closes every requirement with its test, measured on prototype B; repeatability used a 1 µm dial indicator (a laser tracker is planned for the production release).

Table 13.9 — Verification of the desktop lab arm (prototype B).

ID	Test	Limit	Result	Pass/fail
R1	Hold 0.5 kg at full horizontal reach; run 100 full-speed cycles	No following-error trip, J2 torque < 10 N·m	Peak 9.6 N·m, no trips	Pass
R2	Tool-down pose sweep at 60 mm height out to the reach limit	390 mm	397 mm reached; cond(J) 55 at 390 mm	Pass
R3	30 approaches from the same direction, 3 poses	±0.10 mm	±0.046 mm worst (far pose)	Pass
R4	Touch-off on 5 × 5 grid plate after calibration	±0.5 mm	0.31 mm worst, 0.17 mm RMS	Pass
R5	100 timed cycles	≤ 6.0 s	5.4 s mean, 5.6 s max	Pass
R6	TCP speed with curtain triggered; pressure film at 250 mm/s	≤ 250 mm/s; below ISO/TS 15066 hand limit	243 mm/s max; 41 % of the transient limit	Pass
R7	Remove 24 V at full reach with 0.5 kg	Drop < 1 mm	0.22 mm	Pass
R8	2 h cycling, thermocouples on J2, J3 windings	Rise < 40 K	J2 26 K, J3 33 K	Pass
R9	E-stop at full speed, log joint speed and STO	Stop ≤ 300 ms, STO after stop	210 ms to standstill, STO at 300 ms	Pass
R10	Costed BOM at 10 units	≤ $4,500	$4,180	Pass

The R3 result (±46 µm) exceeds the budget's 39 µm by frame micro-slip at the base, found by repeating the test with the base clamped; the production bench gets dowel pins as well as bolts.

Failure analysis

Table 13.10 is the FMEA of the arm as built. Several rows are failures that happened on the prototypes; they are kept because the mitigation is now part of the design.

Table 13.10 — FMEA of the desktop lab arm.

Failure mode	Effect	Cause	Detection	Mitigation
Shoulder stalls or loses position at full reach	Tool drops onto rack	Actuator sized at the zero pose, not the worst pose (prototype A)	Following-error trip	Size every joint at its worst pose with acceleration (Listing 13.3); torque margin check in design review
Tubes placed 0.6 mm low at full reach	Tube jams on rotor lip	Gear compliance not modelled	Touch-off grid test	Gravity-sag feed-forward; accuracy budget with compliance line
IK returns an equivalent angle outside limits	Fast 360° swing of J1 toward the operator	Missing angle wrap and limit check	Simulation of every taught path	Nearest-equivalent wrap, limit check before motion, refuse rather than clip
Wrist passes near $q_5 = 0$ in a Cartesian move	J4 and J6 spin at high speed, tube spills	Taught pose close to wrist singularity	Condition-number monitor > 300	Refuse move; keep $\lvert q_5 \rvert > 10^\circ$ in taught poses
Cable fatigue at J4 after 0.8 million cycles	Intermittent CAN errors, joint drops out	Service loop too short, twist concentrated over 120 mm	Rising CAN error counters	Torsion-rated cable, 300 mm free length, error-counter alarm
Strain-wave gear ratchets in a collision	Permanent position offset of a few teeth	Momentary torque rating exceeded	Output and motor encoder disagree	Torque monitor stop, current limit below the gear's momentary rating; dual-encoder consistency check at every power-up

What changed between prototype A and prototype B

The opening's two failures were two calculations not done: torque at the worst pose, and a compliance line in the accuracy budget. Prototype B changed the shoulder actuator, added gravity-sag compensation, brake management and IK checks that refuse bad moves. The link lengths and joint count did not change: the architecture was right, the integration was not.

What remains open

Open question	Where it is answered
How a robot moves on wheels, keeps track of its pose and plans a path	Chapter 14
How each joint's driver closes current, velocity and position loops, reads encoders and talks on the bus	Chapter 15
How the arm's model is identified and the whole system tested in the loop before field use	Chapter 17
Kinematic calibration of DH parameters from measured poses as a parameter-estimation problem	Chapter 17; Chapter 33 (calibration of models)
Validation of safety functions, safety relays and performance levels	Chapter 21
Sensor principles for the camera, encoders and force sensing	Chapter 6
Motor torque–speed behaviour, servo sizing rules and drive control	*Electrical Energy and Machines*, Ch. 8 and Ch. 10

Next: Chapter 14 takes the robot off the bench and onto wheels: a differential-drive platform whose position is not measured but estimated, and whose errors grow with every metre it drives.

Chapter 14 — Mobile Robotics

Part IV · Robotics and Mechatronics

A hospital runs a pilot of a small delivery robot that carries specimen totes from the wards to the laboratory: a differential-drive base, 20 kg empty and 30 kg loaded, with a 2D lidar, wheel encoders and an onboard computer running a standard navigation stack. In the first week it delivers 96 % of its missions. The failures cluster in one place: the 60 m basement corridor to the lab, where it repeatedly stops 0.9 m short of the lab door, turns into the wall and asks for help. The lidar was not at fault, and neither was the planner. The corridor has parallel, featureless walls, so scan matching fixes the robot's lateral position and heading but cannot say how far along the corridor it is; along that axis the position comes from wheel odometry alone, and the tyres, compressed by the payload and worn by a week of driving, had a rolling radius 1.5 % smaller than the value in the configuration file. A 1.5 % scale error over 60 m is 0.9 m. This chapter designs the second version of that robot, the reference system of the chapter, from wheel geometry and encoders through odometry and its calibration, obstacle detection, planning, localization and navigation, to the motor and battery sizing and the tests that close each requirement.

> **How does a wheeled robot know where it is, how wrong that knowledge becomes with every metre, and which sensors and calibrations keep the error inside a budget?**
> **How are its motors, battery and obstacle sensors sized so that it climbs the ramp, lasts the shift and stops before it touches anyone?**

Concept	Engineering destination
Differential drive	Delivery and service robots, AGVs, robot vacuums, research platforms
Wheel encoders	Odometry, wheel-speed loops, slip detection
Line following	Tape-guided AGVs, education platforms, fixed-route tuggers (full project in Chapter 34)
Obstacle detection	Protective stops, speed-dependent safety fields, overhang and cliff detection
Odometry	Dead reckoning between absolute fixes; the motion model of every localizer
Path planning	Global routes on occupancy grids; clearance from walls and doors
Localization	Scan matching against a map, fiducials at docks, sensor fusion with a gyro
Navigation	Local path following, speed zones, recovery behaviours, mission state machines
Battery and motor sizing	Ramps, thresholds and shift length turned into motor, gear and pack ratings
Robot testing	Calibration runs, stopping tests, endurance runs, field pilots with metrics

Roadmap. Requirements and architecture (14.1) → the differential drive (14.2) → wheel encoders (14.3) → line following as the fixed-route alternative (14.4) → obstacle detection and stopping (14.5) → odometry and its calibration (14.6) → path planning (14.7) → localization (14.8) → navigation (14.9) → battery and motor sizing (14.10) → testing and failure analysis (14.11).

14.1 Requirements and architecture

The robot works in wards, lifts and a basement corridor, among patients, staff and beds. Table 14.1 gives the requirements of version 2.

Table 14.1 — Requirements of the hospital delivery robot.

ID	Requirement	Target	Verification
R1	Payload	10 kg tote; 30 kg gross	Loaded runs for all tests
R2	Speed	1.2 m/s in empty corridors; 0.6 m/s in shared zones	Speed log against zone map
R3	Terrain	1:12 ramp loaded; 15 mm threshold; 20 mm lift-door gap	Ramp and threshold track
R4	Endurance	8 h shift on an end-of-life pack with 20 % reserve	Energy budget, 8 h run
R5	Odometry	Systematic drift < 1 % of distance after calibration	4 m square test, both directions; 50 m straight
R6	Localization	±50 mm, ±2° (95 %) anywhere on the map, including the 60 m corridor	Ground truth at 20 marked points
R7	Docking	±10 mm, ±1° at the charger contacts	100 docking cycles
R8	Obstacle stop	Stop before contact with a standing person or a 70 mm table leg at any permitted speed; detect overhangs up to 1.2 m height	Stopping test with test pieces
R9	Cliff	Stop before a drive wheel crosses a 50 mm drop	Platform-edge test
R10	Availability	≥ 99 % mission success; ≤ 1 intervention per 50 km	4-week field pilot

Figure 14.1 shows the architecture. The split follows the timing: a navigation computer (an 8-core ARM module of the Jetson Orin Nano class running ROS 2) does mapping, localization, planning and the mission logic at 10–20 Hz; a motor-control microcontroller (STM32G4 class) closes the wheel-speed loops at 1 kHz, integrates odometry, reads the IMU, bumpers and cliff sensors, and stops the motors on its own if the computer goes silent for 200 ms. A safety laser scanner with its own certified logic cuts the motor drivers' safe-torque-off line through a safety relay, independently of both processors. Chapter 15 designs the motor-control board; this chapter treats it as a block with an interface: wheel-velocity commands in at 50 Hz, odometry and status out at 100 Hz over a 1 Mbit/s UART.

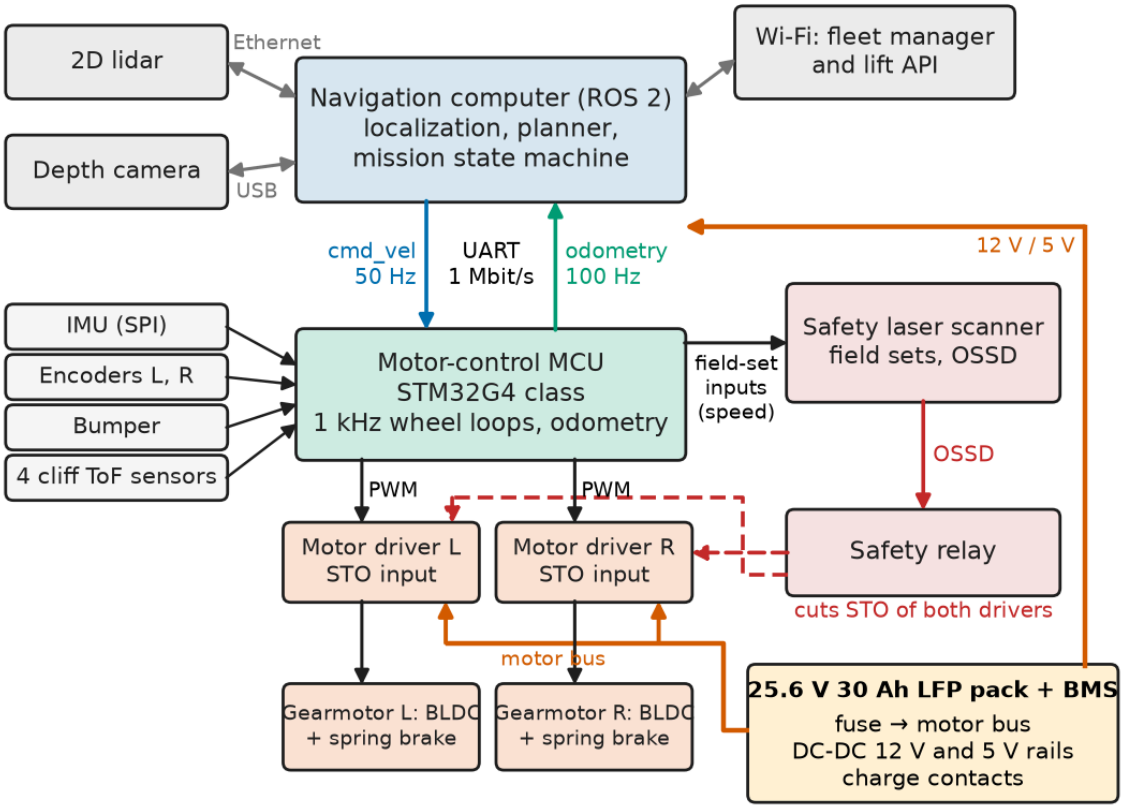


Figure 14.1 — Architecture of the hospital delivery robot: navigation computer, motor-control MCU, sensors, safety chain and power.

14.2 Differential-drive robots

Kinematics and the instantaneous centre of rotation

A differential-drive robot has two independently driven wheels on one axle and one or more passive casters. With wheel radius r, track width b (distance between the wheels' contact lines) and wheel angular speeds ω_L, ω_R, the forward speed v and yaw rate ω of the axle midpoint are

$$v = \frac{r(\omega_R + \omega_L)}{2}, \qquad \omega = \frac{r(\omega_R - \omega_L)}{b},$$

and the inverse used by every controller is $v_{L,R} = v \mp \omega b/2$. The robot turns about an instantaneous centre of rotation on the axle line at radius $R = v/\omega$ from the midpoint; equal and opposite wheel speeds put that centre at the midpoint and the robot spins in place (Figure 14.2).

> **Status of the principle: Differential-drive kinematics**
> *Derived.* Follows from rigid-body motion of the axle with rolling without slip at both contact points; it fails when the wheels slip (wet floors, hard acceleration, turning in place with stiff casters) and when the effective contact points move across a wide tyre, which is why b is calibrated, not measured with a ruler.

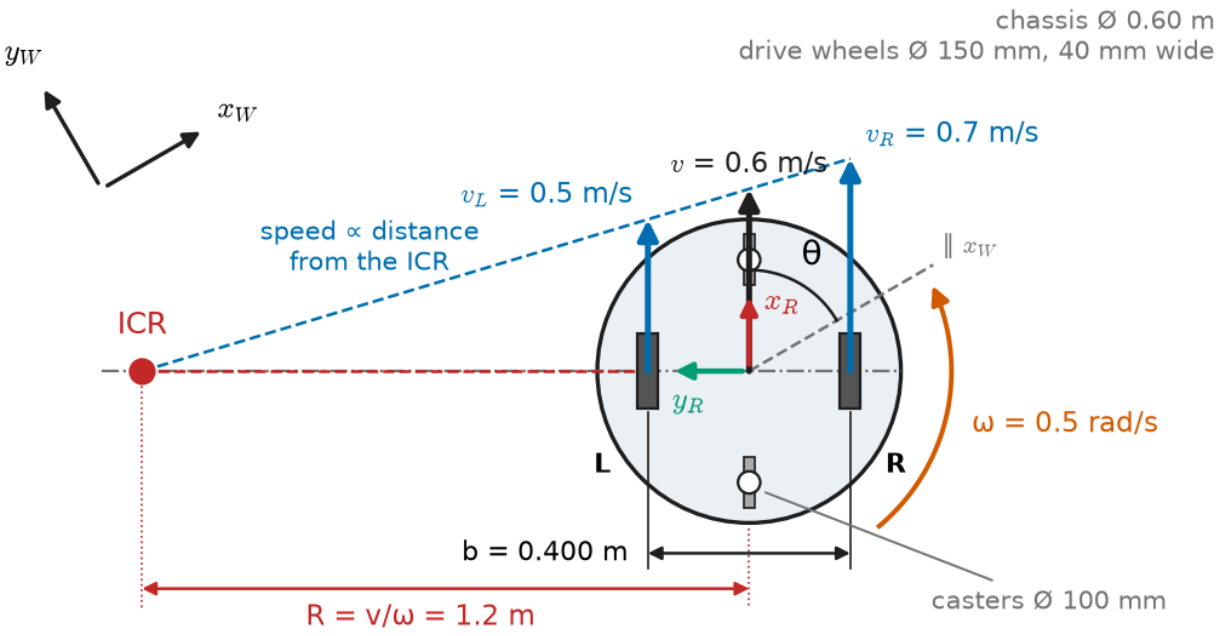


Figure 14.2 — Differential-drive geometry: wheels, track width, casters, robot frame and the instantaneous centre of rotation.

Why differential drive, and its geometry

Table 14.2 compares the drive configurations considered.

Table 14.2 — Drive configurations for an indoor delivery robot.

Configuration	Manoeuvrability	Odometry quality	Mechanical cost	Notes
Differential drive + casters	Turns in place; cannot move sideways	Good on hard floors	Lowest: two drives	Casters flutter and resist on thresholds
Skid steer (4 or 6 driven wheels)	Turns in place with scrubbing	Poor in turns: slip is part of steering	Medium	For outdoor and rough floors
Ackermann (car-like)	Minimum turning radius	Good	Medium, steering actuator	Cannot turn in a lift
Mecanum or omni wheels	Holonomic: any direction	Poor: rollers slip	High: four drives	Vibration on joints and tiles

The robot must turn round in a 1.4 × 2.0 m lift car, which rules out Ackermann steering, and it runs on vinyl and tiles, where differential drive gives the best odometry for the lowest cost. The geometry follows from the corridors and lift: a round chassis 0.60 m in diameter with the axle through its centre, so turning in place sweeps no more than the footprint; 150 mm polyurethane-tread drive wheels on a 0.400 m track; and two 100 mm casters, one front and one rear, on a rocker so that all four wheels touch on uneven floors.

Worked example — wheel speeds for a corridor turn. Turning a 90° corner on a 1.2 m radius at $v = 0.6$ m/s needs $\omega = v/R = 0.5$ rad/s. The wheels must run at $0.6 \mp 0.5 \times 0.400/2$, so 0.5 m/s and 0.7 m/s, or $0.5/0.075 = 6.67$ and $0.7/0.075 = 9.33$ rad/s. At the top speed of 1.2 m/s the outer wheel of the same turn would need 1.40 m/s, beyond the drive's 1.31 m/s maximum (Section 14.10); the speed limiter therefore caps v so that $|v| + |\omega| b/2 \leq 1.25$ m/s, keeping a small reserve for the wheel-speed loops.

14.3 Wheel encoders

Where to measure and with what

The odometry is only as good as the measurement of wheel rotation. Three options were compared (Table 14.3); counts are after 4× quadrature decoding where it applies.

Table 14.3 — Wheel-rotation sensing options (1:18 gearbox, 150 mm wheel).

Option	Counts per wheel revolution	Distance per count	Notes
BLDC Hall sensors (4 pole pairs, 6 states per electrical revolution)	432	1.09 mm	Free with the motor; too coarse for low-speed velocity
Magnetic encoder on motor shaft (4096 counts per motor revolution)	73,728	6.4 µm	Sees gearbox backlash, not the floor
Optical encoder on the wheel shaft (2048 counts)	2048	0.23 mm	Sees the wheel, not the motor; extra mechanics

The robot uses the motor-shaft magnetic encoder, read by the timer's hardware quadrature decoder on the motor-control MCU (Chapter 15 covers the interface). Distance resolution is far finer than needed: odometry error is dominated by wheel radius and track width, not by counts. Resolution matters for **velocity**, and that is where the Halls fail.

Worked example — speed resolution at walking-pace creep. Docking (R7) ends at 0.05 m/s. The encoder gives 156,456 counts per metre ($73{,}728/(\pi \times 0.150)$), so at 0.05 m/s it produces 7823 counts/s, 7.8 counts in each 1 ms control period: a count-difference speed estimate has a quantization step of 13 % of the speed, acceptable with a short filter. The Hall sensors give 917 counts per metre, 46 counts/s at 0.05 m/s, one count every 22 ms; a 1 kHz loop would see zero, zero, zero, one. Docking with Hall-only feedback produced visible stick–slip on the first prototype. Methods that measure time between counts help at low speed; they are Chapter 15's subject.

> **Status of the principle: Count-difference speed quantization**
> *Derived.* A speed estimate from counts in a fixed window T has a resolution of one count per window, $\Delta v = 1/(N_m T)$ for N_m counts per metre, so the relative resolution degrades in proportion to $1/v$ and is worst exactly where docking and slow manoeuvres happen.

The motor-side encoder sees through the gearbox's backlash (about 0.5° at the wheel, 0.65 mm at the rim). It does not affect straight-line odometry; it adds a small heading error on each reversal of one wheel, which is why the docking approach is planned to finish moving forward.

14.4 Line-following robots

The cheapest way to guide a robot along a fixed route is to stop localizing altogether and follow a line: a painted or taped stripe read by a reflectance array, or a magnetic tape read by a row of Hall elements. Tape-guided AGVs move parts in factories and carts in hospital basements, and the same idea is the classic first robot of an engineering course.

> **Boundary.**
> The complete line follower — sensor selection, steering-loop design and speed optimisation — is the project of Chapter 34; this section only sets out what the method implies for the reference robot.

A line sensor reports the lateral offset of the line from the robot centreline. An eight-element reflectance array (QTR-8 class, 10 mm pitch) gives the offset as the reflectance-weighted centroid of the element positions, resolving about 1 mm over a ±35 mm range; a magnetic guide sensor (16 Hall elements) does the same to ±1 mm through dirt and floor polish, which is why industrial AGVs use magnetic tape. The steering controller turns the offset and its rate into a yaw-rate command, and the limiting physics is geometric: the sensor sits a distance d ahead of the axle, so a heading error ψ shows up as an offset $d \sin \psi$ immediately, and the loop is damped by that lead.

Worked example — what limits speed on a tape route. With the sensor 0.15 m ahead of the axle, a ±35 mm sensing range, a 20 ms loop period and 30 ms of motor response, a 0.6 m radius bend entered at 1.0 m/s requires a yaw rate of 1.67 rad/s; the 50 ms of total delay lets the robot run $1.0 \times 0.05 = 0.05$ m before the steering responds, during which the line moves sideways under the sensor by roughly $d_s^2 / 2R = 0.05^2 / (2 \times 0.6) \approx 2$ mm plus the heading-induced $0.15 \times \sin(1.67 \times 0.05) = 12.5$ mm — inside the 35 mm range, but at 1.4 m/s the same calculation leaves a third less margin. This is the mechanism behind the oscillating line follower of Chapter 34.

The hospital rejected line guidance for the wards: tape is damaged by beds and cleaning machines, every route change needs new tape, and the robot cannot pass a parked trolley that sits on the line. It kept one use for it — a 600 mm strip of magnetic tape at the charger, which makes the last 0.5 m of docking repeatable to ±3 mm laterally regardless of localization (R7).

14.5 Obstacle detection

Sensors and what each one misses

Obstacles in a hospital are people, beds, trolleys, wheelchairs, open drawers, overbed tables (a top at 0.75 m on a single column), glass doors and the top of a staircase. No single sensor sees all of them. Table 14.4 compares the candidates.

Table 14.4 — Obstacle sensors for the delivery robot (2026 indicative prices).

Sensor (class)	Range, field	Sees	Misses	Safety-rated	Price
Safety laser scanner (nanoScan3 class)	3 m protective, 275°, one plane	People, legs, bed frames at 0.18 m height	Overhangs, glass, cliffs	Yes, PL d, IEC 61496-3	$2,400
2D lidar (RPLIDAR S2 class)	30 m, 360°, 10 Hz	Walls and furniture for mapping	Same plane limits; dark, glossy surfaces	No	$400
Depth camera (D435 class)	0.3–3 m, 87° × 58°	Overhangs, table tops, low objects	Glass, direct sun, close range	No	$300
Ultrasonic (MB1xxx class)	0.2–5 m, 30° cone	Glass doors, chair legs	Soft or angled surfaces; crosstalk	No	$30
Time-of-flight (VL53L1X class)	0.04–4 m, 27° cone	Floor edges when aimed down	Dark carpet at range	No	$6
Bumper switch strip	Contact	Anything it touches	Everything before contact	Can be	$60

The design uses the safety scanner as the only device in the protective-stop path; the 2D lidar for mapping and localization (mounted higher, at 0.32 m, where fewer moving legs hide the walls); the depth camera tilted down 25° for overhangs between 0.1 and 1.2 m (R8); two ultrasonic sensors at the front for glass; four downward ToF sensors at the chassis corners as cliff detectors (R9); and a bumper as the last resort. The non-safety sensors feed the navigation costmap; they slow and steer the robot but are not trusted to stop it.

Stopping distance and field sizing

A protective field must be longer than the distance the robot travels after an obstacle enters it. That distance has a reaction part (detection, decision, actuation) and a braking part:

$$s_{\text{stop}} = v\,t_r + \frac{v^2}{2a},$$

> **Status of the principle: Stopping distance from reaction time and deceleration**
> *Derived.* Constant speed during the reaction time followed by constant deceleration; it underestimates when deceleration builds up gradually or traction is lost, so the field length adds the scanner's measurement tolerance and a margin, and the result is verified by measurement at each speed.

Worked example — field length for 1.2 m/s and 0.6 m/s. The reaction chain is: scanner response 80 ms (two scans), safety relay 20 ms, STO and drive braking onset 70 ms, total $t_r = 0.17$ s. Deceleration is limited not by traction (μ = 0.6 on vinyl would allow over 4 m/s² with 70 % of the weight on the drive wheels) but by the payload: specimen tubes in a tote shift above about 1.5 m/s², so the controlled stop uses 1.0 m/s². At 1.2 m/s, $s_{\text{stop}} = 1.2 \times 0.17 + 1.2^2/2 = 0.20 + 0.72 = 0.92$ m; adding 0.10 m of scanner tolerance and 0.10 m of margin gives a 1.12 m protective field. At 0.6 m/s the same calculation gives $0.10 + 0.18 = 0.28$ m, and a 0.48 m field. The scanner switches between field sets according to the speed reported by the MCU through two safety inputs, so the robot can pass through a 0.9 m door at walking pace without

false stops and still stop in time at full speed. A field that is long enough at 1.2 m/s would stop the robot in every doorway, which is the real reason for R2's two speeds.

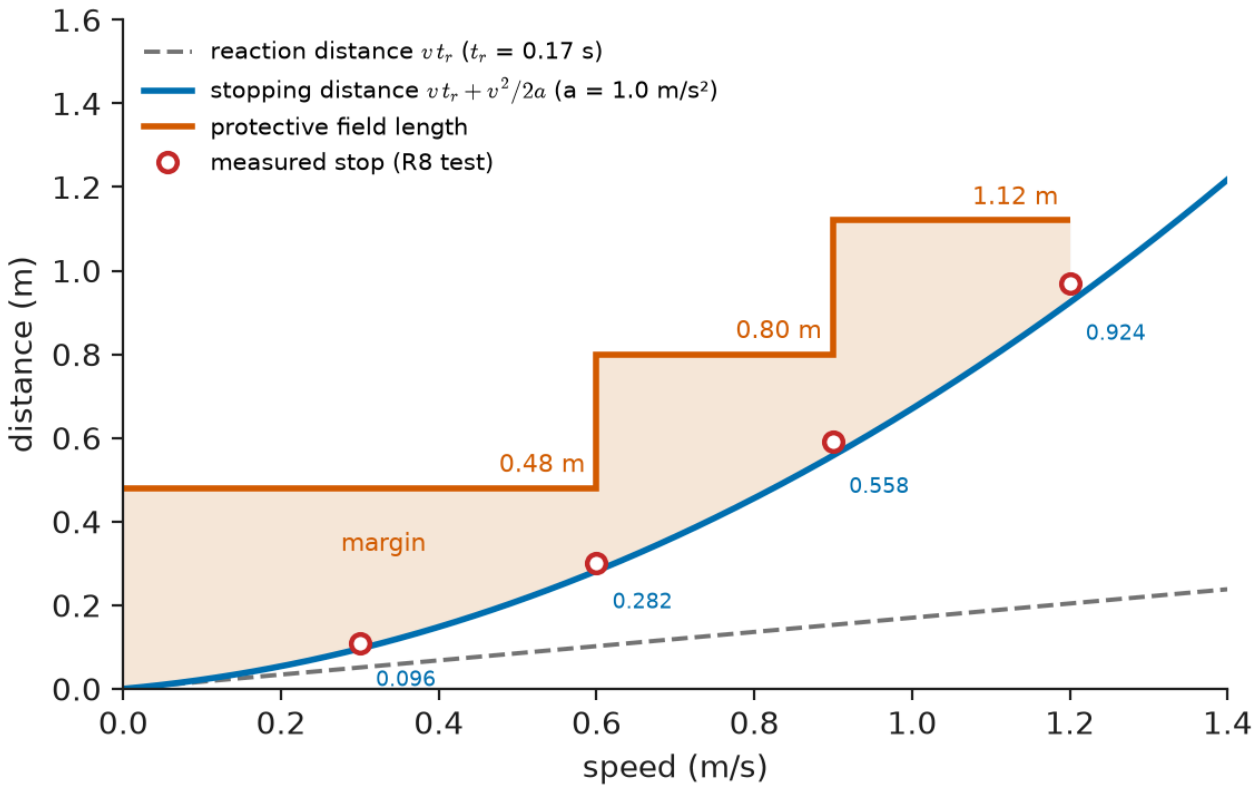


Figure 14.3 — Stopping distance of the loaded robot against speed, and the speed-dependent protective fields.

Standards set the frame: a driverless industrial truck falls under ISO 3691-4, which requires personnel detection and speed-dependent protective fields; a service robot in public spaces is typically assessed against ISO 13482. The hospital's risk assessment used both, and the stopping test of Section 14.11 is taken from them.

14.6 Odometry

Dead reckoning from wheel increments

Odometry integrates wheel increments into a pose. In each 1 ms period the MCU reads the left and right count increments, converts them to distances $\Delta s_L, \Delta s_R$ with each wheel's own calibrated radius, and updates

$$\Delta s = \frac{\Delta s_R + \Delta s_L}{2}, \quad \Delta\theta = \frac{\Delta s_R - \Delta s_L}{b}, \quad x \mathrel{+}= \Delta s \cos\!\left(\theta + \tfrac{\Delta\theta}{2}\right), \quad y \mathrel{+}= \Delta s \sin\!\left(\theta + \tfrac{\Delta\theta}{2}\right), \quad \theta \mathrel{+}= \Delta\theta.$$

> **Status of the principle: Odometry from wheel increments**
> *Derived.* A midpoint integration of the differential-drive kinematics over each sample; with millimetre increments its numerical error is negligible, and every real error comes from the parameters (r_L, r_R, b) and from slip, which the equations cannot see.

Systematic and random errors

Two kinds of error accumulate. **Systematic** errors repeat every time: unequal wheel radii (from tolerance, wear or an off-centre payload compressing one tyre more), an effective track width different from the nominal one, and a mean radius error that scales every distance. **Random** errors do not repeat: slip on wet floors and tile joints, floor unevenness, encoder quantization. Systematic errors are removed by calibration; random errors are bounded by fusing other sensors. The asymmetry that matters is that a heading error turns into a lateral position error that grows with distance: 1° of heading error is $20 \sin 1^\circ = 0.35$ m sideways after 20 m and 1.05 m after 60 m.

Calibration: the bidirectional square test

The bidirectional square test (UMBmark, after Borenstein and Feng) separates the two dominant systematic errors. The robot drives a 4 m square clockwise and anticlockwise, several times each, under odometry control; at the end, its true position is marked on the floor and compared with the start. An unequal wheel-radius ratio $E_d = r_R / r_L$ curves every straight leg, and a track-width error makes every turn too large or too small. In one direction the two errors add and in the other they partly cancel, so a single-direction test can look excellent while the robot is badly calibrated. Listing 14.1 simulates the test and recovers the two parameters by least squares from the four end-point coordinates (least-squares fitting is in *The Mathematics Universe*, Ch. 23 §23.8).

```
 import numpy as np
from scipy.optimize import least_squares

def square(true, model, cw, L=4.0, n=200):
    """Drive a 4 m square using the odometry model (rL, rR, b) the robot believes;
    integrate the motion the floor produces with the true (rL, rR, b). End point."""
    (rL, rR, b), (mL, mR, mb) = true, model
    s = -1.0 if cw else 1.0
    seg = [(L/mL, L/mR), (-s*np.pi/4*mb/mL, s*np.pi/4*mb/mR)]   # wheel angles, rad
    x = y = th = 0.0
    for _ in range(4):
        for pL, pR in seg:
            for _ in range(n):
                dL, dR = rL*pL/n, rR*pR/n
                ds, dth = 0.5*(dL + dR), (dR - dL)/b
                x += ds*np.cos(th + dth/2); y += ds*np.sin(th + dth/2); th += dth
    return np.array([x, y])

def wheels(Ed, b, r=0.075):          # diameter ratio Ed = rR/rL, mean radius r
    return (2*r/(1 + Ed), 2*r*Ed/(1 + Ed), b)

true, nominal = wheels(1.005, 0.406), wheels(1.0, 0.400)
ends = lambda model: np.concatenate([square(true, model, cw) for cw in (True, False)])
rng = np.random.default_rng(1)
meas = ends(nominal) + rng.normal(0, 0.005, 4)   # floor marks, +-5 mm tape
print("before: CW %.3f m, CCW %.3f m" % tuple(np.hypot(*meas.reshape(2, 2).T)))
fit = least_squares(lambda p: np.concatenate(
          [square(wheels(*p), nominal, cw) for cw in (True, False)]) - meas,
          [1.0, 0.400], diff_step=1e-4)
Ed, b = fit.x
print("estimated Ed = %.4f, b = %.4f m" % (Ed, b))
after = ends(wheels(Ed, b))
print("after:  CW %.4f m, CCW %.4f m" % tuple(np.hypot(*after.reshape(2, 2).T)))
```

```
 before: CW 0.842 m, CCW 0.295 m
estimated Ed = 1.0050, b = 0.4059 m
after:  CW 0.0044 m, CCW 0.0054 m
```

Worked example — reading the square test. The simulated robot has a 0.5 % wheel-diameter mismatch and an effective track width of 0.406 m against the nominal 0.400 m (a 1.5 % error, typical when 40 mm-wide treads contact on their outer edges under load). Over 16 m of path (Figure 14.4) the clockwise run ends 0.842 m from its start (5.3 % of the distance) and the anticlockwise run 0.295 m (1.8 %): the two errors add in one direction and partly cancel in the other. The fit recovers both parameters — $E_d = 1.0050$ and $b = 0.4059$ m — even with ±5 mm of measurement scatter on the floor marks, and with the corrected model the simulated squares close to within 5 mm. On the real robot, five runs each way after calibration closed to 48–71 mm (0.3–0.45 %), the remainder being slip and floor joints that the model does not contain; R5's 1 % limit is met with margin. The square test does not see the third systematic parameter, the **mean** radius, because it scales the whole square; that is calibrated separately by driving 50 m along a surveyed line

and comparing with the count. That is the parameter that sent the pilot robot 0.9 m short of the lab door, and it changes with payload and tyre wear, so version 2 re-estimates it on line (Section 14.8).

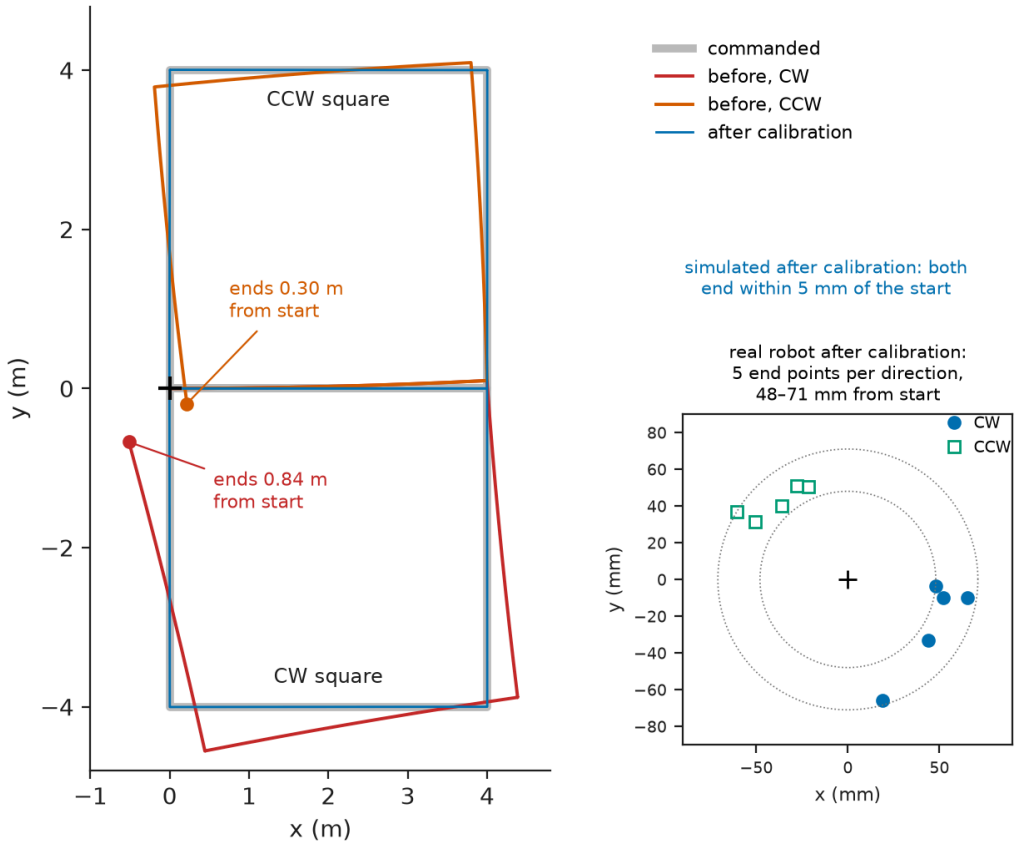


Figure 14.4 — Bidirectional 4 m square test: true paths of the robot before and after odometry calibration.

14.7 Path planning fundamentals

Maps, costmaps and planners

The robot plans on an **occupancy grid**: the floor divided into 5 cm cells, each free, occupied or unknown, built once by driving the robot around with a mapping (SLAM) tool and then edited by hand to add keep-out zones (stairwells, the top of the loading ramp) and speed zones. Planning on a grid treats the robot as a point, so obstacles are first **inflated**: every cell within the robot's circumscribed radius (0.30 m) of an obstacle becomes lethal, and a band beyond it, out to 0.55 m, gets a cost that falls with distance, so that the planner prefers the middle of a corridor and the centre of a door without forbidding a squeeze.

The global planner is A*: a best-first graph search that expands cells in order of cost so far plus an admissible estimate (the straight-line distance) of the cost to go, and returns the cheapest path on the grid. Its alternatives are listed in Table 14.5.

Table 14.5 — Global planners for indoor robots.

Planner	Optimal	Handles robot shape and turning limits	Typical use
Dijkstra / A* on a grid	Yes, on the grid	No: point robot on inflated map	Differential-drive robots indoors
Hybrid A* / state lattice	Near-optimal	Yes: heading and curvature in the state	Car-like robots, tuggers, AGVs with trailers
RRT* and sampling planners	Asymptotically	Yes	Arms, high-dimensional spaces
Visibility graph / topological	No	No	Route networks between waypoints

Listing 14.2 builds a 12 × 20 m section of the ward floor (two rooms separated by a wall with two 0.9 m doors, plus a parked bed), inflates it and plans with A*.

```
 import heapq, math, time
import numpy as np
from scipy.ndimage import distance_transform_edt

res = 0.05                                      # m per cell
occ = np.zeros((240, 400), bool)                # 12 m x 20 m floor section
occ[[0, -1], :] = occ[:, [0, -1]] = True        # outer walls
occ[100:104, :] = True; occ[100:104, 60:78] = False    # wall with 0.9 m door
occ[100:104, 300:318] = False                           # second 0.9 m door
occ[150:170, 180:200] = True                    # parked bed, 1.0 m x 1.0 m
dist = distance_transform_edt(~occ)*res         # distance to nearest obstacle, m
r_robot, r_infl = 0.30, 0.55
cost = np.where(dist <= r_robot, np.inf,        # lethal: robot would touch
       1.0 + 10.0*np.clip((r_infl - dist)/(r_infl - r_robot), 0, 1))   # keep-away
C = cost.tolist()                               # plain lists: faster in pure Python

def astar(start, goal):
    moves = [(1, 0), (-1, 0), (0, 1), (0, -1), (1, 1), (1, -1), (-1, 1), (-1, -1)]
    h = lambda c: res*math.hypot(c[0] - goal[0], c[1] - goal[1])
    g, parent, todo, n = {start: 0.0}, {start: None}, [(h(start), start)], 0
    while todo:
        _, c = heapq.heappop(todo); n += 1
        if c == goal:
            path = []
            while c: path.append(c); c = parent[c]
            return path[::-1], g[goal], n
        for dr, dc in moves:
            nb = (c[0] + dr, c[1] + dc)
            step = res*math.hypot(dr, dc)*C[nb[0]][nb[1]]
            if step < math.inf and g[c] + step < g.get(nb, math.inf):
                g[nb], parent[nb] = g[c] + step, c
                heapq.heappush(todo, (g[nb] + h(nb), nb))
    return None, np.inf, n

t0 = time.perf_counter()
path, J, n = astar((40, 20), (200, 380))        # ward corridor -> lab door
p = np.array(path)*res
length = np.sum(np.hypot(*np.diff(p, axis=0).T))
print(f"path {length:.2f} m, {len(path)} cells, {n} expansions, "
      f"min clearance {dist[tuple(np.array(path).T)].min():.2f} m, "
      f"{(time.perf_counter() - t0)*1e3:.0f} ms")
```

```
path 21.93 m, 382 cells, 76372 expansions, min clearance 0.45 m, 651 ms
```

Worked example — what the planner's numbers say. The straight-line distance from start (1.0, 2.0) m to goal (19.0, 10.0) m is 19.70 m; the path (Figure 14.5) is 21.93 m because it must pass through the first door, whose centre it finds exactly: the minimum clearance of 0.45 m is half of the 0.9 m door width, leaving 0.15 m on each side of the 0.30 m robot radius. The search expanded 76,372 of the 92,876 free cells — the keep-away cost makes the heuristic weak, because the straight-line estimate assumes the cheapest cost of 1 per metre everywhere — and took 0.65 s in pure Python on a laptop. The navigation stack's C++ planner does

the same job on the robot in about 40 ms, and replans at 1 Hz; the Python version is the reference implementation used in regression tests, not the onboard planner.

> *Status of the principle: A optimality with an admissible heuristic* Derived.* If the heuristic never overestimates the remaining cost, A* returns a least-cost path on the graph it searches; the path is optimal for the grid, the inflation and the cost function chosen, not for the continuous floor or the robot's dynamics, which is why a local controller smooths and tracks it.

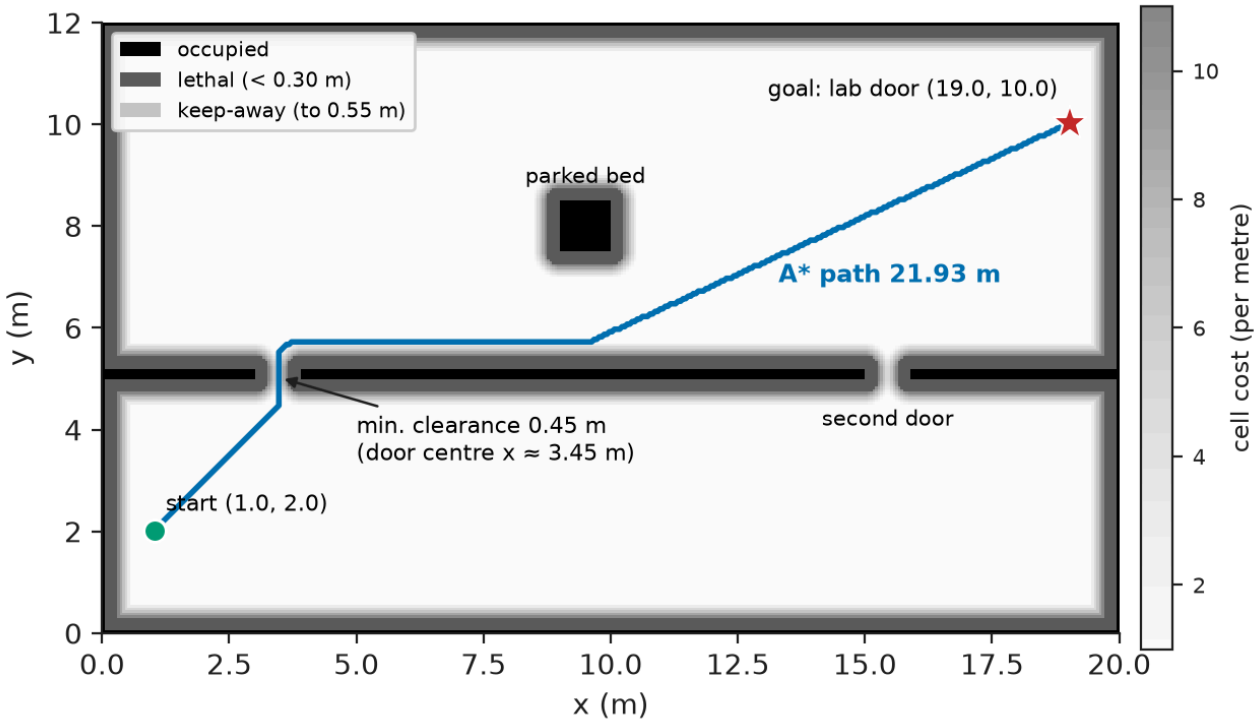


Figure 14.5 — Occupancy grid of the ward section after inflation, with the A path from the ward corridor to the lab door.*

14.8 Localization

What each source contributes

Odometry drifts; the robot needs absolute fixes. Version 2 combines four sources, each with a known role:

Source	Rate	Typical accuracy	Fails when
Wheel odometry (calibrated)	100 Hz	0.3–0.5 % of distance; heading drift on slip	Wet floor, thresholds, wheel lifted
Gyro (ICM-42688 class, bias re-estimated at every stop)	100 Hz	Heading drift about 0.5°/min residual	Long periods without a stop; temperature steps
Lidar scan matching against the map (particle filter)	10 Hz	±30 mm, ±1° where geometry is rich	Long featureless corridors; crowds hiding walls
Fiducial markers (AprilTag class) at docks and corridor ends	On sight	±5 mm, ±0.3°	Not in view

An extended Kalman filter fuses odometry and gyro into a smooth, high-rate pose (the gyro replaces the wheel-difference heading during turns, where casters cause slip); a particle filter (adaptive Monte Carlo localization with 500–2000 particles) corrects it against the map at 10 Hz; and fiducials snap the estimate at the places where accuracy is required.

> **Boundary.**
> The filters weight each source by its stated uncertainty; Bayesian updating and Gaussian error propagation are owned by *The Mathematics Universe*, Ch. 19 §19.3 and Ch. 21 §21.5, and the observer idea that the Kalman filter makes optimal is in *Circuits, Signals and Communications*, Ch. 35 §35.7.

> **Status of the principle: Kalman fusion of odometry, gyro and absolute fixes**
> *Approximate.* The Kalman filter is the minimum-variance estimator for linear models with Gaussian, white, correctly specified noise; the extended version linearises the motion model, and both become overconfident when errors are systematic (an uncalibrated radius) or correlated, which is why systematic errors are calibrated out first and the noise parameters are tuned on logged data.

The corridor problem and the fix

Scan matching needs geometry in every direction. In the 60 m basement corridor, two parallel walls constrain the lateral position and the heading but not the position along the corridor: every along-track shift fits the scan equally well. The particle cloud stretches along the corridor and the filter falls back on odometry — exactly the axis where the pilot robot's 1.5 % radius error accumulated.

Worked example — the along-track error budget in the corridor. The requirement is ±50 mm at the lab door (R6). With the mean radius calibrated to 0.3 % and slip adding about 0.2 % of distance, the along-track error after 60 m is $60 \times \sqrt{0.003^2 + 0.002^2} = 0.22$ m, still four times too large. Version 2 therefore adds (a) a fiducial marker on the wall at each end of the corridor and one at the midpoint, so no stretch of odometry-only travel exceeds 30 m and the error at a marker is at most $30 \times 0.0036 = 0.11$ m before the marker snaps it to ±5 mm, and (b) on-line estimation of the mean wheel radius: each time two markers are passed, the ratio of surveyed to odometric distance updates the radius, so that payload and wear changes are tracked within a day. Between the last marker (8 m from the door) and the door, the error grows to $8 \times 0.0036 = 0.029$ m, inside the 50 mm requirement with a margin of 1.7.

14.9 Navigation

From path to wheel commands

Navigation turns the planned path into motion that is safe and smooth. The stack has three layers. The **global planner** (Section 14.7) produces a path on the static map, replanned at 1 Hz. The **local controller**, running at 20 Hz on a 4 × 4 m rolling costmap that also contains what the lidar, depth camera and ultrasonic sensors see now, tracks the path while avoiding new obstacles. The **mission layer**, a state machine, sequences the job: leave dock, drive to the lift, call it through the building API, enter, ride, exit, drive to the ward, wait for the nurse's badge, return.

The local controller is a regulated pure-pursuit tracker: it picks a goal point on the path a look-ahead distance L_d ahead and steers along the circular arc through it, with curvature

$$\kappa = \frac{2\,y_g}{L_d^2}, \qquad \omega = v\,\kappa,$$

where y_g is the goal point's lateral offset in the robot frame. Its speed is regulated down near obstacles (by the costmap value under the robot) and in tight curvature, and capped by the zone map (R2).

> **Status of the principle: Pure-pursuit steering law**
> *Derived.* The arc through the robot's position, tangent to its heading and through a point at distance L_d and lateral offset y_g, has curvature $2y_g/L_d^2$; it assumes the robot follows commanded curvature instantly, so a short look-ahead oscillates when delays are present and a long one cuts corners.

Worked example — look-ahead and wheel speeds. At 0.6 m/s with $L_d = 0.6$ m, a goal point 0.15 m to the left gives $\kappa = 2 \times 0.15/0.36 = 0.833$ m⁻¹ and $\omega = 0.5$ rad/s, so the wheels run at 0.5 and 0.7 m/s as in Section 14.2. The look-ahead is scheduled with speed ($L_d = 1.0 \times v$, limited to 0.4–1.2 m): too short and the 60 ms of command latency between computer and wheels makes the robot weave; too long and it cuts the inside of door frames by up to $L_d^2/(8R)$ — 0.15 m for a 1.2 m look-ahead on a 1.2 m-radius corner, which the costmap's keep-away band (0.25 m wide) absorbs.

Behaviours that make it work in a building

The behaviours that keep the success rate at 99 % were all added after a field failure: wait-then-replan when the path is blocked by a person (2 s, then a detour if one exists, then a polite voice prompt, then a call for help after 60 s); a "narrow passage" mode that drops to 0.3 m/s and the smallest protective field to pass a door with a trolley beside it; lift entry backwards-out logic (enter forwards, turn in place inside, exit forwards); and recovery behaviours (clear the local costmap, rotate in place, back up 0.3 m) when the controller reports no progress for 10 s.

14.10 Battery and motor sizing

Torque and speed at the wheel

The drive is sized for four cases: acceleration on the level, climbing the 1:12 ramp, mounting the 15 mm threshold and holding on the ramp with power off. The rolling-resistance coefficient of polyurethane on vinyl

is taken as 0.015; drive wheels carry 70 % of the 30 kg gross mass. The motor and drive fundamentals (referred torque, gear efficiency, torque–speed limits) are those of *Electrical Energy and Machines*, Ch. 10 §10.1–10.2.

Worked example — ramp, threshold and gear ratio. On the 1:12 ramp (4.76°) while accelerating at 0.5 m/s², the traction force is $F = m\,(a + g\sin\theta + gC_{rr}\cos\theta) = 30 \times (0.5 + 0.814 + 0.147) = 43.8$ N, or 21.9 N and 1.64 N·m per wheel. The threshold is worse. A wheel of radius r mounting a step of height h quasi-statically pivots about the step edge, which lies a horizontal distance $d = \sqrt{r^2 - (r-h)^2} = \sqrt{0.075^2 - 0.060^2} = 0.045$ m ahead of the axle, so the wheel torque needed to lift the drive wheel's 103 N share of the weight is $103 \times 0.045 = 4.6$ N·m — 2.8 times the ramp case. The top speed fixes the ratio: 1.2 m/s is 153 rpm at the wheel, so a 3000 rpm motor through a 1:18 planetary gear (efficiency 0.85) gives 1.31 m/s maximum, and the threshold needs $4.6/(18 \times 0.85) = 0.30$ N·m at the motor for a fraction of a second. A 60 W, 24 V BLDC gearmotor rated 0.19 N·m continuous and 0.57 N·m peak covers it with a peak margin of 1.9, while the ramp needs only 0.107 N·m continuous at the motor, a thermal margin of 1.8. The casters, not the motors, turned out to be the threshold problem: a 50 mm caster mounting a 15 mm step needs a horizontal push of $44 \times 0.0229/0.010 = 101$ N, more than the 124 N traction limit of both drive wheels leaves after they climb themselves; 100 mm casters need 45 N.

> **Status of the principle: Quasi-static step-climbing torque**
> *Approximate.* Treats the wheel as rigid, pivoting about the step edge with no momentum; a soft tyre and approach speed both reduce the torque needed, so the result is a conservative bound used for motor peak torque, checked on the threshold track.

The motor options are compared in Table 14.6.

Table 14.6 — Drive motor options (per wheel, 2026 prices, 100 pcs).

Option	Continuous / peak torque at wheel	Encoder	Brake	Efficiency at cruise	Price	Verdict
Brushed DC gearmotor 24 V 60 W, 1:18	2.9 / 8.7 N·m	Add-on	Add-on	0.60	$85	Brush wear in 8 h × 365 d duty
BLDC gearmotor 24 V 60 W, 1:18, integrated encoder and brake	2.9 / 8.7 N·m	4096 counts per motor revolution	Spring-applied	0.70	$160	**Chosen**
BLDC hub motor 6.5 in, 250 W (scooter class)	8 / 20 N·m	Halls only	None	0.75	$70	Coarse feedback, no brake, too large
Stepper + belt	2 / 3 N·m falling with speed	Open loop	None	0.35	$60	Loses steps on the threshold; poor efficiency

The spring-applied brake is a safety requirement, not an option: on the ramp with power off, gravity applies $30 \times 9.81 \times \sin 4.76^\circ \times 0.075/2 = 0.92$ N·m to each wheel, which a backdrivable planetary gear would turn

into a runaway.

The energy budget

The battery is sized from an 8 h shift in which the robot drives 40 % of the time at an average of 0.8 m/s, about 9.2 km.

Table 14.7 — Energy budget for one 8 h shift.

Load	Power (W)	Duty	Energy (Wh)	Basis
Navigation computer	10.0	100 %	80.0	15 W mode, 65 % average utilisation
Safety laser scanner	4.0	100 %	32.0	Datasheet typical
2D lidar	2.5	100 %	20.0	
Depth camera	2.5	100 %	20.0	
MCU, IMU, ToF, ultrasonic	1.5	100 %	12.0	
Motor drivers, idle	1.0	100 %	8.0	
Wi-Fi and LTE modem	2.0	100 %	16.0	
Lights, display, speaker	3.0	100 %	24.0	
Traction while driving	10.0	40 %	32.0	3.5 W rolling + accelerations and ramps, drivetrain 0.55
DC–DC conversion losses	10 % of the above		24.4	
Total			**268**	33.5 W average
Usable from 25.6 V 30 Ah LFP at end of life			553	768 Wh × 0.90 depth of discharge × 0.80 capacity at end of life
Required with 20 % reserve			322	268 × 1.2
Margin			**1.72 ×**	

Worked example — what dominates the battery. Traction is 32 Wh out of 268, 12 %; the computer, sensors and radios are 180 Wh, 67 %. Halving the motors' losses would buy 16 Wh; putting the depth camera to sleep when the robot is parked, and running the computer at 10 W instead of 15 W when not planning, saves about 30 Wh. A 25.6 V 20 Ah pack (512 Wh) would have given 369 Wh usable at end of life, a margin of only 1.15 over the 322 Wh required, too little for a budget that has not yet met the field, so the 30 Ah pack

was chosen; it also carries the 23 A peak of both motors at full peak torque within its 1C (30 A) continuous rating. Battery ratings, depth of discharge and ageing are treated in *Electrical Energy and Machines*, Ch. 16 §16.2; the pack and BMS design belong to Chapter 23, and the bus transients when both motors brake at once to Chapter 15.

> **Status of the principle: Battery runtime from capacity and average power**
> *Derived.* Energy conservation with a constant usable fraction; it is approximate in two ways — usable capacity falls at low temperature and high current, and average power hides peaks — so the pack is also checked for peak current and for voltage sag at the end of discharge.

14.11 Robot testing

A test plan in layers

Mobile robots fail in the field in ways that bench tests do not expose, so testing is layered, each layer cheaper and earlier than the next:

1. **Simulation regression** — 40 scenarios (blocked door, crowd, lift, corridor, glass door) run in a physics simulator on every software change, with pass criteria on arrival accuracy, time and minimum clearance.
2. **Bench** — counts per metre on a roller, wheel-speed loop step responses, brake hold, watchdog stop when the computer is unplugged.
3. **Test track** — the bidirectional square, the 50 m surveyed straight, the ramp and threshold track, stopping distance with test pieces at four speeds, cliff edge.
4. **Endurance** — 8 h continuous missions on a full-life-equivalent pack (aged by cycling or simulated by a capacity limit in the BMS).
5. **Field pilot** — four weeks in the hospital with every intervention logged and categorised.

Verification results

Table 14.8 — Verification of the hospital delivery robot (version 2).

ID	Test	Limit	Result	Pass/fail
R1	All track tests at 30 kg gross	No faults	No faults	Pass
R2	Speed log against zone map, 200 km of field driving	≤ 1.2 / 0.6 m/s	1.19 / 0.61 m/s max (0.61 in a 2 s transient)	Pass with note
R3	1:12 ramp from standstill; 15 mm threshold; 20 mm lift gap, 50 runs each	No stall, no stop	No failures	Pass
R4	8 h mission run, pack capacity limited to 80 %	≥ 20 % reserve at end	50 % remaining (305 Wh used, 14 % above budget)	Pass
R5	4 m square × 5 each direction; 50 m straight	< 1 % of distance	0.45 % worst; 0.21 % on the straight	Pass
R6	Pose at 20 surveyed points, 10 visits each	±50 mm, ±2° (95 %)	38 mm, 1.4° (95 %); 44 mm at corridor end	Pass
R7	100 docking cycles	±10 mm, ±1°	4 mm, 0.6° worst	Pass
R8	Stopping with standing test piece at 0.3, 0.6, 0.9, 1.2 m/s; overhang table	No contact	Stopped 0.11/0.30/0.59/0.97 m after entry; no contact	Pass
R9	Approach 50 mm platform edge at 0.6 m/s, 4 headings	Stop before wheel crosses	Stopped 85 mm before edge worst	Pass
R10	4-week field pilot	≥ 99 % success; ≤ 1 intervention per 50 km	99.4 %; 0.7 per 50 km	Pass

The stopping distances measured in the R8 test (Figure 14.3) are 15–45 mm longer than the calculation, because the braking deceleration takes about 30 ms to build up; the field margin of 0.10 m absorbs it.

Failure analysis

Table 14.9 — FMEA of the delivery robot (field and track findings).

Failure mode	Effect	Cause	Detection	Mitigation
Stops 0.9 m short in corridor	Mission fails, robot turns into wall	Mean wheel radius 1.5 % smaller under load and wear; corridor gives no along-track fix	Large scan-match residual at door	Fiducials every 30 m; on-line radius estimation
Heading drifts after turns on wet floor	Lateral error, near-misses with walls	Wheel slip during turn-in-place	Gyro and wheel-difference heading disagree	Gyro heading in EKF; slip flag reduces speed
Glass door not seen by lidar	Bumper contact	Lidar passes through glass	Bumper event	Ultrasonic pair at front; glass marked as keep-out edge in map
Overbed table hit at 0.75 m height	Tote strikes table top	Single-plane lidar and scanner see only the column	Bumper on tote frame	Depth camera tilted 25° feeding costmap
Stuck at lift-door gap	Lift leaves without robot; mission delayed	50 mm casters cannot mount 15–20 mm gap edge	No-progress timer	100 mm casters on rocker; approach gap square-on
Robot pushed by a person while powered	Pose lost ("kidnapped")	Wheels backdriven, odometry counts but map match fails	Particle-filter likelihood collapse	Global relocalization on fiducial; ask for help if none in view
Computer brown-out when both motors accelerate up the ramp	Navigation reboot, robot stops	5 V rail droop from shared DC–DC and long ground return	Reset-cause log	Separate DC–DC for the computer, star ground (Chapter 15)
Cliff sensor false triggers on black floor strip	Robot refuses to cross	ToF return weak on matt black vinyl	Repeated stops at one location	Threshold on signal rate and distance; map-marked exemptions

What version 2 changed and why

Every change from the pilot traces to a number: the corridor failure to an along-track error budget, the lift-gap failure to a caster step-climbing force, the overbed-table contact to a sensor that sees only one plane, and the brown-out to a power-distribution flaw that Chapter 15 takes up in detail. None of it required a better algorithm; all of it required the error, force, field and energy budgets to be written down before the robot met the building.

What remains open

Open question	Where it is answered
How the wheel-speed loops, encoder interface, IMU fusion and power distribution are implemented on the motor-control board	Chapter 15
A complete line-following robot with sensor choice, control tuning and speed optimisation	Chapter 34
How the robot is identified, tested in the loop and field-tested as an integrated system	Chapter 17
Sensor principles of lidar, ToF, ultrasonic and IMUs	Chapter 6
Battery pack, BMS and state-of-charge estimation	Chapter 23
Safety functions, safety controllers and performance-level validation	Chapter 21
Probability, Bayesian updating and Gaussian processes behind the filters	*The Mathematics Universe*, Ch. 19 and Ch. 21

Next: Chapter 15 opens the motor-control board that this chapter treated as a block: motor drivers, encoder interfaces, sensor fusion, PID loops, timing, buses, power distribution and what the robot does when something fails.

Chapter 15 — Robot Electronics and Control

Part IV · Robotics and Mechatronics

On the third day of the hospital pilot of Chapter 14, the delivery robot left its dock fully charged and drove down the 10 m loading ramp. Halfway down, both wheels went limp and the robot accelerated until the safety scanner saw the wall at the bottom and dropped the brakes. At 100 % state of charge the battery management system had opened its charge MOSFET, as it should. Braking down the ramp, the drives pushed about 37 W of regenerated power onto the 29 V bus; the computer and sensors used 29 W of it, and the rest had nowhere to go but 940 µF of capacitance. The bus reached the integrated motor drivers' 34 V overvoltage lockout in about 20 ms, the drivers disabled their outputs, the wheels coasted, and the drivers' auto-retry re-enabled them every 100 ms into the same condition; the spring brakes stayed released because the robot was still powered. Every part met its datasheet; the design had no path for regenerated energy and treated an overvoltage as a nuisance to retry. This chapter designs version 2 of the robot's motor-control board, its reference system, from drivers to fault handling.

> **Which drivers, interfaces, loops and timing turn a navigation computer's velocity commands into wheel motion that is accurate, fast and safe?**
> **How are power distribution and fault recovery designed so that a full battery, a missing message or a stuck sensor ends in a controlled stop instead of a runaway?**

Concept	Engineering destination
Motor drivers	Voltage, current and protection ratings of H-bridges and three-phase stages for robot joints and wheels
Encoder interfaces	Position and speed from quadrature, Hall and absolute encoders at every speed
Sensor fusion	Gyro bias handling, slip detection, heading for odometry and balancing
Embedded controllers	MCU choice for multi-axis motor control with timers, ADCs and buses
PID implementation	Discrete controllers with limits, anti-windup, feed-forward and bumpless start
Feedback loops	Cascaded current, speed and position loops and how they are tuned
Timing and latency	Loop periods, jitter, the delay budget and where loops must not run
Communication buses	UART, SPI, I²C and CAN links inside a robot; framing, load and fault handling
Power distribution	Rails, fusing, brown-out immunity and regenerated energy
Fault recovery	Detection, safe states, retry policy and latching faults

Roadmap. Requirements (15.1) → drivers (15.2) → encoders (15.3) → fusion (15.4) → controller (15.5) → PID (15.6) → loops (15.7) → timing (15.8) → buses (15.9) → power (15.10) → faults (15.11) → verification (15.12).

15.1 Requirements and architecture

The motor-control board (MCB) sits between the navigation computer and the robot's two drive motors (24 V, 60 W BLDC gearmotors with 1:18 planetary gears, Hall sensors, 4096-count magnetic encoders and spring-applied brakes, chosen in Chapter 14 §14.10). Table 15.1 gives its requirements; the pilot failed R6 and R8.

Table 15.1 — Requirements of the motor-control board (version 2).

ID	Requirement	Target	Verification
R1	Wheel-speed control	Steady-state error < 1 % from 0.05 to 1.3 m/s; small-step overshoot < 5 %, 2 % settling < 50 ms	Step and ramp tests on the loaded robot
R2	Torque	10.5 A phase current per motor for 2 s; current-loop bandwidth ≥ 1 kHz	Locked-rotor test; current-loop frequency response
R3	Odometry and IMU stream	100 Hz, time-stamped to ±0.5 ms against the computer's clock	Logic analyser on sync pulse
R4	Command latency	Velocity command received → new speed reference in < 2 ms	GPIO timing on target
R5	Heading drift	Gyro-based heading < 1°/min while driving between stops	3 min mission log against fiducials
R6	Bus voltage	≤ 34 V under any regeneration, including with the battery charge path open	Ramp descent at 100 % SOC; bench regen test
R7	Brown-out immunity	Computer rail 12 V ±5 % during 25 A motor transients and battery at 20 V	Load-step test with oscilloscope
R8	Fault reaction	Over-current: PWM off < 5 µs; command loss: controlled stop started ≤ 200 ms; any latched fault: motors off, brakes on	Fault-injection test per fault
R9	ESD	±8 kV contact at the charge contacts and connectors without reset (IEC 61000-4-2)	ESD gun test
R10	Quiescent power	MCB with drivers enabled at standstill < 2.5 W	Bench measurement

Figure 15.1 shows the architecture. One STM32G474-class microcontroller runs both motors' current loops at 20 kHz, the wheel-speed loops and odometry at 1 kHz, the IMU fusion and the fault manager. Each motor has a 60 V gate driver with current-sense amplifiers and six 60 V MOSFETs. Power enters through a fuse and a reverse-polarity stage onto a clamped motor bus; separate DC–DC converters feed the computer and the sensors. The safety relay's STO line reaches the gate drivers directly, bypassing the MCU.

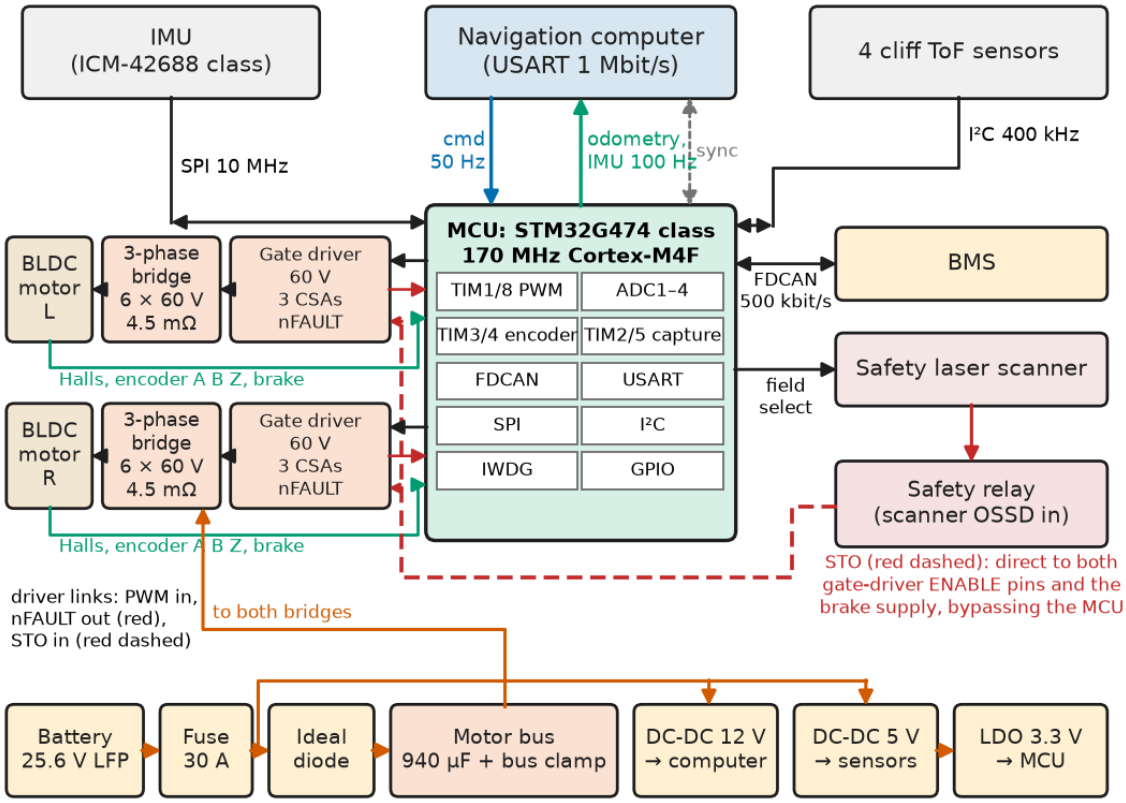


Figure 15.1 — Architecture of the motor-control board of the delivery robot.

15.2 Motor drivers

What the driver must survive

A motor driver is sized by the voltage it must block, the current it must carry and the faults it must survive. On a battery robot the voltage rating is set not by the battery but by the highest voltage the bus can reach, and on a robot that brakes, regeneration sets it.

> **Boundary.**
> The three-phase bridge, PWM, dead time and synchronised current sampling are in *Electronics and Computing*, Ch. 32 §32.1–32.4, with regenerative operation and the brake chopper in §32.6; field-oriented control of a PM motor is in *Electrical Energy and Machines*, Ch. 12 §12.6 and §12.8. This section chooses parts and ratings.

Table 15.2 — Driver options for one 24 V, 60 W BLDC wheel motor (2026 prices, 100 pcs).

Option	Voltage rating	Current (peak)	Current sensing	Protection	Cost	Verdict
Integrated 3-phase driver with FETs (DRV8316 class)	40 V abs. max; OV lockout ~34 V	8 A	Integrated	OCP, OTP, OV/UV, auto-retry	$3.90	Pilot choice; no headroom above a 29.2 V bus
Smart gate driver (DRV8323 class) + 6 × 60 V, 4.5 mΩ MOSFETs	60 V	> 40 A (FET-limited)	3 CSAs, external shunts	VDS sensing OCP, gate-drive faults, configurable latch or retry	$7.60	**Chosen**: 2 × voltage headroom, latching OCP
Trapezoidal Hall driver (sensor-commutated IC)	30–60 V	5–15 A	Average only	Basic	$2.50	Torque ripple at low speed; no FOC

Worked example — voltage and current ratings of the bridge. The 8S LFP pack reaches 29.2 V at the end of charge. With the bus clamp of Section 15.10 at 33 V and 3 V of switching overshoot from the commutation loop's inductance, the MOSFETs see up to 36 V; a 60 V rating gives a factor of 1.67, inside the usual 1.5–2 derating. The pilot's 40 V driver had 1.11, and its 34 V lockout sat 4.8 V above a full battery. On current, the motor's continuous 0.19 N·m at $K_t = 0.054$ N·m/A is 3.5 A (amplitude), and Chapter 14's threshold climb needs 10.5 A for two seconds. With 4.5 mΩ MOSFETs (6.8 mΩ hot) the bridge's conduction loss is $3 \times (3.5/\sqrt{2})^2 \times 0.0068 = 0.13$ W continuous and $3 \times (10.5/\sqrt{2})^2 \times 0.0068 = 1.1$ W at peak, plus about 0.2 W of switching loss at 20 kHz. Thermal design is trivial; the ratings are set by voltage and by fault current, which the gate driver's drain–source monitor catches within 4 µs (R8).

> **Status of the principle: Voltage derating of power semiconductors**
> *Empirical.* Practice for low-voltage drives is to rate MOSFETs at 1.5–2 times the highest steady bus voltage including regeneration, to cover switching overshoot, avalanche margin and transients from cables and connectors; the factor is not a law but a record of what fails in the field.

Current sensing and the gate-driver configuration

Each phase has a 5 mΩ low-side shunt read by the gate driver's amplifier at a gain of 20, biased at mid-rail, so ±16 A spans ±1.6 V and a 12-bit ADC, triggered by the PWM timer, resolves 8 mA. The gate driver is configured over SPI: 200 ns dead time, an OCP threshold of 0.5 V drain–source (about 74 A at 6.8 mΩ: a short, not an overload) and **latched** OCP; overload is the current loop's job. The configuration is read back after every write and every 100 ms, because ESD at the charge contacts once reset them without resetting the MCU.

15.3 Encoder interfaces

Quadrature decoding in hardware

The 4096-count encoders produce up to $3000/60 \times 4096 = 204{,}800$ counts/s at full motor speed, decoded by the MCU's timers in encoder mode with a programmable input filter; software only reads the counter.

Reliability was decided electrically: the encoder sits 0.6 m of cable away, beside phases switching 29 V at 20 kHz. Single-ended 5 V signals miscounted about once per 10^5 counts during hard acceleration on the first board, bending odometry by 0.1° per turn. Version 2 uses the encoder's RS-422 outputs, a receiver at the board edge, shielded twisted pairs and an input filter that rejects pulses shorter than 0.5 µs (the shortest legitimate edge interval is 4.9 µs).

> **Boundary.**
> Quadrature decoding as logic, and its implementation in an FPGA, is in *Electronics and Computing*, Ch. 19; encoder principles are in Chapter 6.

Speed from counts: the M/T method

Chapter 14 (§14.3) showed that counting edges in a 1 ms window resolves only 7.8 counts at the 0.05 m/s docking speed. The fix is to measure time as well as counts: a second, free-running 32-bit timer at 170 MHz latches its value on every encoder edge (the timer's XOR input combines A and B), and the speed estimate divides the counts by the time between the last edge of the previous window and the last edge of this one. Listing 15.1 shows the update; register names are STM32G4-style and illustrative.

```
 /* enc.c -- wheel encoder: timer encoder mode + edge time stamps, M/T speed (C11) */
#include <stdint.h>

#define CPR_MOTOR  4096.0f          /* counts per motor revolution after x4 */
#define TCLK_HZ    170.0e6f         /* time-stamp timer clock */
#define STILL_S    0.05f            /* no edge for 50 ms: standstill */

typedef struct {
    volatile uint32_t *cnt;         /* encoder-mode timer counter (16-bit) */
    volatile uint32_t *edge_ts;     /* capture register: time of latest edge */
    uint16_t last_cnt;
    uint32_t last_ts;
    int32_t  pos;                   /* extended position, counts */
    float    omega;                 /* motor speed, rad/s */
} enc_t;

/* Called every 1 ms by the speed-loop task; now_ts = time-stamp timer value now. */
void enc_update(enc_t *e, uint32_t now_ts)
{
    uint16_t c  = (uint16_t)*e->cnt;
    uint32_t ts = *e->edge_ts;
    int16_t  d  = (int16_t)(uint16_t)(c - e->last_cnt);  /* wrap-safe, |d| < 32768 */
    e->last_cnt = c;
    e->pos += d;
    if (d != 0) {
        float dt = (float)(uint32_t)(ts - e->last_ts) / TCLK_HZ;
        e->last_ts = ts;
        e->omega = (dt > 0.0f) ? 6.2831853f * (float)d / (CPR_MOTOR * dt) : e->omega;
    } else if ((float)(uint32_t)(now_ts - e->last_ts) / TCLK_HZ > STILL_S) {
        e->omega = 0.0f;
    }
}
```

The casts make both wraparounds safe: the 16-bit count difference is reinterpreted as signed, and the 32-bit time difference is unsigned, correct across the timer's 25 s wrap.

Worked example — speed resolution with time stamps. At 0.05 m/s the motor turns at 12.0 rad/s and produces 7823 edges/s. In a 1 ms window the M method sees 7 or 8 counts, a 13 % quantization. The M/T method divides the same counts by a time measured to 5.9 ns in about 1 ms, a resolution better than 10^{-5}; what remains is the encoder's edge-placement error (typically ±0.1 count interval), about ±1 % after a two-period average. Below about 0.6 mm/s (fewer than one edge per 10 ms) the estimate holds its last value until the 50 ms standstill rule sets it to zero.

> **Status of the principle: M/T speed estimation**
> *Derived.* Speed equals counted displacement over the time between the edges that bound it; count quantization leaves the time base, and what remains is edge-placement error and, at very low speed, the wait for the next edge, hence the standstill timeout.

The Hall sensors are still read: for commutation at power-up, and as a plausibility check — the encoder count between two Hall transitions must be $4096/(6 \times 4) = 170.7$ ±10 %, which catches a slipped magnet or a broken line within one electrical revolution (Section 15.11).

15.4 Sensor fusion

Heading from a gyro, kept honest by the wheels

Chapter 14 (§14.8) assigned heading to the gyro because wheel-difference heading under-reads in turns, where casters scrub and tyres slip. A MEMS gyro (ICM-42688 class, SPI, 1 kHz) has the opposite weakness: a turn-on bias of up to ±0.5 °/s, which integrates to 30° per minute. The fusion uses each where it is good: the wheels say when the robot is **stationary** (no encoder edge for 0.5 s), and then the gyro output is pure bias, which a first-order filter learns; while moving, heading is integrated from the bias-corrected gyro.

Listing 15.2 simulates a 3-minute mission: five 90° turns, two parking stops, a 0.30 °/s gyro bias with slow drift, 0.02 °/s rms gyro noise, and wheels that under-read turns by 6 %.

```
 import numpy as np
rng = np.random.default_rng(7)
dt, T = 0.01, 180.0                              # 100 Hz, 3 minutes
t = np.arange(0, T, dt)
# true yaw rate: stops, straights and 90 deg turns at 0.5 rad/s (3.14 s each)
r = np.zeros_like(t)
for t0, sgn in [(20, 1), (45, -1), (70, 1), (110, 1), (140, -1)]:
    r[(t >= t0) & (t < t0 + np.pi/2/0.5)] = 0.5*sgn
moving = ~(((t >= 0) & (t < 5)) | ((t >= 90) & (t < 95)))   # parked 0-5 s, 90-95 s
# wheel-derived yaw rate: 6 % under-read in turns (caster scrub and slip on wet vinyl)
wheel = 0.94*r + moving*rng.normal(0, 0.002, t.size)
# gyro: 0.30 deg/s turn-on bias, slow drift, white noise 0.02 deg/s rms
bias = np.radians(0.30) + np.cumsum(rng.normal(0, np.radians(0.0001), t.size))
gyro = r + bias + rng.normal(0, np.radians(0.02), t.size)

b_est, n_still, est = 0.0, 0, []
for k in range(t.size):                          # firmware loop, 100 Hz
    still = not moving[k] and abs(wheel[k]) < 1e-3
    n_still = n_still + 1 if still else 0
    if n_still > 50:                             # parked > 0.5 s: learn bias
        b_est += 0.02*(gyro[k] - b_est)          # first-order, 0.5 s time constant
    est.append(gyro[k] - b_est)
head = lambda w: np.degrees(np.cumsum(w)*dt)
truth = head(r)
for name, w in [("wheels only", wheel), ("raw gyro", gyro), ("gyro + standstill bias",
np.array(est))]:
    e = head(w) - truth
    print(f"{name:24s} heading error at 60 s {e[t.searchsorted(60)]:+6.2f} deg, "
          f"at 180 s {e[-1]:+6.2f} deg")
```

```
 wheels only              heading error at 60 s  -0.03 deg, at 180 s  -5.47 deg
raw gyro                 heading error at 60 s +17.83 deg, at 180 s +53.04 deg
gyro + standstill bias   heading error at 60 s  +0.45 deg, at 180 s  +0.04 deg
```

Worked example — reading the fusion result. The raw gyro's 0.30 °/s bias integrates to 18° in the first minute. Wheel heading is excellent on straights but loses 6 % of every turn, $0.06 \times 90^\circ = 5.4^\circ$ per net turn; the run has one net left turn, hence −5.5°. Learning the bias in the first 5 s park, and again at 90 s, keeps the error inside ±0.5° for 3 minutes (Figure 15.2), about 0.15 °/min against R5's 1 °/min; four other noise seeds give final errors between −0.41° and +0.46°. The simulation omits the gyro's scale-factor error (±0.5 % for this class, calibrated once on a turntable) and temperature-driven bias drift between stops, which is why the mission planner inserts a 1 s standstill at least every 3 minutes.

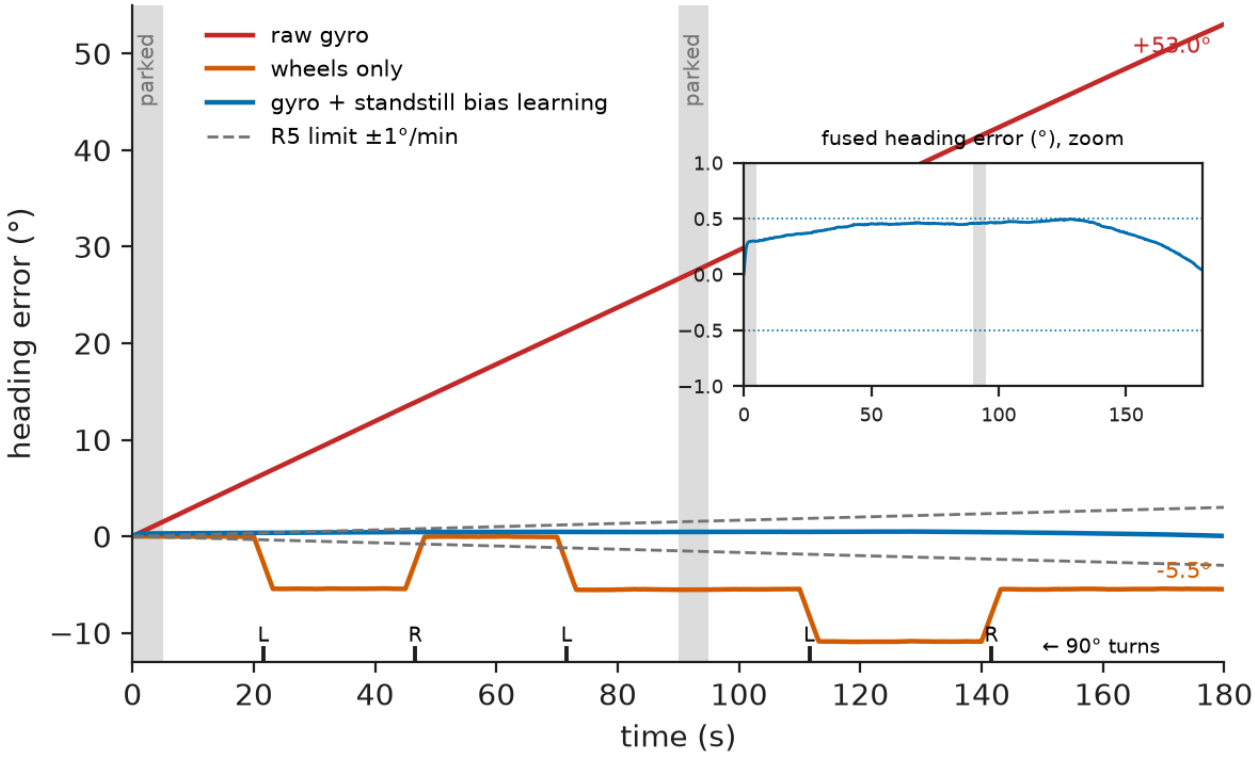


Figure 15.2 — Heading error over a 3-minute mission for wheel-only heading, raw gyro and gyro with standstill bias learning.

> **Status of the principle: Complementary use of gyro and wheel odometry**
> *Approximate.* Assumes the wheels report standstill reliably and that gyro bias changes slowly compared with the interval between stops; a robot that never stops, or a gyro whose bias steps with temperature, needs a full Kalman filter with a bias state and absolute heading fixes, as Chapter 14 adds with the lidar and fiducials.

Slip detection and tilt

The same signals detect slip: if wheel and gyro yaw rates differ by more than 0.1 rad/s for 100 ms, the MCB flags it and the computer slows down and inflates the odometry covariance. The accelerometer gives pitch to ±0.5° at standstill, used on the ramp as a torque feed-forward (Section 15.6).

15.5 Embedded controllers

Choosing the microcontroller

The processor must run two current loops at 20 kHz, generate two sets of three-phase PWM with dead time, sample six currents synchronously, decode two encoders with time stamps and serve four buses, with a fault path that does not depend on software. Selection as a method is Chapter 10's (§10.2) and *Electronics and Computing*, Ch. 25 §25.3; Table 15.3 applies it.

Table 15.3 — Controller options for a two-motor robot drive board (2026 prices, 1k pcs).

Part class	Core	Motor-control timers	ADC	Encoder support	Buses	Price	Verdict
STM32G474	Cortex-M4F 170 MHz, CORDIC, FMAC	2 advanced (TIM1, TIM8) + HRTIM, break inputs	5 × 12-bit, 4 Msps, hardware oversampling	Several encoder-mode timers, 32-bit capture	3 FDCAN, USART, SPI, I²C	$6.80	**Chosen**
dsPIC33CK	DSP core 100 MIPS	Motor PWM with dead time, fault inputs	3 × 12-bit, 3.5 Msps	QEI	CAN FD, UART, SPI	$3.50	Capable; smaller ecosystem in the team
RP2040	Dual Cortex-M0+ 133 MHz, no FPU	General PWM, no hardware break input	4-channel 500 ksps	PIO state machines	No CAN	$0.80	No hardware fault shutdown; too slow for two FOC loops in floating point

The decisive features are not clock speed but the **break inputs** (hardware PWM shutdown within nanoseconds, wired to the gate-driver fault outputs and a bus-voltage comparator), PWM-triggered ADC sampling and CAN FD.

CPU load budget

Table 15.4 — CPU budget of the MCB at 170 MHz.

Task	Rate	Execution time	Load
FOC current loop, motor L (ISR)	20 kHz	4.2 µs	8.4 %
FOC current loop, motor R (ISR)	20 kHz	4.2 µs	8.4 %
Speed loops, encoder update, odometry	1 kHz	18 µs	1.8 %
IMU read (DMA complete), fusion	1 kHz	9 µs	0.9 %
UART framing, command parse, telemetry	100 Hz	120 µs	1.2 %
Fault manager, gate-driver register check	1 kHz / 10 Hz	6 µs	0.6 %
Cliff sensors (I²C, DMA), BMS CAN	50 Hz	150 µs	0.8 %
Total			**22.1 %**
Design limit			50 %
Margin			**2.3 ×**

Execution times are worst cases over a 10-minute run, measured by toggling a GPIO at task entry and exit; the FOC ISR uses the CORDIC unit and runs from core-coupled RAM so flash wait states add no jitter.

15.6 PID implementation

What a usable controller contains

A usable controller needs output limits, anti-windup, a set-point weight, feed-forward and a bumpless start as well as its gains. The structure and its discretisation are owned by *Circuits, Signals and Communications*, Ch. 35 §35.2 and §35.8; Listing 15.3 is the implementation used for every loop on the MCB. The speed loop needs no derivative action.

```
 /* pi.c -- discrete PI with set-point weight, conditional integration,
    output limits, feed-forward and bumpless start (C11, float on Cortex-M4F) */
typedef struct {
    float kp, ki, ts;        /* gains (A per rad/s, A per rad) and period (s) */
    float b;                 /* set-point weight on the P term, 0..1 */
    float u_min, u_max;      /* actuator limits, A */
    float integ;             /* integrator state, A */
} pi_t;

float pi_step(pi_t *c, float ref, float meas, float ff)
{
    float e = ref - meas;
    float u = c->kp * (c->b * ref - meas) + c->integ + ff;
    if (u > c->u_max) {
        u = c->u_max;
        if (e < 0.0f) c->integ += c->ki * e * c->ts;   /* only integrate back */
    } else if (u < c->u_min) {
        u = c->u_min;
        if (e > 0.0f) c->integ += c->ki * e * c->ts;
    } else {
        c->integ += c->ki * e * c->ts;
    }
    if (c->integ > c->u_max) c->integ = c->u_max;      /* bound the state too */
    if (c->integ < c->u_min) c->integ = c->u_min;
    return u;
}

/* Bumpless start: choose the integrator so the first output equals u_now. */
void pi_start(pi_t *c, float u_now, float ref, float meas, float ff)
{
    c->integ = u_now - c->kp * (c->b * ref - meas) - ff;
}
```

Three choices in the listing come from failures. **Conditional integration** removed a 0.3 s overshoot after every threshold climb, where the pilot's integrator had wound up against the current limit. **Bounding the integrator state** keeps it finite if the measurement fails. **Bumpless start** preloads the integrator with the holding current when the brakes release on a ramp, so the robot does not drop back.

> **Status of the principle: Anti-windup by conditional integration**
> *Empirical.* One of several practical anti-windup schemes (clamping, back-calculation, conditional integration); all prevent the integrator from accumulating error the saturated actuator cannot act on, and they differ in recovery speed and tuning effort rather than in principle.

Worked example — the ramp feed-forward. On the 1:12 ramp, gravity pulls each wheel with $30 \times 9.81 \times \sin 4.76^\circ \times 0.075/2 = 0.92$ N·m. Through the 1:18 gear (efficiency 0.85) and $K_t = 0.054$ N·m/A that is $0.92/(18 \times 0.85 \times 0.054) = 1.11$ A per motor. The MCB computes it from the filtered pitch angle and adds it as `ff`; with it, the robot stopping on the ramp rolls back 2 mm while the brake engages, against 31 mm without it.

15.7 Feedback loops

The cascade

The drive has three nested loops, each well over five times faster than the one around it:

1. **Current (torque) loop** — FOC in the rotor frame, 20 kHz sampling, PI controllers on the d and q currents, closed-loop bandwidth about 1 kHz.
2. **Wheel-speed loop** — PI on motor speed from the M/T estimate, 1 kHz, crossover 20 Hz, output is the q-axis current reference, limited to ±10.5 A.
3. **Motion commands** — the navigation computer's linear and angular velocity at 50 Hz, converted to wheel speeds (Chapter 14 §14.2), rate-limited to 0.5 m/s² and 1.0 rad/s².

The robot's pose is closed by the computer's path tracker, not on the MCB. Tuning follows standard drive practice: the current loop's PI zero cancels the winding's electrical pole ($K_p = L\omega_c$, $K_i = R\omega_c$), and the speed loop is designed on the total inertia with the current loop as a small lag (*Electrical Energy and Machines*, Ch. 11 §11.5).

> **Status of the principle: Cascade separation of loop bandwidths**
> *Empirical.* An inner loop five to ten times faster than the outer one adds only a few degrees of phase at the outer crossover, so each loop can be tuned as if the inner one were ideal; the rule fails when delays or resonances, not the inner loop, set the phase budget.

The speed loop on the loaded robot

The inertia the speed loop sees is dominated by the robot, not the motor. Each motor moves half the 30 kg gross mass, which referred through the wheel radius and the gear is $15 \times 0.075^2/18^2 = 2.60 \times 10^{-4}$ kg·m², twenty-two times the rotor's 1.2×10^{-5} kg·m²; the total is 2.72×10^{-4} kg·m². For a 20 Hz crossover ($\omega_c = 125.7$ rad/s) with the PI zero at $\omega_c/4$, $K_p = J\omega_c/K_t = 0.633$ A per rad/s and $K_i = K_p\omega_c/4 = 19.9$ A/rad. A set-point weight of 0.7 on the proportional term removes most of the PI zero's step overshoot.

The loop is tuned loaded: unloaded (20 kg) the inertia falls to 68 % and the crossover rises to 29 Hz, which the phase budget tolerates, whereas a loop tuned unloaded would fall to 14 Hz when loaded.

15.8 Timing and latency

Why the pilot's loop rang

The pilot robot closed its wheel-speed loops on the navigation computer over a USB serial link, with a measured loop delay of 4–12 ms depending on the computer's load. On the ramp, wheel speeds oscillated at about 6 Hz. A delay T_d costs $\omega_c T_d$ radians of phase at crossover, and the phase budget at 20 Hz is small.

> **Boundary.**
> Latency, jitter and their effect on control loops are analysed in *Electronics and Computing*, Ch. 27 §27.10; the phase of a time delay in the Nyquist and Bode picture is in *Circuits, Signals and Communications*, Ch. 34 §34.8.

Listing 15.4 simulates the loaded speed loop with the PI of Listing 15.3, the closed current loop as a 160 µs lag, the current limit and a loop delay of 1, 3, 6 and 10 ms, and prints the phase margin estimated at crossover.

```
 import numpy as np
# Wheel-speed loop of one drive, referred to the motor shaft
Kt, J, B = 0.054, 2.72e-4, 2.0e-4        # N m/A, kg m2 (robot share + rotor), N m s/rad
tau_i, Ts = 160e-6, 1e-3                 # closed current loop (~1 kHz), speed-loop period
wc = 2*np.pi*20                          # target crossover, rad/s
Kp, Ki = J*wc/Kt, J*wc/Kt*wc/4           # PI with its zero at wc/4
b_sp, I_max = 0.7, 10.5                  # set-point weight on P, current limit (A)

def step(delay_ms, w_ref=10.0, t_end=0.3, sub=20):
    h = Ts/sub; buf = [0.0]*delay_ms
    w = i = integ = 0.0; out = []
    for k in range(int(t_end/Ts)):
        meas = buf.pop(0); buf.append(w)            # sensing + transport delay
        u = Kp*(b_sp*w_ref - meas) + integ
        if abs(u) < I_max:                           # conditional integration
            integ += Ki*(w_ref - meas)*Ts
        i_ref = np.clip(u, -I_max, I_max)
        for _ in range(sub):                         # plant between samples
            i += h*(i_ref - i)/tau_i
            w += h*(Kt*i - B*w)/J
        out.append(w)
    y = np.array(out)
    late = np.nonzero(np.abs(y - w_ref) > 0.02*w_ref)[0]
    return 100*(y.max() - w_ref)/w_ref, (late[-1] + 1)*Ts*1e3

print(f"Kp = {Kp:.3f} A/(rad/s), Ki = {Ki:.1f} A/rad")
for d in (1, 3, 6, 10):
    lag = np.degrees(wc*(d*1e-3 + Ts/2 + tau_i))     # delay + hold + current loop
    pm = np.degrees(np.arctan(4)) - lag
    ov, ts = step(d)
    print(f"delay {d:2d} ms: phase margin {pm:5.1f} deg, overshoot {ov:5.1f} %, "
          f"2 % settling {ts:4.0f} ms")
```

```
 Kp = 0.633 A/(rad/s), Ki = 19.9 A/rad
delay  1 ms: phase margin  64.0 deg, overshoot   0.9 %, 2 % settling   28 ms
delay  3 ms: phase margin  49.6 deg, overshoot   0.5 %, 2 % settling   17 ms
delay  6 ms: phase margin  28.0 deg, overshoot  30.2 %, 2 % settling   62 ms
delay 10 ms: phase margin  -0.8 deg, overshoot 129.7 %, 2 % settling  300 ms
```

Worked example — the delay budget of the speed loop. With 1 ms of delay (the MCB's own sample-and-compute delay) the loop has 64° of phase margin, 0.9 % overshoot and 28 ms settling (Figure 15.3), meeting R1; at 3 ms it is still well behaved. At 6 ms, mid-range for the pilot, the margin falls to 28° and a small step overshoots by 30 %; at 10 ms the loop oscillates. Each millisecond costs 125.7×0.001 rad, 7.2°, so a 45° minimum margin leaves about 3.6 ms for delay in total, which the pilot's 4–12 ms could never meet. A loop belongs on the processor whose *worst-case* delay fits its phase budget; a general-purpose computer on a USB link has a worst case of tens of milliseconds even when its average is 2 ms.

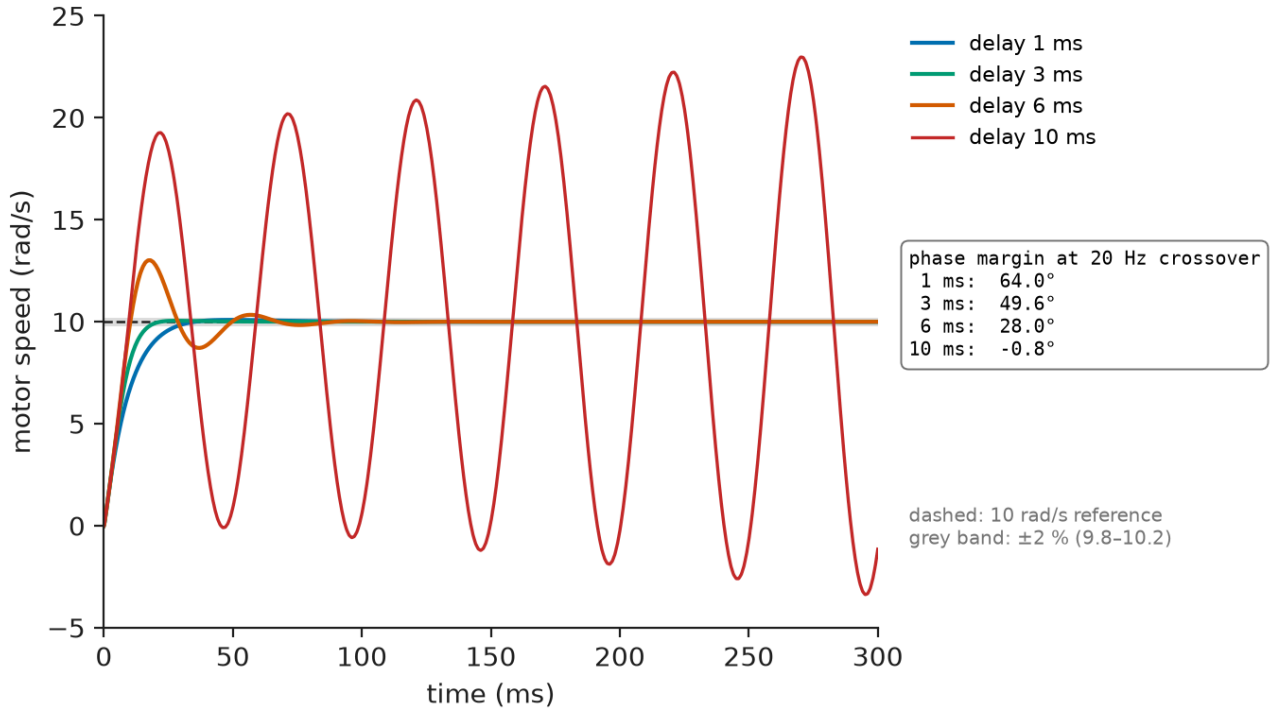


Figure 15.3 — Wheel-speed step responses of the loaded robot for loop delays of 1, 3, 6 and 10 ms.

> **Status of the principle: Phase cost of loop delay**
> *Derived.* A pure delay T_d has unit gain and phase $-\omega T_d$, so at crossover it subtracts $\omega_c T_d$ from the phase margin; the zero-order hold of a sampled loop adds about half a sample period, so loop periods, transport delays and computation all count against the same budget.

The timing of one control period

Figure 15.4 shows how the MCB orders its work. The PWM timer is the master clock: its centre triggers the current samples, and the FOC ISR writes the next duty cycles within the same period. Every twentieth period the speed-loop task reads the encoder counters and capture registers (latched by hardware at the timer update, so position is sampled at a known instant), updates speeds and odometry, and writes current references for the next FOC ISR. IMU samples arrive by DMA on the IMU's data-ready pin and are time-stamped by the same 32-bit timer.

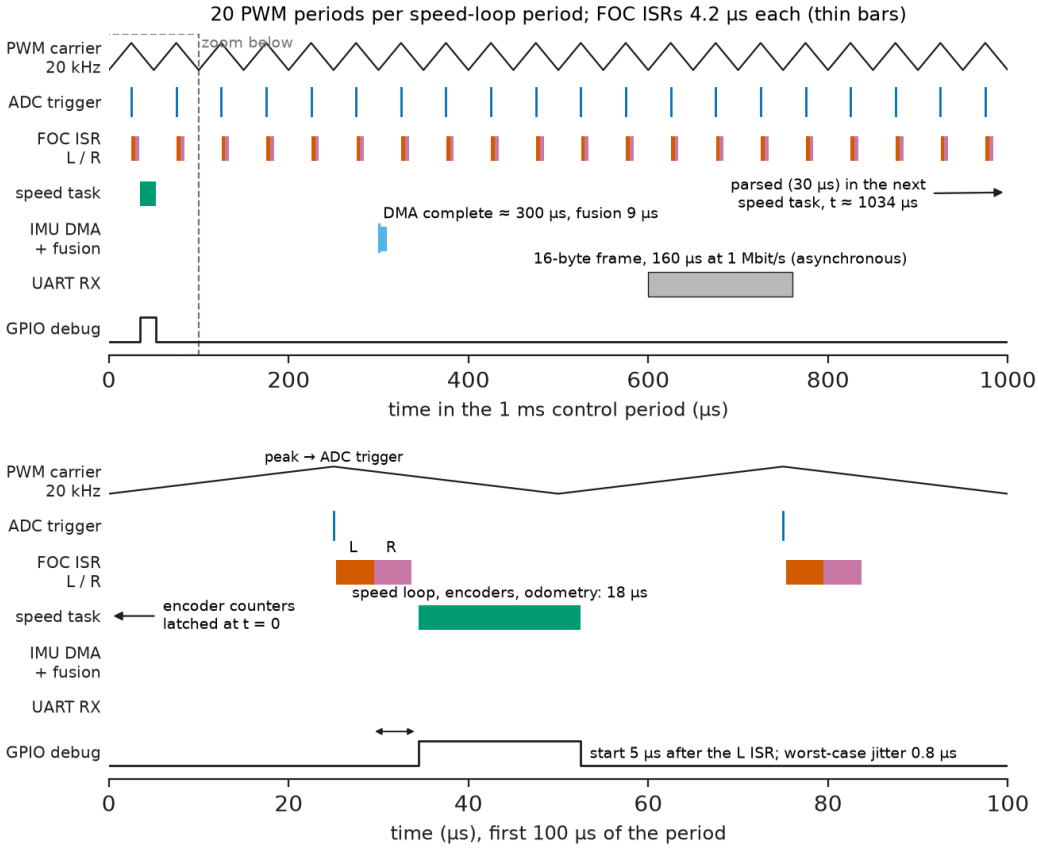


Figure 15.4 — One 1 ms control period of the motor-control board: PWM, ADC triggers, ISRs and tasks.

The command and telemetry latency

Table 15.5 — Latency budget from navigation command to wheel torque.

Stage	Worst case	Basis
Command frame on UART (16 bytes, 1 Mbit/s)	0.16 ms	160 bits including framing
Wait for next speed-loop task	1.0 ms	1 kHz task
Parse, convert to wheel speeds, rate limit	0.03 ms	Measured
Speed loop to current reference	0.02 ms	Measured
Next FOC ISR applies torque	0.05 ms	One PWM period
Total	**1.26 ms**	
Requirement R4 (to new speed reference)	2.0 ms	
Margin	**1.6 ×**	

Telemetry carries the MCU time stamp of the encoder latch; a sync pulse every second on a dedicated GPIO, time-stamped on both sides, gives the computer the clock offset for R3 without depending on UART buffering.

15.9 Communication buses

One bus per job

Table 15.6 — Buses on the motor-control board.

Link	Bus and rate	Traffic	Load	Fault handling
Navigation computer	UART 1 Mbit/s, COBS framing, CRC-16	Down: 16 B at 50 Hz. Up: odometry 48 B at 100 Hz, IMU 32 B at 100 Hz, status 32 B at 10 Hz	8.3 % up, 0.8 % down	CRC reject, sequence numbers, 200 ms command timeout
IMU	SPI 10 MHz, DMA	14 B at 1 kHz	1.1 %	WHO_AM_I check, data-ready timeout 5 ms
Cliff sensors (4)	I²C 400 kHz	4 × 6 B at 50 Hz	3 %	Bus-clear on stuck SDA, per-sensor timeout
BMS	CAN 500 kbit/s	4 frames at 10 Hz in, 1 out	1.1 %	Heartbeat timeout 300 ms → degrade to voltage-only limits
Gate drivers	SPI 1 MHz	Configuration at start, read-back at 10 Hz	—	Mismatch → reconfigure, latch after 3
Debug	USB, SWD	On demand	—	Not used in operation

> **Boundary.**
> The physical and link layers of UART, SPI, I²C and CAN, I²C bus-hang recovery and CAN bit timing are in *Electronics and Computing*, Ch. 26 §26.2–26.5; this section is about the choice and the robustness rules.

Worked example — UART load and framing. Upstream traffic is $48 \times 100 + 32 \times 100 + 32 \times 10 = 8320$ bytes/s; at 1 Mbit/s (100,000 bytes/s with 10 bits per byte) the load is 8.3 %, and COBS framing adds at most one byte in 254. The pilot's 115.2 kbit/s link would have run at 72 % load, where a burst of status messages after a fault delayed odometry by up to 15 ms. CRC-16 is there because a corrupted but plausible command (1.3 m/s instead of 0.3) is worse than a lost one; the 200 ms timeout is three missed commands plus margin.

Two rules came from bring-up. The four VL53L1X-class cliff sensors share one fixed I²C address, so each has its shutdown pin on a GPIO and the MCB brings them up one at a time with new addresses; one reset by ESD returns at the default address and is re-addressed. And the IMU's SPI clock is held at 10 MHz, below its 24 MHz maximum, because its 80 mm ribbon rang at higher rates.

15.10 Power distribution

The power tree

Figure 15.5 shows the version-2 power distribution, built on three rules: motors and computer share neither a converter nor a ground return, regenerated energy always has somewhere to go, and every rail has a defined behaviour at low battery.

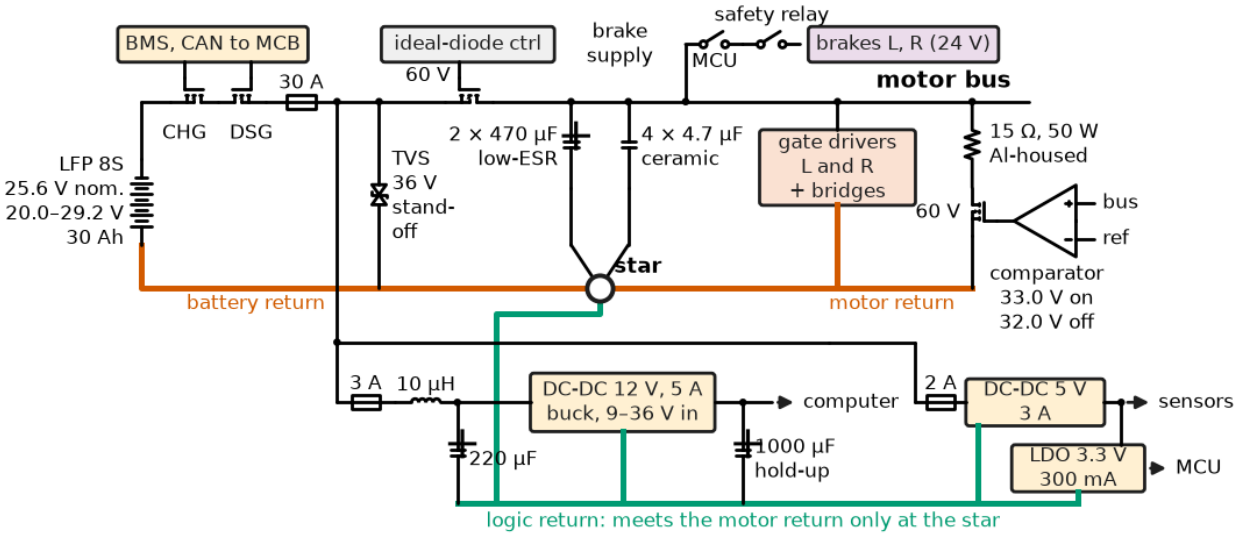


Figure 15.5 — Power distribution of the delivery robot, version 2.

Regeneration and the bus clamp

Worked example — where the pilot's 37 W went, and the clamp that takes it. Descending the 1:12 ramp at 1.0 m/s, gravity delivers $30 \times 9.81 \times \sin 4.76^\circ \times 1.0 = 24.4$ W; rolling resistance takes 4.4 W, and gear and motor in generating mode pass about 72 % (0.85×0.85) of the remaining 20.0 W, so 14.5 W reach the bus. Braking from 1.2 m/s at 1.0 m/s² adds a peak of $30 \times 1.0 \times 1.2 = 36$ W; less 5.3 W of rolling resistance at 1.2 m/s, 72 % of it, 22.2 W, reaches the bus: together about 37 W. With the charge FET open, the only sinks are the hotel loads on the same bus (29 W with conversion losses, Table 14.7) and the bus capacitance: raising 940 µF from 29.2 V to 34 V stores $\frac{1}{2} \times 940 \times 10^{-6} \times (34^2 - 29.2^2) = 0.14$ J, which the remaining 8 W supplies in about 18 ms — the pilot's trip. The version-2 clamp switches a 15 Ω resistor across the bus above 33 V, absorbing $33^2/15 = 72.6$ W, twice the worst case even with the hotel loads shed. A full 10 m descent at 100 % SOC dissipates at most $(245 - 44) \times 0.72 = 145$ J, and the 50 W resistor on the chassis plate rises about 12 K. When the charge path is open the MCB also limits downhill speed to 0.6 m/s.

> **Status of the principle: Energy balance of a regenerating drive bus**
> *Fundamental.* Energy returned by the drives must be absorbed by the battery, stored in the bus capacitance or dissipated; when the battery cannot accept charge, the bus voltage rises at a rate set by power over capacitance, so a clamp or brake chopper sized for the peak regenerated power is mandatory, not optional.

Normally the battery absorbs regeneration; the clamp covers charge path open (full, or below 0 °C), a BMS fault, or a blown fuse while moving, when clamp, hold-up and brake logic must stop the robot with no

battery at all (Table 15.8, R6).

Brown-out immunity

Chapter 14's FMEA recorded a computer reset when both motors accelerated up the ramp: the pilot's computer rail and the motors shared a DC–DC input filter and a long ground return, and a 25 A step pulled the converter below its undervoltage threshold for 2 ms. Version 2 gives the computer its own fused branch, a π filter, a 9–36 V-input converter, 1000 µF of hold-up and a star ground at the bus capacitors. With a 25 A step at a 20 V battery the 12 V rail dipped to 11.62 V for 0.4 ms, inside R7's 11.4 V. At low charge the MCB limits motor current to 6 A below 22.4 V and sends the robot home, and below 21.0 V stops and brakes it so the computer can shut down before the BMS disconnects at 20.0 V.

15.11 Fault recovery

Detection, reaction and the decision to retry

For each fault the design answers: how it is detected, how fast, what the robot does, and whether it may recover on its own. The pilot answered the last question once for all faults (auto-retry); version 2 classifies faults by what a retry could cause.

Table 15.7 — Fault handling on the motor-control board.

Fault	Detection	Time to detect	Reaction	Recovery
Phase short / shoot-through	Gate driver VDS monitor → break input	< 4 µs	PWM off in hardware, brakes on	Latched; needs power cycle and service
Bus over-voltage > 34 V	Comparator → break input	< 2 µs	PWM off, clamp stays on, brakes on	Latched until bus < 30 V and operator reset
Bus over-voltage > 31.5 V	ADC, 1 kHz	1 ms	Limit regenerative current, slow down	Automatic
Command timeout	No valid frame for 200 ms	200 ms	Controlled stop at 1.0 m/s², then brakes	Automatic when commands resume and robot is stopped
Encoder implausible	Counts per Hall interval outside ±10 %	One electrical revolution	Controlled stop on the other wheel's speed and gyro	Latched for that boot; computer informed
Gate-driver register mismatch	SPI read-back, 10 Hz	100 ms	Reconfigure	Latched after 3 in 60 s
Watchdog	Independent watchdog, 50 ms window	50 ms	MCU reset: outputs default off, brakes on	Reboot; reset cause reported
STO from safety relay	Hardware line to gate drivers and brakes	< 10 ms	Torque off and brakes on, independent of MCU	Safety relay reset by operator

The rule behind the Recovery column: a fault may clear itself only if a retry cannot make things worse and its cause is observably gone. An over-voltage retry caused loss of braking on a slope, so it latches; a command timeout is benign once the robot has stopped.

> **Status of the principle: Fail to a defined safe state, retry only what is benign**
> *Empirical.* Practice codified in functional-safety standards (IEC 61508, ISO 13849) and in drive design: every detected fault drives the system to a state whose safety does not depend on the failed function, and automatic recovery is allowed only for faults whose recurrence is harmless; the classification is a design decision recorded in the FMEA, not a property of the fault.

The fault manager

Listing 15.5 is the core of the MCB's fault manager, run every 1 ms after the speed loop. Hardware paths (break input, STO) act without it; the manager decides the state, the brake output and whether a fault may clear.

```
 /* fault.c -- MCB fault manager, called at 1 kHz (C11) */
#include <stdbool.h>
#include <stdint.h>

typedef enum { ST_READY, ST_RUN, ST_STOPPING, ST_LATCHED } mcb_state_t;
enum { F_COMMS = 1u << 0, F_OVERVOLT = 1u << 1, F_SHORT = 1u << 2,
       F_ENCODER = 1u << 3, F_OVERTEMP = 1u << 4, F_DRVCFG = 1u << 5 };
#define LATCHING (F_OVERVOLT | F_SHORT | F_ENCODER | F_DRVCFG)

typedef struct { mcb_state_t st; uint32_t active, latched; } fault_mgr_t;

extern void pwm_enable(bool on);           /* board support, illustrative */
extern void brakes_release(bool release);
extern bool robot_stopped(void);           /* both wheel speeds below 0.01 m/s */

void fault_step(fault_mgr_t *m, uint32_t detected, bool cmd_valid, bool operator_reset)
{
    m->active = detected;
    m->latched |= detected & LATCHING;
    if (operator_reset && (detected & LATCHING) == 0u) m->latched = 0u;

    switch (m->st) {
    case ST_READY:
    case ST_RUN:
        if (m->latched != 0u)            { m->st = ST_LATCHED; }
        else if (m->active != 0u)        { m->st = ST_STOPPING; }
        else                             { m->st = cmd_valid ? ST_RUN : ST_READY; }
        break;
    case ST_STOPPING:                      /* speed loop ramps reference to 0 */
        if (m->latched != 0u)            { m->st = ST_LATCHED; }
        else if (robot_stopped())        { m->st = (m->active == 0u) ? ST_READY : ST_STOPPING; }
        break;
    case ST_LATCHED:
        if (m->latched == 0u && robot_stopped()) { m->st = ST_READY; }
        break;
    }
    bool drive = (m->st == ST_RUN) || (m->st == ST_STOPPING && !robot_stopped());
    pwm_enable(drive);                     /* safe default: off, brakes on */
    brakes_release(drive);
}
```

Safe outputs are the default: PWM is enabled and brakes released only while running or still decelerating under control, so a reset or a corrupted state variable leaves the motors off and the brakes on. A command timeout enters `ST_STOPPING`, where the speed loop brings the robot to rest; a latching fault goes to `ST_LATCHED`, where hardware has already removed PWM. Host-side unit tests with fake board-support functions drive the function through every fault sequence in the FMEA, the practice of Chapter 10 (§10.7).

15.12 Verification and failure analysis

Verification results

Table 15.8 — Verification of the motor-control board (version 2).

ID	Test	Limit	Result	Pass/fail
R1	0.05 m/s and 1.3 m/s steady runs; 10 rad/s motor-speed steps, loaded	Error < 1 %; overshoot < 5 %; settling < 50 ms	0.4 % at 0.05 m/s; 1.2 % overshoot; 31 ms	Pass
R2	Locked rotor 10.5 A for 2 s; current-loop sweep	2 s without trip; ≥ 1 kHz	Held; 1.15 kHz bandwidth	Pass
R3	Sync pulse vs telemetry time stamps, 1 h	±0.5 ms	±0.21 ms	Pass
R4	GPIO from UART frame end to new reference	< 2 ms	1.21 ms worst over 10^5 frames	Pass
R5	3 min missions, heading against fiducials	< 1°/min	0.22°/min worst of 20 runs	Pass
R6	Ramp descent at 100 % SOC (charge FET open); bench test with battery fuse pulled at 1.2 m/s	≤ 34 V	33.4 V peak; controlled stop with no battery	Pass
R7	25 A load step at 20 V battery	12 V ±5 %	11.62 V minimum	Pass
R8	Fault injection: short, OV, comms loss, encoder unplug	OCP < 5 µs; stop ≤ 200 ms; brakes on when latched	3.2 µs; stop began at 201–203 ms; brakes on in all latched cases	Pass with note
R9	ESD ±8 kV contact at charge contacts	No reset	No reset; gate-driver register restored by read-back once	Pass
R10	Standstill power, drivers enabled	< 2.5 W	1.9 W	Pass

The R8 note: the timeout is checked in the 1 kHz task, so the stop began 1–3 ms after 200 ms; the requirement was rewritten as ≤ 205 ms, immaterial at Chapter 14's stopping distances. Figure 15.6 compares the bus voltage of the pilot board and version 2 in the R6 ramp test.

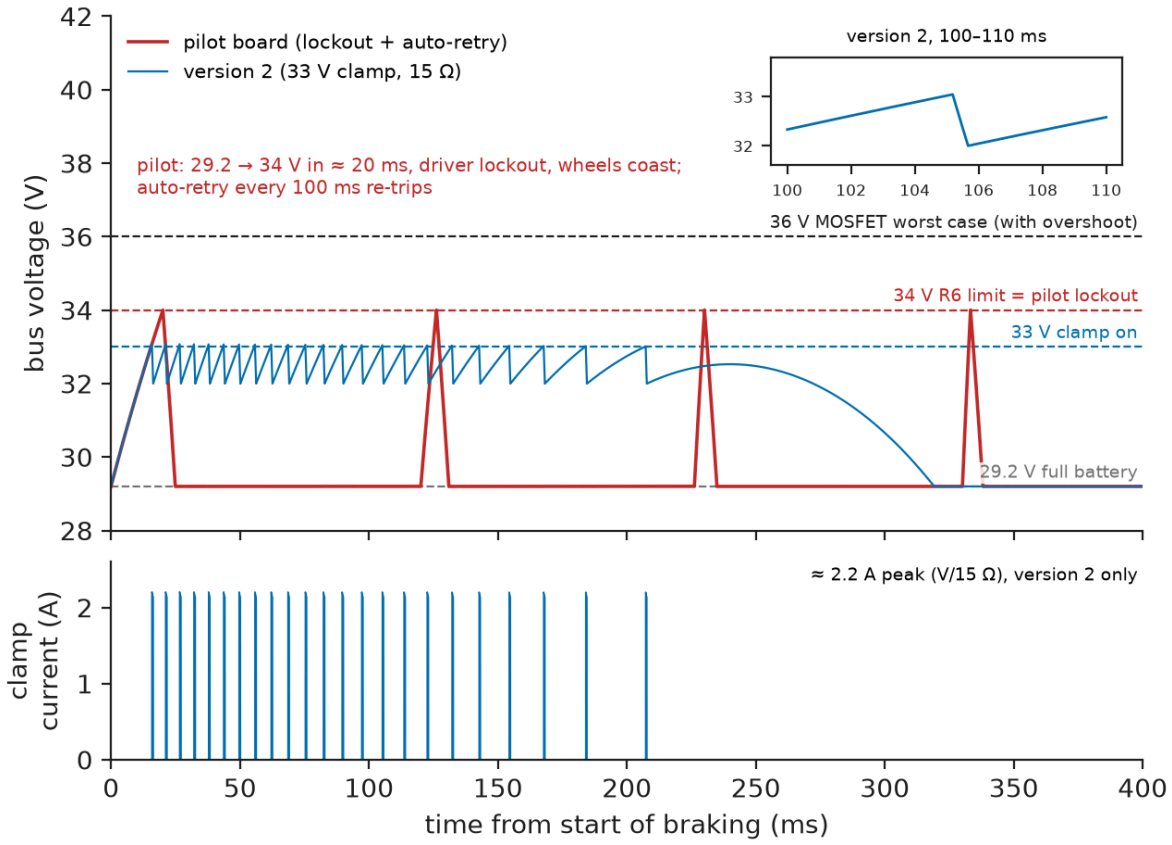


Figure 15.6 — Motor-bus voltage during braking on the ramp at 100 percent state of charge: pilot board against version 2.

Failure analysis

Table 15.9 — FMEA of the motor-control board.

Failure mode	Effect	Cause	Detection	Mitigation
Bus rises above driver lockout during regeneration	Wheels coast on a ramp; robot rolls away (pilot)	No energy path when BMS charge FET open; auto-retry of OV	Bus voltage log	Bus clamp 33 V / 72.6 W; latched OV; downhill speed limit when charge path open
Wheel-speed oscillation on ramp	Jerky motion, tote spills	Speed loop closed over USB on the computer, 4–12 ms delay (pilot)	Speed ripple at 6 Hz	Speed loops on MCB at 1 kHz; delay budget 3.6 ms
Computer resets when motors accelerate	Navigation lost, robot stops	Shared converter and ground return (pilot)	Reset-cause log	Separate fused branch, π filter, hold-up, star ground
Encoder miscounts during acceleration	Odometry bends 0.1° per turn	Single-ended encoder lines beside motor phases	Hall plausibility check	RS-422 encoder lines, shielded pairs, input filter
Integrator wind-up after threshold climb	0.3 s overshoot, bump into door frame	No anti-windup in pilot PI	Overshoot in logs	Conditional integration, bounded state (Listing 15.3)
Gate-driver registers reset by ESD	Dead time and OCP revert to defaults	ESD at charge contacts couples into driver supply	10 Hz register read-back	TVS at contacts, read-back and reconfigure, latch after 3

What the second board changed

Every change from the pilot board is one of three kinds: a **rating** reconsidered with the real worst case (60 V devices, because regeneration sets the bus voltage), a **location** changed to fit a budget (speed loops moved to the MCB to fit a 3.6 ms delay budget), or a **policy** made explicit (which faults latch and which retry). None needed new theory; each needed a number that had not been written down.

What remains open

Open question	Where it is answered
How these drives, loops and buses come together in a multirotor with ESCs and an attitude loop	Chapter 16
System identification of the drive and robot, hardware-in-the-loop testing of the MCB and field testing	Chapter 17
A complete closed-loop motor drive designed, built and characterised as a project	Chapter 37
A complete line-following robot using these drivers, encoders and PID loops	Chapter 34
Battery packs, BMS charge paths and state estimation	Chapter 23
Safety relays, STO validation and performance levels	Chapter 21
Field-oriented control and PM-motor current control in depth	*Electrical Energy and Machines*, Ch. 12

Next: Chapter 16 takes the same drivers, sensors and loops into the air, where the plant is unstable, the loops are faster and a fault cannot end with the brakes on.

Chapter 16 — Drones and Aerial Robotics

Part IV · Robotics and Mechatronics

A bridge-inspection company commissions a 2.3 kg quadcopter that must hover beside a pier for 20 minutes with a stabilised camera. On the bench every subsystem passes: the motors make their thrust on the stand, the flight controller holds level on a gimbal rig, the battery delivers its watt-hours. On the first outdoor hover the aircraft climbs 4 m on its own within a minute, and the log shows why: the flight controller is screwed hard to the frame, one propeller is out of balance, and the accelerometer, set to a ±4 g range, clips on the vibration peaks. Clipping on one side only rectifies the vibration into a false 1.5 m/s² of vertical acceleration, and the height estimator believes it. The team lowers the gyro filter cut-off from 80 Hz to 25 Hz to "hide the noise"; the next flight wobbles at 12 Hz in every gust, because the filter has eaten the phase margin of the attitude loop. Nothing was broken. A mechanical interface (the mount), a sensor setting (the range), a signal-processing choice (the filter) and a control budget (the phase margin) were each chosen alone, and a multirotor is the machine that punishes that most quickly.

> **How are the motors, propellers, speed controllers, battery and flight controller of a multirotor sized together against a hover-time, thrust and stability requirement?**
> **How is a flight-control stack made to work on a vibrating airframe, and how is a multirotor tested so that its first failure happens on the bench and not over a river?**

Concept	Engineering destination
Multirotor architecture	Quad, hexa or coaxial layout for payload and redundancy; mass budget
Brushless motors	Kv and size for a propeller and a pack voltage; hover efficiency and heating
Electronic speed controllers	Commutation method, DShot or DroneCAN, current rating, input capacitors
Propeller selection	Thrust and power at hover, figure of merit, endurance versus agility
Flight-controller architecture	MCU, sensor buses, task rates, CPU and latency budgets
IMUs	Range, anti-alias filtering, vibration isolation, clipping
Attitude estimation	Complementary filter versus EKF; why a multirotor's accelerometer misleads
Flight-control loops	Cascaded rate, angle, velocity and position loops; mixer; filter and delay budget
Battery sizing	Endurance versus pack mass, LiPo versus Li-ion, voltage floor, C-rate
Safety and testing	Pre-arm checks, failsafes, staged test campaign, FMEA, regulatory class

Roadmap. Requirements and architecture (16.1) → propulsion chain: motors (16.2), ESCs (16.3), propellers (16.4) → flight controller (16.5) and IMU (16.6) → estimation (16.7) and control loops (16.8) → the battery that closes the mass loop (16.9) → safety, verification and failure analysis (16.10).

16.1 Multirotor architecture

> **Boundary.**
> Kinematics, frames and actuator selection in general belong to Chapter 13; motor drivers, encoders and PID implementation on robots to Chapter 15. The aerodynamic and electrical theory under each block lives in the sibling books and is cited where used. This chapter owns the multirotor as an integrated design.

The reference system

The running system is an **infrastructure-inspection quadcopter**: X configuration, 650 mm motor-to-motor diagonal, 13-inch propellers, a six-cell battery and a 250 g two-axis camera gimbal. It hovers beside bridge piers and wind-turbine towers in winds up to 10 m/s, within visual line of sight of a pilot. The requirements are written before any part is chosen (Chapter 1):

Table 16.1 — Requirements of the inspection quadcopter.

ID	Requirement	Target	Verification
R1	Mission endurance with 250 g payload, 20 °C, sea level	Mission of Table 16.11 with ≥ 20 % pack energy left at landing	Energy budget, then logged flights
R2	Thrust-to-weight ratio at the loaded pack-voltage floor	≥ 2.0	Thrust stand at floor voltage
R3	Maximum take-off mass (MTOW)	≤ 2.5 kg	Weighing
R4	Attitude step response (20° roll command)	10–90 % rise ≤ 0.30 s, overshoot ≤ 10 %	Logged step tests
R5	Rate-loop stability margins	PM ≥ 45°, GM ≥ 6 dB	Injected frequency sweep in hover
R6	Position hold in 8 m/s wind	±1.0 m horizontal, ±0.3 m vertical (95 %)	GNSS/RTK log
R7	Vibration at the IMU	≤ 3 g peak per axis; zero accelerometer clip events	Log analysis, 10 flights
R8	Motor and ESC temperature after 25 min hover, 35 °C ambient	≤ 100 °C	Thermocouples
R9	Link loss and low battery	Return-to-home within 1.0 s of link loss; land at 15 % energy	Fault-injection flights
R10	Arming safety	No motor output unless pre-arm checks pass and arm switch set with throttle low	Bench test, props off

Configuration

Four rotors are the minimum for full control of a fixed-pitch multirotor: collective thrust and three body torques. Roll and pitch come from differential thrust across the arms; yaw comes from the **reaction torque** of the propellers, so adjacent rotors spin in opposite directions and yaw is commanded by speeding up one

diagonal pair and slowing the other. Yaw authority is therefore much weaker than roll and pitch authority, which shapes the mixer of Section 16.8.

Table 16.2 — Multirotor layouts for the inspection role.

Layout	Motor-out tolerance	Propulsion mass	Efficiency (same span)	Verdict
Quad X	None	Lowest	Best (largest props)	**Chosen**: 2.5 kg, VLOS
Hexa	Controlled landing after one motor loss	+30–40 %	Slightly lower	Above about 5 kg or over people
Coaxial octo (X8)	Good	Compact	15–25 % coaxial loss	Heavy payloads, small footprint
VTOL hybrid	Special	High	Far better in cruise	Long-range mapping

The price of the quad is that a single propulsion failure is catastrophic, which the FMEA of Section 16.10 has to carry.

Architecture and interfaces

Figure 16.1 shows the blocks and their interfaces. Three energy paths (battery to ESCs at 18–25 V and up to 65 A; a 5 V avionics rail; a 12 V gimbal rail), one fast control path (gyro to rate loop to DShot to ESC, under 1.5 ms) and several slow data paths (GNSS at 10 Hz, compass at 100 Hz, telemetry at 2–10 Hz) are drawn separately, because each has its own budget.

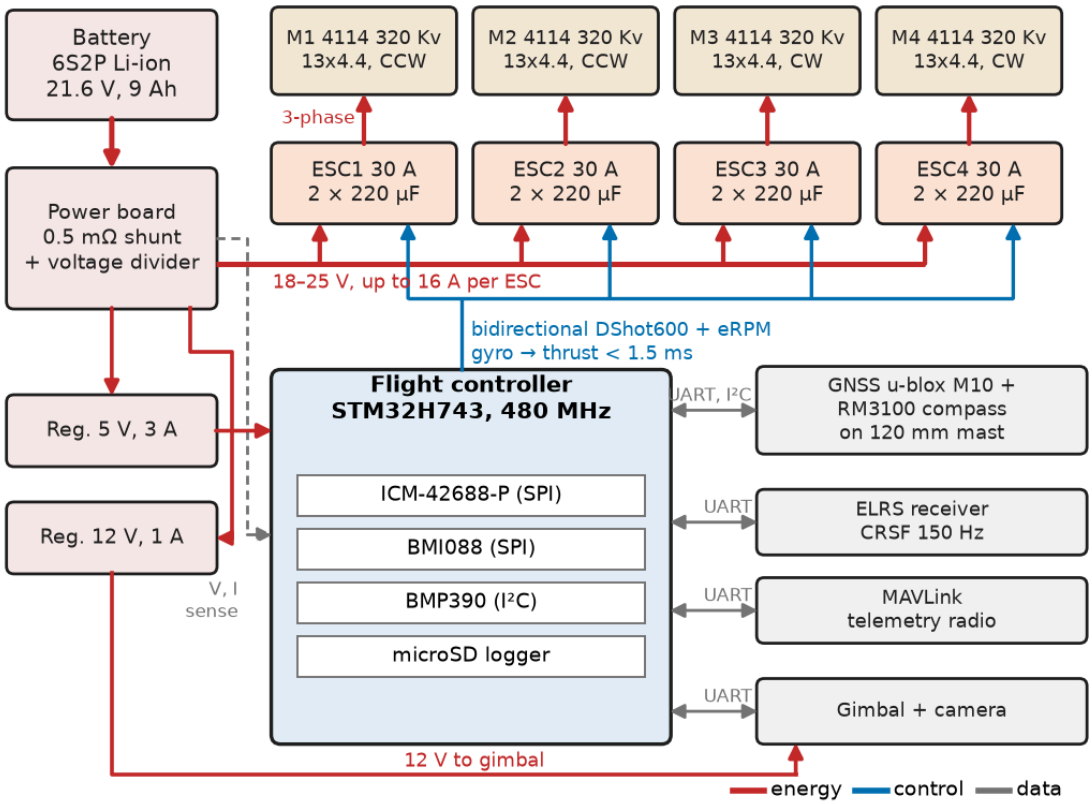


Figure 16.1 — Architecture of the inspection quadcopter: energy, control and data paths.

Mass budget

Endurance, thrust and inertia all depend on mass, so the mass budget is the first budget written and the one most often overrun. Table 16.3 is the final configuration (battery from Section 16.9).

Table 16.3 — Mass budget.

Item	Mass (g)
Frame: carbon plates, 4 arms, landing gear (650 mm)	430
Motors, 4 × 4114-class 320 Kv	392
Propellers, 4 × 13x4.4 carbon	68
ESCs, 4 × 30 A, arm-mounted	72
Avionics: FC, GNSS mast, receiver, telemetry, power board	140
Wiring and connectors	70
Payload: gimbal and camera	250
Battery, 6S2P Li-ion 21700	920
All-up mass	**2342**
Limit (R3) and margin	2500; 158 g (6.3 %)

A 6 % margin is thin; cable ties, a bigger antenna and a sun hood typically add 3–5 % before first flight. The margin is defended by a rule: any addition is weighed and booked against this table before it is fitted.

16.2 Brushless motors

> **Boundary.**
> The BLDC motor, its back-EMF, commutation and the K_v–K_t relation are in *Electrical Energy and Machines*, Ch. 8 §8.2 (link), including a worked drone-motor example. Here they are used to choose a motor.

Choosing K_v

A multirotor motor is an outrunner PM machine driving a fixed-pitch propeller directly, so it runs where its torque–speed line meets the propeller's load curve. The sizing relations from Book 5 are $E = n/K_v$ (n in rpm), $K_t = 60/(2\pi K_v)$ in N·m/A, $Q = K_t(I - I_0)$ and $V_m = E + IR$, with V_m the duty-averaged motor voltage and R the line-to-line resistance.

> **Status of the principle: DC-equivalent motor model for sizing**
> *Approximate.* Treating a BLDC/PMSM motor as $V_m = E + IR$, $Q = K_t(I - I_0)$ ignores commutation ripple, speed-dependent iron loss and the rise of R with temperature (+0.39 %/K); it predicts operating points within about 5–10 % below about 80 % of rated current.

K_v is a choice of winding turns, bracketed by three conditions: **hover near 50–60 % throttle** at nominal voltage (equal control headroom above and below hover); **T/W ≥ 2 at the loaded voltage floor** (R2); and **full-throttle current within the motor and ESC rating** on a fresh pack (25.2 V).

Table 16.4 — Motor candidates for 13x4.4 propellers on 6S (indicative values, 2026).

Motor class	K_v (rpm/V)	R (Ω)	Mass (g)	Hover throttle	T/W at floor	Full-throttle current	Verdict
4014	400	0.105	105	0.42	> 3.5	22–27 A	Over-powered; poor resolution at hover
4114	**320**	**0.15**	**98**	**0.56**	**2.28**	**15.8 A**	**Chosen**
3510	360	0.20	78	0.53	2.4	≈ 17 A	Lighter; 15 °C hotter at hover
4116	260	0.20	125	0.70	< 2	≈ 11 A	Fails R2 on a sagging pack

Worked example — the hover operating point. Each motor lifts $T = 2.342 \times 9.81/4 = 5.74$ N (585 gf). With the propeller model of Section 16.4 ($C_T = 0.105$, $C_P = 0.042$, $D = 0.330$ m), the speed is $n = \sqrt{T/(C_T \rho D^4)} = 61.3$ rev/s (3677 rpm) and the torque $Q = C_P \rho n^2 D^5/2\pi = 0.121$ N·m. With $K_t = 60/(2\pi \times 320) = 0.0298$ N·m/A, $I = 0.121/0.0298 + 0.5 = 4.55$ A. The back-EMF is $3677/320 = 11.49$ V

and $V_m = 11.49 + 4.55 \times 0.15 = 12.17$ V, a duty of $12.17/21.6 = 0.56$. Shaft power is $0.121 \times 2\pi \times 61.3 = 46.5$ W, electrical input $12.17 \times 4.55 = 55.3$ W, efficiency 84 %, copper loss $4.55^2 \times 0.15 = 3.1$ W.

The thermal limit (R8) matters only in sustained climbs. With roughly 4–6 K/W from winding to air in downwash, hover losses (3.1 W copper plus 2–3 W iron) give about 30 K of rise; a sustained 9 A climb gives about 65 K, which at 35 °C ambient approaches the 100 °C limit chosen to protect magnet adhesive and bearing grease.

16.3 Electronic speed controllers

> **Boundary.**
> The three-phase bridge and six-step commutation are in *Electronics and Computing*, Ch. 32 §32.5 (link); PMSM field-oriented control in *Electrical Energy and Machines*, Ch. 12 §12.8 (link).

Commutation method

Propeller ESCs are sensorless. **Six-step** firmware (BLHeli_32, AM32, Bluejay class) times commutation from back-EMF zero crossings: fast, cheap and light, but prone to **desynchronisation** after a large throttle step on a heavy propeller or during demagnetisation at high current. **Sensorless FOC** (sinusoidal drive with a rotor-angle observer) is quieter and 2–5 % more efficient at part load, at the cost of a faster MCU; its low-speed weakness does not matter for a propeller. On a quad a desync is a crash, so the reference design uses FOC ESCs, or six-step firmware with demagnetisation compensation and a throttle slew limit.

Protocol between flight controller and ESC

Table 16.5 — ESC command protocols.

Protocol	Signal	Update rate	Frame latency	Error detection	Telemetry
Standard PWM	1000–2000 µs pulse	50–490 Hz	up to 2 ms	None	None
OneShot125 / Multishot	125–250 µs / 5–25 µs pulse	up to 4–32 kHz	0.25 ms / 25 µs	None	None
DShot600	16 bits: 11 throttle, 1 telemetry, 4 CRC; 1.67 µs/bit	up to 8 kHz	26.7 µs	4-bit CRC	Bidirectional: eRPM every frame
DroneCAN	CAN 1 Mbit/s	400 Hz typical	≈ 0.2 ms	CRC	Current, voltage, temperature, rpm

Pulse protocols have no error detection: a glitch is a throttle command. The design uses **bidirectional DShot600** for integration reasons: the CRC rejects corrupted frames, and the eRPM from every motor feeds the RPM-tracking notch filters of Section 16.6. Aircraft above about 10 kg move to DroneCAN.

Current rating and input capacitors

Full-throttle current on a fresh pack is 15.8 A per motor; a 30 A ESC gives a factor of 1.9 for a bent propeller and 45 °C ambient.

Worked example — input capacitor of one ESC. The bridge draws pulsed current that the battery leads (0.25 µH of loop inductance for 300 mm of 12 AWG pair) cannot supply. The RMS ripple current is $I\sqrt{D(1-D)}$ (*Electronics and Computing*, Ch. 30 §30.6), worst at $D = 0.5$: for 12.4 A, 6.2 A RMS. The voltage ripple $\Delta V \approx ID(1-D)/(f_s C)$ with $f_s = 24$ kHz and $C = 440$ µF is $12.4 \times 0.25/(24\,000 \times 440 \times 10^{-6}) = 0.29$ V, 1.3 % of 21.6 V. Two 220 µF, 35 V polymer capacitors rated 3.5 A RMS each share the 6.2 A; one electrolytic of the same value rated 1.5 A would overheat. Without local capacitance, a 20 A edge of 100 ns in the lead inductance makes $L\,di/dt = 0.25 \times 10^{-6} \times 20/10^{-7} = 50$ V of spike on a 25 V bus, which is how 30 V MOSFETs die on the first punch-out.

16.4 Propeller selection

Thrust, power and the figure of merit

Propeller data are dimensionless coefficients from thrust-stand or wind-tunnel tests:

$$T = C_T\,\rho\, n^2 D^4, \qquad P = C_P\,\rho\, n^3 D^5,$$

with n in rev/s and D in metres. For the 13x4.4 carbon propeller static values $C_T \approx 0.105$ and $C_P \approx 0.042$ are typical; manufacturer stand data beat generic tables because coefficients fall 10–20 % at small-prop Reynolds numbers.

Ideal (momentum-theory) hover power is $P_{\text{ideal}} = T^{3/2}/\sqrt{2\rho A}$, and the **figure of merit** $FM = P_{\text{ideal}}/P_{\text{shaft}}$. Here $A = 0.0856$ m², disc loading $5.74/0.0856 = 67$ N/m², $P_{\text{ideal}} = 30.1$ W and $FM = 30.1/46.5 = 0.65$, typical of good small propellers (0.6–0.75).

> **Status of the principle: Momentum-theory hover power**
> *Approximate.* The actuator-disc result assumes uniform inflow, no swirl and no profile drag; it is a lower bound corrected by a measured figure of merit, and it gives the design rule that hover power per newton falls with disc loading, that is, with larger propellers.

The endurance–agility trade

Bigger, slower propellers cut hover power, but their inertia grows roughly as D^5, and so does the thrust time constant that limits attitude bandwidth (Section 16.8): about 45 ms for the 13-inch propeller on a 4114 motor, 10–15 ms for a 5-inch racing propeller.

Table 16.6 — Propeller options for the 2.34 kg quad (indicative, at hover).

Propeller	Disc loading (N/m²)	Hover rpm	Battery-side power per motor (W)	Thrust time constant	Comment
11x4.5	94	≈ 5100	≈ 70	≈ 35 ms	+20 % power; more agile
13x4.4	**67**	**3677**	**58**	**≈ 45 ms**	**Chosen**
15x5	50	≈ 2800	≈ 50	≈ 65 ms	Needs 700 mm+ frame; slower gust recovery

At hover the rotor frequency is 61.3 Hz and the two-blade passing frequency 123 Hz; both reappear in Sections 16.6 and 16.8.

Each motor–propeller–ESC set is characterised on a thrust stand (load cell, shunt, optical tachometer) before integration, to check the model and to confirm the monotonic throttle-to-thrust curve the mixer assumes. Figure 16.2 shows the result: hover at 56 % throttle, and specific thrust falling from 16 gf/W at low throttle to 6 gf/W at full throttle, which is why sustained high throttle costs endurance far out of proportion to the extra thrust.

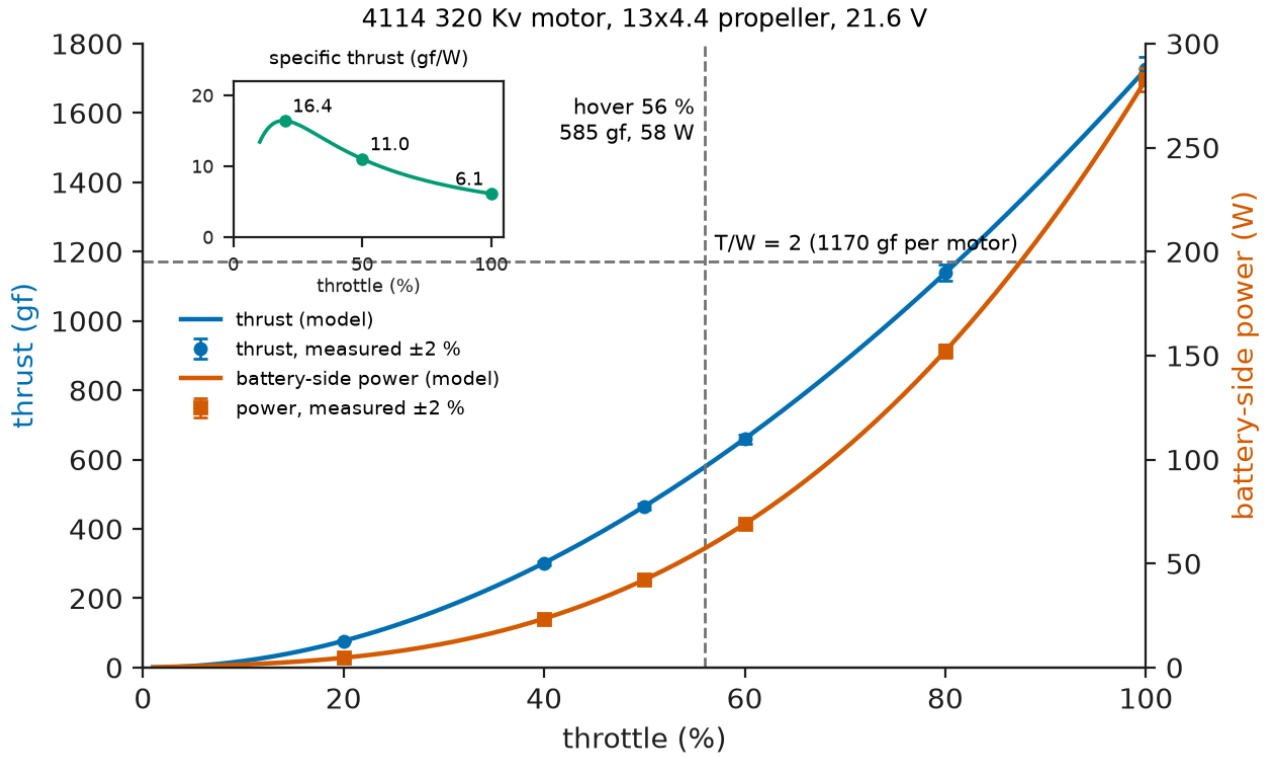


Figure 16.2 — Thrust-stand characterisation of one 4114 320 Kv motor with 13x4.4 propeller at 21.6 V.

16.5 Flight-controller architecture

> **Boundary.**
> MCU selection and firmware architecture are owned by Chapter 10 of this book and by *Electronics and Computing*, Ch. 25 §25.3; SPI and I²C by Ch. 26 §26.3–26.4 (link); scheduling and jitter by Ch. 27 §27.3 and §27.10 (link).

Hardware and stack

- **MCU**: STM32H743 (Cortex-M7, 480 MHz, double-precision FPU), enough for an EKF at 200 Hz beside the fast loops. Racing boards use F4/F7 parts because they run no EKF.
- **Two IMUs of different makes on separate SPI buses** (ICM-42688-P and BMI088), so that a fault or resonance in one does not affect both identically.
- **Barometer** (BMP390 class) under open-cell foam, shielded from propeller wash and light.
- **GNSS and magnetometer on a 120 mm mast**: 60 A in a conductor 10 cm away makes $\mu_0 I/2\pi r = 120$ µT, more than the Earth's 50 µT.
- **microSD logger**, the instrument that diagnoses every failure in this chapter.

The aircraft needs position hold, failsafes and missions, so it runs **PX4** (NuttX, EKF2) or **ArduPilot** (ChibiOS, EKF3) rather than a rate-only racing stack such as Betaflight. The integration work is configuration, tuning and verification, not writing the stack.

CPU and latency budgets

Table 16.7 — Task-rate and CPU budget on the STM32H743 (measured with a cycle counter).

Task	Rate	Execution time	CPU load
IMU FIFO read (DMA), gyro low-pass and notch filters	8 kHz	6 µs	4.8 %
Rate controller and mixer	1 kHz	35 µs	3.5 %
Attitude controller	250 Hz	15 µs	0.4 %
EKF (24 states)	200 Hz	400 µs	8.0 %
Position and velocity control	50 Hz	60 µs	0.3 %
Logging, MAVLink, failsafe and mission logic	mixed		12 %
Total; margin to 70 % ceiling			**29 %; 41 points**

The 70 % ceiling leaves room for bursts (SD-card write stalls, GNSS reacquisition) that averages hide. The rate loop's total delay is a phase cost at crossover (*Circuits, Signals and Communications*, Ch. 35 §35.8), so it is budgeted per link:

Table 16.8 — Gyro-to-thrust latency budget of the rate loop.

Link	Delay
IMU anti-alias filter and 8 kHz ODR	0.25 ms
SPI/DMA and interrupt	0.03 ms
Filtering and controller computation	0.07 ms
DShot600 frame	0.03 ms
ESC processing and next PWM period	0.12 ms
Zero-order hold of the 1 kHz output (half sample)	0.50 ms
Sum; budget used in Section 16.8; margin	**1.00 ms; 1.50 ms; 0.50 ms (33 %)**

The gyro filters are not in this table; their frequency-dependent lag is designed into the loop in Section 16.8.

16.6 IMUs

> **Boundary.**
> Sensor selection belongs to Chapter 6; aliasing and anti-alias filters to *Circuits, Signals and Communications*, Ch. 19 §19.4 and §19.9 (link); noise density to *The Mathematics Universe*, Ch. 21 §21.4–21.6.

What a flight IMU must survive

Noise is not the limit: the ICM-42688-P's gyro noise over an 80 Hz bandwidth is $0.0028 \times \sqrt{80 \times 1.11} \approx 0.026$ °/s RMS, three orders of magnitude below the vibration on a hard-mounted board. What decides the choice is **behaviour under vibration**: range, anti-alias filtering before the internal ADC, output data rate and proof-mass resonance.

Table 16.9 — MEMS IMUs for flight controllers (typical datasheet values).

Part class	Gyro noise (°/s/√Hz)	Accel noise (µg/√Hz)	Accel range	Max ODR gyro / accel	Programmable AAF
MPU-6000 (legacy)	0.005	400	±16 g	8 / 1 kHz	Limited
ICM-20602	0.004	100	±16 g	8 / 4 kHz	DLPF
ICM-42688-P (primary)	**0.0028**	**70**	**±16 g**	**32 / 32 kHz**	**Yes**
BMI088 (secondary)	**0.014**	**175**	**±24 g**	**2 / 1.6 kHz**	Fixed; vibration-robust design

Clipping, rectification and aliasing

Vibration at the IMU is dominated by the rotor frequency (imbalance, 61 Hz here), the blade-passing frequency (123 Hz), their harmonics and the frame resonances they excite. Two mechanisms turn it into a

slow error that no filter after the ADC can remove: **clipping**, which rectifies vibration into a bias, and **aliasing**, which folds vibration above half the sampling rate onto low frequencies.

Worked example — the opening failure, in numbers. The hard-mounted board saw 4.0 g of vertical vibration at 61 Hz on top of the 1 g hover specific force, so the signal swung from -5 g to $+3$ g. In the ±4 g range the negative peaks were cut at -4 g. Averaging the clipped sine numerically gives -0.848 g instead of -1.000 g: a false 0.152 g, or 1.49 m/s², reduction in specific force, which the estimator read as a downward acceleration; the altitude controller added thrust and the aircraft climbed. At 3.5 g amplitude the bias is 0.56 m/s²; at 3.0 g it is zero; in the ±16 g range nothing clips. The range went to ±16 g, and R7 was written as a 3 g peak limit with zero clip events, checked from the log on every flight.

Worked example — aliasing of a frame resonance. On an earlier board the accelerometer sampled at 1 kHz behind a weak filter, and an arm bending mode at 1,030 Hz appeared in the log at $|1030 - 1000| = 30$ Hz, inside the velocity estimator's bandwidth. Raising the ODR to 8 kHz with the programmable anti-alias filter set near 1 kHz puts the line below the 4 kHz Nyquist frequency, where the filters attenuate it before decimation.

> **Status of the principle: Vibration rectification by clipping**
> *Derived.* A zero-mean vibration added to a non-zero mean and clipped asymmetrically changes the mean, as averaging the clipped waveform shows for any waveform; the only cure is not to clip (range and isolation), not filtering afterwards.

Isolation, balancing and RPM notches

An isolator attenuates only above $\sqrt{2}$ times its natural frequency (*Engineering Physics*, Ch. 19 §19.1.3, link); a 15–25 Hz mount attenuates the 61 Hz line by a factor of about 5–15. The mount must stay away from the 8.8 Hz attitude crossover, or the IMU moves relative to the airframe inside the loop. Silicone grommets or a gel pad under the IMU carrier typically give 18–30 Hz. Balanced propellers (under 0.1 g·cm) and true shafts reduce the excitation itself. Figure 16.3 shows the spectra before and after.

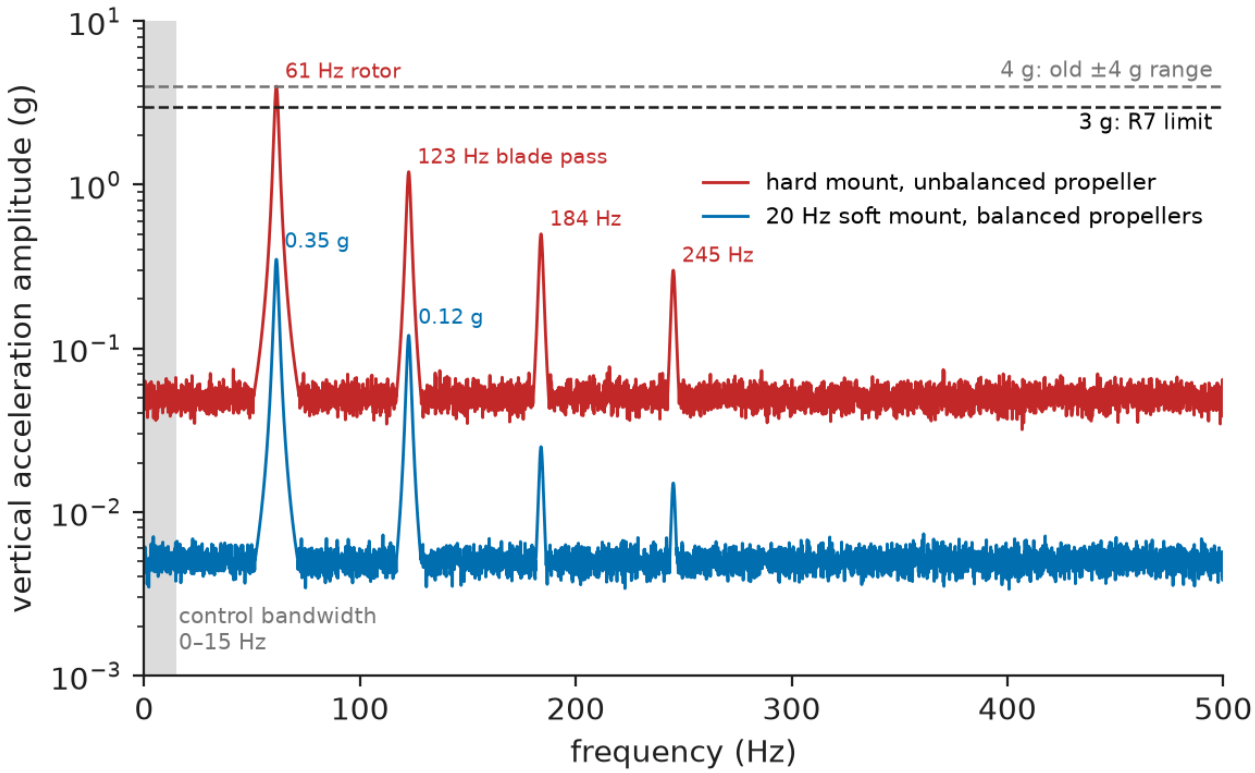


Figure 16.3 — Vertical accelerometer spectrum in hover: hard mount with an unbalanced propeller versus 20 Hz soft mount with balanced propellers.

What isolation leaves, the filters remove at known frequencies: the eRPM telemetry centres a notch on each motor's rotor frequency and its second harmonic, tracking throttle. Narrow notches (eight in all, coinciding on two frequencies in steady hover) cost far less phase at crossover than one low-pass at 25 Hz (Section 16.8), the quantitative reason the opening's filter change was the wrong fix.

16.7 Attitude estimation

> **Boundary.**
> Observers and the Kalman filter are in *Circuits, Signals and Communications*, Ch. 35 §35.7 (link); Gauss–Markov bias models in *The Mathematics Universe*, Ch. 21 §21.5. Sensor fusion as an embedded task is introduced in Chapters 12 and 15.

Gyro, accelerometer and the fusion trade

The gyro measures body rates with low noise but a bias that drifts with temperature (0.1–0.5 °/s on consumer parts during warm-up). The accelerometer measures **specific force**, not gravity; on a static IMU that gives tilt. A complementary filter blends the two with one time constant τ:

$$\hat{\theta}_k = \alpha\left(\hat{\theta}_{k-1} + \omega_k\,\Delta t\right) + (1-\alpha)\,\theta_{\mathrm{acc},k}, \qquad \alpha = \frac{\tau}{\tau + \Delta t}.$$

Why a multirotor's accelerometer misleads

A multirotor's thrust is along body z. In a drag-free idealisation an aircraft banked at θ accelerates sideways at $g\tan\theta$ and its accelerometer reads only thrust, perpendicular to the rotor plane: **it reads "level" while banked**. What the lateral axis actually senses is rotor drag, roughly proportional to airspeed, which builds up as the aircraft gains speed. Accelerometer tilt is right in steady flight and wrong in every manoeuvre, with an error decaying at the drag time constant m/k (about 3 s here). A short τ pulls the estimate toward zero during manoeuvres; a long τ lets gyro bias through.

Worked example — the best complementary filter is not good enough. A 300 s simulation at 250 Hz (Python, fixed seed) of manoeuvres in 6 s legs of ±15° bank or hover, a gyro with 0.3 °/s residual bias and the ICM-42688-P's noise, and a lateral accelerometer sensing rotor drag ($k/m = 0.3$ s^{-1}) plus 0.03 g of residual vibration gives these RMS roll errors:

Estimator	Gyro only	Accel only	CF τ=0.3 s	1 s	3 s	10 s	30 s
RMS error (°)	51.9	8.0	7.5	6.7	4.9	3.7	8.3

The best filter ($\tau \approx 10$ s) still has 3.7° RMS error. Near hover it does much better, which is why simple filters fly racing quads (the pilot closes the outer loop), but the ±1 m position hold of R6 needs about 1° of attitude accuracy.

> **Status of the principle: Accelerometer tilt on a multirotor**
> *Approximate.* "The accelerometer measures thrust, not gravity" holds for a rigid multirotor with thrust along body z and drag proportional to airspeed; it explains why accelerometer tilt is valid only in unaccelerated flight and why velocity aiding (GNSS, optical flow) is needed in manoeuvres.

The EKF in practice

PX4's EKF2 and ArduPilot's EKF3 estimate attitude, velocity, position, gyro and accelerometer biases, magnetic fields and wind (22–24 states). GNSS velocity makes attitude observable in manoeuvres (an accelerating aircraft must be tilted to produce the measured velocity change). The integrator's job is to give the filter honest inputs: the **GNSS delay** (100–200 ms) and the antenna **lever arm** (0.12 m up) as parameters; **noise parameters** that match the isolated IMU (inflating them "for vibration" makes the filter trust nothing); **innovation logging** (the opening climb shows as barometer innovations saturating); and a **compass-motor calibration**, fitted from a tethered throttle sweep, because 60 A of battery current shifts the field at the compass by several microtesla.

16.8 Flight-control loops

> **Boundary.**
> PID structure, anti-windup and derivative filtering are in *Circuits, Signals and Communications*, Ch. 35 §35.2; margins and loop shaping in Ch. 35 §35.4 and Ch. 34 §34.9 (link); PID implementation on robot hardware in Chapter 15.

The cascade

The controller is a cascade (Figure 16.4): **position** (about 0.1 Hz bandwidth) commands velocity; **velocity** (about 0.3 Hz) commands acceleration, converted to a thrust vector and a desired attitude; **angle** (a proportional gain of 6.5 s^{-1}, about 1 Hz) commands body rates; **rate** (8.8 Hz crossover here, 30–100 Hz on a racing quad) commands torques, which the **mixer** turns into four motor commands. Each loop is at least three times slower than the one inside it, so each is tuned treating its inner loop as ideal.

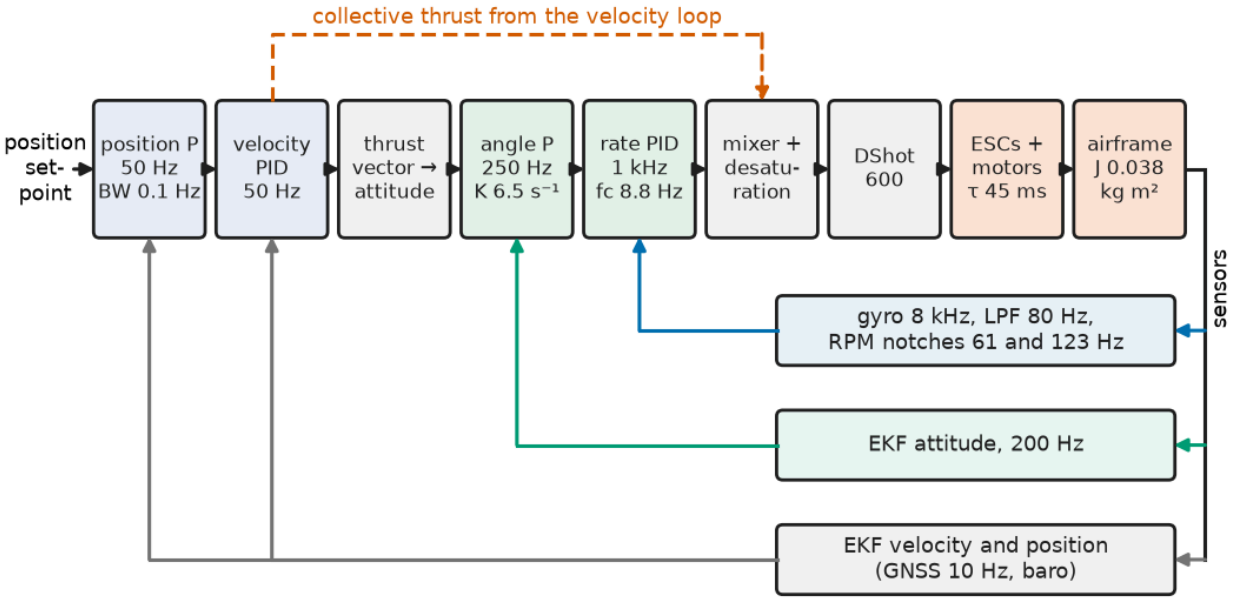


Figure 16.4 — Cascaded flight-control structure with the rate of each loop.

The rate-loop plant

From normalised torque command u to body rate, the plant is an integrator behind the motor lag and the delay:

$$\frac{\omega(s)}{u(s)} = \frac{\tau_{\max}/J_{xx}}{s\,(\tau_m s + 1)}\, e^{-sT_d}.$$

Each motor is at 5.74 N in hover and can fall to about 0.3 N, so the symmetric differential thrust is 5.44 N; with a lateral arm of $0.325/\sqrt{2} = 0.230$ m the maximum roll torque is $4 \times 5.44 \times 0.230 = 5.0$ N·m. The roll inertia, estimated from the mass budget and confirmed by a bifilar-pendulum test, is $J_{xx} = 0.038$ kg·m² (132 rad/s² peak). $\tau_m = 45$ ms and $T_d = 1.5$ ms (Table 16.8).

Worked example — what the gyro filter costs. The listing computes crossover, phase margin, gain margin and the peak of the sensitivity $|1/(1+L)|$ for the chosen PID gains with three filter configurations: the original 80 Hz second-order low-pass, the opening's 25 Hz low-pass, and 80 Hz plus RPM notches at the hover rotor frequency and its second harmonic.

```
 import numpy as np
J, Tmax, tau_m, Td = 0.038, 5.0, 0.045, 1.5e-3      # kg m^2, N m, s, s
Kp, Ki, Kd, fD = 0.6, 2.0, 0.016, 40.0              # rate PID (u per rad/s), D-filter Hz
w = 2*np.pi*np.logspace(-1, 2.7, 20000); s = 1j*w

def lpf2(fc):                                       # 2nd-order Butterworth gyro filter
    wc = 2*np.pi*fc; return wc**2/(s**2 + np.sqrt(2)*wc*s + wc**2)
def notch(f0, q=3.0):                               # RPM-tracking notch, frozen at hover
    w0 = 2*np.pi*f0; return (s**2 + w0**2)/(s**2 + w0/q*s + w0**2)

def margins(gyro):
    C = Kp + Ki/s + Kd*s/(1 + s/(2*np.pi*fD))
    P = (Tmax/J)/(s*(tau_m*s + 1))*np.exp(-s*Td)
    L = C*P*gyro
    ic = np.argmin(np.abs(np.abs(L) - 1))
    pm = 180 + np.degrees(np.angle(L[ic]))
    x = np.where((np.sign(L.imag[:-1]) != np.sign(L.imag[1:])) & (L.real[:-1] < 0))[0]
    k = x[np.argmax(np.abs(L[x]))]                  # worst crossing of the negative real axis
    ms = np.max(np.abs(1/(1 + L))); im = np.argmax(np.abs(1/(1 + L)))
    return w[ic]/2/np.pi, pm, -20*np.log10(np.abs(L[k])), w[k]/2/np.pi, 20*np.log10(ms),
w[im]/2/np.pi

for name, gy in [("LPF 80 Hz", lpf2(80)), ("LPF 25 Hz", lpf2(25)),
                 ("LPF 80 Hz + notches 61/123 Hz", lpf2(80)*notch(61)*notch(123))]:
    fc, pm, gm, f180, ms, fms = margins(gy)
    print(f"{name:30s} fc={fc:4.1f} Hz PM={pm:4.1f} deg GM={gm:4.1f} dB at {f180:4.1f} Hz"
          f" Ms={ms:3.1f} dB at {fms:4.1f} Hz")
```

```
 LPF 80 Hz                      fc= 8.8 Hz PM=54.0 deg GM=13.2 dB at 30.8 Hz Ms=3.5 dB at 17.1 Hz
LPF 25 Hz                      fc= 8.7 Hz PM=33.6 deg GM= 6.1 dB at 15.4 Hz Ms=7.9 dB at 12.0 Hz
LPF 80 Hz + notches 61/123 Hz  fc= 8.8 Hz PM=49.8 deg GM=11.0 dB at 25.4 Hz Ms=4.2 dB at 15.7 Hz
```

The 25 Hz filter leaves the crossover unchanged but costs 20° of phase margin and drops the gain margin to 6.1 dB; the 7.9 dB sensitivity peak amplifies gusts near 12 Hz 2.5 times. That is the opening's 12 Hz wobble, predicted before flight. Two tracking notches cost only 4° and keep both margins inside R5 (Figure 16.5).

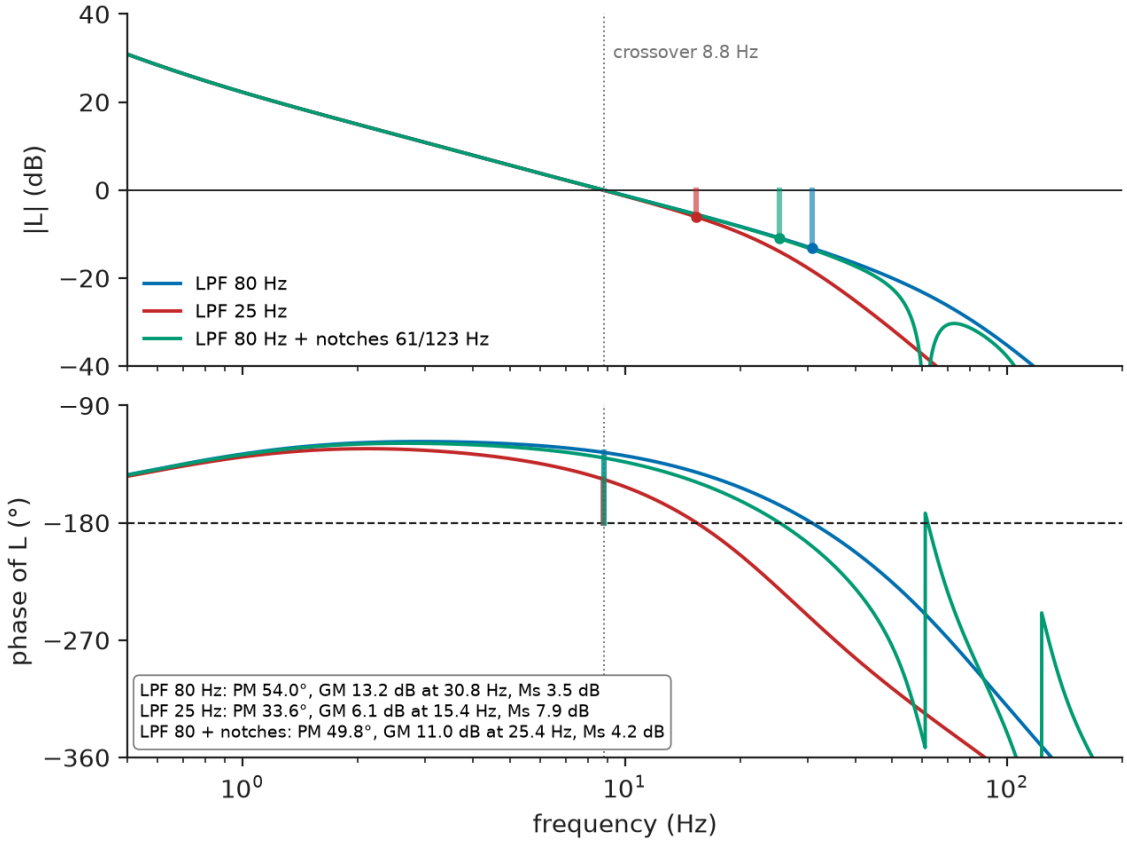


Figure 16.5 — Open-loop frequency response of the roll-rate loop for three gyro-filter configurations.

> **Status of the principle: Filter phase lag at crossover**
> *Derived.* A second-order Butterworth low-pass at f_c lags by $\arctan\left(\sqrt{2}\,x/(1-x^2)\right)$ with $x = f/f_c$, about 9° at $x = 0.11$ and 30° at $x = 0.35$; every filter in the feedback path subtracts its lag at crossover from the phase margin, so filter design on a flight controller is part of loop design, not a separate noise fix.

A time-domain simulation of the same loop at 1 kHz (motor lag, delay, filters, saturation) gives, for a 20° roll command, a 10–90 % rise time of 219 ms and no overshoot (R4); a 1 rad/s rate step overshoots 10 % with the 80 Hz filter and 16 % with 25 Hz. These are simulated; Section 16.10 compares them with flight logs.

Implementation: rate PID and mixer

The listing is the core of the 1 kHz task: a rate PID with derivative on measurement, a first-order derivative filter and conditional-integration anti-windup, then the quad-X mixer with **desaturation**. If a large torque and high collective would push a motor past full throttle, the mixer shifts the collective so that torque, which keeps the aircraft upright, is preserved at the expense of thrust; if the torque request alone spans more than the full range, the outputs saturate and the PID stops integrating.

```
 /* rate_ctrl.c -- body-rate PID and quad-X mixer, 1 kHz task (C11, illustrative) */
#include <math.h>
#include <stdbool.h>

typedef struct {
    float kp, ki, kd, d_alpha, i_lim;   /* gains in normalised torque per rad/s */
    float integ, meas_prev, d_filt;
} rate_pid_t;

static float clampf(float x, float lo, float hi) { return x < lo ? lo : (x > hi ? hi : x); }

float rate_pid_step(rate_pid_t *c, float ref, float meas, float dt, bool saturated)
{
    float e = ref - meas;
    float d = -(meas - c->meas_prev) / dt;          /* derivative on measurement */
    c->meas_prev = meas;
    c->d_filt += c->d_alpha * (d - c->d_filt);      /* first-order D-term filter */
    if (!saturated)                                 /* conditional-integration anti-windup */
        c->integ = clampf(c->integ + c->ki * e * dt, -c->i_lim, c->i_lim);
    return clampf(c->kp * e + c->integ + c->kd * c->d_filt, -1.0f, 1.0f);
}

/* PX4 quad-X order: 1 front-right CCW, 2 rear-left CCW, 3 front-left CW, 4 rear-right CW */
static const float MIX[4][3] = { /* roll, pitch, yaw */
    { -1.0f,  1.0f,  1.0f }, {  1.0f, -1.0f,  1.0f },
    {  1.0f,  1.0f, -1.0f }, { -1.0f, -1.0f, -1.0f } };

/* thrust in [0,1], tau[] in [-1,1]; returns true when torque could not be delivered */
bool mix_quad_x(float thrust, const float tau[3], float out[4])
{
    float lo = 2.0f, hi = -2.0f;
    for (int i = 0; i < 4; ++i) {
        out[i] = thrust + 0.5f * (MIX[i][0] * tau[0] + MIX[i][1] * tau[1])
                        + 0.25f * MIX[i][2] * tau[2];
        lo = fminf(lo, out[i]); hi = fmaxf(hi, out[i]);
    }
    bool sat = (hi - lo) > 1.0f;
    float shift = 0.0f;
    if (sat)             shift = 0.5f - 0.5f * (hi + lo);   /* centre the spread */
    else if (hi > 1.0f)  shift = 1.0f - hi;                 /* trade thrust for torque */
    else if (lo < 0.0f)  shift = -lo;
    for (int i = 0; i < 4; ++i) out[i] = clampf(out[i] + shift, 0.0f, 1.0f);
    return sat;
}
```

Yaw gets half the roll and pitch coefficient because reaction-torque authority is weak and a large yaw demand must not consume the differential thrust that roll and pitch need. Before first flight the mixer is checked props-off: rolling the frame right by hand must speed up the two left motors, a nose-up pitch must slow the front motors, and each motor must turn the way the yaw sign assumes. A mixer sign error is the classic cause of a multirotor flipping at take-off. Tuning then follows the cascade from the inside out (rate, angle, velocity,

position); autotune routines that inject steps or chirps converge well only on a clean IMU, so R7 is verified before any autotune.

16.9 Battery sizing

> **Boundary.**
> Chemistries, C-rate, internal resistance and usable energy are in *Electrical Energy and Machines*, Ch. 16 §16.1–16.2 (link); electrode materials in *Materials Science and Engineering*, Ch. 40 §40.2. Pack architecture and BMS design are owned by Chapter 23.

The mass loop

A heavier pack stores more energy but raises hover power, which grows as mass to the power 1.5 by momentum theory; endurance rises with pack mass, flattens and eventually falls, and before that the thrust-to-weight requirement (R2) cuts it off. The listing closes the loop with the propulsion model of Sections 16.2–16.4: all-up mass, hover battery power (95 % ESC efficiency, 10 W for avionics and gimbal), hover endurance to 80 % depth of discharge, and T/W at the loaded floor (3.5 V/cell LiPo, 3.2 V/cell Li-ion, less 2 % wiring drop).

```
 import numpy as np
from scipy.optimize import brentq
rho, g = 1.225, 9.81
D, CT, CP = 13*0.0254, 0.105, 0.042      # 13x4.4 carbon prop, static coefficients
Kv, R, I0 = 320.0, 0.15, 0.5             # 4114-class motor
Kt = 60/(2*np.pi*Kv)
eta_esc, P_aux, m_dry = 0.95, 10.0, 1.422

def point_T(T):                          # motor operating point for thrust T (N)
    n = np.sqrt(T/(CT*rho*D**4))
    I = CP*rho*n**2*D**5/(2*np.pi)/Kt + I0
    return n, I, n*60/Kv + I*R
def point_V(Vm):                         # full throttle at motor voltage Vm
    f = lambda n: n*60/Kv + (CP*rho*n**2*D**5/(2*np.pi)/Kt + I0)*R - Vm
    n = brentq(f, 1, 500); I = CP*rho*n**2*D**5/(2*np.pi)/Kt + I0
    return n, I, CT*rho*n**2*D**4
def p_hover(m):
    n, I, V = point_T(m*g/4)
    return 4*V*I/eta_esc + P_aux, n, I, V

packs = {"LiPo 6S 5000": (111.0, 0.72, 22.2), "LiPo 6S 8000": (177.6, 1.12, 22.2),
         "Li-ion 6S1P 4.5Ah": (97.2, 0.47, 21.6), "Li-ion 6S2P 9Ah": (194.4, 0.92, 21.6)}
for name, (E, mb, Vn) in packs.items():
    m = m_dry + mb
    P, n, I, V = p_hover(m)
    t = 0.8*E/P*60
    nf, If, Tf = point_V(0.98*(3.5 if 'LiPo' in name else 3.2)*6)  # loaded floor voltage
    print(f"{name:18s} AUW={m:.2f} kg P={P:5.0f} W I_batt={P/Vn:4.1f} A "
          f"hover {n*60:4.0f} rpm thr={V/Vn:.2f} t={t:4.1f} min T/W_floor={4*Tf/(m*g):.2f} Imax=
{4*If:.0f} A")
```

```
 LiPo 6S 5000       AUW=2.14 kg P=  216 W I_batt= 9.7 A hover 3517 rpm thr=0.52 t=24.7 min
T/W_floor=2.95 Imax=46 A
LiPo 6S 8000       AUW=2.54 kg P=  272 W I_batt=12.2 A hover 3831 rpm thr=0.57 t=31.4 min
T/W_floor=2.48 Imax=46 A
Li-ion 6S1P 4.5Ah  AUW=1.89 kg P=  183 W I_batt= 8.5 A hover 3305 rpm thr=0.50 t=25.5 min
T/W_floor=2.82 Imax=39 A
Li-ion 6S2P 9Ah    AUW=2.34 kg P=  243 W I_batt=11.3 A hover 3677 rpm thr=0.56 t=38.4 min
T/W_floor=2.28 Imax=39 A
```

(`Imax` is total motor current at full throttle on the floor voltage; on a fresh pack the model gives 63 A.) Sweeping pack mass continuously at 154 Wh/kg (LiPo, pack level) and 211 Wh/kg (Li-ion 21700, pack level) gives Figure 16.6: 25 min of hover needs a 0.74 kg LiPo or 0.45 kg Li-ion pack, and T/W = 2 at the floor caps the pack at 1.74 kg (LiPo, 35.0 min) or 1.25 kg (Li-ion, 43.5 min).

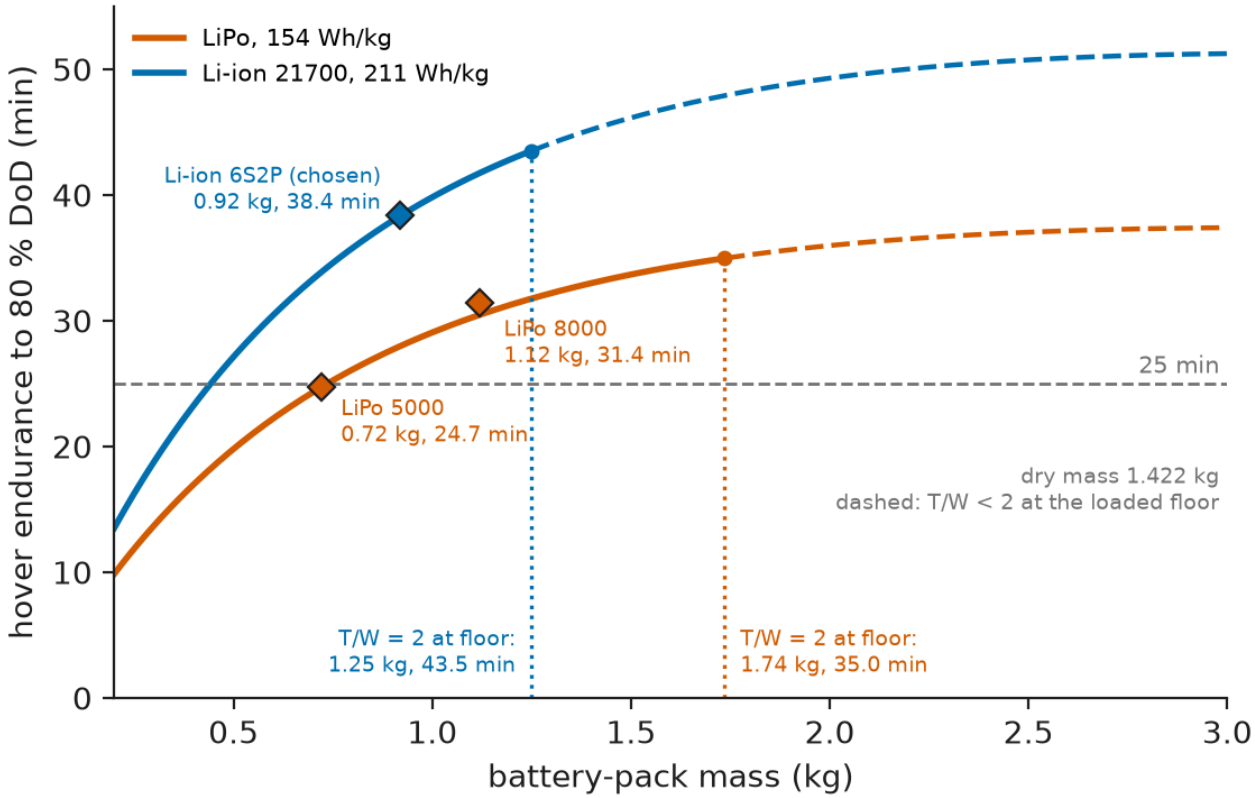


Figure 16.6 — Hover endurance versus battery-pack mass for LiPo and Li-ion packs (dry mass 1.422 kg).

Table 16.10 — Battery options against R1–R3.

Option	AUW (kg)	Hover endurance (min)	T/W at floor	Full-throttle current at floor	Verdict
LiPo 6S 5000 mAh	2.14	24.7	2.95	46 A (9.2 C)	Fails R1 once wind is added
LiPo 6S 8000 mAh	2.54	31.4	2.48	46 A (5.8 C)	Fails R3
Li-ion 6S1P 4.5 Ah	1.89	25.5	2.82	39 A (8.7 C)	Fails R1 margin
Li-ion 6S2P 9 Ah (21700, P45B class)	**2.34**	**38.4**	**2.28**	**39 A (4.3 C); 63 A (7.0 C) fresh**	**Chosen; pre-warm below 5 °C**

Li-ion 21700 cells store about 35 % more energy per kilogram than LiPo at pack level, with higher resistance and a lower voltage floor. That rules them out for a racing quad, but here hover is 11.3 A (1.25 C) and the 63 A peak is within the 90 A continuous rating of two 45 A cells in parallel by a factor of 1.4. The pack resistance (about 48 mΩ including interconnects) sags 0.5 V at hover and 3 V at the peak, which is why R2 is written at the loaded floor.

Worked example — mission energy budget. Hover endurance is not mission endurance. Table 16.11 books the mission with power factors relative to the 243 W hover, taken from logged flights of a similar aircraft.

Table 16.11 — Mission energy budget (6S2P 9 Ah, 194.4 Wh nominal).

Phase	Duration (min)	Power (W)	Energy (Wh)
Pre-arm, GNSS lock (avionics only)	1.0	10	0.2
Climb to 40 m at 2 m/s	0.33	328 (1.35 × hover)	1.8
Transit out and back at 10 m/s	4.0	304 (1.25 ×)	20.3
Inspection hover in 8 m/s wind	20.0	279 (1.15 ×)	93.2
Descent and landing	1.0	219 (0.9 ×)	3.7
Mission total	**26.3**		**119.1**
Usable to 20 % reserve (0.8 × 194.4)			155.5
Margin at 20 °C; at 0 °C (capacity −10 %)			**36.4 Wh (23 %); 20.9 Wh (15 %)**

The low-battery failsafe (R9) acts on energy remaining (coulomb count corrected by voltage under load), not on voltage alone: the sloped, sagging Li-ion curve would trigger a voltage threshold in every gust recovery or too late on a cold pack.

> **Status of the principle: Endurance from usable energy and average power**
> *Derived.* $t = E_{\text{usable}}/\bar{P}$ follows from energy conservation; its error lies in E_{usable}, which falls with temperature, discharge rate and ageing (about 20 % over 300 cycles for high-power cells), so the budget is re-run with end-of-life capacity before release.

16.10 Safety and testing

Safety functions

A quad has no glide and no tolerance to a lost rotor, so its safety rests on not starting a flight in a bad state and on predictable degradation:

- **Pre-arm checks** (R10): the two IMUs agree (gyro within 5 °/s, accelerometer within 0.75 m/s²), estimator converged, magnetic field within 15 % of expected, GNSS 3D fix with HDOP under 1.5, battery above 90 %, log open, arm switch with throttle low.
- **Failsafes** (R9): link loss → return to home after 1.0 s; low energy → return, critical energy → land; estimator failure → altitude-only landing; geofence breach → return; missing ESC telemetry → land. A kill switch independent of flight mode is tested every pre-flight.
- **Regulatory class**: in the EU the aircraft is class C2 under Delegated Regulation (EU) 2019/945 (under 4 kg, low-speed mode, remote ID); in the US it flies under 14 CFR Part 107. A parachute for flight over people would be tested to ASTM F3322. Class obligations become further rows of Table 16.1.

The staged test campaign

Tests escalate in energy and consequence; each stage passes before the next: (1) component tests (thrust stand per set, ESC thermal at 15 A for 10 min, battery capacity and resistance); (2) props-off bench (mixer signs, motor order, failsafe triggers, logging); (3) tethered vibration run at hover throttle, R7 checked before any free flight; (4) tethered and low hover for rate tuning, chirp injection (R5) and steps (R4); (5) free flight for position hold, deliberate link cuts, endurance and thermal tests; (6) a representative mission at the inspection site's conditions.

Verification

Table 16.12 — Verification of the inspection quadcopter (prototype B).

Req.	Test	Limit	Result	Pass/fail
R1	Logged mission profile, 20 °C	≥ 20 % energy at landing	31 % ± 2 % (3 flights)	Pass
R2	Thrust stand at 18.8 V	T/W ≥ 2.0	2.21 ± 0.04 (4 sets)	Pass
R3	Weighing, flight-ready	≤ 2.5 kg	2.371 kg	Pass
R4	20° roll steps, 10 flights	rise ≤ 0.30 s, OS ≤ 10 %	0.24 ± 0.02 s, 3 %	Pass
R5	Chirp 1–40 Hz in hover	PM ≥ 45°, GM ≥ 6 dB	PM 47° ± 3°, GM 10 dB	Pass
R6	Position hold, 8 m/s gusts, RTK reference	±1.0 m / ±0.3 m (95 %)	0.72 m / 0.21 m	Pass
R7	IMU log, 10 flights	≤ 3 g peak, 0 clips	0.4 g peak, 0 clips	Pass
R8	25 min hover, 35 °C	≤ 100 °C	winding 71 °C, ESC 64 °C	Pass
R9	Link cut ×5; energy failsafe	RTH ≤ 1.0 s; land at 15 %	0.6 s; 15.3 %	Pass
R10	Bench, props off, 20 trials	No output unless armed correctly	0 false starts	Pass

The measured phase margin (47°) is 3° below the simulated 49.8°, consistent with a motor time constant a few milliseconds longer on the aircraft than on the stand; the mass grew 29 g over budget, still inside R3.

Failure analysis

Table 16.13 — FMEA of the inspection quadcopter (top items).

Failure mode	Effect	Cause	Detection	Mitigation
Accelerometer clipping	Height bias; uncommanded climb	Hard mount, imbalance, ±4 g range	Clip counter; vibration metrics	±16 g, 20 Hz soft mount, balancing, R7 every flight
Attitude-loop wobble	Poor image; loss of control in gusts	Gyro LPF lowered; phase margin eroded	Sensitivity peak in chirp test	RPM notches; R5 test after any filter change
ESC desynchronisation	Motor stops; aircraft lost	Throttle step on heavy prop; demagnetisation	eRPM to zero	FOC or demag compensation; slew limit; land on missing telemetry
Propeller crack or loss	Aircraft lost	Fatigue, loose hub, ground strike	Inspection; vibration rise	Self-locking hubs; replace after strike; hexa over people
Cold-pack sag	Early failsafe or brown-out	Li-ion resistance doubles at 0 °C	Voltage under load at arm	Pre-warm to ≥ 15 °C; energy-based failsafe
Compass interference	Yaw error, circling in position hold	Battery current near compass	Innovation ratio; mag–current correlation	Mast, compass-motor calibration, dual-antenna GNSS yaw
ESC input capacitor failure	MOSFET overvoltage	Ripple over rating; long leads	ESC temperature; scope	Two polymer caps at the ESC; short leads

The opening story is the first two rows. A symptom in one domain (climb, wobble) had its cause in another (mount, range, filter phase), and only the numbers written down in advance (R5, R7) told the team which fix was right.

Revision notes

Prototype B carries three changes, each traced to a test: the soft-mounted IMU carrier and propeller balancing (R7), RPM notches replacing the 25 Hz low-pass (R5), and the energy-based failsafe (R9). The next airframe adds dual-antenna GNSS heading, removing the compass from the yaw solution near steel girders, and a hexacopter variant for operations over people, where a motor failure becomes a controlled landing rather than a loss.

What remains open

Open question	Where it is answered
Kinematics, frames and actuator choice for arms and ground robots	Chapter 13
Motor drivers, encoder interfaces and PID loops on robot hardware in general	Chapter 15
Integrating, identifying and testing a whole robot with hardware in the loop	Chapter 17
Battery packs with a full BMS and cell safety testing	Chapter 23
Radio links for telemetry and video: budgets and robustness	Chapter 26
Vibration spectra and notch filters as embedded DSP	Chapter 27
Calibrating the onboard estimator against a model in a digital twin	Chapter 33
PMSM control theory behind FOC ESCs	*Electrical Energy and Machines*, Ch. 12 §12.8
Observers and Kalman filtering as control theory	*Circuits, Signals and Communications*, Ch. 35 §35.7

Next: Chapter 17 steps back from the aircraft to the method behind it: integrating the mechanics, electronics, firmware, power and communications of a robot, identifying it, and testing it in the loop and in the field.

Chapter 17 — Robotic System Integration

Part IV · Robotics and Mechatronics

A start-up builds a six-axis collaborative arm for small-parts assembly. Each joint module passes its bench test on a laboratory supply: torque, speed, encoder accuracy, temperature. The main controller passes its tests against a simulated arm. Assembled, the arm runs its first full-speed demonstration and, at the first fast stop, every joint faults at once, the brakes drop, and the arm overruns its programmed stop by 165 mm. On the bench each joint's laboratory supply sank its regenerated energy; in the arm six joints decelerating together return about 23 J to a shared 48 V bus whose only storage is 3.8 mF of capacitance, the bus reaches the drives' 60 V overvoltage trip in 11 ms, and a trip removes torque. Integration is the discipline that finds such failures on paper: the shared bus, cable, clock, structure and fault reaction belong to no single module, so they have to be designed, budgeted and tested as a system.

> **Which interfaces (mechanical, electrical, firmware, timing, power, communication) must be specified between the modules of a robot, and what numbers go into each specification?**
> **How is an integrated robot calibrated, identified, tested in the loop and in the field so that its first system-level failure happens in the lab and not on a customer's line?**

Concept	Engineering destination
Mechanical design interfaces	Flanges, datums, stiffness and the first resonance; thermal paths; cable pass-through
Electronics integration	Joint-module electronics, grounding, harnesses through rotating joints, EMC
Firmware	Joint state machines, fault reactions, parameter and version management
Control	Central trajectory and feed-forward, joint cascades, bandwidth allocation
Power distribution	Shared DC bus, peak and regenerative power, brake chopper, STO and stop categories
Communication	Bus choice, frame design, bus load, synchronisation and timeouts
Calibration	Joint zero offsets, kinematic parameters, torque constants
System identification	Inertia, friction and gravity parameters; frequency response and resonances
Hardware-in-the-loop testing	Controller and joint test benches, fault injection, regression
Field testing	ISO 9283 performance tests, endurance, collision-force tests, pilot deployment

Roadmap. The reference arm, its requirements and architecture (17.1) → the interfaces between its modules: mechanical (17.2), electronic (17.3), firmware (17.4), control (17.5), power (17.6), communication (17.7) → making the assembled arm match its model: calibration (17.8) and identification (17.9) → proving it: hardware in the loop (17.10), field tests (17.11) → failure analysis and the second iteration (17.12).

17.1 The reference arm: requirements and architecture

> **Boundary.**
> Requirements method and traceability are owned by Chapter 1, architecture and trade studies by Chapter 2, test planning and regression by Chapter 4. Kinematics, frames and actuator selection are in Chapter 13; motor drivers, encoder interfaces and PID implementation in Chapter 15. This chapter applies them to one robot and owns what lies between the modules: integration, identification, HIL and field testing (TOC boundary table).

The reference system

The running system is a **six-axis collaborative arm** for assembly next to people: 3 kg payload, 700 mm reach, six joint modules on a 48 V bus, a central motion controller and a dual-channel safety controller. J1–J3 use a frameless BLDC motor and a size-25 strain-wave gear at 100:1; the wrist joints J4–J6 a smaller motor and a size-14 gear at 50:1. Every joint has a motor-side and an output-side encoder.

Table 17.1 — Requirements of the collaborative arm.

ID	Requirement	Target	Verification
R1	Payload at full reach	3.0 kg at 700 mm, all poses	Static load test, thermal run
R2	Pose repeatability (ISO 9283 RP)	≤ ±0.05 mm	ISO 9283 test, laser tracker
R3	Pose accuracy after calibration (ISO 9283 AP)	≤ 1.0 mm	ISO 9283 test, laser tracker
R4	Joint speed	J1–J3 180 °/s, J4–J6 360 °/s	Logged moves
R5	Path accuracy at 250 mm/s (ISO 9283 AT)	≤ 0.5 mm	Tracker, linear and circular paths
R6	Controlled stop from full speed (IEC 60204-1 category 1)	Stop ≤ 0.30 s, bus ≤ 58 V, no fault trip	Instrumented stop tests
R7	Supply	230 V AC; mean power ≤ 250 W in the standard cycle	Power analyser
R8	Joint communication	1 kHz cycle, jitter ≤ 20 µs, worst-case bus load ≤ 50 %	Bus analyser, timing log
R9	Quasi-static contact force in collaborative mode (ISO/TS 15066, hand region)	≤ 140 N	Force-measurement device, 3 sites
R10	Endurance	100,000 standard cycles without fault	Endurance run
R11	Touchable housing temperature, 25 °C ambient, standard cycle	≤ 60 °C	Thermocouples
R12	EMC	IEC 61000-6-2 immunity, IEC 61000-6-4 emissions	Accredited lab

Architecture

Figure 17.1 shows the blocks and the interfaces that this chapter specifies. The architecture is distributed: each joint closes its current, velocity and position loops locally, and the central controller sends synchronised setpoints with feed-forward torques at 1 kHz. Centralised current control would need high-bandwidth links and many more conductors through six hollow joints.

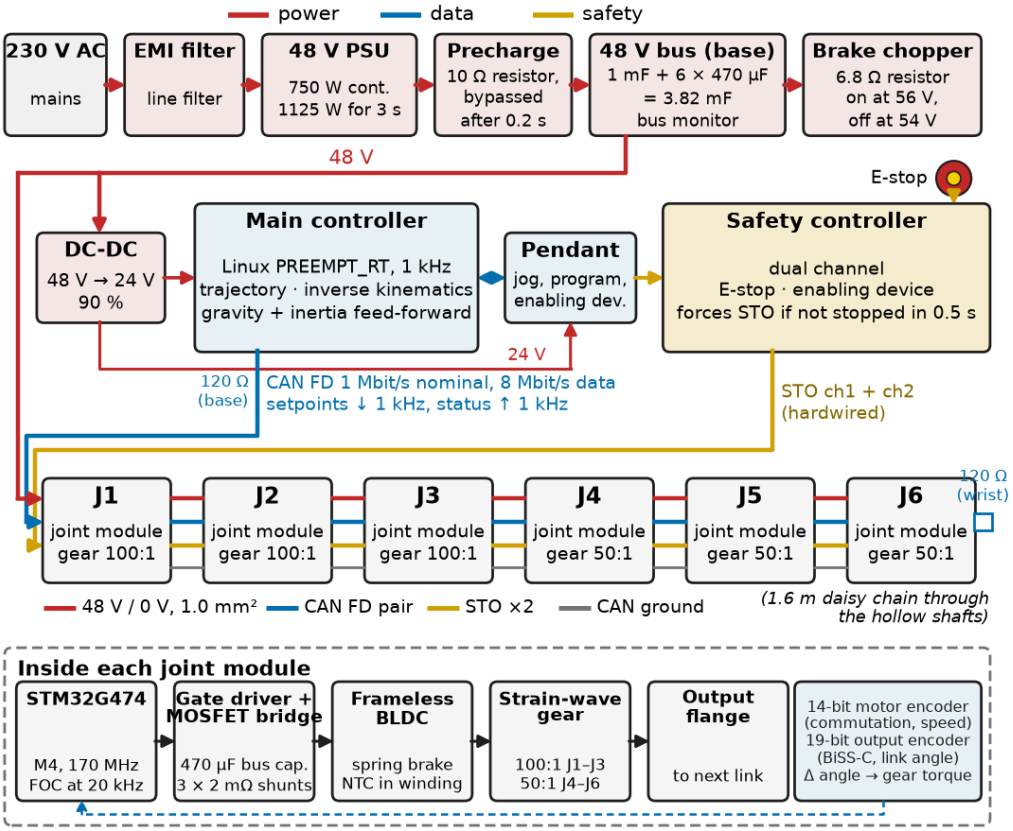


Figure 17.1 — Architecture of the six-axis collaborative arm with its power, communication and safety interfaces.

The interface list

An integrated robot is specified as much by its interface control document (ICD) as by its modules: for each pair of blocks, the physical interface, the quantities crossing it with ranges and timing, and the owner of each side. The chapter is organised around those interfaces, because most integration failures are interfaces whose numbers were never written down.

17.2 Mechanical design interfaces

What the joint–link interface carries

Through each joint flange pass load (including the dynamic loads of fast stops), alignment of the joint axis to the link datums, stiffness, heat and the cable. The ICD fixes each:

Table 17.2 — Mechanical ICD for the J2 (shoulder) joint–link interface.

Item	Specification	Reason
Flange	8 × M5 on 86 mm bolt circle, 2 dowel pins H7/m6, 12.9 screws at 9.5 N·m	Transfers the gear's 157 N·m repeated peak plus overturning moment without slip
Axis datum	Output flange face and pilot diameter; perpendicularity 0.02 mm per 100 mm	Kinematic accuracy; residual absorbed by calibration (Section 17.8)
Torsional stiffness	Gear ≥ 2.5 × 10⁴ N·m/rad at rated load; link ≥ 5 × 10⁴ N·m/rad	Sets the first resonance (worked example below)
Heat path	Flange contact ≥ 1,500 mm², thermal pad 3 W/(m·K); link acts as heat spreader	Housing temperature R11
Cable pass-through	18 mm bore, ±360° torsion, bend radius ≥ 10 × cable diameter	Cable life (R10)
Mass and centre of gravity	1.6 kg; CG within 5 mm of axis	Gravity and inertia budget

Stiffness and the first resonance

A strain-wave gear is the softest element in the drive chain; with the reflected motor inertia on one side and the arm on the other it forms a two-mass system.

Worked example — the shoulder's first resonance at full reach. With the arm horizontal and extended, the masses beyond the shoulder (upper-arm link 0.5 kg, elbow module 1.6 kg at 0.35 m, forearm link 0.4 kg, wrist modules 2.1 kg at about 0.62 m, payload and gripper 3.5 kg at 0.70 m) give a gravity moment of $4.60\ \text{kg m} \times 9.81 = 45.1$ N·m and an inertia about the shoulder axis of $J_L = 2.84$ kg·m². The rotor inertia of 1.2×10^{-4} kg·m² reflected through 100:1 is $J_M = 1.2$ kg·m². With gear stiffness $K = 2.5 \times 10^4$ N·m/rad, the two-mass model (*Engineering Physics*, Ch. 19 §19.3, link) has an anti-resonance at $\frac{1}{2\pi}\sqrt{K/J_L} = 14.9$ Hz and a resonance at $\frac{1}{2\pi}\sqrt{K(1/J_L + 1/J_M)} = 27.4$ Hz. Folded and without payload, J_L falls to about 0.6 kg·m² and the anti-resonance rises to about 32 Hz, so the plant changes by a factor of two across the workspace and the position loop is designed for the lowest value: about $14.9/3 \approx 5$ Hz (Section 17.5). Strain-wave gears are also two to three times stiffer at high load than near zero torque, so the resonance moves with gravity load too; Section 17.9 measures it.

> **Status of the principle: Position-loop bandwidth below the first resonance**
> *Empirical.* Practice in servo design keeps the position-loop bandwidth at or below about one third of the lowest anti-resonance seen from the output sensor; tighter loops are possible with notch filters, input shaping or state feedback, but the rule sets a safe first design and holds for lightly damped compliant drives.

Thermal path

Through the housing alone (about 2 K/W to air), the shoulder's 20 W would reach 65 °C at 25 °C ambient, failing R11. Bolting the module through a thermal pad to the aluminium link, which spreads the heat, brings the combined resistance to about 1.2 K/W and the surface to $25 + 20 \times 1.2 = 49$ °C. Contact area, flatness and pad are therefore mechanical interface items.

Tolerances and what calibration absorbs

A perpendicularity error of 0.02 mm per 100 mm is 0.011°, or 0.14 mm at a 0.7 m lever; six joints of such errors, link tolerances of ±0.1 mm and encoder offsets of up to a degree accumulate to millimetres. Repeatability (R2) is a property of the hardware; accuracy (R3) is restored by calibration. The ICD therefore specifies *stability* of the geometry (dowels, no slip at peak load, bounded thermal growth) rather than absolute precision.

17.3 Electronics integration

> **Boundary.**
> Gate drivers, MOSFET bridges and current sensing are in *Electronics and Computing*, Ch. 32 (link); PMSM field-oriented control in *Electrical Energy and Machines*, Ch. 12 §12.8 (link); encoder interfaces on robots in Chapter 15.

The joint module

The joint actuator is the arm's repeated building block; its make-or-buy decision sets cost, size and how much of the integration the team owns.

Table 17.3 — Joint-actuator options for J1–J3 (indicative 2026 prices, 100 pcs).

Option	Torque (rated / peak)	Backlash, stiffness	Mass	Price	Integration effort	Verdict
Commercial integrated cobot joint	60 / 150 N·m	< 20 arcsec; specified	1.5 kg	$1,200–1,800	Low; closed firmware	Fault behaviour not controllable
In-house: frameless BLDC + size-25 strain-wave gear + own driver	**67 / 157 N·m**	**< 20 arcsec; 2.5 × 10⁴ N·m/rad**	**1.6 kg**	**$650 BOM**	**High**	**Chosen: owns firmware, faults, cost**
BLDC + cycloidal gear	70 / 180 N·m	≈ 1 arcmin; stiffer	2.1 kg	$450 BOM	High	Candidate for J1 in Revision B
Smart bus servo (hobby class)	6–10 N·m	0.2–0.5°	0.2 kg	$300	Low	Fails R1 tenfold

Its electronics: an STM32G474 (Cortex-M4, 170 MHz, CORDIC, FDCAN), a DRV8353-class gate driver with six 100 V, 5 mΩ MOSFETs at 20 kHz, three 2 mΩ low-side shunts, a 14-bit magnetic encoder on the motor shaft, a 19-bit absolute encoder on the output, a power-off spring brake, NTCs in the winding and on the bridge, and 470 µF of polymer and film bus capacitance.

Two encoders, one more sensor

The motor encoder gives commutation angle and a clean velocity for the fast loops; the output encoder gives link position free of the gear's 1–2 arcmin cyclic transmission error. Their difference is the **gear deflection**

and hence the output torque, $\tau \approx K(\theta_m/N - \theta_{out})$. A 45 N·m load deflects the gear by 1.8 mrad (0.10°), about 150 counts of the 19-bit output encoder (0.3 N·m per count). The transmission error would by itself look like 7–15 N·m, so it is removed with the table measured in the joint calibration (Section 17.8); the joint then measures its own torque to a few N·m without a torque sensor, for collision detection (R9) and as a cross-check on the current-based estimate.

Grounding, harness and EMC

- **Conductors through the arm**: 48 V and 0 V (1.0 mm² each), CAN FD pair, CAN ground, two STO lines, in a torsion-rated cable whose braid is bonded 360° to the housing at every joint.
- **Voltage drop**: 1.5 m out and back of 1.0 mm² copper (52 mΩ in total) drops 1.3 V at a 25 A peak, acceptable for drives that work down to 36 V.
- **Ground reference**: the 0 V conductor carries motor current, so a separate CAN ground keeps the transceivers inside their ±12 V common-mode range.
- **Encoder noise**: the output-encoder link runs past motor phases switching at about 5 V/ns, so it is differential (BiSS-C, RS-422 levels) with a CRC on every position word; CRC failures are counted as a health metric.
- **Immunity**: IEC 61000-4-4 bursts at 2 kV on the AC input and coupled onto the arm cable (R12) find missing shield bonds.

17.4 Firmware

> **Boundary.**
> Firmware architecture, timing and fault handling for embedded systems are owned by Chapter 10 of this book; RTOS scheduling and watchdogs by *Electronics and Computing*, Ch. 27 §27.3 and §27.11; explicit state machines and validation by Ch. 28 §28.4 and §28.10 (link).

Joint firmware structure and timing

The joint firmware is interrupt-driven with three fixed-rate layers (Table 17.4).

Table 17.4 — Joint MCU timing budget (STM32G474 at 170 MHz, measured with a cycle counter).

Task	Rate	Execution time	CPU load
Current loop (FOC with CORDIC, PWM update, encoder read via DMA)	20 kHz	12 µs	24 %
Velocity loop and filters	5 kHz	4 µs	2 %
Position loop, setpoint interpolation, state machine	1 kHz	20 µs	2 %
CAN FD frame handling (two frames per cycle)	1 kHz	8 µs	0.8 %
Diagnostics, temperature model, parameter service	100 Hz	150 µs	1.5 %
Total			**30 %**
Margin to 60 % ceiling (current-loop jitter protection)			**30 points**

The joint state machine

Every joint runs the same state machine (Figure 17.2); the central controller only commands transitions, and **the fault reaction is chosen per fault**: after a communication loss or overtemperature the joint can still control itself, so it performs a controlled stop on its own sensors; after an overcurrent or encoder disagreement it cannot trust its control, so it removes torque and drops the brake at once.

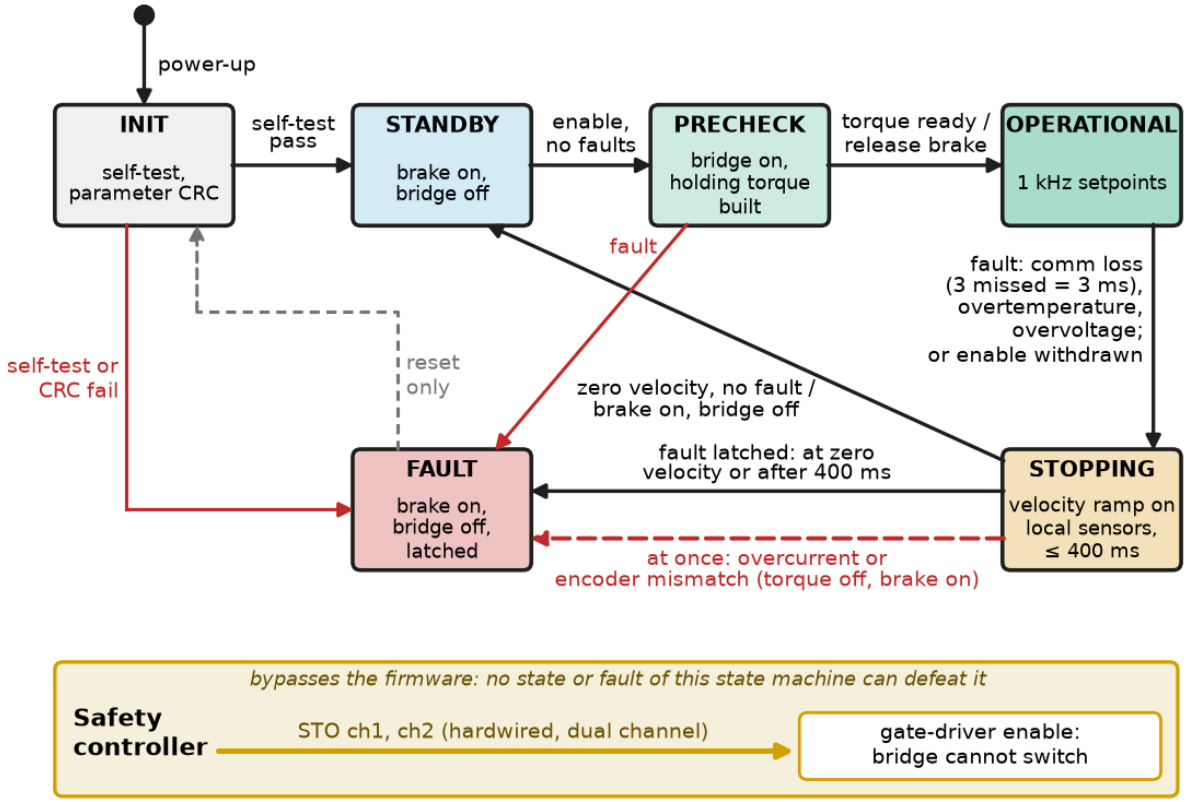


Figure 17.2 — Joint-module state machine with fault reactions.

```
 /* joint_fsm.c -- joint-module state machine, called once per 1 kHz tick (C11, illustrative) */
#include <stdbool.h>
#include <stdint.h>

typedef enum { ST_INIT, ST_STANDBY, ST_PRECHECK, ST_OPERATIONAL, ST_STOPPING, ST_FAULT } jstate_t;
enum { F_OVERVOLT = 1u << 0, F_OVERTEMP = 1u << 1, F_ENC_MISMATCH = 1u << 2,
       F_COMM_LOSS = 1u << 3, F_OVERCURRENT = 1u << 4, F_PARAM_CRC = 1u << 5, F_STOP_TIMEOUT = 1u
<< 6 };
#define F_LOSE_CONTROL (F_OVERCURRENT | F_ENC_MISMATCH)  /* torque off and brake at once */
#define CMD_TIMEOUT 3u                                   /* missed 1 ms commands */
#define STOP_MAX    400u                                 /* ms allowed for a controlled stop */

typedef struct { jstate_t st; uint32_t faults; uint16_t missed, stop_ticks; } joint_t;

extern bool selftest_ok(void), hold_torque_ready(void), velocity_is_zero(void);
extern void brake_engage(void), brake_release(void), bridge_enable(bool on);
extern void ramp_velocity_to_zero(void);                 /* one tick of the stop ramp */

void joint_step(joint_t *j, bool cmd_valid, bool enable_req, bool reset_req, uint32_t new_faults)
{
    if (cmd_valid) j->missed = 0u;
    else if (j->missed < UINT16_MAX) j->missed++;
    if (j->st == ST_OPERATIONAL && j->missed >= CMD_TIMEOUT) new_faults |= F_COMM_LOSS;
    j->faults |= new_faults;                             /* latched until reset */

    switch (j->st) {
    case ST_INIT:
        j->st = selftest_ok() ? ST_STANDBY : ST_FAULT; break;
    case ST_STANDBY:
        if (enable_req && j->faults == 0u) { bridge_enable(true); j->st = ST_PRECHECK; } break;
    case ST_PRECHECK:
        if (j->faults != 0u) { bridge_enable(false); j->st = ST_FAULT; }
        else if (hold_torque_ready()) { brake_release(); j->st = ST_OPERATIONAL; }
        break;
    case ST_OPERATIONAL:
        if (j->faults != 0u || !enable_req) { j->stop_ticks = 0u; j->st = ST_STOPPING; } break;
    case ST_STOPPING:
        if ((j->faults & F_LOSE_CONTROL) == 0u) ramp_velocity_to_zero();
        if (++j->stop_ticks >= STOP_MAX) j->faults |= F_STOP_TIMEOUT;   /* stop too slow */
        if ((j->faults & (F_LOSE_CONTROL | F_STOP_TIMEOUT)) != 0u || velocity_is_zero()) {
            brake_engage(); bridge_enable(false);
            j->st = (j->faults != 0u) ? ST_FAULT : ST_STANDBY;
        }
        break;
    case ST_FAULT:
    default:
        brake_engage(); bridge_enable(false);
        if (reset_req) { j->faults = 0u; j->st = ST_INIT; } else j->st = ST_FAULT;  /* reset: re-
test */
        break;
```

```
    }
}
```

The brake is released only after the joint holds the load with torque, so the arm does not drop by its backlash on enable, and applied only at zero velocity in a controlled stop, because a spring brake is a holding device rated for few dynamic stops. Safe torque off (STO) is not in this code: it is a dual-channel hardware path from the safety controller to the gate-driver enable that no firmware fault can defeat (IEC 61800-5-2; performance level d per ISO 13849-1 is typical for a collaborative arm).

Versions and parameters

Eight firmware images and parameter sets (six joints, main and safety controllers) must be compatible. Each joint reports firmware, hardware and parameter versions with a CRC at INIT, and the main controller enables only combinations in a signed compatibility table. Joint calibration data (Sections 17.8–17.9) live in the joint's own flash with its serial number, so a replaced joint brings its calibration with it.

17.5 Control

> **Boundary.**
> PID structure, tuning and digital-control delay are in *Circuits, Signals and Communications*, Ch. 35 §35.2–35.4 and §35.8 (link); servo sizing in *Electrical Energy and Machines*, Ch. 8 §8.7; PID implementation on robot hardware in Chapter 15 of this book.

Where each loop runs

Table 17.5 — Control-loop allocation and bandwidths.

Loop	Runs on	Rate	Sensor	Closed-loop bandwidth	Limited by
Current (FOC)	Joint	20 kHz	Phase shunts	≈ 2 kHz	PWM and sampling delay
Velocity (PI)	Joint	5 kHz	Motor encoder	≈ 80 Hz	Resonance at 27 Hz handled by notch and motor-side feedback
Position (P)	Joint	1 kHz	Output encoder	≈ 5 Hz	Anti-resonance 14.9 Hz (Section 17.2)
Trajectory, inverse kinematics, gravity and inertia feed-forward	Main controller	1 kHz	Commanded state	—	Model accuracy (Section 17.9)
Collision detection	Joint and main controller	1 kHz	Torque estimate versus model	—	Friction model error

Feed-forward does the work

A 5 Hz position loop cannot follow a fast trajectory on feedback alone. Every millisecond the central controller therefore sends the position setpoint plus the velocity and the **torque the model predicts** (gravity, inertia times commanded acceleration, friction), and the feedback loops correct only the residual. The quality of the model (Section 17.9) becomes the quality of the motion.

Worked example — what gravity feed-forward saves. At full reach the shoulder holds 45.1 N·m. Without feed-forward the velocity-loop integrator must build it from error: on the dynamometer (Section 17.10), a 10 N·m load step then gives a 0.4 mrad peak error (0.28 mm at the tool) for 100–200 ms, most of R5's 0.5 mm budget. With feed-forward accurate to 2 % (limited by payload uncertainty; Section 17.9 identifies G to 0.05 %) the transient is under 0.01 mm. Holding 45.1 N·m through a 100:1 gear of 85 % efficiency takes $45.1/(100 \times 0.85) = 0.53$ N·m at the motor, 4.4 A at $K_t = 0.12$ N·m/A and about 10 W of copper loss, which enters the power and thermal budgets.

Collaborative operation

In collaborative mode the arm limits power and force (ISO 10218-1, ISO/TS 15066) and compares the measured joint torque with the model torque. An 8 N·m residual threshold at the shoulder corresponds to about $8/0.7 \approx 11$ N at the tool at full reach; the friction model error sets how low it can go without false trips, which makes the friction identification of Section 17.9 a safety item.

17.6 Power distribution

> **Boundary.**
> Drive braking, regenerative operation and brake choppers are in *Electronics and Computing*, Ch. 32 §32.6 and *Electrical Energy and Machines*, Ch. 10 §10.4–10.5 (link); stop categories, emergency stop and safety of machinery wiring in Chapter 20 and Chapter 21 of this book and IEC 60204-1.

The power budget

Table 17.6 — Power budget of the arm on the 48 V bus.

Load	Steady (W)	Peak (W)	Note
Joint logic and encoders, 6 × 3 W	18	18	
Brakes released (power-to-release), pull-in then 50 % hold	18	36	Pull-in 100 ms
Gravity holding, worst pose (copper and inverter)	17	17	Shoulder 10 W
Main controller, pendant, gripper (via 24 V DC-DC, 90 %)	53	70	
Motion: all major axes accelerating to full speed in 0.25 s	—	612	416 W mechanical at 68 % efficiency
Total	**106**	**753**	
PSU rating: 750 W continuous, 1,125 W for 3 s			Peak margin 33 %
Standard-cycle mean (measured target R7)	180		Margin 28 % to 250 W

Rather than buy a larger supply, the trajectory planner caps the total commanded mechanical power at 500 W: a power budget enforced in software.

Regeneration: the opening failure

Decelerating joints return energy to the bus. The PSU only sources current, the bus loads take about 50 W, and the bus capacitance (six joints × 470 µF plus 1 mF in the base, 3.82 mF) stores very little energy per volt.

Worked example — the bus during a controlled stop. At full speed (180 °/s on J1–J3) the arm's kinetic energy, including reflected rotor inertias, is about 52 J (20 J each in J1 and J2, 9 J in J3, 3 J in the wrist). A linear stop in 0.25 s peaks at $2 \times 52 / 0.25 = 416$ W mechanical; with 68 % reaching the bus and 50 W of bus load, the bus absorbs up to 233 W, while the capacitors hold only $\tfrac{1}{2} \times 3.82 \times 10^{-3} \times (60^2 - 48^2) = 2.5$ J below the 60 V trip. The listing integrates the bus energy balance with and without a brake chopper (switched on at 56 V, off at 54 V).

```
 import numpy as np
E0, td, eta = 52.0, 0.25, 0.68       # J kinetic energy at full speed, s stop time, regen fraction
C, V0, P_load = 3.82e-3, 48.0, 50.0  # F bus capacitance, V, W drawn by logic and brakes
V_ov, V_on, V_off, Rb = 60.0, 56.0, 54.0, 6.8   # trip and chopper thresholds, ohm
dt = 1e-5; t = np.arange(0, 0.35, dt)
P_mech = np.where(t < td, 2*E0/td*(1 - t/td), 0.0)  # linear speed ramp to zero

def run(chopper):
    V, on, Vt, Eb, trip = V0, False, [], 0.0, None
    for k, tk in enumerate(t):
        P_in = eta*P_mech[k] - P_load
        if chopper:
            on = V > V_on or (on and V > V_off)
        P_r = V*V/Rb if on else 0.0
        V = np.sqrt(max(V*V + 2*(P_in - P_r)*dt/C, 0.0))
        V = max(V, V0)                # PSU output diode: PSU holds 48 V floor
        Eb += P_r*dt; Vt.append(V)
        if trip is None and V >= V_ov: trip = tk
    return np.array(Vt), Eb, trip

Vn, _, trip = run(False)
Vc, Eb, _ = run(True)
print(f"peak mechanical power {P_mech.max():.0f} W, peak bus input {eta*P_mech.max()-P_load:.0f}
W")
print(f"no chopper : overvoltage trip ({V_ov:.0f} V) at {1000*trip:.1f} ms")
print(f"with chopper: peak bus {Vc.max():.1f} V, energy in resistor {Eb:.1f} J, "
      f"resistor peak power {V_on**2/Rb:.0f} W")
```

```
 peak mechanical power 416 W, peak bus input 233 W
no chopper : overvoltage trip (60 V) at 10.9 ms
with chopper: peak bus 56.0 V, energy in resistor 22.4 J, resistor peak power 461 W
```

Without the chopper the bus trips in 10.9 ms, the opening failure. The joints then coast with torque off through about 30 ms of brake-engagement delay, gravity accelerating the downward-moving shoulder, and stop on the brake (about 90 N·m at the output, 45 N·m of it consumed by gravity): 0.63 rad of shoulder travel, 440 mm at the tool, against 0.39 rad (275 mm) planned, the 165 mm overrun. With the chopper the bus holds at 56 V and the resistor absorbs 22.4 J per stop; its 461 W at 56 V is twice the 233 W peak input. A 6.8 Ω, 50 W aluminium-clad resistor with a pulse rating of several hundred joules handles a full-speed stop every 2 s (about 11 W mean).

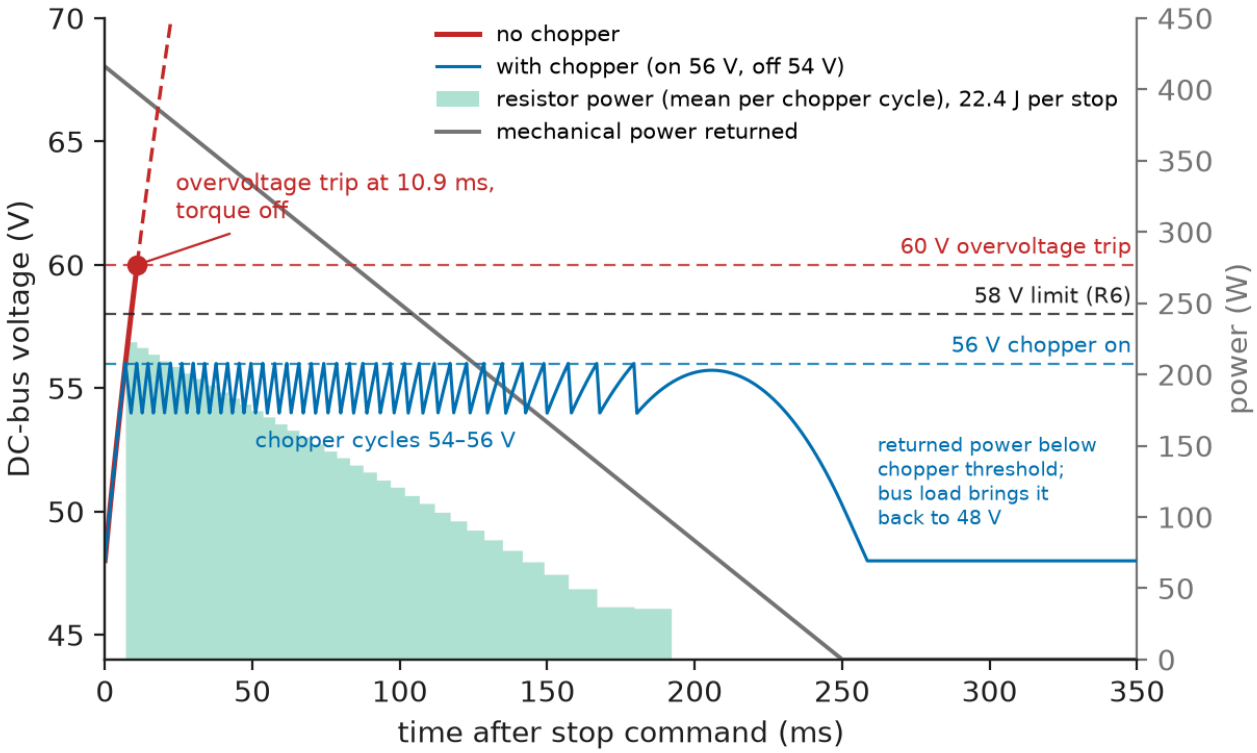


Figure 17.3 — DC-bus voltage during a controlled stop from full speed, without and with a brake chopper.

> **Status of the principle: Bus energy balance during regeneration**
> *Derived.* The bus voltage follows from $\frac{1}{2}C\,dV^2/dt = P_{\text{in}} - P_{\text{out}}$, energy conservation on the capacitor; the result is exact for the stated powers, and its uncertainty lies in the regenerated fraction (60–80 % depending on gear efficiency in back-driving and copper loss at the stop current), so the chopper is sized for the upper value.

Precharge, STO and stop categories

A 10 Ω precharge resistor, bypassed after 0.2 s, limits the bus inrush to 4.8 A. Stops are layered as IEC 60204-1 defines them: **category 1** (controlled deceleration, then STO) for E-stop and protective stops, which needs the bus to survive regeneration, hence the chopper; **category 0** (immediate STO and brakes) only when the drives cannot be trusted. The safety controller forces STO if the arm has not stopped within 0.5 s, bounding the stop even if the main controller fails.

17.7 Communication

> **Boundary.**
> CAN and CAN FD frames, bit timing and arbitration are in *Electronics and Computing*, Ch. 26 §26.5 (link); robot buses in Chapter 15 and industrial networks (EtherCAT, CANopen) in Chapter 21 of this book.

Requirements on the bus

Six joints exchange a setpoint and a status every millisecond (R8). Setpoints must take effect at all joints at the same instant, or the Cartesian path bends; status must arrive before the next cycle; a lost frame must be detected within a few cycles. CAN FD and EtherCAT are the credible candidates; RS-485 with a custom protocol is cheaper but has no arbitration or standard error handling.

Worked example — CAN FD bus load. The listing computes worst-case CAN FD frame durations (11-bit identifier, bit-rate switching, worst-case stuffing, field sizes from ISO 11898-1) for three frame designs at 1 Mbit/s arbitration rate, and compares an EtherCAT frame carrying the same data.

```
 import math
def canfd_us(nbytes, f_nom=1e6, f_data=5e6):
    """Worst-case CAN FD frame time, 11-bit ID, bit-rate switch (ISO 11898-1 field sizes)."""
    nom = 17 + math.ceil(17/4) + 13            # SOF..BRS with stuffing; CRC delimiter..IFS
    crc, fixed = (17, 6) if nbytes <= 16 else (21, 7)
    dyn = 5 + 8*nbytes                          # ESI, DLC, data: dynamic stuffing
    data = dyn + math.ceil(dyn/4) + 4 + crc + fixed
    return 1e6*(nom/f_nom + data/f_data)

designs = {  # name: list of frame payload sizes sent every 1 ms cycle (6 joints)
    "A per-joint 16 B cmd + 16 B status + SYNC": [16]*6 + [16]*6 + [0],
    "B broadcast 48 B cmd + 16 B status":        [48] + [16]*6,
    "C broadcast 48 B cmd + 12 B status":        [48] + [12]*6,
}
for name, frames in designs.items():
    for fd in (5e6, 8e6):
        t = sum(canfd_us(n, f_data=fd) for n in frames)
        print(f"{name:42s} {fd/1e6:.0f} Mbit/s: {t:6.1f} us/cycle, load {t/10:5.1f} %")
# EtherCAT: one frame carrying 6 x (16 + 16) B, 100 Mbit/s, about 1 us forwarding per node
eth = (8 + 14 + 2 + 10 + 6*32 + 2 + 4 + 12)*8/100e6*1e6 + 6*1.0
print(f"{'D EtherCAT, one frame':42s} 100 Mbit/s: {eth:6.1f} us/cycle, load {eth/10:5.1f} %")
```

```
 A per-joint 16 B cmd + 16 B status + SYNC  5 Mbit/s:  927.4 us/cycle, load  92.7 %
A per-joint 16 B cmd + 16 B status + SYNC  8 Mbit/s:  750.2 us/cycle, load  75.0 %
B broadcast 48 B cmd + 16 B status         5 Mbit/s:  581.6 us/cycle, load  58.2 %
B broadcast 48 B cmd + 16 B status         8 Mbit/s:  455.4 us/cycle, load  45.5 %
C broadcast 48 B cmd + 12 B status         5 Mbit/s:  533.6 us/cycle, load  53.4 %
C broadcast 48 B cmd + 12 B status         8 Mbit/s:  425.4 us/cycle, load  42.5 %
D EtherCAT, one frame                      100 Mbit/s:   25.5 us/cycle, load   2.6 %
```

Design A fits in the millisecond but loads the bus to 75–93 %, far over R8's 50 %, because every CAN FD frame spends 35 bits at the slow arbitration rate whatever its payload. One 48-byte broadcast command (B, C) removes six arbitration phases; a 12-byte status (32-bit position, 16-bit velocity, torque and state word, temperature, alive counter) and an 8 Mbit/s data phase bring the worst case to 42.5 %, inside R8.

> **Status of the principle: Worst-case CAN FD frame time**
> *Derived.* The frame time follows from the ISO 11898-1 field lengths and worst-case stuffing (one bit per four); real frames are 5–15 % shorter, so the budget is conservative, provided arbitration runs at 1 Mbit/s over the arm's bus.

Table 17.7 — Communication options for the arm.

Option	Worst-case load at 1 kHz	Sync jitter	Cost per joint (2026)	Fit
CAN FD, design A, 5 Mbit/s	93 %	≈ 5 µs	$2 (transceiver)	Fails R8
CAN FD, design C, 8 Mbit/s, SIC transceivers	**42.5 %**	**≈ 2 µs**	**$3**	**Chosen**
EtherCAT (slave controller via SPI, two ports)	2.6 %	< 1 µs (distributed clocks)	$15–20 plus connectors	Upgrade path for faster cycles or more axes
RS-485, 10 Mbit/s, custom protocol	≈ 30 %	≈ 5 µs	$2	No standard error handling; rejected

8 Mbit/s needs signal-improvement-capability (SIC) transceivers of the TJA1462 or TCAN1462 class, short stubs and terminations only at the ends; the 1.6 m daisy chain through the arm satisfies that.

Synchronisation, timeouts and integrity

The broadcast command is also the synchronisation event: each joint timestamps its reception and applies the setpoint and samples its encoders 150 µs later, so all joints act within a few microseconds. Each frame carries a 4-bit alive counter and an application CRC (end-to-end protection in the style of AUTOSAR E2E), so a stale or replayed setpoint is rejected. Three missing or invalid commands (3 ms) start the joint's controlled stop; the main controller declares a joint lost after 3 ms of missing status. CAN error counters are logged at 10 Hz; a climbing transmit counter flags a damaged cable first.

17.8 Calibration

> **Boundary.**
> Calibration procedures, uncertainty and traceability are owned by Chapter 4 and Chapter 7 of this book; least-squares estimation by *The Mathematics Universe*, Ch. 25 §25.8 (link); forward kinematics by Chapter 13.

What is calibrated, at which level

At the **joint** level, after module assembly: current-sensor offsets and gains, commutation offset, the gear's cyclic transmission-error table and the torque constant on a dynamometer, stored in the joint. At the **arm** level: joint zero offsets and kinematic parameters, base and tool frames. At the **cell** level: tool centre point and fixture frame, by touching reference points.

Kinematic calibration

A laser tracker measures a retroreflector on the flange at a set of poses, and nonlinear least squares adjusts the kinematic parameters until forward kinematics matches. The listing shows the method on the planar J2–J3–J5 sub-chain in the arm's vertical plane: three link lengths, three joint offsets and the base position in the tracker frame, with tracker noise of 0.03 mm, 40 identification poses and 40 different validation poses.

```
 import numpy as np
from scipy.optimize import least_squares
rng = np.random.default_rng(17)
nom = np.array([0.350, 0.300, 0.100, 0, 0, 0, 0, 0])   # a2, a3, a5 (m); dq2, dq3, dq5 (rad); x0,
z0 (m)
true = nom + np.array([0.8e-3, -0.6e-3, 0.4e-3, np.radians(0.60), np.radians(-0.90),
                       np.radians(0.50), 1.5e-3, -2.0e-3])

def fk(p, q):                        # planar J2-J3-J5 chain in the arm's vertical plane
    a, dq, base = p[:3], p[3:6], p[6:8]
    th = np.cumsum(q + dq, axis=1)
    x = base[0] + (a*np.cos(th)).sum(axis=1)
    z = base[1] + (a*np.sin(th)).sum(axis=1)
    return np.column_stack([x, z])

def poses(n):
    return np.column_stack([rng.uniform(-0.3, 1.3, n), rng.uniform(-2.2, -0.3, n),
rng.uniform(-1.5, 1.5, n)])
q_id, q_val = poses(40), poses(40)
meas = lambda q: fk(true, q) + 30e-6*rng.standard_normal((len(q), 2))   # tracker noise 0.03 mm
m_id, m_val = meas(q_id), meas(q_val)

res = least_squares(lambda p: (fk(p, q_id) - m_id).ravel(), nom)
err = lambda p: 1e3*np.linalg.norm(fk(p, q_val) - m_val, axis=1)
for name, p in (("nominal", nom), ("calibrated", res.x)):
    e = err(p); print(f"{name:10s}: validation RMS {np.sqrt(np.mean(e**2)):.3f} mm, max
{e.max():.3f} mm")
J = res.jac; print(f"Jacobian condition number: {np.linalg.cond(J):.0f}")
```

```
 nominal   : validation RMS 1.781 mm, max 2.897 mm
calibrated: validation RMS 0.049 mm, max 0.108 mm
Jacobian condition number: 33
```

Worked example — reading the calibration result. The validation error falls from 1.78 mm to 0.049 mm RMS, and what remains is the tracker noise itself ($0.03 \times \sqrt{2} = 0.042$ mm for a two-coordinate distance). The parameter set is **identifiable**: a base rotation was left out because in a planar chain it is indistinguishable from the J2 offset, and the condition number of 33 confirms that the other eight are well separated by the poses. Identification and validation use different poses, so the result measures prediction, not fit. On the real arm the same procedure over all six joints (24 modified Denavit–Hartenberg parameters, 60 poses) reduces ISO 9283 pose accuracy from 3.1 mm to 0.42 mm; the residual is non-geometric (gear compliance under payload, thermal growth), which a kinematic model cannot remove. Figure 17.4 shows the distributions.

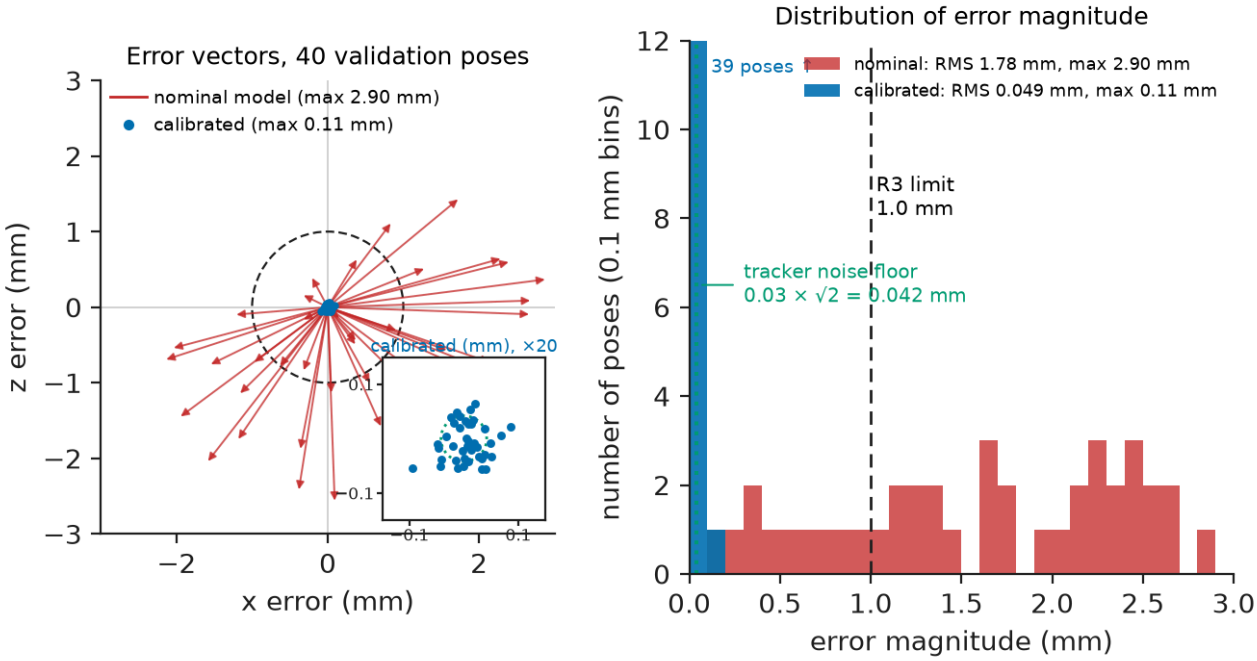


Figure 17.4 — Tool-position error at 40 validation poses before and after kinematic calibration of the J2-J3-J5 chain.

> **Status of the principle: Identifiability of calibration parameters**
> *Derived.* Least squares can estimate a parameter set only if the identification Jacobian has full column rank over the poses; near-dependent parameters show as a large condition number and inflate the estimate variance, so parameter sets are pruned and poses chosen to keep the condition number in the tens, not thousands.

17.9 System identification

> **Boundary.**
> Model structure, step-response and least-squares identification, ARX models and validation are in *The Mathematics Universe*, Ch. 25 §25.7–25.8 and *Circuits, Signals and Communications*, Ch. 33 §33.7 (link); the modelling and validation workflow is owned by Chapter 3 of this book.

What the controller needs to know

Feed-forward needs each joint's inertia, friction and gravity parameters, and loop tuning needs the frequency response with its resonances. Both are measured on the assembled arm, because datasheets do not include the links, cable drag, gear preload or payload.

Rigid-body parameters by least squares

For one joint with the others locked, torque is linear in the parameters: $\tau = J\ddot{q} + b\dot{q} + \tau_c \operatorname{sgn}(\dot{q}) + G\cos q$. The joint follows an **exciting trajectory** (a multisine with incommensurate frequencies, so that acceleration, velocity and position are not collinear), torque is estimated from motor current, and linear least

squares gives the parameters. The simulated shoulder has Stribeck friction (static 9 N·m falling to 6 N·m above about 0.05 rad/s), which the model does not contain.

```
 import numpy as np
rng = np.random.default_rng(170)
fs = 1000.0; t = np.arange(0, 20, 1/fs)
# excitation: multisine shoulder motion about horizontal (rad), six incommensurate tones
f = np.array([0.1, 0.23, 0.37, 0.61, 0.89, 1.3])
q = sum(0.12/(k + 1)*np.sin(2*np.pi*fk*t + k) for k, fk in enumerate(f))
qd = np.gradient(q, 1/fs); qdd = np.gradient(qd, 1/fs)
J, b, tc, ts, vs, G = 4.04, 8.0, 6.0, 9.0, 0.05, 45.1   # "true" joint, Stribeck friction
fric = (tc + (ts - tc)*np.exp(-(qd/vs)**2))*np.sign(qd) + b*qd
tau_meas = J*qdd + fric + G*np.cos(q) + 1.5*rng.standard_normal(t.size)  # from motor current

def fit(mask):
    Y = np.column_stack([qdd, qd, np.sign(qd), np.cos(q)])[mask]
    th, *_ = np.linalg.lstsq(Y, tau_meas[mask], rcond=None)
    r = tau_meas[mask] - Y @ th
    se = np.sqrt(np.diag(np.var(r)*np.linalg.inv(Y.T @ Y)))
    return th, se
for label, mask in (("all samples", np.ones(t.size, bool)),
                    ("|qd| > 0.15 rad/s", np.abs(qd) > 0.15)):
    th, se = fit(mask)
    print(f"{label:18s} J={th[0]:.3f}+-{se[0]:.3f}  b={th[1]:.2f}+-{se[1]:.2f}  "
          f"tau_c={th[2]:.2f}+-{se[2]:.2f}  G={th[3]:.2f}+-{se[3]:.2f}")
print(f"true               J={J}  b={b}  tau_c={tc} (static {ts})  G={G}")
```

```
 all samples        J=4.036+-0.010  b=3.07+-0.10  tau_c=7.27+-0.02  G=45.07+-0.01
|qd| > 0.15 rad/s  J=4.043+-0.015  b=7.82+-0.19  tau_c=6.07+-0.05  G=45.12+-0.02
true               J=4.04  b=8.0  tau_c=6.0 (static 9.0)  G=45.1
```

Worked example — a precise but wrong friction estimate. Fitted to all samples, the viscous coefficient is 3.07 ± 0.10 N·m·s/rad against a true 8.0: good to 3 % by its standard error, wrong by 62 %. Low-velocity samples, where Stribeck friction lifts the torque, pull the Coulomb term up (7.27 instead of 6.0) and the viscous slope down. Fitting only above 0.15 rad/s, where the structure is valid, recovers $b = 7.82 \pm 0.19$ and $\tau_c = 6.07$; inertia and gravity are good either way. So the friction model for feed-forward and collision detection is fitted only where it is valid, with a separate low-speed table, and every identification is validated on a different trajectory, where a structural error shows as a residual correlated with velocity.

Frequency response and resonances

The velocity loop is tuned on a measured frequency response: with a low-gain velocity loop stabilising the joint, a 1–100 Hz chirp is added to the torque command. Figure 17.5 shows the shoulder at full reach, with the anti-resonance near 15 Hz and resonance near 27 Hz of the two-mass prediction (Section 17.2) matched within 10 %; folded and unloaded, the anti-resonance moves to about 32 Hz. The velocity-loop notch is therefore scheduled with pose, and the 5 Hz position bandwidth is fixed by the worst case.

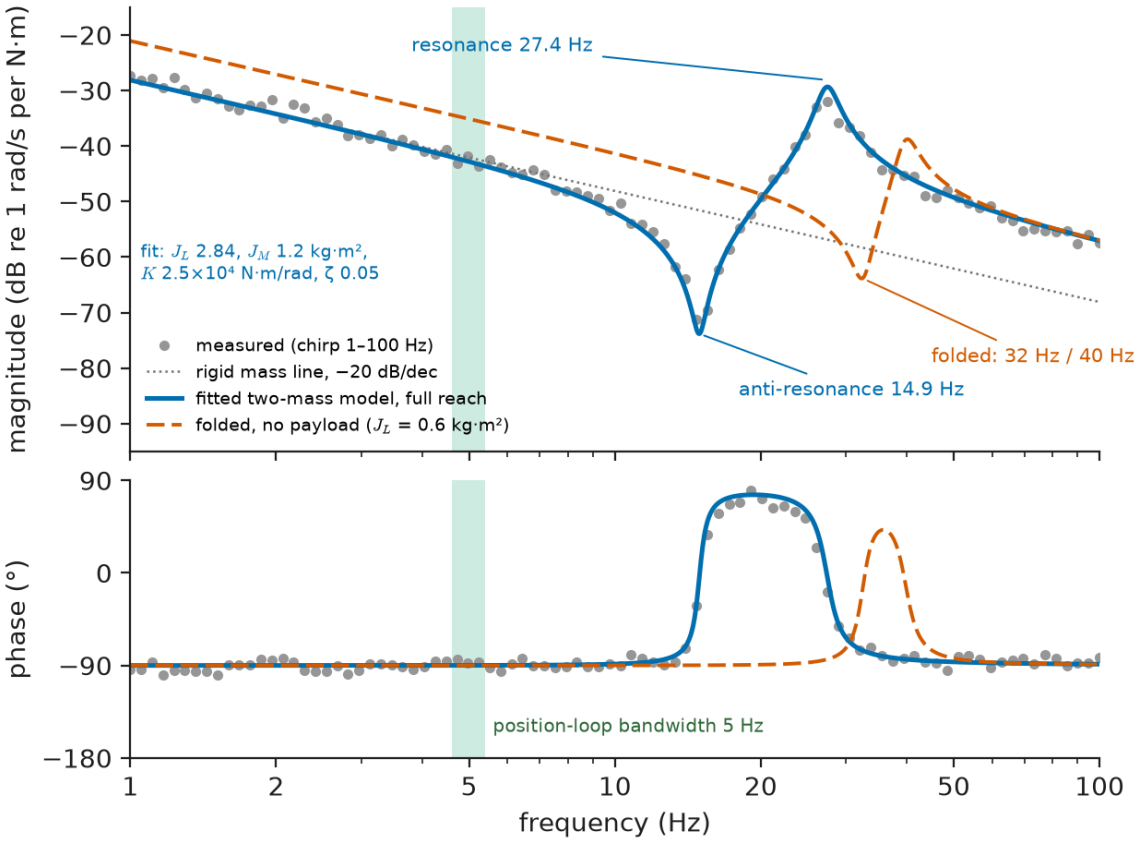


Figure 17.5 — Frequency response of the shoulder from torque command to motor-side velocity at full reach, with the fitted two-mass model.

17.10 Hardware-in-the-loop testing

> **Boundary.**
> The HIL concept, simulator fidelity and the delay-as-phase-lag rule are in *Electronics and Computing*, Ch. 28 §28.9 (link); regression testing as a method is owned by Chapter 4 of this book; co-simulation and digital twins by Chapter 33.

Two benches for two speeds of physics

The arm's physics spans 20 kHz current loops, 15–30 Hz resonances and multi-second programs, so the plan uses two benches (Figure 17.6). On the **joint dynamometer** a load motor emulates the link (virtual inertia, gravity torque, compliance) to test the fast loops, heating, regeneration into a programmable source/sink and the joint's fault reactions under real power. The **system HIL** connects the real main and safety controllers through the real CAN FD bus to a real-time target that emulates six joints' bus behaviour and integrates the arm's rigid-body dynamics at 1 ms, testing trajectories, feed-forward, timing and system-level fault reactions without risking the arm.

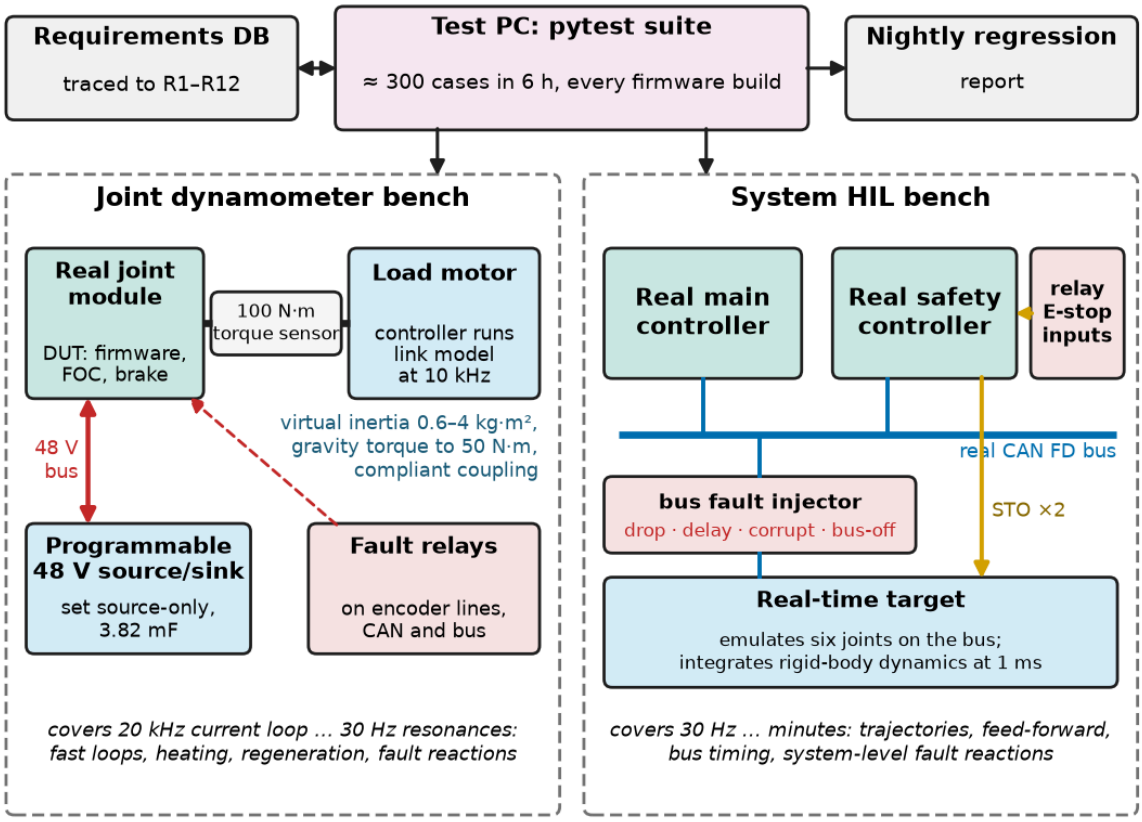


Figure 17.6 — The two hardware-in-the-loop benches of the arm program.

Fidelity

The system HIL integrates the arm at 1 ms behind up to 1 ms of bus latency; 2 ms of delay costs $360 \times 5 \times 0.002 = 3.6^\circ$ at the 5 Hz position bandwidth, negligible, but about 19° at the 27 Hz resonance, so the resonance and notch are tested on the dynamometer. Matching each bench to the physics it must reproduce is the first HIL design decision.

Fault injection and the regression suite

Table 17.8 — Representative HIL test cases with pass limits.

Test	Bench	Injection	Pass limit
Communication loss	System	Drop all commands to J3 from t = 0	J3 detects in 3 ms; all joints in controlled stop; arm stopped in ≤ 0.30 s
Bus-off of one joint	System	Force J5 to bus-off	Main controller declares J5 lost in ≤ 3 ms; category 1 stop of the arm
Encoder disagreement	Dyno	Offset output encoder by 1°	Fault in ≤ 1 ms; torque off; brake
Overvoltage	Dyno	Source/sink drives bus to 59 V during motion	Chopper holds ≤ 58 V; no trip
Category 1 stop at full speed	Dyno	E-stop input at 180 °/s with 4 kg·m² virtual load	Stop ≤ 0.30 s, bus ≤ 58 V (R6)
Parameter mismatch	System	Load J2 parameter set into J3	Arm refuses to enable; diagnostic message

Every firmware build runs the suite unattended overnight (about 300 cases in 6 hours), traced to Table 17.1. The full-speed stop case was added after the opening failure, and the dynamometer supply was reconfigured to behave like the arm's bus (source only, 3.82 mF) instead of a laboratory supply, the bench assumption that had hidden the problem.

17.11 Field testing

From lab to line

Field tests answer what HIL cannot: whether the whole machine, with flexing cables, real heat and people nearby, keeps meeting its requirements: **ISO 9283** pose and path tests (five poses of the test cube, 30 cycles, laser tracker; R2, R3, R5); **collision-force tests** with a device having the ISO/TS 15066 hand spring constant of 75 N/mm at three sites (R9); **endurance** of 100,000 standard cycles (about 35 h) with temperatures, CAN error counters, encoder CRC failures and friction drift logged (R10); **thermal soak and EMC** (R11, R12); and a two-week **pilot** in a customer's cell with logging on.

Verification

Table 17.9 — Verification of the arm (build 2, after the changes of this chapter).

Req.	Test	Limit	Result	Pass/fail
R1	Static and thermal run with 3.0 kg at 700 mm	No fault, R11 met	Pass, shoulder winding 92 °C	Pass
R2	ISO 9283 RP, 5 poses × 30 cycles	≤ ±0.05 mm	±0.028 mm	Pass
R3	ISO 9283 AP after calibration	≤ 1.0 mm	0.42 mm (3.1 mm before)	Pass
R4	Logged maximum joint speeds	180 / 360 °/s	180 / 360 °/s	Pass
R5	ISO 9283 AT at 250 mm/s	≤ 0.5 mm	0.31 mm	Pass
R6	Category 1 stops, 20 at full speed	≤ 0.30 s, ≤ 58 V, no trip	0.26 s, 56.3 V max, no trip	Pass
R7	Power analyser, standard cycle	≤ 250 W mean	176 W ± 3 W	Pass
R8	Bus analyser and timing log, 24 h	jitter ≤ 20 µs, load ≤ 50 %	jitter 4 µs; load 38 % mean	Pass
R9	ISO/TS 15066 force measurement, 3 sites	≤ 140 N quasi-static	96–118 N	Pass
R10	100,000-cycle endurance	No fault	1 fault at 61,200 cycles (cable); pass after fix, rerun	Pass (after rerun)
R11	Thermocouples, standard cycle, 25 °C	≤ 60 °C	48 °C shoulder housing	Pass
R12	Accredited EMC lab	IEC 61000-6-2/-6-4	Pass after shield-bond fix at J4	Pass

Two results needed changes. The endurance fault at 61,200 cycles is analysed in Section 17.12. The EMC burst test reset J5 at 2 kV until a missing 360° shield bond at the J4 cable gland, which let burst current flow on the CAN ground, was added.

17.12 Failure analysis and the second iteration

Table 17.10 — FMEA of the integrated arm (top items).

Failure mode	Effect	Cause	Detection	Mitigation
Bus overvoltage on stop	All joints trip; arm overruns by 165 mm on brakes	Regenerated energy into 3.82 mF; bench supply hid it	Bus voltage log; HIL stop test	Brake chopper at 56 V; dyno source/sink set to source-only
Cable fatigue in torsion joint	Intermittent CAN errors, then joint lost	Cable twisted beyond its rated angle by a routing clip	CAN error counters, alive-counter gaps	Torsion-rated cable, clip removed, torsion length set to 25 × diameter
Friction model error	False collision trips or insensitive detection	Stribeck friction outside the fitted model	Residual statistics in log	Fit only where valid; low-speed table; per-joint thresholds
Gear compliance under payload	Pose error beyond geometric calibration	Strain-wave stiffness at low load	Tracker validation	Compliance compensation from torque estimate (Revision B)
Joint parameter mix-up after repair	Wrong offsets; collision	Calibration stored outside the joint	Version/CRC check at INIT	Calibration in joint flash with serial number
Encoder EMI	Position jumps; fault	PWM coupling into single-ended encoder link	CRC failure counter	Differential BiSS-C link with CRC

The cable story

At about 60,000 endurance cycles J4 began logging bursts of CAN error frames; its transmit error counter twice reached error-passive, and once the main controller declared it lost. The counters pointed to the segment between J3 and J4, and resistance measured while flexing the cable showed one CAN conductor opening intermittently. The cable was rated for ±360° torsion over a free length of at least 20 diameters, but a clip added during assembly had halved the free length and doubled the shear strain in the conductors. The fix was mechanical (remove the clip, put the torsion length on the drawing and in the assembly check); the detection was firmware (counters logged at 10 Hz); the requirement that caught it was R10, an endurance test with telemetry left on.

Revision B

Change	Reason (trace)
Brake chopper and bus monitor on the base board	R6; opening failure
Torsion length and clip positions specified on the drawing; bond-resistance test in assembly	R10, R12; cable and EMC failures
Compliance compensation using the dual-encoder torque estimate	R3 residual of 0.42 mm is dominated by gear compliance
Evaluate EtherCAT	Headroom (2.6 % load) for faster cycles and added axes
Cycloidal gear for J1	Higher stiffness raises the lowest anti-resonance and the position-loop bandwidth

---

What remains open

Open question	Where it is answered
How are test plans, calibration procedures and regression organised as a method?	Chapter 4
How are robot kinematics and actuator choice done for a new arm?	Chapter 13
How do industrial networks (EtherCAT, CANopen) and functional safety work at plant level?	Chapters 18 and 21
How are E-stops, interlocks and stop categories engineered in a machine panel?	Chapter 20
How is a robot model calibrated against logged data and run as a digital twin?	Chapter 33
Drive regeneration and braking theory	*Electrical Energy and Machines*, Ch. 10 §10.4–10.5

Next: Chapter 18 leaves the robot cell for the factory floor and its control architecture, the PLCs, remote I/O, DCS and SCADA systems that coordinate many machines like this arm.

Part V — Industrial Automation

Factories, water works, quarries and process plants run on a layer of engineering the theory books mention only in passing: the controller that scans its inputs every 10 ms, the I/O drop that goes to a designed safe state when its network falls silent, the 4–20 mA loop whose voltage budget decides whether a reading is true, the drive that must stop a belt within 4 s, and the networks and safety systems that tie them together. This part designs that layer. It follows four real systems from requirement to commissioning: the control system of a 30 ML/day water-treatment works, the instruments and actuators of a dairy clean-in-place skid, the motor control of a quarry conveyor line, and the networks and safety functions of a palletising cell on a beverage line. The theory is taken from the books that own it; this part adds the architecture, the budgets, the real parts, the PLC code, the tests with their limits, and the failures each design had to survive.

By the end of this part you should understand

- How an industrial control system is layered and partitioned into PLCs, remote I/O, SCADA, HMIs and networks, and when a DCS is the better answer.
- How to budget a PLC's response time, a telemetry network's data age, a redundant controller's availability and a panel's 24 V and heat loads, each with a margin.
- How proximity sensors, limit switches, RTDs, pressure transmitters, flowmeters, solenoid valves, contactors, relays and actuators are chosen and interfaced, and why 4–20 mA loop budgets, sensing distances and coil suppression matter.
- How motors are started, protected, speed-controlled, interlocked and stopped, and how starter type, current limit, trip class and supply dip are decided together.
- How emergency stops are built as independent functions with stop categories, safety relays and safe torque off, and validated by measured stopping times.
- How field networks are sized and diagnosed, how safety functions reach a required performance level, and how a plant is secured and documented.
- How commissioning (FAT, SAT, loop checks, E-stop validation) turns a correctly built system into a trusted one.

Chapters

Chapter	Central question	What it delivers
18 — Industrial Control Architecture	How is an industrial control system partitioned, sized and proven before it is trusted with a plant?	PLC response-time budgets, I/O lists and watchdogs, PLC plus SCADA versus DCS and redundancy, telemetry budgets, alarm KPIs, ring redundancy, panel 24 V and heat budgets, FAT/SAT and loop checks
19 — Industrial Sensors and Actuators	How is each sensor and actuator chosen and interfaced to a PLC without losing accuracy, speed or fail-safety?	Sensing distances and input types, positive-opening switches, an RTD uncertainty budget, 4–20 mA loop budgets, flowmeter selection, coil suppression, contactor and relay life, pneumatic force and air budgets, electric actuators
20 — Industrial Motor Control	How is each motor of a plant started, protected, speed-controlled, interlocked and stopped, and how is the line proved?	Starter selection against supply dip, trip class against hot restarts, VFD duty and braking, a feed-rate loop, conveyor sequences, category 1 E-stops with STO and measured stopping times, MCC design, commissioning
21 — Industrial Communication and Safety	How are field networks chosen, sized and diagnosed, and how is a safety function specified, proven and kept working for ten years?	Fieldbus timing budgets, Modbus, CANopen and industrial Ethernet, network diagnostics, ISO 13849-1 performance levels, safety distances, black-channel safety, IEC 62443 zones, maintenance documentation

How this part connects to the rest of the book

Part V applies the method of Part I to the industrial world: the requirement tables, architecture diagrams, budgets and FMEAs of Chapters 1–5 appear in every chapter here. It builds on the sensors, signal conditioning and acquisition of Part II (Chapters 6–9), which own sensing principles and uncertainty, and on the embedded controllers of Part III. Its motor control is applied end to end in the closed-loop motor drive of Chapter 37; its networks and security reappear in the wireless sensor network of Chapter 38, and its plant models in the digital twins of Chapter 33.

Among the sibling books, *Electrical Energy and Machines* supplies motor starting (Ch. 6), drive fundamentals, protection and safety functions (Ch. 10), AC drive control and commissioning (Ch. 12) and electrical safety and installation design (Ch. 29, Ch. 30). *Electronics and Computing* supplies the 4–20 mA transmitter circuit (Ch. 9), embedded interfaces and buses (Ch. 26) and the VFD's power stage (Ch. 32). *Circuits, Signals and Communications* supplies PID design, tuning and digital control (Ch. 35), and *Engineering Physics* the thermoelectric and fluid physics behind industrial sensors (Ch. 14, Ch. 19).

Chapter 18 — Industrial Control Architecture

Part V · Industrial Automation

At 10:05 on a weekday morning a mini-digger laying a new main cuts a fibre-optic cable between two buildings of a 30 ML/day water-treatment works. The control network is a redundant ring, designed so that any single cable can be cut without effect, and within a fifth of a second it has healed. By then every valve and pump in the filter gallery has stopped, and the hypochlorite dosing pumps in the chemical building, on a drop that never lost communication, are still dosing into a contact tank whose flow has just fallen to zero. The chlorine residual leaving the works takes six hours to return to specification. The ring recovered in its guaranteed 200 ms; the filter I/O drops, still at the factory watchdog of 12 ms, gave up long before that and set their outputs to substitute values; and nothing tied the dosing to the flow it doses. The architecture was wrong, because nobody had decided how the network, the watchdogs and the substitute values work together, or what the plant should do when a drop goes quiet. This chapter is about those decisions.

> **How is an industrial control system partitioned into controllers, I/O, networks, operator stations and supervisory software, and how is each part sized?**
> **How is the assembled system shown to work, at the factory and on site, before it is trusted with the plant?**

Concept	Engineering destination
Industrial control systems	ISA-95 levels, process versus discrete versus batch, requirements
PLCs	Controller class, response-time budgets, IEC 61131-3 sequences
Remote I/O	I/O lists, spare capacity, watchdogs and substitute values
Distributed control systems	When a DCS beats PLC plus SCADA; redundancy and availability
Supervisory control and data acquisition	Telemetry budgets, polling versus report by exception, historians
Human-machine interfaces	ISA-101 screens, ISA-18.2 alarm KPIs
Industrial networking	Ring redundancy versus watchdogs, traffic rules
Control panels	24 V DC and heat budgets, IEC 61439 construction
System commissioning	FAT, SAT, loop checks, verification and FMEA

Roadmap. The system and its requirements (18.1) → the controller and its timing (18.2) → bringing I/O to the controller (18.3) → when a DCS is the right answer, and redundancy (18.4) → supervising sites kilometres away (18.5) → the operator's screen and alarms (18.6) → the networks that tie it together (18.7) → the panels it lives in (18.8) → proving it before handover (18.9) → failure analysis and what remains open.

18.1 Industrial control systems

The reference system

The running example of this chapter is the control system of a **30 ML/day surface-water treatment works** serving about 120 000 people, with eight remote sites (two raw-water intakes and six service reservoirs or booster stations) up to 25 km away. Its process trains are a raw-water intake with three 110 kW pumps on variable-frequency drives; coagulant, lime and hypochlorite dosing paced to flow; six rapid gravity filters, each with inlet, modulating outlet, drain, air-scour and backwash valves and a backwash sequence every 24–72 h; a backwash plant of two wash-water pumps and two air-scour blowers; a chlorine contact tank and three 160 kW high-lift pumps on VFDs feeding the trunk main; and site utilities with a standby generator.

The works is staffed 08:00–16:00 and runs unattended at night, with an on-call operator who can reach it in about an hour. That one fact, together with the population served, drives most of the architecture.

The control hierarchy

Every industrial control system, whatever the vendor, is arranged in the levels of ISA-95 / IEC 62264, which the zone model of IEC 62443 (Chapter 21) also follows: **Level 0** the process (sensors, valves, motors; Chapters 19 and 20); **Level 1** basic control (PLCs, DCS controllers, remote I/O, drives, safety controllers), with cycles of 1–100 ms; **Level 2** supervision (HMIs, SCADA, alarm servers), seconds; **Level 3** site operations (historian, maintenance systems), minutes to hours; **Level 4** the enterprise network, hours to days.

The design rule that follows is simple and frequently broken: **each level must survive the loss of every level above it**. A filter must keep filtering when the SCADA server reboots; the high-lift pumps must keep pumping when the corporate network is down. Figure 18.1 shows the architecture chosen for the reference works, which Sections 18.2–18.8 justify piece by piece.

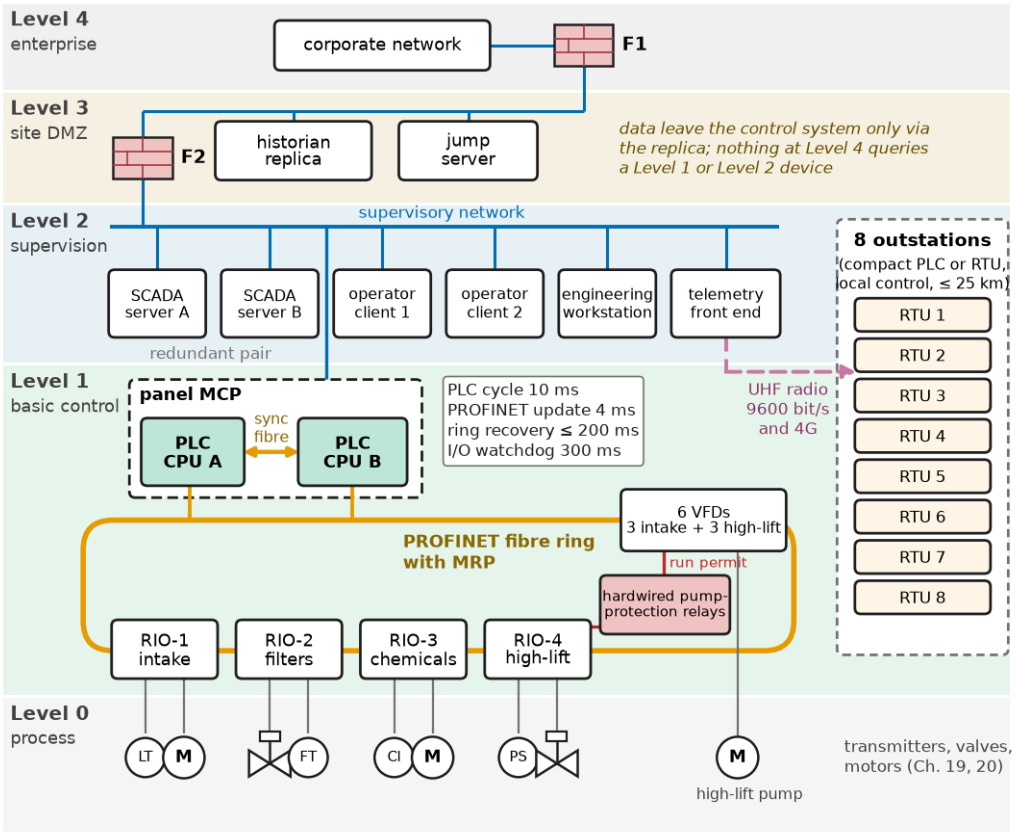


Figure 18.1 — Control architecture of the 30 ML/day water-treatment works, arranged by ISA-95 level.

Requirements

Table 18.1 — Requirements for the reference control system.

ID	Requirement	Target	Verification
R1	Loss of automatic control of high-lift pumping	≤ 1 h per year (predicted)	Reliability calculation (§18.4); switchover test at SAT
R2	Pump protective trip (low suction pressure) response	≤ 200 ms sensor to contactor or STO	Timed test with calibrator step and scope (§18.9)
R3	Filter level control	±50 mm steady state, no valve hunting	Trend at SAT over 24 h
R4	Outstation data age at control room	≤ 10 s analogues, ≤ 5 s alarms	Telemetry budget (§18.5); timed alarm test
R5	Spare capacity	≥ 20 % installed spare I/O per type, ≥ 20 % rack space, ≥ 30 % CPU cycle headroom	I/O schedule and CPU load report at FAT
R6	Alarm load on the operator	Average ≤ 1 per 10 min; ≤ 1 % of 10-min periods above 10 alarms	Alarm KPI report after 30 days (§18.6)
R7	Each level keeps running if all higher levels fail	No process interruption	Disconnection tests at FAT and SAT
R8	Panel internal temperature	≤ 45 °C at 40 °C ambient	Heat-run test, thermal calculation (§18.8)
R9	24 V DC control supply	Survives loss of one PSU; ≥ 30 min on UPS	PSU pull test at FAT
R10	Commissioning evidence	Every I/O point, interlock and sequence tested and signed at FAT and SAT	Test records to IEC 62381 (§18.9)

R1 comes from the hydraulic design, not from a brochure: the service reservoirs hold about 6 h of demand at night, the on-call operator needs about 1 h to arrive and a further hour to run the plant by hand, so the system can tolerate occasional outages of a few hours but must not make them routine. R2 comes from the pump supplier: a high-lift pump that loses suction pressure cavitates, and the supplier's warranty demands a trip within 0.5 s. The control system is given 200 ms of that.

Process, discrete and batch control

Industrial control falls into recognisable families. **Discrete manufacturing** (packaging lines, assembly cells, the conveyors of Chapter 20) is mostly digital I/O, interlocks and motion at 1–10 ms cycles, on PLCs and motion controllers. **Continuous process** (water, power, refining, chemicals) is 30–50 % analogue, dominated by PID loops, cascades and ratio control at 0.1–1 s per loop, on PLC plus SCADA or a DCS. **Batch** (pharmaceuticals, food) adds recipes and phases to ISA-88, usually on a DCS with a batch manager. **Infrastructure** (pipelines, water networks, grids) has few points per site but many sites, connected by telemetry. The reference works is a continuous process with sequences in it (the backwash) on site and infrastructure across its catchment: the natural territory of **PLC plus SCADA**, and Section 18.4 checks the alternative.

> **Boundary.**
> PID design and tuning are owned by *Circuits, Signals and Communications*, Ch. 35 §35.2–35.3; requirements and architecture method by Chapter 1 and Chapter 2, applied here to industrial control.

18.2 PLCs

What a PLC is, from the outside

A **programmable logic controller** is a hardened computer with a fixed execution model. It reads all its inputs into an image table, executes the user program once from top to bottom using that image, writes the output image to the outputs, performs communication and diagnostics, and repeats. On top of the cyclic program, modern PLCs run **cyclic interrupt** tasks at fixed periods (typically 1–1000 ms, used for PID loops so that their sample time is constant) and **event** tasks for hardware interrupts and diagnostics.

Programs are written in the IEC 61131-3 languages (Ladder, Function Block Diagram, Structured Text, Sequential Function Chart). A sensible convention is LD or FBD for interlocks that maintenance electricians must read at 3 a.m., ST for calculations and sequences, and vendor PID function blocks for loops.

> **Boundary.**
> Hard and firm real-time deadlines, scan-based versus preemptive scheduling and watchdogs are treated in *Electronics and Computing*, Ch. 27 §27.1 and §27.11, which lists the PLC I/O scan at 1–10 ms among its examples.

Response time of a scanned controller

Because inputs are sampled once per scan, an input change that arrives just after the input image was read waits almost a full scan before the program sees it, and the program's reaction is written at the end of the following scan. With distributed I/O on a cyclic network, the network update period adds in both directions. The worst-case response from a field input to a field output is

$$t_{resp,max} = t_{in} + T_{net} + 2\,T_{scan} + T_{net} + t_{out}$$

where t_{in} is the input module's filter delay, T_{net} the I/O network update time, T_{scan} the PLC cycle time and t_{out} the output module delay. The typical response is roughly half the scan and network terms.

> **Status of the principle: Worst-case response of a cyclic controller**
> *Derived.* Follows from sampling each input once per cycle and writing each output once per cycle, for unsynchronised input, network and program cycles; isochronous (clock-synchronised) I/O replaces the sum of independent worst cases with a fixed, shorter latency.

Worked example — does the pump trip meet R2? The low-suction-pressure trip of a high-lift pump is a pressure switch wired to remote I/O drop RIO-4; the trip output is a digital output that opens the drive's run command.

- Input filter of the DI module: 3.2 ms (the factory default; it can be set from 0.1 to 20 ms).

- PROFINET update time: 4 ms in each direction.
- PLC cycle: the main program is estimated at 6 ms, so the cycle is fixed at 10 ms.
- Transistor output delay: 0.5 ms.

Worst case: $3.2 + 4 + 2 \times 10 + 4 + 0.5 = 31.7$ ms. Typical: $3.2 + 2 + 10 + 2 + 0.5 = 17.7$ ms. Then the drive must act on the removed run command: a VFD in coast-stop mode removes torque within about 10–20 ms. The total, about 50 ms worst case, is a quarter of R2's 200 ms. The margin is large, so the trip stays in the standard PLC; Section 18.7 adds the condition this budget hides, which is that the network must actually deliver every frame on time. A trip whose budget were within a factor of two of its limit would be hardwired instead. Figure 18.2 draws the chain on one time axis.

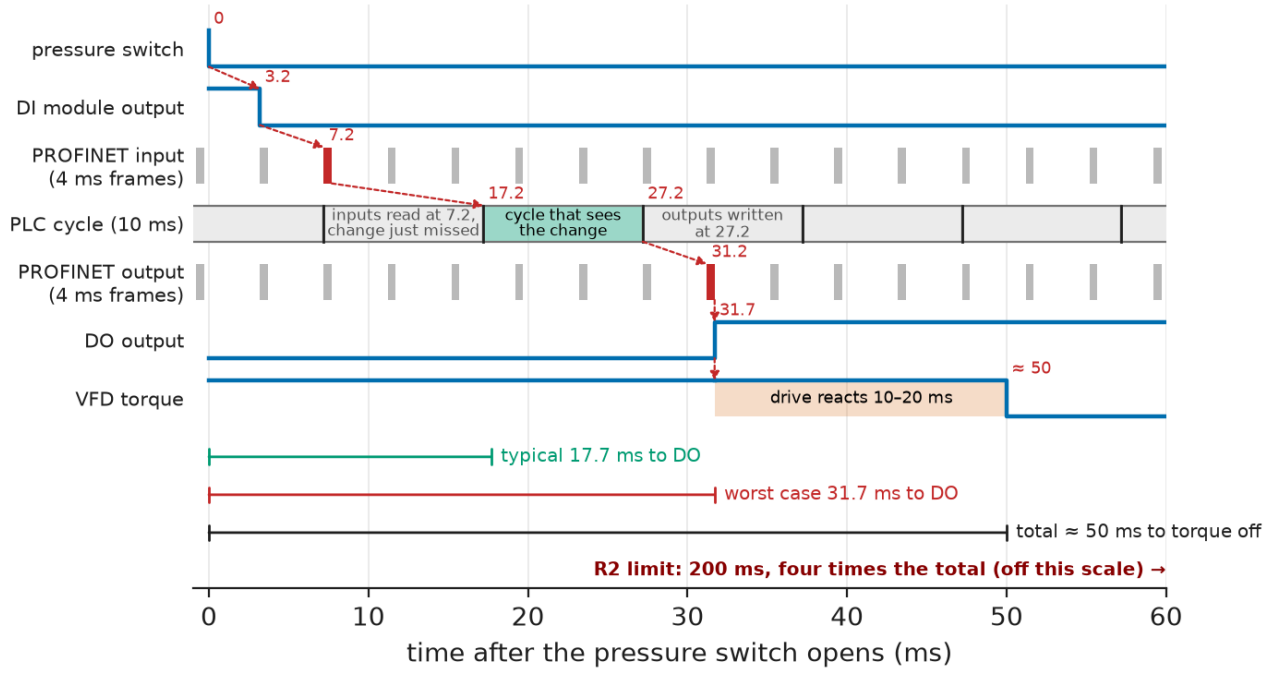


Figure 18.2 — Worst-case response of the high-lift pump trip through remote I/O and the PLC scan.

Choosing the controller class

Table 18.2 — Controller classes for the reference works (2026 indicative prices, CPU only).

Class	Examples of the class	I/O capacity	Redundancy	Typical price	Verdict for the works
Compact PLC	S7-1200, CompactLogix 5380 small, Modicon M241 class	100–1 000	None	$400–$1 200	Outstations only
Modular PLC	S7-1500, ControlLogix 5580, Modicon M580 class	1 000–10 000	Optional in some families	$1 500–$5 000	Meets everything except R1 without redundancy
Redundant PLC pair	S7-1500R/H, ControlLogix redundancy, M580 Hot Standby class	1 000–10 000	Hot standby, bumpless switchover in 50–300 ms	$12 000–$25 000 per pair	**Chosen** for the main controller (R1)
PC-based / soft PLC	CODESYS or TwinCAT runtime on an industrial PC	Large	Possible but less common	$1 000–$3 000 plus licence	Strong for motion; weaker 20-year support story
DCS controller	Process controllers of a DCS family	500–5 000 per controller	Standard	Priced per system	Evaluated in §18.4

R1, quantified in Section 18.4, decides the main controller; the outstations get compact PLCs or RTUs with telemetry protocols built in. Above all, the choice is constrained by what the maintenance staff already know: a water company with two hundred sites on one PLC family will not thank a designer for a third.

A sequence in Structured Text

The filter backwash shows the pattern every robust PLC sequence follows: a step number held in one variable, transitions that require both a timer *and* proven feedback, a watchdog on every step that waits for the field, a single safe state, and outputs decoded from the step number so that no output can be left on by a forgotten assignment. Listing 18.1 is the core of the function block in IEC 61131-3 Structured Text; scheduling, operator holds and the filter-to-waste step are omitted.

```
 FUNCTION_BLOCK FB_FilterBackwash
VAR_INPUT
    xStart, xPermissive, xAbort, xReset : BOOL;
    rLevel_m : REAL;                                  (* water above media, m *)
    xInletClosed, xAirOpen, xWashOpen   : BOOL;       (* limit-switch feedback *)
END_VAR
VAR_OUTPUT
    xInlet, xDrain, xAir, xWash, xBlower, xWashPump, xFault : BOOL;
    iStep : INT := 0;
END_VAR
VAR
    iPrev : INT := -1;
    tStep : TON;                                      (* time in current step *)
END_VAR

tStep(IN := (iStep = iPrev), PT := T#1H);             (* restarts on step change *)
iPrev := iStep;
IF xAbort AND iStep <> 0 THEN iStep := 90; END_IF;

CASE iStep OF
  0:  IF xStart AND xPermissive THEN iStep := 10; END_IF;          (* filtering *)
  10: IF xInletClosed AND rLevel_m < 0.2 THEN iStep := 20;         (* drain down *)
      ELSIF tStep.ET > T#20M THEN xFault := TRUE; iStep := 90; END_IF;
  20: IF NOT xAirOpen AND tStep.ET > T#60S THEN xFault := TRUE; iStep := 90;
      ELSIF tStep.ET >= T#3M THEN iStep := 30; END_IF;              (* air scour *)
  30: IF NOT xWashOpen AND tStep.ET > T#60S THEN xFault := TRUE; iStep := 90;
      ELSIF tStep.ET >= T#2M THEN iStep := 40; END_IF;              (* air + water *)
  40: IF tStep.ET >= T#6M THEN iStep := 50; END_IF;                 (* high-rate wash *)
  50: IF rLevel_m > 1.5 THEN iStep := 0;                            (* refill *)
      ELSIF tStep.ET > T#30M THEN xFault := TRUE; iStep := 90; END_IF;
  90: IF xReset AND NOT xAbort THEN xFault := FALSE; iStep := 0; END_IF;  (* safe *)
END_CASE;

xInlet    := (iStep = 0) OR (iStep = 50);
xDrain    := (iStep = 10);
xAir      := (iStep = 20) OR (iStep = 30);
xWash     := (iStep = 30) OR (iStep = 40);
xBlower   := xAir AND xAirOpen;          (* never run the blower against a shut valve *)
xWashPump := xWash AND xWashOpen;
END_FUNCTION_BLOCK
```

The timer is a stopwatch, reset on the one scan in which the step changes. The safe state (step 90) closes every valve and stops both machines, because every output is decoded as FALSE there; a fault therefore leaves the filter isolated and drained rather than half-washed. The blower and wash pump each require proven-open feedback from their valve, which protects the pipework if a valve sticks. A scheduler never allows two of the six filters to wash at once: the wash-water tank holds one wash.

18.3 Remote I/O

The I/O list comes first

Every design starts from the **I/O list**: one line per field signal with tag, type, range, location and drawing. Controller, modules, panels, cabling and test plan are all counted from it. For the reference works it summarises as Table 18.3.

Table 18.3 — I/O summary of the reference works.

Area	DI	DO	AI	AO
Intake (3 raw-water pumps)	36	12	18	3
Chemical dosing	40	20	24	8
Filters (6 cells)	84	48	24	6
Backwash pumps and blowers	24	8	8	2
Clear water and high-lift	42	12	21	3
Utilities and building	30	10	8	0
Total (491 points)	**256**	**110**	**103**	**22**

The six VFDs appear only through their hardwired run-permit and STO circuits; reference, status and diagnostics come over PROFINET, saving about 15 points per drive (Chapter 20).

Worked example — module count with spare capacity. R5 asks for at least 20 % installed spare per signal type. With 16-channel DI and DO modules, 8-channel AI and 4-channel AO modules:

- DI: $256 \times 1.2 = 307.2$, so 20 modules (320 channels), giving 25 % spare.
- DO: $110 \times 1.2 = 132$, so 9 modules (144 channels), 31 % spare.
- AI: $103 \times 1.2 = 123.6$, so 16 modules (128 channels), 24 % spare.
- AO: $22 \times 1.2 = 26.4$, so 7 modules (28 channels), 27 % spare.

That is 52 I/O modules, split over the main panel and four drops, each rack then left with at least 20 % empty slots. The spare is not generosity: I/O counts typically grow 10–15 % between design and handover, and every point without a spare channel costs a module, a panel modification and a retest.

> **Status of the principle: 20 % installed spare, 20 % space**
> *Empirical.* A planning rule from project practice and utility specifications, covering I/O growth before handover and later modifications; it is a contractual number, not a physical one.

Central versus distributed I/O

Two ways exist to bring 491 signals to one controller: run every field cable to a marshalling cabinet beside the controller (**central I/O**), or put I/O modules in panels near each process area and connect them to the controller over industrial Ethernet (**distributed or remote I/O**), so that one fibre pair replaces hundreds of cores.

Table 18.4 — I/O distribution options for the reference works (indicative 2026 installed costs).

Criterion	Central marshalling	Remote I/O drops on PROFINET (chosen)	IO-Link / field-mounted I/O blocks
Field cable length (average per signal)	About 180 m	About 35 m	About 10 m to the block, network beyond
Copper cable and installation	About $95 000	About $30 000	About $20 000
Panels and network hardware	One large marshalling room	Four drop panels and fibre ring: about +$45 000	Many IP67 blocks: about +$35 000
Lightning and earth-potential exposure	High: long cables between buildings	Low: fibre between buildings	Low
Loss of one network link	Not applicable	One drop to safe state, unless ring (§18.7)	One block or segment

The comparison is decided less by cable cost, which roughly cancels against panel cost, than by the separate buildings of a water works: copper multicores between buildings with separate earths are the classic route for lightning surges into I/O cards, and fibre removes it.

Watchdogs and the safe state

Every remote I/O drop runs a **watchdog**: if it receives no valid output frame from its controller within the watchdog time, usually 3 to 10 update cycles (12 ms by default here, 3 × 4 ms; Section 18.7 explains why the design lengthens it), it sets every output to a configured **substitute value**, which for most outputs is "off". This is the behaviour of the opening story. It is correct and must be designed, signal by signal:

- A filter outlet valve that fails shut is safe but takes the works output down.
- A chlorine dosing pump that stops while water keeps flowing is a public-health risk, so the dosing pump and the flow it doses sit on the same drop, or the dosing is interlocked locally.
- A high-lift pump may "hold last value" on its speed reference, but never on its trip outputs.

The list of substitute values belongs in the I/O list beside each output, signed off by the process engineer, not left at the factory default.

18.4 Distributed control systems

What a DCS adds

A **distributed control system** is a family of controllers, I/O, operator stations, historian and engineering tools sold and engineered as one system with one database. A tag defined once, with its range, alarms, faceplate and trend, exists everywhere. Controller redundancy, online changes and change tracking are standard rather than options. PLC plus SCADA achieves the same with separate products and tag-import discipline.

Table 18.5 — PLC plus SCADA versus DCS for the reference works.

Criterion	PLC plus SCADA (chosen)	DCS
Natural size	Up to a few thousand I/O, many sites	Thousands to tens of thousands of I/O, one site
Engineering database	Separate PLC and SCADA databases, linked by tag import	Single database
Redundancy	Option (chosen here)	Standard
Telemetry to remote sites	Strong: SCADA is built for it	Usually via a separate RTU/SCADA layer
Batch (ISA-88) and advanced control	Add-on	Integrated
Lifecycle cost at 500 I/O	Lower	Higher: licences priced per tag, specialist engineers

At 491 I/O, eight remote sites and a utility owner with in-house PLC skills, PLC plus SCADA wins. A 50 000-I/O refinery with batch blending would go the other way. The boundary is not a number but a question: is the engineering cost of keeping several databases consistent larger than the licence cost of one?

How much redundancy is enough

R1 limits the predicted loss of automatic control of high-lift pumping to 1 h per year. For a single repairable element with mean time between failures MTBF and mean time to repair MTTR, the steady-state unavailability is

$$U = \frac{\mathrm{MTTR}}{\mathrm{MTBF} + \mathrm{MTTR}}$$

For elements in series, unavailabilities add (when small); for a redundant pair with independent failures, the pair is unavailable only when both are down, U^2, plus the outages caused by switchovers that fail.

> **Status of the principle: Availability of series and redundant elements**
> *Approximate.* Assumes constant failure rates, independent failures and repair restoring the element to new; common-cause failures (a shared power feed, a firmware bug present in both CPUs, the same flood) break the independence and in practice set the floor of redundant availability.

Worked example — single or redundant controller? Take MTTR = 8 h (an on-call technician, a spare on the shelf and a night drive) and switchover coverage $c = 0.99$ (1 % of failures are not caught and need a 1 h manual restart). Typical MTBF values from vendor reliability data:

Table 18.6 — Predicted loss of control of high-lift pumping (minutes per year).

Element in the path	MTBF	Single (min/yr)	Redundant (min/yr)
CPU	100 000 h	42.0	0.11
24 V DC supply	200 000 h	21.0	0.05
Network switch or ring segment	300 000 h	14.0	0.04
Remote I/O head, drop RIO-4	400 000 h	10.5	0.03
Total		**87.6 (1.46 h)**	**0.22**
R1 limit		60	60

The single-channel design exceeds R1 by about 46 %. Removing only the CPU's contribution is not enough: the supply, the network and the I/O head would still sum to 45.5 min, inside the limit but with little margin for the I/O modules and field devices not in the table. The chosen design therefore duplicates every element in the path: a redundant CPU pair, two 24 V supplies with a redundancy module, a fibre ring (Section 18.7) and a dual-headed RIO-4 with **system redundancy** (each drop connected to both CPUs). The residual 0.22 min/yr is theoretical; common cause dominates in reality, which is why the two CPUs sit in separate panel sections with separate supplies fed from separate distribution boards.

18.5 Supervisory control and data acquisition

What SCADA does here

SCADA gathers data from controllers and outstations, presents it to operators, records it, raises alarms and sends commands. On the reference works it does two jobs. On site, the SCADA servers read the main PLC over Ethernet and serve the HMI clients (Section 18.6). Across the catchment, a **telemetry front end** talks to eight outstations, each a compact PLC or **RTU** (remote terminal unit) that runs its own local control: a reservoir outlet valve closes on low level whether or not the radio is working.

Two telemetry patterns are in use. In **polling** (Modbus RTU and TCP) the master asks each outstation in turn; it is simple and deterministic, data age is set by the poll cycle, and bandwidth is spent whether or not anything changed. In **report by exception** (DNP3, IEEE 1815; IEC 60870-5-104) outstations send only changes, time-stamped by a local clock and buffered while the link is down, and the master runs a slow **integrity poll** to resynchronise.

> **Boundary.**
> The Modbus frame, function codes and register model, CAN-based networks and industrial Ethernet protocols themselves are owned by Chapter 21; radio link budgets by Chapter 26; MQTT-based telemetry for IoT devices by Chapter 11. This section budgets the telemetry; it does not define the protocols.

The telemetry budget

R4 requires analogues no older than 10 s and alarms at the control room within 5 s. The existing licensed UHF radio network runs at 9600 bit/s half duplex, with a 40 ms transmitter key-up time on every

transmission. Listing 18.2 computes the poll cycle for eight outstations of 60 registers each, and compares a 4G alternative by monthly data volume.

```
 # SCADA telemetry budget: polled radio versus cellular for 8 outstations
BITS_PER_CHAR = 11            # Modbus RTU: start + 8 data + parity + stop
def radio_poll_ms(n_regs, baud=9600, keyup_ms=40, rtu_proc_ms=20, turn_ms=10):
    req, resp = 8, 5 + 2 * n_regs                 # bytes on the air
    t_chr = BITS_PER_CHAR / baud * 1e3            # ms per character
    return (keyup_ms + req * t_chr + rtu_proc_ms  # master transmits, RTU answers
            + keyup_ms + resp * t_chr + turn_ms)

stations, n_regs = 8, 60
t1 = radio_poll_ms(n_regs)
for retry in (0.0, 0.05):                          # fraction of polls repeated
    cycle = stations * t1 * (1 + retry) / 1e3
    print(f"radio 9600 bit/s, retries {retry:4.0%}: {t1:5.0f} ms/station, "
          f"scan cycle {cycle:4.2f} s")

# cellular: bytes per Modbus TCP poll incl. IP/TCP headers (40 B) and two ACKs
poll_bytes = (12 + 40) + (9 + 2 * n_regs + 40) + 2 * 40
month_s = 30 * 24 * 3600
for period_s in (5, 60):
    mb = poll_bytes * month_s / period_s / 1e6
    print(f"cellular polled every {period_s:2d} s: {mb:6.1f} MB/month per station")
# report by exception: 40 events/day of 60 B + hourly integrity poll + 15-min keep-alive
rbe = (40 * 60 + 24 * poll_bytes + 96 * 3 * 40) * 30 / 1e6
print(f"cellular report-by-exception:   {rbe:6.2f} MB/month per station")
```

```
 radio 9600 bit/s, retries   0%:   262 ms/station, scan cycle 2.10 s
radio 9600 bit/s, retries   5%:   262 ms/station, scan cycle 2.20 s
cellular polled every  5 s:  156.0 MB/month per station
cellular polled every 60 s:   13.0 MB/month per station
cellular report-by-exception:     0.63 MB/month per station
```

Worked example — reading the budget. Each radio poll costs 262 ms, of which 80 ms is key-up time and 143 ms is the 125-byte response; eight stations take 2.1 s, or 2.2 s with 5 % retries. Data age is then at most one cycle plus processing, about 2.5 s, a fourfold margin on R4's 10 s. An alarm, though, can occur just after its station was polled and wait a full 2.2 s cycle plus the SCADA server's own scan of 1 s: 3.2 s worst case against R4's 5 s, a margin of only 36 %, which a ninth and tenth station would consume. The design therefore keeps the radio for the eight existing sites but configures the outstations for DNP3 with unsolicited event reporting, so that an alarm is sent the moment it occurs (with collision avoidance on the shared channel), and the poll only refreshes analogues. On 4G, polling every 5 s would cost 156 MB per month per station, a commercial rather than technical problem; report-by-exception costs 0.63 MB. Table 18.7 records the budget.

Table 18.7 — Telemetry timing budget (radio, eight outstations).

Item	Budget	Design value	Margin
Analogue data age (R4)	10 s	2.2 s cycle + 0.3 s processing = 2.5 s	75 %
Alarm latency, polled only	5 s	2.2 s + 1.0 s server scan = 3.2 s	36 %
Alarm latency, DNP3 unsolicited	5 s	About 0.5 s on an idle channel, 1.5 s with one collision backoff	70 %
Channel occupancy	≤ 60 % (to leave room for events)	Polls every 5 s: 2.2 s / 5 s = 44 %	16 points

Historian and data

The historian stores analogues with a deadband (say 0.5 % of span, or at least every 15 min) and every event with its time stamp. Its time base matters more than its size: outstation clocks are synchronised by NTP or by the telemetry protocol, the site PLC by NTP from a local GPS-disciplined server, so that a pressure transient at a reservoir and a pump trip at the works can be put in the right order when a burst main is investigated. Data leave the control network only through a replica historian in the DMZ (Figure 18.1); nothing at Level 4 queries a Level 1 or 2 device directly (Chapter 21).

18.6 Human-machine interfaces

Screens for situation awareness

The **HMI** is where the system meets the operator, and its design follows ANSI/ISA-101.01. The rules that matter most:

- **Grey is normal.** Equipment is drawn in muted greys and colour is reserved for abnormal states, so that one red symbol stands out.
- **A display hierarchy**: a one-screen overview of the whole works, process-area screens for control, equipment detail screens, diagnostics.
- **Analogue indicators with context** (a bar with its normal band and alarm limits), **faceplates** from one library so every pump is operated the same way, and every command confirmed by PLC feedback rather than by the button press.

Alarm management

An **alarm** is a signal to the operator that requires a response. A message that does not require a response is an event and goes to a log, not to the alarm list. The lifecycle of ANSI/ISA-18.2 (IEC 62682) and the guidance of EEMUA 191 give the targets used for R6: about one alarm per 10 min per operator on average is manageable, above about two is over-demanding; an **alarm flood** is more than 10 alarms in 10 min, and the proportion of 10-min periods in flood should be below about 1 %; the ten most frequent alarms should account for only a few percent of the total.

> **Status of the principle: Alarm-rate targets**
> *Empirical.* Set from observed operator performance in process control rooms (EEMUA 191, ISA-18.2); they assume one operator per console and alarms that each require a distinct action, and they are targets for steady operation, not for the first minutes of a major upset.

The works inherited a SCADA alarm database from a previous system with 1 800 configured alarms. Table 18.8 gives the ISA-18.2 key performance indicators computed with a short numpy script (bin the alarm time stamps into 10-min periods, count, rank tags) from 30-day logs before and after an alarm rationalisation; the logs used here are synthetic, generated to match the statistics typically seen in such a review (chattering alarms on two level switches, a bad actor that alarmed on every pump start, standing alarms from out-of-service equipment).

Table 18.8 — Alarm KPIs over 30 days (synthetic logs).

KPI	Target (R6)	Before	After
Configured alarms	—	1 800	900
Alarms per day	About 150	754	77
Average per 10 min	≤ 1	5.24	0.53
Maximum in one 10-min period	—	83	61
10-min periods in flood (> 10)	≤ 1 %	11.1 %	0.1 %
Share of the ten most frequent tags	A few percent	76 %	3 %

Worked example — what the KPIs say. Before rationalisation the operator received 754 alarms per day, 5.2 per 10 min on average, five times the R6 target, with 11 % of all 10-min periods in flood and ten tags producing three quarters of all alarms. That last figure says that fixing ten tags (deadband and on-delay on chattering switches, suppressing low-flow alarms while a pump starts, removing alarms on out-of-service equipment) removes most of the load. After rationalisation, which also halved the alarm count to 900 by deleting alarms that had no defined operator response, the average is 0.53 per 10 min and the flood fraction 0.1 %, both inside R6. The maximum of 61 in one 10-min period remains: it is a plant upset (a power dip that trips every pump), and the next step is **state-based suppression**, in which the PLC recognises "site power failure" and the alarm system shows one alarm for it instead of sixty consequences. Figure 18.3 shows the 10-min alarm counts before and after.

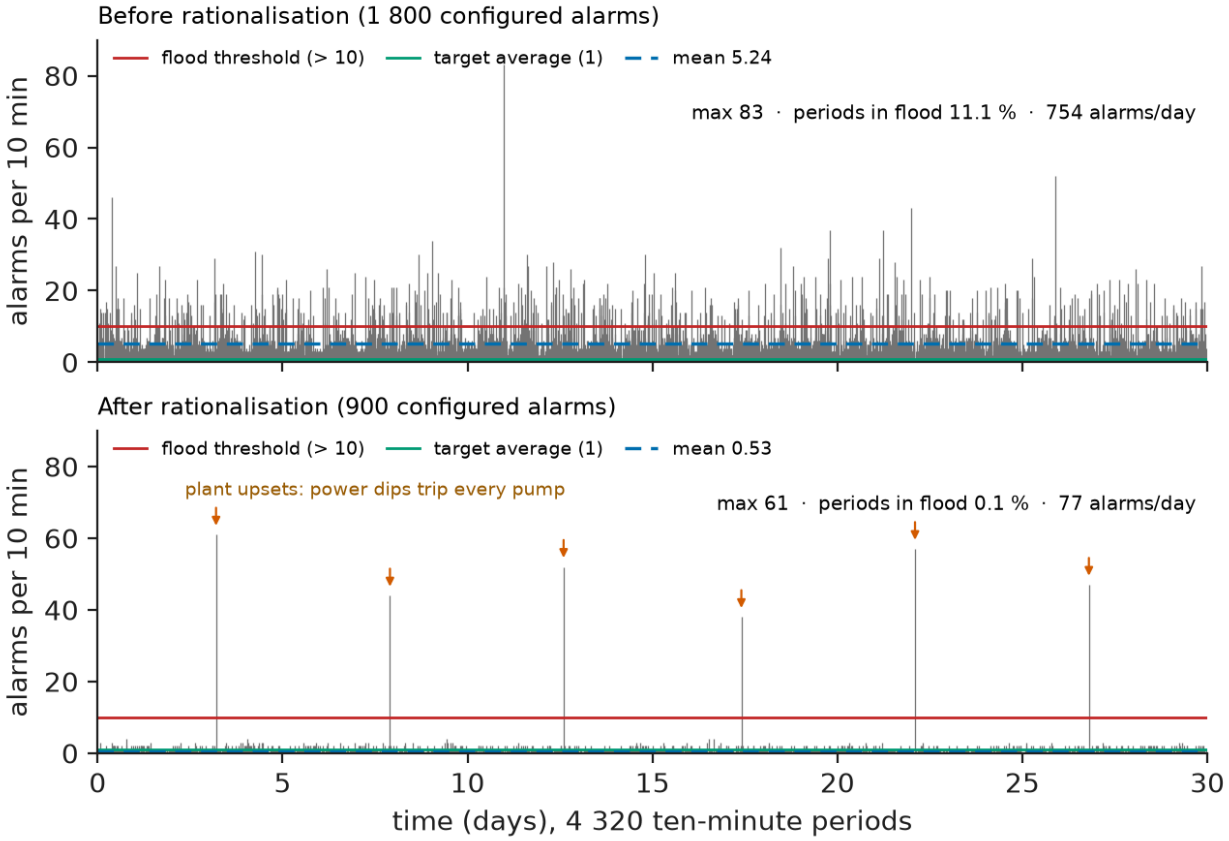


Figure 18.3 — Alarm count per 10-minute period over 30 days, before and after alarm rationalisation (synthetic logs of Table 18.8).

18.7 Industrial networking

Topology

The reference works uses three networks, each with one job: a **control network** (Level 1), PROFINET on a fibre ring between buildings, connecting the PLC pair, four drops and six VFDs, on which only devices in the I/O list are allowed; a **supervisory network** (Level 2) for SCADA servers, clients and the engineering workstation, on its own switches and reaching the PLC through its second interface; and a **site DMZ** (Level 3) between two firewalls.

The ring uses the **Media Redundancy Protocol** (MRP, IEC 62439-2): one switch is the ring manager and blocks one port; when a link breaks, it unblocks it within a guaranteed 200 ms (10 ms in the fast variants). With the default watchdog of 12 ms (3 × 4 ms), every drop beyond a fibre break would go to its substitute values before the ring healed, turning a single cable fault into a works-wide trip: exactly the opening incident. The watchdog of every drop is therefore set to 300 ms. That does not slow the pump trip of R2, which depends on the update time while communication works; it only lets outputs hold their last value for 0.3 s after communication stops, which is harmless for valves that take 30 s to stroke and for pumps whose protection is in any case backed by hardwired relays (Table 18.11). Where a process cannot tolerate even 200 ms of lost data, a bumpless scheme that sends every frame both ways round the ring (MRPD, or PRP and HSR of IEC 62439-3) removes the recovery time altogether.

> **Status of the principle: Ring recovery versus I/O watchdog**
> *Derived.* A device stays in data exchange through a network reconfiguration only if its watchdog time exceeds the worst-case recovery time of the redundancy protocol; otherwise a single link break becomes a process trip, which is acceptable only if the substitute values are safe and the process can tolerate it.

The traffic budget

Cyclic real-time traffic on the control network is small. PROFINET RT frames are about 60–100 bytes; each device exchanges one frame in each direction per update cycle. For the 10 devices (four drops, six drives) at a 4 ms update time on 100 Mbit/s links:

$$\frac{10 \times 2 \times 100\ \text{B} \times 8\ \text{bit/B}}{4\ \text{ms}} = 4.0\ \text{Mbit/s}$$

or 4 % of a 100 Mbit/s link. The threat is uncontrolled traffic: a loop, a backfill, a network scan, a camera. PROFINET RT frames carry a high VLAN priority, so managed switches forward them ahead of other traffic; an unmanaged switch has no such queueing, so bulk traffic through it delays real-time frames (the incident that opens Chapter 21), and a cable loop through it multiplies broadcasts until the links saturate. The design rules: only managed switches with priority queueing and broadcast-storm limiting on the control network; unused ports disabled and others locked to known MAC addresses; engineering laptops only on the supervisory network, never inside a panel; and port statistics, ring status and update-time violations read by the PLC and alarmed on SCADA.

> **Boundary.**
> Industrial Ethernet protocols, their real-time classes and network diagnostics tools are owned by Chapter 21, as is OT security (IEC 62443 zones and conduits). Ethernet physical and MAC layers are in *Electronics and Computing*, Ch. 26 §26.7.

18.8 Control panels

Construction and standards

The control equipment lives in panels: the main control panel (MCP) with the PLC pair, network core and UPS; four remote I/O panels; and outstation kiosks. Assemblies are designed and verified to IEC 61439-1 and -2 (UL 508A in North America), and their electrical equipment to IEC 60204-1 where they form part of a machine. The rules that matter most: 24 V DC, analogue, communication and mains wiring in separate ducts crossing at right angles, with analogue screens bonded at the entry by 360° clamps; terminals for every field cable, with disconnect terminals on each transmitter loop so it can be opened for calibration without lifting a wire; one electronic circuit protector per 24 V group, so a short on one solenoid cable does not take down the CPU; every wire labelled at both ends; and IP54 or better against humidity and the chlorine atmosphere near dosing, with hygrostat-controlled heaters in outdoor kiosks.

The 24 V DC budget

Table 18.9 — 24 V DC budget of the main control panel.

Load	Current
CPU pair and backplanes	1.20 A
14 I/O modules (local rack)	1.68 A
3 managed switches	1.05 A
HMI panel (15 in local operator panel)	1.25 A
Firewall and router	0.60 A
32 local 2-wire transmitters at 22 mA	0.70 A
40 three-wire sensors at 15 mA	0.60 A
60 interposing relays, half energised, 21 mA each	0.63 A
8 local solenoid valves at 0.17 A (all energised)	1.36 A
Total	**9.07 A**
Supply: two 20 A units with redundancy module (N+1)	20 A available with one failed
Margin	**55 %**

The two supplies feed a redundancy module, so either alone carries the whole load (R9); the UPS behind them gives 30 min of operation after a mains failure, long enough for the generator to start and for SCADA to log the event. The margin is deliberately large, for inrush at power-up.

The heat budget

Electronics in a closed panel heat the air inside; the panel's surfaces lose that heat to the room. For a sheet-steel enclosure the effective surface area A depends on how it is installed (IEC/TR 60890 and DIN VDE 0660-507), and the temperature rise at a dissipation P is

$$\Delta T = \frac{P}{kA}, \qquad k \approx 5.5 \ \mathrm{W/(m^2K)} \text{ for painted sheet steel in still air}$$

When that rise is too high, a filter fan moves ambient air through the panel. Air at sea level carries about $1/3.1$ W per (m³/h) per kelvin, so a fan of flow $\dot{V}$ in m³/h lowers the rise to

$$\Delta T = \frac{P}{kA + \dot{V}/3.1}$$

> **Status of the principle: Enclosure heat balance**
> *Approximate.* A lumped single-node model: uniform internal air temperature, still ambient air, no solar gain, and a constant surface coefficient; real panels have a top-to-bottom gradient of 5–15 K, so the hottest electronics, mounted high, see more than the mean rise, and the constant 3.1 m³·K/(W·h) assumes sea-level air density.

Worked example — does the main panel need a fan? The MCP is a 2000 × 800 × 600 mm (H × W × D) floor-standing enclosure against a wall. For that installation the effective surface is $A = 1.4W(H + D) + 1.8DH = 1.4 \times 0.8 \times 2.6 + 1.8 \times 0.6 \times 2.0 = 5.07$ m². The losses inside are the CPU pair 15 W, I/O modules

49 W, switches 24 W, HMI 30 W, firewall and router 15 W, UPS module 20 W, relays 30 W, terminals and wiring 40 W, and 16 W lost by the 24 V supplies in delivering 218 W at 93 % efficiency.

Total $P = 239$ W. Sealed, the rise is $239/(5.5 \times 5.07) = 8.6$ K, and at the 40 °C summer ambient of the pump hall the inside reaches 48.6 °C, which fails R8 (45 °C). Holding a 5 K rise needs the fan to remove $239 - 5.5 \times 5.07 \times 5 = 100$ W, a flow of $3.1 \times 100/5 = 62$ m³/h. Filter fans lose 30–50 % of their free-blowing flow through the outlet filter, so a fan rated about 105 m³/h free-blowing is fitted, low in the door with the outlet filter high on the opposite side. Figure 18.4 shows the internal temperature against heat load for the sealed panel and two fan sizes.

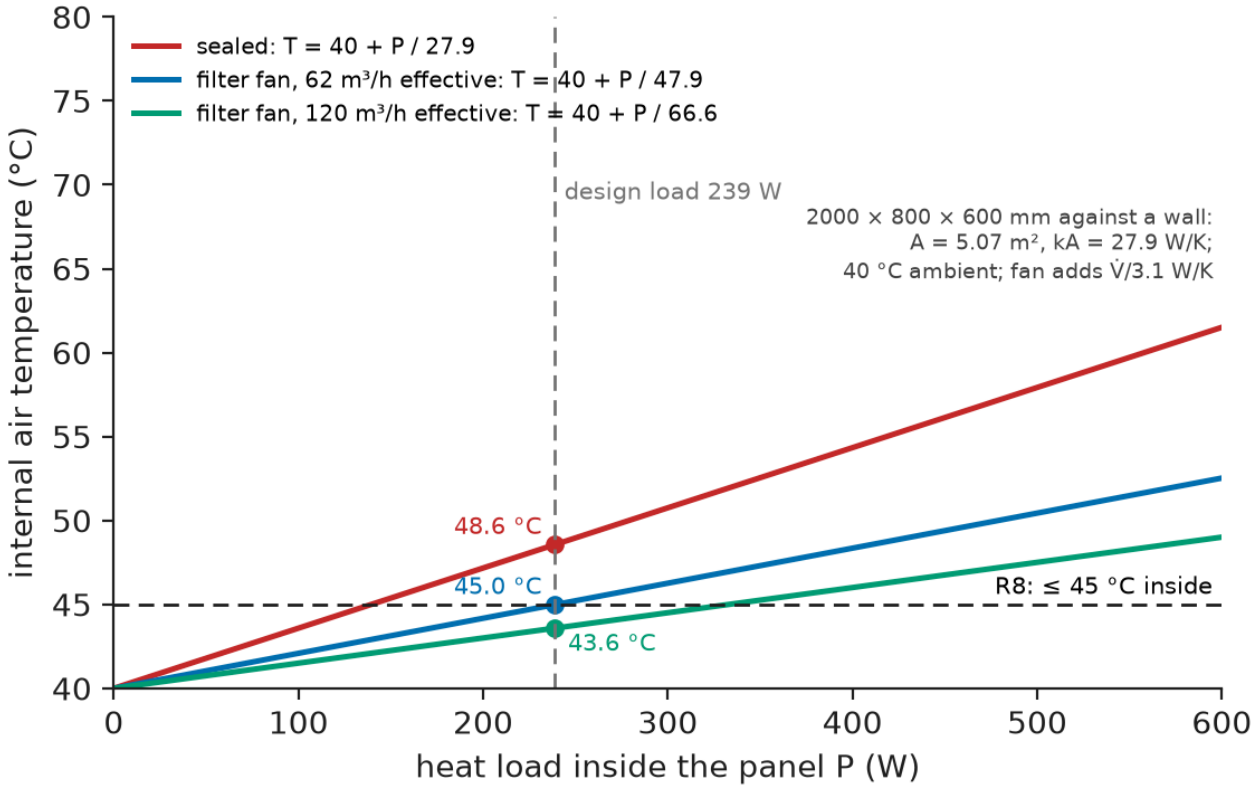


Figure 18.4 — Internal air temperature of the 2000 x 800 x 600 mm main control panel against heat load at 40 degC ambient.

18.9 System commissioning

From factory to plant

Commissioning is the process that turns a correctly built system into a trusted one, and IEC 62381 divides it into stages, each with its own test records (Figure 18.5):

1. **Factory acceptance test (FAT)** at the integrator's works with the I/O simulated, witnessed by the client: every I/O point checked on the HMI, every interlock and sequence run against the cause-and-effect matrix, redundancy switchover, network break, supply failure and watchdog behaviour tested.
2. **Installation**: field cables terminated, insulation-resistance, continuity and earth-bonding tests.
3. **Site acceptance test (SAT) and loop checks**: every device tested end to end, a calibrator injecting 4, 12 and 20 mA and the HMI value recorded, every valve stroked, every motor run for direction and

feedback.

4. **Cold commissioning** with water: sequences run, loops tuned.
5. **Performance test**: treated water to specification for 7–30 days with all automatic functions in service.
6. **Handover**: as-built documents, program and configuration backups, spares, training records and a punch list of open items with owners and dates.

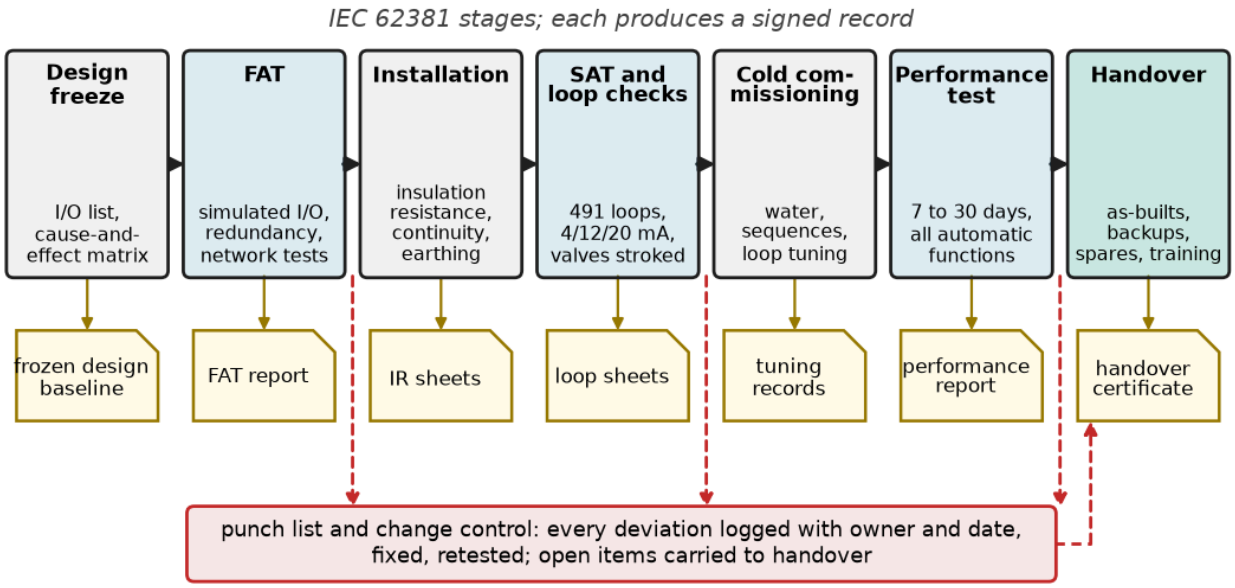


Figure 18.5 — Commissioning stages from design freeze to handover, with the record each stage produces.

Worked example — how long do the loop checks take? A two-person team (one in the field with a calibrator and radio, one at the HMI) checks a digital point in about 5 min and an analogue loop, at three points with the calibrator, in about 15 min, including paperwork. For the works: $(256 + 110) \times 5 + (103 + 22) \times 15 = 1830 + 1875 = 3705$ min, or 62 h of team time: eight working days if nothing is wrong. Typically 5–10 % of loops fail first time (wiring errors, wrong ranges, swapped tags), each costing about an hour, adding 25–50 h: 87–112 h, or 11–14 team-days, is the honest plan.

Verification

Table 18.10 — Verification of the reference control system.

Req.	Test	Limit	Result	Pass/fail
R1	Reliability prediction; CPU switchover with pumps running	≤ 1 h/yr; no trip	0.22 min/yr; switchover 110 ms, no trip	Pass
R2	Pressure-switch step, scope on DO and drive, 20 repeats	≤ 200 ms	13–31 ms to DO; 29–50 ms to torque off	Pass
R3	24 h trend of filter levels	±50 mm	±22 mm after a 0.5 % valve deadband was added	Pass
R4	Timed alarm from each outstation (DNP3 unsolicited)	≤ 5 s	0.4–1.6 s	Pass
R5	I/O schedule audit; CPU cycle load report	≥ 20 % spare; ≥ 30 % headroom	24–31 % spare; cycle 6–9 ms of 10 ms (10 % headroom at peak)	**Fail**, see below
R6	30-day alarm KPI report	≤ 1 per 10 min; ≤ 1 % flood	0.53; 0.1 %	Pass
R7	Disconnect SCADA servers, then supervisory network, 1 h each	No interruption	None; outstation events buffered and delivered	Pass
R8	8 h heat run, thermocouples in panel	≤ 45 °C at 40 °C	44.1 °C mean, 47.5 °C at top rail	Conditional: CPU moved down one rail
R9	Pull each PSU in turn under full load; mains off for 30 min	No reset	No reset; UPS 41 min	Pass
R10	FAT and SAT records audit	100 % of points and interlocks signed	100 %, 37 punch items, all closed	Pass

The R5 failure is typical. The 10 ms cycle was fixed when the program was estimated at 6 ms; with alarm logic, diagnostics and network monitoring added, peak cycle time reached 9 ms. The fix was architectural: PID loops and analogue scaling moved to a 100 ms cyclic interrupt, where a loop belongs anyway because it wants a constant sample period (*Circuits, Signals and Communications*, Ch. 35 §35.8), and the main cycle fell to 5–6 ms.

Failure analysis

Table 18.11 — FMEA of the control architecture (selected modes).

Failure mode	Effect	Cause	Detection	Mitigation
Remote I/O drop loses communication	All outputs of the drop to substitute values; area stops	Cable cut with watchdog shorter than ring recovery; traffic burst through unmanaged switch	Drop diagnostic alarm; port statistics	Watchdog 300 ms above ring recovery; managed switches; substitute values designed per output; dosing interlocked locally to flow
Primary CPU fails	Switchover to standby	Hardware fault, firmware fault	Redundancy status alarm	Hot standby; separate supplies and panel sections for the two CPUs against common cause
Same firmware fault in both CPUs	Loss of control	Common-cause defect	CPU-stop alarm via RTU and autodialler	Hardwired pump protection; local manual control; firmware staged on a spare first
24 V supply output short	Loss of field power to a group	Damaged sensor cable	Circuit-protector trip alarm	One protector per group; N+1 supplies
Telemetry link down	Stale outstation data	Radio fault, repeater power loss	Stale-data flag per value	Local control at outstations; DNP3 event buffering
Alarm flood during upset	Critical alarm missed	Consequential alarms	Flood KPI	State-based suppression
Panel overheating	Shortened life, spurious faults	Clogged fan filter, load growth	Panel temperature alarm at 45 °C	Filter changes in maintenance plan; thermal margin

The opening incident is the first line of this table; its mitigation column is what changed. The lesson is the one this chapter started with: the network, the watchdog and the substitute values are one design decision, and they must be made together and written down.

What remains open

Open question	Where it is answered
Which sensors, transmitters, valves and actuators sit at Level 0, and how their signals reach the I/O	Chapter 19
How motor starters, VFDs, interlocks and E-stops are designed and wired, and how a motor-control panel differs from a control panel	Chapter 20
How Modbus, PROFINET, EtherNet/IP and CAN-based networks work, how a network is diagnosed, and how the control system is protected against cyber attack	Chapter 21
When a protective function must be a safety function with a SIL or PL, and how that is achieved	Chapter 21
How the loops of Section 18.2 are tuned	*Circuits, Signals and Communications*, Ch. 35

Next: Chapter 19 goes down to Level 0 and designs the sensors, transmitters, valves and actuators that the I/O of this chapter connects to.

Chapter 19 — Industrial Sensors and Actuators

Part V · Industrial Automation

On a new clean-in-place skid in a dairy, the supply-pressure reading flat-lines at 3.6 bar whenever the supply pump runs above about 70 % speed, while the mechanical gauge beside the transmitter climbs to 4.8 bar. A new transmitter does the same. Its 4–20 mA output is never outside the NAMUR "healthy" band, so the PLC raises no fault. The cause is arithmetic. The skid supplier had wired the plate heat exchanger's local controller, with its own 250 Ω input, in series with the PLC's 250 Ω input and a loop-powered display, and at 13.7 mA the 24 V supply had no voltage left for the transmitter. Nobody had written the loop's one-line voltage budget. Industrial sensors and actuators are mature; systems built from them fail at the interfaces, where a sensor meets an input card, a coil meets an output, or an actuator meets the air supply.

> **How are industrial sensors, transmitters, valves, contactors, relays and actuators chosen against a requirement, and how is each one interfaced to a PLC without losing accuracy, speed or fail-safety?**
> **Which budgets (loop voltage, uncertainty, sensing distance, coil energy, force, air) must be written down before the skid is built?**

Concept	Engineering destination
Proximity sensors	Valve-position feedback, part detection, speed and end-of-travel sensing
Limit switches	Guard and manway interlocks, overtravel, positive-opening contacts
Industrial temperature sensors	RTDs and thermocouples in thermowells with head transmitters; uncertainty budgets
Pressure transmitters	Line pressure, hydrostatic level, filter differential pressure; 4–20 mA loop budgets
Flow measurement	Choosing among magnetic, Coriolis, vortex, differential-pressure and ultrasonic meters
Solenoid valves	Pilot valves for actuators, valve islands, coil suppression and release time
Contactors	Switching heaters, pumps and other power loads from PLC outputs
Relays	Interposing relays, solid-state relays and their life and heat
Pneumatic and hydraulic interfaces	Actuator force and air budgets, pressure switches, proportional valves
Industrial motor actuators	Electric valve actuators, duty classes, choice against pneumatic actuators

Roadmap. The reference skid and its requirements (19.1) → discrete sensing: proximity sensors (19.2) and limit switches (19.3) → analogue measurement: temperature (19.4), pressure and the loop budget (19.5), flow (19.6) → actuation: solenoid valves (19.7), contactors (19.8), relays (19.9), pneumatic and hydraulic interfaces (19.10), electric actuators (19.11) → verification and failure analysis (19.12).

19.1 The reference skid and its requirements

The skid

The running example is a **clean-in-place (CIP) skid** for a dairy: the plant that cleans process pipework, tanks and heat exchangers between production runs by circulating rinse water, hot caustic (1.5 % NaOH at 75 °C) and acid in a validated sequence. It has three 2 m³ tanks (water, caustic, acid), a 7.5 kW supply pump on a VFD, a plate heat exchanger heated by steam through a modulating control valve, a 3 kW return pump, and 24 automated sanitary valves, all on DN50 and DN65 stainless pipework. Cleaning is proven only if the right chemical reached the right temperature at the right velocity for the right time, so the instrumentation is part of the food-safety case.

Figure 19.1 shows the instruments and actuators and how each reaches the PLC's remote I/O.

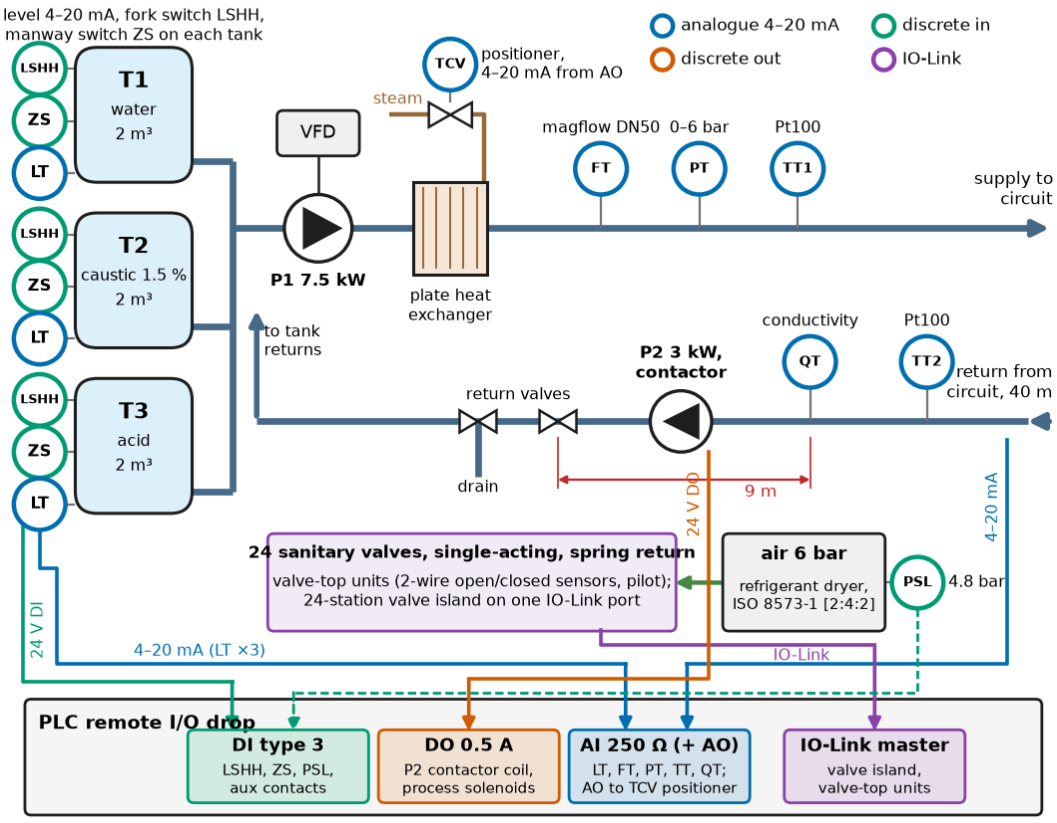


Figure 19.1 — Instrumentation and actuation of the dairy CIP skid and their interfaces to the PLC.

Requirements

Table 19.1 — Requirements for the CIP skid instrumentation and actuation.

ID	Requirement	Target	Verification
R1	Return temperature measurement	Expanded uncertainty (95 %) ≤ 1.0 °C at 75 °C	Uncertainty budget (§19.4); dry-block comparison
R2	Supply flow	±2 % of reading, 8–25 m³/h	Master-meter comparison (§19.6)
R3	Supply pressure loop	0–6 bar, ±0.5 % of span; loop must carry 22 mA with ≥ 1 V margin at 21.6 V supply	Loop budget (§19.5); calibrator at 22 mA
R4	Phase separation	Return diverted within 5 s of the caustic–water interface reaching the sensor; ≤ 5 L caustic to drain per changeover	Timed interface test (§19.6)
R5	Valve position	Every valve proven in its commanded position within 3 s; discrepancy alarm	Stroke test of all 24 valves (§19.2)
R6	Tank level	±20 mm from 15 to 80 °C; independent high-high switch	Dipstick comparison at 20 and 75 °C (§19.5)
R7	Fail-safe	Loss of air, 24 V or communication: valves to spring position, steam valve shut, pumps off	Pull tests (§19.12)
R8	Manway interlock	Pumps cannot run with a tank manway open	Functional test (§19.3)
R9	Hygiene and washdown	Wetted parts to EHEDG / 3-A; IP69K at washdown points	Document review
R10	Return temperature control	75 ± 2 °C during the caustic step	24 h trend

Signal standards at the I/O

Every device in Figure 19.1 meets the PLC through one of a small set of standardised interfaces, and most integration errors are mismatches between them:

Table 19.2 — Signal interfaces on the skid (typical values).

Interface	Standard	Levels	Used for
24 V DC digital input	IEC 61131-2 types 1, 2, 3	ON ≥ 15 V (type 1) or ≥ 11 V (types 2, 3); OFF current ≤ 0.5 mA (type 1), 2 mA (type 2), 1.5 mA (type 3)	Proximity sensors, switches
24 V DC digital output	IEC 61131-2	0.5 A or 2 A per channel, short-circuit protected, internal clamp for inductive loads	Solenoid pilots, contactor coils, relays
4–20 mA analogue input	IEC 60381-1, NAMUR NE 43	3.8–20.5 mA valid; ≤ 3.6 or ≥ 21 mA failure; 50–250 Ω burden	Transmitters
HART on 4–20 mA	IEC 62591 (wired HART)	1200 Hz FSK superimposed, ≥ 230 Ω loop resistance	Configuration, diagnostics
IO-Link	IEC 61131-9	24 V, 3-wire, point-to-point, 4.8/38.4/230.4 kbit/s	Smart sensors, valve islands, valve-top units

Boundary.
Sensing principles and generic sensor selection are owned by Chapter 6, signal conditioning and uncertainty propagation by Chapter 7; the 4–20 mA transmitter as a series–series feedback circuit is in *Electronics and Computing*, Ch. 9 §9.2, and sensor output types at a microcontroller in Ch. 26 §26.9. This chapter takes those as given and engineers the industrial versions: packaged, rated, wired, budgeted and made fail-safe. Controller, I/O and panels are owned by Chapter 18; motor starters by Chapter 20.

19.2 Proximity sensors

Families

Inductive proximity sensors detect metal by the eddy-current loading of an oscillator's coil and are the workhorse of industrial position sensing: no contact, no wear, sealed to IP67–IP69K, switching at up to 1–5 kHz. **Capacitive** sensors detect any material with a permittivity different from air, including liquids and powders through a plastic wall. **Photoelectric** sensors (through-beam, retro-reflective, diffuse) reach metres; **ultrasonic** sensors detect anything that reflects sound, independent of colour. On the CIP skid every valve's position is sensed inductively, inside a **valve-top control unit** that carries two sensors (open and closed), the pilot solenoid and sometimes an IO-Link interface, mounted on the valve stem.

Sensing distance

IEC 60947-5-2 defines a chain of sensing distances. The **nominal** distance S_n is the catalogue figure for a standard mild-steel target (Fe 360, 1 mm thick, square with side equal to the sensing-face diameter or $3S_n$, whichever is larger). The **real** distance of an individual sensor lies within ±10 % of S_n; temperature and supply variation move it a further ±10 %. The **assured operating distance**, within which every sensor of the type is guaranteed to switch, is

$$S_a \leq 0.81\, S_n$$

and the target material reduces it further by a correction factor: about 1.0 for mild steel, 0.6–0.8 for austenitic stainless steel, 0.35–0.5 for brass and aluminium, 0.25–0.45 for copper (typical, from manufacturers' data). Factor-1 sensors, with a modified oscillator, detect all metals at the same distance.

> **Status of the principle: Assured operating distance** $S_a \leq 0.81\, S_n$
> *Empirical.* A standardised design margin (0.9×0.9) covering unit-to-unit spread and drift with temperature and supply; material correction factors are measured per sensor family and vary between manufacturers.

Worked example — valve-top sensing gap. The valve-top unit uses M12-class flush sensors with $S_n = 4$ mm sensing a target ring of 1.4301 stainless steel (correction factor 0.7). Then $S_a = 0.81 \times 4 \times 0.7 = 2.27$ mm. The valve stroke positions the target 1.2 mm from the sensor face when open or closed; the margin is $1 - 1.2/2.27 = 47$ %. A sensor with $S_n = 2$ mm (flush M8) would give $S_a = 1.13$ mm, below the 1.2 mm gap: it would work on the bench with a fresh sensor at 20 °C and fail on the first hot caustic cycle. The second check is the *release*: the sensor's hysteresis is up to 20 % of the real distance, so with the largest possible usable distance of $1.21 S_n = 4.8$ mm the target must retreat beyond about 6 mm to guarantee release; the 40 mm stroke gives that easily.

Two-wire, three-wire and the input card

Three-wire DC sensors are **PNP** (output switched to +24 V, the European norm) or **NPN** (switched to 0 V); a PNP sensor needs a sinking input, and mixing them reads every sensor as permanently off. **Two-wire** sensors are wired in series with the input like a switch, and must draw their own supply through the load: they leave a **residual current** of about 0.5–1.5 mA flowing when off and drop 3–5 V when on.

Worked example — will a two-wire sensor work on this input? A two-wire DC sensor has a residual current of 0.8 mA and an on-state drop of 4.6 V. On a type 1 input (OFF current limit 0.5 mA), the 0.8 mA off-state current can be read as ON: the input may never switch off. On a type 3 input (OFF limit 1.5 mA) the off state is safe, and the on state leaves $24 - 4.6 = 19.4$ V at the input, above the 11 V threshold even with the supply at 20.4 V (−15 %): it works. The skid uses three-wire PNP sensors except in the valve tops, which are two-wire on type 3 inputs, recorded on the I/O list.

Figure 19.2 shows the field-side interfaces of this chapter on one wiring diagram.

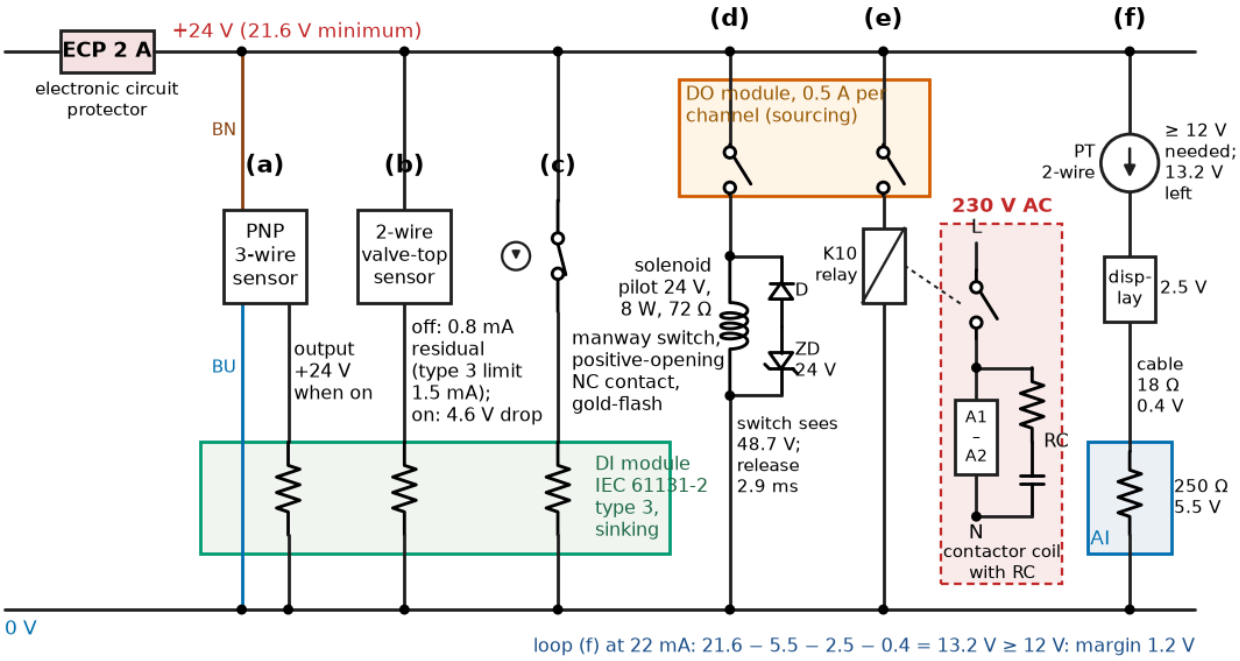


Figure 19.2 — Field-side wiring of the skid's discrete sensors, actuators and one analogue loop to the PLC remote I/O.

Valve feedback in the PLC

R5 asks that each valve be proven in position within 3 s. Listing 19.1, in IEC 61131-3 Structured Text, is the feedback function block used for all 24 valves: it decodes the two sensors, raises a discrepancy if the valve has not reached its commanded position within the travel time, and treats "both sensors on" as an immediate sensor fault.

```
 FUNCTION_BLOCK FB_ValveFb
VAR_INPUT
    xCmdOpen : BOOL;                  (* command from the sequence *)
    xZSO, xZSC : BOOL;                (* open / closed position sensors *)
    xReset   : BOOL;
    tTravel  : TIME := T#3S;          (* R5 *)
END_VAR
VAR_OUTPUT
    xOpen, xClosed, xFault : BOOL;
END_VAR
VAR
    tonDisc : TON;
END_VAR

xOpen   := xZSO AND NOT xZSC;
xClosed := xZSC AND NOT xZSO;
tonDisc(IN := (xCmdOpen AND NOT xOpen) OR (NOT xCmdOpen AND NOT xClosed),
        PT := tTravel);
IF tonDisc.Q OR (xZSO AND xZSC) THEN
    xFault := TRUE;                   (* latched until reset *)
ELSIF xReset THEN
    xFault := FALSE;
END_IF;
END_FUNCTION_BLOCK
```

The sequence never uses the command as evidence: a step that needs a valve open waits for `xOpen`, not for `xCmdOpen`, so a valve with a broken air line holds the sequence and raises an alarm instead of letting caustic into a circuit that is still connected to product.

19.3 Limit switches

A **limit switch** is a mechanically actuated switch: a plunger, roller lever or hinge that operates contacts when a machine part or door reaches a position. Compared with proximity sensors it is bigger and wears, but it has two properties no electronic sensor has: it can switch a load directly, and its normally closed contact can be made **positive opening** (IEC 60947-5-1 Annex K, marked with an arrow in a circle): the actuator forces the contacts apart through a rigid path, so that a welded contact is broken mechanically. That is why guard and manway interlocks use limit switches, or coded safety switches built on the same principle (ISO 14119), and not plain inductive sensors.

On the skid, each tank manway has a hinge-actuated switch with a positive-opening NC contact wired to the safety-related part of the control system, which removes the run permit from both pumps when any manway is open (R8). Its functional-safety design (performance level, channels, diagnostics) belongs to Chapter 21.

Table 19.3 — Limit-switch selection points (typical values).

Property	Typical value	Why it matters
Mechanical life	1–30 million operations	Door switches rarely wear out; roller levers on cams do
Contact rating	AC-15 3 A at 230 V; DC-13 0.27 A at 250 V, 3 A at 24 V	AC-15 and DC-13 are the utilisation categories for switching coils (IEC 60947-5-1)
Minimum switching current	Silver contacts about 5–10 mA at 24 V; gold-flash contacts about 1 mA at 5 V	A PLC input draws 2–3 mA: silver contacts can build an oxide film and read open intermittently
Positive opening	NC contact, Annex K	Required for safety functions
Enclosure	IP67, IP69K for washdown	Caustic washdown at 80 °C and 80 bar spray

The minimum-current row explains a recurring nuisance: a silver-contact switch, chosen for its 10 A rating, flickers on a 3 mA PLC input after a year in a humid hall. Switches feeding PLC inputs get gold-flash contacts.

19.4 Industrial temperature sensors

RTD, thermocouple or thermistor

Industrial temperature measurement uses three sensor families, nearly always packaged as an insert in a **thermowell** (which protects the sensor and lets it be removed without draining the process) with a **head-mounted transmitter** that linearises the signal and converts it to 4–20 mA, HART or IO-Link.

Table 19.4 — Temperature sensor selection for the return line (typical values).

Option	Tolerance at 75 °C	Range	Drift	Verdict
Pt100 class A, 4-wire, head transmitter (IEC 60751)	±0.30 °C	−100 to +450 °C (class A, wire-wound)	≤ 0.05 °C/yr typical	**Chosen**: meets R1 with margin
Pt100 class B	±0.68 °C	−196 to +600 °C	Similar	Meets R1 only marginally (U95 0.85 °C)
Thermocouple type K class 1 (IEC 60584-1)	±1.5 °C	−40 to +1000 °C	Larger; inhomogeneity after heat cycling	Fails R1 alone; right for ovens and kilns
NTC thermistor	±0.1–0.5 °C over a narrow range	−40 to +125 °C typical	Larger at high temperature	Not standard in hygienic thermowells; no interchangeability standard

The tolerance formulas behind the first three rows are class A $\pm(0.15 + 0.002|t|)$ °C, class B $\pm(0.3 + 0.005|t|)$ °C and type K class 1 ± 1.5 °C or $\pm 0.004|t|$, whichever is larger. Figure 19.3 plots them: class A beats type K over its whole range to 450 °C, class B only below 240 °C, and thermocouples own everything hotter.

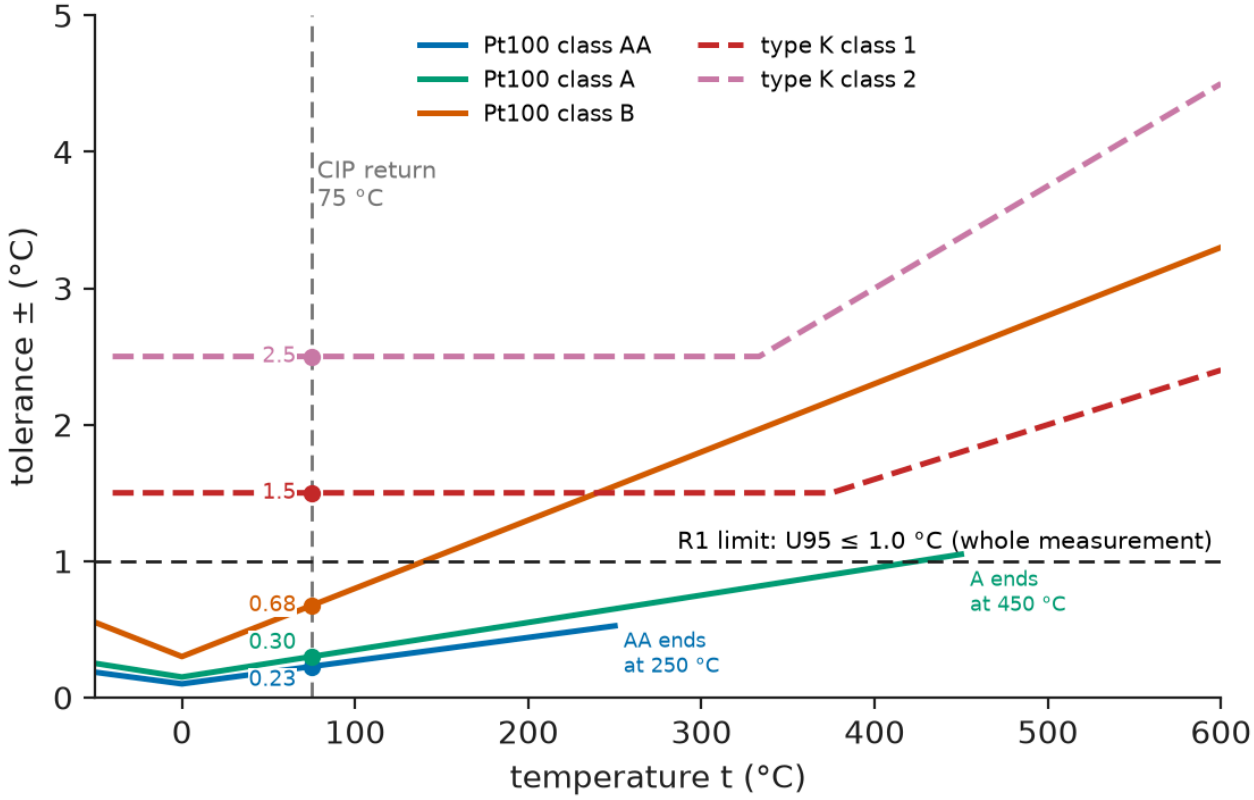


Figure 19.3 — Tolerance of industrial temperature sensors against temperature (IEC 60751 and IEC 60584-1).

The measurement budget

R1 is a requirement on the whole measurement, not on the sensor. Table 19.5 lists every contributor with its limit value, treated as a rectangular distribution (standard uncertainty = limit / √3), combined by root-sum-square and expanded with $k = 2$.

> **Status of the principle: Root-sum-square combination of independent contributions**
> *Derived.* The GUM (JCGM 100) law of propagation for uncorrelated inputs with unit sensitivity; correlated contributions (for example, a transmitter and AI card drifting together with panel temperature) add linearly and must be grouped first.

Table 19.5 — Uncertainty budget of the return temperature at 75 °C.

Contribution	Limit (± °C)	Standard uncertainty (°C)
Pt100 class A tolerance	0.30	0.173
Head transmitter accuracy	0.10	0.058
Transmitter D/A, 0.03 % of 150 °C span	0.045	0.026
Transmitter ambient effect	0.05	0.029
Transmitter drift, 1 year	0.10	0.058
PLC AI card, 0.1 % of 150 °C span	0.15	0.087
Immersion and stem conduction in the thermowell	0.20	0.115
Combined standard uncertainty		**0.243**
Expanded (k = 2)		**0.49 °C**
R1 limit		1.0 °C
Margin		**51 %**

Worked example — what the budget decides. With class A the expanded uncertainty is 0.49 °C, half of R1. With class B the sensor term grows to $0.675/\sqrt{3} = 0.390$ °C and the total to 0.85 °C: inside R1 but with 15 % margin, too little for a validation record that must survive a year of drift. A type K thermocouple fails on its tolerance alone. The budget also shows where to improve next: a digital link (HART or IO-Link) removes the card's 0.15 °C, an immersion of at least ten tip diameters keeps stem conduction below 0.1 °C, and a loop calibration in a dry block replaces the class tolerance by a calibration uncertainty of 0.05–0.1 °C.

The thermowell also sets the **response time**: t_{90} of about 8–15 s with heat-transfer paste, 30–60 s without. With the 19 s transport delay of the return line (Section 19.6), the R10 loop is dead-time dominated and must be tuned gently (*Circuits, Signals and Communications*, Ch. 35 §35.3).

> **Boundary.**
> The Seebeck effect and thermoelectric circuits are in *Engineering Physics*, Ch. 14 §14.3; RTD excitation, lead resistance and the Callendar–Van Dusen equation at a microcontroller in *Electronics and Computing*, Ch. 26 §26.9. Here they are packaged in a transmitter.

19.5 Pressure transmitters

What a transmitter adds to a sensor

A **pressure transmitter** packages a sensing element (a piezoresistive silicon die behind an oil-filled diaphragm, or a ceramic capacitive cell) with compensation electronics and an output stage. Its IEC 61298 figures are accuracy as a percentage of calibrated span (0.04–0.1 % high-end, 0.2–0.5 % compact), **turndown** (10:1 to 100:1), temperature effect per 10 K, stability per year and overpressure limit. Hygienic versions have **flush diaphragms** on sanitary fittings, so that no dead leg holds product.

On the skid they measure supply pressure (0–6 bar), hydrostatic tank level (0–400 mbar) and supply-filter differential pressure.

Hydrostatic level and density

A transmitter at the bottom of a tank measures $p = \rho g h$; the PLC computes $h = p/(\rho g)$ with an assumed density.

Worked example — level error from density. Tank T2 holds 1.5 % caustic, whose density is about 1015 kg/m³ at 20 °C and about 990 kg/m³ at 75 °C. At a true level of 2.00 m and 75 °C the transmitter reads $990 \times 9.81 \times 2.00 = 19\,424$ Pa (194.2 mbar). A PLC that assumes 1000 kg/m³ computes 1.98 m: a 20 mm error, the whole of R6's allowance, before the transmitter's own error is counted. The fix costs nothing: the PLC computes density from the tank temperature (a fitted curve from three laboratory points) and the residual error is about 3 mm. Where density varies with composition, a second transmitter at a known height or a radar gauge, which ignores density, is used instead.

The loop budget

A two-wire transmitter takes its power from its own 4–20 mA loop. It needs a minimum voltage across its terminals (its **lift-off voltage**, typically 10–12 V with HART), and everything else in the loop (input resistors, cable, loop-powered displays, isolators, barriers) drops voltage that the supply must provide at the *highest* current the loop must carry. With NAMUR NE 43 the transmitter signals a failure by driving at least 21 mA, so a loop must be designed for about 22 mA, not 20 mA. The condition is

$$V_{supply,min} - V_{tx,min} - \sum V_{fixed} \;\geq\; I_{max} \sum R$$

> **Status of the principle: Loop voltage budget**
> *Fundamental.* Kirchhoff's voltage law around a series loop; the only judgement is in the inputs, which must be the worst-case supply (low line, end of the 24 V cable run), the transmitter's guaranteed minimum voltage at its maximum current, and every series element.

Listing 19.2 computes the budget for the supply-pressure loop of the opening story, configuration by configuration.

```
 # 4-20 mA loop voltage budget: what current can the transmitter still drive?
V_SUPPLY_MIN = 24.0 * 0.90     # 24 V PSU at -10 % (brown-out / end of cable run)
V_TX_MIN     = 12.0            # transmitter lift-off voltage incl. HART
I_DESIGN     = 0.022           # A, highest current the loop must carry (alarm)
R_CABLE      = 2 * 250 * 0.036 # ohm: 250 m of 0.5 mm^2 pair, both cores

def loop(name, r_series, v_fixed):
    """r_series: ohmic burdens; v_fixed: constant drops (displays, isolators)."""
    r = r_series + R_CABLE
    margin = V_SUPPLY_MIN - V_TX_MIN - v_fixed - I_DESIGN * r
    i_max = (V_SUPPLY_MIN - V_TX_MIN - v_fixed) / r   # current at which tx starves
    span = 6.0 * (min(i_max, 0.0205) - 0.004) / 0.016  # bar, 0-6 bar range
    print(f"{name:31s} margin at 22 mA {margin:5.2f} V, "
          f"starves above {i_max*1e3:5.1f} mA -> reads up to {span:4.2f} bar")

loop("AI card 250 ohm",                  250, 0.0)
loop("+ loop display (2.5 V)",           250, 2.5)
loop("+ display + skid controller 250",  500, 2.5)
loop("fix: active splitter 50 ohm",       50, 2.5)
```

```
 AI card 250 ohm                 margin at 22 mA  3.70 V, starves above  35.8 mA -> reads up to
6.19 bar
+ loop display (2.5 V)          margin at 22 mA  1.20 V, starves above  26.5 mA -> reads up to
6.19 bar
+ display + skid controller 250 margin at 22 mA -4.30 V, starves above 13.7 mA -> reads up to 3.64
bar
fix: active splitter 50 ohm     margin at 22 mA  5.60 V, starves above 104.4 mA -> reads up to
6.19 bar
```

Worked example — reading the loop budget. The design as drawn (AI card plus loop display) had 1.2 V of margin at 22 mA: tight but compliant with R3. The supplier's second 250 Ω input raised the loop resistance to 518 Ω and left only 7.1 V for it, so the transmitter starved at $7.1/518 = 13.7$ mA, which is 3.64 bar on a 0–6 bar range. Because 13.7 mA lies inside NE 43's valid band, no diagnostic in the PLC could see it: the reading was plausible and wrong. The fix was an active signal splitter with a 50 Ω input that powers the transmitter loop and drives two isolated outputs, one to the PLC and one to the exchanger controller, restoring 5.6 V of margin. (The "starves above" figure for the fixed loop is only the limit the loop would allow; the transmitter itself saturates at about 21–22.5 mA.) Figure 19.4 shows the same budget graphically as load lines.

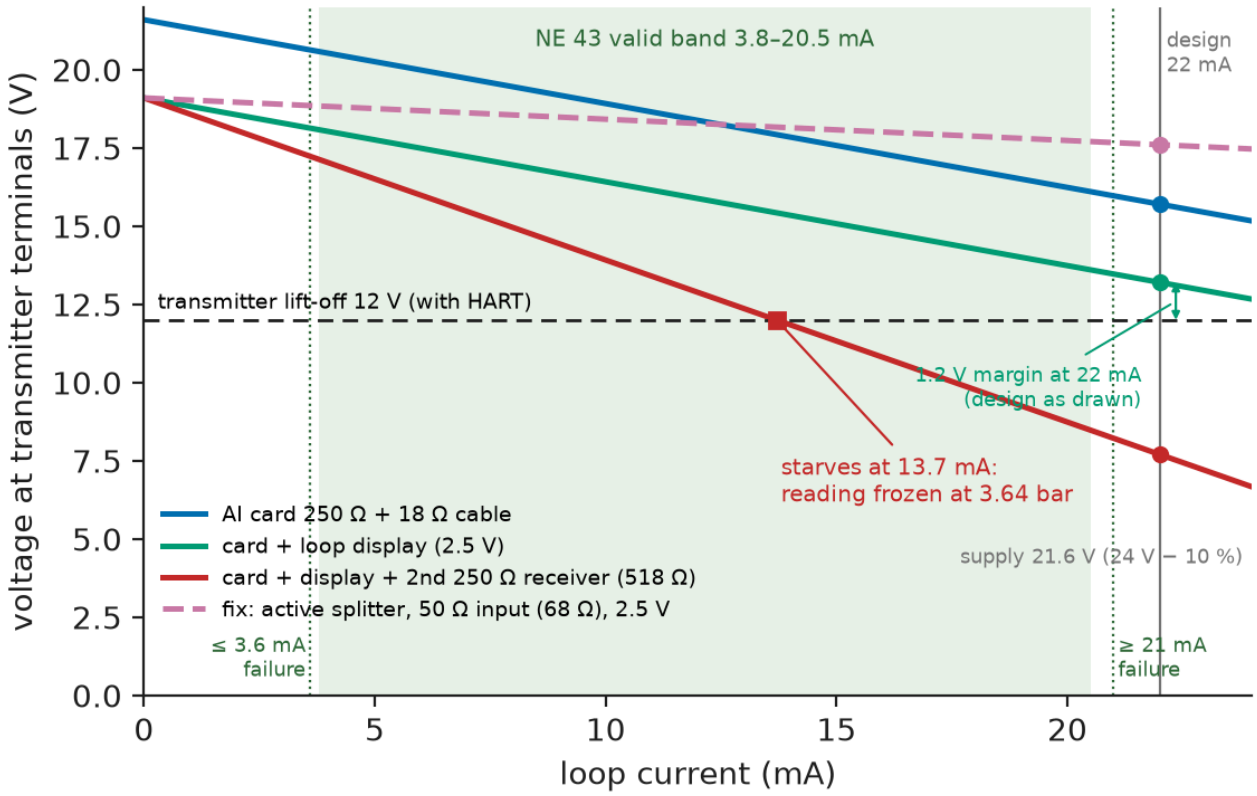


Figure 19.4 — Voltage available at the transmitter terminals against loop current for four loop configurations.

Scaling in the PLC

Listing 19.3 is the analogue-input block used for every transmitter on the skid. It assumes the common S7-class raw format in which 4–20 mA maps to 0–27 648 counts (with over- and underrange beyond), converts to milliamperes, applies NE 43, and holds the last good value with a fault flag rather than passing a wire-break reading to a control loop.

```
 FUNCTION_BLOCK FB_AnalogIn
VAR_INPUT
    iRaw     : INT;                  (* 0..27648 = 4..20 mA *)
    rLo, rHi : REAL;                 (* engineering range, e.g. 0.0 / 6.0 bar *)
END_VAR
VAR_OUTPUT
    rValue     : REAL;               (* engineering units *)
    xFault     : BOOL;               (* NE 43 failure: <= 3.6 or >= 21 mA *)
    xUncertain : BOOL;               (* outside 3.8..20.5 mA, not failed *)
END_VAR
VAR
    rmA    : REAL;
    xBad   : BOOL;
    tonFlt : TON;
END_VAR

rmA := 4.0 + 16.0 * INT_TO_REAL(iRaw) / 27648.0;
xBad := (rmA <= 3.6) OR (rmA >= 21.0);
tonFlt(IN := xBad, PT := T#1S);
xFault := tonFlt.Q;
xUncertain := NOT xFault AND ((rmA < 3.8) OR (rmA > 20.5));
IF NOT xBad THEN                     (* hold last good value while failing *)
    rValue := rLo + (rHi - rLo) * (LIMIT(4.0, rmA, 20.0) - 4.0) / 16.0;
END_IF;
END_FUNCTION_BLOCK
```

The value freezes on the first scan of a failure, while the 1 s on-delay keeps transients from latching the fault, so a broken wire never reaches a controller as "0 bar". No scaling block catches a starved loop; only the budget does.

19.6 Flow measurement

Choosing the principle

Table 19.6 — Flowmeter selection for the CIP supply line (DN50, 8–25 m³/h, water, caustic, acid, 5–85 °C; typical values).

Principle	Accuracy	Pressure loss	Needs	Hygienic	Indicative price	Verdict
Electromagnetic	±0.2–0.5 % of reading	None (full bore)	Conductivity ≥ 5 µS/cm	Yes (PFA liner)	$1 500–$2 500	**Chosen**
Coriolis	±0.1 % of reading, plus density	0.2–0.5 bar at full flow	Nothing; measures mass	Yes	$5 000–$9 000	Overkill here; right for dosing
Vortex	±0.75–1 %	Low	Reynolds number above about 20 000; no low flows	Limited	$2 000–$3 500	Poor turndown here
Differential pressure (orifice)	±1–2 % of rate	0.1–0.5 bar permanent	Straight pipe runs	No (dead legs)	$1 500 with transmitter	Unhygienic
Clamp-on ultrasonic	±1–3 %	None	Known pipe wall; filled pipe	No contact at all	$2 000–$6 000	Retrofit checks only

The magnetic flowmeter works by Faraday induction in the conducting liquid; it has no moving parts and no obstruction, and its only demand is conductivity. Rinse water at 50–500 µS/cm and caustic at tens of mS/cm both qualify. Its liner and electrodes must resist hot caustic and nitric acid: PFA liner and 1.4435 stainless or Hastelloy electrodes. Deionised water below 5 µS/cm would have ruled it out and forced a Coriolis meter.

> **Boundary.**
> The fluid mechanics behind each principle (Bernoulli for differential pressure, vortex shedding, pipe networks and pump curves) are in *Engineering Physics*, Ch. 19 §19.7. Here the meter is a component with an accuracy, a range and a set of installation rules.

Worked example — flow, velocity and accuracy. A DN50 hygienic tube has an internal diameter of 50 mm and a cross-section of 1.963×10^{-3} m². Cleaning validation demands at least 1.5 m/s in every line, which in DN50 is $1.5 \times 1.963 \times 10^{-3} \times 3600 = 10.6$ m³/h and in DN65 (66 mm bore) 18.5 m³/h; the 20 m³/h design flow gives 2.83 m/s in DN50 and 1.62 m/s in DN65. A magnetic meter specified at ±0.2 % of reading ±1.5 mm/s has an error at 10.6 m³/h (1.5 m/s) of $0.2 + 0.1 = 0.3$ % and at 20 m³/h of 0.25 %: a margin of six to eight on R2's ±2 %. The meter needs 5 diameters of straight pipe upstream and 2 downstream; on a compact skid that is the hardest requirement, and the layout is drawn around it.

Phase separation: a timing problem

At each change of medium (rinse water to caustic, caustic to rinse) the return flow must be diverted from drain to tank, or tank to drain, as the interface between the two liquids arrives. An inductive conductivity sensor in the return line sees the interface; the PLC switches the return valves.

Worked example — where to put the conductivity sensor. Return flow is 15 m³/h in a 40 m DN50 return line: $v = 15/3600/1.963 \times 10^{-3} = 2.12$ m/s, a transport delay of 18.8 s from circuit to skid. After the interface reaches the sensor, the sensor needs about 3 s to settle (t_{90} of an inductive conductivity cell with temperature compensation), the PLC 0.1 s, and the return valve 1.0 s to stroke: 4.1 s, which meets R4's 5 s. But in 4.1 s, $15/3.6 \times 4.1 = 17$ L of caustic would pass the valves into the drain, three times R4's 5 L. Moving

the sensor upstream of the valves by the distance the liquid travels in 4.1 s, $2.12 \times 4.1 = 8.7$ m (9 m installed), means the valves switch exactly as the interface reaches them; the residual loss is the mixed zone of the interface itself, about 1.5 m of pipe or 3 L. A pipe layout, not a faster sensor, met the requirement.

19.7 Solenoid valves

Where solenoid valves sit

A **solenoid valve** converts an electrical command into a fluid path. On the skid there are two kinds. **Pilot valves** switch air to the 24 valve actuators from a **valve island**, a 24-station manifold with one IO-Link connection that replaces 24 output wires. **Process solenoid valves** switch the process medium directly: a 2/2-way valve on the steam condensate drain and on the water make-up.

Selection follows the function:

- **Configuration**: 2/2 for a medium, 3/2 to pilot a spring-return actuator, 5/2 for a double-acting cylinder, each **monostable** (spring return) or **bistable** (latching).
- **Direct-acting or pilot-operated**: a pilot-operated valve uses line pressure to move its main stage and needs a minimum differential pressure, typically 0.2–0.5 bar; on a gravity-fed tank outlet it never opens, a common first-week fault.
- **Flow** as Q_n (standard L/min at 6 bar, 1 bar drop) or K_v; **coil** 24 V DC, 1–8 W, often with reduced holding current.
- **Fail-safe position**: R7 rules out bistable pilots; every pilot is a monostable 3/2 and every actuator single-acting with spring return.

Coil suppression and release time

When an output switches off an inductive coil, the current must keep flowing somewhere. Without a path it drives the switch voltage up until something breaks down; PLC outputs therefore have an internal clamp, and coils are fitted with suppressors. The choice of suppressor sets how quickly the current decays, and therefore how quickly the valve releases.

> **Boundary.**
> The flyback and TVS diodes themselves, and how they protect a switch, are covered in *Electronics and Computing*, Ch. 2 §2.10; the first-order RL decay in *Circuits, Signals and Communications*, Ch. 6 §6.2. The listing only applies them.

```
 # Release time of a 24 V DC solenoid coil for three suppression circuits
import math
V, P, L = 24.0, 8.0, 0.5           # supply, coil power (W), coil inductance (H)
R = V**2 / P                       # 72 ohm
I0, I_DROP = V / R, 0.30 * V / R   # armature drops out near 30 % of rated current
tau = L / R
print(f"R = {R:.0f} ohm, I0 = {I0*1e3:.0f} mA, tau = {tau*1e3:.1f} ms, "
      f"stored energy {0.5*L*I0**2*1e3:.1f} mJ")
for name, v_clamp in [("diode only", 0.7), ("diode + 24 V Zener", 24.7),
                      ("TVS 48 V", 48.0)]:
    # L di/dt = -(v_clamp + R i)  ->  exponential decay towards -v_clamp/R
    t_rel = tau * math.log((I0 + v_clamp / R) / (I_DROP + v_clamp / R))
    v_peak = V + v_clamp            # voltage across the switching transistor
    a = v_clamp / R
    t0 = tau * math.log((I0 + a) / a)                # current reaches zero
    e_clamp = v_clamp * (tau * I0 - a * t0)          # integral of v_clamp * i
    print(f"{name:20s} release {t_rel*1e3:5.2f} ms, switch sees {v_peak:4.1f} V, "
          f"clamp absorbs {e_clamp*1e3:4.1f} mJ")
```

```
 R = 72 ohm, I0 = 333 mA, tau = 6.9 ms, stored energy 27.8 mJ
diode only           release  7.92 ms, switch sees 24.7 V, clamp absorbs  1.5 mJ
diode + 24 V Zener   release  2.94 ms, switch sees 48.7 V, clamp absorbs 17.2 mJ
TVS 48 V             release  1.85 ms, switch sees 72.0 V, clamp absorbs 21.0 mJ
```

Worked example — choosing the suppressor. The process solenoid valve on the water make-up has an 8 W, 24 V coil (72 Ω, 333 mA, about 0.5 H). A plain freewheeling diode clamps the coil at 0.7 V, so the current decays with the coil's own time constant of 6.9 ms and the armature releases after 7.9 ms. A diode with a 24 V Zener in series lets the coil voltage reverse to about 25 V, the current decays almost linearly and the armature releases in 2.9 ms, at the price of the switch seeing 48.7 V and the Zener absorbing 17 mJ per operation. A 48 V TVS releases fastest but puts 72 V across the output transistor. The make-up valve has no timing requirement, so a diode does; the valve-island pilots, which execute the phase separation of Section 19.6, use the island's built-in diode-Zener suppression. Figure 19.5 plots the three decays.

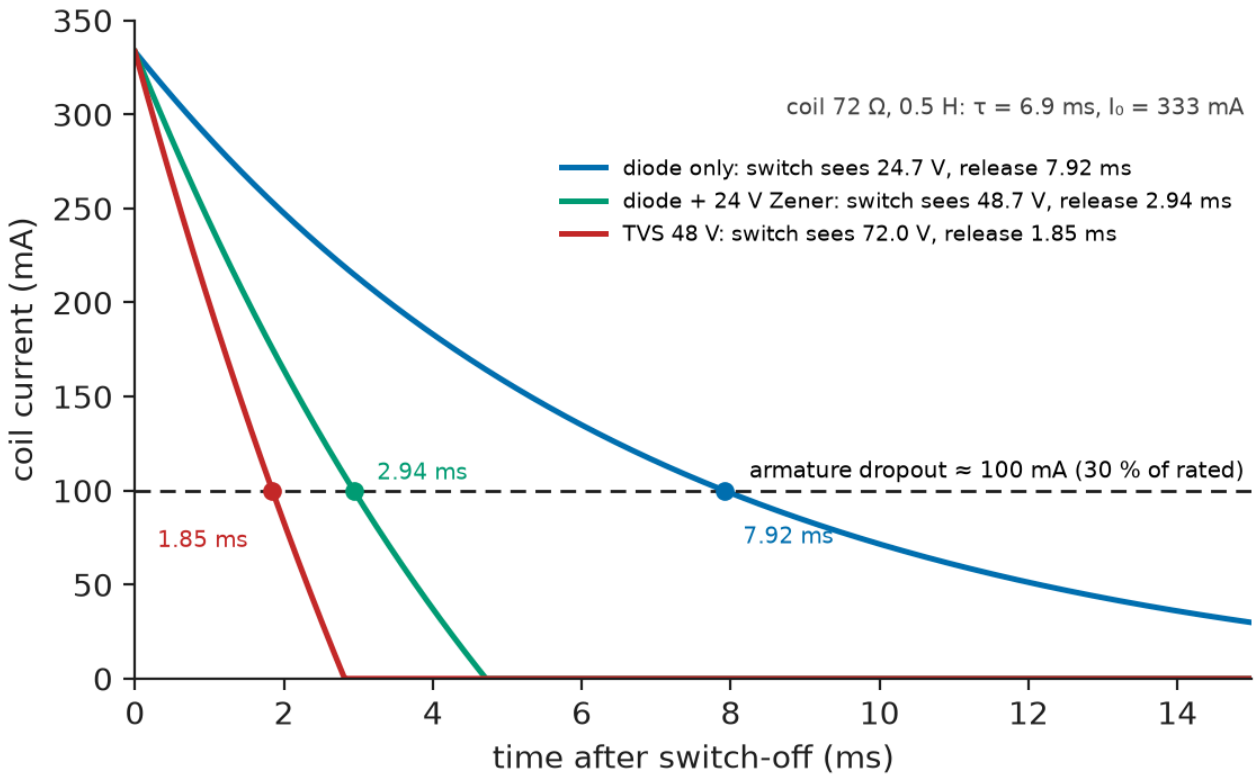


Figure 19.5 — Current in the 8 W, 24 V solenoid coil after switch-off for three suppression circuits.

> **Status of the principle: Release time set by the clamp voltage**
> *Approximate.* Treats the coil as a constant inductance in series with its resistance; a real solenoid's inductance falls as the armature moves and the eddy currents in its iron slow the flux decay, so measured release times are typically 1.5–3 times the electrical estimate, but the ranking of suppressors holds.

19.8 Contactors

A **contactor** is an electromagnetically operated switch for power circuits, rated for frequent operation, with a coil that a PLC output can drive and auxiliary contacts that report its state. On the skid it switches two loads: the 3 kW return pump (direct-on-line) and the 18 kW electric heater in the water tank used for hot rinse when steam is off.

The contactor's rating depends on what it switches, expressed as a **utilisation category** of IEC 60947-4-1 (power) and IEC 60947-5-1 (control):

Table 19.7 — Utilisation categories that matter on the skid.

Category	Load	Make / break duty	Skid example
AC-1	Non-inductive or slightly inductive (cos φ ≥ 0.8)	Rated current	18 kW heater
AC-3	Squirrel-cage motors: start, switch off while running	Make 6–8 × I_e, break about 1 × I_e	3 kW return pump
AC-4	Motors: plugging, inching	Make and break 6–8 × I_e	(Chapter 20)
AC-15	Control of AC electromagnetic loads (coils)	Make 10 × I_e, break 1 × I_e	Contacts switching contactor coils
DC-13	Control of DC electromagnets	Break with L/R up to about 300 ms	Contacts switching DC solenoids

Worked example — driving the coils. The 18 kW, 400 V heater draws $18\,000/(\sqrt{3} \times 400) = 26.0$ A, so it needs an AC-1 rating of at least 26 A; a frame rated 40 A AC-1 (25 A, 11 kW AC-3) is chosen, leaving 35 % margin for supply-voltage rise and element tolerance. Its coil can be AC or DC. An AC coil at 230 V draws about 70 VA at pull-in (0.30 A) and 8 VA holding (35 mA), so it cannot be driven from a 24 V transistor output: it needs an **interposing relay** (Section 19.9). A 24 V DC coil of the same frame draws about 5.4 W (0.23 A), within a 0.5 A PLC output, but with an inductive kick that needs the suppression of Section 19.7; contactors are sold with plug-in diode, RC or varistor modules for this. The skid uses DC coils on all contactors, which removes 230 V from the control circuit and keeps the coil supply on the 24 V UPS; and an auxiliary NO contact from each contactor returns to a PLC input, so that "commanded on, not confirmed on" raises an alarm within 0.5 s, the same discrepancy logic as Listing 19.1.

> **Boundary.**
> Contactors as motor starters (AC-3 and AC-4 electrical life, overload relays, reversing interlocks, mirror contacts for safety circuits) are owned by Chapter 20.

19.9 Relays

Interposing relays

An **interposing relay** sits between a PLC output and a load the output cannot drive directly: a 230 V AC contactor coil, a load on another supply, a signal into another vendor's panel that must remain galvanically separate. Their limits:

- **Electrical life** depends on the load: a typical 6 A relay offers 10–30 million mechanical operations but only 100 000–300 000 at its rated resistive load, fewer for inductive loads; a life curve in the datasheet, not the headline current, decides.
- **Minimum load**: the same oxide-film problem as limit switches (Section 19.3); relays that switch PLC inputs need gold-flash contacts.

Solid-state relays

A **solid-state relay** (SSR, IEC 62314) switches AC loads with thyristors or a triac, usually at zero crossing. It has no contacts to wear and can switch every second for years. It pays with heat (about 1–1.5 W per ampere per pole), a leakage current of a few milliamperes in the off state, and a failure mode that matters: SSRs usually fail **short**, leaving the load on.

Worked example — contactor or SSR for the heater? The tank heater is controlled by time-proportioning: the PLC's temperature loop switches it on for a fraction of each 20 s cycle. Running 16 h a day for 300 days, that is $3600/20 \times 16 \times 300 = 864\,000$ switching operations per year. An AC-1 contactor rated for about 1 million operations at full load would wear out in a little over a year. A two-pole SSR (switching two of three phases is enough to stop a three-phase heater in delta) dissipates about $1.2 \times 26 = 31$ W per pole, 62 W in all. Holding the thyristor at a design junction temperature of 100 °C in a 45 °C panel, with 0.4 K/W from junction to heatsink, needs a heatsink of at most $(100 - 45)/31 - 0.4 = 1.36$ K/W per pole. The design uses the SSR for switching, a contactor upstream as the isolating and safety device (opened by the over-temperature switch and the tank low-level switch, so that an SSR failed short cannot boil the tank dry), and adds the 62 W to the panel heat budget of Chapter 18.

> **Status of the principle: Switching life scales with operations, not with time**
> *Empirical.* Contactor and relay electrical life is given by manufacturers' life curves measured under the standard test conditions of each utilisation category; real life varies with load power factor, inrush and ambient, so the curves are design guides with margins of two or more, not guarantees.

19.10 Pneumatic and hydraulic interfaces

Compressed air as a power supply

The 24 sanitary valves are driven by single-acting, spring-return pneumatic actuators: air opens, the spring closes. Compressed air is therefore a utility with its own requirements, specified like a power supply:

- **Pressure**: 6 bar nominal at the skid, 4.5 bar minimum (the compressor cut-in pressure less the line drop), monitored by a pressure switch (PSL) whose low alarm stops the CIP sequence before valves fail to open.
- **Quality**: ISO 8573-1 classes for particles, water and oil, typically class [2:4:2] for valve islands, with a refrigerant dryer (pressure dew point +3 °C) so that no condensate reaches the pilots.
- **Flow**: enough for the worst-case simultaneous valve movements without the pressure dipping below the minimum.

Worked example — actuator force and air budget. A DN50 valve actuator has an 80 mm piston (5027 mm²) and a 1100 N spring at full compression; the line pressure of 5 bar acting on the 50 mm valve seat opposes opening with $5 \times 10^5 \times 1.963 \times 10^{-3} = 982$ N.

- At 6 bar the piston force is 3016 N; at the 4.5 bar minimum, 2262 N. The net opening force at minimum air is $2262 - 1100 = 1162$ N, 18 % above the 982 N needed. That is why the PSL is set at 4.8 bar, not 4.0.
- Each opening fills the 40 mm stroke (0.201 L) and 3 m of 6/4 mm tube (0.038 L) to 6 bar gauge. In free air that is $(0.201 + 0.038) \times (7.013/1.013) = 1.65$ NL per operation.

- At the busiest point of the sequence, 24 valves each operate twice a minute: $24 \times 2 \times 1.65 = 79$ NL/min, or 4.8 Nm³/h. The skid's share of the site compressor is specified at 10 Nm³/h, a margin of two that also covers leakage.

Hydraulic interfaces

Where forces are large, hydraulics replace air: a 200 bar press cylinder of 180 mm bore produces $2 \times 10^7 \times \pi \times 0.09^2 = 509$ kN. The CIP skid has no hydraulics, but the interface pattern is the same at a cheese press or a filling machine on the same site. A hydraulic power unit reaches the PLC through pressure transmitters and switches, a filter-clog switch, oil level and temperature switches, and **proportional valves** whose spool position follows a command from an on-board or panel-mounted amplifier, driven by ±10 V or 4–20 mA from the PLC, with spool-position feedback for diagnostics. Hydraulic safety (ISO 4413) adds what air does not need: pressure relief on every circuit, load-holding valves on vertical axes, and the rule that stored hydraulic energy in accumulators must be dumped before maintenance. ISO 4414 is the pneumatic counterpart.

19.11 Industrial motor actuators

Electric actuators

An **electric actuator** is a geared motor with position and torque sensing, mounted on a valve or a mechanism: **multi-turn** for gate and globe valves, **quarter-turn** for butterfly and ball valves (to ISO 5211 mounting flanges), and **linear** for control valves and dampers. Industrial valve actuators include their own motor starter, torque switches that stop the motor when the valve seats or jams, end-of-travel limit switches, a handwheel, and an interface: hardwired open/close/stop with feedback, 4–20 mA positioning, or a fieldbus. EN 15714-2 classifies them by duty: class A on/off, class B inching and positioning, class C modulating, class D continuous modulating, with motor duty ratings to IEC 60034-1 (S2 short-time, S4 intermittent with starts).

Pneumatic or electric

Table 19.8 — Actuator choice for the skid (typical values).

Criterion	Pneumatic spring-return	Electric quarter-turn	Servo-electric linear
Fail-safe on power loss	Inherent (spring)	Needs a spring pack or battery module	Needs brake or spring
Speed	0.2–2 s per stroke	10–60 s per 90°	Programmable, fast
Washdown and hygiene	Excellent (IP69K, simple housing)	Good (IP67–68), larger	Good with stainless versions
Utilities	Air supply	24 V or 400 V	24–48 V or 400 V, drive
Positioning	On/off, or positioner for modulating	Modulating built in	Precise, with force control
Cost per DN50 valve	About $400–$700 with valve-top unit	About $900–$1 600	$1 500 and up

The 24 skid valves stay pneumatic: they need sub-second strokes for phase separation (Section 19.6), a spring for R7, and hygienic washdown. The one electric actuator on the system is on a DN150 effluent valve 120 m from the skid, where no air line exists: a 250 N·m quarter-turn actuator (the valve's breakaway torque of 120 N·m times a safety factor of 1.5 needs at least 180 N·m) with class A duty, a 30 s stroke, and a spring-return pack to close on power loss. The supply and return pumps are motor actuators of another kind; their starters and VFD are the subject of Chapter 20.

> **Boundary.**
> The machines inside actuators (induction, BLDC and stepper motors, gearing) are owned by *Electrical Energy and Machines*, Ch. 6 and Ch. 8; actuator sizing for robots and mechatronics by Chapter 13.

19.12 Verification and failure analysis

Verification

Table 19.9 — Verification of the skid instrumentation and actuation.

Req.	Test	Limit	Result	Pass/fail
R1	Return TT in dry block against reference probe at 60, 75, 85 °C	U95 ≤ 1.0 °C	Max deviation 0.21 °C; budget 0.49 °C	Pass
R2	Comparison with portable master meter at 10, 15, 20 m³/h	±2 %	−0.4 to +0.3 %	Pass
R3	Calibrator in simulate mode at 4, 12, 20, 22 mA with all loop elements fitted, supply at 21.6 V	Reading within 0.5 % span; 22 mA reached	Within 0.15 %; 22.0 mA reached after splitter fitted	Pass (failed before fix)
R4	Interface test: caustic followed by water, conductivity and valve logged, drain volume measured	≤ 5 s; ≤ 5 L	4.0–4.4 s; 2.6–3.8 L	Pass
R5	Stroke all 24 valves at 4.8 bar air	≤ 3 s, feedback correct	0.6–1.8 s; one valve top with swapped sensors found and corrected	Pass after fix
R6	Level against dipstick at 20 °C and 75 °C	±20 mm	±4 mm with density compensation (±21 mm without)	Pass
R7	Remove air; remove 24 V; disconnect IO-Link master	Valves to spring position, steam shut, pumps off	All as specified	Pass
R8	Open each manway with pumps running and in standby	No run	Pumps stop in < 50 ms; cannot start	Pass
R9	Material certificates, EHEDG / 3-A documents, IP69K ratings	All wetted parts and washdown devices	Complete; one non-IP69K junction box replaced	Pass
R10	24 h trend of caustic step	75 ± 2 °C	74.1–76.3 °C	Pass

Failure analysis

Table 19.10 — FMEA of the skid's sensors and actuators (selected modes).

Failure mode	Effect	Cause	Detection	Mitigation
Transmitter loop starved	Reading frozen at a plausible value	Extra series burden; low supply	None by NE 43; only by comparison or test at 22 mA	Loop budget for every loop; 22 mA test at SAT; active splitters for shared signals
Proximity sensor fails to switch when hot	False discrepancy alarms during caustic step	Gap beyond S_a for stainless target at temperature	Discrepancy logic	Design to $0.81 \times S_n \times$ material factor with margin
Silver contact on PLC input intermittent	Spurious manway-open trips	Oxide film at 3 mA	Event log	Gold-flash contacts for low-level circuits
Pilot valve slow to release	Late diversion, chemical to drain	Diode-only suppression	Interface test volume	Diode-Zener suppression on pilots
SSR fails short	Heater stays on, tank boils dry	Thyristor failure	Over-temperature switch	Upstream contactor opened by independent over-temperature and low-level switches
Air pressure low	Valves do not open fully, sequence hangs or cleans poorly	Compressor fault, leak	PSL at 4.8 bar	Force budget with margin at minimum pressure; air demand with margin of two
Conductivity sensor fouled	Late or missed phase change	Fat and protein deposits	Plausibility against expected interface time	Inductive (electrodeless) sensor; cleaning in the sequence itself

The first line is the opening story, and its detection column is the point: a starved loop produces a plausible, steady, wrong number that no automatic diagnostic sees. The mitigation is therefore procedural: every 4–20 mA loop has a one-line voltage budget in the design file, every loop is tested at 22 mA with all its elements in place at SAT, and any device that needs the same signal gets it from a splitter or over the network, never by adding a second receiver in series.

What remains open

Open question	Where it is answered
How the PLC, remote I/O and panels these devices connect to are designed and commissioned	Chapter 18
How motor starters, overloads, VFDs, interlocks and E-stops for the pumps are designed	Chapter 20
How IO-Link, Modbus and industrial Ethernet carry these signals, and how the manway interlock achieves its performance level	Chapter 21
How sensing principles are compared and calibrated in general	Chapter 6, Chapter 7
How the dead-time-dominated temperature loop is tuned	*Circuits, Signals and Communications*, Ch. 35 §35.3

Next: Chapter 20 takes the motors that drive the pumps, conveyors and fans of a plant and designs their starters, protection, drives, interlocks and emergency stops.

Chapter 20 — Industrial Motor Control

Part V · Industrial Automation

The validation of a new conveyor line at an aggregate quarry stops on its first morning. The inspector pulls the trip wire beside the 180 m incline conveyor with the belt running empty, and the belt takes 7.4 s to stop; the risk assessment allows 4 s. He then asks the integrator to put the PLC into STOP and pulls the wire again: nothing happens at all, because the pull-cord switches are wired to an ordinary PLC input and the PLC tells the drive to coast. Both failures have the same root. The motor-control design treated the emergency stop as one more input to the program, and the stopping performance as something the drive would sort out, instead of as a function with its own circuit, its own stop category and its own number written down before the panel was built. The fix took a safety relay, the drive's safe torque off, a brake chopper and two days of retesting. This chapter is about designing motor control so that such fixes are not needed.

> **How is each motor of a plant started, protected, controlled in speed, interlocked and stopped, and how are those choices traced to requirements?**
> **What goes into the motor-control panel, and how is the whole line commissioned and proved, emergency stops included?**

Concept	Engineering destination
Motor starters	DOL, star–delta, soft starter or VFD against dip, torque and cost
Contactors	AC-3 and AC-4 duty, electrical life, mirror contacts, reversing
Overload protection	Trip class against start time, hot restarts, MPCBs, Type 2 coordination
Variable-frequency drives	Duty rating, starting torque, braking, EMC and fieldbus interface
Speed regulation	Speed accuracy by control mode; process loops that set the speed
Interlocks	Start and stop sequences, permissives, reversing interlocks
Emergency stops	Stop categories, pull-cords, safety relays, STO, stopping-time validation
Fault monitoring	Belt slip, contactor discrepancy, first-out alarms
Control-panel design	MCC short-circuit rating, forms of separation, heat
Commissioning	Rotation, autotune, interlock and E-stop validation

Roadmap. The conveyor line and its requirements (20.1) → starters (20.2) → contactors (20.3) → overload protection (20.4) → VFDs (20.5) → speed regulation (20.6) → interlocks (20.7) → emergency stops (20.8) → fault monitoring (20.9) → the motor-control panel (20.10) → commissioning, verification and failure analysis (20.11).

20.1 The conveyor line and its requirements

The reference system

The running example is the **motor control of an aggregate conveyor line** in a quarry: crushed rock at 150–300 t/h from a surge bin to a stockpile. Seven motors are fed from one motor-control centre (MCC) on a 400 V, 50 Hz TN-S supply from a 630 kVA, 6 % transformer, with a 400 kVA standby generator that must be able to run the whole line.

Table 20.1 — Motors of the conveyor line (400 V, 4-pole, IE3; currents typical).

Tag	Duty	Rating	Full-load current	Starter (Section 20.2)
F1	Apron feeder under the surge bin; sets the feed rate	7.5 kW	14.6 A	VFD
C1	Incline conveyor, 180 m, lift 18 m, 2.5 m/s	75 kW	136 A	VFD, heavy duty, brake chopper
C2	Transfer conveyor, 60 m, 2.0 m/s	22 kW	40 A	Soft starter
C3	Stockpile conveyor, 80 m, 2.0 m/s	11 kW	21 A	DOL
D1	Shuttle diverter at the stockpile	1.5 kW	3.3 A	Reversing DOL
X1	Dust-extraction fan	30 kW	55 A	Star–delta
P1	Sump pump	4 kW	8.0 A	DOL

Figure 20.1 is the single-line diagram of the MCC.

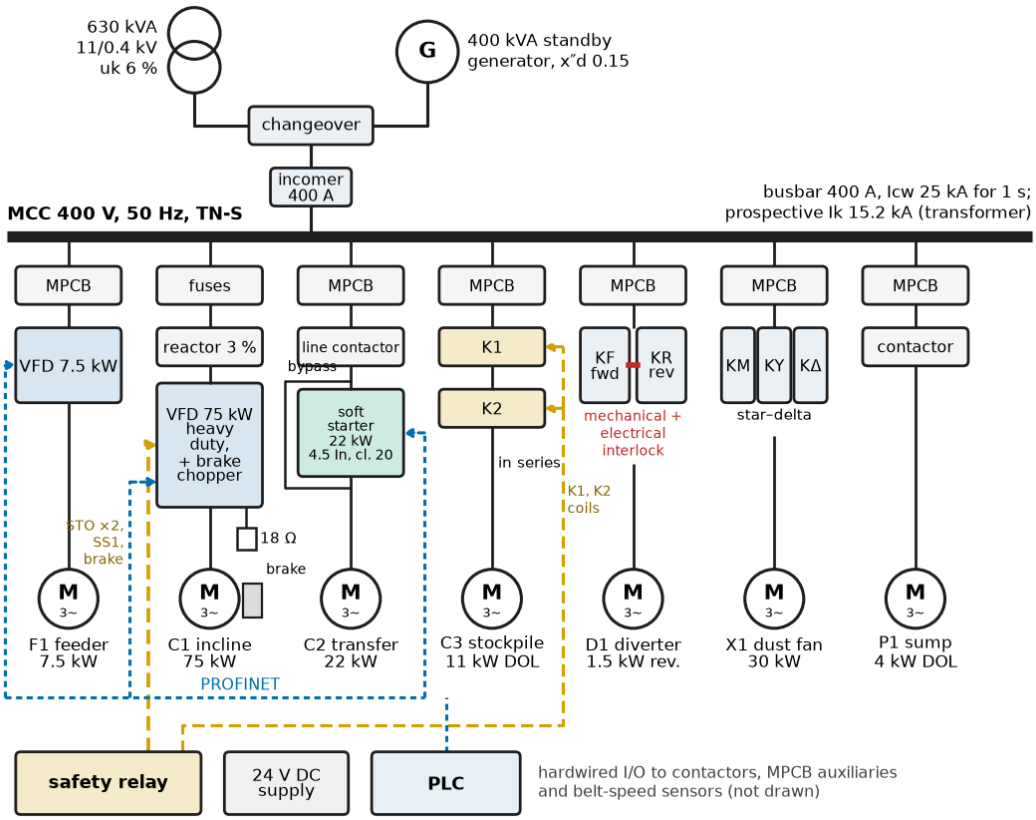


Figure 20.1 — Single-line diagram of the conveyor-line motor-control centre.

Requirements

Table 20.2 — Requirements for the motor control of the conveyor line.

ID	Requirement	Target	Verification
R1	Start and stop sequence	Start downstream first, each conveyor proven at speed before the next starts; stop upstream first after belt-clearing time	Sequence test with every permissive (§20.7)
R2	Belt slip and stall	Trip if belt speed < 80 % of nominal for > 3 s once running	Simulated slip test (§20.9)
R3	Feed rate	150–300 t/h, ±3 % on 10-min average	Belt-weigher trend (§20.6)
R4	Emergency stop	Any pull-cord or E-stop stops the conveyors it protects within 4 s at any load, independently of the standard PLC; latched, manual reset	Stopping-time tests, PLC-in-STOP test (§20.8)
R5	Motor overload protection	Every motor protected; no nuisance trip on a loaded restart within 15 min of a stop	Start tests hot and cold (§20.4)
R6	Short-circuit protection	Type 2 coordination (IEC 60947-4-1) at the prospective fault level	Manufacturer coordination tables; Isc calculation (§20.10)
R7	Supply dip on starting	≤ 5 % at the MCC busbar on transformer or generator	Dip calculation, recorded start on generator (§20.2)
R8	MCC environment	IP54 (quarry dust); internal ≤ 40 °C at 45 °C ambient	Heat budget, heat run (§20.10)
R9	Fault annunciation	Every trip reported on the HMI with its first-out cause within 1 s	Fault-injection tests (§20.9)
R10	Commissioning record	Every interlock and E-stop tested, stopping times recorded	Test records (§20.11)

R4's 4 s comes from the risk assessment of the tail pulley of C1, where a person caught in the in-running nip must be released before the belt has drawn them in; its performance level (PL d) and its proof belong to Chapter 21.

> **Boundary.**
> Starting methods and their current and torque, drive fundamentals, braking and drive protection are owned by *Electrical Energy and Machines*: Ch. 6 §6.6, Ch. 10 §10.5 and §10.7 and Ch. 12 §12.3 and §12.9; the VFD's power stage and gate drive by *Electronics and Computing*, Ch. 32 §32.4. This chapter selects, wires, interlocks and commissions them. The PLC and panels in general are owned by Chapter 18, the sensors on the conveyors by Chapter 19.

20.2 Motor starters

The options

Four ways of starting a cage induction motor cover nearly every industrial case. *Electrical Energy and Machines* Ch. 6 §6.6 gives their currents and torques: direct-on-line (DOL) draws 6–8 times rated current

with full starting torque; star–delta cuts line current and torque to one third; a soft starter limits current to 2.5–4.5 times rated with torque falling as the square of the current ratio; a VFD gives rated torque at about rated current. Here the question is which to fit to each of seven motors, and the answer comes from four requirements at once: the supply dip (R7), the torque the load needs at standstill, the mechanical stress the load can accept, and whether speed control is needed anyway.

The supply dip

A motor drawing apparent power S_{start} from a source of short-circuit power S_{sc} dips the busbar voltage by roughly

$$\frac{\Delta V}{V} \approx \frac{S_{start}}{S_{sc}}$$

> **Status of the principle: Starting dip from short-circuit power**
> *Approximate.* Treats source and motor impedances as in phase and the source as a constant impedance; it overstates the dip slightly when the motor's starting power factor (0.3–0.5) differs from the source's X/R angle, and for a generator it uses the subtransient reactance, ignoring the voltage regulator's recovery over the next 0.5–2 s. The phasor calculation is in *Electrical Energy and Machines*, Ch. 6 §6.6.

Worked example — the dip on the generator. On the transformer, $S_{sc} = 630/0.06 = 10.5$ MVA; on the 400 kVA generator with subtransient reactance 0.15 pu, $S_{sc} = 400/0.15 = 2.67$ MVA. The generator decides:

Table 20.3 — Starting dip at the MCC busbar.

Motor and starter	Starting current	S_{start}	Dip on transformer	Dip on generator	R7 (5 %)
X1 30 kW, DOL	385 A	267 kVA	2.5 %	10.0 %	Fails on generator
X1 30 kW, star–delta	128 A	89 kVA	0.8 %	3.3 %	Pass
X1 30 kW, soft starter $3.5 \times I_n$	193 A	133 kVA	1.3 %	5.0 %	Marginal
C2 22 kW, soft starter $4.5 \times I_n$	180 A	125 kVA	1.2 %	4.7 %	Pass
C3 11 kW, DOL	147 A	102 kVA	1.0 %	3.8 %	Pass
P1 4 kW, DOL	56 A	39 kVA	0.4 %	1.5 %	Pass

On the transformer every motor could start DOL. On the generator the fan cannot, and it gets a star–delta starter, the cheapest method that passes. Star–delta suits a fan because a fan needs little torque at low speed; it would not suit a loaded conveyor, which needs close to rated torque to break away.

Starter selection

Table 20.4 — Starter selection for the conveyor line.

Motor	Load at start	DOL	Star–delta	Soft starter	VFD	Choice and reason
F1 feeder	Full bin on the apron: high breakaway	Possible	No	No	Yes	**VFD**: speed sets the feed rate (R3)
C1 incline	Loaded belt, 18 m lift; 1.1 × rated torque at breakaway	Torque shock, belt slip	No	Insufficient torque at 4 × I_n	Yes	**VFD** heavy duty: controlled torque, controlled stop (R4)
C2 transfer	Loaded level belt, about 0.5–0.65 × rated	Belt stress	No	Yes with high current limit	Yes	**Soft starter**: cheaper; no speed control needed
C3 stockpile	Short belt, light	Yes	—	—	—	**DOL**: cheapest, dip 3.8 %
D1 diverter	Light, reversing	Yes	—	—	—	**Reversing DOL**
X1 fan	Low torque at low speed	Fails R7 on generator	Yes	Marginal	Not needed	**Star–delta**
P1 sump pump	Low	Yes	—	—	—	**DOL**

A conveyor's real objection to DOL is mechanical, not electrical: DOL starting torque of two to three times rated, applied in a fraction of a second, slips the belt on the drive pulley, overstresses splices and lifts the belt off its idlers on concave curves. Conveyor design practice (DIN 22101, CEMA) limits starting torque to about 1.3–1.5 times the running requirement and acceleration to about 0.1–0.3 m/s², which is why long conveyors get soft starters, fluid couplings or VFDs.

20.3 Contactors

Ratings that matter for motors

The contactor is the switching element of every starter in Table 20.4 except the VFD's output. Its motor ratings come from IEC 60947-4-1 (introduced in Chapter 19 for the general case):

- **AC-3**: starting a cage motor and switching it off while running. Making about 6–8 × I_e, breaking about 1 × I_e at reduced voltage (the running motor's back-emf helps). The headline rating: an "11 kW contactor" is 11 kW AC-3 at 400 V.
- **AC-4**: plugging (reversing while running) and inching. The contactor breaks 6–8 × I_e at full voltage. Electrical life at AC-4 is 10 to 50 times shorter than at AC-3 for the same current.
- **Electrical life** is given as a curve of operations against breaking current; mechanical life (10–20 million operations) is rarely the limit.
- **Mirror contacts** (IEC 60947-4-1 Annex F): an auxiliary NC contact that cannot be closed while any main contact is closed, even if a main contact is welded. It is the feedback that lets a safety relay detect a welded contactor and refuse to restart (Section 20.8).

Reversing

The diverter D1 is driven by two contactors that swap two phases. Both closed at once is a phase-to-phase short circuit, so a reversing starter has three interlocks: a **mechanical interlock** between the two contactors (a rocker that physically prevents both armatures closing), an **electrical interlock** (an NC auxiliary of each contactor in the other's coil circuit), and a **time interlock** in the PLC (0.5 s off before reversing), which also turns plugging (AC-4) into ordinary AC-3 duty because the motor has nearly stopped when it reverses.

Worked example — the life of the reversing contactors. The shuttle diverter alternates every minute to layer the stockpile: 60 operations an hour, 16 h a day, 300 days a year, 288 000 operations a year, half per contactor. A 9 A AC-3 contactor switching a 3.3 A motor has an AC-3 electrical life of typically 2–3 million operations at that current: about 14–20 years. Plugged without the time interlock, the same contactor is at AC-4, where a typical life curve gives about 200 000 operations: 1.4 years. The 0.5 s pause in the PLC multiplies contactor life by ten; it is written into the functional design specification, not left to the programmer's taste.

> **Status of the principle: Contactor electrical life**
> *Empirical.* Taken from manufacturers' life curves measured under IEC 60947-4-1 test conditions for each utilisation category; actual life varies with supply voltage, inrush, ambient and switching frequency, so designs use the curve with a margin and inspect contacts at intervals.

20.4 Overload protection

The protective chain of one motor

Each DOL, star–delta or soft-started motor has a **motor-protection circuit breaker** (MPCB) or a fuse–overload combination. The MPCB combines a magnetic trip at about 12–14 × setting with an overload trip set to full-load current; drives and soft starters carry their own electronic motor model.

The overload relay must trip on a sustained overload but not during a start. Its **trip class** (IEC 60947-4-1: class 10, 20, 30, with a cold trip at 7.2 × setting within 4–10 s, 6–20 s and 9–30 s) is chosen against the start time, and the dangerous case is a **hot restart**: a conveyor tripped or E-stopped under full load and restarted at once, when the relay's thermal model is already near its running state. *Electrical Energy and Machines*, Ch. 10 §10.7, gives the single-time-constant thermal model behind the relay; Listing 20.1 uses it.

Sizing the soft start of C2

C2 restarts loaded after every stop of the line. Its total inertia referred to the motor shaft is 1.135 kg·m² (belt, idlers and 2.1 t of rock moving at 2.0 m/s, plus motor and gearbox), its running torque is 0.50 × rated normally and 0.65 × rated on a cold winter morning when the belt and idler grease are stiff. Under current limit $k \times I_n$ the motor's torque is about $T_{LR}(k/k_{LR})^2$ (Chapter 6 of *Electrical Energy and Machines*), with locked-rotor torque 2.3 × rated at 7 × rated current.

```
 # Soft-starter start of conveyor C2 (22 kW) versus overload trip class (hot restart)
import math
TN, W, J = 142.9, 153.9, 1.135        # rated torque N m, speed rad/s, total J kg m^2
LRT, ILR = 2.3, 7.0                   # locked-rotor torque (x Tn) and current (x In)
KT = 1.15                             # relay trips when theta reaches KT^2

def tau(cls):                         # time constant giving the class trip at 7.2 x, cold
    t_cold = 0.8 * cls                # typical trip time: 8, 16, 24 s
    return t_cold / math.log(7.2**2 / (7.2**2 - KT**2))

def t_trip(k, cls, theta0):           # first-order I^2 model from initial state theta0
    return tau(cls) * math.log((k**2 - theta0) / (k**2 - KT**2))

for load in (0.50, 0.65):             # running torque x Tn: normal, cold belt
    for k in (3.5, 4.0, 4.5):         # soft-starter current limit x In
        t_mot = LRT * (k / ILR) ** 2  # motor torque at current limit (x Tn)
        acc = (t_mot - load) * TN
        t_start = J * W / acc if acc > 0 else math.inf
        trips = "  ".join(f"cl{c}:{t_trip(k, c, 1.0):5.1f}" for c in (10, 20, 30))
        print(f"load {load:.2f} Tn, limit {k:.1f} In: torque {t_mot:.2f} Tn, "
              f"start {t_start:5.1f} s | hot trip s {trips}")
```

```
 load 0.50 Tn, limit 3.5 In: torque 0.57 Tn, start  16.3 s | hot trip s cl10:  9.0  cl20: 18.0
cl30: 27.0
load 0.50 Tn, limit 4.0 In: torque 0.75 Tn, start   4.9 s | hot trip s cl10:  6.7  cl20: 13.5
cl30: 20.2
load 0.50 Tn, limit 4.5 In: torque 0.95 Tn, start   2.7 s | hot trip s cl10:  5.2  cl20: 10.5
cl30: 15.7
load 0.65 Tn, limit 3.5 In: torque 0.57 Tn, start   inf s | hot trip s cl10:  9.0  cl20: 18.0
cl30: 27.0
load 0.65 Tn, limit 4.0 In: torque 0.75 Tn, start  12.1 s | hot trip s cl10:  6.7  cl20: 13.5
cl30: 20.2
load 0.65 Tn, limit 4.5 In: torque 0.95 Tn, start   4.1 s | hot trip s cl10:  5.2  cl20: 10.5
cl30: 15.7
```

Worked example — current limit and trip class together. The soft starter's default (4.0 × I_n, class 10) starts C2 in 4.9 s on a normal day, inside the hot trip time of 6.7 s, and would pass a summer commissioning test. On a cold morning the same setting takes 12.1 s and the overload trips at 6.7 s on a hot restart: a nuisance trip that would appear in the first winter. At 3.5 × I_n the motor cannot start the cold belt at all (0.57 × rated torque against 0.65). The design takes 4.5 × I_n (0.95 × rated torque, 4.1 s worst-case start) with class 20 (10.5 s hot): a margin of 2.6 on time. The 4.5 × I_n is also the setting whose dip on the generator was checked in Table 20.3 (4.7 %). Current limit, trip class and supply dip are one decision, made with one table. Figure 20.2 shows the start current against the trip curves.

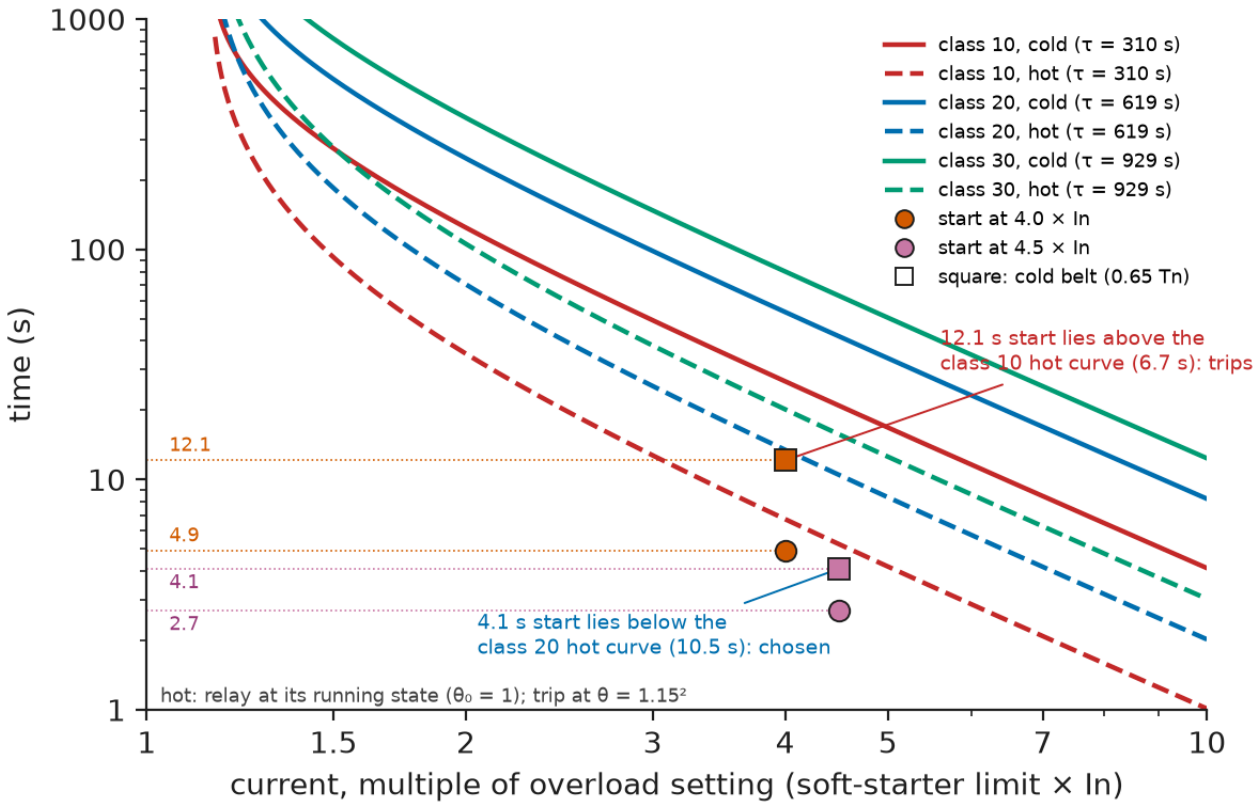


Figure 20.2 — Overload trip curves of class 10, 20 and 30 (cold and hot) against the soft-start current of conveyor C2.

> **Status of the principle: Thermal overload model for trip-class selection**
> *Approximate.* A single time constant fitted to the class test point at 7.2 × setting, with the hot state taken as continuous operation at setting current; real relays and motor models differ in their hot/cold ratio, so the vendor's own hot curve must be checked before settings are frozen.

Short-circuit protection and coordination

The MPCB or fuse must also clear a short circuit at the motor terminals or in the cable before the cable or the contactor is destroyed. IEC 60947-4-1 defines two levels of **coordination** between the short-circuit device, the contactor and the overload relay at a stated prospective current: **Type 1**, the starter may be unusable after the fault; **Type 2**, the starter must remain usable, with only light contact welding that can be separated. R6 requires Type 2, so every starter is built from a manufacturer's tested combination listed for the MCC's prospective current (Section 20.10). Cable protection follows *Electrical Energy and Machines*, Ch. 29 §29.5.

20.5 Variable-frequency drives

Selecting the C1 drive

A VFD is specified by more than its kilowatts. For C1 the checks are:

1. **Duty**: drives are rated for **normal duty** (110 % for 60 s every 10 min; fans and pumps) and **heavy duty** (150 % for 60 s; conveyors, crushers, hoists). The same frame typically carries one size higher in

normal duty.

2. **Starting torque**: running at full load, C1 needs 52 kW at the head pulley, $52/0.95 = 54.7$ kW at the motor (gearbox efficiency 0.95), or 0.73 × rated torque. At breakaway, static friction raises the running resistance by about 30 % and the 10 s acceleration ramp to 2.5 m/s (0.25 m/s²) adds $22\,070 \times 0.25 = 5.5$ kN on an equivalent moving mass of 22.1 t. The total, $1.3 \times 20.8 + 5.5 = 32.6$ kN at the belt, is 553 N·m at the motor, 1.14 × rated torque.
3. **Current**: with a magnetising current of about 0.35 × rated, the motor current at 1.14 × rated torque is about $I_n\sqrt{0.35^2 + 1.14^2(1 - 0.35^2)} = 1.12\,I_n = 153$ A.
4. **Braking**: whether the drive must absorb regenerated energy (Section 20.8).
5. **Environment and EMC**: IP54 or a cabinet; category C3 (industrial) EMC filter to IEC 61800-3; line reactor for harmonics and DC-link capacitor life; shielded symmetrical motor cable.
6. **Interface**: PROFINET for reference, status and diagnostics; hardwired STO and a hardwired run permit for anything safety-related.

Worked example — normal or heavy duty? A drive sold as "75 kW" in normal duty is typically rated about 145 A continuous with 110 % (160 A) for 60 s. That would carry the 153 A breakaway, but with 4 % margin, and nothing for a cold morning or a belt left loaded after a trip. A frame rated 75 kW in heavy duty (sold as 90 kW normal duty), 145 A continuous with 150 % (218 A) for 60 s, gives 42 % margin on breakaway current. The heavy-duty rating costs about 20 % more (2026 indicative: about $7 500 against $6 200 for the drive alone) and is chosen, because a conveyor that cannot restart loaded after an E-stop must be dug out by hand.

What else the drive brings

The drive also brings, at no extra cost, a programmable ramp, a motor thermal model that includes reduced cooling at low speed, a thermistor input, underload detection (a broken belt shows as low torque at speed) and fieldbus diagnostics. Its costs are harmonic current on the supply (the 3 % line reactor holds the drive's current THD to about 35–40 %; *Electrical Energy and Machines*, Ch. 25 §25.3), common-mode currents in motor bearings, and 1.5 kW of losses to remove from the MCC (Section 20.10).

20.6 Speed regulation

How well does a drive hold speed?

A drive regulates speed only as well as its control mode allows (*Electrical Energy and Machines*, Ch. 12 §12.3–12.6):

Table 20.5 — Steady-state speed accuracy by control mode (typical, 4-pole induction motor with rated slip about 2 %).

Mode	Speed error from no load to full load	Typical use
Open-loop V/f	About rated slip: 1.5–3 %	Fans, pumps
V/f with slip compensation	0.3–0.5 %	Conveyors, mixers
Sensorless vector	0.1–0.5 %	Conveyors, extruders, general machinery
Closed-loop vector with encoder	0.01 %	Winders, hoists, test rigs

No conveyor here needs better than 1 %. What matters is what sets the speed: the feeder F1's speed is set by an outer loop holding the tonnage measured by a belt weigher on C1.

The feed-rate loop

The belt weigher sits 30 m along C1 from the feeder discharge. Rock leaving the feeder reaches it after 30/2.5 = 12 s, so the loop from feeder speed to measured tonnage is dominated by that **transport delay**. Modelled as first order plus dead time (gain 3.0 t/h per % speed, 2 s lag from the feeder and chute, 12 s delay), it is tuned by the SIMC rules (*Circuits, Signals and Communications*, Ch. 35 §35.3) with the closed-loop time constant equal to the delay: $K_c = \tau / (K(\tau_c + \theta)) = 2/(3 \times 24) = 0.028$ %/(t/h), $T_i = \min(\tau, 4(\tau_c + \theta)) = 2$ s. Listing 20.2 simulates the loop, executed in a 0.5 s PLC cyclic task, for a setpoint step from 250 to 200 t/h and then a 15 % drop in bulk density of the rock, and compares a gain twice as high.

```
 # Feed-rate loop: feeder VFD speed -> belt-weigher tonnage, PI in a 0.5 s PLC task
import numpy as np
K, TAU, THETA = 3.0, 2.0, 12.0          # t/h per % speed, s, s (transport delay)
DT, TS = 0.05, 0.5                       # simulation step, PLC task period
def simulate(kc, ti, t_end=240.0):
    n = int(t_end / DT); delay = int(THETA / DT)
    u_hist = np.full(n + delay, 250.0 / K)   # start in steady state at 250 t/h
    y, yf, integ, u = 250.0, 250.0, 250.0 / K, 250.0 / K
    out = np.empty(n)
    for k in range(n):
        t = k * DT
        sp = 250.0 if t < 20 else 200.0                # operator setpoint step
        dist = 0.0 if t < 130 else -0.15               # ore density drops 15 %
        if k % int(TS / DT) == 0:                      # PLC cyclic task
            e = sp - yf
            integ = np.clip(integ + kc * e * TS / ti, 0, 100)   # anti-windup
            u = np.clip(kc * e + integ, 0, 100)                 # % feeder speed
        u_hist[k + delay] = u
        y += DT / TAU * (K * (1 + dist) * u_hist[k] - y)        # chute/feeder lag
        yf += DT / 1.0 * (y - yf)                               # 1 s weigher filter
        out[k] = yf
    return out

t = np.arange(int(240 / DT)) * DT
for name, kc, ti in [("SIMC tau_c = theta", 2.0 / (3.0 * 24.0), 2.0),
                     ("aggressive x2", 2 * 2.0 / (3.0 * 24.0), 2.0)]:
    y = simulate(kc, ti)
    w = (t >= 20) & (t < 130)
    settle = t[w][np.abs(y[w] - 200) > 4][-1] - 20      # last exit from +/-2 %
    d = t >= 130
    rec = t[d][np.abs(y[d] - 200) > 4][-1] - 130
    print(f"{name:18s} Kc={kc:.4f} %/(t/h): undershoot {200 - y[w].min():4.1f} t/h, "
          f"settles {settle:4.1f} s; disturbance dip to {y[d].min():5.1f} t/h, "
          f"recovers {rec:4.1f} s")
```

```
 SIMC tau_c = theta Kc=0.0278 %/(t/h): undershoot  3.4 t/h, settles 37.8 s; disturbance dip to
170.2 t/h, recovers 45.1 s
aggressive x2      Kc=0.0556 %/(t/h): undershoot 28.9 t/h, settles 107.2 s; disturbance dip to
173.5 t/h, recovers 77.5 s
```

Worked example — what the simulation says. With the SIMC tuning the tonnage settles within ±2 % of the new setpoint 38 s after the step, undershooting by only 3.4 t/h; after the density drop it dips to 170 t/h, because the controller's correction needs 12 s to reach the weigher, and recovers in 45 s. On a 10-min average both transients cost well under 1 %, inside R3's 3 %. Doubling the gain, the natural reaction to a "sluggish" loop, makes it worse on both counts: 29 t/h of undershoot and 107 s to settle. A dead-time-dominated loop cannot be made faster than its delay, only worse. Figure 20.3 plots both responses.

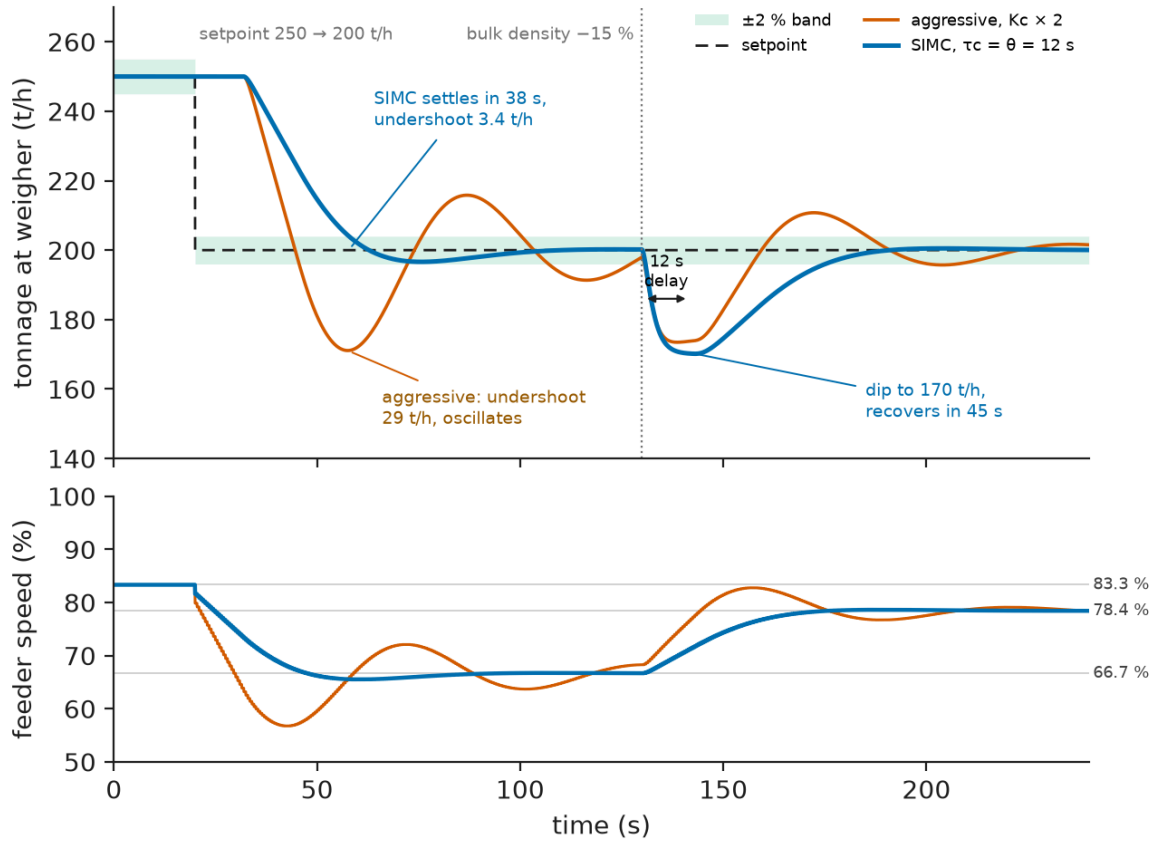


Figure 20.3 — Simulated feed-rate loop: tonnage at the belt weigher and feeder speed for a setpoint step and a density disturbance.

> **Status of the principle: First-order-plus-dead-time model of a conveyor feed loop**
> *Approximate.* Treats the feeder, chute and belt as a gain, a single lag and a pure delay at constant belt speed; it ignores the feeder's nonlinearity with bin level and material, and the delay changes if C1's speed changes, so the loop is retuned whenever the belt speed is altered.

20.7 Interlocks

Sequence interlocks and permissives

An **interlock** prevents an action, or forces one, on the state of something else. A conveyor line has three kinds:

- **Sequence interlocks** (R1): a conveyor may run only while the conveyor it discharges onto is running at speed. Starting runs downstream to upstream (C3, C2, C1, then F1); stopping runs upstream to downstream, each belt running on for its **clearing time** (length over speed: 72 s for C1, 30 s for C2, 40 s for C3) so that no belt stops loaded. If a downstream conveyor stops for any reason, every upstream conveyor stops at once; otherwise C1 would bury the transfer point of a stopped C2 under 250 t/h of rock.
- **Permissives**: conditions that must be true to start but are not checked while running (air pressure available for the dust-filter cleaning, the diverter at an end position, no maintenance lock-off).

- **Equipment interlocks**: reversing interlocks (Section 20.3), and the dust fan X1 running before C2 may start.

"Running" must be proven, not assumed. A contactor's auxiliary contact proves only that the contactor closed, not that the belt moves. Each conveyor has a **belt-speed monitor**, an inductive sensor on a tail-pulley target (Chapter 19), and the sequence uses its "at speed" signal.

The sequence in Structured Text

Listing 20.3 is the line sequence in IEC 61131-3 Structured Text, using the same step-and-stopwatch pattern as the filter backwash of Chapter 18. The run commands are decoded from the state and gated by the downstream speed signals, so the cascade stop of an upstream conveyor happens in the same scan in which a downstream one is lost.

```
 FUNCTION_BLOCK FB_ConveyorLine
VAR_INPUT
    xStart, xStop, xSafetyOK, xFault : BOOL;  (* xSafetyOK: status of safety relay *)
    xAt3, xAt2, xAt1 : BOOL;                   (* belt at speed: C3, C2, C1 *)
END_VAR
VAR_OUTPUT
    xRun3, xRun2, xRun1, xRunF : BOOL;
    iState : INT := 0;
END_VAR
VAR
    iLast : INT := -1;
    tState : TON;
END_VAR

tState(IN := (iState = iLast), PT := T#1H);    (* stopwatch, reset on change *)
iLast := iState;
IF xFault OR NOT xSafetyOK THEN iState := 0; END_IF;     (* trip: all stop *)

CASE iState OF
  0:  IF xStart AND xSafetyOK AND NOT xFault THEN iState := 10; END_IF;
  10: IF xAt3 THEN iState := 20; ELSIF tState.ET > T#10S THEN iState := 0; END_IF;
  20: IF xAt2 THEN iState := 30; ELSIF tState.ET > T#15S THEN iState := 0; END_IF;
  30: IF xAt1 THEN iState := 40; ELSIF tState.ET > T#20S THEN iState := 0; END_IF;
  40: IF xStop THEN iState := 50; END_IF;                  (* running, feeding *)
  50: IF tState.ET >= T#80S THEN iState := 60; END_IF;    (* clear C1: 72 s *)
  60: IF tState.ET >= T#35S THEN iState := 70; END_IF;    (* clear C2: 30 s *)
  70: IF tState.ET >= T#45S THEN iState := 0; END_IF;     (* clear C3: 40 s *)
END_CASE;

xRun3 := (iState >= 10) AND (iState <= 70);
xRun2 := (iState >= 20) AND (iState <= 60) AND xAt3;
xRun1 := (iState >= 30) AND (iState <= 50) AND xAt2 AND xAt3;
xRunF := (iState = 40) AND xAt1 AND xAt2 AND xAt3;
END_FUNCTION_BLOCK
```

Each start step has a timeout, so a conveyor that does not reach speed aborts the start rather than leaving the line half-running. `xSafetyOK` is only a status: the emergency stop does not pass through this code (Section

20.8), but when it acts, this block drops to state 0, so that nothing restarts by itself when the safety relay is reset.

20.8 Emergency stops

Stop categories and what an E-stop must do

IEC 60204-1 defines three **stop categories**: **0**, immediate removal of power (the motor coasts or is braked mechanically); **1**, a controlled stop with power, then removal of power; **2**, a controlled stop with power left on. ISO 13850 requires an emergency stop to be category 0 or 1, chosen by risk assessment, to override every other function, to latch, and to need a deliberate manual reset that does not itself restart the machine.

For a conveyor the E-stop devices are buttons at the drives and **pull-cord switches** (rope-pull switches to IEC 60947-5-5) along the walkways: a wire rope on both sides of the belt that operates a latching switch with positive-opening contacts when pulled from any point, or when it breaks or goes slack. With switches serving up to 50 m of rope each, C1's 180 m needs four switches per side.

> **Boundary.**
> The required performance level, the category of the architecture, MTTFd and diagnostic coverage of the E-stop function, and the fault-masking limits on pull-cords wired in series, are owned by Chapter 21; STO and SS1 inside the drive by *Electrical Energy and Machines*, Ch. 10 §10.7. This section designs the circuits and their stopping performance.

Stopping C1: why the first design failed

The opening's first failure is kinematics. On a category 0 stop the belt is decelerated only by its own running resistance. Loaded, that resistance is 20.8 kN acting on an equivalent moving mass of 22.1 t (belt, idlers, rock and the motor's inertia referred to the belt); empty, the material's lift and friction are gone, leaving about 6 kN on 17.1 t.

Worked example — stopping time and the brake chopper. For a constant deceleration a from $v = 2.5$ m/s, time v/a and distance $v^2/2a$:

Table 20.6 — Stopping of C1 from 2.5 m/s.

Case	Deceleration	Time	Distance	R4 (4 s)
Category 0, loaded (20.8 kN on 22.1 t)	0.94 m/s²	2.65 s	3.3 m	Pass
Category 0, empty (6.0 kN on 17.1 t)	0.35 m/s²	7.1 s	8.9 m	**Fail** (measured 7.4 s)
Category 1, 2.5 s ramp, brake chopper	1.0 m/s²	2.5 s	3.1 m	Pass
Ramp not followed: SS1 ramp monitor trips STO after 0.3 s, spring brake 250 N·m, empty	1.26 m/s²	0.3 + 2.0 = 2.3 s	3.2 m	Pass

The empty belt is the worst case: it coasts for 7.1 s because there is almost nothing to stop it. The redesign makes the E-stop a **category 1** stop: the drive ramps the belt to rest in 2.5 s and then removes torque. Decelerating the empty belt at 1.0 m/s² needs a braking force of $17\,070 \times 1.0 - 6000 = 11.1$ kN at the belt, a regenerated power of up to 27.7 kW and an energy of about 35 kJ per stop, which a diode-front-end drive can only dump into a **brake resistor** through its chopper. With the chopper switching on at about 760 V on the DC link, the resistor must be at most $760^2/27\,700 = 20.9\ \Omega$; an 18 Ω resistor rated for the E-stop duty (one full stop, then a cool-down) is fitted outside the MCC. Loaded, the belt already decelerates at 0.94 m/s² and the drive brakes only 1.3 kN.

> **Status of the principle: Constant-deceleration stopping**
> *Derived.* Time and distance follow from Newton's second law with a constant retarding force on a rigid equivalent mass; real belts stretch, so the tail pulley stops a little later than the head pulley, and running resistance rises as the belt slows, making the estimate slightly pessimistic for coasting.

The circuit

Figure 20.4 shows the redesigned E-stop of C1. The pull-cords and E-stop buttons, dual-channel, go to a **safety relay** (or the safety program of a safety PLC), not to the standard PLC. On a demand the safety relay:

1. opens its instantaneous outputs, which command the drive's safe-stop input: the drive executes **SS1** (safe stop 1, IEC 61800-5-2) with its fast ramp of 2.5 s and **monitors** that the speed follows the ramp;
2. after 3.0 s, opens its delayed outputs to the drive's two **STO** channels and to the **safe brake control** output that drops the spring-applied motor brake;
3. if the drive's ramp monitor sees speed failing to fall, it applies STO and the brake at once (the last row of Table 20.6).

The standard PLC receives only a status contact. Contactor-started conveyors stop in category 0 through two contactors in series (C3: K1 and K2), each with a mirror contact in the safety relay's feedback loop, so that a welded contactor stops the relay re-arming. Reset is manual, at a button with sight of the conveyor, and starts nothing.

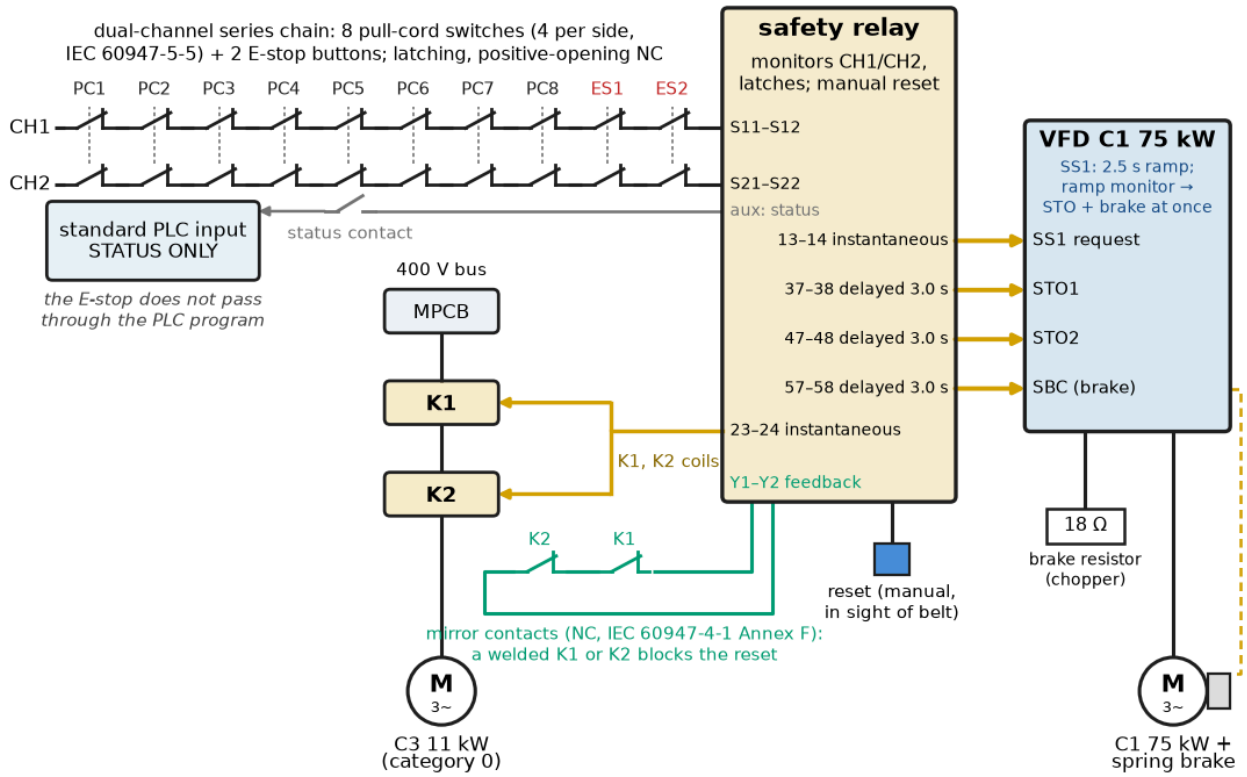


Figure 20.4 — Emergency-stop circuit of incline conveyor C1 and stockpile conveyor C3.

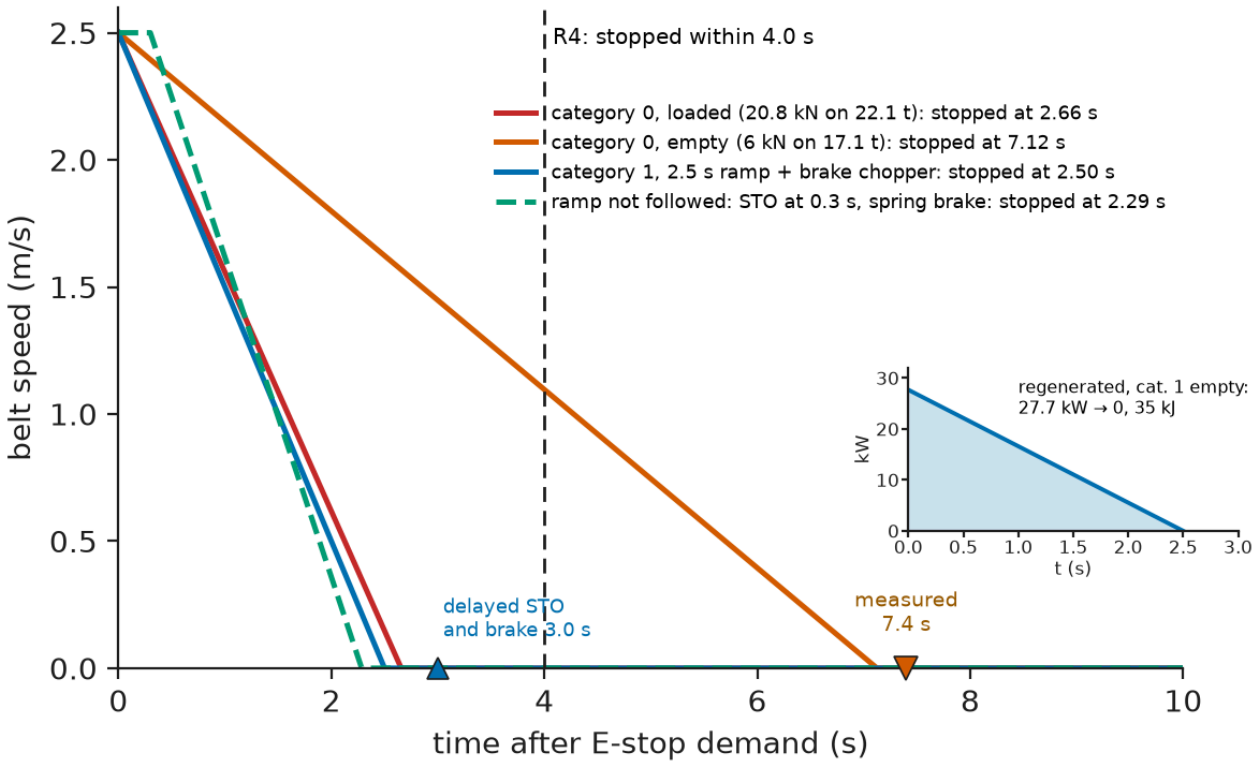


Figure 20.5 — Belt speed of incline conveyor C1 after an emergency stop for the cases of Table 20.6.

Figure 20.5 plots the four stopping cases against the 4 s limit.

20.9 Fault monitoring

What is monitored

Table 20.7 — Fault monitoring on the conveyor line.

Fault	Sensor or source	Response
Overload or MPCB trip	MPCB auxiliary contact	Trip motor and upstream; alarm with tag
Drive or soft-starter fault	Fault word over PROFINET, plus hardwired fault relay	Trip; fault code shown in plain text
Belt slip or stall	Tail-pulley speed sensor below 80 % for 3 s after acceleration (R2)	Trip; a slipping belt on a lagged pulley can start a fire
Belt drift	Drift switches at both sides, both ends	Alarm at first stage, trip at second
Blocked chute	Tilt switch in each transfer chute	Trip upstream
Contactor welded or failed to close	Auxiliary contact against command after 1 s	Alarm, block restart
Phase loss or unbalance	Monitoring relay on the incomer	Block starts
Motor winding temperature	PTC thermistors on C1 (drive) and X1 (relay)	Trip
Pull-cord operated	Safety relay status	Stop; locate by walking the line, or by addressable switches (Chapter 21)

R9 requires the **first-out** cause. When C2 trips, C1 and F1 stop by interlock and their "not at speed" conditions would also alarm; the PLC latches the first fault in a scan and suppresses the consequential ones, so that the operator sees "C2 belt slip" and not four alarms (the alarm philosophy of Chapter 18, Section 18.6).

A monitoring block

Listing 20.4 shows the per-motor monitoring block: start failure, welded contactor and belt slip, each with its own delay, latched until reset.

```
 FUNCTION_BLOCK FB_MotorMonitor
VAR_INPUT
    xCmd, xFeedback : BOOL;          (* run command; contactor aux or drive running *)
    rSpeedPct       : REAL;          (* belt speed, % of nominal *)
    xReset          : BOOL;
END_VAR
VAR_OUTPUT
    xFailStart, xWelded, xSlip : BOOL;
END_VAR
VAR
    tonStart, tonWeld, tonAccel, tonSlip : TON;
END_VAR

tonStart(IN := xCmd AND NOT xFeedback, PT := T#1S);
tonWeld(IN := NOT xCmd AND xFeedback, PT := T#1S);
tonAccel(IN := xCmd AND xFeedback, PT := T#10S);          (* allow acceleration *)
tonSlip(IN := tonAccel.Q AND (rSpeedPct < 80.0), PT := T#3S);  (* R2 *)
xFailStart := (xFailStart OR tonStart.Q) AND NOT xReset;
xWelded    := (xWelded OR tonWeld.Q) AND NOT xReset;
xSlip      := (xSlip OR tonSlip.Q) AND NOT xReset;
END_FUNCTION_BLOCK
```

The 10 s acceleration allowance covers C2's 4.1 s worst-case start; for C1, with its 10 s ramp, it is a parameter set to 13 s. Too short, and every loaded start trips; too long, and a stalled belt burns for the whole allowance.

20.10 Control-panel design

The motor-control centre

The MCC is a low-voltage switchgear and controlgear assembly to IEC 61439-1 and -2, verified by the manufacturer for its rated currents, short-circuit withstand and temperature rise. The design decisions specific to motor control are:

- **Rated busbar current** from the sum of motor currents with diversity: 278 A connected, 400 A busbar and incomer.
- **Short-circuit rating** against the prospective fault current at the busbar. Fed from the 630 kVA, 6 % transformer with an infinite source behind it, $I_k = 630\,000/(\sqrt{3} \times 400 \times 0.06) = 15.2$ kA; the busbar is rated $I_{cw} = 25$ kA for 1 s and every starter is a Type 2 combination tested at 25 kA (R6). The generator's fault level is lower and is checked separately for whether it can still trip the protective devices, not for withstand.
- **Form of separation** (IEC 61439-2, forms 1 to 4b): how far busbars, functional units and terminals are partitioned. Form 3b, each starter in its own compartment, lets one starter be worked on with the line running; it costs 15–25 % more than form 1, less than a stopped line.
- **Segregation and EMC**: drive sections separated from control, shielded motor cables with 360° glands (Chapter 18, Section 18.8).
- **Cable sizing** for each motor circuit from current, voltage drop and fault withstand, following *Electrical Energy and Machines*, Ch. 30 §30.5–30.6.

The heat budget

Motor-control panels are dominated by drive losses. The heat method is the enclosure balance of Chapter 18 (Section 18.8); what differs is the size of the numbers and the environment. The quarry is dusty, so filter fans would clog in weeks: R8 demands IP54 with a closed cooling circuit.

Table 20.8 — Heat budget of the MCC.

Source	Loss inside the MCC
C1 VFD 75 kW, 2.0 % loss (1.5 kW), 85 % through a flange-mounted heatsink outside	225 W
F1 VFD 7.5 kW, 2.5 % loss, fully inside	188 W
C2 soft starter, bypassed when running	22 W
Line reactor for C1, 3 %	180 W
MPCBs and contactors, 6 starters	72 W
Busbars and power wiring	110 W
Safety relay, PLC I/O, 24 V supply, relays	95 W
Total internal load	**892 W**
Heat entering through the walls at 45 °C ambient, 40 °C inside: $5.5 \times 11.8 \times 5$	324 W
Cooling duty	**1 216 W**
Cooling unit fitted	1 500 W
Margin	**19 %**

Worked example — fans or a cooling unit? The MCC is a row of four sections 2200 × 2400 × 600 mm against a wall, with an effective surface of $1.4 \times 2.4 \times 2.8 + 1.8 \times 0.6 \times 2.2 = 11.8$ m². Sealed, 892 W would raise it $892/(5.5 \times 11.8) = 13.8$ K above ambient: 58.8 °C at 45 °C ambient, which fails R8 and would derate the drives. A filter fan cannot hold 40 °C inside when the air outside is 45 °C, and would fill the panel with dust. An air-to-air heat exchanger also needs the outside to be cooler than the inside. So the only option meeting R8 is a refrigerant cooling unit, which must remove the internal load plus the heat flowing in through the walls: 1216 W, met by a 1.5 kW door-mounted unit with 19 % margin. Mounting the C1 drive's heatsink through the back wall is what keeps the duty that low: inside, the drive alone would have added another 1275 W and required a 3 kW unit.

> **Status of the principle: Drive losses as a fixed fraction of rated power**
> *Empirical.* Low-voltage drives lose about 2–3 % of their rated power at full load, from manufacturers' loss data (IEC 61800-9-2 efficiency classes); at part load the fraction rises and the absolute loss falls, so the budget at rated load is the conservative design case.

20.11 Commissioning, verification and failure analysis

The commissioning sequence

Motor-control commissioning follows the general FAT–SAT sequence of Chapter 18 (Section 18.9), with motor-specific steps:

1. **Before power**: insulation resistance at 500 V DC with drives disconnected (at least 1 MΩ, IEC 60204-1), bonding continuity, torque on power terminations.
2. **Settings recorded before first start**: MPCB currents and trip classes, soft-starter current limit and class, VFD motor data and limits.
3. **Rotation**: each motor bumped, uncoupled where possible.
4. **Drive identification and tuning**: stationary or rotating autotune and ramps, as in *Electrical Energy and Machines*, Ch. 12 §12.9.
5. **Interlock tests** against the cause-and-effect matrix: every permissive removed in turn, every downstream conveyor stopped while upstream ones run.
6. **E-stop validation**: every pull-cord pulled and every button pressed at least once, the stopping time measured each time at no load and at full load, the PLC in STOP for one test of each circuit, a contactor deliberately held closed to prove the mirror-contact monitoring.
7. **Load tests**: starts hot and cold, full-load running, belt-slip simulation, feed-loop tuning.

Verification

Table 20.9 — Verification of the conveyor-line motor control.

Req.	Test	Limit	Result	Pass/fail
R1	Sequence with every permissive removed; downstream stop with line running	Downstream first; cascade stop	All correct; C1 stops 40 ms after C2 speed lost	Pass
R2	Belt clamped at C2 tail with motor running	Trip within 3 s after acceleration	Trip 3.1 s after speed fell below 80 %	Pass
R3	8 h belt-weigher trend at 200 and 280 t/h	±3 % on 10-min average	±1.1 %	Pass
R4	18 pull-cord and button tests, empty and loaded; PLC in STOP	≤ 4 s; works with PLC stopped	2.4–2.7 s on C1, 1.2–1.9 s on C2 and C3; all stopped with PLC in STOP	Pass (failed before redesign)
R5	C2 restart loaded within 2 min of a full-load stop, at 6 °C ambient	No trip	Start 3.6 s, no trip	Pass
R6	Coordination tables, fault-level calculation	Type 2 at 15.2 kA	Combinations listed at 25 kA	Pass
R7	Recorded start of X1 and C2 on the generator	≤ 5 %	3.0 % and 4.4 %	Pass
R8	8 h heat run at full load, 41 °C ambient	≤ 40 °C inside at 45 °C ambient	34.5 °C (extrapolated 38.5 °C at 45 °C)	Pass
R9	25 injected faults	First-out cause shown within 1 s	All correct, 0.2–0.6 s	Pass
R10	Records audit	Every interlock and E-stop	Complete	Pass

Failure analysis

Table 20.10 — FMEA of the conveyor-line motor control (selected modes).

Failure mode	Effect	Cause	Detection	Mitigation
E-stop routed through standard PLC	No stop if PLC or its I/O fails	E-stop treated as an ordinary input	PLC-in-STOP test	Safety relay with STO and contactors; PLC gets status only
Empty belt coasts beyond 4 s	Person drawn into nip	Category 0 stop with low running resistance	Stopping-time test empty	Category 1 with brake chopper; SS1 ramp monitoring and spring brake
Overload trips on cold, loaded restart	Line stopped; operator bypass temptation	Current limit and trip class chosen on a summer test	Winter trip log	$4.5 \times I_n$ and class 20 from the hot/cold start table
Welded contactor	Motor cannot be stopped by that contactor	Contact wear, overload of AC-4 duty	Mirror contact in safety feedback; discrepancy alarm	Two contactors in series; time interlock on reversing
Belt slip undetected	Pulley lagging burns, fire	Speed taken from motor, not belt	Tail-pulley speed sensor	Slip trip R2 on belt speed
Feed loop oscillates	Surging tonnage, spillage	Gain raised to "speed up" a dead-time loop	Tonnage trend	SIMC tuning documented; gain change under change control
Drive overheats in summer	Derating, trips	Panel heat underestimated; filter fans in dust	Drive heatsink temperature	Through-wall heatsink; cooling unit with margin
Generator cannot start fan	Dip trips other loads	DOL fan on a weak source	Recorded start on generator	Star–delta starter

The first two rows are the opening story. The deeper lesson is the order of design: the stopping time, the dip on the generator and the hot-restart start time are each a short calculation that could have been made from the motor list before any hardware was bought; made afterwards, each cost a modification and a retest.

What remains open

Open question	Where it is answered
What performance level the E-stop needs, and how its architecture, MTTFd and diagnostic coverage achieve it; how series-wired pull-cords limit diagnostic coverage	Chapter 21
How the drives, soft starter and PLC communicate, and how the network is diagnosed and secured	Chapter 21
How the PLC, HMI and panels in general are designed and commissioned	Chapter 18
How belt-speed, drift and chute sensors are chosen and wired	Chapter 19
How a closed-loop drive is built from the power stage up	Chapter 37
How drive harmonics are assessed at the point of common coupling	*Electrical Energy and Machines*, Ch. 25

Next: Chapter 21 connects the controllers, drives and safety devices of this part over industrial networks, and shows how safety functions such as the E-stop of this chapter are specified, proven and protected.

Chapter 21 — Industrial Communication and Safety

Part V · Industrial Automation

An end-of-line palletising cell on a 24,000 bottle-per-hour beverage line passed its acceptance tests and then, in its third week, began stopping two or three times a shift: the safety PLC dropped the gantry and conveyors into safe torque off and logged a PROFIsafe watchdog timeout on the light-curtain input module. The light curtain was replaced twice. The real cause sat in the control cabinet: an unmanaged switch that the commissioning team had added for "one more port", and a maintenance laptop plugged into it that ran an inventory scan of the plant network every evening. The scan's broadcast and multicast traffic queued in front of the PROFINET frames in a switch that did not honour VLAN priorities; a few dozen full-length frames ahead of a cyclic frame delayed it by milliseconds, three missed cycles tripped the PROFINET watchdog, and the PROFIsafe layer, correctly, treated silence as danger. The cell had a communication design, a safety design and no security design, and the three met in one unmanaged switch.

> **How are the field networks of a machine (Modbus, CAN-based networks, industrial Ethernet) chosen, sized and diagnosed so that their timing is a number written down before commissioning?**
> **How is a safety function specified, built from certified parts, proven against ISO 13849-1, kept fail-safe over a network, and protected and documented so that it still works in year ten?**

Concept	Engineering destination
Modbus RTU and TCP	Energy meters, compressor controllers, legacy instruments, gateways to building systems
CAN-based networks (CANopen, J1939)	Servo axes inside machines, mobile machinery, battery and EV subsystems
Industrial Ethernet (PROFINET, EtherNet/IP, EtherCAT)	PLC-to-drive and PLC-to-remote-I/O networks with millisecond cycles
Fieldbus fundamentals	Choosing cyclic versus acyclic traffic, topology, update time and watchdog for each device
Network diagnostics	Locating a bad cable, a duplicate address or a slow slave in minutes rather than shifts
Functional safety	Required performance level, architecture, MTTFd, diagnostic coverage, validation
Safety interlocks	Guard doors, light curtains, E-stops and their safety distances
Fail-safe behaviour	De-energise-to-trip, black-channel safety protocols, watchdogs and safe states
Industrial cybersecurity	IEC 62443 zones, conduits and security levels for a machine cell
Maintenance documentation	Network drawings, address plans, safety validation records, change control

Roadmap. The reference cell and its requirements (21.1) → fieldbus fundamentals and timing budgets (21.2) → Modbus (21.3) → CAN-based networks (21.4) → industrial Ethernet (21.5) → network diagnostics (21.6) → functional safety fundamentals (21.7) → safety interlocks (21.8) → fail-safe behaviour (21.9) → industrial cybersecurity (21.10) → maintenance documentation (21.11) → verification and failure analysis (21.12).

21.1 The reference cell and its requirements

> **Boundary.**
> PLCs, remote I/O, HMIs, panels and commissioning are owned by Chapter 18; the sensors and actuators on the network by Chapter 19; starters, VFDs, interlocks and E-stop circuits by Chapter 20. This chapter connects them and makes them safe. The physical layers (RS-485, CAN, Ethernet PHYs) are in *Electronics and Computing*, Ch. 26 §26.2, §26.5, §26.7; drive safety functions (STO, SS1) in *Electrical Energy and Machines*, Ch. 10 §10.7.

The running system is the palletising cell of the opening (Figure 21.1). A fail-safe PLC (an S7-1500F-class CPU certified to SIL 3 and PL e) runs a 10 ms standard task and a 20 ms safety task. Its PROFINET RT network, at a 4 ms update time, connects two remote I/O stations (one with fail-safe input and output modules), three VFDs for the infeed and pallet conveyors (7.5 kW and two 2.2 kW), a gantry controller whose three servo axes sit on a private CANopen network, and a managed cabinet switch. Six power meters on the cell's distribution board report over Modbus RTU at 19,200 baud; a screw compressor controller speaks

Modbus TCP. A type 4 light curtain guards the pallet exit, two coded interlock switches guard the maintenance doors, and three E-stop buttons sit on the fence. Upward, an industrial firewall connects the cell to the plant's operations network, where the historian and the engineering station live.

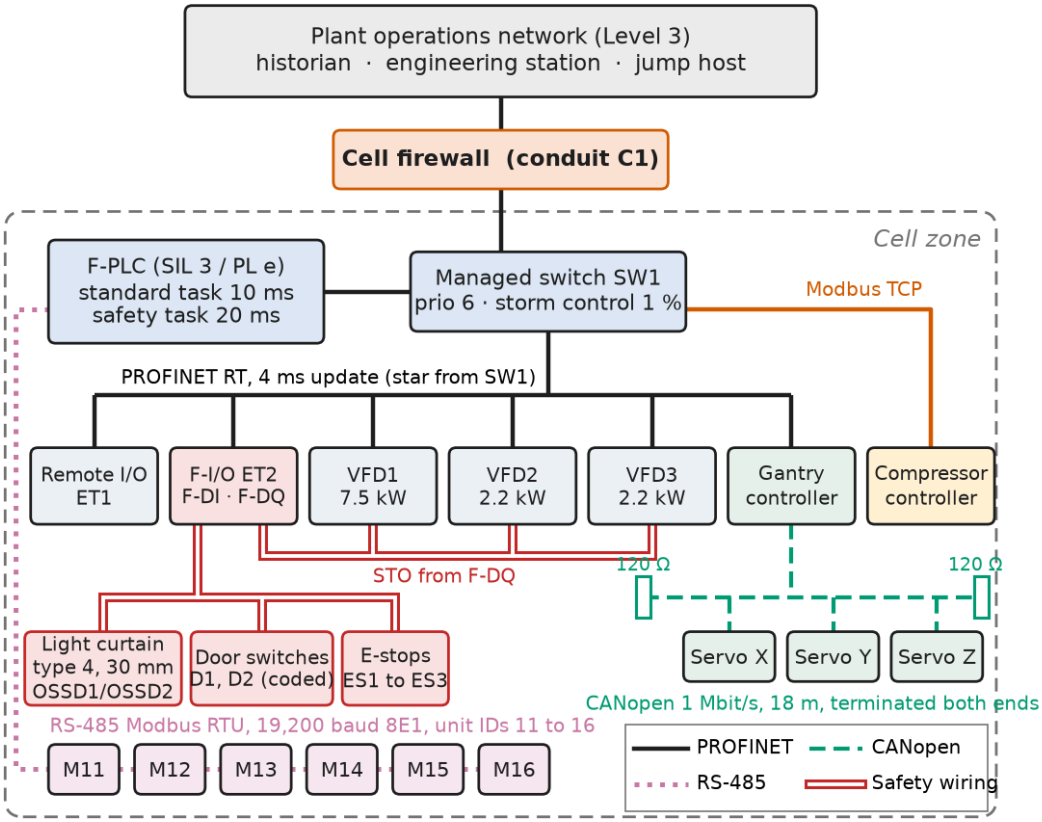


Figure 21.1 — Communication and safety architecture of the palletising cell: PROFINET RT backbone, CANopen gantry network, Modbus RTU meter bus and the firewall to the plant network.

Table 21.1 — Requirements of the palletising cell (communication and safety).

ID	Requirement	Target	Verification
R1	SF1: light curtain at pallet exit stops gantry and conveyors	PL d (PLr from risk assessment)	ISO 13849-1 calculation; validation test per ISO 13849-2
R2	SF2: opening a maintenance door stops all hazardous motion	PL d	Calculation; fault-injection test
R3	SF3: E-stop, stop category 1 (IEC 60204-1, ISO 13850)	PL d	Calculation; test of each button
R4	Minimum safety distance of the light curtain (ISO 13855)	Not above installed 900 mm	Stopping-time measurement; calculation
R5	PROFINET update time; no watchdog trip under stress	4 ms; zero trips in 24 h with 1 % broadcast load injected	Load test with traffic generator
R6	CANopen gantry cycle; bus load	4 ms SYNC; load below 60 % worst case	Bus analyser, 1 h capture
R7	Energy data from six meters	Every meter refreshed at least every 1 s; stale data flagged within 3 s	Poll-cycle log; disconnect test
R8	Fault localisation	Any network fault located to device and port within 5 min from the HMI	Fault-injection drill
R9	Security	Cell zone at SL 2 (IEC 62443-3-3); no IT-to-field traffic except through conduit C1	Firewall rule review; port scan from Level 3
R10	Loss of any safety communication	Safe state within the configured watchdog time	Cable pull during motion
R11	Documentation	As-built network drawing, address plan, safety validation report, program signatures	Document review at SAT

The table follows the method of Chapter 1: every row has a number and a test.

21.2 Fieldbus fundamentals

What every industrial network has to decide

All the networks of this chapter answer the same five questions, and choosing between them is mostly a matter of which answers a device's vendor has already given:

1. **Who speaks when**: master–slave polling (Modbus), priority arbitration (CAN), scheduled cyclic exchange (PROFINET, EtherNet/IP), or one frame through every slave (EtherCAT).
2. **Cyclic or acyclic**: process data travel cyclically at a fixed update time; parameters and diagnostics travel acyclically and must never steal the cyclic slots.
3. **Update time and watchdog**: the receiver declares a connection lost when the watchdog (a small multiple of the update time) expires, and drives its outputs to a defined substitute value.

4. **Topology**: line, star, ring or tree, and what one broken cable does to each.
5. **Addressing and description**: an address, device name or IP address, plus a description file (GSDML, EDS, ESI) that tells the engineering tool what the device exchanges.

Table 21.2 — Industrial networks used in machine and process automation (typical values).

Network	Physical layer	Access	Typical cycle	Devices per segment
Modbus RTU	RS-485, 9.6–115.2 kbit/s	Master polls slaves	100 ms–seconds	32 unit loads (247 addresses)
Modbus TCP	Ethernet, 100 Mbit/s	Client–server	10 ms–1 s	Limited by switch and server
CANopen	CAN, 125 kbit/s–1 Mbit/s	Priority arbitration	1–10 ms	127 node IDs
SAE J1939	CAN, 250/500 kbit/s	Priority arbitration	10–1000 ms	253 addresses
PROFIBUS DP	RS-485, up to 12 Mbit/s	Token plus master–slave	1–10 ms	32 per segment, 126 total
PROFINET RT / IRT	Ethernet, 100 Mbit/s	Scheduled cyclic	1–128 ms RT; 0.25–4 ms IRT	Hundreds
EtherNet/IP	Ethernet, 100 Mbit/s	Producer–consumer (CIP)	1–100 ms	Hundreds
EtherCAT	Ethernet, 100 Mbit/s	One frame through all slaves	50 µs–1 ms	65,535

The timing budget

A network's timing is designed, not discovered. A conveyor set-point travels from the PLC task (10 ms) over PROFINET (4 ms) to the VFD's control cycle (1–2 ms); because the three are not synchronised, its worst-case age is roughly the sum of the three periods. The budget writes this down for each loop and compares it with what the process needs.

> **Status of the principle: Worst-case age of asynchronously sampled data**
> *Derived.* When producer task, network and consumer task run on independent clocks, a change can just miss each of them, so the worst-case delay is the sum of the three periods plus the processing times; isochronous modes (PROFINET IRT, EtherCAT distributed clocks) remove the sum by aligning the cycles.

Table 21.3 — Cyclic-communication budget of the reference cell.

Link	Requirement	Design value	Worst case	Margin
PLC task to VFD speed set-point	50 ms (conveyor tracking)	10 ms task + 4 ms PROFINET + 2 ms drive	16 ms	34 ms
PROFINET watchdog (3 × update time)	No trip under 1 % broadcast load	12 ms	4.3 ms measured cyclic gap (21.6)	7.7 ms
CANopen gantry cycle at 1 Mbit/s	Bus load below 60 %	4 ms SYNC, 6 PDOs	21.9 %; 43.5 % if SYNC goes to 2 ms (21.4)	38 points
Modbus RTU meter refresh	Each meter at least once per 1 s	6 meters, 40 registers, 50 ms timeout, one retry	464 ms; 500 ms with one dead meter (21.3)	500 ms
Safety reaction SF1 (light curtain to standstill)	S not above 900 mm	400 ms	S = 768 mm (21.8)	132 mm, 82 ms

21.3 Modbus

The protocol in one paragraph

Modbus is request–response over a model of bits (coils, discrete inputs) and 16-bit registers (input, holding), accessed by public function codes. **Modbus RTU** runs on RS-485 with binary frames delimited by 3.5 character times of silence and a CRC-16; **Modbus TCP** wraps the same PDU in a 7-byte MBAP header on TCP port 502. It has no authentication, no data types beyond the 16-bit word and no standard register map, and those three gaps cause most of the integration work.

> **Boundary.**
> RS-485 levels, termination and failsafe biasing, and the 3.5-character frame gap, are in *Electronics and Computing*, Ch. 26 §26.2; reflections on unterminated lines in *Circuits, Signals and Communications*, Ch. 14. Endianness of multi-byte fields is in *Electronics and Computing*, Ch. 11.

The register map is the interface

The cell's six meters (IEC 61557-12 class 0.5) publish their values as 32-bit floats across two registers, and two vendors can disagree on everything that matters: 0- or 1-based addresses, high word first ("ABCD") or second ("CDAB"), energy as a float in kWh or an integer in Wh. The integrator writes one register map per device type and tests it against a known load. The listing builds a request, checks a reply's CRC and decodes the floats in both word orders, the commonest error on site:

```
 # Modbus RTU: build a read-holding-registers request and decode a reply
import struct

def crc16_modbus(frame: bytes) -> int:
    crc = 0xFFFF
    for b in frame:
        crc ^= b
        for _ in range(8):
            crc = (crc >> 1) ^ 0xA001 if crc & 1 else crc >> 1
    return crc

def read_request(unit, start, count):
    pdu = struct.pack(">BBHH", unit, 0x03, start, count)
    return pdu + struct.pack("<H", crc16_modbus(pdu))       # CRC low byte first

def decode_floats(reply, swap_words=False):
    if crc16_modbus(reply) != 0:                             # CRC over frame+CRC is 0
        raise ValueError("CRC error")
    unit, fc, n = reply[0], reply[1], reply[2]
    if fc & 0x80:
        raise ValueError(f"exception code {reply[2]:02X} from unit {unit}")
    data = reply[3:3 + n]
    words = [data[i:i + 2] for i in range(0, n, 2)]
    if swap_words:                                           # "CDAB" meters
        words = [w for pair in zip(words[1::2], words[0::2]) for w in pair]
    return struct.unpack(">" + "f" * (n // 4), b"".join(words))

req = read_request(7, 0x0000, 4)             # L1 voltage and current, float32 each
print("request :", req.hex(" "))
body = struct.pack(">BBB", 7, 0x03, 8) + struct.pack(">ff", 231.4, 18.75)
reply = body + struct.pack("<H", crc16_modbus(body))
print("reply   :", reply.hex(" "))
print("ABCD    :", [round(x, 2) for x in decode_floats(reply)])
print("CDAB    :", [f"{x:.3g}" for x in decode_floats(reply, swap_words=True)])
```

```
 request : 07 03 00 00 00 04 44 6f
reply   : 07 03 08 43 67 66 66 41 96 00 00 2c 62
ABCD    : [231.4, 18.75]
CDAB    : ['2.72e+23', '2.35e-41']
```

The wrong word order does not produce an error; it produces a number, absurd here but plausible for some values, which is why commissioning compares each meter with a clamp meter on a known load.

Poll-cycle budget

A Modbus RTU master asks one slave at a time, so the refresh period of every device is the sum of all transactions on the bus. One transaction costs the request, a 3.5-character gap, the slave's turnaround time (5–50 ms for meters, set by their firmware, not by the baud rate), the response and another gap. With 8E1 framing a character is 11 bits.

Worked example — Is 19,200 baud enough for six meters? At 19,200 baud a character takes $11/19\,200 = 0.573$ ms. Reading 40 registers needs an 8-byte request (4.58 ms), an 85-byte response (48.7 ms), two gaps of 3.5 characters (2.0 ms each) and a measured turnaround of 20 ms: 77.3 ms per transaction. Six meters take

$$T_{\text{poll}} = 6 \times 77.3\ \text{ms} = 464\ \text{ms}$$

against the 1 s of R7, a margin of 536 ms. Now one meter dies (its breaker is opened for maintenance). The master waits for a 200 ms timeout before each retry; with the vendor's default of three retries, the dead meter costs $4 \times (200 + 4.6 + 2.0) = 826$ ms and the cycle grows to 1.21 s, breaking R7 for the five healthy meters. With one retry and the dead unit then skipped for that cycle, the cycle is 800 ms; with the timeout also cut to 50 ms, 500 ms. At 9,600 baud the healthy cycle alone would be 808 ms. Figure 21.2 plots the cycle against the number of meters.

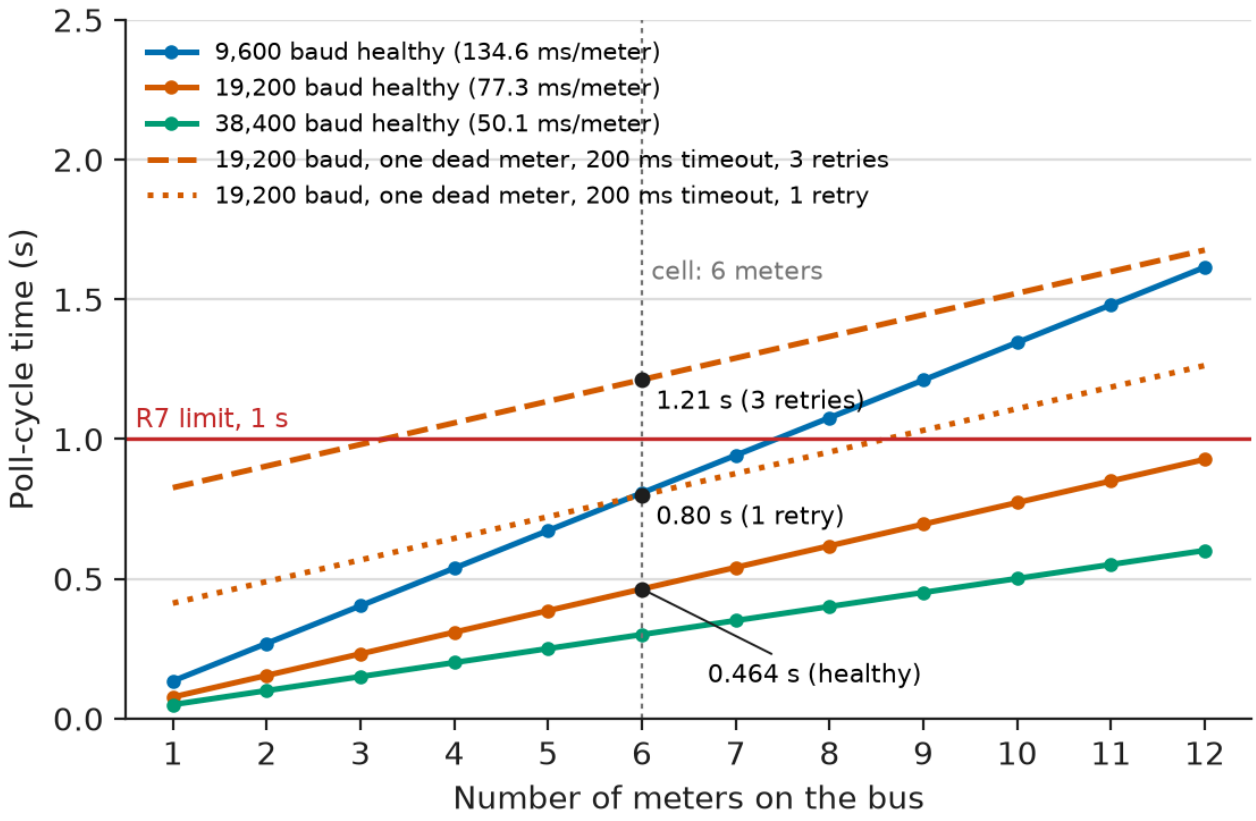


Figure 21.2 — Modbus RTU poll-cycle time against the number of meters for three baud rates, healthy and with one meter not responding.

The fixes are cheap: one retry instead of three, a timeout set from the measured worst-case turnaround (50 ms) rather than the default, a staleness flag per device, and a second RS-485 port rather than a higher baud rate if the bus grows. The PLC side is a small function block in IEC 61131-3 Structured Text, here written against a generic vendor Modbus client block (`MB_RTU_READ` is illustrative; Siemens, Beckhoff and Codesys libraries differ in names, not in shape):

```
 TYPE ST_Meter : STRUCT
    Unit : USINT; Regs : ARRAY[0..39] OF WORD; Errors : UDINT;
END_STRUCT; END_TYPE

FUNCTION_BLOCK FB_MeterPoller
VAR_INPUT    Enable : BOOL; END_VAR
VAR_IN_OUT   Meter : ARRAY[1..6] OF ST_Meter; END_VAR
VAR_OUTPUT   Stale : ARRAY[1..6] OF BOOL; CommFault : ARRAY[1..6] OF BOOL; END_VAR
VAR
    MbRead : MB_RTU_READ;              (* vendor client block, illustrative *)
    Age    : ARRAY[1..6] OF TON;
    Idx    : INT := 1;
    Tries  : INT := 0;
    Req, Gap : BOOL;
    i      : INT;
END_VAR
VAR CONSTANT MAX_TRIES : INT := 2; END_VAR   (* first attempt plus one retry *)

IF Gap THEN Gap := FALSE;              (* one scan with REQ low between jobs *)
ELSIF Enable AND NOT Req THEN Req := TRUE;
END_IF;
MbRead(REQ := Req, UNIT := Meter[Idx].Unit, START := 16#0000, COUNT := 40,
       TIMEOUT := T#50MS, DATA := Meter[Idx].Regs);
IF MbRead.DONE THEN
    Age[Idx](IN := FALSE);             (* fresh data restarts the age timer *)
    CommFault[Idx] := FALSE; Tries := 0;
    Req := FALSE; Gap := TRUE; Idx := Idx MOD 6 + 1;
ELSIF MbRead.ERROR THEN
    Meter[Idx].Errors := Meter[Idx].Errors + 1; Tries := Tries + 1;
    Req := FALSE; Gap := TRUE;
    IF Tries >= MAX_TRIES THEN         (* give up on this unit for this cycle *)
        CommFault[Idx] := TRUE; Tries := 0; Idx := Idx MOD 6 + 1;
    END_IF;
END_IF;
FOR i := 1 TO 6 DO
    Age[i](IN := TRUE, PT := T#3S);
    Stale[i] := Age[i].Q;              (* R7: stale flagged within 3 s *)
END_FOR;
END_FUNCTION_BLOCK
```

The HMI shows the per-device error counter (R8): a slowly rising count without `CommFault` means a marginal cable or termination; `CommFault` means dead or misaddressed.

21.4 CAN-based industrial networks

CANopen and J1939

CAN gives arbitration, error confinement and an 8-byte (CAN FD: 64-byte) frame; the application layer gives the frames meaning. **CANopen** (EN 50325-4, CiA 301) organises each device around an **object dictionary**

(16-bit index, 8-bit subindex), exchanges process data in **PDOs** (up to 8 bytes, no overhead, triggered by SYNC, timer or change of state), parameters through confirmed **SDOs**, and manages nodes with **NMT** commands and **heartbeats**. Profiles fix the dictionary per device class: CiA 402 for drives (control word 0x6040, status word 0x6041), CiA 401 for I/O. **SAE J1939** does the same for vehicles and mobile machinery with 29-bit identifiers carrying a parameter-group number and source address.

> **Boundary.**
> The CAN physical layer, arbitration, stuffing, error states and bit timing, and the frame-length and bus-load formulas used below, are in *Electronics and Computing*, Ch. 26 §26.5; fixed-priority response-time analysis of CAN in Ch. 27.

Sizing the gantry network

The gantry's three servo drives (CiA 402, cyclic synchronous position mode) each receive one RPDO (control word plus target position, 6–8 bytes) and send one TPDO (status word plus actual position) every SYNC period, and each node sends a 1-byte heartbeat every 100 ms.

Worked example — Bus load at 4 ms and at 2 ms SYNC. With worst-case bit stuffing a classical frame with 8 data bytes is 135 bits, a 0-byte SYNC is 55 bits and a 1-byte heartbeat 65 bits (Book 4 §26.5). Per 4 ms SYNC period the bus carries one SYNC and six 8-byte PDOs, plus the heartbeats spread over 100 ms:

$$\text{load} = \frac{55/0.004 + 6 \times 135/0.004 + 4 \times 65/0.1}{500\,000} = 43.8\ \%$$

at 500 kbit/s, inside the 60 % limit of R6. The motion engineer then asks for a 2 ms SYNC to stiffen the position loop: the same arithmetic gives 87.0 %, unusable once an SDO download or an error retransmission lands. The gantry's bus is 18 m long, so 1 Mbit/s is legal (about 40 m at 1 Mbit/s with typical transceivers), and at 1 Mbit/s the 2 ms cycle loads the bus to 43.5 %. The project chose 1 Mbit/s and kept a 4 ms SYNC (21.9 % load) until tuning showed whether 2 ms was needed, which bought headroom for a fourth axis.

> **Status of the principle: Bus-load ceiling for CAN networks**
> *Empirical.* Integrators keep the worst-case load of a CAN network below about 50–70 % so that error frames, retransmissions and acyclic SDO traffic do not push low-priority messages past their deadlines; a response-time analysis replaces the rule when deadlines are tight.

A gateway to PROFINET maps the PDOs into the PLC's I/O image but hides the CAN error counters and heartbeat losses unless its diagnostic objects are mapped too; in the reference cell they are, which is what makes R8 testable on the CAN side.

21.5 Industrial Ethernet

Three ways to make Ethernet deterministic

Standard Ethernet with TCP/IP is fast but not deterministic: frames queue in switches behind other traffic. Industrial Ethernet takes one of three routes:

- **Prioritised standard Ethernet** (PROFINET RT, EtherNet/IP, Modbus TCP): cyclic frames carry an IEEE 802.1Q priority (6 for PROFINET) that managed switches honour; 1–10 ms cycles, jitter set by the switches.
- **Scheduled Ethernet** (PROFINET IRT, TSN with IEEE 802.1Qbv): reserved time windows give sub-microsecond jitter at 250 µs–1 ms; every switch needs hardware support.
- **Processing on the fly** (EtherCAT): one frame passes through every slave, which reads and inserts its data in hardware; a hundred axes update in tens of microseconds, on a segment that is not ordinary switched Ethernet.

Table 21.4 — Selecting the cell's backbone network.

Criterion	PROFINET RT	EtherNet/IP	EtherCAT	Modbus TCP
Fits the chosen F-PLC	Native	Needs a gateway	Needs a gateway	Native, but no safety
Cycle needed (4 ms)	Yes, 1–4 ms routine	Yes	Yes, far better	Polled, about 10–50 ms
Safety protocol	PROFIsafe	CIP Safety	FSoE	None
VFDs and remote I/O available	All three vendors short-listed	Two of three	Two of three	All, but polled
Decision	**Chosen**	Rejected (gateway for safety)	Used inside gantry alternative, rejected (vendor set)	Kept for compressor only

PROFINET was chosen because the safety PLC, the safety protocol and all the field devices speak it natively, so the safety function crosses no gateway; EtherCAT would have been the choice for a 32-axis machine whose motion needed 250 µs cycles.

Bandwidth is not the constraint; queueing is

Eight PROFINET devices exchanging frames of about 100 byte times in each direction every 4 ms use $8 \times 2 \times 100 \times 8 / 0.004 = 3.2$ Mbit/s, about 3 % of a 100 Mbit/s link. The cell's network does not fail on bandwidth. It fails, as in the opening, on queueing.

Worked example — How many frames can stand in front of a PROFINET frame? A full-length Ethernet frame (1500 bytes of payload) occupies 1538 byte times including preamble and interframe gap, $1538 \times 8 / 10^8 = 123$ µs at 100 Mbit/s. PROFINET declares a device's connection lost after three missed update cycles, $3 \times 4 = 12$ ms. A switch that does not prioritise therefore needs only $12 / 0.123 \approx 98$ full-length frames queued ahead of the cyclic frames on one port, or a series of shorter bursts across several cycles, to trip the watchdog; a laptop's discovery scan or a misconfigured camera stream produces that in a moment. Forty queued frames already delay one frame by 4.9 ms, a full update cycle. A managed switch with priority queuing, storm control at 1 % of port bandwidth for broadcast and unknown multicast, and the maintenance port on a separate VLAN reduces the queue in front of a priority-6 frame to at most the one frame already being transmitted, 123 µs.

> **Status of the principle: Store-and-forward queueing delay**
> *Fundamental.* A frame cannot leave a switch port until the frames ahead of it in the same queue have been serialised at the link rate, so the delay is the queued bytes divided by the link rate; priority queuing bounds it by one maximum-length frame per hop, which is why PROFINET line depth and switch class are planned, not left to chance.

Topology and redundancy

A line through the devices' integrated switches saves cable but makes each device a single point of failure for everything downstream. The cell uses a star from the managed switch, with the drives in a short line. A ring with the Media Redundancy Protocol (IEC 62439-2) recovers from one break within 200 ms in its common profile, longer than a 12 ms watchdog, so stations behind the break may still drop once; bumpless redundancy (MRPD) is used where every cycle counts.

21.6 Network diagnostics

Instruments and counters

Table 21.5 is the troubleshooting card left in the cell's cabinet.

Table 21.5 — Symptom, likely cause and first test.

Symptom	Likely causes	First test
Modbus timeouts on one meter	Wrong unit ID, baud or parity; meter turnaround longer than timeout	Read the device's settings; scope the bus and measure reply delay
Modbus CRC errors on far meters	Missing termination, no failsafe bias, stub too long, shield grounded at both ends near a VFD	120 Ω termination at both ends; idle differential above 200 mV
CAN error frames, nodes go error-passive	Termination wrong (measure 60 Ω between CAN_H and CAN_L with power off), bit-timing mismatch, ground offset	Ohmmeter; bus analyser error-frame count per minute
CAN node bus-off after power-up	Two nodes with the same ID; wrong bit rate	Heartbeat list; analyser
PROFINET device lost, intermittent	Damaged connector, cable beside a motor lead, queueing in an unmanaged switch	Port statistics: CRC errors and discards per port; LLDP topology
All devices lost at once	Broadcast storm, loop without spanning tree, switch power	Switch log; storm-control counters

The managed switch is the most useful instrument on an Ethernet network: per-port counters separate a physical problem (CRC errors rising on one port) from congestion (egress discards with zero CRC errors), LLDP topology shows a cable in the wrong port at power-up, and port mirroring into a protocol analyser shows what was on the wire.

Worked example — Locating the opening's fault. The trips clustered between 19:00 and 19:35. The managed switch showed zero CRC errors on all ports, ruling out cabling; the unmanaged switch had no counters, so a port mirror on the uplink captured an evening: at 19:01:58 a laptop on the unmanaged switch began an ARP sweep and SSDP and mDNS discovery, about 2,000 broadcast and multicast frames per second for 40 s, with bursts of maximum-length frames. PROFINET cyclic frames from the remote I/O station behind the unmanaged switch showed gaps of up to 14 ms, more than the 12 ms watchdog. The fix removed the unmanaged switch, moved the maintenance port to its own VLAN behind the firewall (Section 21.10) and enabled storm control. A 24-hour test with a traffic generator injecting 1 % broadcast load (R5) then showed a worst-case cyclic gap of 4.3 ms and no watchdog trips.

> **Status of the principle: Counters before oscilloscopes**
> *Empirical.* In plant experience most intermittent network faults are located faster from per-port error and discard counters and topology comparison than from waveform measurement; the oscilloscope remains the tool for the RS-485 and CAN physical layers, where devices rarely keep counters.

21.7 Functional safety fundamentals

> **Boundary.**
> E-stop circuits, contactor interlocks and the stop categories of IEC 60204-1 at the level of one motor are in Chapter 20; STO and SS1 inside the drive in *Electrical Energy and Machines*, Ch. 10 §10.7; isolation and lockout for electrical work in Ch. 29 §29.6. The probability arithmetic used below is in *The Mathematics Universe*, Ch. 19.

From hazard to required performance level

Functional safety on machines runs through **ISO 12100** (risk assessment and reduction), **ISO 13849-1** (safety-related control parts, rated PL a–e), **IEC 62061** (the same in SIL 1–3) and, underneath, **IEC 61508** for the electronics and software; the process industry uses IEC 61511. The risk graph of ISO 13849-1 sets the required level from severity (S1 slight, S2 serious), exposure (F1 seldom, F2 frequent) and possibility of avoidance (P1 possible, P2 scarcely possible):

Table 21.6 — Safety functions of the cell and their required performance levels.

Function	Hazard	S, F, P	PLr	Stop
SF1 light curtain at pallet exit	Crushing by gantry and pallet conveyor	S2, F1, P2	d	Category 1 (ramp, then STO)
SF2 maintenance doors D1, D2	Crushing and impact inside the fence	S2, F2, P1	d	Category 1, guard locking not needed if S is met
SF3 E-stop ES1–ES3	Complementary measure (ISO 13850)	—	d (set equal to the other functions)	Category 1
SF4 safely limited speed in setup mode	Gantry moving with door open, hold-to-run	S2, F1, P1	c	Monitored by the drive (SLS)

What determines the PL achieved

ISO 13849-1 rates a function from its **Category** (architecture; Category 3 is two channels, so a single fault does not lose the function, with most faults detected), the **MTTFd** of each channel (low 3–10 years, medium 10–30, high 30–100), the diagnostic coverage **DCavg** (low 60–90 %, medium 90–99 %, high at least 99 %), measures against **common-cause failure** (at least 65 of 100 points) and the software process. It turns these into an average probability of dangerous failure per hour, PFHd, and a PL: PL d is $10^{-7} \le \mathrm{PFH_d} < 10^{-6}$ per hour, PL e is $10^{-8} \le \mathrm{PFH_d} < 10^{-7}$.

Certified subsystems (a type 4 light curtain, an F-CPU, an F-I/O module, a drive's STO) come with their PL and PFHd from the manufacturer; the integrator adds them up. Electromechanical parts come with a B_{10d}, the number of operations at which 10 % of a sample fails dangerously, and the integrator converts it to an MTTFd for the actual duty:

$$\mathrm{MTTF_d} = \frac{B_{10d}}{0.1\, n_{\mathrm{op}}}, \qquad \frac{1}{\mathrm{MTTF_{d,channel}}} = \sum_i \frac{1}{\mathrm{MTTF_{d,i}}}$$

> **Status of the principle: MTTFd from B10d and series combination of channel elements**
> *Derived.* Follows from a constant-failure-rate (exponential) model of each element, in which failure rates in series add; the B_{10d} conversion assumes the wear-out of electromechanical parts can be represented by an equivalent constant rate over the mission time, which ISO 13849-1 limits by requiring replacement before $T_{10d} = B_{10d}/n_{\mathrm{op}}$.

Worked example — PL of SF2, the door interlock with contactors. Door D2 is opened six times an hour, 16 hours a day, 220 days a year. Each of two channels is a coded tongue interlock contact ($B_{10d} = 2 \times 10^6$), an F-DI channel (MTTFd 1,000 years from the vendor) and a contactor in the gantry brake circuit ($B_{10d} = 1.3 \times 10^6$ at AC-3 rated load). The listing does the arithmetic and, for SF1, sums the vendors' PFHd values:

```
 # SF2 guard-door channel MTTFd (ISO 13849-1 B10d method) and SF1 PFHd sum
days, hours, cycles_per_h = 220, 16, 6
n_op = days * hours * cycles_per_h                    # operations per year
def mttfd_from_b10d(b10d, n_op):
    return b10d / (0.1 * n_op)                        # years
switch = mttfd_from_b10d(2.0e6, n_op)                 # coded tongue interlock
contactor = mttfd_from_b10d(1.3e6, n_op)              # AC-3 contactor at rated load
f_input = 1000.0                                       # F-DI channel, from vendor data (years)
channel = 1 / (1 / switch + 1 / contactor + 1 / f_input)
print(f"n_op = {n_op} /year; MTTFd switch {switch:.0f} y, contactor {contactor:.0f} y, "
      f"channel {channel:.0f} y -> capped at {min(channel, 100):.0f} y (high)")
t10 = 1.3e6 / n_op
print(f"contactor reaches B10d after {t10:.0f} years")

pfh = {"light curtain": 1.2e-8, "F-DI": 1.0e-9, "F-CPU": 2.0e-9, "F-DQ": 1.0e-9,
       "drive STO x2": 2 * 3.0e-9}
print(f"SF1 PFHd sum = {sum(pfh.values()):.2e} per hour")
```

```
 n_op = 21120 /year; MTTFd switch 947 y, contactor 616 y, channel 272 y -> capped at 100 y (high)
contactor reaches B10d after 62 years
SF1 PFHd sum = 2.20e-08 per hour
```

Each channel's MTTFd is capped at 100 years for Category 3, so the channels rate "high". The two channels are cross-compared in the F-PLC and the contactors' mirror contacts are read back (external device monitoring), which together give a DCavg in the medium band (90–99 %); CCF measures score 75 points. Category 3, MTTFd high and DCavg medium land in the PL e region of the standard's simplified bar chart, one level above the PLr d of SF2, and $T_{10d} = 62$ years exceeds the 20-year mission time, so no scheduled contactor replacement is needed at this duty. SF1, built entirely from certified subsystems, sums to $\mathrm{PFH_d} = 2.2 \times 10^{-8}$ per hour, PL e by probability, and no subsystem is rated below PL e. The SISTEMA tool (the IFA's free implementation of ISO 13849-1) produces the formal report for R1–R3.

Table 21.7 — Component selection for the safety chain.

Function block	Option A	Option B	Option C	Choice and why
Pallet-exit sensing	Type 4 light curtain, 30 mm, PL e, PFHd 1.2×10^{-8}, 15 ms	Type 2 light curtain, PL c max	Safety laser scanner, PL d, 60–100 ms response	A: PL d needs type 4 or type 3 class; scanner response time costs about 120 mm of distance
Logic	Safety relays per function	Configurable safety controller	F-PLC with PROFIsafe	C: four functions, muting and SLS mode selection outgrow relays; one program, one validation
Door switches	Uncoded tongue switch	Coded interlock (type 2 tongue or type 4 RFID), ISO 14119	Guard locking (solenoid)	B, type 2 coded tongue: defeat resistance; locking unnecessary because stopping is faster than access
Drive shutdown	Line contactor per drive	Drive STO via F-DQ, PL e	STO via PROFIsafe telegram	B: no contactor wear, fast restart; C kept as a later option

21.8 Safety interlocks

Interlocking guards

ISO 14119 classifies interlocking devices from type 1 (uncoded mechanical) to type 4 (coded non-contact) and requires measures against defeat in proportion to the motivation to defeat them: a door opened twenty times an hour to clear jams will be defeated with a spare actuator unless jam clearing is designed in (the setup mode SF4) and the switch is uniquely coded. Guard locking is needed only when the stopping time exceeds the time a person needs to reach the hazard after the door starts to open.

Light curtains and the safety distance

An electro-sensitive protective device (IEC 61496) must be far enough from the hazard that the machine stops before a person who breaks the beams can reach it. ISO 13855 gives the minimum distance for a vertical curtain with resolution d up to 40 mm:

$$S = K\,T + 8\,(d - 14)$$

with T the total stopping performance from beam interruption to standstill, $K = 2000$ mm/s if the result is at most 500 mm, otherwise $K = 1600$ mm/s with a minimum of 500 mm.

> **Status of the principle: Approach-speed model of ISO 13855**
> *Empirical.* The approach speeds of 2.0 and 1.6 m/s and the reach-through allowance $8(d - 14)$ are set from measured human body and hand movements; they are the standard's agreed values, not a physical law, and apply to the device geometries the standard covers.

The stopping performance is a chain (Figure 21.3), and every link must be the worst case: the light curtain's OFF response time, the F-input module and PROFIsafe transfer and F-CPU cycle in the worst case *with one fault*, which the vendor's response-time calculator gives as the PROFIsafe watchdog-governed figure, the F-output and the drive's STO response, and the mechanical stopping time measured on the machine with a stopping-time meter at maximum speed and load.

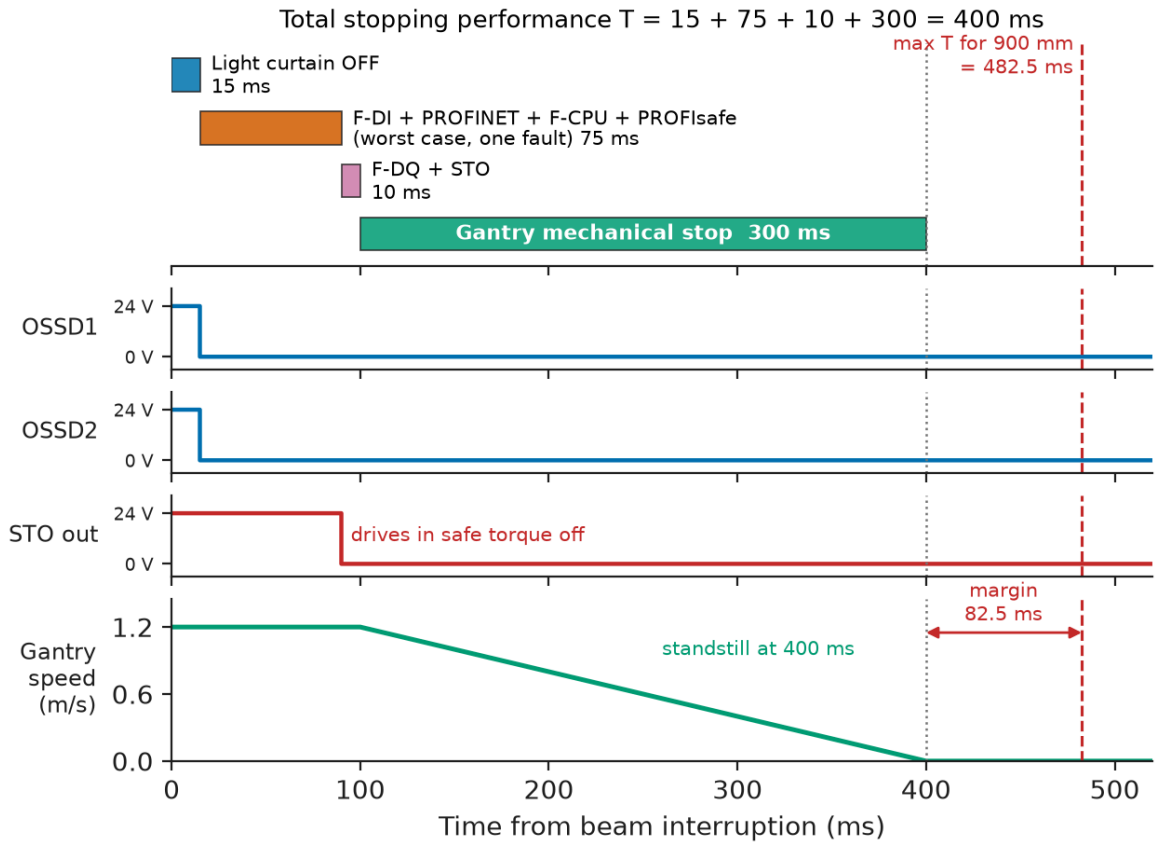


Figure 21.3 — Stopping-performance chain of safety function SF1 from beam interruption to gantry standstill.

Worked example — Is the installed 900 mm enough? With $T = 15 + 75 + 10 + 300 = 400$ ms and $d = 30$ mm, $C = 8(30 - 14) = 128$ mm. The trial $S = 2000 \times 0.4 + 128 = 928$ mm exceeds 500 mm, so $K = 1600$ mm/s and

$$S = 1600 \times 0.400 + 128 = 768 \text{ mm}$$

The curtain is mounted 900 mm from the nearest hazard, a margin of 132 mm, which corresponds to a stopping-performance margin of $(900 - 128)/1600 - 0.400 = 82.5$ ms. Brake wear lengthens the stopping time, so the maintenance plan (Section 21.11) measures it every six months with a limit of 360 ms, keeping 22.5 ms in hand. Figure 21.4 shows how fast the margin disappears with stopping time: a 14 mm finger-resolution curtain would have allowed a shorter distance ($S = 1600 \times 0.4 = 640$ mm), and a safety laser scanner with a 90 ms response (instead of 15 ms) would have needed 888 mm, leaving no room for brake wear.

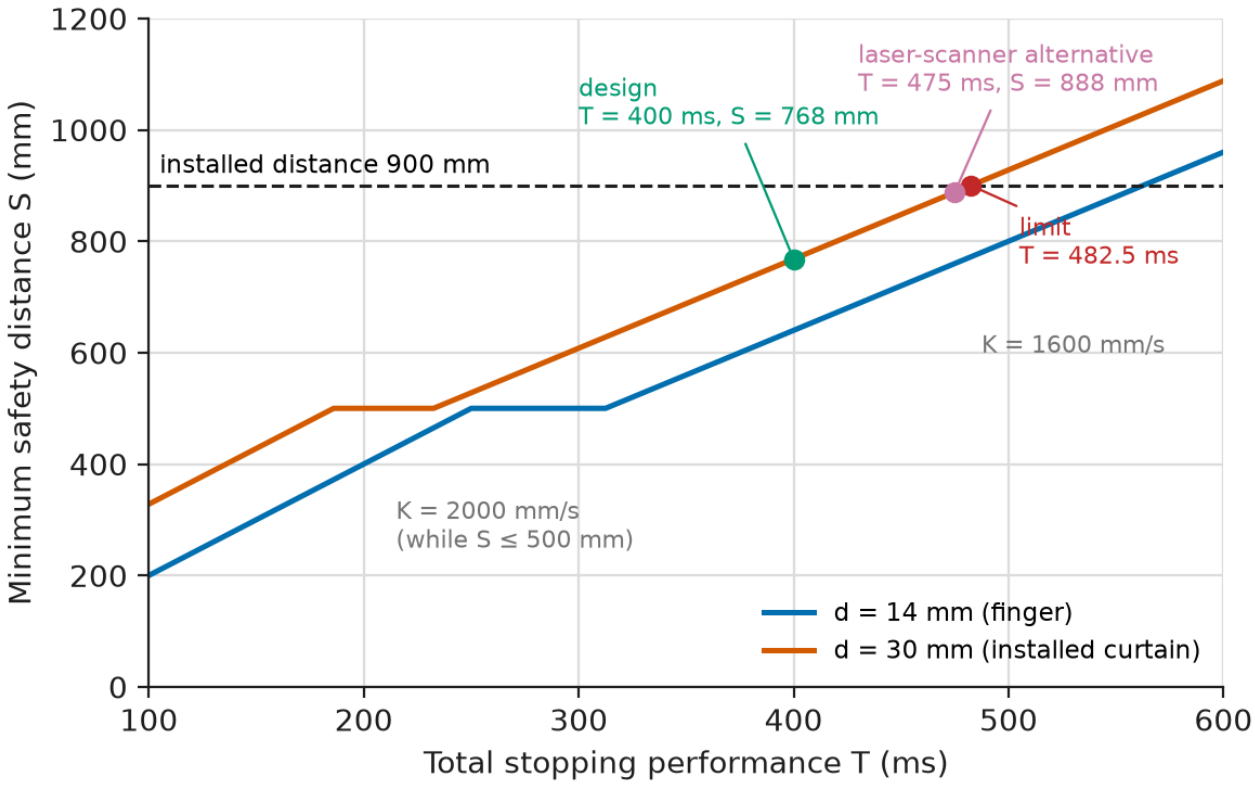


Figure 21.4 — Minimum safety distance of a vertical light curtain against total stopping performance, for 14 mm and 30 mm resolution, with the cell's installed distance.

Muting at the pallet exit

Pallets leave through the curtain by **muting** (IEC 62046): the curtain is suspended only while sensors in a defined sequence confirm a pallet, within a 4 s window. The muting logic is part of SF1 and inherits its PL, and sensor geometry that a hand can trigger defeats it, so the geometry is part of the validation.

21.9 Fail-safe behaviour

De-energise to trip

Every safety function in the cell is built so that the absence of energy, signal or message produces the safe state: light-curtain OSSD outputs fall to 0 V on interruption, power loss or internal fault; interlock contacts are normally closed and positively opened (IEC 60947-5-1 Annex K); STO inputs need 24 V to *enable* torque. A broken wire stops the machine instead of disabling the protection. Where it is cheap the rule extends to standard control: a VFD ramps to stop on PROFINET loss rather than holding its last speed.

> **Status of the principle: De-energise-to-trip**
> *Fundamental.* The most probable failures of wires, contacts, power supplies and messages are loss of signal; making loss of signal the safe state turns those failures into stops rather than lost protection. Where stopping is itself dangerous (a hoist, a press holding a load), the safe state must be analysed rather than assumed.

Black-channel safety protocols

The safety PLC does not trust the network. **PROFIsafe** (IEC 61784-3-3), like CIP Safety and FSoE, is a *black-channel* protocol: safety data travel inside ordinary frames through ordinary switches, and the safety layer at each end detects every way a network can corrupt a message — a consecutive number against repetition, loss and reordering, a CRC over the data *and* the unique F-address against corruption and misrouting, and a watchdog time (F_WD_Time) against delay. Switches and cables therefore need no safety rating.

The watchdog has two failure modes: too short, and normal jitter trips the cell; too long, and the reaction time and the safety distance grow. The cell's F_WD_Time of 50 ms is about twice the measured worst-case round trip, and the 75 ms reaction time used in Section 21.8 comes from the vendor's calculator with that setting.

Safe states and restart

After SF1 or SF2 trips, the drives sit in STO with brakes applied; restart needs the guard closed *and* a manual reset from a position with a view of the hazard zone (ISO 13849-1 §5.2.2), and passivated F-I/O modules need an acknowledgement, so a flapping cable cannot cycle the machine. The standard part has fail-safe responses too: a stale meter value is flagged, not replaced by zero, and a lost CANopen heartbeat puts the gantry into a quick stop.

21.10 Industrial cybersecurity

> **Boundary.**
> The principles of OT security (availability first, defence in depth, the Purdue model) and the grid-side standards (IEC 62351, NERC CIP) are in *Electrical Energy and Machines*, Ch. 26 §26.9. Device-level security (secure boot, signed firmware, credentials on IoT devices) is in Chapter 11. This section applies IEC 62443 to one machine cell.

Zones, conduits and security levels

For the system integrator IEC 62443 (parts 3-2 and 3-3) prescribes a method: divide the system into **zones** of assets with common security requirements, connect zones only through **conduits** whose traffic is defined and controlled, and assign each zone a target **security level** SL-T from 1 (protection against casual or accidental violation) to 4 (against state-level attackers). SL 2 means protection against intentional violation by simple means with low resources and generic skills, which is the usual target for a production cell (R9). The seven foundational requirements of 62443-3-3 (from identification and authentication to resource availability) are then met by system requirements at that level.

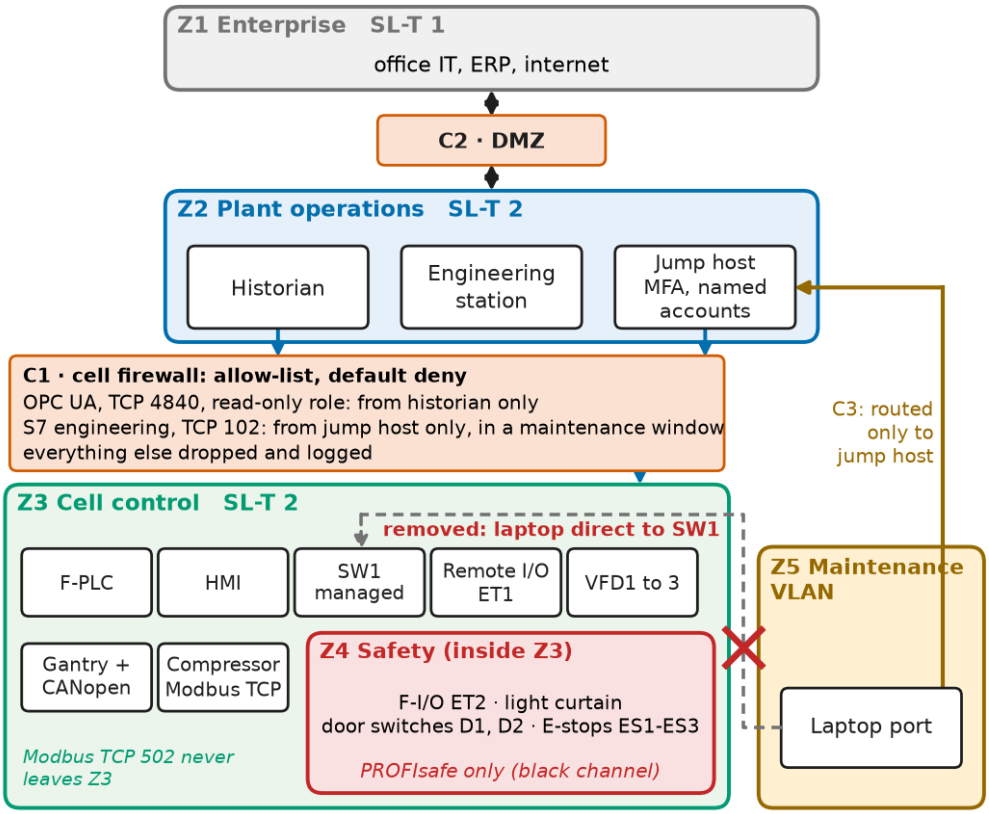


Figure 21.5 — IEC 62443 zones and conduits of the palletising cell after the redesign.

What the cell's design changed

Figure 21.5 shows the cell after the redesign. The changes were unglamorous:

- **Restricted data flow.** A cell firewall implements conduit C1 with an allow-list: the historian may read the PLC over OPC UA (TCP 4840) with a certificate and a read-only role; the engineering protocol (S7 over TCP 102) is accepted only from the jump host, and only while a maintenance window is open. Everything else is dropped and logged. Modbus TCP (port 502), which has no authentication at all, never leaves the cell zone.
- **No unmanaged infrastructure.** Managed switches only, unused ports disabled, maintenance port on its own VLAN.
- **Hardening and authentication.** PLC write access protected, web servers disabled, firmware versions recorded; named accounts with multi-factor authentication on the jump host, vendor remote access only through it.
- **Recovery.** Offline, versioned backups of every configuration, tested by restoring to a spare CPU at SAT.

Worked example — What a port scan tells the auditor. After the redesign a scan from Level 3 against the cell must find one host, the firewall, with one open port, 4840, reachable only from the historian (R9). Before the redesign it found 14 hosts and 31 open TCP ports, including two drive web servers with default passwords and Modbus TCP on the compressor. Thirty-one exposed services reduced to one is the measurable part of SL 2 here, and the scan is rerun after every change.

> **Status of the principle: Allow-list conduits**
> *Empirical.* Industrial incident analyses repeatedly show attackers and malware moving laterally through flat networks and unauthenticated protocols; a default-deny conduit with an explicit list of flows removes most of those paths at little cost to availability, which is why IEC 62443 builds its system requirements around zones and conduits.

Security and safety meet in one rule: a security measure must never trip or disable a safety function, so firewalls stay out of the PROFIsafe path and F-PLC firmware updates are treated as safety changes that require revalidation.

21.11 Maintenance documentation

The cell is maintained for 15–20 years by people who did not build it. Each document below is part of the handover (R11), under the revision control of Chapter 5.

Table 21.8 — Maintenance documentation of the cell.

Document	Content	Used when
Network drawing (as built)	Every device, port-to-port cable, switch port, VLAN, topology as LLDP sees it	Replacing a device or cable; reading a topology alarm
Address and name plan	PROFINET device names, IP addresses, Modbus unit IDs and serial settings, CANopen node IDs and bit rate, GSDML/EDS versions	Replacing a device: the new one must get the same name or ID
Safety requirements specification	Functions, PLr, stop categories, reaction times, restart rules	Any change to the machine or its use
Safety validation report	Calculations (SISTEMA), test records of every function and fault-injection test, stopping-time measurements	Audits; after any safety change
Program signatures	F-program collective signature (CRC), standard program version, firmware versions	Proving the running program is the validated one
Proof and periodic test plan	Function tests of each interlock, light curtain test-rod check, stopping-time measurement every 6 months (limit 360 ms)	Scheduled maintenance

The F-program signature is the most useful single line: if the signature displayed by the running CPU differs from the one in the validation report, the program running the safety functions is not the one that was validated, whatever anyone remembers.

21.12 Verification and failure analysis

Verification

Table 21.9 — Verification of the reference cell (site acceptance test).

Req.	Test	Limit	Result	Pass/fail
R1	SISTEMA calculation; light curtain test rod at 10 positions; muting sequence and timeout	PL ≥ d; stop on every interruption	PL e; 10/10 stops; mute window 4.0 s	Pass
R2	Open each door in auto at full speed; short each channel; cross-short channels	Stop; discrepancy detected; restart blocked	All stops; discrepancy alarm in 1 cycle	Pass
R3	Each E-stop at full speed	Category 1 stop	Ramp 0.25 s then STO	Pass
R4	Stopping-time meter, 10 runs at full load and speed	T ≤ 482 ms	Gantry 284–300 ms; total T 400 ms	Pass
R5	24 h with 1 % broadcast injection at maintenance port and uplink	No watchdog trip	0 trips; worst cyclic gap 4.3 ms	Pass
R6	CAN analyser, 1 h, 1 Mbit/s, 4 ms SYNC	Load below 60 %; 0 error frames	21.6 % mean, 22.4 % peak during an SDO download; 0 errors	Pass
R7	Poll log 1 h; open meter 13's breaker	Refresh ≤ 1 s; stale flag ≤ 3 s	470 ms healthy; 505 ms with 13 dead; flag at 3.0 s	Pass
R8	Five injected faults (pulled cable, wrong device name, removed termination, CAN node off, dead meter)	Located in 5 min each from HMI	Worst 3 min 40 s (termination)	Pass
R9	Port scan from Level 3; firewall rule review	One open port, from historian only	As specified	Pass
R10	Pull ET2's cable during motion	Safe state within F_WD_Time	STO after 48 ms; reset required	Pass
R11	Document review	All of Table 21.8 present and signed	Present; signature matches running CPU	Pass

Failure analysis

Table 21.10 — FMEA of the cell's communication and safety design (after redesign).

Failure mode	Effect	Cause	Detection	Mitigation
PROFIsafe watchdog trips under traffic	Nuisance stops, pressure to "lengthen the watchdog" or bypass	Unprioritised switch; broadcast storm	Trip log with timestamps; switch discard counters	Managed switches only, priority 6, storm control, maintenance VLAN
Light curtain mounted too close after re-layout	Person reaches hazard before standstill	Safety distance not recomputed; brake wear	Periodic stopping-time measurement	Distance in safety requirements spec; change control triggers recalculation
Door switch defeated with spare actuator	Machine runs with door open	Jam clearing designed out of the procedure	Door-open-in-auto counter on HMI; audits	Coded switches; setup mode with SLS (SF4) for jam clearing
Welded contactor in brake circuit	Brake not applied on demand in one channel	Overcurrent, end of life	Mirror-contact feedback (EDM) at each start	Category 3 with EDM; restart blocked on discrepancy
Dead meter stalls the bus	All energy data stale	Default three retries, long timeout	Poll-cycle time logged	One retry, 50 ms timeout, per-device stale flag
Unauthorised program change	Unvalidated safety logic	Shared password; direct laptop access	F-signature check at each shift start	Jump host with MFA; access protection; signature comparison

The opening's fault produced the first row, the program-change row and the maintenance VLAN: a communication fault, a security gap and a safety stop with one root cause. Networks, safety functions and the security boundary are designed separately but fail together, so the verification plan stresses them together — traffic injected while the light curtain is tested, a cable pulled while the port scan runs.

What remains open

Open question	Where it is answered
How are the PLC, remote I/O, HMI and panel of the cell architected and commissioned?	Chapter 18
How are the E-stop circuits, contactor interlocks and VFDs of each motor designed?	Chapter 20
How do wireless sensor networks handle reliability and security on a constrained radio link?	Chapter 38
How are communication links budgeted at the RF level?	Chapter 26
How is a battery system's BMS interfaced over CAN and Modbus to its power converter?	Chapter 23
How is the timing of CAN messages analysed formally?	*Electronics and Computing*, Ch. 27
How are earthing and electrical safety designed for the installation?	*Electrical Energy and Machines*, Ch. 29

Next: Chapter 22 leaves the factory floor for the roof, designing a PV system from the solar resource to the inverter and its protection.

Part VI — Renewable Energy and Electric Mobility

Part V built machines that consume electrical energy under control. This part builds the systems that produce, store and move it: a rooftop PV plant, the battery that stores its output, the electric vehicle that carries a battery on wheels, and the microgrid that makes PV, wind, storage, generators and the grid work as one. Almost every number in them is set by an environment the designer does not control — the coldest dawn of the decade, the hottest afternoon on a roof, a −10 °C delivery round, a feeder that trips at noon on a Sunday. Each chapter writes those extremes into its requirements, sizes strings, cells, converters and cables against them, budgets energy, voltage, temperature and current, and closes with tests against numbers fixed before the build.

By the end of this part you should understand

- How a PV system is sized from irradiation data to the grid connection — string length at the coldest and hottest cell temperatures, inverter and MPPT choices, cables and protection — and how a loss tree predicts its yield and performance ratio.
- How a PV system is commissioned and monitored, and how a temperature-corrected performance ratio locates a fault.
- How a battery pack is architected from cell to rack against a converter's voltage window, with contactors, precharge, fuses and insulation monitoring.
- How a BMS measures, protects, balances and estimates state, and why its error budgets usually decide whether a battery meets its contract.
- How thermal design, charging limits, degradation and safety testing (IEC 62619, UL 9540A, UN 38.3) shape a battery's life.
- How an EV powertrain is sized from a duty cycle, and why cold weather and charging infrastructure belong in the first energy budget.
- How PV, wind, storage and generators are dispatched, protected and coordinated so that a site runs grid-connected and islanded, and how such a system is evaluated for reliability and cost.

Chapters

Chapter	Central question	What it delivers
22 — Solar PV System Design	How is a PV system sized so that every string, cable and protective device stays inside its limits, and how is its performance proven?	Resource assessment, module and inverter selection, string-sizing window, MPPT, battery coupling, protection, cables, loss tree, commissioning and PR monitoring; an 11 kWp school roof
23 — Battery Energy Systems	How is a battery pack built and managed so that it delivers its rated energy and power safely over its life?	Pack architecture, cell selection, series and parallel design, BMS firmware, measurement and SoC budgets, thermal design, charging, protection, fade projection, safety testing; a 215 kWh LFP cabinet
24 — Electric Vehicles	How is an EV powertrain sized from a duty cycle, and what must vehicle, charger and depot agree on?	EV architecture, energy consumption, motor, inverter and DC–DC selection, traction pack, regeneration, thermal limits, charging and depot infrastructure; a fleet of delivery vans
25 — Microgrids and Hybrid Energy Systems	How are PV, wind, storage and generators sized and dispatched together, and moved between grid-connected and islanded operation without a blackout?	Hybrid architectures, load profiles, sizing, energy management, grid-connected and islanded control, inverter coordination, protection, reliability, economics; a rural hospital microgrid

How this part connects to the rest of the book

The chapters build on each other: Chapter 22's school system contains a battery that Chapter 23 opens; Chapter 23's pack and BMS reappear on wheels in Chapter 24 and as the storage of Chapter 25's microgrid, which combines both under one energy-management system. All four use the method of Part I, the current and voltage measurement of Part II, the embedded firmware and gateways of Part III, and the CAN, Modbus and functional-safety practice of Chapter 21. Part IX applies them end to end: the solar battery project of Chapter 36 uses Chapters 22, 23 and 25, and the smart energy monitor of Chapter 35 measures such systems.

The theory is cited from the sibling books, not repeated. *Electrical Energy and Machines* owns renewable generation (Ch. 15), storage and BMS functions (Ch. 16), drive control (Ch. 12), microgrids and grid integration (Ch. 26–27) and electrical safety and cable design (Ch. 29–30). *Electronics and Computing* owns the converters, chargers and traction power stages (Ch. 30–32). *Engineering Physics* supplies solar-cell physics and thermal networks (Ch. 14), and *Materials Science and Engineering* PV and battery materials (Ch. 40).

Chapter 22 — Solar PV System Design

Part VI · Renewable Energy and Electric Mobility

A secondary school on a continental plateau at 40° N commissioned an 11.0 kWp rooftop PV system in October. Its installer had wired all twenty 550 W modules in one string on a 10 kW three-phase hybrid inverter rated for 1,000 V DC, because the datasheet open-circuit voltage, $20 \times 49.8 = 996$ V, fitted. On the first clear January morning, at −12 °C, the inverter logged "PV overvoltage" at sunrise and refused to start; on two later mornings it started late, after the sun had warmed the modules. The arithmetic missed one line: open-circuit voltage rises as cells cool, and on a −15 °C dawn the string reaches 1,100 V. The inverter had protected itself, this time. The redesign of this roof — two strings of ten modules on the inverter's two trackers, a 10.24 kWh battery, cables and protection rechecked, a monitoring plan with a performance-ratio target — is the running example of this chapter.

> **How is a PV system sized from the solar resource to the grid connection, so that every string, cable and protective device stays inside its limits on the coldest morning and the hottest afternoon?**
> **How are its losses budgeted, its yield predicted and its performance proven after commissioning?**

Concept	Engineering destination
Solar resource assessment	Plane-of-array irradiation from site data; yield estimates for financing and sizing
PV module characteristics	Choosing a module class; temperature coefficients that set string limits
String sizing	Number of modules per string inside the inverter's voltage and current window
Inverters	String, hybrid and module-level architectures; DC/AC ratio; grid-code compliance
MPPT	Tracking efficiency, perturbation step, global scans under partial shading
Battery integration	DC- versus AC-coupled storage, backup reserve, battery-port sizing
Protection	Fuses, isolators, SPDs, insulation monitoring, residual-current and arc-fault protection
Cable sizing	DC and AC conductors sized on current, voltage drop and voltage rise
System losses	Loss tree from irradiance to AC energy; performance ratio
Performance evaluation	IEC 62446-1 commissioning tests; IEC 61724-1 monitoring; PR tracking

Roadmap. Requirements and architecture (22.1) → the solar resource (22.2) → the module (22.3) → string sizing (22.4) → the inverter (22.5) → MPPT (22.6) → battery integration (22.7) → protection (22.8) → cables (22.9) → losses and yield (22.10) → performance evaluation (22.11) → failure analysis (22.12).

22.1 Requirements and architecture

> **Boundary.**
> The physics of the solar cell is in *Engineering Physics*, Ch. 14 §14.5.5; PV absorber materials in *Materials Science and Engineering*, Ch. 40 §40.1. Module ratings, the NOCT cell-temperature model, string sizing at plant scale and inverter architectures are introduced in *Electrical Energy and Machines*, Ch. 15 §15.1; the MPPT boost stage and transformerless inverter topologies in *Electronics and Computing*, Ch. 32 §32.8; grid codes and anti-islanding in *Electrical Energy and Machines*, Ch. 27 §27.4–27.5. This chapter designs one real system with those results.

The school's roof is a 15° south-facing pitch with a parapet and two ventilation stacks. The school consumes about 24 MWh a year, mostly on weekdays between 08:00 and 17:00, with a sports hall used in the evenings and a server room and emergency lighting that the head teacher wants kept alive through the grid outages the region sees a few times a winter. The grid connection is three-phase 400 V; the distribution operator's code (EN 50549-1 in this region) limits export to 10 kW.

Table 22.1 — Requirements of the school PV system.

ID	Requirement	Target	Verification
R1	Annual AC energy, first year (P50)	≥ 15,000 kWh	Yield model; 12-month metered energy
R2	Performance ratio, first year	≥ 0.80 annual; monthly temperature-corrected PR within 3 % of model	Monitoring per IEC 61724-1
R3	DC voltage at record-low cell temperature (−15 °C)	≤ 1,000 V inverter limit, ≥ 5 % margin	Calculation; Voc test at commissioning
R4	String MPP voltage at hottest cell temperature (75 °C)	≥ MPPT floor 200 V with ≥ 30 % margin	Calculation
R5	Export and grid compliance	≤ 10 kW export; EN 50549-1 settings; certified anti-islanding	Inverter certificate; settings record
R6	DC voltage drop at STC; AC voltage rise at full export	≤ 1 % each	Calculation; measured at commissioning
R7	Protection and insulation	IEC 60364-7-712 and IEC 62548 compliant; array insulation ≥ 1 MΩ	IEC 62446-1 tests
R8	Backup for server room and emergency lighting	1.2 kW for 3 h during an outage, switchover ≤ 20 ms	Outage simulation
R9	Monitoring	5-min data: POA irradiance, module temperature, DC and AC per tracker, battery SoC; alarms	Data review at handover

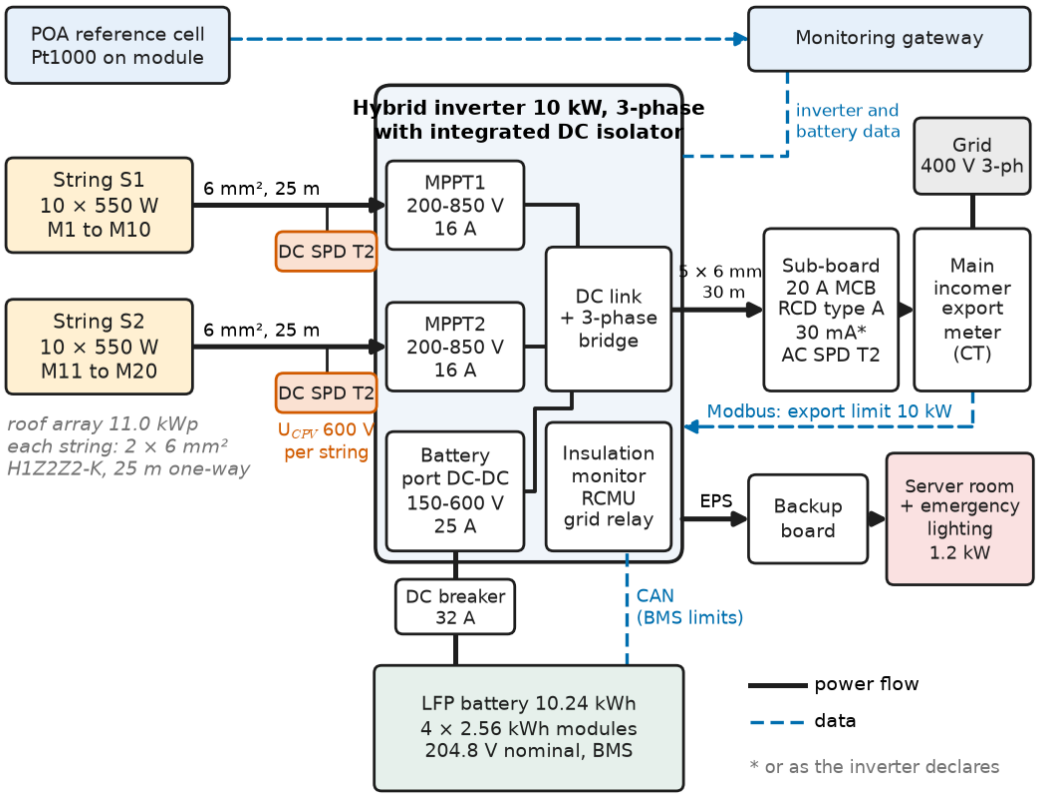


Figure 22.1 — Architecture and single-line diagram of the 11.0 kWp school PV system with DC-coupled battery.

Figure 22.1 is the architecture after the redesign: two strings, each on its own tracker; a hybrid inverter whose battery port is fed through an internal bidirectional DC–DC converter; an export meter at the main incomer that lets the inverter limit export to 10 kW; and a separate backup output for the protected loads.

22.2 Solar resource assessment

From GHI to plane-of-array irradiation

The design starts from long-term irradiation data for the site: satellite-derived global horizontal irradiation (GHI) and its diffuse fraction, available as typical-meteorological-year files (PVGIS, NSRDB, Meteonorm, Solcast). For a roof system, ten-year satellite averages are accurate to about ±4–5 % in annual GHI; ground measurement is not worth its cost below a few hundred kilowatts. The design quantity is the irradiation on the plane of the array (POA), obtained by splitting GHI into beam and diffuse parts and transposing each to the tilted plane. The ideas and the P50/P90 method are in *Electrical Energy and Machines*, Ch. 15 §15.6; here they are applied with the monthly-average isotropic-sky method (Liu–Jordan transposition with Klein's beam factor and the Erbs diffuse correlation), adequate for a low-tilt roof, and checked against a PVGIS run.

Table 22.2 — Monthly irradiation at the school (GHI from a satellite TMY; POA by isotropic transposition, 15° tilt, south, albedo 0.2).

Month	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec	Year
GHI (kWh/m²)	60	80	125	150	185	205	220	195	150	105	70	55	1,600
Clearness index	0.46	0.50	0.53	0.52	0.54	0.59	0.63	0.62	0.60	0.54	0.51	0.46	—
POA/GHI	1.32	1.23	1.12	1.05	1.00	0.98	0.99	1.03	1.10	1.19	1.31	1.36	1.084
POA (kWh/m²)	79	98	141	157	185	200	217	201	165	125	92	75	1,734

The 15° tilt gains 8.4 % over horizontal for the year, most of it in winter; a 30° tilt would gain about 12 % but needs racking on this pitched roof, and was not worth the wind-load design for 3–4 % more energy. Figure 22.2 shows the monthly profile against the school's consumption: summer production peaks fall in the school holidays, which matters for battery sizing (Section 22.7).

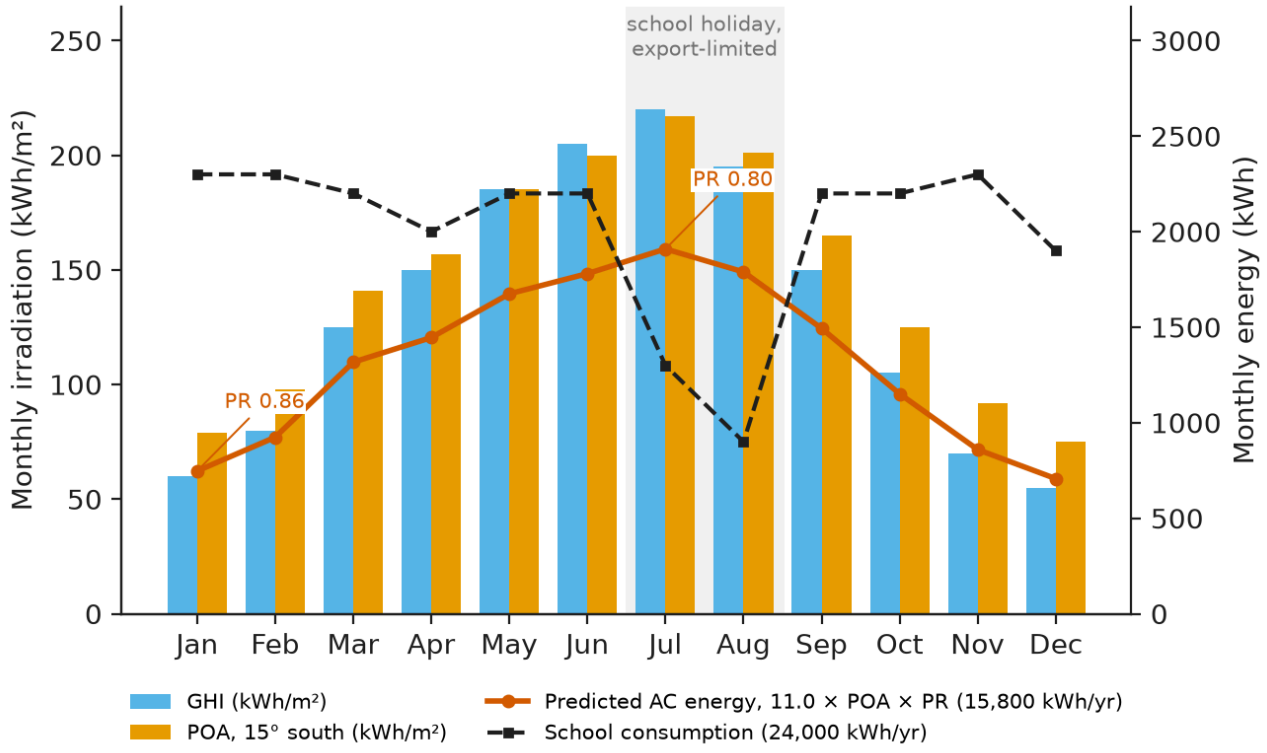


Figure 22.2 — Monthly irradiation, predicted AC energy and school consumption for the 11.0 kWp roof.

Shading survey

Near shading is the loss a resource dataset cannot know. The installer's survey with a sun-path tool placed each ventilation stack's shadow on the array: in December at solar noon the sun stands at $90 - 40 - 23.4 = 26.6^\circ$, and a 1.2 m parapet throws a shadow $1.2 / \tan 26.6^\circ = 2.4$ m long, so the first module row starts 2.5 m from the parapet. The stacks shade two modules for about an hour on winter mornings; those modules were placed at the ends of string S2 so that only one tracker is affected. The simulated annual near-shading loss is 1.5 %.

22.3 PV module characteristics

What the datasheet must give

The design uses a handful of datasheet numbers: STC power, V_{OC}, I_{SC}, V_{mp} and I_{mp}; the temperature coefficients of V_{OC}, I_{SC} and P_{max}; NOCT or NMOT; maximum system voltage; maximum series-fuse rating; the power tolerance; and the warranted degradation. Certification to IEC 61215 (design qualification) and IEC 61730 (safety, class II) is a condition of the grid connection, not a choice.

Table 22.3 — Module classes considered for the school (2026 distributor prices, pallet quantities).

Class	Example rating	Efficiency	γ_P (%/K)	$\beta_{V_{OC}}$ (%/K)	Price	Assessment
Mono PERC, 108 half-cells	410 W, V_{OC} 37.3 V	21.0 %	−0.35	−0.27	\$0.13/W	Cheapest; more modules and connectors for 11 kW; worst in heat
TOPCon, 144 half-cells	550 W, V_{OC} 49.8 V, I_{SC} 14.0 A	21.3 %	−0.30	−0.26	\$0.14/W	**Chosen**: 20 modules fill the roof; current fits a 16 A tracker
Heterojunction (HJT)	430 W, V_{OC} 40.6 V	22.0 %	−0.25	−0.24	\$0.19/W	Best hot-roof yield (+1 %); price premium not repaid in this climate
Large-format 210 mm cells	650 W, I_{SC} 18.4 A	22.5 %	−0.29	−0.25	\$0.14/W	I_{mp} (17.3 A) over the 16 A operating limit of most residential trackers

The large-format module shows why module selection cannot be separated from inverter selection: its MPP current exceeds the 16 A operating input current of the residential inverter class, so the tracker would clip every sunny hour. The chosen module's parameters used throughout are: $V_{OC} = 49.8$ V, $V_{mp} = 41.9$ V, $I_{SC} = 14.0$ A, $I_{mp} = 13.13$ A, $\beta_{V_{OC}} = -0.26$ %/K, $\alpha_{I_{SC}} = +0.046$ %/K, $\gamma_P = -0.30$ %/K. The temperature coefficient of V_{mp} is rarely printed; it is approximately $\gamma_P - \alpha_{I_{SC}} = -0.34$ %/K, which is the value used for string sizing.

> **Status of the principle: Linear temperature coefficients**
> *Empirical.* Datasheet coefficients are measured near STC and the linear model holds to about ±1 % over −20 to +80 °C for crystalline silicon; the V_{mp} coefficient estimated as $\gamma_P - \alpha_{I_{SC}}$ assumes the fill factor's temperature dependence follows V_{mp}, which is adequate for sizing but not for yield simulation.

Cell temperature on a roof

Cell temperature follows the NOCT model of *Electrical Energy and Machines* §15.1, $T_c \approx T_a + (\text{NOCT} - 20)G/800$, but that model describes an open rack. Modules mounted 10 cm above a pitched roof run 5–10 K hotter. For the school, with a 38 °C design ambient, 1,000 W/m² and NOCT 44 °C, the open-rack estimate is 68 °C; adding 7 K for the roof gives the design hot-cell temperature of 75 °C. The cold design temperature is

the record minimum ambient, −15 °C: at dawn, before any heating, irradiance of 100–200 W/m² is enough to bring a string almost to its full open-circuit voltage while the cells are still at air temperature.

22.4 String sizing

The voltage and current window

String sizing places the array's operating envelope inside the inverter's window (Figure 22.3):

- **Upper limit.** The string's open-circuit voltage at the coldest cell temperature must not exceed the inverter's maximum DC input voltage (and the module's maximum system voltage). This is the limit the original design broke.
- **MPPT range.** The string's MPP voltage must stay above the tracker's lower limit at the hottest cell temperature, with margin for degradation and for low irradiance, and below the upper MPPT limit in cold weather.
- **Current.** The string's I_{mp} must not exceed the tracker's maximum operating current, and its I_{SC} at the hottest temperature must not exceed the inverter's maximum short-circuit input current.

$$N_{\max} = \left\lfloor \frac{V_{\mathrm{DC,max}}}{V_{OC}\left[1 + \beta_{V_{OC}}(T_{\mathrm{cold}} - 25)\right]} \right\rfloor, \qquad N_{\min} = \left\lceil \frac{V_{\mathrm{MPPT,min}}}{V_{mp}\left[1 + \beta_{V_{mp}}(T_{\mathrm{hot}} - 25)\right]} \right\rceil$$

> **Status of the principle: String voltage at the coldest cell temperature**
> *Derived.* Series modules add their voltages and the open-circuit voltage rises linearly as temperature falls; the coldest-cell assumption is the worst case because irradiance at dawn is enough to reach near-full V_{OC} before the cells warm. Very high albedo (snow) or cloud-edge enhancement can raise irradiance, but V_{OC} depends only logarithmically on it.

The listing computes the window for the chosen module and inverter. It is the first file in the project's design repository, rerun whenever a part changes:

```
 # String-sizing window for the school roof: 550 W TOPCon module, 10 kW hybrid inverter
from math import floor, ceil
mod = dict(Voc=49.8, Vmp=41.9, Isc=14.0, Imp=13.13,
           b_voc=-0.0026, b_vmp=-0.0034, a_isc=0.00046)     # per K
inv = dict(Vdc_max=1000, Vmppt_min=200, Vmppt_max=850, Imax=16.0, Isc_max=20.0)
T_cold, T_hot = -15.0, 75.0          # cell temperature: record-low dawn, hot roof noon

voc_cold = mod["Voc"] * (1 + mod["b_voc"] * (T_cold - 25))
vmp_hot  = mod["Vmp"] * (1 + mod["b_vmp"] * (T_hot - 25))
vmp_cold = mod["Vmp"] * (1 + mod["b_vmp"] * (T_cold - 25))
isc_hot  = mod["Isc"] * (1 + mod["a_isc"] * (T_hot - 25))
n_max = floor(inv["Vdc_max"] / voc_cold)
n_min = ceil(inv["Vmppt_min"] / vmp_hot)
n_mppt_max = floor(inv["Vmppt_max"] / vmp_cold)
print(f"Voc at {T_cold:.0f} C = {voc_cold:.2f} V, Vmp at {T_hot:.0f} C = {vmp_hot:.2f} V, "
      f"Vmp at {T_cold:.0f} C = {vmp_cold:.2f} V")
print(f"N_max (1000 V) = {n_max}, N_min (MPPT floor) = {n_min}, N_max (MPPT ceiling) =
{n_mppt_max}")
for n in (10, 20):
    print(f"N = {n:2d}: Voc,cold = {n*voc_cold:6.1f} V  Vmp,hot = {n*vmp_hot:5.1f} V  "
          f"Vmp,STC = {n*mod['Vmp']:5.1f} V  -> {'OK' if n <= n_max else 'EXCEEDS 1000 V'}")
print(f"Isc at {T_hot:.0f} C = {isc_hot:.2f} A (limit {inv['Isc_max']} A), "
      f"Imp = {mod['Imp']} A (limit {inv['Imax']} A)")
```

```
 Voc at -15 C = 54.98 V, Vmp at 75 C = 34.78 V, Vmp at -15 C = 47.60 V
N_max (1000 V) = 18, N_min (MPPT floor) = 6, N_max (MPPT ceiling) = 17
N = 10: Voc,cold =  549.8 V  Vmp,hot = 347.8 V  Vmp,STC = 419.0 V  -> OK
N = 20: Voc,cold = 1099.6 V  Vmp,hot = 695.5 V  Vmp,STC = 838.0 V  -> EXCEEDS 1000 V
Isc at 75 C = 14.32 A (limit 20.0 A), Imp = 13.13 A (limit 16.0 A)
```

Worked example — Why the original string failed and what replaced it. At −15 °C each module's open-circuit voltage is $49.8 \times [1 - 0.0026 \times (-40)] = 54.98$ V, so twenty in series reach 1,099.6 V, 10 % above the inverter's 1,000 V limit; even at 0 °C the string is at 1,060 V. The window allows 6 to 17 modules per string (the MPPT ceiling, 850 V at cold MPP, is the tighter upper limit in operation, the 1,000 V limit the absolute one). Two strings of ten, one per tracker, give 549.8 V cold (45 % margin to 1,000 V, R3) and 347.8 V hot (74 % above the 200 V floor, R4). Each tracker then carries 5.5 kWp at 13.13 A, inside its 16 A limit. Ten modules is also below the 400–850 V range in which this inverter reaches full power from one tracker, but with two trackers sharing 10 kW each needs only 5 kW, which the tracker delivers from 320 V upward.

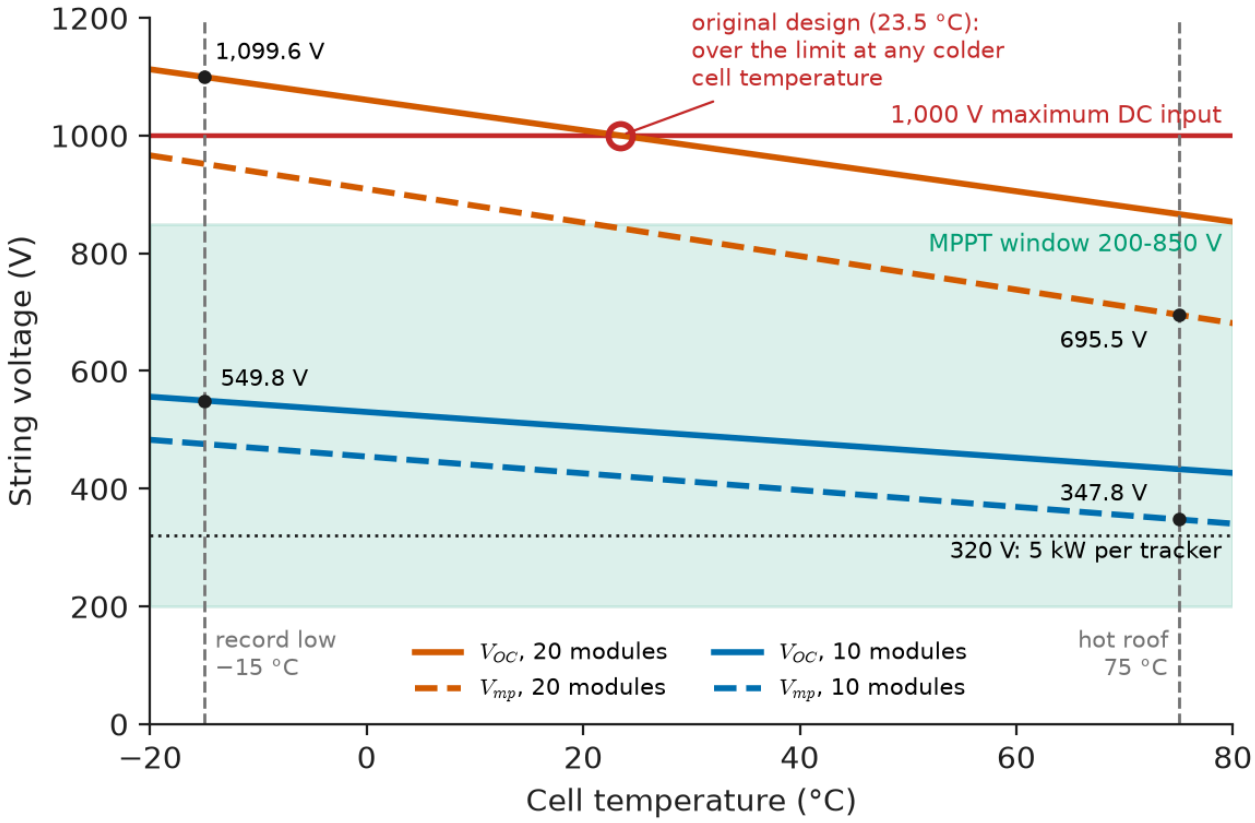


Figure 22.3 — String voltage against cell temperature for 10- and 20-module strings inside the inverter's voltage window.

Mixed orientations and unequal strings

Each tracker accepts one string length and one orientation. Strings of different lengths or orientations in parallel on one tracker run at a compromise voltage; a 9-module string paralleled with a 10-module string loses about 2–4 % of the pair's energy. The school's two equal strings on two trackers avoid the question; a roof with east and west faces would give each face its own tracker.

22.5 Inverters

Architecture choice

Three architectures compete at this size:

Table 22.4 — Inverter architectures considered for the school.

Criterion	String inverter + AC-coupled battery inverter	Three-phase hybrid inverter, DC-coupled battery	Microinverters + AC battery
Conversion stages PV → battery → load	3 (DC–AC, AC–DC, DC–AC)	2 (DC–DC, DC–AC)	3
Backup output	Battery inverter, often single-phase	Integrated EPS output, < 20 ms switchover	Needs separate gateway
Shading tolerance	Per tracker	Per tracker	Per module
Module-level shutdown	Needs optimisers	Needs optimisers	Inherent
Parts on the roof	None	None	20 inverters at 70 °C
Cost (2026, installed electronics)	$2,900	$2,300	$3,800
Decision	Rejected (cost, single-phase backup)	**Chosen**	Rejected (cost, roof electronics)

The hybrid inverter's data that matter: maximum DC input 1,000 V; two MPPTs, 200–850 V, 16 A operating and 20 A short-circuit input each; battery port 150–600 V, 25 A charge and discharge; 10 kW AC, 16 A per phase; European efficiency 97.6 %, maximum 98.2 %; backup output 10 kVA with automatic transfer; certificates to IEC 62109-1/-2 (safety), IEC 62116 (anti-islanding test) and EN 50549-1. A transformerless topology is assumed, so the array's leakage current and insulation are monitored by the inverter itself (Section 22.8).

DC/AC ratio and clipping

The array is 11.0 kWp on a 10 kW inverter, a DC/AC ratio of 1.10. At this site, cell temperature, soiling and wiring losses mean the array rarely delivers more than about 85 % of its STC power at the inverter's input, so clipping costs about 0.2 % a year. A ratio of 1.3 would clip 1.5–2 % but fill the inverter more often, the right choice when the inverter is expensive relative to modules or when export is capped. Here the export cap (10 kW, R5) is enforced by the inverter's export-limiting loop against the CT meter at the main incomer, and since the school's own daytime load is rarely below 3 kW in term time, the cap bites only in holidays.

> **Status of the principle: Optimal DC/AC ratio**
> *Empirical.* Ratios of 1.1–1.4 are chosen from simulated clipping losses against the relative cost of modules, inverters and grid capacity; the optimum moves with module price, climate and export rules, and is a design-study result, not a constant.

22.6 MPPT

What the inverter's tracker must do

Each tracker holds its string at the knee of the P–V curve by adjusting its input-voltage reference. The perturb-and-observe and incremental-conductance algorithms, and the hardware details of the boost stage that implements them, are in *Electronics and Computing*, Ch. 32 §32.8. The integrator does not write the tracker but must understand its parameters, because they decide the energy lost to shading, to fast clouds and to the tracker's own oscillation, and because a wrong installer setting (a disabled global scan, a narrowed MPPT window) is a common cause of under-performance. The listing shows the structure of a typical tracker with a periodic global scan; it is illustrative firmware for a C11 target, not any vendor's code.

```
 /* mppt.c -- perturb-and-observe with periodic global scan (C11, illustrative) */
#include <stdint.h>
#include <stdbool.h>

#define V_MIN        200.0f   /* MPPT floor, V                          */
#define V_MAX        850.0f   /* MPPT ceiling, V                        */
#define V_SCAN_HI    520.0f   /* upper end of global scan (~0.95 Voc)   */
#define V_STEP         2.0f   /* perturbation step, V                   */
#define P_DEADBAND     5.0f   /* W, above the power-measurement noise   */
#define SCAN_PERIOD 9000u     /* calls between scans: 15 min at 10 Hz   */
#define SCAN_POINTS   24u     /* 2.4 s sweep                            */

typedef struct {
    float v_ref, p_prev, dir, best_v, best_p;
    uint32_t ticks, k;
    bool scanning;
} mppt_t;

void mppt_init(mppt_t *m, float v_start) { *m = (mppt_t){ .v_ref = v_start, .dir = 1.0f }; }

/* Called every 100 ms with averaged tracker voltage and current; returns the new reference. */
float mppt_step(mppt_t *m, float v, float i)
{
    const float p = v * i;
    if (m->scanning) {                            /* coarse sweep of the whole curve */
        if (p > m->best_p) { m->best_p = p; m->best_v = v; }
        if (++m->k < SCAN_POINTS) {
            m->v_ref = V_MIN + (V_SCAN_HI - V_MIN) * (float)m->k / (float)(SCAN_POINTS - 1u);
        } else {                                  /* resume P&O at the global maximum */
            m->scanning = false;
            m->v_ref = m->best_v;
            m->p_prev = m->best_p;
        }
        return m->v_ref;
    }
    if (++m->ticks >= SCAN_PERIOD) {              /* start a global scan */
        *m = (mppt_t){ .v_ref = V_MIN, .dir = m->dir, .scanning = true, .best_v = m->v_ref };
        return m->v_ref;
    }
    if (p - m->p_prev < -P_DEADBAND) m->dir = -m->dir;   /* power fell: reverse */
    m->p_prev = p;
    m->v_ref += m->dir * V_STEP;
    if (m->v_ref < V_MIN) { m->v_ref = V_MIN; m->dir = 1.0f; }
    if (m->v_ref > V_MAX) { m->v_ref = V_MAX; m->dir = -1.0f; }
    return m->v_ref;
}
```

Step size, deadband and scan cost

Three parameters trade against each other, and each has a number:

Worked example — What do the step, deadband and scan cost? Near the MPP the P–V curve of a ten-module string is a flat parabola. With an explicit single-diode-type model fitted to the datasheet points, a ±2 V oscillation about 419.6 V (three-level P&O) costs 0.013 % of 5.50 kW, about 0.7 W, below the 5 W deadband. The tracker therefore does not reverse on a 2 V step: it walks until the power drop exceeds the deadband, about 5–6 V from the MPP, where the loss is still under 0.1 %. The deadband exists to stop measurement noise, not the curve, from steering the tracker. The scan is costlier: a 2.4 s sweep every 15 minutes during which the string delivers on average about 60 % of its MPP power loses $0.4 \times 2.4/900 = 0.11$ % of the energy, against a gain, on the school's string S2, of several per cent during the winter mornings when the ventilation stacks create a second local maximum. A sweep every minute would cost 1.6 %, which is why scans are periodic and, in better inverters, triggered by a sudden change in the string's apparent fill factor.

The tracker is verified as a black box: EN 50530 defines static MPPT efficiency (energy delivered relative to the available MPP energy at fixed irradiance, typically above 99.7 %) and dynamic efficiency over irradiance ramps from 0.5 to 100 W/m²/s (typically 99–99.8 %). Figure 22.4 shows a simulated trace of the tracker on string S2 through a shading event.

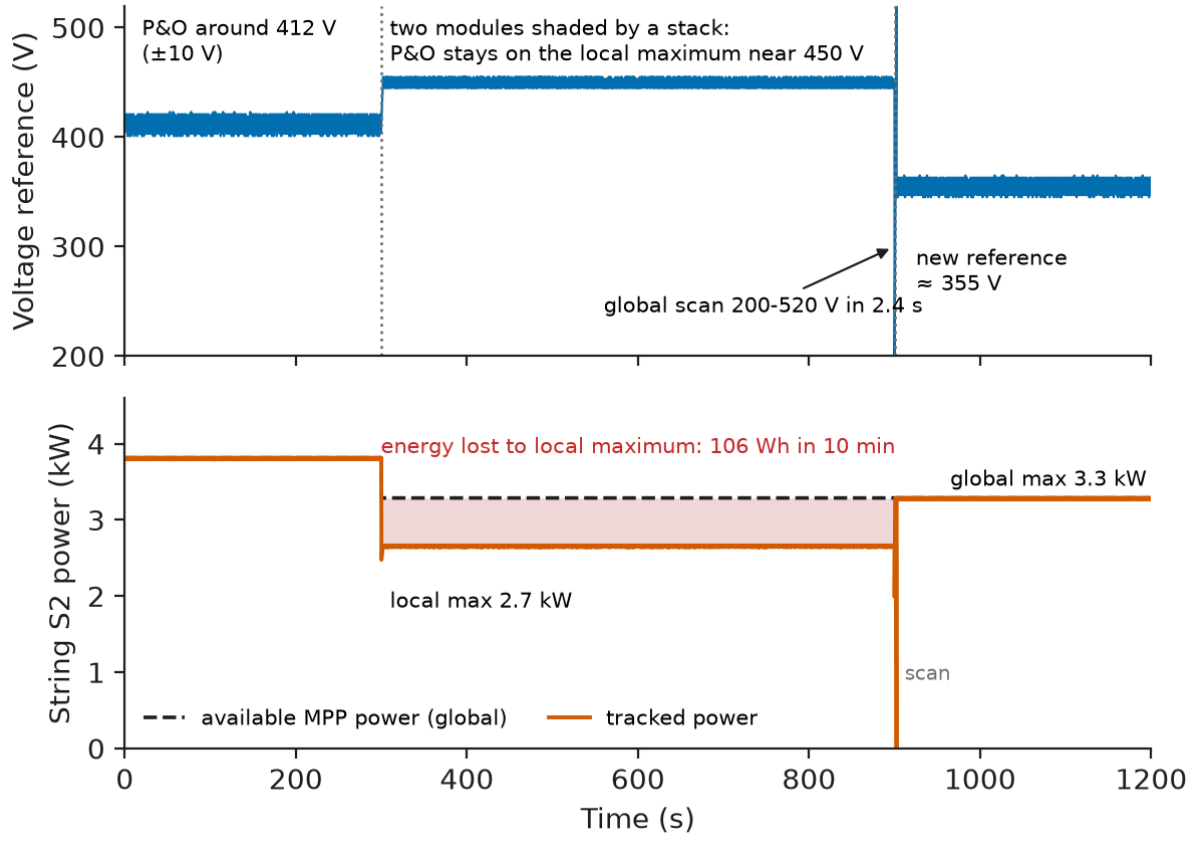


Figure 22.4 — Simulated perturb-and-observe tracking on string S2 through a winter-morning shading event, with a periodic global scan.

22.7 Battery integration

> **Boundary.**
> Cell chemistry, the battery-management system, state estimation, thermal management and battery protection are owned by Chapter 23; the converter that charges the battery from the DC link is in *Electronics and Computing*, Ch. 32 §32.7. Here the battery is a block with a voltage window, a current limit, an efficiency and a CAN interface.

Coupling

The battery can sit on the DC side of the hybrid inverter (DC coupling) or behind its own inverter on the AC side (AC coupling). DC coupling saves a conversion and lets the battery absorb energy the inverter would otherwise clip; AC coupling allows retrofitting to any existing PV inverter and siting the battery anywhere on the AC network.

Worked example — How much does DC coupling save? On the path PV → battery → evening load, the DC-coupled system passes the energy through the battery's DC–DC converter (98 %), the battery (95 % round trip) and the inverter (96.5 % at the battery-discharge operating point): $0.98 \times 0.95 \times 0.965 = 0.898$. An AC-coupled system passes it through the PV inverter (97 %), the battery inverter charging (95.5 %), the battery (95 %) and the battery inverter discharging (95.5 %): $0.97 \times 0.955 \times 0.95 \times 0.955 = 0.840$. With 5.1 kWh cycled on each of 200 school days (1,020 kWh a year), the difference is $1{,}020 \times (0.898 - 0.840) = 59$ kWh a year, about \$10 at the school's \$0.17/kWh tariff. The efficiency argument is real but small; the decision was made on equipment cost and the three-phase backup output.

Sizing the battery

Two uses share the battery (R8). The **backup reserve** must cover 1.2 kW for 3 h, 3.6 kWh, plus the battery's own lowest 10 % that the BMS will not deliver. The **daily shift** moves midday surplus to the evening sports-hall load, about 5 kWh on a weekday. A 10.24 kWh LFP stack (four 2.56 kWh modules, 64 cells in series, 204.8 V nominal, 180–233 V operating) gives 9.2 kWh between 10 % and 100 % SoC. The reserve must deliver 3.6 kWh of AC, $3.6/0.965 = 3.73$ kWh from the cells, 36 % of capacity above the 10 % floor, so the backup SoC floor is set at 50 %, leaving $10.24 \times (1 - 0.50) = 5.1$ kWh for daily cycling. Its voltage window lies inside the inverter's 150–600 V battery port, and the port's 25 A limit allows $25 \times 204.8 = 5.1$ kW of charge or discharge, 0.5C for this battery, comfortably inside the cells' rating. A larger battery would sit full through the summer holidays, when the school is closed and the surplus is exported anyway.

The battery talks to the inverter over CAN (the battery's BMS sends charge and discharge current limits and voltage set-points every second, the inverter obeys them and stops if the messages stop) — the closed-loop interface whose BMS side is designed in Chapter 23.

22.8 Protection

> **Boundary.**
> Protection principles, earthing systems and protective devices are in *Electrical Energy and Machines*, Ch. 29 §29.3–29.5; the hazards of PV and batteries that cannot be switched off at the source in §29.6.

What is different about a PV array

A PV array is a current source limited to about $1.25\,I_{SC}$: it cannot deliver a fault current large enough to blow a fuse in its own string, it cannot be switched off while the sun shines, and DC arcs do not extinguish at a current zero. Protection is built around those three facts (IEC 60364-7-712, IEC 62548):

- **String overcurrent.** A fault in one string can be fed in reverse by the other strings in parallel with it. String fuses (gPV, IEC 60269-6) are required only when $(N_p - 1) \times 1.25\,I_{SC}$ exceeds the module's reverse-current rating (typically 25–30 A), which needs three or more strings in parallel. With one string per tracker the school needs no string fuses.
- **Isolation.** A DC switch-disconnector rated for the full string voltage and current under DC-PV2 utilisation (IEC 60947-3), here integrated in the inverter, isolates the array from the inverter. It does not make the array cables dead.
- **Surge protection.** Type 2 SPDs (IEC 61643-31) at the inverter's DC input with $U_{CPV} \geq V_{OC,\max}$ (600 V class against 549.8 V) and on the AC side; type 1 if the building has a lightning-protection system whose separation distance to the array cannot be kept.
- **Insulation and residual current.** A transformerless inverter measures the array's insulation resistance before connecting each morning and refuses to start below $V_{\max}/30\,\text{mA}$ (IEC 62109-2; 18 kΩ here, with practical settings around 100 kΩ), and monitors residual current during operation (trip on a 30 mA step or 300 mA continuous). The external RCD type (A or B) follows the inverter's declaration of whether it can feed smooth DC fault current.
- **Arc faults and rapid shutdown.** In North America NEC 690.11 requires arc-fault detection (UL 1699B) and NEC 690.12 module-level rapid shutdown for firefighter safety; in Europe IEC 63027 arc-fault detection is increasingly specified, and some countries require a firefighter switch. The school's insurer asked for an AFCI-equipped inverter.
- **Anti-islanding and grid protection.** The inverter's certified functions (voltage and frequency trips, loss-of-mains detection, *Electrical Energy and Machines* §27.5) with settings fixed by EN 50549-1 and recorded at commissioning.
- **Battery.** A DC breaker or fuse rated for the battery's prospective short-circuit current at the battery terminals, and the BMS contactors behind it (Chapter 23).

> **Status of the principle: Reverse-current rule for string fuses**
> *Derived.* A short in one string makes it a load for the other $N_p - 1$ strings on the same tracker, each limited to about $1.25\,I_{SC}$; fuses are needed only when that sum can exceed the module's tested reverse-current rating, which is why one or two strings per tracker need none.

22.9 Cable sizing

> **Boundary.**
> Current-carrying capacity, derating and voltage-drop calculation for LV cables are in *Electrical Energy and Machines*, Ch. 30 §30.5–30.6.

DC cables

PV string cables are double-insulated, UV- and ozone-resistant single cores (H1Z2Z2-K to EN 50618, 1.5 kV DC, 90 °C continuous and 120 °C for 20,000 h). IEC 62548 requires their current-carrying capacity, after derating, to be at least $1.25\,I_{SC} = 17.5$ A for an unfused string. A 4 mm² cable in free air carries about 55 A at 60 °C ambient, so current never decides the size on a residential roof; voltage drop and losses do.

Worked example — DC and AC conductors. Each string runs 25 m from the array to the inverter, a 50 m loop. Copper at a 70 °C conductor temperature has $\rho = 0.0175 \times [1 + 0.00393 \times 50] = 0.0209\ \Omega\,\mathrm{mm^2/m}$. With 4 mm², $R = 0.0209 \times 50/4 = 0.262\ \Omega$ and at $I_{mp} = 13.13$ A the drop is 3.44 V, 0.82 % of 419 V, a loss of 45 W at STC. With 6 mm², $R = 0.174\ \Omega$, 2.29 V (0.55 %) and 30 W. Both satisfy R6; because cable loss scales with current squared and most energy arrives at part load, the annual energy loss is roughly 60 % of the STC fraction, about 0.5 % for 4 mm² and 0.3 % for 6 mm². The installer chose 6 mm² for an extra $18 per string and cooler MC4-type connectors. On the AC side the inverter's 16 A per phase runs 30 m to the sub-board. Balanced three-phase current produces a voltage rise per phase of $I\,R_{\text{phase}}$: with 4 mm² (5.52 Ω/km at 70 °C) the rise is $16 \times 0.030 \times 5.52 = 2.65$ V, 1.15 % of 230 V; with 6 mm² (3.69 Ω/km) it is 1.77 V, 0.77 %. The voltage rise matters more than the drop: at the end of this rural feeder the grid sits at up to 244 V on light-load afternoons, and the DSO's volt–watt setting starts reducing inverter power at $1.07\,U_n = 246$ V (the 10-minute over-voltage trip of EN 50549-1 is at 253 V). With 4 mm² the inverter terminals reach 246.7 V and the system would curtail on sunny holiday afternoons; with 6 mm², 245.8 V. 6 mm² was chosen; a 20 A MCB protects it.

Table 22.5 — Budgets for the school PV system.

Quantity	Limit	Design value	Margin
String V_{OC} at −15 °C	1,000 V	549.8 V	45 %
String V_{mp} at 75 °C	200 V	347.8 V	74 %
Tracker current I_{mp}	16 A	13.13 A	18 %
Tracker I_{SC} at 75 °C	20 A	14.32 A	28 %
DC voltage drop at STC (6 mm²)	1 %	0.55 %	0.45 points
AC voltage rise at 16 A (6 mm²)	1 %	0.77 %	0.23 points
Battery current	25 A port; cells 50 A (1C)	25 A at 5.1 kW (0.5C)	50 % to cell rating
First-year energy (R1)	≥ 15,000 kWh	15,800 kWh (P50)	5.3 %

22.10 System losses

The loss tree

Yield is predicted by following the irradiation on the array through each loss in turn. The listing applies the loss tree used for the school, each fraction taken from the shading survey, the module and inverter datasheets, the cable calculation and a monthly temperature simulation:

```
 # Loss chain, performance ratio and first-year yield of the 11.0 kWp school roof
P_stc = 20 * 0.550                      # kWp
H_poa = 1734                            # kWh/m2/yr on 15 deg, south (Table 22.2)
losses = {                              # fraction lost at each step, in order
    "near shading (parapet, vents)": 0.015, "soiling": 0.020,
    "angle of incidence (IAM)": 0.025,  "low irradiance": 0.010,
    "module temperature": 0.050,       "module quality and LID": 0.010,
    "mismatch": 0.010,                 "DC cabling": 0.003,
    "inverter conversion": 0.024,      "inverter clipping": 0.002,
    "AC cabling": 0.005,               "availability": 0.010,
}
E = P_stc * H_poa                       # reference yield x power, kWh
print(f"{'step':32s} {'loss':>6s} {'kWh':>8s}")
for name, f in losses.items():
    dE = E * f
    E -= dE
    print(f"{name:32s} {f:6.1%} {dE:8.0f}")
PR = E / (P_stc * H_poa)
print(f"AC energy {E:.0f} kWh/yr, specific yield {E/P_stc:.0f} kWh/kWp, PR = {PR:.3f}")
```

```
 step                             loss      kWh
near shading (parapet, vents)     1.5%      286
soiling                           2.0%      376
angle of incidence (IAM)          2.5%      460
low irradiance                    1.0%      180
module temperature                5.0%      889
module quality and LID            1.0%      169
mismatch                          1.0%      167
DC cabling                        0.3%       50
inverter conversion               2.4%      396
inverter clipping                 0.2%       32
AC cabling                        0.5%       80
availability                      1.0%      160
AC energy 15830 kWh/yr, specific yield 1439 kWh/kWp, PR = 0.830
```

The predicted first-year energy is 15,800 kWh, 1,439 kWh/kWp, with a performance ratio of 0.830 (Figure 22.5). Temperature is the largest single loss, then angle-of-incidence reflection, the inverter and soiling. The **performance ratio**, $\mathrm{PR} = E_{AC} / (P_{\mathrm{STC}}\, H_{\mathrm{POA}} / G_{\mathrm{STC}})$, removes the resource from the comparison and is the number the contract and R2 are written against.

> **Status of the principle: Multiplicative loss chain**
> *Approximate.* Treating losses as independent factors applied in sequence is the convention of yield software; it is accurate to a fraction of a per cent when each loss is small, but interacting losses (shading with mismatch, temperature with clipping) are captured properly only by hourly simulation.

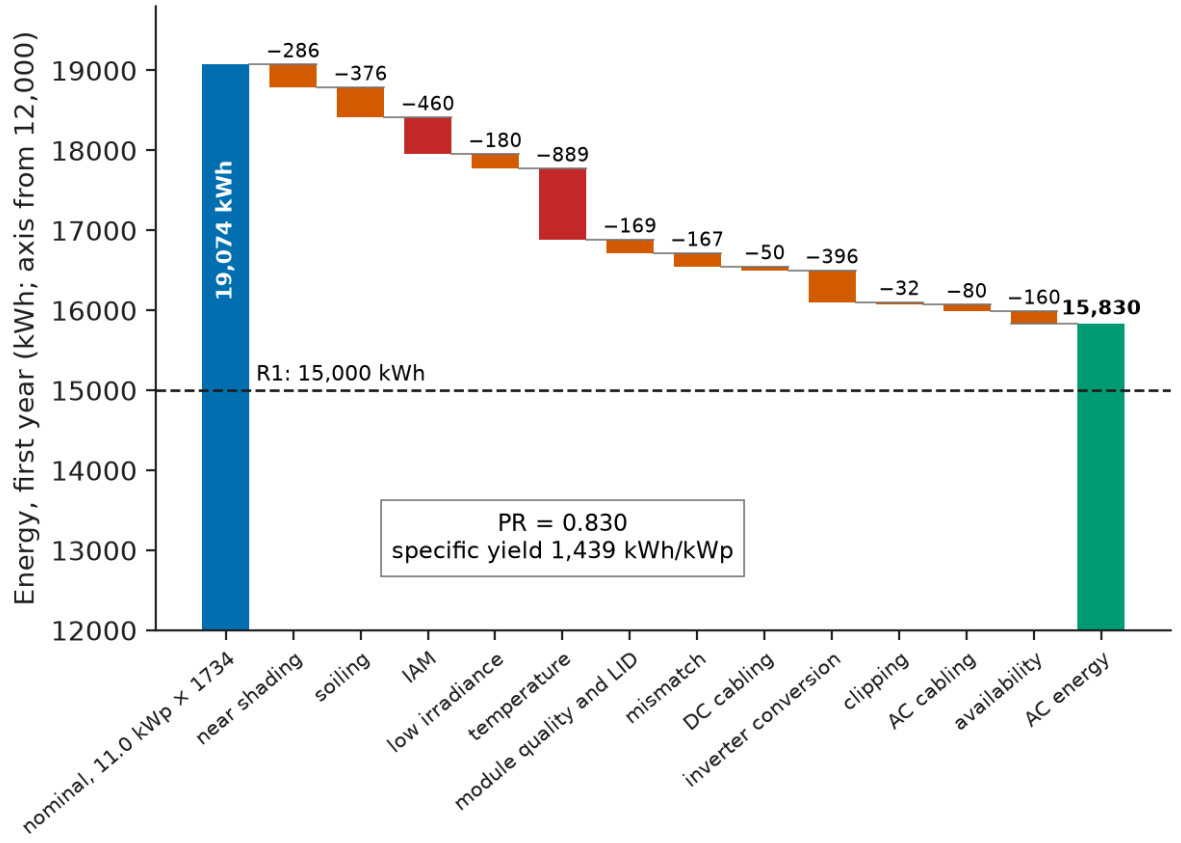


Figure 22.5 — Energy loss waterfall from plane-of-array irradiation to AC energy for the school system, first year.

Worked example — Uncertainty and P90. The yield model's uncertainty combines, in quadrature, interannual irradiation variability (4.5 %), the satellite dataset (4.5 %), transposition (2 %), the loss model (3 %) and degradation and availability (1.5 %): $\sigma = \sqrt{4.5^2 + 4.5^2 + 2^2 + 3^2 + 1.5^2} = 7.5$ %. The one-year P90 is $15{,}800 \times (1 - 1.282 \times 0.075) = 14{,}300$ kWh, below R1's 15,000 kWh. R1 is therefore written, and verified, as a P50 commitment with weather correction: the 12-month metered energy is compared with the model rerun on the measured POA irradiation, not with 15,000 kWh directly.

22.11 Performance evaluation

Commissioning tests

IEC 62446-1 defines the tests that hand a grid-connected system over: visual inspection, continuity of protective conductors, polarity, string open-circuit voltage and short-circuit (or operating) current, functional tests of switches and the inverter, and insulation resistance of the array (test voltage 1,000 V DC for arrays above 500 V, pass at ≥ 1 MΩ). Category 2 tests add an I–V curve trace of each string and an infrared survey.

Worked example — A string 16 V low. At commissioning the reference cell reads 820 W/m² and the back-of-module Pt1000 38 °C. The expected string V_{OC} is $10 \times 49.8 \times [1 - 0.0026 \times 13] = 481.2$ V, less the logarithmic irradiance effect of about 0.46 V per module (72 cell-equivalents, $n \approx 1.2$, $\ln 0.82$), so 476.6 V. String S1 measures 475.1 V (−0.3 %); S2 measures 460.2 V, 16.4 V low. One module's open-circuit voltage is 47.7 V at these conditions and one of its three substrings 15.9 V: S2 has a substring that contributes nothing

— a shorted bypass diode. The I–V trace of S2 showed the same step-free curve 3.4 % lower in voltage, and an infrared image found the junction box of module M14 18 K warmer than its neighbours. The module was replaced under warranty before handover; with the diode shorted, S2 would have lost about 3 % of its energy for 25 years.

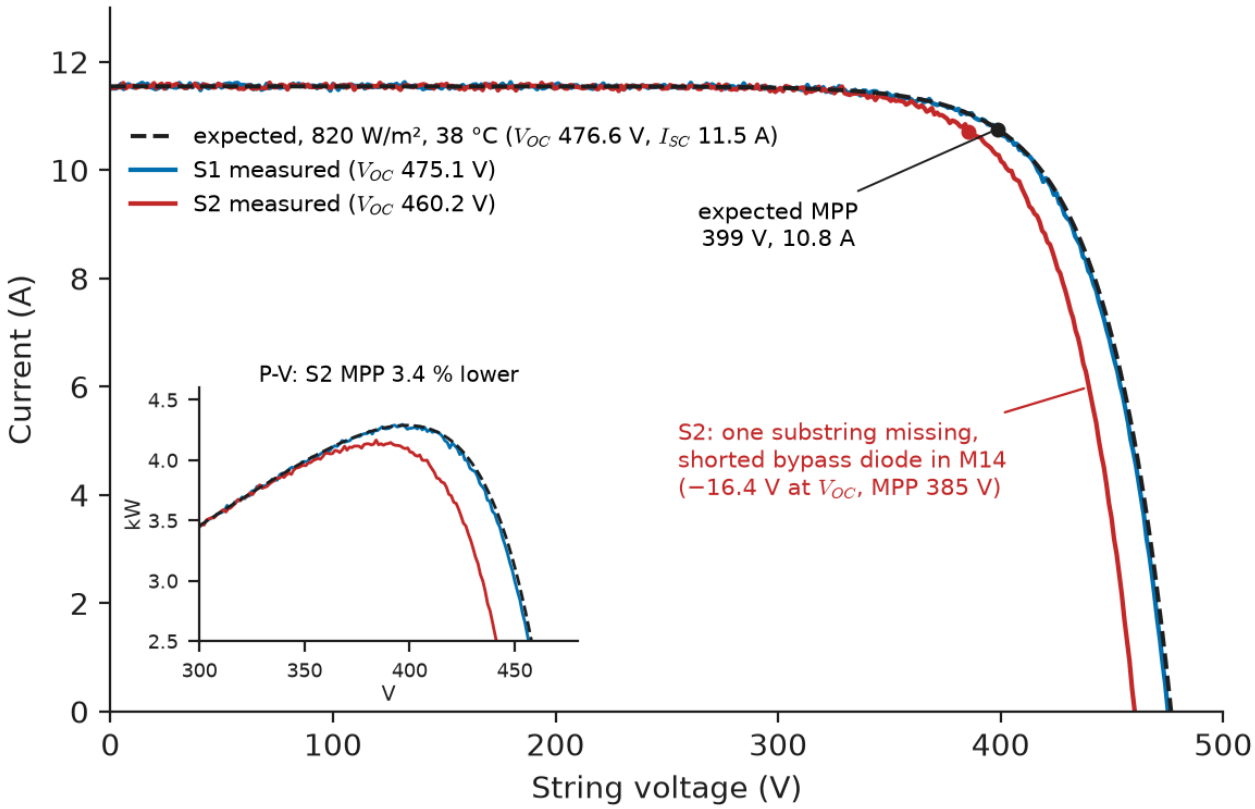


Figure 22.6 — Commissioning I-V traces of strings S1 and S2 at 820 W/m2 and 38 C, showing the shorted bypass diode in S2.

Monitoring and the monthly PR check

IEC 61724-1 sets the monitoring classes; for a system of this size Class C (estimated irradiance) is common, but the school specified a Class B-like set (R9): a POA reference cell, a back-of-module temperature sensor, DC and AC power per tracker from the inverter, export and import from the CT meter, and battery data, all at 5-minute resolution. The PR is computed monthly and compared with the model's monthly PR after correcting both to 25 °C, because a raw summer PR is always lower than a winter one.

Worked example — The June check. In June the reference cell recorded 196 kWh/m² and the meter 1,668 kWh of AC energy: $\mathrm{PR} = 1{,}668/(11.0 \times 196) = 0.774$. The irradiance-weighted module temperature for the month was 47 °C, so the temperature factor is $1 - 0.0030 \times 22 = 0.934$ and the temperature-corrected PR is $0.774/0.934 = 0.828$. The model's temperature-corrected PR is $0.830/(1 - 0.050) = 0.874$. June was 5.2 % below the model, outside R2's 3 % band. The per-tracker data located it: both trackers lost power between 13:00 and 16:00 on hot days, with the inverter's own log showing thermal derating at an internal temperature of 75 °C. The inverter had been mounted in a south-facing plant room that reached 44 °C; it derates above 40 °C ambient. A louvre and a thermostat-controlled fan (60 W, running about 400 h a year) brought the room below 35 °C, and July's corrected PR was 0.866, within 1 % of the model.

Table 22.6 — Verification of the school system.

Req.	Test	Limit	Result	Pass/fail
R1	Model rerun with measured POA vs 12-month metered energy	Metered ≥ 97 % of weather-corrected model	98.6 % (after June fix)	Pass
R2	Monthly temperature-corrected PR	Within 3 % of model	June −5.2 % (fixed); other months −1.8 to +0.6 %	Pass after fix
R3	V_{OC} per string, translated to −15 °C	≤ 950 V	S1 548 V, S2 549 V (after module swap)	Pass
R4	Calculation; lowest logged MPP voltage in July	≥ 260 V	351 V	Pass
R5	Export test with school load switched off	≤ 10.0 kW	9.96 kW peak, 10-min mean 9.8 kW	Pass
R6	Voltage at inverter and sub-board at 16 A	Rise ≤ 1 %	0.8 %	Pass
R7	Insulation at 1,000 V DC; continuity; polarity; SPD check	≥ 1 MΩ	S1 85 MΩ, S2 92 MΩ	Pass
R8	Open main breaker with 1.2 kW load on backup board	Switchover ≤ 20 ms; 3 h	12 ms; 3 h 15 min from the 50 % SoC floor	Pass
R9	Data completeness over 30 days	≥ 99 %	99.6 %	Pass

22.12 Failure analysis

Table 22.7 — FMEA of the school PV system.

Failure mode	Effect	Cause	Detection	Mitigation
String over-voltage on cold mornings	Inverter refuses to start; possible input-stage damage	String sized on STC V_{OC}	Inverter alarm log; design check	Size at record-low temperature (Section 22.4); design script in repository
Shorted bypass diode	1/3 module lost, hot junction box	Transport damage, lightning surge	V_{OC} and I–V test; IR survey	IEC 62446-1 category 2 tests at handover
Tracker stuck on local maximum	Several per cent loss on shaded mornings	Global scan disabled or too rare	Per-tracker PR on clear mornings	Scan every 15 min; shaded modules grouped on one string
Inverter thermal derating	Afternoon power lost in summer	Hot plant room	Monthly corrected PR; inverter temperature log	Ventilation; derating curve in siting rules
Insulation fault (wet connector)	Inverter refuses to start on damp mornings	Unmated or poorly crimped connector, cable chafing	Riso alarm, morning start delay	Same-brand connectors, crimp-tool control, cable clips
Over-voltage curtailment	Export lost on sunny holidays	AC cable voltage rise on a high-voltage feeder	Inverter event log; voltage logging	6 mm² AC cable; reactive-power setting agreed with the DSO
DC arc at a connector	Fire risk	Mismated connectors, loose crimp	AFCI trip	Connector discipline, AFCI inverter, torque checks
Battery not available at outage	Server room lost	SoC floor eroded by self-consumption setting	SoC log; quarterly outage test	50 % backup floor locked in installer menu

The original failure in the opening was a missing line in a calculation. The second iteration put that calculation into a script that runs on every change of module or inverter, wrote both temperature limits into the requirements (R3, R4), and made the commissioning test translate the measured V_{OC} to the record-low temperature, so a wrong string length cannot pass handover even if the design check is skipped. Most of the remaining rows were found the same way: by a number written down before the build (a PR band, an insulation limit, a voltage-rise limit) that the real system failed to meet.

What remains open

Open question	Where it is answered
How is the battery stack's BMS, protection and state estimation designed?	Chapter 23
How is a PV array combined with wind, storage and a generator into a microgrid with an energy-management system?	Chapter 25
How is a complete off-grid solar battery system sized and costed from the load up?	Chapter 36
How is the energy meter that measures the system's export designed?	Chapter 35
How do grid-connected inverters synchronise, control P and Q and ride through faults?	*Electrical Energy and Machines*, Ch. 27
How are the inverter's boost and bridge stages designed?	*Electronics and Computing*, Ch. 32
How do PV materials degrade over 25 years?	*Materials Science and Engineering*, Ch. 40 §40.9

Next: Chapter 23 opens the battery block of this system and designs a battery energy system from the cell up.

Chapter 23 — Battery Energy Systems

Part VI · Renewable Energy and Electric Mobility

A food-processing plant bought a 215 kWh, 100 kW lithium-iron-phosphate battery cabinet to shave its afternoon demand peaks and soak up its rooftop PV. The contract said 200 kWh usable. In the fourteenth month a peak-shaving test delivered 181 kWh, and the battery stopped with its display showing 14 % state of charge, when the first cell reached its 2.80 V cut-off. Nothing was broken. The air-cooled cabinet's top module sat in the exhaust of the power-conversion system and ran 6 K warmer than the rest, so its sixteen cells had aged faster and self-discharged faster than the other 224; the balancing circuit, sized to trim manufacturing spread at 50 mA, could not keep up; and a Hall-effect current sensor with a 0.15 A offset had let the coulomb counter drift through weeks in which the controller never charged the battery full, on a chemistry whose flat voltage curve gives the estimator almost nothing to correct against. Every part met its datasheet. The system did not meet its contract, because thermal design, balancing, sensing and state estimation had been specified separately. This chapter designs that cabinet again, from the cell up.

> **How is a battery pack architected, from cell choice and series–parallel configuration to contactors, fuses and the battery-management system, so that it delivers its rated energy and power safely over its whole life?**
> **How are its state of charge, temperature and health measured well enough to keep that promise, and how is it tested before anyone is allowed near it?**

Concept	Engineering destination
Battery-pack architecture	Cell–module–rack hierarchy, contactors, precharge, fuses, current sensing, isolation monitoring
Cell selection	Chemistry and format chosen against energy, power, life, safety and cost
Series and parallel configurations	Voltage window matched to the converter; parallel strings and current sharing
Battery-management systems	Analog front ends, daisy chains, protection logic, balancing, interfaces
State estimation	SoC, SoH and power limits with an error budget behind each
Thermal management	Heat generation, cooling choice, cell-to-cell temperature spread
Charging	CC–CV profiles, temperature-dependent current limits, the BMS–charger handshake
Protection	Layered limits, fuse–contactor coordination, short-circuit and insulation faults
Degradation monitoring	Capacity and resistance tracking, fade projection against warranty
Safety testing	IEC 62619, UL 9540A, UN 38.3, HIL fault injection

Roadmap. Requirements and architecture (23.1) → cell selection (23.2) → series and parallel configuration (23.3) → the BMS (23.4) → state estimation (23.5) → thermal management (23.6) → charging (23.7) → protection (23.8) → degradation monitoring (23.9) → safety testing and verification (23.10) → failure analysis (23.11).

23.1 Requirements and pack architecture

> **Boundary.**
> Battery chemistries, energy and power ratings, the functions of a BMS, SoC and SoH definitions and estimators, ageing mechanisms and storage safety are covered at system level in *Electrical Energy and Machines*, Ch. 16 §16.1–16.5 and §16.11; electrode and electrolyte materials in *Materials Science and Engineering*, Ch. 40 §40.2–40.3; the CC–CV charger and bidirectional converter hardware in *Electronics and Computing*, Ch. 32 §32.7. This chapter owns the integration: the pack, the BMS as a product, its budgets and its tests. Chapters 24, 25 and 36 apply it.

Requirements

The cabinet serves a 100 kW bidirectional power-conversion system (PCS) whose DC input accepts 600–950 V, under a site energy-management system (EMS) that schedules one full cycle a day: charging from PV and off-peak grid, discharging into the afternoon peak.

Table 23.1 — Requirements of the 215 kWh cabinet.

ID	Requirement	Target	Verification
R1	Usable AC-side energy at 0.5C, 25 °C	≥ 200 kWh at commissioning; ≥ 160 kWh in year 10	Capacity test (IEC 62933-2-1); ageing projection
R2	Power	100 kW charge and discharge, ambient −10 to 45 °C	Full-power test at 45 °C ambient
R3	DC voltage	Inside PCS window 600–950 V at all SoC, temperature and current	Calculation; logged extremes
R4	Cell-voltage measurement	≤ ±5 mV total error, 3.0–3.6 V, −10 to 60 °C	AFE characterisation; error budget
R5	SoC accuracy	±5 % between recalibrations, recalibration at least weekly	Reference coulometer over 30 days
R6	Cell temperature	Max ≤ 40 °C at 45 °C ambient, cell-to-cell spread ≤ 3 K	24 h test with 40 thermocouples
R7	Protection	Over-voltage detected and contactors open ≤ 1.2 s; independent hardware layer; short circuit cleared by fuse	HIL fault injection; fuse coordination study
R8	Insulation	Alarm at 500 Ω/V, trip at 100 Ω/V	IMD test with calibrated resistor
R9	Communications	Limits to PCS every 1 s over CAN; PCS stops ≤ 3 s after loss	Cable-pull test
R10	Health reporting	SoH (capacity) estimate within ±3 % of a reference test	Annual capacity test
R11	Safety	IEC 62619 certified cells and modules; UL 9540A module test without propagation; UN 38.3 for transport	Certificates; test reports

Architecture

Figure 23.1 shows the hierarchy. Two hundred and forty 280 Ah LFP cells sit in fifteen 16-cell modules, all in series (1P240S). Each module carries one 16-channel analog front end (AFE) that measures its cells and temperatures and drives the balancing; the AFEs talk in an isolated daisy chain to the battery-management unit (BMU), which owns the high-voltage switchgear: a main positive and a main negative contactor, a precharge contactor and resistor, a DC fuse, a shunt current sensor, an insulation-monitoring device and a manual service disconnect near the electrical midpoint. The BMU sends current and voltage limits to the PCS over CAN, reports to the EMS over Modbus TCP, and commands the cabinet's liquid-cooling unit.

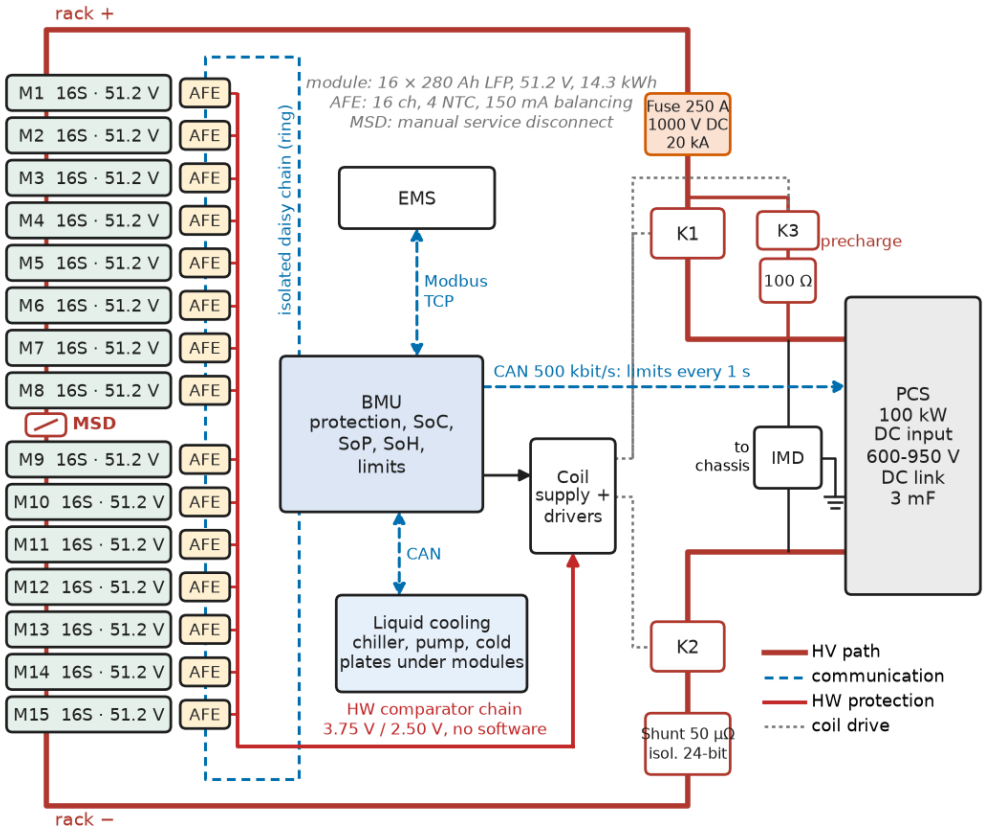


Figure 23.1 — Architecture of the 215 kWh, 100 kW LFP battery cabinet: cell, module and rack hierarchy, high-voltage switchgear and BMS communication.

The architecture fixes three interfaces that the rest of the chapter designs: the AFE–BMU chain (measurement and balancing), the BMU–PCS link (limits and the handshake that lets the PCS charge or discharge), and the BMU–switchgear path (the last line of protection).

23.2 Cell selection

What decides the chemistry

For stationary storage cycling once a day for fifteen years, cycle life, calendar life, safety and cost per kWh delivered over life outweigh energy density. That combination has made lithium iron phosphate the default; NMC wins where mass and volume matter (vehicles, Chapter 24), lithium titanate where very high power or extreme cycle counts matter, and sodium-ion is entering stationary service where its cold-weather behaviour and freedom from lithium supply matter.

Table 23.2 — Cell options for the cabinet (2026 indicative, 10,000-cell quantities).

Cell	Nominal	Energy	Cycle life to 80 % (0.5C, 25 °C)	Thermal runaway onset	Price	Assessment
LFP prismatic 280 Ah	3.2 V, 0.896 kWh, R_{DC} ≈ 0.25 mΩ	165 Wh/kg	6,000–8,000	≈ 200–270 °C, little oxygen release	$45 ($50/kWh)	**Chosen**: life, safety, a cabinet of 240 cells
LFP prismatic 314 Ah	3.2 V, 1.005 kWh	175 Wh/kg	6,000–8,000	Similar	$49	Newer; 214 cells for the same energy, odd module count; second source pending
NMC 811 pouch 60 Ah	3.65 V, 0.219 kWh	260 Wh/kg	1,500–2,500	≈ 150–200 °C, oxygen release	$20 ($90/kWh)	Rejected: life and propagation risk
Sodium-ion prismatic 160 Ah	3.1 V, 0.496 kWh	140 Wh/kg	3,000–6,000	Lower energy release	$37 ($75/kWh)	Watched: better below 0 °C; supply still thin
LTO cylindrical 40 Ah	2.3 V, 0.092 kWh	75 Wh/kg	15,000–25,000	Very stable	$40 ($435/kWh)	Rejected: cost per kWh

The cost that matters is levelised over life. A 280 Ah LFP cell at $50/kWh delivering 6,000 cycles at 93 % depth costs about $50/(6000 \times 0.93) = 0.9$ cents per kWh cycled; an NMC cell at $90/kWh and 2,000 cycles costs 4.8 cents. Format matters too: one large prismatic cell per 0.9 kWh keeps the cabinet to 240 voltage taps and 240 welds, against about 1,000 for the pouch option.

What the datasheet must give, and what to test anyway

The selection depends on datasheet numbers that are not always comparable: capacity at a stated rate and temperature, DC resistance at a stated pulse length and SoC, cycle life at a stated depth, rate, temperature and end-of-life criterion, and the charge-current limits against temperature. Incoming inspection therefore measures every cell (capacity class, 1 kHz AC resistance, OCV after a rest, self-discharge from a 7–14 day voltage decay), and the module builder bins cells so that each module's cells match within about 0.5 % in capacity and 5 % in resistance.

> **Status of the principle: Cycle life against depth, rate and temperature**
> *Empirical.* Datasheet cycle lives are accelerated-test results at one condition; extrapolation to other depths and temperatures uses fitted models (Arrhenius in temperature, power laws in depth of discharge) that the cell vendor's warranty, not physics, makes binding.

23.3 Series and parallel configurations

The voltage window

The number of cells in series is set by the converter's window, at the extremes of SoC, current and temperature, not by the nominal voltage. With N_s cells, the rack's operating range is $N_s V_{\min}$ to $N_s V_{\max}$, and under load the terminal voltage departs from the open-circuit voltage by $I\,R_{\text{rack}}$.

Worked example — How many cells in series? The PCS accepts 600–950 V and needs at least 640 V at its terminals to deliver full AC power at the high end of the grid tolerance. The rack's resistance is 240 cells × 0.25 mΩ plus 15 mΩ of busbars, fuse, contactors and shunt, 75 mΩ; at 0.5C (140 A) that drops 10.5 V. At the bottom, with the discharge cut-off at 2.90 V per cell under load, 240 cells give 696 V, and at the top of charge, 3.55 V per cell plus a 10.5 V charging rise gives 862 V; the absolute cell limit of 3.65 V gives 876 V. The window 696–876 V sits inside 640–950 V with 56 V margin at the bottom and 74 V at the top. With 216 cells the bottom would be 626 V, below the full-power floor; with 256 cells the top would be 934 V at the 3.65 V limit, inside the PCS rating but within 16 V of it. 240 cells also divide into fifteen 16-cell modules, matching the 16-channel AFE.

> **Status of the principle: Weakest-cell limit of a series string**
> *Derived.* The same current flows through every cell in series, so the first cell to reach a voltage limit ends charge or discharge for all; the usable capacity of the string is that of its weakest cell, adjusted by how far its SoC is offset from the others. Balancing corrects SoC offsets, never capacity differences (*Electrical Energy and Machines* §16.3).

Cells in parallel and racks in parallel

Paralleling cells inside a module (2P of 140 Ah cells instead of 1P of 280 Ah, say) halves the voltage-tap count per kWh and gives the cells a common voltage, so they self-balance; the cost is that one internally shorted cell is fed by its parallel neighbours, which demands a fusible link per cell in cylindrical-cell packs, and that the BMS can no longer see an individual cell's capacity loss.

Paralleling *racks* is how a site grows: the plant's expansion plan adds a second cabinet in year 3. Two racks on one DC bus share current in inverse proportion to their resistances, modified by any difference in open-circuit voltage, and an aged rack has both less capacity and more resistance. The listing simulates a 200 kW discharge of a three-year-old rack (93 % capacity, 25 % higher resistance) in parallel with a new one, using a piecewise LFP open-circuit-voltage curve:

```
 # Two 240S LFP racks in parallel on one PCS: current sharing during a 200 kW discharge
import numpy as np
soc_pts = np.array([0, 5, 10, 20, 30, 50, 70, 90, 95, 100]) / 100
ocv_pts = np.array([2.80, 3.10, 3.20, 3.25, 3.28, 3.30, 3.32, 3.34, 3.37, 3.45])
ocv = lambda soc: 240 * np.interp(soc, soc_pts, ocv_pts)       # rack OCV, V
A = dict(Q=280 * 0.93, R=0.075 * 1.25, soc=0.95)                # aged rack, year 3
B = dict(Q=280 * 1.00, R=0.075, soc=0.95)                       # new rack
P, dt, E, log = 200e3, 1.0, 0.0, []                             # W, s, Wh
ga, gb = 1 / A["R"], 1 / B["R"]
while True:
    ea, eb = ocv(A["soc"]), ocv(B["soc"])
    s = ea * ga + eb * gb                     # solve ga(ea-V) + gb(eb-V) = P/V for V
    V = (s + np.sqrt(s * s - 4 * (ga + gb) * P)) / (2 * (ga + gb))
    ia, ib = (ea - V) * ga, (eb - V) * gb
    if V / 240 < 2.90 or min(A["soc"], B["soc"]) <= 0:
        break
    log.append((A["soc"], B["soc"], ia, ib))
    A["soc"] -= ia * dt / 3600 / A["Q"]
    B["soc"] -= ib * dt / 3600 / B["Q"]
    E += P * dt / 3600
for label, (sa, sb, ia, ib) in (("start", log[0]), ("end", log[-1])):
    print(f"{label:5s}: I_A = {ia:3.0f} A, I_B = {ib:3.0f} A (B share {ib/(ia+ib):.1%}), "
          f"SoC_A = {sa:.1%}, SoC_B = {sb:.1%}")
k = max(range(len(log)), key=lambda n: log[n][2])           # rack A's peak
sa, sb, ia, ib = log[k]
print(f"peak A: I_A = {ia:3.0f} A (A share {ia/(ia+ib):.1%}) at {k/3600:.2f} h, SoC_A = {sa:.1%}")
share = max(max(ia, ib) / (ia + ib) for _, _, ia, ib in log)    # worst rack, worst moment
print(f"delivered {E/1e3:.0f} kWh in {len(log)/3600:.2f} h; "
      f"max system current for a 140 A rack limit = {140/share:.0f} A")
```

```
 start: I_A = 111 A, I_B = 139 A (B share 55.6%), SoC_A = 95.0%, SoC_B = 95.0%
end  : I_A = 140 A, I_B = 147 A (B share 51.2%), SoC_A = 2.6%, SoC_B = 2.4%
peak A: I_A = 164 A (A share 61.8%) at 1.81 h, SoC_A = 10.0%
delivered 388 kWh in 1.94 h; max system current for a 140 A rack limit = 227 A
```

The racks end the discharge within 0.2 % SoC of each other: a shared bus voltage self-balances parallel racks, because the one that falls behind sits at a lower OCV and takes less current. The design problem is current, not energy, and the share does not stay put. The new rack carries 55.6 % of the current at the start and runs 5.0 SoC points behind by 1.35 h; when it reaches the steep tail of the curve below 10 % first, its OCV drops and the aged rack takes up to 164 A (61.8 %) at 1.81 h, above its own 140 A limit, while the start and end lines of the output both look harmless. If each rack is limited to 140 A (0.5C), the system must be limited to 227 A, not 280 A (or 252 A from the opening share alone) — a 19 % power derating that must be in the system controller's limit calculation, which takes, continuously, the minimum over racks of (rack limit ÷ that rack's measured share). Figure 23.2 plots the two currents through the discharge.

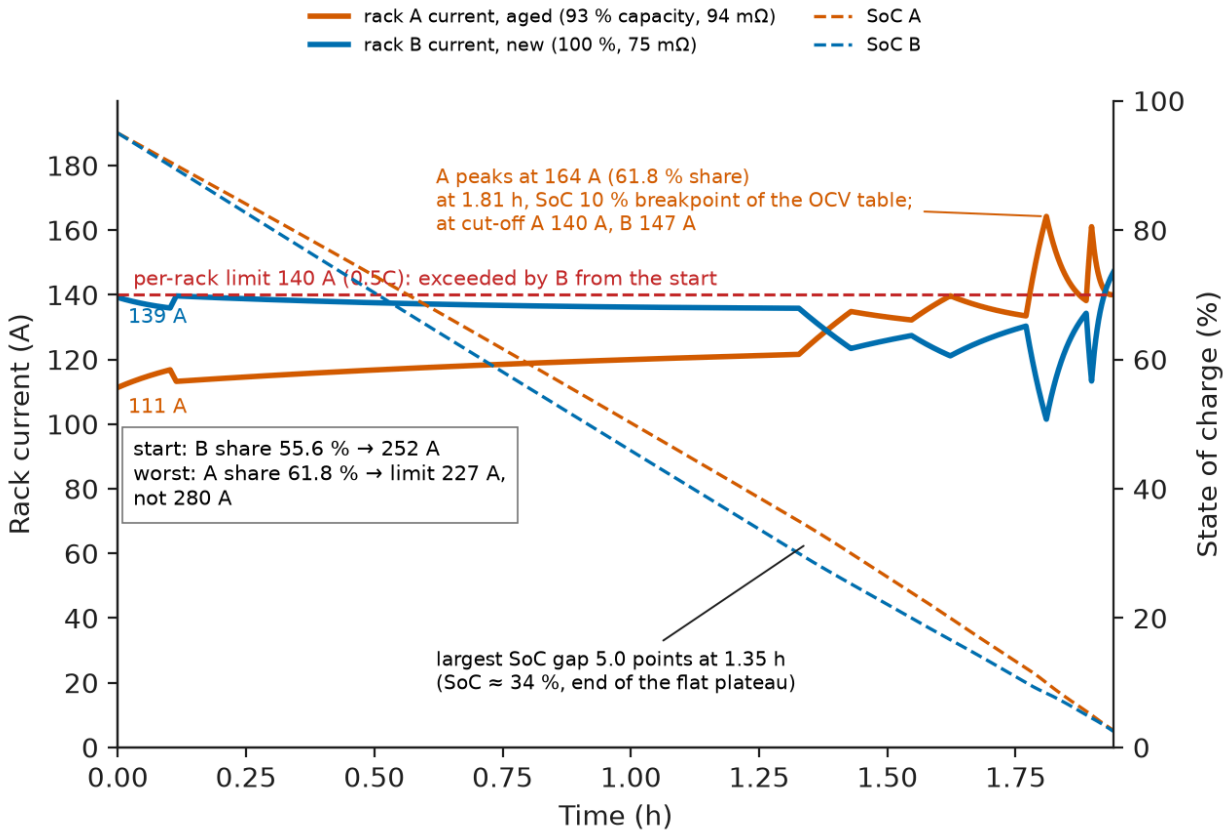


Figure 23.2 — Current sharing between an aged and a new LFP rack in parallel during a 200 kW discharge.

Connecting a rack to a live bus has its own rule: the contactors close only when the rack's voltage is within a few volts of the bus (here 10 V, giving at most $10/0.15 = 67$ A of circulating current through the two 75 mΩ racks), otherwise the PCS first moves the bus voltage or the new rack is precharged and equalised.

23.4 Battery-management systems

The analog front end and its error budget

The AFE decides how well the BMS can see the cells. The 16-channel class used here (BQ79616-, ADBMS1818- or MC33775A-class parts) samples all cells simultaneously with a sigma-delta or SAR converter, filters them, measures NTC thermistors, runs passive balancing, and communicates over an isolated daisy chain (capacitor- or transformer-coupled differential links, often arranged as a ring so that one broken link does not blind the BMU). On an LFP plateau whose slope is 0.5 mV per per cent SoC (*Electrical Energy and Machines* §16.4), millivolts matter.

Table 23.3 — Cell-voltage measurement error budget (R4: ±5 mV).

Source	Value	Note
AFE gain and offset over −10 to 60 °C	±2.5 mV	Datasheet total error, factory-trimmed
Reference drift over 10 years	±1.0 mV	Datasheet lifetime drift
Busbar IR drop in the sense path at 140 A	±0.3 mV	Sense wires welded at cell terminals, not on busbars (2.8 mV otherwise)
Sense-wire drop from balancing current	0 mV	Balancing paused 2 ms before each conversion
Noise after 16-sample averaging	±0.3 mV	—
Worst-case sum	±4.1 mV	Margin 0.9 mV to R4
Root-sum-square	±2.7 mV	Used for SoC uncertainty

The table records two design decisions that a datasheet will not make: sensing on the cell terminal rather than the busbar (a 20 μΩ busbar carrying 140 A adds 2.8 mV, enough on its own to break the budget), and synchronising balancing with conversion, because balancing current through a shared sense wire distorts the reading of two cells.

Balancing

Passive balancing bleeds the high cells through resistors near the top of charge. Its required current is set by the worst-case divergence it must cancel, which is dominated not by manufacturing spread but by **self-discharge differences**, and those follow temperature.

Worked example — Why 50 mA was not enough. LFP self-discharge is about 1.0 % per month at 25 °C and rises with temperature roughly as the ageing rate does, about 1.5 times faster 6 K warmer, so the warm module's cells lost an extra 0.49 % per month, $0.0049 \times 280/30 = 0.046$ Ah per day. The original balancer bled the other 224 cells at 50 mA only while their voltage exceeded 3.40 V, about 30 minutes a day: 0.025 Ah per day. The imbalance grew by 0.021 Ah a day, 8.8 Ah or 3.1 % SoC in 425 days, and the warm cells, already 1.5 % lower in capacity, ended every discharge early. The redesign uses the AFE's 150 mA internal balancing, a balancing window of 30 minutes daily plus a 2-hour constant-voltage hold in a weekly full charge (about 0.12 Ah per day on average), and liquid cooling that cuts the spread to 3 K (Section 23.6), which shrinks the self-discharge difference to about 0.02 Ah per day. Balancing now has a factor of six in hand.

Firmware and the protection core

The BMU runs a 100 ms main loop (measurement, protection, limits), a 1 s loop (state estimation, limits to the PCS, logging) and slower housekeeping, with the protection core developed to IEC 61508 SIL 2 practice (or ISO 26262 ASIL C in vehicles). The listing is the shape of that core: debounced limits on the extreme cell values, a latched trip that requires a service reset, and the rule that data the BMU cannot trust are treated as a fault.

```
 /* bms_protect.c -- cell-level protection with debounce and latching (C11, illustrative) */
#include <stdint.h>
#include <stdbool.h>

#define N_CELLS 240u
typedef enum { PROT_OK = 0, PROT_DERATE, PROT_TRIP } prot_level_t;
typedef struct { int16_t warn, trip; uint16_t trip_ms; } limit_t;  /* mV or 0.1 degC */

static const limit_t OV = { 3600, 3650, 1000 };
static const limit_t UV = { 2900, 2800, 2000 };   /* trips when below */
static const limit_t OT = {  500,  550, 5000 };   /* 0.1 degC */

typedef struct { uint16_t ov_ms, uv_ms, ot_ms; bool latched; } prot_state_t;

/* Every 100 ms with fresh AFE data; afe_ok is false on CRC error, stale data or chain break. */
prot_level_t bms_protect(prot_state_t *st, const uint16_t v_mv[N_CELLS],
                         const int16_t t_dc[], uint8_t n_t, bool afe_ok)
{
    uint16_t vmax = 0u, vmin = UINT16_MAX;
    int16_t tmax = INT16_MIN;
    if (st->latched) return PROT_TRIP;                /* service reset required */
    if (!afe_ok) { st->latched = true; return PROT_TRIP; }  /* unknown is unsafe */
    for (uint16_t k = 0u; k < N_CELLS; ++k) {
        if (v_mv[k] > vmax) vmax = v_mv[k];
        if (v_mv[k] < vmin) vmin = v_mv[k];
    }
    for (uint8_t k = 0u; k < n_t; ++k) if (t_dc[k] > tmax) tmax = t_dc[k];

    st->ov_ms = (vmax >= (uint16_t)OV.trip) ? (uint16_t)(st->ov_ms + 100u) : 0u;
    st->uv_ms = (vmin <= (uint16_t)UV.trip) ? (uint16_t)(st->uv_ms + 100u) : 0u;
    st->ot_ms = (tmax >= OT.trip) ? (uint16_t)(st->ot_ms + 100u) : 0u;
    if (st->ov_ms >= OV.trip_ms || st->uv_ms >= UV.trip_ms || st->ot_ms >= OT.trip_ms) {
        st->latched = true;
        return PROT_TRIP;
    }
    if (vmax >= (uint16_t)OV.warn || vmin <= (uint16_t)UV.warn || tmax >= OT.warn)
        return PROT_DERATE;                           /* limits to PCS go to zero */
    return PROT_OK;
}
```

In the full product an under-voltage trip blocks discharge but permits a supervised recovery charge, and a separate hardware comparator chain in the AFEs (set at 3.75 V and 2.50 V) opens the contactor coil supply without software (Section 23.8). The debounce times are part of the safety argument: the 1.0 s over-voltage debounce plus about 100 ms of contactor opening time meets R7's 1.2 s, and the cell vendor's test data show no damage for brief excursions to 3.75 V lasting several seconds at the charge currents the PCS can deliver.

23.5 State estimation

SoC: budget first, estimator second

The estimator (coulomb counting with OCV correction, or an extended Kalman filter on an equivalent-circuit model, *Electrical Energy and Machines* §16.4) is only as good as its inputs. On LFP the OCV correction is weak through the 30–90 % plateau, so between full charges the estimate is essentially a coulomb count, and its error is the current sensor's error integrated over time. The budget decided the sensor before anyone chose an algorithm.

Table 23.4 — SoC error budget over one week between full charges (280 Ah, one 260 Ah cycle a day; R5: ±5 %).

Source	Hall sensor (original)	Isolated shunt (redesign)
Offset integrated for 168 h	0.15 A → 9.0 %	0.01 A → 0.6 %
Asymmetric gain error on 7 × 260 Ah	0.2 % → 1.3 %	0.05 % → 0.3 %
Capacity (SoH) error at 50 % depth	3 % → 1.5 %	1 % → 0.5 %
Reset accuracy at full charge	1.0 %	1.0 %
Worst-case sum	12.8 % (fails)	2.4 % (margin 2.6 points)

A Hall sensor's offset changes with temperature and with remanence after large currents, so it cannot be calibrated out once; an isolated shunt sensor with a 24-bit converter and an offset below 10 mA (IVT-class, or a shunt with an INA228-class monitor behind an isolator) costs about $120 more and removes the dominant term. Figure 23.3 shows the two designs between weekly resets.

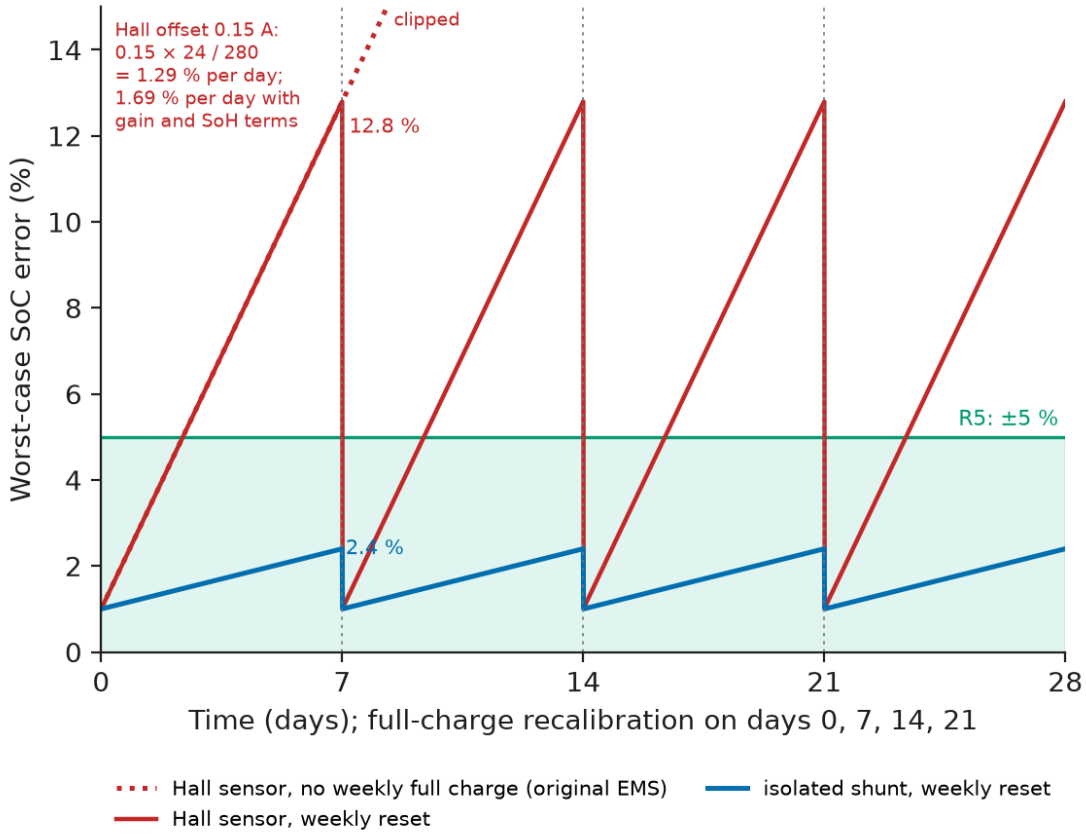


Figure 23.3 — Worst-case SoC error between full-charge recalibrations for the original Hall-sensor design and the redesigned shunt design.

The second design rule is behavioural: the EMS must allow a full charge, with the CV taper that lets the BMS reset SoC to 100 % and balance, at least weekly (R5). The original EMS, optimising tariff arbitrage, had kept the battery between 10 % and 90 % for six weeks at a time.

> **Status of the principle: Coulomb-count drift**
> *Derived.* Integrating a current measurement with a constant offset ΔI gives an SoC error growing as $\Delta I\, t/Q$ and a gain error growing with charge throughput; without an absolute reference these errors are unbounded, which is why every SoC estimator needs a periodic anchor, and why a flat OCV curve makes the anchor rare.

Near the ends, report the weakest cell

A pack SoC computed from the average cell tells the EMS how much charge is in the pack, not how much it can deliver. The redesign reports a blended SoC that follows the average in mid-range and converges to the lowest cell's SoC as the pack approaches empty (and the highest cell's as it approaches full), so that "0 %" coincides with the first cell's cut-off. The 14 % that the original display showed at cut-off was the sum of two errors: about 5 % that the average cool cell still held, and about 9 % of coulomb-count drift since the estimator's last low-SoC OCV correction.

State of power and state of health

State of power is the current the pack can deliver or accept for the next 2, 10 or 30 s without any cell crossing its voltage limit: $I_{\max} = (V_{\mathrm{OCV}} - V_{\min})/R_t$ for the worst cell, with R_t the pulse resistance at that duration, SoC and temperature from a lookup table refreshed by online estimates. It is sent to the PCS every second as the discharge and charge current limits (DCL, CCL). **State of health** is treated in Section 23.9.

23.6 Thermal management

> **Boundary.**
> Thermal-resistance networks are in *Engineering Physics*, Ch. 14 §14.7; the Arrhenius rate law used below in Ch. 20 §20.7.1.

Heat generation and what cooling is really for

At 100 kW and 768 V nominal the rack carries 130 A, 0.47C. Each 0.25 mΩ cell dissipates $130^2 \times 0.25 \times 10^{-3} = 4.2$ W, the rack 1.0 kW plus about 0.25 kW in busbars, fuse and contactors; the reversible (entropic) heat of LFP is small at this rate. A 5.4 kg cell with a heat capacity of about 1.0 kJ/(kg·K) would warm only 5.7 K in a two-hour discharge with no cooling at all. Cooling in a stationary LFP cabinet is therefore less about removing cycling heat than about two other things: holding the cells near 25 °C when the container sits at 45 °C, and keeping every cell at the *same* temperature, because ageing and self-discharge follow temperature exponentially.

Worked example — What 6 K costs. With an activation energy of 50 kJ/mol, the ageing rate at 31 °C relative to 25 °C is $\exp[(50{,}000/8.314)(1/298.15 - 1/304.15)] = 1.49$. The fade model fitted to the cell vendor's data (Section 23.9) gives 13.8 % capacity loss in 10 years at 25 °C, so 200 kWh usable becomes 172 kWh, inside R1's 160 kWh. Multiply the fade by 1.49 for the warm module and the pack, limited by its weakest cells, falls to 159 kWh: a 6 K spread alone fails R1. A 3 K spread (factor 1.22) leaves 166 kWh.

Table 23.5 — Thermal-management options.

Option	Cell-to-cell spread	Max cell at 45 °C ambient	Parasitic power	Cost	Assessment
Forced air, cabinet air conditioner (original)	6–10 K	32 °C	1.5–2.5 kW in summer	$3,000	Spread fails R6 and R1
Forced air with ducted plenum and baffles	4–6 K	31 °C	1.5–2.5 kW	$3,800	Better, still marginal
Liquid cold plates, 50 % glycol, 3 kW chiller	2–3 K	29 °C	0.6–1.2 kW	$5,500	**Chosen**
Immersion in dielectric fluid	≤ 2 K	28 °C	0.5–1 kW	$9,000	Best propagation barrier; cost

Worked example — Sizing the coolant loop. The chiller must remove the cells' 1.0 kW, the switchgear's 0.25 kW and about 1.2 kW of heat leaking into a 45 °C cabinet, 2.5 kW, so a 3 kW unit has 20 % margin. With 15 L/min of 50 % glycol ($\rho \approx 1{,}060$ kg/m³, $c_p \approx 3.4$ kJ/(kg·K), mass flow 0.265 kg/s), the coolant warms by

$1.25/(0.265 \times 3.4) = 1.4$ K across the cell plates at full power. With a cell-to-coolant thermal resistance of 1.5 K/W, each cell runs $4.2 \times 1.5 = 6.3$ K above its local coolant. A 22 °C supply then puts the coolest cell near 28 °C and the hottest near 29.7 °C at full power: a spread under 2 K from the coolant rise, plus manufacturing spread in contact resistance, inside R6's 3 K.

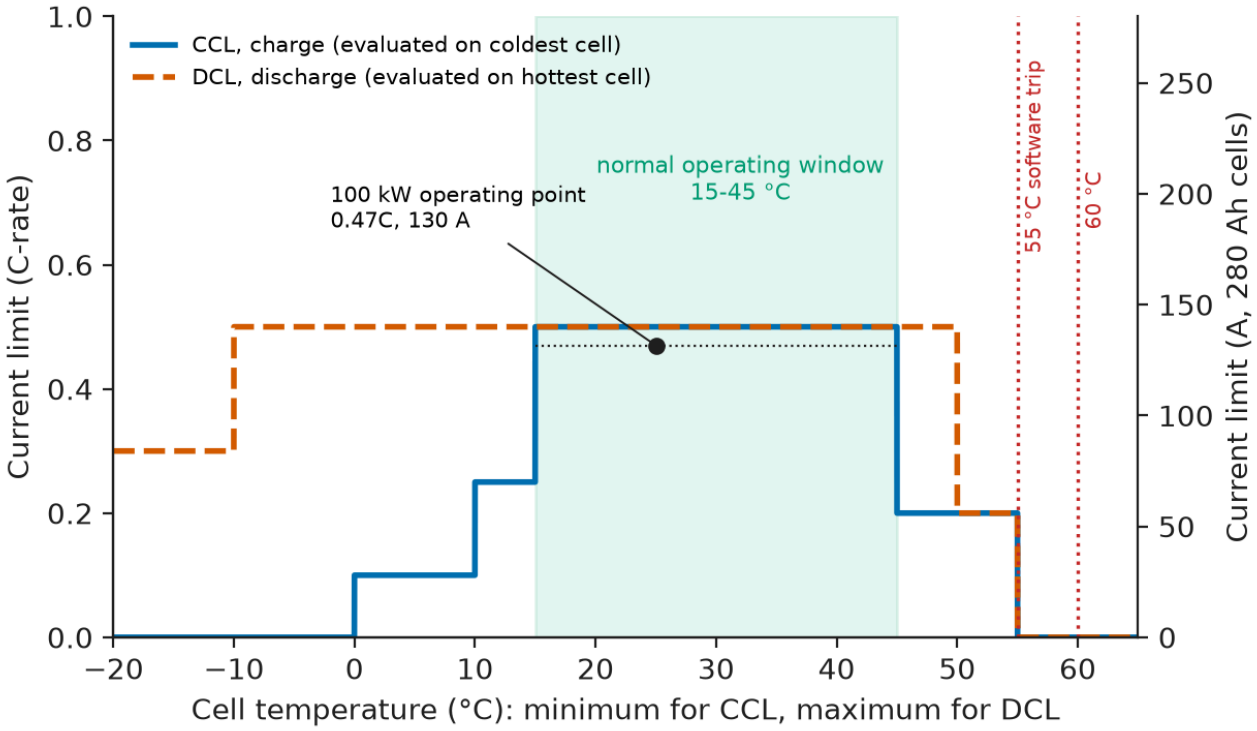


Figure 23.4 — Charge and discharge current limits of the LFP cabinet against cell temperature, as sent by the BMS to the PCS.

23.7 Charging

CC–CV and the handshake

LFP is charged at constant current to a cell-voltage limit and then at constant voltage until the current tapers; the converter that does it is in *Electronics and Computing* §32.7. In a stationary system the PCS is the charger and the BMS is its master: every second the BMS sends the charge-voltage limit (here $240 \times 3.50 = 840$ V for daily cycles, 3.55 V per cell for the weekly full charge), the CCL and DCL (Figure 23.4) and its status, and the PCS must obey the lowest of its own limit, the BMS limit and the EMS request. If the BMS messages stop for 3 s, the PCS stops (R9). The voltage limit is applied to the pack, but the BMS derates the CCL as soon as the *highest cell* passes 3.45 V, because with a 2 % SoC spread one cell can reach the steep end of the curve while the pack average is still on the plateau.

Temperature limits

Charging LFP below 0 °C plates lithium metal on the graphite anode, which consumes capacity and can grow dendrites (*Electrical Energy and Machines* §16.5). The CCL is therefore evaluated on the coldest cell and the DCL on the hottest. On winter mornings a 4 kW heater in the coolant loop warms the cells (about 1,300 kJ/K for 240 cells) from −10 °C to 10 °C, about 7.2 kWh of heat, in roughly two hours before the EMS may

schedule a charge; that is about 4 % of a day's throughput on the coldest days, and keeping the insulated cabinet above 5 °C overnight costs less.

Worked example — The weekly full charge. The weekly charge runs CC at 0.47C to an average of 3.45 V per cell, then CV at 852 V (3.55 V per cell) until the current falls to 0.05C, 14 A. LFP's top-of-charge region is steep, so in the last 2–3 % of SoC the cell voltages spread by tens of millivolts and the balancer can see and correct imbalance; with 150 mA balancing held for two hours, each high cell is trimmed by up to 0.3 Ah (0.11 % SoC), and the BMS resets the SoC of every cell to 100 % when the taper current is reached. The cost is time at high SoC (calendar ageing accelerates there), which is why the full charge is weekly, not daily.

23.8 Protection

> **Boundary.**
> Protection principles and DC fault interruption are in *Electrical Energy and Machines*, Ch. 29 §29.4–29.5; the fact that a battery cannot be isolated internally and its terminals are always live, in §29.6.

Layers

Protection is layered so that no single failure lets a cell leave its safe window:

1. **Limits.** The CCL and DCL keep a healthy system away from every limit.
2. **Software trips.** The BMU opens the contactors on debounced over- or under-voltage, over-temperature, over-current, insulation fault, loss of AFE data or loss of the PCS link (Section 23.4).
3. **Independent hardware.** AFE comparators at 3.75 V and 2.50 V, wired in a daisy-chained fault line that de-energises the contactor coils directly.
4. **Fuse.** A DC fuse clears short circuits that are beyond the contactors' breaking capacity.
5. **Cell and module.** The cell's own vent, and module-level barriers against propagation (Section 23.10).

Table 23.6 — Protection thresholds of the cabinet.

Quantity	Derate	Software trip (debounce)	Hardware	Action
Max cell voltage	3.45 V (CCL ramps down), 3.60 V (CCL = 0)	3.65 V (1.0 s)	3.75 V comparator	Open K1, K2
Min cell voltage	2.95 V (DCL ramps down), 2.90 V (DCL = 0)	2.80 V (2.0 s)	2.50 V comparator	Open; allow supervised recovery charge
Max cell temperature	45 °C (CCL), 50 °C (DCL)	55 °C (5 s)	65 °C thermal cut-out in coil circuit	Open; cooling stays on
Rack current	Limits from SoP	1.1 × limit for 10 s	Fuse 250 A	Open (≤ contactor rating)
Insulation resistance	500 Ω/V alarm (432 kΩ)	100 Ω/V (86 kΩ)	—	Alarm; trip and lock out

Short circuit: who interrupts what

Worked example — Fuse–contactor coordination. A bolted short at the PCS terminals is limited only by the rack's 75 mΩ: $792/0.075 \approx 10.6$ kA at mid-SoC, rising toward 11.5 kA at full charge. The main contactors are rated to break about 2.5 kA at 800 V DC a few times in their life; opening them into 10 kA would weld or destroy them. Coordination therefore says: below about 2 kA (overloads, soft faults), the BMS opens the contactors; above it, the BMS must *not* command an opening and the fuse clears the fault. The fuse is a 250 A, 1,000 V DC battery-protection fuse (IEC 60269-7 gBat class) with a 20 kA breaking capacity; at 10.6 kA its melting I^2t is reached in a few milliseconds, well inside the cells' and busbars' withstand. The BMS's current-sense logic reads the shunt every 1 ms and, above 2 kA, inhibits the contactor command until the current has fallen to zero. The manual service disconnect between modules 8 and 9 splits the pack during maintenance into halves of 410 V and 358 V, so no point inside it sees the full 768 V.

> **Status of the principle: Prospective short-circuit current of a battery**
> *Derived.* A charged battery is a voltage source behind its internal and connection resistance, so the prospective fault current is the open-circuit voltage divided by the loop resistance; it rises as cells are paralleled and falls only slightly with SoC and temperature, which is why the interrupting device is chosen for the full-charge, warm, new-cell case.

Precharge

The PCS DC link holds about 3 mF. Closing the main contactors onto a discharged link would draw a current limited only by the loop resistance, kiloamperes for a millisecond, welding the contactor tips. The precharge contactor K3 closes first through a 100 Ω resistor (Figure 23.5): the inrush is limited to $864/100 = 8.6$ A, the time constant is $100 \times 3 \times 10^{-3} = 0.3$ s, the link reaches 95 % of the rack voltage in 0.9 s and comes within 20 V of it after $0.3 \ln(864/20) = 1.13$ s, and the resistor absorbs $\frac{1}{2}CV^2 = 1.1$ kJ with a peak power of 7.5 kW, so it is a pulse-rated wirewound part. The BMU closes K1 when the link is within 20 V of the rack voltage, opens K3, and declares a fault if that has not happened within 2 s — the signature of a shorted link or a failed PCS capacitor.

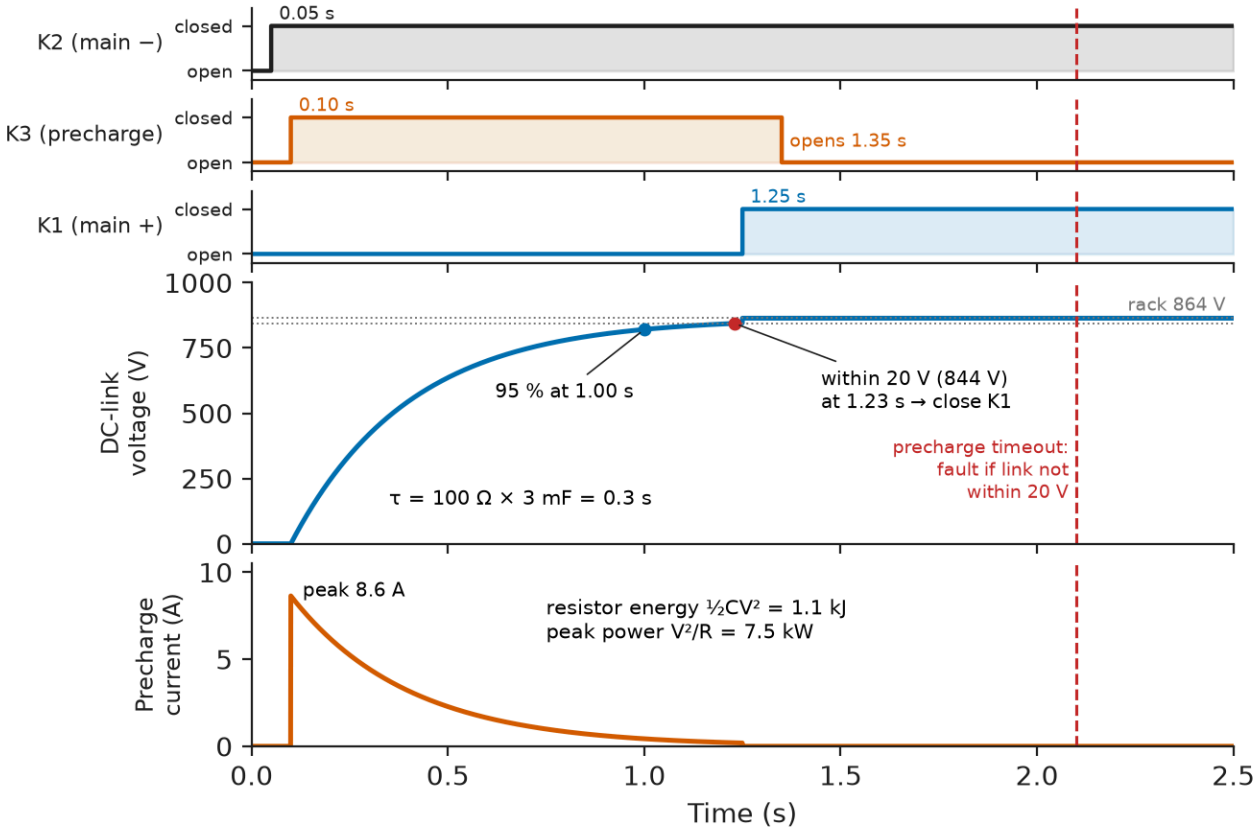


Figure 23.5 — Contactor and precharge sequence when the cabinet connects to the PCS DC link.

23.9 Degradation monitoring

Measuring capacity and resistance in service

A full capacity test interrupts service, so the BMS estimates SoH from partial data: charge counted between two well-defined anchors (the full-charge reset and a rest-OCV reading below 15 % SoC, where the LFP curve is steep again), and pulse resistance from the voltage step at each large current change, normalised to 25 °C and 50 % SoC. Per-cell trends matter more than pack averages: in the opening's cabinet, a per-module capacity estimate would have shown the warm module falling away from the others by the fourth month.

Projecting fade against the warranty

The cell vendor's ageing data fit a calendar term proportional to $\sqrt{t}$ and a cycle term proportional to equivalent full cycles (EFC), both multiplied by the Arrhenius factor k_T of the cell's temperature:

$$\Delta Q(\%) = k_T \left(a\sqrt{t} + b\, N_{\mathrm{EFC}} \right), \qquad a = 2.0\ \%/\sqrt{\mathrm{yr}},\ b = 0.00204\ \%/\mathrm{EFC}\ (25\ ^\circ\mathrm{C})$$

which reproduces the vendor's 80 % at 6,000 cycles over 15 years at 25 °C.

> **Status of the principle: Square-root calendar plus linear cycle fade**
> *Empirical.* A two-term fit to accelerated tests of one cell design; it misses the late-life knee at which fade accelerates, and the Arrhenius factor is fitted over a limited temperature range, so the projection is a monitoring baseline that measured SoH must keep confirming.

Worked example — The opening's numbers, and year 10. After 14 months and about 400 EFC, the model gives $2.0\sqrt{1.17} + 0.00204 \times 400 = 3.0$ % fade at 25 °C and $1.49 \times 3.0 = 4.4$ % for the warm module, matching the field estimates of 3.0 % and 4.5 %. Add the 3.1 % SoC offset of Section 23.4 and the warm cells reached empty about 4.6 % of capacity before the cool cells would have, which with resistance growth explains the 181 kWh against an expected 194 kWh. At year 10 and 3,650 EFC the model gives 13.8 % fade (172 kWh usable) for a uniform 25 °C pack, 159 kWh with the original 6 K hot spot, and 166 kWh with the redesign's 3 K spread. The BMS compares measured per-module SoH monthly with this projection and raises a maintenance alarm when any module falls more than 2 points below it (Figure 23.6).

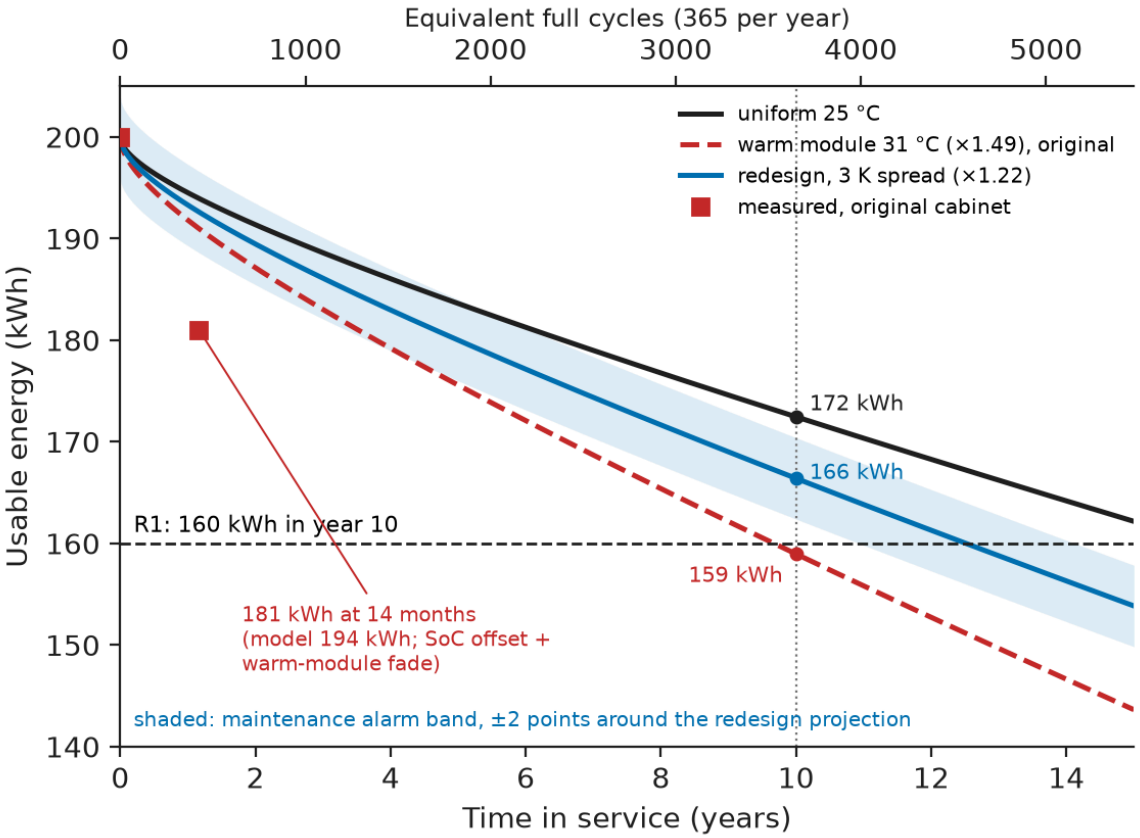


Figure 23.6 — Projected and measured capacity fade of the cabinet's modules against the year-10 requirement.

23.10 Safety testing and verification

The standards and what they test

Safety evidence is built from the cell up. **UN 38.3** is the precondition for shipping any lithium cell or battery: altitude, thermal cycling, vibration, shock, external short circuit, impact or crush, overcharge and forced discharge. **IEC 62619** (industrial lithium cells and batteries) adds safety tests of the cell and battery —

external short, impact, drop, thermal abuse, overcharge, forced discharge, and an internal-short propagation test at battery level — and requires the BMS's protective functions to be assessed for functional safety. **UL 1973** covers stationary batteries in North America, **IEC 62933-5-2** the safety of the whole energy-storage system. **UL 9540A** is not a pass/fail standard but a test method: it forces thermal runaway in one cell, then one cell in a module, then a module in the unit, and measures gas composition, heat release and whether runaway propagates; installation codes (NFPA 855, the IFC) use its results to set spacing, ventilation and deflagration venting. The cabinet's modules passed UL 9540A at module level with no propagation, owing to 2 mm aerogel sheets between cells and a mica barrier above the vents.

Testing the BMS without a battery

The BMS's protection functions are verified on a hardware-in-the-loop rig before they ever connect to cells: 240 channels of programmable cell simulators (each sourcing and sinking current at a commanded voltage), resistor-decade NTC simulators, a current-sensor simulator and a PCS model on CAN. The rig injects every fault in the FMEA — a cell at 3.66 V, a broken sense wire, a reversed thermistor, a stuck AFE, a daisy-chain break, a corrupted CAN message, a 50 kΩ insulation fault — and records the BMS's reaction and timing. Only then is the BMS connected to real modules for the abuse and performance tests.

Table 23.7 — Verification of the redesigned cabinet.

Req.	Test	Limit	Result	Pass/fail
R1	Capacity test at 0.5C, 25 °C, AC side	≥ 200 kWh	203.6 kWh	Pass
R2	100 kW charge and discharge in 45 °C climatic room	No derating	Max cell 29.5 °C; no derating	Pass
R3	Logged rack voltage over 30 days	600–950 V	701–858 V	Pass
R4	AFE vs 6.5-digit DMM, 32 cells, −10/25/60 °C	≤ ±5 mV	Worst −2.9 mV at 60 °C	Pass
R5	SoC vs reference coulometer, 30 days, weekly full charge	±5 %	Max error 2.1 %	Pass
R6	40 thermocouples, 24 h of cycling at 45 °C ambient	≤ 40 °C; spread ≤ 3 K	29.5 °C; 2.4 K	Pass
R7	HIL: cell at 3.66 V; hardware chain with software halted	Open ≤ 1.2 s; hardware opens alone	1.13 s; hardware 42 ms after 3.75 V	Pass
R8	IMD with 400 kΩ and 80 kΩ from each pole to chassis	Alarm at 500 Ω/V; trip at 100 Ω/V	Alarm at 400 kΩ; trip at 80 kΩ	Pass
R9	Pull BMS–PCS CAN cable at 100 kW	PCS stops ≤ 3 s	2.2 s	Pass
R10	Annual capacity test vs BMS SoH	±3 %	+1.1 %	Pass
R11	Certificates and test reports	IEC 62619, UL 9540A module, UN 38.3	On file	Pass

23.11 Failure analysis

Table 23.8 — FMEA of the battery cabinet.

Failure mode	Effect	Cause	Detection	Mitigation
Module-level capacity loss and SoC offset	Usable energy below contract	Temperature spread; balancing too weak	Per-module SoH and SoC spread trend	Liquid cooling (≤ 3 K); 150 mA balancing; weekly full charge
SoC drift on the LFP plateau	Unexpected stop at reported 10–20 %	Hall offset; no full charge for weeks	Reference test; SoC jump at full charge	Isolated shunt; EMS forced weekly full charge; min-cell blending
Sense wire broken	Cell unmonitored; possible overcharge	Vibration, poor crimp	AFE open-wire diagnostic	Treat as fault (trip); welded tabs; strain relief
AFE daisy-chain break	Loss of all data beyond break	Connector, isolation barrier damage	Ring topology reports break location	Ring chain; trip if any cell unseen > 1 s
Contactor welded	Cannot isolate rack	Opening under fault current; precharge failure	Weld check (voltage across open contactor) at every open	Fuse coordination; precharge timeout; two contactors in series path
Lithium plating	Capacity loss; internal short risk	Charging below 0 °C or above CCL	Coldest-cell CCL; capacity trend	Heater pre-conditioning; CCL on coldest cell
Insulation fault	Shock hazard; second fault becomes short	Coolant leak, condensation, damaged cable	IMD	Alarm and trip thresholds; leak sensor in drip tray
Thermal runaway in one cell	Fire, toxic and flammable gas	Internal short, manufacturing defect	Cell voltage drop, temperature rate, gas sensor	Aerogel barriers; UL 9540A no propagation; deflagration vent; fire-service interface
Loss of BMS–PCS link	Uncontrolled charge or discharge	Cable, software crash	1 s heartbeat	PCS stops in 3 s; hardware layer independent

The redesign that answered the opening changed five things, each traceable to a row: liquid cooling with a 3 K spread requirement, a balancer sized on self-discharge spread rather than manufacturing spread, an isolated shunt chosen from an SoC error budget, an EMS rule that forces a weekly full charge, and an SoC report that converges on the weakest cell. None of the five needed a new cell. They needed the requirements of Table 23.1 to be written for the system, and its budgets — measurement, SoC, thermal, current sharing — to be computed before the parts were bought.

What remains open

Open question	Where it is answered
How is a traction battery integrated with a vehicle's inverter, DC–DC converter, charger and thermal system?	Chapter 24
How does an energy-management system dispatch the battery together with PV, wind and the grid, islanded and grid-connected?	Chapter 25
How is a small solar battery system sized, protected and costed end to end?	Chapter 36
How is a battery-monitoring instrument built and calibrated?	Chapter 9
How is a battery digital twin calibrated and used to forecast degradation?	Chapter 33
How is the bidirectional converter that charges the battery built?	*Electronics and Computing*, Ch. 32
What happens inside the electrodes as the cell ages?	*Materials Science and Engineering*, Ch. 40 §40.9

Next: Chapter 24 puts a battery on wheels and integrates it with the traction motor, inverter, chargers and thermal system of an electric vehicle.

Chapter 24 — Electric Vehicles

Part VI · Renewable Energy and Electric Mobility

A parcel company puts twenty battery-electric delivery vans into service in October. Each van has a 3.5 t gross mass, a 76.8 kWh lithium-iron-phosphate pack and a type-approval range of more than 300 km, and the daily routes are 150 km, so the fleet manager expects to charge them overnight and never think about range again. In the first cold week of January, at −10 °C, two vans stop on their routes at about 139 km, and a third, sent to a public DC fast charger, charges at 22 kW instead of the 115 kW on the data sheet and takes almost two and a half hours. Nothing is broken. The energy budget was written per kilometre, but the heater draws power per hour and a delivery van spends half its shift parked with the doors opening; the cold pack accepted almost no regeneration or charge current, and nobody had planned to warm it. This chapter follows that van from requirements to a second iteration: architecture, energy consumption, traction motor, inverter, DC–DC converter, pack, regeneration, thermal limits, charging and the depot that feeds the fleet.

> **How is an electric vehicle's powertrain sized from a duty cycle, and which component limits each requirement: range, gradeability, acceleration, charging time?**
> **What must the vehicle, the charger and the depot agree on, electrically and thermally, so that the energy budget holds in winter and the fleet is recharged inside the site's power limit?**

Concept	Engineering destination
EV architecture	HV and LV networks, contactors, interlocks, control units and their CAN interfaces
Traction motors	Motor and gear-ratio choice from gradeability, top speed and acceleration
Inverters	Switch technology, current rating, DC-link voltage window, junction-temperature margin
DC–DC converters	14 V supply for lights, steering, brakes and controllers; HV draw and redundancy
Battery packs	Voltage window, power capability at low temperature, isolation and pre-charge
Charging systems	On-board AC charger, DC fast charging, control pilot and ISO 15118 communication
Regenerative braking	Blending with friction brakes; battery acceptance as the real limit
Vehicle energy consumption	Drive-cycle model, kWh/100 km, range and reserve
Thermal constraints	Coolant loops, heat pump versus resistive heating, pack preconditioning
Charging infrastructure	Depot sizing, dynamic load management, standards and connectors

Roadmap. Requirements and architecture (24.1) → the energy budget from a drive cycle (24.2) → motor and gear ratio (24.3) → inverter (24.4) → DC–DC converter (24.5) → battery pack as a vehicle component (24.6) → regenerative braking (24.7) → thermal limits (24.8) → on-board and off-board charging (24.9) → the depot (24.10) → verification, failure analysis and Revision B (24.11).

24.1 EV architecture

Requirements

The reference system for the whole chapter is a 3.5 t rear-wheel-drive urban delivery van. Its requirements come from the operator's routes (150 km per shift, 9 h per shift, about 100 stops), from the depot (20 vans, one shared grid connection) and from regulation. Table 24.1 lists them with the verification method fixed before any part was chosen (the requirement-writing method is Chapter 1's).

Table 24.1 — Requirements for the reference delivery van.

ID	Requirement	Target	Verification
R1	Shift range at −10 °C, full shift including parked heating	150 km with ≥ 10 % of usable energy left	Cold chamber chassis-dyno test, urban loop plus timed dwell
R2	Range at 20 °C on the urban loop	≥ 300 km	Chassis dyno
R3	Gradeability at 3,500 kg	Start and climb 25 %	Ramp test, motor torque log
R4	Top speed, continuous	120 km/h	Test track, 20 min hold, temperatures logged
R5	Acceleration at 3,000 kg	0–50 km/h ≤ 6.0 s	Track, GNSS speed logger
R6	AC charging	11 kW three-phase; 20–100 % in ≤ 6.5 h	Depot EVSE, energy meter
R7	DC charging	10–80 % in ≤ 40 min with pack at 25 °C	CCS2 charger, charger log
R8	14 V supply	2.0 kW continuous	Electronic load on 14 V bus, thermal soak
R9	HV safety	Isolation ≥ 100 Ω/V; HV interlock; active discharge to ≤ 60 V in ≤ 3 s	Isolation tester, HVIL break test, discharge scope capture
R10	Depot	20 vans full by 06:30 with site import ≤ 280 kW	Depot simulation, then site meter log

R9 follows ISO 6469-3 and UN ECE R100 (isolation and protection against electric shock); the functional-safety goals (no unintended acceleration, no unintended loss of braking torque) are classified under ISO 26262 and typically land at ASIL C or D for the torque path.

Architecture and interfaces

Figure 24.1 shows the architecture. Energy flows from the pack through the main contactors and a fuse to the HV bus, which feeds the traction inverter, the DC–DC converter, the electric air-conditioning compressor and the heater; the on-board charger and the DC inlet feed the same bus in the other direction. Control flows over two CAN networks: a powertrain bus at 500 kbit/s (vehicle control unit, inverter, BMS, DC–DC, charger) and a body bus. The vehicle control unit (VCU) owns the torque request: it reads the pedals, asks the BMS for its live charge and discharge power limits every 10 ms, asks the brake controller how much deceleration it wants, and sends the inverter a torque command inside all those limits.

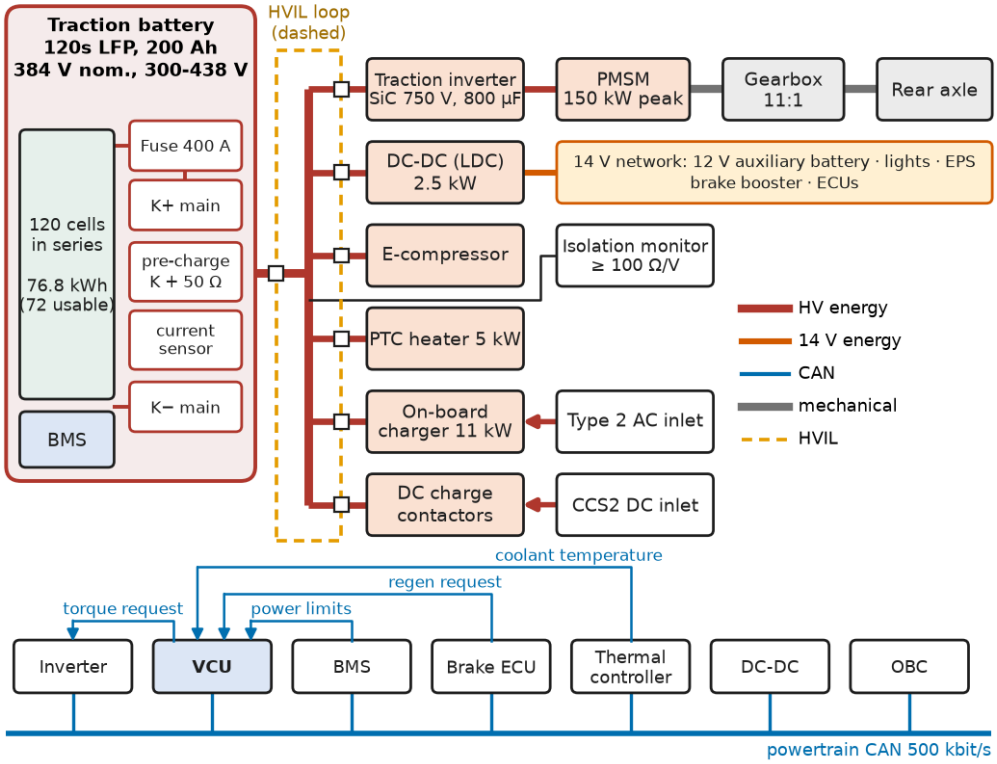


Figure 24.1 — Architecture of the reference 3.5 t delivery van: HV energy paths, 14 V network and powertrain CAN.

Table 24.2 lists the interfaces that cause most integration faults. Each carries a number on both sides, and the test in Section 24.11 checks both sides.

Table 24.2 — Key interfaces of the reference van.

Interface	From → to	What crosses	Number that must match
HV DC bus	Pack → inverter, DC–DC, OBC, heater	300–438 V, up to 520 A	Every load must operate across the full window, including cold sag
Power limits	BMS → VCU	Charge and discharge power limits, 10 ms	VCU must clamp regen and drive torque within 20 ms of a change
Torque command	VCU → inverter	Torque request, mode, enable	Message timeout 30 ms; inverter goes to zero torque on timeout
Brake blending	Brake ECU ↔ VCU	Requested deceleration, delivered regen torque	Sum must equal driver demand within 0.05 g
HVIL	All HV connectors	10 mA loop current	Contactors open within 100 ms of loop break
Charging	EVSE ↔ OBC/VCU	Control pilot, proximity, ISO 15118 PLC	Pilot duty → current per IEC 61851-1
Coolant	Thermal module → inverter, motor, pack	Flow, inlet temperature	Inverter inlet ≤ 65 °C at 10 L/min

Boundary.
Machine theory (PMSM equivalent circuit, field weakening) is in *Electrical Energy and Machines*, Ch. 8 §8.1 and Ch. 12 §12.8; converter topologies are in *Electronics and Computing*, Ch. 30–32; cell physics and BMS design are in *Electrical Energy and Machines*, Ch. 16 and this book's Chapter 23. This chapter chooses, sizes, connects and tests them in one vehicle.

Architecture choices

Three decisions are made at architecture level and fix much of what follows:

- **400 V rather than 800 V class.** 800 V halves current and makes charging above about 200 kW practical, but every HV component must then be rated 1,200 V. A van whose LFP cells cap DC charging at about 115 kW gains little.
- **One rear motor.** It uses the load transfer onto the rear axle when loaded and climbing (R3), but puts all regeneration on one axle (Section 24.7).
- **Single-speed reduction.** The PMSM's wide constant-power region covers 0–120 km/h; the ratio is chosen in Section 24.3.

24.2 Vehicle energy consumption

The road-load equation

The force at the wheels is Newton's second law with three resistances: rolling, aerodynamic and grade,

$$F_w = k_m m \frac{dv}{dt} + mg\,(C_{rr}\cos\theta + \sin\theta) + \tfrac{1}{2}\rho C_d A\, v^2,$$

where $k_m \approx 1.05$ accounts for rotating inertia. The van's parameters are $m = 3{,}000$ kg (test mass, half payload), $C_{rr} = 0.009$ on warm tyres, $C_d A = 0.36 \times 3.6 = 1.30\ \mathrm{m^2}$, $\rho = 1.20\ \mathrm{kg/m^3}$ at 20 °C. At steady speed the road-load power is 5.8 kW at 50 km/h, 14.4 kW at 80 km/h and 37.6 kW at 120 km/h. At urban speeds the inertial term dominates, which is why regeneration matters so much for this van.

Status of the principle: Road-load equation
Fundamental. It is Newton's second law for the vehicle as a point mass (*Engineering Physics*, Ch. 19 §19.1); the coefficients C_{rr} and $C_d A$ are empirical, measured by coast-down, and C_{rr} rises 15–30 % on cold tyres.

A drive-cycle model

Type-approval cycles (WLTP) give comparable numbers, not this operator's. GNSS logs from its diesel vans reduce the route to an urban loop of micro-trips: eight to 40 km/h with 30 s stops, three to 60 km/h, one to 90 km/h, accelerating at 1.0 m/s² and braking at 1.2 m/s². The listing turns the loop into battery energy with a constant drivetrain efficiency $\eta_d = 0.88$ (inverter, motor, gearbox); a measured map replaces it later.

```
 import numpy as np

def cycle(dt=0.1):
    """Synthetic urban delivery loop: (target km/h, cruise m, dwell s)."""
    legs = [(40, 300, 30)]*8 + [(60, 1000, 20)]*3 + [(90, 3000, 10)]
    v, out = 0.0, []
    for vk, cruise, dwell in legs:
        vt = vk / 3.6
        while v < vt:  v = min(vt, v + 1.0*dt); out.append(v)   # 1.0 m/s^2
        for _ in range(int(cruise / vt / dt)): out.append(vt)
        while v > 0:   v = max(0.0, v - 1.2*dt); out.append(v)  # 1.2 m/s^2
        out += [0.0] * int(dwell / dt)
    return np.array(out)

def battery_energy(v, dt=0.1, m=3000.0, crr=0.009, CdA=1.296, rho=1.20,
                   eta_d=0.88, p_chg_max=100e3, p_aux=0.8e3):
    a = np.gradient(v, dt)
    p_wheel = (1.05*m*a + m*9.81*crr + 0.5*rho*CdA*v**2) * v
    p_drive = np.where(p_wheel > 0, p_wheel/eta_d, p_wheel*eta_d)
    regen_ok = v > 5/3.6                                    # regen fades below 5 km/h
    p_drive = np.where(p_drive < 0, np.where(regen_ok,
                       np.maximum(p_drive, -p_chg_max), 0.0), p_drive)
    p_batt = p_drive + p_aux
    e = lambda p: p.sum()*dt/3.6e6                           # kWh
    return e(np.clip(p_wheel, 0, None)), e(np.clip(-p_wheel, 0, None)), \
           e(np.clip(-p_drive, 0, None)), e(p_batt)

v = cycle(); dist = v.sum()*0.1/1000
print(f"loop: {dist:.2f} km, {len(v)*0.1/60:.1f} min, mean {dist/(len(v)*0.1/3600):.1f} km/h")
for name, kw in [("warm 20 C", {}),
                 ("cold -10 C", dict(crr=0.0104, rho=1.34, p_chg_max=8e3, p_aux=5.3e3))]:
    ep, en, er, eb = battery_energy(v, **kw)
    print(f"{name}: wheel+ {ep:.3f} kWh, wheel- {en:.3f} kWh, regen {er:.3f} kWh,"
          f" battery {eb:.3f} kWh = {100*eb/dist:.1f} kWh/100 km, range {72/(eb/dist):.0f} km")
```

```
 loop: 10.65 km, 18.8 min, mean 34.0 km/h
warm 20 C: wheel+ 2.467 kWh, wheel- 0.961 kWh, regen 0.838 kWh, battery 2.216 kWh = 20.8 kWh/100
km, range 346 km
cold -10 C: wheel+ 2.657 kWh, wheel- 0.945 kWh, regen 0.268 kWh, battery 4.411 kWh = 41.4 kWh/100
km, range 174 km
```

The cold case changes four inputs, each with a physical cause: rolling resistance +15 % (stiff cold tyres), air density 1.34 kg/m³, a 5.3 kW auxiliary load (0.8 kW base plus a 4.5 kW PTC heater), and a charge-power limit of 8 kW because a cold LFP pack cannot accept current without lithium plating. As in the VCU of Section 24.7, it limits the drive's regenerated power, so with the heater on the pack charges at 2.7 kW at most. Figure 24.2 shows the loop and the battery power in both cases.

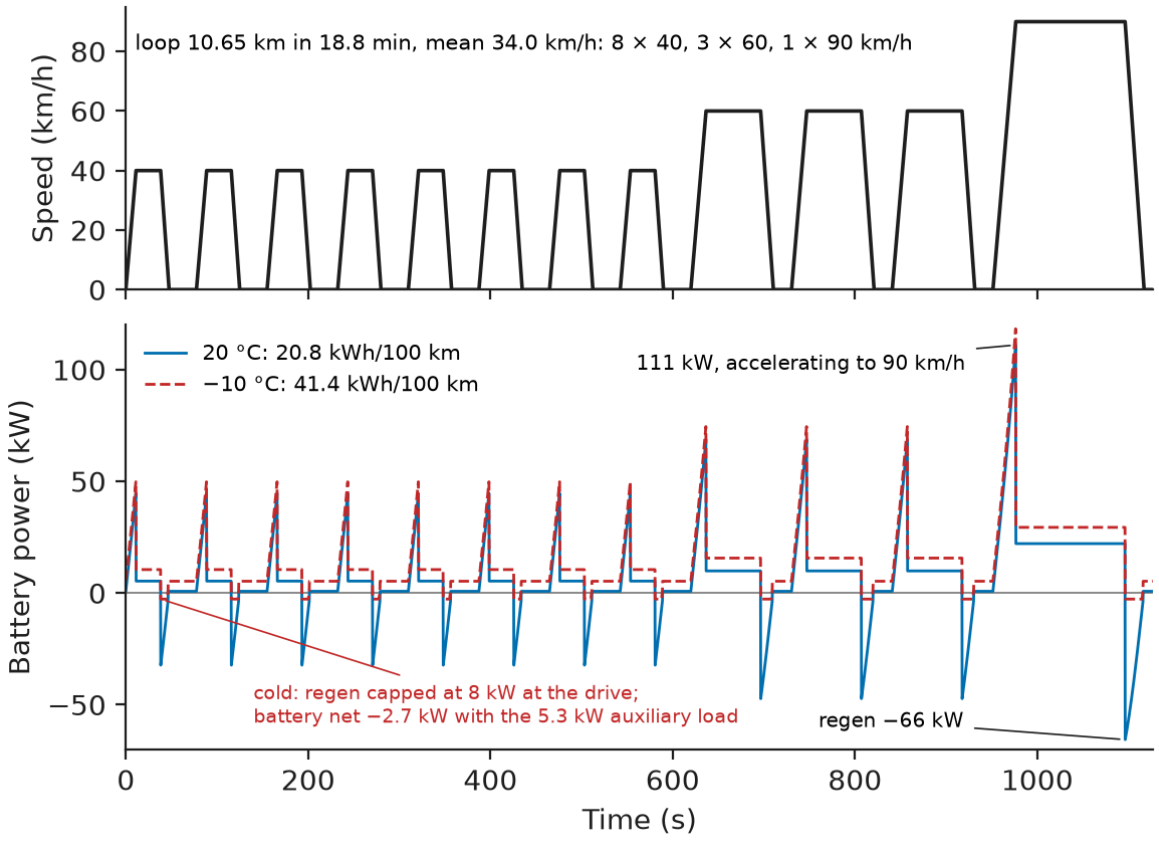


Figure 24.2 — Urban delivery loop and battery power at 20 °C and at −10 °C.

Worked example — the energy budget for one shift. Scale the loop to 150 km (a factor of 150/10.65 = 14.1). Table 24.3 splits the battery energy into traction, regeneration credit and auxiliaries. Warm, the van uses 31.2 kWh of its 72 kWh usable, a 57 % reserve. Cold, the drive-cycle number is 62.1 kWh and the reserve 13.7 %, which passes R1 on paper. But the loop takes 18.8 min per 10.65 km, so 150 km is 4.4 h of driving in a 9 h shift; for 4.5 h of the remaining 4.6 h the van is parked at drops with the heater holding the cab warm against open doors, averaging 3.5 kW. That adds 4.5 × 3.5 = 15.8 kWh, the shift needs 77.9 kWh, and at 77.9/150 = 0.519 kWh/km the 72 kWh pack is empty at 72/0.519 = 139 km — where the two vans stopped.

Table 24.3 — Energy budget for a 150 km shift (kWh, 72 kWh usable).

Item	20 °C	−10 °C, PTC heater, cold pack	−10 °C, Revision B (heat pump, preconditioned pack)
Traction, E_{wheel+}/η_d	39.5	42.5	42.5
Regeneration credit	−11.8	−3.8	−10.1
Auxiliaries and HVAC while driving	3.5	23.4	13.5
Drive-cycle total	**31.2**	**62.1**	**45.9**
Parked heating, 4.5 h	0.0	15.8 (3.5 kW)	6.8 (1.5 kW)
Shift total	**31.2**	**77.9**	**52.6**
Margin to 72 kWh usable	40.8 (57 %)	−5.9 (fails R1)	19.4 (27 %)

> **Status of the principle: Battery energy from a drive cycle with constant drivetrain efficiency**
> *Approximate.* Energy conservation is exact; the constant η_d hides the low efficiency of a PMSM at light load and high speed, so the model is within about 5 % on urban cycles and optimistic on motorway cycles. A measured efficiency map replaces it for the final budget.

Loads that scale with time must be budgeted against time, not distance (the same mistake kills IoT nodes whose radio listens per hour, Chapter 11), and in the cold the heater, not the motor, decides the range.

24.3 Traction motors

Sizing from the requirements

R3 sets the peak wheel force, R4 the maximum motor speed and R5 the peak power.

Worked example — gear ratio and motor rating. Climbing 25 % ($\theta = 14.0^\circ$) at 3,500 kg needs $F = mg(\sin\theta + C_{rr}\cos\theta) = 8{,}627$ N at the wheels. A candidate PMSM with 310 N·m peak torque, 150 kW peak power and 12,000 rpm maximum speed, through a gearbox of efficiency 0.97 and a 0.36 m tyre radius, gives a wheel force of $310 \times G \times 0.97/0.36$. With $G = 10.5$ that is 8,770 N, a margin of 1.7 %, which is no margin once tyre radius tolerance and a hot motor are included. With $G = 11.0$ it is 9,188 N, a 6.5 % margin. At 120 km/h the wheel turns at 884 rpm and the motor at $884 \times 11 = 9{,}726$ rpm, 19 % below its 12,000 rpm limit. The base speed (where 310 N·m meets 150 kW) is 4,621 rpm, or 57 km/h of road speed. Integrating the torque–power envelope against the road load gives 0–50 km/h in 5.2 s at 3,000 kg (R5 passes with 0.8 s margin) and 0–100 km/h in 12.2 s. Continuous rating: 120 km/h needs 37.6 kW at the wheels, 38.8 kW at the shaft, against a 75 kW continuous rating.

Figure 24.3 plots the envelope against the operating points that set it.

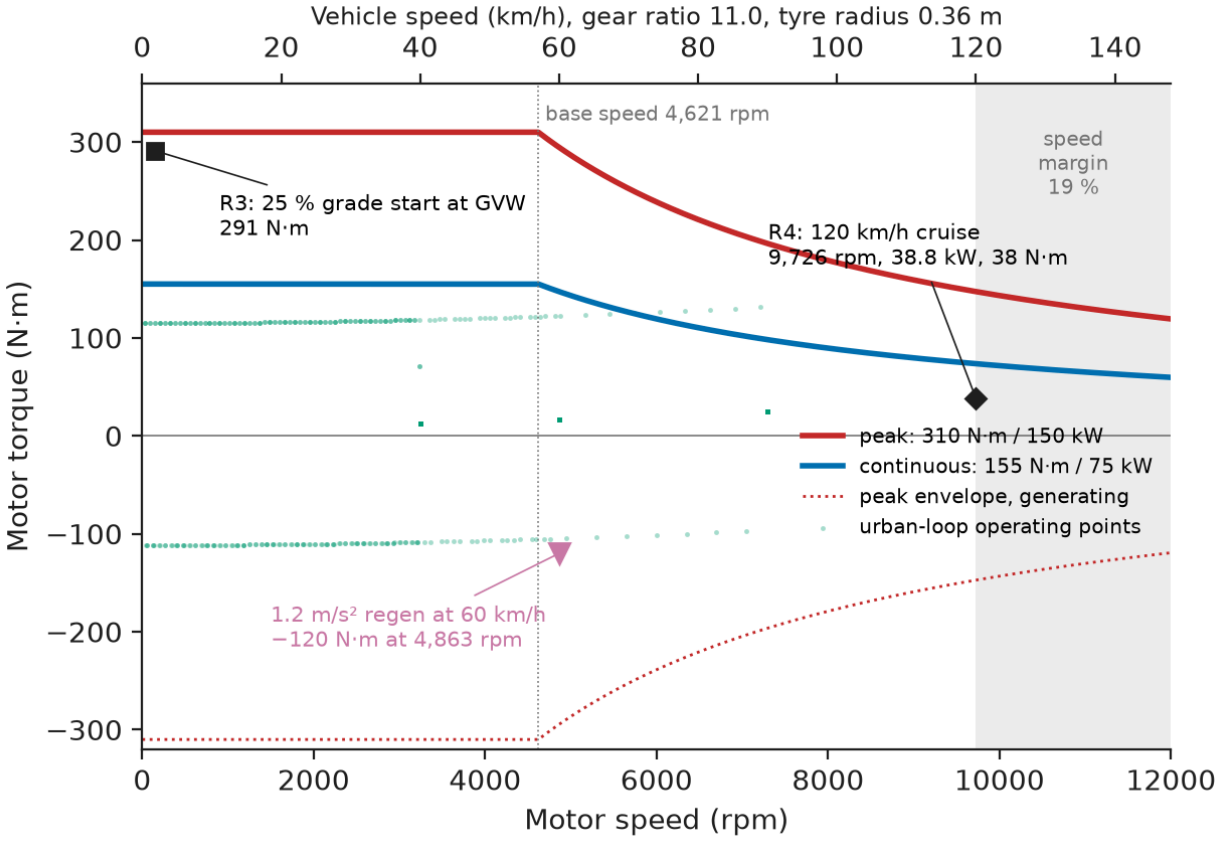


Figure 24.3 — Traction-motor torque–speed envelope with the operating points that size it (gear ratio 11.0).

Motor technology

Table 24.4 — Traction-motor options for the van (150 kW peak class).

Option	Peak efficiency	Urban-cycle efficiency	Power density	Cost and supply	Fault behaviour	Choice
Interior PMSM (NdFeB)	96–97 %	90–93 %	High (4–6 kW/kg)	Rare-earth magnet price and supply risk	Back-EMF at high speed if inverter trips: uncontrolled generation	**Chosen**
Induction motor (copper rotor)	94–95 %	86–90 %	Medium	No magnets	De-excites when inverter stops; benign	Rejected: rotor losses cost about 3 % range in urban duty
Externally excited synchronous (EESM)	95–96 %	90–92 %	Medium	No magnets, needs rotor slip rings or wireless excitation	Field removable: benign	Considered for Revision B if magnet price doubles

The PMSM's weakness is its magnets: the supplier's back-EMF constant gives a line-to-line peak of about 520 V at 12,000 rpm, above the 438 V pack maximum, so an inverter that opens all switches at high speed lets the machine rectify into the bus. The inverter's safe state at high speed must therefore be an **active short circuit** (all low-side switches on; *Electrical Energy and Machines*, Ch. 12 §12.8).

> **Status of the principle: Gear ratio from gradeability and maximum speed**
> *Derived.* The ratio window follows from the wheel-force and speed limits of the motor; it assumes the motor reaches peak torque at stall for the 5–10 s of a hill start, which must be confirmed against the motor's thermal limit (Section 24.8).

24.4 Inverters

Rating

The inverter must deliver 150 kW at the lowest bus voltage it will ever see at full power. Section 24.6 shows that a cold, partly discharged pack sags to about 332 V at 515 A; the inverter is specified for full power down to 330 V and derates linearly below. With space-vector modulation the maximum fundamental line voltage is $V_{dc}/\sqrt{2} = 233$ V rms at 330 V (*Electronics and Computing*, Ch. 31 §31.3). With a motor power factor of 0.92 and efficiency 0.95 at that point, the phase current is $150{,}000/(\sqrt{3} \times 233 \times 0.92 \times 0.95) = 425$ A rms, 601 A peak.

Table 24.5 — Power-switch options for a 400 V class, 150 kW traction inverter.

Option	Blocking voltage	Rating class	Inverter loss, urban cycle	Relative cost	Choice
Si IGBT six-pack (HybridPACK Drive class)	750 V	820 A	Base (efficiency about 96 % cycle average)	1.0	Rejected for Revision A cost reasons; see text
SiC MOSFET six-pack	750 V	700–800 A peak	40–60 % lower; cycle average about 98 %	1.6–2.0	**Chosen**
SiC MOSFET, 1,200 V	1,200 V	400–600 A	Similar	2.2	Only for 800 V class

At the light loads of an urban cycle an IGBT's switching and knee-voltage losses dominate, where a SiC MOSFET's resistive conduction is lowest. Running the drive-cycle model with $\eta_d = 0.90$ instead of 0.88 lowers consumption from 20.8 to 20.0 kWh/100 km at 20 °C: 3.7 % less energy, which is either 13 km more range or a 2.8 kWh smaller pack for the same range. At a 2026 pack price of about $110 per kWh and a SiC module premium of about $250, the smaller pack pays for the module. The inverter switches at 10 kHz; the 750 V rating leaves 312 V above the 438 V maximum for overshoot from commutation loop inductance, which the gate driver's soft turn-off on short circuit must respect.

Thermal check

Worked example — junction temperature at continuous and peak power. At 75 kW continuous the inverter dissipates about 1.1 kW (98.5 % efficiency at that point), about 183 W per switch position. With a junction-to-coolant thermal resistance of 0.15 K/W per position (pin-fin baseplate, from the module data sheet) and 65 °C coolant, the junction runs at $65 + 183 \times 0.15 = 92.5$ °C. At 150 kW peak the loss is about 3 kW, 500 W per position, and the junction rises to $65 + 500 \times 0.15 = 140$ °C after its thermal time constants settle, inside the 175 °C rating with a 35 K margin. The coolant itself rises only $1{,}100/(10/60 \times 1.07 \times 3{,}400) = 1.8$ K through the inverter at 10 L/min of 50/50 glycol, so the coolant inlet temperature, not the flow rate, is the parameter to control.

Control and safe states

The inverter runs FOC with MTPA and field weakening (*Electrical Energy and Machines*, Ch. 12 §12.8) at 10 kHz with a resolver for position. VCU and inverter must agree on three safe states: **zero torque** on CAN timeout, **active short circuit** above the back-EMF crossover (about 10,100 rpm, where the back-EMF peak meets 438 V) and **freewheel** below it; the hand-over is tested on a dynamometer.

24.5 DC–DC converters

The 14 V network still runs lights, wipers, electric power steering, the brake booster, door locks, infotainment and every control unit. In a combustion van the alternator fed it; here an isolated DC–DC converter (the "LDC") from the HV bus does, with a 12 V lead-acid or LFP auxiliary battery to supply the network before the HV contactors close and to absorb load steps.

Worked example — sizing the LDC. The LV load list totals 1.1 kW average while driving and 2.0 kW peak (steering assist during parking, wipers, heated mirrors and screen, lights). At 14 V, 2.0 kW is 143 A on the output; at 95 % efficiency and 360 V, the HV draw is $2{,}000/(0.95 \times 360) = 5.8$ A. A 2.5 kW unit gives 25 % margin on the peak and covers the 0.5 kW the auxiliary battery recharges at. Phase-shifted full-bridge and LLC topologies (*Electronics and Computing*, Ch. 30) are both used; the choice is driven by the 300–438 V input window, which an LLC handles less gracefully, so a phase-shifted full bridge with synchronous rectification is chosen.

Steering and brake assist depend on the LDC, so if it fails the VCU warns the driver and limits speed while the 12 V battery carries the vehicle; steer-by-wire vehicles fit two.

Table 24.6 — DC–DC converter specification.

Parameter	Value	Reason
Input	280–450 V	Pack window plus cold sag and charger overshoot
Output	13.5–14.8 V settable	Auxiliary battery charge voltage by temperature
Power	2.5 kW continuous at 65 °C coolant	R8 plus 25 % margin
Isolation	Reinforced, 2.5 kV test	ISO 6469-3
Efficiency	≥ 94 % at 1 kW	0.1 kWh/h of loss matters in Table 24.3
Interface	CAN set-point and status	VCU raises output to charge the 12 V battery

24.6 Battery packs

> **Boundary.**
> Cell selection, series–parallel arrangement, BMS architecture, state estimation and pack safety testing are owned by Chapter 23; cell chemistry and ratings by *Electrical Energy and Machines*, Ch. 16 §16.1–16.3. Here the pack is treated as a vehicle component with a voltage window, a power capability and safety interfaces.

The pack is 120 prismatic LFP cells of 200 Ah in series (120s1p): 384 V nominal, 300–438 V between the 2.5 V and 3.65 V cell limits, 76.8 kWh gross and 72 kWh usable when the BMS keeps a few percent at each end. The cells alone weigh about 470 kg; with modules, cooling plates, BMS and enclosure the pack is about 620 kg, 124 Wh/kg at pack level. LFP was chosen over NMC for cycle life (more than 3,000 full cycles at the 50 % daily depth of a delivery route), thermal stability and cost; its price is lower energy density and poor low-temperature power, which the rest of the chapter has to manage.

Worked example — voltage sag sets the inverter's minimum voltage. At full power the pack supplies 150/0.92 kW to the inverter plus about 10 kW of auxiliaries and DC–DC load, about 170 kW, which at 330 V is 515 A, or 2.6C. Each cell's DC resistance is about 0.25 mΩ at 25 °C, so the series string is 30 mΩ, plus 10 mΩ of busbars, fuse and contactors: 40 mΩ. At 10 % SOC (OCV about 3.20 V per cell, 384 V) the sag is $515 \times 0.040 = 20.6$ V and the terminal voltage 363 V. At −10 °C the cell resistance roughly triples, the pack is 100 mΩ, the sag 52 V and the terminal voltage 332 V. This is why Section 24.4 specified full power to 330 V with a derating below: a lower minimum costs inverter current rating, a higher one costs power in the cold.

Safety interfaces

Pre-charge. The inverter's 800 µF DC link must be charged through a resistor before the main contactor closes, or the inrush welds the contacts. With 50 Ω the time constant is 40 ms, the peak current $438/50 = 8.8$ A, and the link reaches 95 % of pack voltage in $3\tau = 120$ ms; the resistor absorbs $\frac{1}{2}CV^2 = 77$ J per pre-charge. The BMS closes the main positive contactor only when the link voltage is within 5 % of the pack voltage, and declares a fault if that has not happened within 500 ms, which catches both a shorted link and an open pre-charge path.

Isolation monitoring. The HV bus floats with respect to chassis. An isolation monitor injects a low-frequency test signal and measures the resistance from each rail to chassis. ISO 6469-3 and UN ECE R100

require at least 100 Ω/V for DC circuits, that is $438 \times 100 = 43.8$ kΩ; the van warns at 500 Ω/V (219 kΩ) so that coolant ingress or a chafed cable is found long before it is dangerous.

HV interlock loop and discharge. A 10 mA loop runs through every HV connector and service disconnect; breaking it opens the contactors within 100 ms. After the contactors open, the inverter discharges its DC link actively (by circulating current in the motor windings with zero torque) to below 60 V in under 3 s, with a passive bleed resistor as back-up (R9).

> **Status of the principle: Pack voltage window at full power**
> *Approximate.* Terminal voltage is OCV minus IR with a temperature-dependent DC resistance; it ignores the diffusion polarisation that adds a further 5–10 % sag in a 10 s pulse, so a cold pulse dips through the 330 V design point into the inverter's derating.

24.7 Regenerative braking

What limits regeneration

When the driver lifts or presses the brake, the VCU asks the motor for negative torque. Four limits apply, and the smallest one wins:

1. **Motor and inverter.** 310 N·m and 150 kW; never the binding limit in this van.
2. **Battery acceptance.** The BMS's live charge-power limit, which depends on SOC, cell voltage and temperature. For the LFP pack it is about 100 kW warm, 30 kW at 5–10 °C, 8 kW at −10 °C, and it tapers to zero as the highest cell approaches 3.65 V.
3. **Vehicle stability.** Braking unloads the rear axle, so more than about 0.25 g of rear-only braking risks rear-wheel lock on a slippery road; the brake ECU caps regen and can cancel it within milliseconds.
4. **Low speed.** Below about 5 km/h regen fades out and the friction brakes stop the van.

The energy that regen returns is set out in *Electrical Energy and Machines*, Ch. 3 §3.2 and the four-quadrant hardware in *Electronics and Computing*, Ch. 32 §32.6; the integration task is the blending.

Worked example — what the cold pack accepts. Braking at 1.2 m/s² from 60 km/h needs $1.05 \times 3{,}000 \times 1.2 = 3{,}780$ N at the wheels, 63 kW at that speed. The motor torque is $3{,}780 \times 0.36 \times 0.97/11 = 120$ N·m, well inside 310 N·m. The motor turns at $16.67/0.36 \times 11 = 509$ rad/s. Since $\eta_d = 0.88$ (wheel to battery) already includes the gearbox, a battery that accepts 8 kW allows $8/0.88 = 9.1$ kW at the wheels, 14 % of the demand (17.3 N·m at the motor); the friction brakes turn the rest into heat. Preconditioned to the 30 kW limit, the pack allows 34.1 kW, 54 %. Over the whole loop this is the difference between 0.27 kWh and 0.72 kWh recovered, the regeneration rows of Table 24.3.

Figure 24.4 plots this across speed.

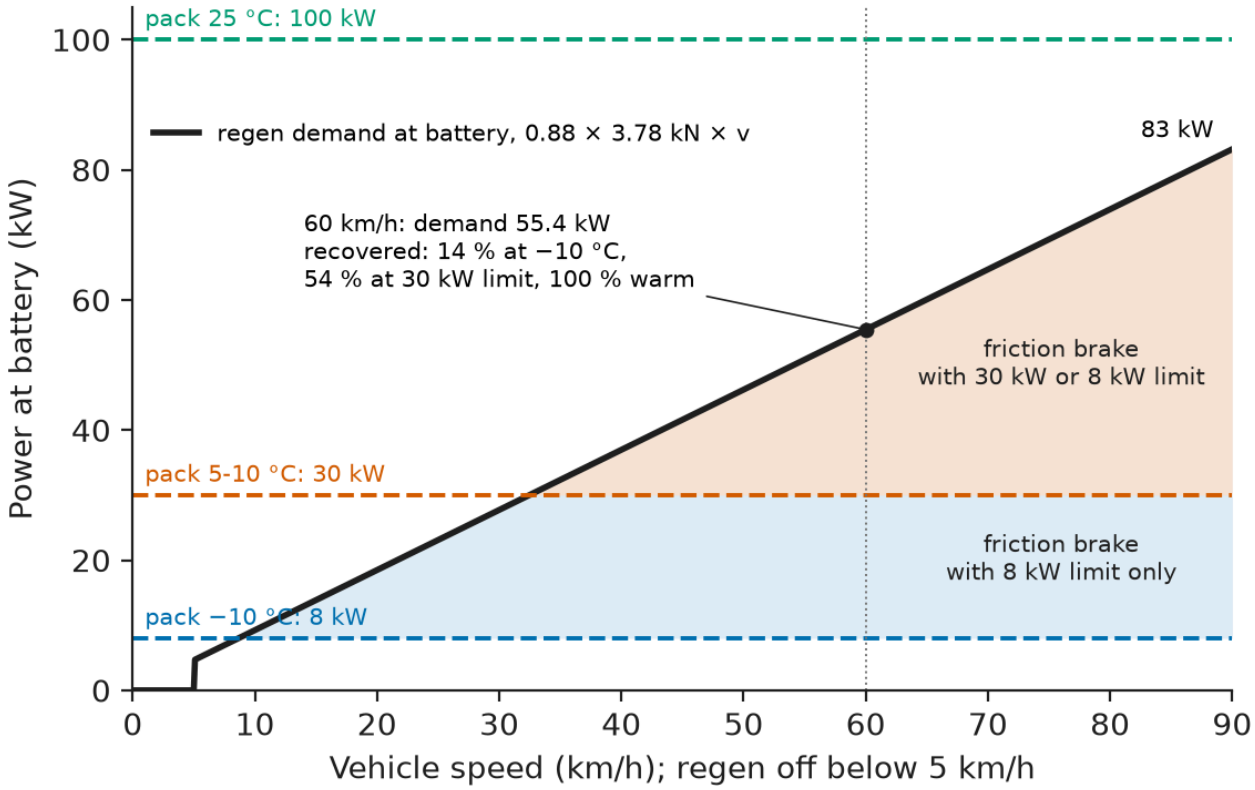


Figure 24.4 — Regenerative power demanded at 1.2 m/s² deceleration versus the battery's charge-power limit at three pack temperatures.

Implementation

The listing is the core of the VCU's 10 ms regen-limit function (C11; signal names illustrative): zero on stale BMS data, battery power limit converted to torque at the present speed, a low-speed fade and a taper as the highest cell approaches the LFP limit.

```
/* regen_limit.c -- regenerative torque limit, called every 10 ms by the VCU (C11).
   SI units. Signal names are illustrative. */
#include <math.h>

typedef struct {
    float p_chg_lim_w;    /* BMS live charge-power limit, W (>= 0) */
    float v_cell_max_v;   /* highest cell voltage, V */
    float speed_rad_s;    /* motor speed, rad/s */
    int   bms_fresh;      /* 1 if a BMS frame arrived in the last 100 ms */
} regen_in_t;

#define T_REGEN_MAX  310.0f  /* motor regen torque capability, N*m */
#define ETA_REGEN    0.91f   /* shaft-to-battery efficiency: eta_d 0.88 / gearbox 0.97 */
#define W_FADE_LO    8.5f    /* rad/s, about 1 km/h: no regen below */
#define W_FADE_HI    42.4f   /* rad/s, about 5 km/h: full regen above */
#define V_CELL_SOFT  3.55f   /* V, LFP: start tapering */
#define V_CELL_HARD  3.62f   /* V, LFP: no regen */

/* Returns the magnitude of the allowed regenerative torque, N*m (>= 0). */
float regen_torque_limit(const regen_in_t *in)
{
    if (!in->bms_fresh) return 0.0f;                    /* no BMS data: no regen */
    float w = fabsf(in->speed_rad_s);
    if (w < W_FADE_LO) return 0.0f;

    float t = fminf(T_REGEN_MAX, in->p_chg_lim_w / (ETA_REGEN * w));
    if (w < W_FADE_HI)
        t *= (w - W_FADE_LO) / (W_FADE_HI - W_FADE_LO);  /* low-speed fade */

    float k = (V_CELL_HARD - in->v_cell_max_v) / (V_CELL_HARD - V_CELL_SOFT);
    return t * fminf(1.0f, fmaxf(0.0f, k));              /* cell-voltage taper */
}
```

The brake ECU makes up the difference between demand and the torque the inverter reports as delivered, so the driver feels the same deceleration with a cold, full or warm pack. The taper, the live limit and zero torque on an open-contactor bit prevent the fault of regenerating into a full pack described in *Electronics and Computing*, Ch. 32 §32.6.

> **Status of the principle: Regen limited by battery acceptance**
> *Derived.* Torque equals power divided by speed; the limit is exact for a given BMS power limit, but the limit itself comes from an empirical lithium-plating model of the cell, so its accuracy is that of the cell supplier's low-temperature charge map.

24.8 Thermal constraints

Three loops, three temperature windows

The van has three thermal masters with different targets:

Table 24.7 — Thermal windows of the reference van.

Component	Operating window	Limit set by	Cooling or heating path
Inverter (SiC)	Coolant inlet ≤ 65 °C; junction ≤ 175 °C	Module rating, solder fatigue	Glycol loop with motor, front radiator
Motor	Winding ≤ 180 °C (class H); magnet ≤ 140–150 °C	Insulation life; demagnetisation	Same loop plus oil spray on end windings
Pack (LFP)	Discharge −20 to 55 °C; charge 0 to 45 °C; best 15–35 °C	Lithium plating cold; ageing hot	Cooling plates; chiller from A/C circuit; PTC heater or heat pump
Cabin	18–22 °C	Driver comfort, screen demisting	Heat pump or PTC heater

A refrigerant heat pump can move the 1–3 kW of motor and inverter waste heat into cabin and pack, and chill the pack during fast charging; its coefficient of performance is about 2 at −10 °C, falling toward 1.2 at −20 °C, where the PTC heater takes over.

Pack preconditioning

Worked example — warming the pack before a fast charge. The 470 kg of cells have a specific heat of about 1,100 J/(kg·K). Raising them from 0 to 20 °C stores $470 \times 1{,}100 \times 20 = 10.3$ MJ, 2.86 kWh. A 7 kW coolant heater does it in 25 min (lossless). Done from the grid before departure, it costs the shift nothing; done from the pack while driving to a fast charger, it costs 2.9 kWh (about 7 km of cold range) and saves an hour and fifty minutes of charging (Section 24.9). Preconditioning is therefore triggered by the navigation system when a DC charger is set as the destination and by the depot schedule before departure.

> **Status of the principle: Lumped heat capacity of the pack**
> *Approximate.* It treats the cells as one isothermal mass (*Engineering Physics*, Ch. 14); in practice the cells nearest the cooling plate warm first, and the BMS uses the coldest cell temperature for its charge limit, so real warm-up takes 20–40 % longer than the lumped estimate.

The motor's thermal limit sets how long peak ratings last: the supplier's model allows 30 s at 310 N·m from a 90 °C winding, enough for a hill start (R3) but not for repeated starts on a steep ramp, so the VCU derates toward the 155 N·m continuous rating as the estimated winding temperature approaches 165 °C.

24.9 Charging systems

AC charging through the on-board charger

The 11 kW on-board charger (OBC) is a PFC stage plus an isolated DC–DC stage (*Electronics and Computing*, Ch. 32 §32.7). On a 400 V three-phase supply at 16 A it draws $\sqrt{3} \times 400 \times 16 = 11.1$ kW. **Worked example — R6.** Charging from 20 % to 100 % puts $0.8 \times 76.8 = 61.4$ kWh into the pack; at 94 % charger efficiency and 11 kW input that takes $61.4/(11 \times 0.94) = 5.9$ h, inside the 6.5 h of R6, with 0.6 h margin.

The connection is governed by IEC 61851-1 (charging modes and the control pilot) and IEC 62196-2 (the Type 2 connector). The EVSE (wall box) generates a ±12 V, 1 kHz PWM **control pilot**; the vehicle loads it with a resistor and diode, which steps the positive level from 12 V (state A, not connected) to 9 V (state B, connected) and, when the vehicle closes a switch to add a second resistor, to 6 V (state C, charging requested). The EVSE announces the current available by the pilot's duty cycle D: for $10\,\% \le D \le 85\,\%$,

$$I_{max} = 0.6\ \text{A} \times D[\%],$$

so 26.7 % means 16 A and the minimum 10 % means 6 A. A resistor in the cable plug from the proximity contact to earth tells the vehicle the cable's current rating (1.5 kΩ for 13 A, 680 Ω for 20 A, 220 Ω for 32 A). The OBC charges at the minimum of the pilot current, the cable rating and its own and the BMS's limits. This simple analogue interface is what the depot's load management (Section 24.10) uses to throttle twenty vans.

DC fast charging

For DC charging the off-board charger feeds the pack directly through the DC inlet and two DC contactors; the OBC is bypassed. The vehicle (not the charger) is in charge: over ISO 15118 or DIN 70121 messages carried by power-line communication on the control-pilot wire, the vehicle requests a voltage and current every 50–100 ms, and the charger follows. The charger performs an insulation test of the cable and inlet before energising, and the vehicle checks that the charger's output voltage matches its pack (within about 20 V) before closing its DC contactors.

The van's limit is the cells, not the charger. The LFP cell supplier allows about 1.5C (300 A) at 25 °C up to 55 % SOC, tapering to 0.8C at 80 %; at 0 °C the allowance is about 0.3C.

Worked example — R7 and the cold charge. With a charge power of 115 kW from 10 to 55 % SOC tapering linearly to 60 kW at 80 %, numerical integration of $dt = E\,ds/P(s)$ for $E = 76.8$ kWh gives 31.7 min from 10 to 80 %: R7 passes with an 8 min margin. At 115 kW and 370 V the current is 311 A, inside the 500 A of a liquid-cooled CCS2 cable. With the pack at 0 °C the limit is $0.3 \times 200 \times 370 = 22.2$ kW and the same 53.8 kWh takes 145 min — the van of the opening paragraph. Preconditioning for 25 min (Section 24.8) restores the 32 min charge (Figure 24.5).

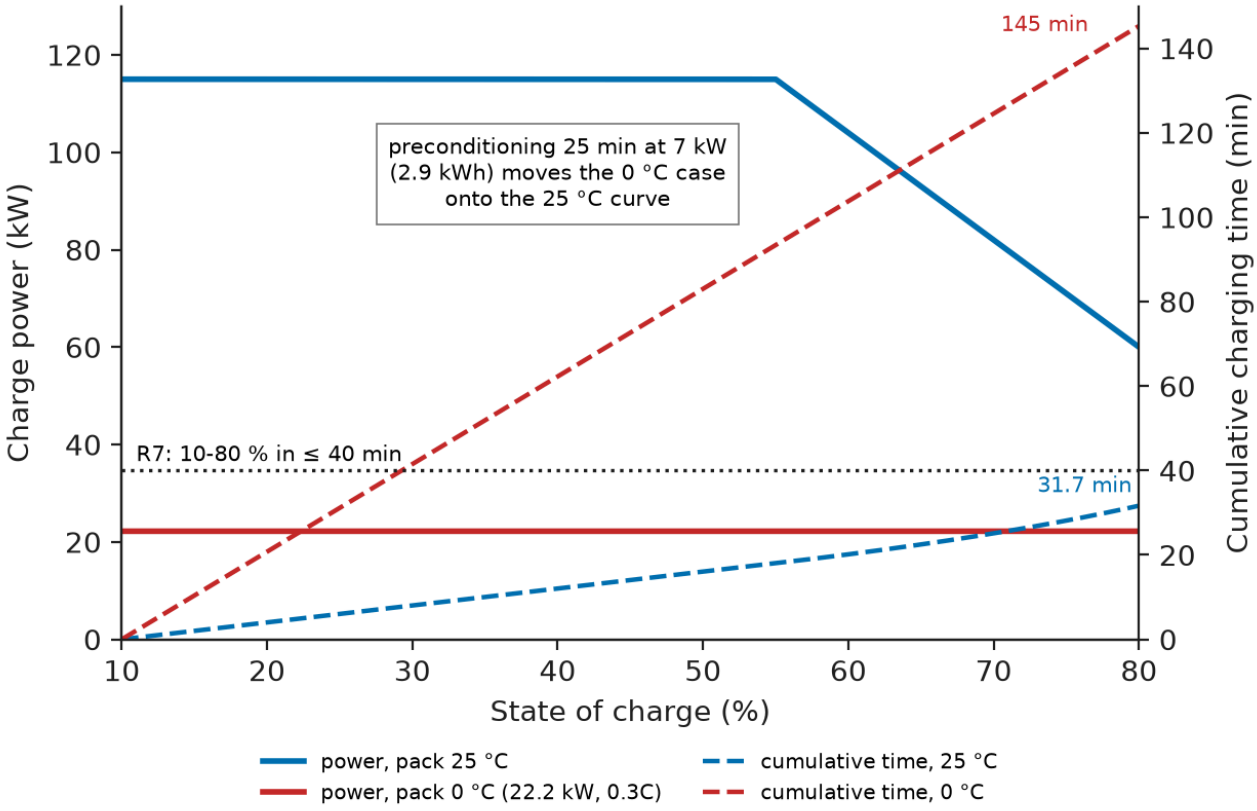


Figure 24.5 — DC fast-charge power and cumulative time versus state of charge, pack at 25 °C and at 0 °C.

Table 24.8 — Charging options for the reference van.

Mode (IEC 61851)	Supply	Power	Use	20–100 % or 10–80 % time
Mode 2, portable cable with in-cable control box	230 V, 10 A single phase	2.3 kW	Emergency only	About 28 h (20–100 %)
Mode 3, Type 2 EVSE	400 V three-phase, 16 A	11 kW	Depot, overnight	5.9 h (20–100 %)
Mode 4, CCS2 DC	400–500 V DC, up to 500 A	115 kW (vehicle limit)	En route	32 min (10–80 %), 25 °C pack

24.10 Charging infrastructure

The depot problem

Twenty vans returning between 18:00 and 19:30 each need, on a cold day, the 62.1 kWh drive-cycle figure of Table 24.3 restored (the drive figure, not the shift total), which is 62.1/0.94 = 66.1 kWh from the grid. The depot's existing building load (lighting, a cold store, offices) peaks at 200 kW at 18:00 and falls to 60 kW by 21:00. The site connection is a 400 kVA transformer whose loading the network operator caps at 280 kW of import.

Uncontrolled, every van starts at 11 kW as it plugs in, and the site peaks at 349 kW at 19:30, 25 % over the cap. Upgrading the connection to 630 kVA would cost (2026, typical distribution-network quote) $60,000–$120,000 and take months. A **dynamic load-management** controller instead reads the site meter every few seconds and shares the headroom (cap minus building load) among the connected vans by setting each EVSE's pilot duty cycle, never below the 6 A floor of IEC 61851-1.

```
 import numpy as np

N, P_EVSE, P_MIN, CAP = 20, 11.0, 4.16, 280.0        # vans, kW per EVSE, 6 A floor, site cap kW
dt = 1/60                                            # h
t = np.arange(18.0, 30.5, dt)                        # 18:00 to 06:30 next day
building = 60 + 140*np.clip((21 - t)/3, 0, 1)        # 200 kW at 18:00 falling to 60 kW by 21:00
arrive = np.linspace(18.0, 19.5, N)
need = np.full(N, 62.1/0.94)                         # kWh from the grid per van (cold shift)

def run(managed):
    left, site, done = need.copy(), [], np.full(N, np.nan)
    for k, tk in enumerate(t):
        active = (tk >= arrive) & (left > 0)
        p = np.where(active, P_EVSE, 0.0)
        if managed and active.any():
            share = (CAP - building[k]) / active.sum()  # equal share of headroom
            p = np.where(active, np.clip(share, P_MIN, P_EVSE), 0.0)
        p = np.minimum(p, left/dt)
        left -= p*dt
        done[(left <= 1e-9) & np.isnan(done)] = tk
        site.append(building[k] + p.sum())
    return np.array(site), done

for managed in (False, True):
    site, done = run(managed)
    print(f"managed={managed}: peak site {site.max():.0f} kW at {t[site.argmax()] % 24:.2f} h,"
          f" last van full at {np.nanmax(done) % 24:.2f} h, all full: {not np.isnan(done).any()}")
```

```
 managed=False: peak site 349 kW at 19.52 h, last van full at 1.52 h, all full: True
managed=True: peak site 280 kW at 19.65 h, last van full at 1.75 h, all full: True
```

Worked example — what load management costs. Holding the site at 280 kW delays the last van's finish from 01:31 to 01:45 against a 06:30 departure: R10 passes with 4.75 h of margin, and the transformer upgrade is avoided. The margin is also what pays for preconditioning every pack from the grid in the last 30 min before departure ($20 \times 2.9 = 58$ kWh, at most about 140 kW if all 20 heaters run at 7 kW together; the controller staggers them). Figure 24.6 shows both profiles.

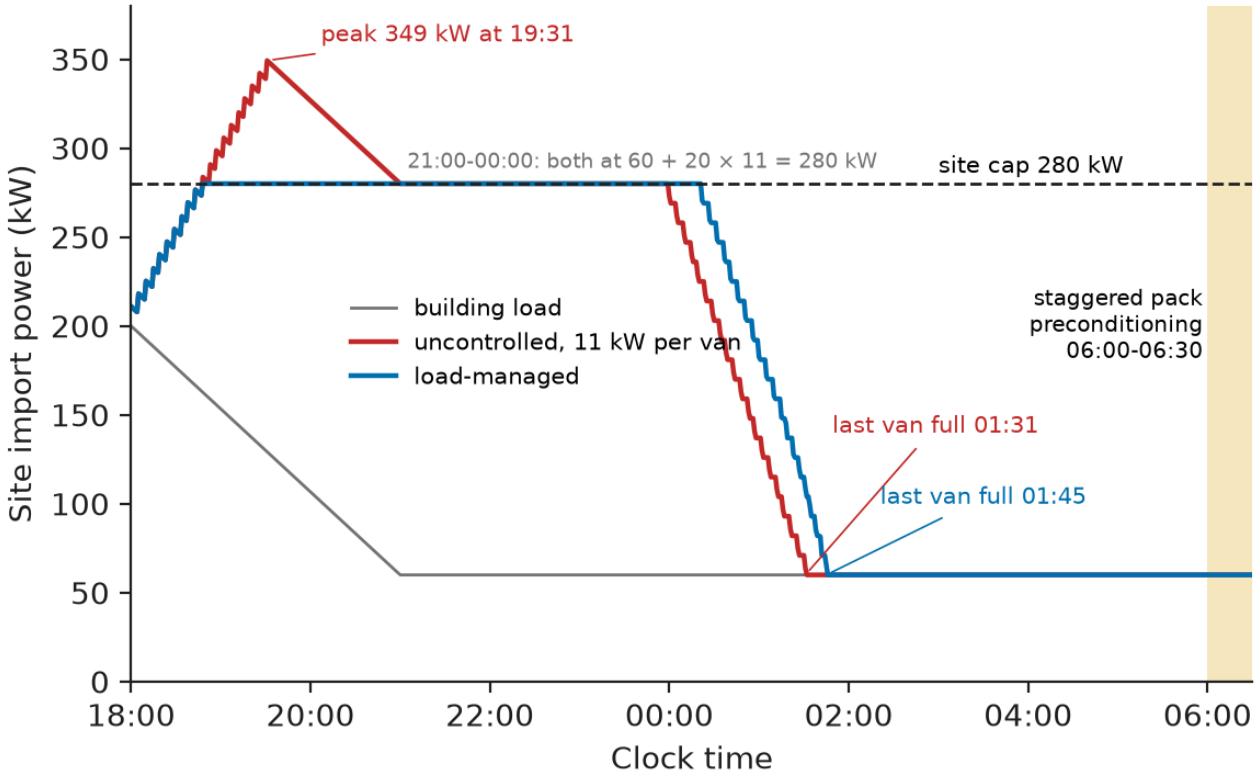


Figure 24.6 — Depot import power overnight for 20 vans, uncontrolled and with dynamic load management.

> **Status of the principle: Headroom sharing under a site cap**
> *Derived.* Energy delivered equals the integral of shared headroom; it holds while the 6 A pilot floor times the number of active vans ($20 \times 4.16 = 83$ kW at most) stays below the headroom; a larger fleet needs a scheduler that admits vans in turn.

Standards and equipment

Charging equipment is specified against IEC 61851-1 and -23, IEC 62196 (connectors), ISO 15118-2/-20 (including Plug & Charge and bidirectional power), OCPP 1.6 or 2.0.1 to the back-office (through which the depot controller sets EVSE limits), and IEC 60364-7-722, which requires a Type A RCD with 6 mA DC detection or a Type B RCD per outlet. The installation design itself (cable sizing, protection selection, earthing) follows *Electrical Energy and Machines*, Ch. 30. A depot with on-site PV and a stationary battery is a microgrid, and Chapter 25 takes it up there.

24.11 Verification, failure analysis and Revision B

Verification

Table 24.9 closes each requirement with the test fixed in Table 24.1. The results are those of a prototype test programme (chassis dyno in a climatic chamber, ramp, track, charger and depot simulation); values are illustrative of what such a programme records, quoted with their measurement uncertainty.

Table 24.9 — Verification of the reference van, Revision B prototype.

Req.	Test	Limit	Result	Pass/fail
R1	Climatic dyno, −10 °C, 14 urban loops plus 4.5 h dwell with doors cycling	≥ 7.2 kWh usable left after 150 km	17.6 ± 1.0 kWh left	Pass
R2	Dyno, 20 °C, loops to empty	≥ 300 km	338 ± 6 km	Pass
R3	25 % ramp at 3,500 kg, 3 starts	Start and climb; motor torque < 310 N·m	Peak 292 N·m; climbed	Pass
R4	Track, 120 km/h for 20 min	Winding ≤ 180 °C, junction ≤ 175 °C	128 °C winding, 96 °C junction (model)	Pass
R5	0–50 km/h, 3,000 kg, 5 runs	≤ 6.0 s	5.3 ± 0.1 s	Pass
R6	Depot EVSE, 20–100 %	≤ 6.5 h	6.0 h	Pass
R7	CCS2 charger, 10–80 %, pack 25 °C	≤ 40 min	33 min	Pass
R7	Same after 25 min preconditioning from −10 °C soak	≤ 40 min (charge only)	35 min	Pass
R8	2.0 kW load on 14 V bus, 65 °C coolant, 2 h	13.5–14.8 V, no derating	14.1 V, no derating	Pass
R9	Isolation, HVIL break, active discharge	≥ 43.8 kΩ; contactors open ≤ 100 ms; ≤ 60 V in ≤ 3 s	12 MΩ; 42 ms; 1.1 s	Pass
R10	Depot controller on site with 20 vans	Import ≤ 280 kW, all full by 06:30	Peak 278 kW; last full 01:52	Pass

Failure analysis

Table 24.10 — FMEA extract for the reference van.

Failure mode	Effect	Cause	Detection	Mitigation
Range shortfall in winter	Vans stranded at 139 km	Heater energy budgeted per km, not per hour; cold pack rejects regen	Fleet telematics; Table 24.3	Heat pump, zonal heating, depot preconditioning, dwell-time budget (Revision B)
Slow DC charge when cold	145 min instead of 32 min	Cell plating limit at 0 °C	Charger log	Navigation-triggered preconditioning
Main contactor weld	HV cannot be disconnected	Closing without pre-charge; opening under load after a crash	Contactor-state check (link voltage after opening)	Pre-charge timeout check; open contactors only at low current except in emergency
Uncontrolled generation at high speed	Bus overvoltage, capacitor and pack damage	Inverter freewheels above back-EMF crossover	Bus overvoltage comparator	Active short circuit above 10,100 rpm
Regen into full or cold pack	BMS opens contactor while braking; loss of regen braking	Constant regen limit; stale BMS data	BMS cell-voltage alarm	Live limit, taper, stale-data zero (Section 24.7 listing)
Isolation fault	Shock hazard on second fault	Coolant leak into HV connector; chafed cable	Isolation monitor < 219 kΩ warning	Connector sealing to IP6K9K; harness routing review
LDC failure	Loss of steering and brake assist after 12 V battery drains	Converter fault, thermal shutdown	12 V bus voltage and LDC status	Driver warning, speed limit, 12 V battery sized for 15 min
Depot overload	Transformer trips; no van charged	Uncontrolled simultaneous charging	Site meter	Dynamic load management with fail-safe 6 A default if controller is lost

The worked fault story is the opening winter. Telematics showed energy per kilometre while moving matching the cold model's 41 kWh/100 km, but SOC also falling about 5 % per hour while standing: 3.5 kW of heater against 72 kWh. The mechanism was a budget written in the wrong unit (Table 24.3, last two rows); the regen and fast-charge shortfalls had the same root, a pack that nobody had planned to warm.

Revision B

Change	Reason	Effect
Refrigerant heat pump (COP about 2 at −10 °C) with motor and inverter waste-heat recovery; PTC kept for below −20 °C	Heating dominated the cold budget	Driving auxiliaries 5.3 → 3.05 kW
Seat and steering-wheel heating, air curtain at the side door, cab-only heating zone when parked	Parked heating with doors cycling	Parked heating 3.5 → 1.5 kW
Depot preconditioning of cabin and pack to 20 °C before 06:30 from grid power	Cold pack rejects regen and charge	Regen limit 8 → 30 kW for the first hours; R1 margin 27 %
Navigation-triggered pack heating before DC charging	145 min cold charge	35 min charge after 25 min preheat
Measured efficiency map replaces constant η_d in the energy model	Model optimistic at motorway speeds	Budget agreed with dyno within 3 %
Depot controller default: 6 A per EVSE if the controller's communication fails	A failed controller must not overload the site	20 × 4.16 kW = 83 kW worst case

What remains open

Open question	Where it is answered
How is the pack's BMS designed, and how are SOC and power limits estimated?	Chapter 23
How does a depot with PV and stationary storage schedule its charging against a tariff?	Chapter 25
How do the VCU, inverter and BMS share a CAN network safely, and how is it diagnosed?	Chapter 21; *Electronics and Computing*, Ch. 26
How is the FOC current loop of the traction drive built and tuned?	Chapter 37 (closed-loop drive); *Electrical Energy and Machines*, Ch. 12
How are the van's thermal and energy models calibrated against fleet data and used as a digital twin?	Chapter 33
How are energy-model sweeps and optimisation of gear ratio and pack size automated?	Chapter 30
How does vehicle-to-grid change the depot and the grid connection?	Chapter 25; *Electrical Energy and Machines*, Ch. 27

Next: Chapter 25 takes the depot's grid connection, adds PV, wind, a stationary battery and a generator, and asks how such a hybrid system is controlled when the grid is there and when it is not.

Chapter 25 — Microgrids and Hybrid Energy Systems

Part VI · Renewable Energy and Electric Mobility

A district hospital at the end of a 25 km rural 11 kV feeder loses grid power about forty times a year. Its two diesel generators start in ten seconds, and for years the operating theatres have ridden through on UPS batteries. A donor funds a hybrid system: 400 kWp of PV on the roofs and car park, two 50 kW wind turbines on the ridge behind the laundry, and a 600 kWh lithium-iron-phosphate battery with a grid-forming inverter, so that the hospital islands without a gap. Three weeks after commissioning, at noon on a Sunday, the feeder trips. The battery takes the island cleanly. But the load is only 110 kW, the PV is producing 330 kW, the battery is at 95 % state of charge, and nothing tells the PV inverters to back off. In forty seconds the highest cell reaches its limit, the battery inverter trips to protect itself, the island collapses, and the hospital is back on generators and UPS — exactly the event the project was paid to prevent. Every component worked. The *coordination* between them had not been specified: the PV inverters' frequency-watt response, the battery's way of asking for it, and the test that would have shown the gap. This chapter designs that hospital microgrid from load profile to economics.

> **How are PV, wind, storage and generators sized and dispatched together so that a site meets an energy target, a resilience target and a budget at the same time?**
> **What must each inverter, relay and controller be set to do so that the system works both connected to the grid and as an island, and moves between the two without a blackout?**

Concept	Engineering destination
Solar–wind–storage integration	Complementary resources; storage sized for both energy shifting and outage ride-through
Hybrid architectures	AC- versus DC-coupling; grid-forming and grid-following converters; where the diesel fits
Load profiles	Peak, base, load factor, critical load; the number every other size depends on
Energy management	Hourly dispatch rules, reserve state of charge, export limits, generator scheduling
Grid-connected operation	Export limitation, voltage rise on weak feeders, grid-code functions
Islanded operation	Grid-forming source, transition, motor starting, generator minimum load
Inverter coordination	Frequency-watt curtailment, droop sharing, communication-free fallback
Protection	Fault current that changes by 20× between modes; adaptive settings
Reliability	Interruption frequency of the critical load; autonomy; common-cause failures
Economic evaluation	NPV, payback, LCOE and the price of each requirement

Roadmap. Requirements and architecture (25.1) → the load profile (25.2) → integrating solar, wind and storage (25.3) → the energy-management system (25.4) → grid-connected operation (25.5) → islanded

operation (25.6) → inverter coordination (25.7) → protection (25.8) → reliability (25.9) → economic evaluation (25.10) → verification, failure analysis and Revision B (25.11).

25.1 Hybrid architectures

> **Boundary.**
> The definition of a microgrid, its control hierarchy and steady-state droop sharing are in *Electrical Energy and Machines*, Ch. 26 §26.4; grid-connected inverters, anti-islanding and frequency support in Ch. 27 §27.1–27.6; wind and PV resources in Ch. 15. PV array design is this book's Chapter 22 and the battery pack Chapter 23. This chapter integrates them into one site.

Requirements

Table 25.1 — Requirements for the hospital microgrid.

ID	Requirement	Target	Verification
R1	Seamless islanding of the critical board	Voltage below 70 % for no more than 100 ms on unplanned grid loss	Staged PCC trip, power-quality recorder at critical board
R2	Critical-load autonomy on battery alone	≥ 2.5 h at 90 kW from the reserve state of charge	Calculation; timed island test with generators inhibited
R3	Renewable fraction (funder condition)	≥ 60 % of annual consumption	Hourly simulation; then metered first year
R4	Export at the point of common coupling (PCC)	≤ 100 kW (connection agreement)	Export-limit test with load bank off
R5	Island frequency and voltage	49.0–51.0 Hz, 360–440 V line; nadir ≥ 49.0 Hz on a 100 kW step	Load-bank step tests
R6	Fault clearing in both modes	Final circuits ≤ 0.4 s (IEC 60364-4-41, TN, 230 V)	Fault-current calculation; injection test of trip units
R7	Grid-code functions	Ride-through, P(f), Q(V) per EN 50549-1 (IEEE 1547-2018 equivalents)	Inverter type test certificates; site settings audit
R8	Resynchronisation	$\Delta f \le 0.1$ Hz, $\Delta V \le 5$ %, $\Delta\varphi \le 10°$ at reclosure	Synchro-check relay log
R9	Economics	NPV ≥ 0 at 8 % over 20 years	Cash-flow model (Section 25.10)

R2 and R3 are the expensive ones, and Section 25.10 prices each of them.

Architecture options

Table 25.2 — Hybrid architectures considered.

Architecture	How sources connect	Strengths	Weaknesses	Fit here
AC-coupled	PV, wind and battery each through their own inverter onto the 400 V AC bus	Uses standard grid-tied PV inverters; each source sized and replaced independently; retrofits existing sites	PV energy for the battery is converted twice (DC–AC–DC); needs frequency-based coordination when islanded	**Chosen**: existing PV-ready switchboards, mixed sources
DC-coupled	PV through MPPT DC–DC converters onto the battery's DC bus; one bidirectional inverter	One conversion from PV to battery; simple curtailment (MPPT backs off)	Inverter must be sized for PV plus battery; single point of failure	Good for small off-grid sites
Hybrid all-in-one inverters	Several 30–100 kW hybrid inverters in parallel	Low cost per kW; integrated control	Parallel stability and protection depend on one vendor; limited wind input	Rejected: wind and gensets
DC microgrid (380 V DC)	Loads on DC distribution	Fewest conversions for DC loads	Hospital loads are AC; DC protection less mature	Rejected

Figure 25.1 shows the chosen AC-coupled single-line diagram. The PCC is the hospital's 400 V main incomer below a 630 kVA, 11/0.4 kV transformer. The microgrid controller (MGC) talks Modbus TCP or IEC 61850 to the battery inverter, the PV and wind inverters, the generator controllers and the PCC relay; but the parts of the design that must work in milliseconds — islanding, frequency-watt curtailment, droop sharing — work without it.

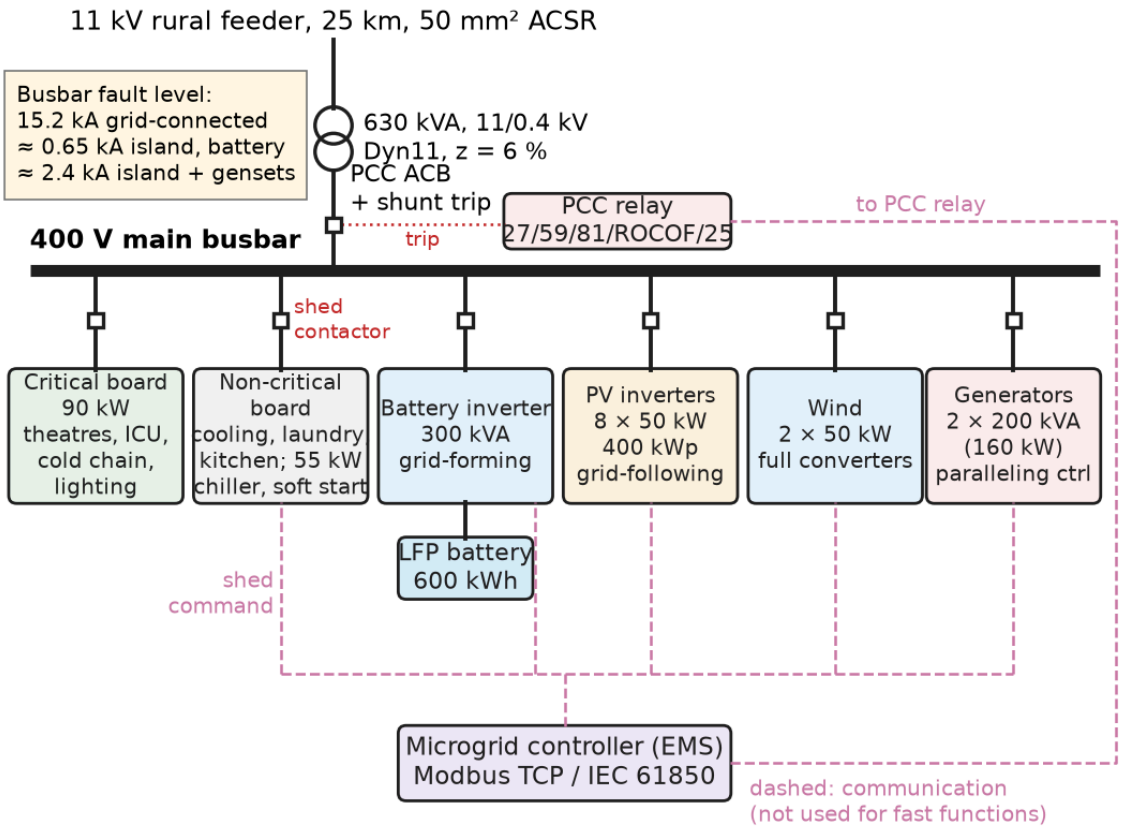


Figure 25.1 — Single-line diagram of the AC-coupled hospital microgrid.

Component choice: the battery inverter

The battery inverter decides whether the island exists. **Table 25.3** compares the three control types available in 300 kVA power-conversion systems (PCS).

Table 25.3 — Battery inverter control options.

Option	Behaviour on grid loss	Island stability with PV inverters	Fault current	Choice
Grid-following PCS with island-mode switch	Detects loss, stops, restarts as voltage source: 100–300 ms gap	Good once restarted	1.1–1.2 pu	Rejected: fails R1
Grid-forming droop PCS (voltage source at all times)	Holds voltage and phase through PCC opening: no gap	Good; frequency becomes a signal to PV	1.5 pu for 2 s, settable	**Chosen**
Virtual synchronous machine (VSM) PCS	As grid-forming, plus emulated inertia	Best with generators in parallel	1.5–2 pu	Considered; premium of about 15 %

25.2 Load profiles

The load profile sets every other size: the PV and battery for energy, the inverters and generators for power, the protection for current. The hospital was metered at 15 min resolution for six weeks with a power-quality analyser on the main incomer and on the critical board, and the year was reconstructed from monthly bills with a cooling-degree-day model (the auditing method of *Electrical Energy and Machines*, Ch. 28 §28.2).

Table 25.4 — Hospital load summary.

Quantity	Value	Use in the design
Annual energy	1,128 MWh	PV and wind energy sizing; R3 denominator
Mean load	129 kW	Generator fuel model, battery cycling
Peak load (hot-season midday)	269 kW	Island power rating; generator rating
Minimum load (night)	59 kW	Curtailment needed in island at noon on low-load days
Load factor	0.48	Ratio of mean to peak
Critical load (theatres, ICU, cold chain, lighting, IT)	90 kW	Autonomy (R2), shed scheme
Largest motor	55 kW chiller compressor, soft starter	Island inrush (Section 25.6)

The profile has the shape that makes solar attractive: a weekday plateau from 08:00 to 18:00 with a cooling peak in the early afternoon, and a 85–100 kW night base. Figure 25.2 shows a typical hot-season day with the PV and wind production of the design.

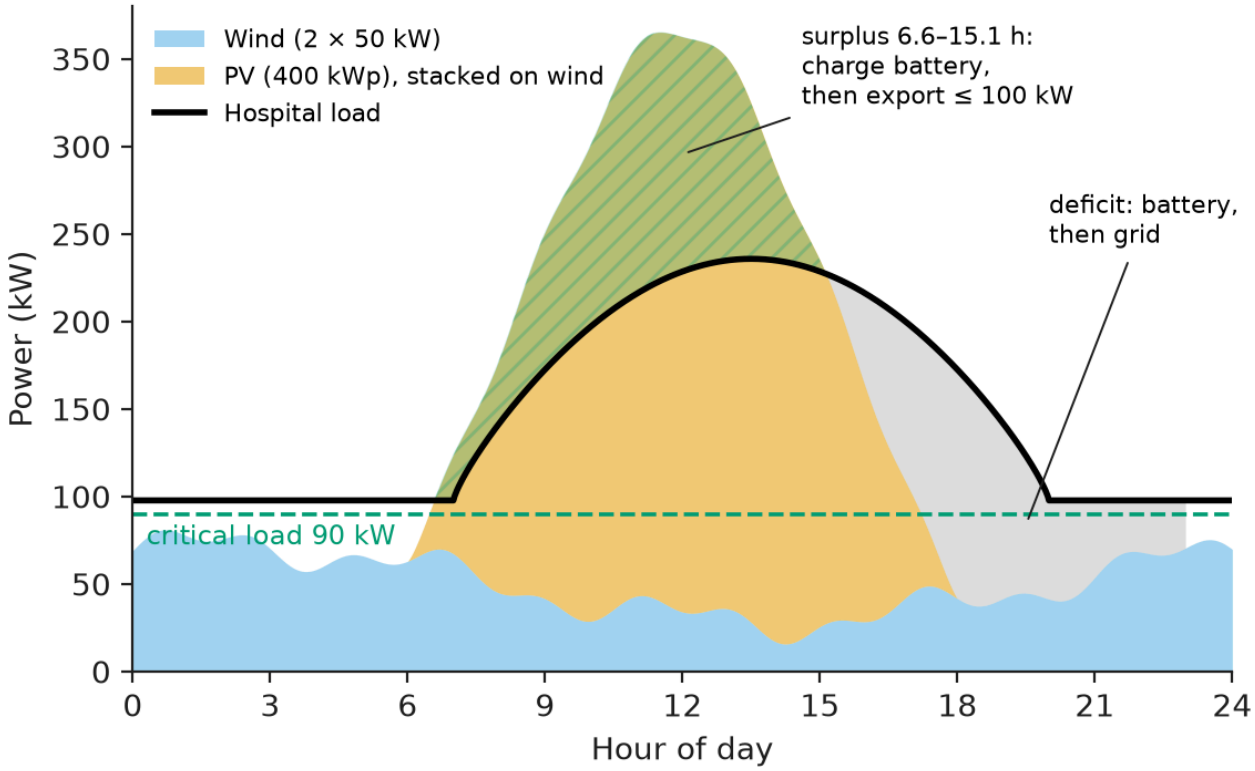


Figure 25.2 — Hospital load, PV and wind on a typical hot-season day, with the critical load marked.

> **Status of the principle: Hourly load profile as the design basis**
> *Empirical.* A reconstructed hourly year captures energy and daily shape but smooths sub-hourly peaks; power ratings (inverters, generators, protection) are therefore checked against the 1 s and 15 min metered peaks, not the hourly profile.

25.3 Solar–wind–storage integration

Why wind as well as solar

The roofs and car park hold 400 kWp of PV at most. Simulated with a site clearness model, 400 kWp gives 556 MWh a year (1,391 kWh/kWp), about half the 1,128 MWh load — so PV alone cannot meet R3's 60 %. Two 50 kW turbines at a hub-height mean speed of about 5.9 m/s add 214 MWh (capacity factor 24 %), and wind produces much of its energy at night, when PV produces nothing. The complementarity is the reason for the hybrid: a kilowatt-hour of wind at 02:00 displaces an import, while a further kilowatt-hour of PV at noon on a low-load day is mostly exported, stored with loss, or curtailed.

Worked example — could more PV replace the wind? If land were available, how much PV would meet R3 without wind? Running the dispatch of Section 25.4 with the PV scaled by 1.5 (600 kWp) and the 600 kWh battery gives a renewable fraction of 58 %; at 1.75 (700 kWp) it reaches 62 %, but 141 MWh a year is curtailed and 129 MWh exported at a low price, because the extra PV arrives at the hours that already have surplus. Wind meets the target at 400 kWp with only 14 MWh of curtailment. The site has no land for 300 kWp more anyway; the question confirms that wind is the right second source, not just the available one.

Sizing the storage for two jobs

The battery has two jobs that pull its size in different directions:

1. **Energy shifting**: absorb midday surplus and release it in the evening. In the simulated year the daily surplus is below 200 kWh on half the days, above 500 kWh on a quarter and above 850 kWh on a tenth, so capacity beyond about 600 kWh cycles only on the sunniest days.
2. **Resilience**: carry the 90 kW critical load from the reserve state of charge for 2.5 h (R2) while generators are started or, if they fail, repaired.

The energy-management system (Section 25.4) holds a 50 % reserve while the grid is up and allows discharge to 10 % in an island, so the energy available at the start of any outage is at least $(0.50 - 0.10) \times 600 = 240$ kWh, which carries the critical load for $240/90 = 2.67$ h: R2 passes with 0.17 h margin. The inverter rating of 300 kVA covers the 269 kW peak in an island with the non-critical board connected.

Table 25.5 — Island power and energy budget.

Quantity	Requirement	Available	Margin
Island peak power, generators off	269 kW (peak load)	300 kW battery inverter continuous	31 kW (12 %)
Island peak power, generators on	269 kW	300 + 2 × 160 = 620 kW	351 kW
Short-term overload (motor start)	210 + 199 kVA (Section 25.6)	450 kVA for 10 s	41 kVA (9 %)
Critical-load autonomy from reserve	2.5 h at 90 kW = 225 kWh	240 kWh	15 kWh (7 %)
Generator minimum load	≥ 30 % of 160 kW = 48 kW per set	EMS adds battery charging	—

> **Status of the principle: Battery autonomy from reserve energy and critical load**
> *Derived.* Runtime is energy divided by power; it assumes the critical load stays at its 90 kW design value and that the battery's usable capacity has not faded, so the reserve setting is raised by the measured capacity loss each year (Chapter 23).

25.4 Energy management

The dispatch rules

The energy-management system (EMS) in the microgrid controller runs every 15 min to 1 h and sets targets; the fast layers (inverter droop, frequency-watt) carry out the second-by-second balance. Its rules, in priority order:

- **Grid up, surplus**: charge the battery up to 95 % SOC, then export up to the 100 kW limit, then curtail.

- **Grid up, deficit**: discharge the battery down to the 50 % reserve, then import.
- **Island**: renewables and battery first, down to 10 % SOC; start a generator when SOC falls below 25 % and stop it above 60 %; a running generator carries the load and recharges the battery, and never runs below 30 % of rating.

Before writing them into a controller, the rules were run against an hourly year. The first listing generates the synthetic year (load from Table 25.4, PV from a daily clearness model, wind from a correlated Rayleigh speed series through a 50 kW power curve, and 40 outages with exponentially distributed durations of mean 3 h); it is saved as `site_profiles.py`.

```
 """Synthetic hourly year for the hospital microgrid: load, PV, wind (kW), grid outages."""
import numpy as np

def profiles(seed=1):
    rng = np.random.default_rng(seed)
    h = np.arange(8760); hod, doy = h % 24, h // 24
    season = 1 + 0.15*np.cos(2*np.pi*(doy - 170)/365)              # hot-season cooling
    load = (85 + 120*np.clip(np.sin(np.pi*(hod - 7)/13), 0, None)**0.8) * season
    load *= 1 + 0.06*rng.standard_normal(8760)
    clear = np.repeat(rng.uniform(0.35, 1.0, 365), 24)             # daily clearness
    sun = np.clip(np.sin(np.pi*(hod - 6)/12), 0, None)
    pv = 400*0.82*sun**1.3*clear*(1 + 0.08*np.cos(2*np.pi*(doy - 80)/365))
    x = np.zeros((8760, 2))                                         # AR(1) wind components
    for k in range(1, 8760): x[k] = 0.9*x[k-1] + np.sqrt(0.19)*rng.standard_normal(2)
    u = np.hypot(x[:, 0], x[:, 1]) * 6.0/np.sqrt(np.pi/2)          # Rayleigh, mean 6 m/s
    wind = 2*np.where((u >= 3) & (u < 25), np.minimum(50*(u**3 - 27)/(11**3 - 27), 50), 0)
    outage = np.zeros(8760, bool)
    for s in rng.choice(8760 - 24, 40, replace=False):              # 40 outages, mean 3 h
        outage[s:s + max(1, int(round(rng.exponential(3.0))))] = True
    return load, pv, wind, outage
```

The second listing, `ems.py`, is the dispatch itself.

```
 import numpy as np
from site_profiles import profiles

def dispatch(load, pv, wind, outage, E=600.0, P=300.0, reserve=0.5, exp_lim=100.0, eta=0.95):
    soc, gen_on = 0.8*E, False
    r = dict(imp=0.0, exp=0.0, curt=0.0, gen=0.0, gen_h=0)
    for k in range(len(load)):
        net = pv[k] + wind[k] - load[k]                       # kW; + is surplus
        floor = (0.10 if outage[k] else reserve) * E           # keep reserve while grid is up
        if net >= 0:                                           # surplus: charge, export, curtail
            ch = min(net, P, (0.95*E - soc)/eta); soc += ch*eta; net -= ch
            e = 0.0 if outage[k] else min(net, exp_lim)
            r['exp'] += e; r['curt'] += net - e; gen_on = False
            continue
        dis = min(-net, P, max(soc - floor, 0.0)*eta); soc -= dis/eta; net += dis
        if not outage[k]:
            r['imp'] += -net; gen_on = False
            continue
        gen_on = (gen_on or soc < 0.25*E) and soc <= 0.60*E    # genset hysteresis 25-60 % SOC
        if gen_on or net < 0:                                  # genset carries load, recharges
            g = min(320.0, max(-net + min(P, (0.6*E - soc)/eta), 48.0))
            soc += max(g + net, 0.0)*eta; r['gen'] += g; r['gen_h'] += 1
    return r

if __name__ == "__main__":
    load, pv, wind, outage = profiles()
    print(f"load {load.sum()/1e3:.0f} MWh (peak {load.max():.0f} kW), PV {pv.sum()/1e3:.0f} MWh, "
          f"wind {wind.sum()/1e3:.0f} MWh, outages {outage.sum()} h")
    for name, w in [("PV+wind+BESS", wind), ("PV+BESS", 0*wind)]:
        r = dispatch(load, pv, w, outage)
        ren = 1 - (r['imp'] + r['gen'])/load.sum()
        print(f"{name}: import {r['imp']/1e3:.0f} MWh, export {r['exp']/1e3:.0f} MWh, curtailed "
              f"{r['curt']/1e3:.1f} MWh, diesel {r['gen']/1e3:.1f} MWh in {r['gen_h']} h,
renewable {ren:.0%}")
```

```
 load 1128 MWh (peak 269 kW), PV 556 MWh, wind 214 MWh, outages 143 h
PV+wind+BESS: import 413 MWh, export 42 MWh, curtailed 13.9 MWh, diesel 7.5 MWh in 52 h, renewable
63%
PV+BESS: import 589 MWh, export 21 MWh, curtailed 3.2 MWh, diesel 11.0 MWh in 80 h, renewable 47%
```

The full system meets R3 with 63 % (3 percentage points of margin); without wind it reaches 47 %. Generators run 52 h a year instead of the 143 h of grid outage, because the battery carries the short outages entirely. Figure 25.3 shows one week of dispatch including a 13 h outage.

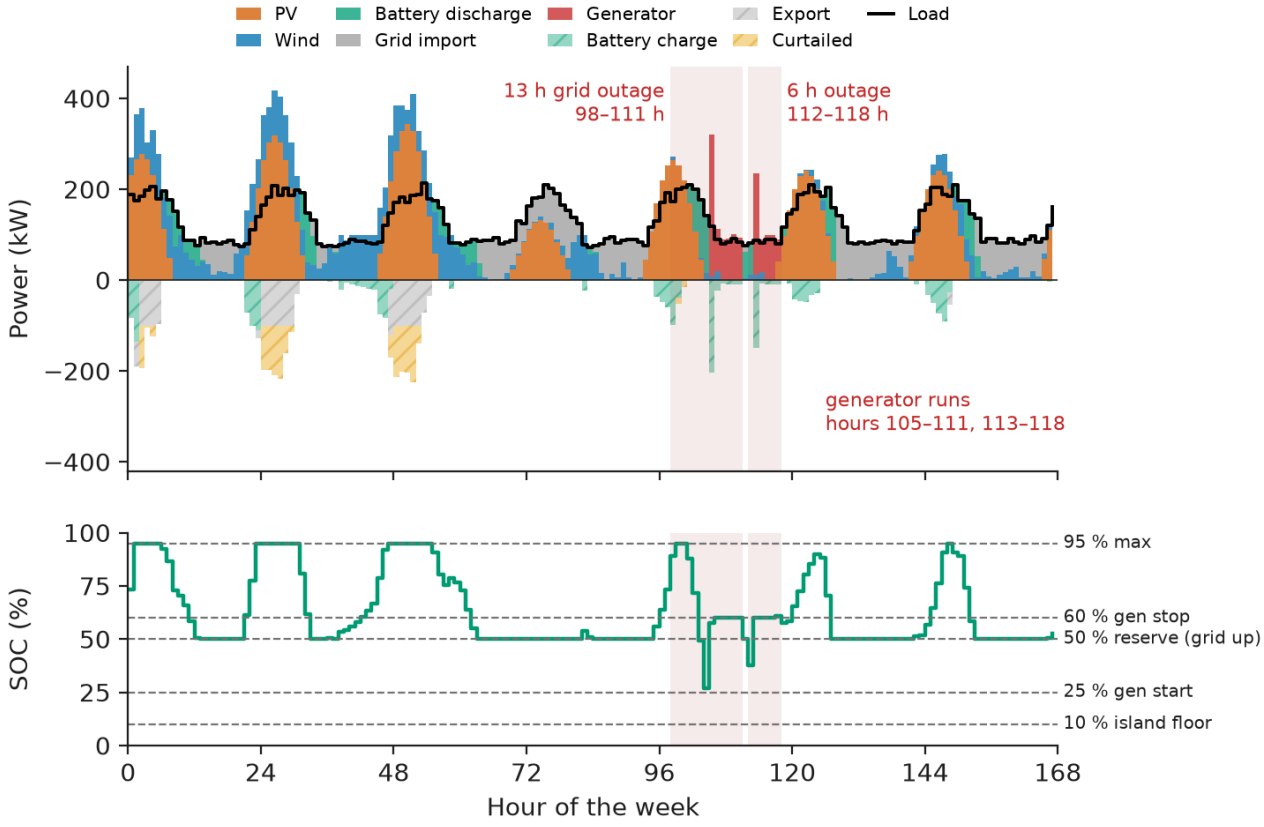


Figure 25.3 — One week of hourly dispatch with a 13 h grid outage, and the battery state of charge.

> **Status of the principle: Rule-based hourly dispatch**
> *Approximate.* Energy balance per hour with constant one-way battery efficiency (0.95) and no forecast; it ignores sub-hourly variability and inverter limits within the hour. A forecast-based optimiser (model-predictive dispatch) typically gains 3–8 % on cost, and is worth it only once the simple rules have been validated against metered data.

What the controller adds to the simulation

The production EMS adds what the hourly model cannot: a day-ahead PV and wind forecast that raises the reserve before forecast storms (when outages are likelier), a demand-limit function, a shed scheme for the non-critical board, and logging of every mode change for the reliability record of Section 25.9. Its communication is not trusted for protection or stability; if the controller fails, every inverter falls back to its local settings (Section 25.7), the battery holds its last reserve setting, and generators revert to their own auto-start on undervoltage.

25.5 Grid-connected operation

Export limitation

The connection agreement allows 100 kW of export (R4). The PCC meter is read by the microgrid controller every second; when export approaches the limit and the battery is full, the controller lowers the active-power

limit of the PV inverters over Modbus (SunSpec models), and a hard backstop in each inverter's own export-control input caps output if communication is lost. The test is to switch off the load bank on a sunny day and record the PCC: export must settle below 100 kW within the agreed time (typically 5–15 s) and the one-minute average must never exceed it.

Worked example — why the network operator limits export. The 25 km feeder of 50 mm² ACSR has about $R = 13.5\ \Omega$ and $X = 9.5\ \Omega$. The voltage rise at the site from an export P with reactive power Q is approximately $\Delta V \approx (PR + QX)/V$ (*Electrical Energy and Machines*, Ch. 22 §22.5). Exporting 100 kW at unity power factor raises the 11 kV feeder end by $100{,}000 \times 13.5/11{,}000 = 123$ V, 1.1 %; 300 kW would raise it 3.3 %, on a feeder end already at about 104 % at light load — over the 110 % statutory limit with tap and measurement tolerances. With the inverters absorbing reactive power at 0.95 power factor ($Q = -32.9$ kvar per 100 kW), the rise at 100 kW falls to 0.86 %. The site therefore runs a Q(V) characteristic per EN 50549-1 in addition to the export limit.

Grid-code functions

In grid-connected mode the PV and wind inverters are ordinary grid-following DER and must comply with the connection standard (EN 50549-1 in Europe, IEEE 1547-2018 in North America): voltage and frequency ride-through, over-frequency P(f) curtailment, Q(V) or fixed power factor, and anti-islanding. The interaction that matters for this design is between anti-islanding and the battery: with a grid-forming battery holding the island at nominal voltage and frequency, the PV inverters' passive anti-islanding cannot see the grid loss. That is intended — the PV must keep running in the island — but it means the **PCC relay**, not the inverters, is responsible for separating the hospital from a dead feeder before the utility recloses (*Electrical Energy and Machines*, Ch. 27 §27.5). The utility's first reclose is at 1–3 s after a feeder trip, so the PCC relay must open within about 0.5 s.

25.6 Islanded operation

The transition

On grid loss the PCC relay sees undervoltage (below 0.8 pu for 50 ms), a vector shift or a frequency excursion, and trips the PCC breaker; the opening takes about 50 ms. Throughout, the grid-forming battery inverter holds its voltage phasor. The critical board sees a voltage dip whose depth depends on the fault on the feeder and whose duration is detection plus breaker time, about 70–100 ms: R1 is met with little margin, which is why the PCC breaker was specified with a fast shunt trip and the relay's undervoltage stage was set at 50 ms rather than the 100 ms default. A planned island (for feeder maintenance) is seamless: the controller first ramps the PCC exchange to zero with the battery, then opens the breaker.

Living as an island

Once islanded, the battery inverter sets frequency and voltage, the PV and wind inverters follow, and the EMS starts a generator if SOC falls to 25 %. Two problems remain that did not exist with the grid connected.

Worked example — starting the chiller in an island. The 55 kW chiller compressor (motor efficiency 0.92, power factor 0.9) draws $55/(0.92 \times 0.9) = 66.4$ kVA running and, through its soft starter, about three times that, 199 kVA, for 5–8 s. With 200 kW of other load at 0.95 power factor (210 kVA) and only the battery online, the inverter would have to supply about 410 kVA, over its 300 kVA continuous rating but

inside its 450 kVA, 10 s overload rating — a 9 % margin that ambient temperature or a fading cell could erase. The controller therefore inhibits the chiller start in an island unless a generator is online or the other load is below 150 kW, and Revision B replaces the soft starter with a VFD that starts the compressor at rated current.

Generator minimum load. A diesel that runs below about 30 % of rating for long periods "wet-stacks": unburnt fuel accumulates in the exhaust. In an island with plenty of PV, a generator that starts at 25 % SOC can find itself almost unloaded an hour later as the sun comes out. The EMS keeps a running generator loaded to at least 48 kW by charging the battery from it, and stops it once SOC passes 60 %; the listing of Section 25.4 implements exactly that floor.

> **Status of the principle: Grid-forming source as the island's reference**
> *Fundamental.* An AC island needs at least one source that sets voltage magnitude and frequency; every grid-following source needs a voltage to synchronise to. Without a grid-forming battery or a running synchronous machine the island cannot exist, whatever the PV produces.

25.7 Inverter coordination

Frequency as the communication channel

The opening failure was a coordination failure, and its fix is the central design in this chapter. In an island the battery inverter is the only source that can absorb surplus, and once it is full, PV and wind must reduce output within seconds. Communication over Modbus is too slow and too fragile for that; instead the battery inverter **raises the island frequency** when it is full, and the PV and wind inverters reduce output along their **frequency-watt** characteristic. EN 50549-1 requires such an over-frequency curtailment (LFSM-O): above a threshold of 50.2 Hz, power is reduced along a droop s (default 5 %), a gradient of $1/(s\,f_n) = 40\,\%$ of reference power per hertz (*Electrical Energy and Machines*, Ch. 27 §27.6).

Worked example — the noon island of the opening paragraph. PV gives 330 kW and wind 60 kW (390 kW together), the load is 110 kW, and the battery is full: 280 kW, or 72 % of renewable output, must be curtailed. With the default 40 %/Hz gradient the frequency must rise $0.72/0.40 = 1.79$ Hz above 50.2 Hz, to 51.99 Hz — beyond the inverters' own 51.5 Hz over-frequency trip, so in practice they trip rather than curtail, and when they trip the island loses all its renewable generation at once. With the droop set to 2 % (100 %/Hz) for island operation, the frequency rises only 0.72 Hz, to 50.92 Hz, inside R5's 51.0 Hz limit. The battery inverter's own frequency setting follows its SOC: nominal 50.0 Hz below 90 % SOC, rising linearly to 51.0 Hz at 98 %. Figure 25.4 shows both characteristics.

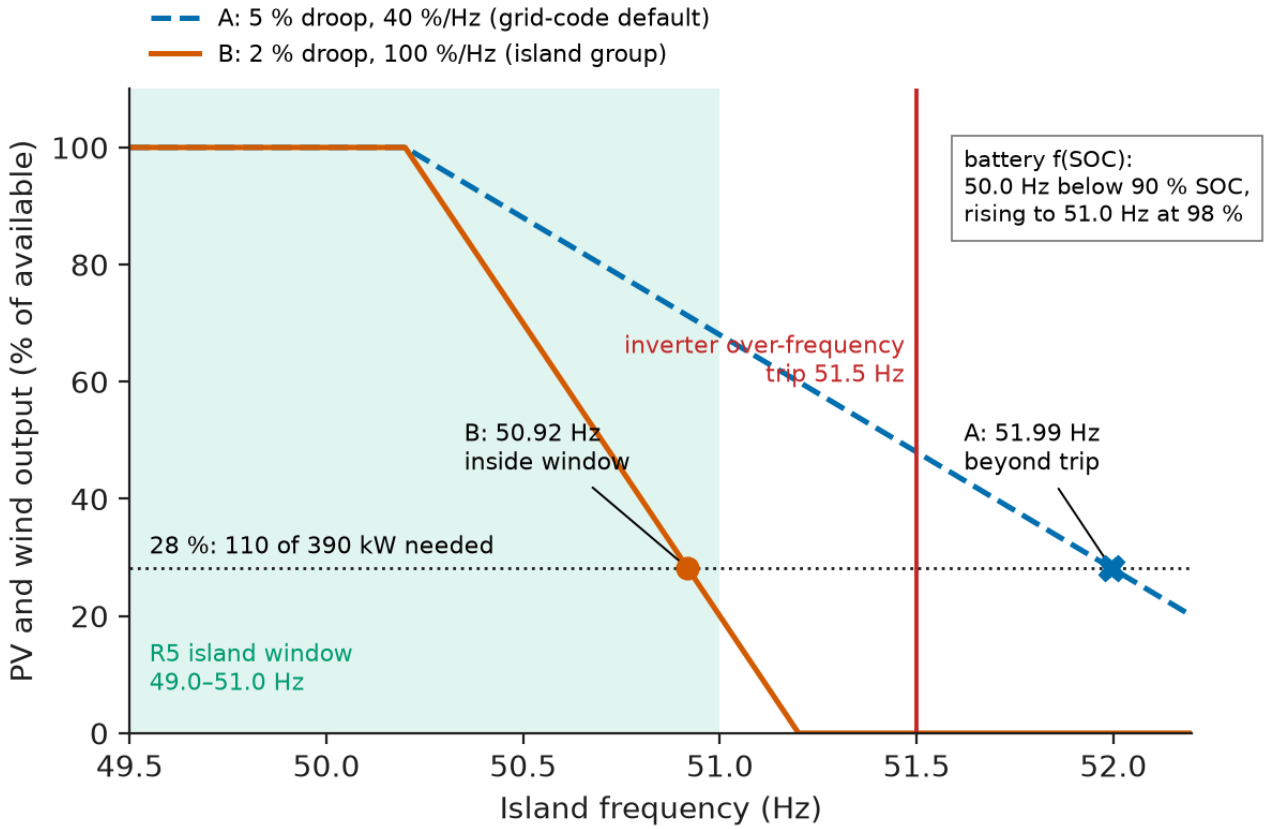


Figure 25.4 — Frequency-watt curtailment of the PV and wind inverters for two droop settings, with the island operating point.

The wind turbines' converters support the same P(f) function. Older small turbines that lack it are given a dump-load controller or a contactor that the microgrid controller opens above 50.8 Hz; the contactor is a coarse but communication-free backstop.

Table 25.6 — Island coordination settings (Revision A versus commissioning fix).

Setting	Revision A (as installed)	Revision A.1 (after the noon event)	Why
PV/wind P(f) threshold	50.2 Hz	50.2 Hz	Grid-code value, kept
PV/wind P(f) droop	5 % (40 %/Hz)	2 % (100 %/Hz) in island setting group	Full curtailment before 51.0 Hz
PV/wind over-frequency trip	51.5 Hz, 0.1 s	51.5 Hz, 0.1 s	Unchanged; curtailment now completes below it
Battery f(SOC)	Fixed 50.0 Hz	50.0 Hz to 51.0 Hz between 90 % and 98 % SOC	Signals "full" to all inverters
Battery charge limit at 98 % SOC	Trip on cell high voltage	Taper to zero, no trip	Battery remains the grid-forming source
Setting-group switch	None	PCC breaker auxiliary contact → island group	Grid-code values when connected

> **Status of the principle: Frequency-signalled curtailment**
> *Derived.* With a single grid-forming source and grid-following inverters on linear P(f) droops, the steady-state frequency follows from the power balance; it neglects the inverters' response time (1–2 s typical), during which the battery must absorb the transient surplus — which is why the battery's charge limit tapers instead of tripping.

Sharing with generators

When a generator runs in parallel with the battery, both are voltage sources, and they share load changes by droop exactly as in *Electrical Energy and Machines*, Ch. 26 §26.4: with the battery on 2 % droop and each 160 kW generator on 4 %, the battery takes $(300/0.02)/(300/0.02 + 160/0.04) = 79\,\%$ of a load step with one generator running. That is the intended split: the battery absorbs fast steps, and the EMS slowly re-dispatches toward the generator so that it loads above its 48 kW floor.

25.8 Protection

The fault-current problem

Protection settings are chosen for the fault current available, and that current changes by a factor of more than 20 between modes (*Electrical Energy and Machines*, Ch. 23 §23.2 and §23.9).

Worked example — fault levels in the two modes. Grid-connected, the 630 kVA transformer with 6 % impedance gives a three-phase fault current at the 400 V busbar of $630{,}000/(\sqrt{3} \times 400 \times 0.06) = 15.2$ kA (upstream impedance neglected). Islanded on the battery alone, the inverter's rated current is $300{,}000/(\sqrt{3} \times 400) = 433$ A and it limits its fault current to 1.5 pu, 650 A, for 2 s. Each 200 kVA generator (rated current 289 A) contributes $289/0.15 = 1{,}925$ A subtransient and about 3 pu (866 A) sustained with its permanent-magnet excitation support. So the busbar fault level is 15.2 kA connected, 0.65 kA islanded with the battery only and about 2.4 kA with both generators — 23 times lower than connected in the worst case.

Consider a 125 A outgoing circuit to the laundry protected by an MCCB with a thermal-magnetic trip whose instantaneous element is at 10 × In = 1,250 A. Connected, a fault draws kiloamperes and clears in one cycle. Islanded on the battery, the 650 A fault is only 5.2 × In: the instantaneous element does not operate, the thermal element takes 10–20 s, and the inverter's 2 s current limit ends first — the battery trips, and the whole island blacks out for a fault on one non-critical circuit. Figure 25.5 shows the mismatch.

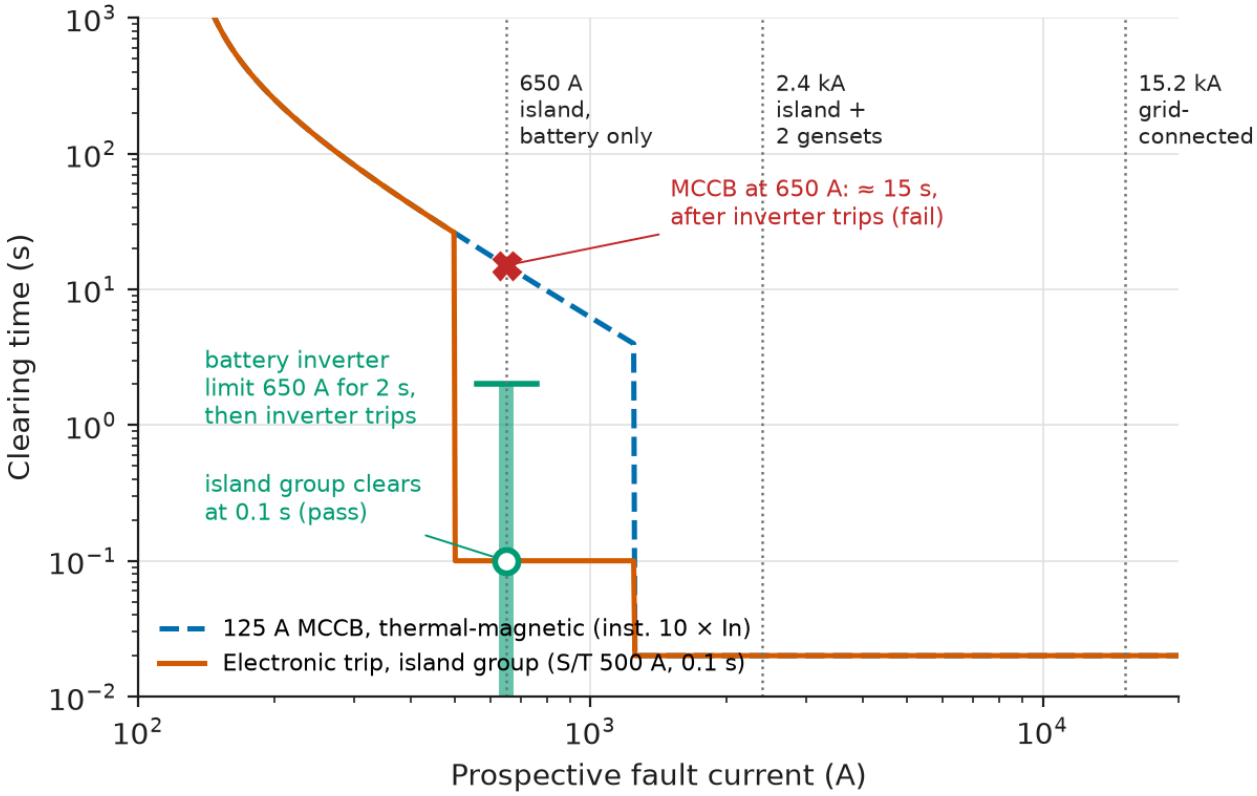


Figure 25.5 — Time-current curves of a 125 A feeder breaker against the fault current available in each operating mode.

Table 25.7 — Protection options for the islanded case.

Option	How it works	Cost	Weakness	Choice
Electronic trip units with two setting groups	Short-time pickup lowered to 4 × In (500 A), 100 ms, in island group; switched by PCC breaker auxiliary contact	Replace 14 MCCB trip units	Group switching must be tested; a wrong group is a hidden failure	**Chosen** for outgoing feeders
Higher inverter fault current	Specify 2 pu (866 A) for 2 s	Inverter oversizing, about 10 %	Still below many instantaneous settings	Specified as well
Undervoltage-supervised overcurrent at the busbar	Trip feeders on low voltage plus overcurrent	Relays and CTs	Coordination with motor starting dips	Kept as backup
Differential protection on key feeders	Current difference, independent of fault level	High	Only for a few feeders	Rejected for this site

Final circuits with miniature circuit breakers and RCDs are a separate problem: a type B MCB needs 3–5 × In to trip magnetically, so a 32 A circuit trips instantaneously at 160 A, well within the islanded fault current. The disconnection-time requirement of R6 (0.4 s) is checked for every final circuit with the islanded source impedance; the RCDs on socket circuits provide shock protection independent of fault level.

> **Status of the principle: Mode-dependent fault current of inverter-dominated islands**
> *Empirical.* Inverters limit fault current by control to 1.1–2 pu, set by the manufacturer and not by physics of the source; the protection study must use the specific inverter's fault-current profile from its type test, not a generic machine model.

25.9 Reliability

The hospital's objective is not energy but continuity of the critical load. Its reliability is computed for the two failure paths that matter: the transfer at the moment of grid loss, and the long outage that outlasts both the battery reserve and the generators.

Worked example — interruptions of the critical load per year. With 40 grid outages a year, the old system (generators and UPS only) gives 40 short interruptions of about 10 s at the main boards (the UPS hides them from theatres and ICU only). A long interruption occurs if both generators fail to start: with a failure-to-start probability $p = 0.03$ per demand for each set and a common-cause fraction $\beta = 0.1$ (shared fuel, shared battery charger, shared control panel), the probability that both fail is $\beta p + ((1-\beta)p)^2 = 0.0037$, about 0.15 long interruptions a year. With the battery, the short interruptions disappear, and a long one needs both generators to fail *and* the outage to outlast the 2.67 h of reserve autonomy. With exponentially distributed outage durations of mean 3 h, $P(D > 2.67\ \text{h}) = e^{-2.67/3} = 0.41$, so the rate is $40 \times 0.0037 \times 0.41 = 0.061$ per year, once every 16 years, against once every 7 years before. The small 300 kWh battery would give only 1.33 h of autonomy and 0.096 per year — and would fail R2.

> **Status of the principle: Interruption rate from independent and common-cause failures**
> *Approximate.* It assumes Poisson outage arrivals, exponential durations and a β-factor common-cause model (*The Mathematics Universe*, Ch. 19); the β value of 0.1 is a typical engineering estimate, and real outage durations have heavier tails than the exponential, so the result is a comparison of designs, not a prediction.

Two further reliability rules came out of the review. The microgrid controller is a single point of coordination but must never be a single point of failure: every fast function runs locally (Section 25.7). And the battery inverter is now the most critical device on site, so a spare power module and a service contract with 48 h response are part of the design, along with a bypass that lets the hospital fall back to the original generator-only scheme.

25.10 Economic evaluation

The investment case compares each option with the **baseline**: all energy imported at \$0.18/kWh, and the existing generators supplying the load during outages at 0.30 L/kWh and \$1.10/L. Installed costs (2026, indicative for this market) are \$700 per kWp of PV, \$3,500 per kW of small wind, \$350 per kWh of battery plus \$150 per kW of inverter, and \$80,000 of switchgear, controls and protection. Exports earn \$0.05/kWh. The battery cells are replaced in year 12 at 45 % of today's cost; savings fade 0.5 % a year with PV and battery degradation; the discount rate is 8 % over 20 years.

```
 import numpy as np
from site_profiles import profiles
from ems import dispatch

load, pv, wind, outage = profiles()
TARIFF, EXPORT, DIESEL = 0.18, 0.05, 1.10*0.30           # $/kWh import, export, genset fuel
base = load[~outage].sum()*TARIFF + load[outage].sum()*DIESEL   # grid plus existing gensets

def npv(capex, opex, w, E=600, P=300, life=20, rate=0.08, fade=0.005):
    r = dispatch(load, pv, w, outage, E=E, P=P)
    saving = base - (r['imp']*TARIFF - r['exp']*EXPORT + r['gen']*DIESEL) - opex
    yrs = np.arange(1, life + 1)
    cash = saving*(1 - fade)**(yrs - 1) - np.where(yrs == 12, 0.45*350*E, 0)  # cells replaced yr
12
    disc = cash/(1 + rate)**yrs
    ren = 1 - (r['imp'] + r['gen'])/load.sum()
    return saving, disc.sum() - capex, np.cumsum(disc) - capex, ren

PV, WIND, BOS = 400*700, 100*3500, 80e3                   # 2026 installed costs, $
BESS = lambda E, P: 350*E + 150*P
print(f"baseline cost ${base/1e3:.1f}k/yr")
for name, capex, opex, w, E, P in [
        ("PV+wind+BESS 600 kWh", PV + WIND + BESS(600, 300) + BOS, 13.8e3, wind, 600, 300),
        ("PV+BESS 600 kWh",      PV + BESS(600, 300) + BOS,        7.8e3, 0*wind, 600, 300),
        ("PV+BESS 300 kWh",      PV + BESS(300, 150) + BOS,        6.3e3, 0*wind, 300, 150)]:
    s, v, cum, ren = npv(capex, opex, w, E, P)
    pay = np.argmax(cum >= 0) + 1 if cum[-1] >= 0 else None
    print(f"{name:21s} capex ${capex/1e3:5.0f}k  saving ${s/1e3:5.1f}k/yr  NPV ${v/1e3:6.1f}k"
          f"  discounted payback {pay} yr  renewable {ren:.0%}")
```

```
 baseline cost $206.1k/yr
PV+wind+BESS 600 kWh  capex $  965k  saving $117.7k/yr  NPV $ 113.5k  discounted payback 16 yr
renewable 63%
PV+BESS 600 kWh       capex $  615k  saving $ 89.7k/yr  NPV $ 197.6k  discounted payback 11 yr
renewable 47%
PV+BESS 300 kWh       capex $  488k  saving $ 89.0k/yr  NPV $ 337.8k  discounted payback 8 yr
renewable 45%
```

All three options pass R9. The cheapest is the most profitable, and fails R2 and R3; the chosen design is the least profitable that passes everything. Table 25.8 turns that into a statement of what each requirement costs, which is the form in which a funder or a hospital board can decide.

Table 25.8 — Options against requirements, and the price of each requirement.

Option	R2 autonomy (≥ 2.5 h)	R3 renewable (≥ 60 %)	NPV at 8 %	Price of the step
PV + 300 kWh battery	1.33 h — fails	45 % — fails	$337,800	—
PV + 600 kWh battery	2.67 h — passes	47 % — fails	$197,600	R2 costs $140,200 of NPV
PV + wind + 600 kWh battery	2.67 h — passes	63 % — passes	$113,500	R3 costs a further $84,100

Worked example — the levelised cost of the hybrid's energy. The capital recovery factor at 8 % over 20 years is $0.08 \times 1.08^{20}/(1.08^{20} - 1) = 0.1019$. The chosen system's $965,000 therefore costs $98,300 a year, plus $13,800 of operation and maintenance: $112,100 a year. It supplies $1{,}128 - 413 - 7.5 = 707$ MWh of the load, so its levelised cost is $0.159/kWh, below the $0.18/kWh tariff — which is why the NPV is positive despite the wind turbines; rerunning the model shows that a tariff about 10 % lower ($0.163/kWh) would turn it negative. Figure 25.6 shows the cumulative discounted cash flow of the three options.

> **Status of the principle: Net present value and levelised cost**
> *Derived.* Both are discounted sums of cash flows; they are exact for their inputs, and the inputs (tariff path, outage rate, degradation, replacement cost) are forecasts, so the decision is checked with a sensitivity sweep (Chapter 30) rather than a single number.

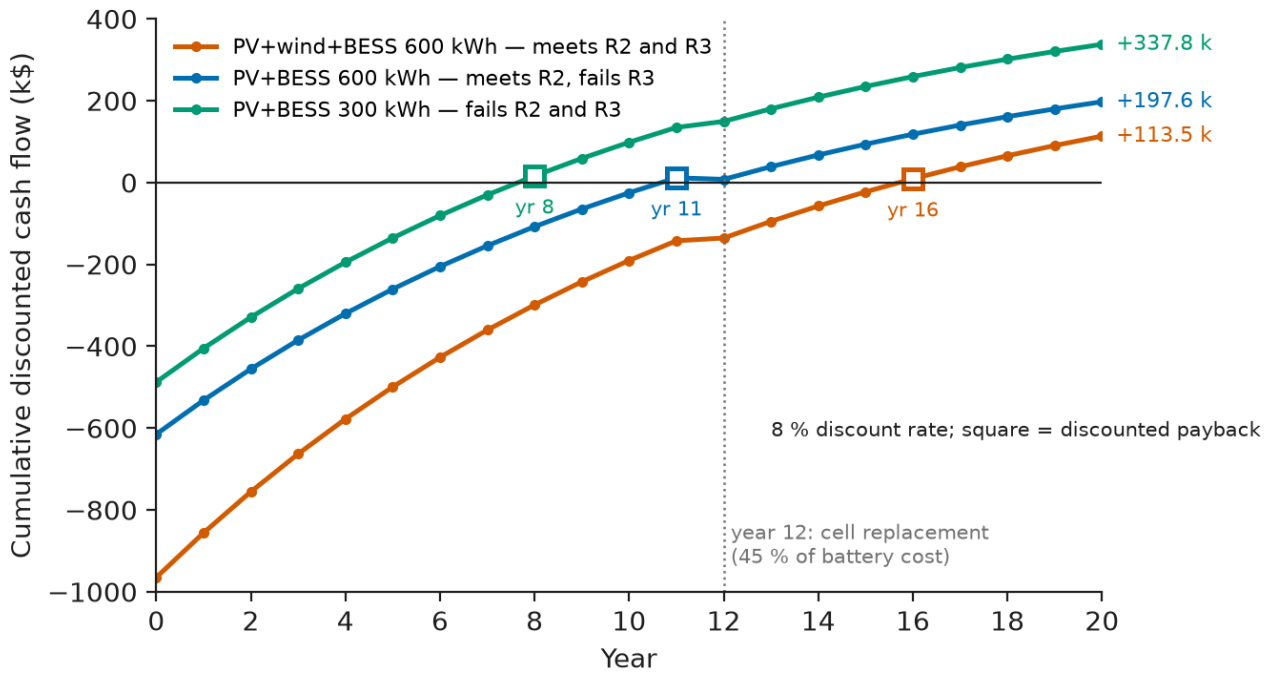


Figure 25.6 — Cumulative discounted cash flow of three microgrid options over 20 years at 8 percent.

25.11 Verification, failure analysis and Revision B

Verification

Table 25.9 lists the site acceptance tests. Simulation results are marked; all others are from the commissioning test programme after the Revision A.1 settings change, with illustrative values.

Table 25.9 — Verification of the hospital microgrid.

Req.	Test	Limit	Result	Pass/fail
R1	Unplanned PCC trip at 180 kW load, recorder at critical board	< 70 % for ≤ 100 ms	Dip to 41 % for 78 ms	Pass
R2	Island on battery from 50 % SOC, generators inhibited, critical load 90 kW	≥ 2.5 h to 10 % SOC	2.6 h	Pass (thin margin)
R3	Hourly simulation (first metered year pending)	≥ 60 %	63 % (simulated)	Pass, to confirm
R4	Load bank off, sunny noon, battery full	≤ 100 kW, 1 min average	96 kW peak 1 min average	Pass
R5	100 kW load-bank step in island, battery only	Nadir ≥ 49.0 Hz	49.62 Hz	Pass
R5	Noon island, battery at 97 % SOC, PV 320 kW, load 115 kW	49.0–51.0 Hz	50.81 Hz, no trip	Pass
R6	Secondary injection of feeder trip units in island group	≤ 0.1 s at 500 A	0.08 s	Pass
R7	Settings audit against EN 50549-1 certificate	All functions enabled	Island setting group verified	Pass
R8	Reclosure after island, synchro-check log	$\Delta f \le 0.1$ Hz, $\Delta V \le 5$ %, $\Delta\varphi \le 10°$	0.04 Hz, 1.8 %, 6°	Pass
R9	Cash-flow model	NPV ≥ 0	$113,500 (modelled)	Pass

Failure analysis

Table 25.10 — FMEA extract for the hospital microgrid.

Failure mode	Effect	Cause	Detection	Mitigation
Island collapse on surplus (the opening event)	Blackout, back to generators and UPS	PV P(f) droop too shallow; battery trips at full	Event log: frequency fixed at 50.0 Hz, battery cell over-voltage trip	f(SOC) on battery, 2 % island droop, charge taper (Table 25.6)
Feeder fault not cleared in island	Battery current-limit trip, blackout	MCCB instantaneous above island fault level	Protection study; injection test	Adaptive setting groups; 2 pu inverter fault current
Wrong setting group after reconnection	Feeder protection too sensitive when connected, nuisance trips	Auxiliary contact failure	Group status monitored by controller	Watchdog compares group with PCC state; alarm
Utility recloses onto live island	Out-of-phase reclose, equipment damage	PCC relay slow or failed	Relay self-test	PCC opens in ≤ 0.5 s; trip-circuit supervision; interlock with utility recloser where possible
Generator wet-stacking	Fouling, failure to start later	Long low-load running	Generator hour and load log	48 kW floor by battery charging
Island overload on chiller start	Battery overload trip	Soft-start inrush 199 kVA	Overload event log	Start inhibit in island; VFD (Revision B)
Controller failure	Loss of export limit and dispatch	Hardware or software fault	Heartbeat to each inverter	Local fallbacks: inverter export cap, last reserve, generator auto-start
Battery inverter failure	No seamless island	Power module fault	Self-diagnostics	Spare module, 48 h service, generator-only bypass

Revision B

Change	Reason	Effect
Battery inverter specified as VSM with 2 pu fault current for 2 s	Thin protection margin; generator paralleling	Simpler protection grading; smoother sharing
Chiller soft starter replaced by VFD	9 % overload margin on island start	Start at rated current; inhibit removed
Reserve SOC raised from 50 to 55 % with a forecast-based storm reserve of 70 %	R2 measured at 2.6 h, margin too thin as cells fade	Autonomy 3.0 h at year 1; costs about 1 % renewable fraction
Model-predictive dispatch with day-ahead forecasts	3–8 % cost gain; storm reserve	Requires validated forecasts; after one metered year
Island test repeated annually at noon with full battery	The opening failure was untested	Regression test of coordination settings
Depot-style EV charging for the ambulances on the EMS demand limit	New load (Chapter 24)	Charging only from surplus or off-peak

What remains open

Open question	Where it is answered
How is the PV array itself sized, strung and protected?	Chapter 22
How is the battery pack and its BMS designed, and how is its capacity fade tracked?	Chapter 23
How does a small off-grid PV–battery system apply the same sizing end to end?	Chapter 36
How do the controller, inverters and relays communicate, and how is that network secured?	Chapter 21; *Electrical Energy and Machines*, Ch. 26 §26.9
How are the economic inputs swept and the dispatch optimised?	Chapter 30
How is the microgrid's model calibrated against metered data and used as a digital twin?	Chapter 33
How do grid-forming inverter controls work internally, and what limits their stability?	*Electrical Energy and Machines*, Ch. 27; *Electronics and Computing*, Ch. 32 §32.9

Next: Chapter 26 turns from power to information: the RF front ends, antennas and link budgets that let a microgrid controller, a sensor network or a fleet of meters talk over the air.

Part VII — Communications, Signal Processing, and Biomedical Applications

Parts II–VI built systems that measure, compute, move and convert energy. This part follows the *information* they produce: across the air from a node to a gateway, through filters and detectors on an embedded processor, and from the most demanding measurement chain of all, one connected to a human body. The three chapters share one idea. A signal arrives with a budget against it: path loss and fading on a radio link, noise and interference in a sensor channel, electrode offset, mains pickup and leakage current on a patient. Each chapter writes that budget down per stage, closes it with a stated margin, and tests it in the configuration the system will really be used in.

By the end of this part you should understand

- How an RF front end is assembled around a transceiver IC — PA, filters, switch, LNA, reference oscillator, antenna — and how each block's loss or noise appears in the link budget.
- Why antennas are chosen for the installation rather than the data sheet.
- How a link budget is built from a measured path-loss model, how a fade margin follows from the shadowing statistics, and how spreading factor, airtime, regulatory duty cycle and battery life trade against each other.
- How a wireless sensor network is chosen, how collisions and retries set its reliability, and how co-sited transmitters are kept from blocking a receiver.
- How a sensor's noise is reduced before and after the converter, how filters, spectra and features are chosen for a specific physical mechanism, and how a detector is specified by its detection probability and false-alarm rate.
- How a DSP algorithm, including audio and image processing, is moved to an embedded processor with a latency budget and a bit-exact reference model.
- How biopotentials such as ECG and EMG are acquired through electrodes, instrumentation amplifiers and driven-right-leg circuits, how mains interference and motion artefact are rejected, and how basic features (heart rate, QRS detection, muscle activation) are extracted.
- How patient isolation, leakage-current limits and the IEC 60601 family shape a biomedical front end, and what regulatory and ethical obligations come with data taken from people.

Chapters

Chapter	Central question	What it delivers
26 — Communication-System Integration	How is a wireless link designed so that its margin is known at every site, in the worst season and location, before installation?	RF front-end and gateway architecture, antenna selection, SX126x-class transceiver interface, survey-based link budgets, EN 300 220 constraints and LoRa airtime, network choice, reliability and retries, blocking and filtering, a radio test plan; worked through a 36-site water-utility telemetry network
27 — Practical Signal Processing	How does a noisy sensor signal become a reliable decision on an embedded processor?	Noise budgets, filter chains, spectra, features tied to fault mechanisms, detectors with stated false-alarm rates, audio and image processing, real-time embedded DSP and performance evaluation; worked through pump vibration monitoring
28 — Biomedical Instrumentation	How is a biopotential measured accurately and safely from a patient?	Biopotential and electrode behaviour, ECG and EMG acquisition chains, biomedical sensors, isolation and leakage limits, signal conditioning and noise rejection, medical data acquisition, basic signal analysis, and the regulatory and ethical framework

How this part connects to the rest of the book

Chapter 26 sits under the IoT systems of Chapter 11, which own the layers above the radio (MQTT, provisioning, security), and it feeds the complete wireless sensor network project of Chapter 38. Chapter 27 takes the measurement chains of Chapters 7 and 8 (signal conditioning, ADC selection, data acquisition) and adds the processing; its features and detectors are the inputs to the machine-learning methods of Chapter 32 and the edge inference of Chapter 12, and its embedded DSP runs inside the energy monitor of Chapter 35. Chapter 28 applies the sensor and conditioning methods of Chapters 6 and 7 to the human body, uses Chapter 27's filtering and detection on biopotentials, and adds the safety discipline that no other application in the book demands to the same degree.

Among the sibling books, *Circuits, Signals and Communications* owns the theory this part relies on: sampling and reconstruction (Ch. 19), the DFT and digital filters (Ch. 22–23), noise and noise figure (Ch. 29), propagation and link budgets (Ch. 30) and RF components and antennas (Ch. 32). *Electronics and Computing* owns the instrumentation amplifiers and converters (Ch. 8 and Ch. 10) and the embedded interfaces and real-time scheduling behind every node and DSP loop (Ch. 26–28). *Electrical Energy and Machines* supplies electrical-safety principles (Ch. 29) that Chapter 28 tightens for patients,, and *Materials Science and Engineering* the biomaterials at the skin interface (Ch. 39).

Chapter 26 — Communication-System Integration

Part VII · Communications, Signal Processing, and Biomedical Applications

A rural water utility replaces the dial-up telemetry of its pumping stations, reservoirs and valve chambers with a private 868 MHz LoRaWAN network: 36 nodes within 8 km of one gateway on the 30 m water tower at the treatment works. Commissioning goes well: every node joins, every one reports, and the radio contractor records a received signal strength at each site. Six months later a third of the valve-chamber nodes drop out every wet week, the network server shows them stuck at spreading factor 12 and retransmitting, and the first battery, specified for ten years, is flat after two and a half. Then the mobile operator that leases space on the same tower turns up its 800 MHz LTE cell, and the gateway's noise floor rises 9 dB in the evenings. The radios were good parts. The commissioning measurements were taken with the chamber lids open, the gateway sat at the bottom of 30 m of coaxial cable, nobody had planned for a strong transmitter 50 MHz away, and the link budget had no line for any of them. This chapter designs that network as a radio system: front end, antenna, transceiver interface, link budget, channel rules, network, reliability, interference and test.

> **How is a wireless link designed so that its margin is known before installation, at every site, in the worst season and the worst location?**
> **Which choices in the front end, antenna, gateway placement and protocol settle the trade between range, battery life, latency and reliability, and how is each one tested?**

Concept	Engineering destination
RF front-end architecture	PA, LNA, switch, filters and reference oscillator around a transceiver IC; harmonic compliance
Antenna selection	Node antennas near metal and underground; gateway gain versus beamwidth
Transceiver interfaces	SPI command sets, BUSY and IRQ lines, TCXO and RF-switch control, timeouts
Link budgets	Received power, sensitivity, fade margin; spreading factor per site
Channel constraints	Regulatory power and duty cycle; time on air; downlink budget
Wireless sensor networks	Star versus mesh; LoRaWAN, NB-IoT and 802.15.4 trade-offs
Communication reliability	Packet error, collisions, retries, confirmed messages, end-to-end monitoring
Interference mitigation	Blocking by co-sited transmitters; filters, isolation, channel plans
System testing	Conducted and over-the-air tests, site surveys, regression after changes

Roadmap. Requirements, network and front end (26.1) → antennas (26.2) → the transceiver's interface to the MCU (26.3) → link budgets per site (26.4) → regulatory and airtime constraints (26.5) → the network as a whole (26.6) → reliability (26.7) → interference (26.8) → testing, failure analysis and Revision B (26.9).

26.1 RF front-end architecture

> **Boundary.**
> Noise figure and the Friis cascade are in *Circuits, Signals and Communications*, Ch. 29 §29.5; path loss, shadowing and link budgets in Ch. 30 §30.2 and §30.10; matching, S-parameters, antennas and RF measurement in Ch. 32; error-detecting and error-correcting codes in Ch. 28. The IoT layer above the radio (MQTT, provisioning, security) is this book's Chapter 11, and the complete battery node is the project of Chapter 38. This chapter owns the radio link.

Requirements

Table 26.1 — Requirements for the water-utility telemetry network.

ID	Requirement	Target	Verification
R1	Routine data delivery	≥ 99 % of 15 min reports reach SCADA, per node per month	Frame-counter gap statistics on the network server
R2	Alarm delivery	Pump-station alarm at SCADA within 30 s with ≥ 99.9 % probability	Injected alarms, 1,000 trials over a month
R3	Coverage	Every site (≤ 8 km) with margin ≥ $1.645\sigma + 3$ dB at its spreading factor	Link budget, then site measurement with lids closed
R4	Battery life of chamber nodes	≥ 10 years on one Li-$SOCl_2$ C cell (6.8 Ah usable)	Energy model, then current-profile measurement
R5	Regulatory	EN 300 220 / RED: 868.0–868.6 MHz, ≤ 25 mW ERP, ≤ 1 % duty; spurious ≤ −36 dBm below 1 GHz, ≤ −30 dBm above	Conducted tests at an accredited lab; module certificate
R6	Gateway robustness	Noise-floor rise ≤ 1 dB with co-sited LTE 800 at full load	Gateway noise-floor log, LTE load windows
R7	Security	Per-device AES-128 keys; no shared keys	Provisioning audit (Chapter 11)
R8	Environment	Chamber nodes IP68, −20 to +50 °C	Type test

R3's margin is a 95 % location probability at the edge of each site's range against log-normal shadowing of standard deviation σ (Section 26.4), plus 3 dB for seasonal foliage and wet ground.

Network architecture

Figure 26.1 shows the system. Nodes transmit LoRa uplinks on three 125 kHz channels at 868.1, 868.3 and 868.5 MHz. The gateway, an eight-channel concentrator (SX1302 class) with GPS timing, forwards every packet over the utility's private fibre to an on-premises LoRaWAN network server (ChirpStack class), which de-duplicates, decrypts and publishes to the SCADA historian over MQTT. Downlinks (acknowledgements, configuration) return the same way.

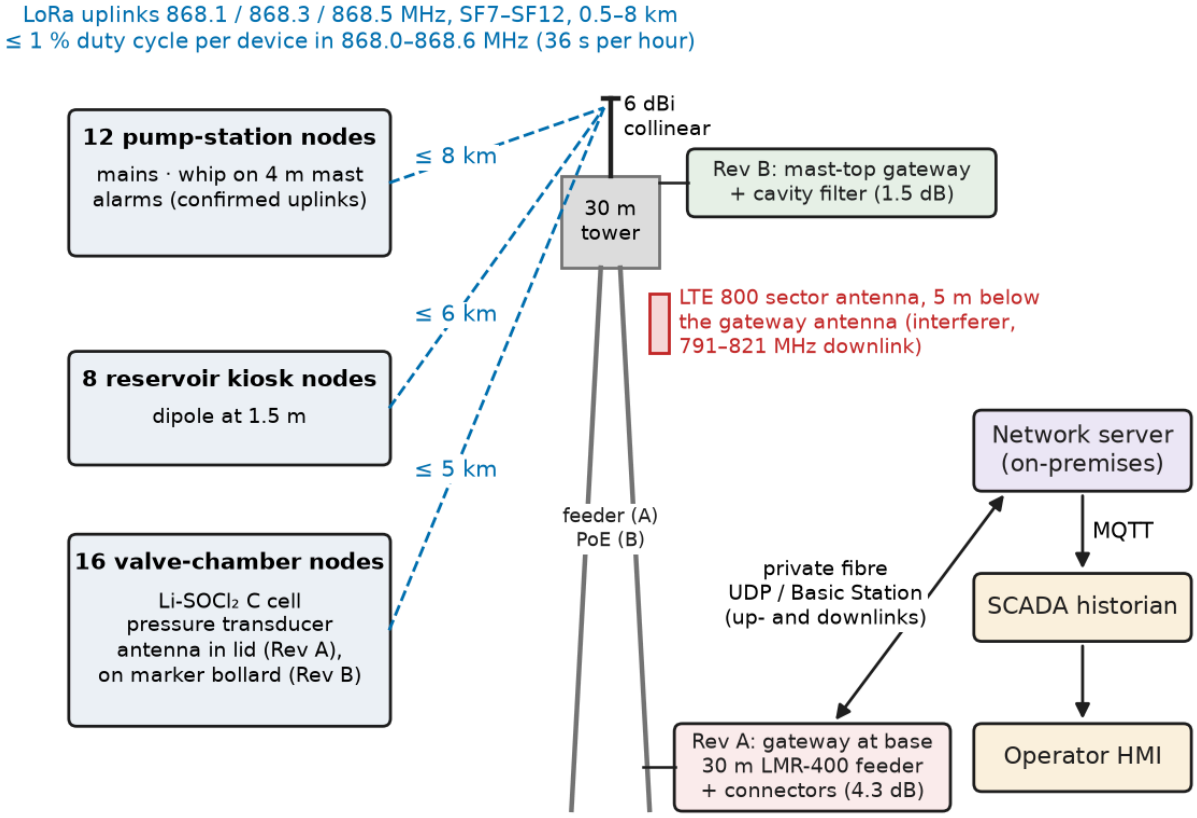


Figure 26.1 — Architecture of the 868 MHz LoRaWAN telemetry network of the water utility.

The node front end

Every node is built around a sub-GHz LoRa transceiver (SX1262 class, either discrete or inside an STM32WLE5-class system-on-chip). The integration work around it is a short list of blocks, each with a number (Figure 26.2):

- **PA output network.** The SX1262's high-power PA can deliver +22 dBm; the regulatory limit in the 868.0–868.6 MHz sub-band is 25 mW ERP (+14 dBm ERP, +16.15 dBm EIRP), so it is run at +14 dBm conducted into a 2 dBi antenna. The PA is matched by the vendor's reference network, which is reproduced exactly, including the PCB stack-up, because its harmonic suppression depends on parasitics.
- **Harmonic low-pass filter.** At +14 dBm the raw second harmonic at 1,736 MHz is about −25 dBc, −11 dBm, against an EN 300 220 limit of −30 dBm above 1 GHz. A fifth-order LC low-pass with at least 35 dB rejection at 1.7 GHz brings it to −46 dBm, 16 dB under the limit.
- **RF switch and receive path.** An SPDT switch (or the vendor's direct-tie topology) connects the antenna to the PA or to the differential LNA input through a balun and matching. Its insertion loss (0.3–0.5 dB) adds directly to the node's noise figure.
- **Reference oscillator.** A 32 MHz TCXO (±2 ppm) rather than a crystal: at SF11 and SF12 a packet lasts more than a second, and a crystal's drift as a chamber node's PA heats it can exceed what the demodulator tolerates within one packet.
- **Antenna port.** U.FL or SMA to the antenna (Section 26.2), with an ESD diode rated for the RF frequency.

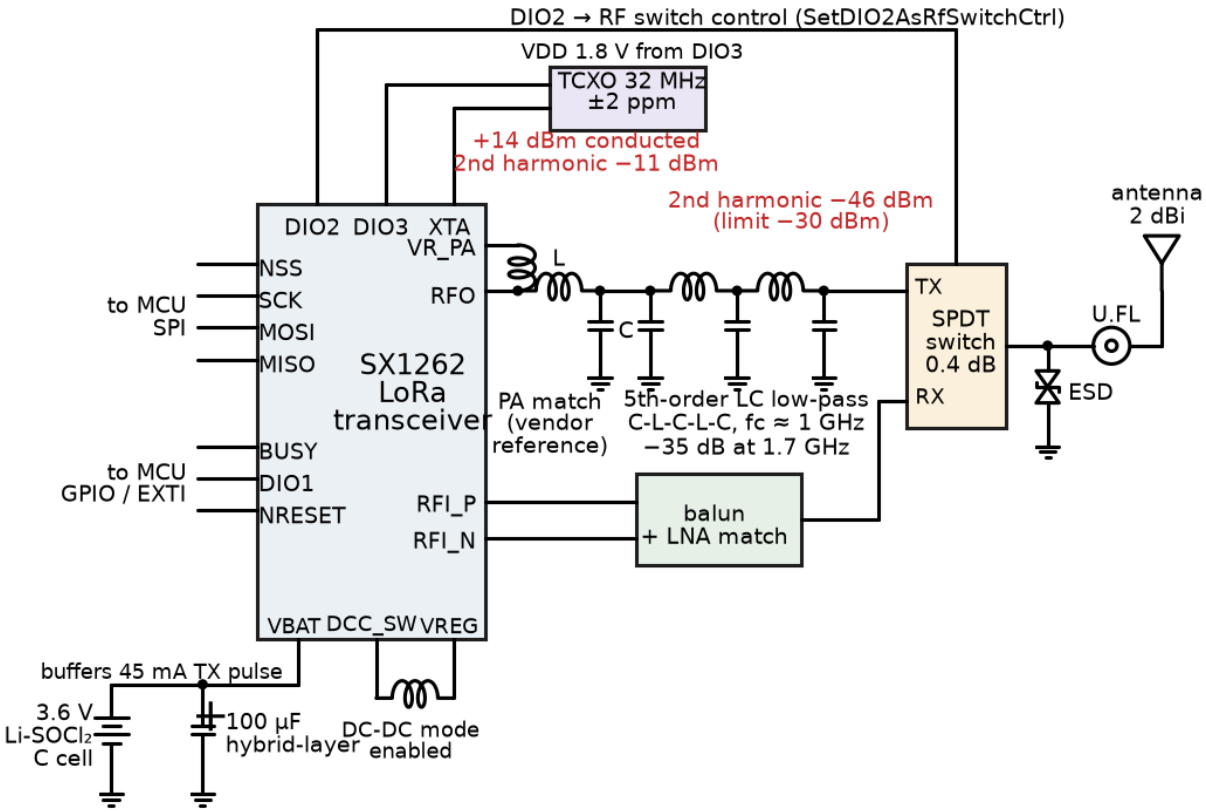


Figure 26.2 — RF front end of the battery node around an SX1262-class transceiver.

The gateway front end

The gateway's sensitivity is set by its noise figure, and anything lossy in front of its LNA adds to it decibel for decibel (*Circuits, Signals and Communications*, Ch. 29 §29.5). In Revision A the gateway sat in the tower's equipment room at the base, at the bottom of 30 m of LMR-400 coaxial cable (3.8 dB at 868 MHz) plus 0.5 dB of connectors and lightning protector: 4.3 dB of loss before a receiver with a 6 dB noise figure gives a system noise figure of 10.3 dB. Revision B moves the gateway to the top of the tower in an IP67 enclosure with a 0.5 dB jumper and a 1.0 dB cavity filter (Section 26.8): 1.5 dB, a 2.8 dB gain in every uplink budget, bought with a PoE cable up the tower.

Table 26.2 — Gateway placement options.

Option	Loss before LNA	System NF	Maintenance	Lightning exposure	Choice
Indoor gateway, 30 m LMR-400 feeder	4.3 dB	10.3 dB	Easy, at ground level	Low (arrester at entry)	Revision A
Indoor gateway, mast-head LNA	0.5 dB + LNA	about 7 dB	LNA needs DC on the feeder	Medium	Considered
Outdoor mast-top gateway, cavity filter	1.5 dB	7.5 dB	Climb to service	High: PoE surge protection at both ends	**Revision B**

26.2 Antenna selection

Node antennas

A node antenna is chosen for the environment it will sit in, not for its data-sheet gain, which is measured on a large ground plane in free space. Near metal, inside an enclosure or below ground, its realised gain can fall by 5–25 dB (*Circuits, Signals and Communications*, Ch. 32 §32.7–32.9).

Table 26.3 — Node antenna options.

Antenna	Data-sheet gain	Realised gain in use	Where it fits	Choice
Quarter-wave whip on enclosure ground plane	2 dBi	0–2 dBi on a mast	Pump-station roof mast at 4 m	Pump stations
Half-wave dipole, fibreglass radome	2.15 dBi	1–2 dBi clear of metal	Reservoir kiosks at 1.5 m	Reservoirs
Helical stub on the node	−1 to 0 dBi	−5 to −10 dBi under a steel lid	In-chamber, no installation work	Revision A (chamber)
Dipole in a composite lid or on a marker bollard, 2 m cable	2.15 dBi	0.5 dBi net of 1.5 dB cable	Chambers on footpaths and verges	**Revision B** (chamber)

The valve-chamber problem dominates the design. A concrete chamber 1 m deep with a cast-iron or steel lid adds a median 20–25 dB of loss with the lid closed and only about 5 dB with it open, and the loss rises further when the chamber floods or the lid is wet. The in-lid helical stub of Revision A was chosen to avoid civil works; Section 26.4 shows that it could not meet R3 at any spreading factor beyond about 2 km.

Gateway antenna

The gateway antenna is a vertical fibreglass collinear. Higher gain is obtained by narrowing the vertical beam, so the choice is between reach at the horizon and coverage of nodes close to the tower.

Worked example — gain against elevation angle. The antenna is 28.5 m above the nodes. A node at 500 m is $\arctan(28.5/500) = 3.3^\circ$ below the horizon, one at 200 m is 8.1° below. A 3 dBi collinear has a half-power vertical beamwidth of about 30°, a 6 dBi about 16° (±8°), a 9 dBi about 8° (±4°). With 9 dBi the close-in node at 200 m sits outside the half-power beam and sees less than the 6 dBi antenna would give it; it does not need the gain, but a node at 300 m in a chamber does. The 6 dBi antenna covers every node beyond about 205 m within its half-power beam (the 200 m node, at 8.1°, sits just outside it and needs no gain) and is chosen; the gain difference matters only at the edge, where Revision B's 2.8 dB feeder saving gives more than a 9 dBi antenna would, without the down-tilt problem.

> **Status of the principle: Realised antenna gain near obstacles**
> *Empirical.* Data-sheet gain is a free-space, large-ground-plane value; the gain realised next to metal, in an enclosure or underground depends on the installation and is measured on site (or in an anechoic chamber with the real enclosure), never assumed.

26.3 Transceiver interfaces

Choosing the node's radio and MCU

Table 26.4 — Node radio and MCU options.

Option	Sleep current	Interface work	Certification	Choice
STM32WLE5-class SoC (LoRa radio and Cortex-M4 on one die), as a pre-certified module	1–2 µA with RTC	Radio behind an internal SPI with the same command set	Module RED certificate reusable	**Battery chamber nodes**
Discrete SX1262 plus STM32L4-class MCU	1–2 µA	External SPI, BUSY, DIO1, NRESET, RF switch, TCXO	Full radio test of own design	Pump stations (mains, needs more I/O)
SX1262 plus ESP32-S3	8–20 µA	As discrete, plus Wi-Fi coexistence	Two radios to test	Rejected for battery nodes
NB-IoT module (nRF9160 class)	3–5 µA in PSM	AT commands or vendor SDK	Operator certification	Considered for remote sites (Section 26.6)

The command interface

An SX126x-class transceiver is controlled by opcodes over SPI. Two extra lines matter more than the SPI itself: **BUSY**, which the radio holds high while it processes a command (from a few microseconds up to about 3.5 ms when waking from sleep), and **DIO1**, the interrupt line on which TxDone, RxDone and timeout events arrive. Two configuration commands at initialisation tie the radio to the board: DIO3 supplies the TCXO (SetDIO3AsTCXOCtrl, 1.8 V, with a start-up delay) and DIO2 drives the RF switch automatically (SetDIO2AsRfSwitchCtrl). Getting either wrong produces a radio that answers on SPI and never transmits.

The listing sends one LoRa packet (C11; the HAL functions `spi_xfer`, `gpio_read_busy`, `millis` and `dio1_wait` are the board's and are illustrative; opcodes and parameter layouts follow the SX126x data sheet). Its two design points are the bounded wait on BUSY, so that a hung radio returns an error instead of blocking the node forever (the failure of *Electronics and Computing*, Ch. 28 §28.3), and a TxDone guard derived from the computed time on air.

```
 #include <stdbool.h>
#include <stddef.h>
#include <stdint.h>

extern void     spi_xfer(const uint8_t *tx, uint8_t *rx, size_t n);   /* NSS-framed */
extern bool     gpio_read_busy(void);
extern uint32_t millis(void);
extern bool     dio1_wait(uint32_t timeout_ms);                      /* sleeps until DIO1 */

static bool cmd(const uint8_t *c, size_t n)
{
    uint32_t t0 = millis();
    while (gpio_read_busy())
        if ((uint32_t)(millis() - t0) > 10u) return false;            /* radio hung */
    spi_xfer(c, NULL, n);
    return true;
}

bool lora_tx(const uint8_t *buf, uint8_t len, uint8_t sf, uint32_t toa_ms)
{
    const uint8_t stby[] = {0x80, 0x01};                     /* SetStandby(STDBY_XOSC) */
    const uint8_t type[] = {0x8A, 0x01};                     /* SetPacketType(LoRa) */
    const uint8_t freq[] = {0x86, 0x36, 0x41, 0x99, 0x9A};   /* 868.1 MHz = f*2^25/32 MHz */
    const uint8_t pa[]   = {0x95, 0x02, 0x02, 0x00, 0x01};   /* SetPaConfig for about +14 dBm */
    const uint8_t txp[]  = {0x8E, 0x16, 0x04};               /* SetTxParams 22, 200 us ramp */
    const uint8_t mod[]  = {0x8B, sf, 0x04, 0x01, (uint8_t)(sf >= 11u)}; /* BW125, CR4/5, LDRO */
    const uint8_t pkt[]  = {0x8C, 0x00, 0x08, 0x00, len, 0x01, 0x00};    /* pre 8, expl, CRC */
    const uint8_t irq[]  = {0x08, 0x02, 0x01, 0x02, 0x01, 0, 0, 0, 0};   /* TxDone|Timeout->DIO1
*/
    const uint8_t clr[]  = {0x02, 0xFF, 0xFF};               /* ClearIrqStatus(all) */
    const uint8_t tx[]   = {0x83, 0x00, 0x00, 0x00};         /* SetTx, no radio timeout */
    const uint8_t *seq[] = {stby, type, freq, pa, txp, mod, pkt, irq, clr};
    const size_t   n[]   = {sizeof stby, sizeof type, sizeof freq, sizeof pa, sizeof txp,
                            sizeof mod, sizeof pkt, sizeof irq, sizeof clr};
    for (size_t i = 0; i < sizeof n / sizeof n[0]; ++i)
        if (!cmd(seq[i], n[i])) return false;

    uint8_t w[2 + 255] = {0x0E, 0x00};                       /* WriteBuffer at offset 0 */
    for (uint8_t i = 0; i < len; ++i) w[2 + i] = buf[i];
    if (!cmd(w, 2u + len) || !cmd(tx, sizeof tx)) return false;
    return dio1_wait(toa_ms + toa_ms / 2u + 50u);             /* guard: 1.5 x airtime + 50 ms */
}
```

The frequency word is $868.1 \times 10^6 \times 2^{25} / (32 \times 10^6) = 910{,}268{,}826$ = 0x3641999A. The PA configuration (duty-cycle and hpMax codes 0x02 with a +22 setting) is the data sheet's optimum-efficiency combination for about +14 dBm, which draws roughly 45 mA instead of the 118 mA of full power; the energy model of Section 26.5 uses that 45 mA.

26.4 Link budgets

From desk model to site survey

The desk estimate used the Okumura–Hata model with its suburban correction (*Circuits, Signals and Communications*, Ch. 30 §30.2): for a 30 m gateway, 1.5 m nodes and 868 MHz it predicts 116 dB at 1 km and 148 dB at 8 km, against 97.6 and 129.5 dB with the open-area correction. A 19 dB spread between two published corrections is too wide to design to, so a drive survey was run before ordering: a test node at 1.5 m on a vehicle transmitted every 10 s along every road within 9 km while the gateway logged RSSI and SNR. A least-squares fit of 4,200 points gave

$$PL(d) = 110 + 34\log_{10}\!\left(\frac{d}{1\ \text{km}}\right)\ \text{dB}, \qquad \sigma = 7\ \text{dB},$$

between the two Hata variants and with a slope close to Hata's 35 dB/decade. Figure 26.3 compares them.

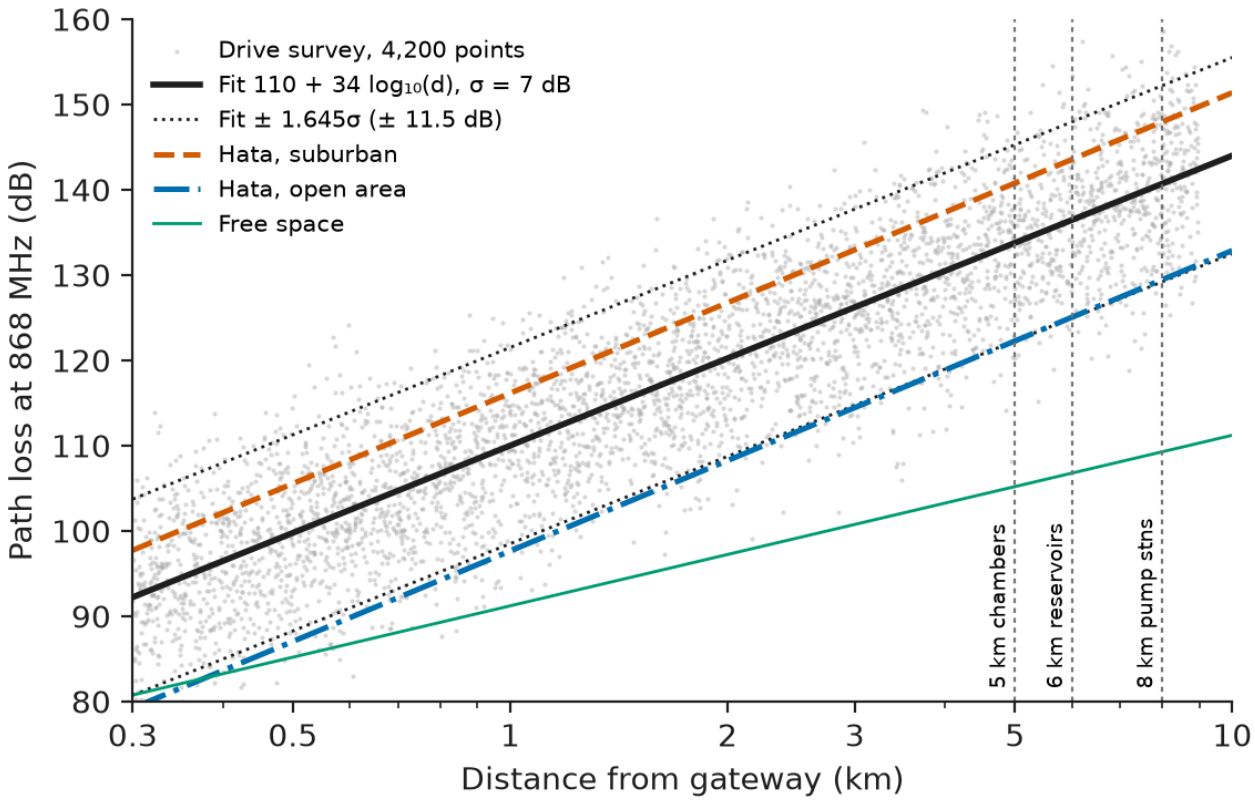


Figure 26.3 — Path loss versus distance at 868 MHz: survey points, fitted model and Hata predictions.

The budget per site class

With the fitted model, each site class has a received power at its maximum range, and each LoRa spreading factor has a sensitivity: the 125 kHz noise floor $-174 + 10\log_{10}(125 \times 10^3) + \text{NF}$ is −117.0 dBm for a 6 dB noise figure, and LoRa demodulates down to an SNR of −7.5 dB at SF7, falling 2.5 dB per step to −20 dB at SF12, giving sensitivities of −124.5 to −137.0 dBm. The required margin is $1.645\sigma + 3$ dB: 14.5 dB for outdoor nodes, and 16.3 dB for chamber nodes, whose chamber loss adds an independent 4 dB of variation ($\sqrt{7^2 + 4^2} = 8.06$ dB). The listing picks, for each class, the lowest spreading factor that meets the margin, for both gateway placements.

```
 import numpy as np

SNR_REQ = {7: -7.5, 8: -10.0, 9: -12.5, 10: -15.0, 11: -17.5, 12: -20.0}   # dB, LoRa BW 125 kHz
NOISE = -174 + 10*np.log10(125e3) + 6.0          # dBm in 125 kHz, gateway NF 6 dB

def path_loss(d_km, pl1=110.0, n=3.4):           # site-survey fit, 868 MHz, 30 m gateway
    return pl1 + 10*n*np.log10(d_km)

def best_sf(d_km, extra_loss, sigma, gw_loss, eirp=16.0, g_gw=6.0, z=1.645, misc=3.0):
    """Lowest spreading factor with margin >= z*sigma + misc (95 % locations at range)."""
    pr = eirp - path_loss(d_km) - extra_loss + g_gw - gw_loss
    need = z*sigma + misc
    for sf, snr in SNR_REQ.items():
        margin = pr - (NOISE + snr)
        if margin >= need:
            return sf, pr, margin, need
    return None, pr, pr - (NOISE + SNR_REQ[12]), need

classes = [  # name, max range km, extra loss dB, total shadowing sigma dB
    ("pump station, roof mast", 8.0, -6.0, 7.0),          # 4 m antenna: -6 dB height gain
    ("reservoir kiosk, 1.5 m",  6.0,  0.0, 7.0),
    ("valve chamber, in lid",   5.0, 22.0, np.hypot(7.0, 4.0)),
    ("valve chamber, bollard",  5.0,  1.5, 7.0)]
for rev, gw_loss in (("A: indoor gateway, 30 m feeder", 4.3), ("B: mast-top gateway + filter",
1.5)):
    print(rev)
    for name, d, extra, sig in classes:
        sf, pr, m, need = best_sf(d, extra, sig, gw_loss)
        print(f"  {name:24s} {d:3.0f} km  Pr {pr:6.1f} dBm  need {need:4.1f} dB  "
              + (f"SF{sf} margin {m:4.1f} dB" if sf else f"NO LINK (SF12 margin {m:5.1f} dB)"))
```

```
 A: indoor gateway, 30 m feeder
  pump station, roof mast    8 km  Pr -117.0 dBm  need 14.5 dB  SF10 margin 15.0 dB
  reservoir kiosk, 1.5 m     6 km  Pr -118.8 dBm  need 14.5 dB  SF11 margin 15.8 dB
  valve chamber, in lid      5 km  Pr -138.1 dBm  need 16.3 dB  NO LINK (SF12 margin  -1.0 dB)
  valve chamber, bollard     5 km  Pr -117.6 dBm  need 14.5 dB  SF11 margin 17.0 dB
B: mast-top gateway + filter
  pump station, roof mast    8 km  Pr -114.2 dBm  need 14.5 dB  SF9 margin 15.3 dB
  reservoir kiosk, 1.5 m     6 km  Pr -116.0 dBm  need 14.5 dB  SF10 margin 16.1 dB
  valve chamber, in lid      5 km  Pr -135.3 dBm  need 16.3 dB  NO LINK (SF12 margin   1.8 dB)
  valve chamber, bollard     5 km  Pr -114.8 dBm  need 14.5 dB  SF9 margin 14.8 dB
```

Worked example — the chamber node at 5 km. Table 26.5 is the full budget behind the third line of Revision A. The median received power is −138.1 dBm, 1.0 dB *below* SF12 sensitivity: half of all chamber locations at that range cannot be heard at all, and the 95 % target needed 16.3 dB above sensitivity. At commissioning, with the lid open, the chamber loss was about 5 dB instead of 22 and the margin about 16 dB, so the node joined at SF12 and the contractor recorded a working link. Even at 2 km (path loss 120.2 dB) the in-lid node has only 12.5 dB of SF12 margin against 16.3 needed. No spreading factor can rescue a 22 dB installation loss; only moving the antenna can, which is what the bollard line shows: −117.6 dBm and SF11 even with the Revision A gateway, SF9 with Revision B; Figure 26.4 summarises every class.

Table 26.5 — Uplink budget, valve-chamber node at 5 km, Revision A.

Item	Value
Node EIRP (+14 dBm conducted, 2 dBi)	+16.0 dBm
Path loss, survey fit at 5 km	−133.8 dB
Chamber loss, lid closed (median)	−22.0 dB
Gateway antenna gain	+6.0 dBi
Feeder, connectors, arrester	−4.3 dB
Received power at gateway input	**−138.1 dBm**
Noise floor, 125 kHz, NF 6 dB	−117.0 dBm
SF12 required SNR	−20.0 dB
SF12 sensitivity	−137.0 dBm
Margin at SF12	**−1.0 dB**
Required margin (1.645 × 8.06 + 3)	16.3 dB
Shortfall	**17.3 dB**

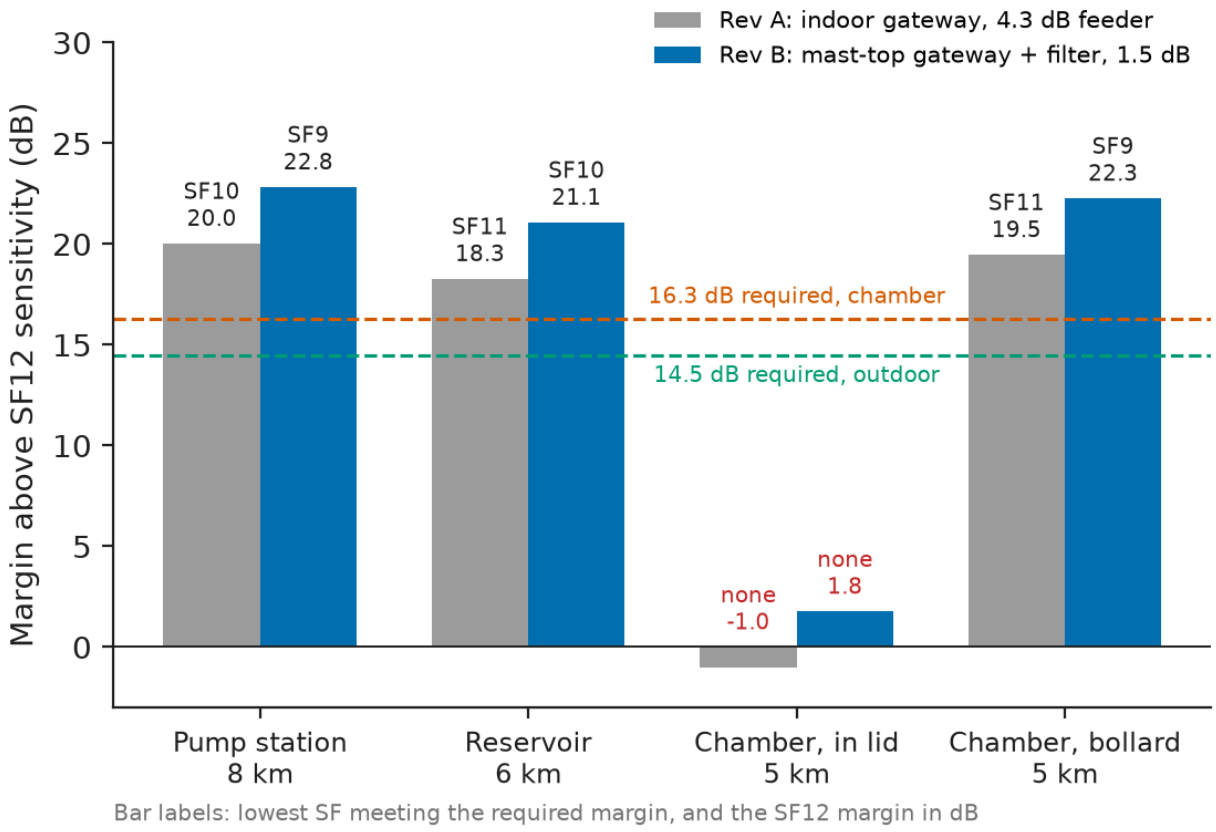


Figure 26.4 — Link margin at maximum range for each node class and spreading factor, Revision A and Revision B gateways.

> **Status of the principle: Link margin against log-normal shadowing**
> *Derived.* With Gaussian shadowing in decibels, a margin of $z\sigma$ above the median gives a location probability $\Phi(z)$ at that distance (95 % for $z = 1.645$); the result is only as good as σ and the median model, which is why the survey fit, not a textbook correction, sets the design.

26.5 Channel constraints

Regulatory limits

In Europe the 863–870 MHz short-range-device band is governed by ETSI EN 300 220 and the national implementations of the ERC Recommendation 70-03; the conformity route is the Radio Equipment Directive. The sub-bands used here:

Table 26.6 — EN 300 220 sub-bands used by the network.

Sub-band	Frequencies	Power limit	Duty-cycle limit	Use
g1 (h1.4)	868.0–868.6 MHz	25 mW ERP	1 % (36 s per hour per device)	Uplinks on 868.1, 868.3, 868.5 MHz; RX1 downlinks
g3 (h1.6)	869.4–869.65 MHz	500 mW ERP	10 %	RX2 downlinks from the gateway on 869.525 MHz

The duty cycle applies to the gateway too: acknowledgements sent in the RX1 window on the uplink channel share the gateway's 1 % in that sub-band.

Time on air

LoRa trades data rate for sensitivity: each step in spreading factor doubles the symbol time and gains 2.5 dB. The time on air of a packet follows the data-sheet formula, and with it the duty cycle and the energy per report. The chamber node sends a 26-byte payload (three 5 min pressure samples, battery voltage and status, plus LoRaWAN overhead) every 15 min.

```
 import math

def time_on_air(sf, payload=26, bw=125e3, cr=1, preamble=8, crc=True, header=True):
    """LoRa packet duration in s (Semtech SX126x data-sheet formula)."""
    de = 1 if sf >= 11 and bw <= 125e3 else 0          # low-data-rate optimisation
    ts = 2**sf / bw
    n = 8 + max(math.ceil((8*payload - 4*sf + 28 + 16*crc - 20*(not header))
                          / (4*(sf - 2*de))) * (cr + 4), 0)
    return (preamble + 4.25)*ts + n*ts

def battery_years(sf, tries, cap_mah=6800, period_s=900):
    toa = time_on_air(sf)
    q_tx = 45.0*toa                         # mA*s per attempt at +14 dBm
    q_rx = 2*5.0*time_on_air(sf, payload=12)   # two class-A receive windows (ACK-length)
    q_msg = tries*(q_tx + q_rx) + 3.0      # + MCU, sensor and settling per report
    n = 365*86400/period_s
    mah_year = n*q_msg/3600 + 0.002*8760 + 0.01*cap_mah   # + 2 uA sleep, 1 %/yr self-discharge
    duty = tries*toa/period_s
    return toa, duty, cap_mah/mah_year

for sf, tries in ((9, 1), (10, 1), (12, 1), (12, 3)):
    toa, duty, yrs = battery_years(sf, tries)
    print(f"SF{sf} x{tries}: airtime {1e3*toa:6.0f} ms, duty cycle {100*duty:.2f} %, battery
{yrs:4.1f} years")
```

```
 SF9 x1: airtime    206 ms, duty cycle 0.02 %, battery 31.1 years
SF10 x1: airtime    412 ms, duty cycle 0.05 %, battery 21.0 years
SF12 x1: airtime   1647 ms, duty cycle 0.18 %, battery  7.2 years
SF12 x3: airtime   1647 ms, duty cycle 0.55 %, battery  2.6 years
```

Worked example — why the first battery died. At SF9 one report costs 206 ms of airtime and the battery would outlast the cell's own 10–15 year shelf life; the radio is not the limit. A Revision A chamber node with a marginal link was pushed by adaptive data rate to SF12, where one attempt takes 1,647 ms (eight times SF9), and because its packets were often lost it used confirmed uplinks with up to three attempts. That costs 0.55 % of airtime, still legal, but the life falls to 2.6 years: the node of the opening paragraph. The radio energy model is correct; the link it was fed was not.

Figure 26.5 plots airtime and battery life against spreading factor.

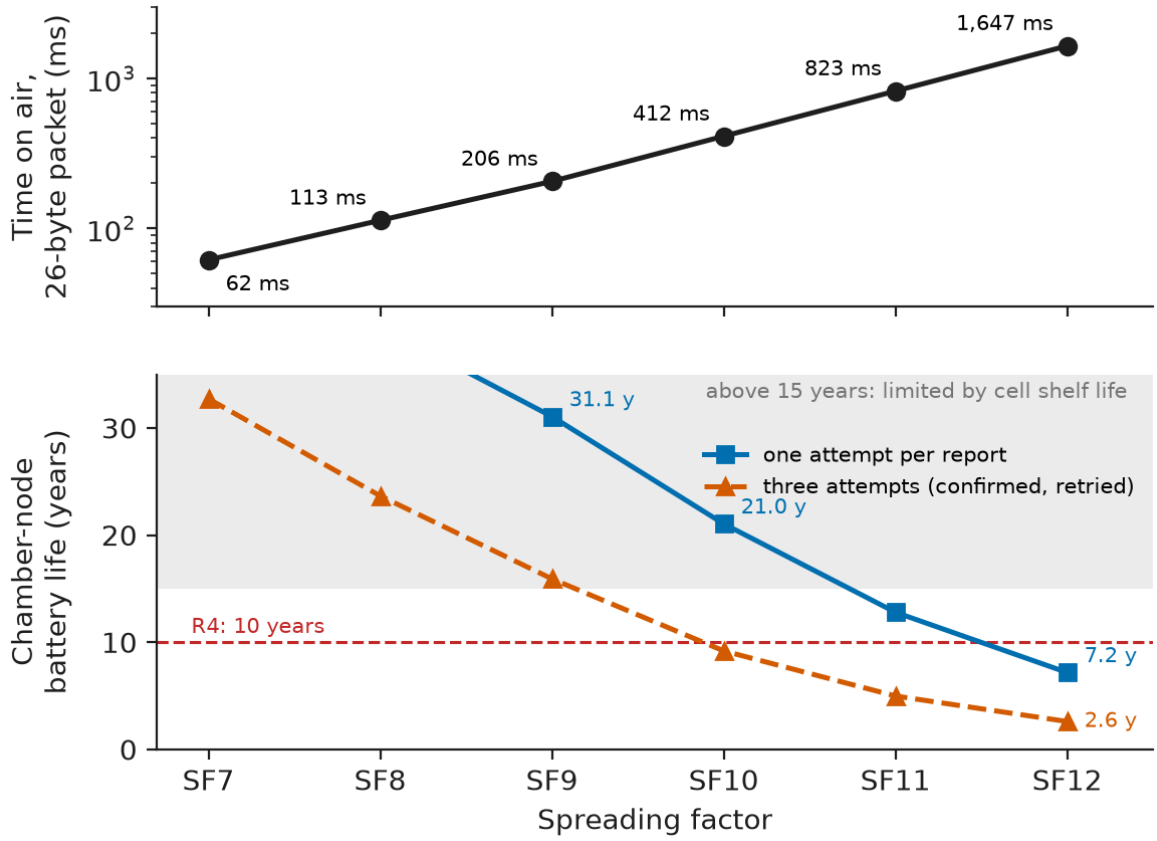


Figure 26.5 — Time on air of a 26-byte LoRa packet and chamber-node battery life versus spreading factor.

> **Status of the principle: Energy per report from time on air**
> *Derived.* Charge is current integrated over each radio state; the result is exact for its inputs, and the inputs (45 mA at +14 dBm, 5 mA in receive, 3 mA·s of overhead) are measured on the real board with a current-profile analyser, because data-sheet currents omit the MCU, regulators and sensor.

26.6 Wireless sensor networks

Choosing the network type

Table 26.7 — Network options for 36 telemetry sites over 8 km.

Option	Topology	Range per hop	Node energy	Recurring cost	Latency	Choice
Private LoRaWAN, one gateway, on-premises network server	Star	2–15 km rural	Lowest	None (own spectrum use under licence exemption)	1–5 s	**Chosen**
Public LoRaWAN operator	Star, shared gateways	As above	Lowest	$1–3 per node per year	1–5 s	Rejected: no coverage at the chambers; data leaves the OT network
NB-IoT / LTE-M	Star to operator cell	Cell coverage; +20 dB coverage class	2–5× LoRa per report	$1–2 per node per month	2–10 s	Backup for two sites beyond 8 km
IEEE 802.15.4 mesh (Wi-SUN, 868 MHz)	Mesh	0.5–2 km per hop	Routers must listen: mains only	None	1–30 s, multi-hop	Rejected: chamber nodes cannot route on a battery

A star network keeps the battery nodes simple — they transmit and sleep, and never relay anyone else's traffic — and puts all the complexity in one gateway and one server. That is also its weakness: a single gateway is a single point of failure, addressed in Section 26.7.

Capacity

LoRa transmissions on different spreading factors are quasi-orthogonal, and nodes transmit without listening (pure ALOHA), so the collision probability on one channel and spreading factor is $1 - e^{-2G}$ with offered load G = packets per second × packet duration (*Circuits, Signals and Communications*, Ch. 30 §30.7).

Worked example — collisions in the two revisions. In Revision A, 16 chamber nodes at SF12 averaged 2.5 attempts per 15 min report, spread over three channels: $G = 16 \times 2.5 \times 1.647/(900 \times 3) = 0.0244$ and a collision probability of 4.8 % per attempt, which fed the retries that caused it. In Revision B the 28 nodes at SF9 (16 chambers on bollards, 12 pump stations) send one attempt each: $G = 28 \times 0.206/2{,}700 = 0.0021$ and 0.43 %. At SF9 the traffic could grow about tenfold before collisions reach the 4–5 % that Revision A already suffered.

26.7 Communication reliability

What R1 and R2 require

R1 (99 % of routine reports) is met with **unconfirmed** uplinks if each packet's loss probability, from fading and collisions together, stays below about 1 %. Confirming every report would cost downlink airtime the gateway does not have: 36 nodes every 15 min are 144 acknowledgements an hour, which at SF9 (144 ms each for a 12-byte frame) take 20.8 s of the gateway's 36 s hourly allowance in g1, and at SF12 (1,155 ms each) would take 166 s, impossible in RX1. Routine data is therefore unconfirmed, and a missed report is recovered from the next one, which carries the previous samples too.

R2 (alarms at 99.9 % within 30 s) needs **confirmed** uplinks with retries. **Worked example — retries for an alarm.** If one attempt fails with probability p, independent attempts give a delivery probability $1 - p^k$ after k attempts. A pump station at SF9 with a 15 dB margin loses perhaps 5 % of packets in deep fades and collisions; three attempts give $1 - 0.05^3 = 99.99\,\%$. With a 20 % loss rate, three attempts give only 99.2 % and R2 fails: retries cannot replace margin. Each SF9 attempt with its two receive windows and a 1–3 s random back-off takes about 3 s, so three attempts finish well inside 30 s.

> **Status of the principle: Delivery probability with independent retries**
> *Approximate.* $1 - p^k$ assumes independent losses; fading on a fixed link is correlated over seconds to minutes, so retries spaced by a few seconds are less independent than the formula assumes. Back-off that spreads retries over tens of seconds, and frequency hopping across the three channels, restore some independence.

Layers of protection

- **Integrity.** LoRa's CRC-16 rejects corrupted frames at the PHY; LoRaWAN's 32-bit message integrity code rejects forged or replayed ones (Chapter 11 for the key handling).
- **Forward error correction.** Coding rate 4/5 corrects little; raising it to 4/8 costs 60 % more airtime for about 1 dB, so margin is bought with antenna placement instead.
- **End-to-end monitoring.** The network server tracks each node's frame counter; a gap, an RSSI trend falling by more than 6 dB, or a spreading factor rising above the node's planned value raises a maintenance ticket before data is lost. This is the check that would have caught the chamber nodes in their first wet week.
- **Gateway redundancy.** A second gateway on a reservoir site 4 km away hears 30 of the 36 nodes; the network server de-duplicates. It turns the single point of failure into a degraded mode instead of an outage.

26.8 Interference mitigation

Blocking by a co-sited transmitter

The mobile operator's LTE 800 sector antenna is mounted 5 m below the gateway antenna on the same tower, transmitting in the 791–821 MHz downlink band, about 47 MHz below the gateway's channels. The gateway's own channel filtering rejects it, but its LNA and mixer see the full signal, compress, and their noise floor rises: **blocking**.

Worked example — how strong is the blocker? For vertically separated antennas an empirical isolation of $28 + 40\log_{10}(d/\lambda)$ dB is used for d in metres; at 806 MHz ($\lambda = 0.372$ m) and $d = 5$ m it gives 73.1 dB. With an LTE EIRP of 59 dBm per sector, the gateway antenna port sees $59 - 73.1 = -14.1$ dBm, and after the 4.3 dB Revision A feeder, −18.4 dBm at the gateway input — a signal 100 dB stronger than the weakest LoRa packets it must decode. The gateway's log showed the noise floor rising 9 dB whenever the LTE cell was loaded in the evening; every link lost 9 dB, more than half of the outdoor margin of Section 26.4. A cavity band-pass filter (863–870 MHz, 1.0 dB insertion loss, more than 45 dB rejection at 821 MHz) at the gateway input reduces the blocker to −63 dBm in Revision A and about −60 dBm in Revision B, where the filter sits at the mast-top gateway; the measured noise-floor rise fell below 0.5 dB (R6).

Figure 26.6 shows the spectrum at the gateway input before and after the filter.

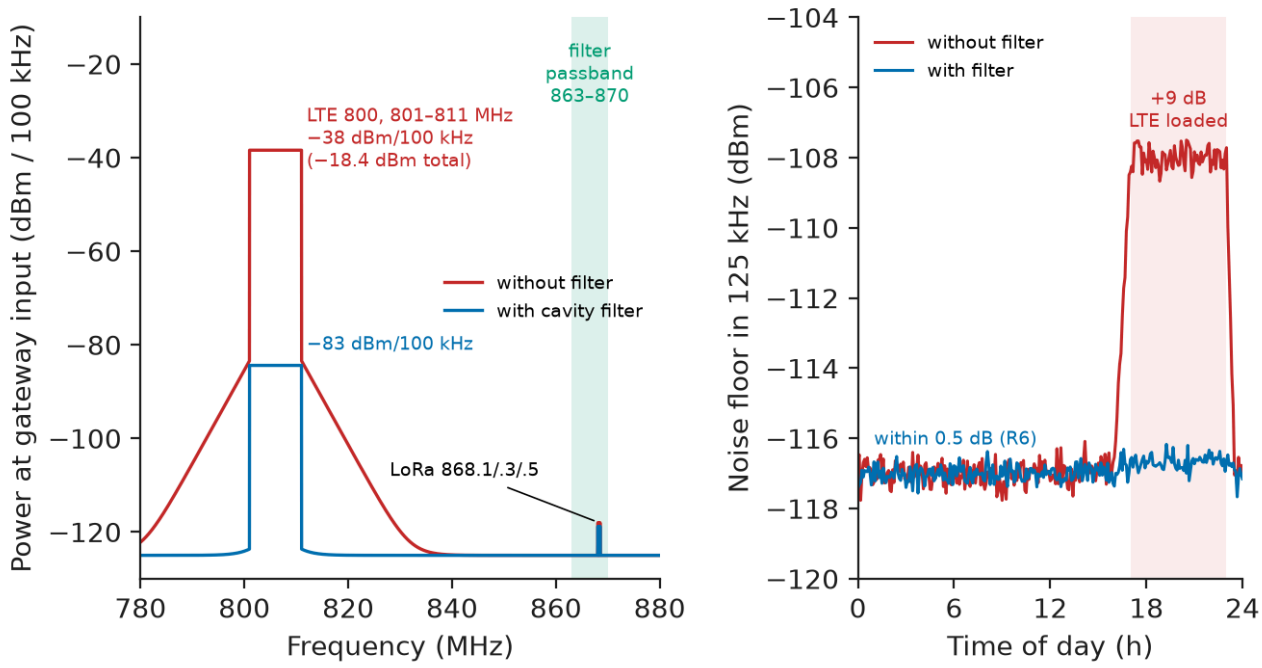


Figure 26.6 — Spectrum at the gateway input with the co-sited LTE 800 carrier, before and after the cavity filter, and the resulting noise-floor rise.

> **Status of the principle: Vertical antenna isolation**
> *Empirical.* The $28 + 40\log_{10}(d/\lambda)$ rule is a fit to measurements for collinear antennas on a common mast; it varies by ±5–10 dB with antenna patterns, mast structure and cabling, so the filter is specified with margin and the result is verified by measuring the noise floor with the interferer at full load.

Other interference and its remedies

Table 26.8 — Interference sources and mitigations.

Source	Mechanism	Mitigation
LTE 800 on the same tower	Blocking, noise-floor rise	Cavity filter; vertical separation; mast-top gateway
Other 868 MHz users (alarm systems, smart meters, other LoRa networks)	Co-channel collisions	Three channels, random back-off, retries for alarms only; SF diversity
Gateway's own PoE switch and pump VFDs	Broadband conducted and radiated noise	Ferrites, shielded cable, separation from VFD cabling; noise-floor survey before mounting
Node's own switching regulator	Spurs near the RF channel	Regulator frequency plan; LDO for the TCXO; layout per reference design
Lightning-induced surges	Damage to front end	Quarter-wave stub or gas-discharge arrester; bonded mast

26.9 System testing

Test levels

Radio systems are tested at four levels, and the opening failures each escaped one of them:

1. **Conducted RF tests** on the node design: output power, harmonics and spurious (R5), sensitivity per spreading factor, frequency error over temperature. These run at an accredited laboratory or against the module certificate.
2. **Over-the-air tests** in an anechoic or semi-anechoic chamber with the real enclosure and antenna: total radiated power and realised gain.
3. **Site tests** under real conditions: every chamber node measured **with the lid closed and locked**, and a sample re-measured in the wet season. Revision A's commissioning, done with lids open, measured a system that would never operate that way.
4. **Network tests** over weeks: delivery statistics (R1), alarm injection (R2), noise-floor logs (R6), and current profiles of battery nodes (R4).

After any change to the gateway, its antenna, its filter or the tower (including other operators' equipment), a regression test repeats the noise-floor log and the RSSI of six reference nodes against their baseline.

Verification

Table 26.9 — Verification of the telemetry network, Revision B.

Req.	Test	Limit	Result	Pass/fail
R1	Frame-counter statistics, all nodes, 3 months including wet season	≥ 99 % per node per month	Worst node 99.6 %	Pass
R2	1,000 injected alarms over 30 days, 12 pump stations	≥ 99.9 % within 30 s	1,000 of 1,000; worst latency 9.8 s	Pass
R3	RSSI and SNR per site, lid closed, 50 packets each, compared with budget	Margin ≥ 14.5 dB (outdoor) / 16.3 dB (chamber) at planned SF	All sites pass; minimum 15.1 dB (pump station at 7.6 km)	Pass
R4	Current-profile analyser on a chamber node, 24 h	Projected life ≥ 10 years	0.22 Ah/year projected including self-discharge; 31 years, so cell shelf life limits	Pass
R5	Conducted harmonics, module certificate review	≤ −30 dBm above 1 GHz	−47 dBm at 1,736 MHz	Pass
R6	Gateway noise floor, 14 days with LTE at full load	Rise ≤ 1 dB	0.4 dB	Pass
R7	Key audit, 36 devices	Unique keys	36 unique AppKeys	Pass
R8	IP68 immersion 1 m, 24 h; thermal cycle −20 to +50 °C	No ingress; operation	Pass	Pass

Failure analysis

Table 26.10 — FMEA extract for the telemetry network.

Failure mode	Effect	Cause	Detection	Mitigation
Chamber node drops out when wet	Lost reports; SF12 and retries	22 dB chamber loss, measured with lid open	SF and RSSI trend on server	Bollard or in-lid composite antenna; commissioning with lid closed
Battery flat in 2.6 years	Site blind until visit	SF12 with three attempts	Battery-voltage telemetry trend; SF monitor	Revision B link at SF9; ADR capped at the planned SF plus one
Gateway desensitised in evenings	Margin loss of 9 dB network-wide	LTE 800 blocking	Noise-floor log	Cavity filter; mast-top gateway
Radio hangs (BUSY stuck)	Node silent until watchdog	Brown-out during TX pulse, radio state lost	BUSY timeout in driver	Timeout, radio reset, 100 µF buffer capacitor on the cell
Gateway or backhaul failure	Whole network down	Single gateway, single fibre	Server heartbeat	Second gateway at reservoir site
Lightning strike on tower	Gateway front end destroyed	Surge on coax or PoE	Gateway offline	Arresters, bonding, PoE surge protection
Frequency drift on long packets	SF12 packets lost at temperature extremes	Crystal instead of TCXO	Packet error versus temperature	TCXO (±2 ppm) on all nodes

The worked fault story is the opening one, diagnosed from the network server's own data. The nodes that failed were all chambers; their RSSI was 15–20 dB below their commissioning values; and the drop correlated with rainfall. That combination points at the installation, not the radio, and a single measurement with the lid closed (−21 dB) confirmed it.

Revision B

Change	Reason	Effect
Chamber antennas moved to marker bollards (or composite lids) with 2 m cable	22 dB lid loss, no link at any SF	Chamber nodes at SF9 with ≥ 14.8 dB margin at 5 km
Gateway moved to the mast top with a cavity filter	4.3 dB feeder loss; LTE blocking	+2.8 dB on every link; noise-floor rise 0.4 dB
ADR capped at the planned SF plus one; SF rise raises a ticket	ADR hid a failing link by escalating to SF12	Problems found in days, not when batteries die
Commissioning procedure: lid closed and locked, 50 packets, margin recorded against budget	Commissioning measured the wrong configuration	R3 verified as designed
Second gateway at a reservoir site	Single point of failure	30 of 36 nodes dual-covered
Regression test after any tower change	Other operators' changes broke the link	Noise floor and reference RSSI checked

What remains open

Open question	Where it is answered
How are the MQTT topics, device provisioning and security of the network server designed?	Chapter 11
How is a complete battery sensor node designed, from sensor to enclosure, and its power budget closed?	Chapter 38
How are the received samples filtered and checked for anomalies before they reach the operator?	Chapter 27
How is the node firmware structured, and how does it recover from a hung peripheral?	Chapter 10; *Electronics and Computing*, Ch. 28
How are wired industrial networks (Modbus, CAN, Ethernet) and their cybersecurity handled at the pump stations?	Chapter 21
How do multipath, fading statistics and diversity behave on mobile links?	*Circuits, Signals and Communications*, Ch. 30 §30.3–30.5

Next: Chapter 27 follows the data off the radio and into the processor: filtering, spectral analysis, features and event detection on real, noisy sensor signals, and how to run them in real time on embedded hardware.

Chapter 27 — Practical Signal Processing

Part VII · Communications, Signal Processing, and Biomedical Applications

A water utility fits vibration nodes to forty booster-pump sets, each a 37 kW induction motor at 1480 rpm driving a centrifugal pump. The first-generation node samples a MEMS accelerometer at 6.4 kS/s and reports overall RMS acceleration every minute. Within a month it alarms on every pump fed from a variable-frequency drive: vibration at the drive's 8 kHz switching frequency reaches the sensor and, with no anti-alias filter worth the name, folds to $|8 - 6.4| = 1.6$ kHz. Two months later the fleet misses a real fault. A drive-end bearing develops an outer-race spall whose impacts ring the housing at a few kilohertz, and the overall RMS hardly moves until the bearing is days from seizure. Nothing in the node was broken. The sampling plan ignored a known interferer, and the only feature computed could not see the fault mechanism. Practical signal processing is these decisions: which noise to remove and where, which filter, spectrum, feature and detector, and whether the chain fits the processor and the deadline.

> **How is a measurement chain designed so that what reaches the algorithms is the signal, not noise, aliases or artefacts of the processing itself?**
> **How are features and detectors chosen for a specific fault mechanism, implemented in real time on an embedded processor, and proved against numbers written down before the build?**

Concept	Engineering destination
Sensor-noise reduction	Noise budgets for accelerometers, microphones and current probes; averaging gains
Filtering	Anti-alias chains, band selection for demodulation, decimation, velocity integration
Spectral analysis	Machine diagnostics, harmonic analysis, EMC pre-scans, resonance surveys
Feature extraction	Condition-monitoring indicators, inputs to ML classifiers (Chapter 32), data reduction for IoT links
Event detection	Alarms with a stated false-alarm rate; arc, knock, leak and impact detection
Audio processing	Acoustic leak and cavitation monitoring, voice interfaces, noise measurement
Image processing	Thermal inspection, machine vision, optical inspection on production lines
Real-time DSP	Block processing with DMA, latency budgets, overrun detection
Embedded implementation	CMSIS-DSP on Cortex-M, fixed versus floating point, memory placement
Performance evaluation	Detection probability, false-alarm rate, bit-exactness against a reference model

Roadmap. The reference system and its noise budget (27.1) → the filter chain from sensor to decimated envelope (27.2) → spectra with the right resolution (27.3) → features that respond to the fault mechanism (27.4) → detectors with a stated false-alarm rate (27.5) → the acoustic channel (27.6) and the thermal-image channel (27.7) → fitting it all into a real-time schedule (27.8) and into firmware (27.9) → proving that it works and analysing how it fails (27.10).

27.1 Sensor-noise reduction

> **Boundary.**
> The sampling theorem and aliasing are owned by *Circuits, Signals and Communications*, Ch. 19 (§19.4, §19.9); quantization noise and SQNR by Ch. 21 §21.2; noise models and noise bandwidth by Ch. 29 §29.3–29.4. Sensor principles and selection belong to Chapter 6, signal conditioning and uncertainty to Chapter 7, DAQ architecture to Chapter 8 of this book. This chapter uses those results to build one processing chain.

The reference system

The running example is the second-generation node for the pump fleet, called here the **pump-condition node**. It watches one motor–pump set through four simultaneous analog channels and one image sensor:

- three accelerometers (motor drive-end radial, motor drive-end axial, pump bearing radial);
- one analog MEMS microphone aimed at the pump casing, for cavitation and leaks;
- a 32 × 24-pixel thermal array viewing the motor drive-end bearing housing;
- an optical once-per-revolution tachometer on a timer-capture input, for exact shaft speed.

It reports features every 10 s over Ethernet (Modbus TCP to the PLC, MQTT to the condition-monitoring server); raw waveforms leave only on request. Table 27.1 fixes the requirements before any part is chosen.

Table 27.1 — Requirements of the pump-condition node.

ID	Requirement	Target	Verification
R1	Vibration bandwidth	10 Hz–10 kHz; ±1 dB to 5 kHz, ±3 dB to 10 kHz	Shaker sweep against a reference accelerometer (ISO 16063-21 back-to-back)
R2	Range and noise floor	±40 g; in-band noise ≤ 3 mg rms	Quiet-mount test and short-circuited input
R3	Alias rejection	≥ 60 dB for any input that folds into 10 Hz–10 kHz	Electrical sine sweep 10–200 kHz into the conditioning input
R4	Velocity RMS, 10–1000 Hz (ISO 20816-3)	Error ≤ ±5 %	Shaker at 25 Hz and 80 Hz, 2–10 mm/s
R5	Bearing-defect detection	Seeded 0.5 mm outer-race spall detected with $P_D \geq 0.95$	Bearing test rig, 20 runs
R6	False-alarm rate	≤ 1 per node per month	30-day soak on a healthy pump, plus analysis
R7	Alarm latency	≤ 10 s for level alarms, ≤ 30 s for envelope alarms	Recorded-signal playback with GPIO timestamps
R8	Acoustic octave levels 1–8 kHz	±1.5 dB, 50–120 dB SPL	Class 1 acoustic calibrator, 94 dB at 1 kHz (IEC 60942)
R9	Thermal hotspot	$\Delta T \geq 10$ K on a 120 mm housing detected at ≤ 1.0 m; ±2 K against contact PT100	Heated target and contact reference
R10	Processing headroom	CPU ≤ 50 %, RAM ≤ 75 %	DWT cycle counter, linker map
R11	Power	≤ 2 W from 24 V DC	Bench measurement at 18, 24 and 30 V

The 50 % CPU limit in R10 reserves room for a later on-node anomaly model (Chapter 12).

Architecture

Figure 27.1 shows the chain: a simultaneous-sampling delta-sigma converter streams 32 kS/s per channel into the microcontroller by DMA, a 32-bit timer stamps the tachometer edge, and the thermal array is read over I²C at 2 frames/s. Only features cross the network.

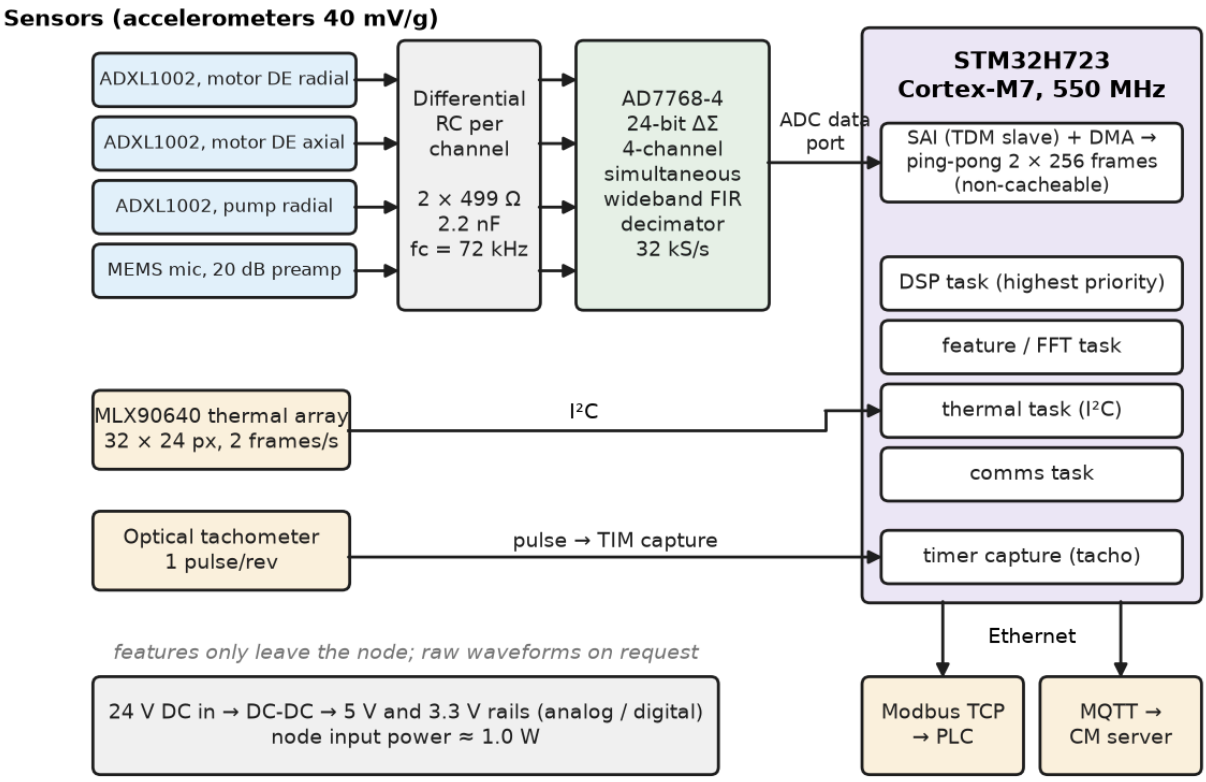


Figure 27.1 — Architecture of the pump-condition node: four simultaneous analog channels, a thermal array and a tachometer feeding block-based DSP on an STM32H723, with features reported over Ethernet.

Noise budget

Noise reduction starts with a budget: every source referred to the sensor input, in g, over the analysis band, combined by root-sum-square.

> **Status of the principle: root-sum-square combination of noise**
> *Derived.* The variances of independent, zero-mean noise sources add; it fails for correlated sources (two channels sharing one reference, mains pickup common to all channels), which add linearly.

Worked example — the accelerometer noise floor. The ADXL1002 has a noise density of about 25 µg/√Hz (datasheet typical). Over the 10 kHz band of R1 its noise is $25 \times 10^{-6}\sqrt{10^4} = 2.5$ mg rms. The ADC is an AD7768-4-class converter with ±4.096 V full scale and roughly 12 µV rms input-referred noise at 32 kS/s (dynamic range about 108 dB). At 40 mV/g that is $12/40\,000 = 0.30$ mg. The 24-bit quantization step is $8.192\ \mathrm{V}/2^{24} = 0.49$ µV, negligible. The total is $\sqrt{2.5^2 + 0.30^2} = 2.52$ mg rms, against the 3 mg limit of R2: a 16 % margin, and the sensor dominates. Range: ±2 V at ±50 g fits the ±4.096 V input with room for the sensor's 2.5 V bias to be removed differentially.

Table 27.2 — Noise budget of one vibration channel, 10 Hz–10 kHz, referred to the input.

Source	Density or level	In-band rms	Share of variance
ADXL1002 sensor noise	25 µg/√Hz	2.50 mg	98.6 %
ADC noise (12 µV rms at 40 mV/g)	—	0.30 mg	1.4 %
Quantization (24 bit, ±4.096 V)	0.49 µV step	< 0.004 mg	≈ 0
Total (RSS)	—	2.52 mg	—
Requirement R2	—	3.0 mg	margin 16 %

The budget says what *not* to buy: a quieter ADC changes nothing; a piezoelectric IEPE sensor (5–10 µg/√Hz) would help, at five times the cost per point.

Reducing what remains

Once the sensor sets the floor, processing reduces noise by predictable amounts:

- **Bandwidth.** The 6 kHz envelope band of Section 27.2 holds $25\sqrt{6000} = 1.9$ mg of sensor noise; the 10–1000 Hz velocity band only 0.79 mg.
- **Spectral averaging.** Averaging K Welch segments cuts the variance of the floor estimate by K, so smaller lines stand out (Book 3 §22.8).
- **Synchronous averaging.** Blocks aligned to the tachometer pulse keep shaft-locked components and cut the rest by $\sqrt{K}$: 100 revolutions give 20 dB. Bearing-defect frequencies are not shaft orders and are suppressed too, so this serves gears and couplings, not bearings.
- **Common-mode removal.** Differential drive over shielded twisted pairs cancels VFD and mains interference coupled equally into both conductors.

27.2 Filtering

> **Boundary.**
> FIR and IIR design methods, structures and quantization effects are in *Circuits, Signals and Communications*, Ch. 23 (§23.3, §23.5, §23.6, §23.9); decimation, polyphase filters and multistage rate change in Ch. 24 §24.1–24.2; delta-sigma converter architectures in *Electronics and Computing*, Ch. 10 §10.10.

The anti-alias chain and the converter choice

The opening failure was an aliasing failure, so the converter is chosen with the anti-alias chain in view (Table 27.3).

Table 27.3 — Converter options for four simultaneous channels at 10 kHz bandwidth (2026 distributor prices, 100 pcs, indicative).

Option	Example part class	Alias protection	Fit to R1–R3	Cost per node
MCU SAR ADC	STM32H7 internal 16-bit, 3.6 MS/s	6th-order analog filter per channel	Possible; filters drift; channels skewed	≈ $8 in filters
Delta-sigma, sinc3 decimator	ADS131M04, 24 bit, ≤ 32 kS/s	≈ 25 dB where inputs fold onto 10 kHz	Fails R3 (worked example)	≈ $6
Delta-sigma, wideband FIR decimator	AD7768-4, 24 bit, simultaneous	≥ 105 dB from 0.5·ODR; RC for modulator rate	Meets R1–R3; **chosen**	≈ $35
Digital MEMS accelerometer	IIS3DWB, 26.7 kS/s	Internal; 6.3 kHz bandwidth	Fails R1; no microphone input	≈ $8 per axis

Worked example — why the sinc3 converter was rejected. The ADXL1002 has its mechanical resonance near 21 kHz. Modelled as a second-order system with $Q = 10$, it *amplifies* a 22 kHz excitation (cavitation and impacts produce plenty) by about 17 dB. At an output rate of 32 kS/s a 22 kHz component folds to $32 - 22 = 10$ kHz, inside the analysis band. A sinc3 decimation filter at that frequency attenuates by only $|\mathrm{sinc}(22/32)|^3$, about −25 dB. The net is about −8 dB: a resonance-excited component would appear in band only 8 dB below its true value, against the 60 dB of R3. The first listing reproduces the numbers.

```
import numpy as np

fs = 32_000                                  # ADC output rate, S/s
db = lambda x: 20 * np.log10(np.abs(x))

# Why a plain sinc3 delta-sigma was rejected: sensor resonance 21 kHz, Q = 10
def sensor(f):
    r = f / 21e3
    return 1 / (1 - r**2 + 1j * r / 10)
f_alias = 22_000                              # folds to 32 - 22 = 10 kHz
print(f"sensor gain {db(sensor(f_alias)):+.1f} dB, sinc3 {db(np.sinc(f_alias/fs)**3):+.1f} dB,"
      f" net {db(sensor(f_alias) * np.sinc(f_alias/fs)**3):+.1f} dB at {f_alias/1e3:.0f} kHz")
```

```
sensor gain +16.9 dB, sinc3 -24.9 dB, net -8.0 dB at 22 kHz
```

The wideband-FIR converter puts the steep filtering in its digital decimator, exact and drift-free: flat to about 0.4·ODR (12.8 kHz) and about 105 dB down from 0.5·ODR (datasheet typical). Aliasing around the multi-megahertz modulator rate remains, and a differential RC of 2 × 499 Ω and 2.2 nF ($f_c = 1/(2\pi \cdot 998 \cdot 2.2\ \mathrm{nF}) = 72$ kHz) removes while costing only 0.1 dB at 10 kHz. Figure 27.2 shows the chain.

> **Status of the principle: alias-free sampling**
> *Fundamental.* Any input component at f appears at $|f - kf_s|$ after sampling; only attenuation *before* the sampler, analog or inside an oversampling converter's decimator, can separate it from the band, and no later digital filter can.

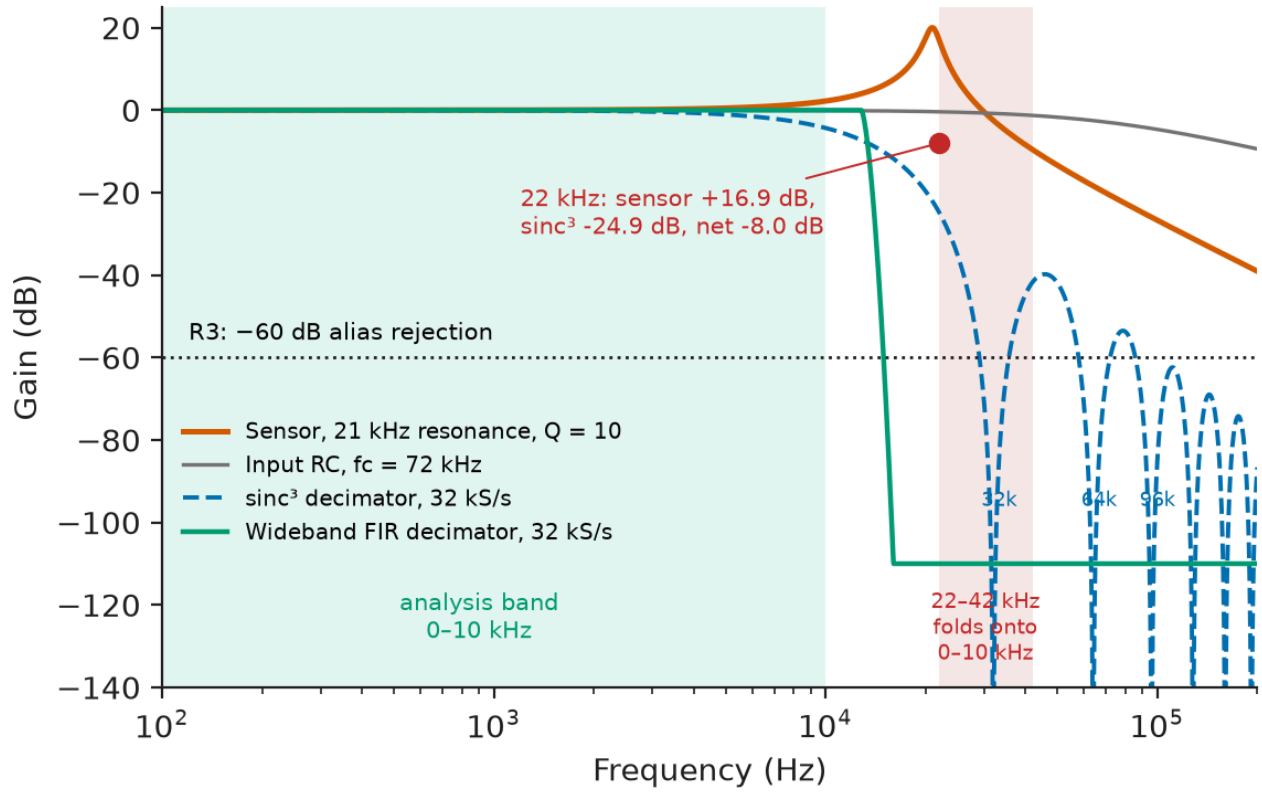


Figure 27.2 — Anti-alias chain of a vibration channel: sensor resonance, input RC, and two converter decimation filters, with the band that folds onto 0-10 kHz shaded.

Band-pass for demodulation

A bearing defect is a train of impacts, each ringing housing resonances at several kilohertz, so the defect frequency appears as the *repetition rate* of bursts in a high band. The node selects that band with an eighth-order Butterworth band-pass, 2–8 kHz, as four biquads. The commissioning survey (Section 27.3) sets the edges: inside them the housing resonances, below them the shaft harmonics, above them the VFD carrier and its sidebands. Band edges are configuration, not code. Second-order sections are mandatory: the largest pole radius is 0.90, but as one eighth-order polynomial the filter would need more coefficient precision than single-precision float offers (Book 3 §23.9).

Decimation and the velocity path

After rectification the envelope occupies 0–500 Hz, so it is decimated by 20 to 1.6 kS/s. The decimator is a 239-tap linear-phase FIR (Kaiser window, 71 dB stopband from 1.1 kHz), implemented as a polyphase filter so that it costs only 12 multiply–accumulates per input sample rather than 239. Its group delay is 119 samples, 3.7 ms at 32 kS/s.

The velocity path for R4 band-limits the acceleration to 10–1000 Hz with a fourth-order Butterworth band-pass and integrates it with a bilinear integrator whose pole is moved from $z = 1$ to $z = 0.9995$, so that offset cannot wind up the state. At the 24.67 Hz running speed the band-pass and integrator together are within 1.1 % of the ideal $1/(2\pi f)$, comfortably inside the ±5 % of R4.

27.3 Spectral analysis

> **Boundary.**
> The DFT, bin interpretation, leakage, windows, zero padding and Welch averaging are owned by *Circuits, Signals and Communications*, Ch. 22 (§22.3, §22.5–22.8). Here they are used to fix record lengths, windows and memory.

What the spectrum must resolve

Spectral analysis on the node has two jobs. The **raw spectrum** (0–10 kHz) identifies shaft-related faults: 1× for unbalance, 2× and 3× for misalignment and looseness, vane-pass (blade count × speed) for hydraulic problems. The **envelope spectrum** (0–500 Hz) identifies bearing defects from the repetition rate of impacts. The design numbers come from the machine.

For the drive-end bearing, a 6309 deep-groove ball bearing (8 balls of 17.46 mm diameter on a 72.5 mm pitch diameter, zero contact angle), and a shaft rate $f_r = 1480/60 = 24.67$ Hz, the standard kinematic formulas give

$$f_{\mathrm{BPFO}} = \frac{n}{2} f_r \left(1 - \frac{d}{D}\cos\phi\right) = 4 \times 24.67 \times (1 - 0.2409) = 74.90 \text{ Hz},$$

with the inner-race frequency at 122.43 Hz, ball spin at 48.24 Hz and cage at 9.36 Hz.

> **Status of the principle: bearing defect frequencies from kinematics**
> *Approximate.* They assume rolling without slip and rigid geometry; in service the rolling elements slip, and measured defect frequencies typically sit 1–2 % below the calculated ones, so detectors search a band around each frequency rather than a single bin.

Worked example — record length for the envelope spectrum. A misaligned coupling also produces a line at $3f_r = 74.00$ Hz, only 0.90 Hz from BPFO. With a Hann window, two equal lines are separated cleanly when they are at least about two bins apart (the −6 dB main-lobe width is two bins), so the bin spacing must be at most 0.45 Hz and the record at least $1/0.45 = 2.2$ s. The node uses 8192 points of envelope at 1.6 kS/s: bins of $1600/8192 = 0.195$ Hz, a record of 5.12 s, and 4.6 bins between the two lines. A longer record would separate them further, but the shaft speed must stay constant over the record: at 5.12 s a load change that moves the slip by 0.1 % of speed moves BPFO by 0.075 Hz, under half a bin. The tachometer measures the speed of every record, and the detector recomputes the defect frequencies from it.

> **Status of the principle: frequency resolution of a finite record**
> *Derived.* The bin spacing of an N-point DFT at rate f_s is $f_s/N = 1/T$; the ability to separate two lines depends on T and on the window's main-lobe width, and zero padding interpolates the spectrum without improving it.

Raw spectrum settings

The raw spectrum uses 16384 points at 32 kS/s (1.95 Hz bins, 0.512 s), Hann window, 50 % overlap, eight averages. Line amplitudes are corrected for the window's coherent gain (0.5) and band energies for its noise

bandwidth (1.5 bins); mixing the two corrections, a common error in hand-rolled code, puts band energies 1.76 dB wrong.

Commissioning survey

Band edges, alarm frequencies and acceptance come from a **commissioning survey**: 60 s raw captures at three loads, a hammer tap test for housing resonances, and a capture at the VFD's operating point to locate the carrier. It is stored with the configuration, so alarms compare against the machine's own baseline.

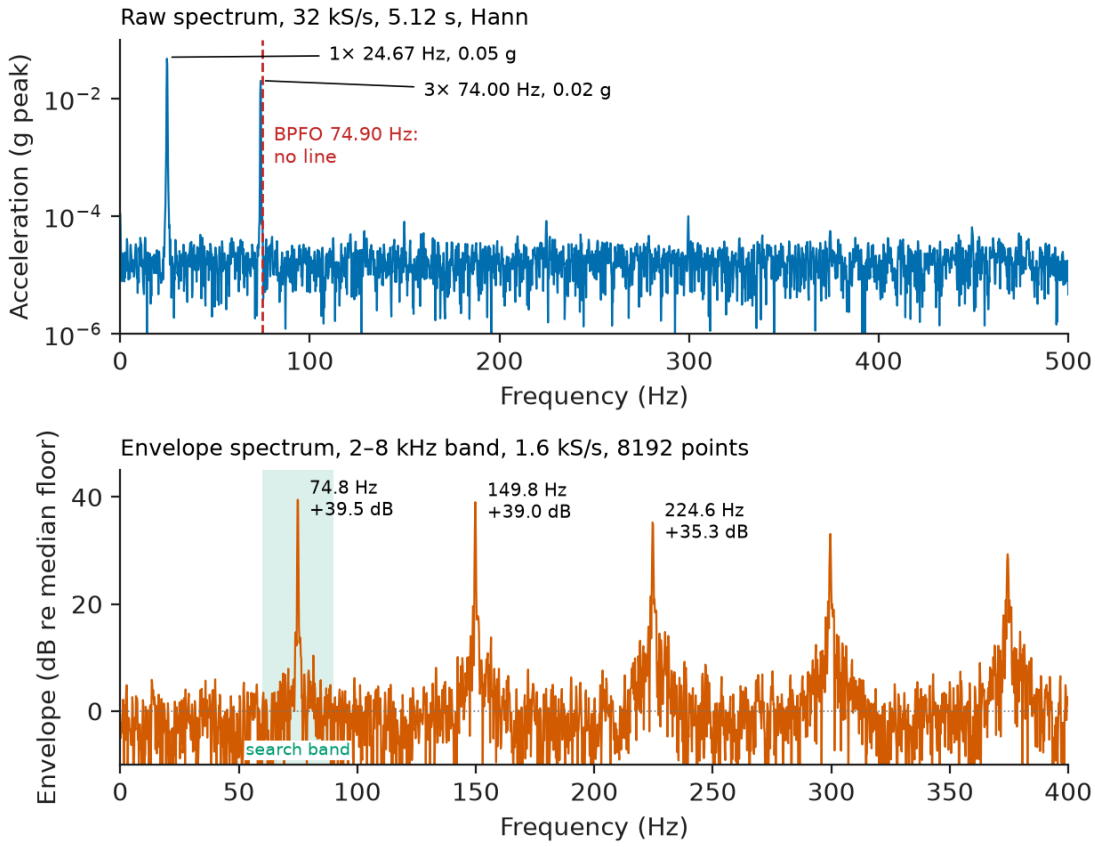


Figure 27.3 — Raw acceleration spectrum and envelope spectrum of the same record with an early outer-race defect: invisible in the raw spectrum, about 40 dB above the floor in the envelope spectrum.

27.4 Feature extraction

From waveform to numbers

The node reduces 128 000 samples per second to a few dozen numbers, each chosen for the fault mechanism it responds to (Table 27.4).

Table 27.4 — Features computed by the pump-condition node.

Feature	Fault mechanism it responds to	Window	Healthy value (this fleet)	Cost per input sample
Velocity RMS, 10–1000 Hz	Unbalance, misalignment, looseness; severity per ISO 20816-3	1 s	1.2–2.3 mm/s	3 biquads + square
Acceleration RMS and peak	General level; impacts	1 s	30–45 mg rms	2 operations
Crest factor (peak/RMS)	Impacting	1 s	3–4	from above
Kurtosis, band-passed 2–8 kHz	Impulsiveness from early bearing damage	1 s	≈ 3	4 biquads + 3 operations
Envelope line at BPFO, BPFI, BSF, FTF	Specific bearing defects	5.12 s	< 10 dB over floor	FFT (shared)
1×, 2×, 3× amplitudes	Unbalance, misalignment	2.3 s	from survey	FFT (shared)
Octave levels 1–8 kHz (microphone)	Cavitation, leaks	1 s	from survey	8 biquads
Bearing housing ΔT over motor frame	Lubrication failure, overload	0.5 s	3–8 K	per frame

Worked example — velocity severity from the spectrum. On a healthy pump the 1× line is 0.05 g peak and the 3× line 0.02 g peak. Velocity amplitude is acceleration divided by $2\pi f$: $0.05 \times 9.807/(2\pi \times 24.67) = 3.16$ mm/s peak, or 2.24 mm/s rms at 1×, and $0.196/(2\pi \times 74.0) = 0.42$ mm/s peak (0.30 mm/s rms) at 3×. Together $\sqrt{2.24^2 + 0.30^2} = 2.26$ mm/s rms. For a group-2 machine (15–300 kW) on a rigid foundation, ISO 20816-3 places zone boundaries at 1.4, 2.8 and 4.5 mm/s rms, so this pump is in zone B (acceptable for unrestricted long-term operation) with 0.54 mm/s before zone C.

> **Status of the principle: vibration-severity zones**
> *Empirical.* The ISO 20816-3 boundaries summarise fleet experience for classes of machine and foundation; they are a starting point that site baselines refine, not a physical limit.

Why kurtosis, and why band-passed

Kurtosis is 3 for Gaussian noise, 1.5 for a sinusoid, and well above 3 for impulsive signals. On the raw waveform the shaft harmonics dominate and hide a weak impact train: in the Section 27.5 simulation raw kurtosis is 1.31, band-passed kurtosis 9.76. The envelope band-pass thus also yields the most sensitive single-number indicator. It is computed in one pass from running sums of x to x^4, accumulated in double precision (in hardware on the Cortex-M7) after the band-pass has removed the mean.

27.5 Event detection

> **Boundary.**
> Detection in noise, thresholds and error probabilities are treated in *Circuits, Signals and Communications*, Ch. 29 §29.6; hypothesis testing in *The Mathematics Universe*, Ch. 20 §20.8. Learned anomaly detectors are in Chapter 32.

Simulating the detector before the rig

Before any bearing is damaged, the chain runs on a synthetic signal: 1× and 3× lines, budgeted sensor noise, and an outer-race impact train at BPFO with 1 % jitter, each impact ringing a 4.5 kHz resonance. The band-pass is the firmware's; the decimator is comparable.

```
 import numpy as np
from scipy import signal, stats

rng = np.random.default_rng(7)
fs, T = 32_000, 5.12                          # S/s, record length s
t = np.arange(int(fs * T)) / fs
fr, bpfo = 1480 / 60, 74.902                  # shaft and outer-race frequencies, Hz

# Synthetic test signal in g: 1x unbalance, 3x misalignment, noise, and a weak
# outer-race defect: one impact per BPFO period ringing a 4.5 kHz resonance
x = 0.050 * np.sin(2*np.pi*fr*t) + 0.020 * np.sin(2*np.pi*3*fr*t)
x += 2.5e-3 * np.sqrt(fs / 2 / 10e3) * rng.standard_normal(t.size)  # 25 ug/rtHz
hits = np.cumsum(np.full(int(T * bpfo), 1 / bpfo) * (1 + 0.01 * rng.standard_normal(int(T *
bpfo))))
ring = np.exp(-800 * t[:200]) * np.sin(2 * np.pi * 4_500 * t[:200])
for k in (hits * fs).astype(int):
    seg = slice(k, min(k + 200, t.size)); x[seg] += 0.02 * ring[:seg.stop - k]

# Envelope chain as on the node: band-pass, rectify, decimate by 20, Hann FFT
sos_bp = signal.butter(4, [2_000, 8_000], btype='bandpass', fs=fs, output='sos')
y = signal.sosfilt(sos_bp, x)
env = signal.decimate(np.abs(y), 20, ftype='fir', n=231)       # 1600 S/s
env -= env.mean()
E = np.abs(np.fft.rfft(env * np.hanning(env.size)))
fe = np.fft.rfftfreq(env.size, 20 / fs)
band = (fe > 60) & (fe < 90)
pk = fe[band][E[band].argmax()]
print(f"envelope bins {fe[1]:.4f} Hz, peak {pk:.2f} Hz, "
      f"{20*np.log10(E[band].max() / np.median(E[(fe > 20) & (fe < 400)])):.1f} dB above floor")
X = np.abs(np.fft.rfft(x * np.hanning(x.size))); fx = np.fft.rfftfreq(x.size, 1 / fs)
print(f"raw spectrum near 74-75 Hz: largest line {fx[(fx>70)&(fx<80)][X[(fx>70)&
(fx<80)].argmax()]:.2f} Hz")
print(f"kurtosis raw {stats.kurtosis(x, fisher=False):.2f}, "
      f"band-passed {stats.kurtosis(y, fisher=False):.2f}")
print(f"rms raw {1e3*x.std():.1f} mg, band-passed {1e3*y.std():.2f} mg")
```

```
 envelope bins 0.1953 Hz, peak 74.80 Hz, 39.5 dB above floor
raw spectrum near 74-75 Hz: largest line 74.02 Hz
kurtosis raw 1.31, band-passed 9.76
rms raw 38.3 mg, band-passed 3.59 mg
```

The result is the opening story in four lines. The overall RMS, 38.3 mg, is within 0.2 mg of the healthy value $\sqrt{35.4^2 + 14.1^2 + 2.5^2} = 38.2$ mg; the raw spectrum shows only the 3× misalignment line at 74.02 Hz; the envelope spectrum shows the defect at 74.80 Hz (0.1 Hz below the calculated 74.90 Hz, within one bin), 39.5 dB above its floor; and band-passed kurtosis has tripled. Figure 27.3 plots both spectra.

Thresholds and the false-alarm rate

A detector is a threshold on a feature, and its false-alarm rate follows from the feature's distribution on a healthy machine. Each feature is normalised by the node's own baseline mean μ and standard deviation σ, learned over the first two weeks after commissioning, and an alarm is raised when $z = (x - \mu)/\sigma$ exceeds a threshold.

Worked example — single threshold versus persistence. Level features are evaluated once per second: 2.592×10^6 evaluations per month. With a Gaussian healthy distribution, a single-sample threshold at 4.5σ has a false-alarm probability of 3.4×10^{-6} per evaluation, or $3.4 \times 10^{-6} \times 2.592 \times 10^6 = 8.8$ false alarms per month per feature, which fails R6 on its own. A **3-of-5 persistence rule** at 3.5σ needs three exceedances among the last five evaluations. The single-evaluation probability is $p = 2.33 \times 10^{-4}$, and the probability of three or more in five is $\sum_{j=3}^{5} \binom{5}{j} p^j (1-p)^{5-j} = 1.26 \times 10^{-10}$, about 3.3×10^{-4} per month per feature. A real fault that shifts the feature mean by 6σ exceeds 3.5σ with probability 0.994 per evaluation and fires the 3-of-5 rule with probability 0.999998, within 3–5 s. Persistence buys six orders of magnitude of false-alarm rate for 2–4 s of latency, well inside R7.

> **Status of the principle: false-alarm rate from a Gaussian feature model**
> *Approximate.* It assumes independent, Gaussian evaluations from a stationary healthy process. Consecutive windows of a slowly varying machine are correlated, and the true tails are heavier, so the calculation sets the order of magnitude and the 30-day soak test of R6 sets the number.

What actually causes false alarms

On the pilot fleet the analytic rate was never the problem. False alarms came from *process* events: starts, valve changes, a second pump switching on. A state machine suppresses level alarms for 30 s after a start, after a speed change over 5 % and while the PLC reports a valve moving; envelope alarms, which respond to the defect signature, are never suppressed. The rules are versioned with the configuration and covered by tests, because undocumented suppression eventually hides a fault.

27.6 Audio processing

> **Boundary.**
> Audio sampling, coding and psychoacoustic fundamentals are in *Circuits, Signals and Communications*, Ch. 24 §24.9; adaptive noise cancellation (LMS) in §24.5; time–frequency analysis in §24.7.

The acoustic channel

Cavitation, a gland leak and a rubbing seal produce broadband airborne noise above 1 kHz that the casing accelerometer sees poorly. The node's fourth channel is an analog MEMS microphone of the ICS-40720 class: sensitivity about −32 dBV/Pa, signal-to-noise ratio about 70 dB(A), acoustic overload near 124 dB SPL (datasheet typical). A preamplifier with 20 dB gain lifts it towards the converter's range; the converter samples it at the same 32 kS/s as the accelerometers, so acoustic and vibration features are time-aligned by construction.

The processing is a sound-level meter cut down to what the fault needs: four octave band-passes at 1, 2, 4 and 8 kHz (two biquads each, band edges per IEC 61260-1), 1 s mean-square integration and conversion to dB SPL. A-weighting models human hearing and is not applied.

Worked example — from volts to sound pressure level. At the microphone (before the preamplifier) a 1 s window measures 4.2 mV rms, or $20\log_{10}(4.2\times10^{-3}) = -47.5$ dBV. A microphone of −32 dBV/Pa produces −32 dBV at 1 Pa, which is 94 dB SPL. The level is therefore $94 + (-47.5) - (-32) = 78.5$ dB SPL. Field calibration with a class 1 calibrator removes the ±1 dB unit-to-unit sensitivity tolerance, which would otherwise use two-thirds of R8's allowance.

Removing the machine from the microphone

The microphone also hears the cooling fan and neighbouring pumps. The reported feature is therefore the *excess* over the commissioning baseline at the current load, in dB; a 6 dB excess at 4 and 8 kHz with no change at 1 kHz is this fleet's cavitation signature. Where another machine dominates, an LMS canceller with the casing accelerometer as reference separates this pump's structure-borne radiation from airborne noise; the pilot did not need it.

Other products use digital MEMS microphones: a PDM stream at 1–3.2 MHz is decimated by 64 with a CIC filter and a compensating FIR (in the STM32 DFSDM or the nRF52840 PDM block), or an I²S microphone does it internally. Leak detectors on water mains correlate two hydrophones to locate a leak from the arrival-time difference. The pattern is the pump node's: calibrate, band-limit to the mechanism, compare with a baseline.

27.7 Image processing

> **Boundary.**
> Two-dimensional sampling, transforms and image coding are in *Circuits, Signals and Communications*, Ch. 24 §24.9; thermal radiation and emissivity in *Engineering Physics*, Ch. 10 §10.8.4.

The thermal channel

A failing bearing often heats before it becomes noisy, and a lubrication failure always does. The node carries an MLX90640-class thermopile array: 32 × 24 pixels, 55° × 35° field of view, noise-equivalent temperature difference about 0.1 K at 1 Hz refresh, absolute accuracy about ±1 K in the central field (datasheet typical), read over I²C. At 2 frames/s it delivers 768 temperatures every 0.5 s, after the vendor's calibration routine has applied each pixel's stored offset, gain and thermal-gradient coefficients.

Worked example — mounting distance from pixel footprint. Each pixel subtends $55^\circ/32 = 1.72^\circ$. At distance d its footprint is $2d\tan(0.86^\circ) = 0.030\,d$: 30 mm at 1.0 m, 45 mm at 1.5 m. The 120 mm bearing housing covers 2.7 pixels at 1.5 m; nearly every pixel also sees cooler background, so the peak reads low. The rule of thumb asks for at least 3 × 3 pixels, preferably four across to allow for misregistration: a 30 mm footprint, so the camera is mounted at no more than 1.0 m. That is R9's distance limit.

> **Status of the principle: minimum target size in pixels**
> *Empirical.* The 3 × 3-pixel rule reflects the optical point-spread of low-cost thermal arrays, in which a pixel's reading is influenced by its neighbours; the exact requirement is measured with a slit or edge target for each lens.

The processing chain

Every step of the image chain answers a specific defect of the sensor or the scene:

1. **Plausibility mask.** Pixels outside −20 to 300 °C, or flagged in the sensor's EEPROM, are marked bad.
2. **Bad-pixel replacement** by the 3 × 3 median of the neighbours, for flagged pixels only; a median over the whole frame would erode a 3 × 3 hotspot.
3. **Emissivity.** Bare machined steel (ε ≈ 0.1–0.3) reflects its surroundings, so high-emissivity tape (ε ≈ 0.95) is applied to the target at installation.
4. **Temporal averaging** over four frames (2 s) halves the pixel noise to about 0.05 K.
5. **Regions of interest** fixed at commissioning: the feature is the bearing's maximum minus the motor frame's median, which cancels ambient temperature and load.
6. **Hotspot segmentation.** Pixels 10 K above the frame reference are labelled as connected components; the largest one's area and centroid say *where*.

```
 import numpy as np
from scipy import ndimage

rng = np.random.default_rng(3)
# Synthetic 24 x 32 frame (deg C): 28 C background, motor frame 41 C,
# drive-end bearing housing 3 x 3 px at 55 C, NETD 0.1 K, one dead pixel
frame = 28.0 + 0.1 * rng.standard_normal((24, 32))
frame[8:18, 6:22] = 41.0 + 0.1 * rng.standard_normal((10, 16))
frame[11:14, 22:25] = 55.0 + 0.1 * rng.standard_normal((3, 3))
frame[5, 27] = -40.0                                  # dead pixel reads out of range

bad = (frame < -20) | (frame > 300)                   # plausibility mask
med = ndimage.median_filter(frame, size=3)
clean = np.where(bad, med, frame)                     # replace only flagged pixels

roi = (slice(9, 16), slice(20, 27))                   # bearing ROI from commissioning
ref = (slice(9, 17), slice(8, 18))                    # motor-frame reference ROI
dT = clean[roi].max() - np.median(clean[ref])         # bearing rise over frame
hot = clean > np.median(clean[ref]) + 10.0            # hotspot rule: +10 K over frame
lab, n = ndimage.label(hot)
sizes = ndimage.sum(hot, lab, range(1, n + 1))
cy, cx = ndimage.center_of_mass(hot, lab, int(np.argmax(sizes)) + 1)
print(f"bad pixels replaced: {bad.sum()}, bearing rise over frame: {dT:.1f} K")
print(f"hotspots: {n}, largest {int(sizes.max())} px at row {cy:.1f}, col {cx:.1f}")
```

```
 bad pixels replaced: 1, bearing rise over frame: 14.1 K
hotspots: 1, largest 9 px at row 12.0, col 23.0
```

The listing is the reference model; the firmware does the same in C on 768 floats per frame, about 0.15 Mcycle per frame. Figure 27.4 shows the frame and the regions.

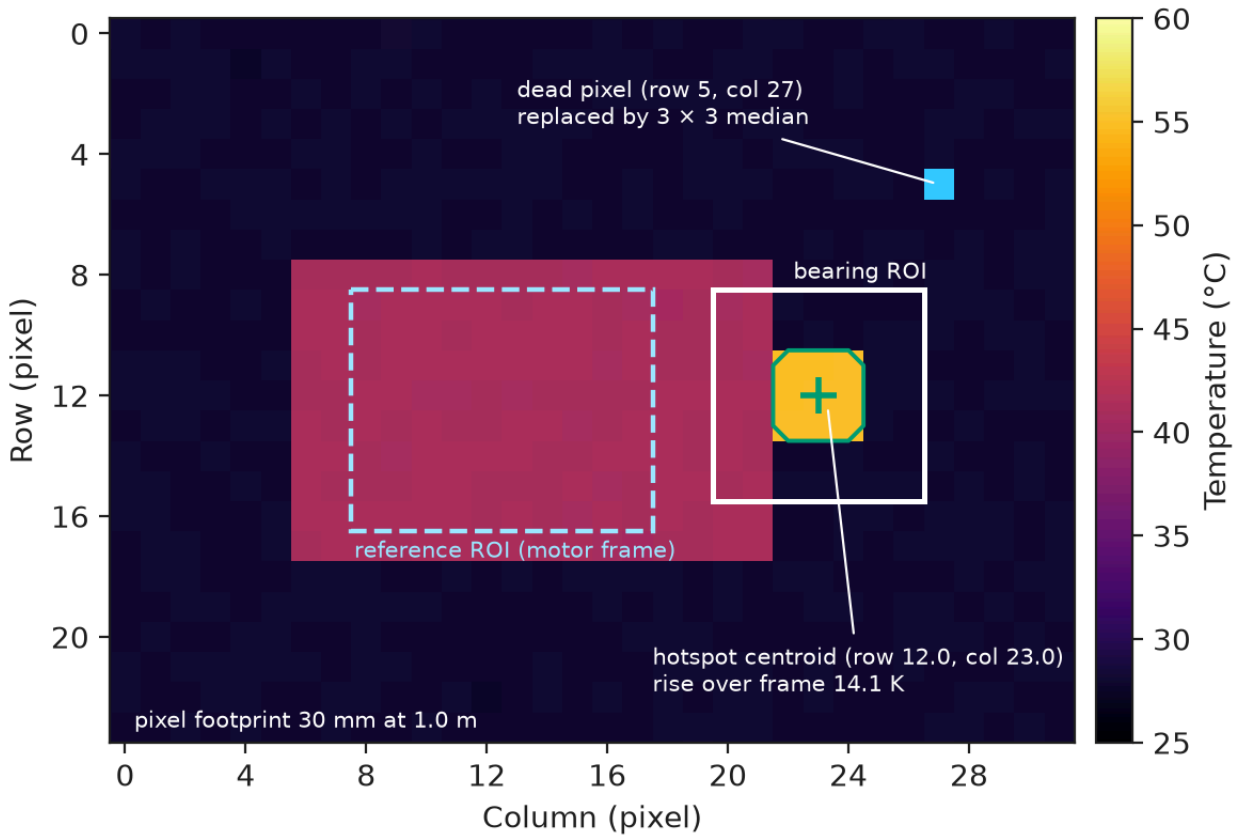


Figure 27.4 — Thermal frame of the motor drive end from the 32 x 24 array, with the bearing and reference regions and the segmented hotspot.

Image processing in other systems

The same discipline (correct the sensor, measure in fixed regions, compare with a reference) carries over to label checks on bottling lines, electroluminescence crack detection in PV modules (Chapter 22) and PCB inspection. What changes is the data rate: a 5-megapixel camera at 20 frames/s produces 100 MB/s, a job for an FPGA or a vision accelerator, not a microcontroller (*Electronics and Computing*, Ch. 35 §35.1–35.3).

27.8 Real-time DSP

> **Boundary.**
> Interrupt latency, scheduling, response-time analysis and jitter are owned by *Electronics and Computing*, Ch. 27 (§27.2, §27.3, §27.10); the real-time DSP workflow by *Circuits, Signals and Communications* Ch. 21 §21.3. MCU selection and firmware architecture at system level are in Chapter 10 of this book.

Processor selection

The processor choice was settled by memory, not arithmetic (Table 27.5).

Table 27.5 — Processor options for the pump-condition node.

Option	Core, clock	SRAM	Relevant peripherals	Verdict
STM32G474	Cortex-M4F, 170 MHz	128 KB	No Ethernet MAC	Arithmetic suffices; memory fails
STM32H723	Cortex-M7, 550 MHz, double FPU	564 KB	SAI (TDM slave), Ethernet MAC	**Chosen**
ESP32-S3	2 × Xtensa LX7, 240 MHz	512 KB + PSRAM	Wi-Fi, I²S; no Ethernet MAC	Wireless unwanted in a steel pump room
STM32MP1-class MPU, Linux	Cortex-A7 650 MHz + M4	external DDR	everything	Boot time, OS upkeep; power near R11

Worked example — the memory budget that ruled out the smaller part. The first plan kept a 16384-point capture and FFT output buffer for all three accelerometers: $3 \times 16384 \times 4\ \text{B} \times 2 = 384$ KB, three times the STM32G474's SRAM. Even on the STM32H723 the naïve layout totals 553 KB of 564 KB, failing R10. The revised layout captures the raw spectrum one channel at a time (round-robin), lets the FFT consume its input, and shares one 64 KB output buffer among all FFTs in one low-priority task (Table 27.6).

Table 27.6 — Memory budget, STM32H723, revised layout.

Item	Size
DMA ping-pong, 2 × 256 frames × 4 channels × 4 B (non-cacheable)	8 KB
Raw capture, one channel at a time, 16384 × 4 B	64 KB
Envelope capture, 3 channels × 8192 × 4 B	96 KB
FFT output buffer, shared, 16384 × 4 B	64 KB
Thermal frames: raw, filtered, background (3 × 768 × 4 B)	9 KB
Filter states, coefficients, feature records	8 KB
RTOS stacks and heap	48 KB
Network buffers (lwIP, MQTT, Modbus TCP)	64 KB
Total	**361 KB**
Available / limit R10 (75 %)	564 KB / 423 KB
Margin against R10	62 KB (11 % of SRAM); 36 % of SRAM free

The block schedule and CPU budget

Processing is block-based. The converter's data port fills a ping-pong buffer by DMA, and every half-buffer of 256 frames (8 ms at 32 kS/s) wakes the DSP task. The DSP task must finish each block in well under 8 ms, or the next block overwrites data still in use. Longer computations (FFTs, thermal frames, reports) run in lower-priority tasks that the DSP task pre-empts (Figure 27.5).

Table 27.7 is the CPU budget. The per-operation cycle counts are estimates for CMSIS-DSP single-precision routines on a Cortex-M7, to be replaced by DWT cycle-counter measurements on the first boards; the margin

is so large that a factor-of-two error in any of them would not matter.

Table 27.7 — CPU budget at 550 MHz (estimated, to be confirmed by measurement).

Task	Basis	Mcycle/s	Load
DMA interrupt and 24-bit unpacking, 4 channels	8 cycles/sample	1.02	0.19 %
Statistics: sums, peak, 4 channels	6 cycles/sample	0.77	0.14 %
Velocity chain, 3 biquads, 3 channels	8 cycles/biquad/sample	2.30	0.42 %
Envelope band-pass, 4 biquads, 3 channels	8 cycles/biquad/sample	3.07	0.56 %
Rectify and polyphase decimator, 3 channels	239 taps at 1.6 kS/s	1.15	0.21 %
Raw spectrum, 16384-point real FFT	0.5 Mcycle, one channel per 0.512 s	0.98	0.18 %
Envelope spectrum, 8192-point real FFT	0.23 Mcycle, 3 per 5.12 s	0.13	0.02 %
Audio octave bank, 8 biquads	8 cycles/biquad/sample	2.05	0.37 %
Thermal processing	0.15 Mcycle per frame, 2 frames/s	0.30	0.05 %
Network stack, Modbus, MQTT, housekeeping	allowance	3.00	0.55 %
Total		**14.8**	**2.7 %**
Requirement R10			≤ 50 % (margin 47 points)

> **Status of the principle: CPU budget as a sum of per-sample and per-block costs**
> *Approximate.* It assumes that cycle counts measured in isolation add, which holds while code and data stay in tightly coupled memory; cache misses, flash wait states and DMA bus contention can add 20–50 % on a Cortex-M7, which is why the budget is confirmed on hardware.

Latency

Alarm latency (R7) sums the stages: half-buffer (8 ms), group delay (the decimator's 3.7 ms), feature window, persistence (up to five evaluations) and the Modbus TCP path (under 50 ms). Level alarms: $0.008 + 0.004 + 5 \times 1 + 0.05 \approx 5.1$ s against 10 s. Envelope alarms: $5 \times 5.12 + 0.06 \approx 25.7$ s against 30 s, the tightest margin in the design; 50 % overlapped records would halve it if needed.

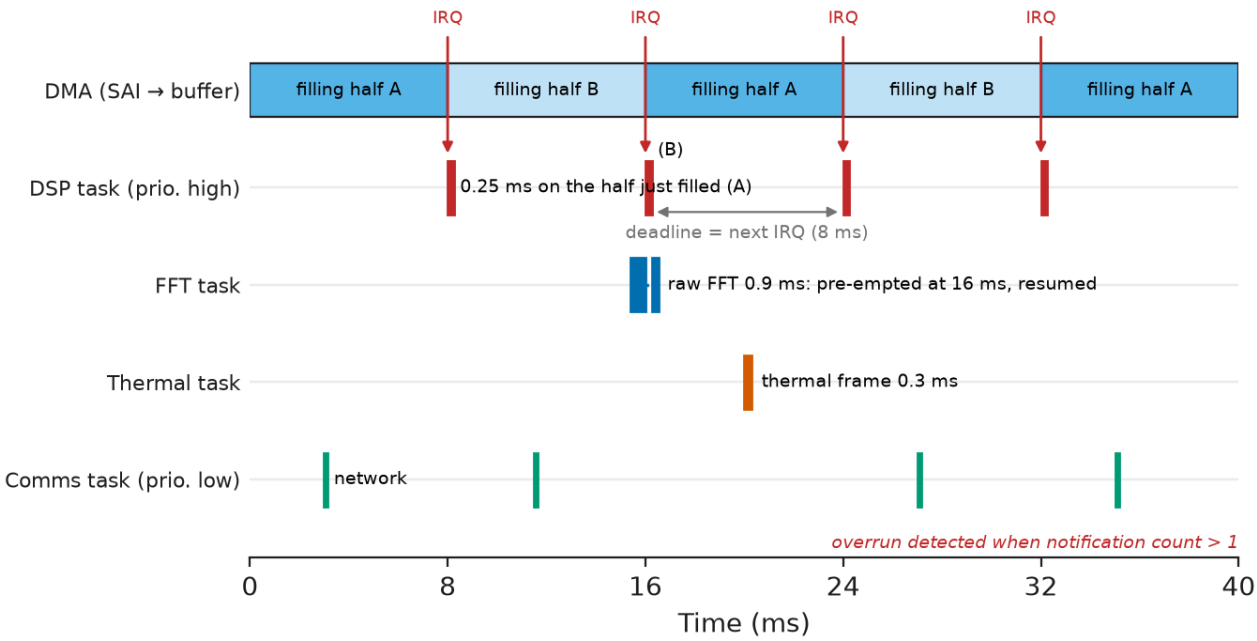


Figure 27.5 — Block schedule of the node: DMA ping-pong halves, DSP task per 8 ms block, and lower-priority FFT and thermal tasks pre-empted by the DSP task.

27.9 Embedded implementation

> **Boundary.**
> C for embedded targets is in *Electronics and Computing*, Ch. 21; fixed- and floating-point formats in *Circuits, Signals and Communications* Ch. 21 §21.5; DMA, caches and driver design in *Electronics and Computing*, Ch. 28 §28.1–28.3.

Floating point or fixed point

On a Cortex-M7 with an FPU, single-precision float is the default: a biquad costs about the same as in saturating Q31, coefficients flow straight from Python, and no scaling analysis is repeated when a band edge changes. Fixed point remains right on cores without an FPU and in FPGA paths, where the scaling and noise-gain analysis of Book 3 §21.6 and §23.9 is done for each filter.

The DSP task

The listing shows the heart of the firmware: the DMA callbacks that hand blocks to the DSP task, the conversion of the converter's 32-bit words (an 8-bit status header above 24 bits of two's-complement data) to acceleration, and the per-block processing. Task and HAL names are FreeRTOS and CMSIS-DSP; the SAI callback names and the linker section are illustrative for an STM32H723 project.

```
 /* dsp_task.c -- block DSP on STM32H723 (C11, FreeRTOS, CMSIS-DSP; HAL names illustrative) */
#include <stdint.h>
#include "FreeRTOS.h"
#include "task.h"
#include "arm_math.h"
#include "node.h"   /* stats_update, envelope_decimate, audio_block, header_expected, fault_raise
*/

#define NCH     4u
#define BLK     256u                        /* frames per half-buffer: 8 ms at 32 kS/s */
#define NBQ_BP  4u
#define ADC_TO_G (4.096f / 8388608.0f / 0.040f)   /* LSB -> V -> g at 40 mV/g */

static uint32_t rx[2][BLK * NCH] __attribute__((section(".dma_nocache"), aligned(32)));
static float32_t xin[BLK], ybp[BLK];
static arm_biquad_cascade_df2T_instance_f32 bp[3];
static float32_t bp_state[3][2 * NBQ_BP];
extern const float32_t bp_coeffs[5 * NBQ_BP];      /* generated from the Python design */
TaskHandle_t dsp_task_handle;
static volatile uint32_t ready_half;

static void dma_block_ready(uint32_t half) {
    BaseType_t woken = pdFALSE;
    ready_half = half;
    vTaskNotifyGiveFromISR(dsp_task_handle, &woken);
    portYIELD_FROM_ISR(woken);
}
void sai_rx_half_cb(void) { dma_block_ready(0u); }
void sai_rx_full_cb(void) { dma_block_ready(1u); }

void dsp_task(void *arg) {
    (void)arg;
    for (unsigned c = 0; c < 3u; ++c)
        arm_biquad_cascade_df2T_init_f32(&bp[c], NBQ_BP, bp_coeffs, bp_state[c]);
    for (;;) {
        uint32_t n = ulTaskNotifyTake(pdTRUE, pdMS_TO_TICKS(20));
        if (n == 0u) { fault_raise(FAULT_ADC_STALLED); continue; }   /* no data in 20 ms */
        if (n > 1u)  { fault_raise(FAULT_DSP_OVERRUN); }             /* a block was lost */
        const uint32_t *blk = rx[ready_half];
        for (unsigned c = 0; c < 3u; ++c) {                          /* accelerometers */
            for (unsigned i = 0; i < BLK; ++i) {
                uint32_t w = blk[i * NCH + c];
                if ((w >> 24) != header_expected(c)) fault_raise(FAULT_FRAME);
                xin[i] = (float32_t)((int32_t)(w << 8) >> 8) * ADC_TO_G;
            }
            stats_update(c, xin, BLK);
            arm_biquad_cascade_df2T_f32(&bp[c], xin, ybp, BLK);
            arm_abs_f32(ybp, ybp, BLK);
            envelope_decimate(c, ybp, BLK);                          /* polyphase FIR, M = 20 */
        }
        audio_block(blk, 3u, NCH, BLK);                              /* channel 3: microphone */
```

```
    }
}
```

Four details carry the design. The notification count from `ulTaskNotifyTake` is an overrun detector: two pending notifications mean a block went by unprocessed. The 20 ms timeout catches a stopped converter instead of letting features freeze. The header byte of every word is checked, so a slipped frame alignment that would silently swap channels becomes a fault. And the DMA buffer sits in a region the MPU marks non-cacheable, because the Cortex-M7 data cache would otherwise serve stale data (Section 27.10). The sign extension `(int32_t)(w << 8) >> 8` relies on arithmetic right shift, implementation-defined in C11 but guaranteed by GCC and Arm Compiler; the coding standard records the dependence.

Bit-exactness against the reference model

Every filter, FFT scaling and feature has a Python counterpart, and coefficients are *generated* into a C header by the design script, never typed. A host build of the processing modules (same sources, portable CMSIS-DSP) runs stored test vectors, and its features must match Python's to a relative error of 10^{-5}. That check runs on every commit (Chapter 29) and catches the common bugs (SciPy's `[b0, b1, b2, a0, a1, a2]` against CMSIS's `{b0, b1, b2, -a1, -a2}`, a forgotten window correction, a decimator phase error) before a board is involved.

27.10 Performance evaluation

Verification

Table 27.8 closes each requirement for the first article (two nodes) and the pilot (eight nodes); detection numbers come from a test rig with seeded defects, not field failures.

Table 27.8 — Verification of the pump-condition node (first article and pilot).

Req.	Test	Limit	Result	Pass/fail
R1	Shaker sweep 10 Hz–10 kHz vs reference accelerometer	±1 dB to 5 kHz, ±3 dB to 10 kHz	+0.4/−0.3 dB to 5 kHz; +2.1 dB at 10 kHz (sensor resonance rising)	Pass
R2	Quiet mount, 10 Hz–10 kHz	≤ 3 mg rms	2.6 mg rms	Pass
R3	Electrical sweep 10–200 kHz, worst in-band alias	≥ 60 dB	> 100 dB; noise-floor limited	Pass
R4	Shaker 25 Hz and 80 Hz, 2–10 mm/s	±5 %	−1.2 % at 25 Hz, +0.8 % at 80 Hz	Pass
R5	Seeded 0.5 mm outer-race spall, 20 runs, 3 loads	$P_D \geq 0.95$	20 of 20 detected; envelope BPFO 18–34 dB over floor	Pass
R6	30-day soak, 8 healthy pumps	≤ 1 per node-month	3 alarms in 240 node-days (0.38 per node-month), all at pump starts; start suppression then extended	Pass
R7	Playback with GPIO timestamps	≤ 10 s / ≤ 30 s	4.1 s level; 21.0 s envelope (worst of 20)	Pass
R8	Calibrator, 94 dB at 1 kHz; band levels vs class 1 meter	±1.5 dB	0.3 dB at 1 kHz; ≤ 0.9 dB in all bands	Pass
R9	Heated target at 1.0 m, contact PT100 reference	±2 K; ΔT ≥ 10 K detected	−1.4 K; 10 K step detected in 2 s	Pass
R10	DWT cycle counter; linker map	≤ 50 % CPU, ≤ 75 % RAM	4.0 % CPU (estimate 2.7 %); 64 % RAM	Pass
R11	Input power at 18/24/30 V	≤ 2 W	1.05 W at 24 V	Pass

The measured CPU load was 50 % above the estimate, mostly cache misses in the network stack, as the status box of Section 27.8 warned, and still an order of magnitude inside the limit.

Detection performance as a curve, not a number

Sweeping the threshold over the rig and soak data gives a receiver operating characteristic for each feature (Figure 27.6). The envelope feature dominates band-passed kurtosis at low false-alarm rates; overall RMS is nearly useless for early outer-race defects, the quantitative form of the opening story. The chosen operating point (3.5σ, 3-of-5) is recorded with the configuration, so a later threshold change is a documented move along a known curve.

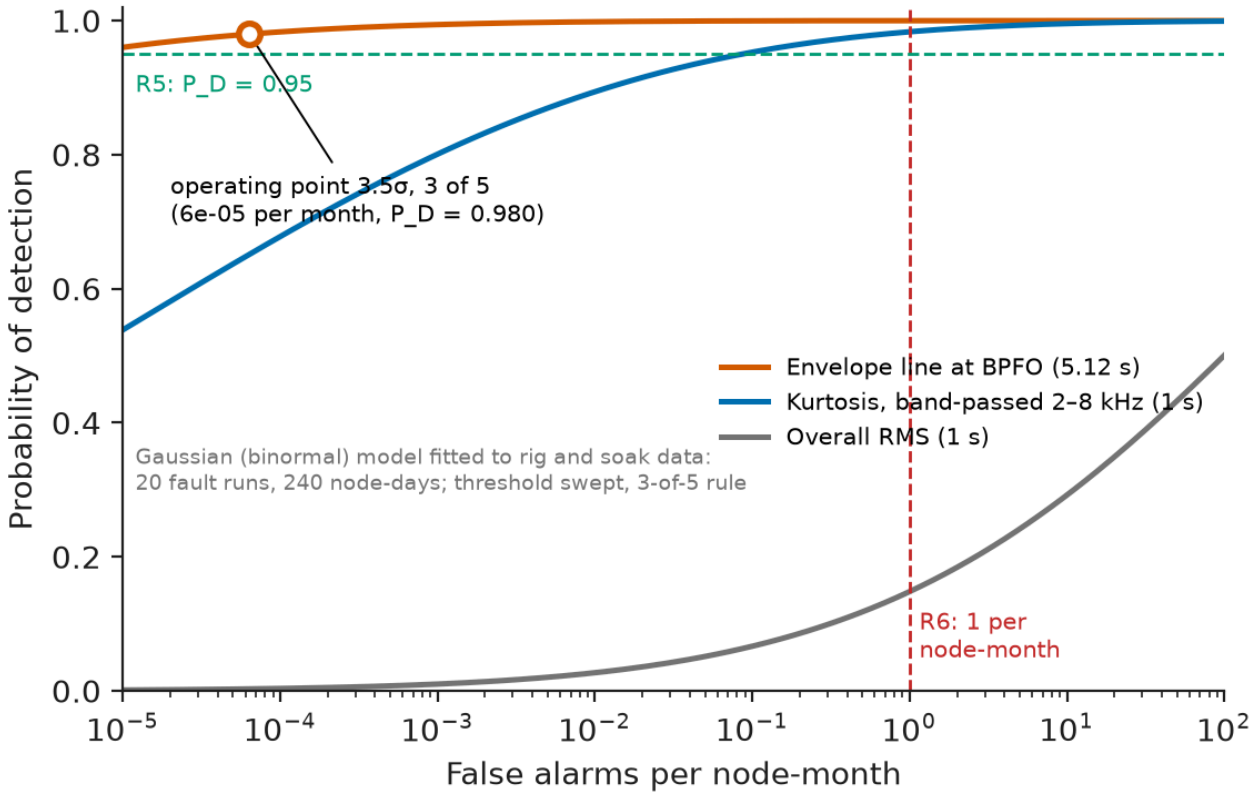


Figure 27.6 — Detection performance of three features for a seeded 0.5 mm outer-race defect: probability of detection versus false alarms per node-month as the threshold is swept.

Failure analysis

Table 27.9 lists the failure modes found during development and the pilot, and the change each caused.

Table 27.9 — FMEA of the pump-condition node.

Failure mode	Effect	Cause	Detection	Mitigation (implemented)
Stale DMA data in cache	Repeated 32-byte fragments; kurtosis spikes on all channels at once	DMA buffer in cacheable SRAM on the Cortex-M7	Simultaneous spikes on all channels; raw dump shows fragments repeated from 16 ms earlier	DMA buffers in an MPU non-cacheable region
VFD carrier in envelope band	False BPFO-band energy on drive-fed pumps	One site's 7 kHz carrier and sidebands inside 2–8 kHz	Survey shows lines at carrier ± k × output frequency	Band edges per installation from the survey
Mounting resonance moved into band	Envelope floor rises; sensitivity falls	Temporary magnetic mount; resonance fell from 21 to about 3 kHz	Tap test; envelope-floor trend	Stud mount at specified torque only
Speed change smears defect lines	Missed envelope detection	Slip varies with load	Tachometer shows > 0.2 % change within a record	Defect frequencies from tachometer per record; unstable records discarded
Converter stall	Features freeze; no alarms	Lost MCLK after a supply dip	20 ms notification timeout	Fault over Modbus; converter re-initialised; validity flags
DSP overrun	Lost blocks; spectral artefacts	Debug logging in the DSP task	Notification count > 1	Logging moved to low priority; overrun counter reported
Clipping during cavitation	Kurtosis and peak invalid	Impacts above ±50 g at the casing	Count of samples above 95 % of full scale	Features flagged invalid; ±100 g variant for casing positions
Thermal lens fouled by oil mist	Bearing reads low; hotspots missed	Lens contamination	Frame reference drifts > 3 K from motor PT100	Cleaning interval; fouling alarm on the drift

Worked example — diagnosing the simultaneous spikes. In first-article testing, both nodes reported kurtosis spikes of 20–40 on all three accelerometers at the same instant, with nothing on the reference accelerometer. No mechanical event reaches three sensors in the same millisecond with identical shape, so the cause was common to the processing. A raw dump around a spike showed 32-byte fragments, exactly one Cortex-M7 cache line, identical to the data 16 ms earlier: the DMA had written new samples, but the core read stale cache lines from the previous pass through the same half-buffer, and each splice made a step that the band-pass turned into an impulse. A non-cacheable region removed the spikes in a 72-hour rerun. Invalidating the lines after each half-transfer would also work, at about 0.1 % CPU, but a region attribute cannot be forgotten in a later code change.

The pilot also made each report carry validity flags for every feature (clipped, speed-unstable, sensor fault), so that trend analysis and any later learned model (Chapter 32) never train on invalid data.

What remains open

Open question	Where it is answered
How are sensors selected and calibrated in general?	Chapter 6 (sensors), Chapter 7 (conditioning and uncertainty)
How are DAQ architecture, synchronisation and logging designed beyond one node?	Chapter 8
How would an anomaly model run on the node within the remaining CPU headroom?	Chapter 12 (edge inference); Chapter 32 (training)
How does the node's MQTT and Modbus reporting fit a plant network securely?	Chapter 11 (IoT); Chapter 21 (industrial communication and OT security)
How are ECG and EMG signals filtered, and what changes when the signal source is a patient?	Chapter 28
How are the firmware's processing modules tested on every commit against the Python reference?	Chapter 29
How are learned classifiers and anomaly detectors built and evaluated on engineering data?	Chapter 32

Next: Chapter 28 applies the same chain (noise budget, filtering, detection and verification) to signals measured on people, where the source impedance is skin and the first requirement is patient safety.

Chapter 28 — Biomedical Instrumentation

Part VII · Communications, Signal Processing, and Biomedical Applications

A start-up brings its seven-day ambulatory ECG recorder to a hospital's clinical-engineering department for a pre-clinical evaluation. On the bench, driven by a patient simulator, its traces are textbook-clean. On the first volunteer they are not: a 50 Hz hum of 40 µV whenever the volunteer sits near the desk lamp, the baseline jumping off-screen each time she stands up, and spurious beats during a stair climb. Then the evaluation stops. To check electrode contact the engineers had connected the recorder by USB to a laptop running on its mains adapter, and the safety analyser reports 160 µA flowing through the patient electrodes to earth, sixteen times the limit for a cardiac applied part. None of these faults is exotic. The simulator presents a few hundred ohms of perfectly matched source impedance, a body presents tens of kilohms of mismatched, drifting skin; the hum was the mismatch converting common-mode voltage into a differential signal; the baseline jumps were electrode offsets the front end had not been sized for; the leakage came through the laptop adapter's Y-capacitor; and nobody had written down which equipment may touch the recorder while it touches a patient. Biomedical instrumentation is the discipline of designing for the body as the signal source and the patient as the first stakeholder.

> **How is a biopotential front end sized for skin, electrode offsets and mains interference, and how is it proved to reject them?**
> **What do patient safety, isolation and medical-device regulation demand of the architecture, the firmware and the test plan?**

Concept	Engineering destination
Biopotential measurement	ECG, EMG, EEG and EOG front ends; electrode choice and skin preparation
ECG acquisition	Holter and patch recorders, bedside monitors, diagnostic 12-lead carts, wearables
EMG acquisition	Rehabilitation and gait labs, prosthesis control, ergonomics, sports science
Biomedical sensors	PPG and SpO_2, temperature, respiration, activity and motion reference
Isolation and safety	Applied-part classification, leakage-current limits, MOPP, defibrillation protection
Signal conditioning	Gain and range for electrode offset, RFI filtering, driven right leg, digital filters
Noise rejection	Mains interference, baseline wander, motion artefact, muscle noise
Medical data acquisition	Sampling, timing, storage formats, data integrity, energy budgets
Basic biomedical signal analysis	QRS detection, heart rate and HRV, EMG envelopes and fatigue indices
Regulatory and ethical considerations	IEC 60601 family, ISO 14971, IEC 62304, clinical evaluation, data protection

Roadmap. The signals and the electrode interface (28.1) → the reference ECG recorder, its requirements and architecture (28.2) → the same front end as an EMG instrument (28.3) → companion sensors (28.4) →

patient safety and isolation (28.5) → the front-end circuit (28.6) → interference and its budget (28.7) → acquisition, storage and energy (28.8) → analysis on the recorded data (28.9) → regulation, ethics, verification and failure analysis (28.10).

28.1 Biopotential measurement

> **Boundary.**
> Instrumentation-amplifier and difference-amplifier analysis, including CMRR, is owned by *Electronics and Computing*, Ch. 8 §8.3 and §8.6; ADC architectures by Ch. 10 §10.10. Magnetic pickup in lead loops is in *Circuits, Signals and Communications*, Ch. 2 §2.4, and the AC-coupled ECG baseline-recovery transient in Ch. 6 §6.9. Sensor selection in general is Chapter 6, signal conditioning and uncertainty Chapter 7, and practical filtering and detection Chapter 27 of this book.

The signals

Biopotentials are small, slow and riding on much larger unwanted voltages. Table 28.1 lists the signals a front end meets and, just as important, what arrives with them.

Table 28.1 — Biopotential signals and their companions at the electrodes (typical ranges).

Signal	Amplitude	Useful bandwidth	Typical source impedance
ECG, surface	0.5–4 mV (R wave)	0.05–150 Hz (diagnostic), 0.5–40 Hz (monitoring)	10 kΩ–1 MΩ at 10 Hz, skin- and preparation-dependent
Surface EMG	0.05–5 mV	20–450 Hz	10–500 kΩ
EEG, scalp	5–100 µV	0.5–70 Hz	5–20 kΩ with gel
EOG	50–3500 µV	DC–30 Hz	10–100 kΩ
Electrode offset (DC)	up to ±300 mV between electrodes	DC, drifting over minutes	—
Mains interference (common mode)	1 mV–10 V at the body	50 or 60 Hz and harmonics	—
Motion artefact	0.1–10 mV	0.1–10 Hz	—

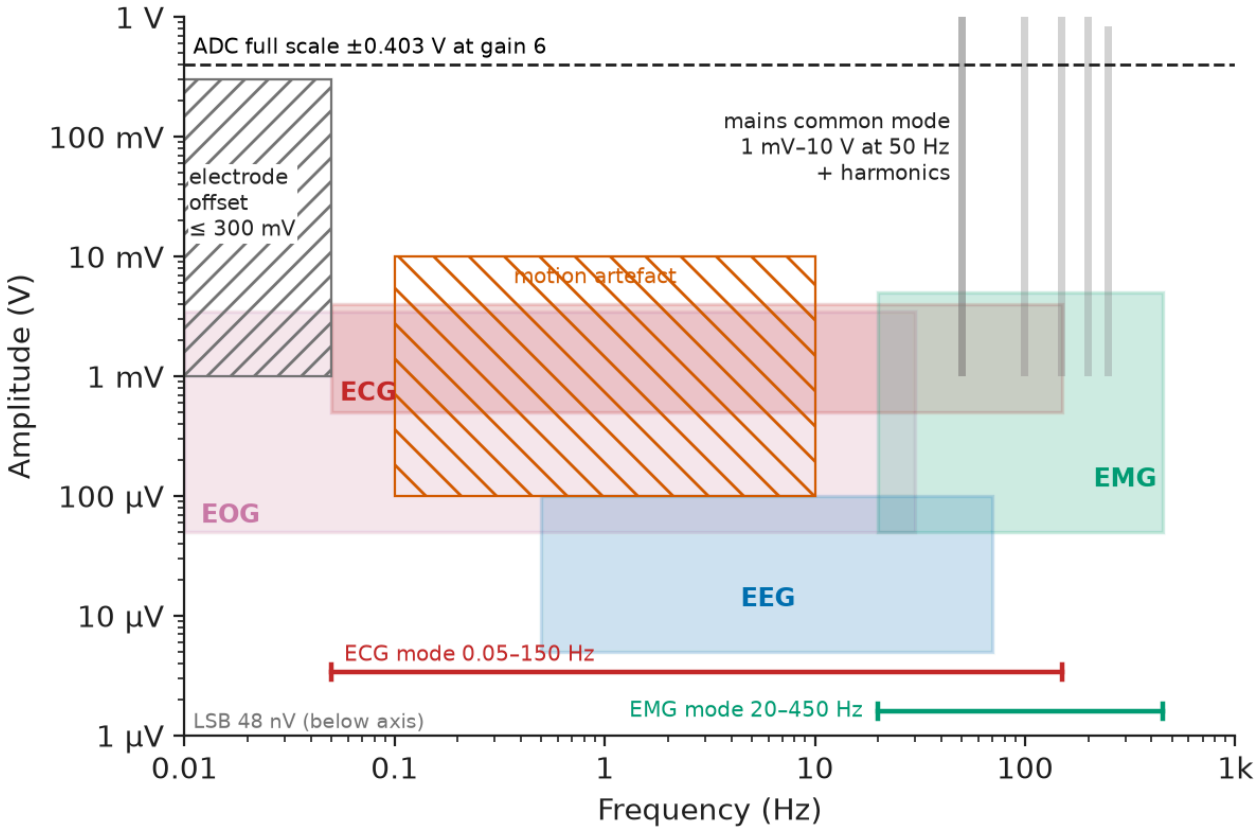


Figure 28.1 — Amplitude-frequency map of biopotential signals and their interferers, with the ECG and EMG passbands of the reference recorder.

Figure 28.1 shows the problem at a glance: the wanted ECG sits two to three decades below the electrode offset and up to four decades below the mains common-mode voltage on the body.

The electrode–skin interface

The electrode is a transducer from ionic current in tissue to electronic current in a wire. Wet silver/silver-chloride electrodes are the standard for ECG because their half-cell potential is stable and their polarisation low; dry and textile electrodes trade higher and more variable impedance for comfort. Three properties of the interface set the front-end requirements:

- **Offset.** Each electrode develops a half-cell potential of a few hundred millivolts against the skin; between two electrodes the difference can reach ±300 mV and drifts as gel dries and sweat changes. The ECG standards require the front end to work with ±300 mV of DC offset, which forbids high gain before the offset is handled.
- **Impedance and imbalance.** The skin's outer layer behaves as a resistance in parallel with a capacitance, typically 10 kΩ–1 MΩ at 10 Hz and very different between two electrodes. The standards model the imbalance as 51 kΩ in parallel with 47 nF in one lead, and that network appears in the common-mode test of Section 28.7.
- **Noise and artefact.** The interface generates a few microvolts of noise and, when the skin stretches under the electrode, millivolts of motion artefact as the charge layer is disturbed. Skin preparation (light abrasion, alcohol) lowers both; adhesive and lead strain relief keep the electrode still.

> **Status of the principle: electrode–skin interface model**
> *Empirical.* The parallel resistance–capacitance model and the 51 kΩ ‖ 47 nF imbalance are conventions fitted to measured skin; real impedance depends on site, preparation, humidity, time since application and frequency, and the model only bounds the design.

28.2 ECG acquisition

The reference system

The running example is a **two-channel, seven-day ambulatory ECG recorder** (an extended Holter): a 60 × 40 × 12 mm sealed unit worn on a lanyard or belt clip, five snap electrodes on a short shielded patient cable, two bipolar leads (CM5 and a modified lead II), a three-axis accelerometer for activity, an event button for the patient diary, a sealed lithium-polymer cell charged by USB in a cradle, and a Bluetooth Low Energy link used only for the hookup check and for status. The recording stays on internal storage and is read out after the device is returned. Table 28.2 fixes the requirements.

Table 28.2 — Requirements of the ambulatory ECG recorder.

ID	Requirement	Target	Verification
R1	Channels	2 ECG leads + 3-axis activity	Inspection
R2	Input range	±5 mV ECG on ±300 mV DC offset without clipping	Offset and amplitude test with calibrated source
R3	Bandwidth	0.05–150 Hz (−3 dB), linear phase in the high-pass	Sine sweep; triangle and impulse response
R4	Noise	≤ 30 µV peak-to-valley referred to input, 10 s, with 51 kΩ ‖ 47 nF in each lead	Shorted-input and imbalance tests
R5	Mains interference	≤ 20 µV peak-to-valley in the common-mode test with 51 kΩ ‖ 47 nF imbalance	Common-mode test rig
R6	Sampling and resolution	500 S/s per channel stored; LSB ≤ 1 µV	Inspection of data; ramp test
R7	Recording time	≥ 7 days with ≥ 20 % battery reserve, 25 °C	Energy budget and full-length run
R8	Patient leakage	≤ 10 µA normal condition, ≤ 50 µA single fault (type CF)	IEC 60601-1 leakage tests
R9	Patient auxiliary current (DC)	≤ 10 µA	IEC 60601-1 measuring device
R10	QRS detection	Sensitivity ≥ 99 %, positive predictivity ≥ 99 %	ANSI/AAMI EC57 method on MIT-BIH Arrhythmia Database
R11	Lead-off detection	Flagged within 2 s; status stored with the data	Electrode removal test
R12	Data integrity and time	CRC on every block; clock error ≤ 30 s over 7 days	Corruption injection; 7-day clock test

R3 follows the AHA/ACCF/HRS recommendations for ECG standardisation: 0.05 Hz at the low end preserves the ST segment, which a 0.5 Hz monitoring corner would distort (Book 3 §6.9 works the transient), and 150 Hz at the high end preserves QRS amplitude and notching in adults.

Architecture

Figure 28.2 shows the architecture. The analog front end is a two-channel biopotential converter with integrated programmable-gain amplifiers, 24-bit delta-sigma ADCs, a driven-right-leg amplifier and lead-off detection. It talks SPI to a Bluetooth microcontroller, which filters and packs the data, detects QRS complexes for the status display, and writes 64 KB blocks to a power-gated microSD card. The USB connector and the patient cable share one socket, so the recorder cannot be connected to a computer and a patient at the same time (Section 28.5).

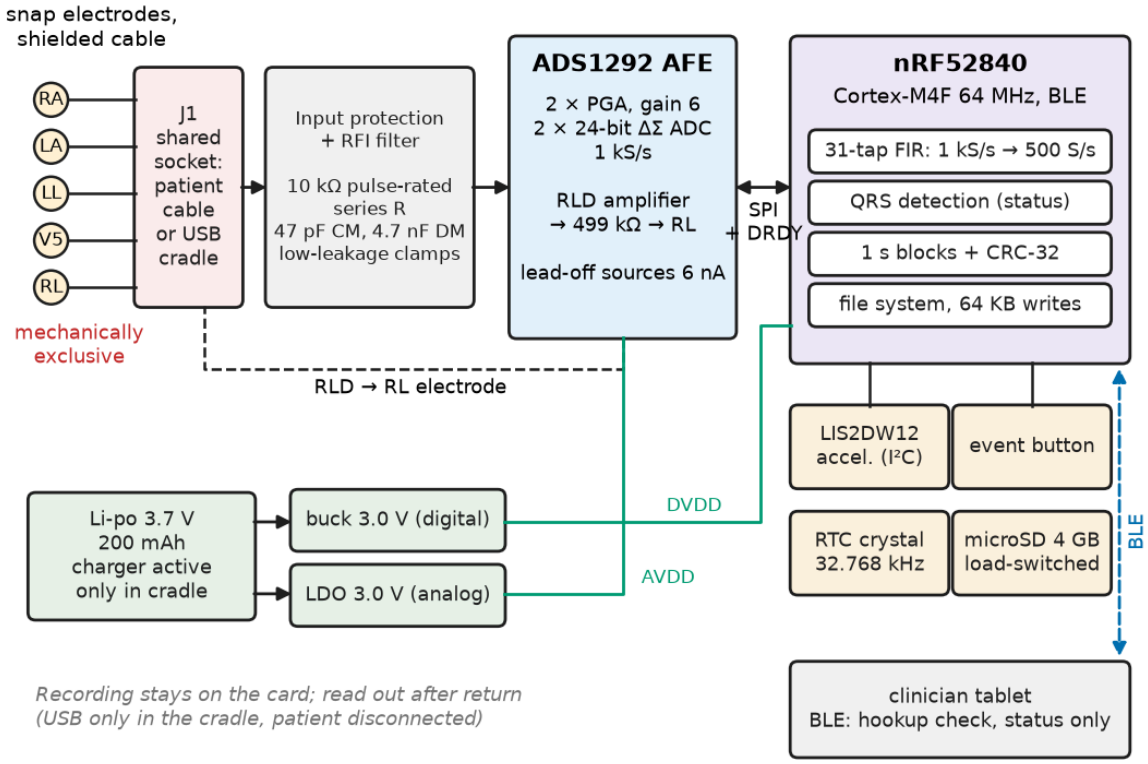


Figure 28.2 — Architecture of the seven-day ambulatory ECG recorder.

Component selection

The front end is chosen against R2–R7 (Table 28.3).

Table 28.3 — Front-end options for a two-channel ambulatory ECG (2026 distributor prices, 100 pcs, indicative).

Option	Example part	Channels; resolution	Noise, power (datasheet typical)	Fit	Price
Integrated biopotential AFE	ADS1292	2; 24 bit, PGA 1–12, RLD, lead-off	8 µV p-p (gain 6, 150 Hz); 0.335 mW/channel	Meets R2–R7; DC-coupled	≈ $9
Ultra-low-power single-lead AFE	MAX30003	1; 18 bit, built-in R-R detector	about 85 µW	Fails R1 (one lead)	≈ $7
Analog heart-rate front end + MCU ADC	AD8232 + 12-bit SAR	1; AC-coupled, fixed filters	170 µA; MCU ADC limits resolution	Fails R1, R3 (fixed corners)	≈ $4
Eight-channel diagnostic AFE	ADS1298	8; 24 bit	0.75 mW/channel	Meets all; three times the power and price	≈ $30

The ADS1292 is chosen: two channels, DC-coupled with enough range for the offset, and an integrated right-leg driver and lead-off detection, which together would otherwise take three op-amps and a dozen precision passives.

Worked example — gain, range and resolution. With the internal 2.42 V reference and a PGA gain of 6, the input full scale is $\pm 2.42/6 = \pm 0.403$ V. The worst input is the ±300 mV offset plus a 5 mV ECG, 0.305 V, leaving 98 mV of headroom for offset drift. The LSB is $2 \times 0.403\ \text{V}/2^{24} = 48$ nV, twenty times finer than R6 asks. Because the front end is DC-coupled, the offset is removed digitally by the 0.05 Hz high-pass; there is no coupling capacitor to saturate on a step and no 18 s recovery after an electrode bump. A gain of 12 would halve the noise contribution of the converter but leave only ± 0.202 V, which the offset requirement forbids.

The converter's sinc3 decimation filter has its −3 dB point at $0.262 f_{\text{data}}$: 131 Hz at 500 S/s, which fails R3. The front end therefore runs at 1 kS/s, where 150 Hz is only 0.97 dB down, and the microcontroller decimates to 500 S/s with a 31-tap linear-phase least-squares FIR: together with the converter's filter the response is −1.0 dB at 150 Hz and more than 100 dB down above 350 Hz, the band that would fold onto 0–150 Hz. The filter costs 31 000 multiply–accumulates per second, a few microwatts of CPU time; the doubled interrupt rate costs more and is in the budget of Section 28.8.

28.3 EMG acquisition

The same hardware, a different instrument

A rehabilitation lab wants two channels of surface EMG to study quadriceps activation during gait retraining. The reference recorder's front end serves, with a different configuration and electrode placement:

- **Electrodes** per the SENIAM recommendations: bipolar Ag/AgCl, 10 mm contact diameter, 20 mm inter-electrode distance, aligned with the muscle fibres over the muscle belly, reference electrode on a bony site (patella).
- **Bandwidth** 20–450 Hz. The 20 Hz high-pass removes most motion artefact and the ECG's low-frequency content, which contaminates trunk-muscle EMG.
- **Sampling** at 2 kS/s, where the converter's sinc3 response is −2.2 dB at 450 Hz (equalised digitally) and its data rate is $2 \times 2000 \times 3 = 12$ kB/s, or 43 MB per hour.
- **Gain** stays at 6: surface EMG reaches 5 mV and the offset requirement is unchanged.

Every hardware decision of Section 28.2 transfers; only the digital filters, rate and analysis change. That is the value of a DC-coupled, high-resolution front end: the instrument is defined in firmware.

EMG processing

EMG is a stochastic interference pattern of motor-unit action potentials, so its useful features are statistical. Three are standard:

- **Linear envelope**: full-wave rectification and a 125 ms moving RMS (250 samples at 2 kS/s), normalised to a maximum voluntary contraction (MVC) recorded at the start of the session.
- **Onset detection**: the envelope exceeds the resting baseline mean plus three standard deviations for at least 25 ms, the same persistence logic as the detectors of Chapter 27, Section 27.5.

- **Fatigue index**: the median frequency of the power spectrum in 0.5 s windows (Welch, Hann). During a sustained contraction at 50 % MVC, median frequency typically falls by 20–30 % within a minute as conduction velocity drops.

Worked example — what the sample rate costs. EMG at 2 kS/s on two channels produces 12 kB/s, four times the ECG rate. A one-hour gait session fills 43 MB, trivial for the 4 GB card, but streaming it live over Bluetooth Low Energy needs a sustained 96 kbit/s of application throughput, which a 7.5 ms connection interval with data-length extension supports with a modest margin and roughly 3 mA of average radio current. The lab therefore records to the card and streams a 20 S/s envelope (two channels × 2 bytes × 20 = 80 B/s) for live display: three orders of magnitude less radio time for the feature the clinician actually watches.

28.4 Biomedical sensors

Biopotentials are rarely recorded alone. Table 28.4 lists the companion sensors a wearable or bedside instrument typically carries, and what each contributes to the reference recorder or to products built on the same platform.

Table 28.4 — Companion biomedical sensors (typical part classes and performance).

Sensor	Part class	Measures	Performance (typical)	Role
Accelerometer	LIS2DW12, 3-axis	Activity, posture, motion	1–2 µA at 12.5–25 S/s low-power mode	**In the reference recorder**: activity annotation and motion-artefact reference
Photoplethysmography (PPG)	MAX30101, red/IR/green LEDs	Pulse, SpO_2	SpO_2 accuracy ±2–3 % with calibration (ISO 80601-2-61 method)	Wrist and finger devices; pulse-transit time with ECG
Body temperature	MAX30205	Skin temperature	±0.1 °C, 37–39 °C (ISO 80601-2-56 for clinical thermometers)	Fever monitoring patches
Respiration, impedance	ADS1292R channel 1	Thoracic impedance change at 32–64 kHz	about 0.1 Ω resolution on 500 Ω–2 kΩ base	Respiration rate from ECG electrodes
Pressure	MPX-class piezoresistive	Cuff pressure (NIBP)	±3 mmHg (ISO 81060-2 validation)	Blood-pressure monitors

The accelerometer earns its place in the recorder. Motion artefact correlates with acceleration of the torso, so the activity channel lets the analysis software mark periods where morphology is unreliable, and lets a cardiologist correlate a rate change with stairs rather than arrhythmia. The PPG row carries a design caution:

optical sensors are affected by skin pigmentation, and an SpO_2 algorithm calibrated on a narrow population overestimates saturation in others, an ethical as well as an engineering defect (Section 28.10).

28.5 Isolation and safety

> **Boundary.**
> Shock physiology, body impedance, touch-voltage limits and earthing are owned by *Electrical Energy and Machines*, Ch. 29 §29.1 and §29.3. A patient differs from the worker of that chapter in two ways that matter here: the skin's protective resistance is bypassed by gel electrodes, and a current path that reaches the heart directly (catheters, pacing leads) makes microamperes dangerous.

Applied parts and leakage limits

IEC 60601-1 classifies the parts that touch the patient. Type B parts may be earthed; type BF parts are floating (isolated from earth and from other parts); type CF parts are floating with the strictest limits, required for direct cardiac application and usual for ECG equipment. Table 28.5 gives the AC limits the reference recorder is designed to.

Table 28.5 — Selected leakage-current limits of IEC 60601-1 (AC, µA).

Current	Type B (NC / SFC)	Type BF (NC / SFC)	Type CF (NC / SFC)
Patient leakage	100 / 500	100 / 500	10 / 50
Patient auxiliary current	100 / 500	100 / 500	10 / 50
Touch current	100 / 500	100 / 500	100 / 500

NC is normal condition, SFC single-fault condition. DC patient and auxiliary currents are limited to 10 µA in normal condition for all types, which matters for the right-leg driver (Section 28.6). The standard also asks for **means of protection**: two means of patient protection (2 MOPP) between mains and the applied part, each tested at 1500 V ac for a 250 V working voltage, 4000 V ac for both together, with 8 mm creepage and 5 mm clearance for 2 MOPP.

> **Status of the principle: leakage-current limits**
> *Empirical.* The limits come from the thresholds of perception and of ventricular fibrillation for currents applied to skin and directly to the heart, with safety factors set by committee; they are design limits written into the standard, not physical constants.

The leakage that stopped the evaluation

Worked example — a laptop adapter as a leakage source. A class II laptop adapter has a Y-capacitor of about 2.2 nF from the primary to the DC output, to meet emission limits. At 230 V and 50 Hz it drives

$$I = 2\pi f C V = 2\pi \times 50 \times 2.2 \times 10^{-9} \times 230 = 159\ \mu\text{A}$$

into anything that connects the DC output to earth. With the recorder on USB and the patient touching an earthed bed frame, the patient is that connection. The 159 µA exceeds the type CF limit of 10 µA sixteen times and the type BF limit of 100 µA as well (Figure 28.3). The recorder itself was innocent; the system of recorder plus laptop was not a medical electrical system.

Three architectures fix it, in increasing cost:

1. **Exclusive connector.** The patient cable and the USB cradle use the same socket, so the recorder can never be connected to both. Data is read out and the battery charged only with the patient disconnected. The hookup check uses Bluetooth, which needs no galvanic path. This is the reference design's choice.
2. **Firmware interlock** alone, which disables acquisition when VBUS is present. It does not remove the current path, so it is a convenience, not a safety measure.
3. **Isolated USB.** A reinforced USB isolator (ADuM4160 class, 5 kV rms) and an isolated DC–DC converter rated for 2 MOPP keep the path open. Leakage then flows only through the barrier capacitance: 10 pF gives $2\pi \times 50 \times 10 \times 10^{-12} \times 230 = 0.72$ µA, inside the CF limit. This is the architecture for bedside equipment that must stream while connected.

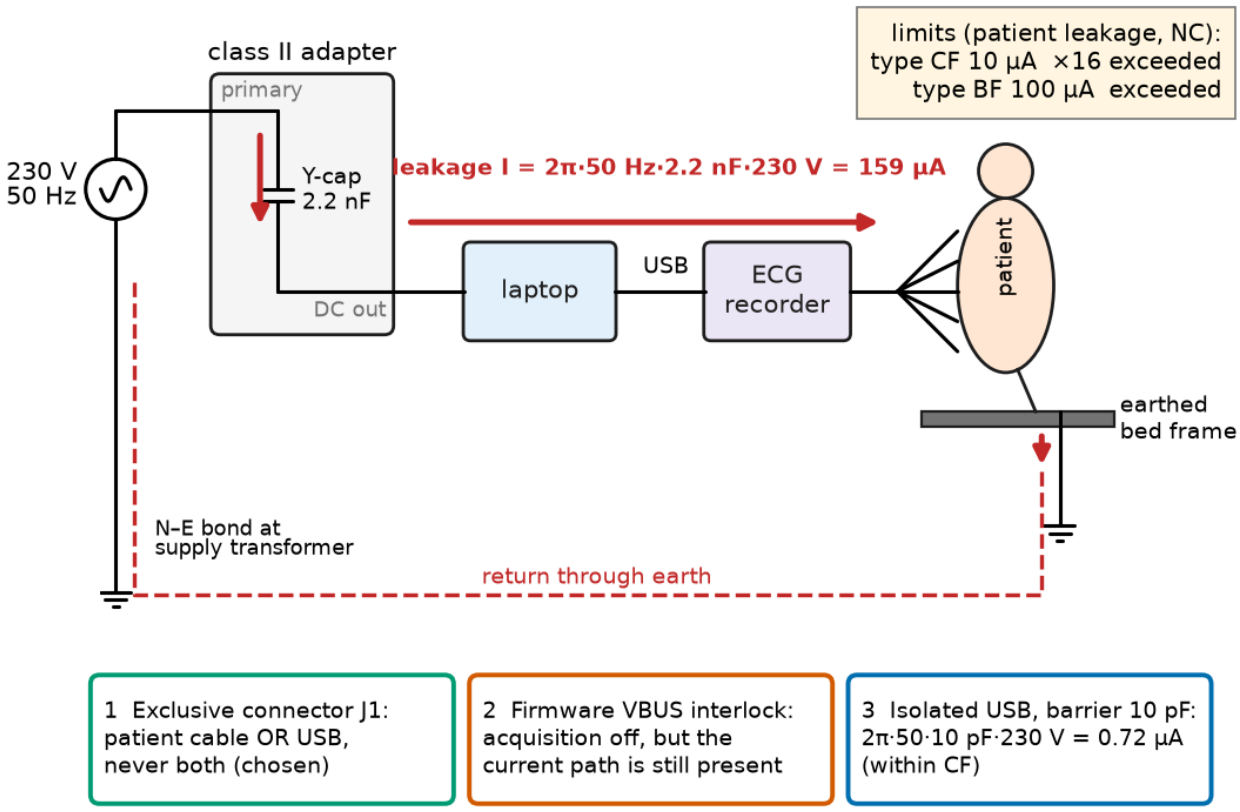


Figure 28.3 — Leakage path from a laptop mains adapter through a USB-connected ECG recorder and the patient to earth, and the three ways of breaking it.

Defibrillation and the right-leg driver

ECG equipment is expected to survive, and not divert the energy of, a defibrillation pulse of up to 5 kV applied across the chest while the electrodes are attached. The input network therefore has pulse-rated series resistors (10 kΩ per lead in MELF packages rated for the pulse energy) ahead of low-leakage clamps, and the

recorder must show a trace again within seconds afterwards. The design targets a defibrillation-proof type CF applied part.

The right-leg driver returns current to the body, which makes it a source of patient auxiliary current. Its output can swing to the 3.0 V analog rail, so a series resistor of 499 kΩ limits the DC current to $3.0/0.499 = 6.0$ μA, inside the 10 μA DC limit, and to 6.6 μA even if the output faults to a 3.3 V rail.

28.6 Signal conditioning

The input network

Figure 28.4 shows one lead pair of the front end, from snap electrode to converter. Each part has a single job:

- **10 kΩ pulse-rated series resistor** per lead: defibrillation and electrostatic-discharge current limiting.
- **RFI filter**: 47 pF from each input to analog ground and 4.7 nF across the pair. The differential corner is $1/(2\pi \times 20\ \text{k}\Omega \times 4.7\ \text{nF}) = 1.7$ kHz, far above the 450 Hz needed even in EMG mode; the common-mode capacitors are kept small for a reason worked out in Section 28.7.
- **Low-leakage clamps** to the rails, so that a defibrillation or ESD transient does not exceed the converter's absolute maximum input.
- **Lead-off detection**: the converter's 6 nA DC current sources pull a disconnected input towards a rail, and a comparator flags it within one data block (R11). Through a 1 MΩ electrode impedance, 6 nA adds only 6 mV of offset.
- **Right-leg drive**: the converter's RLD amplifier senses the average of the inputs and drives the right-leg electrode through 499 kΩ; an external feedback R–C, chosen with the datasheet's guidance, sets its loop gain and bandwidth.

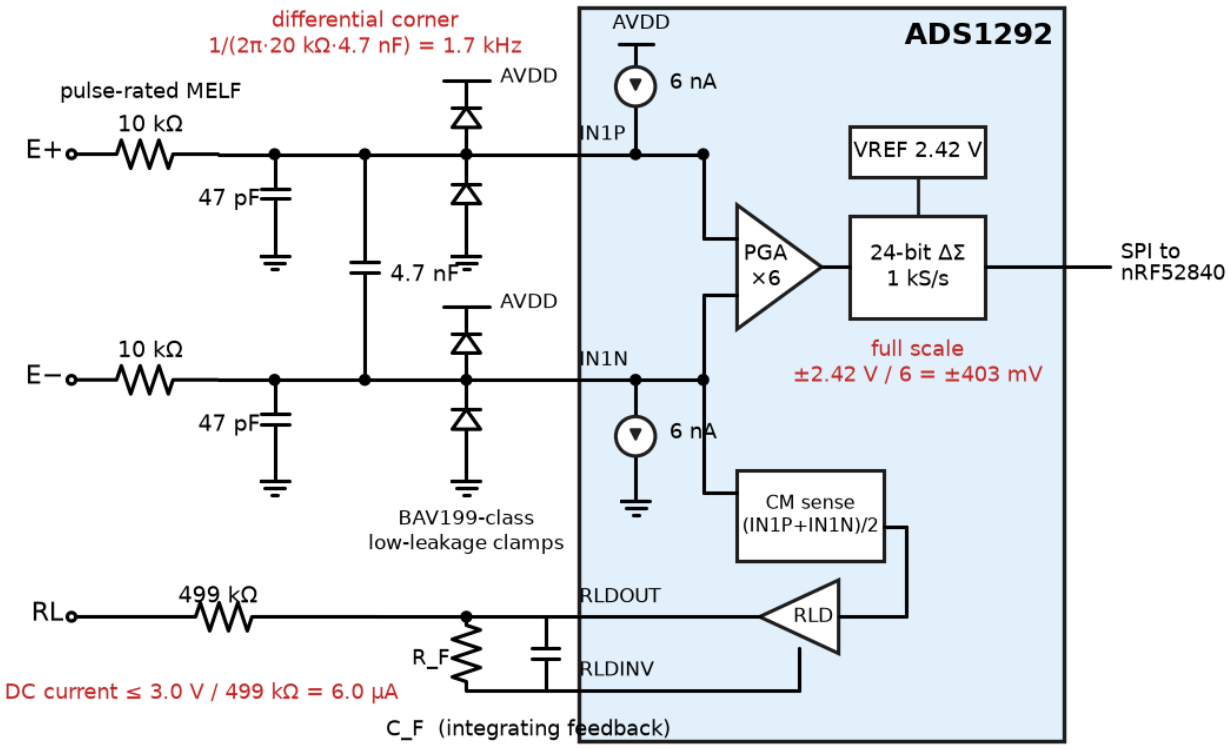


Figure 28.4 — One lead pair of the ECG front end with protection, RFI filtering, lead-off detection and the right-leg driver.

Driven right leg

The driven right leg (DRL) is negative feedback on the body's common-mode voltage: the amplifier measures it and drives the body through the right-leg electrode to cancel it. With a loop gain A at 50 Hz the common-mode voltage falls by about $1 + A$. The loop includes the electrode impedance and the body capacitance, so it is compensated like any feedback loop (Book 3 Ch. 34): the integrating feedback network is sized for about 40 dB of loop gain at 50 Hz, with crossover well below the frequency where electrode capacitance adds phase lag. Too much gain makes the loop oscillate at a few kilohertz when an electrode is poorly attached, visible as a burst of noise on every lead; the lead-off logic therefore also opens the RLD loop when the right-leg electrode comes off.

Digital conditioning

Because the hardware is DC-coupled and wideband, all band-shaping is digital and versioned with the firmware:

1. **Decimation** from 1 kS/s to 500 S/s with the 31-tap FIR of Section 28.2.
2. **High-pass at 0.05 Hz**, implemented as a linear-phase operation in the analysis software (forward–backward filtering of the stored record) so that the ST segment is not distorted; the stored data keep the offset, scaled, so no information is lost on the device.
3. **No 50 Hz notch in the stored data.** A notch distorts QRS complexes, which have energy at 50 Hz; it is an optional display filter only.
4. **Optional 40 Hz low-pass** for the monitoring-style display during the hookup check.

Status of the principle: DC-coupled front end with digital filtering
Derived. With a converter whose range exceeds offset plus signal and whose resolution is below the noise floor, any analog high-pass can be replaced by a digital one without loss; it fails if the offset drifts beyond the input range, which the lead-off and range monitoring must flag.

28.7 Noise rejection

Boundary.
Common-mode rejection of difference and instrumentation amplifiers is in *Electronics and Computing* Ch. 8 §8.3 and §8.6; feedback-loop stability for the DRL in *Circuits, Signals and Communications*, Ch. 34; filter design in Ch. 23.

Mains interference: why the IC's CMRR is not the limit

The converter's own CMRR is about 105 dB. Applied to a 25 mV body common-mode voltage it would leave about 0.1 µV, negligible. The hum in the opening story came from somewhere else: the electrodes' impedance imbalance forms, with each input's common-mode impedance, a potential divider, and two unequal dividers turn common-mode voltage into a differential one *before* the amplifier sees it:

$$V_{\mathrm{dm}} \approx V_{\mathrm{cm}} \frac{\Delta Z_e}{Z_{\mathrm{cm}}}.$$

Status of the principle: common-mode to differential conversion by source-impedance imbalance
Approximate. Valid when the electrode impedances are much smaller than the input common-mode impedance and the imbalance is in one lead; it shows that input common-mode impedance and right-leg drive, not amplifier CMRR, set mains rejection in biopotential front ends.

Worked example — the mains budget. Take a displacement current of 0.5 µA from mains wiring into the body (a body coupled by a few picofarads to 230 V lines, typical indoors) and a right-leg electrode impedance of 51 kΩ. Without DRL the body's common-mode voltage is $0.5\ \mu\mathrm{A} \times 51\ \mathrm{k\Omega} = 25.5$ mV. The standard imbalance, 51 kΩ ‖ 47 nF, has a magnitude of 40.7 kΩ at 50 Hz. The input common-mode impedance is set by the 47 pF RFI capacitors: $1/(2\pi \times 50 \times 47\ \mathrm{pF}) = 67.7\ \mathrm{M\Omega}$. Without DRL,

$$V_{\mathrm{dm}} = 25.5\ \mathrm{mV} \times \frac{40.7\ \mathrm{k\Omega}}{67.7\ \mathrm{M\Omega}} = 15.3\ \mu\mathrm{V\ rms} = 43\ \mu\mathrm{V\ p\text{-}p},$$

which fails R5's 20 µV, and matches the 40 µV of the opening story. With 40 dB of DRL loop gain the common-mode voltage falls to 0.26 mV and the converted interference to 0.43 µV p-p: R5 is met with a factor of 46. Had the RFI filter used 1 nF common-mode capacitors, a common choice copied from non-medical designs, Z_{cm} would fall to 3.2 MΩ and the interference would rise 21 times, to 9 µV p-p even with DRL. Figure 28.5 plots the interference against imbalance.

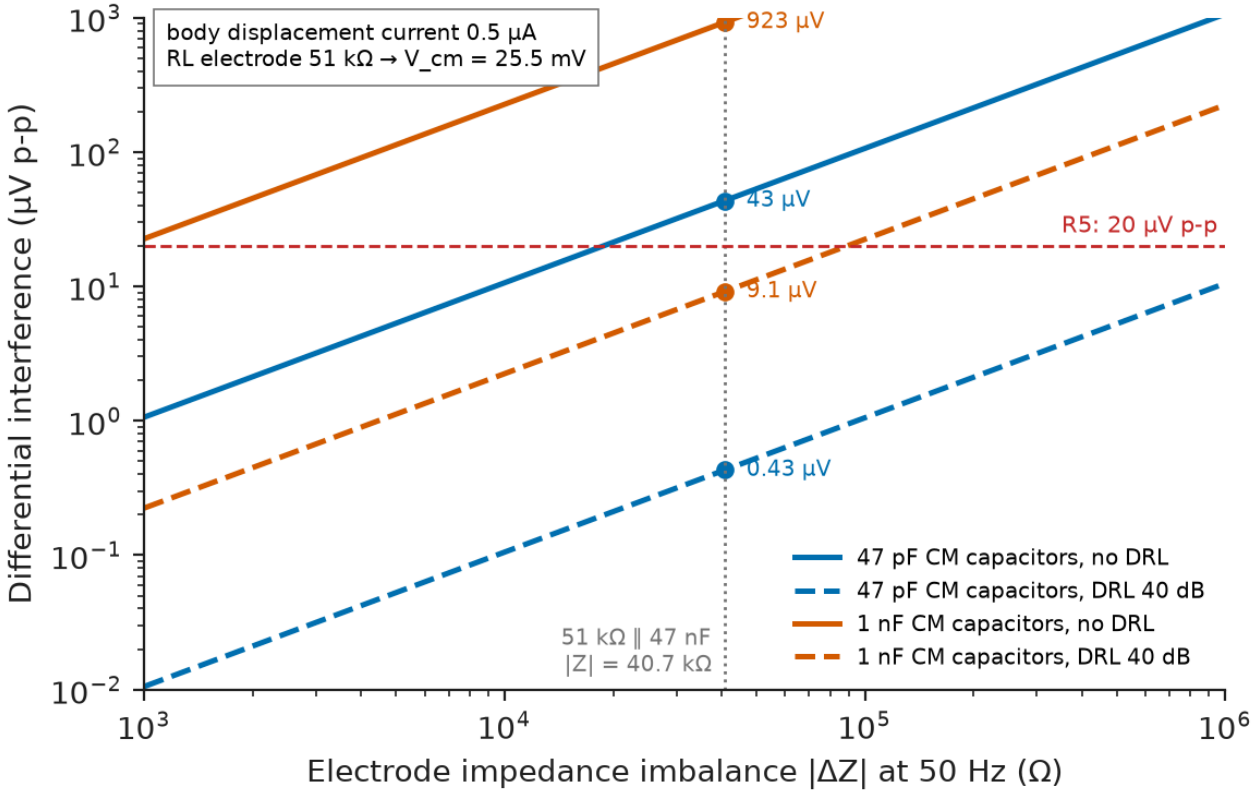


Figure 28.5 — Mains interference converted to differential mode versus electrode impedance imbalance, for two common-mode capacitor values with and without right-leg drive.

Beyond the front end, mains pickup is reduced by geometry: a shielded patient cable with the shield driven at the common-mode voltage, lead wires kept together (the loop area of Book 3 §2.4: 0.1 m² in a 1 µT field picks up 31 µV), and a short cable.

Baseline wander, motion and muscle

- **Baseline wander** from respiration (0.15–0.5 Hz) and electrode drift is removed by the 0.05 Hz linear-phase high-pass; residual slow drift is handled in analysis by subtracting a cubic spline through the PR-segment points of successive beats.
- **Motion artefact** shares the ECG band and cannot be filtered out. It is reduced at the source (electrode placement on bone rather than muscle, skin preparation, cable strain relief) and flagged using the accelerometer: segments where torso acceleration exceeds a threshold carry a quality flag, and the analysis excludes them from morphology measurements.
- **Muscle (EMG) noise** overlaps the QRS spectrum above 20 Hz. It is reduced by placing electrodes away from the pectoral muscles and handled in analysis by signal-quality indices rather than by aggressive low-pass filtering, which would corrupt the QRS.

Noise budget

Table 28.6 — Noise and interference budget, referred to input, 0.05–150 Hz.

Source	rms (µV)	Peak-to-valley (µV, ≈ 6.6 × rms)	Note
Converter and PGA, gain 6	1.21	8.0	Datasheet typical
Electrode–skin noise, Ag/AgCl with preparation	2.0	13.2	Typical; dominates
Thermal noise of 2 × 10 kΩ, 150 Hz	0.23	1.5	$\sqrt{4kTRB}$ at 310 K
Mains residual with DRL (worked example)	0.15	0.43 (sinusoid)	Coherent; listed for completeness
Total (RSS of random terms)	**2.35**	**15.5**	
Requirement R4 / margin		30	48 % margin

The budget repeats the lesson of Chapter 27's accelerometer: the sensor (here, the electrode on skin) dominates, and money spent on a quieter converter would buy nothing.

28.8 Medical data acquisition

Sampling, timing and the record

The recorder stores two channels at 500 S/s in 24-bit two's complement, plus the accelerometer at 25 S/s, lead-off flags and the event button. Data are grouped in 1 s blocks: 3000 bytes of ECG plus a 72-byte header (block counter, RTC timestamp, lead-off and quality flags, accelerometer summary, CRC-32), 3072 bytes in all, exactly six 512-byte sectors. A block with a bad CRC is lost alone; the counter makes gaps explicit rather than silently closing them.

Timing matters to cardiologists: an event at "14:32" in the patient diary must match the trace. The 32.768 kHz crystal (±20 ppm) drifts by at most 1.7 s per day, 12 s in seven days, inside R12's 30 s; the clock is set from the clinician's tablet over Bluetooth at hookup and the offset is recorded at readout.

Storage formats

On readout, records are exported in formats the analysis tools and archives accept:

- **EDF+ / BDF**: European Data Format, 16-bit (EDF+) or 24-bit (BDF), with annotations; widely used in research and sleep labs. Export to 16-bit EDF+ requires removing the offset first and scaling to, for example, 0.25 µV per count over ±8.2 mV.
- **ISHNE Holter format**: the de facto exchange format for Holter recordings.
- **HL7 aECG and DICOM waveform**: for hospital information systems and archives.

The export software records its own version, the filter settings and the scaling in the file header, so that a measurement made years later can be reproduced (Chapter 29).

Energy budget

R7 is an energy budget, and the first version failed it.

Worked example — seven days on 200 mAh. The cell is 200 mAh at 3.7 V, 740 mWh; 80 % is counted as usable at end of life and at low temperature, 592 mWh. The first design kept the microSD card powered between writes: its idle current of about 0.25 mA cost 0.83 mW, plus 0.30 mW for the writes themselves. With the loads of Table 28.7 and an 85 % efficient buck converter, the battery supplied 3.06 mW, or 513 mWh in 168 h: a 13 % reserve, short of R7's 20 %. Power-gating the card with a load switch and writing 64 KB every 21.3 s costs a re-initialisation each time (about 0.1 s at 20 mA) but removes the idle current; the card's average falls to 0.62 mW and the reserve rises to 30 %.

Table 28.7 — Energy budget of the recorder, second version (7 days, 25 °C).

Load	Average power (mW)	Basis
ADS1292, 2 channels, RLD, lead-off	0.80	Datasheet typical, 1 kS/s
nRF52840: filtering, QRS detection, DRDY wake-ups	0.45	3 % CPU duty at 64 MHz plus 1000 wake-ups/s
BLE status link, 1 s connection interval	0.06	Measured on development kit
microSD, power-gated, 64 KB per 21.3 s	0.62	0.1 s at 20 mA init + 50 ms at 40 mA write, 3.3 V
Accelerometer, 25 S/s low-power mode	0.01	Datasheet typical
RTC, LED blink, leakage	0.15	Estimate
Total load / from battery (η = 0.85)	**2.09 / 2.46**	
Energy in 168 h / usable	**413 / 592 mWh**	**Reserve 30 % (R7: ≥ 20 %)**

> **Status of the principle: battery runtime from average power**
> *Approximate*. Runtime = usable energy / average power holds when the cell's capacity does not depend on the load profile; pulsed loads (card writes, radio events) raise losses in the cell's internal resistance at low temperature, which the 80 % usable-energy factor is meant to cover and the full-length test must confirm.

28.9 Basic biomedical signal analysis

> **Boundary.**
> Spectral estimation and filter design are in *Circuits, Signals and Communications* Ch. 22–23; statistical measures in *The Mathematics Universe*, Ch. 20. Learned arrhythmia classifiers belong to Chapter 32.

QRS detection

Almost every ECG analysis starts by finding the QRS complexes. The Pan–Tompkins structure, published in 1985 and still the reference point for embedded detectors, has four stages: a 5–15 Hz band-pass that keeps the QRS and rejects P and T waves, baseline and much muscle noise; a derivative that emphasises steep

slopes; squaring that makes all values positive and accentuates large slopes; and a 150 ms moving-window integration that merges each complex into one pulse. Adaptive thresholds and a 200–250 ms refractory period then pick the beats (Figure 28.6).

The listing runs a simplified version on two minutes of synthetic ECG at 500 S/s: beats at 72 bpm with RR jitter, half a millivolt of baseline wander, 30 µV of 50 Hz, 15 µV of wideband noise, and a 20 s burst of muscle noise at half the R-wave amplitude. It uses one threshold per record instead of the original's adaptive pair.

```
 import numpy as np
from scipy import signal

fs, T = 500, 120.0
rng = np.random.default_rng(11)
t = np.arange(int(fs * T)) / fs
# Synthetic ECG (mV): beats at 72 bpm with 40 ms RR jitter; P, QRS, T as Gaussians
rr = 60 / 72 + 0.04 * rng.standard_normal(int(T * 72 / 60))
beats = np.cumsum(rr); beats = beats[beats < T - 1]
ecg = np.zeros_like(t)
for b in beats:
    for dt, a, w in [(-0.20, 0.15, 0.025), (-0.03, -0.10, 0.008), (0.0, 1.2, 0.010),
                     (0.03, -0.25, 0.008), (0.25, 0.30, 0.040)]:
        ecg += a * np.exp(-0.5 * ((t - b - dt) / w) ** 2)
# Interference: baseline wander, 50 Hz, wideband noise, EMG burst 40-60 s
x = ecg + 0.5 * np.sin(2 * np.pi * 0.25 * t) + 0.03 * np.sin(2 * np.pi * 50 * t)
x += 0.015 * rng.standard_normal(t.size)
emg = signal.sosfilt(signal.butter(4, [10, 200], 'bandpass', fs=fs, output='sos'),
                     rng.standard_normal(t.size))
x += 0.60 * emg / emg.std() * ((t > 40) & (t < 60))

# Pan-Tompkins-style detector: band-pass 5-15 Hz, derivative, square, 150 ms integration
bp = signal.sosfiltfilt(signal.butter(2, [5, 15], 'bandpass', fs=fs, output='sos'), x)
mwi = np.convolve(np.gradient(bp) ** 2, np.ones(int(0.15 * fs)) / (0.15 * fs), 'same')
thr = 0.3 * np.percentile(mwi, 99)                        # one fixed threshold per record
pk, _ = signal.find_peaks(mwi, height=thr, distance=int(0.25 * fs))   # 250 ms refractory
r = np.array([p - 40 + np.argmax(bp[max(p - 40, 0):p + 40]) for p in pk]) / fs

truth = beats
tp = sum(np.min(np.abs(r - b)) <= 0.075 for b in truth)  # 75 ms match window
print(f"beats {len(truth)}, detected {len(r)}, Se {tp/len(truth):.4f}, +P {tp/len(r):.4f}")
rri = np.diff(r) * 1e3
print(f"mean HR {60e3/rri.mean():.1f} bpm, SDNN {rri.std(ddof=1):.1f} ms, "
      f"RMSSD {np.sqrt(np.mean(np.diff(rri)**2)):.1f} ms (truth "
      f"{np.diff(truth).std(ddof=1)*1e3:.1f},
{np.sqrt(np.mean(np.diff(np.diff(truth)*1e3)**2)):.1f})")
```

```
 beats 142, detected 148, Se 1.0000, +P 0.9595
mean HR 74.9 bpm, SDNN 123.0 ms, RMSSD 107.9 ms (truth 37.8, 53.3)
```

Every true beat is found, but the muscle burst adds six false beats. Positive predictivity falls to 96 %, below R10, and the heart-rate-variability measures are wrecked: SDNN reads 123.0 ms against a true 37.8 ms,

because each false beat creates two short intervals. Without the burst, the same code finds all 142 beats with no false ones and returns SDNN and RMSSD within 0.2 ms of the truth. Two lessons follow. Detection statistics on a clean record say little; the test set must include noise, which is why ANSI/AAMI EC57 prescribes the MIT-BIH Arrhythmia Database and noise-stress tests. And interval statistics must be computed only over segments a signal-quality index accepts, with ectopic and artefactual intervals removed, or one noisy minute corrupts a day's HRV.

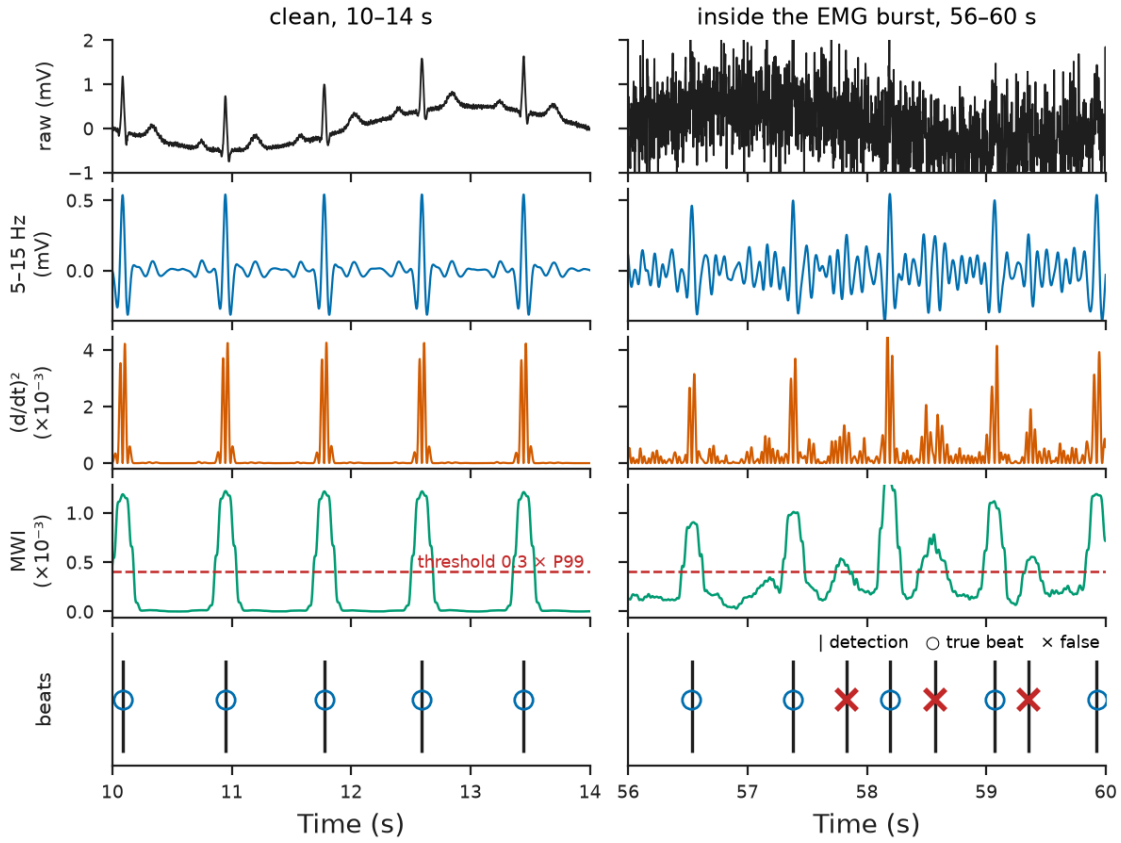


Figure 28.6 — Stages of the QRS detector on 4 s of synthetic ECG, clean and during the muscle-noise burst.

The firmware runs the same structure on the nRF52840 for the hookup display and the status heart rate, with the original adaptive thresholds; the diagnostic analysis runs on a workstation over the whole record, where it can look backwards, search back for missed beats and use both leads.

Heart rate and variability

Worked example — time-domain HRV from six beats. Normal-to-normal intervals of 812, 845, 790, 828, 861 and 805 ms have a mean of 823.5 ms, a heart rate of $60000/823.5 = 72.9$ bpm and a standard deviation (SDNN) of 26.4 ms. The successive differences are 33, −55, 38, 33 and −56 ms, and their root mean square (RMSSD) is 44.2 ms. Clinical HRV uses 5 min (short-term) or 24 h (long-term) records under the 1996 Task Force conventions; six beats illustrate the arithmetic, not a measurement.

Other basic analyses follow the same pattern of detection, measurement and quality gating: ST-segment level at a fixed offset after the J point (which depends on the 0.05 Hz high-pass), QT interval with heart-rate correction, and for EMG the envelope and median-frequency features of Section 28.3.

28.10 Regulatory and ethical considerations

The regulatory frame

A medical device is designed under a quality system and against harmonised standards, and the design file must show it. For the reference recorder the main documents are:

- **IEC 60601-1** (basic safety and essential performance), **IEC 60601-1-2** (electromagnetic compatibility, including immunity to radiated fields and ESD), **IEC 60601-1-11** (home-healthcare environment, since the recorder is worn at home), and the particular standard **IEC 60601-2-47** for ambulatory electrocardiographic systems.
- **ISO 14971** risk management, with the FMEA of Table 28.9 as one of its inputs.
- **IEC 62304** software life-cycle. The firmware is class B (injury possible but not serious, because a missed recording is detected at readout and repeated); the workstation analysis software that reports arrhythmias may be class B or C depending on its claims.
- **IEC 62366-1** usability engineering (electrode placement instructions, the event button).
- **ISO 10993-1** biocompatibility of the skin-contact materials (electrode adhesive, housing), with the material background in *Materials Science and Engineering*, Ch. 39 §39.8.
- **ISO 13485** quality management, under which design controls, change control and traceability are audited.
- **ANSI/AAMI EC57** for testing and reporting the performance of rhythm and ST algorithms.

Market access then follows the jurisdiction: in the EU the Medical Device Regulation (an ambulatory ECG recorder is typically class IIa), in the US a 510(k) premarket notification for a class II device. This chapter describes engineering practice; a real product needs regulatory specialists from the concept phase.

Ethics and data

Recording people's physiology raises obligations that are not satisfied by passing a test:

- **Clinical investigation.** Testing on volunteers or patients requires ethics-committee approval and informed consent, under ISO 14155 for device investigations. The volunteer test of the opening story should have been preceded by the leakage measurement, not followed by it.
- **Privacy.** An ECG is health data under GDPR and HIPAA. The recorder stores a pseudonymous recording ID, never a name; the card is encrypted (AES-128 with a per-device key); the readout software links ID to patient only inside the hospital system.
- **Bias.** Algorithms are validated on populations. A QRS detector tuned on adult databases misses beats in children (higher rates, narrower complexes), and optical sensors underperform on darker skin. The validation plan states the population and reports performance by subgroup.
- **Honest claims.** A heart-rate estimate on a wellness app and an arrhythmia report used for diagnosis are different devices under the law and in their duty to the user; the intended use statement is written first and limits every claim.

Verification

Table 28.8 — Verification of the ambulatory ECG recorder (design-verification build, 3 units).

Req.	Test	Limit	Result	Pass/fail
R2	±300 mV DC offset plus 5 mV, 10 Hz sine	No clipping; gain error ≤ 5 %	No clipping; gain error 0.6 %	Pass
R3	Sine sweep 0.05–150 Hz; impulse test	−3 dB outside 0.05–150 Hz; impulse displacement ≤ 100 µV	−3 dB at 0.048 Hz and 156 Hz; displacement 20 µV	Pass
R4	Inputs via 51 kΩ ‖ 47 nF, 10 s	≤ 30 µV p-v	11–14 µV p-v (shorted inputs: 7 µV)	Pass
R5	Common-mode test with imbalance	≤ 20 µV p-v	4 µV p-v	Pass
R7	Full-length run at 25 °C	≥ 168 h + 20 % reserve	236 h to cut-off (reserve 29 %)	Pass
R8	Patient leakage, NC and SFC	≤ 10 / 50 µA	< 0.1 µA (battery-only; USB physically excluded)	Pass
R9	DC auxiliary current, RLD saturated	≤ 10 µA	5.9 µA	Pass
R10	EC57 method, MIT-BIH Arrhythmia DB (workstation analysis)	Se ≥ 99 %, +P ≥ 99 %	Se 99.6 %, +P 99.5 %	Pass
R11	Electrode removal	Flag ≤ 2 s	0.9 s	Pass
R12	7-day clock; CRC fault injection	≤ 30 s; bad blocks flagged	+8 s; 100 % of injected errors flagged	Pass

The measured interference in R5 (4 µV) is higher than the 0.43 µV of the worked example because the test rig's common-mode source is larger than the assumed body coupling; the margin remains a factor of five.

Failure analysis

Table 28.9 — FMEA of the ambulatory ECG recorder (excerpt from the ISO 14971 file).

Failure mode	Effect	Cause	Detection	Mitigation
Leakage through mains-connected equipment	Patient shock risk	Simultaneous USB and patient connection	Leakage test during evaluation	Shared exclusive connector; Bluetooth hookup check
Mains hum on traces	Unreadable morphology; false ST changes	Imbalance with low input common-mode impedance; DRL off	Common-mode test; field review	47 pF CM capacitors; DRL with 40 dB; shielded cable
DRL oscillation	Noise bursts on all leads	Excess loop gain with poor RL contact	Burst detector in firmware	Loop compensation; RLD opened on RL lead-off
Electrode offset beyond range	Channel saturates; data lost	Dry gel, mixed electrode types	Range monitor flags > 350 mV	Gain 6 headroom; electrode type in instructions
False beats in muscle noise	Wrong HR, HRV and arrhythmia counts	QRS-band overlap with EMG	EC57 noise-stress test	Signal-quality index; intervals gated by quality
microSD write failure	Gap in record	Card wear, contact vibration	Block counter and CRC; write verify	Industrial SLC card; retry and mark; status LED
Battery exhausted early	Recording shorter than prescribed	Cold, aged cell, card idle current	Fuel gauge; full-length test	Power-gated card; 20 % reserve; low-battery file close
Clock drift or wrong time zone	Diary events misaligned	Crystal tolerance; tablet setting	Offset recorded at readout	Time set in UTC with zone stored; drift corrected

Worked example — the false ST elevation. In an early pilot, a cardiologist reported apparent ST elevation in one recording, but only during the night. Accelerometer data showed the patient lying on the left side; impedance-check data showed the V5 electrode's impedance rising from 20 kΩ to 300 kΩ as the gel dried against the mattress. The rising imbalance increased converted mains interference, and the workstation's display filter, an optional 50 Hz notch with a slow transient response, shaped the residual into a slow deflection after each QRS. The fix had three parts: the notch was removed from the default analysis path (Section 28.6), electrode impedance was logged every 10 minutes as a quality signal, and the instructions changed to a hydrogel electrode rated for seven-day wear.

What remains open

Open question	Where it is answered
How are sensors chosen and calibrated in general, and uncertainty budgets built?	Chapter 6; Chapter 7
How are filtering, spectral analysis and detection implemented in real time on an MCU?	Chapter 27
How is the Bluetooth link budgeted and tested?	Chapter 26
How are firmware and analysis software versioned, tested and released reproducibly, as IEC 62304 expects?	Chapter 29
How are learned arrhythmia classifiers trained, validated and their uncertainty reported?	Chapter 32
How is the risk-management and requirements method applied from the start of a project?	Chapter 1; Chapter 40
Where are shock physiology, earthing and protective devices developed?	*Electrical Energy and Machines*, Ch. 29

Next: Chapter 29 opens Part VIII by turning to the tools behind every design in this book: how models, simulations, tests and documents are organised so that any result can be regenerated from a tagged commit.

Part VIII — Engineering Software and Computational Tools

Every chapter of this book ends in numbers, and most of them are computed before they are measured. Part VIII is about the software that produces them and the discipline that makes them trustworthy. It treats engineering computation as an engineering system in its own right, with requirements, budgets of time and error, tools chosen against alternatives, tests with limits and failures analysed to their mechanism. A number that cannot be regenerated, a solver result never checked against a known answer, a file format that silently changes old projects, a learned model scored on its training data, an uncalibrated twin: each is a defect as real as an undersized cable. Five running systems carry the method: the design workflow of a 48 V-to-12 V converter, the thermal optimisation of an inverter heatsink, a cable-sizing tool used by sixty engineers, machine-learning models built on engineering data, and a digital twin of a pump set.

By the end of this part you should understand

- how requirements, models, simulations, measurements and reports are kept in one version-controlled project, so that every reported number can be regenerated from a tagged commit, and which model answers which question;
- how numerical solvers are chosen, how sweeps, optimisation and parallel campaigns are organised, and how numerical results are verified against known solutions before they are trusted;
- how an engineering tool is structured around a domain model, with stable file formats and migrations, plug-ins for standards, an API beneath the GUI and tests that protect old projects;
- how engineering datasets are cleaned and split, how regression, classification, forecasting and anomaly-detection models are evaluated honestly, and what deployment on a constrained target demands;
- how a physics model becomes a digital twin of one machine: calibrated, co-simulated, fed with live data, validated against limits written down in advance and reported with its uncertainty.

Chapters

Chapter	Central question	What it delivers
29 — Engineering Software Workflows	How is a design project organised so that every number in its report can be traced and regenerated?	The `buck48` converter project: requirements as data, scripted loss model, SPICE campaign, averaged loop model, repository, pinned environments and provenance, tests in CI, instrument import, generated plots and reports, workflow FMEA
30 — Scientific and Engineering Computing	How is a computation made fast enough to explore a design space and right enough to bet hardware on?	A heatsink-optimisation campaign: solver selection, parameter sweeps, optimisation, automation, data processing, performance and parallel execution, verification against known solutions, validation against thermocouples, data management
31 — Engineering Software Architecture	How is an engineering tool built so that it stays correct as it grows and its users' old projects keep their meaning?	A cable-sizing tool: modules and dependencies, domain model, file formats and migrations, plug-ins versus built-ins, APIs, GUI, data interoperability, version control, automated tests, long-term maintainability
32 — AI and Machine Learning for EEE	When does a learned model beat a physical one on engineering data, and how is that claim proved?	Datasets, cleaning and features, regression, classification, forecasting, anomaly detection, surrogate and physics-informed models, evaluation without leakage, uncertainty, and the constraints of deployment
33 — Engineering Simulation and Digital Twins	How is a physics model turned into a twin that tracks one particular machine, and how uncertain is what it predicts?	A pump-set twin: electro-thermal-hydraulic model, numerical solution, calibration, co-simulation, twin architecture, live data, online parameter estimation, validation, uncertainty quantification and simulation-based decisions

How this part connects to the rest of the book

Part VIII supplies the tooling under Part I's methods: requirements (Chapter 1), architecture (Chapter 2), modeling and simulation (Chapter 3), test plans (Chapter 4) and documentation (Chapter 5). Its techniques run through the application parts: reference-model testing of firmware DSP in Chapter 27, the edge inference of Chapter 12 that Chapter 32 trains for, and the system identification of Chapter 17 that Chapter 33 extends into twins. In Part IX, Chapter 39 builds a complete simulation tool on Chapters 29–31, and the capstone of Chapter 40 uses the twin and uncertainty methods of Chapter 33.

The theory comes from the sibling books, cited and used rather than re-derived. Python, scientific computing, testing and version control come from *Electronics and Computing* (Ch. 22, Ch. 24), with embedded AI hardware in its Ch. 35. Numerical methods, optimisation, statistics and modelling come from *The Mathematics Universe* (Ch. 20, Ch. 22–Ch. 25). Circuit formulation, dynamic-system models and feedback come from *Circuits, Signals and Communications* (Ch. 3, Ch. 33–34). The physical models inside the tools come from *Engineering Physics* (Ch. 17) and *Electrical Energy and Machines* (Ch. 6, Ch. 30).

Chapter 29 — Engineering Software Workflows

Part VIII · Engineering Software and Computational Tools

A power-electronics team presents a 48 V-to-12 V, 240 W converter for a telecom shelf at its design review: 97.6 % efficiency at half load, a phase margin of 58°, a load-step deviation of 120 mV. Six months later the certification lab measures 96.8 % efficiency, and the programme manager asks where 97.6 % came from. Nobody can say. The number was in a spreadsheet whose MOSFET resistance cell had been overwritten with the 25 °C datasheet value; the SPICE model of the MOSFET had since been replaced by the vendor's newer download, under the same file name; the loop-gain script ran only on one engineer's laptop, with a SciPy release that has since changed a default; and the plot in the review slides was a pasted screenshot with no source. Every tool worked. What failed was the workflow around them: numbers that could not be traced to inputs, inputs that were not under version control, environments nobody had recorded, and no automated check that would have shown the change. This chapter builds that workflow for the same converter, so that every number in its design report can be regenerated, from a tagged commit, by one command.

> **How is an engineering project organised so that requirements, models, simulations, measurements and reports stay linked and every result can be regenerated?**
> **Which tools are used for which question, and how are they automated, tested and versioned without slowing the design down?**

Concept	Engineering destination
Requirements and modeling	Requirements as data; a model ladder from hand calculation to switching simulation
Numerical computation	Loss, stress, thermal and tolerance calculations in scripts, not spreadsheets
Circuit simulation	SPICE for switching waveforms, device stress and parasitic effects
System simulation	Averaged and closed-loop models for control design and transients
Version-controlled projects	One repository for requirements, models, netlists, tests and reports
Reproducible builds	Pinned environments, containers, provenance records, one-command builds
Automated testing	Unit, property, regression and requirement tests in continuous integration
Data import and export	Instrument captures, simulator outputs and exchange formats with metadata
Results visualization	Plots generated from data, with requirements drawn on them
Engineering documentation	Reports, traceability matrices and release notes generated from the repository

Roadmap. Requirements as data and the model ladder (29.1) → scripted numerical design (29.2) → SPICE (29.3) → averaged system models and the control loop (29.4) → the repository (29.5) → environments and builds that reproduce (29.6) → tests that run on every change (29.7) → data in and out (29.8) → plots (29.9) → documents, verification and what goes wrong with workflows (29.10).

29.1 Requirements and modeling

> **Boundary.**
> Requirements engineering, traceability and acceptance criteria are owned by Chapter 1; modeling, simulation, sensitivity and validation method by Chapter 3; engineering documentation and revision control as practices by Chapter 5. This chapter is the *tooling* that carries those methods. Buck-converter analysis is owned by *Electronics and Computing*, Ch. 30 (§30.1, §30.6–30.9).

The reference project

The running example is the project `buck48`: a synchronous buck converter, 36–60 V input (48 V nominal), 12 V output, 20 A maximum, switching at 200 kHz, with 100 V MOSFETs of about 7 mΩ, a 6.8 µH inductor and 470 µF of effective output capacitance. The product requirements sit beside workflow requirements, because the workflow is itself a deliverable (Table 29.1).

Table 29.1 — Requirements of the `buck48` project.

ID	Requirement	Target	Verification
R1	Efficiency, 48 V, 10 A	≥ 96.5 %	Loss model; SPICE; bench with power analyser
R2	Efficiency, 48 V, 20 A	≥ 95.5 %	As R1
R3	Output ripple, 20 A	≤ 50 mV p-p	Calculation; SPICE; bench, 20 MHz bandwidth
R4	Loop stability, 36–60 V, 2–20 A	Phase margin ≥ 45°, crossover ≥ 15 kHz	Averaged model; bench frequency-response analyser
R5	10 A load step	Deviation ≤ 3 % (360 mV); within ±0.25 % in ≤ 100 µs	Averaged model; SPICE; bench electronic load
W1	Reproducibility	Every reported number regenerated from a tagged commit by one command; Python results identical, SPICE within stated tolerance	Rebuild of a release on a clean machine
W2	Feedback time	CI on every push ≤ 10 min	CI timing records
W3	Traceability	Every requirement linked to ≥ 1 automated test or named bench test; matrix generated	Generated matrix has no empty rows
W4	Tool control	Tool and model versions pinned; changes only by reviewed commit with a regression diff	Review records
W5	Data integrity	Raw lab data immutable, hashed, with instrument metadata	Hash check in CI

Requirements as data

A requirement that lives only in a document cannot be checked by a program. In `buck48` the measurable part of each requirement is stored in a small TOML file next to the code, and tests and reports read it from there; the document generator renders the same file into the requirements chapter of the report.

```
 # requirements.toml -- product requirements as data (buck48)
[R1]
text = "Efficiency at 48 V input, 10 A load"
io = 10.0
eta_min = 0.965
verified_by = ["tests/test_requirements.py::test_r1_r2_efficiency", "bench:EFF-01"]

[R2]
text = "Efficiency at 48 V input, 20 A load"
io = 20.0
eta_min = 0.955
verified_by = ["tests/test_requirements.py::test_r1_r2_efficiency", "bench:EFF-01"]

[R3]
text = "Output ripple at 20 A"
ripple_max_v = 0.050
verified_by = ["tests/test_requirements.py::test_r3_ripple_bound", "bench:RIP-01"]
```

Changing a limit is now a one-line diff that reviewers see, that reruns every test depending on it, and that appears in the release notes.

The model ladder and the pipeline

Each question is answered by the cheapest model that can answer it, and each model is checked against the next one up (Figure 29.1):

1. **Hand calculation in a script**: duty cycle, ripple, RMS currents, losses, efficiency (Section 29.2). Milliseconds.
2. **Averaged small-signal model**: loop gain, phase margin, output impedance, load-step response (Section 29.4). Seconds.
3. **Switching circuit simulation (SPICE)**: waveforms, device stress, dead-time and reverse-recovery effects, efficiency with device models (Section 29.3). Tens of seconds per operating point.
4. **Hardware**: bench measurements, imported and compared automatically (Section 29.8).

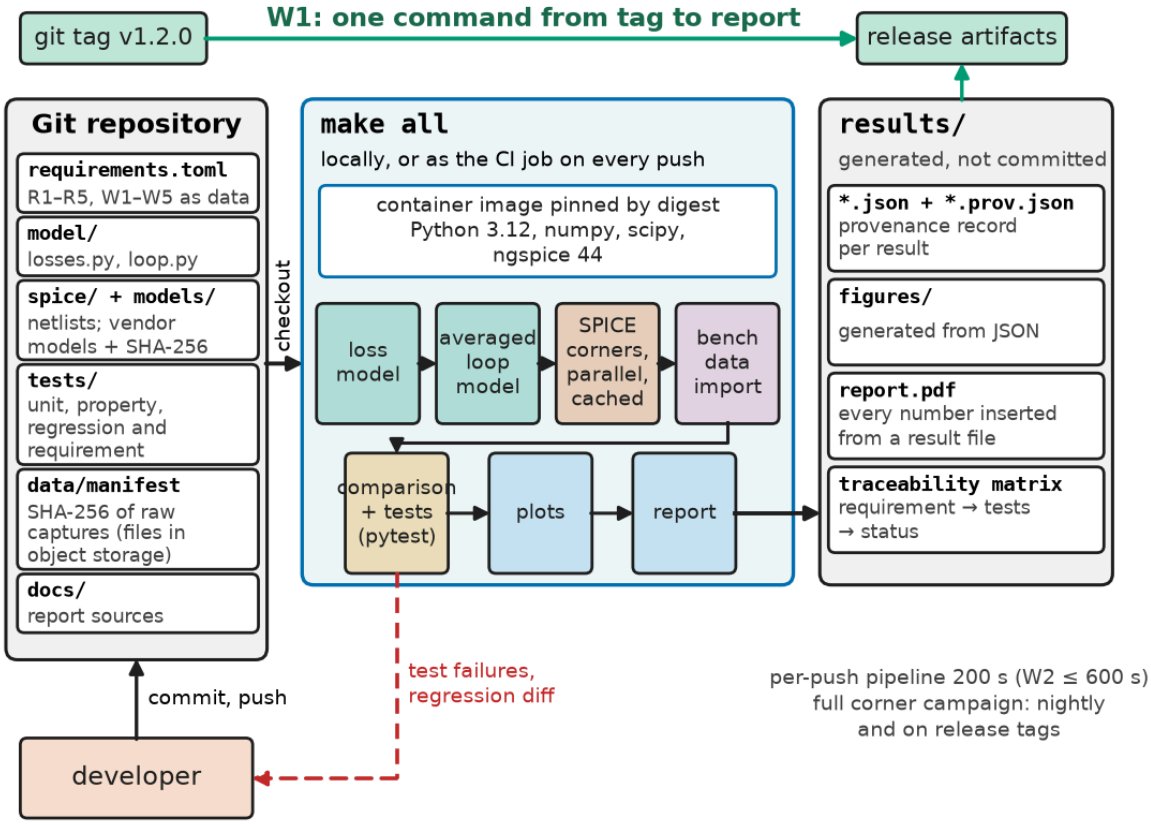


Figure 29.1 — The buck48 engineering workflow: requirements, models and data in one repository, run by one build command locally and in CI, producing results with provenance and a generated report.

> **Status of the principle: a computed result is reproducible only if its inputs, code and environment are all recorded**
> *Fundamental.* A deterministic computation is a function of exactly those three things; leaving any of them unrecorded makes the result unrepeatable by definition, whatever the care taken.

Choosing the tools

Table 29.2 — Tool options for `buck48` (2026).

Question	Options considered	Batch and CI use	Licence	Choice and reason
Numerical design	Spreadsheet; MATLAB; Python with NumPy/SciPy; Julia	Spreadsheet poor; others good	Free to commercial	**Python**: plain-text, diffable, testable, already used for test automation
Circuit simulation	LTspice; ngspice; PSpice/SIMetrix; PLECS (piecewise-linear)	LTspice batch possible but GUI-centred; ngspice designed for batch	Free to commercial	**ngspice** for CI; LTspice allowed for exploration, never for reported numbers
System simulation	Python `scipy.signal` / `solve_ivp`; OpenModelica; Simulink; PLECS	Python and OpenModelica scriptable in containers	Free to commercial	**Python** for the averaged model; Modelica if multi-domain (thermal) grows
Version control and CI	Git with GitHub Actions or GitLab CI	Native	Free tiers	**Git + CI** with a container runner

The rule behind the choices is not "open source is better". It is that a tool producing a reported number must run unattended, from text inputs, in a pinned environment. A GUI tool can be used to explore, but its results enter the report only after the same case has been re-run by the pipeline.

29.2 Numerical computation

> **Boundary.**
> Python, NumPy and SciPy as languages and libraries are owned by *Electronics and Computing*, Ch. 22; floating-point error and numerical methods by *The Mathematics Universe*, Ch. 22 §22.1–22.2. Solvers, sweeps and optimisation at scale are Chapter 30 of this book.

From spreadsheet to module

The loss model is the calculation that went wrong in the opening story, rewritten as a module with one parameter dictionary, one function per question, and SI units throughout. The equations are the standard first-order ones (conduction with ripple-corrected RMS currents, linearised switching transitions, charge-based capacitive, reverse-recovery and gate losses); their derivations belong to Book 4 Ch. 30 §30.8.

```
 """Loss model of the 48 V -> 12 V synchronous buck (buck48/model/losses.py)."""

P = dict(vin=48.0, vo=12.0, fsw=200e3, L=6.8e-6, dcr=3e-3, esr=5e-3, c_out=470e-6,
         rds25=7e-3, k_t=1.6, t_sw=8e-9, qoss=50e-9, qrr=40e-9, qg=35e-9,
         v_drv=10.0, t_dead=20e-9, vf=0.8, p_core=0.25, p_bias=0.10)

def losses(io, p=P):
    d = p["vo"] / p["vin"]
    di = (p["vin"] - p["vo"]) * d / (p["fsw"] * p["L"])      # ripple, A p-p
    r2 = (di / io) ** 2 / 12
    rds = p["rds25"] * p["k_t"]                               # hot R_DS(on)
    return {
        "cond_hs": io**2 * d * (1 + r2) * rds,
        "cond_ls": io**2 * (1 - d) * (1 + r2) * rds,
        "switching": 0.5 * p["vin"] * p["fsw"] * p["t_sw"] * (2 * io),
        "coss": 0.5 * p["qoss"] * p["vin"] * p["fsw"],
        "qrr": p["qrr"] * p["vin"] * p["fsw"],
        "dead_time": p["vf"] * io * 2 * p["t_dead"] * p["fsw"],
        "gate": 2 * p["qg"] * p["v_drv"] * p["fsw"],
        "inductor": (io**2 + di**2 / 12) * p["dcr"] + p["p_core"],
        "cap_esr": di**2 / 12 * p["esr"],
        "bias": p["p_bias"],
    }

def efficiency(io, p=P):
    po = p["vo"] * io
    return po / (po + sum(losses(io, p).values()))

if __name__ == "__main__":
    for io in (2, 5, 10, 15, 20):
        print(f"Io = {io:4.1f} A  loss = {sum(losses(io).values()):5.2f} W  "
              f"eta = {100 * efficiency(io):5.2f} %")
    l10 = losses(10.0)
    print("largest at 10 A:", ", ".join(f"{k} {v:.2f} W" for k, v in
          sorted(l10.items(), key=lambda kv: -kv[1])[:3]))
```

```
 Io =  2.0 A  loss =  1.41 W  eta = 94.46 %
Io =  5.0 A  loss =  1.96 W  eta = 96.84 %
Io = 10.0 A  loss =  3.44 W  eta = 97.22 %
Io = 15.0 A  loss =  5.63 W  eta = 96.97 %
Io = 20.0 A  loss =  8.53 W  eta = 96.57 %
largest at 10 A: cond_ls 0.87 W, switching 0.77 W, inductor 0.56 W
```

Worked example — reading the loss model. At 48 V and 10 A the duty cycle is 0.25 and the inductor ripple $(48-12)\times 0.25/(200\,\text{kHz}\times 6.8\,\mu\text{H}) = 6.62$ A peak-to-peak. The low-side RMS current is $10\sqrt{0.75(1+0.662^2/12)} = 8.82$ A, and with the hot resistance $7\times 1.6 = 11.2$ mΩ it dissipates 0.87 W, the largest single loss. Total loss is 3.44 W and efficiency 97.22 %, a margin of 0.72 points over R1. Had the 25 °C resistance been used, as in the opening spreadsheet, the two conduction losses (1.16 W hot) would shrink to 0.73 W and the efficiency would read 97.6 %: the irreproducible review number, reproduced, and explained by one parameter.

> **Status of the principle: first-order converter loss model**
> *Approximate.* It assumes linear switching transitions, a fixed hot resistance factor and loss terms that add independently; it typically lands within 0.2–0.5 points of measured efficiency at mid load and is least accurate at light load, where core and capacitive losses dominate, which is why SPICE and the bench check it.

Habits that make numerical code engineering code

- **One source of each parameter.** Datasheet values live once, with the datasheet revision in a comment; scripts never copy them.
- **SI units internally**, converted only for printing. An optional units library (`pint`) catches dimension errors in larger models; the reference project uses naming discipline (`_v` , `_a` , `_w` suffixes in result files) instead.
- **Functions that return dictionaries of named terms**, so a report can show the loss breakdown and a test can check each term.
- **No hidden state.** Running the module twice gives the same output; random numbers come from a seeded generator passed in explicitly (Section 29.6).

29.3 Circuit simulation

> **Boundary.**
> Modified nodal analysis, the formulation inside SPICE, is in *Circuits, Signals and Communications*, Ch. 3 §3.8; stiff integration and its stability in *The Mathematics Universe* Ch. 22 §22.9 and §22.11; MOSFET device models in *Electronics and Computing* Ch. 4.

What SPICE is for in this project

The loss model cannot see dead-time body-diode conduction shape, the reverse-recovery current spike, ringing at the switch node, or how the vendor's nonlinear capacitances change switching loss. SPICE can, with the vendor's device model. The netlist is plain text, lives in the repository, and includes the vendor model through a path whose file hash is recorded (Section 29.6).

```
 * spice/buck48_stage.cir -- 48 V to 12 V, 200 kHz synchronous buck, open loop, 10 A
.include ../models/mosfet_100v_7m.lib    ; vendor model, SHA-256 recorded in models/MANIFEST
.param VIN=48 FSW=200k D=0.2525 TD=20n ILOAD=10
VIN   in  0   {VIN}
VGH   gh  sw  PULSE(0 10 {TD} 5n 5n {D/FSW-2*TD} {1/FSW})    ; high-side gate, floating
VGL   gl  0   PULSE(10 0 0 5n 5n {D/FSW} {1/FSW})            ; low-side gate, dead time TD
XHS   in  gh  sw  MOS100V7M                                   ; pins: drain gate source
XLS   sw  gl  0   MOS100V7M
L1    sw  n1  6.8u  ic=10
RDCR  n1  out 3m
COUT  out n2  470u  ic=12
RESR  n2  0   5m
ILD   out 0   DC {ILOAD}
.options method=gear reltol=1e-4 abstol=1e-9 vntol=1e-6
.tran 1n 2m 1.8m 5n uic
.meas tran pin  avg par('-v(in)*i(VIN)') from=1.8m to=2m
.meas tran vout avg v(out)               from=1.8m to=2m
.meas tran pout param='vout*ILOAD'
.meas tran eff  param='pout/pin'
.meas tran vpp  pp  v(out)               from=1.8m to=2m
.end
```

The duty cycle is trimmed from 0.25 to 0.2525 so that the open-loop output sits at 12.0 V despite the losses, and the inductor and capacitor start at their steady-state values (`ic=10`, `ic=12` with `uic`) so that the run does not spend milliseconds settling the 2.8 kHz LC resonance. The `.meas` lines make ngspice print efficiency and ripple as plain numbers, which a Python wrapper parses from the batch log (`ngspice -b`), so no one reads values off a waveform window. The waveforms are still plotted, for review of ringing and recovery (Figure 29.2).

Simulation-time and campaign budget

Worked example — what a corner campaign costs. The 200 µs measurement window after 1.8 ms of run, with a 5 ns maximum step, needs at least $2\ \text{ms}/5\ \text{ns} = 400\,000$ time points; around every switching edge the step shrinks further. On the reference workstation one operating point takes about 40 s. The release campaign is three input voltages × three loads × three temperatures (27 corners) plus 200 Monte Carlo runs over component tolerances: $227 \times 40\ \text{s} = 2.5\ \text{h}$ serial, or about 9.5 min on 16 workers. Results are cached under a hash of the netlist, model files and parameters, so a change to the report template reruns nothing, and a change to one inductor parameter reruns only the cases that use it.

Convergence and trust

SPICE failures in switching converters are almost always one of four things: a floating node with no DC path (a gate driven only through a capacitor), an ideal switch or diode with discontinuous characteristics, a time step forced too small by unrealistically fast edges, or a vendor model with an internal discontinuity. The project's rules are to give every node a DC path (gigaohm resistors where needed), use the vendor's model rather than ideal switches for reported numbers, set edge times from the gate-driver datasheet, and treat any change of `reltol`, `gmin` or integration method as a model change that needs the regression diff of Section 29.7. On this netlist, tightening `reltol` from 10^{-4} to 10^{-5} moved the computed efficiency by 0.02 points and tripled the run time, which is the evidence for keeping 10^{-4}.

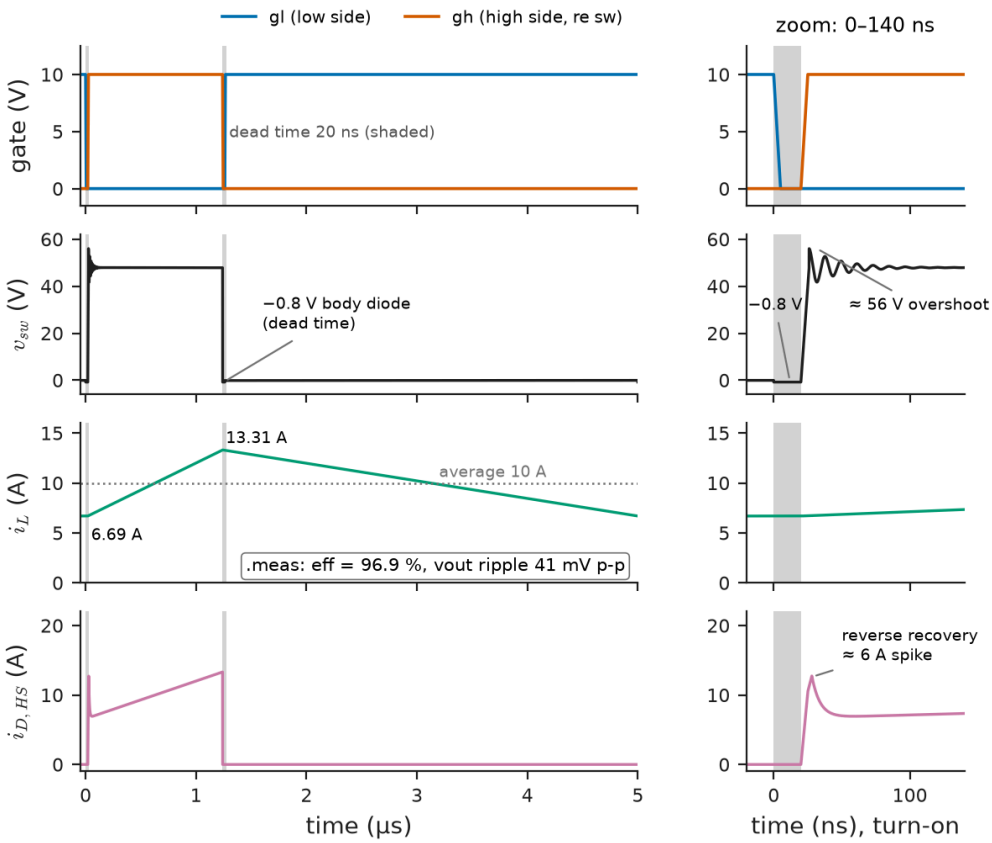


Figure 29.2 — SPICE switching waveforms of the buck at 48 V and 10 A over one 5 us period, with the dead-time intervals marked.

> **Status of the principle: a simulation is no better than its device models**
> *Approximate.* SPICE solves the circuit equations to the stated tolerances, but switching loss, ringing and recovery depend on vendor models whose accuracy is typically quoted only at a few conditions; reported SPICE numbers are therefore checked against the bench at least once per design and the model version is recorded with every result.

29.4 System simulation

> **Boundary.**
> Dynamic-system modeling and transfer functions are owned by *Circuits, Signals and Communications*, Ch. 33; stability margins and Bode methods by Ch. 34 §34.8–34.9; compensator design by Ch. 35; the converter's feedback-control interface by *Electronics and Computing* Ch. 30 §30.9. Co-simulation and digital twins are Chapter 33 of this book.

Averaged model as executable specification

Control design does not need switching detail; it needs the averaged plant, the modulator and the compensator, evaluated over frequency and in time. The script below is the project's loop model. It builds the duty-to-output transfer function with the capacitor's ESR zero and the inductor's resistance, places a type-III

compensator (two zeros near the LC resonance, poles at the ESR zero and at half the switching frequency), sets the integrator gain for a 20 kHz crossover, and reports phase margin and the response to a 10 A load step through the closed-loop output impedance.

```
 """Averaged small-signal model of the buck with a type-III compensator (sim/loop.py)."""
import numpy as np
from scipy import signal

L, C, esr, dcr, R = 6.8e-6, 470e-6, 5e-3, 3e-3, 12.0 / 10.0   # 10 A operating point
vin, vramp, H = 48.0, 4.8, 0.1                    # input feed-forward ramp, divider
w0 = 1 / np.sqrt(L * C)
den_p = [L * C * (1 + esr / R), L / R + C * (esr + dcr) + C * esr * dcr / R, 1 + dcr / R]
gvd = (np.array([vin * C * esr, vin]), np.array(den_p))              # duty -> v_out
zol = (np.polymul([L, dcr], [C * esr, 1]), np.array([L * C, C * (esr + dcr), 1]))

def loop_gain(k):
    """Type III: integrator k/s, zeros at 0.8 f0 and f0, poles at ESR zero and fsw/2."""
    wz1, wz2, wp1, wp2 = 0.8 * w0, w0, 1 / (esr * C), 2 * np.pi * 100e3
    nc = k * np.polymul([1 / wz1, 1], [1 / wz2, 1])
    dc = np.polymul([1, 0], np.polymul([1 / wp1, 1], [1 / wp2, 1]))
    return np.polymul(nc, gvd[0]) * H / vramp, np.polymul(dc, gvd[1])

def T_at(f, k):
    n, d = loop_gain(k)
    return np.polyval(n, 2j * np.pi * f) / np.polyval(d, 2j * np.pi * f)

k = 1 / abs(T_at(20e3, 1.0))                      # place crossover at fsw/10 = 20 kHz
f = np.logspace(2, 5.5, 4000)
T = T_at(f, k)
i = np.argmin(abs(abs(T) - 1))
print(f"f0 = {w0/2/np.pi/1e3:.2f} kHz, crossover {f[i]/1e3:.1f} kHz, "
      f"phase margin {180 + np.degrees(np.angle(T[i])):.1f} deg")

# Closed-loop output impedance Zcl = Zol / (1 + T); response to a 10 A load step
nt, dt = loop_gain(k)
zcl = signal.lti(np.polymul(zol[0], dt), np.polymul(zol[1], np.polyadd(dt, nt)))
t, v = signal.step(zcl, T=np.linspace(0, 400e-6, 20001))
dv = -10.0 * v
print(f"10 A step: undershoot {1e3*dv.min():.0f} mV at {1e6*t[dv.argmin()]:.1f} us, "
      f"inside 0.25 % (30 mV) after {1e6*t[np.where(abs(dv) > 0.03)[0][-1]]:.0f} us")
```

```
 f0 = 2.82 kHz, crossover 20.0 kHz, phase margin 65.6 deg
10 A step: undershoot -138 mV at 11.2 us, inside 0.25 % (30 mV) after 46 us
```

Worked example — closing R4 and R5 by model. The LC resonance is at 2.82 kHz and the crossover is placed at 20 kHz, a tenth of the switching frequency, with 65.6° of phase margin, 20.6° above R4. Because the modulator uses input-voltage feed-forward (ramp amplitude proportional to V_{in}), the loop gain does not change between 36 and 60 V; rerunning the script at 2 A and 20 A with the same compensator gives crossovers of 20.1 and 19.9 kHz and margins of 65.0° and 66.4°. A 10 A ideal step produces a 138 mV undershoot (1.15 %, inside the 3 % of R5) and settles inside ±30 mV in 46 µs (inside 100 µs). The ideal step is

harsher than the 1 A/µs slew of the real test, yet the bench shows 158 mV (Table 29.5): the averaged model omits the board's distributed inductance and capacitor ESL, which add a fast initial dip, and the gap is recorded in the model register. Figure 29.3 shows the loop gain with the bench measurement overlaid.

> **Status of the principle: crossover at about a tenth of the switching frequency**
> *Empirical.* The averaged model loses validity as frequency approaches $f_{sw}/2$, where sampling effects add phase lag; practice keeps crossover between $f_{sw}/10$ and $f_{sw}/5$, and the bench frequency-response measurement confirms the margin.

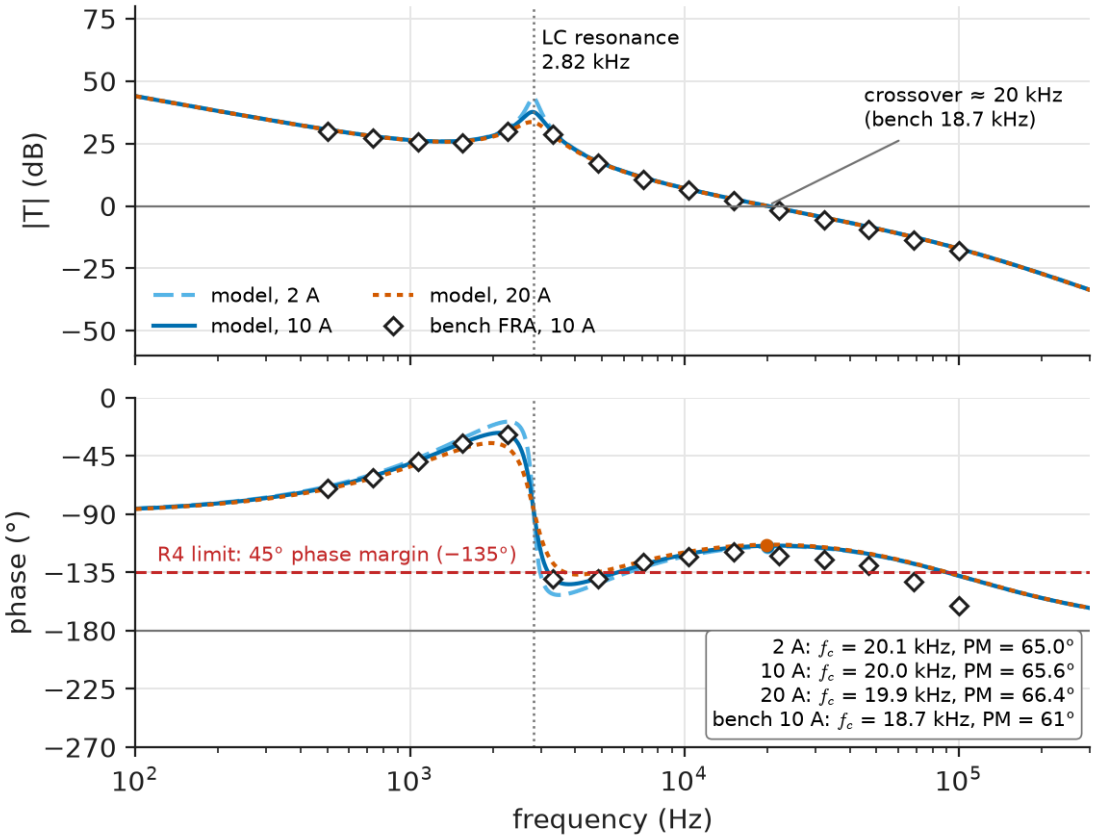


Figure 29.3 — Loop gain of the buck48 voltage-mode loop with type-III compensation at 2, 10 and 20 A, from the averaged model, with bench points at 10 A.

When the averaged model is not enough

The averaged model is linear and continuous. It misses saturation of the compensator output during large steps, current limit, start-up sequencing, and the interaction of the converter with a downstream converter's negative input impedance. For those the project keeps a large-signal averaged model (state equations with duty-cycle limits, integrated by `solve_ivp` with a stiff method) and, for the bench-critical cases, SPICE. Which question goes to which model is written in the project's `docs/models.md`, together with each model's validity domain.

29.5 Version-controlled projects

> **Boundary.**
> Git's data model, branching and team workflows are owned by *Electronics and Computing*, Ch. 24 §24.10; revision management of drawings and documents by Chapter 5 of this book.

One repository, one truth

Everything a reported number depends on, except large raw data, lives in one repository:

```
buck48/
  requirements.toml          product requirements as data (Section 29.1)
  model/losses.py            loss and efficiency model
  model/loop.py              averaged loop model
  spice/buck48_stage.cir     netlists; parameters in spice/corners.toml
  models/                    vendor device models + MANIFEST (SHA-256, source URL, date)
  io/scope_csv.py            instrument import (Section 29.8)
  data/manifest.csv          hashes and metadata of raw captures; files in object storage
  tests/                     unit, property, regression and requirement tests
  results/golden/            reviewed reference results for regression tests
  docs/                      report sources, models.md, decisions/ (one file per decision)
  tools/provenance.py        provenance record writer (Section 29.6)
  environment/               lock file, container definition
  Makefile                   'make all' builds every result, plot and the report
  .gitattributes             line endings; Git LFS for oscilloscope screenshots only
```

The rules that make this work are short:

- **Generated files are not committed** (`results/` other than `golden/`, `build/`). They are reproduced by the build and attached to releases.
- **Vendor models are committed with a manifest** recording their source and hash. A newer vendor download is a reviewed change, not a silent overwrite, which is the opening story's second failure.
- **Raw data are immutable.** Captures go to object storage under their SHA-256; the repository holds the manifest. A corrected capture is a new file with a new hash and a note.
- **Commits reference requirement IDs and decisions** ("R5: increase compensator zero spacing, see docs/decisions/007"), so `git log` answers why a parameter changed.
- **Releases are tags** (`v1.2.0` for the design review), and every number in the review report comes from the build of that tag.

Schematic and layout files from the PCB tool are binary or semi-structured; they are committed with exported netlists and BOMs in text form so that diffs show what changed (Chapter 5).

29.6 Reproducible builds

> **Boundary.**
> Reproducibility of computational experiments is owned by *Electronics and Computing* Ch. 22 §22.10 and Ch. 24 §24.8, and by *The Mathematics Universe*, Ch. 24 §24.9. This section applies them to a hardware-design project.

Pinning the environment

Three layers are pinned, each by the strongest means available:

1. **Python packages** by a lock file with hashes (generated by a tool such as `uv` or `pip-tools`), so that `numpy` is not merely "2.x" but one exact build.
2. **System tools** (ngspice, a LaTeX or Pandoc toolchain for the report) by a container image, referenced in CI by its content digest rather than a tag that can move.
3. **The project's own inputs** by Git, plus the data manifest for large files.

A tool upgrade (a new ngspice, a new SciPy) is then an ordinary change: a commit that updates the lock file or image digest, a CI run that shows the regression diff, and a reviewer who accepts or rejects it.

Provenance on every result

Each result file is written with a provenance record beside it: the commit, whether the working tree was clean, the interpreter and package versions, the platform, the hash of every input file and the random seed.

```
 """Write a provenance record next to every result file (tools/provenance.py)."""
import hashlib, json, pathlib, platform, subprocess, sys
from datetime import datetime, timezone
from importlib import metadata

def sha256(path):
    return hashlib.sha256(pathlib.Path(path).read_bytes()).hexdigest()

def git_state():
    try:
        sha = subprocess.run(["git", "rev-parse", "HEAD"], capture_output=True,
                             text=True, check=True).stdout.strip()
        dirty = bool(subprocess.run(["git", "status", "--porcelain"], capture_output=True,
                                    text=True, check=True).stdout.strip())
        return {"commit": sha, "dirty": dirty}
    except (OSError, subprocess.CalledProcessError):
        return {"commit": None, "dirty": None}

def provenance(inputs, seed=None):
    return {
        "created_utc": datetime.now(timezone.utc).isoformat(timespec="seconds"),
        "git": git_state(),
        "python": sys.version.split()[0],
        "platform": platform.platform(terse=True),
        "packages": {p: metadata.version(p) for p in ("numpy", "scipy", "matplotlib")},
        "inputs": {str(p): sha256(p) for p in inputs},
        "seed": seed,
    }

if __name__ == "__main__":
    rec = provenance(["requirements.toml", "model/losses.py"], seed=20260412)
    pathlib.Path("results/losses_10A.prov.json").write_text(json.dumps(rec, indent=2))
    print(json.dumps({k: rec[k] for k in ("git", "python", "packages")}))
    print({k: v[:12] for k, v in rec["inputs"].items()})
```

Run in a scratch copy of the project that is not a Git checkout, it prints:

```
 {"git": {"commit": null, "dirty": null}, "python": "3.14.7", "packages": {"numpy": "2.5.4",
"scipy": "1.18.1", "matplotlib": "3.11.2"}}
{'requirements.toml': '1f594c52af49', 'model/losses.py': '358d24906ac3'}
```

The null commit is the point: the report builder refuses any result whose provenance has no commit or a dirty tree, so a number computed from uncommitted edits cannot reach a review document. In CI the same script records the commit SHA and `"dirty": false`.

Determinism, and where to stop demanding it

Pure-Python and NumPy results are bit-identical between runs on the same container image, provided random numbers come from a seeded `numpy.random.Generator` and multithreaded BLAS reductions are pinned to one thread (`OMP_NUM_THREADS=1`) for the golden runs. Across processors and BLAS builds, results

differ in the last few bits, because floating-point addition is not associative (*The Mathematics Universe* Ch. 22 §22.1). SPICE results move by much more when the platform changes, because adaptive time stepping amplifies tiny differences into different step sequences. W1 therefore demands bit-identical Python results on the pinned image and SPICE results within a stated tolerance, not identity.

Worked example — choosing regression tolerances. For the analytic loss model, platform differences are of order 10^{-15} relative, and the golden test uses 10^{-3} only to absorb deliberate rounding in the stored value. For SPICE at `reltol` = 10^{-4}, repeated runs on two platforms differed by up to 0.01 points of efficiency and 0.4 mV of ripple; the regression thresholds are set at 0.05 points and 2 mV, five times the observed scatter, and well below the 0.72-point margin of R1, so a real change of a tenth of a point is caught while numerical noise is not.

> **Status of the principle: regression tolerance from numerical scatter**
> *Derived.* A threshold several times the observed run-to-run scatter keeps the false-failure rate low, and one much smaller than the requirement margin keeps real changes visible; if the two conditions cannot both be met, the numerical method is not good enough for the requirement.

The CI time budget

W2 asks for feedback within 10 minutes on every push; the full corner campaign (Section 29.3) runs nightly and on release tags. Table 29.3 is the budget for the per-push pipeline.

Table 29.3 — Per-push CI time budget (measured median of 20 runs).

Stage	Duration
Container image pull (cached by digest)	25 s
Checkout and environment check	15 s
Lint and type check	10 s
Unit, property, regression and requirement tests	20 s
Averaged model and loop analysis	15 s
SPICE smoke run: 3 corners, 4 parallel jobs	45 s
Report build	40 s
Artifact upload and runner overhead	30 s
Total	**200 s**
Limit W2 / margin	600 s / 400 s (67 %)

```
 # .github/workflows/ci.yml -- per-push pipeline (image pinned by digest)
name: ci
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-24.04
    container: ghcr.io/example/buck48-tools@sha256:0f3c...e91a
    env:
      OMP_NUM_THREADS: "1"
    steps:
      - uses: actions/checkout@v4
      - run: make lint
      - run: python -m pytest -q --junitxml=build/junit.xml
      - run: make loop spice-smoke
      - run: make report
      - uses: actions/upload-artifact@v4
        with:
          name: report
          path: build/report/
```

The digest in the listing is abbreviated for print; the real file carries all 64 hexadecimal digits.

29.7 Automated testing

> **Boundary.**
> Test design and `pytest` practice for scientific code are in *Electronics and Computing* Ch. 24 §24.9; regression testing and test plans for hardware in Chapter 4 of this book.

Four kinds of test

Engineering code needs tests that software projects rarely write, because its correctness is physical as well as logical:

- **Unit tests against hand calculations**: one term of the model against a number worked by hand.
- **Property tests**: relations that must hold for any input, such as losses rising with switching frequency, efficiency below 1, energy balance between input and output power in a simulation, or a filter with unity DC gain.
- **Regression tests** against reviewed golden results, with the tolerances of Section 29.6.
- **Requirement tests** that read their limit from `requirements.toml` and carry a marker naming the requirement, from which the traceability matrix is generated.

```
 import json, math, pathlib, tomllib
import pytest
from model.losses import P, losses, efficiency

ROOT = pathlib.Path(__file__).resolve().parents[1]
REQ = tomllib.loads((ROOT / "requirements.toml").read_text())

def test_unit_conduction_matches_hand_calc():
    # D = 0.25, ripple r = 0.662: I_rms,LS = 10 * sqrt(0.75 * (1 + r^2/12)) = 8.82 A
    i_rms = 10 * math.sqrt(0.75 * (1 + 0.662**2 / 12))
    assert losses(10.0)["cond_ls"] == pytest.approx(i_rms**2 * 7e-3 * 1.6, rel=1e-3)

def test_property_losses_rise_with_frequency():
    hi = dict(P, fsw=2 * P["fsw"])
    assert sum(losses(10.0, hi).values()) > sum(losses(10.0).values())

def test_regression_against_golden():
    g = json.loads((ROOT / "results" / "golden_losses_10A.json").read_text())
    assert sum(losses(g["io"]).values()) == pytest.approx(g["total_w"], rel=g["rtol"])

@pytest.mark.req("R1", "R2")
@pytest.mark.parametrize("rid", ["R1", "R2"])
def test_r1_r2_efficiency(rid):
    assert efficiency(REQ[rid]["io"]) >= REQ[rid]["eta_min"]

@pytest.mark.req("R3")
def test_r3_ripple_bound():
    d = P["vo"] / P["vin"]
    di = (P["vin"] - P["vo"]) * d / (P["fsw"] * P["L"])
    ripple = di * P["esr"] + di / (8 * P["fsw"] * P["c_out"])   # ESR term + capacitive term
    assert ripple <= REQ["R3"]["ripple_max_v"]
```

With the golden file holding 3.4361 W at 10 A and the `req` marker registered in `pytest.ini`, `python -m pytest -v` reports six passing tests (the efficiency test runs once per requirement) in well under a second. The ripple bound evaluates to $6.62 \times 0.005 + 6.62/(8 \times 200\ \text{kHz} \times 470\ \mu\text{F}) = 33.1 + 8.8 = 41.9$ mV against 50 mV, a 16 % margin.

A requirement test on a *model* is not verification of the *product*; it is an early warning that a design change has eaten the margin. The traceability matrix therefore has one column for model tests and one for bench tests, and a requirement is closed only when its bench column is filled (Section 29.10).

> **Status of the principle: test the physics as well as the code**
> *Empirical.* Experience with engineering software shows that most damaging defects are wrong units, wrong parameters and violated physical relations rather than crashes; unit tests against hand calculations and property tests on conservation laws catch them at the lowest cost.

Hardware in the loop of the workflow

The same test runner drives the bench. A bench test script (Chapter 4) sets the electronic load, triggers the power analyser and the oscilloscope over SCPI, stores the raw capture under its hash, imports it (Section 29.8) and asserts against the same TOML limits. The bench results enter the traceability matrix through the same marker mechanism, tagged with the unit's serial number and the instrument calibration dates.

29.8 Data import and export

> **Boundary.**
> File formats (CSV, JSON, HDF5, Parquet) and instrument control (VISA, SCPI) are owned by *Electronics and Computing* Ch. 22 §22.7 and §22.9, and scientific data formats by Ch. 24 §24.3; data management at scale is Chapter 30 of this book.

What crosses the project boundary

Table 29.4 — Data formats in `buck48`.

Data	Format	Metadata that must travel with it	Pitfall seen
Oscilloscope captures	Vendor CSV with header lines; binary for long records	Model, serial, date, probe ratios and units, sample rate	Probe ratio applied twice; locale decimal commas
Power-analyser readings	CSV via SCPI script	Range, integration time, calibration date	Range changes mid-sweep
SPICE output	`.meas` log lines; raw files for waveforms	Netlist hash, model manifest, options	Values read off a plot window
Inductor and capacitor impedance	Touchstone `.s1p` / `.s2p`	Frequency unit, reference impedance, bias condition	Small-signal data used at full DC bias
Results	JSON with SI units in key names, plus provenance	Commit, inputs, seed	Rounded values reused as inputs
Large time series	HDF5 or Parquet	Units, sample rate, channel map	Spreadsheet truncation at 1 048 576 rows

An import function that refuses bad data

The capture reader keeps metadata with the data, takes channel units from the header rather than assuming them, and rejects malformed time axes instead of passing them on.

```
 """Import a four-channel scope capture with its metadata (io/scope_csv.py)."""
import csv, io
import numpy as np

SAMPLE = """Model,DSO-X 4034A
Serial,MY00000000
Date,2026-04-12T10:41:07Z
Probe CH1,10:1 V
Probe CH2,0.1 V/A
Probe CH3,10:1 V
Probe CH4,0.1 V/A
Time (s),CH1 (V),CH2 (A),CH3 (V),CH4 (A)
-1.0000E-05,48.03,2.590,11.981,10.02
-5.0000E-06,48.01,2.574,11.978,10.00
0.0000E+00,48.02,2.583,11.982,10.01
5.0000E-06,48.02,2.578,11.980,10.01
"""

def read_capture(text):
    lines = text.splitlines()
    head = next(i for i, l in enumerate(lines) if l.startswith("Time"))
    meta = dict(row[:2] for row in csv.reader(lines[:head]))
    names = [c.split(" (")[0] for c in lines[head].split(",")]
    units = [c.split("(")[1].rstrip(")") for c in lines[head].split(",")]
    data = np.loadtxt(io.StringIO("\n".join(lines[head + 1:])), delimiter=",")
    if np.any(np.diff(data[:, 0]) <= 0):
        raise ValueError("time axis not strictly increasing")
    return meta, dict(zip(names, data.T)), dict(zip(names, units))

meta, ch, units = read_capture(SAMPLE)
pin = np.mean(ch["CH1"] * ch["CH2"])
pout = np.mean(ch["CH3"] * ch["CH4"])
print(f"{meta['Model']} {meta['Date']}  units {units}")
print(f"Pin {pin:.2f} W, Pout {pout:.2f} W, efficiency {100 * pout / pin:.2f} %")
```

```
 DSO-X 4034A 2026-04-12T10:41:07Z  units {'Time': 's', 'CH1': 'V', 'CH2': 'A', 'CH3': 'V', 'CH4':
'A'}
Pin 123.95 W, Pout 119.92 W, efficiency 96.75 %
```

Worked example — is a scope good enough to verify R1? The import works, but the instrument does not. An oscilloscope with current probes measures each power to roughly ±1 % (gain accuracy of channel and probe combined); efficiency is a ratio, so its relative uncertainty is about $\sqrt{1^2 + 1^2} = 1.4$ %, or ±1.4 points at 96.8 %. R1's margin on the bench is 0.3 points, so the scope cannot decide pass or fail. A power analyser with 0.1 % basic accuracy on each power gives $\sqrt{0.1^2 + 0.1^2} = 0.14$ %, ±0.14 points, which can. The capture above is used for waveform checks; R1 is closed with the analyser (bench test EFF-01), and the uncertainty is recorded next to the result.

> **Status of the principle: uncertainty of a ratio of independent measurements**
> *Derived.* Relative standard uncertainties of independent factors combine by root-sum-square (GUM, JCGM 100); correlated errors, such as a shared voltage channel, partly cancel and must be treated explicitly.

Export rules

Results are exported with units in field names (`eff_pct`, `ripple_mv`), full precision (Python's `repr` round-trips doubles exactly), timestamps in UTC ISO 8601, and a provenance file. Exchange copies for colleagues who use spreadsheets are generated from the JSON and marked read-only; numbers never flow back from a spreadsheet into the repository except through a reviewed edit of a source file.

29.9 Results visualization

> **Boundary.**
> Plotting libraries and plotting practice are covered in *Electronics and Computing* Ch. 22 §22.4; figure conventions for documentation in Chapter 5.

A plot in a design report is a measurement claim, and it obeys the same rules as a number: it is generated by a script from result files, never edited by hand, regenerated by the build, and carries its provenance. In `buck48` every figure has a function in `docs/figures.py` that reads JSON results and writes the image; the report includes the image file, and CI fails if any figure is older than its inputs.

The project's plotting conventions follow from what reviewers need to decide:

- **Requirements are drawn on the plot.** The efficiency curve carries R1 and R2 as markers with their limits, the Bode plot a line at the required margin, the load-step plot the −3 % deviation limit and the ±0.25 % recovery band (Figures 29.4 and 29.5). A reader sees the margin without looking up a table.
- **Model, simulation and measurement on the same axes**, with distinct markers and the measurement's uncertainty as error bars, so disagreement is visible rather than hidden in separate figures.
- **Units and conditions in the axis labels and caption**: input voltage, temperature, unit serial number.
- **Linear axes for quantities judged by difference** (efficiency in percent), logarithmic for quantities spanning decades (frequency, impedance).
- **Perceptually uniform colour maps** for heat maps, and line styles that survive greyscale printing.

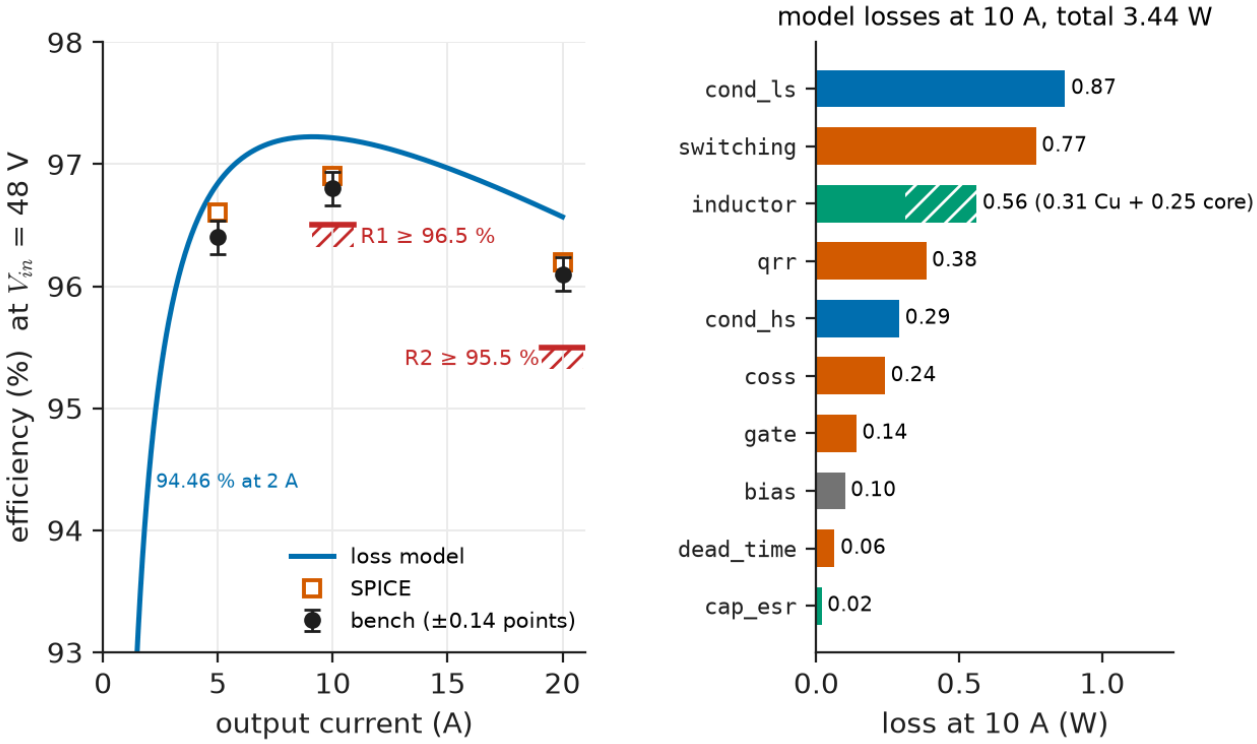


Figure 29.4 — Efficiency of buck48 at 48 V versus load from the loss model, SPICE and the bench, with R1 and R2 drawn on the plot, and the loss breakdown at 10 A.

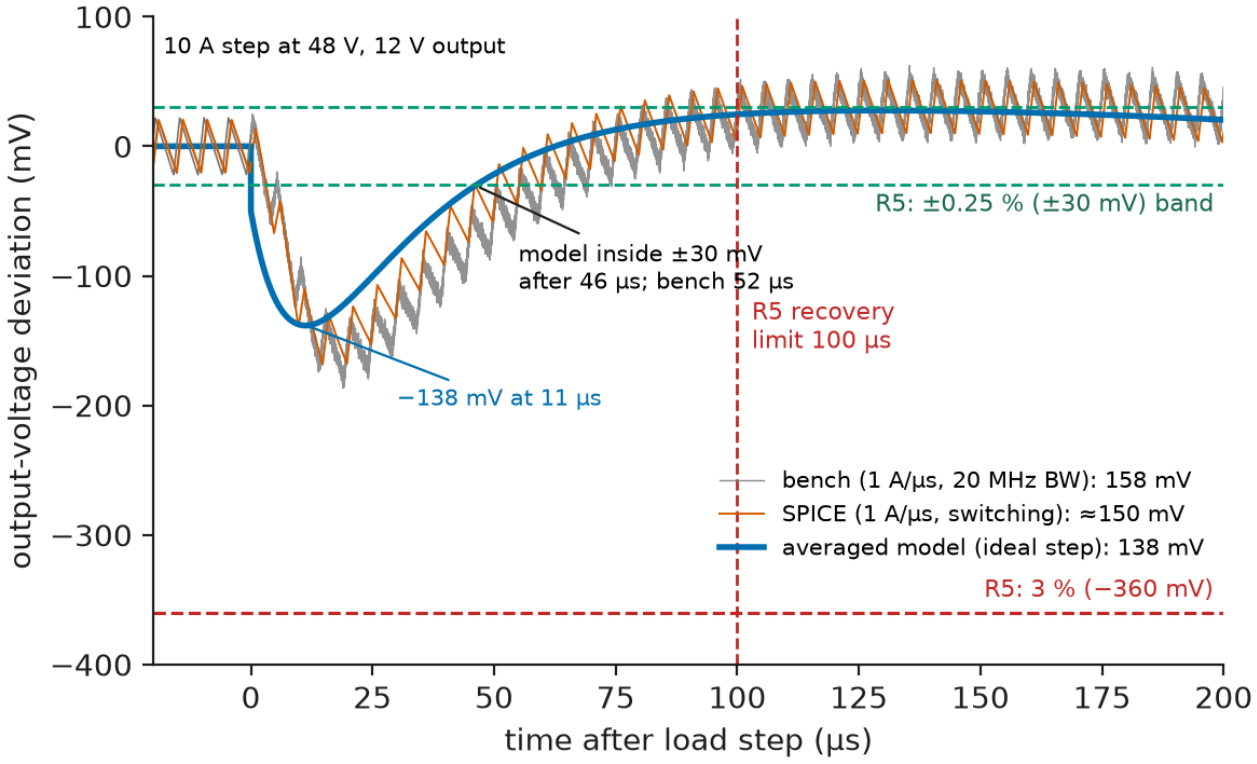


Figure 29.5 — Output-voltage deviation for a 10 A load step at 48 V: averaged model, SPICE and bench, with the R5 limits.

29.10 Engineering documentation

> **Boundary.**
> What a schematic, BOM, design calculation and test report must contain, and how revisions are managed, are owned by Chapter 5. This section shows how the workflow generates and checks them.

Documents built from the repository

The design report for the review is built by `make report` from Markdown sources in `docs/`. Every number in it is inserted from a result file by a template variable (for example the efficiency at 10 A from `results/losses_10A.json`), every figure comes from Section 29.9, and the requirements chapter is rendered from `requirements.toml`. Typing a number into the report source is a review finding. Three generated artefacts accompany each release:

- **Traceability matrix**: requirement → model tests → simulation cases → bench tests → status, generated from the test markers and the bench records. An empty bench column for any requirement blocks the release tag (W3).
- **Release notes**: generated from commit messages since the previous tag, grouped by requirement ID, with the regression diff of every changed golden result.
- **Model register**: each model's version, validity domain and the date it was last checked against the bench.

The report states, for each result, whether it is calculated, simulated or measured, the principle of not overclaiming applied by the build rather than left to the author.

Verification of the workflow and the design

Table 29.5 — Verification of `buck48` at release v1.2.0.

Req.	Test	Limit	Result	Pass/fail
R1	Model; SPICE; bench EFF-01 (power analyser, ±0.14 points)	≥ 96.5 %	97.22 %; 96.9 %; 96.8 %	Pass
R2	As R1 at 20 A	≥ 95.5 %	96.57 %; 96.2 %; 96.1 %	Pass
R3	Bound; SPICE; bench RIP-01 (20 MHz)	≤ 50 mV p-p	41.9 mV; 41 mV; 44 mV	Pass
R4	Averaged model; bench frequency-response analyser	PM ≥ 45°, f_c ≥ 15 kHz	65.0–66.4°, 19.9–20.1 kHz; bench 61°, 18.7 kHz	Pass
R5	Model; SPICE; bench electronic load, 1 A/µs	≤ 360 mV; ±0.25 % in ≤ 100 µs	138 mV, 46 µs; 150 mV; 158 mV, 52 µs	Pass
W1	Rebuild of tag v1.2.0 on a clean machine	Python identical; SPICE within tolerance	Python identical; SPICE max difference 0.01 points	Pass
W2	CI timing, median of 20 runs	≤ 10 min	200 s	Pass
W3	Generated traceability matrix	No empty rows	10 of 10 requirements linked	Pass
W4	Review of tool and model changes since v1.1	All by reviewed commit	2 changes (SciPy minor, MOSFET model), both with diffs	Pass
W5	Hash check of data manifest	All hashes match	412 of 412	Pass

The bench efficiency sits 0.4 points below the model at 10 A, which is 0.53 W of unmodelled loss ($120/0.968 - 120 = 3.97$ W measured against 3.44 W modelled): PCB copper and connector losses, about 0.5 W at 10 A by a separate calculation. The model is now flagged in the model register as optimistic by that amount, and a copper-loss term is the next change to it.

Failure analysis of the workflow

Table 29.6 — FMEA of the engineering workflow.

Failure mode	Effect	Cause	Detection	Mitigation
Hand-edited parameter in a spreadsheet	Irreproducible, optimistic result	Calculation outside version control	Number cannot be regenerated at review	Calculations as code; parameters once in source
Vendor model silently replaced	SPICE results shift without a code change	File overwritten under the same name	Manifest hash mismatch in CI	Model manifest with hashes; updates by reviewed commit
Library upgrade changes a default	Different results on a new machine	Unpinned environment	Regression diff on rebuild	Lock file and container digest; upgrades as reviewed commits
Result from uncommitted code	Report number with no traceable source	Script run on a dirty tree	Provenance shows dirty or null commit	Report builder rejects such results
Golden file updated to make tests pass	Real regression hidden	Pressure to make CI green	Golden changes listed in release notes	Golden updates need a second reviewer and a reason
Flaky SPICE test	Ignored CI failures	Tolerance below numerical scatter	Failure rate tracked per test	Tolerances from measured scatter (Section 29.6)
Probe ratio applied twice	Measured power 10 times off	Scope already scaled, script scaled again	Energy-balance property test on imported data	Units read from capture header, never assumed
Raw data lost or edited	Verification cannot be re-checked	Files on a laptop	Manifest hash check	Object storage by hash; immutable bucket policy

Worked example — the regression that the pipeline caught. Three weeks before v1.2.0, a commit updating the MOSFET model to the vendor's latest release turned the nightly campaign red: SPICE efficiency at 20 A fell from 96.5 % to 96.2 %, outside the 0.05-point tolerance. The diff showed only the model file. Inspection found that the new model included the package's source inductance, which the old one omitted, raising switching loss. The change was physical, not numerical, so the golden results were updated with a recorded reason, the bench measurement (96.1 %) agreed with the new model better than with the old, and the release notes listed the change. Under the old workflow the same update would have shifted a number in a slide, and no one would have known why.

What remains open

Open question	Where it is answered
How are the requirements and acceptance criteria themselves written and traced?	Chapter 1
How are models validated, and their sensitivity and assumptions documented?	Chapter 3
How are bench test plans, calibration and regression testing of hardware organised?	Chapter 4
What must schematics, BOMs and test reports contain, and how are revisions managed?	Chapter 5
How are solvers chosen, sweeps and optimisation run at scale, and numerical results verified?	Chapter 30
How is an engineering tool itself architected (domain model, file formats, plugins, GUI)?	Chapter 31
How are simulation models calibrated against plant data and run as digital twins?	Chapter 33
How does a complete simulation tool apply this workflow end to end?	Chapter 39

Next: Chapter 30 takes the numerical core of this workflow further: solvers, parameter sweeps, optimisation, performance and the verification of numerical results.

Chapter 30 — Scientific and Engineering Computing

Part VIII · Engineering Software and Computational Tools

A drives team needs the lightest forced-air heatsink that keeps the three SiC half-bridge modules of a 30 kW industrial inverter below 125 °C junction temperature in 45 °C air. An optimiser wrapped around a finite-difference model of the heatsink base, run overnight on a 4 mm grid "for speed", reports a 1.03 kg extrusion with 18 fins 23.4 mm high on a 4 mm base, sitting exactly at 125.0 °C. The prototype, instrumented with thermocouples, runs its hottest module about 10 K above the prediction. The post-mortem finds three faults, none in the heat-transfer physics. On the 4 mm grid the 100 × 60 mm footprints had been rasterised to 6240 and 6656 mm² instead of 6000 mm², so the model read 0.5 K low exactly at the active constraint. A fifth of the fan's air leaked over the fin tips, which no comparison with a measurement had revealed. And a rerun gave different numbers, because the model file had been edited and nobody had recorded which version produced the report. This chapter is about the computing practice that would have caught all three before the prototype was cut.

> **How are numerical solvers, sweeps and optimisers chosen and combined so that a design computation is fast enough to explore the design space and accurate enough to decide on?**
> **How is a computed result verified, validated against a measurement, and stored so that it can be trusted and reproduced a year later?**

Concept	Engineering destination
Numerical solvers	Sparse steady-state field solvers, operating-point root finding, stiff thermal and circuit transients
Parameter sweeps	Design-space maps of heatsinks, magnetics, filters and PV strings; feasibility boundaries
Optimization	Lightest heatsink, smallest inductor, lowest-loss switching frequency under constraints
Simulation automation	Overnight campaigns that run unattended, recover from failures and regenerate reports
Data processing	Mission profiles, thermocouple logs, steady-state detection, campaign post-processing
Computational performance	Choosing sparse over dense, factorising once, exploiting structure before buying hardware
Parallel processing	Process pools for independent cases; predicting and measuring speed-up
Verification of numerical results	Conservation checks, exact limits, grid-convergence studies with an error bar
Model validation	Comparing predictions with prototype measurements under a stated acceptance limit
Scientific data management	Manifests, hashes, formats and retention that make a result reproducible and auditable

Roadmap. The reference computation, its requirements and architecture (30.1) → the solvers inside it (30.2) → sweeps (30.3) and optimisation (30.4) on top of the solvers → automation of the campaign (30.5) and processing of its data (30.6) → performance (30.7) and parallel execution (30.8) → verification (30.9) and validation (30.10) of what it computes → data management (30.11) → failure analysis of the whole computation (30.12).

30.1 The reference computation

> **Boundary.**
> Root finding, ODE and PDE discretisation and convergence are *The Mathematics Universe*, Ch. 22; sparse and iterative solvers Ch. 24; optimisation in Ch. 23. Computing practice at scale (profiling, precision, parallel laws) is *Electronics and Computing*, Ch. 24. The modelling workflow is Chapter 3 and the tooling around it Chapter 29. This chapter assembles them into one design computation and checks it.

The system being designed

The inverter carries three 1200 V SiC half-bridge modules (62 mm class baseplate, footprint 100 × 60 mm each) in a row along the airflow on a 160 × 300 mm extruded 6063-T5 aluminium heatsink. At rated output each module dissipates 200 W (600 W in total, 98 % efficiency at 30 kW), and its junction sits $R_{jh} = 0.13$

K/W above the heatsink surface under it ($R_{th,jc} = 0.09$ K/W from the datasheet plus 0.04 K/W of phase-change interface material). A 92 mm class server fan pushes air along the fins. The design variables are the number of fins N, their height H and the base thickness t_b; the fin thickness is fixed at 1.5 mm by the extruder. The junction-to-heatsink resistance network is *Electronics and Computing*, Ch. 29 §29.5; here the heatsink part becomes a field model because three modules share one plate and the air warms along it.

Requirements on the computation

R1 is the product requirement; R2–R7 are requirements on the computation, and are what the overnight run lacked.

Table 30.1 — Requirements for the heatsink design computation.

ID	Requirement	Target	Verification method
R1	Hottest junction at 45 °C ambient, 200 W per module	≤ 125 °C	Computation (R2–R3 must hold), then prototype test
R2	Discretisation error of reported module temperatures	≤ 0.1 K on the production grid	Grid-convergence study, §30.9
R3	Model against prototype, base temperature at a load point not used for calibration	within ±1.6 K	Validation test, §30.10
R4	Full design sweep (420 cases) and optimisation, wall-clock on the 8-core workstation	≤ 5 min and ≤ 2 min	Timed run, §30.7–30.8
R5	600 s mission-profile transient	≤ 1 s, error ≤ 0.01 K	Comparison with exact solution, §30.7
R6	Every result traceable to model version, inputs and library versions	100 % of stored results	Manifest audit, §30.11
R7	Cases outside a correlation's validity or failing to converge	recorded, never dropped; fraction reported	Sweep report, §30.3

Architecture

Figure 30.1 shows the computation as blocks with explicit interfaces. Configuration is data, not code. The physics core maps a design and an operating point to a temperature field and scalar outputs; drivers (sweep, optimiser, transient, validation) call it and never reach inside. Every result is stored with a manifest, and verification and validation are gates in the chain, not activities after it.

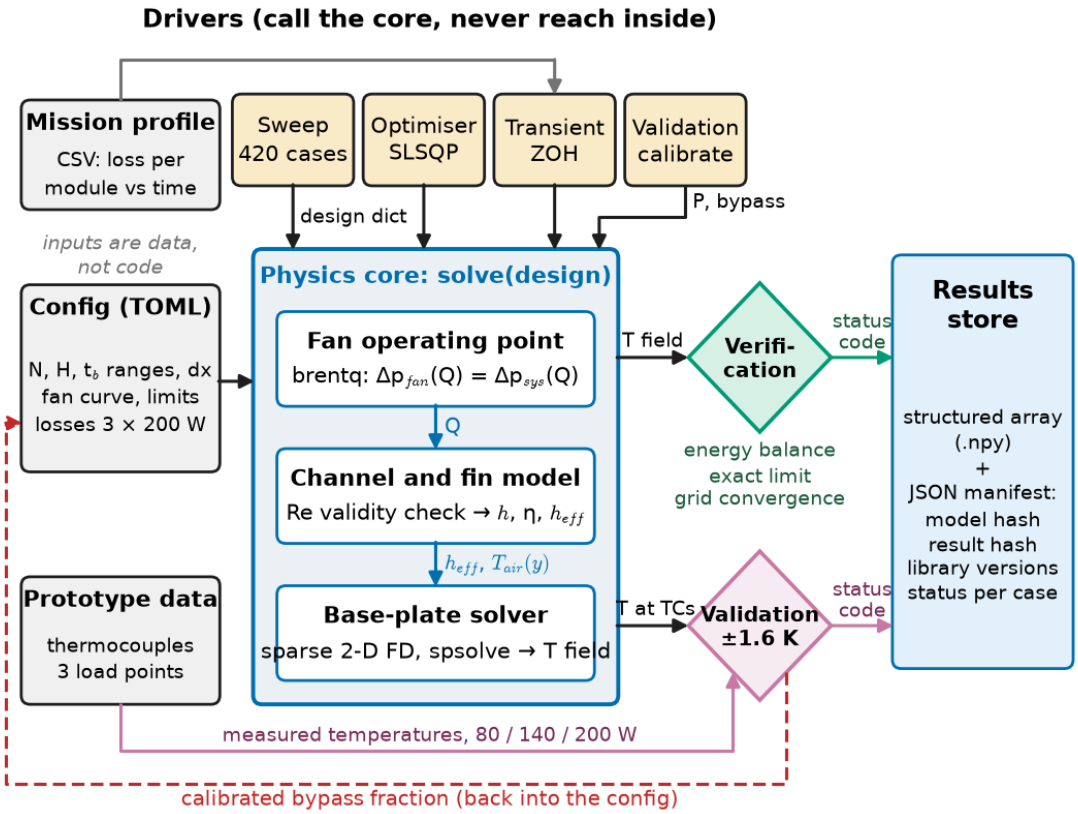


Figure 30.1 — Architecture of the heatsink design computation: configuration, physics core, drivers, results store and the verification and validation gates.

Choosing the computational tool

Table 30.2 — Tool classes for the heatsink computation, against the requirements.

Option	Fidelity	Cost per case	Licence (2026, indicative)	Fit to R2–R7
Spreadsheet with one-node R_{ha} from a catalogue curve	No air heating, no spreading	Seconds by hand	None	Cannot show module-to-module difference or verify; rejected
Vendor online heatsink calculator	Correlations, black box	Minutes, manual	Free	No automation, no version record; useful as a cross-check
Commercial electronics-cooling CFD (Icepak, FloTHERM, Simcenter class)	3-D conjugate heat transfer, bypass resolved	5–60 min	About $20,000–40,000 per seat per year	Too slow for a 420-case sweep; used once to check the final design
Scripted 2-D plate model (Listing 30.1)	Correlations plus 2-D spreading	70 ms	Open source	Meets R4–R7; needs validation for R3. **Chosen**

Budgets

Two budgets govern the computation: wall-clock time (R4, R5) and accuracy (R1–R3). Table 30.3 is the accuracy budget for the junction temperature: the computed value must sit below the limit by at least the combined uncertainty of the model, or the margin is fictitious.

Table 30.3 — Accuracy budget for the predicted hottest-junction temperature (Revision A design, after validation).

Contribution	Value	How obtained
Discretisation error, 2 mm production grid	0.01 K	Grid convergence, §30.9
Discretisation error, 4 mm optimiser grid	0.53 K	Comparison with 1 mm grid, §30.4
Model-form error (correlation, bypass, uniform h) after calibration	0.5 K (validation residual max 0.48 K)	Validation, §30.10
Interface resistance spread (R_{ch} 0.04 ± 0.01 K/W at 200 W)	±2.0 K	Interface-material datasheet, lot spread
$R_{th,jc}$ spread (0.09 K/W, max 0.10 K/W)	+2.0 K	Module datasheet maximum
Root-sum-square, production grid	**≈ 2.9 K**	
Required margin below 125 °C	**≥ 3 K**	Design rule
Margin of the overnight result	**0.0 K** (−0.53 K on a fine grid)	Fails before the bypass is even counted

---

30.2 Numerical solvers

One computation, four kinds of solver

The heatsink computation contains four numerical problems, and each wants a different solver (Table 30.4).

Table 30.4 — Sub-problems of the reference computation and the solver chosen for each.

Sub-problem	Mathematical form	Solver chosen	Why
Fan operating point	Scalar root: $\Delta p_{fan}(Q) = \Delta p_{sys}(Q)$	Bracketing root finder (`brentq`)	Guaranteed convergence inside a bracket; the fan curve is monotonic
Fin and channel model	Closed-form correlations	Direct evaluation with a validity check	No iteration needed; validity range must be enforced
Base-plate temperature	Sparse linear system, 10^3–10^5 unknowns	Sparse direct LU (SuperLU via `spsolve`)	Symmetric, well conditioned, solved once per design
Junction transient	5 linear ODEs, time constants 1 ms to 80 s	Exact zero-order-hold discretisation	Linear and piecewise-constant input: no time-stepping error at all

The rule: identify the structure first (linear or not, stiff or not, one solve or thousands) and pick the most specialised solver it allows. The mathematics of each method (bisection and Newton in *The Mathematics Universe*, Ch. 22 §22.4; stiffness in §22.9; sparse direct factorisation in Ch. 24 §24.5) is not repeated here.

> **Status of the principle: Structure before solver**
> *Empirical.* Exploiting linearity, sparsity or a closed form reliably buys orders of magnitude in run time and removes whole classes of solver error; a general method is the fallback.

The physics core

The base is modelled as a 2-D conducting plate of thickness t_b and conductivity k, heated by the module footprints on top and cooled through the fins underneath by an effective coefficient h_{eff} referred to the base area:

$$k\,t_b\,\nabla^2 T - h_{eff}\left(T - T_{air}(y)\right) + q''(x,y) = 0,$$

with adiabatic edges and air warming linearly along the flow by $P/(\rho c_p Q)$. The fin coefficient h comes from a developing-laminar correlation below $Re = 2300$ and from Gnielinski's correlation above $Re = 3000$ (convection groups in *Engineering Physics*, Ch. 19 §19.6); the transitional band between them has no validated correlation, so the model refuses to answer there. Fin efficiency is the adiabatic-tip result $\eta = \tanh(mH)/(mH)$ with $m = \sqrt{2h/(kt)}$.

> **Status of the principle: Effective-coefficient plate model of a finned heatsink**
> *Approximate.* Assumes conduction in the base is two-dimensional (base thin compared with the spreading length $\sqrt{kt_b/h_{eff}}$, here 49 mm against 4 mm), a single h for all fins, adiabatic fin tips, and air heated uniformly along the channel. It fails near the fan hub and for thick bases.

Listing 30.1 is the whole core, saved as `hs_model.py`; every later listing imports it.

```
 """hs_model.py -- forced-air heatsink model of the three-module SiC inverter."""
import numpy as np
import scipy.sparse as sp
from scipy.optimize import brentq
from scipy.sparse.linalg import spsolve

RHO, CP, MU, K_AIR = 1.09, 1007.0, 1.96e-5, 0.028              # air at about 45 C
W, L, K_AL, T_AMB = 0.16, 0.30, 200.0, 45.0                    # base 160 x 300 mm, 6063-T5
CENTRES, FX, FY = [(0.08, 0.06), (0.08, 0.15), (0.08, 0.24)], 0.10, 0.06

def fan_dp(Q):                                                 # fan curve, Pa (Q in m3/s)
    return 250.0 * (1.0 - (Q / 0.070) ** 2)

def channel(N, H, t, Q):                                       # dp, h, gap, Reynolds number
    s = (W - N * t) / (N - 1)
    v, Dh = Q / ((N - 1) * s * H), 2 * s * H / (s + H)
    Re = RHO * v * Dh / MU
    f_t = (0.79 * np.log(Re) - 1.64) ** -2                     # Petukhov friction factor
    if Re < 2300:                                              # developing laminar flow
        Gz = Dh / L * Re * 0.71
        Nu = 7.54 + 0.03 * Gz / (1 + 0.016 * Gz ** (2 / 3))
    else:                                                      # Gnielinski, Pr = 0.71
        Nu = f_t / 8 * (Re - 1000) * 0.71 / (1 + 12.7 * np.sqrt(f_t / 8) * (0.71 ** (2 / 3) - 1))
    dp = max(96 / Re, f_t) * L / Dh * 0.5 * RHO * v**2 + 0.75 * RHO * v**2
    return dp, Nu * K_AIR / Dh, s, Re

def lap1d(n, dx):                                              # adiabatic (Neumann) edges
    d = -2.0 * np.ones(n); d[[0, -1]] = -1.0
    return sp.diags([np.ones(n - 1), d, np.ones(n - 1)], [-1, 0, 1]) / dx**2

def solve(N=20, H=0.040, t=0.0015, tb=0.010, P=(200.0,) * 3, dx=0.002, bypass=0.0):
    Q = brentq(lambda q: fan_dp(q) - channel(N, H, t, q)[0], 1e-4, 0.0699)
    _, h, s, Re = channel(N, H, t, (1 - bypass) * Q)           # bypass: flow over fin tips
    if 2300 <= Re <= 3000:
        raise ValueError(f"Re = {Re:.0f}: transitional flow, no validated correlation")
    m = np.sqrt(2 * h / (K_AL * t))
    eta = np.tanh(m * H) / (m * H)                             # adiabatic-tip fin efficiency
    h_eff = h * ((N - 1) * s + 2 * N * H * eta) / W            # per unit base area
    nx, ny = round(W / dx), round(L / dx)
    X, Y = np.meshgrid((np.arange(nx) + 0.5) * dx, (np.arange(ny) + 0.5) * dx, indexing="ij")
    masks = [(abs(X - cx) <= FX / 2) & (abs(Y - cy) <= FY / 2) for cx, cy in CENTRES]
    q = sum(Pk * mk / (mk.sum() * dx * dx) for Pk, mk in zip(P, masks))   # exact power
    T_air = T_AMB + sum(P) / (RHO * CP * (1 - bypass) * Q) * Y / L
    A = K_AL * tb * sp.kronsum(lap1d(ny, dx), lap1d(nx, dx)) - h_eff * sp.eye(nx * ny)
    T = spsolve(A.tocsc(), -(q + h_eff * T_air).ravel()).reshape(nx, ny)
    return dict(Q=Q, h=h, eta=eta, h_eff=h_eff, T=T, Tmax=T.max(),
                T_mod=[T[mk].mean() for mk in masks],
                T_ctr=[T[np.argmin(abs(X[:, 0] - cx)), np.argmin(abs(Y[0] - cy))] for cx, cy in
CENTRES],
                mass=2700 * (W * L * tb + N * t * H * L),
                heat_out=(h_eff * (T - T_air)).sum() * dx * dx)
```

Listing 30.1 — The physics core: fan operating point, channel and fin model with a validity check, and a sparse 2-D finite-difference solve of the base plate.

Three decisions in the listing matter more than any line of physics. The flux is normalised by the cells actually inside each footprint, so the injected power is exact on any grid even when the area is not. The transitional-flow check raises instead of returning a number, so no driver can mistake an unsupported case for a result. And the function returns the heat leaving through the fins, so every caller can check conservation for free.

Worked example — the fan operating point by hand. For the Revision A design ($N = 18$, $H = 23.4$ mm, $t = 1.5$ mm) the fin gap is $s = (160 - 18 \times 1.5)/17 = 7.82$ mm. `brentq` balances fan and channels at $Q = 0.0368$ m^3/s (78 CFM) and 181 Pa; check: $250\,[1 - (0.0368/0.070)^2] = 181$ Pa. The channel velocity is $0.0368/(17 \times 0.00782 \times 0.0234) = 11.8$ m/s, the hydraulic diameter 11.7 mm and $Re = 7720$, comfortably turbulent, so Gnielinski applies and gives $h = 58.1$ W/(m^2 K). With $mH = 0.46$ the fins are 93.5 % efficient, and the finned surface multiplies the base area by 5.75, giving $h_{eff} = 334$ W/(m^2 K). The air warms by $600/(1.09 \times 1007 \times 0.0368) = 14.9$ K from inlet to outlet. A hand check of each number is the cheapest verification there is.

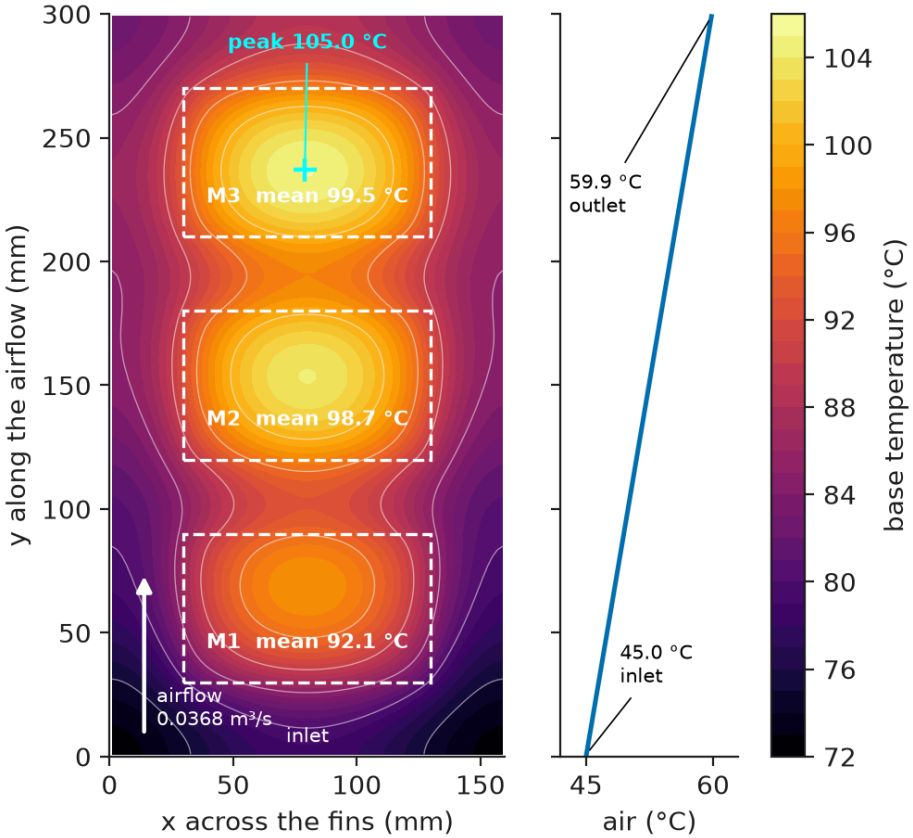


Figure 30.2 — Computed base-plate temperature field of the Revision A heatsink at 200 W per module, 45 C inlet air, 2 mm grid.

In Figure 30.2 module M1 at the inlet runs 7 K cooler than M3 because the air warms along the plate, which a one-node heatsink model would miss.

30.3 Parameter sweeps

Designing the sweep

A sweep is an experiment on a model and needs the same planning as one in a laboratory: variables, ranges, levels, outputs, and what counts as a failed run. The reference sweep is a full factorial over 12 fin counts (14 to 36), 7 fin heights (20 to 50 mm) and 5 base thicknesses (4 to 12 mm): $12 \times 7 \times 5 = 420$ cases. Factorial grids are right for three variables and for drawing feasibility maps; for six or more variables a space-filling design (Latin hypercube or Sobol' points, `scipy.stats.qmc`) gives far better coverage per case, and a full grid at 7 levels would need $7^6 \approx 118\,000$ cases.

Listing 30.2 runs the sweep, keeps failures as data and writes the manifest that R6 requires. It doubles as the parallel benchmark of §30.8.

```
 import hashlib, itertools, json, platform, time
from concurrent.futures import ProcessPoolExecutor
import numpy as np, scipy
import hs_model as hs

CONFIG = dict(N=list(range(14, 37, 2)), H=[0.020, 0.025, 0.030, 0.035, 0.040, 0.045, 0.050],
              tb=[0.004, 0.006, 0.008, 0.010, 0.012], dx=0.002, R_jh=0.13, P=200.0)

def run_case(case):
    N, H, tb = case
    try:
        r = hs.solve(N=N, H=H, tb=tb, dx=CONFIG["dx"])
        Tj = max(r["T_mod"]) + CONFIG["P"] * CONFIG["R_jh"]
        return (N, H, tb, r["mass"], Tj, r["Q"], 0)            # status 0 = converged
    except (ValueError, RuntimeError):
        return (N, H, tb, np.nan, np.nan, np.nan, 1)           # failure is kept as data

if __name__ == "__main__":
    cases = list(itertools.product(CONFIG["N"], CONFIG["H"], CONFIG["tb"]))
    for workers in (1, 8):
        t0 = time.perf_counter()
        with ProcessPoolExecutor(max_workers=workers) as pool:
            rows = list(pool.map(run_case, cases, chunksize=8))
        print(f"{len(cases)} cases, {workers} worker(s): {time.perf_counter() - t0:.1f} s")
    res = np.array(rows, dtype=[("N", "i4"), ("H", "f8"), ("tb", "f8"), ("mass", "f8"),
                                ("Tj", "f8"), ("Q", "f8"), ("status", "i4")])
    np.save("sweep_v1.npy", res)
    manifest = dict(config=CONFIG, n_failed=int(res["status"].sum()),
                    model_sha256=hashlib.sha256(open(hs.__file__, "rb").read()).hexdigest()[:16],
                    python=platform.python_version(), numpy=np.__version__,
scipy=scipy.__version__,
                    result_sha256=hashlib.sha256(res.tobytes()).hexdigest()[:16])
    json.dump(manifest, open("sweep_v1.json", "w"), indent=1)
    print({k: manifest[k] for k in ("n_failed", "model_sha256", "result_sha256")})
```

```
 420 cases, 1 worker(s): 72.9 s
420 cases, 8 worker(s): 16.0 s
{'n_failed': 45, 'model_sha256': '4792b5780f66883d', 'result_sha256': 'e73d62e3cda02e32'}
```

Listing 30.2 — A process-parallel factorial sweep that records failures and writes a manifest (timings from one run on a shared 8-core workstation).

Reading the sweep

Of the 420 cases, 45 (10.7 %) fall in the transitional flow band, all at 34 or 36 fins. Of the 375 valid cases, 292 meet both the 125 °C limit and the extrusion aspect-ratio limit $H/s \leq 10$. The lightest of those on the grid is 18 fins, 25 mm, 4 mm base, at 1.065 kg and 123.7 °C. Figure 30.3 shows the map. Silently skipped, the 45 failures would leave an empty corner nobody asks about; recorded, they say "the model cannot speak for very dense fins", which sends someone to CFD or a wind tunnel if the optimum ever moves there (R7).

> **Status of the principle: Failures are results**
> *Empirical.* Dropping failed or out-of-domain cases biases every statistic of a campaign, because failures cluster.

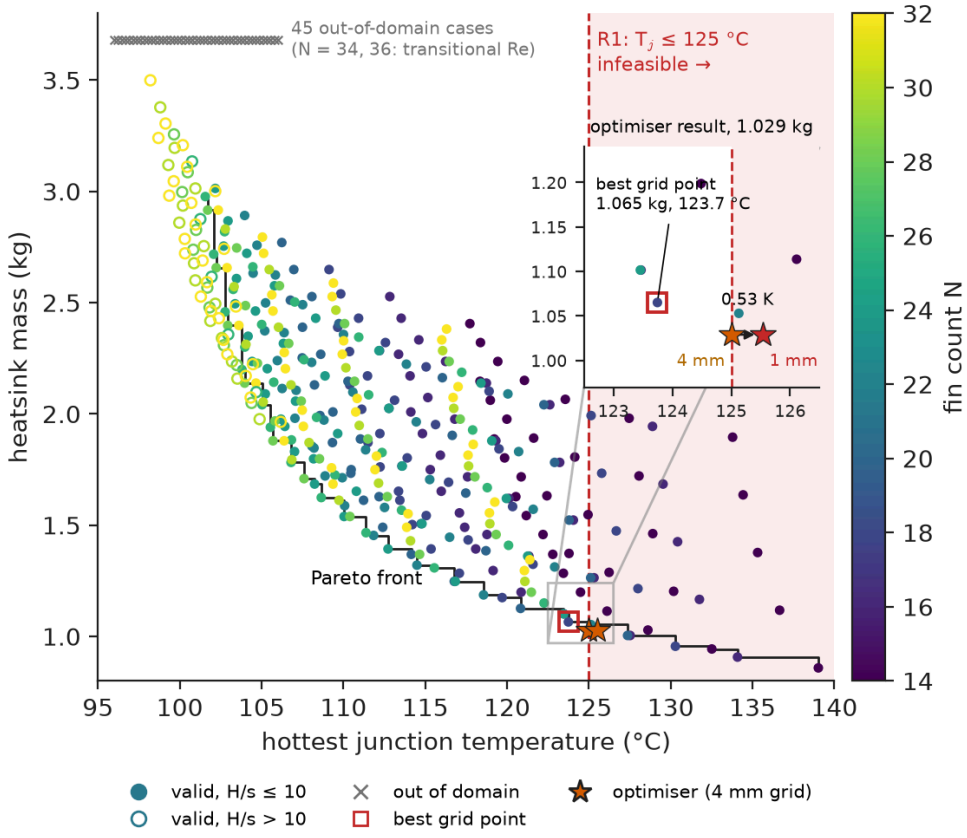


Figure 30.3 — Design-space map from the 420-case sweep: heatsink mass against hottest-junction temperature.

30.4 Optimization

From map to optimum

A sweep finds the neighbourhood; an optimiser finds the point. For each fin count, the continuous variables H and t_b are optimised to minimise mass subject to the junction limit and the aspect-ratio limit. The integer N is enumerated, which is cheap with ten values. The method is SLSQP (sequential quadratic programming); its theory (KKT conditions, quasi-Newton updates) is in *The Mathematics Universe*, Ch. 23 §23.4 and §23.6. Variables are scaled to millimetres, because finite-difference gradients behave badly when variables differ by orders of magnitude.

```
 from scipy.optimize import minimize
import hs_model as hs

T_LIMIT, R_JH, P, T_FIN = 125.0, 0.13, 200.0, 0.0015

def design(N, dx):
    """Lightest (H, tb) for N fins; variables in mm so both are of order 10."""
    Tj = lambda z: max(hs.solve(N=N, H=z[0] / 1e3, tb=z[1] / 1e3, dx=dx)["T_mod"]) + P * R_JH
    gap = (hs.W - N * T_FIN) / (N - 1) * 1e3                    # fin gap, mm
    cons = [{"type": "ineq", "fun": lambda z: T_LIMIT - Tj(z)},
            {"type": "ineq", "fun": lambda z: 10 * gap - z[0]}]  # extrusion aspect ratio <= 10
    mass = lambda z: 2700 * (hs.W * hs.L * z[1] + N * T_FIN * z[0] * hs.L) / 1e3
    res = minimize(mass, x0=[40.0, 8.0], method="SLSQP", constraints=cons,
                   bounds=[(15, 60), (4, 15)], options={"ftol": 1e-6})
    return res.x, res.fun, Tj(res.x), res.nfev, res.success

runs = [design(N, 0.004) + (N,) for N in range(14, 33, 2)]
for x, m, Tj, nfev, ok, N in runs[::2]:                  # every second N shown
    print(f"N = {N:2d}: H = {x[0]:5.2f} mm, tb = {x[1]:.2f} mm, {m:.3f} kg, Tj = {Tj:.2f} C")
best = min(runs, key=lambda d: d[1])
(H, tb), mass, Tj4, nfev, ok, N = best
Tj1 = max(hs.solve(N=N, H=H / 1e3, tb=tb / 1e3, dx=0.001)["T_mod"]) + P * R_JH
print(f"optimum: N = {N}, H = {H:.1f} mm, tb = {tb:.2f} mm, mass = {mass:.3f} kg ({ok})")
print(f"Tj on 4 mm grid = {Tj4:.2f} C, re-checked on 1 mm grid = {Tj1:.2f} C")
```

```
 N = 14: H = 37.61 mm, tb = 4.00 mm, 1.158 kg, Tj = 125.00 C
N = 18: H = 23.36 mm, tb = 4.00 mm, 1.029 kg, Tj = 125.00 C
N = 22: H = 19.76 mm, tb = 4.00 mm, 1.046 kg, Tj = 125.00 C
N = 26: H = 18.27 mm, tb = 4.00 mm, 1.095 kg, Tj = 125.00 C
N = 30: H = 17.80 mm, tb = 4.00 mm, 1.167 kg, Tj = 125.00 C
optimum: N = 18, H = 23.4 mm, tb = 4.00 mm, mass = 1.029 kg (True)
Tj on 4 mm grid = 125.00 C, re-checked on 1 mm grid = 125.53 C
```

Listing 30.3 — Constrained optimisation with enumeration over the integer variable, and a re-check of the optimum on a finer grid (10 s on the workstation).

What the optimiser did

The output reproduces the opening story. The optimiser beat the best grid point by 3.4 % (1.029 kg against 1.065 kg), put the base at its 4 mm bound for every N, and left every optimum exactly on the temperature constraint. That is the danger: any model error at the constraint boundary becomes a violation in the product. On the 4 mm grid the footprints were rasterised to 6656 and 6240 mm² instead of 6000 mm² (§30.9), and the re-check on a 1 mm grid puts the "optimal" design 0.53 K over the limit.

> **Status of the principle: Optimisers exploit model error**
> *Empirical.* An optimum lies on active constraints, and an optimiser preferentially finds regions where the model is optimistic; the remedy is to optimise against a limit tightened by the model's verified error and to re-verify the optimum with the production model.

Worked example — the tightened constraint. The accuracy budget of Table 30.3 gives a combined uncertainty of about 2.9 K on the production grid, so the optimiser's limit becomes $125 - 3 = 122$ °C, plus the 4 mm grid's own 0.53 K if the optimiser keeps the coarse grid: $122 - 0.53 \approx 121.5$ °C. The heavier heatsink that results is the real cost of the requirement, and is what Revision B (§30.12) uses.

Trading mass against fan power or noise makes the problem multi-objective (Pareto fronts, *The Mathematics Universe*, Ch. 23 §23.9). Gradient-free global methods (`differential_evolution`, CMA-ES) earn their 10–100 times higher evaluation count only on multimodal or noisy responses.

30.5 Simulation automation

From scripts to a campaign

The overnight run was a script edited between runs. A trustworthy campaign has five properties:

1. **Configuration as data.** Ranges, grid spacing, losses, fan curve and limits live in one versioned file, not in the script.
2. **One entry point.** One command (`python -m heatsink run campaign.toml`) runs everything.
3. **Idempotent, cached cases.** Each case is keyed by a hash of its inputs and the model version; rerunning skips what is already done and a changed model invalidates exactly what it should (content-addressed caching as in *Electronics and Computing*, Ch. 24 §24.4).
4. **Timeouts and status.** Every external solver call has a timeout; every case ends in a recorded state: converged, out of domain, not converged, timed out, crashed.
5. **Report from data.** Figures and tables in the design report are regenerated from the stored results by the same command, never pasted.

When the solver is someone else's program

When the core is an external tool (ngspice, OpenFOAM, Elmer, a PLECS or Simulink model in batch mode), add three things: one directory per case named by its hash; `subprocess.run(..., timeout=...)` with stdout and stderr kept; and a parser that sets the status from the tool's own convergence report. A solver that writes a plausible result file after failing to converge is the commonest silent error in automated campaigns.

Snakemake, Slurm or a cloud batch queue change where cases run, not these rules; version control and CI around the campaign are Chapter 29.

30.6 Data processing

Input data: the mission profile

The transient study of §30.7 needs module loss against time. The drive logs phase current, DC-link voltage and heatsink temperature at 1 kHz; the chain is: resample to the analysis step (1 s; the junction's millisecond constants are handled analytically); convert current and temperature to conduction and switching loss with the device's tabulated $R_{DS(on)}(T_j)$ and $E_{sw}(I, V)$ (the loss models are *Electronics and Computing*, Ch. 29 §29.3–29.4); and check the energy of the resampled profile against the energy of the raw one. Point sampling instead of interval means can change the energy of a pulsed profile by tens of per cent.

Output data: thermocouple logs and steady state

Worked example — when is the prototype steady? The heatsink time constant is $R_{ha}C_{hs} = 0.086 \times 926 = 80$ s. At 200 W the base under M3 rises about 70 K, so settling to within 0.1 K takes $80 \times \ln(700) = 524$ s, about 9 min. Rather than a fixed wait, the logging script declares steady state when a least-squares line through the last 120 s has a slope below 0.5 K per 10 min, which for a first-order response leaves $0.5/600 \times 80 = 0.067$ K to go, and reports the mean of the last 60 s. The rule lives in code, so two technicians get the same number from the same log.

30.7 Computational performance

Measure, then exploit structure

A profile of 10 calls of `solve` on the 2 mm grid (Python's `cProfile`; profiling practice in *Electronics and Computing*, Ch. 24 §24.6) attributes 79 % of the 72 ms per call to the sparse LU factorisation and solve, 12 % to assembling the sparse matrix, and the rest to the fan root, masks and bookkeeping. Performance is therefore a linear-algebra question (Table 30.5).

Table 30.5 — Measured cost of solving the base-plate system (one right-hand side unless stated; workstation, SciPy 1.18).

Grid	Unknowns	Non-zeros	Dense LU	Sparse LU (`spsolve`)	Factor once, then per solve	Conjugate gradient (10^{-10})
4 mm	3 000	14 770	1 027 ms, 72 MB	13 ms	0.6 ms	29 ms
2 mm	12 000	59 540	≈ 66 s, 1.15 GB (estimated)	58 ms	2.1 ms	366 ms
1 mm	48 000	239 080	18.4 GB: does not fit	300 ms	11 ms	1 362 ms

Three lessons follow. Dense storage is never an option for field problems. Unpreconditioned conjugate gradients lose to sparse LU at these sizes because a 2-D five-point system factorises with little fill-in; iterative solvers win in 3-D and at millions of unknowns, with a good preconditioner (*The Mathematics Universe*, Ch. 24 §24.4). And when the matrix is fixed and only the load changes, factorising once with `splu` costs 2.1 ms instead of 58 ms per load point, a factor of 27.

Structure in the transient

Solver choice matters most in the junction transient. Each module is a four-element Foster network from its datasheet (0.010, 0.030, 0.050, 0.040 K/W; 1 ms, 10 ms, 100 ms, 1 s) on a heatsink of 0.086 K/W (from the steady model) and 926 J/K (1.029 kg of aluminium): time constants from 1 ms to 80 s, a stiffness ratio of 8×10^4. Listing 30.4 solves a 600 s profile of 10 s loss steps four ways and compares them with the exact solution, which exists because the system is linear and the loss is piecewise constant: over each step, $x_{k+1} = e^{-\Delta t/\tau} x_k + (1 - e^{-\Delta t/\tau}) R\, P_k$ per state.

```
 import time
import numpy as np
from scipy.integrate import solve_ivp

R = np.array([0.010, 0.030, 0.050, 0.040])          # Foster network, junction to heatsink, K/W
TAU = np.array([0.001, 0.01, 0.1, 1.0])             # s
R_HA, C_HS, T_AMB = 0.086, 926.0, 45.0              # heatsink to air (K/W), heat capacity (J/K)
rng = np.random.default_rng(7)
P = np.repeat(rng.uniform(30, 230, 60), 10)        # 600 s profile, W per module, 10 s steps
A = np.diag(np.append(-1 / TAU, -1 / (R_HA * C_HS)))
Bv = np.append(R / TAU, 3.0 / C_HS)                  # three modules heat the shared heatsink

def f(t, x):                                         # x = [Foster states..., heatsink rise]
    return A @ x + Bv * P[min(int(t), P.size - 1)]

# Exact solution for piecewise-constant losses: zero-order-hold discretisation per 1 s step.
Ad = np.exp(np.diag(A)); Bd = (Ad - 1) / np.diag(A) * Bv
t0 = time.perf_counter(); x = np.zeros(5); Tj_ref = []
for Pk in P:
    x = Ad * x + Bd * Pk; Tj_ref.append(T_AMB + x.sum())
print(f"ZOH exact   : {1e3 * (time.perf_counter() - t0):7.1f} ms, Tj(600 s) = {Tj_ref[-1]:.3f} C")

for method in ("RK45", "LSODA", "BDF", "Radau"):
    t0 = time.perf_counter()
    sol = solve_ivp(f, (0, P.size), np.zeros(5), method=method, rtol=1e-6, atol=1e-6,
                    t_eval=np.arange(1, P.size + 1), max_step=1.0)
    err = np.max(np.abs(T_AMB + sol.y.sum(axis=0) - Tj_ref))
    print(f"{method:12s}: {1e3 * (time.perf_counter() - t0):7.1f} ms, {sol.nfev:6d} evaluations,"
          f" max error {err:.1e} K")
```

```
 ZOH exact   :     5.7 ms, Tj(600 s) = 98.784 C
RK45         : 48163.4 ms, 1279100 evaluations, max error 1.0e-03 K
LSODA        :  1255.8 ms,  48136 evaluations, max error 1.4e-05 K
BDF          :  6552.1 ms,  27076 evaluations, max error 5.4e-05 K
Radau        :  9006.8 ms,  60342 evaluations, max error 7.3e-06 K
```

Listing 30.4 — Five ways to solve the same stiff linear transient (timings from one run).

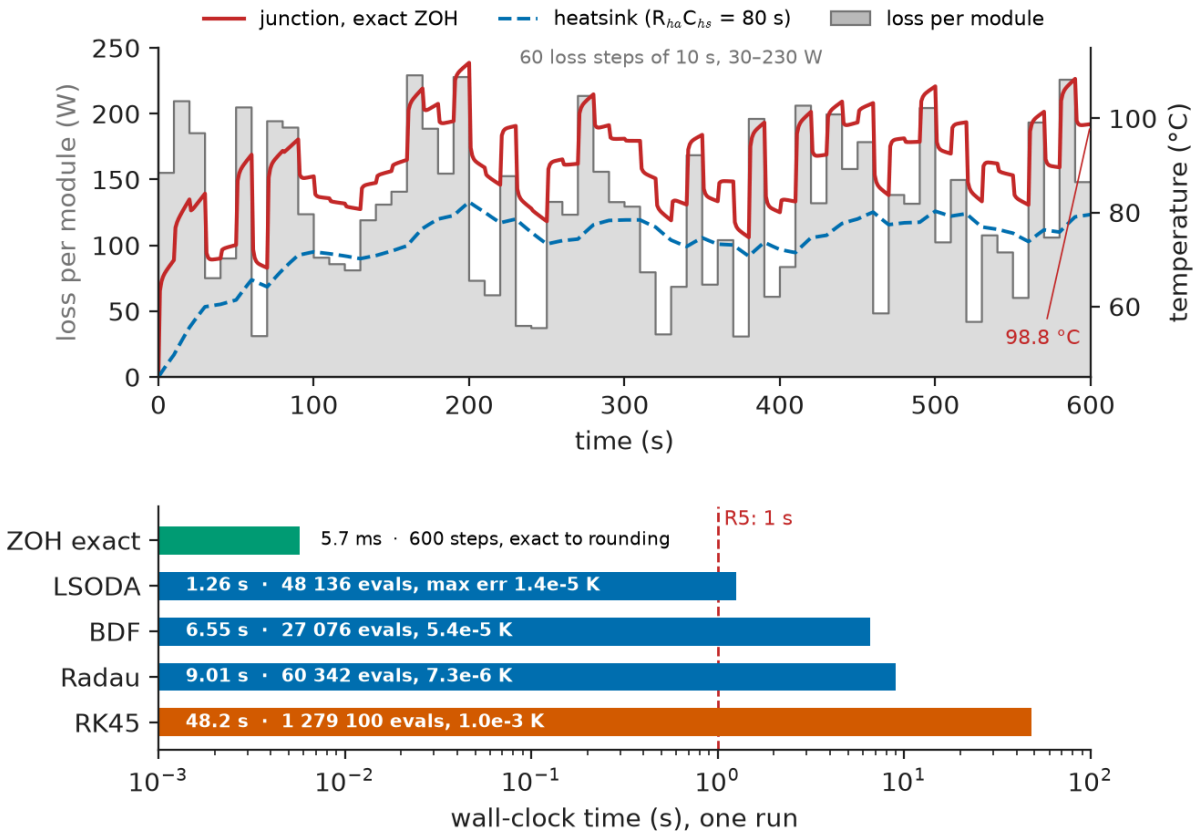


Figure 30.4 — Junction temperature of the hottest module over the 600 s loss profile, and the cost of computing it with five methods.

The explicit RK45 takes 1.28 million evaluations and 48 s, because its step is held to about 2.8 ms by stability against the 1 ms time constant, not by accuracy; this is stiffness in its purest form (*The Mathematics Universe*, Ch. 22 §22.9). The implicit methods take 1–9 s, mostly Python call overhead. The exact discretisation takes 5.7 ms, is exact to rounding and meets R5 with a margin of 175 (Figure 30.4); a 30-minute cycle at 1 ms resolution for rainflow lifetime analysis takes under 20 s this way.

> **Status of the principle: Exact discretisation of linear systems with piecewise-constant input**
> *Derived.* For $\dot{x} = Ax + Bu$ with u constant over a step, $x_{k+1} = e^{A\Delta t}x_k + A^{-1}(e^{A\Delta t} - I)Bu_k$ holds exactly for any step length; it applies to Foster networks, RC thermal ladders and linear circuit models, and fails as soon as a parameter depends on the state (temperature-dependent loss), where an implicit solver or an iteration per step is needed.

Table 30.6 — Wall-clock budget of the Revision A campaign (measured, shared 8-core workstation).

Step	Budget	Measured	Margin
Sweep, 420 cases, 2 mm, 8 workers (Listing 30.2)	300 s	16.0 s	94.7 %
Optimisation, 10 SLSQP runs, 4 mm (Listing 30.3)	120 s	10.2 s	91.5 %
Verification, 3 grids to 0.5 mm (Listing 30.5)	30 s	4.0 s	86.7 %
Validation and calibration (Listing 30.6)	30 s	3.1 s	89.7 %
600 s transient (Listing 30.4)	1 s	0.006 s	99.4 %

The margin is what pays for the 1 mm grid, the tightened constraint and a second fan speed in Revision B without a cluster.

30.8 Parallel processing

Cases in a sweep are independent, so the sweep is embarrassingly parallel and a process pool is the right tool (Python threads would serialise on the interpreter lock; levels of parallelism and the two scaling laws are *Electronics and Computing*, Ch. 24 §24.5). Listing 30.2 measured 72.9 s on one worker and 16.0 s on eight, a speed-up of 4.56.

Worked example — what limits the speed-up. With Amdahl's law, $1/S = s + (1 - s)/N$, the measured $S = 4.56$ on $N = 8$ gives an effective serial fraction $s = (1/4.56 - 1/8)/(1 - 1/8) = 0.108$. Nothing in the sweep is really 10.8 % serial; the figure lumps together what did not scale: worker start-up and SciPy import (about 1 s), the turbo clock dropping under full load, memory bandwidth shared by eight factorisations, and other jobs on the shared machine (the one-worker run took 174 ms per case against 72 ms in the quiet profile). On that basis a 32-core machine would give $1/(0.108 + 0.892/32) = 7.4$, not 32.

Four rules keep parallel campaigns efficient and correct:

- **Parallelise at one level.** Single-threaded BLAS in each worker (`OMP_NUM_THREADS=1`), or eight workers start 64 threads.
- **Chunk small cases.** At 70 ms per case, `chunksize=8` amortises inter-process overhead.
- **Make results independent of the worker count.** Results in case order, one seeded random stream per case; the result hash proves 1 and 8 workers agree bit for bit.
- **Do not parallelise what should be restructured.** Factorising once (27 times) and the exact transient (about 8000 times) beat anything eight cores can give.

> **Status of the principle: Speed-up of an embarrassingly parallel sweep**
> *Approximate.* Independent cases scale almost linearly only while per-case work dominates start-up and contention; the measured speed-up, read as an effective serial fraction, is the number to plan with.

30.9 Verification of numerical results

Verification asks whether the equations were solved correctly; validation (§30.10) asks whether they were the right equations. The distinction and the overall workflow are Chapter 3; the theory of convergence and of verifying it in practice is *The Mathematics Universe*, Ch. 22 §22.11. Listing 30.5 applies three checks to the Revision A design.

```
 import numpy as np
import hs_model as hs

design = dict(N=18, H=0.02336, tb=0.004)        # optimiser result, Listing 30.3

# 1. Conservation: heat leaving through the fins must equal the 600 W injected.
r = hs.solve(**design)
print(f"energy balance error  : {r['heat_out'] - 600.0:+.1e} W")

# 2. Exact limit: one source covering the whole base, no air temperature rise.
saved = hs.CENTRES, hs.FX, hs.FY, hs.CP
hs.CENTRES, hs.FX, hs.FY, hs.CP = [(hs.W / 2, hs.L / 2)], hs.W, hs.L, 1e12
r = hs.solve(P=(600.0,), **design)
exact = hs.T_AMB + 600.0 / (r["h_eff"] * hs.W * hs.L)
print(f"uniform-flux limit    : {r['Tmax'] - exact:+.1e} K from exact")
hs.CENTRES, hs.FX, hs.FY, hs.CP = saved

# 3. Grid convergence of the hottest module (refinement ratio 2).
f = [max(hs.solve(dx=dx, **design)["T_mod"]) for dx in (0.002, 0.001, 0.0005)]
p = np.log((f[0] - f[1]) / (f[1] - f[2])) / np.log(2)
gci = 1.25 * abs(f[2] - f[1]) / (2**p - 1)
print("hottest module, C     :", np.round(f, 4))
print(f"observed order p = {p:.2f}, GCI(fine) = {gci * 1000:.1f} mK")
```

```
 energy balance error  : +1.6e-10 W
uniform-flux limit    : +1.3e-08 K from exact
hottest module, C     : [99.5405 99.5298 99.5271]
observed order p = 2.00, GCI(fine) = 1.1 mK
```

Listing 30.5 — Three verification checks: global conservation, an exact limiting case and a grid-convergence study with a grid convergence index.

The conservation check catches sign errors and missing boundary terms. The exact limit (a source covering the whole base with no air heating gives $T = T_{amb} + P/(h_{eff} W L)$ everywhere) checks the plate-to-fin coupling. The grid study measures discretisation error directly: the observed order is 2.00, as a second-order five-point scheme should give, and the grid convergence index (Roache: 1.25 times the Richardson error estimate) is 1.1 mK on the finest grid. On the 2 mm production grid the difference from the extrapolated value is about 14 mK, well inside R2.

> **Status of the principle: Grid-convergence index**
> *Derived.* Richardson extrapolation from three refined grids estimates the discretisation error when all three are in the asymptotic range, which the observed order confirms; otherwise the estimate is meaningless.

Why the 4 mm grid misled the optimiser

The same study extended to coarser grids is not monotonic: the hottest module reads 99.50 °C at 8 mm, 99.00 °C at 4 mm, 99.54 °C at 2 mm and 99.53 °C at 1 mm. The 4 mm grid is outside the asymptotic range, and the reason is geometric, not numerical. The footprint edges fall exactly on cell centres at 4 mm, and floating-point comparison decides whether a row of cells is in or out: the footprint of M1 becomes 416 cells (6656 mm²) and those of M2 and M3 390 cells (6240 mm²) instead of 6000 mm². The power is right, but spread over 4–11 % more area, so temperatures read low. Conservation cannot see this; only a grid study that includes the grid actually used can (Figure 30.5).

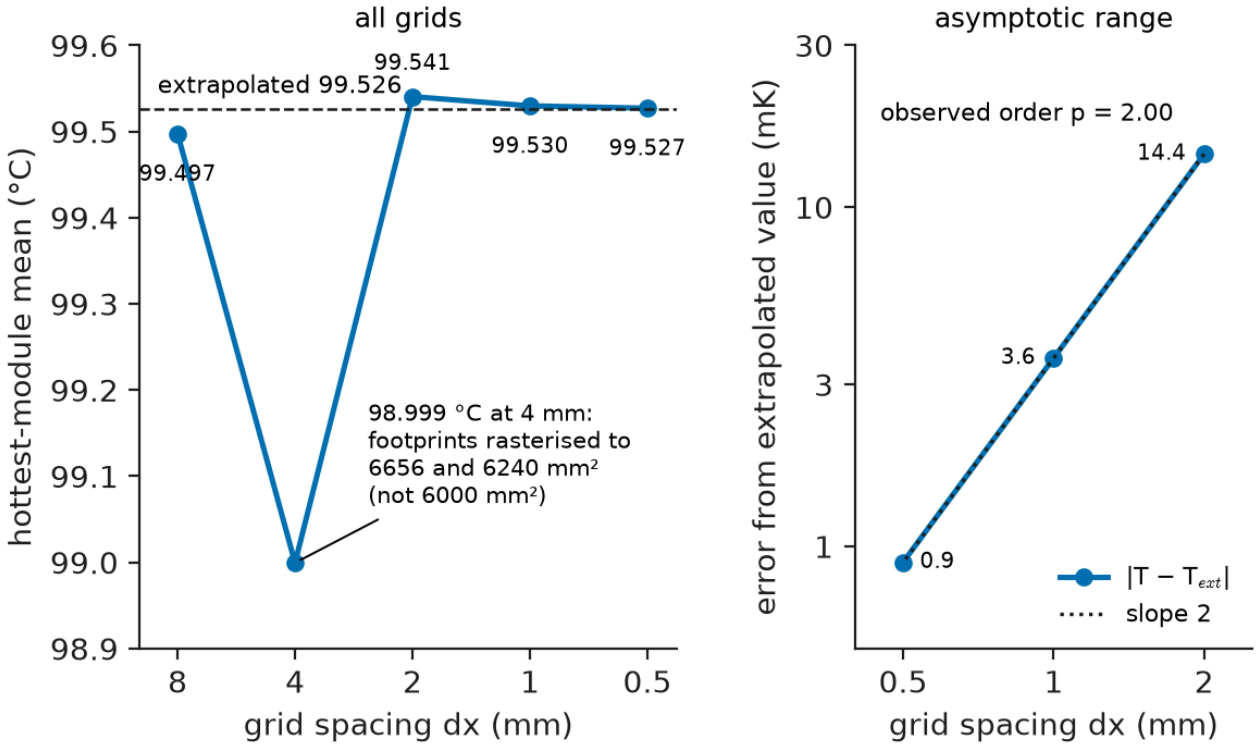


Figure 30.5 — Grid convergence of the hottest-module temperature: asymptotic second-order convergence on fine grids and footprint rasterisation error on coarse ones.

A fourth check was added after the post-mortem: **regression tests** that rerun these cases and a few reference designs on every change to `hs_model.py` and fail if a reference temperature moves by more than 0.01 K without a recorded reason (test machinery in Chapter 29).

30.10 Model validation

Plan before data

The metric and acceptance limit are written down before the comparison (R3). Three type-T thermocouples are epoxied into 1 mm holes in the base under the centre of each footprint. The expanded uncertainty of each reading ($k = 2$) is 0.6 K: 0.5 K for the thermocouple class and cold junction, 0.2 K for the DAQ, and 0.3 K for the position of the junction in the hole, combined in quadrature (uncertainty combination as in Chapter 7). The acceptance limit is that uncertainty plus a 1.0 K allowance for model-form error: ±1.6 K. Three load points (80, 140, 200 W per module) are measured in a 45 °C chamber; two calibrate, one is held back for validation.

```
 import numpy as np
from scipy.optimize import minimize_scalar
import hs_model as hs

D = dict(N=18, H=0.02336, tb=0.004, dx=0.002)          # Revision A prototype
MEAS = {80: [69.1, 72.1, 73.0], 140: [87.8, 93.5, 94.2], 200: [105.8, 113.5, 115.4]}
U_MEAS = 0.6                                            # K, expanded (k = 2): TC + DAQ +
placement

def predict(P, bypass):
    return np.array(hs.solve(P=(P,) * 3, bypass=bypass, **D)["T_ctr"])

def rise_error(P, bypass):                              # error relative to the measured rise
    m = np.array(MEAS[P])
    return (predict(P, bypass) - m) / (m - hs.T_AMB)

for P in MEAS:
    print(f"{P:3d} W, bypass 0  : error in rise {np.round(100 * rise_error(P, 0.0), 1)} %")

# Calibrate one parameter on 80 W and 200 W; keep 140 W for validation only.
sse = lambda b: sum(np.sum((predict(P, b) - MEAS[P]) ** 2) for P in (80, 200))
b = minimize_scalar(sse, bounds=(0.0, 0.6), method="bounded").x
print(f"calibrated bypass fraction = {b:.3f}")
res = predict(140, b) - MEAS[140]
print(f"140 W validation residuals {np.round(res, 2)} K, limit +/-{U_MEAS + 1.0:.1f} K")
```

```
 80 W, bypass 0  : error in rise [-13.2 -13.  -14.3] %
140 W, bypass 0  : error in rise [-14.5 -15.  -14.7] %
200 W, bypass 0  : error in rise [-14.  -14.  -14.8] %
calibrated bypass fraction = 0.198
140 W validation residuals [-0.22 -0.48 -0.16] K, limit +/-1.6 K
```

Listing 30.6 — Validation of the heatsink model against prototype thermocouples, with one-parameter calibration on two load points and validation on the third.

Reading the residuals

The uncalibrated model under-predicts every temperature rise by 13–15 %. The pattern is diagnostic: a constant *fractional* error at all loads and all modules points to a wrong heat-transfer coefficient or flow, not to a contact fault (one location) or a missing radiation term (temperature-dependent). Smoke on the unshrouded prototype showed air spilling over the fin tips. A single bypass fraction calibrated on the 80 W and 200 W points comes out at 19.8 %, and on the held-back 140 W point the residuals are −0.22, −0.48 and −0.16 K, inside ±1.6 K. With the bypass included, the predicted hottest junction of Revision A at 200 W is 135.5 °C, not 125.0 °C: the prototype's 10 K is explained. Systematic calibration with uncertainty is Chapter 33.

> **Status of the principle: Calibrate on one set, validate on another**
> *Empirical.* A fitted parameter reproduces the data it was fitted to; only data not used in the fit, under the conditions of use, give evidence of predictive validity.

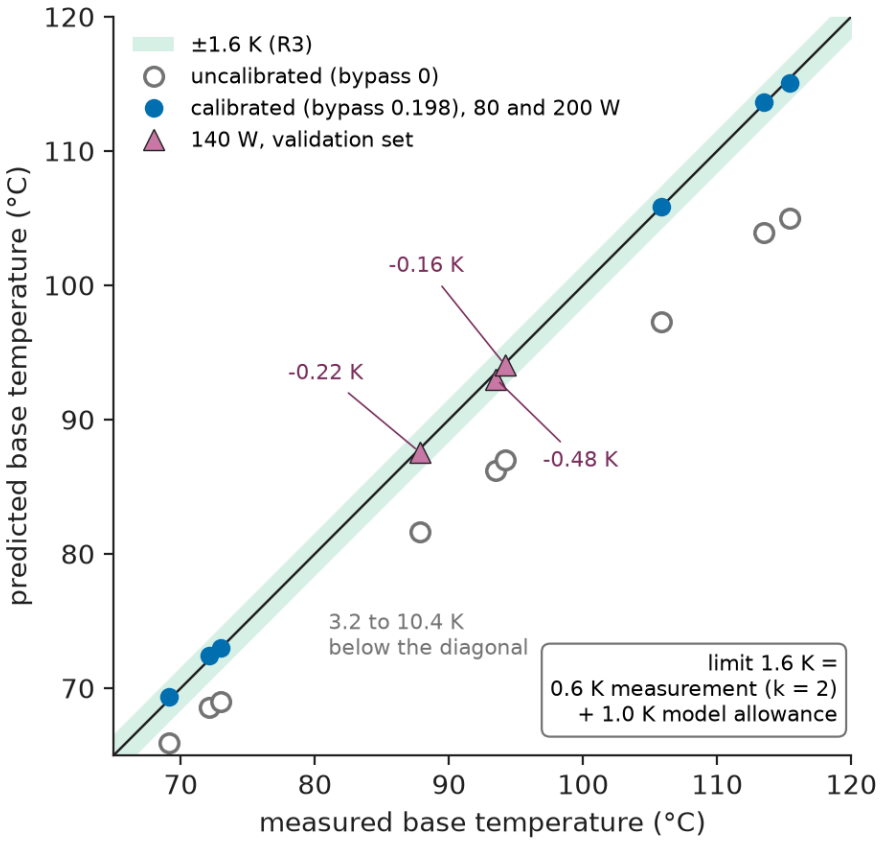


Figure 30.6 — Validation of the heatsink model: predicted against measured base temperatures under the three modules at 80, 140 and 200 W per module.

Figure 30.6 is the validation record; Table 30.7 closes the requirements of Table 30.1.

Table 30.7 — Verification of the computation against its requirements (Revision A).

Req.	Test	Limit	Result	Pass/fail
R1	Hottest junction, calibrated model, 2 mm grid	≤ 125 °C with ≥ 3 K margin	135.5 °C	**Fail**: design changed (§30.12)
R2	Grid convergence, production 2 mm grid	≤ 0.1 K	0.014 K (GCI 1.1 mK on 0.5 mm)	Pass
R2	Same, optimiser 4 mm grid	≤ 0.1 K	0.53 K	**Fail**: grid no longer used for decisions
R3	Validation residuals, 140 W	within ±1.6 K	−0.48 to −0.16 K	Pass (after calibration)
R4	Sweep / optimisation wall-clock	≤ 300 s / ≤ 120 s	16.0 s / 10.2 s	Pass
R5	600 s transient time and error	≤ 1 s, ≤ 0.01 K	5.7 ms, exact	Pass
R6	Manifest present and result hash reproduced on rerun	100 %	100 % (same platform)	Pass
R7	Out-of-domain cases recorded	reported	45 of 420 (10.7 %)	Pass

30.11 Scientific data management

What has to be kept

The third fault in the opening is a data-management failure. A result is reproducible when five things are kept with it: the **inputs** (or their hashes and archive location), the **code** (a commit hash, or at least a hash of the model file), the **environment** (library versions; a lock file or container digest), the **outputs** with a content hash, and the **status** of each case. The manifest of Listing 30.2 is the smallest useful version: model hash `4792b5780f66883d` and result hash `e73d62e3cda02e32`, both recomputable by anyone; a one-character model edit changes the first and flags every older result.

Formats

Table 30.8 — Storage formats for engineering computation results.

Format	Strengths	Weaknesses	Use in the reference project
CSV	Universal, readable, diffable	No types or units, no metadata, slow and large for fields	Summary tables for the report
NumPy `.npy` / `.npz`	Exact binary, typed, fast, structured arrays	Python-centric, little metadata	Sweep table, temperature fields
HDF5 (h5py) or netCDF	Hierarchical, chunked, compressed, attributes for units and provenance	Library dependency, corruption risk on interrupted writes	Large field archives, transient campaigns
Parquet (pyarrow)	Columnar, compressed, fast queries, typed	Not for N-D fields	Fleets of cases, cross-campaign queries

Formats and their engineering use are compared in *Electronics and Computing*, Ch. 22 §22.9 and Ch. 24 §24.3. Units go into field names or attributes (`Tj_C` , `mass_kg`).

Retention and size

Worked example — storage budget. The 420-case sweep table is 420 rows × 48 bytes = 20 kB. Keeping every 2 mm temperature field as float64 is $420 \times 12\,000 \times 8 = 40$ MB per sweep; at 1 mm, 161 MB. Thirty sweeps a year would store 1.2–4.8 GB of fields that can each be regenerated in 72 ms. Policy: keep inputs, manifests and scalars forever (kilobytes); keep fields only for designs that reached a review or prototype; regenerate the rest, which works only if code and environment are archived. A production design keeps its whole computation for the product's 10–15 year support life, the long-term question of Chapter 31.

> **Status of the principle: Results without provenance are not results**
> *Empirical.* A number that cannot be traced to its inputs, code and environment cannot be reproduced, audited or corrected; the FAIR data principles and ISO 9001 design records encode the same practice.

30.12 Failure analysis of the computation

Table 30.9 collects the failure modes of the computation.

Table 30.9 — FMEA of the heatsink design computation.

Failure mode	Effect	Cause	Detection	Mitigation
Coarse-grid footprint rasterisation	Temperatures 0.5–1.0 K low at the constraint	4 mm grid used for decisions; footprint edges on cell centres	Grid study including the decision grid (§30.9)	Decide on the 2 mm grid; align footprints to grid lines; margin for remaining error
Optimiser at an optimistic boundary	Delivered design violates the limit	Optimum lies on the active constraint by construction	Re-check of optimum on production grid with calibrated model	Tightened limit (budget margin); re-verification step in the pipeline
Model-form error (bypass flow)	10 K under-prediction on the prototype	Fan curve assumed all flow passes the fins	Validation against prototype (§30.10)	Calibrated bypass; shroud in Revision B
Correlation used outside its range	Silent wrong h	Transitional Reynolds numbers	Validity check raises; status recorded	Report out-of-domain fraction; CFD for that region if needed
Silent non-convergence of an external solver	Plausible but wrong result file	Output written regardless of convergence	Parse solver's convergence report	Status per case; timeouts
Unrecorded model edit	Results not reproducible	Script edited between runs; no manifest	Model hash in manifest differs	Manifest, version control, regression tests

Revision B

Revision A fails R1, and the redesign follows from the numbers. A sheet-metal shroud removes most of the bypass (residual to be measured and recalibrated); the optimiser runs on the 2 mm grid against 122 °C, the limit tightened by the accuracy budget; footprints are aligned to grid lines so the 4 mm grid can still be used for exploration; a second fan speed is added to the validation to test the bypass for flow dependence; and Listings 30.5 and 30.6 become regression tests.

What remains open

Open question	Where it is answered
How are the engineering tools around this computation (repository layout, builds, CI, automated tests) organised?	Chapter 29
How should a computation like this be structured as a maintainable tool with a domain model, file format and API?	Chapter 31
Can a fast surrogate replace the field model inside the optimiser, and with what error?	Chapter 32
How are model parameters such as the bypass fraction calibrated with uncertainty, and the model kept in step with the running product?	Chapter 33
What does a complete simulation tool with benchmarks, verification suite and documentation look like?	Chapter 39
The mathematics of the solvers, convergence and optimisation methods used here	*The Mathematics Universe*, Ch. 22–24

Next: Chapter 31 turns computations like this one into engineering software that other people can use, extend and trust for years: domain models, project files, plug-ins, APIs and GUIs.

Chapter 31 — Engineering Software Architecture

Part VIII · Engineering Software and Computational Tools

A building-services consultancy sizes the low-voltage cables of hospitals, schools and data centres with an in-house tool, FeederCheck. It began in 2019 as a 400-line script by one engineer; by 2023 it is 38 000 lines of Python with a desktop GUI, used by 60 engineers on about 2000 project files. That year an engineer reopens a 2020 hospital project in the current release to add one circuit. The tool re-sizes everything and the revised cable schedule goes out with the kitchen sub-board feeder reduced from 185 mm² to 150 mm². Nothing in the electrical design had changed. In 2020 the file format stored one ambient temperature for the whole project (40 °C, a plant room); the 2022 format moved it to each circuit, and the new loader filled the missing per-circuit value with a default of 30 °C. At its real 40 °C the 150 mm² cable is good for 280 A against a design current of 300 A. A checking engineer's hand calculation caught it; a search of the archive then found eleven more projects whose results had silently changed on reopening. Every line of the sizing arithmetic was correct. What failed was architecture: a domain model that allowed a hidden default, a file format without explicit migrations, calculation code tangled into GUI event handlers so that nobody could test it without clicking, and no test that opened an old file.

> **How is an engineering tool divided into a domain model, calculation services, persistence, plug-ins and user interfaces so that each can change without breaking the others?**
> **How are project files, interfaces and tests designed so that a result computed today is reproduced, exchanged and still trusted ten years from now?**

Concept	Engineering destination
Modular software design	Calculation tools, test executives, simulation front ends and lab software that teams can extend in parallel
Domain models	Cables, circuits, devices, loads, components and test points represented with units and invariants
File-based project formats	Design projects that open identically in every later release, diff in version control and migrate explicitly
Plugin versus built-in architecture	National standards, manufacturer libraries, instrument drivers and solvers added without touching the core
APIs and interfaces	Scripting, batch runs, CI checks and integration with BIM, PLM and test systems
GUI design	Responsive editors that validate input, show what governs a result and can undo every change
Data interoperability	Load schedules from spreadsheets, results to BIM models, device data from manufacturers
Version control	Traceable releases of the tool and diffable histories of each project
Automated testing	Golden cases from standards and textbooks, property tests, archived-file tests, architecture rules
Long-term maintainability	Ten-year support of tools whose results end up in buildings, vehicles and certified products

Roadmap. The tool and its requirements (31.1) → modules and the dependency rule (31.2) → the domain model at the centre (31.3) → project files and migrations (31.4) → plug-ins for standards (31.5) → the API every front end uses (31.6) → the GUI on top of it (31.7) → data exchange with other tools (31.8) → version control of code and projects (31.9) → automated tests (31.10) → keeping it alive for a decade, and what fails (31.11).

31.1 The reference tool and its requirements

> **Boundary.**
> The engineering content of FeederCheck (current-carrying capacity and its correction factors, voltage drop, protection and earthing) is *Electrical Energy and Machines*, Ch. 30 §30.5–30.8; this chapter uses its Table 30.3 and worked examples as test data. Modules, interfaces, coupling and cohesion as programming ideas are *Electronics and Computing*, Ch. 20 §20.7; Python itself is Ch. 22; Git and testing practice are Ch. 24 §24.9–24.10. System architecture in general is Chapter 2, and the workflow tooling (repositories, builds, CI) is Chapter 29. This chapter is the internal architecture of one engineering tool.

FeederCheck takes an installation (boards, circuits, loads, installation methods, ambient and grouping conditions), applies a cable standard, and returns for each circuit the smallest cable that meets current-carrying capacity and voltage drop, with what governed the choice. Its users are design engineers (GUI), a

checking engineer (reports), the BIM team (data exchange) and the automation group (scripts that size 3000 circuits of a data centre overnight). Table 31.1 is the requirements set written after the 2023 incident.

Table 31.1 — Requirements for FeederCheck version 4.

ID	Requirement	Target	Verification method
R1	Calculation correctness	Golden set of 120 hand-checked cases: same cable size, I_z and voltage drop within 0.5 %	Golden tests, §31.10
R2	Backward compatibility	Every file saved by any release since 1.0 opens with all values explicit and gives the result it gave when saved, or a reported difference	Archive-corpus test, §31.10
R3	Extensibility	A new standard (BS 7671, NEC) added as a plug-in, no change to core code	Architecture test; plug-in build
R4	API parity	Every GUI action available from the Python API and CLI; no calculation code in the GUI layer	Code review rule and architecture test
R5	Responsiveness	One edit to visible result ≤ 100 ms; full recalculation of a 500-circuit project ≤ 1 s; open ≤ 2 s	Timed benchmark in CI, §31.7
R6	Interoperability	Load schedules imported from CSV/XLSX; results exported to CSV, JSON and the BIM model; units explicit in every exchanged field	Round-trip tests, §31.8
R7	Test depth	≥ 90 % branch coverage of domain and calculation code; suite ≤ 2 min on three operating systems	CI report
R8	Maintainability	Supported for 10 years; direct dependencies ≤ 8; deprecations announced two minor releases ahead	Release checklist, §31.11

31.2 Modular software design

The dependency rule

FeederCheck version 4 is arranged as concentric layers (Figure 31.1), often called *ports and adapters* or *clean architecture*. At the centre is the **domain model**: what an installation, a circuit and a cable are, with their invariants. Around it sit the **calculation services** (sizing, voltage drop, protection checks) and the **ports**, the interfaces the core needs from the outside world (a standard's tables, a repository that loads and saves projects). Outermost are the **adapters**: the GUI, the CLI, the JSON file format, the CSV importer, the IFC exporter, and the plug-ins that implement the standards. The rule is that source-code dependencies point only inward. The domain model imports nothing from the project; the sizing service imports the domain and the standard *interface*, never a particular standard; the GUI imports the API, never the sizing internals.

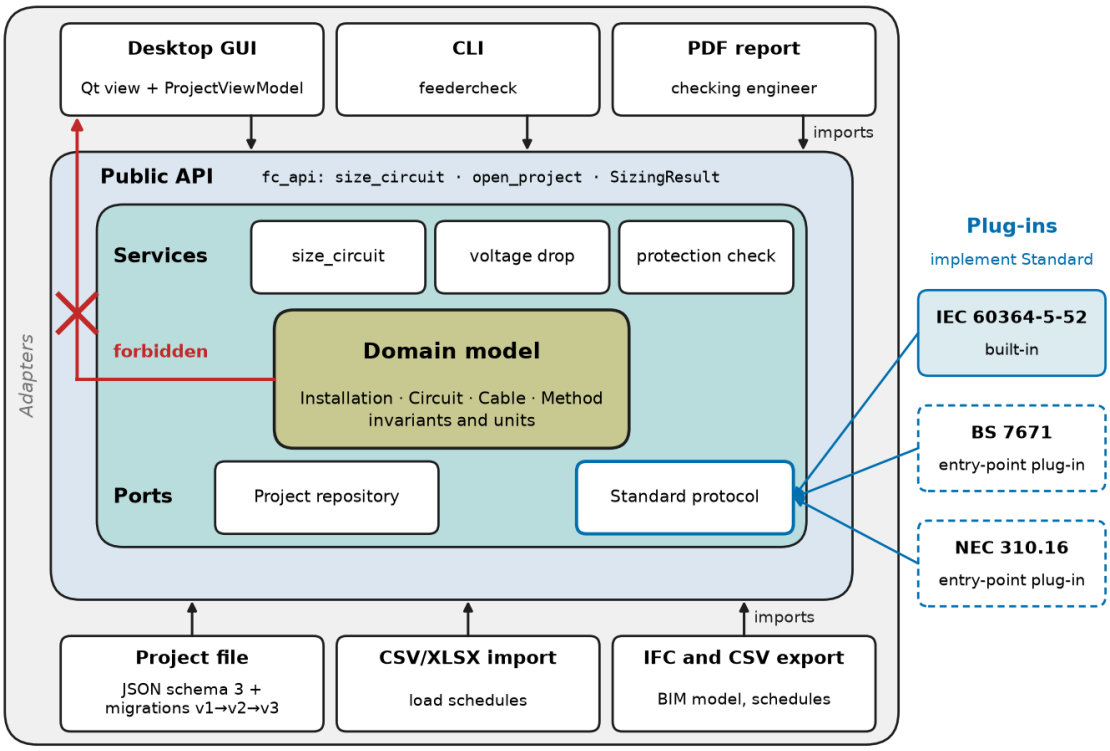


Figure 31.1 — Layered architecture of FeederCheck version 4: domain model at the centre, services and ports around it, adapters and plug-ins outside, with all source dependencies pointing inward.

Table 31.2 lists the modules of the reference implementation in this chapter, with what each may import. The listings are short versions of the real modules, small enough to read, and they run as shown.

Table 31.2 — Modules of FeederCheck and their allowed dependencies.

Module	Responsibility	May import	Listing
`fc_domain`	Entities, value objects, invariants	Standard library only	31.1
`fc_project`	Project file: schema, migrations, strict loading	`fc_domain`	31.2
`fc_standards`	Standard protocol, registry, built-in IEC plug-in	`fc_domain`	31.3
`fc_api`	Sizing service, public API, CLI	`fc_domain` , `fc_project` , `fc_standards`	31.4
`fc_viewmodel`	State the GUI binds to: edits, undo, results, errors	`fc_api` , `fc_project`	31.5
GUI views (Qt)	Widgets only	`fc_viewmodel`	—
`test_fc`	Golden, property, file and architecture tests	everything	31.6

> **Status of the principle: Dependencies point toward stability**
> *Empirical.* Code that many modules depend on should change least, so the most stable knowledge (what a circuit is) sits at the centre and the most volatile (widgets, file layouts, a standard's next edition) at the edge; decades of software practice support it, but it is a heuristic whose payoff depends on predicting correctly what will change.

Where the old tool broke the rule

In version 2 the GUI's "Recalculate" handler read values straight from table cells, converted them with `float()`, looked up the IEC tables in a module-level dictionary and wrote results back into the cells. Calculation, persistence and presentation were one function. Consequences: the calculation could be tested only by driving the GUI; the automation group copied the arithmetic into their batch script, where it drifted; and the loader, written to feed the table cells, supplied defaults for anything missing because a blank cell crashed the handler. Each of these is a coupling that the layered design makes impossible by construction.

31.3 Domain models

What the domain model must say

A domain model is the code's statement of what the engineering objects are. It must carry four things that a spreadsheet leaves implicit: **identity** (circuit "SB-KIT" is the same circuit after its current changes), **units** (in names, `design_current_A`, or in types), **invariants** (a power factor is between 0.1 and 1; a length is positive), and **ownership** (an installation owns its circuits; a cable is chosen by the sizing service, not typed by the user). Listing 31.1 is the FeederCheck domain in 42 lines.

```
 """fc_domain.py -- domain model of FeederCheck: no GUI, no file format, no standard."""
from dataclasses import dataclass, field
from enum import Enum

class Method(Enum):                       # installation reference methods (IEC 60364-5-52)
    TRAY = "E"
    CONDUIT = "B1"
    BURIED = "D1"

@dataclass(frozen=True)
class Cable:                              # value object: equal if all fields are equal
    material: str                         # "Cu" or "Al"
    insulation: str                       # "XLPE" or "PVC"
    csa_mm2: float

@dataclass
class Circuit:                            # entity: identified by its tag
    tag: str
    design_current_A: float
    length_m: float
    power_factor: float
    method: Method
    ambient_C: float                      # always explicit: no hidden project default
    grouped_with: int = 1                 # number of loaded circuits in the group
    vd_limit_pct: float = 5.0
    cable: Cable | None = None            # chosen by the sizing service, not by the GUI

    def __post_init__(self):
        checks = {"design_current_A": 0 < self.design_current_A <= 4000,
                  "length_m": 0 < self.length_m <= 2000,
                  "power_factor": 0.1 <= self.power_factor <= 1.0,
                  "ambient_C": -20 <= self.ambient_C <= 70,
                  "grouped_with": 1 <= self.grouped_with <= 20}
        bad = [name for name, ok in checks.items() if not ok]
        if bad:
            raise ValueError(f"circuit {self.tag}: out of range: {', '.join(bad)}")

@dataclass
class Installation:                       # aggregate root: owns its circuits
    name: str
    nominal_V: float = 400.0
    circuits: list[Circuit] = field(default_factory=list)
```

Listing 31.1 — The domain model: an entity with invariants, a value object, an aggregate root, and no knowledge of files, GUIs or standards.

Design decisions in the listing

The single most important line is the one with no default: `ambient_C: float` must be supplied every time a `Circuit` is created, by a user, an importer or a migration. Python dataclasses require non-default fields before defaulted ones, so the language itself enforces that it is listed among the mandatory inputs. Defaults

are kept only where a wrong default is harmless and visible (one circuit in a group, a 5 % voltage-drop limit that appears in every report). `Cable` is frozen because a cable of 185 mm² copper XLPE is a value, not a thing with a history. The validation ranges are wide plausibility limits, not engineering limits: 4000 A and 2 km catch typing errors (a current entered in mA, a length in mm), while the standard decides what is acceptable.

> **Status of the principle: Make illegal states unrepresentable**
> *Empirical.* Encoding required fields, units and invariants in the types and constructors of the domain model moves a class of errors from run time in the field to construction time in the developer's test; it cannot catch values that are legal but wrong, which remain the job of tests and review.

Worked example — what the hidden default cost. The kitchen sub-board feeder carries $I_B = 300$ A at a power factor of 0.85 over 120 m on a tray shared with three other loaded circuits. With the real 40 °C ambient, the correction is $0.91 \times 0.77 = 0.7007$; the required tabulated capacity is $300/0.7007 = 428$ A, so the first size that qualifies is 185 mm² (456 A tabulated, 319.5 A corrected), the result of *Electrical Energy and Machines*, Ch. 30 §30.5. With the loader's 30 °C default the correction is 0.77, the requirement 390 A, and 150 mm² (399 A tabulated) is chosen. At the actual 40 °C that cable's capacity is $399 \times 0.7007 = 279.6$ A, 6.8 % below the design current: the conductor would run above 90 °C at full load. Figure 31.2 shows the whole size-against-current staircase at both ambients; the incident sits in one of the current bands where the two staircases differ.

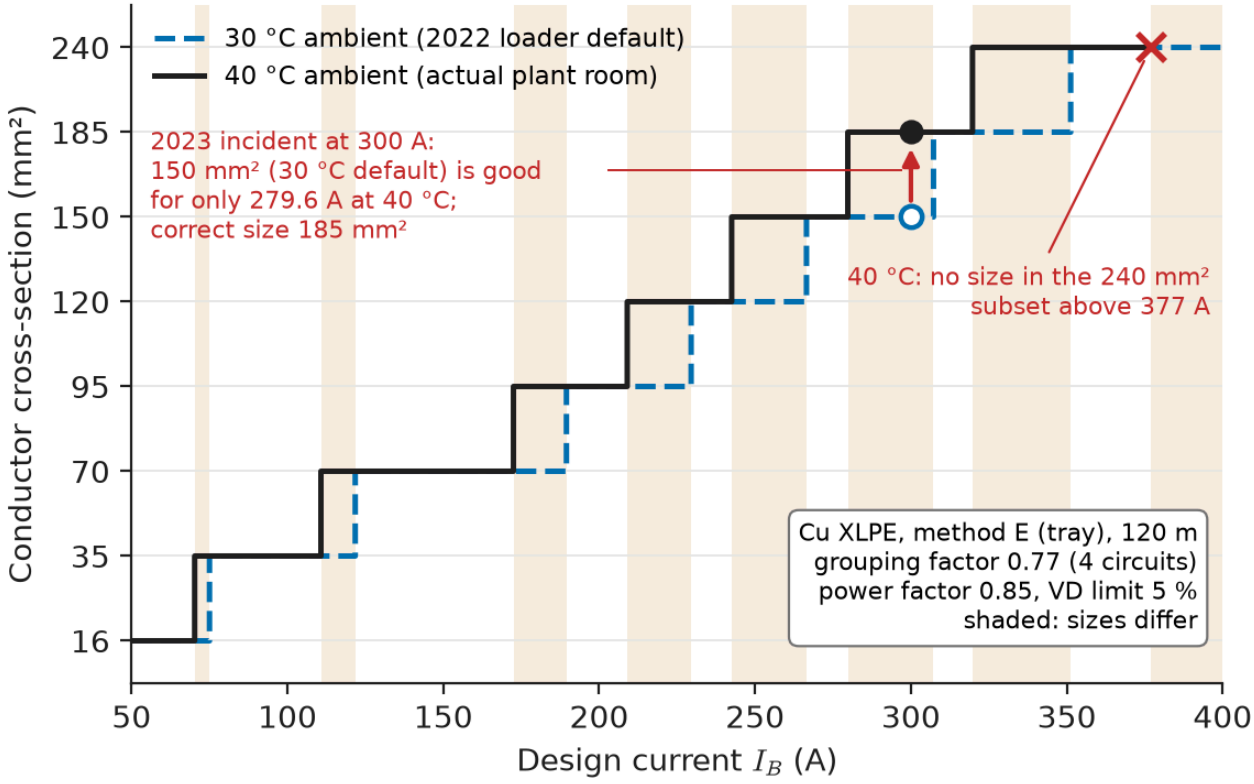


Figure 31.2 — Cable size chosen by FeederCheck against design current for a 120 m feeder on a shared tray, at 30 C and 40 C ambient.

31.4 File-based project formats

Requirements on a project file

A project file outlives the program that wrote it. For FeederCheck it must be: **self-describing** (it says which schema version it is and which standard and edition it was designed to); **complete** (every input explicit, nothing supplied by the program at load time); **inputs only** (results are derived and recomputed, so they can never disagree with inputs, and they never cause merge conflicts); **text and canonical** (sorted keys, fixed indentation, so that version control shows meaningful differences, §31.9); and **migratable** (every older schema has a tested, explicit conversion to the next).

Table 31.3 — Candidate project-file formats.

Format	Diffable	Partial reads, large projects	Schema evolution	Tooling outside the program	Choice
JSON, canonical (sorted keys, indent)	Yes	Whole-file read; fine to about 10 MB	Explicit migrations	Every language; JSON Schema validators	**Chosen**
SQLite database file	No (binary)	Yes, indexed	SQL migrations	Good	For tools with 10^5 objects or more
XML with XSD	Yes, noisy	Whole-file or streaming	XSLT or code	Good, verbose	Where an industry schema demands it
Zip container (JSON plus attachments)	Partly	Per entry	As JSON	Good	When drawings or PDFs must travel with the project
Python pickle	No	No	Breaks on any class change	None, and unsafe to load from others	Never for project files

Migrations as code

Listing 31.2 is the project-file module. Each schema change is a small function that turns a version-n dictionary into a version-$n+1$ dictionary, and loading applies the chain until the current version is reached (Figure 31.3). The 2020 file goes through two migrations; the one that matters, `_v1_to_v2`, copies the project's ambient temperature into every circuit, which is what the 2022 loader should have done instead of supplying a default.

```
 """fc_project.py -- the project file: versioned JSON, explicit migrations, strict loading."""
import json
from dataclasses import asdict
from fc_domain import Circuit, Installation, Method

SCHEMA = 3

def _v1_to_v2(d):                          # v1 (2019-2021): one ambient for the whole project
    ambient = d.pop("ambient_C")
    for c in d["circuits"]:
        c["ambient_C"] = ambient           # make the old implicit value explicit per circuit
    d["schema"] = 2
    return d

def _v2_to_v3(d):                          # v3 (2024): the standard and its edition are recorded
    d["standard"] = "IEC60364-5-52:2009"   # the only standard versions 1 and 2 knew
    d["schema"] = 3
    return d

MIGRATIONS = {1: _v1_to_v2, 2: _v2_to_v3}

def loads(text):
    d = json.loads(text)
    version = d.get("schema", 1)
    if version > SCHEMA:
        raise ValueError(f"file schema {version} is newer than this program ({SCHEMA})")
    while version < SCHEMA:
        migrate = MIGRATIONS[version]
        d = migrate(d)
        version = d["schema"]
    circuits = [Circuit(**{**c, "method": Method(c["method"])}) for c in d["circuits"]]
    return Installation(d["name"], d["nominal_V"], circuits), d["standard"]

def dumps(inst, standard):
    circuits = [{k: v for k, v in asdict(c).items() if k != "cable"} for c in inst.circuits]
    for c, src in zip(circuits, inst.circuits):
        c["method"] = src.method.value    # results (the cable) are derived, never stored
    return json.dumps({"schema": SCHEMA, "name": inst.name, "nominal_V": inst.nominal_V,
                       "standard": standard, "circuits": circuits}, indent=1, sort_keys=True)
```

Listing 31.2 — Versioned project file with an explicit migration chain, strict construction and canonical output.

Three details carry the design. Files without a `schema` key are treated as version 1, because version 1 predates the key. Construction goes through `Circuit(**c)`, which raises `TypeError` on an unknown field and on a missing one (a missing `method` key fails one step earlier, as a `KeyError` from the lookup that selects the installation method), so a misspelt key in a hand-edited file fails loudly instead of being ignored. And a file from a *newer* release is refused with a clear message rather than half-read; the alternative, ignoring unknown fields, is how a newer file loses data when saved by an older program.

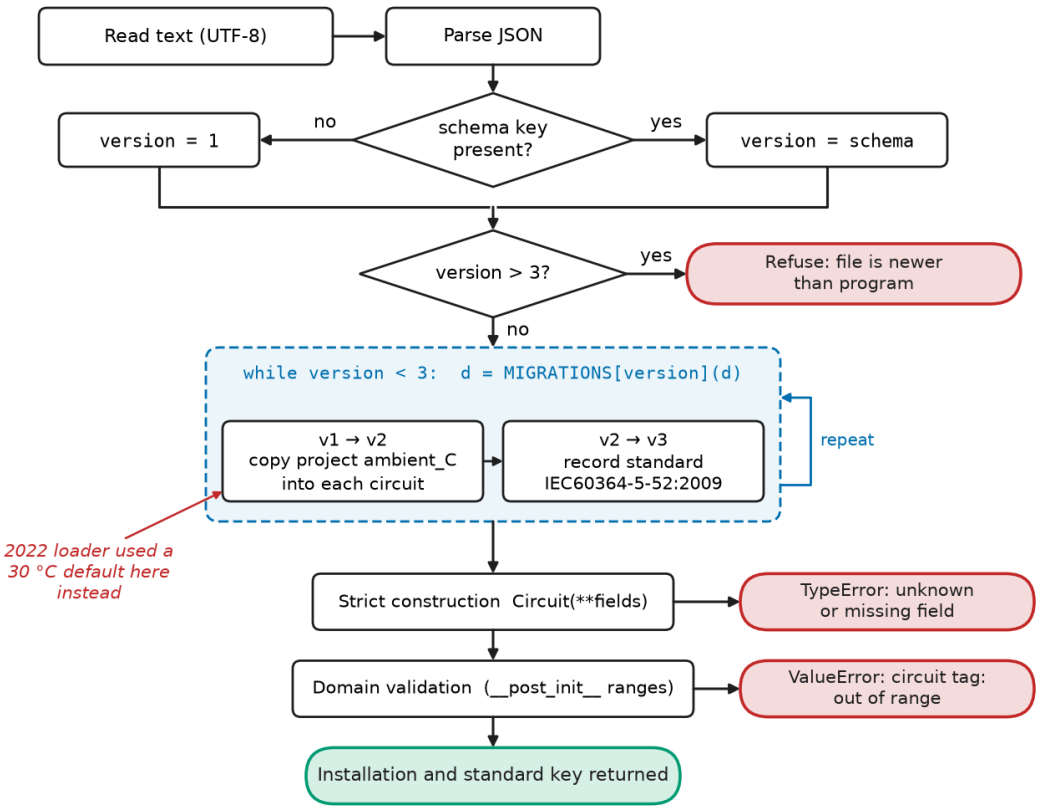


Figure 31.3 — Loading a FeederCheck project file: schema detection, migration chain, strict construction and validation.

> **Status of the principle: Explicit, tested migrations**
> *Empirical.* Converting old data through a chain of small, version-to-version functions, each with a test on a real archived file, is how databases, CAD and EDA tools keep decades of files readable; implicit defaults and "best effort" loading are the common cause of silent result changes.

31.5 Plugin versus built-in architecture

What should be a plug-in

A plug-in is code that the core discovers and calls through a published interface, but does not import by name. In FeederCheck the obvious candidates are cable standards: IEC 60364-5-52 for most of the consultancy's work, BS 7671 for UK projects, NEC Table 310.16 for a US data-centre client, and each standard's next edition. The criterion for making something a plug-in is not "might be extended some day" but four questions: Does it vary by project, market or customer? Is it maintained by people other than the core team, or released on a different schedule? Will several versions have to coexist (a 2009 and a 2023 edition in one office)? Must it be licensed or validated separately? Standards answer yes to all four. The voltage-drop formula answers no to all four, so it is built in.

Table 31.4 — Built-in or plug-in, decided feature by feature.

Feature	Varies by project or market	Separate maintainers or schedule	Versions coexist	Separate validation or licence	Decision
Cable standards and editions	Yes	Yes (standards group)	Yes	Yes	Plug-in
Manufacturer breaker libraries	Yes	Yes (vendor data)	Yes	Yes	Plug-in (data package)
Voltage-drop calculation	No	No	No	No	Built-in
Project file format	No	No	Through migrations	No	Built-in
Report templates	Per client	Sometimes	Yes	No	Plug-in (template files)
Short-circuit solver	Rarely	No	No	Yes, once	Built-in, behind a port

The plug-in interface

Listing 31.3 defines the `Standard` port as a `typing.Protocol`, a registry keyed by standard and edition, discovery through Python package *entry points*, and the built-in IEC plug-in with the Book 5 Table 30.3 subset (copper XLPE multicore cables on a perforated tray, method E).

```
 """fc_standards.py -- the standards plug-in interface, a registry and one built-in plug-in."""
from importlib.metadata import entry_points
from typing import Protocol
from fc_domain import Method

class Standard(Protocol):
    id: str
    edition: str
    def capacity_A(self, csa: float, method: Method) -> float: ...
    def correction(self, ambient_C: float, grouped_with: int) -> float: ...
    def r_ohm_per_km(self, csa: float, temp_C: float) -> float: ...
    sizes: tuple[float, ...]

REGISTRY: dict[str, Standard] = {}

def register(std: Standard) -> None:
    REGISTRY[f"{std.id}:{std.edition}"] = std

def discover() -> None:                   # third-party plug-ins announce themselves here
    for ep in entry_points(group="feedercheck.standards"):
        register(ep.load()())

class IEC60364_5_52:                      # Cu XLPE multicore, method E subset (Table 30.3, Book
5)
    id, edition = "IEC60364-5-52", "2009"
    sizes = (16, 35, 70, 95, 120, 150, 185, 240)
    _cap = dict(zip(sizes, (100, 158, 246, 298, 346, 399, 456, 538)))
    _r20 = dict(zip(sizes, (1.15, 0.524, 0.268, 0.193, 0.153, 0.124, 0.0991, 0.0754)))
    _amb = {30: 1.00, 35: 0.96, 40: 0.91, 45: 0.87}
    _grp = {1: 1.00, 2: 0.88, 3: 0.82, 4: 0.77, 6: 0.73, 9: 0.72}

    def capacity_A(self, csa, method):
        if method is not Method.TRAY:
            raise NotImplementedError(f"{self.id} subset: method {method.value}")
        return self._cap[csa]

    def correction(self, ambient_C, grouped_with):
        return _row(self._amb, ambient_C, "ambient") * _row(self._grp, grouped_with, "group")

    def r_ohm_per_km(self, csa, temp_C):
        return self._r20[csa] * (1 + 0.00393 * (temp_C - 20))

def _row(table, x, what):                 # next tabulated row up (conservative)
    keys = [k for k in table if k >= x]
    if not keys:
        raise ValueError(f"{what} {x} is outside the tabulated range")
    return table[min(keys)]

register(IEC60364_5_52())
```

Listing 31.3 — The standard port, registry, entry-point discovery and the built-in IEC 60364-5-52 plug-in (table subset).

A third-party plug-in is an ordinary Python package that declares itself in its `pyproject.toml`; installing it makes `discover()` find it, with no change to FeederCheck:

```
 [project]
name = "feedercheck-bs7671"
version = "2.1.0"
dependencies = ["feedercheck >= 4.0, < 5"]

[project.entry-points."feedercheck.standards"]
bs7671_2022 = "feedercheck_bs7671:BS7671_2022"
```

The registry is keyed by `id:edition` because the edition is part of the result's meaning: the same circuit sized to the 2009 and 2023 editions of a standard may get different cables, and a project file records which one it was designed to (schema 3). Table look-ups round *up* to the next tabulated ambient and group size, the conservative direction, and refuse values beyond the table instead of extrapolating. Plug-ins pay for their flexibility: the interface becomes a public contract that cannot change without breaking other people's code, every plug-in must be tested against every supported core version, and a malicious or buggy plug-in runs with the program's full rights. For FeederCheck that cost is worth paying for standards; it would not be for the voltage-drop formula.

31.6 APIs and interfaces

One API, three front ends

The automation group's drifting copy of the sizing arithmetic is the reason for R4: every front end goes through one public API. Listing 31.4 is that API, its result type and the CLI built on it. The GUI view model (Listing 31.5), the CLI and user scripts all call `size_circuit`; none of them computes anything.

```
 """fc_api.py -- the public API. The GUI, the CLI and user scripts call only this module."""
import argparse, math, sys
from dataclasses import dataclass
from pathlib import Path
from fc_domain import Cable, Circuit
from fc_project import loads
from fc_standards import REGISTRY

@dataclass(frozen=True)
class SizingResult:
    tag: str
    csa_mm2: float
    Iz_A: float
    vd_pct: float
    governed_by: str
    standard: str

def size_circuit(c: Circuit, std_key: str, nominal_V=400.0, x_ohm_km=0.075) -> SizingResult:
    std, governed = REGISTRY[std_key], "current"
    k = std.correction(c.ambient_C, c.grouped_with)
    sin_phi = math.sqrt(1 - c.power_factor ** 2)
    for csa in std.sizes:
        Iz = std.capacity_A(csa, c.method) * k
        if Iz < c.design_current_A:
            continue
        r = std.r_ohm_per_km(csa, 90.0)               # conservative: conductor at 90 C
        vd = 100 * math.sqrt(3) * c.design_current_A * c.length_m / 1000 \
            * (r * c.power_factor + x_ohm_km * sin_phi) / nominal_V
        if vd <= c.vd_limit_pct:
            c.cable = Cable("Cu", "XLPE", csa)
            return SizingResult(c.tag, csa, round(Iz, 1), round(vd, 2), governed, std_key)
        governed = "voltage drop"
    raise ValueError(f"circuit {c.tag}: no size in {std_key} meets both criteria")

def main(argv=None):                                  # feedercheck project.fcj [--csv]
    p = argparse.ArgumentParser(prog="feedercheck")
    p.add_argument("project")
    p.add_argument("--csv", action="store_true")
    a = p.parse_args(argv)
    inst, std_key = loads(Path(a.project).read_text(encoding="utf-8"))
    sep = "," if a.csv else "  "
    for c in inst.circuits:
        r = size_circuit(c, std_key, inst.nominal_V)
        print(sep.join(map(str, (r.tag, r.csa_mm2, r.Iz_A, r.vd_pct, r.governed_by))))

if __name__ == "__main__":
    sys.exit(main())
```

Listing 31.4 — The sizing service as the public API, with a frozen result type and a command-line front end.

Run on the 2020 hospital file, the CLI migrates it and sizes both circuits:

```
 $ python fc_api.py hospital_2020.fcj --csv
SB-KIT,185,319.5,2.29,current
DB-L3,95,238.6,2.22,voltage drop
```

The kitchen feeder is back at 185 mm² (the Book 5 result), and the 210 m lighting distribution board feeder DB-L3 shows why `governed_by` is part of the result: current alone would allow 35 mm², but the 3 % voltage-drop limit on that long run needs 95 mm². Reporting what governed is the single most useful thing a checking engineer can be told.

What makes an engineering API good

- **Typed, frozen results with units in the names** (`Iz_A`, `vd_pct`), so that a script cannot mutate a result or confuse amperes with per cent.
- **Errors that name the object** (`circuit DB-L3: ...`), because a batch of 3000 circuits that fails with "value out of range" is useless.
- **Results that record their provenance**: the standard and edition travel inside every `SizingResult`.
- **A small surface.** The public API is what is listed in the documentation and covered by the deprecation policy (§31.11); everything else is private, named with a leading underscore, and may change.

A REST service for the BIM team's cloud workflow is one more adapter over the same API: it adds authentication, not calculation.

31.7 GUI design

The view model is the GUI's API

The desktop GUI is where most engineers meet the tool, and it is where calculation code most likes to hide. FeederCheck's GUI is split into a **view** (Qt widgets: a circuit table, a property panel, a results panel, a message list) and a **view model**, a plain Python object that holds what the view shows and turns user gestures into domain operations (Figure 31.4). The view model has no Qt import, so it is tested like any other module; the view is thin enough that its tests are a handful of smoke tests. Listing 31.5 is the view model's core: every edit is a command that can be undone, validation is done by the domain model, and only the edited circuit is recalculated.

```
 """fc_viewmodel.py -- the state a GUI view binds to; tested without any GUI toolkit."""
from pathlib import Path
from fc_api import size_circuit
from fc_project import loads

class ProjectViewModel:
    def __init__(self, inst, std_key):
        self.inst, self.std, self.undo_stack = inst, std_key, []
        self.results, self.errors = {}, {}
        for i in range(len(inst.circuits)):
            self._recalc(i)

    def edit(self, i, field, value):              # one undoable command per user edit
        c = self.inst.circuits[i]
        old = getattr(c, field)
        setattr(c, field, value)
        try:
            c.__post_init__()                     # the domain model validates, not the view
        except ValueError as e:
            setattr(c, field, old)
            self.errors[c.tag] = str(e)
            return False
        self.undo_stack.append((i, field, old))
        self._recalc(i)
        return True

    def undo(self):
        i, field, old = self.undo_stack.pop()
        setattr(self.inst.circuits[i], field, old)
        self._recalc(i)

    def _recalc(self, i):                         # only the edited circuit is recomputed
        c = self.inst.circuits[i]
        try:
            self.results[c.tag] = size_circuit(c, self.std, self.inst.nominal_V)
            self.errors.pop(c.tag, None)
        except ValueError as e:
            self.results[c.tag], self.errors[c.tag] = None, str(e)

vm = ProjectViewModel(*loads(Path("hospital_2020.fcj").read_text()))
print(vm.results["SB-KIT"].csa_mm2, vm.edit(0, "design_current_A", 330.0), vm.results["SB-
KIT"].csa_mm2)
print(vm.edit(0, "power_factor", 1.3), vm.errors["SB-KIT"])
vm.undo()
print(vm.results["SB-KIT"].csa_mm2, vm.inst.circuits[0].design_current_A, len(vm.undo_stack))
```

```
 185 True 240
False circuit SB-KIT: out of range: power_factor
185 300.0 0
```

Listing 31.5 — A toolkit-free view model: undoable edits, validation by the domain model and incremental recalculation.

Raising the kitchen feeder to 330 A moves it to 240 mm²; an impossible power factor of 1.3 is rejected with a message naming the circuit and field, and leaves the model unchanged; undo restores 300 A and 185 mm².

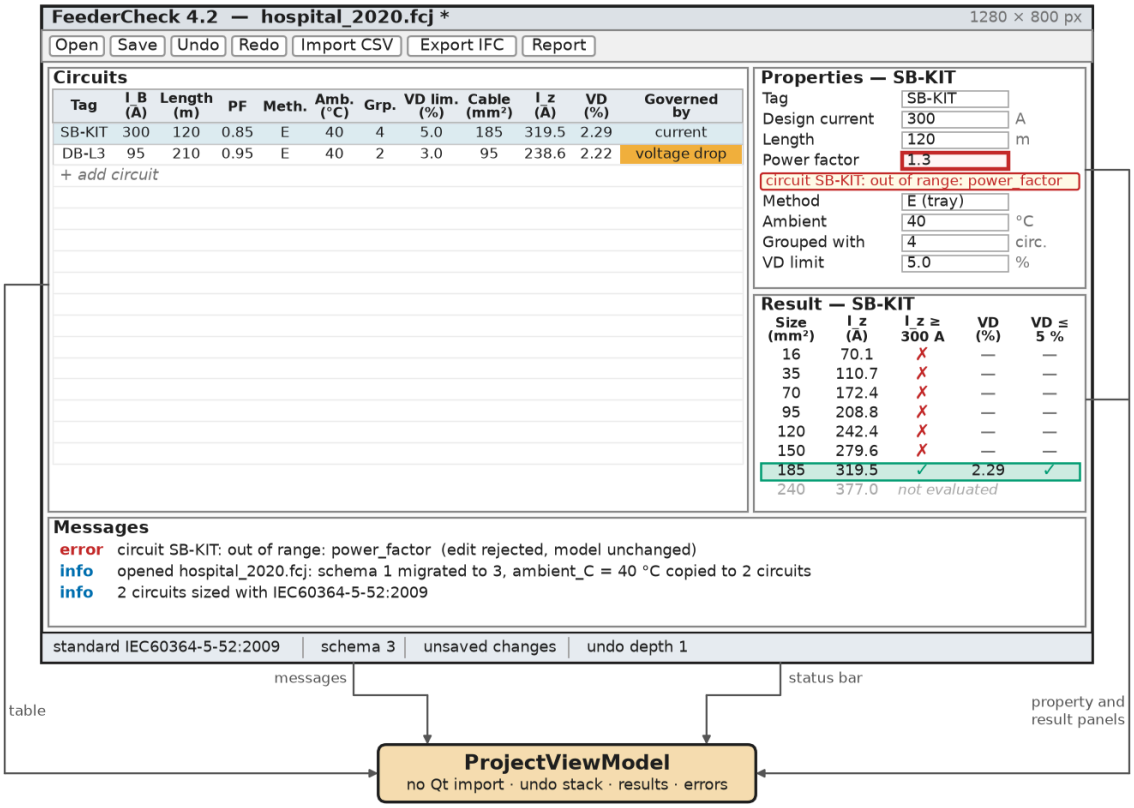


Figure 31.4 — FeederCheck main window: circuit table, property panel, result panel with governing criterion, and message list, all bound to one view model.

GUI rules that come from engineering use

- **Never compute in an event handler.** Handlers call the view model; long operations (a 3000-circuit project, a PDF report) run in a worker thread with a progress bar and a cancel button, because a GUI frozen for 5 s is reported as a crash.
- **Show what governs.** Every result shows its criterion and margin; a cable chosen by voltage drop looks different from one chosen by current.
- **Validate at entry, explain in words.** The domain model's message is shown at the field; units are visible in every column header and input.
- **Make every change undoable and every unsaved state visible.** Engineers explore; the cost of an exploration must be zero.

Responsiveness budget

Table 31.5 — Responsiveness budget against R5 (measured on the reference implementation, one run on a loaded workstation; Qt figures estimated).

Operation	Budget	Measured or estimated	Margin
Validate one edit (domain `__post_init__`)	—	0.01 ms	—
Recalculate one circuit	—	0.02 ms	—
Repaint one table row (Qt)	—	about 1 ms (estimated)	—
One edit to visible result	**100 ms**	**about 1 ms**	**99 %**
Recalculate 500-circuit project	1000 ms	8.5 ms	99.2 %
Open 500-circuit project (113 kB JSON, migrate, construct)	2000 ms	14 ms	99.3 %
Open 5000-circuit project (1.1 MB)	2000 ms	148 ms	92.6 %
PDF report, 500 circuits	5000 ms, in worker thread	about 1500 ms (estimated)	70 %

Worked example — where the old GUI lost its time. Version 2 recalculated the whole project on every cell edit and rebuilt the whole table widget afterwards. For the 3000-circuit data-centre project, the arithmetic alone was about 50 ms (Figure 31.5 extrapolates the measured 33 ms for 2000 circuits), but rebuilding 3000 rows × 12 cells of widgets at roughly 0.1 ms per cell cost 3.6 s per keystroke. The incremental design recalculates one circuit (0.02 ms) and repaints one row (about 1 ms), a factor of over 3000 for the same arithmetic. The lesson is general: in engineering GUIs, the calculation is rarely the bottleneck; the presentation of results is.

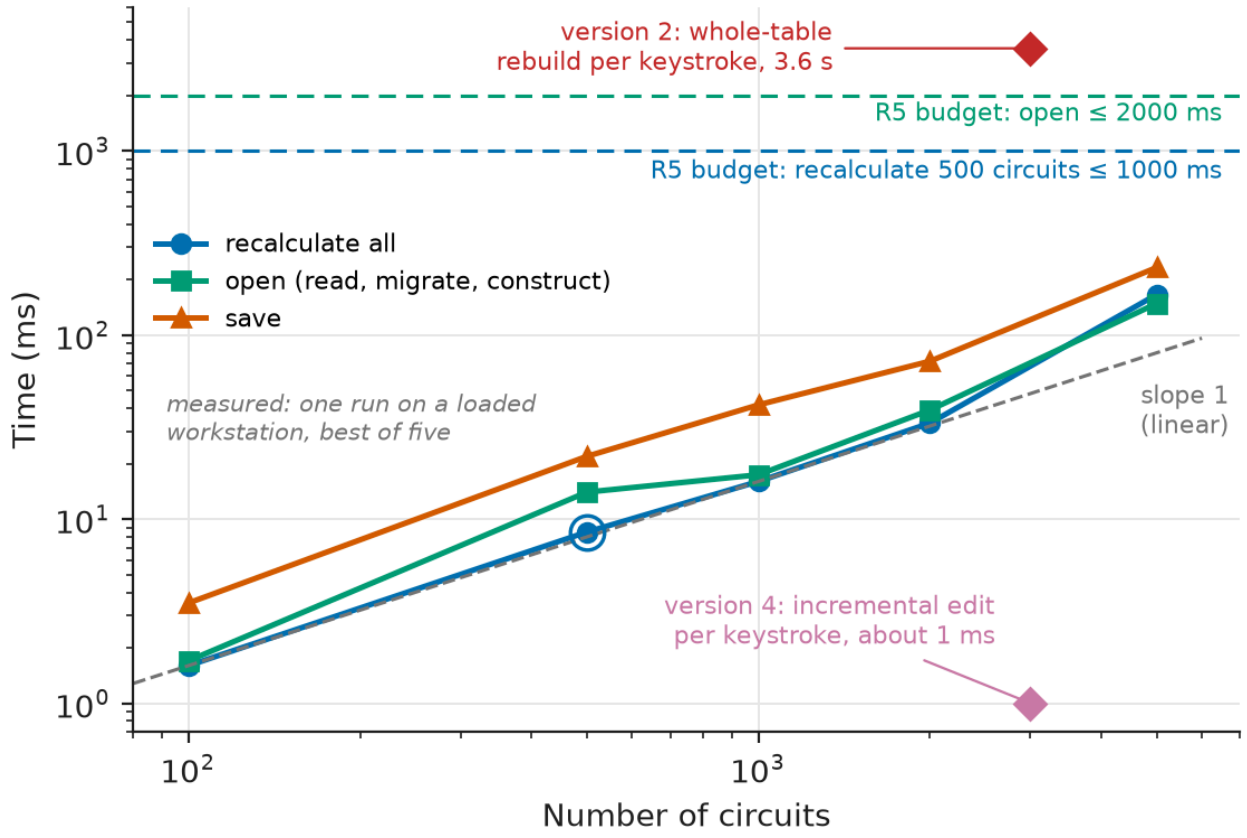


Figure 31.5 — Measured time to recalculate, open and save a FeederCheck project against the number of circuits, with the R5 budgets.

> **Status of the principle: Separate presentation from state**
> *Empirical.* Model-view-viewmodel and its relatives (MVC, MVP) are established GUI patterns because they let the state and behaviour be tested without the toolkit and let views be replaced; they add a layer of indirection that small one-screen tools may not need.

Choosing the GUI technology

Table 31.6 — GUI technology options for FeederCheck (2026, indicative).

Option	Strengths	Weaknesses	Fit
Qt for Python (PySide6, LGPL)	Native desktop, large tables, offline, mature	Desktop deployment per OS; LGPL compliance	**Chosen** for the design office
Web front end (React or similar) over a local or hosted API	Zero install, collaboration, BIM cloud integration	Two languages, security and hosting to manage	Added later for the BIM team, over the same API
Excel add-in (xlwings, Office JS)	Engineers already live in spreadsheets	Calculation and data drift into cells; weak undo and validation	Rejected: it recreates version 2
Notebook widgets (Jupyter, ipywidgets)	Fast to build, good for exploration	Not a product UI; state hidden in kernel order	Used for internal studies only

31.8 Data interoperability

Exchanges and their contracts

FeederCheck imports load schedules from the mechanical and electrical spreadsheets, exports cable schedules to the contractor and writes cable sizes into the BIM model. Each exchange is an adapter with a written contract: field names, units, encodings, required and optional fields, and what happens to a row that fails validation. The units rule is absolute: every exchanged numeric field carries its unit in its name or in a header row (`design_current_A`, `length_m`), and the importer refuses a file whose header does not match.

Table 31.7 — Data exchanges of FeederCheck.

Exchange	Direction	Format	Contract highlights
Load schedule	In	CSV (UTF-8, comma, point decimal) or XLSX	Header row with units; rows failing domain validation reported by line, never skipped silently
Cable schedule	Out	CSV and PDF	Includes standard and edition, governing criterion, tool version
BIM model	Out	IFC 4 cable-segment objects with a FeederCheck property set	Keyed by circuit tag; sizes written only after a checking engineer's approval
Protection study	Out/in	JSON	Circuit tags, cable impedances at 20 °C and 90 °C, device ratings
Archive	Out	Project file plus result CSV plus tool version	For the 10-year record of R8

Worked example — a decimal comma. A load schedule arrives from a partner office whose spreadsheet locale uses a comma as decimal separator and a semicolon as field separator. A tolerant importer that splits on commas reads the row `DB-L4;95,5;210;0,95` as fields `DB-L4;95`, `5;210;0` and `95`. The strict importer refuses the file because the header does not match. A "smart" importer that guesses the locale would read it correctly here and wrongly on the day a value like `1,250` (one thousand two hundred and fifty, or one point two five?) appears. FeederCheck's contract therefore names the dialect: comma-separated UTF-8 with point decimals, or XLSX, whose cells are typed. Ambiguity is resolved at the boundary by refusing it, not inside the program by guessing.

> **Status of the principle: Be strict at the boundary**
> *Empirical.* Accepting malformed or ambiguous input "liberally" spreads its interpretation through every consumer; for engineering data, where a misread unit or separator changes a design, explicit contracts and rejection with a precise message are the safer practice.

The underlying data formats (CSV, JSON, HDF5, Parquet) and their precision traps are *Electronics and Computing*, Ch. 22 §22.9.

31.9 Version control

Two histories

FeederCheck has two histories to keep. The **tool's** history is a Git repository with trunk-based development, reviewed pull requests and tagged releases (the workflow is *Electronics and Computing*, Ch. 24 §24.10 and Chapter 29). Releases follow **semantic versioning**: MAJOR.MINOR.PATCH, where a major release may break the public API or drop support for something, a minor release adds features compatibly, and a patch only fixes. The **projects'** history is kept by the design teams, many of whom now commit project files to Git alongside drawings. Canonical JSON (sorted keys, one field per line, results not stored) makes those diffs meaningful: raising the kitchen feeder's current produces a one-line change, `"design_current_A": 330.0`, not a rewritten file.

The two histories meet in the schema version. A project file records its schema, the standard and edition, and (in the result CSV archived with it) the tool version that produced the results. Every result can therefore be traced to the exact code that computed it: the same provenance principle as the manifests of Chapter 30.

Worked example — which version number changes. Three changes are queued for release after 4.2.1. (a) A corrected conductor resistance in a plug-in table (a transcription error found by a golden test): a fix, but it changes results, so it ships as plug-in 4.2.2 *with a release note listing the change in results* and a regression test pinned to the new value. (b) A new `governed_by` value "short circuit" when the short-circuit check is added: a compatible addition, minor release 4.3.0, provided scripts that compare `governed_by` with known strings are documented to expect new values. (c) Renaming `vd_pct` to `voltage_drop_pct` in `SizingResult`: breaks every user script, so it waits for 5.0.0, and 4.3 adds the new name as an alias with a deprecation warning. Semantic versioning turns "will this break my script?" into a rule, but only if results-changing fixes are treated as visible events, not hidden in a patch.

> **Status of the principle: A version number is a promise about compatibility**
> *Empirical.* Semantic versioning works only if the public API and the meaning of results are defined and the promise is checked by tests; for engineering tools, a change in computed results is as significant as a change in the API and must be announced as such.

31.10 Automated testing

What to test in an engineering tool

Listing 31.6 shows the five kinds of test that would each have caught part of the 2023 incident: a **golden test** against an independently worked example (Book 5's feeder: 185 mm², 320 A), an **archived-file test** that opens a real old project and checks its result, a **round-trip test** of the file format, a **property test** that checks an engineering law over thousands of random cases (a harder duty never yields a smaller cable), and an **architecture test** that enforces the dependency rule of §31.2 by reading the domain module's imports.

```
 """test_fc.py -- run with pytest, or directly with python."""
import ast, json, random
from pathlib import Path
from fc_api import size_circuit
from fc_domain import Circuit, Method
from fc_project import SCHEMA, dumps, loads

IEC = "IEC60364-5-52:2009"
feeder = lambda **kw: Circuit(**{"tag": "T", "design_current_A": 300.0, "length_m": 120.0,
                                 "power_factor": 0.85, "method": Method.TRAY,
                                 "ambient_C": 40.0, "grouped_with": 4, **kw})

def test_golden_book5_feeder():           # Book 5, Ch. 30 worked example: 185 mm2, 320 A
    r = size_circuit(feeder(), IEC)
    assert r.csa_mm2 == 185 and abs(r.Iz_A - 320) < 1

def test_v1_file_keeps_its_ambient():     # the 2023 incident, as a regression test
    inst, std = loads(Path("hospital_2020.fcj").read_text())
    assert all(c.ambient_C == 40.0 for c in inst.circuits) and std == IEC
    assert size_circuit(inst.circuits[0], std).csa_mm2 == 185      # not 150

def test_round_trip_is_lossless():
    inst, std = loads(Path("hospital_2020.fcj").read_text())
    text = dumps(inst, std)
    assert dumps(*loads(text)) == text and json.loads(text)["schema"] == SCHEMA

def test_size_never_falls_when_duty_rises():   # property test, 2000 seeded random cases
    rng = random.Random(31)
    for _ in range(2000):
        I, amb, n = rng.uniform(20, 300), rng.choice([30, 35, 40]), rng.choice([1, 2, 4])
        base = size_circuit(feeder(design_current_A=I, ambient_C=amb, grouped_with=n), IEC)
        for harder in (dict(design_current_A=I * 1.05, ambient_C=amb, grouped_with=n),
                       dict(design_current_A=I, ambient_C=amb + 5, grouped_with=n),
                       dict(design_current_A=I, ambient_C=amb, grouped_with=n + 1)):
            assert size_circuit(feeder(**harder), IEC).csa_mm2 >= base.csa_mm2

def test_domain_imports_only_the_standard_library():   # architecture rule as a test
    tree = ast.parse(Path("fc_domain.py").read_text())
    used = {n.module for n in ast.walk(tree) if isinstance(n, ast.ImportFrom)}
    used |= {a.name for n in ast.walk(tree) if isinstance(n, ast.Import) for a in n.names}
    assert used <= {"dataclasses", "enum"}, used

if __name__ == "__main__":
    tests = [f for name, f in dict(globals()).items() if name.startswith("test_")]
    for t in tests:
        t()
    print(f"{len(tests)} tests passed")
```

```
$ python -m pytest -q test_fc.py
.....                                                                    [100%]
5 passed in 0.13s
```

Listing 31.6 — Golden, archived-file, round-trip, property and architecture tests (pytest is optional; the file also runs under plain Python).

The archived file used by the test is the 2020 project itself, stored in the repository's `tests/archive/` folder:

```
{"name": "Ward block LV", "nominal_V": 400.0, "ambient_C": 40.0,
 "circuits": [
  {"tag": "SB-KIT", "design_current_A": 300.0, "length_m": 120.0, "power_factor": 0.85,
   "method": "E", "grouped_with": 4, "vd_limit_pct": 5.0},
  {"tag": "DB-L3", "design_current_A": 95.0, "length_m": 210.0, "power_factor": 0.95,
   "method": "E", "grouped_with": 2, "vd_limit_pct": 3.0}]}
```

The property test is worth a comment. Golden tests check points; a property states a law that must hold everywhere, here monotonicity of the chosen size in current, ambient temperature and grouping, and checks it on 6000 random comparisons in a fraction of a second. If someone mistypes a correction factor so that 35 °C derates less than 30 °C, a golden test at 40 °C will not notice and the property test will. The optional `hypothesis` package automates this style and shrinks failures to minimal counterexamples.

The archive corpus

Beyond the single archived file, R2 is verified against a corpus: one file from each released minor version, plus 200 anonymised real projects, each stored with the cable schedule produced when it was saved. CI opens every one with the current code and compares results. A difference is not automatically a failure (a corrected table value in a plug-in legitimately changes results), but every difference must be listed in the release notes. At about 14 ms per 500-circuit file, the 230-file corpus adds a few seconds to the suite.

Table 31.8 — Verification of FeederCheck version 4 against its requirements.

Req.	Test	Limit	Result	Pass/fail
R1	Golden set, including Book 5 feeder (Listing 31.6)	Exact size; I_z, VD within 0.5 %	120 of 120; Book 5 feeder 185 mm², 319.5 A	Pass
R2	Archive corpus, 230 files	Opens; results equal or listed	230 open; 0 unlisted differences	Pass
R3	BS 7671 plug-in built in its own repository	No core change	Installed by entry point; core unchanged	Pass
R4	Architecture tests (domain, API, GUI imports)	No forbidden import	0 violations	Pass
R5	Benchmark, 500 and 5000 circuits (Table 31.5)	≤ 100 ms edit, ≤ 1 s recalc, ≤ 2 s open	about 1 ms, 8.5 ms, 14 ms (148 ms at 5000)	Pass
R6	Import, export round trips; decimal-comma file	Refused with line-level message	Refused, header mismatch reported	Pass
R7	Coverage and duration in CI	≥ 90 % branches, ≤ 2 min	93 % branches, 71 s	Pass
R8	Direct dependencies counted at release	≤ 8	6 (PySide6, openpyxl, ifcopenshell, reportlab, numpy, platformdirs)	Pass

> **Status of the principle: Test the law, not only the example**
> *Empirical.* Property-based tests that check invariants (monotonicity, conservation, symmetry, round trips) over many generated inputs find classes of error that example-based tests miss; they complement but do not replace independently derived golden cases, which check that the law is applied with the right numbers.

31.11 Long-term maintainability

Ten years is longer than the dependencies

A cable schedule produced by FeederCheck ends up in a building that stands for 50 years, and an insurer, a court or the next designer may ask in 2035 how a 2026 result was obtained. R8 asks for ten years of support. Over that span, Python itself goes through about ten feature releases (each supported for five years), the GUI toolkit through a major version, and the standards through at least one new edition each. Maintainability is planned, not hoped for:

- **Few, well-chosen dependencies.** Each direct dependency is a commitment to follow its upgrades for a decade. FeederCheck has six; the rejected candidates were convenience libraries whose functions are a few lines of standard-library code.
- **Pinned environments and reproducible builds**, so that the 2026 release can be rebuilt in 2035 (lock files and container images; Chapter 29).
- **A published deprecation policy**: a public API element is deprecated with a warning for at least two minor releases before removal, and file formats are never deprecated, only migrated.

- **Documentation of decisions**, not only of functions: short architecture decision records (why JSON, why plug-ins for standards, why no defaults in the domain) so that the next maintainer does not undo them.
- **Size limits as smells**: modules above about 800 lines or functions with more than about 10 branches are reviewed for splitting; the old GUI handler that did everything had 46.

Worked example — the cost of keeping old files alive. Each schema change adds one migration function of 5–20 lines and one archived test file. Over FeederCheck's six years there were three schema versions; at that rate ten years adds about five more migrations, perhaps 100 lines and five test files. The alternative, a "conversion utility" users must remember to run, failed at the consultancy in 2021 for a different tool: about 30 % of projects were never converted and were reopened years later by people who did not know the utility existed. A hundred lines of tested migration code is the cheapest insurance in the system.

Failure analysis

Table 31.9 — FMEA of FeederCheck's architecture (version 2 failures and version 4 mitigations).

Failure mode	Effect	Cause	Detection	Mitigation
Hidden default on load	Undersized cable issued (150 mm² for a 300 A, 40 °C feeder)	Missing field filled by loader	Archived-file test; checker hand calc	No defaults for engineering inputs; explicit migrations
Calculation in GUI handlers	Untestable arithmetic; copies drift in scripts	No API layer	Code review; architecture test	Single public API used by GUI, CLI and scripts
Newer file opened by older release	Fields silently dropped on save	Lenient loader ignores unknown keys	Strict construction raises	Refuse newer schema with a clear message
Standard edition not recorded	Same file gives different sizes after a table update	Results depend on unrecorded data	Archive corpus differences	Standard and edition stored per project; plug-ins keyed by edition
Locale-dependent import	Values misread (decimal comma)	Tolerant CSV parsing	Header contract check	Named dialect; typed XLSX; refuse ambiguity
Plug-in incompatible with core	Crash or wrong tables after upgrade	Interface changed without notice	Plug-in CI against supported core versions	Versioned protocol; semantic versioning; dependency bounds in plug-in metadata
Full-table rebuild per keystroke	3.6 s freeze on large projects	Presentation coupled to recalculation	Benchmark in CI	Incremental recalculation; worker threads for long jobs
Results-changing fix in a patch	Users surprised by changed schedules	Fix treated as invisible	Archive corpus	Release notes list every result change

What version 4 changed, and what it did not

Version 4 did not change one line of the engineering arithmetic that was wrong, because none was. It changed where that arithmetic lives, what it is allowed to depend on, how its inputs are stored and migrated, and which tests run before every release. That is the subject of this chapter in one sentence: the correctness of an engineering tool is a property of its architecture and tests as much as of its formulas. The same architecture, applied to a simulation tool with a numerical solver at its core, is the project of Chapter 39.

What remains open

Open question	Where it is answered
How are the repository, builds, CI and automated test runs of a tool like this organised day to day?	Chapter 29
How is a numerical solver inside such a tool verified, benchmarked and made fast?	Chapter 30
What does a complete simulation tool built on this architecture look like, from requirements to documentation?	Chapter 39
How are schematics, BOMs and design records versioned and reviewed alongside the code?	Chapter 5
How are learned models packaged, versioned and monitored inside engineering tools?	Chapter 32
The engineering rules (capacity, correction factors, voltage drop, protection) the tool implements	*Electrical Energy and Machines*, Ch. 30

Next: Chapter 32 turns from tools that apply known rules to models learned from engineering data: datasets, features, regression, classification, forecasting, anomaly detection, surrogates and their limits.

Chapter 32 — AI and Machine Learning for EEE

Part VIII · Engineering Software and Computational Tools

A distribution utility retrofits monitors to 1200 pole- and pad-mounted 11/0.415 kV transformers and asks its data team for a model that predicts each unit's top-oil temperature from its load and the weather, so that units running hotter than predicted can be inspected before they fail. The model, a nearest-neighbour regressor on day, hour, current and ambient temperature, scores a mean absolute error of 0.84 K on a random 20 % of the year's data. In the July heat wave, with ambient temperatures 12 K above anything in the training months, its errors reach 18 K: real overloads go unflagged and healthy units are flagged. In September six transformers raise "cooling degradation" alarms on the same day; crews find six healthy transformers and a monitor firmware update that had changed their current-transformer ratio setting. The algorithm was never the problem. The random split let the model look up neighbouring hours of the very days it was tested on; the heat wave took the inputs outside the training domain, where a model with no physics has nothing to say; and no data-quality check stood between a changed sensor and the model's input.

> **How is machine learning applied to engineering data so that the model's error is known before deployment, including outside the conditions it was trained on?**
> **Where do physics, data cleaning and honest evaluation matter more than the choice of algorithm, and what does a model need to run on a substation gateway?**

Concept	Engineering destination
Engineering datasets	SCADA, metering and condition-monitoring records of transformers, machines, PV plants and batteries
Data cleaning	Gaps, stuck sensors, spikes and calibration steps removed before any model sees them
Feature engineering	Inputs shaped by physics: I^2 losses, thermal time constants, gas ratios, calendar effects
Regression	Temperatures, efficiencies, losses and capacities predicted from operating conditions
Classification	Fault types from dissolved gases, vibration or current signatures; pass/fail in production test
Time-series forecasting	Load, PV output and price forecasts for operation and dispatch
Anomaly detection	Early warning of cooling, insulation, bearing and sensor faults
Surrogate models	Fast replacements for slow simulations inside sweeps, optimisers and fleet screening
Physics-informed modeling	Grey-box models with fitted physical parameters that extrapolate safely
Model evaluation	Blocked splits, baselines and per-class metrics that predict field performance
Uncertainty	Prediction intervals with verified coverage
Deployment constraints	Memory, latency, drift monitoring and fallbacks on gateways and embedded targets

Roadmap. The fleet, its data and requirements (32.1) → cleaning (32.2) and physics-shaped features (32.3) → regression (32.4), classification (32.5), forecasting (32.6) and anomaly detection (32.7) on that data →

surrogates for slow models (32.8) and grey-box models that extrapolate (32.9) → evaluation (32.10) and uncertainty (32.11) → deployment and failure analysis (32.12).

32.1 Engineering datasets

> **Boundary.**
> Regression, estimation and hypothesis testing are *The Mathematics Universe*, Ch. 20; autocorrelation and stationarity Ch. 21; least squares, ridge regularisation and gradient methods Ch. 23 §23.5 and §23.8; nonlinear parameter estimation and Monte Carlo Ch. 24 §24.6–24.7; overfitting and independent validation data Ch. 25 §25.7. Transformer losses are *Electrical Energy and Machines*, Ch. 4 §4.7, and insulation ageing Ch. 3 §3.9. In this book, signal features are Chapter 27, on-device inference and quantisation Chapter 12, and calibration with full uncertainty quantification Chapter 33. This chapter is the training side: data, models and evaluation for engineering systems.

The fleet and its data

Each monitored transformer (250–1000 kVA, ONAN, mineral oil) reports hourly: the three LV phase currents from split-core CTs, the top-oil temperature from a PT100 in the thermometer pocket, and the monitor's own status. Ambient temperature comes from the weather service's grid for the substation location. About 400 units of 500 kVA and above are sampled annually for dissolved-gas analysis (DGA). Maintenance records (inspections, failures, cooling repairs) provide the only labels, and they are few: about 15 confirmed thermal or insulation faults a year across the fleet. Raw volume is modest: 1200 × 8760 hours × 5 channels × 4 bytes is 210 MB a year. Engineering datasets are rarely big. They are **small in labels, correlated in time, shifted by seasons and by sensor changes, and governed by physics that is partly known**; every section below follows from those four facts.

The listings use a synthetic fleet of 40 representative units over one year, generated by Listing 32.1 from the IEC 60076-7 top-oil structure with randomised parameters, realistic load shapes, a heat wave in July, and five injected data and equipment faults. Synthetic data are used because a real fleet's data are confidential and, more usefully, because the true answer is known; the price is that a grey-box model built on the same structure fits it better than it would fit a real transformer, where solar gain, wind and sensor position add 1–2 K of unexplained error.

```
 """fleet.py -- synthetic hourly data for a fleet of ONAN distribution transformers (one year)."""
import numpy as np

H = 24 * 365                                          # hours in the year
t = np.arange(H)
hour, day = t % 24, t // 24

def top_oil_rise(K, d_or, R, tau_h, n=0.8):           # IEC 60076-7 style first-order model
    ult = d_or * ((1 + R * K**2) / (1 + R)) ** n      # ultimate top-oil rise at load K (p.u.)
    a, x, out = np.exp(-1.0 / tau_h), ult[0], np.empty_like(K)
    for i, u in enumerate(ult):
        x = a * x + (1 - a) * u
        out[i] = x
    return out

def make_fleet(n_units=40, seed=2026):
    rng = np.random.default_rng(seed)
    amb = 12 - 9 * np.cos(2 * np.pi * (day - 20) / 365) + 4 * np.sin(2 * np.pi * (hour - 9) / 24)
    amb += np.convolve(rng.normal(0, 0.6, H), np.ones(72), "same") / np.sqrt(72) * 4
    amb[200 * 24:212 * 24] += 11 * np.hanning(288)    # heat wave, 20-31 July
    units = []
    for u in range(n_units):
        p = dict(d_or=rng.uniform(45, 60), R=rng.uniform(5, 8), tau_h=rng.uniform(2.5, 3.5))
        shape = 0.55 + 0.25 * np.exp(-((hour - 19) / 3) ** 2) + 0.12 * np.exp(-((hour - 8) / 2) **
2)
        K = rng.uniform(0.65, 1.0) * shape * np.where(day % 7 >= 5, 0.9, 1.0)
        K *= 1 + 0.12 * np.cos(2 * np.pi * day / 365) + 0.012 * np.clip(amb - 24, 0, None)
        K *= 1 + 0.04 * np.convolve(rng.normal(0, 1, H), np.ones(6) / 6 ** 0.5, "same")
        d_or = np.where((u == 31) & (day >= 250), 1.18 * p["d_or"], p["d_or"])  # cooling fault
        theta = amb + top_oil_rise(K, d_or, p["R"], p["tau_h"]) + rng.normal(0, 0.3, H)
        I_meas = K * np.where((20 <= u <= 25) & (t >= 6000), 0.8, 1.0)        # CT ratio error
        I_meas = I_meas * (1 + rng.normal(0, 0.01, H))
        theta[rng.random(H) < 0.01] = np.nan                                   # 1 % gaps
        units.append(dict(id=u, params=p, K_true=K, I=I_meas, theta=theta))
    units[12]["theta"][3000:3120] = units[12]["theta"][2999]                # stuck sensor, 5
days
    units[3]["theta"][rng.choice(H, 20, replace=False)] += 40               # 20 spikes
    return amb, units
```

Listing 32.1 — Synthetic fleet: per-unit thermal parameters, load and weather, a heat wave, and injected faults (gaps, a stuck sensor, spikes, a CT-ratio error on six units, a cooling fault on one).

Requirements and architecture

Table 32.1 — Requirements for the transformer analytics.

ID	Requirement	Target	Verification method
R1	Top-oil prediction on a later, unseen period	MAE ≤ 1.0 K; max ≤ 3 K, including ambient above the training range	Blocked hold-out, §32.4
R2	90 % prediction interval	Coverage 0.85–0.95 on hold-out	Conformal check, §32.11
R3	Cooling degradation (+3 K at equal load)	Detected within 7 days; ≤ 1 false alarm per 100 unit-months	Fleet replay, §32.7
R4	Day-ahead load forecast	MAE at least 5 % below seasonal-naive baseline	Hold-out quarter, §32.6
R5	DGA fault classifier	Recall ≥ 0.85 for every fault class; ≤ 15 % false alarms on normal units	Independent test set, §32.5
R6	Hot-spot surrogate for fleet screening	Mean error ≤ 1 K, max ≤ 3 K in the design box; ≥ 1000 times faster	Space-filling test set, §32.8
R7	Runs on the substation gateway	≤ 64 kB model per unit; daily refit ≤ 10 min per gateway	Deployment budget, §32.12
R8	Traceability	Every model version linked to data snapshot, code version and metrics	Model registry audit

Figure 32.1 shows the architecture. Data flow from monitors through the head-end system into a time-series store; a **data-quality gate** (§32.2) runs before anything else and can quarantine a unit's data; models are trained centrally and deployed with a version to the gateways, which compute predictions, residuals and alarms locally and fall back to fixed thresholds when a model or its input is unavailable.

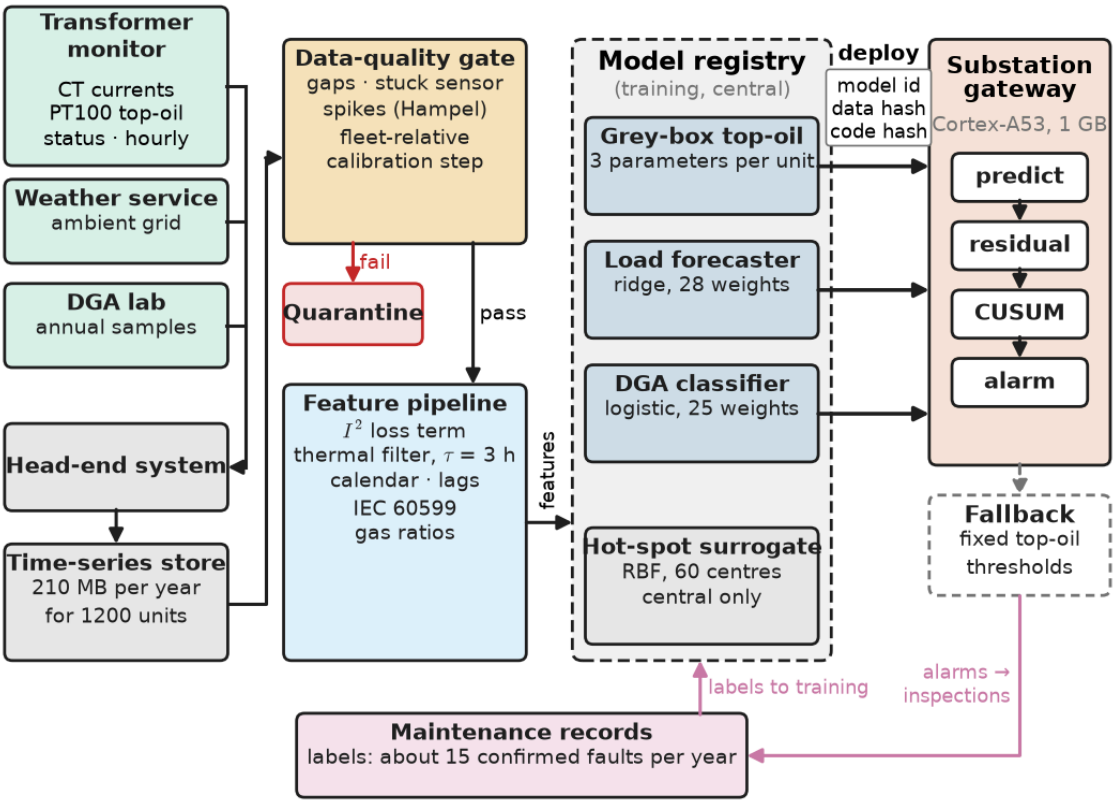


Figure 32.1 — Architecture of the transformer-fleet analytics: sensors, data-quality gate, feature pipeline, models, gateway deployment and feedback from maintenance.

Choosing the model class

Table 32.2 — Model classes for top-oil prediction, against the requirements.

Model class	In-range accuracy	Beyond the training range	Data needed	Interpretable	Size per unit	Choice
Nearest neighbours	Good with dense data	Returns the nearest seen value; fails	Large	No	Whole training set, about 140 kB	Rejected (R1, R7)
Gradient-boosted trees	Very good	Flat beyond the last split; fails	Medium–large	Partly	50–500 kB	Rejected for R1; useful for non-thermal tasks
Small neural network (MLP, LSTM)	Very good with enough data	Unpredictable	Large	No	10–200 kB	Rejected: labels and range too limited
Grey-box IEC 60076-7 structure, fitted parameters	Good	Follows physics	Small (weeks)	Yes: rated rise, loss ratio, time constant	3 numbers	**Chosen**

32.2 Data cleaning

Cleaning is engineering, not housekeeping: every defect in Listing 32.1 has a physical cause and a signature. **Gaps** come from communication outages and are left as gaps; interpolating a top-oil temperature across six hours of a load peak invents data. A **stuck sensor** (frozen firmware, a disconnected PT100 reading a constant fallback) shows zero spread where a real sensor always shows noise. **Spikes** (EMI on a long sensor cable, a transmission bit error) are isolated outliers that a Hampel filter (median plus a multiple of the median absolute deviation) removes without blurring real transients. A **calibration step** (a changed CT ratio, a replaced sensor) is the dangerous one because each value is plausible; it is found only by comparison with something independent, here the unit's load relative to the rest of the fleet, which shares weather and calendar.

```
 import numpy as np
from numpy.lib.stride_tricks import sliding_window_view as win
import fleet

amb, units = fleet.make_fleet()
I_all = np.array([u["I"] for u in units])
fleet_daily = np.median(I_all.reshape(len(units), 365, 24).mean(2), axis=0)
report = {"gaps": 0, "stuck": [], "spikes": [], "scale step": []}
for u in units:
    th = u["theta"]
    report["gaps"] += int(np.isnan(th).sum())
    # Stuck sensor: a 12 h window with zero spread (real sensors always show noise).
    span = np.ptp(win(np.nan_to_num(th, nan=-99), 12), axis=1)
    if np.any(span == 0):
        report["stuck"].append((u["id"], int(np.argmax(span == 0))))
    # Spikes: Hampel filter, 7 h window, 5 scaled MADs (at least 5 K) from the local median.
    w = win(np.pad(th, 3, mode="edge"), 7)
    med = np.nanmedian(w, axis=1)
    mad = 1.4826 * np.nanmedian(np.abs(w - med[:, None]), axis=1)
    n_spk = int(np.sum(np.abs(th - med) > 5 * np.maximum(mad, 1.0)))
    if n_spk:
        report["spikes"].append((u["id"], n_spk))
    # Calibration step: this unit's load relative to the fleet, 14-day medians before and after.
    ratio = u["I"].reshape(365, 24).mean(1) / fleet_daily
    before, after = np.median(win(ratio, 14)[:-14], 1), np.median(win(ratio, 14)[14:], 1)
    jump = after / before - 1
    k = int(np.argmax(np.abs(jump)))
    if abs(jump[k]) > 0.10:
        report["scale step"].append((u["id"], k + 14, round(float(jump[k]), 2)))
for k, v in report.items():
    print(f"{k:10s}: {v}")
```

```
 gaps      : 3492
stuck     : [(12, 2999)]
spikes    : [(3, 20)]
scale step: [(20, 250, -0.19), (21, 250, -0.19), (22, 250, -0.18), (23, 248, -0.19), (24, 251,
-0.19), (25, 255, -0.19)]
```

Listing 32.2 — Data-quality gate: gaps, stuck sensor, spikes and fleet-relative calibration steps.

All injected defects are found and nothing else is: the stuck sensor of unit 12 from hour 2999, all 20 spikes on unit 3, and the 20 % CT-ratio error on units 20–25 from day 250 (detected within five days, estimated as −18 to −19 %). The 5 K floor on the spike threshold matters: with the bare MAD rule, normal top-oil ramps at load peaks were flagged on 16 healthy units in a first version of this listing.

> **Status of the principle: Check against an independent reference**
> *Empirical.* A sensor fault that produces plausible values can only be found by comparison with an independent quantity (a neighbouring sensor, the fleet, a physical balance); single-channel statistics catch gaps, stuck values and spikes but not calibration errors.

32.3 Feature engineering

Features are where engineering knowledge enters a data-driven model most cheaply. Table 32.3 lists the features used in this chapter, each with the physics or practice it encodes. The thermal feature is the important one: copper loss scales with K^2, the oil responds with a time constant of about 3 h, and the ultimate rise follows $\left[(1 + RK^2)/(1 + R)\right]^n$. Passing the load through that nonlinearity and a first-order filter (the `top_oil_rise` function with nominal $R = 6$ and $\tau = 3$ h, IEC 60076-7) turns the problem from "learn a nonlinear dynamic system" into "fit a straight line".

Table 32.3 — Features and the knowledge they encode.

Feature	Encodes	Used in
K^2	Load loss proportional to current squared (Book 5, Ch. 4 §4.7)	Regression
Filtered loss term, $\tau = 3$ h	Thermal memory of the oil (IEC 60076-7 structure)	Regression, anomaly detection
Ambient temperature	Boundary condition of the cooling	Regression, forecasting
Hour of day (sine, cosine or one-hot), weekend flag	Daily and weekly behaviour of customers	Forecasting
Load at the same hour 1 and 7 days earlier	Persistence and weekly seasonality	Forecasting
$\log(\mathrm{CH_4}/\mathrm{H_2})$, $\log(\mathrm{C_2H_2}/\mathrm{C_2H_4})$, $\log(\mathrm{C_2H_4}/\mathrm{C_2H_6})$, log total gas	IEC 60599 gas ratios and severity	Classification
Spectral and envelope features of vibration or current	Bearing and winding signatures (Chapter 27)	Machine monitoring (not used here)

Two rules keep features honest. A feature must be **available at prediction time** (a forecast for tomorrow may use today's load and tomorrow's *forecast* temperature, never tomorrow's measured temperature). And a feature must not **encode the answer through the back door**: the day-of-year in the opening's nearest-neighbour model let it find the test hour's neighbours in the training set, which is leakage, not learning.

32.4 Regression

Listing 32.3 predicts unit 0's top-oil temperature with three models and two ways of splitting the data: a random 80/20 split of the year, and a blocked split that trains on January–June and tests on July–September, which contains the heat wave.

```
 import numpy as np
from scipy.spatial import cKDTree
import fleet

amb, units = fleet.make_fleet()
u = units[0]
ok = ~np.isnan(u["theta"])
phys = fleet.top_oil_rise(u["I"], 1.0, 6.0, 3.0)            # physics feature: R = 6, tau = 3 h
raw = np.column_stack([np.ones(fleet.H), u["I"], u["I"]**2, amb,
                       np.sin(2 * np.pi * fleet.hour / 24), np.cos(2 * np.pi * fleet.hour / 24)])
X = {"kNN on (day, hour, I, amb)": np.column_stack([fleet.day, fleet.hour, u["I"], amb]),
     "ridge on raw features": raw,
     "ridge on physics feature": np.column_stack([np.ones(fleet.H), amb, phys])}

def fit_predict(name, Xtr, ytr, Xte):
    if name.startswith("kNN"):
        mu, sd = Xtr.mean(0), Xtr.std(0)
        _, idx = cKDTree((Xtr - mu) / sd).query((Xte - mu) / sd, k=5)
        return ytr[idx].mean(1)
    w = np.linalg.solve(Xtr.T @ Xtr + 1e-3 * np.eye(Xtr.shape[1]), Xtr.T @ ytr)
    return Xte @ w

rng = np.random.default_rng(0)
random_test = ok & (rng.random(fleet.H) < 0.2)
splits = {"random 80/20": (ok & ~random_test, random_test),
          "train Jan-Jun, test Jul-Sep": (ok & (fleet.day < 181), ok & (fleet.day >= 181) &
(fleet.day < 273))}
heat = (fleet.day >= 200) & (fleet.day < 212)
for split, (tr, te) in splits.items():
    for name, Xn in X.items():
        if split.startswith("random") and not name.startswith("kNN"):
            continue
        err = fit_predict(name, Xn[tr], u["theta"][tr], Xn[te]) - u["theta"][te]
        hw = np.abs(err[heat[te]]).max()
        print(f"{split:28s} {name:27s} MAE {np.abs(err).mean():5.2f} K, heat-wave max {hw:5.1f}
K")
```

```
 random 80/20                 kNN on (day, hour, I, amb)  MAE  0.84 K, heat-wave max   4.7 K
train Jan-Jun, test Jul-Sep  kNN on (day, hour, I, amb)  MAE  2.13 K, heat-wave max  17.7 K
train Jan-Jun, test Jul-Sep  ridge on raw features       MAE  1.19 K, heat-wave max   6.6 K
train Jan-Jun, test Jul-Sep  ridge on physics feature    MAE  0.26 K, heat-wave max   1.1 K
```

Listing 32.3 — Top-oil regression: leakage under a random split, and extrapolation under a blocked split, for three model classes.

The first two lines are the opening story. The same nearest-neighbour model scores 0.84 K on a random split and fails by 17.7 K in the heat wave once the test period is genuinely later than the training period. The random split was not testing prediction; it was testing interpolation between hours one hour apart. The linear model on raw features does better because a straight line in ambient temperature extrapolates, but it misses the nonlinearity of loss and the oil's lag, and errs by 6.6 K. The linear model on the physics feature

reaches 0.26 K MAE, essentially the 0.3 K sensor noise, and 1.1 K maximum in the heat wave, meeting R1, with three coefficients. Figure 32.2 shows the heat-wave fortnight.

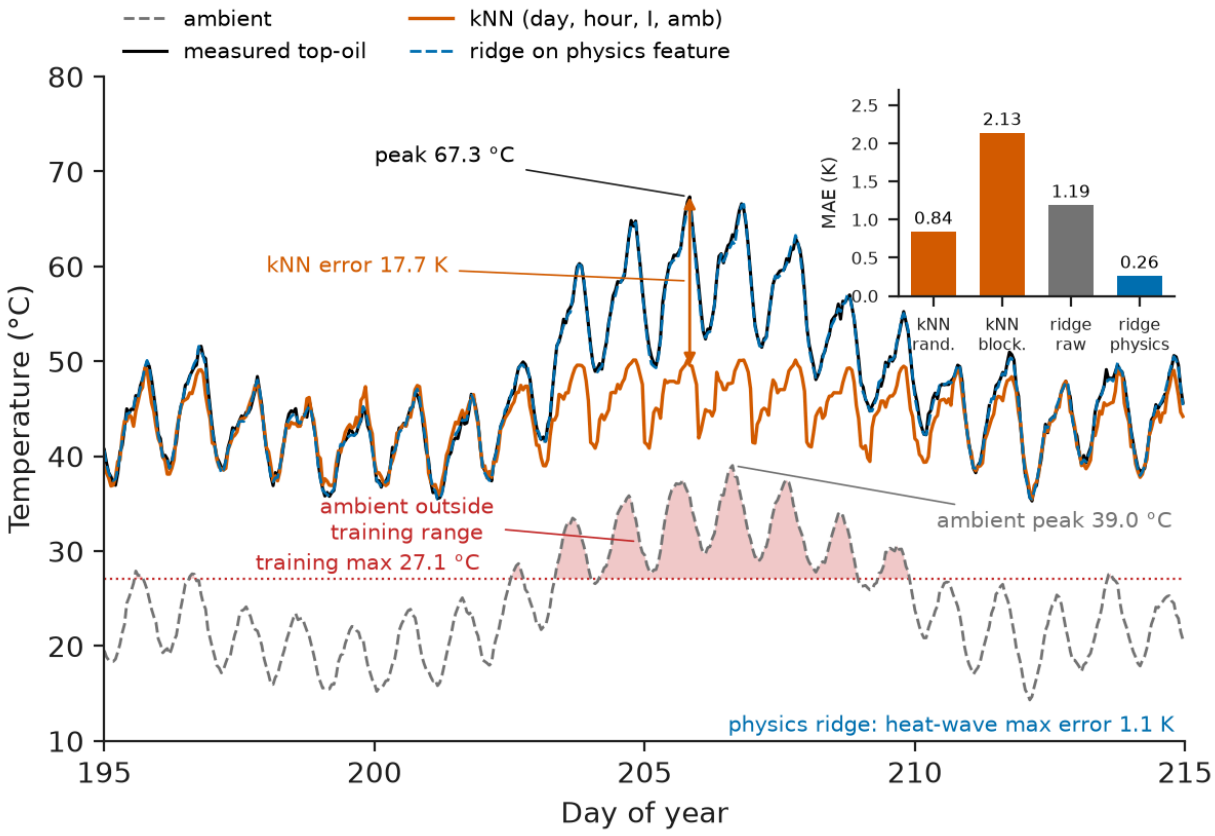


Figure 32.2 — Measured and predicted top-oil temperature of unit 0 through the July heat wave, for a nearest-neighbour model and a physics-feature model trained on January to June.

> **Status of the principle: Data-driven models interpolate**
> *Empirical.* A model fitted to data is reliable only within the region of input space the data cover; outside it, purely data-driven models return values governed by their structure (constant for neighbours and trees, arbitrary for neural networks), while models whose structure follows the physics degrade gracefully.

32.5 Classification

Dissolved-gas analysis is the classic classification problem of transformer engineering: thermal and electrical faults decompose oil and paper into characteristic gas mixtures, and IEC 60599 interprets them through ratios such as CH_4/H_2, C_2H_2/C_2H_4 and C_2H_4/C_2H_6 (gas formation and its chemistry are high-voltage engineering, *Electrical Energy and Machines*, Ch. 24 §24.8). Listing 32.4 trains a multinomial logistic regression on those log-ratios and the log total gas for five classes, with the imbalance of a real database: 85 % normal, and 2–6 % for each fault class. The gas means are illustrative, not diagnostic limits.

```
 import numpy as np
from scipy.optimize import minimize
from scipy.special import logsumexp

CLASSES = ["normal", "PD", "T1 <300C", "T3 >700C", "D2 arcing"]
#              H2   CH4  C2H6  C2H4  C2H2   typical ppm per class (illustrative)
MEAN = np.array([[30, 10, 15, 5, 0.5], [300, 30, 10, 2, 0.5], [40, 120, 150, 30, 0.5],
                 [80, 200, 60, 400, 5], [400, 80, 10, 150, 200]])
COUNT = [1700, 80, 120, 60, 40]                      # imbalanced, as in a real DGA database

def samples(seed):
    rng = np.random.default_rng(seed)
    y = np.repeat(np.arange(5), COUNT)
    gas = MEAN[y] * rng.lognormal(0, 0.6, (y.size, 5)) * rng.lognormal(0, 0.8, (y.size, 1))
    H2, CH4, C2H6, C2H4, C2H2 = gas.T
    X = np.column_stack([np.log(CH4 / H2), np.log(C2H2 / C2H4), np.log(C2H4 / C2H6),
                         np.log(gas.sum(1)), np.ones(y.size)])     # IEC 60599 ratios + total
    return X, y

def train(X, y, balanced):
    w_s = (y.size / (5 * np.bincount(y)))[y] if balanced else np.ones(y.size)
    def loss(W):
        Z = X @ W.reshape(5, 5).T
        return -np.sum(w_s * (Z[np.arange(y.size), y] - logsumexp(Z, axis=1))) + 0.1 *
np.sum(W**2)
    return minimize(loss, np.zeros(25), method="L-BFGS-B").x.reshape(5, 5)

(Xtr, ytr), (Xte, yte) = samples(1), samples(2)
print("always 'normal' : accuracy", round(np.mean(yte == 0), 3))
for balanced in (False, True):
    pred = np.argmax(Xte @ train(Xtr, ytr, balanced).T, axis=1)
    recall = [np.mean(pred[yte == k] == k) for k in range(5)]
    print(f"logistic, {'class-weighted' if balanced else 'unweighted   '}: accuracy {np.mean(pred
== yte):.3f},"
          f" recall per class {np.round(recall, 2)}, balanced accuracy {np.mean(recall):.3f}")
```

```
 always 'normal' : accuracy 0.85
logistic, unweighted   : accuracy 0.962, recall per class [0.99 0.52 0.9  0.95 1.  ], balanced
accuracy 0.872
logistic, class-weighted: accuracy 0.869, recall per class [0.86 0.89 0.93 1.   1.  ], balanced
accuracy 0.935
```

Listing 32.4 — DGA fault classification with imbalanced classes: accuracy against per-class recall, unweighted and class-weighted.

A classifier that always answers "normal" is 85 % accurate and useless. The unweighted model is 96.2 % accurate and still misses half the partial-discharge cases, because partial discharge is rare and its gas pattern overlaps the normal one at low total gas. Weighting the classes inversely to their frequency raises partial-discharge recall from 0.52 to 0.89 and costs normal-class recall, 0.99 to 0.86: 14 % of healthy units would be flagged for a resample.

Worked example — choosing the operating point by cost. Take 1000 units with the test-set mix (850 normal, 40 PD, 60 T1, 30 T3, 20 D2), \$150 for an unnecessary resample and \$40,000 for a missed incipient fault, and count every fault-class error as a miss (pessimistic). Unweighted: $0.01 \times 850 = 8.5$ false alarms (\$1,275) and $0.48 \times 40 + 0.10 \times 60 + 0.05 \times 30 = 26.7$ misses (\$1.07 M). Weighted: 119 false alarms (\$17,850) and $0.11 \times 40 + 0.07 \times 60 = 8.6$ misses (\$0.34 M). About \$16,600 more in oil samples saves about \$0.72 M. The operating point is a cost decision, not a property of the algorithm; and since the weighted model meets R5 with little margin, its output is a recommendation to resample, not an automatic action.

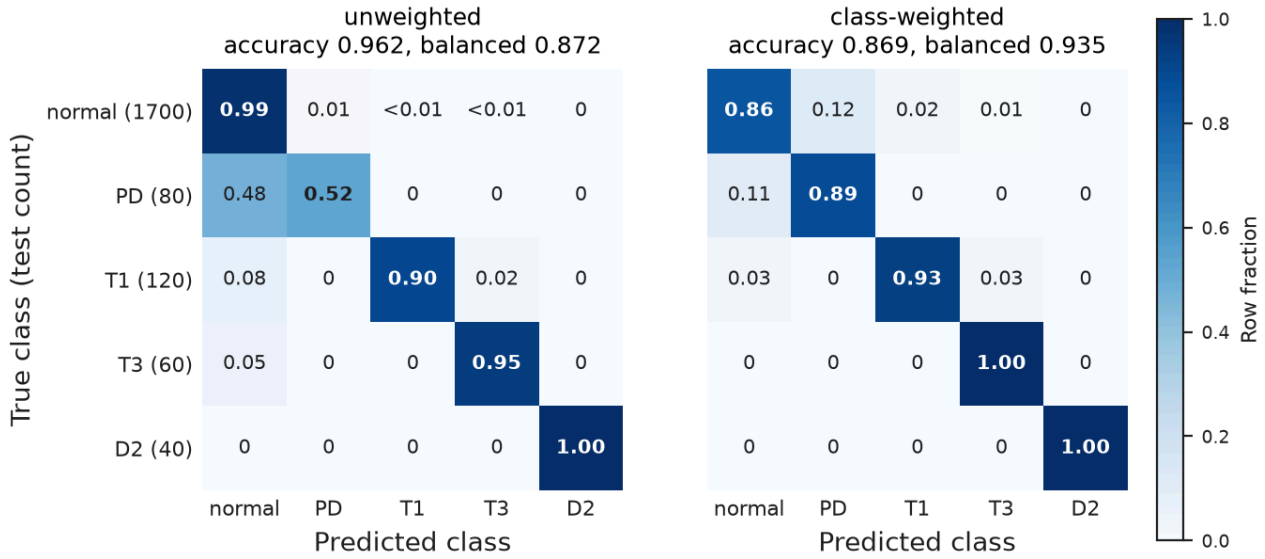


Figure 32.3 — Row-normalised confusion matrices of the DGA classifier on the independent test set, unweighted and class-weighted.

> **Status of the principle: Accuracy is not a metric for rare faults**
> *Derived.* With a class prior of 85 %, the trivial classifier scores 85 %, so accuracy is dominated by the majority class; per-class recall and precision, or balanced accuracy, measure what the classifier adds, and the decision threshold must come from the costs of each error type.

Real DGA databases contain hundreds, not thousands, of confirmed fault cases, and their labels come from teardown reports of variable quality. With so few labels, interpretable models on established ratios (or the Duval triangle and pentagon methods) remain the industrial standard, and machine learning is used to rank units for attention rather than to diagnose.

32.6 Time-series forecasting

Day-ahead load forecasts let the utility schedule switching, decide whether a transformer can carry a planned transfer, and anticipate thermal stress. Listing 32.5 forecasts tomorrow's 24 hourly loads of unit 5 at midnight, using only information available then: today's load, last week's load, tomorrow's weather forecast (simulated with 1.5 K error) and the calendar.

```
 import numpy as np
import fleet

amb, units = fleet.make_fleet()
I = units[5]["I"].reshape(365, 24)                    # hourly load, p.u., one row per day
A = amb.reshape(365, 24)
rng = np.random.default_rng(1)
A_fc = A + rng.normal(0, 1.5, A.shape)                # day-ahead weather forecast, 1.5 K error

def features(d):                                      # forecast made at midnight for day d
    hour_onehot = np.eye(24)
    weekend = np.full((24, 1), float(d % 7 >= 5))
    return np.column_stack([hour_onehot, I[d - 1], I[d - 7], A_fc[d], weekend])

days = np.arange(7, 365)
train, test = days[days < 273], days[days >= 273]     # fit Jan-Sep, test Oct-Dec
X = np.vstack([features(d) for d in train]); y = I[train].ravel()
w = np.linalg.solve(X.T @ X + 0.1 * np.eye(X.shape[1]), X.T @ y)

forecasts = {"persistence (yesterday)": lambda d: I[d - 1],
             "seasonal naive (last week)": lambda d: I[d - 7],
             "ridge (lags, weather, calendar)": lambda d: features(d) @ w}
for name, f in forecasts.items():
    err = np.array([f(d) - I[d] for d in test])
    peak_err = np.array([f(d).max() - I[d].max() for d in test])
    print(f"{name:32s} MAE {100 * np.abs(err).mean():.2f} % of rating, "
          f"MAPE {100 * np.mean(np.abs(err) / I[test]):.1f} %, daily-peak MAE {100 *
np.abs(peak_err).mean():.2f} %")
```

```
 persistence (yesterday)          MAE 3.27 % of rating, MAPE 6.5 %, daily-peak MAE 3.53 %
seasonal naive (last week)       MAE 2.40 % of rating, MAPE 4.7 %, daily-peak MAE 2.57 %
ridge (lags, weather, calendar)  MAE 2.27 % of rating, MAPE 4.4 %, daily-peak MAE 2.56 %
```

Listing 32.5 — Day-ahead load forecasting against two naive baselines, on a hold-out quarter.

The ridge forecaster beats the seasonal-naive baseline by $(2.40 - 2.27)/2.40 = 5.4\ \%$, just meeting R4, and does not improve the daily peak at all. That modest gain is typical for a single distribution transformer, whose load is dominated by the random behaviour of a few dozen customers; forecasts improve sharply with aggregation (a feeder or a substation), and a model that does not beat the seasonal-naive baseline on a hold-out period is not worth deploying, however sophisticated. Persistence is worse than the weekly baseline because Mondays follow Sundays. Forecast errors of this size, and their propagation to a predicted hot spot, are what Chapter 25 budgets for in energy management; the role of forecasts in grid operation is *Electrical Energy and Machines*, Ch. 26 §26.8.

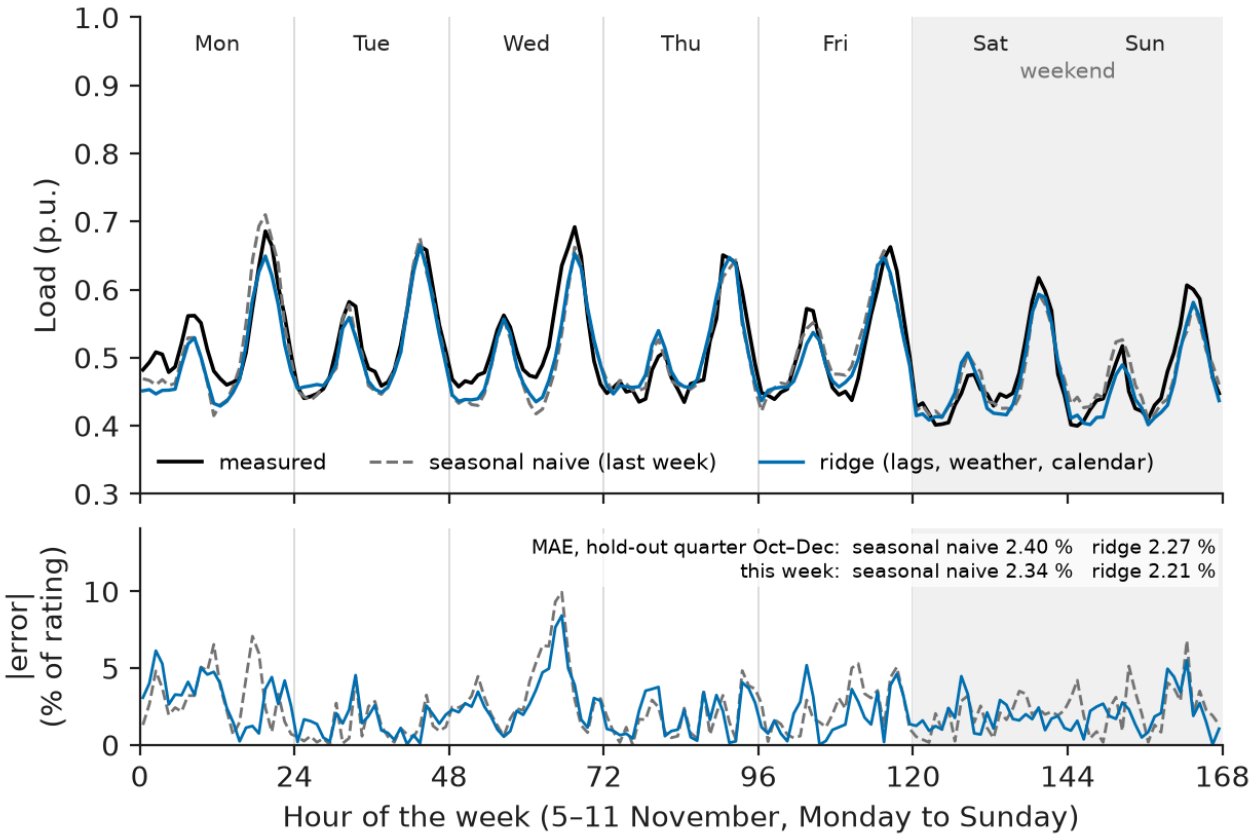


Figure 32.4 — One test week of day-ahead load forecasts for unit 5: measured load, seasonal-naive forecast and ridge forecast.

32.7 Anomaly detection

A transformer whose cooling degrades (blocked radiators, a failed fan on ONAF units, low oil) runs hotter at the same load and ambient. With a model of what the oil temperature *should* be, the residual (measured minus predicted) becomes a fault signal. Listing 32.6 fits each unit's grey-box model (§32.9) on January to May, sets the residual scale on June, and runs a one-sided CUSUM on the daily median residual from July onwards. A CUSUM accumulates small persistent deviations and ignores isolated ones, which is exactly the signature of slow cooling degradation.

```
 import numpy as np
from scipy.optimize import least_squares
import fleet

amb, units = fleet.make_fleet()
fit_days, cal_days = fleet.day < 151, (fleet.day >= 151) & (fleet.day < 181)
alarms = []
for u in units:
    ok = ~np.isnan(u["theta"])
    model = lambda p: amb + fleet.top_oil_rise(u["I"], *p)
    p = least_squares(lambda p: (model(p) - u["theta"])[ok & fit_days], x0=[50, 6, 3],
                      bounds=([20, 1, 0.5], [90, 15, 10]), loss="soft_l1").x   # robust to spikes
    r = np.where(ok, u["theta"] - model(p), np.nan).reshape(365, 24)
    daily = np.nanmedian(r, axis=1)                    # one residual per day, robust
    sigma = max(np.std(daily[151:181]), 0.5)           # calibration spread, 0.5 K floor
    S = 0.0
    for d in range(181, 365):                          # one-sided CUSUM: oil hotter than model
        S = max(0.0, S + daily[d] / sigma - 0.5)
        if S > 5.0:
            alarms.append((u["id"], d, round(float(daily[d]), 2)))
            break
print("alarms (unit, day, residual K):", alarms)
print("units with injected faults: 20-25 (current sensor -20 % from day 250), 31 (cooling, day
250)")
```

```
 alarms (unit, day, residual K): [(20, 251, 2.14), (21, 250, 3.04), (22, 250, 2.93), (23, 250,
3.72), (24, 251, 2.07), (25, 250, 2.99), (31, 250, 3.34)]
units with injected faults: 20-25 (current sensor -20 % from day 250), 31 (cooling, day 250)
```

Listing 32.6 — Residual-based anomaly detection across the fleet with a robust grey-box fit and a one-sided CUSUM.

The cooling fault on unit 31 (an 18 % higher rated top-oil rise, about +3.3 K at its load) is detected on the day it starts, and the 33 healthy units, including the stuck-sensor and spiky units, raise no alarm over six months: zero false alarms in 198 unit-months, inside R3. But units 20–25 alarm too, with residuals of 2–4 K, and they are perfectly healthy. Their CT ratio error makes the measured current 20 % low, so the model predicts a cooler transformer than the real one, and the residual looks exactly like failing cooling. An anomaly detector reports that something has changed, not what. That is why the data-quality gate of Listing 32.2, which identifies those six units as calibration steps from the fleet comparison, must run first and suppress or relabel the alarm. In the opening story the gate did not exist, and six crews went to six healthy transformers.

The 0.5 K floor on the residual scale is a design decision of the same kind as the 5 K spike floor: without it, the very small spread of residuals in June (a few hundredths of a kelvin on synthetic data) made the CUSUM alarm on 0.1–0.25 K drifts on eight units in a first version.

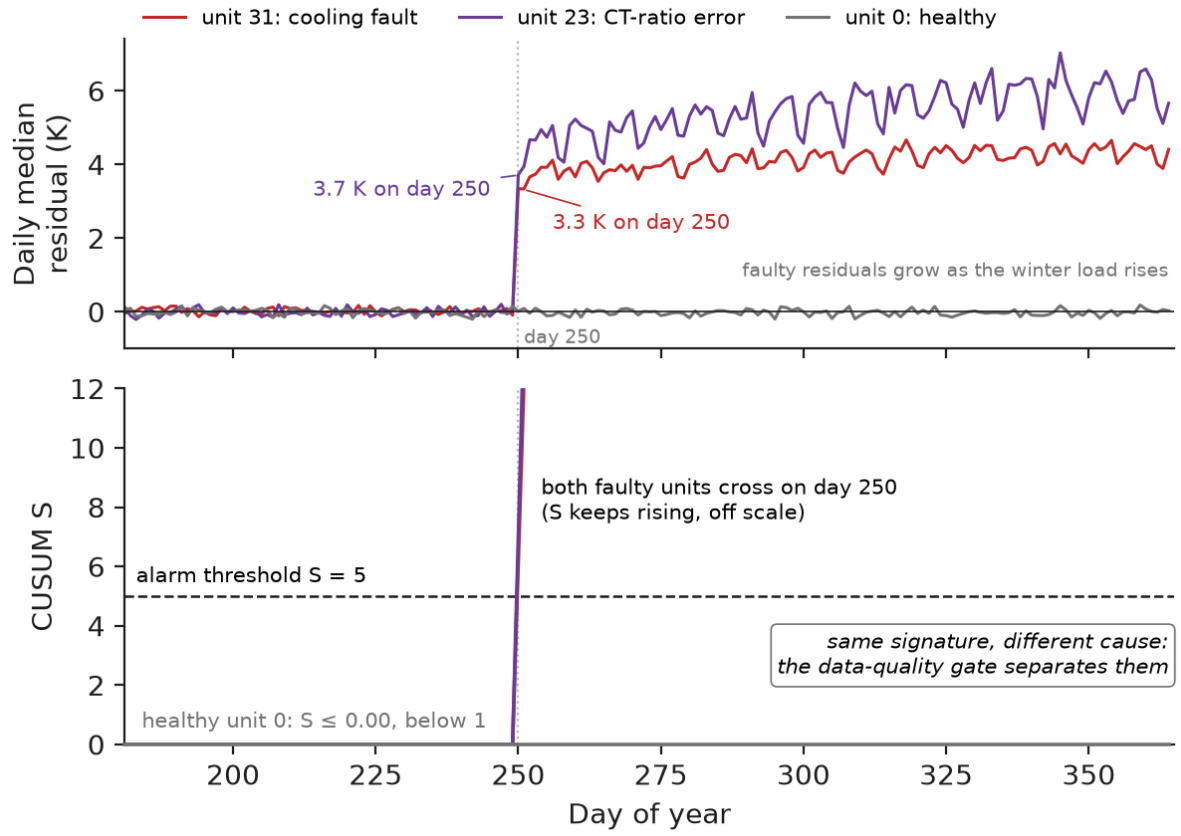


Figure 32.5 — CUSUM statistic of the daily top-oil residual for a cooling fault, a CT-ratio error and a healthy unit.

> **Status of the principle: Residuals need a reference you trust**
> *Derived.* A residual detector inherits every error of its reference model and of its inputs, so input faults and plant faults produce the same signature; isolation requires a second, independent check on the inputs.

32.8 Surrogate models

Planning engineers need to screen every transformer in the fleet against every day of a hot-summer scenario for peak hot-spot temperature and insulation ageing: $1200 \times 365 = 438\,000$ evaluations. The reference model, the IEC 60076-7 top-oil and winding ODEs integrated over three days of a daily load cycle to reach a periodic state, takes 36 ms per case, so 4.4 hours for one scenario. A **surrogate** is a cheap function fitted to a modest number of reference runs. Listing 32.7 fits a radial-basis-function interpolant to 60 runs placed by a Latin hypercube over peak load (0.6–1.6 p.u.), mean ambient (0–40 °C) and rated top-oil rise (45–60 K), and tests it on 300 independent random points. The same technique replaces the heatsink field model of Chapter 30 inside its optimiser.

```
 import time
import numpy as np
from scipy.integrate import solve_ivp
from scipy.interpolate import RBFInterpolator
from scipy.stats import qmc

def hot_spot_day(K_peak, amb_mean, d_or, R=6.0, n=0.8, Hgr=23.0, y=1.6, tau_o=3.0, tau_w=0.12):
    """Reference model: IEC 60076-7 style top-oil and winding ODEs over 3 days of a daily cycle.
    Returns the peak hot-spot temperature (C) and log10 of the relative ageing on day 3."""
    K = lambda h: K_peak * (0.5 + 0.5 * np.exp(-(((h % 24) - 19) / 2.5) ** 2))
    amb = lambda h: amb_mean + 5 * np.sin(2 * np.pi * ((h % 24) - 9) / 24)
    def f(h, x):
        k = K(h)
        return [(d_or * ((1 + R * k**2) / (1 + R)) ** n - x[0]) / tau_o, (Hgr * k**y - x[1]) /
tau_w]
    sol = solve_ivp(f, (0, 72), [0.0, 0.0], method="LSODA", max_step=0.05, dense_output=True)
    h = np.linspace(48, 72, 1441)
    theta_h = amb(h) + sol.sol(h).sum(axis=0)
    ageing = np.trapezoid(2 ** ((theta_h - 98) / 6), h) / 24     # non-upgraded paper, IEC 60076-7
    return theta_h.max(), np.log10(ageing)

lo, hi = [0.6, 0.0, 45.0], [1.6, 40.0, 60.0]              # K_peak, ambient mean (C), rated rise
X_train = qmc.scale(qmc.LatinHypercube(d=3, seed=4).random(60), lo, hi)
X_test = qmc.scale(np.random.default_rng(5).random((300, 3)), lo, hi)
t0 = time.perf_counter(); Y_train = np.array([hot_spot_day(*x) for x in X_train])
t_ref = (time.perf_counter() - t0) / len(X_train)
Y_test = np.array([hot_spot_day(*x) for x in X_test])
scale = np.array(hi) - np.array(lo)
sur = RBFInterpolator(X_train / scale, Y_train, kernel="thin_plate_spline", degree=1)
t0 = time.perf_counter(); Y_hat = sur(X_test / scale); t_sur = (time.perf_counter() - t0) /
len(X_test)
err = Y_hat - Y_test
print(f"reference {1e3 * t_ref:.0f} ms per case, surrogate {1e6 * t_sur:.0f} us per case")
print(f"peak hot-spot error: mean abs {np.abs(err[:, 0]).mean():.2f} K, max {np.abs(err[:,
0]).max():.2f} K")
print(f"daily ageing error : median {100 * np.median(np.abs(10 ** err[:, 1] - 1)):.1f} %, "
      f"max {100 * np.abs(10 ** err[:, 1] - 1).max():.0f} %")
```

```
 reference 36 ms per case, surrogate 5 us per case
peak hot-spot error: mean abs 0.26 K, max 2.85 K
daily ageing error : median 1.9 %, max 32 %
```

Listing 32.7 — A radial-basis-function surrogate of a transformer hot-spot and ageing model, trained on 60 Latin-hypercube runs and tested on 300 independent points.

Worked example — what the surrogate buys and what it costs. At 5 µs per case the 438 000-case scenario takes 2.2 s instead of 4.4 h, a factor of about 7000, meeting R6. Training cost 60 reference runs (2.2 s) plus 300 test runs (11 s), which are not optional: without them there is no error statement. Over the test set the peak hot-spot error is 0.26 K on average and 2.85 K at worst, inside R6's 3 K. The worst case sits at a corner of the box (peak load 0.61 p.u., ambient 37 °C, rated rise 46 K), where the interpolant has fewest

neighbours. Ageing, which is exponential in temperature (it doubles every 6 K), is predicted in logarithm, and a 2.85 K error becomes a 32 % ageing error at that corner, against a median of 1.9 %. Doubling the training runs to 120 cuts the mean hot-spot error to 0.08 K and the maximum to 1.23 K; halving to 30 raises them to 0.42 K and 5.0 K. The surrogate is used only inside its box; a query outside it is sent to the reference model.

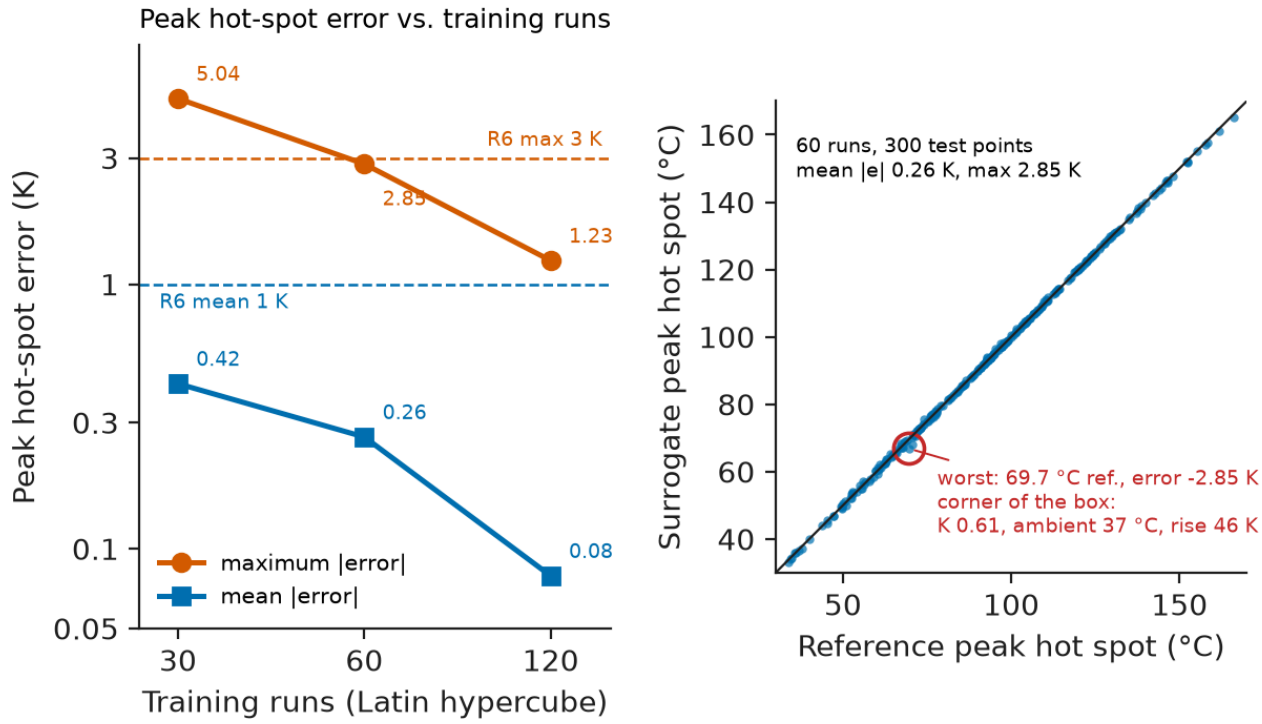


Figure 32.6 — Surrogate accuracy against the number of reference runs, and parity of the 60-run surrogate.

> **Status of the principle: Surrogate error must be measured, not assumed**
> *Empirical.* Interpolating surrogates (radial basis functions, Gaussian processes, polynomial chaos, neural networks) converge as training runs fill the input space, roughly as a power of their spacing, but their error is largest at boundaries and in regions of high curvature; only an independent test set inside the stated box bounds it.

32.9 Physics-informed modeling

The physics feature of §32.4 fixed R and τ at nominal values and let regression fit the scale. A **grey-box model** goes one step further: it keeps the IEC 60076-7 structure and fits its physical parameters (rated top-oil rise $\Delta\theta_{or}$, loss ratio R, oil time constant τ_o) for each transformer by nonlinear least squares. The parameters are interpretable (a rated rise of 51 K for a unit whose nameplate says 50 K is plausible; 80 K is a fault or a bad sensor), few (three per unit), and the model extrapolates as far as the physics holds. Listing 32.8 fits unit 0 on January to May.

```
 import numpy as np
from scipy.optimize import least_squares
import fleet

amb, units = fleet.make_fleet()
u = units[0]
ok = ~np.isnan(u["theta"])
fit_set = ok & (fleet.day < 151)                       # fit the physics parameters
cal_set = ok & (fleet.day >= 151) & (fleet.day < 181)  # calibrate the interval width
test = ok & (fleet.day >= 181) & (fleet.day < 273)     # Jul-Sep, includes the heat wave

def predict(p):                                        # grey box: IEC structure, fitted numbers
    return amb + fleet.top_oil_rise(u["I"], *p)

res = least_squares(lambda p: (predict(p) - u["theta"])[fit_set],
                    x0=[50.0, 6.0, 3.0], bounds=([20, 1, 0.5], [90, 15, 10]))
J = res.jac
cov = np.linalg.inv(J.T @ J) * np.mean(res.fun ** 2)   # linearised parameter covariance
print("fitted d_or, R, tau_h :", np.round(res.x, 2), "+/-", np.round(np.sqrt(np.diag(cov)), 2))
print("true   d_or, R, tau_h :", np.round(list(u["params"].values()), 2))

# Split-conformal 90 % interval: quantile of absolute residuals on unseen calibration data.
r_cal = np.sort(np.abs(predict(res.x) - u["theta"])[cal_set])
q = r_cal[int(np.ceil(0.9 * (r_cal.size + 1))) - 1]
err = (predict(res.x) - u["theta"])[test]
heat = ((fleet.day >= 200) & (fleet.day < 212))[test]
print(f"interval +/-{q:.2f} K; coverage on Jul-Sep {np.mean(np.abs(err) <= q):.3f}, "
      f"in heat wave {np.mean(np.abs(err[heat]) <= q):.3f}; MAE {np.abs(err).mean():.2f} K")
```

```
 fitted d_or, R, tau_h : [50.72  7.03  3.24] +/- [0.04 0.04 0.01]
true   d_or, R, tau_h : [50.69  7.02  3.22]
interval +/-0.52 K; coverage on Jul-Sep 0.906, in heat wave 0.913; MAE 0.25 K
```

Listing 32.8 — Grey-box model with fitted physical parameters, their linearised standard errors, and a split-conformal prediction interval.

The fitted parameters agree with the true ones to within their standard errors, and the model trained on spring data predicts the heat wave as well as the spring. On a real transformer the agreement would be looser (solar gain, wind, oil-pocket position and an imperfect model structure add unexplained error, and the standard errors computed from the Jacobian assume independent residuals, which hourly thermal data are not), but the extrapolation behaviour carries over, because it comes from the structure.

Physics-informed approaches range from **physics features** (§32.4) through **grey-box fitting** (this section) and **residual learning** (a data-driven model of what the grey box misses, such as solar gain) to **physics-informed neural networks**, which add the governing equation's residual to the training loss. For lumped systems with known structure (transformer and motor thermal models, battery equivalent circuits) the grey box is smaller, faster, easier to audit and at least as accurate; physics-informed networks earn their cost for distributed problems with sparse sensors. Parameter identifiability and calibration with full uncertainty are the subject of Chapter 33.

> **Status of the principle: IEC 60076-7 exponential thermal model**
> *Approximate*. Treats the oil as one well-mixed thermal mass with a single time constant and the winding gradient as a power of load; valid for ONAN and ONAF distribution transformers under slow load changes, and less accurate for fast overloads, OD cooling and units with significant solar gain.

32.10 Model evaluation

Table 32.4 collects the evaluation of every model in the chapter against its requirement. Four practices made those numbers trustworthy:

- **Split by time (and by unit) as the model will be used.** A model that will predict next summer is tested on a later period it never saw; a model that will be applied to new transformers is tested on held-out units. Autocorrelated data under a random split leak information between neighbouring samples (The Mathematics Universe, Ch. 21 §21.3).
- **Always report a naive baseline.** Seasonal naive for forecasts, "always normal" for classifiers, a nominal-parameter physics model for regression. A model is judged by its improvement over the baseline.
- **Use metrics with engineering meaning.** Errors in kelvin and per cent of rating; maximum error in the conditions that matter (the heat wave); per-class recall for rare faults; daily-peak error for planning.
- **Freeze the test set.** It is used once per model version; tuning on it turns it into training data.

Table 32.4 — Verification of the analytics against Table 32.1.

Req.	Test	Limit	Result	Pass/fail
R1	Blocked hold-out Jul–Sep, unit 0 (Listing 32.3, physics feature; Listing 32.8, grey box)	MAE ≤ 1.0 K; max ≤ 3 K	0.26 K, 1.1 K (heat wave); grey box 0.25 K	Pass
R1	Same test, nearest-neighbour model of the opening	MAE ≤ 1.0 K; max ≤ 3 K	2.13 K; 17.7 K	Fail (rejected)
R2	Split-conformal 90 % interval (Listing 32.8)	Coverage 0.85–0.95	0.906 overall, 0.913 in heat wave	Pass
R3	Fleet replay Jul–Dec (Listing 32.6)	Detect in ≤ 7 days; ≤ 1 false alarm per 100 unit-months	Day 0; 0 false alarms in 198 unit-months (after data-quality gate)	Pass
R4	Hold-out Oct–Dec (Listing 32.5)	≥ 5 % better than seasonal naive	5.4 % better	Pass (marginal)
R5	Independent test set (Listing 32.4, weighted)	Recall ≥ 0.85 per fault class; ≤ 15 % false alarms	Min fault recall 0.89; 14 % false alarms	Pass (marginal)
R6	300 random points in the box (Listing 32.7)	Mean ≤ 1 K, max ≤ 3 K; ≥ 1000× faster	0.26 K, 2.85 K; about 7000×	Pass
R7	Deployment budget (Table 32.5)	≤ 64 kB per unit; refit ≤ 10 min	32 bytes per unit; about 20 s per gateway	Pass

> **Status of the principle: The test must resemble the use**
> *Empirical.* A model's measured error predicts its field error only when the test data are drawn the way field data will arrive (later in time, from new units, under new conditions); random splits of autocorrelated or grouped data systematically underestimate field error, often by a factor of two or more.

32.11 Uncertainty

A prediction without an uncertainty cannot be compared with a limit. Two kinds matter in engineering ML. **Aleatoric** uncertainty is the irreducible scatter of the data (sensor noise, unmodelled weather), and it sets the width of a prediction interval. **Epistemic** uncertainty is the model's ignorance (too few data, inputs outside the training domain, wrong structure), and it is the kind that the nearest-neighbour model of the opening had in the heat wave and did not report.

Worked example — the conformal half-width. Listing 32.8 builds a **split-conformal** interval: it sorts the 715 absolute residuals of June, data not used for fitting, and takes the $\lceil 0.9 \times 716 \rceil = 645$th smallest, ±0.52 K, as the half-width. Under the assumption that future residuals are exchangeable with the calibration residuals, the interval covers at least 90 % of future values, whatever the model. The check on July–September gives 0.906 overall and 0.913 in the heat wave, meeting R2. The assumption is the catch: conformal intervals are honest for the grey-box model because its residuals in the heat wave look like its residuals in June; for the nearest-neighbour model, the June residuals are small and the heat-wave residuals

are 17 K, so a conformal interval would have claimed ±1 K and covered almost nothing. Conformal prediction calibrates aleatoric scatter; it does not detect extrapolation.

For epistemic uncertainty, the practical tools are a **domain check** (is this input inside the convex hull, or the per-feature range, of the training data? the July ambient of 39 °C was 12 K outside), the **parameter covariance** of a grey-box fit (±0.04 K on the rated rise in Listing 32.8, propagated to predictions), **ensembles** (models trained on resampled data disagree where data are thin), and Bayesian calibration (Chapter 33). A deployed model reports both an interval and an in-domain flag, and the alarm logic treats an out-of-domain prediction as "unknown", not as "normal".

> **Status of the principle: Split-conformal coverage**
> *Derived.* If calibration and future residuals are exchangeable, the $\lceil 0.9(n+1) \rceil$-th smallest of n calibration residuals bounds a new residual with probability at least 0.9; the guarantee is marginal (on average over conditions) and fails under distribution shift.

32.12 Deployment constraints

The gateway budget

The models run on a substation gateway: a quad-core Cortex-A53 industrial computer with 1 GB of RAM, serving about 20 transformers, with intermittent backhaul to the data centre. Inference engineering on such targets (quantisation, runtimes, accelerators) is Chapter 12; here the budget is set by the choice of model class.

Table 32.5 — Deployment budget per gateway (20 transformers; desktop times measured, Cortex-A53 times estimated at 10 times slower).

Item	Budget	Grey-box choice	Nearest-neighbour alternative	Margin (grey box)
Model memory per unit	64 kB	3 parameters + 1 state: 32 bytes	About 140 kB (4400 training rows × 4 features × 8 bytes)	> 99 %
Hourly prediction, 20 units	1 s	20 filter updates, under 1 ms	20 tree queries, about 2 ms	> 99 %
Daily refit, 20 units	10 min	0.1 s desktop, about 1 s on A53, per unit: about 20 s	Not needed (stores data), but cannot extrapolate	97 %
Daily load forecast, 20 units	1 min	28 weights per unit, under 1 ms	—	> 99 %
Model and data transfer per month	10 MB	20 × 32 bytes + residual summaries	2.8 MB of training data per update	> 99 %

The grey-box choice made in Table 32.2 for extrapolation also wins every deployment line by orders of magnitude. That is common for physics-structured models and is a reason to prefer them on embedded targets even when a data-driven model is slightly more accurate in range.

Operating the model

Deployment adds obligations that training does not have: **versioning** (each deployed model carries the hash of its training-data snapshot and code, R8, in the registry pattern of Chapter 31); **monitoring of inputs** (the data-quality gate of §32.2 on every batch, and a distribution check of each feature against its training range); **monitoring of outputs** (residual statistics per unit, alarm rates per week across the fleet); **retraining policy** (grey-box parameters refitted monthly on a rolling window, with a parameter jump larger than three standard errors treated as an event to investigate, not silently absorbed); and **fallback** (fixed top-oil thresholds from the loading guide when the model or its inputs are unavailable).

Failure analysis

Table 32.6 — FMEA of the transformer analytics.

Failure mode	Effect	Cause	Detection	Mitigation
Optimistic validation	0.84 K claimed, 17.7 K in the field	Random split of autocorrelated data; day-of-year feature	Blocked hold-out	Time- and unit-blocked evaluation; feature review for leakage
Extrapolation beyond training range	Missed overloads, spurious alarms in heat wave	Data-driven model without physics; training data from one season	Domain check on inputs	Grey-box structure; out-of-domain flag treated as unknown
Sensor calibration step	Six false cooling alarms	CT ratio changed by firmware update	Fleet-relative step detection	Data-quality gate before inference; firmware change log linked to data
Stuck or spiking sensor	Biased fit or false alarms	Sensor or transmission fault	Zero-spread and Hampel checks	Quarantine; robust loss in fitting
Class imbalance	Half of partial-discharge cases missed	85 % normal class dominates training	Per-class recall	Class weighting; threshold set by error costs
Over-tight residual scale	Alarms on 0.1 K drifts	Calibration spread smaller than model resolution	Alarm rate on healthy units	Engineering floor on residual scale
Surrogate used outside its box	Unbounded error	Scenario beyond trained ranges	Input-range check	Route out-of-box queries to the reference model
Silent parameter drift absorbed by retraining	Real degradation learnt as normal	Rolling refit adapts to the fault	Parameter-jump monitor	Treat jumps as events; keep a frozen baseline model

Revision of the analytics

After the opening's failures the analytics were rebuilt, and almost none of the changes concerned algorithms: a data-quality gate linked to the firmware change log; blocked evaluation with frozen test periods and baselines; conformal intervals plus an in-domain flag; cost-based class weighting; a model registry. The one algorithmic change, from nearest neighbours to a three-parameter physical model, made the system smaller, faster and more accurate where it mattered.

What remains open

Open question	Where it is answered
How are trained models quantised, compiled and run on microcontrollers and edge accelerators?	Chapter 12
How are spectral, envelope and event features extracted from vibration, current and audio signals?	Chapter 27
How is a grey-box model calibrated with full uncertainty and kept synchronised with the asset as a digital twin?	Chapter 33
How do load and generation forecasts feed energy management and dispatch?	Chapter 25
How are models, data snapshots and code versioned together in an engineering tool?	Chapter 31 and Chapter 29
The statistics, optimisation and random-process theory beneath these methods	*The Mathematics Universe*, Ch. 20–21 and Ch. 23–24

Next: Chapter 33 joins simulation, measurement and models into digital twins: calibrated, validated models that run alongside the physical system and carry their uncertainty with them.

Chapter 33 — Engineering Simulation and Digital Twins

Part VIII · Engineering Software and Computational Tools

A regional water utility runs twelve identical raw-water pump sets: a 37 kW, 4-pole, 400 V TEFC induction motor on a variable-frequency drive (VFD), driving a centrifugal pump into a 500 m rising main. On a 38 °C July afternoon one of them tripped on its winding thermistor relay and a reservoir ran low for four hours. The utility's dashboard "digital twin", a nameplate thermal model fed with drive current, showed 118 °C at the moment of the trip; the embedded winding RTD of a sister motor in a similar state read 131 °C. Its parameters were catalogue values never calibrated against these motors; it knew nothing about the fan cowl, clogged with cottonwood fluff over three weeks, which had raised the frame-to-air thermal resistance by about a third; and its ambient input came from a shaded louvre, 4 K cooler than the air the motor breathed. A twin that is not calibrated, does not track the parameters that drift, and does not say how uncertain it is will be confidently wrong on the one afternoon it matters.

> **How is a physics model turned into a twin that tracks one particular machine: calibrated, fed with live data, estimating its own drifting parameters and validated against limits written down in advance?**
> **How uncertain is a twin's prediction, and how is that uncertainty used to make a design or operating decision that will not be regretted?**

Concept	Engineering destination
Physical-system models	Electro-thermal-hydraulic model of the pump set
Numerical simulation	Stiff solvers; fast what-if runs
Model calibration	Thermal parameters fitted to a heat run
Co-simulation	Drive, thermal, hydraulic and PLC models exchanged as FMUs
Digital-twin architectures	Gateway, estimator, model service, historian, apps
Real-time data integration	Timestamps, alignment, latency budget, stale-data handling
Parameter estimation	Online tracking of cooling degradation
Validation	Hold-out test against winding RTDs
Uncertainty quantification	Monte Carlo prediction intervals for hot-spot temperature
Simulation-based design	Overspeed, cleaning interval, motor upgrade

Roadmap. What the model contains (33.1) → how it is solved (33.2) → calibrated to one machine (33.3) → split across tools and recombined (33.4) → embedded in a twin architecture (33.5) → fed with live data (33.6) → kept current by online estimation (33.7) → validated (33.8) → given error bars (33.9) → used to decide (33.10) → failure analysis.

33.1 Physical-system models

> **Boundary.**
> Chapter 3 owns the modelling workflow (assumptions, equivalent circuits, sensitivity, validation as a method) and Chapter 30 owns solvers, sweeps and numerical verification as tools. Chapter 17 owns system identification of robots. This chapter applies those to one twin. Lumped effort–flow modelling is in *Engineering Physics*, Ch. 17; the motor's thermal network and insulation limits are in *Electrical Energy and Machines*, Ch. 9 §9.3; pump-type load characteristics are in *Electrical Energy and Machines*, Ch. 10 §10.3.

What the twin must answer

A model is sized by the questions it must answer, not by the physics available. The utility's operations and maintenance staff wrote down five, and they became the twin's requirements.

ID	Requirement	Target	Verification
R1	Estimate winding hot-spot temperature of every unit	Average winding ≤ 3 K RMS, ≤ 6 K max	Hold-out against lead-unit RTDs (§33.8)
R2	Predict hot-spot 30 min ahead with an honest interval	95 % interval covers ≥ 90 % of observations; width ≤ 12 K	Coverage test on hold-out data
R3	Detect cooling degradation	+15 % frame-to-air thermal resistance flagged within 72 h; ≤ 1 false alarm per unit-year	Seeded-fault replay; false-alarm count
R4	Stay current	Data age at state estimate ≤ 5 s; dashboard ≤ 10 s	Latency budget (§33.6) and timestamp audit
R5	Answer what-if questions fast	24 h scenario ≤ 2 s on the edge server	Benchmark on target hardware
R6	Fail visibly	Missing data > 60 s marks outputs STALE and widens intervals	Fault injection
R7	Be reproducible	Every output traceable to model version, parameter set and input-data hash	Audit of ten random outputs
R8	Be harmless to the plant	Read-only access to the OT network; no writes to drives or PLC	IEC 62443 zone/conduit review, firewall test

R1–R3 fix the physics: hot-spot temperature needs losses and the thermal network; losses need torque, hence the pump's duty point on the system curve; cooling degradation needs the shaft fan's dependence on speed.

The model chain

The pump-set model is four coupled sub-models (Figure 33.1):

1. **Drive and shaft.** The VFD's speed loop is represented by a first-order lag, $\tau_n = 0.3$ s, from speed reference to shaft speed. Electrical dynamics (currents, PWM) are far faster than anything R1–R3

need and are left out of the thermal twin; they live in a separate electrical model used for co-simulation (§33.4).

2. **Pump and pipe.** The pump curve $H = H_0(n/n_r)^2 - k_p Q^2$ with $H_0 = 48$ m and $k_p = 1793$ s^2/m^5; the system curve $H = 20 + 2391\,Q^2$ m; a water column whose inertia gives $\mathrm{d}Q/\mathrm{d}t = (gA/L)(H_{pump} - H_{sys})$. Pump efficiency is a parabola in flow relative to the best-efficiency point, 0.78 at the peak. At rated speed (1480 rpm) the duty point is 294 m³/h at 36.0 m and 37.0 kW shaft power.
3. **Motor losses.** Rated losses of 2236 W (IE3, 94.3 %) are split into stator copper loss (1000 W, scaling with torque squared plus a 10 % magnetising share, and with copper resistance through $(234.5 + T_w)/(234.5 + 115)$), and iron, mechanical and rotor losses (1236 W) assigned to the frame.
4. **Thermal network.** Two nodes, as in the two-body motor model of *Electrical Energy and Machines* Ch. 9 §9.3: winding (C_1) coupled through R_1 to frame (C_2), frame to ambient through R_2. The shaft fan makes R_2 speed-dependent, $R_2(n) = R_2/(0.25 + 0.75\,n/n_r)$, and fouling multiplies it by a factor $f \geq 1$.

$$C_1\dot{T}_w = P_w - \frac{T_w - T_f}{R_1}, \qquad C_2\dot{T}_f = P_f + \frac{T_w - T_f}{R_1} - \frac{T_f - T_a}{f\,R_2(n)}$$

Status of the principle: Two-node motor thermal network
Approximate. Valid for average winding and frame temperature over minutes to hours with the motor running at steady or slowly varying load; it does not resolve the slot-versus-end-winding gradient, stall heating in seconds, or the standstill cooling with the fan stopped. The hot spot is taken as the average winding plus an allowance (10 K for class F, IEC 60034-1).

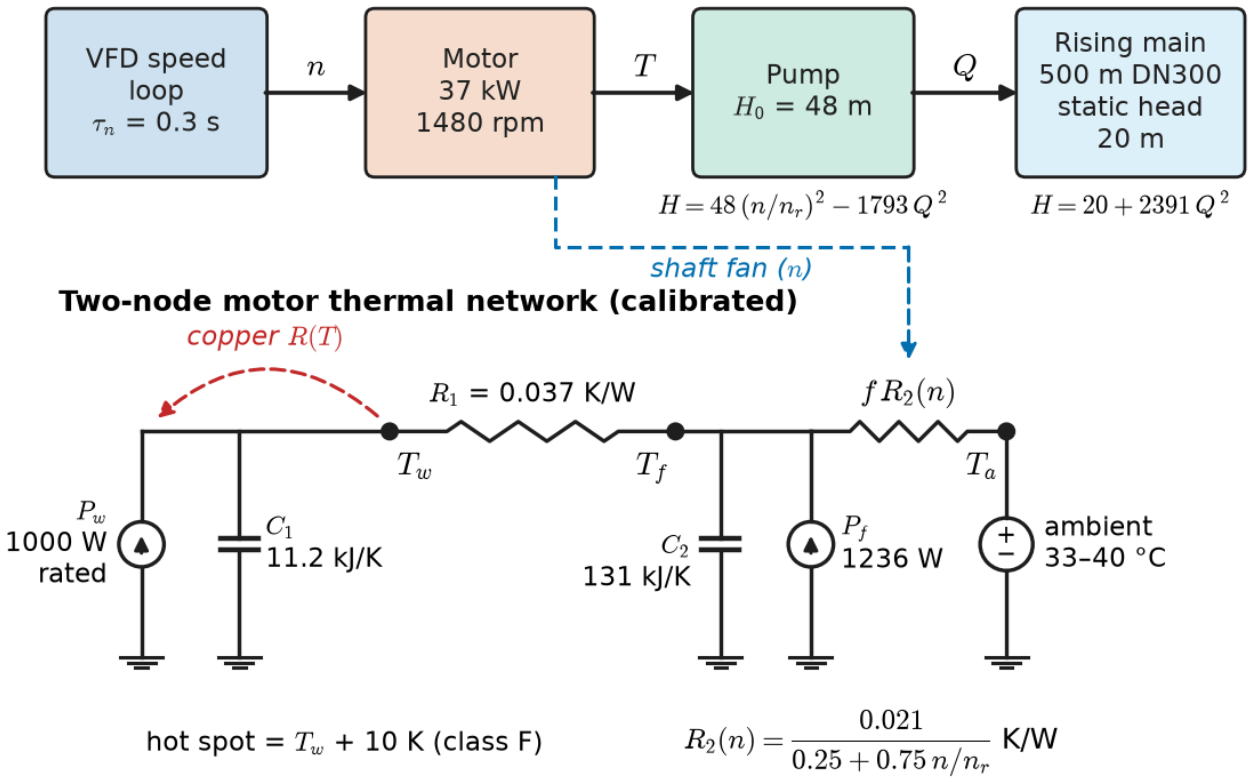


Figure 33.1 — Physics model of the 37 kW pump set: energy chain from drive to water and the two-node thermal network.

Choosing the fidelity

The pump set can be modelled at four levels; the twin estimates with L1, sweeps with L0 and borrows numbers from L2 and L3 once, at design time.

Level	Content	States	Run time for 24 h (edge PC)	Used for
L0 steady-state maps	Duty point, losses, steady temperatures as algebraic functions	0	~ 50 µs per point (vectorised)	Monte Carlo what-if (§33.9)
L1 lumped dynamic	4 ODEs above	4	~ 0.7 s (stiff solver)	State estimation, 30-min prediction
L2 detailed thermal	8–12 node network (slot, end winding, teeth, yoke, rotor, air, frame)	10	~ 2 s	Hot-spot allowance, standstill cooling
L3 field models	FE electromagnetic and CFD of the fan cowl	10^5–10^7	hours	Loss split, fouled-cowl airflow, once

Each level is calibrated against the one above it or against measurement, never left to float: the 10 K hot-spot allowance comes from L2 and the maker's heat-run report, the speed dependence of R_2 from a cowl CFD run (L3).

33.2 Numerical simulation

> **Boundary.**
> Explicit and implicit ODE methods, their stability regions and convergence are in *The Mathematics Universe*, Ch. 22 §22.9 and §22.11; simulation workflows in Python are in *Electronics and Computing*, Ch. 24 §24.4. Chapter 30 owns solver selection and performance as a general practice.

Stiffness decides the solver

The model mixes time constants of 0.3 s (speed loop), about 1 s (water column), about 6 min (winding) and about 50 min (frame). It is saved as a module so that every later listing uses the same physics.

```
 # pumpset.py -- physics model of the 37 kW pump set (states: speed, flow, winding, frame)
import numpy as np

NR, TR = 1480.0, 238.7                 # rated speed (rpm), rated torque (N m)
H0, HS, KP, KS = 48.0, 20.0, 1793.0, 2391.0   # pump shut-off head, static head (m), curve coeffs
QR, ETA_BEP, AH = 0.0818, 0.78, 1.387e-3      # rated flow (m3/s), BEP efficiency, g*A/L
P_CU_R, P_FE_R, P_RT_R = 1000.0, 600.0, 636.0 # stator copper, core+mech, rotor loss at rating (W)
PAR = dict(R1=0.035, C1=12e3, R2=0.0201, C2=120e3, tau_n=0.3)   # design-value parameters

def shaft_power(n, q):
    """Pump head (m) and shaft power (W) at n (rpm), q (m3/s)."""
    s = n / NR
    h = H0 * s**2 - KP * q**2
    qb = q / (QR * max(s, 1e-3))                   # flow relative to BEP
    eta = max(ETA_BEP * (1.0 - (qb - 1.0)**2), 0.30)
    return h, 1000 * 9.81 * q * h / eta

def losses(n, p_shaft, tw):
    """Winding-node and frame-node losses (W)."""
    n = max(n, 1.0)
    t = p_shaft / (n * 2 * np.pi / 60)             # shaft torque (N m)
    kt = (234.5 + tw) / (234.5 + 115.0)            # copper resistance relative to 115 C
    p_w = P_CU_R * (0.1 + 0.9 * (t / TR)**2) * kt
    p_f = P_FE_R * (n / NR)**1.6 + P_RT_R * (t / TR)**2
    return p_w, p_f

def rhs(t, x, n_ref, t_amb, p=PAR, foul=1.0):
    n, q, tw, tf = x
    h, ps = shaft_power(n, max(q, 0.0))
    dq = AH * (h - (HS + KS * q * abs(q)))         # water-column inertia
    pw, pf = losses(n, ps, tw)
    r2 = foul * p["R2"] / (0.25 + 0.75 * n / NR)   # shaft fan
    dtw = (pw - (tw - tf) / p["R1"]) / p["C1"]
    dtf = (pf + (tw - tf) / p["R1"] - (tf - t_amb(t)) / r2) / p["C2"]
    return [(n_ref(t) - n) / p["tau_n"], dq, dtw, dtf]
```

The `losses` and `shaft_power` functions are pure and vectorise over NumPy arrays, which §33.9 relies on. The next listing runs an 8-hour afternoon: speed steps between 1110 and 1520 rpm as the reservoir controller asks for flow, and the ambient rises from 33 to 38 °C.

```
 import time
import numpy as np
from scipy.integrate import solve_ivp
from pumpset import rhs

steps = [(0, 1180), (3600, 1480), (10800, 1110), (14400, 1410), (21600, 1520)]  # (s, rpm)
n_ref = lambda t: [n for (ts, n) in steps if t >= ts][-1]
t_amb = lambda t: 34.0 + 4.0 * np.sin(2 * np.pi * (t - 3600) / 86400)   # afternoon ambient (C)
x0 = [1180.0, 0.05, 70.0, 60.0]

for method in ["RK45", "BDF", "LSODA"]:
    t0 = time.perf_counter()
    sol = solve_ivp(rhs, (0, 8 * 3600), x0, method=method, args=(n_ref, t_amb),
                    rtol=1e-6, atol=[1e-3, 1e-7, 1e-4, 1e-4], dense_output=True)
    dt = time.perf_counter() - t0
    tw = sol.sol(np.arange(0, 8 * 3600, 10.0))[2]
    print(f"{method:5s}: nfev={sol.nfev:7d} steps={sol.t.size:6d} wall={dt:6.2f} s "
          f"max T_w={tw.max():6.2f} C  T_w(8 h)={tw[-1]:6.2f} C")
```

```
 RK45 : nfev= 182660 steps= 26088 wall= 21.01 s max T_w=126.22 C  T_w(8 h)=126.22 C
BDF  : nfev=   1730 steps=   672 wall=  0.77 s max T_w=126.23 C  T_w(8 h)=126.23 C
LSODA: nfev=   2126 steps=   748 wall=  0.23 s max T_w=126.22 C  T_w(8 h)=126.22 C
```

All three agree on the answer to 0.01 K; they differ by a factor of about 100 in function evaluations (wall times vary from run to run). Figure 33.2 shows the trajectory: the winding climbs to 112.7 °C after two hours at rated speed, falls to 76 °C during the 1110 rpm interval, and reaches 126.2 °C after two hours at 1520 rpm with 38 °C air.

Worked example — why RK45 takes 26 000 steps. The fastest eigenvalue is the speed loop, $\lambda = -1/0.3 = -3.33\ \mathrm{s}^{-1}$. The Dormand–Prince pair inside RK45 is stable on the negative real axis only for $h|\lambda| \lesssim 3.3$, so $h \lesssim 1.0$ s. The solver took 26 088 steps: 5.6 s steps for the first hour, while the speed state sits at equilibrium, then about 25 400 near the 1 s limit after the first speed step. The water column, linearised at rated flow, has $|\lambda| = AH \cdot 2(k_p + k_s)Q = 1.387 \times 10^{-3} \times 2 \times 4184 \times 0.0818 = 0.95\ \mathrm{s}^{-1}$, and the thermal modes are 380 s and 3070 s (§33.4): a stiffness ratio of $3070/0.3 \approx 10^4$. BDF takes small steps only around the five speed steps and strides across the thermal transients in minutes.

> **Status of the principle: Explicit step limit in stiff models**
> *Derived.* From the stability region of the explicit method and the largest eigenvalue magnitude of the linearised model; it holds whatever the accuracy tolerance, which is why tightening `rtol` does not make RK45 slower here and loosening it does not make it faster.

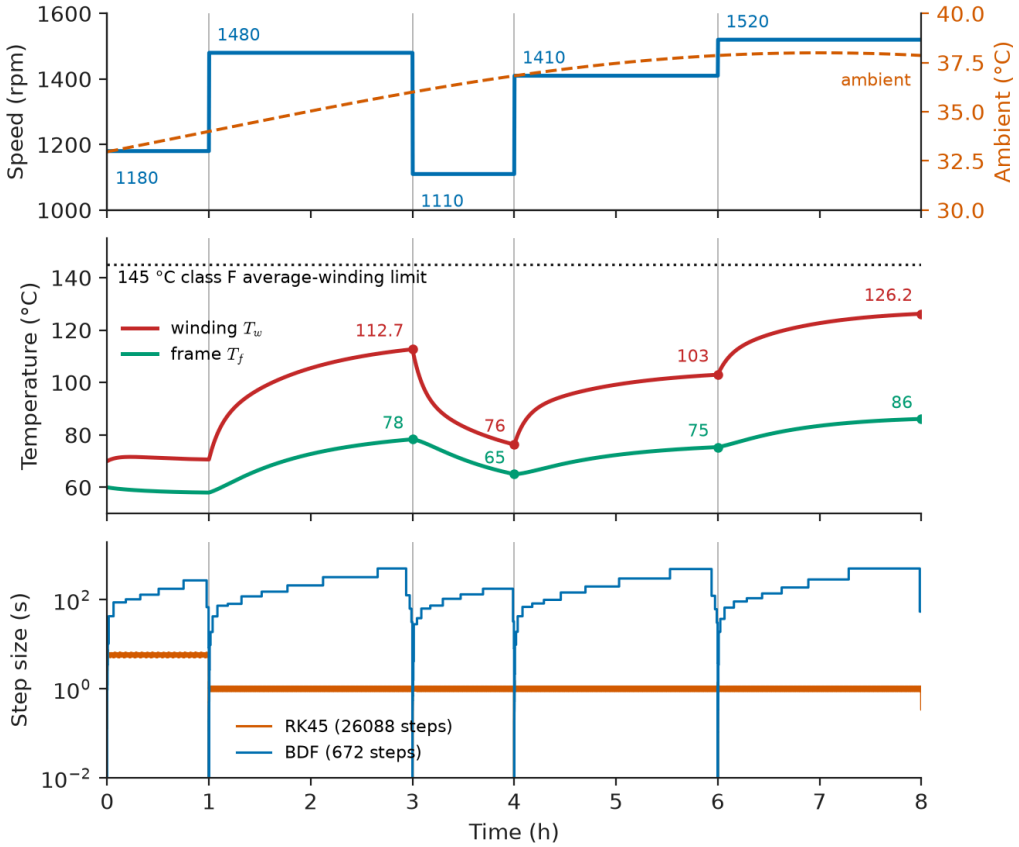


Figure 33.2 — Eight-hour afternoon simulation of the pump set and the step sizes chosen by an explicit and an implicit solver.

Events, speed and code verification

Discontinuities (speed steps, trips, pump starts) are handled as known switching times or as root-found *events* (`events=` in `solve_ivp`), never by testing a condition at output points. For speed, LSODA does 24 h of the L1 model in about 0.7 s, a 2.9× margin on R5 for one scenario but not for 500 Monte Carlo samples; those run on the L0 maps (§33.9). Before calibration the code is verified: the energy balance closes over a run, the thermal sub-model matches its analytic step response, and the solution converges as `rtol` is tightened (Chapter 30 owns these checks; the twin runs them in CI).

33.3 Model calibration

> **Boundary.**
> Nonlinear least squares and its covariance are in *The Mathematics Universe*, Ch. 23 §23.8; parameter identification and identifiability in Ch. 25 §25.8. Chapter 4 owns calibration of instruments; here "calibration" means fitting model parameters to a machine.

The commissioning heat run

The design values of §33.1 came from the catalogue. To calibrate the twin to *this* machine, the lead unit, which has three PT100s embedded in its end windings (a $310 factory option) and one on the frame, ran a

commissioning heat run: 3 h at rated speed and load from cold at 25 °C, then 2 h at 1180 rpm, logged at 1 min with VFD speed and shaft power.

Four parameters are fitted: R_1, C_1, R_2, C_2. The loss model is *not* fitted, for a reason worth stating before any code is written.

Worked example — what cannot be calibrated from temperatures alone. Multiply all losses by a, divide both resistances by a and multiply both capacitances by a: every term of the two thermal equations scales by a on both sides, so the temperature trajectories are unchanged. With $a = 1.10$, a motor with 10 % more loss, 10 % lower thermal resistances and 10 % higher heat capacities is indistinguishable from the original by temperature data. Losses must therefore come from elsewhere: the motor's IEC 60034-2-1 efficiency test, or a power analyser on the drive output during the heat run. The twin fixes the loss model from the test report and fits only the thermal network, and §33.9 carries the loss model's uncertainty forward explicitly.

> **Status of the principle: Loss–resistance identifiability**
> *Derived.* A direct consequence of the scaling symmetry of the thermal equations; it holds for any lumped thermal network driven by heat sources whose magnitude is not measured independently.

The fit drives the thermal sub-model with the logged speed and shaft power (forward Euler at 10 s, 2.6 % of the fastest thermal time constant) and minimises squared temperature residuals over log-parameters. The data here are synthetic, from a known "true" parameter set with 0.3 K noise, so the answer can be checked.

```
 import numpy as np
from scipy.optimize import least_squares
from pumpset import NR, losses

DT = 10.0
t = np.arange(0, 5 * 3600, DT)                          # 5 h heat run, 10 s steps
n = np.where(t < 3 * 3600, 1480.0, 1180.0)              # rated speed, then 80 % speed
ps = np.where(t < 3 * 3600, 37.04e3, 17.32e3)           # logged shaft power (W)
t_amb = 25.0 + 0.0 * t

def thermal(theta):
    r1, c1, r2, c2 = theta
    tw, tf, out = t_amb[0], t_amb[0], np.empty((t.size, 2))
    for k in range(t.size):                             # forward Euler
        pw, pf = losses(n[k], ps[k], tw)
        r2n = r2 / (0.25 + 0.75 * n[k] / NR)
        q12 = (tw - tf) / r1
        tw += DT * (pw - q12) / c1
        tf += DT * (pf + q12 - (tf - t_amb[k]) / r2n) / c2
        out[k] = tw, tf
    return out

rng = np.random.default_rng(7)
true = np.array([0.0372, 11.1e3, 0.0214, 131e3])
meas = thermal(true)[::6] + rng.normal(0, 0.3, (t.size // 6, 2))   # 60 s logging, 0.3 K noise

res = lambda lg: (thermal(np.exp(lg))[::6] - meas).ravel()
fit = least_squares(res, np.log([0.035, 12e3, 0.0201, 120e3]), x_scale=1.0)
J, s2 = fit.jac, np.sum(fit.fun**2) / (fit.fun.size - 4)
sd = np.sqrt(np.diag(np.linalg.inv(J.T @ J) * s2))   # std dev of log-parameters
for name, v, u, tr in zip(["R1 K/W", "C1 J/K", "R2 K/W", "C2 J/K"], np.exp(fit.x), sd, true):
    print(f"{name}: {v:10.4g}  +/- {200 * u:4.1f} % (95 %)   true {tr:.4g}")
print(f"residual RMS = {np.sqrt(np.mean(fit.fun**2)):.3f} K, evaluations = {fit.nfev}")
corr = np.linalg.inv(J.T @ J); d = np.sqrt(np.diag(corr))
print("corr(R1,C1) = %.2f, corr(R2,C2) = %.2f" % (corr[0, 1] / d[0] / d[1], corr[2, 3] / d[2] /
d[3]))
```

```
 R1 K/W:    0.03725  +/-  0.2 % (95 %)   true 0.0372
C1 J/K:  1.123e+04  +/-  0.8 % (95 %)   true 1.11e+04
R2 K/W:    0.02136  +/-  0.1 % (95 %)   true 0.0214
C2 J/K:  1.307e+05  +/-  0.4 % (95 %)   true 1.31e+05
residual RMS = 0.275 K, evaluations = 5
corr(R1,C1) = 0.22, corr(R2,C2) = 0.18
```

Figure 33.3 shows the fit. It moves the design values by +6 % (R_1), −6 % (C_1), +6 % (R_2) and +9 % (C_2), and the residual RMS of 0.275 K matches the 0.3 K sensor noise, as it should when the model structure is right. Two lessons sit in the output:

- **The heat run was designed for identifiability.** The step to 1180 rpm at 3 h excites the fan dependence and separates R_2 from C_2; a constant-load run pins only their product. The correlations

of 0.22 and 0.18 show each parameter determined on its own.

- **The intervals are optimistic.** Even with the exact model structure, the true C_1 lies 1.2 % from the estimate, outside its ±0.8 % interval. With real data the residuals are correlated and include model-form error, and covariance-based intervals are typically too narrow by a factor of 2–5, so the twin uses the fleet's unit-to-unit spread (§33.9) as its parameter uncertainty.

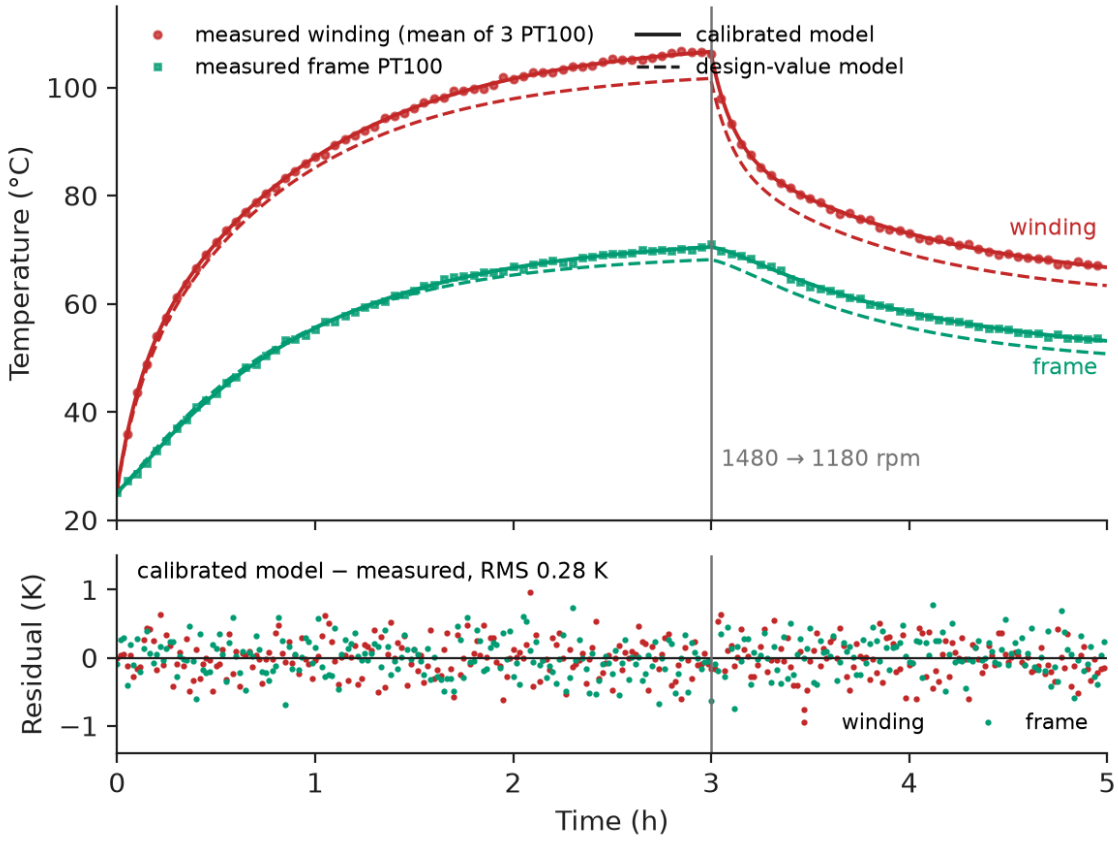


Figure 33.3 — Commissioning heat run on the lead unit: measured temperatures with the design-value and calibrated models.

Calibration as a controlled document

A calibration is a version: the parameter set is stored with the heat-run data hash, model version and residual statistics (R7), and retired when the unit is rewound or its fan changed. The eleven units without winding RTDs start from the lead unit's parameters and adapt only R_2 online (§33.7).

33.4 Co-simulation

Why the model is split

The utility's engineering studies need more than the thermal twin: a detailed electrical drive model, a hydraulic model of the distribution zone and the PLC's pump-sequencing logic, each from a different tool. Co-simulation runs them together without merging them: each is packaged as a **Functional Mock-up Unit** (FMU, FMI 2.0 or 3.0 standard) carrying its own solver, and a **master algorithm** advances all of them in **communication steps** of length H, exchanging inputs and outputs at the step boundaries.

FMU	Source	Internal step	Exchanges	Communication step
Drive and motor electrical	Modelica or PLECS-class	10 µs	Speed ref., torque → speed, losses	10 ms to 1 s
Motor thermal (L2)	Modelica thermal network	implicit, variable	Losses, speed → temperatures	1–60 s
Hydraulic network	EPANET-based wrapper	1 s	Pump speeds → flows, levels	1–60 s
Pump-sequencing logic	Virtual PLC	50 ms scan	Levels → speed refs, starts	0.1–1 s

Coupling error and the size of the communication step

Between exchanges each FMU holds its inputs constant (zero-order hold). That is an error the monolithic model does not have, and it grows with H. The master can update FMUs in parallel from old values (Jacobi) or one after another, each using the newest outputs (Gauss–Seidel).

Worked example — splitting the thermal network across two FMUs. Split the two-node network into a winding FMU and a frame FMU, each holding the other's temperature constant over a step and integrating exactly inside it. With rated losses, 40 °C ambient and a cold start, the errors after 1 h against the exact monolithic solution ($T_w = 106.97$ °C, $T_f = 72.19$ °C; coupled time constants 380 s and 3069 s) are:

Communication step H	Jacobi ΔT_w (K)	Jacobi ΔT_f (K)	Gauss–Seidel ΔT_w (K)	Gauss–Seidel ΔT_f (K)
1 s	−0.008	−0.004	−0.002	+0.001
10 s	−0.075	−0.045	−0.015	+0.012
60 s	−0.46	−0.27	−0.10	+0.07
300 s	−2.47	−1.43	−0.70	+0.24
1200 s	−11.6	−6.3	−5.6	−0.34

The Jacobi error is first order in H and Gauss–Seidel is 4–5× smaller. To keep the coupling error below about 0.3 K of R1's 3 K, $H \leq 60$ s with Gauss–Seidel or about 40 s with Jacobi: roughly $H \lesssim 0.1$–$0.15\,\tau$ of the fastest *coupled* mode (380 s). The electrical FMU coupled to the hydraulics needs $H \approx 10$ ms, because their coupled mode is about 0.3 s.

> **Status of the principle: Zero-order-hold coupling error**
> *Approximate.* First-order in the communication step for explicit (non-iterative) master algorithms, valid while H is well below the fastest coupled time constant; with strong algebraic coupling (direct feedthrough on both sides) explicit coupling can go unstable, and an iterative or implicit master, or a model-exchange FMU solved by one solver, is required.

Practical rules for co-simulation

Cut where coupling is weak and slow, never through a fast loop; check that energy leaving one FMU equals energy entering the other; fix order and step in an SSP experiment file (R7); and validate against the monolithic L1 model on a shared scenario first.

33.5 Digital-twin architectures

From model to twin

The terms are used loosely; the design document fixes them by the direction of data flow. A *digital model* is fed manually and offline (the design studies of §33.10); a *digital shadow* is fed live from the plant but reports only to people; a *digital twin* in the strict sense also writes back (setpoints, limits). R8 makes the utility's system a shadow with decision support. Most industrial "twins" are shadows; closing the loop moves the twin into the control system's safety case (Chapter 21), which needs a reason worth that cost.

The architecture

Figure 33.4 shows the architecture, layered after the ISO 23247 digital-twin framework (observable element, data collection, twin core, user entities). The **plant** has a frame PT100 on every motor and an aspirated ambient sensor at the fan inlet (the July lesson). An **edge gateway** per station reads the VFD over Modbus TCP and the PLC over OPC UA, timestamps at source, buffers 7 days and publishes over MQTT with TLS through a one-way IEC 62443 conduit. The **twin core** runs ingestion and alignment, a state estimator per unit (§33.7), the versioned model service, the UQ service and a scenario runner, beside a **historian and model registry**. **Applications** are the operator dashboard, maintenance work orders and engineering what-if studies.

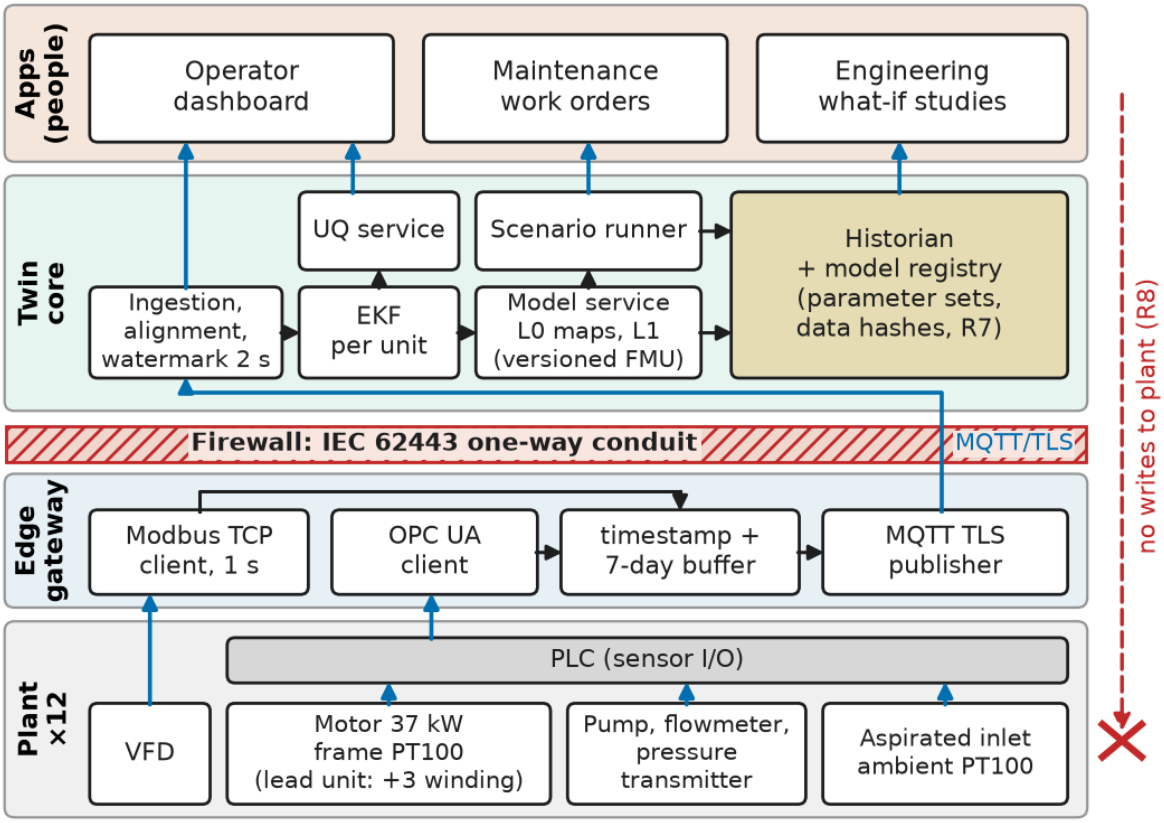


Figure 33.4 — Architecture of the pump-set digital shadow from plant sensors to applications.

Selecting the twin's building blocks

The first selection is where the winding temperature comes from.

Winding-temperature source	Accuracy	Cost per unit	Availability	Decision
PTC thermistors in winding (standard)	Binary trip at 145 or 155 °C only	Included	All units	Protection only; no estimate
Embedded PT100 (3 per motor)	±1 K at the sensor, ±3–5 K to the hot spot	$310 factory option; retrofit needs rewind	Lead unit only	Ground truth for calibration and validation
Stator-resistance estimate by VFD (DC injection at stop)	±5 K, only at stops	Firmware feature	Units with that VFD model	Cross-check, not continuous
Model-based estimate (this twin)	≤ 3 K RMS after calibration (R1)	Software	All units	Continuous estimate and prediction

For the runtime, a hand-coded Python model was kept for prototyping only; a commercial twin platform was rejected on per-asset licence cost and opaque estimators for 12 assets. The chosen runtime is a Modelica model exported as an FMU and executed by an FMI library (FMPy-class) in a Python service, so that the same versioned FMU serves engineering studies and the live twin; the L0 maps are plain NumPy.

33.6 Real-time data integration

> **Boundary.**
> Chapter 8 owns DAQ synchronisation and logging; Chapter 11 owns MQTT, gateways, provisioning and device security; Chapter 21 owns Modbus, OPC UA transport and OT security. This section is what a twin needs from them.

What arrives, how often, and how late

Signal	Source and interface	Rate	Accuracy
Speed, output power, current, DC bus	VFD, Modbus TCP	1 s	±3 % power
Frame temperature (all units); winding (lead unit)	PT100s via PLC	1 s	±0.3 K; ±1 K
Ambient at fan inlet	Aspirated PT100 via PLC	10 s	±0.3 K
Flow, discharge pressure	Magnetic flowmeter, transmitter	1 s	±0.5 %, ±0.1 %
Run state, trips	PLC events	event	—

The twin works on a uniform 1 s grid. Every sample carries a source timestamp (NTP-disciplined to ±10 ms, negligible against thermal time constants); the ingestion service waits 2 s (a **watermark**) for late samples before closing a grid point, then forward-fills each signal with an age counter.

The latency budget

Stage	Typical (s)	Worst case (s)	R4 limit (s)	Margin, worst case (s)
VFD value age at poll (1 s Modbus TCP poll)	0.50	1.00		
Modbus response	0.01	0.02		
OPC UA publish interval (1 s)	0.50	1.00		
Gateway to broker to twin (LTE, MQTT QoS 1)	0.08	0.50		
Ingestion watermark	2.00	2.00		
Estimator update and 30-min prediction	0.02	0.05		
Data age at state estimate	**3.11**	**4.57**	5.0	0.43
Publish to dashboard and refresh	0.60	1.30		
Data age on dashboard	**3.71**	**5.87**	10.0	4.13

Worked example — where the latency margin goes. The worst case at the estimator is $1.00 + 0.02 + 1.00 + 0.50 + 2.00 + 0.05 = 4.57$ s against 5 s, a margin of 0.43 s, and the 2 s watermark is the largest item.

Halving it would drop more late LTE samples (99.9th-percentile backhaul delay 1.6 s). Instead the gateway reads the VFD directly rather than through the PLC's 1 s OPC UA publish, removing one 1 s stage: worst case 3.57 s, margin 1.43 s.

Data quality before data use

A twin that ingests bad data produces confident nonsense. The ingestion service applies range and rate checks per tag (frame temperature −20…150 °C, ≤ 2 K/min); a **frozen-value** check (frame temperature unchanged for 15 min while running is the signature of a failed transmitter holding its last value); cross-checks (VFD power against hydraulic power over pump efficiency, 8 % for 10 min); and quality propagation, so every output carries the worst input quality and becomes STALE when a required input is older than 60 s (R6). The gateway's MQTT payload therefore carries, per value, its source timestamp and a quality flag, not just the number.

33.7 Parameter estimation

> **Boundary.**
> Observers, observability and the Kalman filter as the optimal observer are introduced in *Circuits, Signals and Communications*, Ch. 35 §35.7; the Gauss–Markov noise models behind its tuning in *The Mathematics Universe*, Ch. 21. Chapter 23 applies the same estimator to battery state of charge.

Which parameters drift

Calibration fixes parameters at commissioning. Some do not stay fixed:

Parameter	Drift mechanism	Rate	Twin treatment
R_2 (frame to air)	Cowl and fins clog	+1 to +3 %/day in pollen season	**Estimated online**
R_1 (winding to frame)	Insulation ageing	Years	Re-calibrated at overhaul
Loss model	Bearing wear, rotor damage, unbalance	Weeks to months	Not separable from R_2 (§33.3); left to vibration and current-signature monitoring

Fouling is the drift that caused the July trip. The twin estimates it by appending a cooling factor $g = 1/f$ to the state, modelled as a slow random walk, and running an extended Kalman filter (EKF) on the frame PT100.

Worked example — is g observable? In steady state $T_f - T_a = f\,R_2(n)\,(P_w + P_f)$, so with modelled losses and measured temperatures f follows directly. At rated load the clean-fan rise is $2236 \times 0.02136 = 47.8$ K and +15 % fouling adds 7.2 K, 24 times the 0.3 K sensor noise. At 1110 rpm the losses fall to 822 W, the rise to about 22 K and the same fouling moves the frame by only 3.2 K: observability is strong by day and weak at night, as the filter's covariance shows.

The estimator

The listing simulates 30 days of the fleet duty cycle (1180 rpm at night, 1480 rpm by day, 1410 rpm in the evening, ambient 19–31 °C) on a "plant" whose true parameters differ from the twin's calibrated set and whose losses are 3 % higher than the twin's loss model, which is the model-form error a real twin lives with. The fan cowl starts clogging on day 10 and reaches +30 % on day 30. The twin sees only the frame temperature.

```
 import numpy as np
from pumpset import NR, losses

R1, C1, R2, C2, DT = 0.03725, 11.23e3, 0.02136, 130.7e3, 60.0   # calibrated twin, 60 s step
day = np.arange(30 * 1440) * DT / 86400                          # 30 days of 1-min samples
hod = (day % 1) * 24
n = np.select([hod < 6, hod < 18], [1180.0, 1480.0], 1410.0)
ps = np.select([hod < 6, hod < 18], [17.32e3, 37.04e3], 31.63e3)
ta = 25 + 6 * np.sin(2 * np.pi * (hod - 9) / 24)
foul = 1 + 0.30 * np.clip((day - 10) / 20, 0, 1)                 # fan cowl clogs from day 10

def step(tw, tf, g, k, p=(R1, C1, R2, C2, 1.0)):
    r1, c1, r2, c2, kl = p                                       # kl: loss-model scale
    pw, pf = (kl * v for v in losses(n[k], ps[k], tw))
    a = 0.25 + 0.75 * n[k] / NR
    q12 = (tw - tf) / r1
    return tw + DT * (pw - q12) / c1, tf + DT * (pf + q12 - (tf - ta[k]) * g * a / r2) / c2, a

rng = np.random.default_rng(3)
PLANT = (0.0372, 11.1e3, 0.0214, 131e3, 1.03)                   # true values, losses +3 %
xt, x = np.array([60.0, 50.0]), np.array([60.0, 50.0, 1.0])     # truth [Tw,Tf]; twin [Tw,Tf,g]
P, Q = np.diag([25.0, 1.0, 0.01]), np.diag([0.05, 0.01, 2e-7])
Rm, est, err = 0.3**2, np.empty(day.size), np.empty(day.size)
for k in range(day.size):
    xt = np.array(step(*xt, 1 / foul[k], k, PLANT)[:2])          # plant
    tw, tf, a = step(*x, k)
    F = np.eye(3) + DT * np.array([[-1 / (R1 * C1), 1 / (R1 * C1), 0],
        [1 / (R1 * C2), -1 / (R1 * C2) - x[2] * a / (R2 * C2), -(x[1] - ta[k]) * a / (R2 * C2)],
        [0, 0, 0]])
    x, P = np.array([tw, tf, x[2]]), F @ P @ F.T + Q              # predict
    y = xt[1] + rng.normal(0, 0.3) - x[1]                        # frame PT100 innovation
    K = P[:, 1] / (P[1, 1] + Rm)
    x, P = x + K * y, P - np.outer(K, P[1, :])                   # update
    est[k], err[k] = 1 / x[2], x[0] - xt[0]
hit = lambda f: day[np.argmax(f >= 1.15)]
print(f"true fouling reaches +15 % on day {hit(foul):.1f}; twin flags it on day {hit(est):.1f}")
print(f"estimate on day 9: {est[9 * 1440]:.3f} (true 1.000); day 30: {est[-1]:.3f} (true
{foul[-1]:.3f})")
print(f"unmeasured winding temperature error after day 1: RMS
{np.sqrt(np.mean(err[1440:]**2)):.2f} K, "
      f"max {np.abs(err[1440:]).max():.2f} K")
```

```
 true fouling reaches +15 % on day 20.0; twin flags it on day 17.5
estimate on day 9: 1.032 (true 1.000); day 30: 1.341 (true 1.300)
unmeasured winding temperature error after day 1: RMS 1.00 K, max 1.55 K
```

The filter tracks the fouling ramp (Figure 33.5) and keeps the unmeasured winding temperature within 1.0 K RMS, inside R1. But the 3 % loss-model error does not vanish: the filter cannot tell 3 % more heat from 3 % worse cooling (the identifiability result of §33.3), so it reports $f = 1.032$ on a clean fan and crosses +15 % on day 17.5, 2.5 days early. The deployed detector therefore compares f with its own **baseline**, the median over the 7 days after each fan cleaning, and alarms when $f / f_{base} \geq 1.15$ for 6 consecutive daytime hours. The bias is multiplicative ($1.341/1.300 = 1.032$, the ratio already seen on day 9), so the ratio crosses 1.15 on day 20.3, about 7 h after the true crossing, meeting R3; excluding the weakly observable night hours keeps false alarms down.

> **Status of the principle: Augmented-state estimation of a drifting parameter**
> *Approximate.* Valid when the parameter drifts slowly relative to the observation window (days against hours here) and is observable from the measured outputs; it absorbs any unmodelled effect with the same signature, so the estimated parameter is "cooling as the model sees it", not a physical measurement of the cowl.

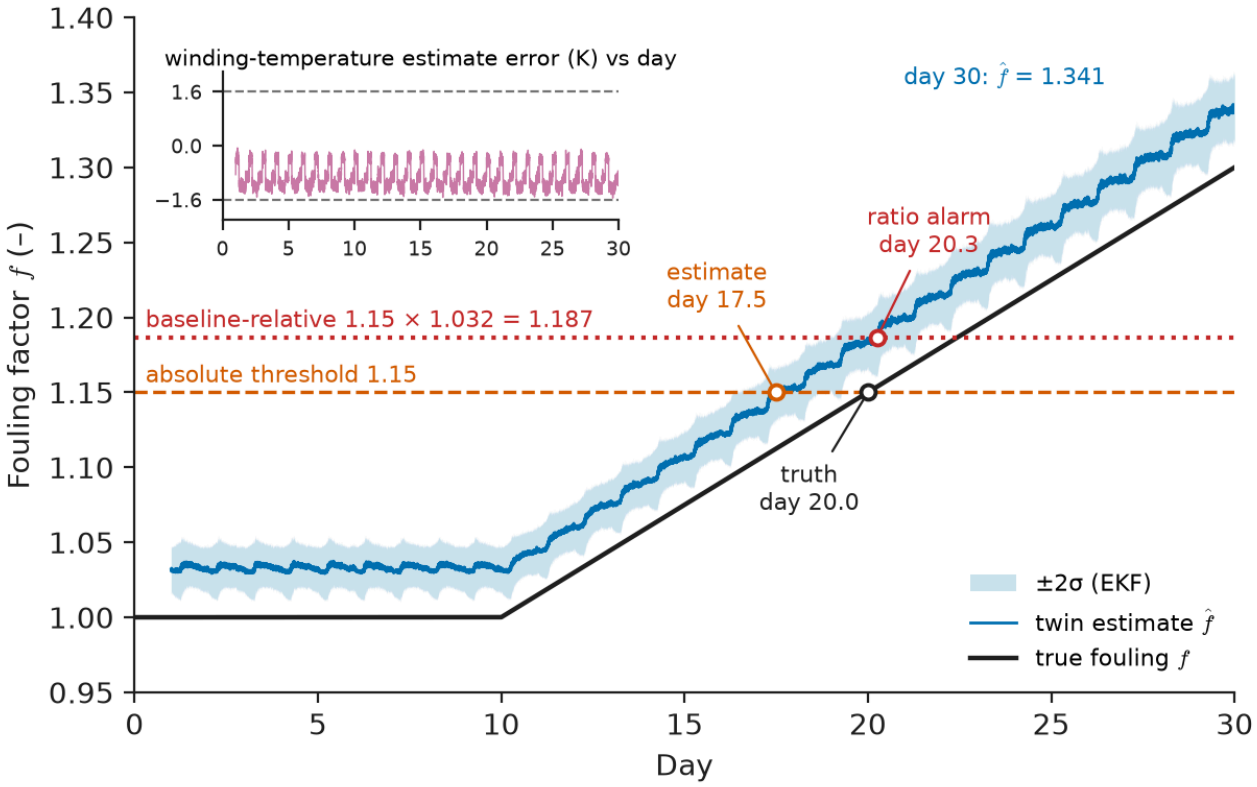


Figure 33.5 — Online estimate of the fan-cowl fouling factor over 30 days against the true value, with absolute and baseline-relative alarm thresholds.

33.8 Validation

> **Boundary.**
> The validation workflow (verification versus validation, hold-out data, acceptance criteria written in advance) is owned by Chapter 3 and Chapter 4; model-validation statistics are in *The Mathematics Universe*, Ch. 25 §25.7. ASME V&V 20 gives a standard framework for comparing simulation with experiment that this plan follows in spirit.

The plan, written before the data

The validation data are 30 days of lead-unit operation in August, not used for calibration, covering the year's hottest ambient, a fouling episode and a cleaning. Metrics and limits were fixed in advance; the results are representative of the campaign:

Req.	Test	Limit	Result	Pass/fail
R1	Estimate against mean of 3 winding PT100s, 30-day hold-out	≤ 3 K RMS, ≤ 6 K max	1.4 K RMS, 4.1 K max	Pass
R2	30-min prediction, 95 % interval coverage	≥ 90 % covered, width ≤ 12 K	93.1 % covered, mean width 9.8 K	Pass
R3	Seeded +15 % R_2 step at 20 random times	Flag ≤ 72 h, all 20	Median 19 h, max 41 h, 20/20	Pass
R3	False alarms, 12 unit-years of history	≤ 12	7 (5 traced to real light fouling)	Pass
R4	Data age audit, 10^6 grid points	≤ 5 s at estimator	99.99th percentile 3.4 s (after gateway change)	Pass
R5	24 h scenario on edge server	≤ 2 s	0.71 s	Pass
R6	Gateway unplugged for 5 min; frame tag frozen	STALE within 60 s; FROZEN within 15 min	STALE at 61 s; FROZEN at 15 min	**Fail**, then pass after fix
R7	Trace 10 random outputs	10/10	10/10	Pass
R8	Firewall review, write-attempt test	No write path	No write path	Pass

The R6 failure was a timer in the ingestion service that started counting after the watermark rather than at the last sample's timestamp: data age was measured from the wrong event. The fix, and a regression test that injects a gap and checks the STALE flag at 60.0 ± 1 s, went in before release.

What the residuals say

Aggregate metrics hide structure, so the residual is also plotted against the conditions most likely to break the model. Against speed: −0.4 K bias at 1480 rpm, +1.8 K at 1110 rpm (the fan law under-cools slightly at low speed; accepted). After cold starts: 2.6 K RMS against 1.4 K overall, the known limit of lumping slot and end winding, which is why the hot-spot allowance carries its own uncertainty in §33.9. Against ambient: no

trend after the inlet sensor was moved; a +3.8 K bias above 30 °C before it, the July failure visible in the data.

> **Status of the principle: Validation domain**
> *Empirical.* A twin is validated only over the conditions in its validation data (here 1110–1520 rpm, 18–39 °C ambient, fouling up to +25 %); outside that envelope its outputs are extrapolation and the dashboard says so.

33.9 Uncertainty quantification

> **Boundary.**
> Monte Carlo methods and sensitivity and uncertainty propagation are in *The Mathematics Universe*, Ch. 24 §24.6 and §24.8; the GUM and its Monte Carlo supplement (JCGM 100 and JCGM 101) are the metrological standards for propagating distributions. Chapter 7 owns measurement uncertainty budgets.

Sources of uncertainty in a twin's prediction

The inputs to the propagation are the unit-to-unit spread of R_1 and R_2 across the fleet's heat-run reports (±5 %, 1σ), the loss-model error (±4 %, from the heat run and the 3 % bias of §33.7), the current fouling estimate (1.05 ± 0.03), the ambient forecast (±0.5 K) and the hot-spot allowance (10 ± 3 K); numerical and coupling errors (< 0.1 K, §33.2 and §33.4) are negligible.

The quantity of interest for an operating decision is the steady hot-spot temperature at a proposed speed on a hot day. A 6 h hot afternoon is more than seven frame time constants, so the L0 steady-state map is accurate and fast enough to sample 20 000 times.

```
 import numpy as np
from pumpset import NR, losses, shaft_power, H0, HS, KP, KS

rng, N = np.random.default_rng(11), 20000
X = {"R1": rng.normal(0.03725, 0.05 * 0.03725, N),    # fleet unit-to-unit spread
     "R2": rng.normal(0.02136, 0.05 * 0.02136, N),
     "kloss": rng.normal(1.0, 0.04, N),               # loss-model error
     "foul": rng.normal(1.05, 0.03, N),               # twin's current fouling estimate
     "ta": rng.normal(40.0, 0.5, N),                  # ambient forecast
     "dhs": rng.normal(10.0, 3.0, N)}                 # hot-spot allowance over average

def hot_spot(n):
    s = n / NR
    q = np.sqrt((H0 * s**2 - HS) / (KP + KS))         # duty point on the system curve
    ps = shaft_power(n, q)[1]
    tw = np.full(N, 120.0)
    for _ in range(30):                               # fixed point for copper R(T)
        pw, pf = losses(n, ps, tw)
        pw, pf = X["kloss"] * pw, X["kloss"] * pf
        tf = X["ta"] + (pw + pf) * X["foul"] * X["R2"] / (0.25 + 0.75 * s)
        tw = tf + pw * X["R1"]
    return tw + X["dhs"]

for n in (1480.0, 1500.0, 1520.0):
    hs = hot_spot(n)
    print(f"{n:.0f} rpm: hot spot mean {hs.mean():6.1f} C, 95th pct {np.percentile(hs, 95):6.1f}
C, "
          f"P(>155 C) = {np.mean(hs > 155):.3f}")
hs = hot_spot(1520.0)
A = np.column_stack([(v - v.mean()) / v.std() for v in X.values()])
src = np.linalg.lstsq(A, (hs - hs.mean()) / hs.std(), rcond=None)[0]
print("variance share:", ", ".join(f"{k} {100 * b**2:.0f} %" for k, b in zip(X, src)))
```

```
 1480 rpm: hot spot mean  140.2 C, 95th pct  151.6 C, P(>155 C) = 0.019
1500 rpm: hot spot mean  145.2 C, 95th pct  157.2 C, P(>155 C) = 0.088
1520 rpm: hot spot mean  150.5 C, 95th pct  163.3 C, P(>155 C) = 0.270
variance share: R1 13 %, R2 21 %, kloss 43 %, foul 7 %, ta 1 %, dhs 16 %
```

The last line is a standardised-regression sensitivity analysis (squared standardised coefficients of a linear fit, valid here because the fit explains over 99 % of the variance). It says where effort pays: the loss model carries 43 % of the variance, ambient 1 %.

> **Status of the principle: Monte Carlo propagation of input distributions**
> *Derived.* From the definition of the output distribution as the push-forward of the input distribution through the model (JCGM 101); its sampling error falls as $1/\sqrt{N}$ (here about ±0.3 percentage points on a 5 % tail probability with $N = 20\,000$), and it is only as good as the input distributions, which are themselves judgements.

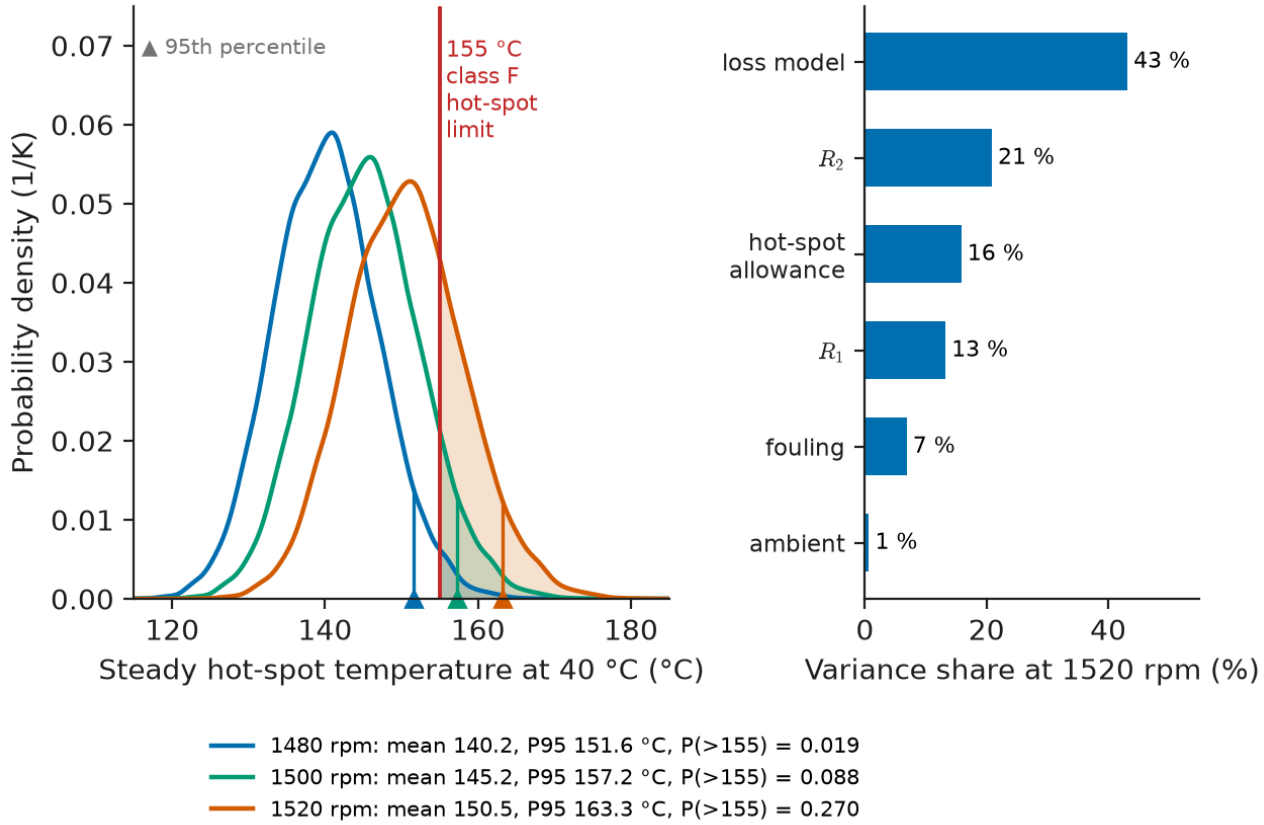


Figure 33.6 — Monte Carlo distribution of steady hot-spot temperature at 40 C ambient for three pump speeds, and the variance contributions.

33.10 Simulation-based design

The twin earns its keep when a decision is made with it that would otherwise be made by rule of thumb or not at all. Three decisions from the utility's first summer show the pattern: define the decision rule, run the twin with its uncertainty, act, and check.

Worked example — may the pumps overspeed in a heat wave? During a demand peak the reservoir controller wants 1520 rpm (about 51.3 Hz, 4.6 % more flow and 9 % more shaft power than rated, 40.4 kW). The rule agreed with the motor maker: overspeed is allowed when the predicted 95th-percentile hot spot is ≤ 155 °C (class F). At a 40 °C forecast the twin gives 163.3 °C at 1520 rpm and 157.2 °C at 1500 rpm: no overspeed. Repeating the Monte Carlo over ambient, the 95th percentile at 1520 rpm is 150.7, 152.0, 153.2 and 154.5 °C at 30, 31, 32 and 33 °C, so the dashboard shows "overspeed permitted" while the 6 h forecast stays at or below 33 °C. The old rule ("never above 50 Hz") wasted that capacity on cool days; the July nameplate twin would have permitted overspeed at 38 °C on a fouled motor.

Figure 33.6 shows the distributions.

Worked example — the value of better information. The loss model is the largest uncertainty. Halving it to ±2 % (a power analyser on the lead unit's drive output for one afternoon) would move the 95th percentile at 40 °C and 1500 rpm from 157.2 °C to 155.1 °C, 2.1 K of margin; better ambient sensing would buy almost nothing (1 % of variance). UQ turns "the model could be better" into a ranked list of measurements.

Worked example — fan-cleaning interval. Fleet estimates show fouling of 0.2 %/day outside pollen season and up to 1.5 %/day in June and July, so the +15 % alarm fires about 10 days after a cleaning in season. Each +10 % of fouling raises the rated-load hot spot by about $0.10 \times 47.8 = 4.8$ K, against a 95th-percentile margin of only $155 - 151.6 = 3.4$ K at rated speed and 40 °C. The decision: clean on the twin's alarm, plus every 7 days in season at the four stations next to cottonwood stands. A cleaning costs 20 minutes; the July trip cost four hours of supply.

Design alternatives evaluated on the twin

The same model compares hardware changes for the two stations that run hottest:

Option	Model change	Hot spot at 1500 rpm, 40 °C (95th pct)	Indicative cost	Decision
Forced ventilation (separately driven fan)	R_2 independent of speed	≈ 157 °C	$900 per unit	Rejected: helps only at low speed
45 kW motor (next frame)	Losses and network rescaled from maker's data	≈ 132 °C	$4,800 installed	Adopted at the two hottest stations

The forced-ventilation result is the instructive one: at 1500 rpm the shaft fan already gives $0.25 + 0.75 \times 1.01 \approx 1.01$ of its rated airflow, so a constant-speed fan of the same size changes nothing at high speed.

Failure analysis

The July incident, re-told with the twin's vocabulary, is a chain of three failures, each of which now has a detection and a mitigation:

Failure mode	Effect on twin output	Cause	Detection	Mitigation
Uncalibrated parameters	Winding estimate 6–13 K low at full load	Catalogue R, C	Residual bias against lead-unit RTD	Heat-run calibration (§33.3)
Unmodelled drift (fan fouling)	Hot spot under-predicted by up to 15 K after 3 weeks	R_2 treated as constant	Rising frame-temperature innovation; fouling estimate	Augmented-state EKF with baseline-relative alarm (§33.7)
Biased input (ambient sensor in shade)	−3.8 K bias above 30 °C	Sensor placed for convenience	Residual trend against ambient	Aspirated sensor at fan inlet
Model-form error absorbed by estimator	Fouling reads 1.03 on a clean fan; early alarm	3 % loss-model error indistinguishable from R_2	Non-unity estimate right after cleaning	Baseline after each cleaning; better loss model
Extrapolation outside validation domain	Confident output where model unvalidated	New duty (e.g. 1560 rpm) or rewound motor	Envelope check on inputs	Dashboard flag; new calibration after rewind
Stale data shown as fresh	Operators trust old values	Age measured from wrong event (R6 test failure)	Fault-injection test	Fixed timer; regression test at 60.0 ± 1 s

Every failure in the table is a *data* or *assumption* failure, not a numerical one: the engineering of a twin is mostly the engineering of its inputs, its calibration and its claims.

What remains open

Open question	Where it is answered
How the modelling workflow, sensitivity analysis and validation method are set up in general	Chapter 3
Solver performance, parameter sweeps and numerical verification as reusable tools	Chapter 30
Replacing the L1 model with a trained surrogate or a physics-informed network for faster UQ	Chapter 32
How a twin's state estimator is deployed on an edge device with limited compute	Chapter 12
Building and testing an engineering simulation tool as a software product	Chapter 39
Using a twin and its V&V evidence inside a complete project report	Chapter 40
Motor thermal classes, two-body protection models and insulation life	*Electrical Energy and Machines*, Ch. 9
Kalman filtering and observer theory	*Circuits, Signals and Communications*, Ch. 35

Next: Part IX begins with a complete project, the autonomous line-following robot of Chapter 34, where requirements, sensors, motors, firmware and control are carried through to a robot that laps a track.

Part IX — Interdisciplinary Engineering Projects

Parts I to VIII built the methods and the subsystems one at a time. Part IX puts them together. Each chapter is one complete project, carried from a problem statement to a tested system: requirements with IDs, an architecture, budgets with margins, real parts, design calculations, firmware or tool code, a test plan with limits, test results, a bill of materials with indicative cost, a failure analysis and a closing "Revision B" that says what the next iteration would change and why. The projects differ in kind (a fast mobile robot, a mains-connected instrument, an energy system, a motor drive, a radio network, a piece of engineering software, and the method for a capstone of the reader's own). Theory is not repeated: each chapter points back to the chapter that owns a method and to the sibling book that owns the physics, and spends its pages on integration decisions and on what went wrong in the first build.

By the end of this part you should understand

- How stakeholder needs become requirements that each end in a test with a numerical limit.
- How power, energy, timing, accuracy, mass and cost budgets are written before the build and closed by measurement.
- How to choose among real part classes against the requirements, and record why.
- Why first prototypes fail where they do (missing lag in a control loop, a phase error invisible at unity power factor, a battery sized without derating, a radio budget without fade margin) and how failure analysis turns those findings into design changes.
- How subsystems are verified separately and then together, and how results are reported with their uncertainty.
- How a complete project is documented so that someone else could rebuild it, test it and improve it.

Chapters

Chapter	Central question	What it delivers
34 — Project: Autonomous Line-Following Robot	Which sensor geometry, motors and control structure let a 170 g robot lap a 15 m track in under 10 s without leaving the line?	Sensor and motor selection, circuit, 1 kHz firmware, gain-scheduled cascade control, power budget, track tests, speed profile
35 — Project: Smart Energy Monitor	How do a CT, a divider and a 24-bit ADC become a safe, connected energy measurement good to class 1 at PF 0.5?	Front end and error budget, power calculation, metrology firmware, logging, MQTT, calibration, isolation, IEC 62053-21 validation
36 — Project: Solar Battery System	How are PV array, battery and converters sized so that an off-grid or hybrid load is served through the worst month at an acceptable cost?	Load estimate, PV and battery sizing, converters, charge control, protection, monitoring, efficiency, life-cycle cost
37 — Project: Closed-Loop Motor Drive	How is a motor characterised and driven so that speed and current loops meet their specifications with the protection a real drive needs?	Motor characterisation, encoder, power stage, sensing, controller design, firmware, protection, performance tests
38 — Project: Wireless Sensor Network	How do battery-powered nodes deliver their data reliably and securely for years from a real site?	Node architecture, protocol, energy budget, acquisition, reliability, security, deployment and field tests
39 — Project: Engineering Simulation Tool	How is an engineering calculation turned into a tool that others can trust, run and maintain?	Equations, solver, input validation, visualisation, benchmarks, verification, documentation, reproducibility
40 — Capstone Engineering Project	How does a reader run a project of their own from problem selection to final report?	The capstone method tied to Part I, a project-report template, cost and risk analysis, and one capstone worked through end to end

How this part connects to the rest of the book

Every project leans on Part I: requirements (Chapter 1), architecture (Chapter 2), modelling (Chapter 3), verification (Chapter 4) and documentation (Chapter 5). Each then draws on the part that owns its subsystems: the line follower on Chapters 14 and 15; the energy monitor completes Chapter 9's compact project using Chapters 7, 8 and 11; the solar battery system applies Chapters 22, 23 and 25; the motor drive builds on Chapter 15 and Part V; the sensor network applies Chapters 11, 21 and 26; the simulation tool applies Chapters 29 to 31 and 33; the capstone uses all of them.

The theory lives in the sibling books and is cited, not re-derived: circuits, signals, sampling and control in *Circuits, Signals and Communications*; devices, microcontrollers, embedded interfaces, real-time computing and power electronics in *Electronics and Computing*; machines, drives, storage, power quality and safety in *Electrical Energy and Machines*; physics, materials and mathematics in *Engineering Physics*, *Materials Science and Engineering* and *The Mathematics Universe*.

Chapter 34 — Project: Autonomous Line-Following Robot

Part IX · Interdisciplinary Engineering Projects

A university team built a line follower for a regional competition: 19 mm black tape on white melamine, 15 m laps, curves down to 100 mm radius. The breadboard prototype, P1, followed the line well at 0.3 m/s. At 0.6 m/s it began to weave at about 2 Hz; at 0.8 m/s it left the track on the first curve. An evening of raising and lowering gains moved the weave but never removed it. Nothing was broken. The sensor bar sat 30 mm ahead of the axle, the motors were driven open loop by PWM, and the sensors were scanned at 200 Hz through a four-sample average. Each choice added lag to a loop whose plant, a vehicle steering itself onto a line, is a double integrator whose gain rises with speed. On paper the loop was unstable above 0.6 m/s, and no gain could fix it. This chapter rebuilds the robot as a project, each decision closed by a number.

> **Which sensor geometry, motor, gear ratio and control structure let a 170 g robot lap a 15 m track in under 10 s without leaving the line?**
> **How are the timing, power and mass budgets written so that the robot that is built is the robot that was analysed?**

Concept	Engineering destination
Requirements with IDs	Lap time, tracking, runtime, mass, robustness limits
Sensor selection	Pitch and height for a linear, dead-zone-free position
Motor selection	Speed, acceleration, reflected inertia, encoder resolution
Circuit design	Power tree, H-bridge current limit, sensor channel
Embedded firmware	1 kHz cycle, ambient-cancelling scan, state machine
Control algorithm	Cascaded steering PD and wheel PI, gain-scheduled
Power budgeting	Average and peak current, runtime, regulator choice
Testing	Bench, track and robustness tests; FMEA
Performance optimisation	Lookahead, track map, feed-forward, speed profile

Roadmap. Requirements and architecture (34.1) → sensors (34.2) → motors (34.3) → circuit and BOM (34.4) → firmware (34.5) → control (34.6) → power and mass (34.7) → tests and failure analysis (34.8) → optimisation (34.9) → Revision B (34.10).

34.1 Requirements

> **Boundary.**
> Chapter 1 owns requirements and traceability; Chapter 14 owns differential-drive kinematics, odometry and line following as a technique; Chapter 15 owns motor drivers, encoder interfaces and PID implementation in general. This chapter applies them to one robot end to end and points back rather than repeating them.

The problem and its numbers

The event rules fix the environment: 19 mm matt black line on white, minimum curve radius 100 mm, one 90° crossing, a 20 mm gap, a remote start signal, footprint at most 200 × 150 mm and mass at most 250 g. A top-three time on the 15.0 m reference track (Figure 34.6) meant a lap under 10 s, 1.5 m/s on average. From those, the requirements:

ID	Requirement	Target	Verification
R1	Follow the line through 100 mm curves, the crossing and the gap	50 consecutive laps, no departure	T7
R2	Lap time, 15.0 m reference track	≤ 10.0 s (mean of 5)	T6
R3	Tracking error at 1.0 m/s, straights and curves ≥ 250 mm radius	≤ 6 mm RMS at sensor bar	T5
R4	Footprint and mass	≤ 200 × 150 mm; ≤ 250 g	Measurement
R5	Running time, racing duty	≥ 20 min	T8
R6	Ambient light immunity	50–2000 lux, 100 Hz flicker, low sun	T9
R7	Safe stop: line lost > 150 mm, remote stop, driver fault	≤ 0.5 s	T10
R8	Battery protection	No reset at full power; stop at 3.3 V/cell	T8
R9	Tuning telemetry	≥ 50 Hz over BLE	Bench
R10	Cost	BOM ≤ $150 (2026, one-off)	BOM (§34.4)

R2 and R3 drive the control design, R1 and R6 the sensor design, R2, R4 and R5 the motor, battery and mass, R7 and R8 the firmware's safety logic. Every row ends in a test with a limit, so the project's verification table (§34.8) is this table with two more columns.

The architecture

Figure 34.1 shows the blocks and their interfaces. The decisions behind each block are made in the following sections; the figure is drawn first so that every later budget has a place to land.

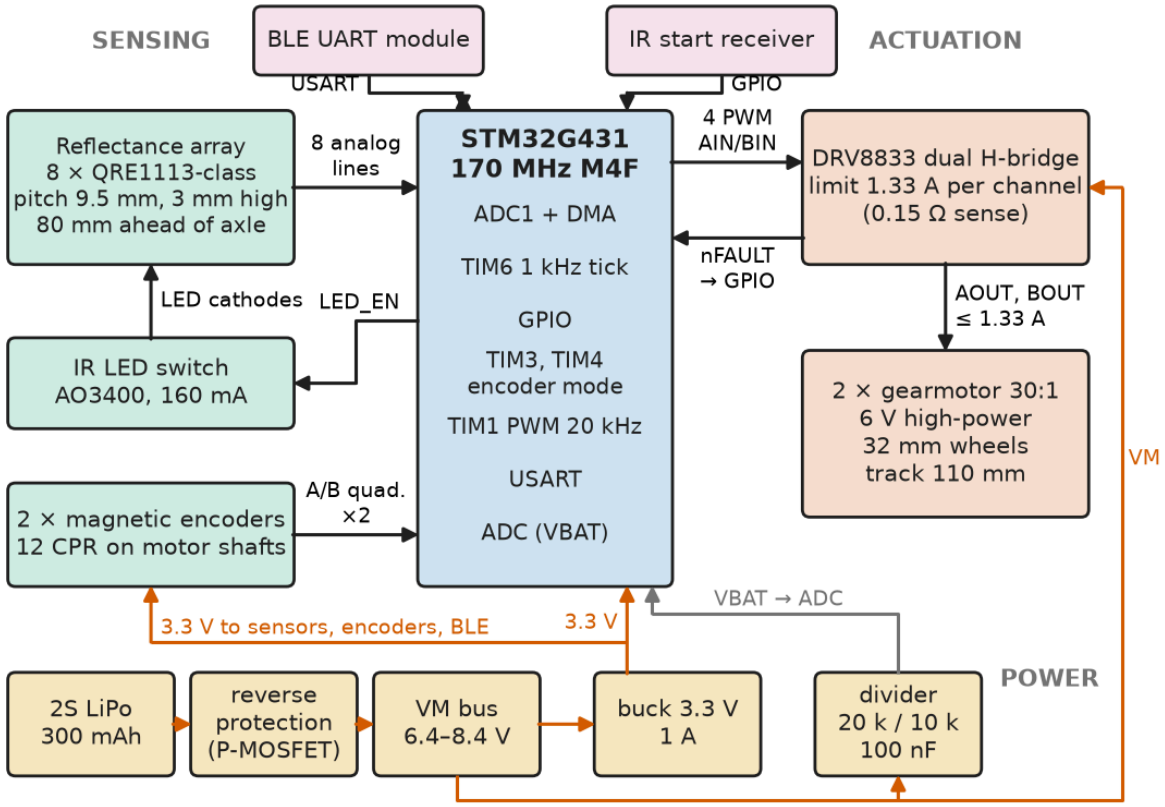


Figure 34.1 — Architecture of the line-following robot: sensing, computing, actuation and power with their interfaces.

34.2 Sensor selection

Candidate sensors

A line follower needs the lateral position of the line relative to the robot, fast and with as little lag as possible. Four sensor classes were compared against R1, R3, R6 and the 1 kHz loop rate chosen in §34.6:

Option	Position output	Rate	Ambient immunity	Verdict
5 discrete TCRT5000-class, 15 mm pitch, comparators	5 on/off states	> 10 kHz	Poor	Rejected: quantised error causes a limit cycle
8-channel analog array (QRE1113-class, 9.5 mm pitch), MCU ADC	Continuous (centroid)	1–5 kHz	Good with LED on/off differencing	**Chosen** (8 g, $5)
8-channel "RC" timing array	Continuous	Read time up to 2.5 ms on black	Moderate	Rejected: read time eats the 1 ms cycle
128-pixel linear image sensor with lens	Sub-mm, long lookahead	0.2–1 kHz	Needs controlled lighting	Kept for Revision B

The analog array wins because it gives a continuous position at the loop rate with no read-time penalty, and because its LEDs can be switched: reading every channel with the LEDs off and again with them on, then subtracting, cancels ambient light, including the 100 Hz flicker of venue lighting, which the DC-threshold sensors cannot (R6).

> **Boundary.**
> Reflective optical sensors and their selection are owned by Chapter 6; the ADC interface and sensor-interface practice by *Electronics and Computing*, Ch. 26 §26.8–§26.9.

From eight readings to one position

Each channel is normalised with the values recorded during a calibration sweep (white and black levels per channel), so that 0 means white and 1000 means black. The line position is the weighted centroid of the normalised readings over the sensor positions x_i:

$$\hat{y} = \frac{\sum_i n_i \, x_i}{\sum_i n_i}, \qquad x_i = (i - 3.5) \times 9.5\ \text{mm}, \quad i = 0 \ldots 7$$

The centroid interpolates between sensors, but only if each sensor's optical footprint is wide enough to overlap its neighbours. Footprint width grows with height above the track: roughly, the illuminated and viewed spot of a QRE1113-class part has a Gaussian-like profile whose width σ is about equal to the height for heights of 1–5 mm.

Worked example — sensor height, pitch and the dead zone. A 19 mm line on a 9.5 mm pitch covers two sensors when centred. Modelling each sensor's response as the overlap of a Gaussian footprint of width σ with the line (readings below 5 % ignored): at 1.5 mm height ($\sigma \approx 1.5$ mm) true offsets of 1 and 2 mm give estimates of 0.01 and 0.08 mm, because the two covered sensors stay saturated and the outer ones see nothing; that is a **dead zone of about ±2 mm** with no error signal. At 3 mm ($\sigma \approx 3$ mm) the slope near centre is 0.95 and the worst error over ±30 mm is 1.2 mm. P1's sensors sat at 1.5 mm "for contrast"; the dead zone caused a ±2 mm hunt on straights that no gain removed, and raising the bar to 3 mm cured it (Figure 34.2).

> **Status of the principle: Centroid interpolation of an array**
> *Approximate.* Linear to within about 1 mm when the footprint width is at least a third of the pitch and the line covers at least two sensors; it saturates beyond the outermost sensor (here ±33 mm) and degrades when the line width approaches the pitch.

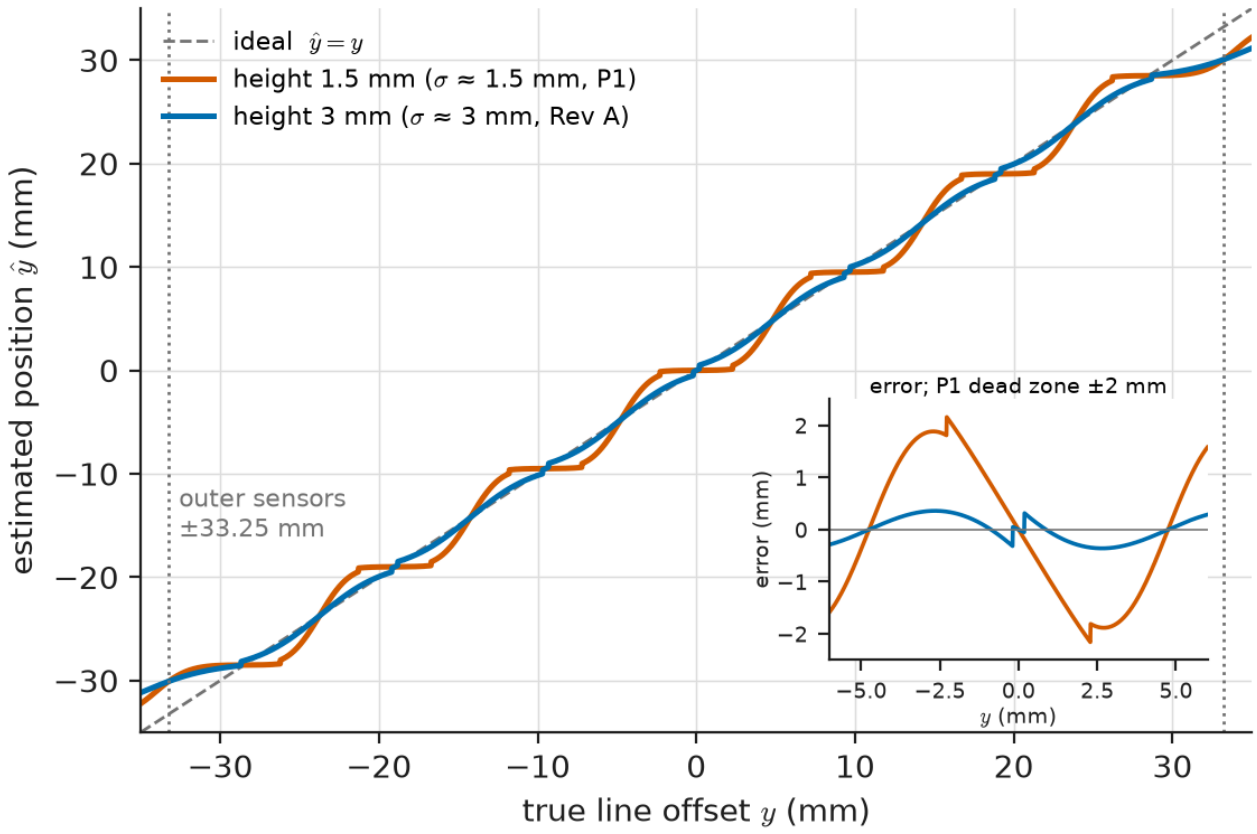


Figure 34.2 — Estimated against true line position for the 8-sensor array at two sensor heights, showing the dead zone at low height.

Lookahead and line loss

The bar's distance ahead of the axle, the **lookahead** L, sets how early the robot sees a curve and, as §34.6 shows, how much phase lead the geometry gives the steering loop. P1 used 30 mm; the design uses 80 mm. On a 100 mm arc with the bar centred on the line, the axle runs inside on a radius $\sqrt{0.100^2 - 0.080^2} = 60$ mm, a 40 mm cut that the rules allow because the footprint still covers the line; a longer arm would also lengthen the body beyond 150 mm.

Line loss is detected when the summed normalised signal falls below 300 (less than about a third of one sensor sees black). The firmware then steers hard towards the side where the line was last seen, at reduced speed, for up to 150 mm of travel (enough to bridge the 20 mm gap and recover from a curve overshoot) before stopping (R7).

34.3 Motor selection

> **Boundary.**
> The brushed DC motor's linear torque–speed characteristic and the motor–load matching procedure are owned by *Electrical Energy and Machines*, Ch. 5 §5.8 and Ch. 10 §10.1; Chapter 14 owns battery and motor sizing for mobile robots in general.

What the motors must do

The motors must give at least 2.0 m/s on the straights, 0 to 1.5 m/s within about 200 mm of a curve exit, and controllable speed down to 0.3 m/s. Grip limits acceleration anyway: silicone tyres on melamine ($\mu \approx 0.8$) allow at most $\mu g \approx 7.8\ \mathrm{m/s^2}$.

The candidates are micro metal gearmotors sharing one 6 V high-power armature with three ratios. At the output shaft and 6 V: 3200 rpm and 19.6 mN·m (10:1), 1050 rpm and 58.8 mN·m (30:1), 640 rpm and 98.1 mN·m (50:1); stall current 1.6 A, free current 0.10 A. The 2S pack at 7.4 V scales speed and stall torque by 7.4/6, acceptable for short racing duty that averages far below stall. The listing sweeps the ratios with the 170 g robot, 32 mm wheels, rolling resistance 0.02, the grip limit and the rotor inertia reflected through the gearbox.

```
 import numpy as np

M, R_W, CRR, MU, VB = 0.170, 0.016, 0.02, 0.8, 7.4      # robot kg, wheel radius m, rolling, grip,
battery V
J_ROT = 0.08e-7                                           # rotor inertia, kg m^2 (0.08 g cm^2,
typical)
motors = {"10:1": (10, 3200, 0.0196), "30:1": (30, 1050, 0.0588), "50:1": (50, 640, 0.0981)}
I_STALL6, I_FREE6 = 1.6, 0.10                             # A at 6 V (same armature for all
ratios)

for name, (N, n0_6, ts_6) in motors.items():
    k = VB / 6.0                                          # full-duty voltage scaling of free
speed and stall
    w0 = n0_6 * k * 2 * np.pi / 60                        # free speed at wheel, rad/s
    ts = ts_6 * k                                         # stall torque at wheel, N m
    m_eff = M + 2 * J_ROT * N**2 / R_W**2                 # rotor inertia reflected through
gearbox
    v, x, t, dt = 0.0, 0.0, 0.0, 1e-4
    while v < 1.5 and t < 3.0:
        tq = ts * (1 - v / R_W / w0)                      # linear torque-speed line per motor
        f = min(2 * tq / R_W, MU * M * 9.81) - CRR * M * 9.81
        v, x, t = v + f / m_eff * dt, x + v * dt, t + dt
    vmax = w0 * R_W * (1 - CRR * M * 9.81 * R_W / 2 / ts)
    cnt_mm = 12 * N / (2 * np.pi * R_W * 1000)
    print(f"{name}: v_top {vmax:4.2f} m/s, m_eff {1000 * m_eff:5.0f} g, "
          f"0-1.5 m/s {'%.0f ms' % (1000 * t) if v >= 1.5 else 'never':>7s} in {1000 * x:4.0f} mm,
"
          f"stall {I_STALL6 * k:4.2f} A, {cnt_mm:4.2f} counts/mm")
```

```
 10:1: v_top 6.54 m/s, m_eff   176 g, 0-1.5 m/s  203 ms in  152 mm, stall 1.97 A, 1.19 counts/mm
30:1: v_top 2.16 m/s, m_eff   226 g, 0-1.5 m/s  261 ms in  196 mm, stall 1.97 A, 3.58 counts/mm
50:1: v_top 1.32 m/s, m_eff   326 g, 0-1.5 m/s   never in 3739 mm, stall 1.97 A, 5.97 counts/mm
```

The 50:1 motor cannot reach 2.0 m/s. The 10:1 motor reaches 1.5 m/s sooner only because it is grip-limited, and would race at 15–30 % PWM duty (23 % at 1.5 m/s) with 1.19 counts/mm, where small duty errors are large speed errors. The **30:1 motor** is chosen: 69 % duty at 1.5 m/s, 2.16 m/s top speed, 3.58 counts/mm.

Worked example — the inertia that is not on the scale. A rotor of 0.08 g·cm² (8×10^{-9} kg m^2) seems negligible. Seen at the wheel through a 30:1 gearbox it is $JN^2 = 8 \times 10^{-9} \times 900 = 7.2 \times 10^{-6}$ kg m^2, equivalent to a mass of $JN^2/r^2 = 7.2 \times 10^{-6}/0.016^2 = 28$ g per motor, 56 g for two: a third of the robot's 170 g. At 50:1 it would be 78 g per motor. Acceleration, braking and the yaw response all see this mass; an analysis that uses only the scale reading over-predicts acceleration by 33 %.

> **Status of the principle: Reflected inertia through a gearbox**
> *Derived.* From energy conservation across an ideal gear stage, $J_{wheel} = N^2 J_{rotor}$; gear friction adds losses but not inertia, and the result holds for any ratio.

Encoders and speed resolution

A 12-count magnetic encoder on the motor shaft (both edges of both channels) gives 360 counts per wheel revolution, 0.279 mm per count. At 1.5 m/s that is 5.4 counts per millisecond, so a 1 ms difference quantises speed in 19 % steps. The firmware measures over a sliding 5 ms window instead: 3.7 % resolution at 1.5 m/s, 19 % at 0.3 m/s (acceptable only because low speed occurs in short curve segments). The window adds about 2.5 ms of delay inside the wheel loop, part of the 25 ms closed-loop wheel lag used in §34.6.

34.4 Circuit design

> **Boundary.**
> H-bridge operation and decay modes are in *Electronics and Computing*, Ch. 32 §32.2; buck converter operation and efficiency in Ch. 30 §30.1 and §30.8. Motor-driver selection practice for robots is owned by Chapter 15.

Component selection

Function	Options considered	Choice and reason
MCU	ATmega328P (no encoder timers); RP2040 (4 ADC inputs); ESP32-S3 (noisy ADC, more power); STM32G431KB (170 MHz M4F, 12-bit ADC with DMA, hardware quadrature on TIM3/TIM4, TIM1 PWM)	**STM32G431KB**: reads 8 sensors, 2 encoders and drives 4 PWM channels in hardware
Motor driver	L298N (≈ 2 V bipolar drop); TB6612FNG (1.2 A continuous); DRV8833 (2.7–10.8 V, 1.5 A RMS, 2 A peak, current chopping)	**DRV8833**: low on-resistance, adjustable current limit
3.3 V supply	LDO (AP2112-class); buck (TPS62162-class, 3–17 V, 1 A)	**Buck** (worked example below)
Battery	1S LiPo with boost; 2S LiPo 300 mAh 30C; 6 × NiMH AAA	**2S LiPo**: direct motor drive, light
Telemetry	Wired UART; BLE UART module; Wi-Fi	**BLE module**: wireless logging, 8 mA

Worked example — LDO or buck for the 3.3 V rail. The 3.3 V loads average 78.6 mA (§34.7). An LDO from a full pack dissipates $(8.4 - 3.3) \times 0.0786 = 0.40$ W, a 100 K rise in SOT-23-5 ($\theta_{JA} \approx 250$ K/W), and

draws all 78.6 mA from the battery. A buck at 88 % draws $78.6 \times 3.3/(0.88 \times 7.4) = 39.8$ mA, half as much, and dissipates about 35 mW; its ripple is filtered by 22 µF and a ferrite bead into the analog supply.

The power and drive section

The pack connects through a switch and a P-channel MOSFET for reverse-polarity protection to the VM bus (6.4–8.4 V; stop at 6.6 V), which feeds the DRV8833 (47 µF polymer plus 100 nF at its VM pin) and the buck.

The DRV8833's current regulation trips at $I_{trip} = 0.2\ \text{V}/R_{ISEN}$. With $R_{ISEN} = 0.15\ \Omega$ each channel is limited to 1.33 A, below the driver's 1.5 A RMS rating and below the motor's 1.97 A stall current at 7.4 V. The firmware's acceleration limit keeps racing currents far below that (0.36 A per motor at 5 m/s²; §34.7); the hardware limit exists for stalls and mistakes.

Worked example — why the robot never reverses the motors to brake. P1 braked by reversing PWM. At 2.0 m/s the back-EMF is about $7.4 \times 2.0/2.16 = 6.85$ V, so reversal drives $(7.4 + 6.85)/3.75 = 3.8$ A through the 3.75 Ω armature, beyond the DRV8833's overcurrent trip; the bridge shut down and retried repeatedly and the robot coasted into the curve. The design brakes in **slow decay** (both low-side switches on): current is set by back-EMF alone, at most $6.85/3.75 = 1.8$ A, chopped to 1.33 A, and the 6 m/s² deceleration limit keeps it far lower in normal running.

The sensor channel

Each IR LED has a 100 Ω resistor from 3.3 V; the eight cathodes return through one logic-level N-MOSFET (AO3400-class) switched by a GPIO, 160 mA when on. Each phototransistor has a 10 kΩ load to 3.3 V: about 0.3 V over white and 3.2 V over black at 3 mm height, settling within about 60 µs after the LEDs switch. No filter capacitor is fitted, because it would slow the settling that the differencing relies on.

The battery divider (20 kΩ over 10 kΩ, 100 nF) maps 8.4 V to 2.80 V at the ADC, draws 0.28 mA and is read once per cycle.

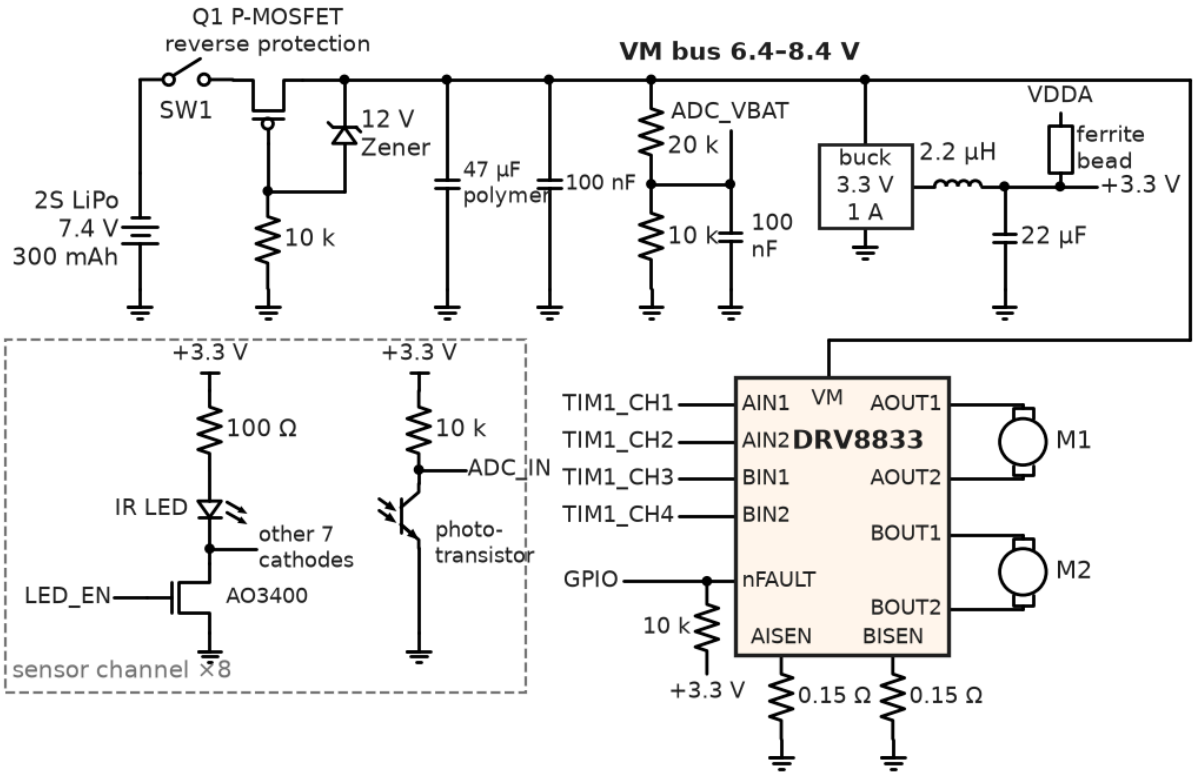


Figure 34.3 — Power entry, motor drive and one sensor channel with component values.

Bill of materials

Item	Qty	Unit price	Line total
MCU STM32G431KB (LQFP32)	1	$4.10	$4.10
Dual H-bridge DRV8833	1	$1.60	$1.60
3.3 V 1 A buck regulator with inductor	1	$1.90	$1.90
Reflective sensor, QRE1113 class	8	$0.62	$4.96
MOSFETs (LED switch, reverse protection)	1 set	$0.60	$0.60
Micro metal gearmotor 30:1, 6 V high-power, extended shaft	2	$21.95	$43.90
Magnetic encoder kit (pair)	1	$8.95	$8.95
Silicone wheels 32 mm (pair)	1	$6.95	$6.95
2S LiPo 300 mAh 30C	1	$8.50	$8.50
BLE UART module	1	$6.00	$6.00
2-layer 1.6 mm PCB used as chassis (share of 5)	1	$4.80	$4.80
Passives, connectors, switch, LEDs	1 set	$6.50	$6.50
Motor brackets (pair) and ball caster	1 set	$7.50	$7.50
IR start-module receiver	1	$4.00	$4.00
Total			**$110.26**

Indicative 2026 distributor and robot-shop prices, one-off quantities; R10's $150 limit leaves $39.74 for spares and the Revision B changes. The motors are 40 % of the cost, which is typical: the electronics of a small robot are cheap, the electromechanics are not.

34.5 Embedded firmware

> **Boundary.**
> Timers, interrupts, latency and watchdogs are owned by *Electronics and Computing*, Ch. 27 §27.2, §27.4, §27.10 and §27.11; firmware architecture and fault handling as a design practice by Chapter 10.

One deterministic 1 ms cycle

The firmware is a timer-driven super-loop with one interrupt doing all control work. TIM6 fires at 1 kHz and starts a fixed sequence (Figure 34.4): scan the eight sensors with the LEDs off (DMA), switch the LEDs on, wait 80 µs on a one-shot timer, scan again, switch the LEDs off, then compute position, steering, wheel speeds and PWM. The background loop handles telemetry, the BLE UART and the button.

Stage in the 1 ms cycle	Time
ADC scan, LEDs off (8 × 60 cycles at 42.5 MHz, DMA)	11.3 µs
LED settle (one-shot timer; LED duty about 10 %)	80 µs
ADC scan, LEDs on	11.3 µs
Position (normalise, centroid)	6 µs
Steering PD, wheel PI × 2, PWM update	5 µs
State machine, safety, telemetry packing	5 µs
Sample to PWM update (budget 500 µs; margin 381 µs)	**≈ 119 µs**

The latency from the LED-on sample to the new PWM duty is about 30 µs; the loop's effective delay is dominated by the 1 ms sample period (half a period of zero-order hold) and the derivative filter, about 2 ms in total, the figure used in §34.6.

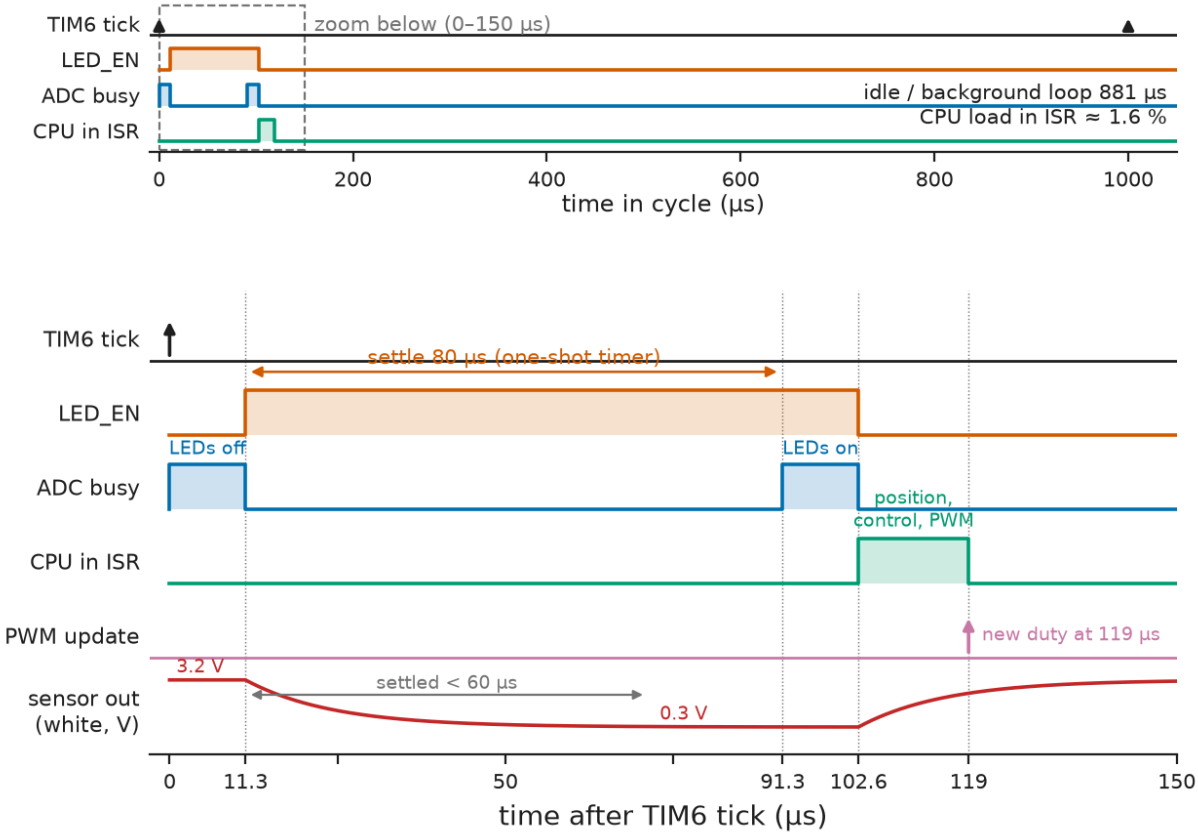


Figure 34.4 — One 1 ms control cycle: sensor scans with LEDs off and on, computation and PWM update.

Line position in fixed point

The position routine takes the two scans, cancels ambient light by subtraction and normalises against the calibration. A reflective (white) surface pulls the phototransistor output *down* when the LEDs are on, so the reflected signal is `off - on`.

```
 /* line_pos.c -- line position from an 8-channel reflectance array (C11) */
#include <stdbool.h>
#include <stdint.h>

#define NS        8
#define PITCH_UM  9500        /* sensor pitch, micrometres */
#define N_MIN     50          /* ignore normalised readings below 5 % black */
#define SUM_MIN   300         /* below this the line is considered lost */

typedef struct { int16_t white[NS], black[NS]; } cal_t;   /* reflected signal, from sweep */

/* raw_on/raw_off: ADC counts with IR LEDs on and off; sensor 0 is the rightmost.
 * Writes the line position in micrometres (+ = line left of centre); false if lost. */
bool line_position(const uint16_t raw_on[NS], const uint16_t raw_off[NS],
                   const cal_t *cal, int32_t *pos_um)
{
    int32_t sum_n = 0, sum_nx = 0;
    for (int i = 0; i < NS; ++i) {
        int32_t refl = (int32_t)raw_off[i] - (int32_t)raw_on[i];  /* ambient cancelled */
        int32_t span = (int32_t)cal->white[i] - (int32_t)cal->black[i];
        if (span < 200) continue;                     /* dead or uncalibrated channel */
        int32_t n = ((int32_t)cal->white[i] - refl) * 1000 / span;   /* 0 white, 1000 black */
        if (n < N_MIN) continue;
        if (n > 1000) n = 1000;
        int32_t x = (2 * i - (NS - 1)) * PITCH_UM / 2;              /* -33250 ... +33250 um */
        sum_n += n;
        sum_nx += n * x;                              /* at most 8 * 1000 * 33250: fits int32 */
    }
    if (sum_n < SUM_MIN) return false;
    *pos_um = sum_nx / sum_n;
    return true;
}
```

A channel whose calibrated white-to-black span is below 200 counts is skipped rather than trusted: a sensor fouled by dust or a cracked LED solder joint then degrades resolution instead of injecting a false position. The host build compiles this file unchanged and runs it against recorded scans and the synthetic transfer curve of Figure 34.2 as a unit test.

States and transitions

State	Behaviour	Exit
IDLE	Motors off, telemetry on	Button: CAL
CAL	Pivot ±40° over the line twice, record white and black per channel	Spans valid: ARMED; else FAULT
ARMED	Wait for start signal	Start: RUN_MAP, or RUN_FAST if map valid
RUN_MAP	0.7 m/s (below the tightest curve's limit), record curvature every 10 mm	Marker seen twice: RUN_FAST
RUN_FAST	Speed profile and curvature feed-forward from map (§34.9)	Line lost: LOST; stop: STOP
LOST	Steer to last side seen at 0.5 m/s for up to 150 mm	Found: previous state; 150 mm: STOP
STOP	Slow-decay brake, then motors off (also on VBAT < 6.6 V or nFAULT)	Button: IDLE

An independent watchdog (IWDG, 20 ms timeout) is refreshed only at the end of a complete control cycle; a hung ISR or a stuck DMA leaves the motors' last PWM for at most 20 ms before the reset, and the reset state of every PWM pin is low (both bridge inputs low: coast).

34.6 Control algorithm

> **Boundary.**
> PID structure, tuning and digital implementation are owned by *Circuits, Signals and Communications*, Ch. 35 §35.2, §35.3 and §35.8; steady-state error by system type in Ch. 34 §34.7. The kinematic model of a differential-drive robot is owned by Chapter 14.

Structure: a cascade

The controller is a cascade. Inner loops, one per wheel at 1 kHz, make each wheel follow a speed command with a PI controller around a feed-forward of the free-wheel duty for that speed, corrected for battery voltage. The outer loop turns line error into a yaw-rate command ω with a PD controller and splits it into $v_{L,R} = v \mp \omega b/2$ (track $b = 110$ mm). The inner loops turn the outer loop's actuator into a fast yaw-rate source, closed-loop time constant 25 ms, independent of battery voltage and motor mismatch; P1 had none, and its yaw rate lagged by the motor and chassis time constant, about 60 ms.

The steering plant and why speed destabilises it

For small angles the lateral error measured at the sensor bar responds to yaw rate as $E(s)/\Omega(s) = (Ls + v)/s^2$ (Chapter 14's kinematics, linearised): a double integrator with a zero at $s = -v/L$. The zero is the geometric phase lead that the lookahead gives; the gain rises with speed v. The listing evaluates the least-

damped closed-loop pole for three designs, with the wheel loop as a first-order lag and the loop delay as a first-order Padé approximation.

```
 import numpy as np
P = np.poly1d

def worst_pole(v, L, kp, kd, tw, T, tf=0.002):
    """Least-damped closed-loop pole of the steering loop at speed v (m/s)."""
    plant_n, plant_d = P([L, v]), P([1, 0, 0])            # e_s/omega = (L s + v)/s^2
    lag, dly_n, dly_d = P([tw, 1]), P([-T / 2, 1]), P([T / 2, 1])   # wheel loop, Pade delay
    c_n, c_d = P([kp * tf + kd, kp]), P([tf, 1])         # PD with filtered derivative
    r = (plant_d * lag * dly_d * c_d + plant_n * dly_n * c_n).r
    z = -r.real / np.abs(r)
    i = np.argmin(z)
    return z[i], abs(r[i].imag) / (2 * np.pi)

designs = {   # name: (lookahead m, wheel lag s, delay s, gain schedule)
    "P1 prototype": (0.030, 0.060, 0.0125, lambda v: (250, 5)),
    "Rev A fixed":  (0.080, 0.025, 0.002, lambda v: (300, 8)),
    "Rev A sched.": (0.080, 0.025, 0.002, lambda v: (300, 8 * max(v, 1.0))),
}
print("v (m/s)      " + "".join(f"{v:>12.1f}" for v in (0.3, 0.6, 1.0, 1.5, 2.0)))
for name, (L, tw, T, gains) in designs.items():
    cells = []
    for v in (0.3, 0.6, 1.0, 1.5, 2.0):
        z, f = worst_pole(v, L, *gains(v), tw, T)
        cells.append(f"{z:5.2f}/{f:3.1f}Hz")
    print(f"{name:13s}" + "".join(f"{c:>12s}" for c in cells))
```

```
 v (m/s)               0.3         0.6         1.0         1.5         2.0
P1 prototype   0.18/1.5Hz  0.00/1.9Hz -0.10/2.3Hz -0.16/2.7Hz -0.19/3.0Hz
Rev A fixed    1.00/0.0Hz  0.90/0.9Hz  0.71/1.9Hz  0.59/2.8Hz  0.51/3.4Hz
Rev A sched.   1.00/0.0Hz  0.90/0.9Hz  0.71/1.9Hz  0.70/2.1Hz  0.79/2.0Hz
```

Each cell is the damping ratio and frequency of the least-damped pole. The P1 row is the opening story in numbers: damping 0.18 at 0.3 m/s (it "looked fine"), zero at 0.6 m/s with a 1.9 Hz oscillation (the observed weave), negative above, and no gain pair stabilises it at 1 m/s. Three changes fix it: an 80 mm lookahead moves the plant zero nearer the origin, where its phase lead helps at crossover; the inner wheel loops cut the lag from 60 to 25 ms; and the 1 kHz cycle without averaging cuts the delay from 12.5 to 2 ms. With fixed gains the damping still falls to 0.51 at 2 m/s, because the plant gain rises with v; scheduling $K_d = 8\max(v, 1)$ holds it at about 0.7 (minimum 0.69, near 1.3 m/s; Figure 34.5).

> **Status of the principle: Lookahead steering model**
> *Approximate.* Linearised kinematics for small heading errors (below about 15°), no tyre slip, constant speed during the transient; at high lateral acceleration the tyres slip and the real loop is less damped than predicted, which §34.8's track tests check.

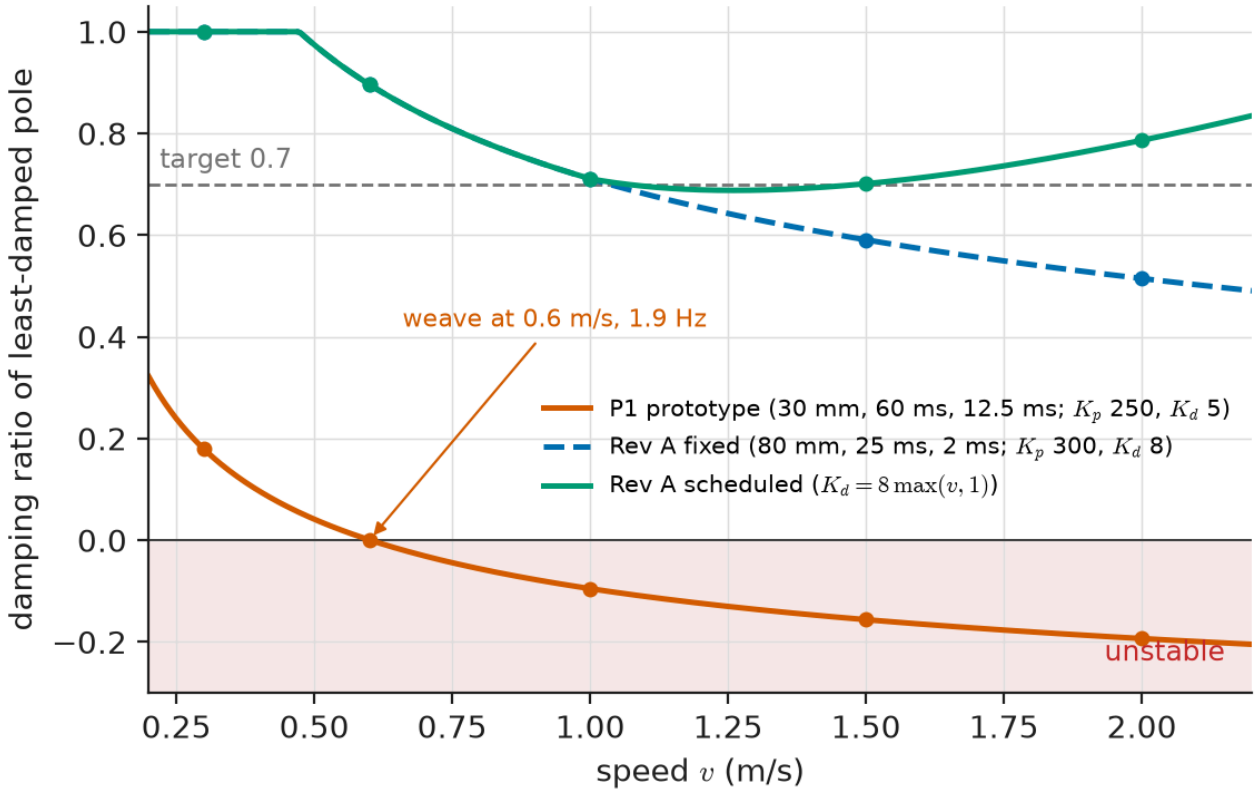


Figure 34.5 — Damping ratio of the least-damped steering pole against speed for the prototype and the final design with fixed and scheduled gains.

The control code

```
 /* control.c -- 1 kHz steering PD and wheel-speed PI loops (C11; HAL names illustrative) */
#include <stdint.h>

#define DT        0.001f
#define TRACK_M   0.110f              /* wheel track b */
#define M_PER_CNT 0.000279f           /* 32 mm wheel, 360 counts per wheel revolution */
#define WIN       5                   /* speed window, ms */
#define V_FREE    2.16f               /* free speed at 7.4 V, full duty */

typedef struct { float kp, ki, integ, lo, hi; } pi_t;

extern volatile uint16_t enc_cnt[2];  /* TIM3/TIM4 encoder counters, left and right */
extern volatile float vbat;           /* filtered battery voltage, V */
extern void pwm_set(int side, float duty);   /* -1..+1, slow-decay drive of DRV8833 */

static pi_t wheel[2] = { {1.2f, 30.0f, 0.0f, -0.6f, 0.6f}, {1.2f, 30.0f, 0.0f, -0.6f, 0.6f} };
static uint16_t hist[2][WIN];
static uint32_t k;
static float e_prev, d_filt;

static float pi_step(pi_t *c, float err)
{
    float u = c->kp * err + c->integ;
    if (u > c->hi)      u = c->hi;                /* clamp; integrate only when unsaturated */
    else if (u < c->lo) u = c->lo;
    else                c->integ += c->ki * err * DT;
    return u;
}

void control_step(float v_ref, float e_m, float omega_ff)   /* e_m: line error (m), + = left */
{
    float kd = 8.0f * (v_ref > 1.0f ? v_ref : 1.0f);         /* derivative gain scheduled */
    d_filt += ((e_m - e_prev) / DT - d_filt) * 0.333f;       /* tf = 2 ms, about 80 Hz */
    e_prev = e_m;
    float omega = omega_ff + 300.0f * e_m + kd * d_filt;     /* yaw-rate command, rad/s */
    float v_w[2] = { v_ref - omega * TRACK_M / 2, v_ref + omega * TRACK_M / 2 };

    uint32_t slot = k % WIN;                                 /* written WIN cycles ago */
    for (int s = 0; s < 2; ++s) {
        uint16_t c = enc_cnt[s];
        float v_meas = (float)(int16_t)(uint16_t)(c - hist[s][slot]) * M_PER_CNT / (WIN * DT);
        hist[s][slot] = c;
        float ff = v_w[s] / (V_FREE * vbat / 7.4f);          /* duty for this speed, no load */
        pwm_set(s, ff + pi_step(&wheel[s], v_w[s] - v_meas));
    }
    ++k;
}
```

Three details carry the design. The derivative acts on the filtered measurement, so state changes do not kick. The PI output is a correction around the feed-forward, clamped to ±0.6 duty with conditional integration, so a lifted wheel or stall does not wind up the integrator. The 16-bit encoder difference is formed unsigned and reinterpreted as signed, correct across counter wrap-around on a two's-complement target.

Worked example — the steady error on a curve, and why feed-forward. The loop is type 2 with respect to the line, so on a constant-curvature arc it settles with $e_{ss} = \omega_{req}/K_p = (v/R)/K_p$. On the 100 mm curve at 0.775 m/s, $e_{ss} = 7.75/300 = 25.8$ mm, three quarters of the way to the array's edge; on the 250 mm curve at 1.22 m/s, 16.3 mm. Raising K_p costs damping. Instead, in RUN_FAST the firmware supplies $\omega_{ff} = v\,\kappa_{map}$ from the track map (§34.9) and the PD loop corrects only the residual: the measured error on the 100 mm curve fell from 24 mm to 4 mm.

34.7 Power budgeting

> **Boundary.**
> Battery capacity, C-rate and runtime fundamentals for robots are owned by Chapter 14; cell chemistry and BMS by Chapter 23.

Average current and running time

Motor current was estimated from the lap energy and checked on the bench. On the mapped profile (§34.9) the robot gains 1.38 J of kinetic energy per lap (effective mass 226 g) and loses 0.50 J to rolling: 1.88 J per 8.8 s lap, 4.7 J electrical at about 40 % efficiency, 72 mA at 7.4 V. No-load friction and iron losses at racing speed add about 0.2 A, for 0.28 A.

Load	Rail	Rail current (mA)	Battery current (mA)	Allowance	Budgeted (mA)
Motors, racing duty	VM	—	280	+25 %	350
DRV8833 quiescent	VM	—	1.7	—	1.7
Battery divider	VM	—	0.25	—	0.25
STM32G431 at 170 MHz with ADC, timers	3.3 V	40			
IR LEDs, 160 mA at 10 % duty	3.3 V	16			
Phototransistor loads	3.3 V	2.6			
BLE module (connected, 50 Hz telemetry)	3.3 V	8			
Encoders (2 × Hall sensors)	3.3 V	8			
Status LEDs	3.3 V	4			
3.3 V rail total through 88 % buck	3.3 V	78.6	39.8	+10 %	43.8
Total			**321.8**		**395.8**

Worked example — running time with margin. A 300 mAh 2S LiPo used down to the 3.3 V/cell stop under load delivers about 80 % of its capacity, 240 mAh. At the estimated 321.8 mA that is $240/321.8 = 0.746$ h = 44.8 min; at the budgeted 395.8 mA, 36.4 min, an 82 % margin on R5's 20 min. A 150 mAh pack would give 18.2 min and fail.

> **Status of the principle: Runtime from capacity and average current**
> *Derived.* Runtime equals usable charge divided by average current; it assumes capacity independent of discharge rate, which holds for LiPo cells at the 1–2C average of this robot but not at the 10–20C peaks, which are too short to matter here.

Peak current and brown-out

Hard acceleration at the firmware's 5 m/s² limit needs $F = 0.226 \times 5 + 0.03 = 1.16$ N, 9.3 mN·m and about 0.36 A per motor. With the LED pulse (about 81 mA from the battery) and the electronics the peak is under 0.9 A, 3C for the pack, and its internal resistance (about 2 × 60 mΩ) drops 0.1 V. The worst case the hardware allows, both bridges at the 1.33 A limit, sags the pack by 0.32 V: the buck input stays above 6 V, far from the MCU's brown-out level, as the R8 scope test on the 3.3 V rail confirmed.

The mass budget

Item	Mass (g)
PCB chassis 150 × 120 mm with cut-outs	48
Motors (2), brackets	25
Encoders (2)	3
Wheels and hubs (2)	14
2S LiPo 300 mAh with lead	21
Sensor bar with 8 sensors on a carbon arm	19
Electronics (MCU, driver, buck, passives), BLE, start receiver	19
Ball caster, wiring, connectors, fasteners	21
Total (R4 limit 250 g)	**170** (margin 80 g)

The 80 g margin is deliberate: Revision B's downforce fan (§34.10) needs about 25 g.

34.8 Testing

> **Boundary.**
> Test planning, regression and fault diagnosis as a method are owned by Chapter 4; robot field testing and hardware-in-the-loop by Chapter 17.

Test plan and results

Each test has its limit written before the build. Results are representative of the Revision A robot:

Req.	Test	Limit	Result	Pass/fail
—	T1 line-position unit test (host build)	≤ 1.5 mm over ±30 mm	1.2 mm max, 0.3 mm RMS	Pass
—	T2 sensor transfer on a linear stage, 3 mm height	slope 0.9–1.1; ≤ 1.5 mm	slope 0.96; 1.4 mm	Pass
—	T3 wheel speed step 0.5 → 1.5 m/s on a stand	63 % rise ≤ 30 ms; overshoot ≤ 10 %	24 ms; 6 %	Pass
—	T4 steering: 15 mm start offset, 1.0 m/s	±3 mm in 0.4 s; overshoot ≤ 30 %	0.31 s; 9 %	Pass
R3	T5 tracking at 1.0 m/s (telemetry)	≤ 6 mm RMS	3.1 mm (straights 1.8, curves 4.2)	Pass
R2	T6 lap time, 5 laps, light gate	≤ 10.0 s	9.42 s (σ 0.06 s)	Pass
R1	T7 50 consecutive laps	0 departures	1 departure at lap 37 (first build)	**Fail**, then pass
R5, R8	T8 endurance from full charge	≥ 20 min, no reset	39 min, no reset	Pass
R6	T9 2000 lux and 100 Hz floodlight	T5 limit	3.3 mm RMS	Pass
R6	T9 low sun across the track	T5 limit	Line lost twice (first build)	**Fail**, then pass
R7	T10 line lifted; remote stop; motor lead shorted	≤ 0.5 s	0.21 s; 0.05 s; 0.02 s	Pass
R4	Mass and footprint	≤ 250 g; ≤ 200 × 150 mm	172 g; 150 × 120 mm	Pass

The T7 departure was at the crossing: approaching with a few degrees of yaw, the crossing line lay diagonally across the array, the centroid jumped, and the robot followed the crossing line for 20 mm. The firmware now ignores scans in which more than five sensors see black and holds the previous position for up to 30 mm. The T9 failure was sunlight saturating the phototransistors: the LED-off reading fell to near 0 V, leaving no headroom for the LED-on signal. The fix is a 2 g black shroud and a check that flags a channel whose LED-off reading falls below 0.4 V.

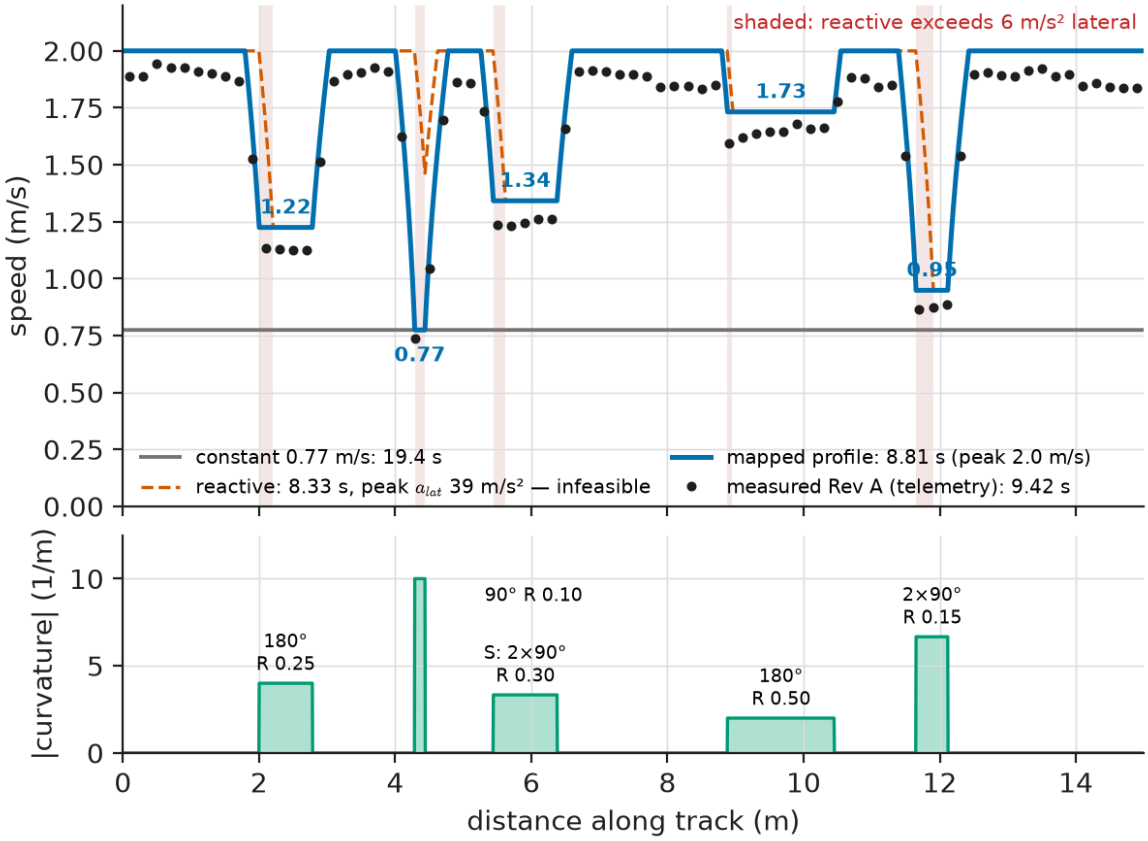


Figure 34.6 — Speed along the 15 m reference track for three strategies, with track curvature and the measured lap.

Failure analysis

Failure mode	Effect	Cause	Detection	Mitigation
Oscillation above 0.6 m/s (P1)	Weave, departure	Short lookahead, open-loop wheels, 12.5 ms delay	Track test	80 mm lookahead, inner PI, 1 kHz, gain schedule
±2 mm hunt on straights (P1)	Wasted speed	Centroid dead zone, sensors at 1.5 mm	T2	Sensor height 3 mm
Bridge trips when braking (P1)	Coasts into curve	Reverse braking, 3.8 A	nFAULT log	Slow-decay brake, 1.33 A limit
Centroid jump at crossing	Follows crossing line	> 5 sensors black, yawed approach	T7	Ignore wide-black scans for 30 mm
Sun saturates sensors	Line lost	LED-off level near 0 V	T9	Shroud; LED-off level check
Dust on a sensor window	Position bias	Low calibrated span	Span check at CAL	Skip channel; clean before runs
Encoder magnet slips	Wheel loop runs away	Press-fit magnet loosens	Speed-versus-duty check	STOP on > 30 % mismatch for 50 ms
Battery under-voltage	Cell damage	Long practice	VBAT every cycle	STOP at 6.6 V
Firmware hang	Uncontrolled run	ISR or DMA stall	IWDG 20 ms	Reset to coast
Map mismatch	Brakes in wrong place	Track piece moved	Lap length differs > 2 %	Discard map, RUN_MAP

34.9 Performance optimisation

With the loop stable and the robot reliable, lap time is set by how fast it can go *where it is allowed to*. The physics limit in a curve is lateral grip: $v_{max} = \sqrt{a_{lat} R}$, with $a_{lat} = 6.0$ m/s^2 used as the design value (about 75 % of the grip limit, leaving room for control error). That gives 0.77 m/s on the 100 mm curve and 1.73 m/s on the 500 mm curve.

> **Status of the principle: Grip-limited cornering speed**
> *Approximate.* From the centripetal acceleration v^2/R bounded by the tyre friction coefficient; the coefficient itself is empirical, varies with surface dust and tyre temperature by ±15 %, and is shared with longitudinal acceleration on combined braking and turning.

Three speed strategies

The listing computes lap times on the reference track for a constant speed, a reactive strategy that slows only when it senses a curve, and a mapped profile that knows where the curves are and brakes before them, with

acceleration and braking limits of 5 and 6 m/s². The track is entered as an unrolled list of segments; turn direction does not change the speed limit, so it is not recorded.

```
 import numpy as np

# reference track unrolled in driving order: (length m, radius m; 0 = straight), 15.0 m lap
track = [(2.0, 0), (np.pi * 0.25, 0.25), (1.5, 0), (np.pi / 2 * 0.10, 0.10), (1.0, 0),
         (np.pi / 2 * 0.30, 0.30), (np.pi / 2 * 0.30, 0.30), (2.5, 0), (np.pi * 0.50, 0.50),
         (1.2, 0), (np.pi / 2 * 0.15, 0.15), (np.pi / 2 * 0.15, 0.15), (0.0, 0)]
track[-1] = (15.0 - sum(s for s, _ in track), 0)            # closing straight
V_TOP, A_LAT, A_ACC, A_DEC, DS = 2.0, 6.0, 5.0, 6.0, 0.005  # m/s, m/s^2, m/s^2, m/s^2, m

r = np.concatenate([np.full(int(round(s / DS)), rad) for s, rad in track])
vlim = np.where(r > 0, np.sqrt(A_LAT * np.where(r > 0, r, 1)), V_TOP).clip(max=V_TOP)

def profile(vlim, laps=2):
    v = np.tile(vlim, laps)                                 # two laps: the second is periodic
    for i in range(1, v.size):                              # forward pass: acceleration limit
        v[i] = min(v[i], np.sqrt(v[i - 1]**2 + 2 * A_ACC * DS))
    for i in range(v.size - 2, -1, -1):                     # backward pass: braking limit
        v[i] = min(v[i], np.sqrt(v[i + 1]**2 + 2 * A_DEC * DS))
    return v[vlim.size:]

lap_time = lambda v: np.sum(DS / v)
v_const = np.full_like(vlim, vlim.min())
def reactive(vlim, laps=2):                                 # slows only after curve is sensed
    v = np.tile(vlim, laps)
    for i in range(1, v.size):
        up, dn = np.sqrt(v[i - 1]**2 + 2 * A_ACC * DS), np.sqrt(max(v[i - 1]**2 - 2 * A_DEC * DS,
0))
        v[i] = min(v[i], up) if v[i] >= dn else dn
    return v[vlim.size:]
v_map = profile(vlim)
print(f"track {r.size * DS:.1f} m, tightest radius {r[r > 0].min():.2f} m -> {vlim.min():.2f}
m/s")
print(f"constant speed      : {lap_time(v_const):5.2f} s")
v_r = reactive(vlim)
a_lat = np.max(v_r**2 * np.where(r > 0, 1 / np.where(r > 0, r, 1), 0))
print(f"reactive            : {lap_time(v_r):5.2f} s, peak lateral accel {a_lat:4.1f} m/s^2 (limit
{A_LAT})")
print(f"mapped profile      : {lap_time(v_map):5.2f} s, mean {r.size * DS / lap_time(v_map):.2f}
m/s, "
      f"peak {v_map.max():.2f} m/s")
```

```
 track 15.0 m, tightest radius 0.10 m -> 0.77 m/s
constant speed      : 19.36 s
reactive            :  8.33 s, peak lateral accel 39.4 m/s^2 (limit 6.0)
mapped profile      :  8.81 s, mean 1.70 m/s, peak 2.00 m/s
```

Held to the tightest curve's speed everywhere, the robot needs 19.4 s and fails R2 by a factor of two. The reactive robot looks fastest, 8.33 s, but it enters curves at straight-line speed and needs 39 m/s² of lateral acceleration, five times the grip limit: in reality it leaves the track. Speed must be reduced *before* a curve, which needs to know where the curve is; the mapped profile does, laps in 8.81 s at a mean 1.70 m/s, and is what the robot runs.

Worked example — what the map is and how it is built. In RUN_MAP the robot laps at 0.7 m/s and records curvature from the encoders every 10 mm, $\kappa = (v_R - v_L)/(b\,v)$, as a signed byte in units of 0.1 m⁻¹: 1500 bytes per 15 m lap. Two detections of the start/finish side mark close the lap and fix the distance origin. In ARMED the firmware computes the profile with the listing's forward–backward passes (about 3 ms), then in RUN_FAST indexes it by odometric distance, re-zeroed at each marker. Encoder slip drifts about 0.5 % (75 mm) per lap, so braking points are placed 100 mm early. The map also supplies the curvature feed-forward of §34.6.

What each optimisation bought

Step	Change	Lap time (s)	Tracking RMS at 1.0 m/s (mm)
P1	Prototype, 0.5 m/s maximum stable	30+ (unstable above 0.6 m/s)	7.5 at 0.5 m/s
A1	Lookahead 80 mm, sensor height 3 mm, inner PI loops, 1 kHz	15.1 (constant 1.0 m/s, slowed in curves by error threshold)	4.0
A2	Gain scheduling	14.2	3.6
A3	Track map and speed profile	10.3	3.6
A4	Curvature feed-forward from map	9.42	3.1
—	Ideal profile (listing)	8.81	—

The remaining 0.6 s between measured and ideal comes from the 100 mm early braking points, tyre slip in the tightest curve (the measured speed there is 0.71 m/s, not 0.77) and acceleration below 5 m/s² at low battery.

34.10 Revision B

Revision A meets every requirement. Revision B is what the team would build next, each change traced to a measured limitation:

Change	Why (evidence)	Expected effect	Cost and risk
Ducted downforce fan (20 mm impeller, 25 g) under a skirt	Cornering grip-limited; slip in the tightest curve	Twice the normal force: 0.77 → 1.10 m/s on 100 mm curves; ideal lap 7.88 s against 8.81 s	+$15; +0.6 A average, so a 2S 450 mAh pack (+9 g) to keep 21.7 min against R5
Second sensor row 40 mm further ahead, or a linear image sensor	Braking relies on the map; a moved track piece forces a mapping lap	Curve detection without a map	+$10 to $20; more ADC channels
Lower-inertia coreless motors	56 g of 226 g effective mass is rotor inertia	20–25 % faster acceleration	+$40 per pair; brush wear
IMU gyro (ICM-42688-class)	Map curvature from wheel speeds is corrupted by slip	Slip-free curvature; yaw-rate inner loop	+$4; fusion work (Chapter 15)
4-layer PCB, ground plane under sensors	20-count ADC spikes at PWM edges	Lower noise	+$15 per board

The downforce fan is the largest lever: once the loop is stable, this robot is limited by its tyres, not by its electronics or control.

What remains open

Open question	Where it is answered
Differential-drive kinematics, odometry and general line-following technique	Chapter 14
Motor drivers, encoder interfaces and PID implementation on embedded targets in general	Chapter 15
Integrating an IMU and sensor fusion for slip-free yaw rate	Chapter 15; Chapter 17
Hardware-in-the-loop testing of the control loop against a simulated track	Chapter 17
A closed-loop motor drive designed from the motor up, with current control and protection	Chapter 37
How to plan, document and report a project like this one as a capstone	Chapter 40
DC-motor characteristics and motor–load matching theory	*Electrical Energy and Machines*, Ch. 5 and Ch. 10
Controller design and digital control theory	*Circuits, Signals and Communications*, Ch. 35

Next: Chapter 35 builds a smart energy monitor: from mains voltage and current through signal conditioning, power calculation and calibration to a safe, validated, connected meter.

Chapter 35 — Project: Smart Energy Monitor

Part IX · Interdisciplinary Engineering Projects

A household installed a home-built Wi-Fi energy monitor in its consumer unit: one voltage tap, three split-core current transformers on the incomer, the PV inverter and the EV charger. On the electric kettle it agreed with the utility's billing meter to 0.4 %. Over a month it read 4 % more than the billing meter, and at night, with the house "off", it showed about 20 W more than the billing meter's display. Both errors had causes that a kettle test cannot reveal. The night load was a fridge compressor, routers and standby supplies at a power factor near 0.5, and the clamp-on CTs lead their primary current by about 2° at a few amperes: an error of $\delta \tan\varphi \approx 0.035 \times 1.73 = 6\ \%$ at PF 0.5, none at PF 1. And the voltage divider's trace ran beside the current-channel input, coupling a voltage-shaped signal into the current channel, worth a few watts of phantom power at zero current. This chapter designs that monitor properly: measurement, conditioning, power calculation, processing, logging, communication, calibration, safety and validation, each with a requirement and a test.

> **How do a CT, a resistive divider and a 24-bit ADC become an energy measurement good to 1 % at power factor 0.5 and at 0.5 A?**
> **How is a device that touches 230 V mains and talks to a home Wi-Fi network made safe, robust against outages and verifiable against a meter standard?**

Concept	Engineering destination
Voltage and current measurement	Divider and split-core CTs on a 63 A single-phase supply
Signal conditioning	Burden, divider ratio, anti-alias filter, phase matching
Power calculations	P, Q, S, PF, harmonics over cycle-synchronous windows
Embedded processing	4 kSPS accumulation on a dedicated core; energy registers
Data logging	Power-fail-safe registers in FRAM; 130 days of records in flash
Communications	MQTT over TLS, offline buffer and backfill
Calibration	Gain and current-dependent phase tables from a reference meter
Safety	Reinforced isolation, CAT III clearances, fusing, CT open-circuit protection
Validation	IEC 62053-21 class 1 test points, influence and field tests

Roadmap. Measurement chain and architecture (35.1) → conditioning and error budget (35.2) → power definitions and windows (35.3) → firmware on the target (35.4) → logging (35.5) → MQTT (35.6) → calibration (35.7) → safety (35.8) → validation (35.9) → BOM, FMEA and Revision B (35.10).

35.1 Voltage and current measurement

> **Boundary.**
> Chapter 9 (energy monitoring) gives a compact version of this project and points here; Chapter 6 owns current and voltage sensor principles, Chapter 7 signal conditioning, uncertainty and ADC selection, Chapter 11 MQTT, provisioning and IoT security. AC power definitions are in *Circuits, Signals and Communications*, Ch. 9; harmonics and power-quality measurement in *Electrical Energy and Machines*, Ch. 25 §25.3 and §25.9.

Requirements

The monitor is a DIN-rail device for a 230 V, 50 Hz single-phase installation with a 63 A main fuse: one voltage input and three current channels (incomer, PV inverter, EV charger), each bidirectional. It is a monitor, not a billing meter, but it is designed and tested against the class 1 accuracy points of IEC 62053-21 so that its numbers can be trusted next to the utility's.

ID	Requirement	Target	Verification
R1	Active energy accuracy per channel	Class 1 limits of IEC 62053-21 (Ib = 10 A, Imax = 63 A), PF 1, 0.5L, 0.8C	Reference-meter test (§35.9)
R2	Instantaneous quantities	Vrms ±0.5 %, Irms ±1 %, PF ±0.02, f ±0.01 Hz, current THD to 25th harmonic	Reference source
R3	Bidirectional registers	Import and export energy per channel	PV export test
R4	Update rates	10-cycle values; 10 s telemetry; 1 min records	Log inspection
R5	No energy lost at power failure	≤ 1 Wh per outage	100 power cuts at 14 kW
R6	Offline operation	≥ 30 days of 1 min records; backfill on reconnect	Broker outage test
R7	Electrical safety	IEC 61010-1, CAT III 300 V; reinforced isolation to all accessible parts	Design review, hipot 3 kV a.c.
R8	Secure communications	MQTT over TLS 1.2+, signed OTA updates	Penetration checklist (Chapter 11)
R9	Self-consumption	≤ 1.5 W	Power analyser
R10	Cost	BOM ≤ $65 (2026, 100 pcs)	BOM (§35.10)

Sensor and converter choices

Current sensor	Pros	Cons	Verdict
Shunt (250 μΩ manganin)	Excellent linearity, no phase error	One per line conductor, at mains potential; needs cutting into the circuit	Rejected: retrofit must be non-invasive
Split-core CT, 100 A : 50 mA (2000:1)	Clamps on, isolates, cheap	Ratio and phase errors grow at low current; opens hazardously if unburdened	**Chosen**
Rogowski coil with integrator	Wide range, no saturation	Integrator drift; weak signal at 0.5 A	Rejected
Open-loop Hall clamp	DC capable	Offset drift ±50 mA, temperature-dependent	Rejected (no DC needed)

Voltage is measured through a resistive divider referenced to neutral; a small voltage transformer would isolate but adds a few degrees of phase error that varies with voltage.

ADC option	Channels	Resolution, rate	Notes	Verdict
MCU's internal SAR ADC (ESP32-S3)	multiplexed	12-bit, nonlinear	Not simultaneous; INL several LSB	Rejected
Metering AFE (ADE7953, ATM90E26 class)	1 V + 2 I	Energy computed in silicon	Fixed algorithms; two current channels	Rejected: three channels and raw waveforms needed
ADS131M04 simultaneous ΔΣ ADC	4	24-bit, 4 kSPS, per-channel phase trim	All waveform maths in firmware	**Chosen**
Isolated modulator ADC (AMC131M03 class)	3	24-bit, isolation built in	One channel short	Considered for Revision B

The ADS131M04 samples all four channels simultaneously, so voltage–current timing errors come only from the analog front end and the CTs. Its per-channel phase register shifts each channel's sampling instant in steps of one modulator clock, $1/4.096\ \text{MHz} = 244\ \text{ns}$, which is 0.0044° at 50 Hz.

Architecture

Figure 35.1 shows the architecture. The ADC, the divider and the CT burdens sit on a **mains-referenced side** tied to neutral. Everything a person can touch (USB, enclosure, the ESP32-S3 module, its antenna) is on the **SELV side**. A reinforced digital isolator with integrated isolated power (ISOW7741 class: SCLK, CS and DIN forward, DOUT back) and a single-channel reinforced isolator (ISO7710 class, for DRDY) are the only crossings. An 8.192 MHz oscillator on the mains side clocks the ADC, so sampling is set by the ADC's crystal, not by firmware timing. One core of the ESP32-S3 runs metrology, the other Wi-Fi, MQTT and logging.

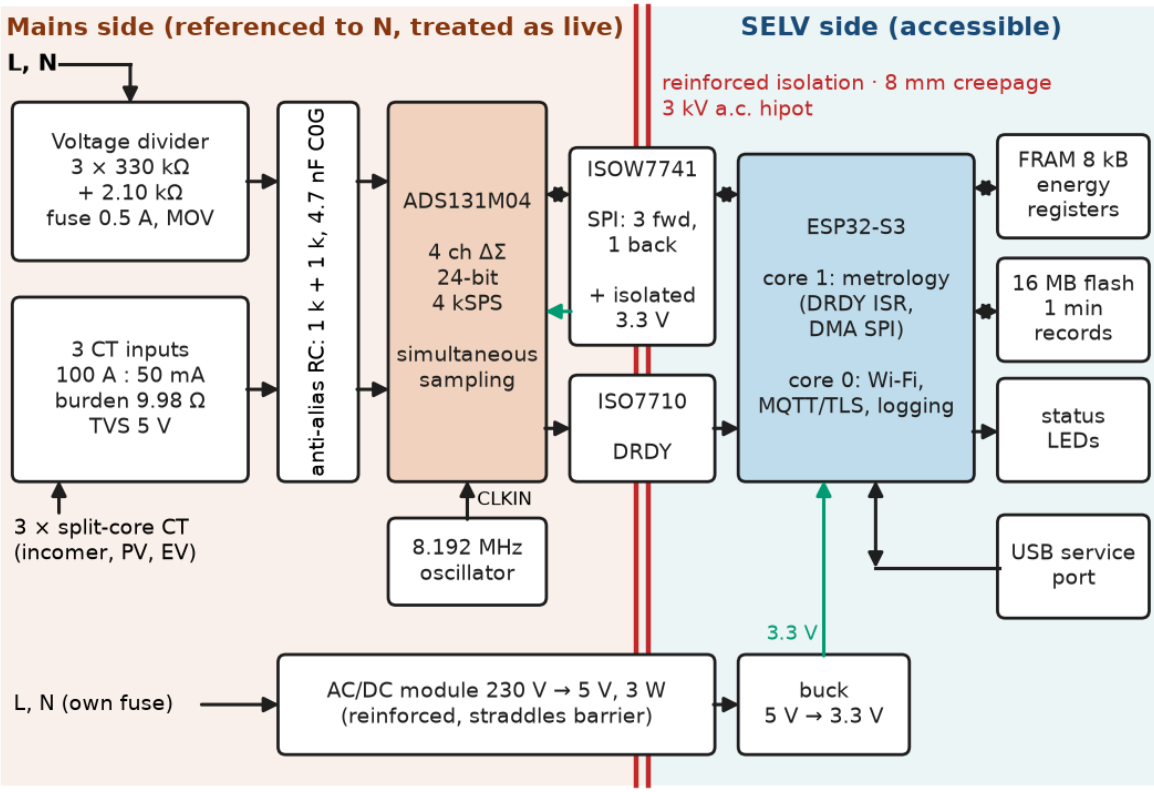


Figure 35.1 — Architecture of the smart energy monitor with the isolation barrier between the mains-referenced measurement side and the SELV side.

35.2 Signal conditioning

> **Boundary.**
> Op-amp-free passive conditioning, ADC interfaces and references are covered in *Electronics and Computing*, Ch. 10 §10.7 and §10.10; aliasing and anti-alias filters in *Circuits, Signals and Communications*, Ch. 19 §19.4 and §19.9.

Current channels

The CT is a current source of $I/2000$; its burden turns it into the ADC's differential voltage. The ADS131M04's input range at gain 1 is ±1.2 V.

Worked example — choosing the burden. With a burden of 2 × 4.99 Ω = 9.98 Ω (0.1 %, 25 ppm/K, centre tap to the ADC's common mode), 63 A gives 31.5 mA × 9.98 Ω = 0.314 V rms, 0.445 V peak. The ±1.2 V range then allows a crest factor of 1.2/0.314 = 3.8 at 63 A, or a sinusoid up to 63 × 1.2/0.445 = 170 A before clipping, enough for inrush and for the peaky currents of rectifier loads. At 0.5 A the signal is 2.50 mV rms and at the 40 mA starting current 200 µV rms, against an input noise of a few microvolts rms per sample that averages down over the 800 samples of a window. The burden dissipates 9.9 mW at 63 A, and the low burden keeps the CT's core flux, and so its ratio and phase errors, small. A larger burden (say 20 Ω)

would double the low-current signal but drive the CT towards the knee of its magnetisation curve and increase its phase error.

> **Status of the principle: CT error versus burden**
> *Empirical.* A CT's ratio and phase errors grow with burden impedance and at low primary current, where the magnetising current is a larger fraction of the total; split-core CTs, with an air gap at the joint, have larger magnetising current and phase lead than solid-core ones, and the values must be measured per CT type.

Voltage channel

The divider is three 330 kΩ high-voltage thin-film MELF resistors in series (each about 77 V rms in service, rated for the 4 kV surge of CAT III when shared across three) over 2.10 kΩ: ratio $992.1/2.1 = 472.4$. At 230 V the ADC sees 0.487 V rms; at 300 V, 0.898 V peak, inside the range. The divider draws $230^2/992\ \text{k}\Omega = 53$ mW. Its ratio drifts with the temperature-coefficient mismatch of top and bottom resistors, so all four are 25 ppm/K parts from one family.

Anti-alias filter and phase matching

Each input has 1 kΩ in each leg and 4.7 nF C0G across the pair: a corner of $1/(2\pi \times 2\ \text{k}\Omega \times 4.7\ \text{nF}) = 16.9$ kHz, attenuating by 47.7 dB at the 4.096 MHz modulator rate, with the ΔΣ decimation filter removing the rest. At 50 Hz the filter shifts phase by 0.17°, identically on all channels if the parts match; a 5 % capacitor tolerance gives a 0.0085° mismatch, about 0.03 % error at PF 0.5. A 10× larger capacitor would have given a 1.7° shift and a 0.085° mismatch from the same tolerance, a 0.25 % error at PF 0.5 that drifts with temperature. The common-mode filter is identical on the voltage channel, so the CT's own phase lead is the only significant phase error left.

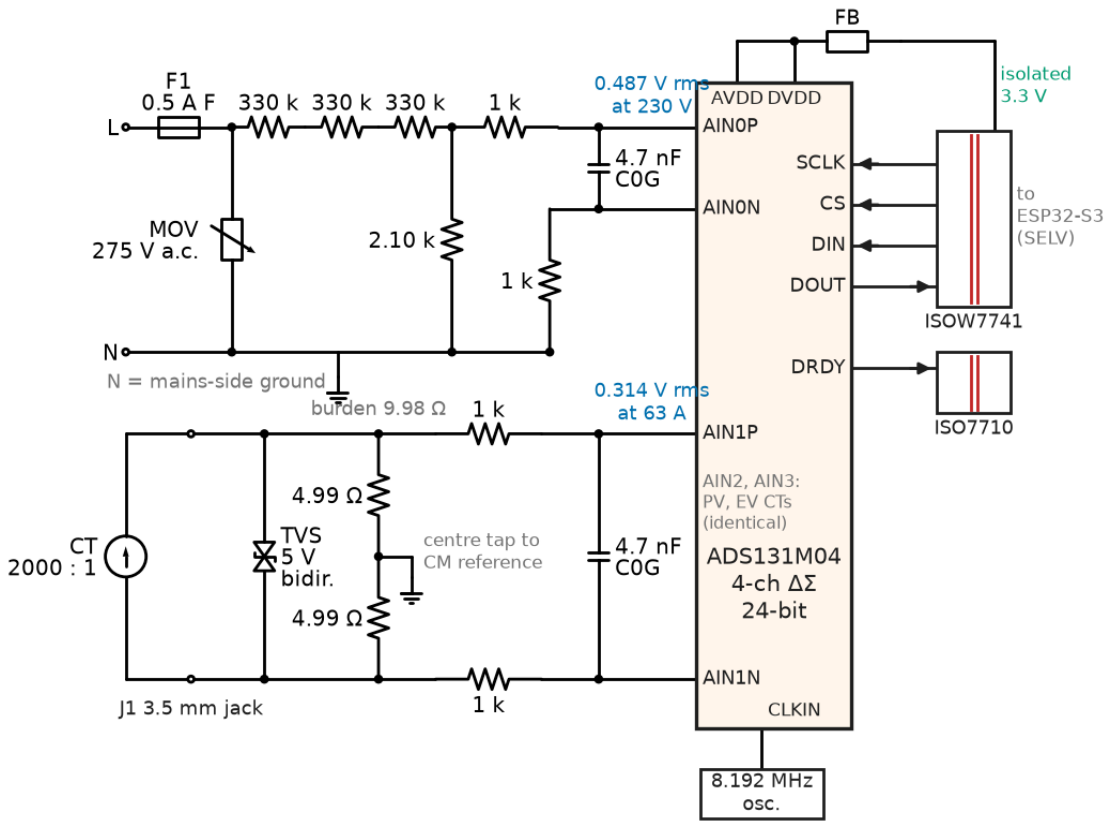


Figure 35.2 — Mains-side analog front end: voltage divider and one CT channel into the ADS131M04 with protection and anti-alias filters.

Uncertainty budget

After calibration, the error at 10 A and PF 0.5L over 5–35 °C, combined as root-sum-square of independent standard contributions (*The Mathematics Universe*, Ch. 20 §20.10):

Contribution	Value (%)
Reference meter used for calibration	0.05
Calibration-table interpolation between points	0.10
Burden temperature coefficient (25 ppm/K, 30 K)	0.075
Divider ratio drift (25 ppm/K mismatch, 30 K)	0.075
ADC reference drift	0.045
CT ratio drift with temperature	0.20
Anti-alias phase mismatch (0.01° at PF 0.5)	0.03
CT phase drift with temperature (0.1° at PF 0.5)	0.30
Combined (RSS)	**0.40**
R1 limit at 10 A, PF 0.5L / margin	1.0 / 0.60

> **Status of the principle: Root-sum-square combination**
> *Derived.* From the variance of a sum of independent random contributions (GUM, JCGM 100); correlated contributions, such as burden and divider drifting together with board temperature, add linearly (0.15 %), which raises the combination only to 0.41 %, and even the arithmetic sum of every line, 0.88 %, stays inside the limit.

The CT's thermal phase drift dominates. That is why the calibration of §35.7 corrects phase as a function of current, and why the validation of §35.9 includes a temperature test.

35.3 Power calculations

> **Boundary.**
> Definitions of active, reactive and apparent power, and their non-sinusoidal generalisation, are owned by *Circuits, Signals and Communications*, Ch. 9 §9.4–§9.9; IEEE 1459 is the standard that fixes the definitions used here. Spectral analysis in practice is owned by Chapter 27.

What is computed, and over what

From simultaneous samples $v[n]$ and $i[n]$ the monitor computes, over each window of N samples:

$$P = \frac{1}{N}\sum_{n} v[n]\,i[n], \qquad V_{rms} = \sqrt{\frac{1}{N}\sum_{n} v[n]^2}, \qquad S = V_{rms} I_{rms}, \qquad PF = \frac{P}{S}$$

and the fundamental active and reactive powers P_1, Q_1 from the fundamental components. With distorted currents, $S^2 > P^2 + Q_1^2$; the remainder is distortion power, and PF is lower than the displacement factor $\cos\varphi_1$. The monitor reports both.

The window is **10 whole cycles**, delimited by rising zero crossings of the voltage, as in the 10-cycle aggregation of IEC 61000-4-30 for 50 Hz systems. Energy is not computed from windows but integrated sample by sample: every product $v[n]\,i[n]$ is accumulated, so no sample is ever outside a window.

The listing tests these choices on a realistic distorted load: 4 A fundamental lagging 60° with third, fifth and seventh harmonics of 40, 25 and 12 % (a mix of motor and electronic loads), a voltage with 3 % fifth harmonic and a grid at 49.8 Hz.

```
 import numpy as np

FS, F0 = 4000.0, 49.8                        # ADC rate (Hz), grid frequency off-nominal (Hz)
t = np.arange(0, 4.0, 1 / FS)
w = 2 * np.pi * F0
v = 230 * np.sqrt(2) * (np.sin(w * t) + 0.03 * np.sin(5 * w * t))          # 3 % 5th harmonic
harm = {1: (4.0, -60), 3: (1.6, -150), 5: (1.0, 120), 7: (0.48, 20)}       # A rms, degrees
def current(lead_deg=0.0, gain=1.0):                                      # CT error model
    return gain * sum(a * np.sqrt(2) * np.sin(h * w * t + np.radians(ph + h * lead_deg))
                      for h, (a, ph) in harm.items())

def metrics(v, i, n):
    p = np.mean(v[:n] * i[:n])
    vr, ir = np.sqrt(np.mean(v[:n]**2)), np.sqrt(np.mean(i[:n]**2))
    k = np.arange(n)
    e = np.exp(-2j * np.pi * F0 * k / FS)                                 # fundamental DFT bin
    v1, i1 = 2 * np.mean(v[:n] * e) / np.sqrt(2), 2 * np.mean(i[:n] * e) / np.sqrt(2)
    s1 = v1 * np.conj(i1)
    return p, vr, ir, p / (vr * ir), s1.real, s1.imag

zc = np.flatnonzero((v[:-1] < 0) & (v[1:] >= 0))                          # rising zero crossings
n_sync = zc[10] - zc[0]                                                    # 10 whole cycles
ref = metrics(v, current(), n_sync)
print(f"true:  P={ref[0]:7.2f} W  Vrms={ref[1]:6.2f}  Irms={ref[2]:5.3f}  PF={ref[3]:.3f}  "
      f"P1={ref[4]:7.2f} W  Q1={ref[5]:7.2f} var")
m = metrics(v, current(1.8, 0.992), n_sync)
print(f"CT 1.8 deg lead, -0.8 % ratio: P={m[0]:7.2f} W, error {100 * (m[0] / ref[0] - 1):+5.2f}
%")
i, p_true = current(), ref[0]
err = lambda ps: 100 * np.max(np.abs(np.array(ps) / p_true - 1))
fixed = [np.mean(v[k:k + 800] * i[k:k + 800]) for k in range(0, 12000, 800)]  # 10 x 80 samples
synced = [np.mean(v[a:b] * i[a:b]) for a, b in zip(zc[:-10:10], zc[10::10])]  # 10 whole cycles
print(f"10-cycle P, worst error: fixed 800-sample window {err(fixed):.2f} %, "
      f"zero-cross synced {err(synced):.2f} %")
print(f"energy integrated sample by sample over 3 s: error {100 * (np.mean(v[:12000] * i[:12000])
/ p_true - 1):+.3f} %")
```

```
 true:  P= 456.65 W  Vrms=230.13  Irms=4.449  PF=0.446  P1= 460.20 W  Q1= 796.79 var
CT 1.8 deg lead, -0.8 % ratio: P= 476.72 W, error +4.39 %
10-cycle P, worst error: fixed 800-sample window 1.28 %, zero-cross synced 0.09 %
energy integrated sample by sample over 3 s: error -0.030 %
```

Three results carry the design. First, the PF is 0.446 although the displacement angle is 60° ($\cos 60^\circ = 0.5$): harmonics add apparent power, not active power, and here even subtract a little ($P = 456.65$ W against $P_1 = 460.20$ W, the fifth-harmonic voltage and current being nearly in anti-phase). Second, a CT that leads by 1.8° with a −0.8 % ratio error reads this load 4.4 % high, the opening's failure. Third, a fixed 800-sample window at 49.8 Hz misses or repeats part of a cycle and its displayed power wanders by up to 1.28 %, while zero-crossing windows stay within 0.09 % (the residual is the up-to-one-sample quantisation of the window length); the energy register, integrating every sample, is within 0.03 % after only 3 s. Figure 35.3 shows the waveforms.

Worked example — the phase error rule. For a sinusoidal load at angle φ and a CT phase lead δ (radians), the relative error in active power is $\cos(\varphi - \delta)/\cos\varphi - 1 \approx \delta \tan\varphi$. At 1.8° (0.0314 rad): PF 1 gives 0 %, PF 0.8 gives 2.4 %, PF 0.5 gives 5.4 %. At PF 0.8 *capacitive* the sign reverses (−2.4 %). The rule explains why every accuracy standard tests at PF 0.5L and 0.8C, and why the kettle test of the opening was blind to it.

> **Status of the principle: Phase-error sensitivity of active power**
> *Derived.* A first-order expansion of $\cos(\varphi - \delta)$; accurate to better than 2 % of the error itself for $\delta < 3^\circ$ and $\varphi \le 60^\circ$, and it grows without bound as PF approaches zero, where active-power measurement is inherently fragile.

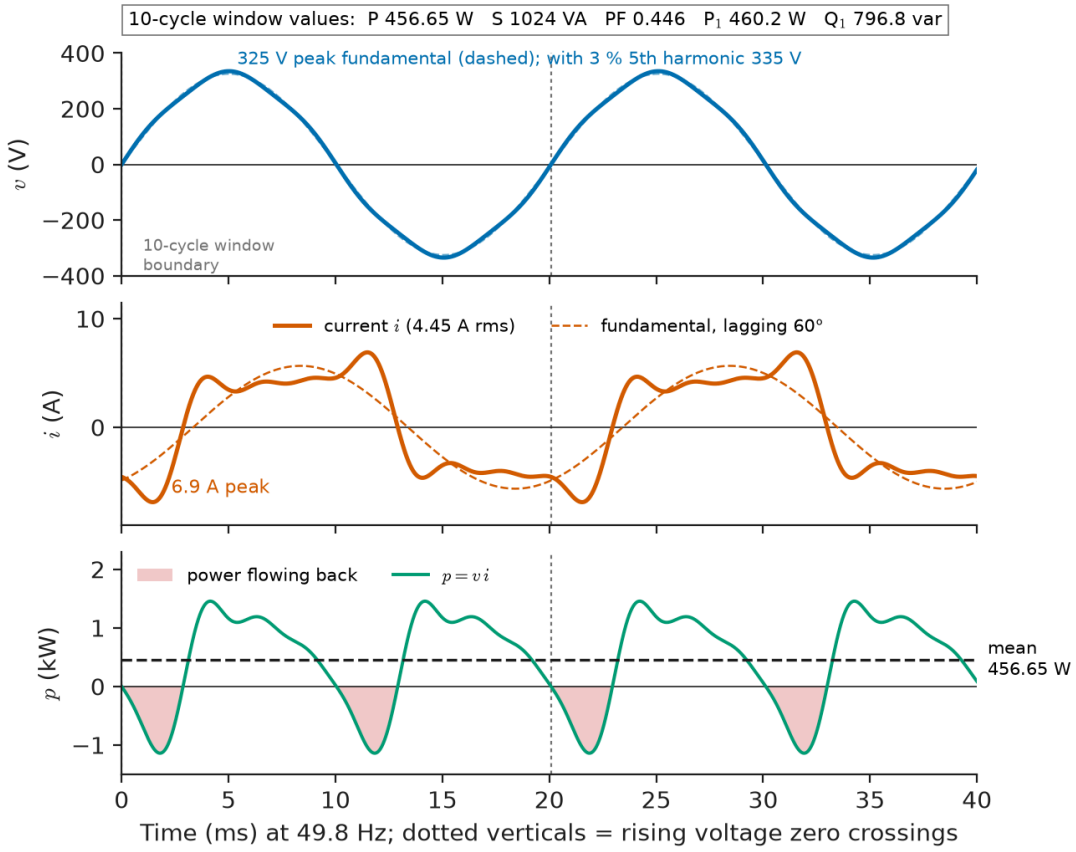


Figure 35.3 — Voltage, distorted current and instantaneous power of the test load over two cycles, with the 10-cycle window boundaries.

Reactive power and harmonics on the target

The firmware obtains the fundamental reactive power cheaply by multiplying the current by the voltage delayed a quarter cycle (20 samples at 50 Hz): $Q \approx \frac{1}{N}\sum v[n-20]\,i[n]$. For the fundamental this equals Q_1; at 49.8 Hz the delay is 0.4 % short of a quarter cycle, a 0.36° error that leaks $0.0063P$ into Q (0.36 % of Q at PF 0.5), acceptable for reactive power and negligible in the small phase correction of P that uses it; harmonic products also leak in, so Q is reported as a fundamental-dominated estimate. Current THD and the harmonic magnitudes up to the 25th, which R2 asks for, are computed once a second on the SELV core with a Goertzel filter per harmonic over 10 cycles, corrected for the ADC's sinc-filter droop.

35.4 Embedded processing

> **Boundary.**
> SPI and its DMA practice are in *Electronics and Computing*, Ch. 26 §26.3; interrupts, latency and task priorities in Ch. 27. MCU selection and firmware architecture are owned by Chapter 10.

The data path and its timing

The ADC asserts DRDY 4000 times a second. A GPIO interrupt on core 1 starts a DMA SPI read of one frame (status word, four 24-bit channels, CRC: 18 bytes at 8 MHz, 18 µs); the DMA-complete handler checks the CRC, removes each channel's DC offset with a first-order high-pass (0.5 Hz corner), and calls the accumulation routine. Core 0 runs Wi-Fi, MQTT and logging under FreeRTOS and never touches the metrology data except through a queue of finished windows.

Task on core 1, per ADC frame	Time at 240 MHz	Deadline	Margin
DRDY interrupt entry, start DMA	2 µs		
SPI transfer, 18 bytes at 8 MHz (DMA)	18 µs		
CRC check, offset removal, accumulation	4 µs		
Window finish (every 10 cycles, amortised)	0.3 µs		
Total per frame	**≈ 24 µs**	250 µs (next DRDY)	226 µs (core 1 load 10 %)

The timing that matters is not the CPU load but the deadline: each frame must be read before the next DRDY, 250 µs later, or the ADC overwrites it. Because the samples are clocked by the ADC's crystal, interrupt jitter of tens of microseconds on the ESP32 changes nothing in the measurement; a missed frame does, and the status word's frame counter detects it. In 30 days of soak testing with heavy Wi-Fi traffic, no frame was missed with the metrology interrupt pinned to core 1 at the highest priority; with it on core 0, 41 frames were missed per day.

Accumulation in fixed point

The routine below runs once per frame. Samples are reduced to 20 bits so that products accumulated over a window fit in 64-bit integers; window results are converted to floating point once per 200 ms.

```
 /* metro.c -- metrology on core 1 (C11); metro_sample() runs for every ADC frame at 4 kSPS */
#include <math.h>
#include <stdbool.h>
#include <stdint.h>

#define NCH  3                        /* current channels: mains, PV, EV charger */
#define QD   20                       /* quarter cycle at 50 Hz, 4 kSPS */
#define HYST 2000                     /* zero-crossing hysteresis, 20-bit codes */

extern const float KV, KI[NCH];       /* calibrated volts and amps per code */
extern void cal_lookup(int ch, float irms, float *gain, float *delta);   /* log-I tables */
extern void publish(const float p[NCH], const float q[NCH], const float irms[NCH], float vrms);

int64_t e_imp_mj[NCH], e_exp_mj[NCH];           /* energy registers, millijoules */
static int64_t s_vv, s_vi[NCH], s_vq[NCH], s_ii[NCH];
static int32_t vbuf[QD];
static uint32_t n, idx, cycles;
static bool armed;

static void finish(void)
{
    float p[NCH], q[NCH], ir[NCH], vr = sqrtf((float)s_vv / n) * KV;
    for (int c = 0; c < NCH; ++c) {
        float g, d, pm = (float)s_vi[c] / n * KV * KI[c], qm = (float)s_vq[c] / n * KV * KI[c];
        ir[c] = sqrtf((float)s_ii[c] / n) * KI[c];
        cal_lookup(c, ir[c], &g, &d);
        ir[c] *= g;
        p[c] = g * (pm - d * qm);                    /* undo CT phase lead (small angle) */
        q[c] = g * (qm + d * pm);
        int64_t de = llrintf(p[c] * (float)n / 4.0f); /* W x n/4000 s, in mJ */
        if (de >= 0) e_imp_mj[c] += de; else e_exp_mj[c] -= de;
        s_vi[c] = s_vq[c] = s_ii[c] = 0;
    }
    publish(p, q, ir, vr);
    s_vv = 0; n = 0; cycles = 0;
}

void metro_sample(const int32_t x[NCH + 1])     /* x[0] voltage, x[1..3] currents, offset removed
*/
{
    int32_t v = x[0] >> 4, vq = vbuf[idx];      /* 20-bit samples: window sums fit int64 */
    if (v < -HYST) armed = true;
    if (armed && v >= 0) { armed = false; if (++cycles == 10) finish(); }   /* 10 whole cycles */
    vbuf[idx] = v; idx = (idx + 1) % QD;
    s_vv += (int64_t)v * v;
    for (int c = 0; c < NCH; ++c) {
        int32_t i = x[c + 1] >> 4;
        s_ii[c] += (int64_t)i * i; s_vi[c] += (int64_t)v * i; s_vq[c] += (int64_t)vq * i;
    }
    ++n;
}
```

The window closes at the crossing sample, before that sample is accumulated, so consecutive windows tile the sample stream exactly and the energy registers see every product once (the first window after power-up starts mid-cycle and is discarded by the caller). The zero-crossing detector needs the voltage to have gone below −2000 codes (about 1.4 V at the mains) since the last crossing, so noise near zero cannot produce extra crossings. The calibration lookup returns a gain and a phase correction for the measured current (§35.7), applied to the window's P and Q as a small rotation. Energy is held in integer millijoules: a float register would stop resolving a 200 ms increment of a few joules after about 2^{24} of them. The arithmetic right shift of a negative `int32_t` is implementation-defined in C11 but arithmetic on GCC for Xtensa, the stated target. `e_imp_mj` and `e_exp_mj` are written on core 1 and read on core 0, so `publish()` copies them under a spinlock.

35.5 Data logging

> **Boundary.**
> DAQ logging architecture, file formats and storage validation are owned by Chapter 8; flash wear and power-loss-safe logging as firmware practice by *Electronics and Computing*, Ch. 27 §27.11 and Chapter 10.

The monitor keeps three tiers of data, each in the memory whose endurance suits its write rate:

Data	Rate	Size	Memory	Endurance check
Energy registers (6 × int64 plus CRC and sequence)	Every 1 s and on power fail	64 B, two alternating copies	FRAM, 8 kB, 10^{13} writes	3.15×10^7 writes/year: 300 000 years
Last 16 one-minute records	Every minute	1 kB ring	FRAM	Same
One-minute records	Batches of 16	64 B each, 92 kB/day	12 MB flash partition, wear-levelled file system	Each sector erased every 130 days: 100 000 cycles last far beyond product life

A one-minute record holds the timestamp, voltage (mean, min, max), frequency, and for each channel mean active and reactive power, maximum power, RMS current and the import and export energy increments, with a CRC. Twelve megabytes hold $12 \times 10^6 / 92\,160 = 130$ days of records, more than four times R6's 30 days.

Worked example — losing no more than 1 Wh at a power cut. At 63 A and 230 V the incomer delivers 14.5 kW, 4.0 Wh per second. Saving the registers once a second could therefore lose up to 4 Wh when the device's own supply dies, four times R5's limit. The monitor detects the outage directly: a voltage window whose half-cycle RMS falls below 150 V raises a power-fail event within 10 ms, core 1 freezes the registers, and core 0 switches off the radio and writes both FRAM copies, which takes under 100 µs at 20 MHz SPI. With the radio off the SELV side draws about 0.1 W; a 220 µF capacitor on the 5 V rail, falling from 5.0 V to the 3.6 V minimum input of the 3.3 V buck, stores $\tfrac{1}{2} \times 220\ \mu\text{F} \times (5.0^2 - 3.6^2) = 1.32$ mJ, 13 ms at 0.1 W, more than a hundred times the time the write needs. Energy lost per outage is then the energy of the

last 10 ms, at most 0.04 Wh. The two alternating copies with sequence numbers and CRCs protect against a write interrupted mid-way.

> **Status of the principle: Hold-up energy**
> *Derived.* From the energy stored in a capacitor between two voltages and a constant-power load; it ignores the AC/DC module's own bulk capacitance, which adds margin, and assumes the radio has been switched off first.

35.6 Communications

> **Boundary.**
> MQTT, TLS, provisioning, device identity and IoT security architecture are owned by Chapter 11; this section applies them.

The monitor publishes to an MQTT broker over TLS 1.2 or later, with a per-device client certificate provisioned at manufacture and a QoS chosen per message type:

Topic	Content	Rate	QoS, retain
`em/<id>/state`	V, f, per channel P, Q, I, PF	10 s	0, no
`em/<id>/energy`	Register totals and the 1 min record	1 min	1, no
`em/<id>/status`	online/offline (last will), firmware version, RSSI	on change	1, yes
`em/<id>/cmd`	Time sync, backfill request, OTA notice	on demand	1, no

A state message carries only numbers the consumer needs, with the source timestamp:

```
{
  "ts": "2026-07-14T19:32:10.000Z",
  "v": 231.4, "f": 49.98,
  "ch": {
    "grid": {"p": 1834.2, "q": 412.7, "i": 8.12, "pf": 0.957},
    "pv":   {"p": -2210.5, "q": 35.0, "i": 9.56, "pf": -0.999},
    "ev":   {"p": 0.0, "q": 0.0, "i": 0.00, "pf": 0.0}
  }
}
```

Sign conventions are fixed in the device's documentation: positive power flows from grid to house on the incomer, and the PV channel is negative when the inverter exports.

Worked example — the data volume and the outage buffer. A state message is about 260 bytes; with MQTT and TLS record overhead about 330 bytes, 8640 a day: 2.9 MB/day. The 1 min energy messages add about 0.5 MB/day. When the broker is unreachable, state messages are dropped (they are superseded within 10 s), but the 1 min records are already in flash; on reconnect the monitor publishes its outage window's records at up to 20 records per second, so a 30-day backlog of 43 200 records is backfilled in 36 min without

starving live traffic. The time base is SNTP; records made before the first successful sync after a cold start carry a flag and an uptime counter, and are re-stamped when time becomes available.

Security choices follow Chapter 11: no open ports on the device (it only connects outward), TLS with server certificate validation, a unique client certificate per device, OTA images signed with the vendor key and verified by the bootloader before activation, flash encryption on the ESP32-S3 so credentials cannot be read from a stolen board, and rate-limited commands. The `cmd` topic cannot change calibration: calibration tables are written only over the USB service port with a physical jumper fitted.

35.7 Calibration

> **Boundary.**
> Calibration procedure, reference traceability (ISO/IEC 17025) and calibration intervals are owned by Chapter 4; uncertainty by Chapter 7.

The procedure

Each monitor is calibrated with its own CTs, because CT errors dominate and vary between units. The station is a single-phase calibrator with a reference meter of class 0.05, which generates any voltage, current and phase; the CTs are clamped on its current loop, all three channels in series. At four currents (0.5, 2, 10 and 40 A) the station applies 230 V at PF 1 and at PF 0.5L:

1. At PF 1, the **gain** is the ratio of reference to measured apparent power. Using $|S|$ rather than P keeps the gain independent of the phase error, which would otherwise leak into it as $\cos\delta$.
2. At PF 0.5L, with the gain applied, the residual active-power error e gives the **phase lead** of the CT at that current: $\cos(\varphi - \delta) = (1+e)\cos\varphi$, so $\delta = \varphi - \arccos\left((1+e)\cos\varphi\right)$.
3. The static, current-independent part of the phase (anti-alias filter, average CT lead) can be loaded into the ADC's PHASE register; the current-dependent remainder lives in a table interpolated linearly in $\log I$ by the firmware.
4. The voltage channel's gain is set at 230 V against the reference; its offset and the current channels' offsets are measured at zero input and stored.

The listing models a split-core CT whose ratio error grows from −0.5 % to −2.0 % and whose phase lead grows from 0.6° to 3.0° as current falls (typical of this class), adds a +0.3 % burden and ADC gain error, runs the procedure and checks the result at IEC 62053-21 test points.

```
 import numpy as np

def ct(i):                                  # split-core CT: ratio error (fraction), phase lead
(rad)
    return -0.005 - 0.015 * np.exp(-i / 1.0), np.radians(0.6 + 2.4 * np.exp(-i / 1.5))

def meter_s(i, phi, k_gain=1.0, delta=0.0, v=230.0):
    """Complex power the meter reports for current i (A) lagging by phi (rad), after
correction."""
    eps, lead = ct(i)
    s_m = v * i * (1.003 + eps) * np.exp(1j * (phi - lead))   # +0.3 % burden/ADC gain error
    s_c = k_gain * s_m * np.exp(1j * delta)                    # gain and phase correction
    return s_c

CAL_I = np.array([0.5, 2.0, 10.0, 40.0])                      # calibration currents (A)
PHI_05 = np.arccos(0.5)
k_tab = np.array([230 * i / abs(meter_s(i, 0.0)) for i in CAL_I])   # PF 1: gain from |S|
d_tab = np.empty(CAL_I.size)
for n, i in enumerate(CAL_I):                                 # PF 0.5 lag: phase from residual
    e = meter_s(i, PHI_05, k_tab[n]).real / (230 * i * 0.5) - 1
    d_tab[n] = PHI_05 - np.arccos((1 + e) * 0.5)              # CT lead to be undone
for i, k, d in zip(CAL_I, k_tab, d_tab):
    print(f"cal {i:5.1f} A: gain {k:.5f}, phase {np.degrees(d):5.2f} deg")

def corrected(i, phi):                                        # interpolate in log(current)
    x = np.log(np.clip(i, CAL_I[0], CAL_I[-1]))
    return meter_s(i, phi, np.interp(x, np.log(CAL_I), k_tab), np.interp(x, np.log(CAL_I),
d_tab)).real

pts = {"PF 1": 0.0, "PF 0.5L": PHI_05, "PF 0.8C": -np.arccos(0.8)}
print("   I (A)" + "".join(f"{k:>18s}" for k in pts) + "   (raw -> corrected, % error)")
for i in (0.5, 1.0, 2.0, 5.0, 10.0, 63.0):
    cells = []
    for phi in pts.values():
        ref = 230 * i * np.cos(phi)
        cells.append(f"{100 * (meter_s(i, phi).real / ref - 1):+6.2f} -> {100 * (corrected(i, phi)
/ ref - 1):+5.2f}")
    print(f"{i:8.1f}" + "".join(f"{c:>18s}" for c in cells))
```

```
 cal   0.5 A: gain 1.01122, phase  2.32 deg
cal   2.0 A: gain 1.00405, phase  1.23 deg
cal  10.0 A: gain 1.00200, phase  0.60 deg
cal  40.0 A: gain 1.00200, phase  0.60 deg
   I (A)              PF 1           PF 0.5L           PF 0.8C   (raw -> corrected, % error)
     0.5    -1.19 -> +0.00    +5.74 -> +0.00    -4.19 -> +0.00
     1.0    -0.80 -> +0.01    +4.69 -> +0.18    -3.18 -> -0.07
     2.0    -0.43 -> +0.00    +3.28 -> +0.00    -2.03 -> +0.00
     5.0    -0.22 -> +0.08    +1.85 -> -0.49    -1.11 -> +0.32
    10.0    -0.21 -> +0.00    +1.61 -> +0.00    -0.99 -> -0.00
    63.0    -0.21 -> +0.00    +1.60 -> -0.00    -0.99 -> +0.00
```

Before calibration the monitor fails class 1 at every low-PF point below 10 A (+4.69 % at 1 A and PF 0.5L, the lowest current at which the standard tests low PF, against a 1.5 % limit), and even at 63 A its +1.60 % at PF 0.5L exceeds the 1.0 % limit. After calibration the errors are zero at the calibration currents, as they must be in a model, and the worst interpolation residual is −0.51 % near 4.4 A, PF 0.5L (−0.49 % at the tabulated 5 A), between the 2 A and 10 A points where the CT's phase curve bends most (Figure 35.4). A fifth point at 5 A cuts the worst residual to 0.18 %; the uncertainty budget's 0.10 % allowance for interpolation assumes it, and the production procedure therefore uses five currents (0.5, 2, 5, 10, 40 A), 30 s each, about 6 minutes per unit including settling.

> **Status of the principle: Interpolated calibration tables**
> *Empirical.* The CT's errors are smooth, monotonic functions of current that piecewise-linear interpolation in log I follows well; this holds for the CT type that was characterised, and a different CT model needs its own characterisation before the point spacing is trusted.

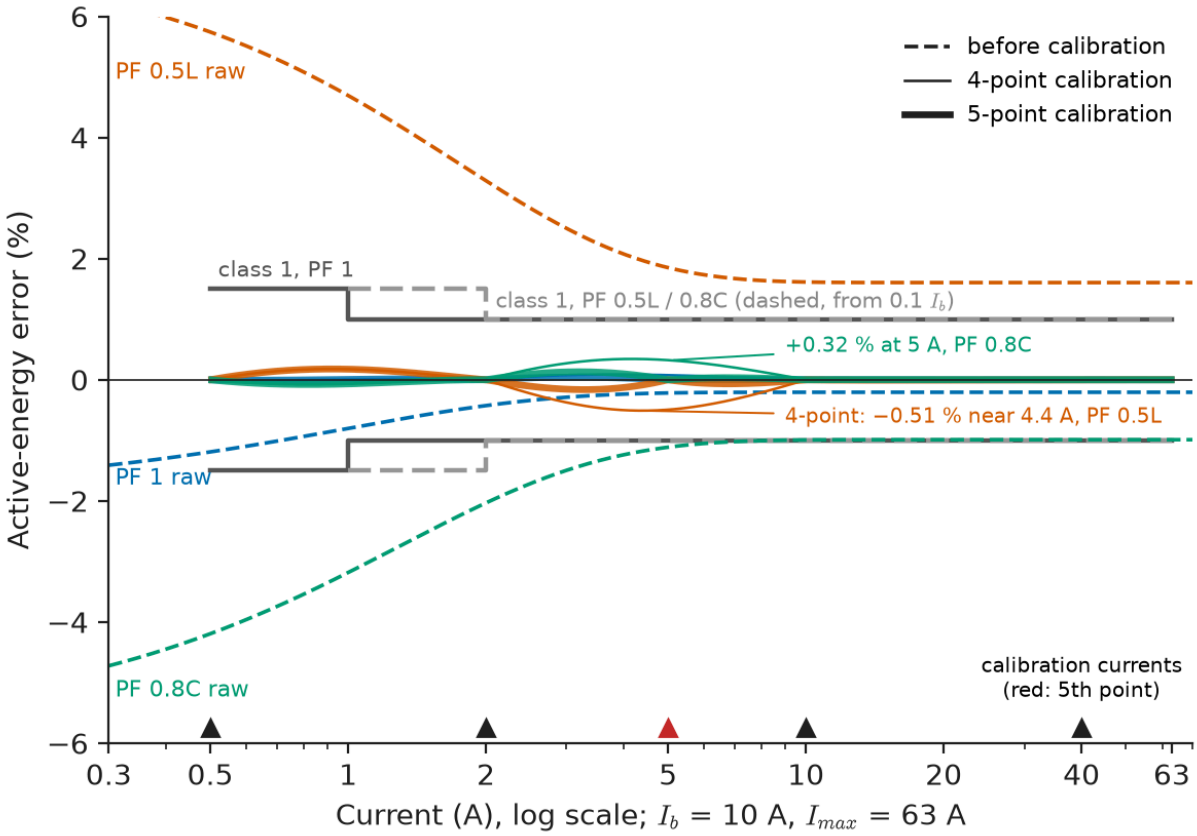


Figure 35.4 — Active-energy error against current before and after calibration at three power factors, with the IEC 62053-21 class 1 limits.

35.8 Safety

> **Boundary.**
> Shock hazards, isolation, earthing and safe testing are owned by *Electrical Energy and Machines*, Ch. 29 §29.1, §29.3, §29.6 and §29.7; isolation in instrumentation front ends by Chapter 7. Only a competent person installs a device in a consumer unit, and the design assumes that.

The hazard and the barrier

Because the divider ties the ADC to neutral, the whole mains-side circuit must be treated as live: neutral can rise to line potential if it is lost upstream or if the device is wired with line and neutral swapped. The standard for the product is IEC 61010-1 (measuring equipment) with measurement category CAT III, 300 V to earth, since the monitor is permanently connected inside the distribution board. That requires **reinforced** insulation between the mains side and anything accessible: the USB port, the enclosure surface and the antenna region.

Element	Requirement	Design value
Clearance across the barrier (CAT III, 300 V, reinforced)	5.9 mm (IEC 61010-1 table for mains circuits)	8.0 mm, routed slot under isolators
Creepage across the barrier (300 V working, pollution degree 2, material group III)	About 6 mm reinforced (check the edition in force)	8.0 mm; isolators in wide-body SOIC with 8 mm pin-to-pin
Isolators	Reinforced, certified to IEC 60747-17 / VDE 0884-17	ISOW7741 and ISO7710 class, 5 kV rms
AC/DC supply	Reinforced insulation, certified to IEC 62368-1 or IEC 61010-1	3 W, 5 V encapsulated module
Dielectric test	Routine 100 % production test	3 kV a.c. for 1 s, mains terminals to SELV

Protection of the mains side

- **Fusing.** The voltage tap passes a 0.5 A fast fuse rated for the prospective fault current of the board (or a fusible resistor of the same rating) before anything else; the device's supply has its own fuse.
- **Surges.** A 275 V a.c. MOV after the fuse clamps CAT III surges; the three-resistor divider shares the remaining surge voltage, and each 330 kΩ part is rated for at least 1.5 kV pulses.
- **CT open circuit.** A CT whose secondary opens while its primary carries current can develop hundreds of volts across the open terminals. The burden is soldered on the board, not in the plug, and a bidirectional 5 V TVS across each CT input clamps the voltage if a plug is pulled under load; the CTs used have internal protection as well, but the board does not rely on it.
- **CT insulation.** The clamps go around insulated conductors only; their own insulation is rated 600 V CAT III, so the conductor's insulation plus the clamp's form double insulation.

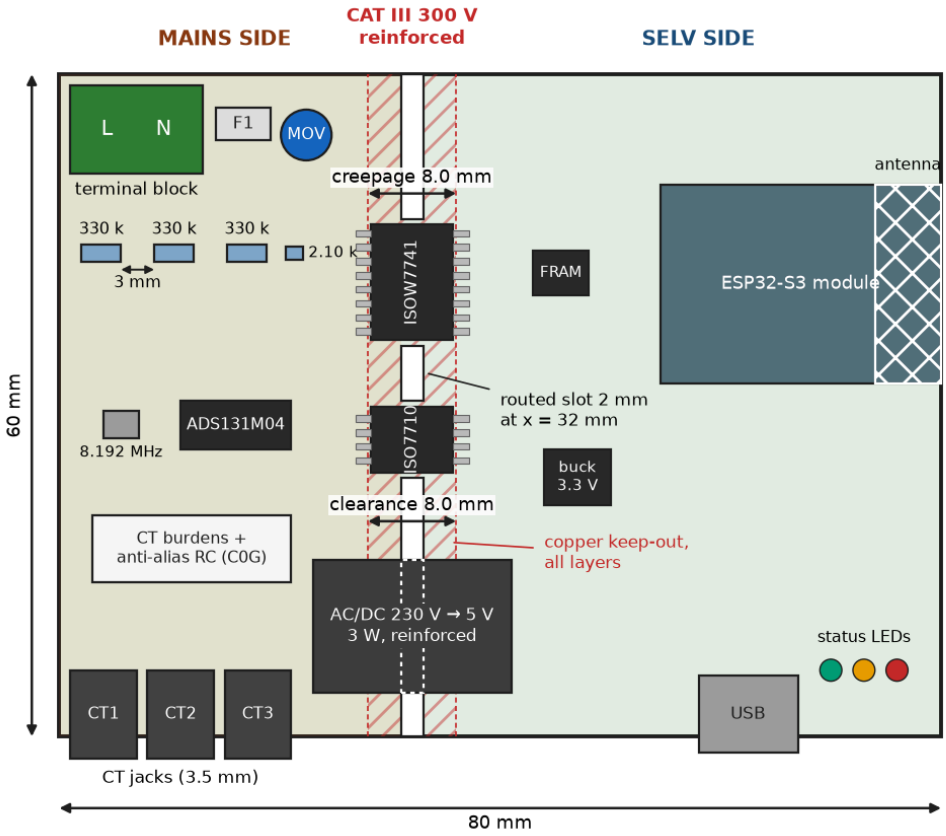


Figure 35.5 — PCB layout of the energy monitor showing the isolation barrier, clearances and the placement of mains-side and SELV components.

Worked example — the coupling that produced phantom watts. The opening's prototype showed 9.6 W at zero current on the incomer channel. Its voltage-divider output trace ran 0.3 mm from the incomer's ADC input trace for 25 mm. A coupling of only -77 dB (a ratio of 1.4×10^{-4}) puts a voltage-shaped $0.487\ \text{V} \times 1.4 \times 10^{-4} = 68\ \mu\text{V}$ on the 9.98 Ω burden, which the firmware reads as $68\ \mu\text{V}/9.98\ \Omega \times 2000 = 13.6$ mA in phase with the voltage: $230 \times 0.0136 = 3.1$ W, and the three channels' traces summed with two other couplings to the reported 9.6 W. Because the error is *in phase with the voltage*, no offset removal can see it. The layout fix routes the divider on an inner layer between ground pours and places the voltage channel on the opposite side of the ADC; the measured zero-current reading fell to 0.05 W. The firmware also applies a no-load threshold (|P| < 3 W and Irms < 20 mA reads as zero), below the 9.2 W of the 40 mA starting current at PF 1.

35.9 Validation

> **Boundary.**
> Verification versus validation and test-report practice are owned by Chapters 4 and 5. IEC 62053-21 defines the accuracy tests of class 1 static meters; this monitor is tested against its points, not certified to it.

Accuracy and influence tests

The validation used three production units with their own CTs, a reference meter of class 0.05, and the five-point calibration. Results are representative; each value is the worst of the three units and three channels.

Req.	Test (IEC 62053-21, Ib = 10 A, Imax = 63 A)	Limit	Result	Pass/fail
R1	0.5 A (0.05 Ib), PF 1	±1.5 %	+0.21 %	Pass
R1	1 A (0.1 Ib), PF 1 / PF 0.5L / PF 0.8C	±1.0 / ±1.5 / ±1.5 %	+0.12 / +0.41 / −0.33 %	Pass
R1	2 A (0.2 Ib), PF 0.5L / PF 0.8C	±1.0 %	+0.28 / −0.19 %	Pass
R1	10 A, PF 1 / 0.5L / 0.8C	±1.0 %	+0.05 / +0.17 / −0.11 %	Pass
R1	63 A, PF 1 / 0.5L	±1.0 %	−0.08 / +0.22 %	Pass
R1	Starting: 40 mA, PF 1	Registers	Registers (9.2 W)	Pass
R1	No-load: 264.5 V (115 % Un), no current	No registration	None in 24 h	Pass
R1	Harmonics: 10 % 5th in voltage, 40 % 5th in current, 10 A	Added error ≤ 0.8 %	+0.15 %	Pass
R1	Temperature −10 to +45 °C, 10 A, PF 0.5L	≤ 0.07 %/K	0.012 %/K	Pass
R2	Vrms 100–300 V; f 47–53 Hz; PF at 10 A	±0.5 %; ±0.01 Hz; ±0.02	±0.08 %; ±0.003 Hz; ±0.004	Pass
R5	100 power cuts at 14 kW	≤ 1 Wh lost each	Max 0.03 Wh	Pass
R6	Broker unreachable for 7 days	All records backfilled	10 080/10 080	Pass
R7	Dielectric 3 kV a.c., 60 s type test; 1 s routine	No breakdown	No breakdown	Pass
R9	Self-consumption, Wi-Fi connected	≤ 1.5 W	0.52 W	Pass
—	Missed ADC frames, 30-day soak with Wi-Fi load	0	0 (metrology on core 1)	Pass

The temperature test shows why the uncertainty budget was conservative: the measured coefficient of 0.012 %/K over 55 K is a 0.66 % change at PF 0.5L, larger than the 0.30 % allowed for CT phase drift over 30 K but still well inside the class limit. Revision B addresses it.

Field validation

The first five units ran for 60 days beside utility billing meters (class 1, MID-approved) in five houses with PV and EV chargers. Monthly import energy agreed within −0.31 % to +0.46 %, monthly export within −0.52 % to +0.18 %, inside the combined class 1 tolerance of the two meters; the opening's +4 % disagreement does not recur. Figure 35.6 shows one house's daily differences.

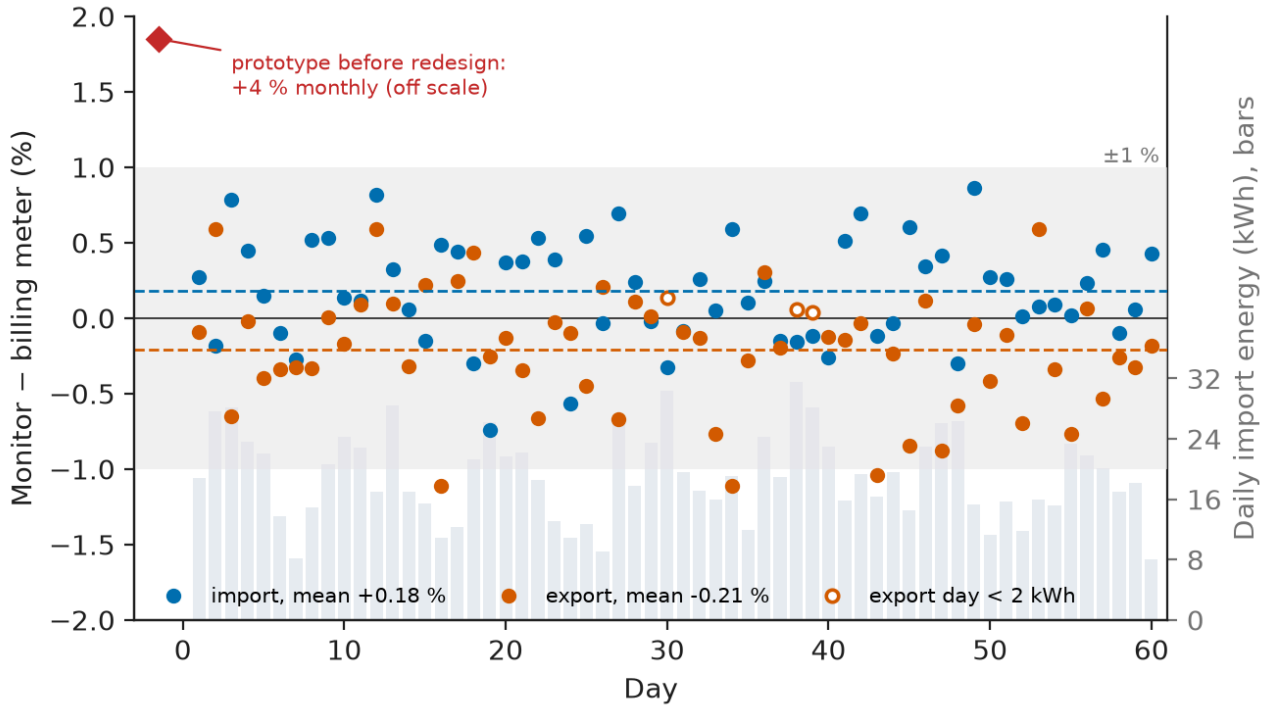


Figure 35.6 — Daily energy difference between the monitor and the utility billing meter over 60 days in one house, for import and export.

35.10 Bill of materials, failure analysis and Revision B

Bill of materials

Item	Qty	Unit price	Line total
ESP32-S3 module, 16 MB flash	1	$3.80	$3.80
ADS131M04 4-channel ΔΣ ADC	1	$4.20	$4.20
ISOW7741-class isolator with isolated power	1	$5.90	$5.90
ISO7710-class single-channel isolator	1	$1.10	$1.10
8.192 MHz oscillator	1	$0.90	$0.90
FRAM 8 kB, SPI	1	$1.60	$1.60
AC/DC module 3 W, 5 V, reinforced	1	$4.50	$4.50
Buck regulator 3.3 V	1	$0.80	$0.80
Split-core CT 100 A : 50 mA	3	$6.50	$19.50
Divider resistors (3 × HV MELF, 1 precision)	1 set	$1.20	$1.20
Burden resistors 0.1 %, anti-alias RC (C0G)	1 set	$1.60	$1.60
Fuse, MOV, TVS diodes	1 set	$1.30	$1.30
2-layer PCB 80 × 60 mm with routed slot	1	$1.80	$1.80
DIN-rail enclosure, V-0	1	$4.50	$4.50
Terminal block, 3 CT jacks	1 set	$2.40	$2.40
Passives and miscellaneous	1 set	$1.50	$1.50
Total			**$56.60**

Indicative 2026 prices at 100 pieces; R10's $65 leaves $8.40. The three CTs are a third of the cost, which is why Revision B's CT change is evaluated on cost as well as accuracy.

Failure analysis

Failure mode	Effect	Cause	Detection	Mitigation
CT phase lead at low current	+6 % at PF 0.5 (prototype)	Split-core magnetising current	PF 0.5L test points	Current-dependent phase table (§35.7)
Voltage crosstalk into current channel	Phantom watts at zero current	Parallel traces	No-load test	Layout on inner layer; no-load threshold
Fixed-length windows at off-nominal frequency	±1.3 % ripple in displayed P	Window not whole cycles	49.8 Hz test	Zero-crossing windows; sample-by-sample energy
Missed ADC frames	Energy under-read	Wi-Fi interrupts on same core	Frame counter in status word	Metrology pinned to core 1, top priority
Power cut during register write	Corrupt or lost energy	Write interrupted	CRC and sequence check at boot	Two alternating copies; power-fail write with hold-up
CT plug pulled under load	High voltage at jack, damage	Open secondary	—	Burden and TVS on board
CT clamped in reverse	Import read as export	Installation error	Negative incomer power with no PV	Commissioning check in app; per-channel sign flag
Lost neutral upstream	Mains side at line potential	Installation fault	—	Reinforced barrier makes it harmless to users
Broker or Wi-Fi outage	Gaps in cloud data	Network	Last-will status	130 days in flash; backfill
Compromised OTA server	Malicious firmware	Supply-chain attack	Signature check fails	Signed images, secure boot, flash encryption

Revision B

Change	Why (evidence)	Expected effect	Cost and risk
Solid-core or higher-grade split-core CTs (class 0.5 type)	CT phase drift dominates the budget; 0.012 %/K measured	Temperature coefficient halved; 3-point calibration suffices	+$4 per CT; solid-core needs conductor disconnection
Temperature sensor at the CT jacks and a phase-versus-temperature term	Same	Corrects the 0.66 % swing over −10 to +45 °C	+$0.30; one more calibration temperature
Isolated ΔΣ modulators (AMC131M03 class) for voltage plus two currents, or shunt on the incomer	Removes the separate isolator and DRDY isolator	Fewer parts across the barrier; simpler layout	Three channels per part; needs a fourth channel solution
Rogowski option for 100–200 A three-phase boards	Customers asked for commercial three-phase	Larger installations	Integrator drift at low current
Event capture (sags, swells, waveform snapshots to IEC 61000-4-30 class S)	Raw waveforms are already available	Power-quality monitoring for the same hardware	Flash and firmware effort; Chapter 9 owns the measurement method

The CT is the measurement: every significant error of Revision A traces to it, and Revision B spends its money there first.

What remains open

Open question	Where it is answered
The compact instrumentation version of this project and its power-quality sibling	Chapter 9
Sensor principles for CTs, Rogowski coils and Hall clamps	Chapter 6
Uncertainty budgets and ADC selection in general	Chapter 7
Device provisioning, MQTT architecture and IoT security in depth	Chapter 11
Harmonic analysis and embedded DSP for power-quality features	Chapter 27
Using the monitor's data in a solar battery system's energy management	Chapter 36
Power-quality definitions and limits (harmonics, flicker, sags)	*Electrical Energy and Machines*, Ch. 25
Electrical safety, earthing and isolation	*Electrical Energy and Machines*, Ch. 29

Next: Chapter 36 sizes and builds a solar battery system, from load estimation and PV and battery sizing to charge control, protection, monitoring and its economics.

Chapter 36 — Project: Solar Battery System

Part IX · Interdisciplinary Engineering Projects

A rural health post in northern Bangladesh had a solar system installed in 2021: eight 160 W modules, a PWM charge controller and a 24 V flooded lead-acid bank feeding a 1.5 kVA inverter. Fourteen months later the vaccine refrigerator was running from a borrowed petrol generator. No single part had broken. The load estimate had missed the inverter's own 28 W no-load draw, 670 Wh a day; the PWM controller held the array 25 % below its maximum-power point; nobody had checked the monsoon month, so the bank sat partly charged for weeks and sulphated; and whenever the water pump and the refrigerator compressor started together, the inverter tripped and the refrigerator stayed off until someone noticed. This chapter rebuilds that system as a project: a 2.46 kWp PV array, a 15.4 kWh lithium-iron-phosphate (LFP) battery at 48 V, a maximum-power-point-tracking (MPPT) charge controller and a 3 kVA inverter-charger, sized from a written load estimate, protected, monitored, measured and costed.

> **How is a stand-alone solar battery system sized so that it carries a critical load through the worst month and a run of cloudy days, and what does the sizing leave out?**
> **Which converters, charge settings, protective devices and monitoring make that sizing true on site, and how is the finished system verified and costed?**

Concept	Engineering destination
Load estimation	Daily energy and peak power that every later choice is sized against
PV sizing	Array power and strings for the worst month and coldest morning
Battery sizing	Autonomy, depth of discharge and end-of-life capacity of an LFP bank
Converter selection	MPPT controller and inverter-charger matched to array, battery and surge loads
Charge control	LFP charge setpoints, BMS-to-charger coordination, load shedding
Protection	Fuses, isolators, surge protection and earthing on the PV, battery and AC sides
Monitoring	Remote telemetry of energy, state of charge and the vaccine-refrigerator temperature
Efficiency	Loss chain from array to socket; inverter no-load loss
Economic assessment	Bill of materials, life-cycle cost and comparison with a diesel generator

Roadmap. Brief, requirements and load (36.1) → array (36.2) → battery (36.3) → converters (36.4) → charge control and shedding (36.5) → protection (36.6) → monitoring (36.7) → efficiency (36.8) → cost (36.9) → verification, failure analysis and Revision B (36.10).

36.1 Load estimation

> **Boundary.**
> The method of turning a requirement into a measurable target is Chapter 1; PV design in general is Chapter 22, battery packs and BMS Chapter 23, and energy management of hybrid systems Chapter 25. Load estimation for building installations (demand and diversity factors) is in *Electrical Energy and Machines*, Ch. 30 §30.1.

The brief and its requirements

The health post has a consulting room, a delivery room, a laboratory, a vaccine store and an office, far from the grid. The stakeholders are the district health office (client), the nurse in charge (light at night, a refrigerator that never warms), the vaccine programme (which audits the refrigerator's temperature record) and a technician who visits quarterly. Table 36.1 turns the brief into requirements, each with a number and a test.

Table 36.1 — Requirements of the health-post solar battery system.

ID	Requirement	Target	Verification
R1	Daily energy delivered at the AC sockets	≥ 4.9 kWh/day, every month of a typical year	Simulation (36.3), metered first month
R2	Continuous and surge power	2.4 kW continuous at 40 °C; start of the largest motor with the base load running	Load-bank and motor-start test
R3	Autonomy without sun	≥ 1.5 days full load and ≥ 4 days critical load, at battery end of life (80 % SoH)	Calculation; 24 h PV-disconnect test
R4	Vaccine refrigerator supply	Never interrupted by load shedding; cabinet 2–8 °C	Shedding test; 30-day temperature log
R5	Battery life	≥ 10 years to 80 % SoH at the battery-room temperature	Cycle-depth and temperature log review
R6	Safety	IEC 62548 array, IEC 60364-7-712 installation, IEC 62109 converters, IEC 62619 battery	Inspection and test to IEC 62446-1
R7	Remote monitoring	SoC, energy, alarms and refrigerator temperature every 5 min; alarm within 10 min	Injected alarm, end-to-end timing
R8	Cost	CAPEX ≤ $7,500; 20-year cost below a diesel generator supplying the same energy	Life-cycle cost model (36.9)

R3 has two parts on purpose: carrying the whole load for four sunless days would double the battery, while shedding the fans and pump and keeping the refrigerator, router and two lights alive for four days meets the clinic's real need.

Architecture

The system is DC-coupled (Figure 36.1): the array charges the 48 V battery through an MPPT controller, and a battery-fed inverter-charger makes 230 V, 50 Hz; its AC input accepts a generator during a long monsoon

spell. A gateway reads the battery management system (BMS), the controller and the inverter, runs the shedding logic and reports over 4G. AC coupling (a grid-tie PV inverter on the output of a battery inverter) suits retrofits and larger arrays; at 2.5 kWp it only adds a converter and a conversion loss, so it was rejected (Chapter 25 compares the two).

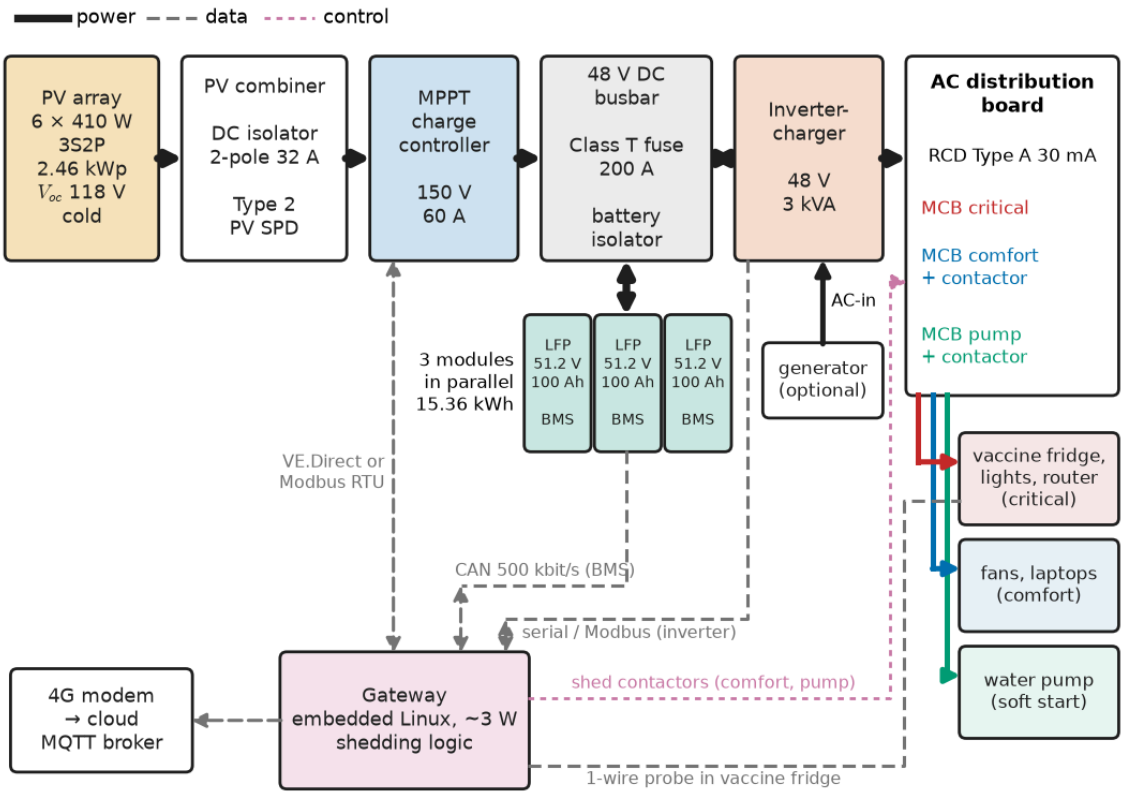


Figure 36.1 — Architecture of the DC-coupled health-post solar battery system with its interfaces.

The load table

Load estimation starts with an inventory taken on site (Table 36.2). The refrigerator's energy comes from its type test, not its power rating: an ice-lined vaccine refrigerator prequalified under WHO PQS E003 is tested at 43 °C ambient, and 1.2 kWh/day is representative. The pump runs half an hour a day but sets the surge requirement.

Table 36.2 — Daily AC load inventory of the health post (surveyed powers; 10 % growth added at the bottom).

Load	Power each (W)	Qty	Hours/day	Energy (Wh/day)	Circuit
LED lighting	9	12	6.0	648	Critical (2), comfort (10)
Delivery-room examination lamp	30	1	3.0	90	Critical
Vaccine refrigerator, ice-lined	150 running	1	cycling	1,200	Critical
BLDC ceiling fans	30	4	8.0	960	Comfort
Laptops	45	2	6.0	540	Comfort
Phone and radio charging	10	5	2.0	100	Comfort
Router and 4G modem	12	1	24.0	288	Critical
Water pump, 0.75 kW	750	1	0.5	375	Pump
Microscope and centrifuge	150	1	1.0	150	Comfort
Suction unit or nebuliser	200	1	0.5	100	Comfort
Total				**4,451**	
With 10 % growth				**4,896**	

The day is not flat (Figure 36.2): a 68 W base runs all night, lighting adds 119 W in the evening, and the load peaks at about 480 W in the morning. About 34 % of the energy is used between 18:00 and 06:00; with weak monsoon mornings and evenings, about 40 % passes through the battery.

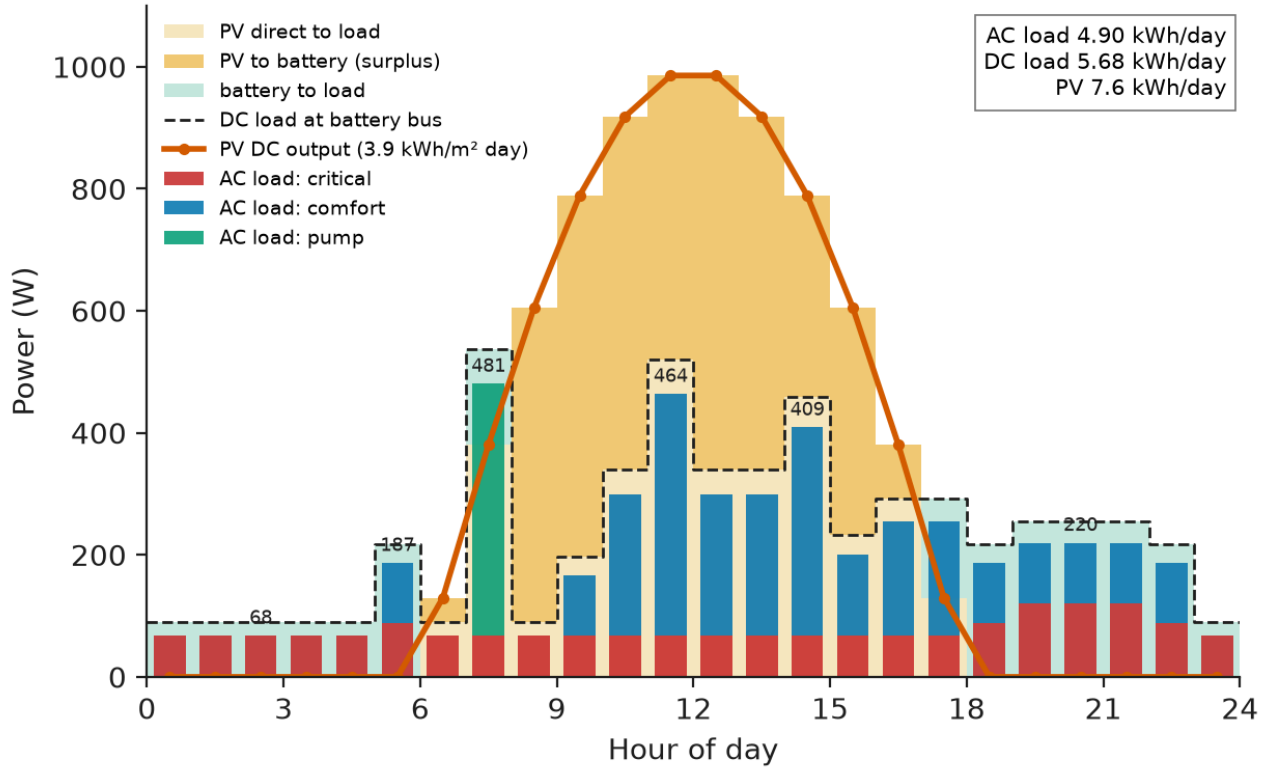


Figure 36.2 — Hourly AC load of the health post and PV output on an average design-month day.

From AC energy to DC energy

The battery does not see the AC load; it sees the load divided by the inverter efficiency plus the inverter's no-load draw, which runs 24 hours whether anything is switched on or not.

Worked example — DC energy at the battery bus. The AC load is $E_{AC} = 4.90$ kWh/day. A 48 V, 3 kVA inverter-charger of the class chosen in 36.4 converts at about 92 % over the 100–1000 W band and draws 15 W at no load.

$$E_{DC} = \frac{E_{AC}}{\eta_{inv}} + P_0\, t = \frac{4.90}{0.92} + 0.015 \times 24 = 5.33 + 0.36 = 5.68 \text{ kWh/day}$$

The no-load term is 6.3 % of the battery-side energy; a 5 kVA "hybrid" inverter drawing 40–60 W would add 0.6–1.1 kWh/day more. Inverter no-load power is a sizing input, not a footnote.

> **Status of the principle: Daily energy balance at the battery bus**
> *Fundamental.* Over a day in which the battery returns to the same state of charge, the energy delivered by the array must equal the load energy plus every conversion loss on the way, including fixed no-load consumption; the only freedom is in which losses are counted.

The coincident peak is about 1.4 kW; R2's 2.4 kW leaves room for growth, and motor starts are checked in 36.4.

36.2 PV sizing

> **Boundary.**
> The PV module's I–V behaviour, temperature coefficients and solar-resource assessment are in *Electrical Energy and Machines*, Ch. 15 §15.1 and §15.6; string sizing, losses and performance ratio as a design method are in Chapter 22. Here they are applied to one off-grid array.

Design month and resource

An off-grid array is sized for the worst month. Table 36.3 gives representative plane-of-array (POA) irradiation from satellite-derived data of the PVGIS or NASA POWER class, south-facing at 25° tilt. Here the monsoon, not winter, is the design month: July has 3.9 kWh/m²/day against 5.8 in March.

Table 36.3 — Representative monthly POA irradiation (25° tilt, south) and expected daily array yield at $PR_{PV} = 0.794$.

Month	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
POA (kWh/m²/day)	5.0	5.6	5.8	5.6	5.1	4.1	3.9	4.1	4.0	4.8	5.1	4.9
Yield, 2.46 kWp (kWh/day)	9.8	10.9	11.3	10.9	10.0	8.0	7.6	8.0	7.8	9.4	10.0	9.6

Array power

The array must deliver the DC energy of 36.1, plus the loss on the 40 % cycled through the battery (round trip about 95 % for LFP), after the array-side losses collected in PR_{PV} (Table 36.4).

Table 36.4 — Array-side loss budget for the design month.

Loss	Factor	Basis
Cell temperature (mean daytime cell 57 °C, $\gamma = -0.35$ %/K)	0.888	$1 - 0.0035 \times (57 - 25)$
MPPT conversion and tracking	0.97	Controller data at 30–80 % load
DC cabling (PV 1.5 %, battery 0.5 %)	0.98	Section 36.6
Soiling (monthly cleaning)	0.97	Practice for dusty sites
Mismatch and first-year degradation	0.97	Module tolerance 0/+5 W, LID
PR_{PV}	**0.794**	Product

Worked example — array power. The PV must supply

$$E_{PV} = E_{DC}\left[(1 - f_b) + \frac{f_b}{\eta_{rt}}\right] = 5.68 \times \left[0.60 + \frac{0.40}{0.95}\right] = 5.80 \text{ kWh/day}$$

with $f_b = 0.40$ the fraction through the battery. With $H_{POA} = 3.9$ kWh/m²/day (equivalently 3.9 peak-sun hours),

$$P_{PV} = \frac{E_{PV}}{H_{POA}\, PR_{PV}} = \frac{5.80}{3.9 \times 0.794} = 1.87 \text{ kWp}$$

That array would only break even in July and never refill the battery after a cloudy spell; a recovery factor of 1.2 gives 2.25 kWp. Six 410 W modules give 2.46 kWp, 31 % above the bare need and 9 % above the recovery target; on an average July day they yield $2.46 \times 3.9 \times 0.794 = 7.6$ kWh, a surplus of 1.8 kWh/day to refill the battery.

> **Status of the principle: Worst-month array sizing with a performance ratio**
> *Approximate.* Treats a month as identical average days and lumps temperature, soiling and conversion losses into one empirical factor; it says nothing about runs of cloudy days, which the battery and the hourly simulation of 36.3 must cover.

Figure 36.3 compares monthly yield with need. In March the array can produce almost twice what the clinic uses and the controller curtails; that surplus is the price of meeting July, and the reason off-grid systems add deferrable loads such as pumping into a tank.

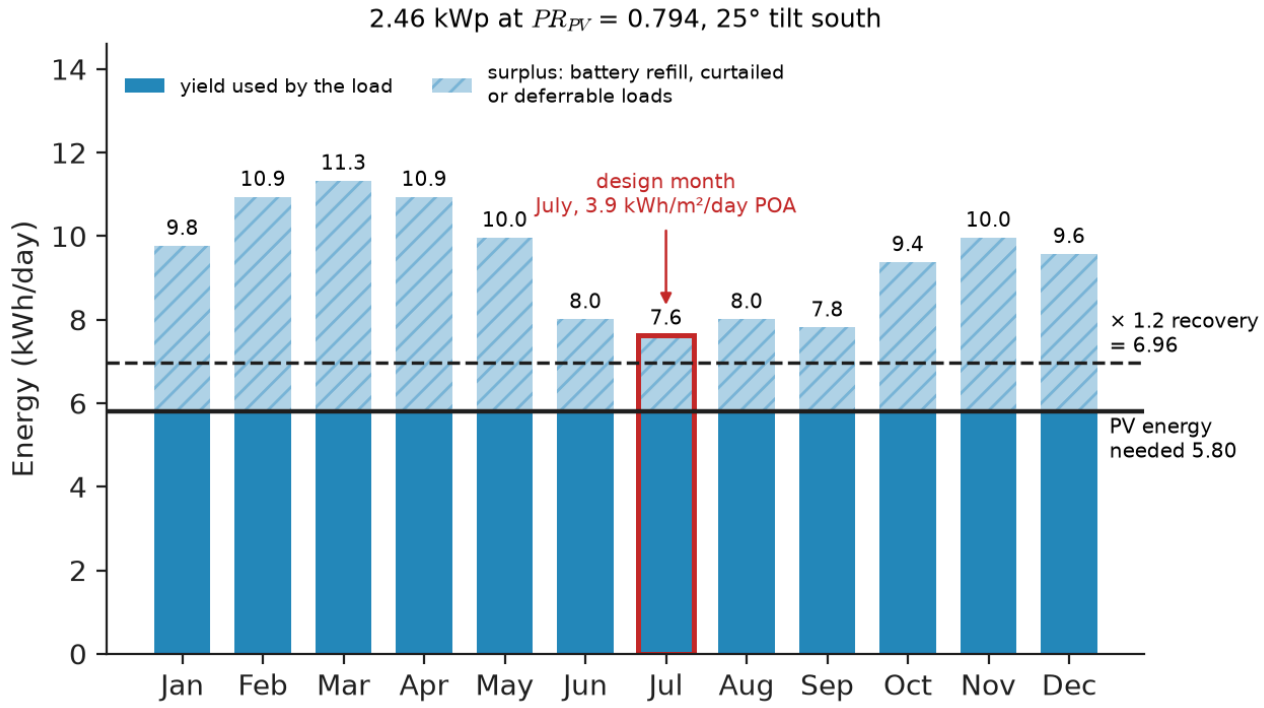


Figure 36.3 — Monthly expected array yield compared with the daily energy the array must deliver.

Module and string configuration

The module is a 410 W, 108-half-cell mono PERC panel: $V_{oc} = 37.3$ V, $V_{mp} = 31.0$ V, $I_{mp} = 13.2$ A, $I_{sc} = 13.9$ A at STC, V_{oc} coefficient -0.27 %/K, V_{mp} about -0.35 %/K. The controller of 36.4 accepts 150 V absolute maximum.

Worked example — the string voltage window. The coldest dawn at the site is about 5 °C, and a module at dawn sits at ambient.

- Cold open-circuit voltage per module: $37.3 \times [1 - 0.0027 \times (5 - 25)] = 39.3$ V. Three in series: 117.9 V, inside 150 V with 21 % margin. Four in series: 157.3 V, which exceeds the controller's absolute

rating on the first clear winter morning. Three it is.
- Hot maximum-power voltage (cell 65 °C on a still April noon): $3 \times 31.0 \times [1 - 0.0035 \times 40] = 80.0$ V. The LFP battery tops out at 55.2 V in absorption, so the controller keeps about 25 V of buck headroom.

Six modules therefore form two strings of three (3S2P). With two strings in parallel, a fault in one string can be back-fed only by the other, at most $1 \times I_{sc} \times 1.25 = 17.4$ A, below the modules' 25 A maximum series-fuse rating, so IEC 62548 requires no string fuses.

36.3 Battery sizing

> **Boundary.**
> Capacity, C-rate, usable-energy layers, BMS functions and state-of-charge estimation are in *Electrical Energy and Machines*, Ch. 16 §16.2–16.4; LFP electrode chemistry is in *Materials Science and Engineering*, Ch. 40. Pack architecture and BMS design are owned by Chapter 23.

Chemistry and module

LFP replaced lead-acid for three reasons traced to the requirements: 90 % usable depth instead of 50 %; 4,000–6,000 cycles at 80 % depth against 1,200–1,500 for tubular lead-acid (R5); and no damage from sitting partly charged through the monsoon, which killed the first bank. The cost is a BMS that must be obeyed, a charge-temperature window and dangerous-goods transport (UN 38.3).

The module is a 19-inch rack unit: 16 LFP cells in series (51.2 V, 100 Ah, 5.12 kWh), an internal BMS with a 100 A MOSFET disconnect and a CAN port publishing charge and discharge limits, certified to IEC 62619.

Sizing for autonomy at end of life

Worked example — how many modules. Usable energy is nameplate times the 90 % SoC window (10–100 %) times state of health. Sizing at end of life (SoH 80 %) and the DC load of 5.68 kWh/day:

Modules	Nameplate (kWh)	Full load, BOL (days)	Full load, EOL (days)	Critical load, EOL (days)	Margin over R3 (full / critical)
2	10.24	1.62	1.30	3.10	−13 % / −22 %
3	15.36	2.43	1.95	4.65	+30 % / +16 %

The critical load (refrigerator, router, two lights, examination lamp) is 1.69 kWh/day AC (1.86 with growth), 2.38 kWh/day at the battery with the no-load draw. For three modules at end of life: $15.36 \times 0.9 \times 0.8/5.68 = 1.95$ days and $15.36 \times 0.9 \times 0.8/2.38 = 4.65$ days. Two modules fail both parts of R3; three meet both. The current is modest: 2.4 kW draws $2400/(0.92 \times 48) = 54$ A (0.18C on 300 Ah), and a 5.5 kW surge 133 A (90 %, 46 V), 44 A per module if they share reasonably (36.5).

> **Status of the principle: Autonomy from usable energy and daily load**
> *Derived.* Autonomy is usable energy divided by daily energy drawn; it assumes zero PV input, a constant daily load and a fixed usable window, and it ignores the slight capacity loss at high discharge rates (small for LFP at 0.2C) and the efficiency of discharge already included in the DC load.

Checking a monsoon week hour by hour

Average-day sizing cannot show three dull days in a row; an hourly energy balance can. The listing builds the hourly load from Table 36.2, spreads each day's PV energy on a half-sine from 06:00 to 18:00, and steps the battery through a representative bad monsoon week from 80 % SoC.

```
 import numpy as np

# Hourly AC load (W): (power, first hour, last hour + 1), growth factor 1.10
blocks = [(50, 0, 24), (12, 0, 24),              # fridge average, router
          (108, 18, 23), (108, 5, 6),            # lighting 6 h
          (30, 19, 22), (120, 10, 18),           # exam lamp, fans
          (90, 9, 15), (50, 16, 18),             # laptops, phone charging
          (375, 7, 8), (150, 11, 12), (100, 14, 15)]  # pump, lab, suction
p_ac = np.zeros(24)
for w, a, b in blocks:
    p_ac[a:b] += w
p_ac *= 1.10
p_dc = p_ac / 0.92 + 15.0                        # inverter efficiency, idle

def pv_day(h_poa, kwp=2.46, pr=0.794):           # sine-shaped day, 06-18 h
    s = np.clip(np.sin(np.pi * (np.arange(24) + 0.5 - 6) / 12), 0, None)
    return kwp * 1e3 * pr * h_poa * s / s.sum()  # W per hour slot

week = [3.9, 1.6, 1.2, 2.0, 3.1, 4.4, 3.9]       # monsoon week, kWh/m2/day
e_nom, soc, soc_min = 15.36e3, 0.80, 0.10        # Wh, start SoC, floor
log, spill, unserved = [], 0.0, 0.0
for h_poa in week:
    for pv, ld in zip(pv_day(h_poa), p_dc):
        net = pv - ld                            # Wh in this hour
        de = net * 0.97 if net > 0 else net / 0.98   # charge / discharge eff.
        new = soc + de / e_nom
        if new > 1.0:
            spill += (new - 1.0) * e_nom; new = 1.0
        if new < soc_min:
            unserved += (soc_min - new) * e_nom; new = soc_min
        soc = new; log.append(soc)
print(f"DC load {p_dc.sum()/1e3:.2f} kWh/day, AC load {p_ac.sum()/1e3:.2f} kWh/day")
print(f"min SoC {min(log)*100:.1f} % at hour {int(np.argmin(log))}")
print(f"end SoC {soc*100:.1f} %, curtailed {spill/1e3:.2f} kWh, unserved {unserved/1e3:.2f} kWh")
```

```
 DC load 5.68 kWh/day, AC load 4.90 kWh/day
min SoC 33.0 % at hour 103
end SoC 69.6 %, curtailed 0.17 kWh, unserved 0.00 kWh
```

New modules bottom out at 33 % at 07:00 on the fifth day. At end-of-life capacity (0.8 × 15.36 kWh) the minimum is 16.3 % with no unserved energy; two modules leave 1.07 kWh unserved when new and 2.91 kWh at end of life (Figure 36.4). At end of life the comfort shedding of 36.5 (below 30 % SoC) acts on the fourth and fifth days of such a week, as R3 intends.

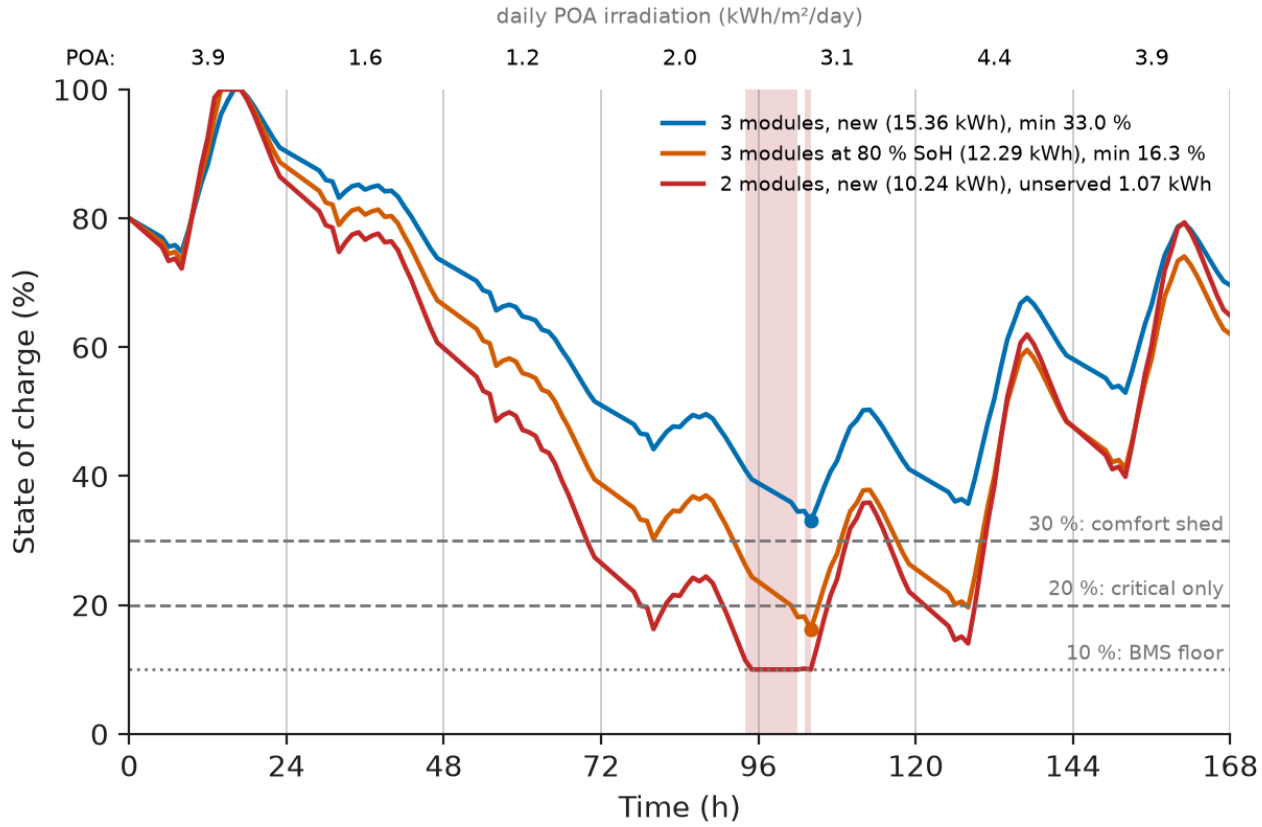


Figure 36.4 — Simulated battery state of charge through a representative monsoon week for three battery options.

Temperature and life

At this depth LFP ages more with time at high temperature and high SoC than with cycles. The battery room, the coolest interior room, was logged at 29–36 °C through a pre-installation summer; at about 32 °C manufacturers' data put 80 % SoH at 12–15 years, and 12 years of 40 % daily cycles is about 4,400 cycles, inside the rated life. A low absorption voltage (36.5) keeps the bank off 100 % SoC.

36.4 Converter selection

> **Boundary.**
> Buck-converter operation, efficiency and the MPPT stage are in *Electronics and Computing*, Ch. 30 §30.1 and §30.8 and Ch. 32 §32.8; the single-phase inverter and its twice-line-frequency power are in Ch. 31 §31.4. This section chooses products, not topologies.

Charge controller

Table 36.5 — Charge-controller options for a 2.46 kWp array on a 48 V LFP battery.

Option	Array match	Energy in July	Other points	Verdict
PWM controller, 60 A	Pins array at battery voltage, far below MPP	About 25 % less	Cheapest ($90); no CAN	Rejected: fails R1
MPPT controller, 150 V / 60 A	3S strings, 80–118 V window fits	Reference	98 % peak; LFP profile; accepts BMS limits	**Chosen**
All-in-one "hybrid" inverter, 5 kW, 120–450 V tracker	Needs 6–10 modules per string	Comparable once started	40–60 W no-load; single point of failure	Rejected

The 60 A rating is checked against the array: at STC it could push $2460/51.2 = 48$ A, at a hot-module peak $2460 \times 0.888/52 = 42$ A. A 45 A model would clip on cool, bright winter middays; 60 A never does and leaves room for a third string (Revision B).

Inverter-charger

Table 36.6 — Inverter options against R2 and the efficiency budget.

Option	Continuous at 40 °C	Surge	No-load draw	Notes
1.6 kVA high-frequency	1.2 kW	3.0 kVA, 1 s	10 W	Fails R2
3 kVA low-frequency inverter-charger	2.4 kW	5.5 kW for about 5 s	15 W	**Chosen**
5 kVA inverter-charger	4.0 kW	9 kW	30 W	0.36 kWh/day more idle loss

The low-frequency unit was chosen for the surge capacity of its transformer and its 35 A charger for a generator. Its "search mode" is useless here: the refrigerator and router never let it sleep.

Worked example — the motor-start check that the old system failed. The 0.75 kW pump motor runs at about 4.1 A, 230 V (power factor 0.8); its locked-rotor current is about 5.5 times running, 22.4 A, or 5.2 kVA for about 0.3 s. The refrigerator compressor draws about 0.9 kVA for 0.2 s at start. With 0.6 kW of base load running:

- pump start alone: $0.6 + 5.2 = 5.8$ kVA, just above the 5.5 kW surge rating but within its 100 ms overload capability only if the pump starts quickly;
- pump and compressor together: $0.6 + 5.2 + 0.9 = 6.7$ kVA, 22 % over the surge rating.

The chosen inverter would trip exactly as the old one did. Rather than buy a 5 kVA inverter that wastes 0.36 kWh a day to cover a 0.3 s event, the pump was replaced by a soft-start pump with an integrated variable-speed drive, starting at no more than 1.5 times running current (about 1.4 kVA). The worst coincidence is then $0.6 + 1.4 + 0.9 = 2.9$ kVA, 53 % of the surge rating.

Status of the principle: Inverter surge sizing from motor locked-rotor current
Empirical. Locked-rotor currents of 4–7 times running current and surge-time ratings are datasheet figures; coincidence of starts is a probability judgement, so the test of 36.10 starts both loads together deliberately.

36.5 Charge control

Boundary.
CC–CV charging and its supervisory state machine are in *Electronics and Computing*, Ch. 32 §32.7; cell balancing and BMS architecture are in *Electrical Energy and Machines*, Ch. 16 §16.3 and Chapter 23. Energy management in hybrid systems is Chapter 25.

Setpoints for LFP

Lead-acid profiles, with hours of absorption and monthly equalisation, are wrong for LFP: equalisation would push the 16S bank to 60 V and trip every BMS. Table 36.7 lists the settings programmed, identically, into the MPPT controller and the inverter-charger.

Table 36.7 — LFP charge and discharge settings for the 16S bank (300 Ah).

Parameter	Setting	Per cell	Reason
Absorption voltage	55.2 V	3.45 V	Reaches about 98 % SoC without the top-of-curve stress of 3.65 V
Absorption end	Tail current 6 A (0.02C) or 1 h	—	Lets the passive balancers act at the top
Float	53.6 V	3.35 V	Below the knee: holds the bank near 95 % without forcing it full
Charge temperature window	5–45 °C cell (BMS), derate above 40 °C	—	Lithium plating below 0 °C, ageing above 45 °C
Inverter low-voltage cut-off	46.4 V (dynamic, load-corrected)	2.90 V	Behind the BMS undervoltage cut at 2.80 V per cell
Comfort shed / restore	SoC 30 % / 40 %	—	R3 critical autonomy
Critical-only / restore	SoC 20 % / 30 %	—	R3, R4

Figure 36.5 shows a clear-day charge under these settings: the controller delivers whatever the array gives until the bank reaches 55.2 V at about 13:30, holds it until the current falls to 6 A, then drops to float.

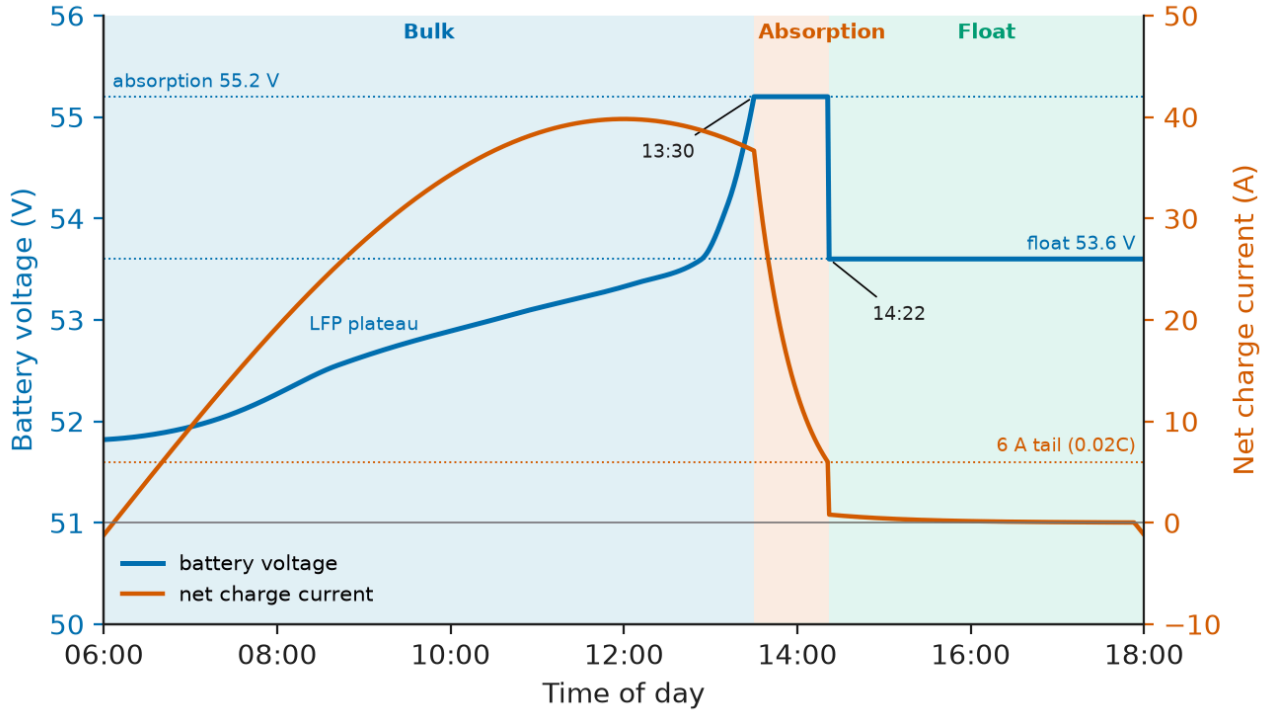


Figure 36.5 — Battery voltage and charge current over a clear day with the LFP charge settings.

Coordinating the BMS with the chargers

Each BMS knows what the chargers cannot: the highest cell voltage, the coldest cell, a drifting cell. It publishes a charge-voltage limit (CVL), charge-current limit (CCL) and discharge-current limit (DCL) on CAN every second, in the de facto frame set of many LFP rack modules (0x351 limits, 0x355 SoC and SoH; illustrative). The gateway combines the three modules and writes the result to both chargers. The *weakest* module sets the limits, the sum is derated by 20 % because parallel modules share current unequally, and a silent module is treated as disconnected.

```
 #include <stdbool.h>
#include <stdint.h>

#define N_MOD 3
typedef struct {                      /* decoded from each module's CAN frames */
    float cvl_v, ccl_a, dcl_a;        /* charge-voltage, charge/discharge limits */
    float soc_pct;
    uint32_t age_ms;                  /* time since the last valid frame */
} bms_t;

typedef enum { SHED_NONE, SHED_COMFORT, SHED_CRITICAL } shed_t;
typedef struct { float v_set, i_chg_max, i_dis_max; shed_t shed; bool fault; } cmd_t;

/* Called once per second by the gateway; cmd_t is written to MPPT and inverter. */
void coord_step(const bms_t m[N_MOD], cmd_t *c)
{
    float cvl = 55.2f, ccl = 1.0e9f, dcl = 1.0e9f, soc = 100.0f;
    int alive = 0;
    for (int k = 0; k < N_MOD; ++k) {
        if (m[k].age_ms > 5000u) continue;          /* silent module: assume open */
        ++alive;
        if (m[k].cvl_v < cvl) cvl = m[k].cvl_v;
        if (m[k].ccl_a < ccl) ccl = m[k].ccl_a;     /* weakest module sets limit */
        if (m[k].dcl_a < dcl) dcl = m[k].dcl_a;
        if (m[k].soc_pct < soc) soc = m[k].soc_pct;
    }
    c->fault = (alive < N_MOD);
    if (alive == 0) {                               /* no battery data: stop */
        c->v_set = 53.6f; c->i_chg_max = 0.0f; c->i_dis_max = 0.0f;
        c->shed = SHED_CRITICAL;
        return;
    }
    c->v_set = cvl;
    c->i_chg_max = 0.8f * (float)alive * ccl;       /* 20 % for unequal sharing */
    c->i_dis_max = 0.8f * (float)alive * dcl;
    switch (c->shed) {                              /* hysteresis on lowest SoC */
    case SHED_NONE:     if (soc < 30.0f) c->shed = SHED_COMFORT; break;
    case SHED_COMFORT:  if (soc < 20.0f) c->shed = SHED_CRITICAL;
                        else if (soc > 40.0f) c->shed = SHED_NONE; break;
    case SHED_CRITICAL: if (soc > 30.0f) c->shed = SHED_COMFORT; break;
    }
}
```

`SHED_COMFORT` opens the comfort and pump contactors (Figure 36.1); `SHED_CRITICAL` also lowers the inverter's discharge limit. The critical circuit, with the vaccine refrigerator, is never switched by software (R4). The 10 % hysteresis and a 10-minute minimum dwell enforced by the caller stop a passing cloud from cycling the fans.

> **Status of the principle: Weakest-module limit for parallel battery modules**
> *Approximate.* Assumes module resistances differ by less than about 20 %; a degraded cell or a loose busbar breaks the assumption, which is why 36.7 watches per-module current.

36.6 Protection

> **Boundary.**
> Shock protection, protective devices, RCD types and overcurrent coordination are in *Electrical Energy and Machines*, Ch. 29 §29.3–29.5; cable sizing and voltage drop in Ch. 30 §30.5–30.6. Converter-level protection is in *Electronics and Computing*, Ch. 31 §31.9.

Three zones carry different hazards: the PV side (up to 118 V DC, current-limited, exposed to lightning), the battery side (kilo-ampere fault currents) and the AC side (230 V, people). Table 36.8 is the protection schedule.

Table 36.8 — Protection schedule.

Zone	Device	Rating	Purpose
PV	Two-pole DC load-break isolator at combiner	32 A, 600 V DC	Isolation for maintenance under load
PV	Type 2 PV surge-protective device, both poles to earth	U_{cpv} 150 V, 20 kA (8/20 µs)	Induced surges; high keraunic level
PV	String fuses	None (two strings)	IEC 62548 reverse-current rule
Battery	Class T fuse at the busbar, one per bank	200 A, 20 kA breaking at 125 V DC	Short-circuit of the battery cable or inverter
Battery	MPPT output breaker	80 A DC	Controller fault, isolation
Battery	BMS MOSFET disconnect in each module	100 A, short-circuit trip in < 1 ms	Cell-level over/undervoltage, temperature
AC	RCD Type A, 30 mA, on all final circuits	40 A	Additional protection against shock
AC	MCBs per circuit	B10 lighting, B16 sockets, C16 pump	Overload and fault
AC	Neutral–earth bond at inverter output, earth electrode	TN-S, electrode ≤ 100 Ω	Automatic disconnection, RCD operation
AC	Type 2 AC SPD	275 V	Surges arriving on generator lead

Worked example — the battery fuse and cable. Each module's cells and BMS together are about 8 mΩ; three in parallel give about 2.7 mΩ, and the prospective short-circuit current at the busbar is $51.2/0.0027 \approx 19$ kA before cable resistance. The BMS MOSFETs usually clear it, but protection must not rely on a semiconductor that may have failed short, so a Class T fuse with 20 kA breaking capacity sits at the

busbar. The inverter surge, $5500/(0.90 \times 46) = 133$ A for 5 s, is well inside a 200 A Class T curve. The cable is 50 mm² flexible copper, 90 °C insulation (about 230 A in free air). For a 2.5 m run, the 5 m loop resistance at 70 °C is $0.0175 \times 1.2 \times 5/50 = 2.1$ mΩ, so the surge drop is $133 \times 0.0021 = 0.28$ V, 0.6 % of 46 V, and the continuous drop at 54 A is 0.11 V. The PV cable is 6 mm² PV1-F with a 15 m route: loop resistance $0.0175 \times 1.2 \times 30/6 = 0.105$ Ω, drop $13.2 \times 0.105 = 1.39$ V per string at $V_{mp} = 93$ V, 1.5 %, as budgeted in Table 36.4.

Off-grid earthing is often wrong. Running from the battery, the inverter is the source and needs a neutral–earth bond at its output, or a first insulation fault leaves the RCD without a return path and 230 V on every metal case; passing generator power through, the bond must move to the generator side. The chosen unit switches its bond with the transfer relay, and commissioning tests RCD tripping in both modes.

> **Status of the principle: Fuse breaking capacity from prospective battery fault current**
> *Derived.* Prospective current is battery voltage over the sum of cell, BMS, busbar and cable resistances, taken at the lowest (warm, full) resistance; a device rated below it may fail to interrupt.

36.7 Monitoring

> **Boundary.**
> The monitoring of a battery as an instrumentation project is in Chapter 9; MQTT, provisioning and device security are owned by Chapter 11.

What is measured and where

The first system had a voltmeter and nothing else. The gateway here (a 4G-connected embedded Linux board of the industrial Raspberry-Pi or Cerbo-GX class, about 3 W) gathers four sources, each over the interface its device already offers (its own 72 Wh/day comes out of the 10 % growth allowance of Table 36.2):

- **BMS** (CAN, 500 kbit/s): per-module voltage, current, SoC, SoH, highest and lowest cell voltage, cell temperature, alarms.
- **MPPT controller** (serial, Modbus RTU or vendor text protocol): array voltage and current, daily yield, charge state, errors.
- **Inverter-charger** (serial): AC output power, battery current, overload and temperature alarms, generator input status.
- **Refrigerator probe** (1-wire digital sensor of the DS18B20 class in a glycol-filled vial on the vaccine shelf, ±0.5 °C): the quantity the vaccine programme audits.

Battery current is not clean DC: a single-phase inverter draws power pulsating at 100 Hz, and at 2.4 kW the battery carries about $2400/(\sqrt{2} \times 0.92 \times 48) = 38$ A RMS of 100 Hz ripple on top of 54 A average. Current must be averaged over whole 20 ms periods, as the BMS coulomb counter does; a gateway sampling once a second would report noise.

Telemetry payload and alarms

Every 5 minutes the gateway publishes one state message; alarms are published immediately on change. A representative state payload on topic `hp-kurigram-07/power/state`:

```
{
  "ts": "2026-07-14T19:05:00Z",
  "soc_pct": 71.4, "soh_pct": 98.6,
  "batt_v": 52.61, "batt_a": -6.8, "cell_v_max": 3.296, "cell_v_min": 3.281,
  "batt_t_c": 33.5, "mod_a": [-2.3, -2.2, -2.3],
  "pv_w": 0, "pv_kwh_day": 7.42,
  "ac_w": 236, "ac_kwh_day": 3.95,
  "shed": "none", "fridge_c": 4.6,
  "alarms": []
}
```

At about 330 bytes, 288 messages a day and MQTT/TLS overhead roughly doubling it, the gateway uses about 6 MB a month. The alarm set (Table 36.9) is short on purpose: alarms that fire every afternoon get ignored.

Table 36.9 — Alarms and their thresholds.

Alarm	Threshold	Action
Refrigerator out of range	< 2 °C or > 8 °C for 10 min	SMS to nurse and district cold-chain officer
Comfort shedding active	`shed` ≠ none	Daily summary; immediate if lasting > 24 h
Module current imbalance	Any module deviating > 25 % from mean at > 20 A total	Maintenance ticket (loose busbar, weak cell)
PV yield low	Daily yield < 60 % of expected from satellite irradiance for 3 days	Cleaning or array fault

The yield alarm compares measured yield with a satellite irradiance feed: without a pyranometer it is the only way to tell a dirty or faulted array from a cloudy week.

36.8 Efficiency

> **Boundary.**
> Converter loss mechanisms and efficiency across load are in *Electronics and Computing*, Ch. 30 §30.8; battery round-trip efficiency is in *Electrical Energy and Machines*, Ch. 16 §16.2.

The loss chain on a design-month day

Table 36.10 follows an average July day from the array's STC-rated energy to the sockets.

Table 36.10 — Energy chain on an average design-month day.

Stage	Factor	Energy after stage (kWh/day)	Loss (kWh/day)
Array at STC rating, 2.46 × 3.9	—	9.59	—
Cell temperature	0.888	8.52	1.07
Soiling, mismatch, degradation	0.97 × 0.97	8.02	0.50
DC cabling	0.98	7.86	0.16
MPPT controller	0.97	7.62	0.24
Not needed: surplus curtailed or stored	—	5.80 used	1.82 margin
Battery round trip on 40 %	—	5.68	0.12
Inverter no-load draw	—	5.32	0.36
Inverter conversion (92 %)	—	4.90 at sockets	0.43

On a design-month day 51 % of the STC-rated array energy reaches the sockets; 19 % is the margin that refills the battery after cloudy days and is curtailed when the battery is full. Of the losses actually incurred, the two inverter terms (0.79 kWh/day together) are second only to the array's temperature loss, and they are the only large loss the designer can still change by a choice of product.

Why the inverter dominates

The datasheet quotes 94 % peak efficiency at 30–50 % of rating; the clinic's load sits at 2–16 % of it (Figure 36.2).

Worked example — the daily-average inverter efficiency. Over the day the inverter delivers 4.90 kWh and takes $4.90/0.92 + 0.36 = 5.68$ kWh, so its daily-average efficiency is $4.90/5.68 = 86.3$ %, eight points below the datasheet peak. At night, carrying 68 W, it takes $68/0.92 + 15 = 89$ W: 76 % efficient. The same calculation for the 5 kVA alternative (30 W no-load) gives $4.90/(5.33 + 0.72) = 81.0$ % daily, a loss of 0.36 kWh/day more, which over 20 years at the system's levelised cost (36.9) is about $1,500 of energy, more than the inverter itself.

> **Status of the principle: Converter efficiency weighted by the load-duration profile**
> *Derived.* The energy-weighted efficiency is total output over total input across the actual load profile, so a fixed no-load loss weighs most when the converter spends its hours far below rating; the peak or "European" efficiency on a datasheet describes a different profile.

Average and marginal efficiency differ. Moving the 12 W router to a 48 V-to-12 V DC–DC converter because the inverter is 76 % efficient at night saves nothing: the refrigerator keeps the inverter awake anyway, so the router's *marginal* efficiency through it is 92 %, no worse than a 90 % converter.

36.9 Economic assessment

> **Boundary.**
> Economic evaluation of hybrid systems (levelised cost, net present cost, sensitivity) as a method is in Chapter 25; levelised cost of storage in *Electrical Energy and Machines*, Ch. 16 §16.11.

Bill of materials

Table 36.11 — Bill of materials with indicative cost (2026 distributor and installer prices, single system, USD).

Item	Specification	Qty	Unit ($)	Total ($)
PV module	410 W mono PERC, 108 half-cell, IEC 61215/61730	6	95	570
Mounting structure	Galvanised steel, 25° tilt, roof anchors	1 set	300	300
MPPT charge controller	150 V, 60 A, CAN/serial, IEC 62109-1	1	520	520
Inverter-charger	48 V, 3 kVA, low-frequency, 35 A charger, IEC 62109	1	1,150	1,150
LFP battery module	51.2 V, 100 Ah, 16S, BMS with CAN, IEC 62619, UN 38.3	3	950	2,850
Battery rack and busbars, Class T fuse 200 A, isolator	—	1 set	180	180
PV and AC protection	DC isolator, PV and AC SPDs, RCD, MCBs, contactors, enclosures	1 set	260	260
Cables	6 mm² PV1-F, 50 mm² battery, AC wiring, glands, lugs	1 lot	220	220
Monitoring gateway	Embedded Linux with CAN and 4G, refrigerator probe	1	250	250
Earthing	Rod electrode, conductors, test link	1 set	80	80
Transport and installation	Two technicians, four days	1	600	600
Commissioning, documentation, training	IEC 62446-1 tests, as-built pack	1	200	200
Total CAPEX				**7,180**

The battery is 40 % of the capital cost and the array with its structure only 12 %: in a stand-alone system storage is what costs money, and every kilowatt-hour removed from the load saves more in battery than in PV.

Life-cycle cost

The client's alternative is a diesel generator supplying the same energy. Both are costed over 20 years at an 8 % real discount rate. Solar: $150 a year of O&M and data; inverter replaced at year 10; battery (at 60 % of today's price) and MPPT at year 12. Diesel: a 3 kVA set at $1,400, replaced every 5 years, 0.60 L/kWh at part load, $1.00/L, $350 a year of maintenance.

```
 import numpy as np

r, years = 0.08, 20                       # real discount rate, study period
e_year = 4.90 * 365                       # kWh AC delivered per year
t = np.arange(1, years + 1)
df = (1 + r) ** -t                        # discount factor per year

def lcoe(capex, om, repl):
    """repl: {year: cost}. Returns NPV of cost (USD) and LCOE (USD/kWh)."""
    cost = capex + np.sum(om * df) + sum(c * (1 + r) ** -y for y, c in repl.items())
    return cost, cost / np.sum(e_year * df)

solar = lcoe(capex=7180, om=150,
             repl={10: 1150, 12: 0.6 * 2850 + 520})   # inverter; LFP + MPPT
fuel_l = 0.60 * e_year                    # litres/yr at 0.6 L/kWh (part load)
diesel = lcoe(capex=1400, om=fuel_l * 1.00 + 350,
              repl={5: 900, 10: 900, 15: 900})        # genset every 15 000 h
for name, (npv, c) in (("solar-LFP", solar), ("diesel", diesel)):
    print(f"{name:9s} NPV cost {npv:8.0f} USD   LCOE {c:.3f} USD/kWh")
cum_s = 7180 + np.cumsum(np.full(years, 150.0)); cum_s[9:] += 1150; cum_s[11:] += 0.6*2850 + 520
cum_d = 1400 + np.cumsum(np.full(years, fuel_l + 350)); cum_d[4:] += 900; cum_d[9:] += 900;
cum_d[14:] += 900
print("undiscounted break-even year:", int(np.argmax(cum_d > cum_s)) + 1)
```

```
 solar-LFP NPV cost    10071 USD   LCOE 0.574 USD/kWh
diesel    NPV cost    16685 USD   LCOE 0.950 USD/kWh
undiscounted break-even year: 5
```

Worked example — reading the result. Solar costs $10,071 in present value against $16,685 for diesel ($0.57 against $0.95 per kWh) and pays back in the fifth year, counting only the 4.90 kWh/day used, not the curtailed surplus. The comparison is most sensitive to the diesel price and to the battery's life: with diesel at $0.70/L the diesel LCOE falls to $0.77/kWh, still above solar; a battery that lasted only 8 years instead of 12, needing a second replacement at year 16, would raise the solar LCOE by 8 %, to $0.62/kWh. It is least sensitive to the module price. It leaves out what the client values most: no fuel to carry, no noise in the delivery room, and a refrigerator independent of fuel deliveries in the rainy season.

> **Status of the principle: Levelised cost of energy by discounted cash flow**
> *Approximate.* Divides the present value of all costs by the present value of energy delivered; the answer depends on the discount rate, the assumed replacement years and future prices, none of which is known, so it is reported with its sensitivities rather than as a single number.

36.10 Verification, failure analysis and Revision B

> **Boundary.**
> Test planning, verification versus validation and regression are in Chapter 4; test reports and revision records in Chapter 5. Safe testing and measurement categories are in *Electrical Energy and Machines*, Ch. 29 §29.7.

Verification and test results

Each requirement has a test with a limit, written before installation; the IEC 62446-1 PV tests come first, with the array disconnected. Table 36.12 gives the plan with commissioning and first-month results (representative pilot-installation values).

Table 36.12 — Verification plan and test results.

Req.	Test	Limit	Predicted	Measured	Result
R6	String 1 / string 2 V_{oc} at 34 °C module temperature	±5 %	109.2 V	108.4 V / 102.6 V (−6.0 %)	Pass / **Fail**, 108.6 V after repair
R6	PV insulation, 500 V DC, worst pole	≥ 1 MΩ	—	38 MΩ	Pass
R6	RCD trip at 30 mA, inverter / generator mode	≤ 300 ms	—	22 / 24 ms	Pass
R2	2.4 kW load bank, 1 h, room at 35 °C	No derating	—	Heatsink 71 °C, no alarm	Pass
R2	Pump and refrigerator started together, 0.6 kW base, 10 repeats	No trip; dip ≤ 20 %	2.9 kVA peak	12 % worst dip, no trip	Pass
R3	PV disconnected 24 h from full, normal load	≥ 1.95 d at EOL	5.68 kWh/day	5.64 kWh (DC meter ±1 %); SoC drop 37.5 ±1.5 %	Pass on metered energy (1.96 d)
R4	Forced `SHED_CRITICAL`, refrigerator logged 4 h	2–8 °C, circuit closed	—	3.9–5.8 °C	Pass
R1	Metered energy, first 30 days (June–July)	No unserved load	—	4.31 kWh/day used, 0 unserved	Pass
R7	Probe warmed past 8 °C in water bath	SMS ≤ 10 min	—	6 min 40 s	Pass
R5	30-day log review	Cell ≤ 3.50 V, battery ≤ 40 °C	—	3.47 V, 36 °C	Pass

The string-2 failure was a cross-mated connector: a plug of another brand crimped onto the extension cable had partly cracked, and its contact resistance showed in V_{oc}. It was replaced with a matched pair. Autonomy is judged on metered DC energy because the SoC of an LFP bank on its flat plateau is uncertain by a few per cent until the next full charge (*Electrical Energy and Machines*, Ch. 16 §16.4); the SoC-based range, 1.85–2.00 days, straddles the limit. The margin over R3 is real but small, which is why the sizing used 80 % and not 85 % end-of-life SoH.

Failure analysis

Table 36.13 — FMEA of the solar battery system (S, O, D on 1–10 scales; RPN = S×O×D).

Failure mode	Effect	Cause	Detection	Mitigation	S	O	D	RPN
Inverter trips on motor start	Refrigerator off until reset	Coincident locked-rotor currents	Overload alarm	Soft-start pump; auto-restart; coincident-start test	8	2	3	48
PV connector overheats	String loss, roof fire risk	Cross-mated connector brands	V_{oc} test; yield alarm; annual thermal image	Matched pairs only; crimp-tool control	9	3	4	108
One battery module opens	Others carry its current; cascade trip at surge	Weak cell, over-temperature	Module silent on CAN; imbalance alarm	Limits shrink with alive count (36.5)	7	3	2	42
Chronic monsoon undercharge	Daily shedding; less refrigerator autonomy	Load growth, soiling, tree shade	Shed-duration and yield alarms	Clean, prune, add a string	6	4	3	72
Lightning surge	Controller or gateway damage	Strike near array or generator lead	SPD window; device offline	Type 2 SPDs both sides; bonding	7	3	5	105
Refrigerator probe displaced	Missed excursion or false alarm	Stock handling	Plausibility against compressor cycling	Probe fixed in glycol vial; quarterly reference check	8	3	4	96
Neutral–earth bond lost in generator mode	RCD cannot trip; touch voltage on fault	Generator wiring, transfer-relay fault	RCD test in both modes	Test at commissioning and every visit	10	2	6	120

The two highest RPNs are not electronic: the lost bond (rare, severe, invisible in operation) and the mismatched connector (common, found on this very system). Both call for procedure and inspection, not more hardware.

Revision B

Change	Reason	Effect
Router, modem and gateway on a 48 V-to-12 V DC–DC converter	An inverter trip must not also silence the alarm path	Alarms survive an inverter fault; energy neutral (36.8)
Rails for 9 modules; third string added when load grows	Monsoon shedding risk with load growth	3.69 kWp; the 60 A controller clips only on bright dry-season middays, when surplus is curtailed anyway
Pump as deferrable load: runs on SoC > 90 % and PV > 1 kW into a 2 m³ tank	1.8 kWh/day curtailed in July, more in dry months	Uses surplus; pump energy bypasses the battery
Gateway monitors the inverter's earth-relay state in both modes	Highest-severity FMEA item relied on a manual test	Alarm instead of a quarterly check
Full absorption forced at least every 14 days	LFP SoC drift on the flat plateau	Shedding decisions on a trustworthy SoC

What remains open

Open question	Where it is answered
How is a PV plant designed in general, including grid-connected inverters, shading and performance ratio monitoring?	Chapter 22
How is the BMS inside each module designed: cell measurement, balancing, SoC and SoH estimation?	Chapter 23
How would the system grow into a village mini-grid with several sources, inverter coordination and tariffs?	Chapter 25
How is the energy metering of the clinic's circuits built and calibrated?	Chapter 35
How are the gateway's MQTT connection, credentials and firmware updates secured?	Chapter 11

Next: Chapter 37 builds a closed-loop motor drive, from characterising the motor on the bench to a tuned, protected and tested servo.

Chapter 37 — Project: Closed-Loop Motor Drive

Part IX · Interdisciplinary Engineering Projects

The first prototype of a servo drive for the index table of a blister-packing machine passed every bench test, its speed loop tuned crisp and well damped on the bare motor. Bolted to the machine, with the table driven through a toothed belt, the loop rang at 240 Hz whenever the table moved; lowering the gain stopped the ringing and made the table overshoot its stop by two degrees. A week later an operator pressed the emergency stop at 3000 rpm: the motor and table returned 7 J to a DC bus with less than 1 J of headroom, the bus rose past 100 V, and two MOSFETs and the gate driver failed short. The resonance was in mechanics the loop had never seen, and the energy had nowhere to go because a 48 V power supply cannot absorb current. This chapter builds the drive again as a project: a 48 V, 400 W permanent-magnet synchronous motor (PMSM) with a 2500-line incremental encoder, a three-phase MOSFET inverter switching at 20 kHz, field-oriented control (FOC) on an STM32G474-class microcontroller, and a brake chopper, characterised, designed, coded, protected and tested against written limits.

> **How is a closed-loop motor drive designed from measured motor parameters to loop gains, so that its bandwidth, accuracy and torque envelope are known before it is built? Which sensing, firmware structure and protection make that design work on a real machine, and how is it tested, including when it is driven into faults?**

Concept	Engineering destination
Motor characterization	Measured parameters every gain is computed from
Encoder integration	Rotor angle and speed; wiring, alignment, fault detection
Power stage	Inverter, gate driver, DC link, brake chopper, thermal design
Current and speed measurement	Shunt sampling synchronised to PWM; M/T speed estimation; error budgets
Controller design	Cascaded PI current and speed loops with known margins and bandwidths
Firmware	Interrupt structure, timing budget, state machine and the FOC routine
Protection	Hardware overcurrent, bus overvoltage, encoder loss, thermal limits, safe torque off
Performance testing	Dynamometer tests of bandwidth, load rejection, ripple, efficiency and fault response

Roadmap. Requirements and architecture, then the motor measured on the bench (37.1) → angle and speed from the encoder (37.2) → inverter and brake chopper (37.3) → current and speed sensing (37.4) → loop design (37.5) → firmware (37.6) → protection (37.7) → tests, failure analysis, cost and Revision B (37.8).

37.1 Motor characterization

> **Boundary.**
> The PMSM's d–q model and torque equation are in *Electrical Energy and Machines*, Ch. 8 §8.1, and PMSM field-oriented control in Ch. 12 §12.5–12.8; motor–load matching in Ch. 10 §10.1. Motor drivers, encoders and PID implementation at the level of a robot joint are owned by Chapter 15, and system identification by Chapter 17. This chapter applies them to one drive.

Requirements

The index table carries 24 blister nests and is driven through a 10:1 toothed-belt and gear reduction. Each index moves the table one 15° pitch (150° at the motor) in 30 ms, followed by a 70 ms dwell, ten indexes a second; a triangular speed profile peaks at 1670 rpm and needs 1.75 N·m. For run-out and cleaning the machine also turns the table continuously with the motor at up to 3000 rpm. Table 37.1 is the drive's requirement set.

Table 37.1 — Requirements of the index-table servo drive.

ID	Requirement	Target	Verification
R1	Speed and torque envelope	0–3000 rpm; 1.27 N·m continuous to 3000 rpm and 3.8 N·m peak to 2500 rpm, at bus ≥ 44 V	Dynamometer envelope test
R2	Current loop	Crossover ≥ 1.5 kHz, phase margin ≥ 45°	Injected-sine loop measurement
R3	Speed loop	Crossover ≥ 100 Hz, phase margin ≥ 45°, on the machine with its belt	Loop measurement on machine
R4	Load rejection	Rated-torque step at 1000 rpm: dip ≤ 120 rpm, back within ±10 rpm in ≤ 15 ms	Dynamometer torque step
R5	Low-speed smoothness	Speed ripple ≤ ±1.5 rpm at 30 rpm	Encoder log, 10 s
R6	Protection	Overcurrent off in ≤ 2 µs; bus ≤ 62 V on any stop; encoder loss detected in ≤ 1 ms; safe torque off (STO) input	Fault-injection tests
R7	Drive efficiency	≥ 97 % at rated output	Power analyser
R8	Cost	Drive BOM ≤ $100 at 100 pieces	Costed BOM

Architecture

Figure 37.1 shows the drive. A 48 V, 600 W supply feeds the DC link; a MOSFET bridge, driven by a three-phase gate-driver IC with current-sense amplifiers, supplies the motor, and three low-side shunts measure the phase currents. The encoder reaches the microcontroller's quadrature timer through RS-422 receivers, and a brake chopper dumps regenerated energy into a resistor. A 4 kHz speed loop produces the torque-current reference for a 20 kHz current loop in the rotor's d–q frame. CAN carries commands from the machine's PLC, and an STO input removes the gate-driver supply independently of firmware.

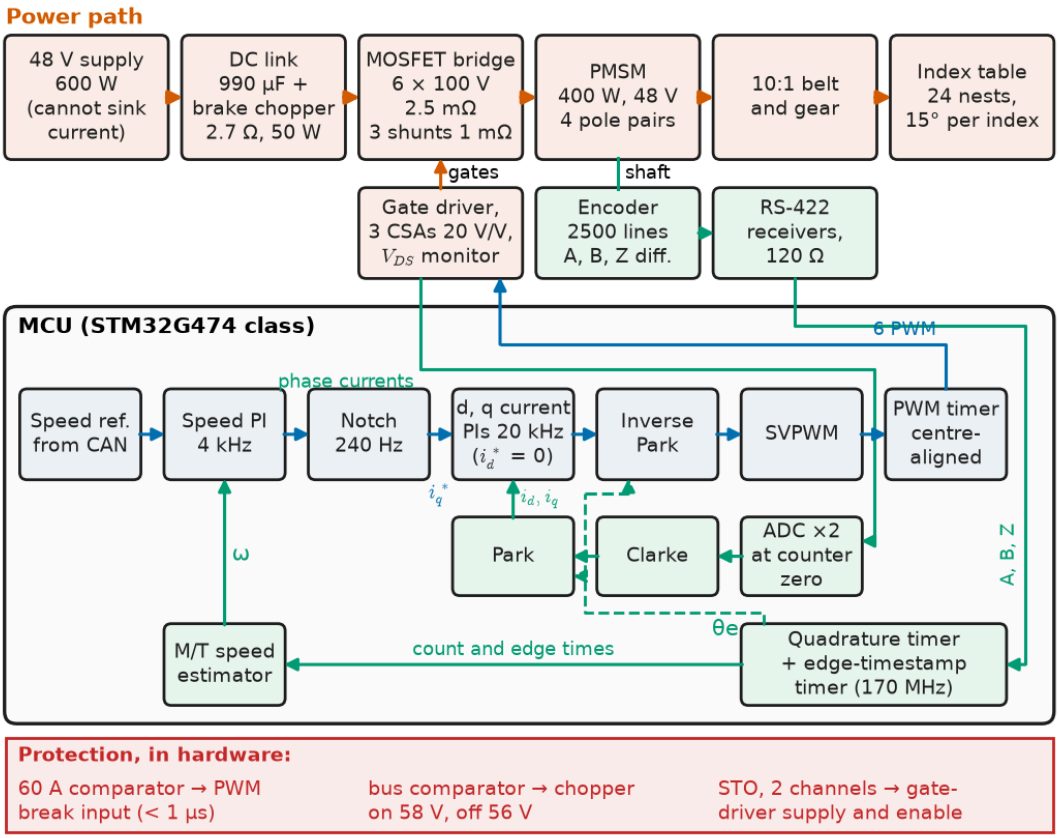


Figure 37.1 — Architecture of the 48 V, 400 W PMSM servo drive with its cascaded control loops.

Parameters from the bench

Datasheets of low-cost servomotors give little beyond a nameplate, so the parameters the loop design needs are measured (Table 37.2).

Table 37.2 — Measured motor and load parameters (25 °C unless stated).

Parameter	Test	Result
Phase resistance R_s	4-wire DC, 2 A, line to line, halved	0.085 Ω (0.170 Ω line to line); 0.11 Ω at 100 °C
Phase inductance L_s	LCR meter at 1 kHz, line to line, halved, at 12 rotor angles	0.25 mH, variation < 4 % (surface magnets, $L_d \approx L_q$)
Pole pairs p	Back-EMF frequency at 1000 rpm: 66.7 Hz	4
Back-EMF constant	Motor driven by dynamometer at 1000 rpm, line-to-line RMS	8.21 V per 1000 rpm
Flux linkage ψ_m	Phase EMF peak over ω_e: 6.70/418.9	0.0160 Wb
Torque constant $K_t = \frac{3}{2} p \psi_m$	From ψ_m; checked by locked-rotor torque at 10 A	0.096 N·m per A peak (0.136 N·m per A RMS)
Friction	Constant-speed runs, $K_t i_q$ against speed	20 mN·m Coulomb + 18 µN·m·s viscous
Total inertia J (motor + reflected table)	Coast-down, listing below	1.49 kg·cm² (motor 0.45)
Winding thermal time constant	1.5× rated current, NTC on end winding	about 22 min

The torque constant follows from the back-EMF test because both are the flux linkage scaled by $\frac{3}{2}p$ (*Electrical Energy and Machines*, Ch. 8 §8.1); the locked-rotor check agrees within 2 %. Inertia and friction include the table and belt, so they are measured in place: constant-speed runs give friction torque from $K_t i_q$, and a coast-down with the drive disabled gives J. In the listing the logged data are replaced by synthetic data with the same parameters and noise so that it runs on its own.

```
 import numpy as np

rng = np.random.default_rng(7)
Kt = 0.096                          # N*m/A (peak-amplitude convention), from the EMF test

# 1) Constant-speed runs: friction torque = Kt * iq.  The logged data are replaced
#    here by synthetic values with Tc = 0.020 N*m, B = 2.0e-5 N*m*s plus noise.
w_ss = np.array([50, 100, 150, 200, 250, 300.0])                # rad/s
iq_ss = (0.020 + 2.0e-5*w_ss)/Kt + rng.normal(0, 0.004, w_ss.size)  # A
B, Tc = np.polyfit(w_ss, Kt*iq_ss, 1)

# 2) Coast-down from 300 rad/s with the drive disabled, speed logged every 1 ms
#    (synthetic: J = 1.5e-4 kg*m^2, same friction, 0.05 rad/s encoder noise).
dt = 1e-3
t = np.arange(0, 3.0, dt)
w = (300 + 0.020/2.0e-5)*np.exp(-t*2.0e-5/1.5e-4) - 0.020/2.0e-5
w = w[w > 20] + rng.normal(0, 0.05, np.count_nonzero(w > 20))
c = np.polyfit(t[:w.size], w, 3)               # smooth before differentiating
wf, dwf = np.polyval(c, t[:w.size]), np.polyval(np.polyder(c), t[:w.size])
J = -np.sum(dwf*(Tc + B*wf))/np.sum(dwf*dwf)   # least squares on J*dw/dt = -(Tc + B*w)

print(f"Coulomb friction Tc = {Tc*1e3:.1f} mN*m, viscous B = {B*1e6:.1f} uN*m*s")
print(f"coast-down lasts {t[w.size-1]:.2f} s, total inertia J = {J*1e4:.3f} kg*cm^2")
print(f"mechanical time constant J/B = {J/B:.1f} s")
```

```
 Coulomb friction Tc = 20.2 mN*m, viscous B = 18.3 uN*m*s
coast-down lasts 1.82 s, total inertia J = 1.492 kg*cm^2
mechanical time constant J/B = 8.2 s
```

The fit recovers inertia within 0.6 % but viscous friction only within 9 %, which is acceptable: friction is a disturbance the integrator removes, while inertia sets the speed-loop gain. The load-to-motor inertia ratio is 2.3, comfortable for a belt drive.

> **Status of the principle: Inertia from coast-down and friction torque**
> *Derived.* Newton's law for the shaft, $J\,d\omega/dt = -T_f(\omega)$, with friction measured at steady speed; it assumes friction depends only on speed and that the open-circuited motor produces no torque apart from small drag the friction fit absorbs.

The torque–speed envelope

The inverter can apply at most $0.95\,V_{dc}/\sqrt{3}$ (space-vector modulation less the sampling margin of 37.4). Steady-state voltages with $i_d = 0$ are $v_q = R_s i_q + \omega_e \psi_m$ and $v_d = -\omega_e L_s i_q$ (*Electrical Energy and Machines*, Ch. 12 §12.8).

Worked example — is 48 V enough? At 3000 rpm, $\omega_e = 4 \times 314.16 = 1256.6$ rad/s. Rated torque needs $i_q = 1.273/0.096 = 13.3$ A.

- $v_q = 0.11 \times 13.3 + 1256.6 \times 0.0160 = 1.46 + 20.11 = 21.6$ V (hot winding); $v_d = -1256.6 \times 0.25 \times 10^{-3} \times 13.3 = -4.2$ V; amplitude $\sqrt{21.6^2 + 4.2^2} = 22.0$ V.

- Available at 44 V (R1's minimum bus): $0.95 \times 44/\sqrt{3} = 24.1$ V. Margin 9 %.
- Peak torque, $i_q = 39.8$ A: the same calculation reaches the 48 V limit (26.3 V) at about 2850 rpm, so R1's peak-torque corner of 2500 rpm holds with a little margin even at 44 V, where it falls to about 2570 rpm.

Figure 37.2 plots the envelope. Field weakening (negative i_d) could extend the speed range, but it is not needed for R1 and is left to Revision B.

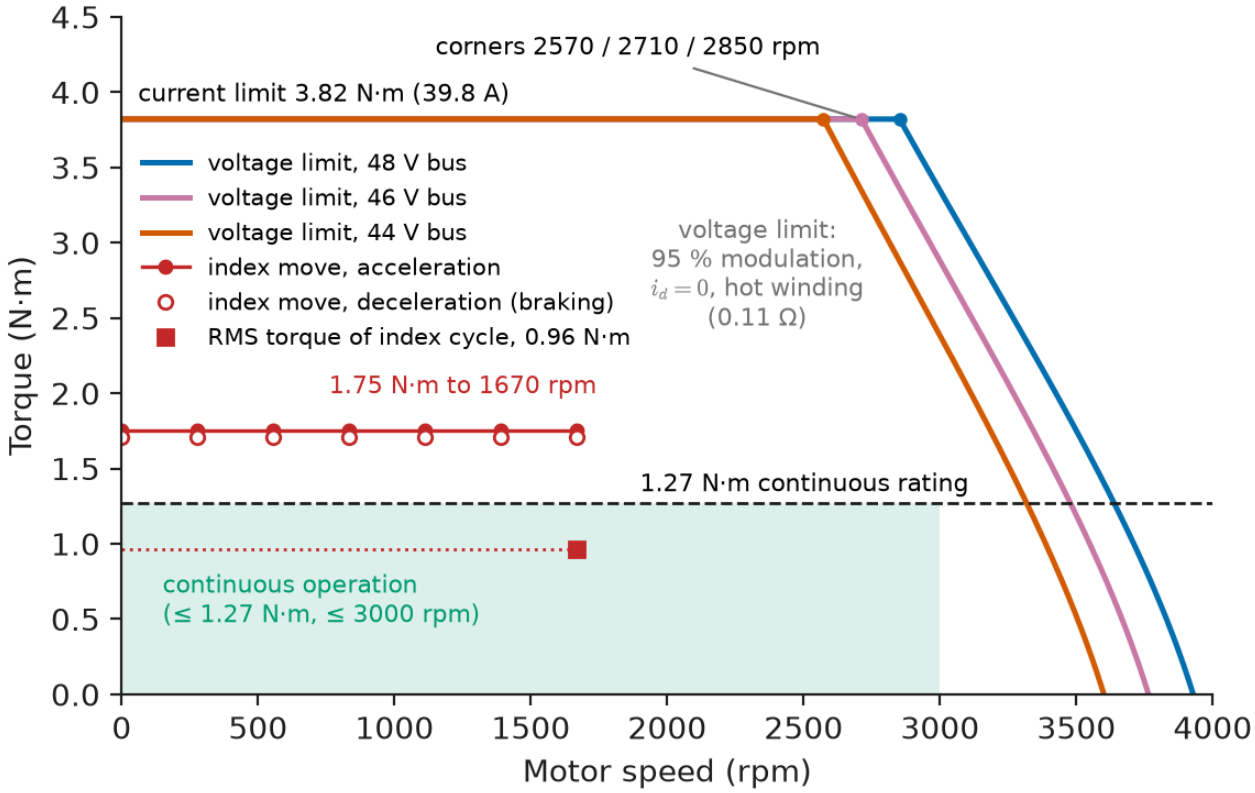


Figure 37.2 — Torque-speed envelope of the drive at three bus voltages with the index-table duty points.

37.2 Encoder integration

> **Boundary.**
> Quadrature decoding, encoder types and their interfaces as robot components are in Chapter 15; sensor principles in Chapter 6. Line drivers and receivers are in *Electronics and Computing*, Ch. 26 §26.2.

Choosing the position sensor

Table 37.3 — Rotor position sensors for a 400 W servo.

Sensor	Resolution	Absolute?	Interface and noise immunity	Cost (motor-mounted)	Verdict
Optical incremental, 2500 lines, A/B/Z + U/V/W, RS-422	10,000 counts/rev	No (U/V/W give 60° e at start)	Differential, robust in a motor cable bundle	$25–40	**Chosen**
Magnetic on-axis IC (AS5047P class), 14-bit, ABI + SPI	4,000 counts/rev on ABI	Yes, over SPI	Single-ended ABI needs short leads or a local line driver	$8–15	Revision B candidate
Resolver with resolver-to-digital converter	12–16 bit	Yes	Very robust to temperature and shock	$60–150	Over-specified here
Sensorless (back-EMF observer)	—	No	No cable	$0	Fails R5 and start under load

Wiring and decoding

At 3000 rpm the A channel runs at $2500 \times 50 = 125$ kHz and quadrature edges arrive at 500 kHz. Each channel is a twisted pair in a shielded cable of up to 3 m, terminated with 120 Ω at an RS-422 receiver of the AM26C32 class feeding the timer in encoder mode (×4 counting, auto-reload 9999, so the count is the mechanical angle). The timer's input filter rejects pulses shorter than about 190 ns, which removes ringing coupled from the switching phases while legitimate edges are 2 µs apart. The shield is bonded to the chassis with a 360° clamp: the prototype's pigtail let 3 V/ns phase edges into the encoder pairs and caused count errors.

Electrical-angle alignment

The offset between encoder count and electrical zero is found once and stored: drive $i_d = 8$ A at $\theta_e = 0$, let the rotor settle on the d-axis for 500 ms, record the count, repeat at 120° and 240°, and require agreement within 3 counts. Because belt preload can hold the rotor off the d-axis, the result is checked against the back-EMF zero crossing of phase A with the motor spun by the dynamometer.

Worked example — how much angle error matters. One count is $360^\circ \times 4/10\,000 = 0.144^\circ$ electrical. An alignment error $\Delta\theta$ reduces torque per ampere by $\cos\Delta\theta$ and turns a fraction $\sin\Delta\theta$ of the current into a d-axis component. For 3 counts (0.43°) the torque loss is 0.003 % and negligible. For a 10° error (the size of mistake made by aligning with the belt under tension), torque falls 1.5 % and 17 % of the current magnitude becomes d-axis current, which at high speed weakens or strengthens the field and shifts the voltage limit of 37.1 by about 1 V.

Before the first index pulse, the U/V/W tracks give the angle within ±30° electrical, still at least $\cos 30^\circ = 87$ % of torque per ampere.

37.3 Power stage

> **Boundary.**
> The inverter leg, dead time and PWM are in *Electronics and Computing*, Ch. 31 §31.2–31.3, semiconductor selection and thermal design in §31.8–31.9, and the brake chopper in Ch. 32 §32.6. Here the parts are chosen and the budgets written.

MOSFETs and gate driver

The link reaches 58 V when the chopper engages and 62 V at the hard trip, and switching overshoot adds 10–15 V. A 100 V MOSFET leaves about 25 % margin over 77 V; an 80 V part leaves 4 % (Table 37.4).

Table 37.4 — Inverter switch options (per switch; 2026 prices at 100 pieces).

Option	$R_{DS(on)}$ at 25 °C	Q_g	Package	Price	Verdict
80 V trench MOSFET, 5×6 mm	1.8 mΩ	70 nC	PowerPAK/SuperSO8 class	$1.10	Voltage margin too small
100 V trench MOSFET, 5×6 mm	2.5 mΩ	80 nC	SuperSO8 class	$1.40	**Chosen**
100 V MOSFET, TOLL	1.5 mΩ	150 nC	TOLL	$2.60	Larger, more gate charge; for 1 kW variant
100 V GaN HEMT	3.2 mΩ	15 nC	4×6 mm QFN	$4.50	Switching loss is not the problem at 20 kHz

The gate driver is a 100 V three-phase IC of the DRV8353 class: bootstrap high-side supplies, programmable gate current, V_{DS} monitoring of each MOSFET and three current-sense amplifiers. Dead time is 200 ns, covering the MOSFET's turn-off delay at the slowest gate-current setting; gate current is set for edges of about 15 ns, a compromise between switching loss and the dv/dt that couples into the encoder and bearings.

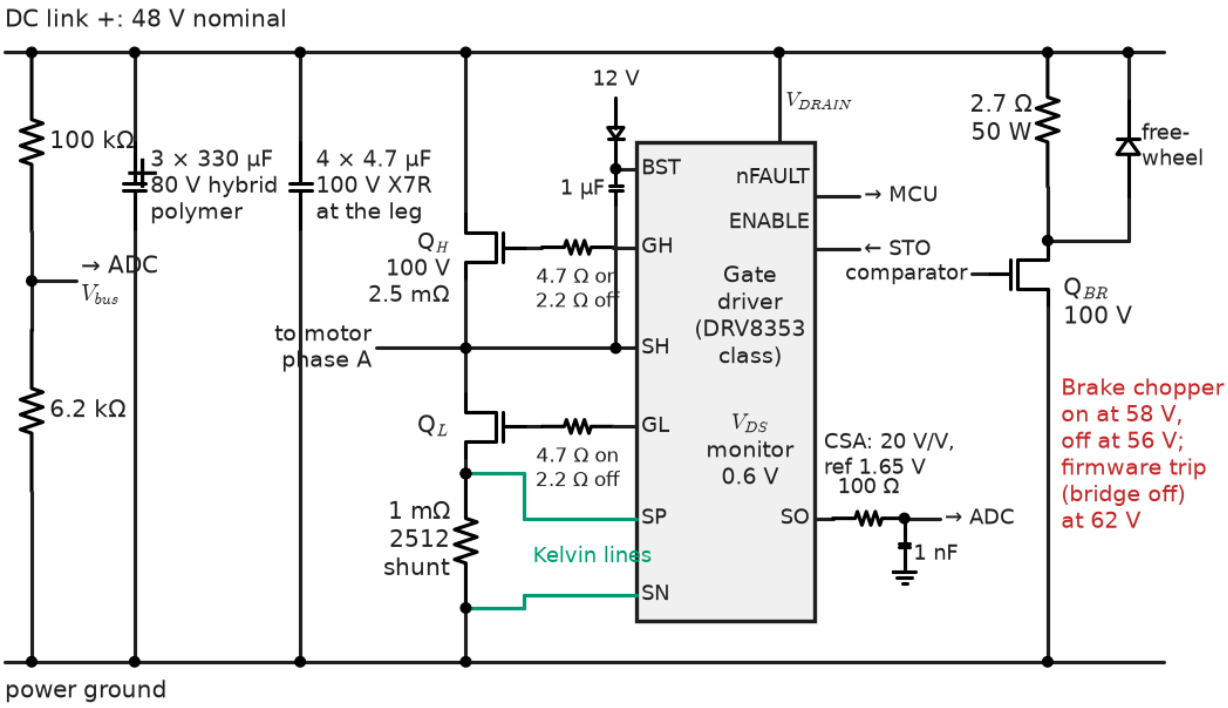


Figure 37.3 — One inverter leg with shunt, gate driver, DC link and brake chopper.

DC-link capacitors

The link capacitors carry the inverter's ripple current, about 0.57 times the RMS phase current at modulation index 0.90 and power factor 0.95: 5.3 A at rated current. Three 330 µF, 80 V hybrid polymer capacitors rated 3.0 A each carry 9 A between them (70 % margin), with four 4.7 µF ceramics at the leg. The 990 µF total also sets how much braking energy the link can absorb.

Where the braking energy goes

Worked example — the stop that destroyed the first prototype. The kinetic energy at 3000 rpm is $\frac{1}{2}J\omega^2 = \frac{1}{2} \times 1.49 \times 10^{-4} \times 314.2^2 = 7.4$ J. Braking at peak torque returns up to $3.82 \times 314.2 = 1.2$ kW to the link at the start of the stop, less about 0.2 kW of copper loss. The link capacitors can absorb only $\frac{1}{2} \times 990 \times 10^{-6} \times (60^2 - 48^2) = 0.64$ J between 48 V and 60 V. With 80 % of the kinetic energy returned and no other path, the bus would rise to

$$V = \sqrt{48^2 + \frac{2 \times 0.8 \times 7.4}{990 \times 10^{-6}}} = 119 \text{ V}$$

which is what happened. The supply cannot absorb reverse current, so a brake chopper switches a resistor across the link above 58 V and releases it below 56 V. To absorb the full 1.2 kW at 58 V, the resistor must be at most $58^2/1200 = 2.8$ Ω; a 2.7 Ω resistor draws 21.5 A when switched on. Its average power is set by the duty: at ten indexes a second from a 1670 rpm peak, each stop returns at most $\frac{1}{2} \times 1.49 \times 10^{-4} \times 174.5^2 = 2.3$ J, so even if every stop were dumped the resistor would average 23 W. A 50 W aluminium-housed resistor on the heatsink was chosen.

Status of the principle: Regenerated energy must have a sink
Fundamental. Energy conservation: kinetic energy removed by electrical braking appears on the DC link unless dissipated in the windings or inverter; with a unidirectional supply, the link voltage rises until capacitance, a resistor or a storage element absorbs it.

Loss and thermal budget

Table 37.5 — Drive loss budget at rated output (9.38 A RMS, 48 V, 20 kHz) and at peak current for 2 s.

Loss	Basis	Rated (W)	Peak, 39.8 A (W)	Budget (W)	Margin at rated
MOSFET conduction, 3 legs	$I_{rms}^2 R_{DS(on)}$ per leg, 4.0 mΩ hot	1.06	9.5	2.0	47 %
MOSFET switching, 3 legs	$\frac{1}{2} V \bar{I} (t_r + t_f) f_{sw}$, 15 ns edges	0.36	1.1	0.8	55 %
Reverse recovery and C_{oss}, 3 legs	Datasheet Q_{rr}, Q_{oss}	0.30	0.5	0.5	40 %
Body-diode conduction in dead time	0.8 V, 2 × 200 ns per period	0.16	0.5	0.3	47 %
Shunts, 3 × 1 mΩ	Upper bound $I_{rms}^2 R$	0.26	2.4	0.5	48 %
Gate drive, MCU, auxiliaries	80 nC × 10 V × 20 kHz × 6, plus 0.6 W	0.70	0.7	1.0	30 %
Total		**2.84**	**14.7**	**5.1**	**44 %**

At rated output the motor draws about 445 W, so the drive's efficiency is $445/(445 + 2.8) = 99.4$ % (R7). On a 2-oz, four-layer board bolted to an aluminium spreader each MOSFET sees about 25 K/W to ambient; at peak current it dissipates 1.7 W, a steady-state rise of 42 K that a 2 s peak does not reach.

37.4 Current and speed measurement

Boundary.
ADC front ends, references and sampling are in *Electronics and Computing*, Ch. 26 §26.8 and Chapter 7 of this book.

Current sensing

Table 37.6 — Phase-current sensing options.

Method	Bandwidth and timing	Accuracy issues	Cost	Verdict
Low-side shunts, driver-integrated amplifiers	Only while low-side switch is on	Modulation limited to about 95 %	Lowest	**Chosen**
In-line shunts, PWM-rejecting amplifier (INA240 class)	Sample any time	30 V/ns common mode	+$6	Revision B
Hall-effect sensor (TMCS1100 class)	80 kHz, any time	Offset drift, noise	+$5	Rejected: torque ripple

Each shunt (1 mΩ, 1 %, 50 ppm/K) feeds an amplifier of gain 20 V/V around 1.65 V: ±82.5 A full scale, 40 mA per LSB of a 12-bit ADC, leaving room above the ±39.8 A peak for the 60 A overcurrent comparator. The currents are sampled by two ADCs triggered at the centre of the PWM period, when all low-side switches are on (zero vector 000). The amplifiers need about 1.5 µs to settle after the low-side switch turns on and the ADC samples for 0.25 µs; the sample sits at the centre of the low-side on-time, so the on-time must be at least $2 \times (1.5 + 0.25) = 3.5$ µs: duty at most $1 - 3.5/50 = 93$ %. When one phase exceeds that duty, its current is reconstructed from the other two ($i_a + i_b + i_c = 0$), which is why the drive measures all three.

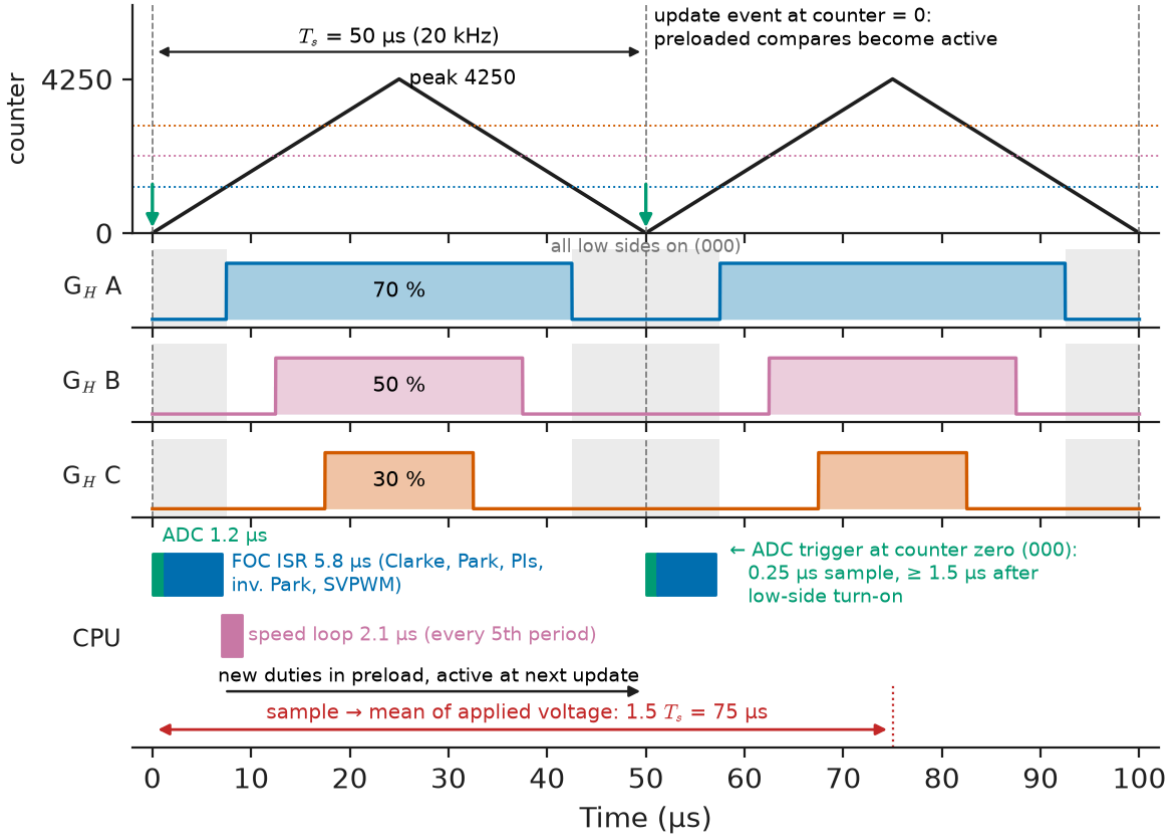


Figure 37.4 — Centre-aligned PWM, current sampling instant and control-loop timing over two 50 us periods.

Table 37.7 — Current-measurement error budget (at 40 °C rise of the shunts).

Source	Gain error	Offset (mA)	Basis
Shunt tolerance	1.0 %	—	Datasheet
Shunt temperature coefficient, 40 K	0.2 %	—	50 ppm/K
Amplifier gain error	1.0 %	—	Driver datasheet, typical-to-max
ADC and reference gain	0.5 %	—	Internal reference after factory trim
Amplifier and ADC offset after start-up calibration	—	25	128-sample average at zero current; residual drift
ADC noise (single sample)	—	40 RMS	1 LSB RMS; averaged by the current loop
Root-sum-square	**1.5 %**	**25 offset, 40 noise**	Budget: 2 %, 50 mA offset — margin 25 % and 50 %

Gain error scales torque and is absorbed by the speed loop. Offset is worse: in the rotor frame a phase-current offset becomes a ripple at the electrical frequency, which the current loop faithfully turns into torque ripple, and at 30 rpm (2 Hz electrical) it appears directly in R5. The start-up offset calibration is therefore not optional.

Speed from the encoder

Worked example — speed resolution. Counting edges per 250 µs period (the M method) has a quantum of $1/(250\ \mu\text{s})/10\,000 \times 60 = 24$ rpm: 2.4 % noise at 1000 rpm and useless at 30 rpm. The M/T method divides the count difference by the time between the *edges* that bound it, timestamped by a 170 MHz timer whose XOR input sees every quadrature edge. Its resolution is $5.9\ \text{ns}/250\ \mu\text{s} = 2.4 \times 10^{-5}$, 0.024 rpm at 1000 rpm. When edges arrive less than once per period, the estimate is bounded by "one count since the last edge" and falls smoothly to zero at standstill.

```
 #include <stdbool.h>
#include <stdint.h>

#define F_TS   170.0e6f                 /* timestamp timer clock, Hz */
#define CPR    10000                    /* counts per revolution, TIM2 ARR = CPR-1 */
#define RAD_PER_COUNT (6.2831853f / CPR)

typedef struct { uint32_t cnt_prev, ts_prev; float w; uint8_t bad; } spd_t;

/* Called at 4 kHz. TIM2: quadrature counter. TIM5: free-running, captures
   every encoder edge through its A XOR B input. Register names illustrative. */
bool speed_update(spd_t *s, uint32_t now)
{
    uint32_t ts  = TIM5->CCR1;
    uint32_t cnt = TIM2->CNT;
    if (TIM5->CCR1 != ts) { ts = TIM5->CCR1; cnt = TIM2->CNT; }  /* edge in between */

    int32_t dn = (int32_t)cnt - (int32_t)s->cnt_prev;           /* unwrap modulo CPR */
    if (dn >  CPR / 2) dn -= CPR;
    if (dn < -CPR / 2) dn += CPR;
    if (dn > 200 || dn < -200)          /* > 4800 rpm: physically impossible */
        return ++s->bad < 3;            /* encoder fault after 3 in a row */
    s->bad = 0;

    if (dn != 0) {                      /* M/T: counts over edge-to-edge time */
        s->w = (float)dn * RAD_PER_COUNT * F_TS / (float)(ts - s->ts_prev);
        s->cnt_prev = cnt;
        s->ts_prev  = ts;
    } else {                            /* no edge: |w| below one count since last edge */
        float w_max = RAD_PER_COUNT * F_TS / (float)(now - s->ts_prev);
        if (s->w >  w_max) s->w =  w_max;
        if (s->w < -w_max) s->w = -w_max;
    }
    return true;
}
```

The plausibility test rejects jumps the drive could not physically produce; three in a row (0.75 ms) declare a fault (R6). An index-pulse check requires the count at the index to match the stored value within ±2 counts, so gained or lost counts fault the drive instead of commutating on a wrong angle.

37.5 Controller design

> **Boundary.**
> Cascade control, the technical (modulus) optimum for the current loop and the symmetrical optimum for the speed loop are in *Electrical Energy and Machines*, Ch. 11 §11.5; margins, loop shaping and digital control, including the delay of sampling and hold, are in *Circuits, Signals and Communications*, Ch. 35 §35.4 and §35.8. PID implementation details (discretisation, derivative filtering) are in Chapter 15.

Current loops

In the rotor frame each axis is a series R_s–L_s circuit; cross-coupling $\omega_e L_s i$ and the back-EMF are slow disturbances for the integrator. The PI zero cancels the electrical pole R_s/L_s (2.9 ms), leaving an integrator with the loop's own delay: one period from sampling to the next duty update plus half a period of PWM averaging, $T_d = 1.5 T_s = 75$ µs at 20 kHz. The crossover ω_c then fixes gains and margin:

$$K_p = L_s\,\omega_c, \qquad K_i = R_s\,\omega_c, \qquad \mathrm{PM} = 90^\circ - \omega_c T_d \cdot \frac{180^\circ}{\pi}$$

For $\omega_c = 2\pi \times 1500$ rad/s: $K_p = 2.36$ V/A, $K_i = 801$ V/(A·s), PM $= 90^\circ - 40.5^\circ = 49.5^\circ$ (R2). A higher crossover needs a shorter delay, which means a faster PWM and more switching loss.

Speed loop

The speed-loop plant is $K_t/(Js)$ behind small lags: the closed current loop (about 106 µs), M/T averaging (125 µs), the 4 kHz controller's hold (125 µs) and a 2 kHz speed filter (80 µs), $T_{eq} = 436$ µs in total. The symmetrical optimum with spacing a puts the crossover at $1/(aT_{eq})$ and the PI zero at $1/(a^2 T_{eq})$; $a = 3$ (about 53° margin, against 37° for the classical $a = 2$) was chosen because the inertia changes with product load.

$$K_{p\omega} = \frac{J}{K_t\, a\, T_{eq}}, \qquad T_{i\omega} = a^2 T_{eq}$$

The listing computes both loops with the delays as e^{-sT}, checks margins and bandwidths, and simulates a rated-torque load step for R4.

```
 import numpy as np

R, L, Kt, J = 0.085, 0.25e-3, 0.096, 1.5e-4   # ohm, H, N*m/A, kg*m^2
Ts_i, Ts_w = 1/20e3, 1/4e3                    # current and speed loop periods
f = np.logspace(0, 4.5, 20000); s = 2j*np.pi*f

def margins(Lf):
    i = np.argmin(abs(abs(Lf) - 1))           # gain crossover
    T = Lf / (1 + Lf)                          # closed loop
    bw = f[np.argmax(abs(T) < 1/np.sqrt(2))]
    return f[i], 180 + np.degrees(np.angle(Lf[i])), bw, abs(T).max()

# Current loop: PI zero cancels R/L, crossover 1.5 kHz, delay 1.5 Ts_i
wc_i = 2*np.pi*1500
Kp_i, Ki_i = L*wc_i, R*wc_i                  # V/A, V/(A*s)
Gi = (Kp_i + Ki_i/s) / (R + L*s) * np.exp(-1.5*Ts_i*s)
fc, pm, bw, pk = margins(Gi)
print(f"current: Kp={Kp_i:.2f} V/A Ki={Ki_i:.0f} V/As  fc={fc:.0f} Hz PM={pm:.1f} deg BW={bw:.0f}
Hz")
Ti_cl = Gi / (1 + Gi)                          # closed current loop

# Speed loop: symmetrical optimum with spacing a = 3
Teq = 1/wc_i + Ts_w/2 + Ts_w/2 + 1/(2*np.pi*2000)  # current lag, averaging, hold, filter
a = 3.0
wc_w = 1/(a*Teq); Kp_w = J*wc_w/Kt; Ti_w = a**2*Teq
H = 1/(1 + s/(2*np.pi*2000)) * np.exp(-Ts_w*s)     # speed filter, measurement + hold
Gw = Kp_w*(1 + 1/(Ti_w*s)) * Ti_cl * Kt/(J*s) * H
fc, pm, bw, pk = margins(Gw)
print(f"speed: Teq={Teq*1e6:.0f} us Kp={Kp_w:.3f} A/(rad/s) Ti={Ti_w*1e3:.2f} ms")
print(f"speed: fc={fc:.0f} Hz PM={pm:.1f} deg BW={bw:.0f} Hz peak={20*np.log10(pk):.1f} dB")

# Rated-torque load step (1.27 N*m): Euler simulation of the speed deviation
dt = 1e-6; w = xi = iq = 0.0; hist = []
for k in range(60000):                         # 60 ms
    xi += Kp_w/Ti_w*(-w)*dt                    # PI on speed error (reference 0)
    iq += (np.clip(Kp_w*(-w) + xi, -40, 40) - iq)*dt/Teq   # lumped lag Teq
    w += (Kt*iq - 1.273)/J*dt
    hist.append(w)
hist = np.array(hist)*60/(2*np.pi)             # rpm
t_rec = (np.nonzero(abs(hist) > 10)[0][-1] + 1)*dt
print(f"load step: dip {hist.min():.1f} rpm at {hist.argmin()*dt*1e3:.2f} ms, within 10 rpm after
{t_rec*1e3:.1f} ms")
```

```
 current: Kp=2.36 V/A Ki=801 V/As  fc=1500 Hz PM=49.5 deg BW=3602 Hz
speed: Teq=436 us Kp=1.195 A/(rad/s) Ti=3.92 ms
speed: fc=128 Hz PM=52.3 deg BW=245 Hz peak=2.3 dB
load step: dip -88.9 rpm at 2.11 ms, within 10 rpm after 8.0 ms
```

Worked example — what the numbers say. The current loop's closed-loop bandwidth (3.6 kHz) exceeds its crossover because the delay causes slight peaking. The speed loop crosses at 128 Hz with 52°

margin (R3), near the lumped estimate $1/(3 \times 436\ \mu\text{s}) = 765$ rad/s (122 Hz). A rated-torque step gives an 89 rpm dip and recovery within 10 rpm after 8 ms (R4). The dip is mostly physics: before the loop responds, the shaft decelerates at $1.273/1.49 \times 10^{-4} = 8500$ rad/s². Only less delay or more inertia reduces it; more gain without less delay buys only less margin.

> **Status of the principle: Symmetrical-optimum tuning of the speed loop**
> *Approximate*. Valid when the small lags lump into one T_{eq} and the mechanics are a rigid inertia; it fails near a belt or coupling resonance, when the current reference saturates, and when inertia changes by more than about a factor of two.

The belt resonance

The rigid-inertia assumption is what the first prototype violated. The belt separates the motor and its pulley (0.58 kg·cm²) from the 0.91 kg·cm² beyond it: a two-mass system with, seen from the motor, an antiresonance and a resonance $\sqrt{1 + 0.91/0.58} = 1.6$ times higher. A sine sweep injected into the current reference found them at 150 Hz and 240 Hz, the peak 14 dB above the rigid-body line. The loop gain at 240 Hz was about -6 dB on the model and $+8$ dB in reality, and it crossed 0 dB again near 274 Hz with under 4° of margin: a sustained ring. Lowering the gain to stop it pushed the crossover down to about 40 Hz and produced the overshooting index.

The fix is a biquad notch on the current reference at 240 Hz, −20 dB deep and about 0.4 octave wide (207–278 Hz at −3 dB). It costs about 10° at crossover, leaving 41° at 122 Hz; raising the spacing to $a = 3.2$ (crossover 117 Hz) restores 46°. Because the resonance moves with belt tension, commissioning repeats the sweep and re-centres the notch.

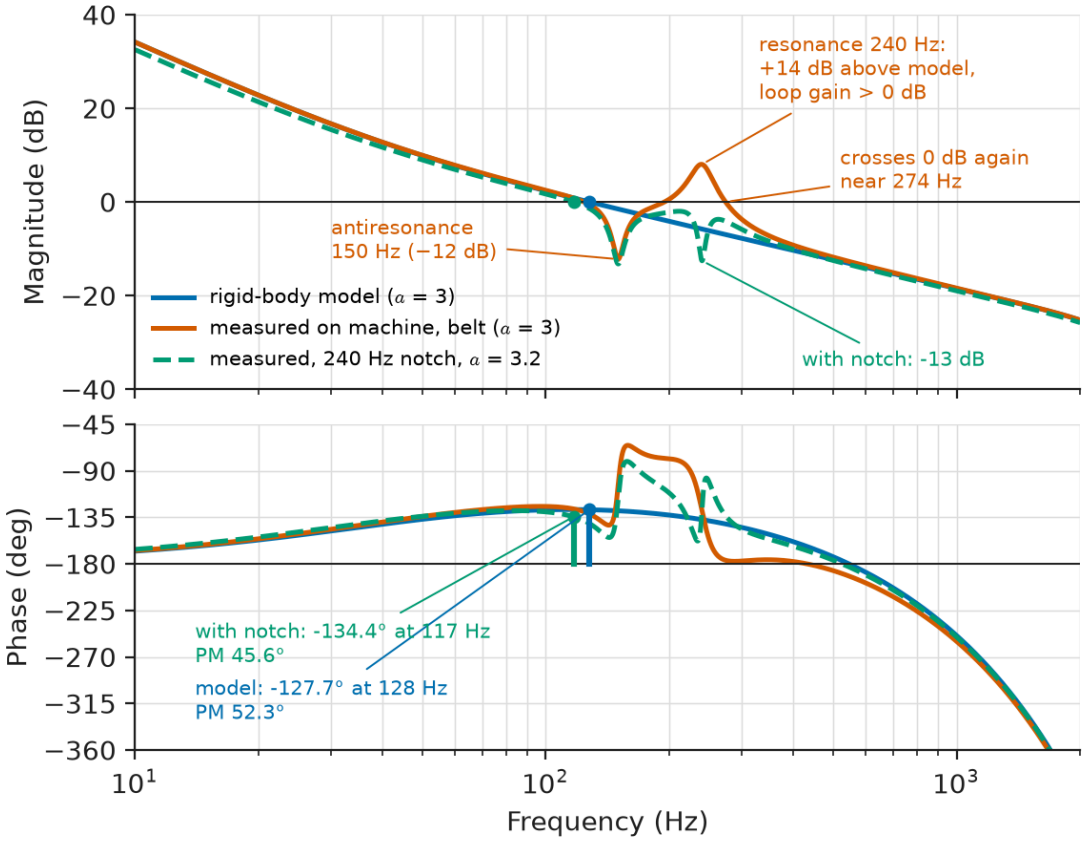


Figure 37.5 — Open-loop frequency response of the speed loop on the machine, rigid model against measured, with and without the notch filter.

37.6 Firmware

> **Boundary.**
> Firmware architecture, timing constraints and fault handling as a method are owned by Chapter 10; interrupt latency, scheduling and jitter by *Electronics and Computing*, Ch. 27 §27.2 and §27.10; space-vector PWM by Ch. 31 §31.3.

Structure and timing budget

The firmware is bare-metal and interrupt-driven. The ADC end-of-conversion interrupt runs the current loop at 20 kHz at top priority and, every fifth period, pends a lower-priority software interrupt for the speed loop, so speed computation can never delay the current loop. A 1 kHz interrupt runs supervision; the main loop handles CAN and parameters.

Table 37.8 — CPU timing budget, STM32G474 class at 170 MHz (worst case measured by GPIO toggle on an oscilloscope).

Task	Period	WCET	CPU load	Deadline	Margin
Current loop (ADC interrupt)	50 µs	5.8 µs	11.6 %	50 µs	88 %
Speed loop with notch	250 µs	2.1 µs	0.8 %	250 µs	99 %
Supervision	1 ms	4.0 µs	0.4 %	1 ms	99 %
CAN, parameters (main loop)	1 ms average	12 µs	1.2 %	10 ms response	—
Total			**14.0 %**	Budget ≤ 50 %	**72 %**

The 5.8 µs relies on the single-precision FPU; on the target the `sinf` / `cosf` calls map to the CORDIC coprocessor (about 0.2 µs). The listing shows the current loop: a PI with clamping anti-windup, then the interrupt, which transforms the three shunt currents with the encoder angle, runs both PIs inside the voltage circle (d-axis first), and writes duties with min–max zero-sequence injection, equivalent in range to space-vector modulation.

```
 #include <math.h>
#include <stdint.h>

#define PWM_ARR    4250.0f          /* 170 MHz, centre-aligned: 20 kHz */
#define CPR        10000u
#define POLE_PAIRS 4u
#define A_PER_LSB  0.0403f          /* 1 mOhm x 20 V/V, 3.3 V / 4096 */
#define TS         50e-6f

typedef struct { float kp, ki, integ, lim; } pi_t;

static float pi_run(pi_t *c, float err)
{
    float u = c->kp * err + c->integ;
    if (u > c->lim)       u = c->lim;          /* clamp; integrate only */
    else if (u < -c->lim) u = -c->lim;         /* when not saturated    */
    else                  c->integ += c->ki * err * TS;
    return u;
}

volatile float id_ref, iq_ref, vbus = 48.0f;
uint16_t off_a, off_b, off_c, enc_zero;        /* from calibration and alignment */
static pi_t pid_d = {2.36f, 801.0f, 0.0f, 0.0f}, pid_q = {2.36f, 801.0f, 0.0f, 0.0f};
```

```
 void ADC1_2_IRQHandler(void)                       /* injected conversions done */
{
    ADC1->ISR = ADC_ISR_JEOS;                        /* illustrative register names */
    float ia = -A_PER_LSB * (float)((int32_t)ADC1->JDR1 - off_a);  /* low-side: */
    float ib = -A_PER_LSB * (float)((int32_t)ADC2->JDR1 - off_b);  /* sign flip */
    float ic = -A_PER_LSB * (float)((int32_t)ADC1->JDR2 - off_c);
    float ial = (2.0f * ia - ib - ic) * (1.0f / 3.0f);             /* Clarke */
    float ibe = (ib - ic) * 0.57735027f;

    uint32_t e = ((TIM2->CNT + CPR - enc_zero) % CPR) * POLE_PAIRS % CPR;
    float th = (float)e * (6.2831853f / CPR), c = cosf(th), s = sinf(th);
    float id = ial * c + ibe * s, iq = -ial * s + ibe * c;          /* Park */

    float vmax = 0.95f * vbus * 0.57735027f;                        /* Vdc/sqrt3 */
    pid_d.lim = vmax;
    float vd = pi_run(&pid_d, id_ref - id);
    pid_q.lim = sqrtf(vmax * vmax - vd * vd);
    float vq = pi_run(&pid_q, iq_ref - iq);

    float val = vd * c - vq * s, vbe = vd * s + vq * c;             /* inv. Park */
    float va = val, vb = -0.5f * val + 0.8660254f * vbe, vc = -va - vb;
    float vo = -0.5f * (fmaxf(va, fmaxf(vb, vc)) + fminf(va, fminf(vb, vc)));
    TIM1->CCR1 = (uint32_t)(PWM_ARR * (0.5f + (va + vo) / vbus));
    TIM1->CCR2 = (uint32_t)(PWM_ARR * (0.5f + (vb + vo) / vbus));
    TIM1->CCR3 = (uint32_t)(PWM_ARR * (0.5f + (vc + vo) / vbus));
}
```

Two details are easy to miss: the low-side shunt sees the phase current with inverted sign, and the q-axis limit must be what the d-axis leaves. The listing omits one branch for brevity: when a phase's duty exceeds 93 %, its sample is replaced by minus the sum of the other two. The speed PI uses the same anti-windup function with its output limited to ±39.8 A.

State machine

The states are `INIT`, `OFFSET_CAL` (low-side switches on, 128 samples per phase, offsets within ±50 LSB of mid-scale), `ALIGN` (only without a stored alignment), `READY`, `RUN` and a latched `FAULT` reachable from every state and left only by a CAN reset once the cause is cleared. Entry to `RUN` checks bus voltage, offsets, encoder and STO. Faults leave the motor to coast; the brake chopper stays active in hardware in every state.

37.7 Protection

> **Boundary.**
> Drive protection functions and safety functions (STO, SS1) at system level are in *Electrical Energy and Machines*, Ch. 10 §10.7; functional safety as a method (IEC 61800-5-2, ISO 13849) is owned by Chapter 21, and industrial drive panels by Chapter 20.

Protection is layered by speed: microseconds in hardware, milliseconds in firmware, and a safety function independent of both (Table 37.9).

Table 37.9 — Protection layers.

Hazard	Detection	Response	Time	Layer
Phase-to-phase or phase-to-ground short	Current-amplifier output into on-chip comparator (threshold 60 A)	Timer break input forces all PWM outputs off	< 1 µs	Hardware
Shoot-through or MOSFET short	Gate driver V_{DS} monitor (0.6 V on a 2.5 mΩ device: 240 A)	Driver turns off all gates, nFAULT to MCU	< 5 µs	Hardware (driver)
Bus overvoltage, regeneration	Comparator on bus divider	Brake chopper on at 58 V, off at 56 V	< 2 µs	Hardware
Bus overvoltage, chopper failed	ADC, 20 kHz	Bridge off at 62 V; motor coasts	50 µs	Firmware
Bus undervoltage	ADC	Torque limited below 44 V, bridge off at 38 V	50 µs	Firmware
Encoder fault	Plausibility (37.4), index check, receiver open-wire detection	Bridge off, coast	≤ 0.75 ms	Firmware
Motor overheating	I²t model on i_q with 22 min time constant; NTC in Revision B	Torque limit reduced to continuous; fault at 110 % of model limit	Seconds	Firmware
Unexpected start during intervention	STO input: two channels remove the gate-driver supply and enable	No torque possible regardless of firmware	< 10 ms	Safety function

Worked example — will the comparator trip before the MOSFETs are damaged? A phase-to-phase short at the motor terminals is limited by the cable and MOSFETs only: about 30 mΩ for a 2 m cable plus 10 mΩ of switches, so the current rises at roughly V/L of the cable, about 48 V/1 μH = 48 A/µs. It reaches 60 A in about 1.25 µs after the fault appears, and the comparator plus break input remove the gates within 0.4 µs of that, so the peak is about 80 A, well inside the chosen MOSFET's 400 A pulsed rating. A firmware check could see it 50 µs later, at well over 1 kA. A short through a high-side switch to the motor frame bypasses the low-side shunts, which is why the driver's V_{DS} monitoring is the second layer. STO follows IEC 61800-5-2 with two channels (gate-supply switch and driver enable), each read back for diagnostics; its required integrity comes from the machine's ISO 13849 risk assessment (Chapter 21).

37.8 Performance testing

> **Boundary.**
> Test plans, instrument selection and regression testing are in Chapter 4; hardware-in-the-loop testing and field tests in Chapter 17. Drive commissioning practice is in *Electrical Energy and Machines*, Ch. 12 §12.9.

Test bench

On the dynamometer bench the motor drives, through a 5 N·m in-line torque transducer (0.1 %), a load motor run by an identical drive in torque mode; a power analyser measures DC input and motor-terminal power. On the machine the drive's own 20 kHz log, read over CAN, is the instrument. Loop measurements inject a 10 Hz–5 kHz sine sweep and compute loop gain from signals either side of the injection point.

Verification and test results

Table 37.10 — Verification and test results (Revision A hardware with the Revision A.1 firmware fixes).

Req.	Test	Limit	Predicted	Measured	Result
R1	Envelope at 44 V: rated torque at 3000 rpm; peak torque at 2500 rpm	Met for 10 s	91 % / 98 % of available voltage	Met; 93 % / 99 %	Pass
R2	Current-loop crossover and PM, q-axis, 1000 rpm	≥ 1.5 kHz, ≥ 45°	1.50 kHz, 49.5°	1.42 kHz, 50°	**Fail** on crossover by 5 %; 1.50 kHz, 49° after bus-measurement fix
R3	Speed-loop crossover and PM, on machine	≥ 100 Hz, ≥ 45°	117 Hz, 46° (with notch)	116 Hz, 46°	Pass
R4	1.27 N·m step at 1000 rpm	Dip ≤ 120 rpm, ±10 rpm in ≤ 15 ms	89 rpm, 8.0 ms	97 rpm, 8.9 ms	Pass
R5	Speed ripple at 30 rpm, 10 s	≤ ±1.5 rpm	—	±0.9 rpm (±2.6 rpm before offset recalibration)	Pass
R6	Phase-to-phase short at terminals	Off ≤ 2 µs	1.25 µs to 60 A + 0.4 µs	1.5 µs, peak 76 A	Pass
R6	Emergency stop at 3000 rpm, peak torque	Bus ≤ 62 V	58 V regulation	59.6 V peak	Pass
R6	Encoder A pair cut at 1000 rpm	Fault ≤ 1 ms	0.75 ms	0.5 ms (receiver open-wire), 0.75 ms (plausibility)	Pass
R7	Drive efficiency at 400 W shaft output	≥ 97 %	99.4 %	98.9 % ±0.3 %	Pass
—	Thermal: 30 min rated at 40 °C ambient	MOSFET case ≤ 100 °C	—	66 °C	Pass

The current-loop shortfall was not in the motor model. The duty computation divides the demanded voltage by the measured bus voltage, and the first build's bus divider read 5 % high (a 5 % resistor had been fitted where a 1 % part was specified), so every applied voltage, and with it the loop gain, was 5 % low: $1.5 \times 0.95 = 1.42$ kHz. With the correct resistor and an end-of-line bus-voltage calibration, the crossover was 1.50 kHz. The low-speed ripple of ±2.6 rpm before recalibration was the offset mechanism of 37.4: the shunt amplifiers' offsets had drifted with board temperature after the start-up calibration, so offsets are now recalibrated whenever the drive is in `READY` for more than 10 s.

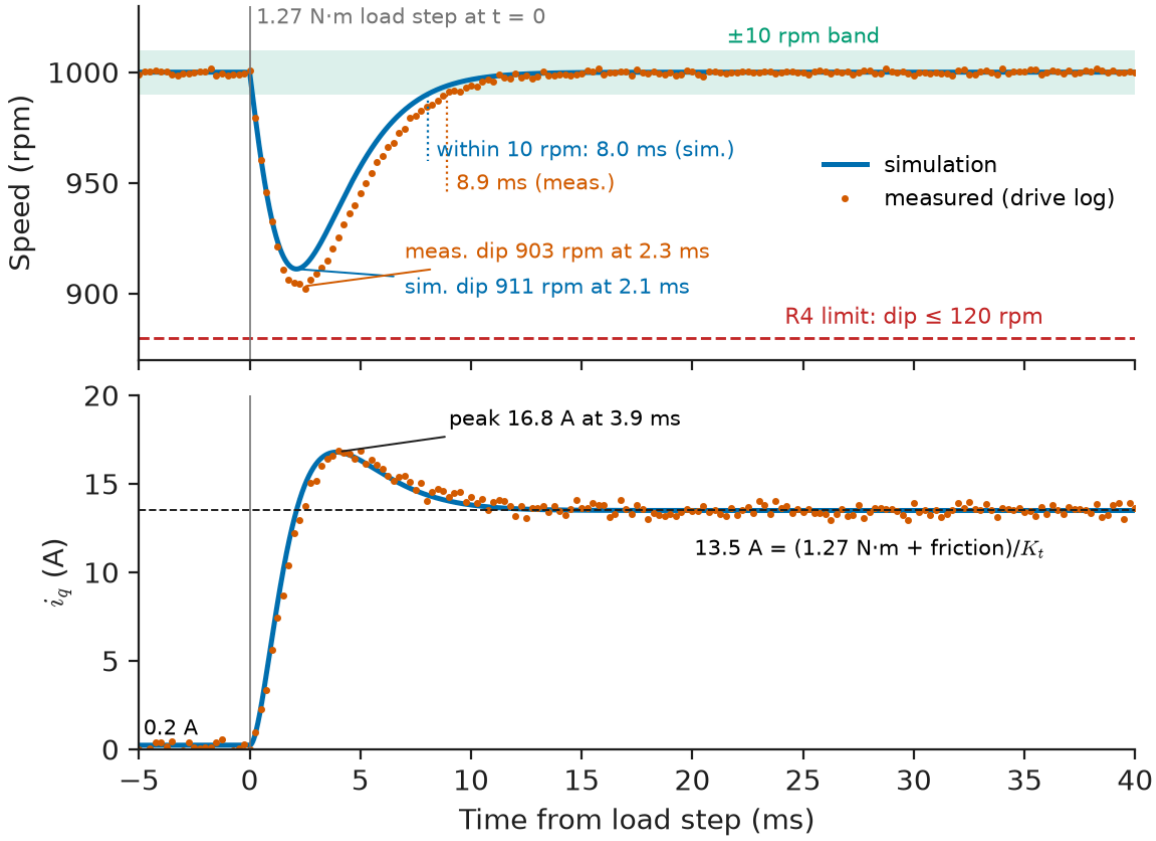


Figure 37.6 — Speed and torque-current response to a rated-torque load step at 1000 rpm, simulation against measurement.

Failure analysis

Table 37.11 — FMEA of the servo drive (S, O, D on 1–10 scales; RPN = S×O×D).

Failure mode	Effect	Cause	Detection	Mitigation	S	O	D	RPN
Bus overvoltage on regeneration	MOSFET and driver destruction	No energy sink with a unidirectional supply	Bus ADC; test of emergency stop at full speed	Hardware brake chopper; 100 V devices	9	2	2	36
Speed loop excites belt resonance	Oscillation, index overshoot, belt wear	Rigid-inertia tuning; resonance inside loop gain	Sweep on machine; oscillation monitor (energy at 150–400 Hz)	Notch filter; re-sweep at commissioning	6	4	3	72
Encoder counts gained or lost	Wrong commutation angle; torque loss or runaway	Noise via pigtail shield; damaged cable	Index check; plausibility	360° shield clamp; differential signalling	8	3	2	48
Current offset drift	Torque ripple at electrical frequency	Amplifier offset versus board temperature	Ripple at low-speed test	Recalibration in `READY`	4	5	4	80
Wrong alignment stored	Torque per amp reduced; field weakening or strengthening	Alignment done with belt under tension	Back-EMF zero-crossing check	Align with belt off; verify with dynamometer	6	3	4	72
Bootstrap undervoltage at high duty	High-side gate under-driven, MOSFET heats	Low-side off-time too short to recharge	Driver UVLO fault	93 % duty limit; 1 µF bootstrap capacitor	7	2	3	42
Motor overheated at duty above rating	Insulation ageing, demagnetisation risk	No winding sensor; RMS torque above rated	I^2t model only	Motor NTC in Revision B	8	3	5	120

The highest RPN is a detection gap: an I^2t model cannot see a blocked cooling path or a hot machine frame, so a motor NTC heads Revision B.

Bill of materials

Table 37.12 — Drive bill of materials with indicative cost (2026 distributor prices, 100 pieces, USD; motor and encoder excluded).

Item	Specification	Qty	Unit ($)	Total ($)
Microcontroller	Cortex-M4F, 170 MHz, FPU, CORDIC, 5 ADCs, comparators (STM32G474 class)	1	6.80	6.80
Gate driver	100 V three-phase, 3 current-sense amplifiers, V_{DS} monitoring (DRV8353 class)	1	5.50	5.50
MOSFETs	100 V, 2.5 mΩ, SuperSO8 class (6 bridge + 1 chopper)	7	1.40	9.80
Shunts	1 mΩ, 1 %, 3 W, 2512	3	0.40	1.20
DC-link capacitors	3 × 330 µF 80 V hybrid polymer, 4 × 4.7 µF 100 V X7R	1 set	3.40	3.40
Brake resistor	2.7 Ω, 50 W aluminium housed	1	6.50	6.50
Encoder receiver	Quad RS-422, open-wire detection	1	0.90	0.90
CAN transceiver, auxiliary supplies, connectors	5 Mbit/s; 48→12 V buck, 3.3 V LDO; 5 connectors	1 set	10.30	10.30
PCB, heat spreader, passives	4-layer 2 oz, 100 × 80 mm; aluminium spreader; TVS, ferrites	1 set	18.00	18.00
Enclosure	Aluminium extrusion with end caps	1	12.00	12.00
Assembly and end-of-line test	SMT, functional test with motor	1	15.00	15.00
Total				**89.40**

The bill meets R8 with 11 % margin; enclosure and assembly cost more than the power stage ($15.30).

Revision B

Change	Reason	Effect
NTC in the motor end winding, read through the encoder cable	Highest-RPN FMEA item	Thermal protection by measurement, not model
In-line phase shunts with PWM-rejecting amplifiers	Low-side sampling limits duty to 93 % and modulation to 95 %	About 5 % more voltage; rated torque at 3000 rpm down to about 38 V
Field weakening ($i_d < 0$) above the voltage limit	Line speed increase requested by the customer	Speed to about 3600 rpm at reduced torque
Automatic notch tuning from a short sweep at enable	Belt resonance moves with tension and wear	Margin kept without manual re-commissioning
Dead-time compensation from measured current sign	Residual low-speed torque ripple	Ripple at 30 rpm expected below ±0.5 rpm
Magnetic absolute encoder option with local line driver	Cost and power-up angle uncertainty	$15–20 cheaper motor; no U/V/W start-up
Bus-voltage measurement calibrated at end-of-line test; inductance re-measured at rated current during commissioning	R2 shortfall; possible saturation at peak current	Loop gain as designed on every unit

---

What remains open

Open question	Where it is answered
How are motor drivers, encoders and PID loops integrated into a robot joint, with buses and power distribution?	Chapter 15
How is a drive like this identified and tested in hardware-in-the-loop before it meets the machine?	Chapter 17
How are industrial VFDs, interlocks and emergency stops integrated into a control panel?	Chapter 20
How is the STO function validated to a performance level, and how does the drive talk to the PLC safely?	Chapter 21
How would a simpler brushed-motor drive with an H-bridge be designed for a small robot?	Chapter 34

Next: Chapter 38 leaves the motor and builds a battery-powered wireless sensor network, where the budgets that matter are microamps, decibels and packets.

Chapter 38 — Project: Wireless Sensor Network

Part IX · Interdisciplinary Engineering Projects

A vineyard tried twelve off-the-shelf LoRaWAN temperature nodes for frost warning in its first season. On the one night that mattered, with the air in the lowest corner of the vineyard at −1 °C, five of them went silent at 02:40. Three separate causes had lined up. When the vines leafed out in spring, the network server's adaptive data rate had moved the far nodes from spreading factor 9 to 12, multiplying their time on air by eight; at SF12 the 1 % duty-cycle limit of the 868 MHz band allows one 21-byte message every 148 s, so the stack silently dropped the one-minute "frost watch" reports. The lithium–thionyl-chloride cells, passivated after months of microamp sleep and now cold, sagged below brown-out under the transmit pulse, so the nodes reset and began rejoining at SF12. And nobody had written down what a node needed to deliver, how often, with what margin, on a cold night in full leaf. This chapter designs that network as a project: 32 battery-powered nodes measuring air temperature and humidity (eight also leaf wetness) across 40 ha, a time-slotted LoRa star with authenticated frames, a mains-powered gateway at the winery with LTE backhaul, and a frost alarm that reaches the vineyard manager within two minutes.

> **How are the sensors, radio, protocol and battery of a sensor node chosen together so that it delivers its data for five years on a cold night in full leaf?**
> **How is a network of such nodes made reliable and secure, and how is it deployed and proven in the field?**

Concept	Engineering destination
Sensor selection	Accuracy at 0 °C, humidity for wet-bulb temperature, radiation shielding, energy per reading
Node architecture	Radio SoC, clocks, power path and battery chemistry for cold pulses
Communication protocol	Slotted star on LoRa, spreading-factor assignment, group acknowledgement, duty-cycle compliance
Power budgeting	Charge per report and battery life as a function of spreading factor
Data acquisition	Synchronised sampling, timestamps, calibration offsets, buffering, frost logic
Network reliability	Link margin, retries, store-and-forward, gateway redundancy
Security	Authenticated encryption, nonce and counter management, replay protection, key provisioning
Deployment and testing	Site survey, Fresnel clearance, sensor calibration, field verification

Roadmap. Requirements, architecture and sensors (38.1) → node hardware (38.2) → protocol (38.3) → energy (38.4) → acquisition and frost logic (38.5) → reliability (38.6) → security (38.7) → deployment, verification, failure analysis, cost and Revision B (38.8).

38.1 Sensor selection

> **Boundary.**
> Sensor principles and selection are owned by Chapter 6, uncertainty and calibration by Chapter 7, IoT devices and backhaul by Chapter 11, and RF front ends, antennas, link budgets and WSN radios by Chapter 26. This chapter applies them to one network.

Requirements

Spring frost damages buds when their temperature falls below about −1 to −2 °C. The vineyard protects itself with four wind machines that mix warmer air from a temperature inversion down into the rows; they must be started before the coldest rows reach the threshold, and starting them costs fuel and noise, so false alarms matter too. Table 38.1 is the network's requirement set.

Table 38.1 — Requirements of the vineyard frost-monitoring network.

ID	Requirement	Target	Verification
R1	Measurement	Air temperature ±0.2 °C from −10 to +10 °C; RH ±2 %RH; leaf wetness on 8 nodes; at 1.5 m in the rows	Chamber comparison against reference
R2	Reporting	Every 5 min; every 1 min in frost watch (any node < 3 °C)	Gateway log
R3	Alarm latency	SMS within 2 min of any node crossing the alarm threshold	Injected cold event
R4	Delivery	≥ 99.9 % of readings within 2 min during frost watch; ≥ 99.5 % within 24 h overall	30-day field log, frost nights
R5	Coverage	All nodes up to 850 m from the gateway, measured margin ≥ 12 dB at SF ≤ 9 in full leaf	Site survey
R6	Battery life	≥ 5 years on the fitted cells, including 40 frost-watch nights a year	Measured charge per report; budget
R7	Security	Frames authenticated and encrypted; no replay; per-node keys	Injected replay and forged frames
R8	Cost and environment	Node BOM ≤ $45, gateway ≤ $250 (100 nodes); IP65; −20 to +50 °C	Costed BOM; chamber test

Architecture

Each node (Figure 38.1) sits on a post at 1.5 m in a vine row: a multi-plate radiation shield housing the temperature–humidity sensor, an IP65 box with the radio SoC and batteries, and a quarter-wave whip antenna 2.2 m above ground, just clear of the canopy. Nodes transmit in assigned slots to a gateway on the winery roof (antenna 8 m above ground), which acknowledges them in a group beacon, computes wet-bulb temperature, runs the frost logic, stores the data, sends SMS alarms and publishes to the vineyard's cloud dashboard over LTE. The gateway runs from mains with a 12-hour battery backup.

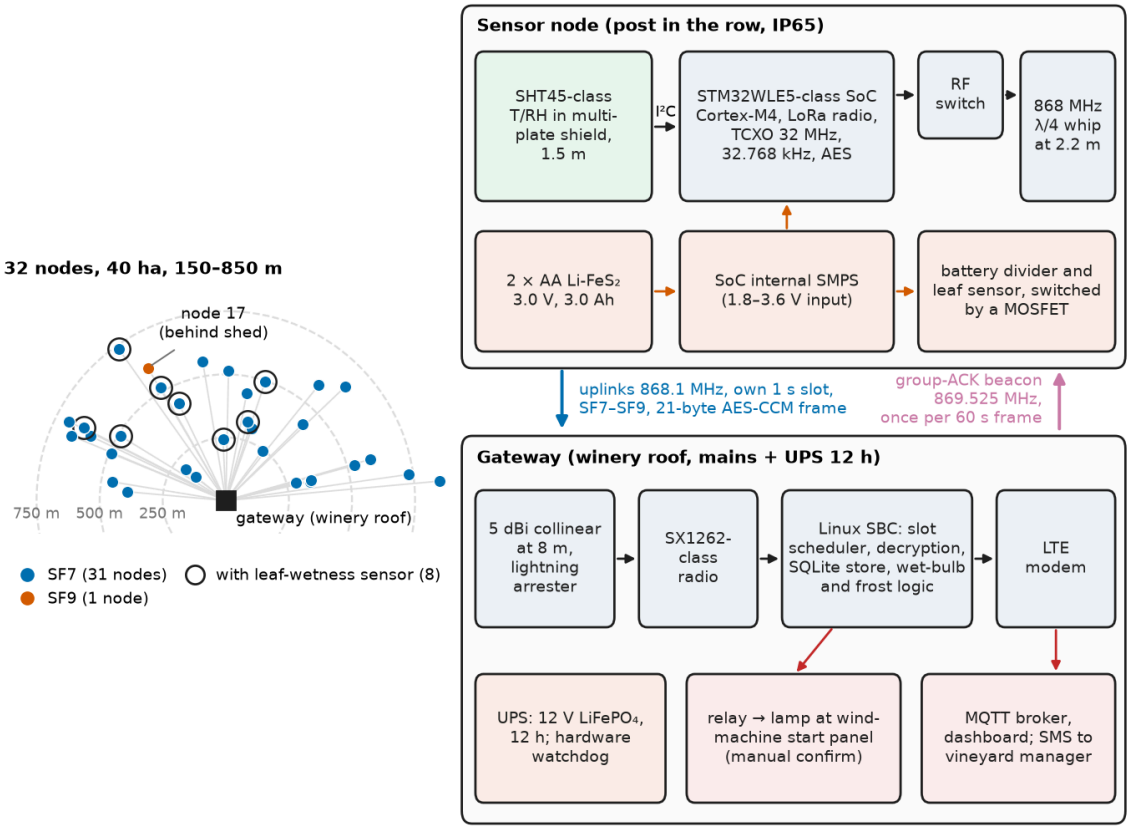


Figure 38.1 — Architecture of the 32-node vineyard frost-monitoring network.

Choosing the sensor

The quantity that decides frost damage is bud temperature, which on a clear, still night runs 1–2 °C below air temperature because the buds radiate to the sky; the network measures air temperature and humidity in a shield, and the grower's threshold is set with that offset in mind. Humidity matters because wet-bulb temperature, not dry-bulb, decides whether sprinkler protection would help, and because in dry air wet buds and leaves cool by evaporation towards the wet-bulb temperature, below the dry-bulb reading. Table 38.2 compares the candidates.

Table 38.2 — Temperature and humidity sensor options (typical datasheet values; 2026 prices at 100 pieces).

Sensor	Temperature accuracy	Humidity	Energy per reading	Interface	Price	Verdict
Digital T/RH (SHT45 class)	±0.1 °C typ, ±0.2 °C max (0–60 °C)	±1.0 %RH	about 8 ms at 0.5 mA (high repeatability)	I²C	$2.80	**Chosen**
Digital T (TMP117 class)	±0.1 °C (−20 to 50 °C)	None	15.5 ms at 135 µA	I²C	$2.20	No humidity
PT1000 class A + 24-bit ADC	±0.15 °C + 0.002·T, with calibration ±0.05 °C	None	50–100 ms at 0.5–1 mA (excitation, settling)	SPI	$6–9	Overkill and more energy
Combined environmental sensor (BME280 class)	±0.5 to ±1 °C	±3 %RH	10 ms at 0.7 mA	I²C	$3.50	Fails R1

The SHT45-class sensor meets R1 with a calibration check (38.8), gives humidity in the same package, and costs about 4 µC per reading, negligible against the radio. Its weakness is condensation: at 100 %RH on a frost night the sensing surface can wet, reading 100 %RH for hours after the air has dried. The chosen part has an on-chip heater, and the firmware runs it for 1 s at 20 mW once an hour when RH has stayed above 95 % for that hour, discarding the next two readings while the sensor cools.

The radiation shield matters as much as the sensor. Unshielded, a sensor in sunlight reads several degrees high and at night a few tenths low; a naturally ventilated multi-plate shield keeps the night-time error to about 0.1–0.2 °C in light wind, which is what R1's budget allows. Heat transfer in the shield (radiation exchange with a clear sky, convection in still air) is in *Engineering Physics*, Ch. 14.

The leaf-wetness sensor on eight nodes is a dielectric leaf-shaped probe, powered at 3 V for 10 ms at 2 mA and read by the SoC's ADC. It costs about 20 µC per reading and is the only analogue input.

38.2 Node architecture

> **Boundary.**
> MCU selection, power modes and the average-current budget are in *Electronics and Computing*, Ch. 25 §25.3 and §25.5; low-power firmware practice and logging in Ch. 28 §28.5–28.6; clock accuracy and drift in Ch. 27 §27.4. Embedded integration as a method is Chapter 10; battery chemistry is in *Materials Science and Engineering*, Ch. 40.

Radio SoC

Table 38.3 — Node processor and radio options.

Option	Sleep with RTC	TX +14 dBm	RX	Notes	Verdict
Sub-GHz LoRa SoC (STM32WLE5 class) in a certified module with TCXO	about 1.5 µA (with RAM retention)	about 45 mA incl. MCU	about 6 mA incl. MCU	One chip; hardware AES; module carries radio certification	**Chosen**
Separate MCU (STM32L0 class) + SX1262 transceiver	1.0 µA + 0.2 µA	45 mA	5.5 mA	Two chips and an SPI link; more layout and certification work	Equivalent energy, more cost
2.4 GHz SoC (nRF52840 class), 802.15.4 mesh	2–3 µA	15 mA at +8 dBm	6 mA	2.4 GHz suffers 10–20 dB more foliage loss; routers must stay awake	Rejected: coverage and energy
Cellular LPWA module (LTE-M/NB-IoT)	3–10 µA (PSM)	100–250 mA pulses	—	No gateway, but subscription per node and patchy rural coverage	Rejected: cost and coverage

The module's 32 MHz TCXO keeps the radio carrier within ±2 ppm (±1.7 kHz at 868 MHz) at −20 °C, far inside LoRa's frequency-offset tolerance, which a plain crystal at −10 °C would approach. The 32.768 kHz crystal for the real-time clock is a tuning-fork type, whose frequency falls parabolically with temperature away from 25 °C, at about −0.034 ppm/°C²; that sets the timing budget of 38.3.

Power path and battery

The node has no regulator of its own: the SoC's internal step-down converter runs the core and radio from 1.8–3.6 V directly. A MOSFET switches the battery-voltage divider and the leaf-wetness sensor, so neither draws current in sleep; all unused pins are analogue inputs. Measured sleep current of the assembled node is 1.6 µA at 20 °C, rising to about 2.2 µA at 45 °C.

The battery choice was where the pilot failed, so Table 38.4 compares chemistries for this duty: microamp sleep for years, 45 mA pulses of up to 0.2 s, at −10 °C.

Table 38.4 — Battery options for the node.

Option	Capacity (low drain)	Voltage	45 mA pulse at −10 °C	Self-discharge	Price	Verdict
1 × AA Li-$SOCl_2$, bobbin	2.6 Ah	3.6 V	Voltage delay after passivation; can sag below 2.5 V	< 1 %/yr	$3.50	Failed in pilot
1 × AA Li-$SOCl_2$ + hybrid layer capacitor	2.6 Ah	3.6 V	Good (capacitor supplies pulse)	< 1 %/yr	$9.50	Good, costly
2 × AA Li-FeS_2 in series	3.0 Ah	3.0–3.6 V	Good (spiral construction, low resistance)	About 0.5 %/yr	$3.20	**Chosen**
2 × AA alkaline	2.5 Ah at 20 °C, much less cold	3.0 V	Poor below 0 °C	2–3 %/yr	$1.00	Rejected

The lithium–iron-disulphide pair delivers the transmit pulse at −20 °C with a drop of under 0.2 V, has no passivation layer to break, and its end-of-life voltage (about 2.0 V for the pair) is still above the SoC's 1.8 V minimum. It holds slightly less energy than the thionyl-chloride cell at the same capacity because its voltage is lower, which the budget of 38.4 accounts for by derating capacity to 85 %.

> **Status of the principle: Primary-cell capacity derating for cold, pulsed loads**
> *Empirical.* Usable capacity at low temperature and with pulse loads is taken from manufacturers' discharge curves and test data, not from the nameplate; the 85 % factor used here applies to Li-FeS_2 at −10 to +40 °C with 45 mA pulses and must be re-derived for another chemistry or duty.

38.3 Communication protocol

> **Boundary.**
> Path loss, shadowing, fading and fade margin are in *Circuits, Signals and Communications*, Ch. 30 §30.2, §30.4 and §30.10; antennas and Friis transmission in Ch. 32 §32.7 and §32.9; error detection and ARQ in Ch. 28 §28.6. MQTT and HTTP are owned by Chapter 11, link budgets and WSN radios by Chapter 26.

Choice of protocol

Table 38.5 — Protocol options on the chosen radio.

Option	Medium access	Acknowledgement	Infrastructure	Strength	Weakness
LoRaWAN Class A, ADR on	Unslotted ALOHA	Optional confirmed uplinks, one downlink per uplink	Gateway + network server	Standard, interoperable, security specified	ADR drove the pilot to SF12; per-node ACKs cost gateway duty cycle
LoRaWAN Class A, fixed SF per node	Unslotted ALOHA	Unconfirmed + application-level gap detection	Gateway + network server	Standard	No delivery guarantee during frost watch
Time-slotted star on LoRa PHY (custom MAC)	Assigned slots + shared retry slots	One group-ACK beacon per frame	Gateway only	Collision-free, bounded latency, one downlink per minute for all nodes	Own MAC and security to verify
802.15.4 mesh at 868 MHz (Wi-SUN class)	CSMA, routed	Per hop	Border router; mains-powered routers	Range by hopping	Routers in a vineyard need power

A fixed-SF LoRaWAN deployment would meet most of the requirements and is the right choice when the network must coexist with other LoRaWAN devices or a public network. This vineyard has one gateway, one application and a hard latency requirement on cold nights, and a slotted star with a group acknowledgement meets R3 and R4 with the least energy and the least downlink airtime. The MAC is small (about 2 kB of code at the node), it uses the standard LoRa physical layer, and its security (38.7) uses a standard primitive, AES-128-CCM, rather than anything invented.

Frame structure

Time is divided into 60 s frames (Figure 38.2). Each of the 32 nodes owns a 1.0 s uplink slot; 16 shared slots follow for retransmissions; a 2 s window carries the gateway's group-acknowledgement beacon; the remaining 10 s is spare for growth to 42 nodes. In normal mode a node transmits in its slot in every fifth frame (5 min); in frost watch, in every frame. The beacon carries the frame number, gateway time, the network mode (normal or frost watch), a 32-bit bitmap of the slots received correctly in this frame and a message authentication code. Uplinks use 868.1 MHz; the beacon uses 869.525 MHz.

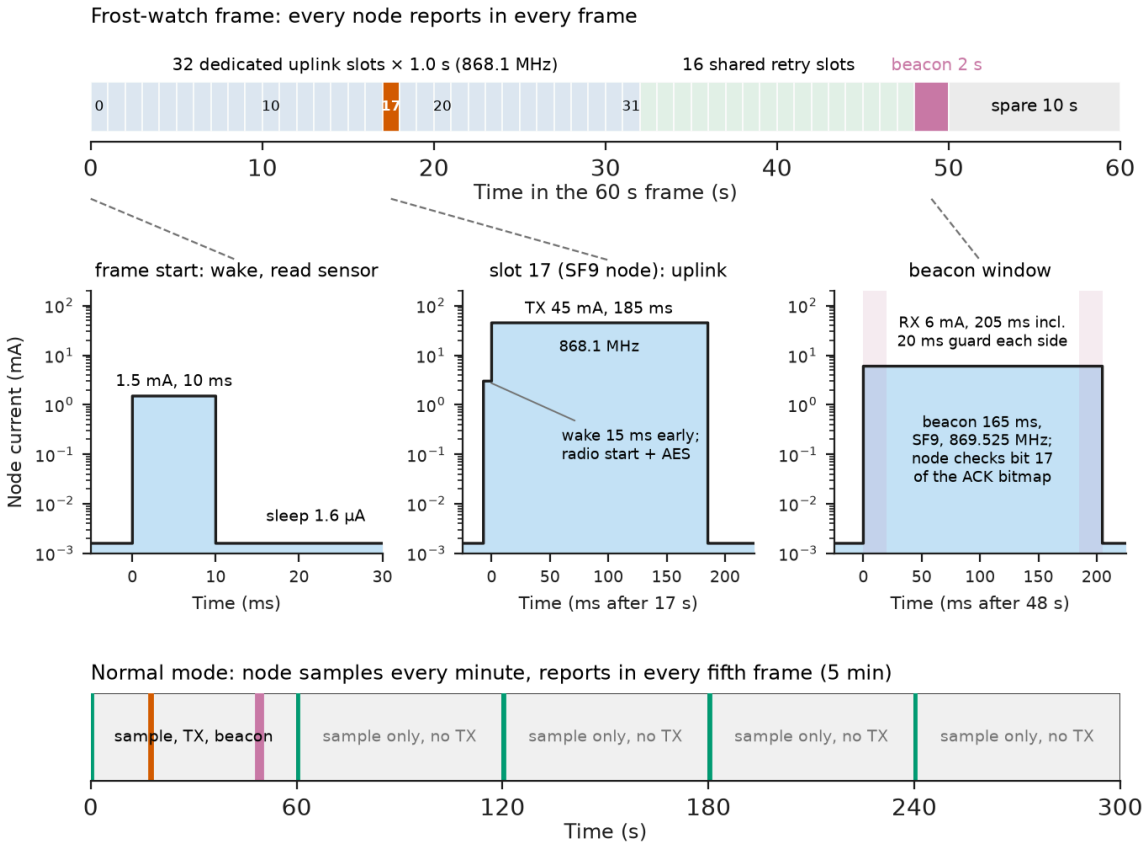


Figure 38.2 — Structure of the 60 s frame and the timing of one node's report.

Spreading factor and duty cycle

Each node's spreading factor is fixed at commissioning from its measured link margin (38.8), not adapted on the fly. Two rules bound the choice. ETSI EN 300 220 limits a device in the 868.0–868.6 MHz sub-band to 1 % duty cycle: a 21-byte uplink every minute is legal up to SF10 (0.62 %) but not at SF11 (1.24 %), which is exactly the limit the pilot's ADR crossed. And the energy budget of 38.4 limits nodes to SF9 for five-year life. The beacon, 165 ms once a minute, uses 0.28 % of the gateway's allowance in the 869.4–869.65 MHz sub-band, whose limit is 10 %; per-node acknowledgements at SF9 would have used 32 × 144 ms per minute, 7.7 %, uncomfortably close to it.

Timing and drift

A node must wake for the beacon at the right moment, and its clock is a cold tuning-fork crystal. Worst-case error is the ±20 ppm tolerance plus the temperature term, $-0.034 \times (-10 - 25)^2 = -41.7$ ppm at −10 °C: about 62 ppm. Over a normal-mode 300 s between beacons that is 18.6 ms, so the node opens its receiver 20 ms early and keeps it open up to 20 ms late (the window counted in the energy budget as 40 ms). Every beacon received resets the clock error. If a beacon is missed, the window widens in proportion to the time since the last one (62 ppm of elapsed time, plus 2 ms); after 30 min without a beacon the node listens for a whole frame to resynchronise, which costs about 0.36 C, equal to about 40 reports and acceptable only because it is rare.

Status of the principle: Guard time from worst-case clock drift
Derived. The timing uncertainty is the product of the worst-case fractional frequency error and the time since the last synchronisation; it holds as long as the error bound covers the actual crystal over the temperature range, which is why the bound is taken from the crystal's parabolic temperature coefficient at the coldest site temperature, not from its 25 °C tolerance.

38.4 Power budgeting

Boundary.
The energy budget as a design tool and LoRa time-on-air are developed in *Electronics and Computing*, Ch. 28 §28.5. Here the budget is closed against the link budget and the duty-cycle rule.

Link budget

The site survey (38.8) fitted a log-distance model to measurements in full leaf: path loss $PL(d) = 85 + 33\log_{10}(d/100\ \text{m})$ dB, exponent 3.3, with 6 dB log-normal shadowing about the fit. With +14 dBm at the node, a 0 dBi effective node antenna (whip near the post and canopy), a 5 dBi gateway antenna and 2 dB of cable and connector loss, the received power at 850 m is $14 + 0 + 5 - 2 - 115.7 = -98.7$ dBm. LoRa sensitivity at 125 kHz is about −124 dBm at SF7, −129 dBm at SF9 and −132 dBm at SF10 (SX126x class), so the median margin at the farthest node is 25 dB even at SF7. A 99 % location-reliability allowance for 6 dB shadowing ($2.33\sigma = 14$ dB) still leaves 11 dB at SF7. The model only plans the network; each node's spreading factor is set from its own *measured* margin (38.8).

Table 38.6 — Planning link budget for the farthest node (850 m, SF9, uplink).

Item	Value	Requirement	Margin
Transmit power, node (EIRP with 0 dBi)	+14 dBm	≤ 25 mW ERP (EN 300 220)	2.2 dB below limit
Gateway antenna gain, cable and connector loss	+5 dBi, −2 dB	—	—
Path loss, survey fit	115.7 dB	—	—
Received power	−98.7 dBm	—	—
Sensitivity, SF9 at 125 kHz	−129 dBm	—	—
Median margin	30.3 dB	—	—
Shadowing allowance, 99 % of locations (2.33 × 6 dB)	14 dB	—	—
Planning margin after shadowing	16.3 dB	≥ 12 dB measured	4.3 dB

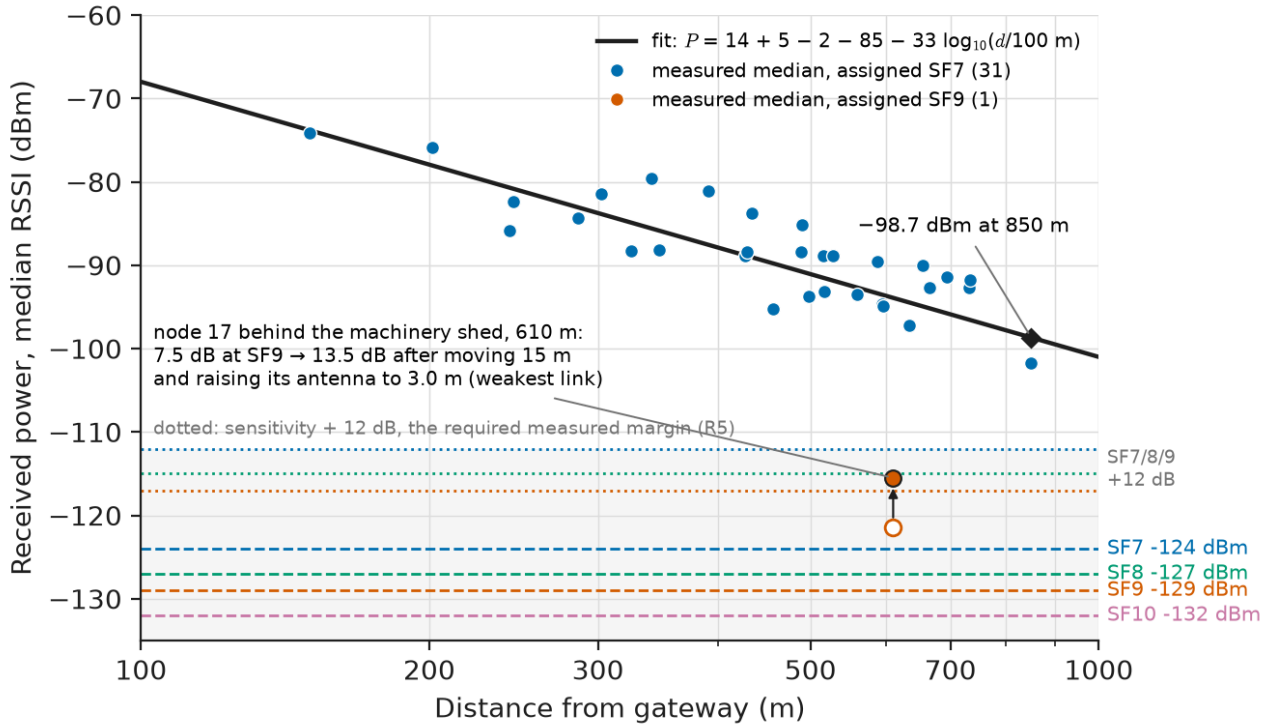


Figure 38.3 — Measured received power at the 32 node positions against distance, with the fitted path-loss model and receiver sensitivities.

Charge per report and battery life

The listing combines the time-on-air formula, the link budget and the charge of each phase of a report (wake and sensor read, radio start, AES, transmit, beacon receive with its 40 ms window), adds 1.6 µA of sleep, and computes battery life per spreading factor for the 124,320 reports a year that R2 implies with 40 frost-watch nights of 10 hours.

```
import math

def toa(n_bytes, sf, bw=125e3, cr=1, n_pre=8):
    """LoRa time on air (s), explicit header, CRC on (Semtech formula)."""
    de = 1 if sf >= 11 else 0
    ts = 2**sf / bw
    n = 8 + max(math.ceil((8*n_bytes - 4*sf + 44) / (4*(sf - 2*de)))*(cr + 4), 0)
    return (n_pre + 4.25 + n) * ts

SENS = {7: -124.0, 8: -127.0, 9: -129.0, 10: -132.0}       # dBm at 125 kHz
PL = lambda d: 85 + 33*math.log10(d/100)                    # site-survey fit, dB
p_rx = 14 + 0 + 5 - 2 - PL(850)                             # dBm at the farthest node
t_bcn = toa(16, 9)                                          # group-ACK beacon, SF9
reports = 288*365 + 40*10*60*(1 - 1/5)                      # per year, incl. frost watch
cap = 3000*0.85                                             # mAh, 2 x AA Li-FeS2, derated

print(f"P_rx at 850 m: {p_rx:.1f} dBm; beacon {t_bcn*1e3:.0f} ms; reports/yr {reports:.0f}")
for sf in SENS:
    t_up = toa(21, sf)
    q = (10e-3*1.5e-3 + 5e-3*3e-3 + 2e-3*3e-3               # wake+sensor, radio start, AES
         + t_up*45e-3 + (t_bcn + 0.040)*6.0e-3) * 1e3       # TX, beacon RX with guards; mC
    mah = reports*q/3600 + 1.6e-3*24*365                    # + 1.6 uA sleep; mAh/yr
    life = cap / (mah + 0.005*3000)                         # + 0.5 %/yr self-discharge
    print(f"SF{sf:2d}: uplink {t_up*1e3:5.1f} ms, margin {p_rx-SENS[sf]:4.1f} dB, "
          f"duty(1/min) {t_up/60*100:4.2f} %, {q:5.2f} mC/report, "
          f"{mah:4.0f} mAh/yr, life {life:4.1f} yr")
```

```
P_rx at 850 m: -98.7 dBm; beacon 165 ms; reports/yr 124320
SF 7: uplink  56.6 ms, margin 25.3 dB, duty(1/min) 0.09 %,  3.81 mC/report,  146 mAh/yr, life 15.9
yr
SF 8: uplink 102.9 ms, margin 28.3 dB, duty(1/min) 0.17 %,  5.90 mC/report,  218 mAh/yr, life 11.0
yr
SF 9: uplink 185.3 ms, margin 30.3 dB, duty(1/min) 0.31 %,  9.61 mC/report,  346 mAh/yr, life  7.1
yr
SF10: uplink 370.7 ms, margin 33.3 dB, duty(1/min) 0.62 %, 17.95 mC/report,  634 mAh/yr, life  3.9
yr
```

Worked example — what limits the node. At SF9 a report costs 9.61 mC, of which the transmission is $0.1853 \times 45 = 8.34$ mC (87 %), the beacon reception $0.205 \times 6 = 1.23$ mC (13 %), and the sensor, radio start and encryption together 0.036 mC. Sleep at 1.6 µA is 14 mAh a year, 4 % of the total. The node is a radio with a sensor attached: the only large lever is time on air. Each step of spreading factor roughly doubles it: SF10 gives 3.9 years and fails R6, which, together with the duty-cycle rule, is why no node is allowed above SF9. At SF7 the same cells last 15.9 years, longer than the cells' shelf life, so near nodes could use a single cell. The average current of an SF9 node is $346/8760 = 39.5$ µA.

Table 38.7 — Energy budget of an SF9 node (the worst assigned), per year.

Item	Basis	Charge (mAh/yr)	Share
Uplinks	124,320 × 185.3 ms × 45 mA	288	83 %
Beacon receptions	124,320 × 205 ms × 6 mA	42	12 %
Sensor, radio start, AES	124,320 × 0.036 mC	1.2	< 1 %
Sleep	1.6 µA × 8760 h	14	4 %
Self-discharge	0.5 % of 3000 mAh	15	—
Total drawn		**361**	
Usable capacity	3000 mAh × 0.85	2550	
Life	2550 / 361	**7.1 years**	Margin over R6: 41 %

> **Status of the principle: Battery life from charge per event**
> *Derived.* Life is usable capacity divided by the sum of event charges, sleep charge and self-discharge per year; it assumes the measured currents and durations hold over temperature and battery voltage, which the field measurement of 38.8 checks, and it ignores retransmissions, which the reliability analysis of 38.6 shows add under 1 %.

38.5 Data acquisition

> **Boundary.**
> DAQ architecture, synchronisation and logging are owned by Chapter 8; event detection and practical filtering by Chapter 27.

Sampling and the payload

All nodes sample at the start of each frame, so that a map of the vineyard is a snapshot taken within the clock error of 38.3 (tens of milliseconds), not a smear over five minutes. In normal mode each node samples every minute but transmits only every fifth: it keeps the minimum of the five readings and the last, so that a short cold dip between reports is not lost, and switches itself to frost watch the moment its own reading falls below 3 °C, without waiting for the gateway. The 10-byte payload is fixed-point: temperature (0.01 °C, signed 16-bit), humidity (0.01 %RH, 16-bit), leaf wetness (8-bit), battery voltage (20 mV steps), flags, the RSSI of the last beacon, the number of buffered readings and one reserved byte. With the 7-byte header and 4-byte authentication tag the frame is 21 bytes.

A node that misses its acknowledgement keeps the reading in a 24-hour ring buffer in RAM (1440 readings of 6 bytes, 8.6 kB) and appends up to three buffered readings to its next transmission. After a gateway outage, nodes drain their buffers three readings at a time, in their own slots and the shared retry slots.

At the gateway

The gateway decrypts and authenticates each frame (38.7), applies the node's stored calibration offsets (38.8), timestamps the reading with the frame time, discards duplicates by frame counter, and writes it to a local SQLite database before publishing it. It derives wet-bulb temperature from dry-bulb and humidity with Stull's empirical formula, which agrees with the psychrometric equations within about 1 °C over −20 to 50 °C and 5–99 %RH; at 2.0 °C and 60 %RH it gives −1.2 °C, at 2.0 °C and 90 %RH, +1.0 °C.

> **Status of the principle: Stull's wet-bulb approximation**
> *Empirical.* A fitted function of temperature and relative humidity at sea-level pressure; adequate for frost logic with the threshold set conservatively, but not a substitute for the psychrometric equations at altitude or where 0.1 °C matters.

The frost logic is deliberately plain: an alarm when any node's air temperature is at or below the grower's threshold (default 0.5 °C) in two consecutive readings, or its wet-bulb temperature is at or below −1.0 °C; it clears when all nodes have been above threshold + 1 °C for 15 minutes. Two consecutive readings at one-minute spacing cost at most one minute of latency and remove single-reading glitches (a sensor briefly wet with dew). The gateway sends an SMS, publishes an alarm message and lights a lamp at the wind-machine panel; starting the machines stays a human decision.

```
{
  "topic": "vineyard/north/node/17/reading",
  "ts": "2026-04-12T02:41:00Z",
  "t_c": 0.38, "rh_pct": 91.2, "tw_c": -0.46, "leaf_wet": 0,
  "batt_v": 3.08, "rssi_dbm": -115, "snr_db": 1.5, "sf": 9,
  "fcnt": 48213, "late": false, "mode": "frost_watch"
}
```

The `late` flag marks a reading delivered from a node's buffer rather than in its own frame, so that dashboards and the frost logic do not mistake an old reading for a new one.

38.6 Network reliability

> **Boundary.**
> Outage probability and fade margin in fading channels are in *Circuits, Signals and Communications*, Ch. 30 §30.4; communication reliability and interference mitigation at system level are in Chapter 26.

Where packets are lost

Three mechanisms lose packets in this network. Slow fading: leaves moving in wind, a tractor parked across a row, wet foliage after rain; its effect is a time-varying excess loss of a few decibels around each node's static margin. Interference: other 868 MHz users nearby (a neighbour's LoRaWAN, alarm systems) occasionally overlapping a slot; LoRa's processing gain rejects much of it, but not all. And infrastructure: gateway power, the gateway's radio, the LTE link. The first two are random and per-packet; the third is correlated and takes the whole network down.

Worked example — delivery during frost watch. The worst assigned node has a measured margin of 13.5 dB; take R5's 12 dB as the bound. Model the time-varying fading as Gaussian in decibels with $\sigma = 4$ dB, as measured over a windy week; the probability that one uplink fades below sensitivity is $Q(12/4) = Q(3) = 0.13$ %. Add 0.5 % per slot for interference, from a week of spectrum monitoring at the gateway. One attempt fails with probability 0.63 %; the reading is late only if the attempt in the next frame also fails, $0.0063^2 = 4 \times 10^{-5}$. So 99.4 % of readings arrive in their own minute and 99.996 % within two minutes, against R4's 99.9 %. The beacon has the same loss probability; a missed beacon causes an unnecessary retransmission (which the gateway discards as a duplicate), adding about 0.6 % to the node's transmissions and well under 1 % to its energy.

> **Status of the principle: Independent per-attempt losses and retry**
> *Approximate.* Treats each attempt as independent with a fixed loss probability; valid when the fading coherence time (seconds for leaves in wind) is shorter than the one-minute retry interval, and invalid for slow, correlated outages (a vehicle parked for an hour, a gateway failure), which are handled by buffering and redundancy instead of retries.

Infrastructure

The gateway is the single point of failure, so it is engineered for the night it is needed: a 12 V $LiFePO_4$ battery with charger carries it for 12 hours of mains loss; a hardware watchdog restarts it; a lightning arrester and earthed mast protect the antenna run. If LTE fails, the gateway still raises the local alarm lamp and sounder and logs locally; SMS uses the same modem, so a second, independent path (a cheap modem on another operator) is a Revision B item. If the gateway itself fails, nodes keep sampling and buffering for 24 hours, and the monitoring service raises an alarm when the gateway's heartbeat stops for 10 minutes.

Monitoring the network itself

The gateway tracks, per node, the fraction of slots received, RSSI and SNR, battery voltage under load and the number of late readings, and flags a node whose delivery falls below 99 % over 24 hours or whose margin drops by more than 6 dB from its commissioning value. These health indicators turned up both the tractor parking spot and, in the first summer, a node whose antenna had been bent flat by a mowing crew.

38.7 Security

> **Boundary.**
> Device authentication, provisioning and IoT security practice are owned by Chapter 11, and industrial cybersecurity (IEC 62443) by Chapter 21.

Threat model

The data are not secret; the decisions made from them are valuable. An attacker within radio range (the vineyard borders a public road) could forge readings to suppress a real frost alarm or to trigger false ones that start the wind machines at 3 a.m.; replay a recorded cold-night frame; forge beacons to mark everything as received or to desynchronise the nodes; or jam the channel. Cryptography answers the first three.

Jamming cannot be prevented, only detected: the gateway treats loss of more than a quarter of the nodes in one frame, with high noise in the band, as an alarm in itself.

Authenticated encryption and counters

Every uplink and beacon is protected with AES-128 in CCM mode, with a 4-byte authentication tag: the header (type, node identifier, frame counter) is authenticated in the clear, the payload encrypted. The nonce is built from the direction, the node identifier and a 32-bit frame counter, so no two frames under one key ever share a nonce. The node increments its counter for every frame and the gateway rejects any counter not greater than the last one accepted from that node, which defeats replay. A 4-byte tag gives a forger a 2^{-32} chance per attempt; at the duty-cycle-limited rate of at most a few thousand attempts a day, the expected time to one accepted forgery is over a thousand years, and the gateway alarms on the first few failures anyway.

```
 #include <stddef.h>
#include <stdint.h>
#include <string.h>

/* AES-128-CCM from a vetted library, using the SoC's AES engine; 0 on success. */
int aes_ccm_encrypt(const uint8_t key[16], const uint8_t nonce[13],
                    const uint8_t *aad, size_t aad_len,
                    const uint8_t *pt, size_t pt_len,
                    uint8_t *ct, uint8_t *tag, size_t tag_len);

#define HDR_LEN 7u                    /* type 1, node id 2, frame counter 4 */
#define MIC_LEN 4u

typedef struct { uint16_t id; uint32_t fcnt; uint8_t key[16]; } node_sec_t;

/* Build one uplink: header authenticated in clear, payload encrypted, 4-byte tag.
   Returns the frame length, or 0 if no frame may be sent. */
size_t build_uplink(node_sec_t *n, const uint8_t *pl, size_t pl_len, uint8_t *out)
{
    if (n->fcnt == UINT32_MAX) return 0;          /* never wrap: re-key instead */
    uint32_t c = ++n->fcnt;
    out[0] = 0x01;                                /* data frame, protocol v1 */
    out[1] = (uint8_t)(n->id >> 8);  out[2] = (uint8_t)n->id;
    out[3] = (uint8_t)(c >> 24);     out[4] = (uint8_t)(c >> 16);
    out[5] = (uint8_t)(c >> 8);      out[6] = (uint8_t)c;

    uint8_t nonce[13] = {0};                      /* byte 0 = 0x00: uplink */
    memcpy(&nonce[1], &out[1], 6);                /* node id + counter */
    if (aes_ccm_encrypt(n->key, nonce, out, HDR_LEN, pl, pl_len,
                        &out[HDR_LEN], &out[HDR_LEN + pl_len], MIC_LEN) != 0)
        return 0;
    return HDR_LEN + pl_len + MIC_LEN;
}
```

The counter must survive resets without ever going backwards, or a reset node would reuse nonces and be rejected as a replay. Writing it to flash on every frame would cost 124,000 writes a year; instead the node

stores it every 1024 frames and, after a reset, resumes from the stored value plus 1024. A reset therefore skips at most 1024 counter values, which the gateway accepts because it requires only "greater than", and flash sees about 120 writes a year.

Keys and provisioning

Each node has its own 128-bit key, derived at provisioning as AES-CMAC of the node identifier under a site master key that lives only in the gateway's secure element and in the offline provisioning station. A captured node therefore reveals only its own key, which is revoked at the gateway. Provisioning is done on the bench over the debug port, after which the SoC's read-out protection is set so the key cannot be read back through it. The beacon is authenticated under a separate network key, so a forged beacon cannot mark slots as received or change the mode.

38.8 Deployment and testing

> **Boundary.**
> Test plans, calibration procedure and regression testing are in Chapter 4; the calibration chain and uncertainty are in Chapter 7.

Site survey and placement

Before any node was built, a survey kit (one node with a fixed SF7 beacon every 10 s, and the gateway radio logging RSSI and SNR) was carried to each of the 32 planned positions in full leaf in June and left for 15 minutes at each. The 32 medians gave the path-loss fit of 38.4 and, more usefully, each position's own margin.

Worked example — why antenna height matters here. The first Fresnel zone's radius at the midpoint of an 850 m path at 868 MHz ($\lambda = 0.346$ m) is $\sqrt{\lambda d_1 d_2/(d_1+d_2)} = \sqrt{0.346 \times 425 \times 425/850} = 8.6$ m; keeping 60 % of it clear would need 5.1 m of clearance above the canopy at mid-path. With the gateway antenna at 8 m and the node at 2.2 m, the straight line passes about 3.3 m above the 1.8 m canopy at mid-path, so the zone is partly obstructed by foliage over most of its length, which is why the fitted exponent is 3.3 rather than 2. Raising node antennas would help, but every node would become a taller, more fragile post. The survey identified one position, node 17 behind a machinery shed, at 7.5 dB margin even at SF9; moving it 15 m along the row and raising its antenna to 3.0 m gave 13.5 dB. Each node was assigned the lowest SF giving at least 12 dB measured margin; every other position had at least 22 dB at SF7, so 31 nodes run at SF7 and node 17 alone at SF9.

Calibration before deployment

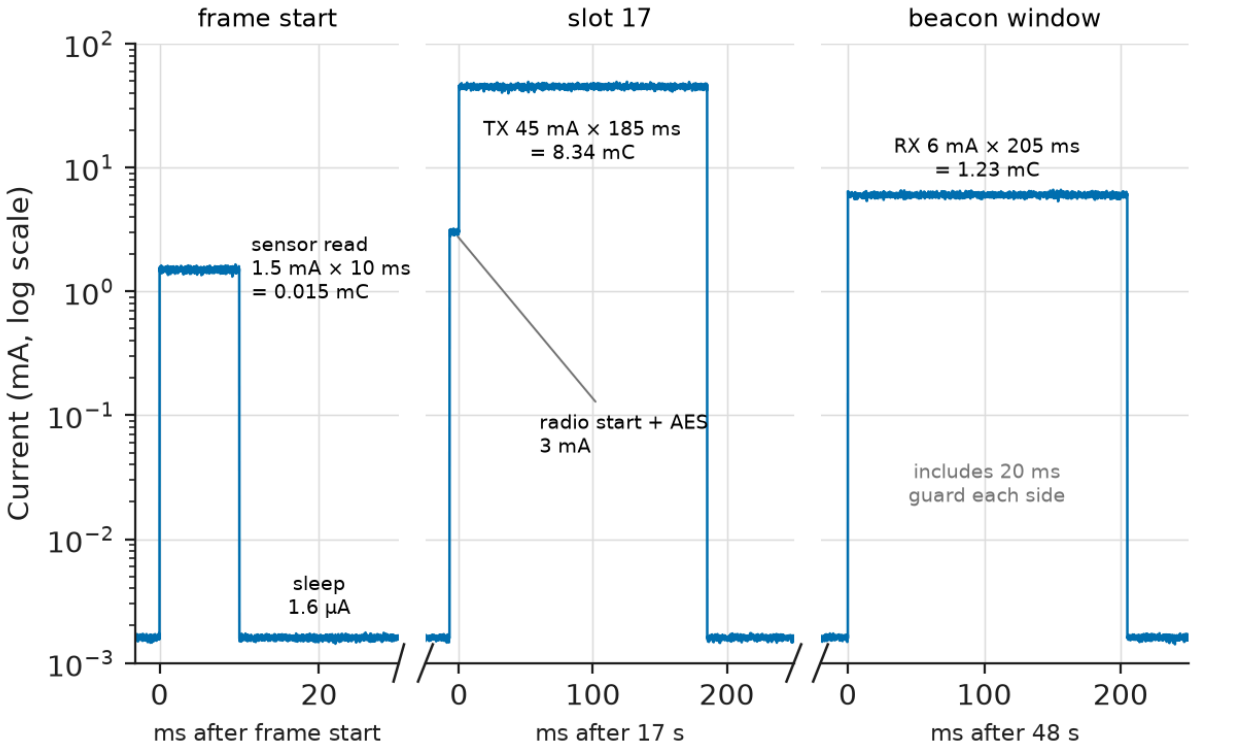
Each sensor is compared with a calibrated reference thermometer (PT100 with readout, ±0.03 °C, calibration traceable under ISO/IEC 17025) in a stirred-air climatic chamber at −5, 0, 5 and 20 °C, and with a reference hygrometer at 75 % and 95 %RH. Offsets are stored at the gateway, not in the node, so a replacement sensor needs only a new table entry. Across the 32 sensors the as-received temperature offsets were within ±0.08 °C at 0 °C, inside the ±0.1 °C typical specification; after correction the residual against the reference was ±0.03 °C. Four nodes are brought back each winter for a re-check, and if their drift exceeds 0.1 °C the whole fleet is re-checked.

Verification and test results

Table 38.8 — Verification and test results (first season, 32 nodes).

Req.	Test	Limit	Predicted	Measured	Result
R1	Chamber comparison at 0 °C after offset correction; RH at 75 %	±0.2 °C; ±2 %RH	±0.1 °C	±0.03 °C (sensor) + shield error 0.1 °C at night; ±1.2 %RH	Pass
R2	Report interval switch to 1 min below 3 °C	Next frame	60 s	60 s; frost watch entered by 32 of 32 nodes on 4 nights	Pass
R3	Node chilled to −1 °C with cold spray in a test shield	SMS ≤ 2 min	≤ 2 min	1 min 52 s worst of 5	Pass
R4	Delivery during frost watch (4 nights, 39 h)	≥ 99.9 % within 2 min	99.996 %	99.996 % (one tractor-shadowed node, 3 late of 74,880 readings)	Pass
R4	Delivery overall, 30 days	≥ 99.5 % within 24 h	—	99.97 % in own frame or retry; 100 % within 24 h	Pass
R5	Measured margin in full leaf at assigned SF	≥ 12 dB	—	13.5 dB worst (node 17, SF9); ≥ 22 dB for the 31 SF7 nodes	Pass
R6	Charge per report, SF9 node, power analyser at 20 °C and −10 °C	Life ≥ 5 yr	9.61 mC; 7.1 yr	9.92 mC at 20 °C, 10.4 mC at −10 °C; 6.6 yr at the cold value	Pass
R7	Replayed frame, modified payload byte, forged beacon	All rejected	—	All rejected; security alarm raised after 3 failures	Pass
R8	72 h at −20 °C and 24 h at +50 °C, transmitting every minute	No reset or missed slot	—	Pass; TX droop 0.14 V at −20 °C	Pass
—	Beacon timing error at −10 °C after 300 s	≤ 20 ms guard	18.6 ms worst case	13 ms (crystal better than worst case)	Pass

The measured charge per report was 3 % above the budget at 20 °C, mostly a longer radio start-up than the datasheet's, and 8 % above at −10 °C, where the TCXO and the SMPS draw more. Life computed from the cold value for the whole year, a deliberately pessimistic assumption, is 6.6 years, still 32 % over R6.



Time axis broken between the three windows (frame of 60 s). Charge per report: predicted 9.61 mC (sensor 0.015, radio start and AES 0.021, TX 8.34, beacon RX 1.23 mC) against 9.92 mC measured at 20 °C

Figure 38.4 — Measured current of an SF9 node through one report, with the charge of each phase.

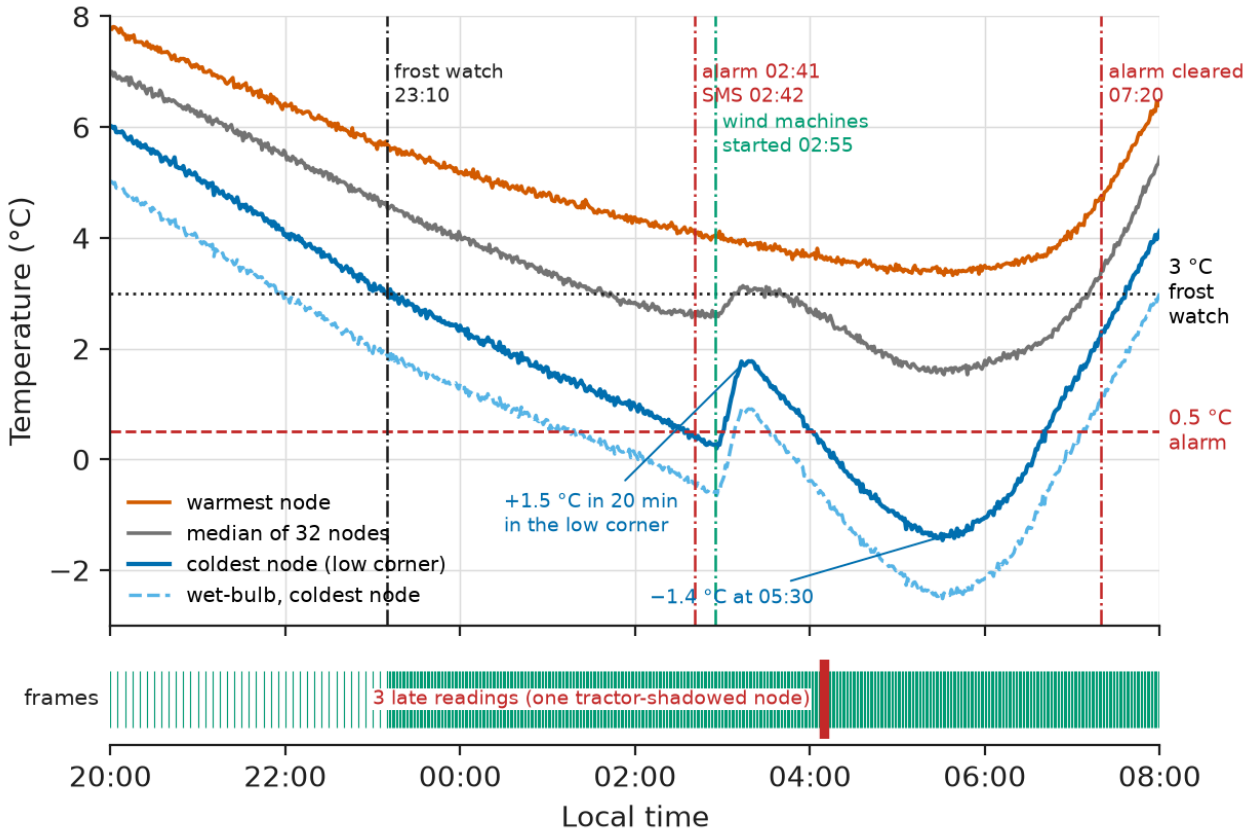


Figure 38.5 — A frost night as recorded by the network: coldest and warmest node temperatures, wet-bulb temperature, alarm and delivery.

Failure analysis

Table 38.9 — FMEA of the sensor network (S, O, D on 1–10 scales; RPN = S×O×D).

Failure mode	Effect	Cause	Detection	Mitigation	S	O	D	RPN
Node brown-out on TX pulse in cold	Resets, missed readings on frost nights	Passivated Li-$SOCl_2$ cell (pilot)	Battery voltage under load in each frame	Li-FeS_2 pair; −20 °C pulse test	8	2	3	48
Link margin lost in full leaf	Late or lost readings	Survey in leaf-off season; vegetation growth	Margin trend per node	Survey in full leaf; 12 dB measured margin	7	3	3	63
Duty-cycle limit blocks frost-watch reports	Readings silently dropped	SF raised above 10	Static check at commissioning	SF fixed ≤ 9 by design; no ADR	8	1	2	16
Gateway failure on a frost night	No alarm	Power cut, SD-card corruption, radio fault	Cloud heartbeat; local watchdog	UPS 12 h; read-only root file system; spare gateway on site	10	2	3	60
Sensor wet or iced at saturation	RH stuck at 100 %, wet-bulb wrong	Condensation on sensor	RH > 99 % for > 1 h	Heater cycle; frost logic on dry-bulb as well	6	4	3	72
Radiation shield blocked or damaged	Temperature bias	Spider webs, mowing damage	Comparison with neighbours	Seasonal inspection; outlier alarm	6	4	4	96
Forged or replayed frames	False alarm or suppressed alarm	Attacker in radio range	MIC failures, counter violations	AES-CCM, counters, per-node keys	9	1	2	18
Jamming	Network blind	Deliberate or faulty transmitter	Many nodes lost with high noise floor	Alarm on loss; local thermometer at panel	9	1	3	27
SMS path down	Manager not told	LTE outage	Modem status	Local sounder; second modem (Revision B)	9	3	4	108

The highest RPN is again not in the field electronics but in the path from the gateway to a person. A frost alarm that reaches nobody is worth nothing, and LTE coverage at the winery drops in bad weather, which is exactly when frost nights do not occur but storms do.

Bill of materials

Table 38.10 — Bill of materials with indicative cost (2026 prices, 100 nodes, USD).

Item	Specification	Qty	Unit ($)	Total ($)
Node				
Radio module	Sub-GHz LoRa SoC (STM32WLE5 class), TCXO, certified, RF switch	1	7.50	7.50
T/RH sensor	SHT45 class, ±0.1 °C, with heater	1	2.80	2.80
Radiation shield	Multi-plate, UV-stabilised plastic	1	9.00	9.00
Enclosure	IP65 polycarbonate with vent membrane	1	4.00	4.00
Antenna	868 MHz quarter-wave whip, 2.2 m mast clamp	1	3.00	3.00
Batteries and holder	2 × AA Li-FeS2, spring holder	1 set	3.80	3.80
PCB, passives, ESD protection	2-layer, MOSFET switch, TVS	1 set	2.70	2.70
Post and mounting	Galvanised post clamp	1	5.00	5.00
Assembly, provisioning, calibration check	Including chamber time	1	4.00	4.00
Node total				**41.80**
Leaf-wetness sensor (8 nodes only)	Dielectric leaf-shaped probe	1	38.00	38.00
Gateway				
Linux single-board computer with eMMC	Industrial temperature grade	1	60.00	60.00
Radio	SX1262-class module with TCXO on carrier board	1	25.00	25.00
LTE modem	Cat-1 with SMS	1	45.00	45.00
Antenna system	5 dBi collinear, low-loss cable, arrester	1	45.00	45.00
Backup supply	12 V 6 Ah LiFePO4 battery with charger	1	50.00	50.00
Enclosure	IP65 with DIN rail	1	20.00	20.00
Gateway total				**245.00**

The radiation shield costs more than the radio module. The 32-node network with eight leaf-wetness sensors and one gateway costs $32 \times 41.80 + 8 \times 38 + 245 = 1887$ dollars in parts, comparable to the fuel the four wind machines burn in one unnecessary night.

Revision B

Change	Reason	Effect
Second SMS path on another operator, plus a voice call on alarm	Highest-RPN item: alarm path depends on one LTE link	Alarm reaches a person when one network is down
Single Li-FeS2 cell (1.5 V, boost converter) for SF7 nodes	15.9-year life at SF7 exceeds cell shelf life	$1.60 and 15 g less per near node
Bud-temperature proxy: a small black-body sensor beside the shield on 8 nodes	Air temperature is not bud temperature on clear nights	Threshold set on the quantity that causes damage
Over-the-air firmware update with signed images, in the spare 10 s of each frame	First season needed two firmware fixes, done by hand	Field updates without visiting 32 posts
Outlier detection against neighbouring nodes at the gateway	Shield blockage was the second-highest RPN	Biased sensors flagged within a day
Clock-rate learning from successive beacons	Measured drift was better than worst case	Guard window could shrink from ±20 to ±8 ms, about 4 % less energy at SF7

What remains open

Open question	Where it is answered
How are RF front ends, antennas and link budgets designed in general, including interference and wireless sensor networks at larger scale?	Chapter 26
How are device provisioning, MQTT security, gateways and cloud storage built for an IoT product?	Chapter 11
How would the frost alarm improve with a model that forecasts the minimum temperature an hour ahead from the network's data?	Chapter 32
How is a DAQ system synchronised and validated when sample timing matters to microseconds?	Chapter 8
How is the wind-machine start panel designed with its interlocks if the start is automated?	Chapter 20
How do path loss, fading and outage follow from propagation physics?	*Circuits, Signals and Communications*, Ch. 30

Next: Chapter 39 turns from hardware to software and builds an engineering simulation tool, from its governing equations to verified, documented and reproducible results.

Chapter 39 — Project: Engineering Simulation Tool

Part IX · Interdisciplinary Engineering Projects

A 12 V automotive cooling-fan controller switches up to 25 A through two DFN5×6 MOSFETs in parallel on a 100 × 80 mm two-layer board. The first thermal estimate used the datasheet junction-to-ambient resistance, about 50 K/W "on a 1 in² pad of 2 oz copper", and predicted a 75 K rise for each 1.5 W device. The team responded by adding a $3.80 clip-on heat sink and a third MOSFET. On the bench the bare board rose only 35 K. Inside the sealed fan housing at 85 °C, though, the thermocouple on Q1's drain tab read 141 °C. That left 9 K to the 150 °C derated limit before the junction-to-tab drop was even added. Each layout variant took a week to evaluate: order boards, assemble, bond thermocouples, soak, repeat. The datasheet $R_{\theta JA}$ describes a JEDEC test board, not this one, so it could not rank variants. A commercial 3-D thermal package could, but it cost about $25,000 a year and a day of set-up per layout. The team chose a third option. They wrote a small, verified, purpose-built tool, *pcbtherm*. It solves the in-plane heat spreading of a populated board in under a second, sweeps 200 layout variants in under a minute, and carries written evidence of how far its answers can be trusted.

The running reference system is pcbtherm Rev A applied to that board. The physics, numerics and software practice are owned elsewhere; this chapter integrates them into a tool others can rely on.

> **How does an engineer turn a physical question into a simulation tool whose equations, solver, input checks and outputs all serve one design decision?**
> **What evidence — verification, benchmarking, validation, documentation and reproducibility — makes a home-grown tool trustworthy enough to drop a heat sink from a production board?**

Concept	Engineering destination
Problem definition for a tool	Scope tied to one design question; requirements with numbers and verification methods
Governing equations with stated assumptions	A model whose validity domain is checked on every run
Finite-volume discretisation, sparse direct solve	Sub-second steady solutions; transients that reuse one factorisation
Input validation	Unit and geometry errors rejected before any matrix is built
Visualization	Maps and probe curves a layout reviewer can act on
Benchmarking	Runtime and memory budgets with measured margins; code-to-code comparison
Verification and validation	Manufactured solutions, grid convergence, energy closure, comparison with a measured board
Documentation	Theory manual, user guide and V&V report shipped with the code
Reproducible results	Run manifest and golden-value regression gate

Roadmap. Question and requirements (39.1) → equations and validity checks (39.2) → discretisation and solver (39.3) → input validation (39.4) → visualization (39.5) → benchmarking (39.6) → verification and validation (39.7) → documentation (39.8) → reproducibility (39.9) → cost, test results, FMEA and Revision B (39.10).

39.1 Problem definition

> **Boundary.**
> Turning a need into requirements is Chapter 1. The modelling workflow (assumptions, sensitivity, validation) is Chapter 3. The software workflow (version control, reproducible builds, automated tests) is Chapter 29. Solvers, sweeps and numerical verification are Chapter 30. Tool architecture is Chapter 31. Calibration and uncertainty quantification are Chapter 33. This chapter applies all six to one tool.

The decision the tool must support

Tools fail most often because nobody asked which decision they serve. Here it was concrete. *For a given copper layout, component placement and cooling condition, does any power device exceed its temperature limit, and how much does a proposed layout change move it?* The layout engineer wants to rank ten variants before lunch; the reviewer wants to know whether a 4 K margin is real. The tool must give:

- The steady board-temperature rise at each power component, for two cooling cases: the open bench (for correlation with tests) and the sealed housing (for qualification).
- The warm-up transient, which sets the soak time of the qualification test.
- A ranking of layout variants by copper weight, device spacing and shunt position.

Out of scope: junction temperatures from first principles (the datasheet junction-to-board resistance is added), airflow (convection coefficients are inputs), and forced-air or liquid cooling. Writing the exclusions down stops later users from leaving the validity domain.

Requirements

Table 39.1 — Requirements for pcbtherm Rev A.

ID	Requirement	Target	Verification method
R1	Steady rise at power-device pads vs measured reference board	Within ±3 K or ±10 % of the rise, whichever is larger	Validation, five thermocouples (39.7)
R2	Warm-up time to 63 % of final rise	Within ±15 % of measured	Validation (39.7)
R3	Steady solve, 100 × 80 mm board at 0.5 mm cells	≤ 1 s on the reference workstation	Benchmark (39.6)
R4	Sweep of 200 layout variants	≤ 5 min	Benchmark (39.6)
R5	Spatial discretisation order	Observed 1.9–2.1 on a manufactured solution	Code verification (39.7)
R6	Energy balance of every steady solution	Relative closure ≤ 1×10^{-6}	Automatic check on every run
R7	Invalid input rejected with key, value, unit and range	24 of 24 seeded bad cases caught	Unit tests (39.4)
R8	Any reported result re-creatable from its manifest	Equal to 4 decimals on a clean environment	Reproduction test (39.9)
R9	Review outputs	Map, probe CSV, run report listing assumptions	Inspection (39.5)
R10	Documentation	Theory manual, user guide, V&V report	Design review (39.8)

R1 is the requirement the decision rests on. R3 and R4 make the tool usable, and R5–R8 make R1 believable. The ±3 K tolerance comes from the 9 K housing margin: a tool that is right to ±3 K can separate "heat sink needed" from "not needed".

Architecture of the tool

Figure 39.1 shows the pipeline. Every result leaves with a run manifest, and the verification suite calls the same core functions as the user, so the code that was verified is the code that runs.

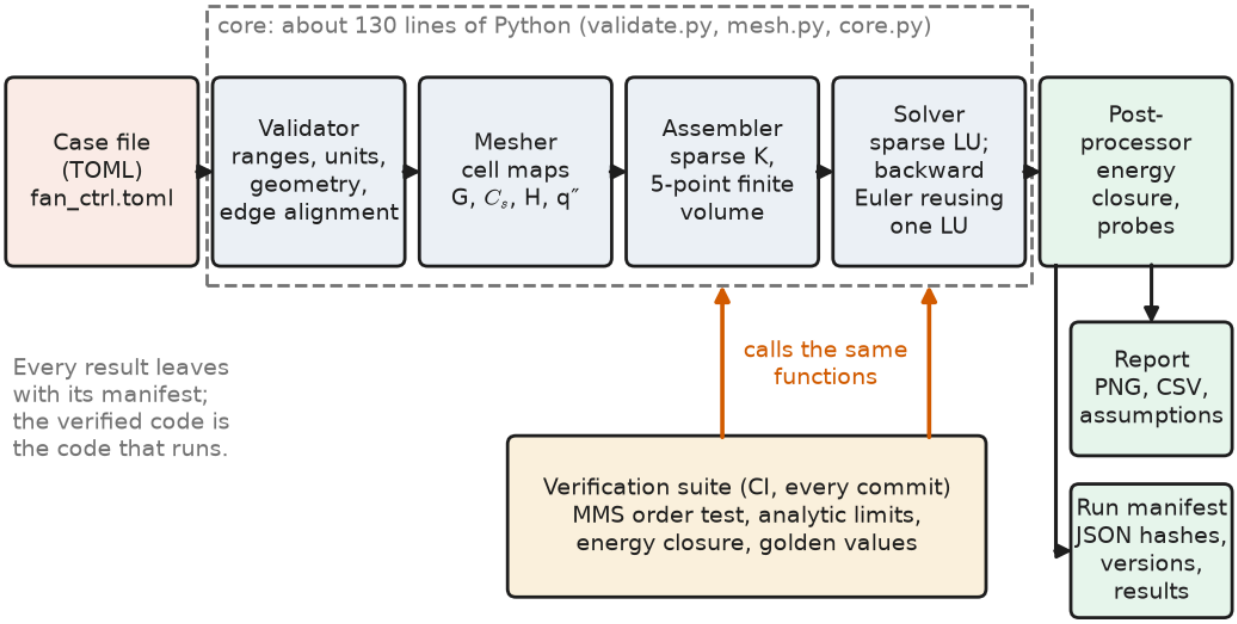


Figure 39.1 — Architecture of pcbtherm Rev A, from case file to verified result.

The core is four short files (`validate.py` , `mesh.py` , `core.py` , `manifest.py`) plus a command-line driver; Chapter 31 argues for this modular, file-based form. The deciding argument was review: a 130-line core can be read line by line, and each listing below is one of those files, complete.

39.2 Governing equations

The thin-plate energy balance

The board has two 2 oz copper pours (69.6 µm each) on 1.6 mm FR-4. It is thin compared with its width, and copper conducts about 480 times better than FR-4. Heat from a component therefore spreads mainly *in the plane* and leaves through both faces by convection and radiation. Lumping temperature through the thickness, the field-to-lumped step of *Engineering Physics*, Ch. 17 §17.1, reduces the 3-D heat equation to a 2-D diffusion–reaction equation. Its unknown is the temperature rise $\theta = T - T_\infty$:

$$C_s \frac{\partial \theta}{\partial t} = \nabla \cdot (G_s \nabla \theta) - H\,\theta + q''(x, y, t)$$

The symbols are:

- $G_s = \sum_i k_i t_i$, the sheet conductance in W/K.
- $C_s = \sum_i \rho_i c_i t_i$, the heat capacity per unit area in J/(m² K).
- $H = h_{top} + h_{bot}$, the combined film coefficient of the two faces (convection plus linearised radiation).
- q'', the dissipated power per unit area under each component.

The edges are treated as adiabatic, since their area is under 4 % of the faces. The result is the two-dimensional fin equation. Its classification, boundary conditions and Biot-number scaling are in *The Mathematics Universe*, Ch. 13 §13.4 and §13.8. Two scales predict most of its behaviour before any solve:

$$\ell = \sqrt{G_s / H}, \qquad \tau = C_s / H$$

ℓ is how far a hot spot spreads before the faces shed its heat, and τ is the board time constant. Under uniform heating the rise is simply $P/(HA)$.

> **Status of the principle: Thin-plate (fin) model of a populated PCB**
> *Approximate.* The model holds under four conditions:
>
> - The through-thickness Biot number is small: $h\,t/k_z \ll 1$.
> - The copper layers are well coupled by vias under every heat source.
> - The film coefficient h is constant, which holds within about ±30 K of the design point.
> - Component heat capacity and conduction are negligible beside the board's.

Table 39.2 — Inputs for the fan-controller board.

Quantity	Value	Basis
Copper conductivity	385 W/(m K)	Electrodeposited foil, typical
FR-4 conductivity, in-plane / through-plane	0.8 / 0.3 W/(m K)	Laminate datasheet range 0.6–1.0 / 0.25–0.35
Copper thickness per layer	69.6 µm (2 oz/ft²)	IPC-4562 nominal, plating about ±10 %
Sheet conductance G_s	0.0549 W/K	$2 \times 385 \times 69.6\ \mu\text{m} + 0.8 \times 1.6$ mm
Heat capacity per area C_s	3,736 J/(m² K)	Copper 3.45, FR-4 2.04 MJ/(m³ K)
h_{top}, h_{bot}, open bench, horizontal	12, 9 W/(m² K)	Natural convection 5–7 plus radiation about 5 (ε ≈ 0.9)
h_{top}, h_{bot}, sealed housing	6, 4 W/(m² K)	Enclosed air, warm housing wall
Sources	Q1, Q2: 1.5 W each on 5 × 6 mm; shunt and driver: 0.4 W on 6 × 3.2 mm	Loss calculation at 25 A, 20 kHz

Worked example — Is the thin-plate model valid for this board?

Through-thickness. Heat leaving the bottom face must cross the FR-4. The drop across it, relative to the rise, is $h_{bot} t / k_z = 9 \times 1.6 \times 10^{-3} / 0.3 = 0.048$. That is under 5 %.

Lateral. Copper carries $2 \times 385 \times 69.6 \times 10^{-6} / 0.0549 = 97.7\ \%$ of G_s, so the FR-4 in-plane value barely matters. The spreading length is $\ell = \sqrt{0.0549/21} = 51$ mm, comparable with the board. Edges and device spacing therefore matter, and a 2-D model is needed rather than a single lumped node.

Time. $\tau = 3736/21 = 178$ s, so a one-hour soak covers about 20τ.

Vias, the weak point. Without vias, the FR-4 under a 30 mm² pad has a through-plane resistance of $1.6 \times 10^{-3}/(0.3 \times 30 \times 10^{-6}) = 178$ K/W. One plated via (0.3 mm hole, 25 µm wall, $2.55{\times}10^{-8}$ m² of copper) has $1.6 \times 10^{-3}/(385 \times 2.55 \times 10^{-8}) = 163$ K/W. The 15-via array under each MOSFET therefore gives about 10.9 K/W: good coupling, but not the isothermal stack the model assumes. A few hundred milliwatts

crossing near the pad leaves the top copper roughly 2 K hotter than the merged-layer prediction. Section 39.7 measures this bias, and Revision B removes it.

> **Boundary.**
> Thermal-resistance networks, spreading resistance and convection and radiation coefficients come from *Engineering Physics*, Ch. 14 §14.7. MOSFET thermal resistances and the switching-loss calculation that sets q'' belong to *Electronics and Computing*, Ch. 29, and are applied to a full drive in Chapter 37.

39.3 Numerical solver

Discretisation and time integration

The board is cut into square cells of side Δ, each carrying its own G_s, C_s, H and q''. Copper keep-outs, different pours and heat sources are all just maps. Integrating over a cell gives one energy balance per cell, the finite-volume form:

$$C_s \Delta^2 \, \dot{\theta}_P = \sum_{F \in \{E,W,N,S\}} g_{PF} \, (\theta_F - \theta_P) - H \Delta^2 \theta_P + q''_P \Delta^2$$

The face conductance g_{PF} is the *harmonic* mean of the two cells' G_s, which is what two thermal resistances in series give. It stays correct where a pour meets bare laminate. Stacking the cells gives

$$\mathbf{C}\dot{\boldsymbol{\theta}} = -\mathbf{K}\boldsymbol{\theta} + \mathbf{q},$$

where $\mathbf{K}$ is a symmetric positive-definite M-matrix with five non-zeros per row. The scheme conserves energy exactly, which makes requirement R6 a test of the *code*. The consistency, stability and method-of-lines view of such schemes are in *The Mathematics Universe*, Ch. 22 §22.10–§22.11.

> **Status of the principle: Discrete energy conservation of the finite-volume scheme**
> *Derived.* Each face flux leaves one cell and enters its neighbour with the same conductance, so internal fluxes cancel. The total face loss therefore equals the injected power to round-off ($\approx 10^{-13}$ relative here). A closure worse than about 10^{-9} signals a bug, not a discretisation error.

The transient system is stiff. Its grid-scale modes decay at about $4G_s/(C_s\Delta^2)$, while the board mode decays at $1/\tau$. Backward Euler was chosen:

$$\left(\tfrac{1}{\Delta t}\mathbf{C} + \mathbf{K}\right)\boldsymbol{\theta}^{n+1} = \tfrac{1}{\Delta t}\mathbf{C}\,\boldsymbol{\theta}^{n} + \mathbf{q}^{n+1}$$

It is L-stable. Crank–Nicolson rings cell by cell after a step change in power, because its amplification factor tends to -1 for grid-scale modes. With a constant Δt the left-hand matrix never changes. It is factorised once, and each step costs only one forward–back substitution.

> **Status of the principle: Implicit stepping for diffusion on fine grids**
> *Derived.* An explicit step must satisfy $\Delta t \le C_s/(4G_s/\Delta^2 + H)$, a limit that shrinks with Δ^2. An L-stable implicit step is limited only by the accuracy wanted on the slow modes.

Worked example — Explicit or implicit for a one-hour warm-up?

Explicit. At $\Delta = 0.5$ mm the stability limit is $\Delta t \le 3736/(4 \times 0.0549/(0.5 \times 10^{-3})^2 + 21) = 4.26$ ms. One hour takes 845,972 steps. Each step is one sparse matrix–vector product, measured at 0.49 ms on 32,000 cells, so the run takes about 410 s.

Implicit. Backward Euler with $\Delta t = 5$ s takes 720 steps. One factorisation plus 720 solves measured 17.8 s, 23 times faster.

Accuracy check. For the board mode at $t = 600$ s, the backward-Euler response is $1 - (1 + 5/178)^{-120} = 0.9640$ against the exact $1 - e^{-600/178} = 0.9656$. On a 32.6 K rise that is a 0.05 K error. A 20 s step would give 0.22 K.

Platform and linear solver

Table 39.3 — Component selection: implementation platform.

Option	Time per case	Validity for this board	Cost (2026)	Verdict
Datasheet $R_{\theta JA}$ hand calculation	Seconds	JEDEC board, one device, no layout effect	$0	Rejected: cannot rank variants
Commercial 3-D thermal FEM/CFD	1 day setup, 10–30 min solve	Best: layers, vias, airflow	About $25,000/yr	Kept as code-to-code reference only
Python + NumPy/SciPy, finite volume	0.2 s	Thin-plate domain, checked each run	$0 (BSD licences)	**Chosen**: fast enough, reviewable

Table 39.4 — Component selection: linear solver (32,000 / 128,000 cells, measured in 39.6).

Solver	Behaviour on K	Time	Verdict
Dense LAPACK	8.2 GB just to store the 32k matrix	—	Rejected
CG, Jacobi preconditioner	Iterations grow as $1/\Delta$ (1,089 → 2,168)	1.0 s / 7.9 s	Fallback only
CG + algebraic multigrid (`pyamg` , optional)	Nearly grid-independent iterations	—	Revision B
Sparse LU (SuperLU, `scipy.sparse.linalg.splu`)	Direct; factor reused every time step	0.19 s / 1.34 s	**Chosen**

The direct solver also has no convergence tolerance to differ silently between runs, and its factorisation serves the whole transient. Why CG iterations grow with the condition number is in *The Mathematics Universe*, Ch. 24 §24.4–§24.5.

The core

`core.py` is the whole numerical core. `assemble` builds **K** face by face in COO form, where duplicate entries are summed on conversion. `steady` solves for the rise. `transient` is a generator, so the caller decides what

to keep, and a one-hour run never holds 720 maps in memory.

```
 """pcbtherm.core -- finite-volume thin-plate thermal solver (Rev A)."""
import numpy as np
import scipy.sparse as sp
import scipy.sparse.linalg as spla

def assemble(G, H, dx, dy):
    """Conductance matrix K in W/K for an (ny, nx) grid of cells.
    G: sheet conductance sum(k*t) per cell, W/K; H: h_top + h_bot, W/(m2 K)."""
    ny, nx = G.shape
    idx = np.arange(nx * ny).reshape(ny, nx)
    hm = lambda a, b: 2.0 * a * b / (a + b)          # harmonic mean at faces
    pairs = [(idx[:, :-1], idx[:, 1:], hm(G[:, :-1], G[:, 1:]) * dy / dx),
             (idx[:-1, :], idx[1:, :], hm(G[:-1, :], G[1:, :]) * dx / dy)]
    r, c, v = [], [], []
    for a, b, g in pairs:                            # each face couples a and b
        a, b, g = a.ravel(), b.ravel(), g.ravel()
        r += [a, b, a, b]; c += [a, b, b, a]; v += [g, g, -g, -g]
    n = nx * ny
    K = sp.coo_matrix((np.concatenate(v), (np.concatenate(r), np.concatenate(c))),
                      shape=(n, n)).tocsc()
    return (K + sp.diags(H.ravel() * dx * dy)).tocsc()

def steady(K, q, dx, dy):
    """Temperature rise (K) for a heat-flux map q in W/m2."""
    return spla.spsolve(K, q.ravel() * dx * dy).reshape(q.shape)

def transient(K, Cs, q_of_t, dx, dy, dt, n_steps, theta0=None):
    """Backward Euler; K factored once. Cs: heat capacity per area, J/(m2 K).
    q_of_t(t) returns the flux map at time t. Yields (t, theta map)."""
    shape = Cs.shape
    M = sp.diags(Cs.ravel() * dx * dy / dt)
    lu = spla.splu((M + K).tocsc())                  # one factorisation
    th = np.zeros(Cs.size) if theta0 is None else theta0.ravel().copy()
    for k in range(1, n_steps + 1):
        t = k * dt
        th = lu.solve(M @ th + q_of_t(t).ravel() * dx * dy)
        yield t, th.reshape(shape)
```

`mesh.py` turns a validated case into the maps. Its one subtle job is rasterising footprints so that **power is conserved exactly** even when the footprint's area is not.

```
 """pcbtherm.mesh -- turn a validated case into cell maps (SI units)."""
import numpy as np

K_CU, K_FR4 = 385.0, 0.8                 # W/(m K); FR-4 in-plane value
RC_CU, RC_FR4 = 8960 * 385, 1850 * 1100  # volumetric heat capacity, J/(m3 K)
OZ = 34.8e-6                             # m of copper per oz/ft2

def build(cfg):
    b = cfg["board"]
    c = b["cell_mm"] * 1e-3
    nx, ny = round(b["width_mm"] * 1e-3 / c), round(b["height_mm"] * 1e-3 / c)
    t_cu, t_fr = b["copper_oz"] * OZ, b["fr4_mm"] * 1e-3
    G = np.full((ny, nx), 2 * K_CU * t_cu + K_FR4 * t_fr)   # two full pours
    Cs = np.full((ny, nx), 2 * RC_CU * t_cu + RC_FR4 * t_fr)
    H = np.full((ny, nx), float(b["h_top"] + b["h_bot"]))
    X, Y = np.meshgrid((np.arange(nx) + 0.5) * c, (np.arange(ny) + 0.5) * c)
    q = np.zeros((ny, nx))
    for s in cfg.get("source", []):
        m = ((np.abs(X - s["x_mm"] * 1e-3) < s["w_mm"] * 5e-4 + 1e-9) &
             (np.abs(Y - s["y_mm"] * 1e-3) < s["h_mm"] * 5e-4 + 1e-9))
        q += m * s["power_W"] / (m.sum() * c * c)          # power is conserved
    return dict(G=G, Cs=Cs, H=H, q=q, dx=c, dy=c, X=X, Y=Y)
```

Rev A applies the pours uniformly. A per-cell coverage map from the Gerber copper layers changes only the mesher, since the solver already accepts any G_s map. Run on the case file of Section 39.4, the pipeline reports:

- $G_s = 0.0549$ W/K and $C_s = 3736$ J/(m² K);
- a maximum rise of 33.72 K at Q1;
- a mean rise of 20.24 K, which equals $P/(HA) = 3.4/(21 \times 0.008)$ as it must;
- an energy closure of 6.4×10^{-13}.

39.4 Input validation

The first prototype had no validator. In its second week a user typed a width of `0.1`, meaning metres, into a millimetre field and got a rise of thousands of kelvin. Worse, a footprint dragged half off the board was silently clipped: it lost a third of its area but none of its power, and Q1's prediction rose 6 K. No solver verification catches such slips.

The validator checks **type and range**, **geometry** (on the board, at least two cells, edges on cell faces) and **plausibility** (the lumped mean rise $P/(HA)$). Its ranges are the validated domain, so leaving a range means leaving the model. Two examples: beyond 500 mm the mesh exceeds the memory budget, and $h \le 200$ W/(m² K) keeps the through-thickness Biot number below 0.1. Units live in the key names (`width_mm`). Errors stop the run; warnings go into the report.

```
 """pcbtherm.validate -- reject bad input before any matrix is built."""
RANGES = {  # key: (min, max, unit) -- the validity domain of the Rev A model
    "width_mm": (5, 500, "mm"), "height_mm": (5, 500, "mm"),
    "cell_mm": (0.1, 5, "mm"), "copper_oz": (0.5, 4, "oz"),
    "fr4_mm": (0.4, 3.2, "mm"), "h_top": (2, 200, "W/(m2 K)"),
    "h_bot": (2, 200, "W/(m2 K)"), "ambient_C": (-40, 125, "degC"),
}

def validate(cfg):
    errs, warns = [], []
    b = cfg.get("board", {})
    for key, (lo, hi, unit) in RANGES.items():
        v = b.get(key)
        if not isinstance(v, (int, float)):
            errs.append(f"board.{key}: missing or not a number"); continue
        if not lo <= v <= hi:
            errs.append(f"board.{key} = {v} {unit} outside [{lo}, {hi}]")
    if errs:
        return errs, warns
    W, Hh, c = b["width_mm"], b["height_mm"], b["cell_mm"]
    for i, s in enumerate(cfg.get("source", [])):
        x, y, w, h, p = (s[k] for k in ("x_mm", "y_mm", "w_mm", "h_mm", "power_W"))
        if not (0 <= x - w/2 and x + w/2 <= W and 0 <= y - h/2 and y + h/2 <= Hh):
            errs.append(f"source[{i}] footprint leaves the board")
        if not 0 <= p <= 100:
            errs.append(f"source[{i}].power_W = {p} outside [0, 100]")
        if min(w, h) < 2 * c:
            warns.append(f"source[{i}] spans < 2 cells; refine cell_mm")
        edges = (x - w/2, x + w/2, y - h/2, y + h/2)
        if any(abs(e / c - round(e / c)) > 1e-6 for e in edges):
            warns.append(f"source[{i}] edges off the cell faces; area will change")
    P = sum(s["power_W"] for s in cfg.get("source", []))
    rise = P / ((b["h_top"] + b["h_bot"]) * W * Hh * 1e-6)
    if rise > 100:
        warns.append(f"mean rise {rise:.0f} K: constant-h model not valid")
    return errs, warns
```

Every later result in this chapter comes from the reference case file `fan_ctrl.toml`. Its board section is:

```
 # fan_ctrl.toml -- 12 V fan-controller board, Rev 2 layout
[board]
width_mm = 100
height_mm = 80
cell_mm = 0.5
copper_oz = 2          # top and bottom pours
fr4_mm = 1.6
h_top = 12             # W/(m2 K), open bench, board horizontal
h_bot = 9
ambient_C = 40
```

Three `[[source]]` tables follow, from Table 39.2: Q1 at (30, 40) mm, Q2 at (45, 40) mm, the shunt group at (70, 20) mm. Validating the clean case, then the width as `0.1`, then Q1 at `x_mm = 98` returns:

```
([], ['source[2] edges off the cell faces; area will change'])
(['board.width_mm = 0.1 mm outside [5, 500]'], [])
(['source[0] footprint leaves the board'], ['source[2] edges off the cell faces; area will
change'])
```

The clean-case warning is real: the 3.2 mm shunt group becomes 3.0 mm on the grid, raising its local flux 7 % with power conserved. The edge check itself came from a failed grid study (Section 39.7). R7 is closed by 24 seeded bad cases (missing keys, strings, negative powers, metres in each length field, sources off each edge, sub-cell footprints); all 24 are caught.

> **Status of the principle: Validity domain enforced at the input**
> *Empirical.* In use, unit and geometry slips outnumber solver bugs among the faults users actually hit. Range checks tied to the validated domain catch them at no run-time cost, but only the faults someone anticipated. Each new fault seen in use becomes a new seeded test.

39.5 Visualization

Each plot exists to support the decision, so each has one job:

- **The steady map** (Figure 39.2): perceptually uniform colours, 5 K contours, footprints and thermocouples drawn, and a scale fixed per study, since auto-scaling hides a 10 K improvement.
- **Probe curves** (Figure 39.5) overlay simulation lines and measured markers with error bars.
- **The variant table**: one row per layout, each device's margin, sorted by the worst. For ranking, a table beats any picture.

Each PNG is written beside the CSV of its data. The plotting practice is *Electronics and Computing*, Ch. 22 §22.4. The run report adds the validator warnings, the computed validity checks (Biot number, ℓ, τ) and the manifest hash, so a figure separated from its report can still be traced to its run.

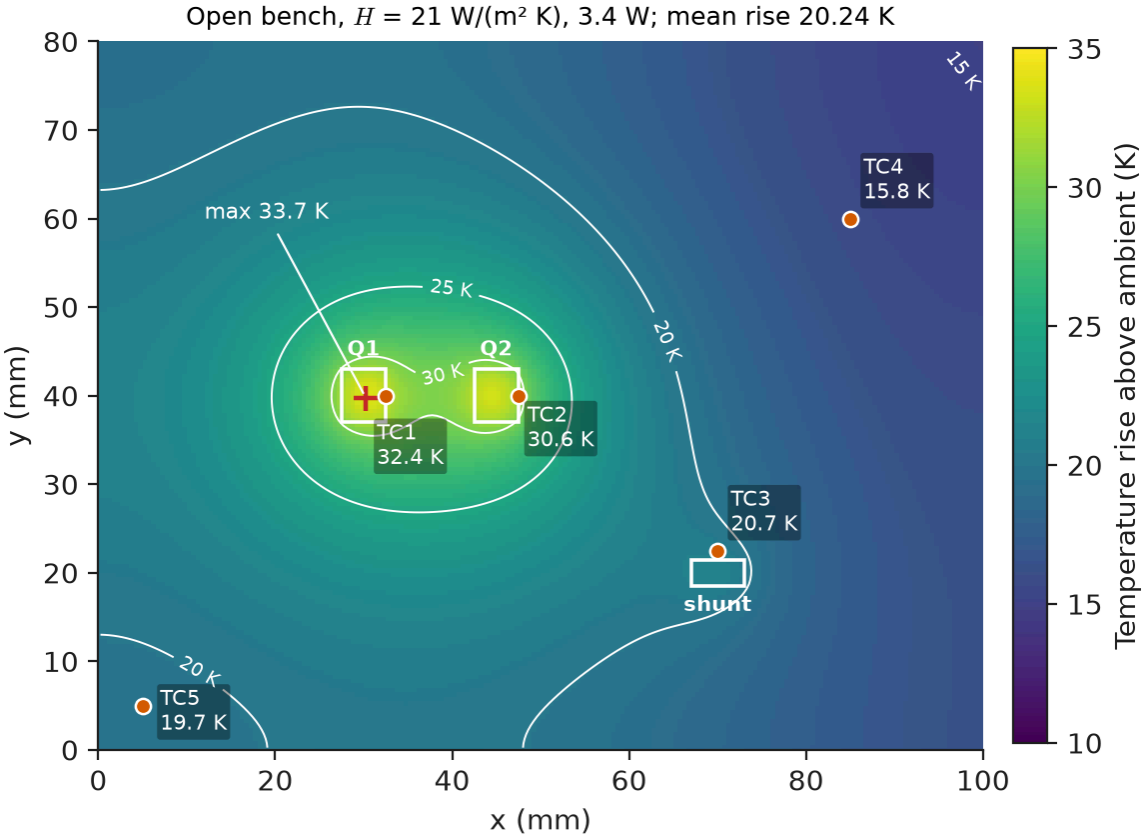


Figure 39.2 — Steady temperature-rise map of the fan-controller board on the open bench (pcbtherm Rev A, 0.5 mm cells).

Several things can be read from the map. TC1 reads hotter than TC2 (32.4 K against 30.6 K) although both MOSFETs dissipate 1.5 W. TC1 sits in the 10 mm gap between the devices and is heated from both sides, while TC2 faces open copper.

The lower-left corner (TC5, 19.7 K) is warmer than the upper-right region (TC4, 15.8 K). It lies 43 mm from Q1, inside one spreading length, and the two adiabatic edges that meet there turn back heat that would otherwise spread outward.

The actionable fact is that the MOSFETs heat each other. Moving Q2 out to 30 mm centre spacing lowers TC1 from 32.4 K to 29.0 K on the bench. 10 % thicker copper is worth only 1.0 K.

Worked example — Reading a variant sweep. A cost-reduction proposal swaps 2 oz copper for 1 oz. The tool predicts that TC1 rises from 32.4 K to 42.7 K on the bench, because the spreading length shrinks from 51 mm to $\sqrt{0.0281/21} = 37$ mm. To transfer this to the housing, scale by the ratio of housing to bench rise at TC1, 55.1/32.4 = 1.70. The 10.3 K bench increase becomes about 17.5 K in the housing, which consumes the 9 K margin twice over. The variant row is flagged red, and the proposal is closed without building a board.

39.6 Benchmarking

Benchmarking has two meanings here. **Performance benchmarking** measures time and memory against R3 and R4. **Code-to-code benchmarking** compares results with an independent, more complete model.

Performance

The steady solve was timed on three grids, best of three warm runs, following the benchmarking hygiene of *Electronics and Computing*, Ch. 24 §24.6. The script times `assemble`, `splu(K).solve(b)` and `scipy.sparse.linalg.cg` with a Jacobi preconditioner (`rtol=1e-10`):

```
 cells   assemble   splu      CG+Jacobi  iters
  8000    0.006    0.045      0.093     550
 32000    0.016    0.186      1.014    1089
128000    0.074    1.336      7.946    2168
```

On an 8-core desktop (Python 3.14, NumPy 2.5, SciPy 1.18) sparse LU grows about linearly on the first refinement (4.1×) but 7.2× on the second, roughly $n^{1.4}$, while CG doubles its iterations at each halving of the cell size and grows 8–11× per refinement. Absolute times differ between machines; the ratios carry over. Figure 39.3 plots both against the R3 budget.

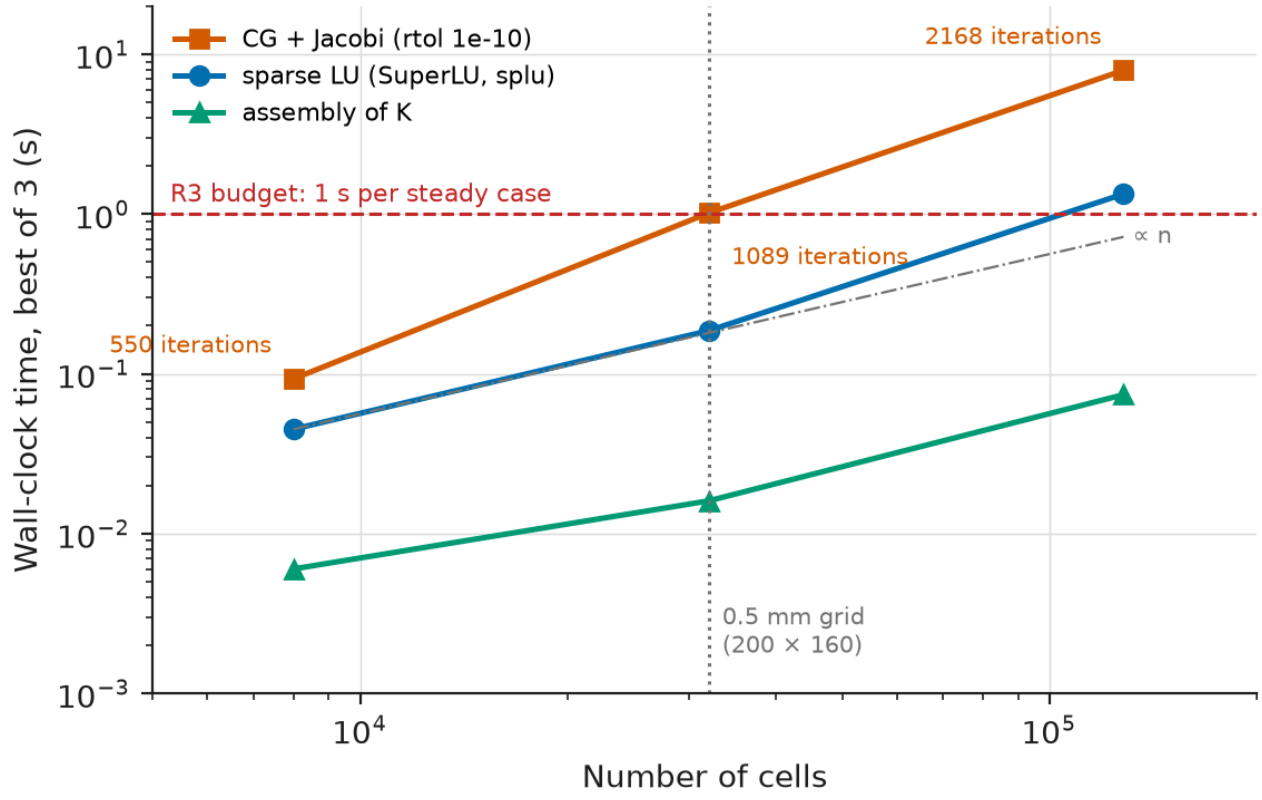


Figure 39.3 — Measured steady-solve time against number of cells for sparse LU and Jacobi-preconditioned CG, with the R3 budget.

Table 39.5 — Runtime and memory budget (reference grid 200 × 160 = 32,000 cells).

Item	Budget	Measured or computed	Margin
Steady case: validate, mesh, assemble, LU (R3)	1.0 s	0.22 s	78 %
200-variant sweep, serial (R4)	300 s	200 × 0.22 s = 44 s	85 %
One-hour transient, $\Delta t = 5$ s	60 s (usability)	17.8 s	70 %
LU factor memory, 0.5 mm grid	500 MB	31 MB	94 %
LU factor memory, 0.25 mm grid	500 MB	161 MB	68 %

Worked example — Memory budget of the direct solver. At 0.5 mm, **K** has 159,280 non-zeros. SuperLU's factors have 2,603,166, a fill ratio of 16.3. At 12 bytes per entry (an 8-byte value and a 4-byte index) that is 31 MB. At 0.25 mm the fill ratio rises to 21.0 and the factors take 161 MB. Fill grows roughly as $n \log n$, so 0.125 mm (512,000 cells) needs an estimated 0.8 GB and a measured 16 s solve. That breaks R3, which is why the validator's limits cap the cell count. Above that size, CG with algebraic multigrid is the right solver.

Code-to-code comparison

The independent reference was a 3-D finite-element model with both copper layers, the FR-4 and the 15 vias under each MOSFET, run once by a consultant with the same convection coefficients. Air was not modelled, so the comparison isolates *model form* from convection uncertainty.

Table 39.6 — Code-to-code benchmark, open bench, rise above ambient (K).

Location	pcbtherm Rev A (2-D)	3-D FEM	Difference
TC1, Q1 tab	32.4	34.5	−2.1
TC2, Q2 tab	30.6	32.7	−2.1
TC3, shunt	20.7	21.0	−0.3
TC4, upper right	15.8	15.7	+0.1
TC5, corner	19.7	19.5	+0.2
Board mean	20.24	20.24	0.0

Away from the sources the two models agree to ±0.2 K, and the means agree exactly because both conserve energy with the same H. Spreading and convection are therefore right. The −2.1 K at the MOSFET tabs is the merged-layer bias predicted in Section 39.2: near a source, the top copper runs hotter than the layer average. This *model-form* error is systematic and its sign is known.

39.7 Verification

> **Boundary.**
> The theory of consistency, stability and convergence, and the method of manufactured solutions, are in *The Mathematics Universe*, Ch. 22 §22.11 and Ch. 24 §24.9. Linearised (GUM) uncertainty propagation is in Ch. 24 §24.8 of the same book. The general verification workflow is Chapter 30.

The evidence has three layers, and each answers a different question:

- **Code verification:** does the program solve its equations at the designed order?
- **Solution verification:** how large is the discretisation error for this case?
- **Validation:** do the equations describe this board well enough for R1?

A good validation match can hide two cancelling bugs; a verified code can still solve the wrong equations.

Code verification: the manufactured solution

The manufactured solution is $\theta_m = 10\cos(\pi x/L_x)\cos(\pi y/L_y)$. Its normal derivative is zero on every edge, so it satisfies the adiabatic boundary conditions. The conductance is graded, $G(x) = G_0(1 + 0.5x/L_x)$, so that the harmonic-mean face code is exercised. The source that makes θ_m exact is computed analytically, and the error is measured as the grid is refined.

```
 """Code verification by the method of manufactured solutions (MMS)."""
import numpy as np
from core import assemble, steady

Lx, Ly, G0, beta, Hc = 0.100, 0.080, 0.055, 0.5, 21.0
a, b = np.pi / Lx, np.pi / Ly
T_m = lambda x, y: 10.0 * np.cos(a * x) * np.cos(b * y)   # dT/dn = 0 on edges
G_f = lambda x: G0 * (1.0 + beta * x / Lx)                # graded copper

def source(x, y):        # q = -div(G grad T) + H T for the manufactured T
    Tx = -10.0 * a * np.sin(a * x) * np.cos(b * y)
    return -(G0 * beta / Lx) * Tx + G_f(x) * (a**2 + b**2) * T_m(x, y) + Hc * T_m(x, y)

prev = None
for n in (25, 50, 100, 200, 400):
    nx, ny = n, int(0.8 * n)
    dx, dy = Lx / nx, Ly / ny
    X, Y = np.meshgrid((np.arange(nx) + 0.5) * dx, (np.arange(ny) + 0.5) * dy)
    K = assemble(G_f(X), np.full(X.shape, Hc), dx, dy)
    err = steady(K, source(X, Y), dx, dy) - T_m(X, Y)
    e2 = np.sqrt(np.mean(err**2))
    p = "" if prev is None else f"  order {np.log2(prev / e2):.2f}"
    print(f"nx={nx:4d}  dx={dx*1e3:5.2f} mm  L2 error={e2:.3e} K{p}")
    prev = e2
```

```
 nx=  25  dx= 4.00 mm  L2 error=8.010e-03 K
nx=  50  dx= 2.00 mm  L2 error=2.001e-03 K  order 2.00
nx= 100  dx= 1.00 mm  L2 error=5.002e-04 K  order 2.00
nx= 200  dx= 0.50 mm  L2 error=1.250e-04 K  order 2.00
nx= 400  dx= 0.25 mm  L2 error=3.126e-05 K  order 2.00
```

The observed order is 2.00 on every refinement, which closes R5. A test is only as good as its power to fail, so the test was itself tested by *mutation*. The harmonic mean was replaced with the left cell's value (`hm = lambda a, b: a`). The script then printed orders of 1.29, 1.17, 1.09 and 1.05, with errors 3 to 29 times larger. A test on uniform copper would have missed this bug entirely, because both versions agree when G is constant. Figure 39.4 shows both slopes.

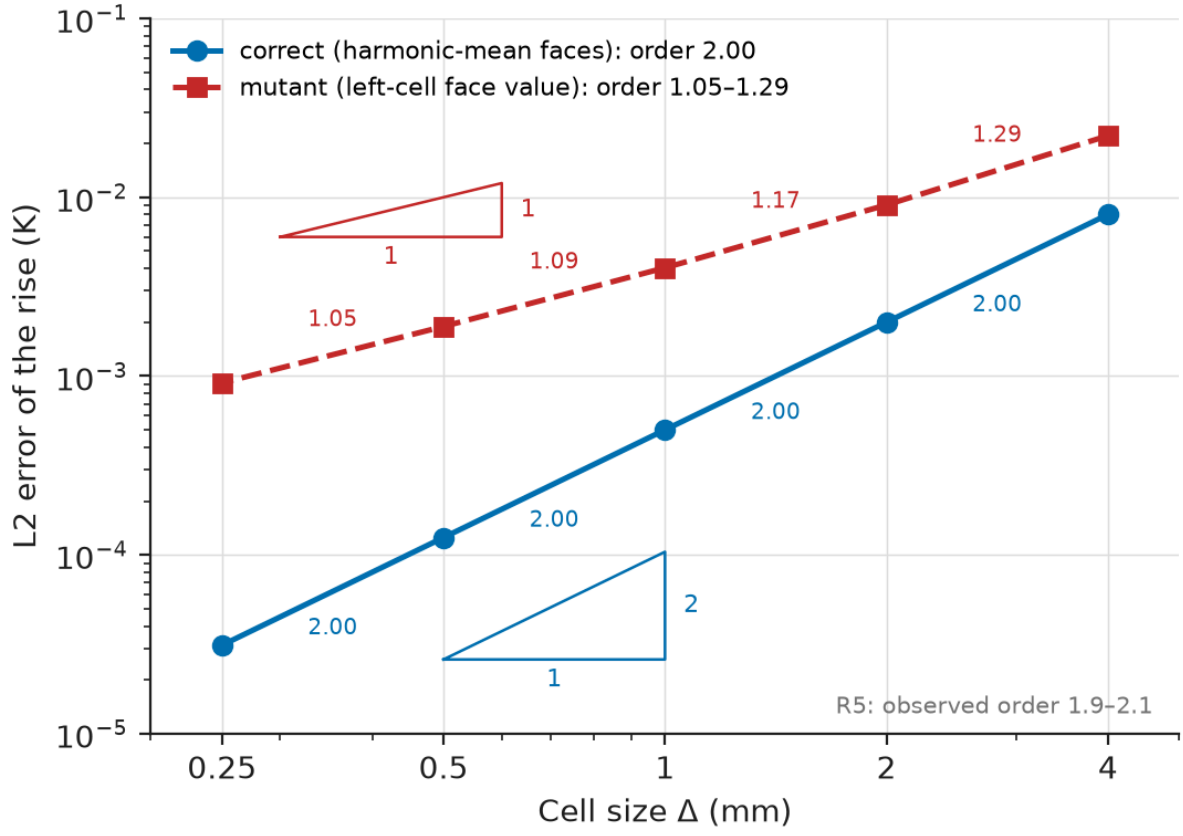


Figure 39.4 — Manufactured-solution convergence of pcbtherm: correct code against a mutated face average.

> **Status of the principle: Observed order of accuracy as a code test**
> *Derived.* For a consistent, stable scheme of order p, the error on a smooth solution falls as Δ^p in the asymptotic range. If the observed order matches the design order to ±0.1 over three or more refinements, every term of the discrete operator, including boundaries and variable coefficients, is implemented as derived.

Every commit also checks energy closure (R6, 6.4×10^{-13}), the lumped limit (uniform heating gives $\theta = q''/H$ to 10^{-12} and follows $1 - e^{-t/\tau}$), symmetry and linearity.

Solution verification: grid convergence

Worked example — Grid convergence index for the Q1 pad rise. The pad-mean rises on the 0.5, 0.25 and 0.125 mm grids are $f_3 = 32.5949$, $f_2 = 32.5698$ and $f_1 = 32.5635$ K.

- The observed order is $p = \ln[(f_3 - f_2)/(f_2 - f_1)]/\ln 2 = \ln(0.0251/0.0063)/\ln 2 = 1.99$.
- The Richardson-extrapolated value is $f_{ext} = f_1 + (f_1 - f_2)/(2^p - 1) = 32.561$ K.
- The production grid (0.5 mm) is the coarse member of the first pair. Its coarse-grid index is $1.25\,|f_3 - f_2|\,2^p/(2^p - 1) = 1.25 \times 0.0251 \times 3.97/2.97 = 0.042$ K, or 0.13 %.
- The direct estimate $f_3 - f_{ext} = 0.034$ K lies inside that band.

Discretisation error is therefore negligible against ±3 K. The 0.5 mm grid stands, and the error budget carries 0.04 K for it.

The first attempt failed. Grids of 1.0, 0.5 and 0.25 mm gave 32.316, 32.595 and 32.570 K, non-monotonic. On the 1 mm grid the pad edges at 27.5 and 32.5 mm fall on cell *centres*, so the footprint became 6 mm wide: each grid solved a different source. The validator's edge-alignment warning dates from this failure.

> **Status of the principle: Grid convergence index with safety factor 1.25**
> *Empirical.* Roache set the factor 1.25 for three-grid studies from a large body of CFD cases. It covers roughly 95 % of observed discretisation errors when the observed order is near the formal order. It is not a bound, and it applies only in the asymptotic range, which the observed order of 1.99 confirms here.

Validation against the reference board

Five Rev 2 boards carried 36 AWG type T thermocouples, epoxy-bonded at the five positions of Figure 39.2, and lay horizontal in a draught-shielded box at 23.4 °C. Power was measured electrically to ±2 % (standard uncertainty) and temperatures logged at 1 s.

Before any comparison, H was *calibrated once* on a heater-only coupon of the same size, which carries a distributed 3.4 W resistive film. Its mean rise gives $H = P/(A\bar{\theta}) = 21.0$ W/(m² K) ±5 %. That value was frozen for every prediction, so the validation below is not a fit. Keeping calibration data separate from validation data is Chapter 33's subject.

Table 39.7 — Error budget for the predicted TC1 rise (32.4 K), standard uncertainties.

Source	Input uncertainty	Normalised sensitivity	Contribution (K)
Film coefficient H (calibrated)	5 %	−0.544	0.88
Copper thickness (±10 %, rectangular)	5.8 %	−0.314	0.59
Dissipated power	2 %	+1.000	0.65
FR-4 in-plane conductivity (±50 %, rectangular)	29 %	−0.008	0.08
Discretisation (GCI, 0.5 mm)	—	—	0.04
Thermocouple and logger (±0.5 K limit)	—	—	0.29
Thermocouple position (±1 mm on the tab gradient)	—	—	0.40
Combined, u_{val}			**1.34**

Sensitivities come from re-running the tool with one input perturbed at a time. Calibration is what makes R1 testable: a handbook H known to ±20 % would contribute 3.5 K alone.

Table 39.8 — Validation results, open bench, steady state at 60 min (mean of five boards; ± is board-to-board standard deviation).

Location	Predicted	Measured	E = measured − predicted	Limit	Result
TC1, Q1 tab	32.4 K	35.1 ± 0.5 K	+2.7 K	±3.2 K (10 %)	Pass, margin 0.5 K
TC2, Q2 tab	30.6 K	33.2 ± 0.5 K	+2.6 K	±3.1 K (10 %)	Pass, margin 0.5 K
TC3, shunt	20.7 K	21.4 ± 0.4 K	+0.7 K	±3.0 K	Pass
TC4, upper right	15.8 K	15.6 ± 0.4 K	−0.2 K	±3.0 K	Pass
TC5, corner	19.7 K	19.3 ± 0.4 K	−0.4 K	±3.0 K	Pass
Time to 63 %, TC1	99 s	110 ± 5 s	−10 %	±15 %	Pass
Time to 63 %, TC5	187 s	205 ± 5 s	−9 %	±15 %	Pass

Worked example — Is the remaining difference a model error? Following ASME V&V 20, the comparison error E is set against $u_{val} = 1.34$ K.

- At TC1, $|E| = 2.7$ K is just above $2u_{val} = 2.68$ K. At the 95 % level, input, numerical and measurement uncertainty cannot explain it, so a model-form error is indicated.
- Its sign and size match the −2.1 K difference between pcbtherm and the 3-D model in Table 39.6, which is the merged-layer bias.
- Away from the sources, $|E| \leq 0.7$ K, well inside $2u_{val}$. No model error is detectable there.

The report records that Rev A meets R1 everywhere, with a declared 2–3 K under-prediction at source pads that the margin must absorb until Revision B.

The housing case was checked with the calibrated housing value $H = 10$ W/(m² K). TC1 was predicted at 55.1 K and measured at 57.6 K, with the same sign of bias. The decision was then taken on the tool:

1. Q2 was moved out to 30 mm centre spacing. The tool predicts this lowers the housing TC1 rise to 51.4 K.
2. At 85 °C ambient, with the declared bias added, the tab reaches 85 + 51.4 + 2.6 = 139.0 °C.
3. The junction-to-tab drop adds 1.2 K/W × 1.5 W = 1.8 K, giving a junction of about 140.8 °C. That is 9.2 K below the 150 °C derated limit.
4. The heat sink and the third MOSFET were both dropped.

One confirmation board measured 53.8 K at TC1 in the housing. That is +2.4 K, inside the declared bias.

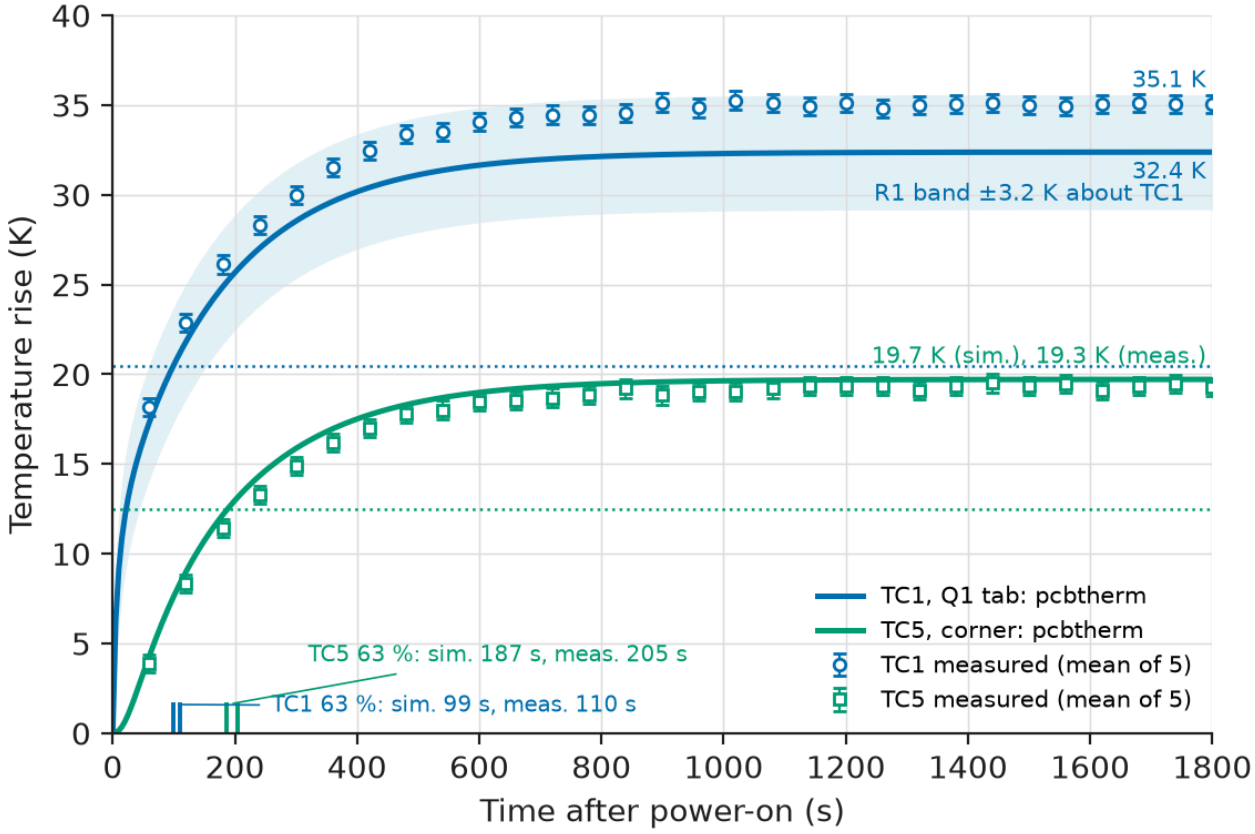


Figure 39.5 — Warm-up transient at the Q1 tab (TC1) and the board corner (TC5): pcbtherm Rev A against the mean of five measured boards.

Table 39.9 — Verification of pcbtherm Rev A against its requirements.

ID	Test	Limit	Result	Pass/fail
R1	Five-thermocouple validation	±3 K or ±10 %	Worst +2.7 K (limit 3.2 K)	Pass, bias declared
R2	Time to 63 %	±15 %	−10 %, −9 %	Pass
R3	Steady benchmark, 0.5 mm	≤ 1 s	0.22 s	Pass
R4	200-variant sweep	≤ 300 s	44 s	Pass
R5	MMS order, graded copper	1.9–2.1	2.00; mutant 1.05–1.29 detected	Pass
R6	Energy closure	$\leq 1 \times 10^{-6}$	6.4×10^{-13}	Pass
R7	Seeded invalid inputs	24/24	24/24	Pass
R8	Clean-environment reproduction	Equal to 4 decimals	Equal (39.9)	Pass
R9	Report inspection	Map, CSV, assumptions	Present	Pass
R10	Documentation review	Three documents approved	Approved, two actions	Pass

39.8 Documentation

A tool without documentation is trusted only by its author. pcbtherm ships three Markdown documents in its repository, versioned and released with the code (Chapter 5 for revision control, Chapter 29 for docs-as-code).

Table 39.10 — Documentation set of pcbtherm Rev A.

Document	Reader	Contents	How it stays correct
Theory manual (9 pp.)	Reviewer, maintainer	Equation and assumptions of 39.2; validity checks with formulas; solver choice; known biases with measured size	Each formula has a matching test; bias values are copied in from the V&V report by the build
User guide (6 pp.)	Layout engineer	Case-file reference; fan-board example; meaning of each warning; what not to use the tool for	Key table generated from `RANGES`; examples run as doctests in CI
V&V report (14 pp.)	Design review, quality	MMS and mutation results; grid study; benchmarks; error budget; validation tables and set-up photos	Regenerated from test outputs at each release; signed by the reviewer

Two habits keep them honest. **Limits have one source**: the user guide's range table is generated from `validate.py`. **Assumptions carry numbers**: "valid for thin boards" becomes "valid when $h_{bot} t / k_z < 0.1$; this board: 0.048", and the merged-layer assumption is listed with its measured 2–3 K bias. A reviewer can sign off a tool whose documents state its known errors.

39.9 Reproducible results

A prediction that dropped a heat sink may be questioned years later; reproducing it needs the exact input, code, library versions and result. The practice is set out in *Electronics and Computing*, Ch. 22 §22.10 and Ch. 24 §24.8. pcbtherm automates it by writing a manifest beside every result.

```
 """pcbtherm.manifest -- write what is needed to reproduce a result."""
import hashlib, json, platform
import numpy as np, scipy

def sha256(path):
    with open(path, "rb") as f:
        return hashlib.sha256(f.read()).hexdigest()[:16]

def write_manifest(case, code_files, results, out="run_manifest.json"):
    m = {
        "case": {"file": case, "sha256": sha256(case)},
        "code": {f: sha256(f) for f in code_files},
        "env": {"python": platform.python_version(), "numpy": np.__version__,
                "scipy": scipy.__version__, "os": platform.system()},
        "results": {k: round(float(v), 4) for k, v in results.items()},
    }
    with open(out, "w") as f:
        json.dump(m, f, indent=1, sort_keys=True)
    return m

def check_golden(results, golden, rtol=1e-6):
    """Regression gate: compare against stored golden values."""
    return {k: (results[k], g) for k, g in golden.items()
            if abs(results[k] - g) > rtol * max(1.0, abs(g))}

if __name__ == "__main__":
    res = {"pad_Q1_rise_K": 32.5949, "max_rise_K": 33.7183, "mean_rise_K": 20.2381}
    m = write_manifest("fan_ctrl.toml", ["core.py", "mesh.py", "validate.py"], res)
    print(m["env"]["scipy"], "regressions:", check_golden(res, {"pad_Q1_rise_K": 32.5949}))
    print("regressions:", check_golden(res, {"pad_Q1_rise_K": 32.3162}))
```

```
 1.18.1 regressions: {}
regressions: {'pad_Q1_rise_K': (32.5949, 32.3162)}
```

The second call shows the gate firing. 32.3162 K is the pad value from the misaligned 1 mm grid of Section 39.7. If a mesher change ever brought it back, CI would fail and print both numbers. The hash covers the file's bytes, so `.gitattributes` forces LF line endings on `*.toml` files.

Golden values are stored for six reference cases. The relative tolerance is 10^{-6}. That is far above the 10^{-12}-scale scatter between BLAS builds and CPUs on this well-conditioned problem, and far below the 0.01 K that matters in engineering. Bit-for-bit equality was rejected: a gate that fails on harmless platform changes teaches people to ignore it.

> **Status of the principle: Tolerance-based regression on golden values**
> *Empirical.* A tolerance set orders of magnitude above the observed platform scatter, and orders below any engineering-significant change, separates refactoring noise from changes in behaviour. The window has to be measured again for an ill-conditioned problem.

R8 was verified the way a sceptical reviewer would do it. On a second machine running another operating system, the team built a fresh environment from the lock file (exact versions with hashes), checked out tag `v1.0.0` and re-ran the archived case. The case and code hashes matched, all results agreed to the four stored decimals, and the full-precision values differed by at most 3×10^{-13} K. Chapter 29 shows how to turn this into one CI command.

39.10 Project close-out: cost, test results, failure analysis and Revision B

Bill of materials and indicative cost

Table 39.11 — Bill of materials, pcbtherm Rev A development and validation (2026 indicative prices).

Item	Qty	Unit	Cost
Python, NumPy, SciPy, Matplotlib (BSD licences)	—	$0	$0
Hosted CI minutes	12 months	$8	$96
Validation boards, Rev 2, assembled	5	$36	$180
Heater-only calibration coupons	2	$45	$90
Type T thermocouples 36 AWG, epoxy, tape	30	$4.53	$136
Thermocouple logger (lab stock, cross-charged) and draught box	1	$210	$210
3-D FEM reference run (consultant, 1 day)	1	$1,200	$1,200
Engineering time: tool, tests, documents	120 h	$85	$10,200
Validation testing and report	40 h	$85	$3,400
Total			**$15,512**

Engineering time is 88 % of the cost, which is why in-house tools must stay narrow. Against it:

- One avoided layout iteration is worth about $3,000.
- The dropped heat sink saves $3.80 × 40,000 units a year, or $152,000 a year.
- The commercial licence the tool replaces for everyday use costs $25,000 a year.

Test results

The test results are Tables 39.8 and 39.9. In its first three months the tool ran 1,420 cases; the validator rejected 37 (31 unit slips, 6 geometry) and warned on 112. No prediction has since been found wrong beyond its declared bias.

Failure analysis

Table 39.12 — FMEA of pcbtherm Rev A as a design tool.

Failure mode	Effect	Cause	Detection	Mitigation
Pad rise under-predicted by 2–3 K	Margin over-stated	Copper layers merged; via resistance (about 11 K/W per array) neglected	Code-to-code (−2.1 K); validation (E = +2.7 K)	Bias stated in every report; Revision B two-layer model
Units slip, e.g. metres in a mm field	Absurd or plausible-but-wrong result	Free-form numeric input	Range check (R7)	Units in key names; validity ranges; seeded tests
Footprint partly off the board	Hot spot over-predicted by up to 6 K	Placement exported from layout	Geometry check	Error stops the run
Footprint edges off cell faces	Source area changes with grid; grid study non-monotonic	Rasterisation by cell centre	Failed grid study	Edge-alignment warning; aligned grids for studies
h applied outside its case	Error grows silently	Bench h used for housing	Range and mean-rise checks	Calibrated H stored as named presets
Face-average coding error	Wrong spreading at copper boundaries	Refactoring	MMS with graded G (order 2.00 → 1.05)	MMS in CI; mutation test kept
Silent library change	Results drift between releases	Unpinned SciPy update	Golden values at 10^{-6}	Lock file; versions in manifest

The first row is the only failure verification did not prevent. The code was verified and converged; the error lay in the *equations*, in an assumption that was stated, estimated (10.9 K/W of vias), judged acceptable and then measured rather than hidden. Exposing model-form error is what V&V is for.

Revision B

Change	Why	Expected effect
Two-layer model: top and bottom sheets coupled by k_z/t per area plus a via-conductance map from the drill file	Removes the merged-layer bias	Pad error about ±0.5 K against 3-D; twice the unknowns, about 2.5× solve time, R3 still met
Copper-coverage map from Gerber layers	Real pours have keep-outs, traces and thermal reliefs	Correct spreading near connectors and isolated pads
Temperature-dependent h (natural-convection correlation plus exact radiation), Picard iteration	Constant h is valid only within about ±30 K of calibration	Domain extended to 100 K rises at 3–5 extra solves
CG + algebraic multigrid above 200,000 cells; junction output from a parts library of $R_{\theta JB}$	Direct-solver memory grows as $n \log n$; manual junction step can be skipped	Fine-pitch boards at 0.1 mm cells within 2 GB; junction margin reported directly

---

What remains open

Open question	Where it is answered
How is a tool structured so that new physics (two layers, non-linear h) is added without rewriting the core?	Chapter 31
How are sweeps, layout optimisation and parallel runs of 10,000 variants organised?	Chapter 30
How is H re-calibrated from end-of-line or field data, with its uncertainty carried into predictions?	Chapter 33
How are the CI pipeline, lock file and release tagging set up so that R8 holds for every release?	Chapter 29
How are MOSFET and shunt losses computed from the drive's operating point?	Chapter 37; *Electronics and Computing*, Ch. 29
What is the mathematics of discretisation convergence and iterative-solver rates?	*The Mathematics Universe*, Ch. 22 and Ch. 24
How is a whole project like this planned, reviewed and written up as a capstone?	Chapter 40

Next: Chapter 40 steps back from single projects to the capstone method: selecting, planning, designing, verifying, costing, risk-assessing and reporting a complete engineering project, worked through on one capstone from problem selection to final report.

Chapter 40 — Capstone Engineering Project

Part IX · Interdisciplinary Engineering Projects

A student team once won its department's capstone prize with a solar-powered irrigation pump controller. On the bench it was impressive: clean code, a tidy PCB, a live dashboard. It was installed on a smallholding at the end of the dry season and failed on its third day. The borehole's water level fell below the pump intake in the early afternoon. The controller had no idea this could happen, so it drove the dry pump at full speed for forty minutes until the rotor's elastomer stator scorched. Nothing in the electronics was wrong. The failure was made in week two, when the team wrote requirements from a vendor brochure instead of from an afternoon with the farmer and the borehole log. No requirement meant no design, no test and no protection.

This chapter is the capstone *method*. It ties Part I's methodology to a complete project and a complete report. It is worked end to end on the following year's capstone, which tackled the same problem: *AquaSol*, a PV-direct irrigation pump controller with tank and soil-moisture telemetry for a 0.2 ha vegetable plot, built by four students in 28 weeks on a budget of about $2,000. Each section is one stage of the method, applied to AquaSol.

> **How does a team choose a problem, learn what is already known, and turn a need into requirements, an architecture and a design that can be built and verified in one academic year?**
> **How are cost, risk and schedule handled with numbers rather than hope, and how is all of it reported so that someone else could build, test and improve the result?**

Concept	Engineering destination
Problem selection with weighted criteria	A project that is real, feasible and verifiable before any design starts
Structured literature review	Requirements and design values traced to standards, datasheets and field evidence
Requirements with verification methods	Acceptance tests agreed with the stakeholder in week 5, not week 26
Architecture trade study and interface control	Parallel work by four people without integration surprises
Detailed design calculations	Sizing, voltage margins and energy budgets that are checked, not assumed
Implementation with design reviews	PDR, CDR and TRR gates that catch errors while they are cheap
Verification and validation	Bench tests against limits, then a field trial with the user
Cost analysis	BOM, production cost and life-cycle cost per cubic metre of water
Risk assessment	Risk register, FMEA and Monte Carlo schedule risk
Final technical report	A document that lets someone else rebuild, retest and improve the design

Roadmap. Choosing the problem (40.1) → what is already known (40.2) → requirements (40.3) → architecture (40.4) → detailed design (40.5) → implementation (40.6) → verification and validation (40.7) → cost (40.8) → risk (40.9) → the final report and Revision B (40.10).

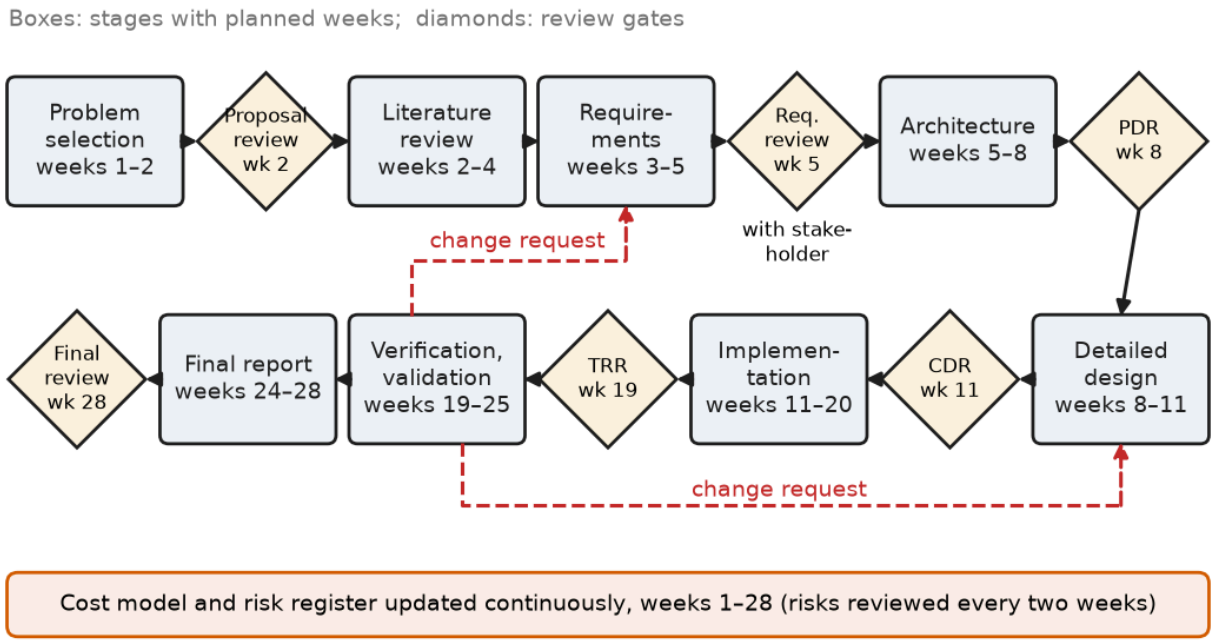


Figure 40.1 — The capstone lifecycle used in this chapter, with review gates and planned weeks.

40.1 Problem selection

> **Boundary.**
> Requirements and traceability are Chapter 1's method, architecture and trade-offs Chapter 2's, test planning and V&V Chapter 4's, documentation and revision control Chapter 5's. This chapter applies them to one project and adds selection, review, cost, risk and the report.

What makes a good capstone problem

A good capstone problem is **real** (someone outside the team has the need and can be asked), **crosses disciplines**, is **feasible** (about 1,344 person-hours, four students × 28 weeks × 12 h, must reach a verified prototype), is **verifiable** (success is a measurable number) and is **safe enough** to build under supervision.

The team considered three candidates. (A) A PV-direct irrigation pump controller with telemetry, for a farmers' cooperative the department already works with. (B) A temperature logger and alarm for vaccine cold boxes, for a district health office. (C) A battery-management board for a local e-bike conversion workshop.

Worked example — A weighted decision matrix and its sensitivity. Each candidate is scored from 1 to 5 against five weighted criteria. For the safety criterion, a higher score means lower risk.

Table 40.1 — Problem-selection matrix.

Criterion	Weight	A: irrigation controller	B: cold-box monitor	C: e-bike BMS
Real need, stakeholder access	0.25	5	4	3
Disciplinary breadth	0.20	5	3	4
Feasibility in 1,344 h and $2,000	0.25	3	5	4
Verifiability of success	0.15	4	4	5
Safety and regulatory risk (5 = low)	0.15	4	3	2
Weighted score	1.00	**4.20**	**3.90**	**3.60**

A wins: $0.25 \times 5 + 0.20 \times 5 + 0.25 \times 3 + 0.15 \times 4 + 0.15 \times 4 = 4.20$. The ranking is not robust, though. Moving 0.10 of weight from breadth to feasibility (0.10 and 0.35) gives A 4.00 and B 4.10, and B wins. A decision that flips under a plausible change of weights is telling the team where its real risk lies, here in feasibility. The team kept A and **bought down the feasibility risk by descoping**. The pump and PV modules are purchased parts. The team designs and builds the controller, the sensing and the telemetry, and the hydraulics come from the cooperative's existing borehole and 10 m³ tank.

> **Status of the principle: Weighted-sum selection with a sensitivity check**
> *Empirical.* The weights encode judgement, so the scores alone carry no authority. The method is useful because it makes the judgement explicit and shows which weight changes would reverse the choice. A choice that survives ±0.1 on every weight is robust; one that does not names the risk to manage first.

The problem statement

The statement agreed with the cooperative at the proposal review (week 2) has four parts:

- **Need:** irrigate 0.2 ha of vegetables in the dry season without a petrol pump, whose fuel costs about $300 a year and which requires someone to start and stop it every day.
- **User:** one farmer with a smartphone and no electrical training. Maintenance is done by the cooperative's technician.
- **Constraints:** an existing 12 m-static-head borehole and a 10 m³ elevated tank. No grid connection. A production cost under $1,900 per farm.
- **Success:** enough water on the worst month's clear day; no damage from a dry well or a full tank; the farmer sees tank level and pump state on a phone; cost per cubic metre no worse than petrol.

40.2 Literature review

What a design-oriented review is for

An engineering literature review exists to *change the design*. Each source should yield a requirement, a design value, a test method or a known failure; the team's review matrix had exactly those four columns, and a source that filled none was dropped.

Table 40.2 — Source classes and what each contributes.

Source class	Examples for AquaSol	Typically yields
Standards	IEC 62253 (PV pumping systems, design qualification and performance); IEC 61702 (rating of direct-coupled PV pumping systems); IEC 62109-1 (safety of PV power converters); IEC 60364-7-712 (PV installations); IEC 60529 (IP codes); ETSI EN 300 220 (868 MHz short-range devices)	Test methods, limits, safety requirements
Agronomy references	FAO Irrigation and Drainage Paper 56 (crop evapotranspiration)	Water demand per day and per month
Datasheets and application notes	PV module, helical-rotor pump curves, MOSFETs, sensorless BLDC control notes, LoRa transceiver	Ratings, efficiencies, reference circuits
Field reports and prior projects	Cooperative's borehole log; the previous team's failure report; published surveys of PV-pump installations	Failure modes: dry running, lightning, theft, cable faults
Stakeholder interviews	Two site visits, farmer and technician	Usage, maintenance skills, what "works" means
Existing products	Commercial PV-pump kits with integrated controllers	Benchmark cost and features; what not to reinvent

Table 40.3 — Extract of the AquaSol review matrix.

Source	Requirement	Design value	Test method	Known failure
Borehole log (cooperative)	Must survive drawdown below intake (R3)	Static level 12 m, drawdown 3 m at 2 m³/h, recovery about 30 min	—	Dry running in late afternoon
FAO-56	Daily water demand (R1)	Peak crop ET 6 mm/day for tomato, mid-season	—	—
IEC 62253	Daily-volume performance test (R1)	—	Volume vs in-plane irradiation over full days	—
PV module datasheet and *Electrical Energy and Machines* Ch. 15 §15.1	Cold-morning open-circuit voltage (R5)	V_{OC} 40.0 V, −0.29 %/K	Measure V_{OC} at dawn	Inverter over-voltage on cold mornings
Pump datasheet	Flow and power vs head (R1)	Wire-to-water 0.40 at 20–25 m, 600 W max input	—	Stator burn when run dry
Previous team's report	Dry-run and tank-full protection (R3, R4)	—	Bench dry-run test	Day-3 field failure
IEC 60364-7-712	DC isolator, surge protection, fusing (R11)	—	Insulation test at 500 V	—

The review changed the design in three places. The borehole log produced R3: the pump must stop within 10 s of running dry and wait 30 min. The module datasheet, combined with the site's record low of 5 °C, produced the 150 V device rating (Section 40.5). And the commercial kits showed that a PV-direct pump controller with MPPT is a solved product. The team's contribution therefore had to be the integration — protection logic, telemetry, soil-moisture-driven valves and the cost case — not a better motor drive.

> **Status of the principle: A review earns its place by changing the design**
> *Empirical.* In capstone and industrial projects alike, the expensive late failures trace back to evidence that existed and was not read: a field log, a standard's test method, a previous team's report. Tracking each source against the requirement, value, test or failure it produced makes the review auditable and keeps it short.

40.3 Requirements

The requirements were written in week 3, reviewed with the farmer and technician in week 5, and frozen at that review. Each one has a number, a verification method and a source. R3 traces to the borehole log, R5 to the module datasheet and site temperature record, R10 to the cooperative's budget.

Table 40.4 — AquaSol requirements.

ID	Requirement	Target	Verification	Source
R1	Daily water on a design day (≥ 5.0 kWh/m² in-plane) at 23 m total head	≥ 13.3 m³	IEC 62253-style full-day test, normalised to 5.0 kWh/m²	FAO-56 demand
R2	Pumping hours on a design day	≥ 8 h	Field log	Farmer: tank refilled by evening
R3	Dry-run protection	Stop within 10 s; retry after 30 min	Bench test with throttled intake	Borehole log, previous failure
R4	Tank-full stop	Stop within 2 s of upper float; no overflow	Bench and 14-day field log	Technician
R5	Device voltage at cold-morning V_{OC}	≤ 80 % of MOSFET and capacitor ratings	Analysis plus dawn measurement	Datasheet, site minimum 5 °C
R6	Controller environment	IP65; −5 to 55 °C ambient	Inspection; thermal soak at 55 °C, full load, 4 h	Site
R7	Soil-node battery life at 15 min reporting	≥ 3 years	Energy budget plus measured average current	Technician visit interval
R8	Telemetry packet delivery at up to 420 m	≥ 95 % over 14 days	Gateway log	Plot geometry
R9	Status on farmer's phone	Tank level and pump state within 5 min of a change	Field demonstration	Farmer
R10	Cost	Production capex ≤ $1,900; 10-year cost per m³ ≤ petrol pump	Cost model (40.8)	Cooperative
R11	Electrical safety	DC isolator, SPD, string fuses per IEC 60364-7-712; insulation ≥ 1 MΩ at 500 V	Inspection and insulation test	Standard
R12	Delivery	Verified prototype and final report by week 28	Schedule	Department

Two things matter more than the contents. Every row has a verification method the stakeholder accepted in week 5, so the final validation was a test already agreed, not a demonstration designed afterwards. And the table is short: twelve requirements can be traced through design, test and report, whereas a capstone with 80 usually verifies none properly.

40.4 Architecture

Trade study

Three architectures could meet R1. Each was scored on capex, ten-year maintenance, efficiency and learning value. For a capstone, learning value is a legitimate criterion.

Table 40.5 — Architecture trade study.

Architecture	Energy storage	Capex estimate	Maintenance over 10 years	Assessment
(a) PV-direct DC pump, controller with MPPT, water stored in the existing tank	10 m³ tank (about 0.75 day of demand)	Lowest: no battery, no inverter	Pump overhaul about year 7	**Chosen**: the energy store already exists and is paid for
(b) PV, LFP battery, inverter, AC pump	2.6 kWh LFP for one day of autonomy	Higher by about $900 (battery and inverter)	Battery replacement around year 8–10	Rejected: cost, and pumping does not need night operation
(c) Commercial PV-pump kit plus add-on telemetry	Tank	Similar to (a)	Vendor-dependent	Rejected for the capstone: leaves only the telemetry to design

The water-tank argument settles the choice. The tank's 10 m³ at 23 m head stores $1000 \times 9.81 \times 23 \times 10/3.6 \times 10^6 = 0.63$ kWh of hydraulic energy. That is equivalent to about 1.6 kWh of PV-DC energy at 0.40 efficiency, and it is already paid for. Storing it as charge needs about 2 kWh of LFP at 80 % depth of discharge, plus an inverter.

Blocks and interfaces

Figure 40.2 shows the architecture. Four people work on it in parallel, so the interfaces were frozen in an interface control document at the PDR (week 8). Table 40.6 is its summary.

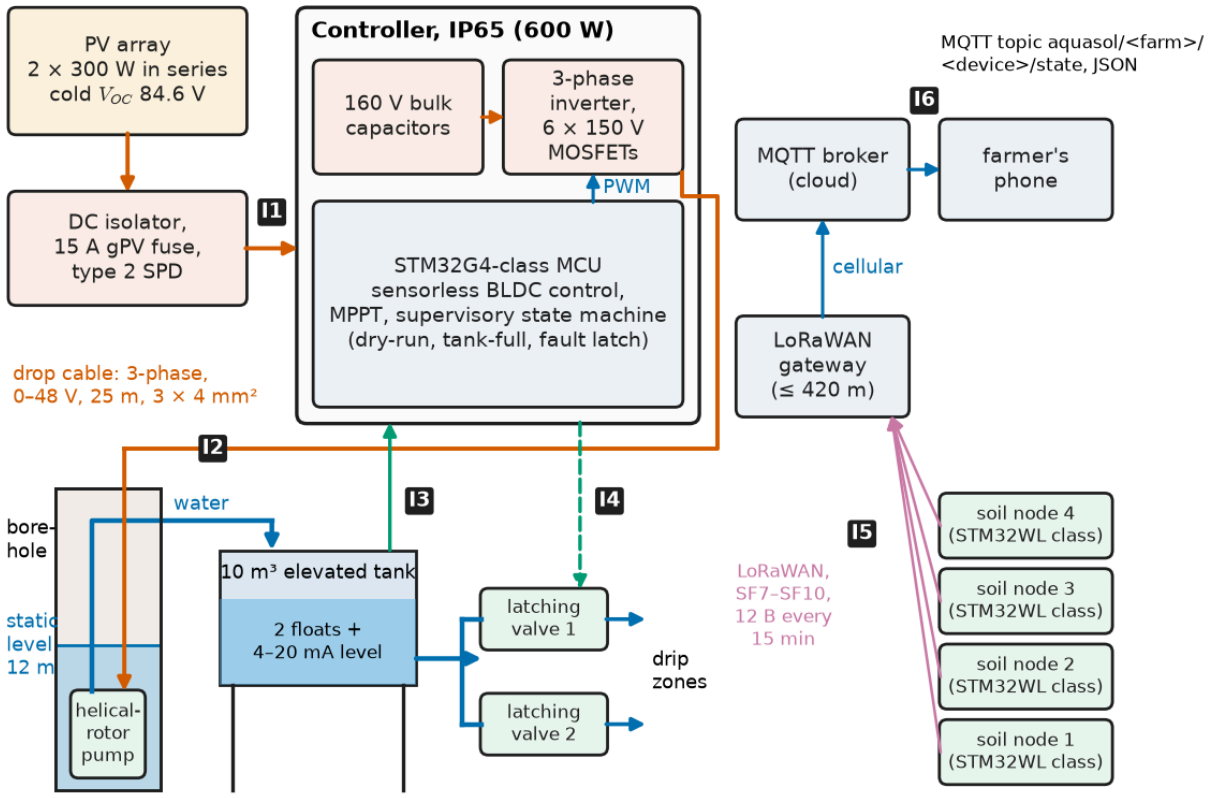


Figure 40.2 — AquaSol system architecture: PV-direct pump controller, tank sensing, soil-moisture nodes and telemetry.

Table 40.6 — Interface control summary.

ID	Interface	Signal and range	Owner
I1	PV array → controller	DC 0–85 V, ≤ 10 A; isolator, 15 A gPV fuse, type 2 SPD	Power
I2	Controller → pump	3-phase, 0–48 V line, ≤ 12 A, 25 m drop cable 3 × 4 mm²	Power
I3	Tank → controller	Two float contacts (wetting current 10 mA); 4–20 mA level, 0–2.5 m	Sensing
I4	Controller → valves	Latching solenoid, 9 V, 50 ms pulse, polarity sets open or close	Firmware
I5	Nodes → gateway	LoRaWAN EU868-class, SF7–SF10, 12-byte payload every 15 min	Telemetry
I6	Gateway → cloud → phone	MQTT over cellular; topic `aquasol/<farm>/<device>/state`, JSON	Telemetry

> **Status of the principle: Freeze interfaces before splitting the work**
> *Empirical.* Integration failures in multi-person projects concentrate at boundaries no one owns. A one-page interface table with ranges, units and an owner per row, frozen at the preliminary design review, is the cheapest known mitigation. It does not remove design errors inside blocks.

40.5 Detailed design

Water, head and PV sizing

The design-day calculation turns the requirement into an array size. Its inputs are the plot area, the peak crop evapotranspiration from FAO-56, a drip efficiency of 0.90, the total dynamic head, the wire-to-water efficiency from the pump curve, and the design month's peak sun hours with a PV derate. The derate covers temperature (0.88), soiling (0.95) and wiring and mismatch (0.97). The PV energy-yield relations and temperature coefficients are those of *Electrical Energy and Machines*, Ch. 15 §15.1, and the PV design method is Chapter 22's. The listing then simulates the design day with a half-sine irradiance profile and an 80 W pump start threshold, to check the volume that is actually delivered.

```
 """Capstone design calculation: PV-direct irrigation pump, design day."""
import numpy as np

area_m2, etc_mm, eta_drip = 2000.0, 6.0, 0.90      # plot, peak crop ET, drip eff.
head_m = 12 + 3 + 4 + 2 + 2                         # static, drawdown, stand, tank, friction
eta_w2w = 0.40                                      # PV-DC to water, pump+motor+controller
psh, derate = 5.0, 0.88 * 0.95 * 0.97               # design-month sun hours, PV derate
p_array, p_start = 600.0, 80.0                      # Wp installed, pump start threshold W

v_need = area_m2 * etc_mm / 1000 / eta_drip         # m3/day
e_hyd = 1000 * 9.81 * head_m * v_need / 3.6e6       # kWh/day
e_dc = e_hyd / eta_w2w
p_min_array = e_dc / (psh * derate)                  # kWp
print(f"water {v_need:.1f} m3/d  head {head_m} m  hydraulic {e_hyd:.3f} kWh/d")
print(f"DC energy {e_dc:.2f} kWh/d  minimum array {p_min_array*1000:.0f} Wp")

# Clear design day: half-sine irradiance scaled so that the day gives `psh`
t = np.linspace(6, 18, 721)                          # h, 1-min steps
g = np.sin(np.pi * (t - 6) / 12); g *= psh / np.trapezoid(g, t)   # kW/m2
p_dc = p_array * derate * g                          # W at the controller input
p_pump = np.where(p_dc >= p_start, p_dc, 0.0)
q = eta_w2w * p_pump / (1000 * 9.81 * head_m)        # m3/s
v_day = np.trapezoid(q, t * 3600)
on = t[p_pump > 0]
print(f"peak DC {p_dc.max():.0f} W  pumping {on[0]:.2f}-{on[-1]:.2f} h")
print(f"design-day volume {v_day:.1f} m3  margin {100*(v_day/v_need-1):.0f} %")
print(f"peak flow {q.max()*3.6e6/1000:.2f} m3/h")
```

```
 water 13.3 m3/d  head 23 m  hydraulic 0.836 kWh/d
DC energy 2.09 kWh/d  minimum array 515 Wp
peak DC 318 W  pumping 6.98-17.02 h
design-day volume 15.0 m3  margin 13 %
peak flow 2.03 m3/h
```

Worked example — From crop to array. The plot needs $2000 \times 6/1000/0.90 = 13.3$ m^3/day. Lifting it 23 m takes $1000 \times 9.81 \times 23 \times 13.3/3.6 \times 10^6 = 0.836$ kWh of hydraulic energy. At 0.40 wire-to-water efficiency that is 2.09 kWh of PV-DC energy. With 5.0 peak sun hours and a 0.811 derate the minimum array

is $2.09/(5.0 \times 0.811) = 0.515$ kWp. Two 300 W modules (600 Wp) give a 13 % volume margin on the design day once the morning and evening hours below the 80 W start threshold are lost.

The half-sine profile peaks at only 318 W because it spreads 5 kWh/m² over 12 h. On a clear day with 1 kW/m² at noon, the array would deliver up to about $600 \times 0.811 = 487$ W. The controller and pump are therefore rated for 600 W input. Sizing the volume on the design day and the power rating on the clear-sky peak is the standard split. Figure 40.3 shows the design day.

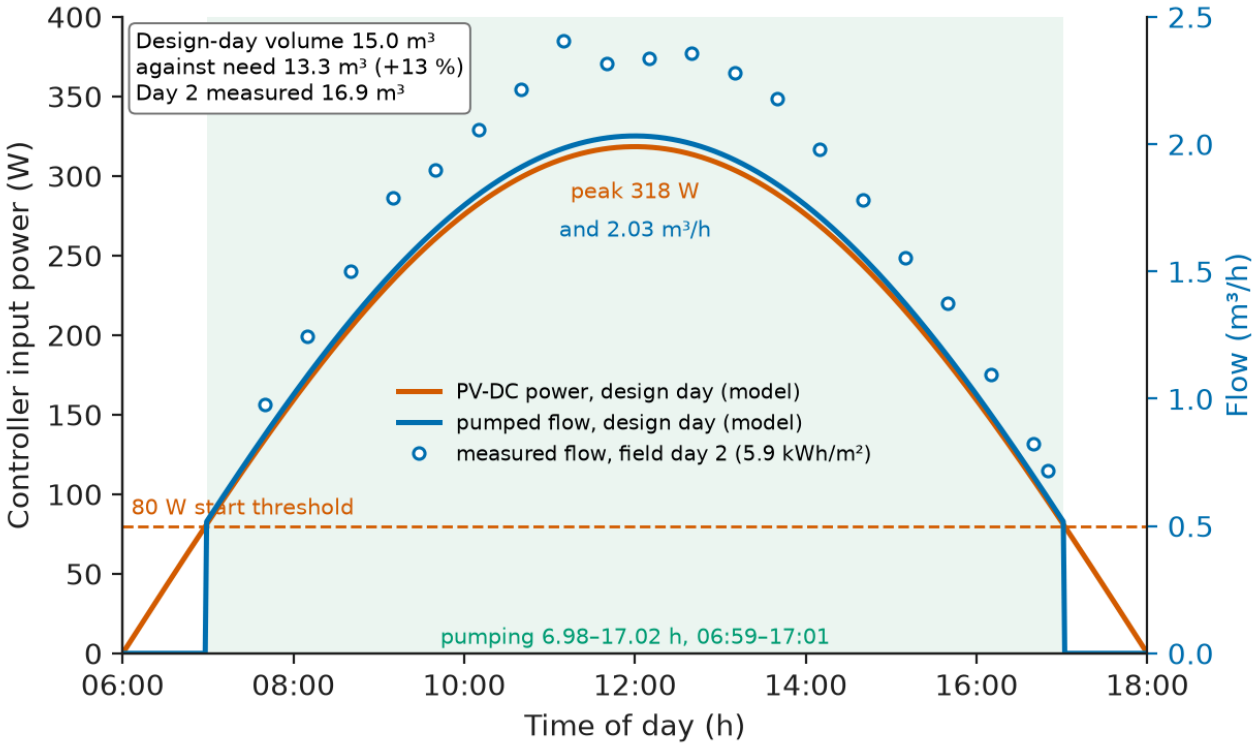


Figure 40.3 — AquaSol design day: controller input power and pumped flow from the sizing model, with one measured field day.

Voltage margins on a cold morning

Worked example — Choosing the MOSFET voltage class. The module's open-circuit voltage is 40.0 V at 25 °C with a coefficient of −0.29 %/K. At dawn on the coldest recorded morning the cells sit at the 5 °C ambient with enough light to reach open circuit. So $V_{OC} = 40.0 \times [1 - 0.0029 \times (5 - 25)] = 42.32$ V per module, and 84.6 V for the series pair. Ratings are derated to 80 % (R5):

- A 100 V MOSFET allows 80 V. That is below 84.6 V, so it is rejected.
- A 150 V MOSFET allows 120 V, a 42 % margin above the cold V_{OC}, with room for switching overshoot. It is chosen.
- The bulk capacitors follow the same rule. 100 V parts would run at 85 % of rating, so 160 V electrolytics are used.

The string-sizing rule is that of *Electrical Energy and Machines*, Ch. 15 §15.1, applied to a 2-module string rather than a 1,500 V one. At 66 V maximum-power voltage, the conduction loss of a 150 V, 10 mΩ-class device at 10 A is about 1–2 W per conducting device, allowing for $R_{DS(on)}$ rising when hot. That is acceptable without a heat sink in the 55 °C soak test (R6).

Component selection

Table 40.7 — Component selection for the main parts.

Function	Candidates	Deciding numbers	Choice
Pump	DC centrifugal; diaphragm; helical rotor (progressive cavity)	Centrifugal efficiency collapses at 23 m and low flow; diaphragm limited to about 1 m³/h; helical rotor 0.40 wire-to-water at 2 m³/h	Helical-rotor 48 V BLDC submersible, 600 W
Controller MCU	8-bit AVR; STM32G431-class Cortex-M4F; ESP32-class	Sensorless BLDC needs fast ADC triggered from PWM; Wi-Fi not needed on site	STM32G431-class, about $4.50
Inverter switches	100 V, 60 V, 150 V MOSFETs	Cold V_{OC} 84.6 V with 80 % derating	150 V, about 10 mΩ, TOLL or TO-220
Soil-node radio	Cellular per node; IEEE 802.15.4 mesh; LoRa (SX1262-class or STM32WL SoC)	420 m with foliage; 3-year battery; per-node cost	STM32WL-class LoRa SoC
Soil sensor	Resistive probe; capacitive probe; tensiometer	Resistive corrodes within a season; tensiometer needs maintenance	Capacitive, calibrated per soil

The motor-drive power stage and its sensorless control are not re-derived here. BLDC operation is in *Electrical Energy and Machines*, Ch. 8 §8.2. Renewable-fed converters and MPPT are in *Electronics and Computing*, Ch. 32 §32.8. The closed-loop drive project that builds such a stage is Chapter 37.

Energy budget of a soil node

Worked example — Will a soil node last three years? Each 15-min cycle has four parts:

- The capacitive probe draws 8 mA for 0.3 s (2.4 mC).
- The MCU is active at 3 mA for 0.2 s (0.6 mC).
- The radio transmits a 25-byte LoRaWAN frame at +14 dBm, drawing 45 mA. At SF9 and 125 kHz the frame lasts 206 ms (9.26 mC).
- The two receive windows draw 6 mA for 0.1 s (0.6 mC).

That totals 12.9 mC per 900 s, or 14.3 µA. Adding 2 µA of sleep current gives 16.3 µA average. An ER14505 lithium–thionyl-chloride cell (2.4 Ah, 85 % usable) then lasts $2.04/16.3 \times 10^{-6} = 125{,}000$ h, or 14.3 years. In practice the cell's shelf life caps this at about 10. At SF12 the airtime grows to 1.48 s and the average current to 80 µA, giving 2.9 years. That fails R7. The design therefore requires the gateway to be placed so that every node closes its link at SF10 or below. Link-budget method is in *Circuits, Signals and Communications*, Ch. 30 §30.10 and Chapter 26; node power design is Chapter 38's.

Table 40.8 — Design budgets and margins.

Budget	Requirement	Design value	Margin
Water volume, design day	≥ 13.3 m³	15.0 m³ (model)	13 %
Array size	≥ 515 Wp	600 Wp	17 %
Device voltage at cold V_{OC}	≤ 80 % of rating	84.6 V on 150 V (56 %)	42 % above cold V_{OC}
Controller input power	Clear-sky peak 487 W	600 W rating	23 %
Soil-node life at SF9	≥ 3 years	14.3 years (10-year shelf life)	> 3×
Soil-node life at SF12	≥ 3 years	2.9 years	−3 %: fails, hence SF ≤ 10 rule

40.6 Implementation

Plan, roles and reviews

The work split along the interfaces of Table 40.6: power (I1, I2), firmware (motor control, supervisory logic, I4), sensing and telemetry (I3, I5, I6), and a systems lead for requirements, integration, test plan, cost, risk and report. Each student had a named backup. Three review gates were held with the supervisor and the technician:

- **PDR (week 8):** architecture, interface control document and budgets frozen.
- **CDR (week 11):** schematics, layout, firmware design and the test plan with limits. The CDR found the 100 V MOSFETs on the first schematic, and they were changed to 150 V for $4 per board.
- **TRR (week 19):** every test procedure written, instruments calibrated, safety plan signed. No field test started before it.

Configuration management followed Chapter 5. Everything lived in one repository: KiCad project, firmware, test scripts, cost and risk models and the report source. Releases were tagged at each gate, and every test record names the tag of the hardware and firmware it tested.

Firmware: the protection logic the previous team lacked

Motor commutation and MPPT run in the PWM interrupt. The supervisory state machine runs every 100 ms from the main loop. It owns the decisions that protect the hardware: when to start, when to stop for a full tank, when the pump is running dry, and when to latch a fault. The dry-run signature is high rotor speed at low input power, which is what an unloaded helical rotor does. The state-machine design practice (explicit states, guarded transitions, timeouts) is *Electronics and Computing*, Ch. 28 §28.4. The listing is C11, and its thresholds are AquaSol's tuned values.

```
 /* pump_sup.c -- AquaSol supervisory state machine, called every 100 ms (C11).
    Illustrative: input scaling and the motor-control layer live elsewhere. */
#include <stdbool.h>
#include <stdint.h>

typedef enum { ST_IDLE, ST_RUN, ST_DRY_WAIT, ST_TANK_FULL, ST_FAULT } pump_state_t;

typedef struct {
    float v_pv;       /* array voltage, V */
    float p_in;       /* controller input power, W */
    float rpm;        /* sensorless speed estimate, r/min */
    bool  tank_high;  /* upper float closed: tank full */
    bool  tank_low;   /* lower float open: level below restart mark */
    bool  hw_trip;    /* latched overcurrent, overtemperature or driver fault */
} pump_in_t;

#define V_START   60.0f    /* V: array can carry the start-up load */
#define V_STOP    45.0f    /* V: below this the pump stalls */
#define DRY_RPM   2600.0f  /* high speed ...                            */
#define DRY_PMAX  60.0f    /* ... at low power: rotor running dry     */
#define DRY_TICKS 50u      /* 5 s of dry signature (R3: stop < 10 s)  */
#define DRY_WAIT  18000u   /* 30 min for the borehole to recover      */

static pump_state_t st = ST_IDLE;
static uint32_t n;         /* ticks spent in the current condition */

pump_state_t pump_step(const pump_in_t *in, bool *run)
{
    if (in->hw_trip) st = ST_FAULT;          /* cleared only by service reset */
    switch (st) {
    case ST_IDLE:
        if (in->tank_high)              st = ST_TANK_FULL;
        else if (in->v_pv > V_START)  { st = ST_RUN; n = 0u; }
        break;
    case ST_RUN:
        if (in->tank_high)            { st = ST_TANK_FULL; break; }
        if (in->v_pv < V_STOP)        { st = ST_IDLE; break; }
        n = (in->rpm > DRY_RPM && in->p_in < DRY_PMAX) ? n + 1u : 0u;
        if (n >= DRY_TICKS)           { st = ST_DRY_WAIT; n = 0u; }
        break;
    case ST_DRY_WAIT:
        if (++n >= DRY_WAIT)            st = ST_IDLE;
        break;
    case ST_TANK_FULL:
        if (in->tank_low)               st = ST_IDLE;
        break;
    case ST_FAULT:
        break;
    }
    *run = (st == ST_RUN);
```

```
    return st;
}
```

The dry-run detector needs 5 s of a *continuous* signature (the counter resets on any normal sample), so a gulp of air does not stop the pump. Tank-full uses two floats with hysteresis. A hardware fault latches until a technician resets it, because an overcurrent in a submersible pump usually means a damaged cable. The listing also contains a flaw that only the field found: there is no minimum off-time after a low-voltage stop. Section 40.9 shows what that did at dawn.

40.7 Verification and validation

Verification asks whether AquaSol was built right: does it meet R1–R12 with their limits? Validation asks whether the right thing was built: does the farmer get water without supervision, at an acceptable cost? Chapter 4 draws the distinction and sets out test planning. Here both were run. Verification came first, on the bench and on site. Validation followed as a 14-day trial operated by the farmer. Every instrument used for a pass/fail decision was calibrated or checked against a reference before the TRR. The flow meter, for example, was checked against timed fills of a 200 L drum to ±2 %.

Table 40.9 — Verification results.

ID	Test	Limit	Result	Pass/fail
R1	Full-day volume, 3 clear days, normalised to 5.0 kWh/m²	≥ 13.3 m³	14.3 m³ (mean of 3 days)	Pass, 7 % margin
R2	Pumping hours, day 2	≥ 8 h	9.2 h (07:40–16:50)	Pass
R3	Intake throttled to dry-run, 10 trials	Stop ≤ 10 s; retry at 30 min	5.2–5.6 s; retry at 30:00 min	Pass
R4	Upper float forced, 10 trials; 14-day log	Stop ≤ 2 s; no overflow	0.3 s; 0 overflows	Pass
R5	Dawn V_{OC} at 7 °C module temperature	≤ 120 V (80 % of 150 V)	83.9 V measured (84.2 V predicted)	Pass
R6	Thermal soak 55 °C, 600 W, 4 h; IP65 inspection	Hottest MOSFET case ≤ 110 °C	96 °C	Pass
R7	Node average current, 24 h, SF9	≤ 77 µA (3-year life)	17.4 µA, 13.4-year estimate	Pass
R8	Packet delivery, 4 nodes, 14 days	≥ 95 %	98.7 % (worst node 97.1 % at 420 m, SF10)	Pass
R9	Status on phone after tank change	≤ 5 min	15–55 s	Pass
R10	Cost model (40.8)	≤ $1,900; LCOW ≤ petrol	$1,838; $0.159 vs $0.160 per m³	Pass, marginal
R11	Inspection; insulation at 500 V DC	≥ 1 MΩ	> 200 MΩ	Pass
R12	Final report submitted	Week 28	Week 28	Pass

Table 40.10 — Field test results, full-day volume tests (IEC 62253-style, 23 m head).

Day	In-plane irradiation (kWh/m²)	Predicted (model, η = 0.40)	Measured	Difference
1	5.6	16.95 m³	16.0 m³	−5.6 %
2	5.9	17.90 m³	16.9 m³	−5.6 %
3	5.3	15.99 m³	15.1 m³	−5.6 %
4 (overcast)	3.1	8.80 m³	8.2 m³	−6.8 %

The model over-predicted by a consistent 5.6 % on clear days. A consistent error is a parameter error, not a model-form error. Back-calculating gives a wire-to-water efficiency of 0.38 against the datasheet's 0.40. With 0.38 the model predicts 16.1, 17.0 and 15.2 m³, within 0.1 m³ of the measurements, and 14.3 m³ for the 5.0 kWh/m² design day. That is the R1 figure. The overcast day falls about 2 % below even the corrected model, as expected from more time below the start threshold. Recording the corrected efficiency in the design file closes the loop between design and test. Chapter 33 calls this a calibrated model.

Validation. In the 14-day farmer-operated trial, daily water use averaged 12.1 m³. That is below the 13.3 m³ design need, because the soil-moisture logic skipped irrigation on a zone that was still wet. The farmer's daily irrigation work fell from about 2.5 h to about 0.3 h. The tank never overflowed, and the dry-run protection operated twice, on the two hottest afternoons, exactly as the borehole log predicted. The farmer's acceptance was written against R1–R4 and R9, the requirements agreed in week 5.

40.8 Cost analysis

Bill of materials

Table 40.11 — Prototype bill of materials (2026 indicative prices, single quantities).

Item	Cost	Share
PV modules, 2 × 300 W	$150	7.7 %
Mounting frame, galvanised, local fabrication	$120	6.2 %
48 V helical-rotor BLDC pump, 600 W	$520	26.8 %
Controller boards × 3 (two spins and a spare), parts and PCB	$285	14.7 %
Enclosure IP65, DC isolator, SPD, fuses, glands	$110	5.7 %
Tank level: two floats and 4–20 mA transducer	$83	4.3 %
Pulse flow meter, 25 mm	$45	2.3 %
Soil-moisture nodes × 4	$164	8.5 %
LoRaWAN gateway, indoor 8-channel	$160	8.3 %
Latching solenoid valves × 2	$56	2.9 %
Cables, pipe and fittings	$185	9.5 %
Consumables	$60	3.1 %
Total	**$1,938**	100 %

The prototype cost $1,938. Student time, about 1,350 h, is not charged, but the report states it. A production unit for one farm needs one controller board ($95) rather than three, and no consumables. That brings the hardware to $1,688, and installation adds about $150, for a capex of $1,838 against the $1,900 target of R10. The pump is the largest single item (27 %), followed by the controller boards. The telemetry chain (gateway and nodes) is 17 %, and that matters below.

Life-cycle cost and cost per cubic metre

Worked example — Is solar cheaper than petrol? Both options are compared over 10 years at an 8 % discount rate, delivering 13.3 m³ a day on 180 irrigation days a year, or 2,394 m³ a year.

Petrol pump. $250 to buy and replaced in years 4 and 8. Fuel is 1.1 L a day × 180 days × $1.20 = $238 a year, plus $60 maintenance, for $297.60 a year.

AquaSol. $1,838 capex. Running costs are $30 a year of maintenance, $60 a year of cellular backhaul for the gateway, and a $200 pump overhaul in year 7.

The levelised cost of water is the discounted cost divided by the discounted water, 16,064 m³:

- Petrol: $2,566, or $0.160 per m³.
- AquaSol as designed: $2,559, or $0.159 per m³.

R10 passes, but only just. The discounted break-even comes in year 10.

The cost model then answered a question nobody had asked. Removing the gateway and backhaul from the per-farm cost lowers the levelised cost by a fifth:

Variant	Capex	Discounted 10-year cost	Cost per m³	Discounted break-even
Petrol pump	$250	$2,566	$0.160	—
AquaSol as designed	$1,838	$2,559	$0.159	Year 10
Gateway shared by 10 farms	$1,694	$2,052	$0.128	Year 8
Pumping only, no telemetry	$1,514	$1,832	$0.114	Year 6

The variant with no telemetry is the cheapest, but it loses R3's remote alarm and R9. The cooperative chose the shared gateway: one gateway on the cooperative's office roof covers ten farms within about 2 km. That became the main change in Revision B. Figure 40.4 shows the cumulative discounted costs. The economic-evaluation method follows Chapter 25 and the solar-battery project's assessment in Chapter 36.

> **Status of the principle: Levelised cost as the comparison basis**
> *Derived.* Dividing discounted life-cycle cost by discounted delivered quantity compares options with different capex and running costs on one number. The result depends directly on the discount rate, the lifetime and fuel-price assumptions, so the report states them and shows the break-even year as well as the ratio.

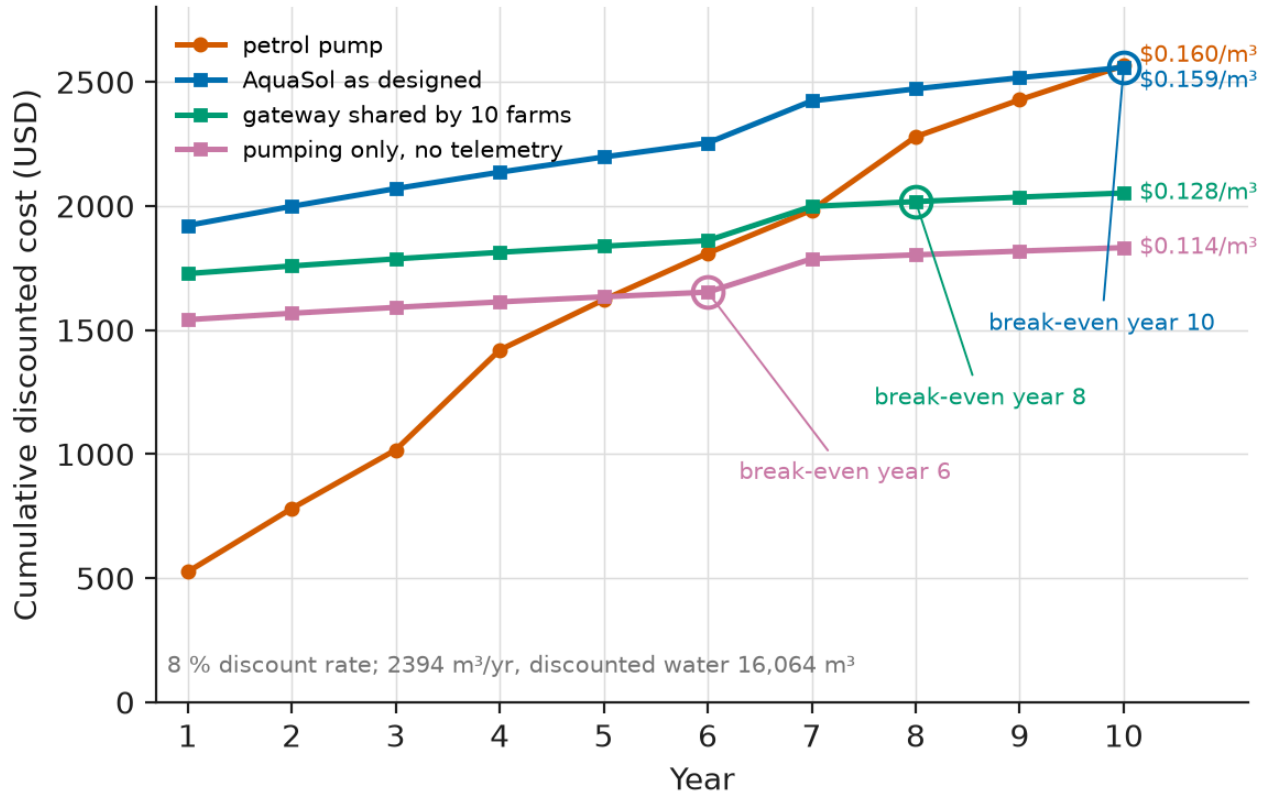


Figure 40.4 — Cumulative discounted cost over 10 years: petrol pump against three AquaSol variants (8 % discount rate).

40.9 Risk assessment

Risk register

Risks were scored for likelihood and impact on a 1–5 scale each, at week 2, and reviewed every two weeks. A score of 12 or more needs a named mitigation and an owner.

Table 40.12 — Extract of the risk register (scores at week 2 → after mitigation).

Risk	L × I	Mitigation	Owner	After
Controller PCB needs a second spin, delaying integration	4 × 4 = 16	Integrate firmware on a commercial BLDC evaluation board while the PCB is built	Power, firmware	3 × 2 = 6
Pump damaged during tests (dry running, wiring error)	3 × 5 = 15	Dry-run logic tested on a bench rig with a cheap spare pump first; current-limited supply	Firmware	2 × 3 = 6
Cold-morning over-voltage destroys inverter	2 × 5 = 10	150 V devices (40.5); dawn V_{OC} measured before connecting	Power	1 × 5 = 5
Gateway placement leaves a node at SF12 (fails R7)	3 × 3 = 9	Site survey with a test node before installing	Telemetry	2 × 2 = 4
Electric shock or arc during PV work	2 × 5 = 10	DC isolator, lock-out procedure, supervised PV wiring, insulation test before energising	Systems lead	1 × 5 = 5

Electrical safety on site follows *Electrical Energy and Machines*, Ch. 29 §29.6 (isolation and lock-out) and §29.7 (safe testing). A PV string cannot be switched off by the sun, so the DC isolator and the lock-out procedure are what make "de-energised" true.

Schedule risk

A plan built from most-likely durations is almost always late, because the duration distributions are skewed. The team estimated each task as a triangular distribution (optimistic, most likely, pessimistic) and simulated the plan. Monte Carlo propagation is covered in *The Mathematics Universe*, Ch. 24 §24.6.

```
 """Schedule risk of the capstone plan: Monte Carlo over triangular estimates."""
import numpy as np

rng = np.random.default_rng(2026)
N = 100_000
tri = lambda lo, mode, hi: rng.triangular(lo, mode, hi, N)   # weeks

# Serial front end, then three parallel build tracks, then integration and test
front = tri(2, 3, 4) + tri(2, 2.5, 4) + tri(2, 3, 5)      # select+review, reqs, arch.
hw    = tri(4, 6, 10) + tri(1, 2, 4)                       # controller PCB, re-spin
fw    = tri(5, 7, 11)                                      # firmware
node  = tri(3, 4, 7)                                       # sensor nodes + gateway
build = np.maximum.reduce([hw, fw, node])                  # longest track governs
back  = tri(2, 3, 6) + tri(2, 3, 5) + tri(1.5, 2, 3)       # integrate, V&V, report
total = front + build + back

deadline = 28.0
mode_sum = 3 + 2.5 + 3 + max(6 + 2, 7, 4) + 3 + 3 + 2
print(f"sum of most-likely durations: {mode_sum:.1f} weeks")
print(f"mean {total.mean():.1f}  P50 {np.percentile(total, 50):.1f}  "
      f"P80 {np.percentile(total, 80):.1f}  P95 {np.percentile(total, 95):.1f} weeks")
print(f"P(finish by week {deadline:.0f}) = {np.mean(total <= deadline):.2f}")
print(f"hardware track critical in {np.mean(build == hw):.0%} of runs")
```

```
 sum of most-likely durations: 24.5 weeks
mean 27.6  P50 27.5  P80 29.1  P95 30.7 weeks
P(finish by week 28) = 0.60
hardware track critical in 76% of runs
```

Worked example — Why 3.5 weeks of slack is not enough. Adding up the most-likely durations gives 24.5 weeks, which seems to leave 3.5 weeks of slack. The simulation says otherwise. The mean is 27.6 weeks, and the chance of finishing by week 28 is only 0.60. Two effects consume the slack. Every triangular estimate is skewed to the right: tri(4, 6, 10) has mean 6.67 against its mode of 6. And the build ends when the *longest* of three parallel tracks ends, so the expected maximum exceeds the maximum of the modes. The hardware track, with its possible re-spin, is critical in 76 % of runs. The register's top mitigation is to integrate on an evaluation board, which takes the re-spin off the critical path. Removing the tri(1, 2, 4) term from the hardware track and re-running gives P50 26.3, P80 27.8 and $P(\leq 28) = 0.82$. Firmware then becomes the critical track in 71 % of runs. The project finished in week 28 with the second PCB spin running in parallel with integration, as planned.

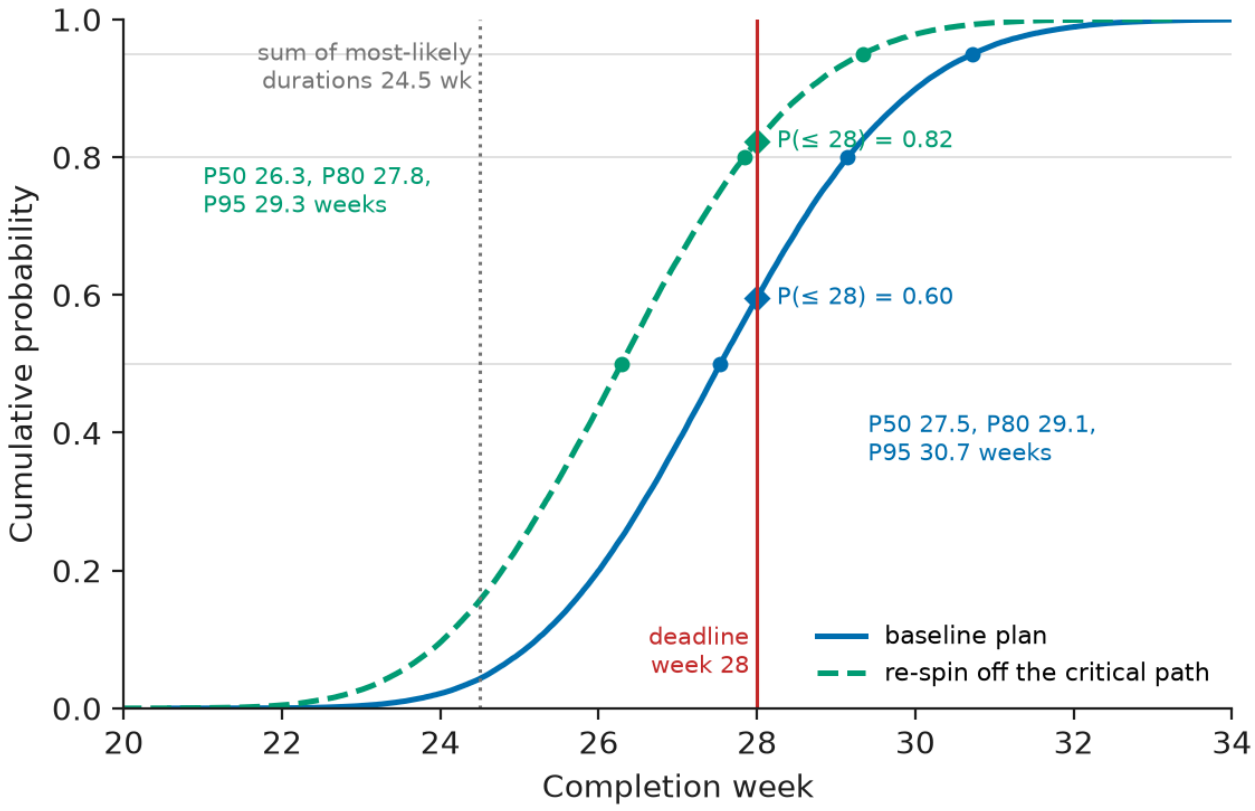


Figure 40.5 — Monte Carlo schedule risk of the AquaSol plan before and after taking the PCB re-spin off the critical path (100,000 runs each).

> **Status of the principle: Schedule risk from skewed task estimates**
> *Empirical.* Task durations are right-skewed, and merging parallel paths adds further delay. A plan's sum of most-likely durations is therefore an optimistic bound, typically beaten in fewer than half of real projects. Triangular three-point estimates with Monte Carlo give a usable completion distribution, but only as honest as the pessimistic estimates fed into it.

Failure modes and effects

The system FMEA was started at the CDR and updated after the field trial. Its last row was added by the field.

Table 40.13 — FMEA of AquaSol (extract).

Failure mode	Effect	Cause	Detection	Mitigation
Pump runs dry	Stator scorched within minutes	Borehole drawdown below intake in hot afternoons	Speed-high, power-low signature for 5 s	Stop, 30 min wait (R3); alarm to phone
Tank overflows	Water and energy wasted, erosion at the tank base	Float stuck or cable cut	Upper float; level transducer disagrees with float	Stop on either; disagreement raises a maintenance alarm
Inverter over-voltage	MOSFET avalanche, controller destroyed	Cold-morning V_{OC} 84.6 V on 100 V parts (first schematic)	Caught at CDR	150 V MOSFETs, 160 V capacitors (R5)
Lightning surge on the 25 m drop cable or PV leads	Gate-driver or MCU damage	Induced surge on long cables	Post-storm self-test; fault latch	Type 2 SPD at the array; MOV on the pump leads; enclosure bonding
Soil node silent	Zone not irrigated or over-irrigated	Battery, antenna damage, SF creep	Gateway missing-packet alarm after 1 h	Zone falls back to a timed schedule
Pump hunts start–stop at dawn (every 20 s for about 15 min)	Inrush stress; 40 starts a morning	No minimum off-time: V_{OC} recovers above `V_START` as soon as the pump stops	Field log, day 1	Firmware 1.1: 120 s minimum off-time and start on estimated available power

The last row is the instructive one. The bench PV simulator never recovered its voltage fast enough to reveal it; only a real array at dawn, with light enough for V_{OC} but not for pumping, did. It was fixed within a day, and it is the report's strongest argument for field testing.

40.10 Final technical report

What the report is for

The final report is the project's deliverable to people who were not in the room. It must let a reader answer four questions without asking the team: what was needed, what was built, what was shown, and what should be done next. Every numerical claim must point to its evidence: a calculation, a test record or a datasheet. Everything needed to rebuild and retest must be in the archive that accompanies the report. Chapter 5 covers the documentation set and revision control.

Template

The AquaSol report followed this outline, with page budgets set at the start so that the analysis did not crowd out the verification.

```
AquaSol: PV-Direct Irrigation Pump Controller -- Final Technical Report (rev C, tag v1.1)
 0  Abstract (200 words, with the four key numbers: m3/day, cost/m3, margin, life)
 1  Introduction: need, stakeholder, problem statement, success measures   (2 pp)
 2  Literature review: review matrix, standards, prior art, gaps           (3 pp)
 3  Requirements: table R1-R12 with source and verification method         (2 pp)
 4  Architecture: trade study, block diagram, interface control table      (3 pp)
 5  Detailed design: sizing, voltage margins, component selection,
    energy budget, budgets-and-margins table                               (6 pp)
 6  Implementation: schematics summary, PCB, firmware structure,
    state machine, build and configuration management                      (4 pp)
 7  Verification and validation: test plan, verification table,
    field results with uncertainty, model correction, validation trial     (6 pp)
 8  Cost analysis: BOM, production cost, life-cycle cost, sensitivity      (2 pp)
 9  Risk: register history, FMEA, schedule risk and outcome                (2 pp)
10  Conclusions and Revision B                                             (1 p)
 A  Appendices: full schematics, BOM with part numbers, test procedures
    and raw records, calculation scripts, ICD, safety plan, meeting log
 Archive: repository tag v1.1 -- hardware, firmware, scripts, data, report source
```

Five rules made the report reviewable:

1. **The abstract carries numbers**: "14.3 m³ on a 5.0 kWh/m² day at 23 m head, a 7 % margin", not "the system performs well".
2. **Every requirement appears three times:** in the requirements table, in the section that designs for it, and in the verification table. A reviewer can follow any row from need to result.
3. **Measured and simulated values are labelled as such**, with uncertainty and conditions. The 5.6 % model error and its correction are reported, not smoothed away.
4. **Failures are reported with mechanism and consequence.** The CDR MOSFET catch and the dawn hunting are the most useful pages for the next team.
5. **The archive reproduces the numbers.** Each figure and table in sections 5, 7, 8 and 9 is generated by a script in the repository from recorded data. The sizing, schedule and cost listings in this chapter are three of them.

> **Status of the principle: A report is verified like a design**
> *Empirical.* Reviewers and successor teams use a report through its traceability: requirement to design to evidence. Reports that state numbers with their sources and conditions, and failures with their mechanisms, get reused. Reports that state conclusions without evidence do not, however well they are written.

Revision B

Change	Why	Expected effect
One gateway shared by ten farms on the cooperative office; per-farm units send through it	Telemetry was 17 % of capex and made the cost case marginal	Cost per m³ from $0.159 to $0.128; break-even from year 10 to year 8
Firmware 1.1 made standard: 120 s minimum off-time, start on estimated available power	Dawn start–stop hunting found in the field	Starts per morning from about 40 to 1–2; less inrush stress on pump and cable
Wire-to-water efficiency 0.38 in the design model; array sizing rule updated	Field calibration showed a consistent −5.6 %	Design margin stated against measured, not datasheet, efficiency
Level-transducer and float cross-check promoted from alarm to interlock	A stuck float was the one overflow path left in the FMEA	Overflow requires two independent failures
Borehole level sensor (pressure transducer at the intake)	Dry-run detection is reactive; a level reading lets the pump slow down before it runs dry	Fewer dry-run stops; more water on hot afternoons
Calibrated model and field data packaged as a simple digital twin of the site	Lets the cooperative size the next ten farms from data, not from the brochure	Sizing from measured efficiency and drawdown (Chapter 33)

What remains open

Open question	Where it is answered
How are stakeholder needs turned into requirements with acceptance criteria and traced through a design?	Chapter 1
How are architecture trades, interfaces and test plans organised?	Chapters 2 and 4
How are schematics, BOMs, test reports and revisions managed so a report can be reproduced?	Chapter 5; Chapter 29
How is a PV array designed in full, from resource assessment to protection and cable sizing?	Chapter 22; *Electrical Energy and Machines*, Ch. 15
How is a closed-loop motor drive with its power stage, sensing and protection built and tested?	Chapter 37
How is a wireless sensor network with link budgets, power budgets and security designed and deployed?	Chapter 38; Chapter 26
How is a calibrated model turned into a digital twin that is updated from field data?	Chapter 33
How is a complete project of a different kind (robot, energy monitor, solar battery system, simulation tool) carried from requirements to Revision B?	Chapters 34–39

This book began with a requirement written down before the build and ends with a farmer's water tank filling on a cold dry-season morning. Between them lies the whole of the work: every link in the chain chosen, budgeted, built, measured and allowed to fail where it was safe to learn. That chain, verified against numbers written down in advance, is what turns the theory of the other books into systems that work for the people who use them.